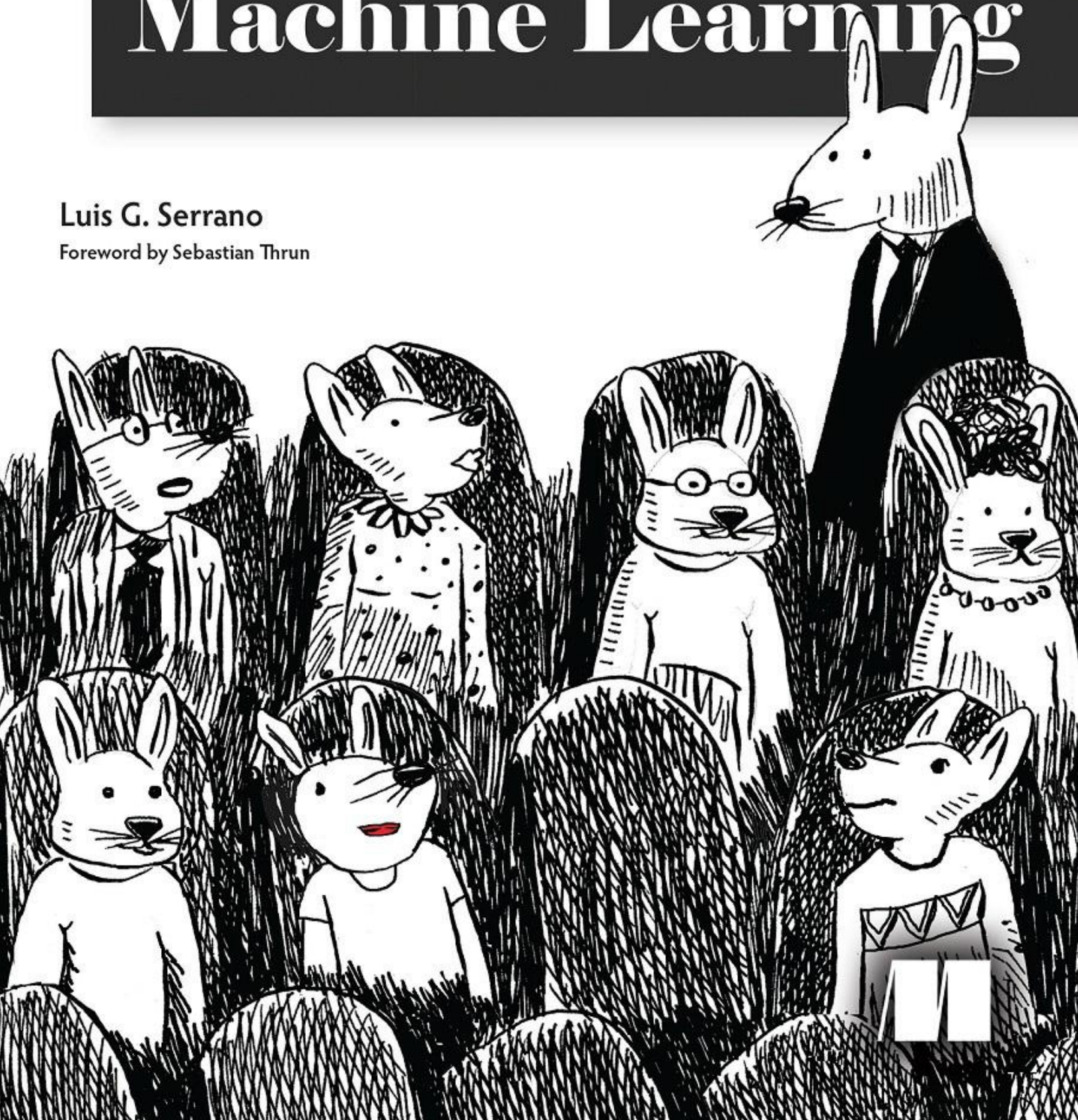


grokking

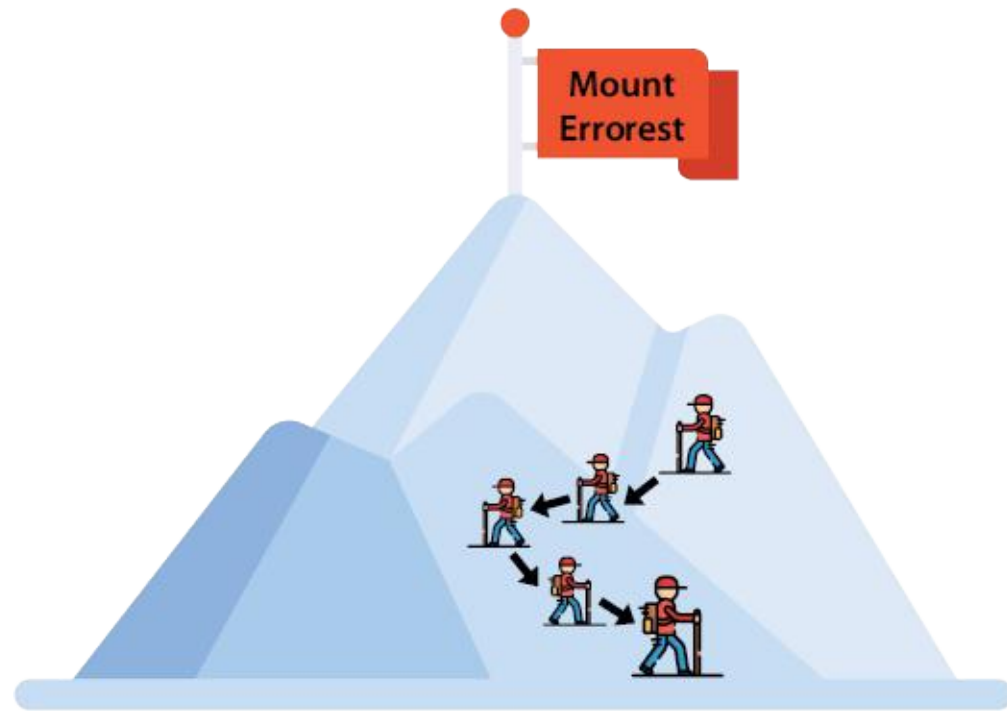
Machine Learning

Luis G. Serrano

Foreword by Sebastian Thrun



bên trong bìa trước



Cách để từ trên núi xuống là bước một bước nhỏ theo hướng khiến chúng ta phải đi xuống nhiều nhất và tiếp tục làm điều này trong một thời gian dài.



Học máy Grokking



Luis G. Serrano



Để bình luận, hãy truy cập [liveBook](#)



Quản lý

Đảo trú ẩn



Để biết thêm thông tin về điều này và các tiêu đề Manning khác, hãy truy cập www.manning.com



Bản quyền

Để biết thông tin trực tuyến và đặt hàng những cuốn sách này và các cuốn sách Manning khác, vui lòng truy cập www.manning.com. Nhà xuất bản cung cấp giảm giá cho những cuốn sách này khi đặt hàng với số lượng.

Để biết thêm thông tin, vui lòng liên hệ



Phòng Kinh doanh Đặc biệt
Manning Publications Co. 20
Baldwin Road PO Box 761 Shelter
Island, NY 11964 Email:
order@manning.com



©2021 của Manning Publications Co. Mọi quyền được bảo lưu.



Không phần nào của ấn phẩm này được phép sao chép, lưu trữ trong hệ thống truy xuất hoặc truyền đi dưới bất kỳ hình thức nào hoặc bằng phương tiện điện tử, cơ khí, sao chụp hoặc cách khác mà không có sự cho phép trước bằng văn bản của nhà xuất bản.

Nhiều tên gọi được nhà sản xuất và người bán sử dụng để phân biệt sản phẩm của họ được coi là nhãn hiệu. Trong trường hợp những ký hiệu đó xuất hiện trong sách và Manning Publications đã biết về khiếu nại về nhãn hiệu, thì các ký hiệu đó đã được in bằng chữ hoa đầu tiên hoặc toàn bộ chữ hoa.

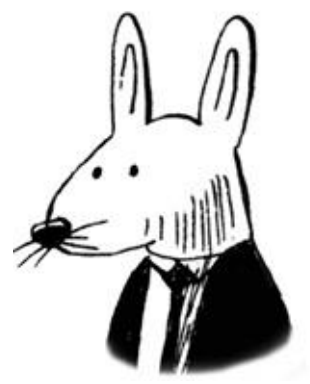
☐ Nhận thức được tầm quan trọng của việc bảo tồn những gì đã được viết ra, chính sách của Manning là yêu cầu những cuốn sách chúng tôi xuất bản được in trên giấy không chứa axit và chúng tôi nỗ lực hết mình vì mục đích đó. Cũng thừa nhận trách nhiệm của chúng ta trong việc bảo tồn tài nguyên của hành tinh, sách Manning được in trên giấy được tái chế và xử lý ít nhất 15% mà không sử dụng clo nguyên tố.



Manning Publications Co.
20 Baldwin Road Technical
PO Box 761
Shelter Island, NY 11964

Biên tập viên phát triển: Marina Michaels
Biên tập viên phát triển kỹ thuật: Kris Athi
Người biên tập đánh giá: Aleksander Dragosavljević
Biên tập sản xuất: Keri Hales
Biên tập viên sao chép: Pamela Hunt
Người hiệu đính: Jason Everett
Người hiệu đính kỹ thuật: Karsten Strøbæk, Shirley Yap
Người sắp chữ: Dennis Dalinnik
Thiết kế bìa: Leslie Haines

ISBN: 9781617295911



vấn đề trước mắt

lời nói đầu

lời nói đầu

lời cảm ơn

về cuốn sách này

về tác giả

□ □

1 Học máy là gì? Đó là điều bình thường, ngoại trừ việc được thực hiện bằng máy tính

Tôi có cần nền tảng toán học và mã hóa chuyên sâu để hiểu về học máy không?

Được rồi, vậy chính xác thì học máy là gì?

Làm thế nào để chúng ta khiến máy móc đưa ra quyết định dựa trên dữ liệu? Khung ghi nhớ- xây dựng- dự đoán

2 loại hình học máy

Sự khác biệt giữa dữ liệu được dán nhãn và không được gắn nhãn là gì?

Học có giám sát: Nhánh học máy hoạt động với dữ liệu được dán nhãn

Học không giám sát: Nhánh học máy hoạt động với dữ liệu không được gắn nhãn

Học tăng cường là gì?

3 Vẽ một đường gần các điểm của chúng ta: Hồi quy tuyến tính

Vấn đề: Chúng ta cần dự đoán giá nhà

Giải pháp: Xây dựng mô hình hồi quy giá nhà đất

Làm sao để máy tính vẽ được đường này: Thuật toán hồi quy tuyến tính

Làm thế nào để chúng tôi đo lường kết quả của chúng tôi?Chức năng lỗi

Ứng dụng thực tế: Sử dụng Turi Create để dự đoán giá nhà ở Ấn Độ

Nếu dữ liệu không nằm trong một dòng thì sao?Hồi quy đa thức

Tham số và siêu tham số

Ứng dụng của hồi quy

❑4 Tối ưu hóa quá trình đào tạo: *Underfitting, overfitting, testing và normalization*

Một ví dụ về trang bị thiếu và trang bị quá mức bằng cách sử dụng hồi quy đa thức

Làm thế nào để chúng ta có được máy tính để chọn đúng mô hình?Bằng cách thử nghiệm

Chúng ta đã vi phạm quy tắc vàng ở đâu và làm cách nào để khắc phục nó?Bộ xác thực

Một cách bằng số để quyết định mức độ phức tạp của mô hình của chúng ta: Biểu đồ độ phức tạp của mô hình

Một cách khác để tránh trang bị quá mức: Chính quy hóa

Hồi quy đa thức, kiểm tra và chuẩn hóa với Turi Create

❑5 Dừng đường kẻ để phân chia điểm: *Thuật toán perceptron*

Vấn đề: Chúng ta đang ở một hành tinh xa lạ và chúng ta không biết ngôn ngữ của họ!

Làm thế nào để chúng ta xác định liệu một bộ phân loại là tốt hay xấu?Chức năng lỗi

Làm thế nào để tìm được một bộ phân loại tốt?Thuật toán perceptron

Mã hóa thuật toán perceptron

Ứng dụng của thuật toán perceptron

❑6 Một cách tiếp cận liên tục để phân chia các điểm: *Phân loại logistic*

Bộ phân loại logistic: Phiên bản liên tục của bộ phân loại perceptron

Làm thế nào để tìm một bộ phân loại logistic tốt?Thuật toán hồi quy logistic

Mã hóa thuật toán hồi quy logistic

Ứng dụng thực tế: Phân loại đánh giá IMDB bằng Turi Create

Phân loại thành nhiều lớp: Hàm softmax

❑7 Bạn đo lường các mô hình phân loại như thế nào?Độ chính xác và những người bạn của nó

Độ chính xác: Mô hình của tôi có thường xuyên đúng không?

Làm thế nào để khắc phục vấn đề về độ chính xác?Xác định các loại lỗi khác nhau và cách đo lường chúng

Một công cụ hữu ích để đánh giá mô hình của chúng tôi: Đặc tính vận hành máy thu

□8 **Sử dụng xác suất ở mức tối đa: Mô hình Bayes ngây thơ**

Ồm hay khỏe? Câu chuyện với định lý Bayes là người hùng

Ca sử dụng: Mô hình phát hiện thư rác

Xây dựng mô hình phát hiện thư rác bằng dữ liệu thực

□9 **Phân chia dữ liệu bằng cách đặt câu hỏi: Cây quyết định**

Vấn đề: Chúng tôi cần giới thiệu ứng dụng cho người dùng dựa trên những gì họ có thể tải xuống

Giải pháp: Xây dựng hệ thống đề xuất ứng dụng

Ngoài những câu hỏi như có/không

Ranh giới đồ họa của cây quyết định

Ứng dụng thực tế: Lập mô hình tuyến tính với Scikit-Learn

Cây quyết định cho hồi quy

Ứng dụng

10 **Kết hợp các khối xây dựng để có thêm sức mạnh: Mạng lưới thần kinh**

Mạng lưới thần kinh với một ví dụ: Một hành tinh ngoài hành tinh phức tạp hơn

Đào tạo mạng lưới thần kinh

Mã hóa mạng lưới thần kinh trong Keras

Mạng lưới thần kinh cho hồi quy

Các kiến trúc khác cho các bộ dữ liệu phức tạp hơn

11 **Tìm ranh giới với kiểu dáng: Hỗ trợ máy vectơ và phương thức kernel**

Sử dụng hàm lỗi mới để xây dựng các bộ phân loại tốt hơn

Mã hóa máy vectơ hỗ trợ trong Scikit-Learn

Huấn luyện SVM có ranh giới phi tuyến: Phương pháp kernel

12 **Kết hợp các mô hình để tối đa hóa kết quả: Ensemble learning**

Với một chút giúp đỡ từ bạn bè của chúng tôi

Đóng góp: Tham gia ngẫu nhiên một số người học yếu để xây dựng một người học mạnh

AdaBoost: Kết nối những người học yếu một cách thông minh để xây dựng một người học giỏi

Tăng cường độ dốc: Sử dụng cây quyết định để xây dựng những người học giỏi

XGBoost: Một cách cực kỳ hiệu quả để tăng cường độ dốc

Ứng dụng của phương pháp tập hợp

13 Áp dụng tất cả vào thực tế: Một ví dụ thực tế về kỹ thuật dữ liệu và học máy

Bộ dữ liệu Titanic

Dọn dẹp tập dữ liệu của chúng tôi: Thiếu giá trị và cách xử lý chúng

Kỹ thuật tính năng: Chuyển đổi các tính năng trong tập dữ liệu của chúng tôi trước khi đào tạo mô hình

Đào tạo mô hình của chúng tôi

Điều chỉnh các siêu tham số để tìm ra mô hình tốt nhất: Tìm kiếm lưới

Sử dụng xác thực chéo K-Fold để sử dụng lại dữ liệu của chúng tôi dưới dạng đào tạo và xác thực

□ □

Phụ lục A. Lời giải bài tập

Phụ lục B. Toán học đằng sau việc giảm độ dốc: Xuống núi bằng cách sử dụng đạo hàm và độ dốc

Phụ lục C. Tài liệu tham khảo

□ □

chỉ mục

lời nói đầu

Bạn có nghĩ rằng học máy phức tạp và khó thành thạo không? Không phải vậy! Hãy đọc cuốn sách này!

Luis Serrano là một bậc thầy khi giải thích mọi thứ bằng tiếng Anh đơn giản. Tôi gặp anh ấy lần đầu tiên khi anh ấy dạy học máy trên Udacity. Anh ấy khiến học sinh của chúng tôi cảm thấy rằng toàn bộ quá trình học máy cũng đơn giản như việc cộng hoặc trừ các con số. Và trên hết, anh ấy đã làm cho tài liệu trở nên thú vị. Các video anh sản xuất cho Udacity vô cùng hấp dẫn và luôn nằm trong số những nội dung được yêu thích nhất trên nền tảng này.

Cuốn sách này tốt hơn! Ngay cả những người sợ hãi nhất cũng sẽ thích thú với tài liệu được trình bày trong tài liệu này, vì Serrano làm sáng tỏ một số bí mật được giấu kín nhất của xã hội học máy. Anh ấy sẽ hướng dẫn bạn từng bước qua từng thuật toán và kỹ thuật quan trọng trong lĩnh vực này. Bạn có thể trở thành người đam mê học máy ngay cả khi bạn không thích môn toán. Serrano giảm thiểu *kauderwelsch* toán học mà rất nhiều học giả khó tính trong chúng ta yêu thích, thay vào đó dựa vào trực giác và những giải thích thực tế.

Mục tiêu thực sự của cuốn sách này là giúp bạn tự mình nắm vững những phương pháp này. Vì vậy, cuốn sách chứa đầy những bài tập thú vị, trong đó bạn có thể tự mình thử những kỹ thuật thần bí (và hiện đã được làm sáng tỏ). Bạn muốn xem chương trình truyền hình Netflix mới nhất hay dành thời gian áp dụng học máy vào các vấn đề về thị giác máy tính và hiểu ngôn ngữ tự nhiên? Nếu là trường hợp thứ hai thì cuốn sách này là dành cho bạn. Tôi không thể diễn tả được cảm giác thú vị như thế nào khi chơi với công nghệ máy học mới nhất và xem máy tính của bạn làm nên điều kỳ diệu dưới sự giám sát của bạn.

Và vì học máy là công nghệ hot nhất xuất hiện trong vài năm qua nên giờ đây bạn sẽ có thể tận dụng các kỹ năng mới tìm được trong công việc của mình. Vài năm trước, *New York Times* tuyên bố rằng trên thế giới chỉ có 10.000 chuyên gia về máy học, với hàng triệu vị trí còn trống. Ngày nay vẫn vậy! Hãy đọc cuốn sách này và trở thành một kỹ sư máy học chuyên nghiệp. Bạn được đảm bảo sở hữu một trong những kỹ năng được yêu cầu cao nhất trên thế giới hiện nay.

Với cuốn sách này, Luis Serrano đã thực hiện một công việc đáng ngưỡng mộ khi giải thích các thuật toán phức tạp và làm cho hầu hết mọi người đều có thể tiếp cận được chúng. Nhưng anh ấy không thỏa hiệp về chiều sâu. Thay vào đó, ông tập trung vào việc trao quyền cho người đọc thông qua một chuỗi các dự án và bài tập mang tính khai sáng. Theo nghĩa này, đây không phải là một cách đọc thụ động. Để được hưởng lợi đầy đủ từ cuốn sách này, bạn phải làm việc. Tại Udacity, chúng tôi có câu nói: Bạn sẽ không giảm cân bằng cách xem người khác tập thể dục. Để tìm hiểu về máy học, bạn phải học cách áp dụng nó vào các vấn đề trong thế giới thực. Nếu bạn sẵn sàng làm điều này thì đây chính là cuốn sách của bạn – dù bạn là ai!

Sebastian Thrun, Tiến sĩ

Người sáng lập, Udacity

Giáo sư phụ trợ, Đại học Stanford

lời nói đầu

Tương lai là ở đây và tương lai đó có một cái tên: học máy. Với các ứng dụng trong hầu hết mọi ngành, từ y học đến ngân hàng, từ ô tô tự lái đến gọi cà phê, mối quan tâm đến học máy đã tăng lên nhanh chóng mỗi ngày. Nhưng *máy học* là gì?

Hầu hết, khi tôi đọc một cuốn sách về máy học hoặc tham dự một bài giảng về máy học, tôi thấy một biển công thức phức tạp hoặc một biển dòng mã. Trong một thời gian dài, tôi cứ tưởng đây là machine learning, và machine learning chỉ dành cho những người có kiến thức vững chắc về cả toán và khoa học máy tính.

Tuy nhiên, tôi bắt đầu so sánh học máy với các môn học khác, chẳng hạn như âm nhạc. Lý thuyết và thực hành âm nhạc là những môn học phức tạp. Nhưng khi nghĩ đến âm nhạc, chúng ta không nghĩ đến nốt nhạc và thang âm; chúng ta nghĩ về những bài hát và giai điệu. Và rồi tôi tự hỏi, liệu máy học có giống nhau không? Nó thực sự chỉ là một đống công thức và mật mã, hay có một giai điệu nào đó đằng sau nó?



Music



Machine learning

Hình FM.1 Âm nhạc không chỉ có thang âm và nốt nhạc. Có một giai điệu đằng sau tất cả các kỹ thuật. Tương tự như vậy, học máy không chỉ có công thức và mã. Ngoài ra còn có một giai điệu, và trong cuốn sách này, chúng tôi hát nó.

Với suy nghĩ này, tôi bắt đầu hành trình tìm hiểu giai điệu của học máy. Tôi nhìn chăm chăm vào các công thức và mã trong nhiều tháng. Tôi đã vẽ rất nhiều sơ đồ. Tôi viết nguệch ngoạc những bức vẽ lên khăn ăn và cho gia đình, bạn bè và đồng nghiệp xem. Tôi đã đào tạo các mô hình trên các tập dữ liệu nhỏ và lớn. Tôi đã thử nghiệm. Sau một thời gian, tôi bắt đầu lắng nghe giai điệu của máy học. Đột nhiên, một số hình ảnh rất đẹp bắt đầu hình thành trong tâm trí tôi. Tôi bắt đầu viết những câu chuyện gắn liền với tất cả các khái niệm về học máy. Giai điệu, hình ảnh, câu chuyện—đó là cách tôi thích thú khi học bất kỳ chủ đề nào, và chính những giai điệu, hình ảnh và câu chuyện đó mà tôi chia sẻ với bạn trong cuốn sách này. Mục tiêu của tôi là làm cho việc học máy trở nên hoàn toàn dễ hiểu đối với mọi người và cuốn sách này là một bước trong hành trình đó—một bước mà tôi rất vui khi bạn đồng hành cùng tôi!

lời cảm ơn

Trước hết, tôi xin cảm ơn người biên tập của tôi, Marina Michaels, nếu không có người đó thì cuốn sách này sẽ không tồn tại. Cách tổ chức, biên tập kỹ lưỡng và thông tin đầu vào có giá trị của cô đã giúp định hình *Grokking Machine Learning*. Tôi cảm ơn Marjan Bace, Bert Bates và những người còn lại trong nhóm Manning vì sự hỗ trợ, tính chuyên nghiệp, những ý tưởng tuyệt vời và sự kiên nhẫn của họ. Tôi xin cảm ơn những người chứng minh kỹ thuật của tôi, Shirley Yap và Karsten Strøbæk; biên tập viên phát triển kỹ thuật của tôi, Kris Athi; và những người phản biện đã cho tôi những phản hồi tuyệt vời và sửa chữa nhiều sai sót của tôi. Tôi cảm ơn biên tập viên sản xuất, Keri Hales, biên tập viên sao chép, Pamela Hunt, biên tập đồ họa, Jennifer Houle, người hiệu đính, Jason Everett, và toàn bộ nhóm sản xuất vì công việc tuyệt vời của họ trong việc biến cuốn sách này thành hiện thực. Tôi cảm ơn Laura Montoya vì sự giúp đỡ của cô ấy về ngôn ngữ hòa nhập và đạo đức AI, Diego Hernandez vì những bổ sung có giá trị cho

code, và Christian Picón vì sự giúp đỡ to lớn của anh ấy về các khía cạnh kỹ thuật của kho và các gói.

Tôi biết ơn Sebastian Thrun vì công việc xuất sắc của ông về dân chủ hóa giáo dục. Udacity là nền tảng đầu tiên cho tôi tiếng nói để giảng dạy thế giới và tôi muốn cảm ơn những đồng nghiệp và sinh viên tuyệt vời mà tôi đã gặp ở đó. Alejandro Perdomo và nhóm Máy tính Zapata xứng đáng được cảm ơn vì đã giới thiệu cho tôi thế giới học máy lượng tử. Tôi cũng xin gửi lời cảm ơn đến nhiều nhà lãnh đạo và đồng nghiệp tuyệt vời mà tôi đã gặp ở Google và Apple, những người đã đóng góp rất nhiều cho sự nghiệp của tôi. Đặc biệt cảm ơn Roberto Cipriani và nhóm tại Paper Inc. vì đã cho phép tôi trở thành thành viên của gia đình và vì công việc tuyệt vời mà họ đã làm trong cộng đồng giáo dục.

Tôi muốn cảm ơn nhiều cố vấn học thuật đã định hình sự nghiệp và cách suy nghĩ của tôi: Mary Falk de Losada và nhóm của cô ấy tại Olympic Toán học Colombia, nơi tôi lần đầu tiên bắt đầu yêu thích toán học và có cơ hội gặp gỡ những người cố vấn tuyệt vời và tạo dựng tình bạn kéo dài suốt đời; cố vấn tiến sĩ của tôi, Sergey Fomin, người có công trong việc giáo dục toán học và phong cách giảng dạy của tôi; cố vấn của thầy tôi, Ian Goulden; Nantel và François Bergeron, Bruce Sagan và Federico Ardila, cùng nhiều giáo sư và đồng nghiệp mà tôi có cơ hội làm việc cùng, đặc biệt là những người ở Đại học Waterloo, Michigan, Quebec ở Montreal và York; và cuối cùng là Richard Hoshino cùng nhóm và sinh viên tại Đại học Quest, những người đã giúp tôi kiểm tra và cải thiện tài liệu trong cuốn sách này.

Gửi tới tất cả những người đánh giá: Al Pezewski, Albert Nogués Sabater, Amit Lamba, Bill Mitchell, Borko Djurkovic, Daniele Andreis, Erik Sapper, Hao Liu, Jeremy R. Loscheider, Juan Gabriel Bono, Kay Engelhardt, Krzysztof Kamyczek, Matthew Margolis, Matthias Busch, Michael Bright, Millad Dagdoni, Polina Keselman, Tony Holdroyd và Valerie Parham-Thompson, những gợi ý của bạn đã giúp cuốn sách này trở nên hay hơn.

Tôi xin cảm ơn vợ tôi, Carolina Lasso, người đã hỗ trợ tôi ở mọi bước của quá trình này bằng tình yêu và lòng tốt; mẹ tôi, Cecilia Herrera, người đã nuôi dạy tôi bằng tình yêu thương và luôn khuyến khích tôi theo đuổi đam mê của mình; bà tôi, Maruja, vì là thiên thần nhìn tôi từ trên trời; người bạn thân nhất của tôi, Alejandro Morales, vì đã luôn ở bên tôi; và những người bạn đã soi sáng con đường và soi sáng cuộc đời tôi, tôi cảm ơn và yêu các bạn bằng cả trái tim.

YouTube, blog, podcast và mạng xã hội đã cho tôi cơ hội kết nối với hàng nghìn tâm hồn thông minh trên khắp thế giới. Tâm trí tò mò với vô tận

niềm đam mê học tập, những nhà giáo dục đồng nghiệp luôn hào phóng chia sẻ kiến thức và hiểu biết sâu sắc của họ, tạo thành một nhóm điện tử truyền cảm hứng cho tôi mỗi ngày và mang lại cho tôi năng lượng để tiếp tục giảng dạy và học tập. Tôi xin cảm ơn những ai đang chia sẻ kiến thức của mình với thế giới hoặc những người nỗ lực học hỏi mỗi ngày.

Tôi cảm ơn bất cứ ai ngoài kia đang nỗ lực biến thế giới này thành một nơi công bằng và hòa bình hơn. Đối với bất kỳ ai đấu tranh cho công lý, vì hòa bình, vì môi trường và vì cơ hội bình đẳng cho mọi người trên Trái đất bất kể chủng tộc, giới tính, nơi sinh, điều kiện và lựa chọn của họ, tôi xin gửi lời cảm ơn từ tận đáy lòng mình.

Và cuối cùng nhưng không kém phần quan trọng, cuốn sách này được dành riêng cho bạn, người đọc. Các bạn đã chọn con đường học tập, con đường hoàn thiện, con đường cảm thấy thoải mái trong khó khăn, và điều đó thật đáng ngưỡng mộ. Tôi hy vọng cuốn sách này là một bước đi tích cực trên con đường theo đuổi đam mê của bạn và tạo ra một thế giới tốt đẹp hơn.

về cuốn sách này

Cuốn sách này dạy cho bạn hai điều: các mô hình học máy và cách sử dụng chúng. Mô hình học máy có nhiều loại khác nhau. Một số trong số chúng trả về câu trả lời xác định, chẳng hạn như có hoặc không, trong khi những câu khác trả về câu trả lời dưới dạng xác suất. Một số sử dụng phương trình; những người khác sử dụng câu lệnh if. Một điểm chung của chúng là đều trả về một câu trả lời hoặc một dự đoán. Nhánh học máy bao gồm các mô hình trả về dự đoán được đặt tên phù hợp là *học máy dự đoán*. Đây là loại máy học mà chúng tôi tập trung vào trong cuốn sách này.

Cuốn sách này được tổ chức như thế nào: Lộ trình

LOẠI CHƯƠNG

Cuốn sách này có hai loại chương. Phần lớn trong số chúng (chương 3, 5, 6, 8, 9, 10, 11 và 12) đều chứa một loại mô hình học máy. Mô hình tương ứng trong mỗi chương được nghiên cứu chi tiết, bao gồm ví dụ, công thức, mã và bài tập để bạn giải. Các chương khác (chương 4, 7 và 13) chứa các kỹ thuật hữu ích được sử dụng để đào tạo, đánh giá và cải thiện các mô hình học máy. Đặc biệt, chương 13 chứa một ví dụ chi tiết về một tập dữ liệu thực, trong đó bạn sẽ có thể áp dụng tất cả kiến thức bạn đã thu được vào

các chương trước.

LỘ TRÌNH HỌC TẬP ĐƯỢC KHUYẾN NGHỊ

Bạn có thể sử dụng cuốn sách này theo hai cách. Cách tôi khuyên bạn nên xem xét nó một cách tuyến tính, từng chương một, bởi vì bạn sẽ thấy rằng sự xen kẽ giữa các mô hình học tập và các kỹ thuật học tập để đào tạo chúng là rất bổ ích. Tuy nhiên, một lộ trình học tập khác là trước tiên hãy học tất cả các mô hình (chương 3, 5, 6, 8, 9, 10, 11 và 12), sau đó học các kỹ thuật để đào tạo chúng (chương 4, 7 và 13). Và tất nhiên, vì tất cả chúng ta đều học theo những cách khác nhau nên bạn có thể tạo ra lộ trình học tập của riêng mình!

PHỤ LỤC

Cuốn sách này có ba phụ lục. Phụ lục A chứa lời giải cho các bài tập của từng chương. Phụ lục B chứa một số dẫn xuất toán học hình thức hữu ích nhưng mang tính kỹ thuật cao hơn phần còn lại của cuốn sách. Phụ lục C chứa danh sách các tài liệu tham khảo và tài nguyên mà tôi đề xuất nếu bạn muốn nâng cao hiểu biết của mình.

YÊU CẦU VÀ MỤC TIÊU HỌC TẬP

Cuốn sách này cung cấp cho bạn một khuôn khổ vững chắc về học máy dự đoán. Để tận dụng tối đa cuốn sách này, bạn nên có tư duy trực quan và hiểu biết tốt về toán tiểu học, chẳng hạn như đồ thị đường thẳng, phương trình và xác suất cơ bản. Sẽ rất hữu ích (mặc dù không bắt buộc) nếu bạn biết cách viết mã, đặc biệt là bằng Python, vì bạn có cơ hội triển khai và áp dụng một số mô hình trong các bộ dữ liệu thực trong suốt cuốn sách. Sau khi đọc cuốn sách này, bạn sẽ có thể làm được những việc sau:

- Mô tả các mô hình quan trọng nhất trong học máy dự đoán và cách chúng hoạt động, bao gồm hồi quy tuyến tính và logistic, Bayes ngây thơ, cây quyết định, mạng lưới thần kinh, máy vector hỗ trợ và các phương pháp tập hợp. Xác định điểm mạnh và điểm yếu của họ và những thông số họ sử dụng. Xác định cách sử dụng các mô hình này trong thế giới thực và hình thành các cách tiềm năng để áp dụng học máy vào bất kỳ vấn đề cụ thể nào mà bạn muốn giải quyết.
-

- Tìm hiểu cách tối ưu hóa các mô hình này, so sánh và cải thiện chúng để xây dựng các mô hình học máy tốt nhất có thể. Mã hóa các mô hình, dù bằng tay hay sử dụng gói hiện có và sử dụng chúng để đưa ra dự đoán trên các tập dữ liệu thực.

Nếu bạn đang nghĩ đến một tập dữ liệu hoặc một vấn đề cụ thể, tôi mời bạn suy nghĩ về cách áp dụng những gì bạn học được trong cuốn sách này vào đó và sử dụng nó làm điểm khởi đầu để triển khai và thử nghiệm các mô hình của riêng bạn.

Tôi rất vui mừng được bắt đầu cuộc hành trình này với bạn và tôi hy vọng bạn cũng hào hứng như vậy!

NGUỒN LỰC KHÁC

Cuốn sách này là độc lập. Điều này có nghĩa là ngoài những yêu cầu được mô tả trước đó, mọi khái niệm chúng tôi cần đều được giới thiệu trong cuốn sách. Tuy nhiên, tôi bao gồm nhiều tài liệu tham khảo mà tôi khuyên bạn nên kiểm tra xem bạn có muốn hiểu các khái niệm ở mức độ sâu hơn hay muốn khám phá các chủ đề khác. Các tài liệu tham khảo đều có trong phụ lục C và tại liên kết này: <http://serrano.academy/grokking-machine-learning>.

Đặc biệt, một số nguồn tài liệu của riêng tôi đi kèm với tài liệu của cuốn sách này. Trong trang của tôi tại <http://serrano.academy>, bạn có thể tìm thấy rất nhiều tài liệu dưới dạng video, bài đăng và mã. Các video này cũng có trong kênh YouTube của tôi www.youtube.com/c/LuisSerrano, tôi khuyên bạn nên xem qua kênh này. Trên thực tế, hầu hết các chương trong cuốn sách này đều có video tương ứng mà tôi khuyên bạn nên xem khi đọc chương đó.

CHÚNG TÔI SẼ VIẾT MÃ

Trong cuốn sách này, chúng ta sẽ viết mã bằng Python. Tuy nhiên, nếu kế hoạch của bạn là học các khái niệm mà không cần mật mã, bạn vẫn có thể theo dõi cuốn sách mà bỏ qua mật mã. Tuy nhiên, tôi khuyên bạn ít nhất hãy xem mã để làm quen với nó.

Cuốn sách này đi kèm với một kho lưu trữ mã và hầu hết các chương sẽ cho bạn cơ hội viết mã các thuật toán từ đầu hoặc sử dụng một số gói Python rất phổ biến để xây dựng các mô hình phù hợp với các tập dữ liệu nhất định. Kho lưu trữ GitHub là www.github.com/luisguiserrano/manning và tôi liên kết các sổ ghi chép tương ứng trong suốt cuốn sách. Trong README của kho lưu trữ, bạn sẽ tìm thấy hướng dẫn

để các gói cài đặt chạy mã thành công.

Các gói Python chính mà chúng tôi sử dụng trong cuốn sách này như sau:

- **NumPy**: để lưu trữ mảng và thực hiện các phép tính toán học phức tạp
- **Pandas**: để lưu trữ, thao tác và phân tích các tập dữ liệu lớn
- **Matplotlib**: để vẽ đồ thị dữ liệu
- **Turi Create**: để lưu trữ và thao tác dữ liệu cũng như đào tạo các mô hình machine learning
- **Scikit-Learn**: để đào tạo các mô hình machine learning
- **Keras (TensorFlow)**: để đào tạo mạng lưới thần kinh
-
-

Giới thiệu về mã

Cuốn sách này chứa nhiều ví dụ về mã nguồn phù hợp với văn bản thông thường. Trong cả hai trường hợp, mã nguồn được định dạng theo kiểu a để tách nó khỏi văn bản thông thường. Đôi khi mã cũng **in bold** để đánh dấu mã đã thay đổi so với các bước trước đó trong chương, chẳng hạn như khi một tính năng mới thêm vào dòng mã hiện có.

Trong nhiều trường hợp, mã nguồn gốc đã được định dạng lại; chúng tôi đã thêm dấu ngắt dòng và thụt lề lại để phù hợp với không gian trang có sẵn trong sách. Ngoài ra, các nhận xét trong mã nguồn thường bị xóa khỏi danh sách khi mã được mô tả trong văn bản. Chú thích mã đi kèm với nhiều danh sách, làm nổi bật các khái niệm quan trọng.

Mã cho các ví dụ trong cuốn sách này có sẵn để tải xuống trên trang web Manning (<https://www.manning.com/books/grokking-machine-learning>) và từ GitHub tại www.github.com/luisguiserrano/manning.

diễn đàn thảo luận liveBook

Mua *Grokking Machine Learning* bao gồm quyền truy cập miễn phí vào diễn đàn web riêng do Manning Publications điều hành, nơi bạn có thể đưa ra nhận xét về cuốn sách, đặt câu hỏi kỹ thuật và nhận trợ giúp từ tác giả cũng như từ những người dùng khác. Để truy cập diễn đàn, hãy truy cập <https://livebook.manning.com/#!/book/grokking-machine-learning/discussion>. Bạn cũng có thể tìm hiểu thêm về các diễn đàn của Manning và các quy tắc ứng xử tại <https://livebook.manning.com/#!/discussion>.

Cam kết của Manning với độc giả của chúng tôi là cung cấp một địa điểm để có thể diễn ra cuộc đối thoại có ý nghĩa giữa từng độc giả và giữa độc giả và tác giả. Đây không phải là cam kết về số lượng tham gia cụ thể của tác giả, sự đóng góp của họ cho diễn đàn là tự nguyện (và không được trả phí). Chúng tôi khuyên bạn nên thử hỏi tác giả một số câu hỏi khó để tác giả không mất hứng thú! Diễn đàn và kho lưu trữ các cuộc thảo luận trước đó sẽ có thể truy cập được từ trang web của nhà xuất bản miễn là cuốn sách được in.

về tác giả

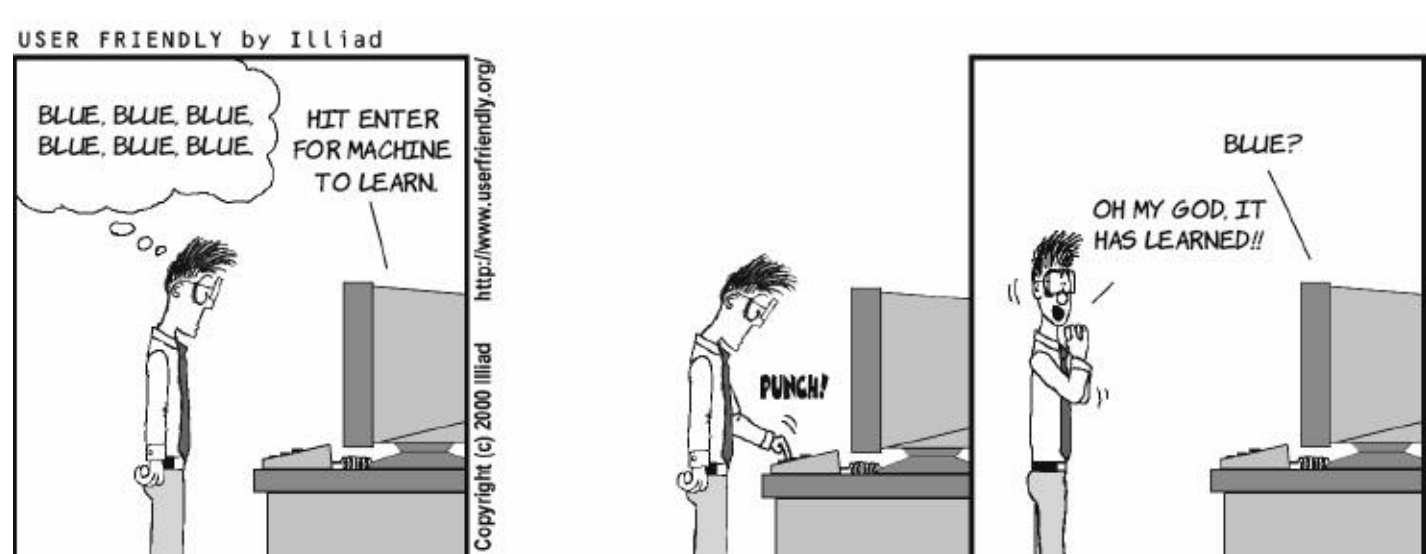


LUIS G. SERRANO là nhà khoa học nghiên cứu về trí tuệ nhân tạo lượng tử tại Zapata Computing. Trước đây, anh từng đảm nhiệm vị trí Kỹ sư máy học tại Google, Nhà giáo dục trí tuệ nhân tạo hàng đầu tại Apple và Trưởng phòng nội dung về trí tuệ nhân tạo và khoa học dữ liệu tại Udacity. Luis có bằng Tiến sĩ toán học tại Đại học Michigan, bằng cử nhân và thạc sĩ toán học tại Đại học Waterloo, đồng thời làm nghiên cứu sinh sau tiến sĩ tại Phòng thí nghiệm Combinatoire et d'Informatique Mathématique tại Đại học Quebec ở Montreal. Luis duy trì một kênh YouTube nổi tiếng về học máy với hơn 85.000 người đăng ký và hơn 4 triệu lượt xem, đồng thời là diễn giả thường xuyên tại các hội nghị khoa học dữ liệu và trí tuệ nhân tạo.

1 Học máy là gì? Đó là điều bình thường, ngoại trừ việc được thực hiện bằng máy tính

Trong chương này

- Học máy là gì thì học máy khó (spoiler: không) chúng ta học gì trong cuốn sách này trí tuệ
- nhân tạo là gì và nó khác với học máy như thế nào con người suy nghĩ như thế nào và làm
- cách nào chúng ta có thể đưa những ý tưởng đó vào máy một số ví dụ cơ bản về học máy trong đời thực
-
-



Tôi rất vui khi được đồng hành cùng bạn trong hành trình học tập!

Chào mừng đến với cuốn sách này! Tôi rất vui khi được tham gia cùng bạn trong hành trình tìm hiểu về học máy. Ở cấp độ cao, học máy là một quá trình trong đó máy tính giải quyết vấn đề và đưa ra quyết định theo cách giống như con người.

Trong cuốn sách này, tôi muốn gửi đến bạn một thông điệp: học máy rất dễ dàng! Bạn không cần phải có nền tảng toán học và lập trình nặng nề mới có thể hiểu được nó. Bạn thực sự cần một số kiến thức toán học cơ bản, nhưng các thành phần chính là lẽ thường, trực giác trực quan tốt và mong muốn học và áp dụng những phương pháp này vào bất kỳ lĩnh vực nào mà bạn đam mê cũng như lĩnh vực mà bạn muốn thực hiện.

cải tiến trên thế giới. Tôi thực sự rất vui khi viết cuốn sách này, bởi vì tôi thích nâng cao hiểu biết của mình về chủ đề này và tôi hy vọng bạn sẽ vui vẻ khi đọc nó và đi sâu vào học máy!

Học máy có ở khắp mọi nơi

Học máy có ở khắp mọi nơi. Câu nói này dường như ngày càng đúng hơn. Tôi gặp khó khăn khi tưởng tượng ra một khía cạnh nào đó của cuộc sống mà học máy không thể cải thiện bằng cách này hay cách khác. Đối với bất kỳ công việc nào đòi hỏi sự lặp lại hoặc xem xét dữ liệu và thu thập kết luận, học máy có thể giúp ích. Trong vài năm qua, học máy đã chứng kiến sự phát triển vượt bậc nhờ những tiến bộ về sức mạnh tính toán và tính phổ biến của việc thu thập dữ liệu. Chỉ cần nêu tên một số ứng dụng của học máy: hệ thống đề xuất, nhận dạng hình ảnh, xử lý văn bản, ô tô tự lái, nhận dạng thư rác, chẩn đoán y tế ... danh sách vẫn tiếp tục. Có lẽ bạn có một mục tiêu hoặc một lĩnh vực mà bạn muốn tạo ra ảnh hưởng (hoặc có thể bạn đã tạo được ảnh hưởng rồi!). Rất có thể, học máy có thể được áp dụng cho lĩnh vực đó—có lẽ đó là điều đưa bạn đến với cuốn sách này. Hãy cùng nhau tìm hiểu nhé!

Tôi có cần nền tảng toán học và mã hóa chuyên sâu để hiểu về máy học không?

Không. Học máy đòi hỏi trí tưởng tượng, sự sáng tạo và đầu óc trực quan. Học máy là việc thu thập các mẫu xuất hiện trên thế giới và sử dụng các mẫu đó để đưa ra dự đoán trong tương lai. Nếu bạn thích tìm kiếm các mẫu và phát hiện các mối tương quan thì bạn có thể học máy. Nếu tôi nói với bạn rằng tôi đã ngừng hút thuốc, ăn nhiều rau hơn và tập thể dục, bạn dự đoán điều gì sẽ xảy ra với sức khỏe của tôi sau một năm nữa? Có lẽ nó sẽ được cải thiện. Nếu tôi nói với bạn rằng tôi đã chuyển từ mặc áo len đỏ sang áo len xanh, bạn dự đoán điều gì sẽ xảy ra với sức khỏe của tôi sau một năm nữa? Có lẽ nó sẽ không thay đổi nhiều (có thể, nhưng không dựa trên thông tin tôi cung cấp cho bạn). Việc phát hiện ra những mối tương quan và mô hình này chính là nội dung của máy học. Sự khác biệt duy nhất là trong học máy, chúng ta gán các công thức và con số vào các mẫu này để máy tính phát hiện ra chúng.

Cần có một số kiến thức toán học và mã hóa để thực hiện học máy, nhưng bạn không cần phải là chuyên gia. Nếu bạn là chuyên gia trong một trong hai hoặc cả hai, bạn chắc chắn sẽ thấy kỹ năng của mình sẽ được khen thưởng. Nhưng nếu không, bạn vẫn có thể học máy học và tiếp tục học toán cũng như viết mã. Trong cuốn sách này, chúng tôi giới thiệu tất cả các khái niệm toán học mà chúng tôi cần vào thời điểm chúng tôi cần chúng. Khi nói đến mã hóa, bạn viết bao nhiêu mã trong machine learning là tùy thuộc vào bạn. Công việc học máy bao gồm từ những người viết mã cả ngày cho đến những người không hề viết mã. Nhiều gói, API và công cụ giúp chúng tôi thực hiện học máy với mức mã hóa tối thiểu. Mỗi ngày, công nghệ máy học được cung cấp nhiều hơn cho mọi người trên thế giới và tôi rất vui vì bạn đã tham gia!

Công thức và mã rất thú vị khi được xem như một ngôn ngữ

Trong hầu hết các sách về học máy, các thuật toán được giải thích bằng toán học bằng cách sử dụng các công thức, đạo hàm, v.v. Mặc dù những mô tả chính xác về các phương pháp này có tác dụng tốt trong thực tế, nhưng bản thân một công thức có thể gây nhầm lẫn hơn là mang tính minh họa. Tuy nhiên, giống như một bản nhạc, một công thức có thể che giấu một giai điệu đẹp để đằng sau sự nhầm lẫn. Ví dụ: hãy xem công thức này: $\sum_{i=1}^4 i$. Thoạt nhìn trông có vẻ xấu nhưng nó biểu thị một tổng rất đơn giản, cụ thể là $1 + 2 + 3 + 4$. Còn $\sum_{i=1}^n w_i$ thì sao? Đó đơn giản là tổng của nhiều (n). Nhưng khi nghĩ về tổng của nhiều số, tôi thích tưởng tượng những thứ như $3 + 2 + 4 + 27$ hơn là $\sum_{i=1}^n w_i$. Mỗi khi nhìn thấy một công thức, ngay lập tức tôi phải tưởng tượng ra một ví dụ nhỏ về nó, khi đó hình ảnh đó hiện rõ hơn trong đầu tôi. Khi tôi nhìn thấy điều gì đó như $P(A|B)$, tôi nghĩ đến điều gì? Đó là một xác suất có điều kiện, vì vậy tôi nghĩ ra một số câu như "Xác suất để một sự kiện A xảy ra khi một sự kiện B khác đã xảy ra." Ví dụ: nếu A tượng trưng cho mưa hôm nay và B tượng trưng cho việc sống trong rừng mưa Amazon, thì công thức $P(A|B) = 0,8$ chỉ đơn giản có nghĩa là "Xác suất hôm nay trời mưa vì chúng ta sống trong rừng mưa Amazon là 80%".

Nếu bạn yêu thích các công thức, đừng lo lắng—cuốn sách này vẫn có chúng. Nhưng chúng sẽ xuất hiện ngay sau ví dụ minh họa chúng.

Hiện tượng tương tự cũng xảy ra với mã. Nếu chúng ta nhìn mã từ xa, nó có thể trông phức tạp và chúng ta có thể khó tưởng tượng rằng ai đó có thể nhét tất cả những thứ đó vào đầu họ. Tuy nhiên, mã chỉ đơn giản là một chuỗi các bước và thông thường mỗi bước này đều đơn giản. Trong cuốn sách này, chúng ta sẽ viết mã nhưng nó sẽ được chia thành các bước đơn giản và mỗi bước sẽ được giải thích cẩn thận bằng các ví dụ hoặc hình ảnh minh họa. Trong vài chương đầu tiên, chúng ta sẽ viết mã

mô hình từ đầu để hiểu cách chúng hoạt động. Tuy nhiên, trong các chương sau, các mô hình trở nên phức tạp hơn. Đối với những điều này, chúng tôi sẽ sử dụng các gói như Scikit-Learn, Turi Create hoặc Keras, những gói đã triển khai hầu hết các thuật toán học máy một cách rõ ràng và mạnh mẽ.

*Được rồi, **vậy chính xác học máy là gì?***

Để định nghĩa học máy, trước tiên hãy định nghĩa một thuật ngữ tổng quát hơn: trí tuệ nhân tạo.

Trí tuệ nhân tạo là gì?

Nhân tạo trí tuệ (AI) là một thuật ngữ chung được chúng tôi định nghĩa như sau:

trí tuệ nhân tạo Tập hợp tất cả các nhiệm vụ mà máy tính có thể đưa ra quyết định

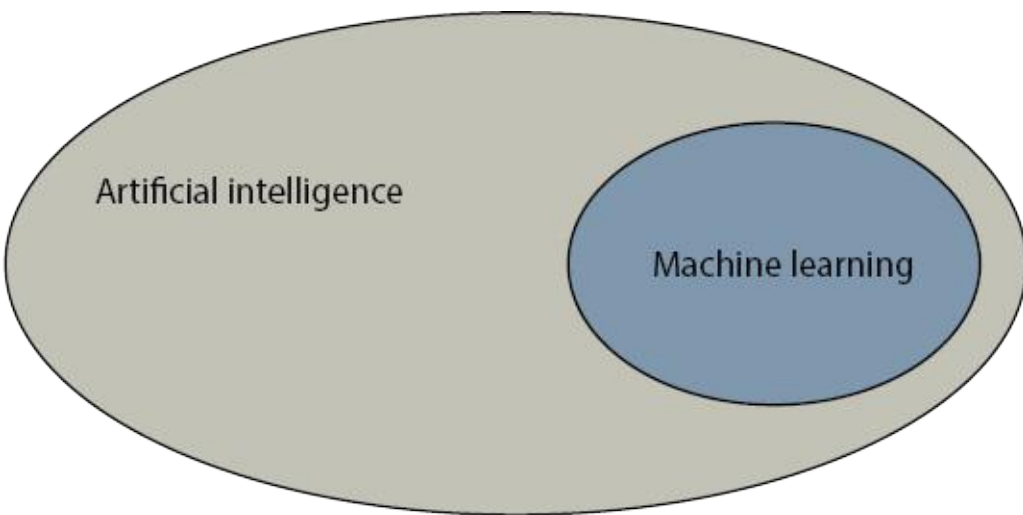
Trong nhiều trường hợp, máy tính đưa ra những quyết định này bằng cách bắt chước cách con người đưa ra quyết định. Trong các trường hợp khác, chúng có thể bắt chước các quá trình tiến hóa, quá trình di truyền hoặc quá trình vật lý. Nhưng nói chung, bất cứ khi nào chúng ta thấy máy tính tự giải quyết một vấn đề, có thể là lái ô tô, tìm đường giữa hai điểm, chẩn đoán bệnh nhân hoặc giới thiệu một bộ phim, chúng ta đang xem xét trí tuệ nhân tạo.

Học máy là gì?

Học máy tương tự như trí tuệ nhân tạo và thường định nghĩa của chúng bị nhầm lẫn. *Học máy* (ML) là một phần của trí tuệ nhân tạo và chúng tôi định nghĩa nó như sau:

học máy Tập hợp tất cả các nhiệm vụ trong đó máy tính có thể đưa ra quyết định *dựa trên dữ liệu*

Điều này có nghĩa là gì? Cho phép tôi minh họa bằng sơ đồ trong hình 1.1.



Hình 1.1 Học máy là một phần của trí tuệ nhân tạo.

Hãy quay lại xem con người đưa ra quyết định như thế nào. Nói chung, chúng tôi đưa ra quyết định theo hai cách sau:

- Bằng cách sử dụng logic và lý luận
- Bằng cách sử dụng kinh nghiệm của chúng tôi

Ví dụ, hãy tưởng tượng rằng chúng ta đang cố gắng quyết định nên mua chiếc xe nào. Chúng ta có thể xem xét cẩn thận các đặc điểm của chiếc xe, chẳng hạn như giá cả, mức tiêu thụ nhiên liệu và điều hướng, đồng thời cố gắng tìm ra sự kết hợp tốt nhất giữa chúng để điều chỉnh phù hợp với ngân sách của chúng ta. Đó là sử dụng logic và lý luận. Thay vào đó, nếu chúng ta hỏi bạn bè xem họ sở hữu chiếc xe nào, họ thích và không thích điều gì ở chúng, rồi lập một danh sách thông tin và sử dụng danh sách đó để quyết định, thì chúng ta đang sử dụng kinh nghiệm (trong trường hợp này là kinh nghiệm của bạn bè chúng ta).

Học máy đại diện cho phương pháp thứ hai: đưa ra quyết định bằng kinh nghiệm của chúng ta. Trong biệt ngữ máy tính, thuật ngữ *trải nghiệm* là *dữ liệu*. Vì vậy, trong học máy, máy tính đưa ra quyết định dựa trên dữ liệu. Vì vậy, bất cứ khi nào chúng ta có máy tính để giải quyết một vấn đề hoặc đưa ra quyết định chỉ sử dụng dữ liệu, chúng ta đang thực hiện học máy. Thông thường, chúng ta có thể mô tả học máy theo cách sau:

Học máy là lẽ thường, ngoại trừ việc được thực hiện bởi máy tính.

Đi từ việc giải quyết vấn đề bằng bất kỳ phương tiện cần thiết nào đến giải quyết vấn đề chỉ sử dụng dữ liệu có thể tưởng như là một bước nhỏ đối với máy tính nhưng lại là một bước tiến lớn đối với nhân loại (hình 1.2). Ngày xưa ngày xưa, nếu chúng ta muốn một chiếc máy tính thực hiện một nhiệm vụ, chúng ta phải viết một chương trình, cụ thể là cả một bộ hướng dẫn để máy tính tuân theo. Quá trình này tốt cho các tác vụ đơn giản, nhưng một số tác vụ

quá phức tạp đối với framework này. Ví dụ: hãy xem xét nhiệm vụ xác định xem một hình ảnh có chứa quả táo hay không. Nếu chúng ta bắt đầu viết một chương trình máy tính để phát triển nhiệm vụ này, chúng ta sẽ nhanh chóng nhận ra rằng nó thật khó.



Hình 1.2 Học máy bao gồm tất cả các nhiệm vụ trong đó máy tính đưa ra quyết định dựa trên dữ liệu. Giống như cách con người đưa ra quyết định dựa trên kinh nghiệm trước đó, máy tính có thể đưa ra quyết định dựa trên dữ liệu trước đó.

Hãy lùi lại một bước và đặt câu hỏi sau đây. Làm thế nào chúng ta, với tư cách là con người, biết được hình dạng của một quả táo? Cách chúng ta học hầu hết các từ không phải nhờ ai đó giải thích cho chúng ta ý nghĩa của chúng; chúng tôi đã học chúng bằng cách lặp lại. Chúng ta đã nhìn thấy nhiều đồ vật trong thời thơ ấu và người lớn sẽ cho chúng ta biết những đồ vật này là gì. Để tìm hiểu quả táo là gì, chúng tôi đã nhìn thấy rất nhiều quả táo trong suốt nhiều năm khi nghe từ *quả táo*, cho đến một ngày nó kêu lên và chúng tôi biết quả táo là gì. Trong học máy, đó là những gì chúng ta yêu cầu máy tính thực hiện. Chúng tôi hiển thị cho máy tính nhiều hình ảnh và cho nó biết hình ảnh nào có quả táo (tạo thành dữ liệu của chúng tôi). Chúng tôi lặp lại quá trình này cho đến khi máy tính nắm bắt được đúng mẫu và thuộc tính tạo nên một quả táo. Khi kết thúc quá trình, khi chúng tôi đưa vào máy tính một hình ảnh mới, nó có thể sử dụng các mẫu này để xác định xem hình ảnh có chứa quả táo hay không. Tất nhiên, chúng ta vẫn cần lập trình cho máy tính để nó nắm bắt được những mẫu hình này. Để làm được điều đó, chúng ta có một số kỹ thuật mà chúng ta sẽ học trong cuốn sách này.

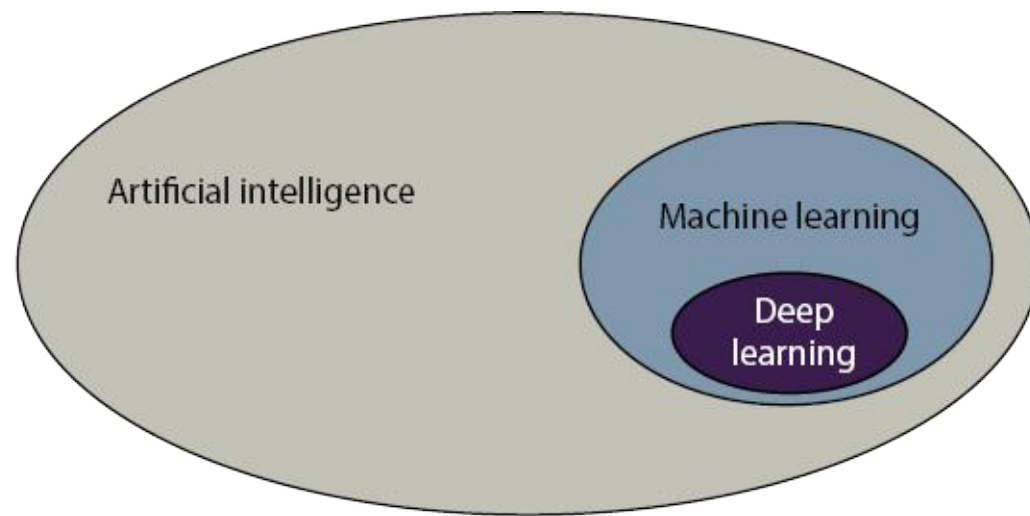
Và bây giờ chúng ta đã đến phần đó, học sâu là gì?

Giống như học máy là một phần của trí tuệ nhân tạo, học sâu là một phần của học máy. Trong phần trước, chúng ta đã biết mình có một số kỹ thuật sử dụng để khiến máy tính học từ dữ liệu. Một trong những kỹ thuật này đã hoạt động rất tốt nên nó có lĩnh vực nghiên cứu riêng

được gọi là *học sâu* (DL), mà chúng tôi định nghĩa như sau và được minh họa trong hình 1.3:

học sâu Lĩnh vực học máy sử dụng một số đối tượng nhất định được gọi là *mạng lưới thần kinh*

Mạng lưới thần kinh là gì? Chúng ta sẽ tìm hiểu về chúng trong chương 10. Học sâu được cho là loại học máy được sử dụng nhiều nhất vì nó hoạt động rất tốt. Nếu chúng ta đang xem xét bất kỳ ứng dụng tiên tiến nào, chẳng hạn như nhận dạng hình ảnh, tạo văn bản, chơi cờ vây hoặc ô tô tự lái, rất có thể chúng ta đang xem xét học sâu theo cách này hay cách khác.



Hình 1.3 Học sâu là một phần của học máy.

Nói cách khác, deep learning là một phần của machine learning, và machine learning là một phần của trí tuệ nhân tạo. Nếu cuốn sách này nói về phương tiện giao thông thì AI sẽ là phương tiện, ML sẽ là ô tô và DL sẽ là Ferrari.

Làm cách nào để máy móc đưa ra quyết định dựa trên dữ liệu? Khung ghi nhớ-công thức-dự đoán

Trong phần trước, chúng ta đã thảo luận rằng học máy bao gồm một tập hợp các kỹ thuật mà chúng ta sử dụng để khiến máy tính đưa ra quyết định dựa trên dữ liệu. Trong phần này, chúng ta tìm hiểu ý nghĩa của việc đưa ra quyết định dựa trên dữ liệu và cách thức hoạt động của một số kỹ thuật này. Để làm được điều này, hãy phân tích lại quy trình con người sử dụng để đưa ra quyết định dựa trên kinh nghiệm. Đây là cái được gọi là **khuôn khổ ghi nhớ-xây dựng-dự đoán**, được minh họa trong hình 1.4. Mục tiêu của học máy là

dạy máy tính cách suy nghĩ theo cùng một cách, theo cùng một khuôn khổ.

Con người suy nghĩ như thế nào?

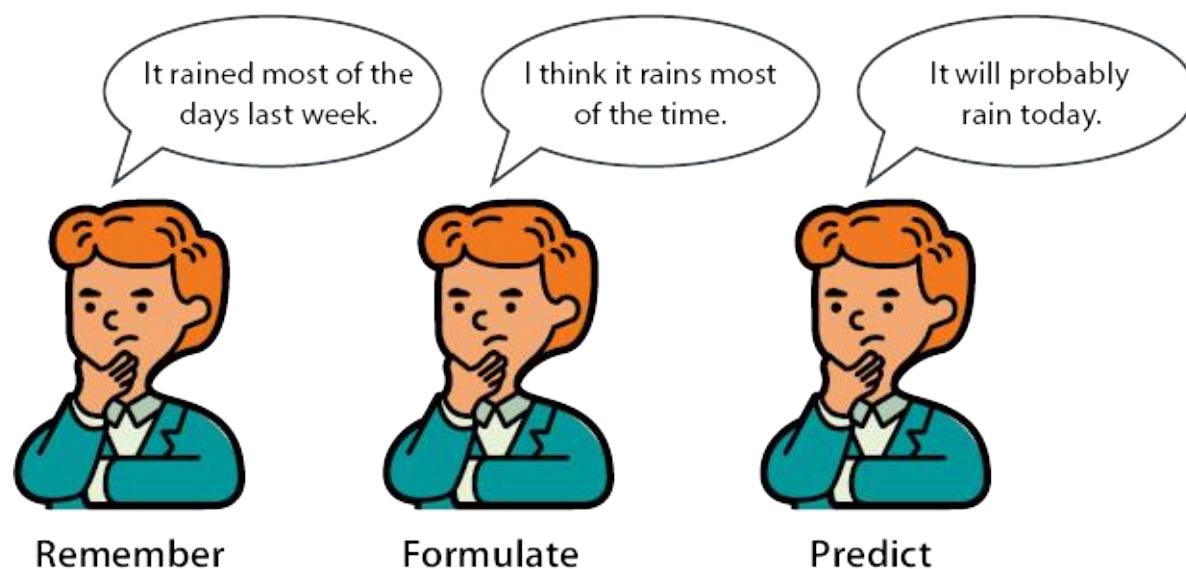
Với tư cách là con người, khi chúng ta cần đưa ra quyết định dựa trên kinh nghiệm của mình, chúng ta thường sử dụng khuôn khổ sau:

1. Chúng tôi **nhớ** những tình huống tương tự trong quá khứ.
2. Chúng tôi **xây dựng** một quy tắc chung.
3. Chúng tôi sử dụng quy tắc này để **dự đoán** điều gì có thể xảy ra trong tương lai.

Ví dụ: nếu câu hỏi là “Hôm nay trời có mưa không?”, quy trình đoán như sau:

1. Chúng tôi **nhớ** rằng tuần trước trời hầu như mưa.
2. Chúng tôi **cho rằng** rằng ở nơi này trời hầu như lúc nào cũng mưa.
3. Chúng tôi **dự đoán** hôm nay trời sẽ mưa.

Chúng tôi có thể đúng hoặc sai, nhưng ít nhất chúng tôi đang cố gắng đưa ra dự đoán chính xác nhất có thể dựa trên thông tin chúng tôi có.



Hình 1.4 Khung ghi nhớ-công thức-dự đoán là khung chính chúng tôi sử dụng trong cuốn sách này. Nó bao gồm ba bước: (1) Chúng tôi nhớ dữ liệu trước đó; (2) chúng tôi xây dựng một quy tắc chung; và (3) chúng ta sử dụng quy tắc đó để đưa ra dự đoán về tương lai.

Một số thuật ngữ học máy—mô hình và

thuật toán

Trước khi đi sâu vào các ví dụ minh họa các kỹ thuật được sử dụng trong học máy, hãy định nghĩa một số thuật ngữ hữu ích mà chúng tôi sử dụng trong suốt cuốn sách này. Chúng tôi biết rằng trong học máy, chúng tôi yêu cầu máy tính học cách giải quyết vấn đề bằng cách sử dụng dữ liệu. Cách máy tính giải quyết vấn đề là sử dụng dữ liệu để **xây dựng mô hình**. Mô hình là gì? Chúng tôi xác định một mô hình như sau:

mô hình Một bộ quy tắc thể hiện dữ liệu của chúng tôi và có thể được sử dụng để đưa ra dự đoán

Chúng ta có thể coi một mô hình như một sự thể hiện thực tế bằng cách sử dụng một bộ quy tắc bắt chước dữ liệu hiện có càng sát càng tốt. Trong ví dụ về mưa ở phần trước, mô hình này là sự thể hiện của chúng ta về thực tế, đó là một thế giới mà trời mưa hầu hết thời gian. Đây là một thế giới đơn giản với một quy tắc: trời mưa hầu hết thời gian. Cách trình bày này có thể chính xác hoặc không, nhưng theo dữ liệu của chúng tôi, đó là cách trình bày chính xác nhất về thực tế mà chúng tôi có thể hình thành. Sau này chúng tôi sử dụng quy tắc này để đưa ra dự đoán về dữ liệu chưa nhìn thấy.

thuật toán là quy trình chúng tôi sử dụng để xây dựng mô hình. Trong ví dụ hiện tại, quy trình này rất đơn giản: chúng tôi xem xét số ngày trời mưa và nhận ra đó là phần lớn. Tất nhiên, các thuật toán học máy có thể phức tạp hơn thế nhiều, nhưng xét cho cùng, chúng luôn bao gồm một tập hợp các bước. Định nghĩa của chúng tôi về thuật toán như sau:

thuật toán Một quy trình hoặc một tập hợp các bước được sử dụng để giải quyết một vấn đề hoặc thực hiện tính toán. Trong cuốn sách này, mục tiêu của thuật toán là xây dựng một mô hình.

Nói tóm lại, mô hình là thứ chúng tôi sử dụng để đưa ra dự đoán và thuật toán là thứ chúng tôi sử dụng để xây dựng mô hình. Hai định nghĩa này rất dễ nhầm lẫn và thường được thay thế cho nhau, nhưng để hiểu rõ hơn, chúng ta hãy xem một vài ví dụ.

Một số ví dụ về mô hình mà con người sử dụng

Trong phần này chúng ta tập trung vào một ứng dụng phổ biến của học máy: phát hiện thư rác. Trong các ví dụ sau, chúng tôi sẽ phát hiện email spam và không phải thư rác. Những email không phải thư rác còn được gọi là *ham*.

spam và hamspam là thuật ngữ phổ biến được sử dụng cho email rác hoặc email không mong muốn, chẳng hạn như thư dây chuyền, quảng cáo, v.v. Thuật ngữ này xuất phát từ Monty năm 1972

Bản phác thảo Python trong đó mọi món trong thực đơn của một nhà hàng đều chứa Thư rác như một thành phần. Trong số các nhà phát triển phần mềm, thuật ngữ *ham* được dùng để chỉ những email không phải thư rác.

VÍ DỤ 1: MỘT NGƯỜI BẠN QUA EMAIL PHÁP LÝ

Trong ví dụ này, người bạn Bob của chúng tôi thích gửi email cho chúng tôi. Rất nhiều email của anh ấy là thư rác, dưới dạng các chuỗi thư. Chúng tôi bắt đầu thấy hơi khó chịu với anh ấy. Hôm nay là thứ Bảy và chúng tôi vừa nhận được thông báo về email từ Bob. Chúng ta có thể đoán được email này là thư rác hay ham muốn mà không cần nhìn vào nó không?

Để tìm ra điều này, chúng tôi sử dụng phương pháp ghi nhớ-công thức-dự đoán. Đầu tiên, chúng ta hãy **nhớ** chẳng hạn như 10 email gần đây nhất mà chúng ta nhận được từ Bob. Đó là dữ liệu của chúng tôi. Chúng tôi nhớ rằng sáu trong số đó là thư rác và bốn trong số đó là giảm bông. Từ thông tin này, chúng ta có thể **xây dựng** mô hình sau:

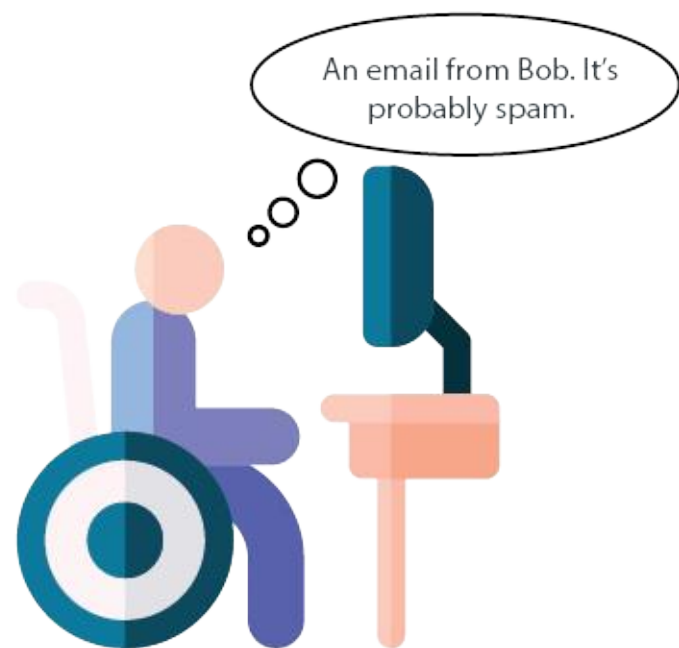
Mô hình 1: Cứ 10 email Bob gửi cho chúng tôi thì có 6 email là thư rác.

Quy tắc này sẽ là mô hình của chúng tôi. Lưu ý, quy tắc này không cần phải đúng. Nó có thể cực kỳ sai lầm. Nhưng với dữ liệu của chúng tôi, đó là dữ liệu tốt nhất mà chúng tôi có thể nghĩ ra, vì vậy chúng tôi sẽ chấp nhận nó. Ở phần sau của cuốn sách này, chúng ta học cách đánh giá các mô hình và cải thiện chúng khi cần thiết.

Bây giờ chúng ta đã có quy tắc, chúng ta có thể sử dụng nó để **dự đoán** liệu email đó có phải là thư rác hay không. Nếu 6 trong số 10 email của Bob là thư rác thì chúng ta có thể giả định rằng email mới này có 60% là thư rác và 40% có khả năng là thư rác. Đánh giá theo quy tắc này, sẽ an toàn hơn một chút khi nghĩ rằng email đó là thư rác. Vì vậy, chúng tôi dự đoán email đó là thư rác (hình 1.5).

Một lần nữa, dự đoán của chúng tôi có thể sai. Chúng tôi có thể mở email và nhận ra đó là ham. Nhưng chúng tôi đã đưa ra dự đoán *theo hiểu biết tốt nhất của mình*. Đây chính là nội dung của máy học.

Có thể bạn đang nghĩ, chúng ta có thể làm tốt hơn không? Có vẻ như chúng tôi đang đánh giá mọi email từ Bob theo cùng một cách, nhưng có thể có nhiều thông tin hơn có thể giúp chúng tôi phân biệt email spam và email ham. Hãy thử phân tích các email nhiều hơn một chút. Ví dụ: hãy xem thời điểm Bob gửi email để xem liệu chúng tôi có tìm thấy mẫu hay không.



Hình 1.5 Một mô hình học máy rất đơn giản

VÍ DỤ 2: MỘT NGƯỜI BẠN EMAIL PHÁT HIỆN THEO MÙA

Hãy xem xét kỹ hơn những email mà Bob đã gửi cho chúng ta vào tháng trước. Cụ thể hơn, chúng ta sẽ xem anh ấy đã gửi chúng vào ngày nào. Dưới đây là các email có ngày tháng và thông tin về việc bị spam hoặc ham:

- Thứ hai: Ham Thứ
- ba: Ham Thứ bảy:
- Thứ rác Chủ nhật:
- Thứ rác Chủ nhật:
- Thứ rác Thứ tư: Ham
- Thứ sáu: Ham Thứ
- bảy: Thứ rác Thứ ba:
- Ham Thứ năm: Ham
-
-
-

Bây giờ mọi chuyện đã khác. Bạn có thể nhìn thấy một mô hình? Có vẻ như mọi email Bob gửi trong tuần đều là email tôi, và mọi email anh ấy gửi vào cuối tuần đều là thư rác. Điều này có lý—có thể trong tuần anh ấy gửi email công việc cho chúng tôi, trong khi vào cuối tuần, anh ấy có thời gian gửi thư rác và quyết định tự do chuyển vùng. Vì vậy, chúng ta có thể **xây dựng** một quy tắc hoặc mô hình có tính giáo dục cao hơn như sau:

Mẫu 2: Đêm giao thừa email mà Bob gửi trong tuần là email tôi tệ và những email anh ấy gửi

vào cuối tuần là thư rác.

Bây giờ chúng ta hãy xem hôm nay là ngày gì. Nếu hôm đó là Chủ nhật và chúng ta vừa nhận được email từ Bob thì chúng ta có thể **dự đoán** một cách chắc chắn rằng email anh ta gửi là thư rác (hình 1.6). Chúng ta đưa ra dự đoán này và không cần nhìn, chúng ta gửi email vào thùng rác và tiếp tục công việc hàng ngày của mình.



Hình 1.6 Một mô hình học máy phức tạp hơn một chút

VÍ DỤ 3: Mọi chuyện đang trở nên phức tạp!

Bây giờ, giả sử chúng ta tiếp tục với quy tắc này và một ngày nọ, chúng ta nhìn thấy Bob trên phố và anh ấy hỏi: "Tại sao bạn không đến dự tiệc sinh nhật của tôi?" Chúng tôi không biết anh ấy đang nói về điều gì. Hóa ra Chủ nhật tuần trước anh ấy đã gửi cho chúng tôi lời mời đến dự tiệc sinh nhật của anh ấy và chúng tôi đã bỏ lỡ! Tại sao chúng ta lại bỏ lỡ nó? Bởi vì anh ấy đã gửi nó vào cuối tuần và chúng tôi cho rằng đó sẽ là thư rác. Có vẻ như chúng ta cần một mô hình tốt hơn. Hãy quay lại xem email của Bob—đây là bước **ghi nhớ** của chúng ta. Hãy xem liệu chúng ta có thể tìm thấy một mô hình hay không.

- 1 KB: Hàm 2
- KB: Hàm 16
- KB: Thư rác 20
- KB: Thư rác 18
- KB: Thư rác 3
- KB: Hàm 5 KB:
- Hàm
-

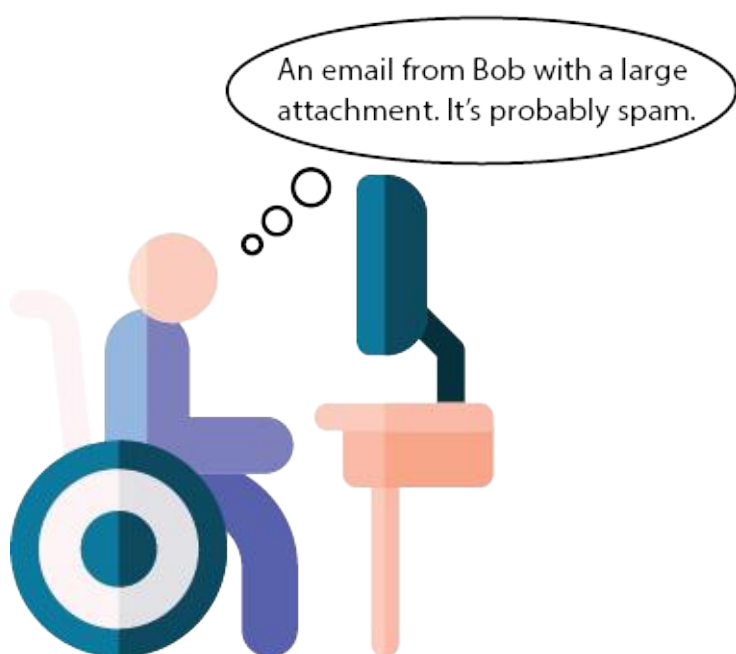
- 25 KB: Thư rác
- 1 KB: Hàm 3
- KB: Hàm

Chúng ta thấy gì? Có vẻ như những email lớn có xu hướng là thư rác, trong khi những email nhỏ hơn có xu hướng trở thành thư rác. Điều này có ý nghĩa vì các email spam thường có tệp đính kèm lớn.

Vì vậy, chúng ta có thể **xây dựng** quy tắc sau:

Mô hình 3: Bất kỳ email nào có kích thước 10 KB trở lên là thư rác và bất kỳ email nào có kích thước nhỏ hơn 10 KB đều là ham.

Bây giờ chúng ta đã xây dựng được quy tắc của mình, chúng ta có thể đưa ra **dự đoán**. Chúng tôi xem email chúng tôi nhận được hôm nay từ Bob và kích thước là 19 KB. Vì vậy, chúng tôi kết luận đó là thư rác (hình 1.7).



Hình 1.7 Một mô hình học máy khác phức tạp hơn một chút

Đây là kết thúc của câu chuyện phải không? Thậm chí không gần gũi.

Nhưng trước khi tiếp tục, hãy lưu ý rằng để đưa ra dự đoán, chúng tôi đã sử dụng ngày trong tuần và kích thước của email. Đây là những ví dụ về **tính năng**. Một tính năng là một trong những khái niệm quan trọng nhất trong cuốn sách này.

tính năng Bất kỳ thuộc tính hoặc đặc điểm nào của dữ liệu mà mô hình có thể sử dụng để đưa ra dự đoán

Bạn có thể tưởng tượng rằng còn có nhiều tính năng khác có thể cho biết email có phải là thư rác hay không. Bạn có thể nghĩ thêm được không? Trong các đoạn tiếp theo, chúng ta sẽ thấy thêm một số tính năng.

VÍ DỤ 4: THÊM?

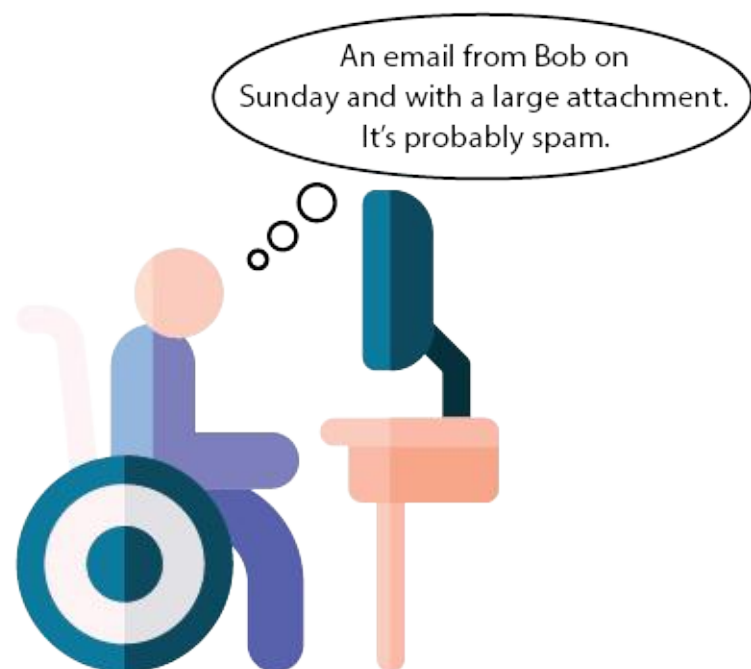
Hai trình phân loại của chúng tôi hoạt động tốt vì chúng loại trừ các email lớn và email được gửi vào cuối tuần. Mỗi người trong số họ sử dụng chính xác một trong hai tính năng này. Nhưng điều gì sẽ xảy ra nếu chúng ta muốn một quy tắc hoạt động với cả hai tính năng? Các quy tắc như sau có thể hoạt động:

Mô hình 4: Nếu email có dung lượng lớn hơn 10 KB hoặc được gửi vào cuối tuần thì email đó được phân loại là thư rác. Nếu không, nó được phân loại là giảm bông.

Mẫu 5: Nếu email được gửi trong tuần thì phải lớn hơn 15 KB mới bị phân loại là thư rác. Nếu nó được gửi vào cuối tuần thì nó phải lớn hơn 5 KB để được phân loại là thư rác. Nếu không, nó được phân loại là giảm bông.

Hoặc chúng ta có thể còn phức tạp hơn nữa.

Mẫu 6: Xét số ngày, trong đó Thứ Hai là 0, Thứ Ba là 1, Thứ Tư là 2, Thứ Năm là 3, Thứ Sáu là 4, Thứ Bảy là 5 và Chủ Nhật là 6. Nếu cộng số ngày và kích thước của email (tính bằng KB) và kết quả là 12 trở lên thì email đó được phân loại là thư rác (hình 1.8). Nếu không, nó được phân loại là giảm bông.



Hình 1.8 Một mô hình học máy thậm chí còn phức tạp hơn

Tất cả đều là mô hình hợp lệ. Chúng ta có thể tiếp tục tạo ra ngày càng nhiều mô hình bằng cách thêm các lớp phức tạp hoặc bằng cách xem xét nhiều tính năng hơn nữa. Bây giờ câu hỏi là, mô hình nào là tốt nhất? Đây là lúc chúng ta bắt đầu cần đến sự trợ giúp của máy tính.

S một số ví dụ về kiểu máy mà máy sử dụng

Mục tiêu là làm cho máy tính suy nghĩ theo cách chúng ta nghĩ, cụ thể là sử dụng khuôn khổ ghi nhớ-công thức hóa-dự đoán. Tóm lại, đây là những gì máy tính thực hiện trong mỗi bước:

Ghi nhớ: Nhìn vào một bảng dữ liệu khổng lồ.

Công thức: Tạo mô hình bằng cách xem xét nhiều quy tắc và công thức, đồng thời kiểm tra xem mô hình nào phù hợp với dữ liệu nhất.

Dự đoán: Sử dụng mô hình để đưa ra dự đoán về dữ liệu trong tương lai.

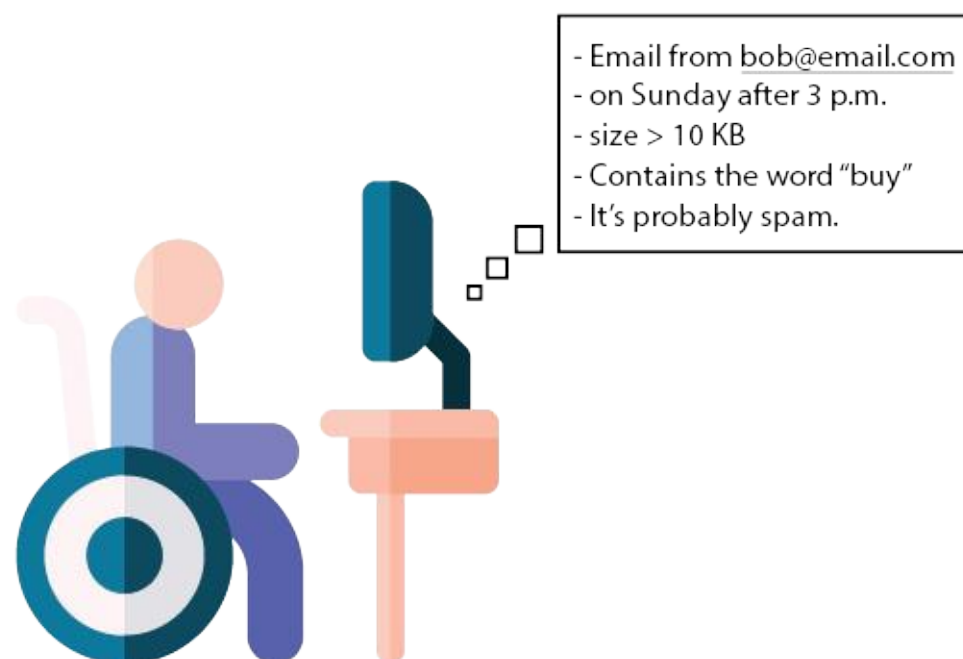
Quá trình này không khác nhiều so với những gì chúng tôi đã làm trong phần trước. Sự tiến bộ lớn ở đây là máy tính có thể xây dựng các mô hình một cách nhanh chóng bằng cách thực hiện nhiều công thức và kết hợp các quy tắc cho đến khi tìm thấy một mô hình phù hợp với dữ liệu hiện có. Ví dụ: chúng tôi có thể xây dựng trình phân loại thư rác với các tính năng như người gửi, ngày và giờ trong ngày, số từ, số lỗi chính tả và sự xuất hiện của một số từ nhất định như *mua* hoặc *thắng*. Một mô hình có thể dễ dàng trông giống như câu lệnh logic sau:

Mẫu 7:

- Nếu email có từ hai lỗi chính tả trở lên thì email đó được phân loại là thư rác. Nếu nó có tệp đính kèm lớn hơn 10 KB thì nó được phân loại là thư rác. Nếu người gửi không có trong danh sách liên hệ của chúng tôi, nó sẽ được phân loại là thư rác. Nếu có dòng chữ *mua* và *thắng* thì bị coi là thư rác. Nếu không, nó được phân loại là giảm bông.
-

Nó cũng có thể trông giống như công thức sau:

Mẫu 8: Nếu (kích thước) + 10 (số lỗi chính tả) – (số lần xuất hiện của từ “mẹ”) + 4 (số lần xuất hiện của từ “mua”) > 10 thì chúng tôi phân loại tin nhắn là thư rác (hình 1.9). Nếu không, chúng tôi phân loại nó là giảm bông.



Hình 1.9 Một mô hình học máy phức tạp hơn nhiều, được tìm thấy bởi máy tính

Bây giờ câu hỏi là, quy tắc nào là tốt nhất? Câu trả lời nhanh là câu trả lời phù hợp nhất với dữ liệu, mặc dù câu trả lời thực sự là câu trả lời khái quát nhất cho dữ liệu mới. Vào cuối ngày, chúng tôi

có thể đưa ra một quy tắc phức tạp, nhưng máy tính có thể xây dựng và sử dụng nó để đưa ra dự đoán một cách nhanh chóng. Câu hỏi tiếp theo của chúng ta là làm cách nào để xây dựng mô hình tốt nhất? Đó chính xác là những gì cuốn sách này nói về.

Tóm tắt

- Học máy thật dễ dàng! Bất cứ ai cũng có thể học và sử dụng nó, bất kể nền tảng của họ. Tất cả những gì cần thiết là mong muốn học hỏi và những ý tưởng tuyệt vời để thực hiện! Học máy cực kỳ hữu ích và được sử dụng trong hầu hết các ngành. Từ khoa học đến công nghệ đến các vấn đề xã hội và y học, học máy đang tạo ra ảnh hưởng và sẽ tiếp tục như vậy. Học máy là lẽ thường, được thực hiện bởi máy tính. Nó bắt chước cách con người suy nghĩ để đưa ra quyết định nhanh chóng và chính xác. Giống như con người đưa ra quyết định dựa trên kinh nghiệm, máy tính có thể đưa ra quyết định dựa trên dữ liệu trước đó. Đây chính là nội dung của máy học.

•

•

Máy móc việc học sử dụng khuôn khổ ghi nhớ-công thức-dự đoán, như sau :

- **Ghi nhớ:** xem dữ liệu trước đó. **Xây dựng:** xây dựng mô hình hoặc quy tắc dựa trên dữ liệu này. **Dự đoán:** sử dụng mô hình để đưa ra dự đoán về dữ liệu trong tương lai.

2 loại hình học máy

Trong chương này

- ba loại học máy khác nhau: học có giám sát, không giám sát và học tăng cường sự khác biệt giữa dữ liệu được gắn nhãn và không được gắn nhãn sự khác biệt giữa hồi quy và phân loại
- cũng như cách chúng được sử dụng
-



Như chúng ta đã học ở chương 1, học máy là điều bình thường đối với máy tính. Học máy gần như bắt chước quá trình con người đưa ra quyết định dựa trên kinh nghiệm, bằng cách đưa ra quyết định dựa trên dữ liệu trước đó.Đương nhiên, việc lập trình máy tính để bắt chước quá trình suy nghĩ của con người là một thách thức, bởi vì máy tính được thiết kế để lưu trữ và xử lý các con số chứ không phải đưa ra quyết định.Đây là nhiệm vụ mà học máy hướng tới giải quyết.Học máy được chia thành nhiều nhánh, tùy thuộc vào loại quyết định được đưa ra.Trong chương này, chúng ta sẽ xem xét tổng quan một số nhánh quan trọng nhất trong số các nhánh này.

Machine learning có ứng dụng trong nhiều lĩnh vực, chẳng hạn như sau

:

- Dự đoán giá nhà dựa trên quy mô, số phòng và vị trí của ngôi nhà Dự đoán giá thị trường chứng khoán ngày hôm nay dựa trên giá của ngày hôm qua và các yếu tố khác của thị trường Phát hiện email rác và không phải thư rác dựa trên các từ trong e-mail và người gửi
-

- Nhận dạng hình ảnh là khuôn mặt hoặc động vật, dựa trên các pixel trong hình ảnh Xử lý tài
- liệu văn bản dài và xuất ra bản tóm tắt Đề xuất video hoặc phim cho người dùng (ví dụ: trên
- YouTube hoặc Netflix) Xây dựng chatbot tương tác với con người và trả lời câu hỏi Đào tạo
- xe tự lái để tự điều hướng trong thành phố Chẩn đoán bệnh nhân ốm yếu hoặc khỏe mạnh
- Phân chia thị trường thành các nhóm tương tự dựa trên vị trí, khả năng tiếp thu và sở thích
- Chơi các trò chơi như cờ vua hoặc cờ vây
-
-
-

Hãy thử tưởng tượng xem chúng ta có thể sử dụng máy học như thế nào trong từng lĩnh vực này. Lưu ý rằng một số ứng dụng này khác nhau nhưng có thể được giải quyết theo cách tương tự. Ví dụ: dự đoán giá nhà đất và dự đoán giá cổ phiếu có thể được thực hiện bằng các kỹ thuật tương tự. Tương tự như vậy, việc dự đoán liệu một email có phải là thư rác hay không và dự đoán liệu giao dịch thẻ tín dụng là hợp pháp hay gian lận cũng có thể được thực hiện bằng các kỹ thuật tương tự. Còn việc phân nhóm người dùng ứng dụng dựa trên điểm giống nhau của họ thì sao? Điều đó nghe có vẻ khác với việc dự đoán giá nhà đất, nhưng nó có thể được thực hiện tương tự như việc nhóm các bài báo theo chủ đề. Còn việc chơi cờ thì sao? Điều đó nghe có vẻ khác với tất cả các ứng dụng trước đây nhưng nó có thể giống như chơi cờ vây.

Các mô hình học máy được nhóm thành nhiều loại khác nhau tùy theo cách chúng vận hành. Ba họ mô hình học máy chính là

- *học có giám sát, học không giám*
- *sát và học tăng cường.*
-

Trong chương này, chúng tôi tổng quan về cả ba. Tuy nhiên, trong cuốn sách này, chúng tôi chỉ tập trung vào học có giám sát vì đây là cách học tự nhiên nhất để bắt đầu học và được cho là được sử dụng nhiều nhất hiện nay. Hãy tra cứu các loại khác trong tài liệu và tìm hiểu về chúng vì chúng đều thú vị và hữu ích! Trong các tài nguyên ở phụ lục C, bạn có thể tìm thấy một số liên kết thú vị, bao gồm một số video do tác giả tạo.

Sự khác biệt giữa dữ liệu được gắn nhãn và không được gắn nhãn là gì?

Dữ liệu là gì?

Chúng ta đã nói về dữ liệu trong chương 1, nhưng trước khi đi xa hơn, trước tiên hãy thiết lập một định nghĩa rõ ràng về *dữ liệu* trong cuốn sách này. Dữ liệu chỉ đơn giản là thông tin. Bất cứ khi nào chúng ta có một bảng chứa thông tin, chúng ta có dữ liệu. Thông thường, mỗi hàng trong bảng của chúng ta là một điểm dữ liệu. Ví dụ: giả sử chúng ta có một tập dữ liệu về thú cưng. Trong trường hợp này, mỗi hàng đại diện cho một con vật cưng khác nhau. Mỗi thú cưng trong bảng được mô tả bằng những đặc điểm nhất định của thú cưng đó.

Và tính năng là gì?

Trong chương 1, chúng tôi đã định nghĩa các đặc điểm là các thuộc tính hoặc đặc điểm của dữ liệu. Nếu dữ liệu của chúng ta nằm trong một bảng thì các đối tượng sẽ là các cột của bảng. Trong ví dụ thú cưng của chúng tôi, các đặc điểm có thể là kích thước, tên, loại hoặc trọng lượng. Các đặc điểm thậm chí có thể là màu sắc của các pixel trong hình ảnh của thú cưng. Đây là những gì mô tả dữ liệu của chúng tôi. Tuy nhiên, một số tính năng rất đặc biệt và chúng tôi gọi chúng là *nhãn*.

Nhãn?

Cách này ít đơn giản hơn một chút vì nó phụ thuộc vào bối cảnh của vấn đề mà chúng ta đang cố gắng giải quyết. Thông thường, nếu chúng ta đang cố gắng dự đoán một đặc điểm cụ thể dựa trên những đặc điểm khác thì đặc điểm đó là nhãn. Nếu chúng ta đang cố gắng dự đoán loại thú cưng (ví dụ: mèo hoặc chó) dựa trên thông tin về thú cưng đó thì nhãn là loại thú cưng (mèo/chó). Nếu chúng tôi đang cố gắng dự đoán liệu thú cưng bị bệnh hay khỏe mạnh dựa trên các triệu chứng và thông tin khác thì nhãn là trạng thái của thú cưng (ốm/khỏe). Nếu chúng ta đang cố gắng dự đoán tuổi của thú cưng thì nhãn là tuổi (một con số).

Dự đoán

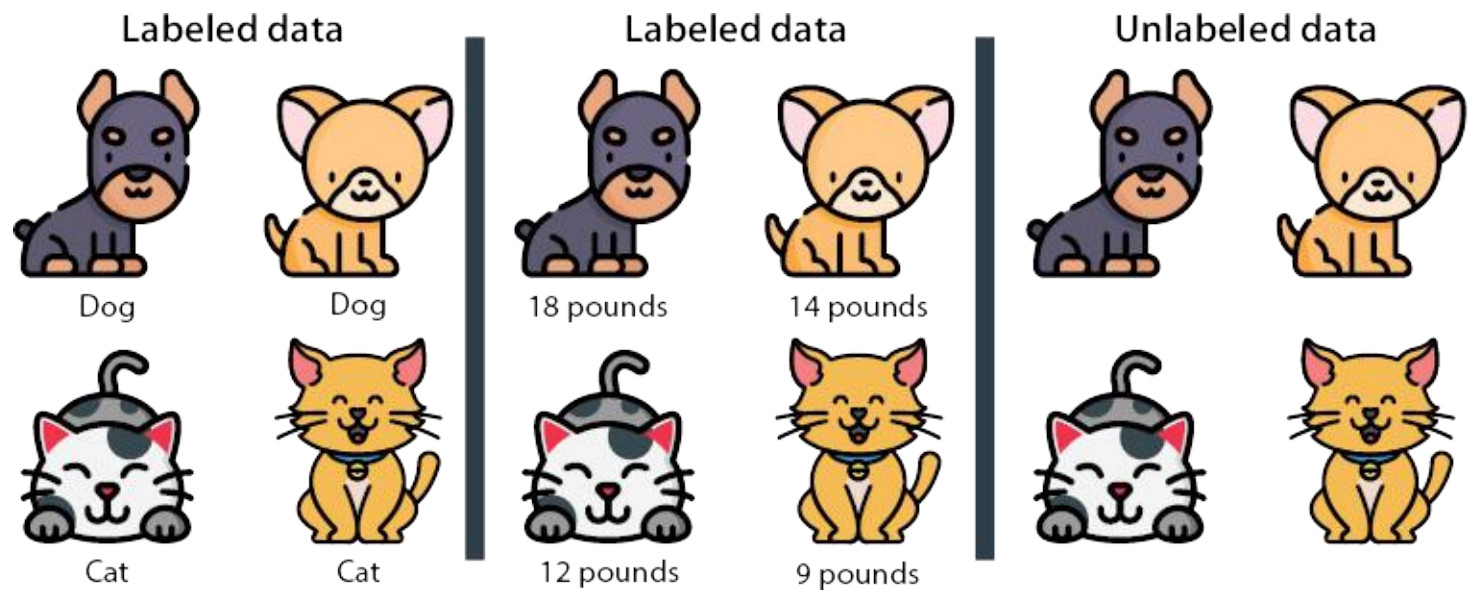
Chúng ta đang sử dụng khái niệm đưa ra dự đoán một cách tự do, nhưng bây giờ chúng ta hãy ghi rõ nó lại. Mục tiêu của mô hình học máy dự đoán là đoán các nhãn trong dữ liệu. Dự đoán mà mô hình đưa ra được gọi là *dự đoán*.

Bây giờ chúng ta đã biết nhãn là gì, chúng ta có thể hiểu có hai loại dữ liệu chính: dữ liệu *được gắn nhãn* và *không được gắn nhãn*.

Dữ liệu được gắn nhãn và không được gắn nhãn

Dữ liệu được gắn nhãn là dữ liệu đi kèm với nhãn. Dữ liệu không được gắn nhãn là dữ liệu không có nhãn. Một ví dụ về dữ liệu được gắn nhãn là một tập dữ liệu gồm các email đi kèm với một cột ghi lại xem các email đó là thư rác hay ham muốn hay một cột ghi lại xem email đó có liên quan đến công việc hay không. Một ví dụ về dữ liệu không được gắn nhãn là tập dữ liệu gồm các email không có cột cụ thể mà chúng tôi muốn dự đoán.

Trong hình 2.1, chúng ta thấy ba tập dữ liệu chứa hình ảnh của thú cưng. Tập dữ liệu đầu tiên có một cột ghi lại loại thú cưng và tập dữ liệu thứ hai có một cột chỉ định trọng lượng của thú cưng. Hai cái này là ví dụ về dữ liệu được dán nhãn. Tập dữ liệu thứ ba chỉ bao gồm các hình ảnh, không có nhãn, khiến nó trở thành dữ liệu không được gắn nhãn.



Hình 2.1 Dữ liệu được gắn nhãn là dữ liệu đi kèm với thẻ hoặc nhãn. Nhãn đó có thể là một loại hoặc một số. Dữ liệu không được gắn nhãn là dữ liệu không có thẻ. Tập dữ liệu bên trái được dán nhãn và nhãn là loại vật nuôi (chó/mèo). Tập dữ liệu ở giữa cũng được dán nhãn và nhãn là trọng lượng của thú cưng (tính bằng pound). Tập dữ liệu bên phải không được gắn nhãn.

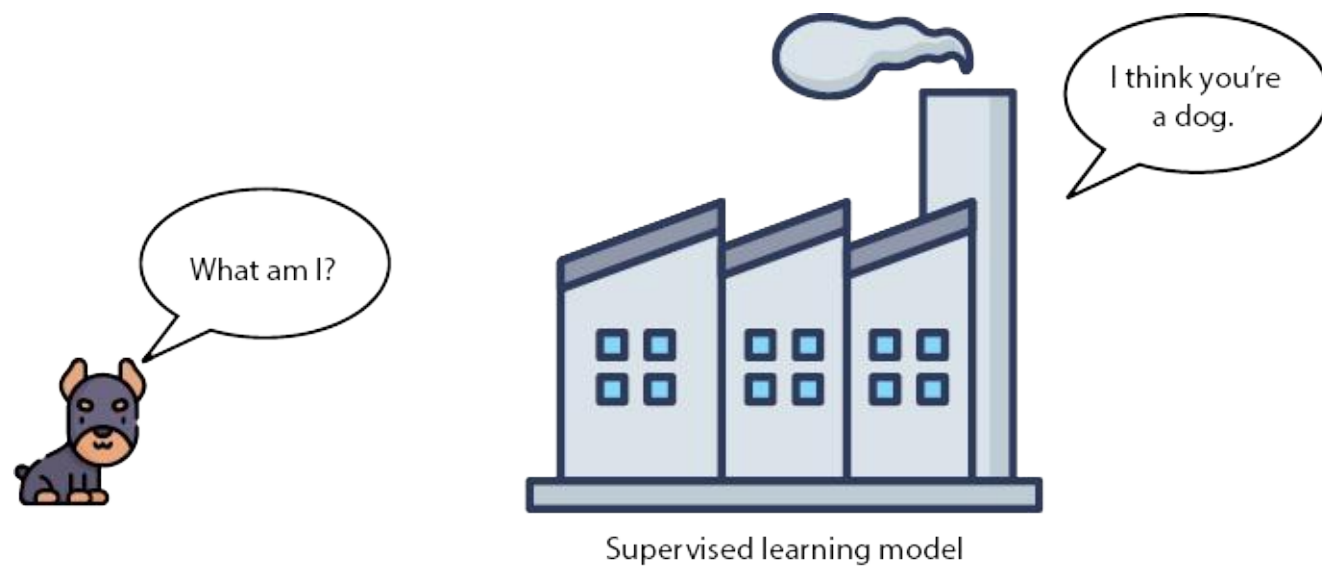
Tất nhiên, định nghĩa này có một số điểm mơ hồ, bởi vì tùy thuộc vào vấn đề, chúng ta quyết định liệu một tính năng cụ thể có đủ tiêu chuẩn làm nhãn hay không. Do đó, việc xác định xem dữ liệu được gắn nhãn hay không được gắn nhãn nhiều lần phụ thuộc vào vấn đề mà chúng ta đang cố gắng giải quyết.

Dữ liệu được gắn nhãn và không được gắn nhãn tạo ra hai nhánh khác nhau của máy học được gọi là *học được giám sát* và *không được giám sát*, được xác định trong ba phần tiếp theo.

Học có giám sát: Nhánh học máy hoạt động với dữ liệu được gắn nhãn

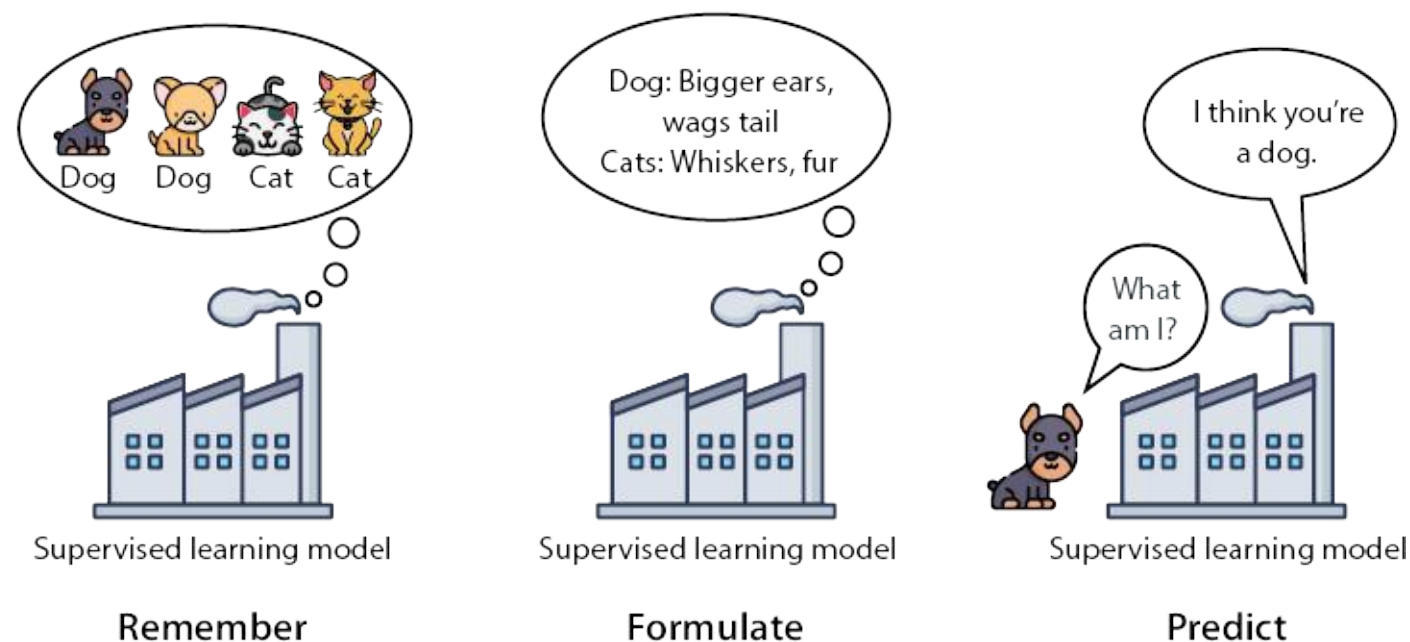
Chúng ta có thể tìm thấy việc học có giám sát trong một số ứng dụng phổ biến nhất hiện nay, bao gồm nhận dạng hình ảnh, các hình thức xử lý văn bản khác nhau và hệ thống đề xuất. Học có giám sát là một loại học máy sử dụng dữ liệu được dán nhãn. Tóm lại, mục tiêu của mô hình học có giám sát là dự đoán (đoán) nhãn.

Trong ví dụ ở hình 2.1, tập dữ liệu bên trái chứa hình ảnh của chó và mèo và các nhãn là “chó” và “mèo”. Đối với tập dữ liệu này, mô hình học máy sẽ sử dụng dữ liệu trước đó để dự đoán nhãn của các điểm dữ liệu mới. Điều này có nghĩa là, nếu chúng ta đưa vào một hình ảnh mới *không* nhãn, mô hình sẽ đoán xem hình ảnh đó là chó hay mèo, từ đó dự đoán nhãn của điểm dữ liệu (hình 2.2).



Hình 2.2 Mô hình học có giám sát dự đoán nhãn của điểm dữ liệu mới. Trong trường hợp này, điểm dữ liệu tương ứng với một con chó và thuật toán học có giám sát được huấn luyện để dự đoán rằng điểm dữ liệu này thực sự tương ứng với một con chó.

Nếu bạn nhớ lại chương 1, khuôn khổ chúng ta đã học để đưa ra quyết định là ghi nhớ-xây dựng-dự đoán. Đây chính xác là cách học có giám sát hoạt động. Đầu tiên, mô hình **ghi nhớ** tập dữ liệu về chó và mèo. Sau đó, nó **xây dựng** một mô hình hoặc một quy tắc cho những gì nó tin là cấu thành nên một con chó và một con mèo. Cuối cùng, khi một hình ảnh mới xuất hiện, mô hình sẽ **dự đoán** về nhãn của hình ảnh đó là gì, cụ thể là con chó hay con mèo (hình 2.3).



Hình 2.3 Một mô hình học có giám sát tuân theo khuôn khổ ghi nhớ-công thức-dự đoán từ chương 1. Đầu tiên, nó ghi nhớ tập dữ liệu. Sau đó, nó xây dựng các quy tắc về những gì sẽ tạo nên một con chó và một con mèo. Cuối cùng, nó dự đoán điểm dữ liệu mới là chó hay mèo.

Bây giờ, hãy lưu ý rằng trong hình 2.1, chúng ta có hai loại tập dữ liệu được gắn nhãn. Trong tập dữ liệu ở giữa, mỗi điểm dữ liệu được gắn nhãn trọng lượng của con vật. Trong tập dữ liệu này, nhãn là số. Trong tập dữ liệu bên trái, mỗi điểm dữ liệu được gắn nhãn loại động vật (chó hoặc mèo). Trong tập dữ liệu này, nhãn là trạng thái. **Số và trạng thái là hai loại dữ liệu mà chúng ta sẽ gặp trong các mô hình học có giám sát. Chúng tôi gọi loại đầu tiên là dữ liệu số và loại thứ hai là dữ liệu phân loại.**

dữ liệu số là bất kỳ loại dữ liệu nào sử dụng các số như 4, 2,35 hoặc -199. Ví dụ về dữ liệu số là giá cả, kích thước hoặc trọng lượng.

dữ liệu danh mục là bất kỳ loại dữ liệu nào sử dụng danh mục hoặc trạng thái, chẳng hạn như nam/nữ hoặc mèo/chó/chim. Đối với loại dữ liệu này, chúng tôi có một tập hợp các danh mục hữu hạn để liên kết với từng điểm dữ liệu.

Th dẫn đến hai loại mô hình học có giám sát sau đây

S:

mô hình hồi quy là loại mô hình dự đoán **dữ liệu số**. Đầu ra của mô hình hồi quy là một *con số*, chẳng hạn như trọng lượng của con vật.

mô hình phân loại là loại mô hình dự đoán **dữ liệu phân loại**. Đầu ra của mô hình phân loại là một *danh mục* hoặc *trạng thái*, chẳng hạn như loại

động vật (mèo hoặc chó).

Hãy xem xét hai ví dụ về mô hình học tập có giám sát, một hồi quy và một phân loại.

Mô hình 1: mô hình giá nhà đất (hồi quy). Trong mô hình này, mỗi điểm dữ liệu là một ngôi nhà. Nhãn của mỗi ngôi nhà là giá của nó. Mục tiêu của chúng tôi là khi một ngôi nhà mới (điểm dữ liệu) xuất hiện trên thị trường, chúng tôi muốn dự đoán nhãn của nó, cụ thể là giá của nó.

Mô hình 2: mô hình phát hiện thư rác qua email (phân loại). Trong mô hình này, mỗi điểm dữ liệu là một email. Nhãn của mỗi email là thư rác hoặc ham. Mục tiêu của chúng tôi là khi một email mới (điểm dữ liệu) đến hộp thư đến của chúng tôi, chúng tôi muốn dự đoán nhãn của nó, cụ thể là, đó là thư rác hay ham.

Lưu ý sự khác biệt giữa mô hình 1 và 2.

- Mô hình giá nhà ở là mô hình có thể trả về một số từ nhiều khả năng, chẳng hạn như 100 USD, 250.000 USD hoặc 3.125.672,33 USD. Vì vậy, đây là mô hình *hồi quy*. Mặt khác, mô hình phát hiện thư rác chỉ có thể trả về hai thứ: thư rác hoặc ham. Vì vậy, đây là mô hình *phân loại*.

•

Trong các phần phụ sau, chúng tôi sẽ trình bày chi tiết hơn về hồi quy và phân loại.

Mô hình hồi quy dự đoán các con số

Như chúng ta đã thảo luận trước đây, các mô hình hồi quy là những mô hình mà nhãn mà chúng ta muốn dự đoán là một con số. Con số này được dự đoán dựa trên các tính năng. Trong ví dụ về nhà ở, các đặc điểm có thể là bất cứ thứ gì mô tả một ngôi nhà, chẳng hạn như kích thước, số phòng, khoảng cách đến trường học gần nhất hoặc tỷ lệ tội phạm trong khu vực lân cận.

Những nơi khác mà người ta có thể sử dụng mô hình hồi quy như sau:

- **Thị trường chứng khoán:** dự đoán giá của một cổ phiếu nhất định dựa trên giá cổ phiếu khác và các tín hiệu thị trường khác
- **Y học:** dự đoán tuổi thọ dự kiến của bệnh nhân hoặc thời gian hồi phục dự kiến, dựa trên các triệu chứng và tiền sử bệnh của bệnh nhân
- **Doanh số bán hàng:** dự đoán số tiền dự kiến mà khách hàng sẽ chi tiêu, dựa trên nhân khẩu học và hành vi mua hàng trong quá khứ của khách hàng
-

- **Đề xuất video:** dự đoán lượng thời gian dự kiến mà người dùng sẽ xem video, dựa trên thông tin nhân khẩu học của người dùng và các video khác mà họ đã xem

Phương pháp phổ biến nhất được sử dụng để hồi quy là hồi quy tuyến tính, sử dụng các hàm tuyến tính (đường hoặc đối tượng tương tự) để đưa ra dự đoán của chúng tôi dựa trên các đặc điểm. Chúng ta nghiên cứu hồi quy tuyến tính ở chương 3. Các phương pháp phổ biến khác được sử dụng để hồi quy là hồi quy cây quyết định mà chúng ta học ở chương 9 và một số phương pháp tổng hợp như rừng ngẫu nhiên, AdaBoost, cây tăng cường độ dốc và XGBoost mà chúng ta học ở chương 12.

Mô hình phân loại dự đoán trạng thái

Các mô hình phân loại là những mô hình trong đó nhãn mà chúng ta muốn dự đoán là trạng thái thuộc một tập hữu hạn các trạng thái. Các mô hình phân loại phổ biến nhất dự đoán “có” hoặc “không”, nhưng nhiều mô hình khác sử dụng tập hợp trạng thái lớn hơn. Ví dụ chúng ta thấy trong hình 2.3 là một ví dụ về phân loại vì nó dự đoán loại vật nuôi, cụ thể là “mèo” hoặc “chó”.

Trong ví dụ về nhận dạng email spam, mô hình dự đoán trạng thái của email (cụ thể là spam hoặc ham) từ các tính năng của email. Trong trường hợp này, đặc điểm của email có thể là số từ trên đó, số lỗi chính tả, người gửi hoặc bất kỳ điều gì khác mô tả email.

Một ứng dụng phổ biến khác của phân loại là nhận dạng hình ảnh. Các mô hình nhận dạng hình ảnh phổ biến nhất lấy các pixel trong hình ảnh làm đầu vào và đưa ra dự đoán về những gì hình ảnh mô tả. Hai trong số các bộ dữ liệu nổi tiếng nhất về nhận dạng hình ảnh là MNIST và CIFAR-10. MNIST chứa khoảng 60.000 hình ảnh đen trắng 28 x 28 pixel của các chữ số viết tay được gắn nhãn 0–9. Những hình ảnh này đến từ sự kết hợp của nhiều nguồn, bao gồm Cục điều tra dân số Hoa Kỳ và kho lưu trữ các chữ số viết tay do học sinh trung học Mỹ viết. Bạn có thể tìm thấy tập dữ liệu MNIST trong liên kết sau: <http://yann.lecun.com/exdb/mnist/>. Tập dữ liệu CIFAR-10 chứa 60.000 hình ảnh màu 32 x 32 pixel về các vật thể khác nhau. Những hình ảnh này được dán nhãn bằng 10 đồ vật khác nhau (do đó có tên là 10), cụ thể là máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu và xe tải. Cơ sở dữ liệu này được duy trì bởi Viện Nghiên cứu Nâng cao Canada (CIFAR) và có thể tìm thấy trong liên kết sau: <https://www.cs.toronto.edu/~kriz/cifar.html>.

Một số ứng dụng mạnh mẽ khác của mô hình phân loại sau:

- **Phân tích cảm xúc:** dự đoán xem đánh giá phim là tích cực hay tiêu cực, dựa trên các từ trong đánh giá **Lưu lượng truy cập trang web:** dự đoán liệu người dùng có nhấp vào liên kết hay không, dựa trên nhân khẩu học của người dùng và tương tác trong quá khứ với trang web
- **Mạng xã hội:** dự đoán liệu người dùng sẽ kết bạn hoặc tương tác với người dùng khác hay không, dựa trên nhân khẩu học, lịch sử và bạn bè chung của họ **Đề xuất video:** dự đoán liệu người dùng có xem video hay không, dựa trên nhân khẩu học của người dùng và các video khác mà họ đã xem

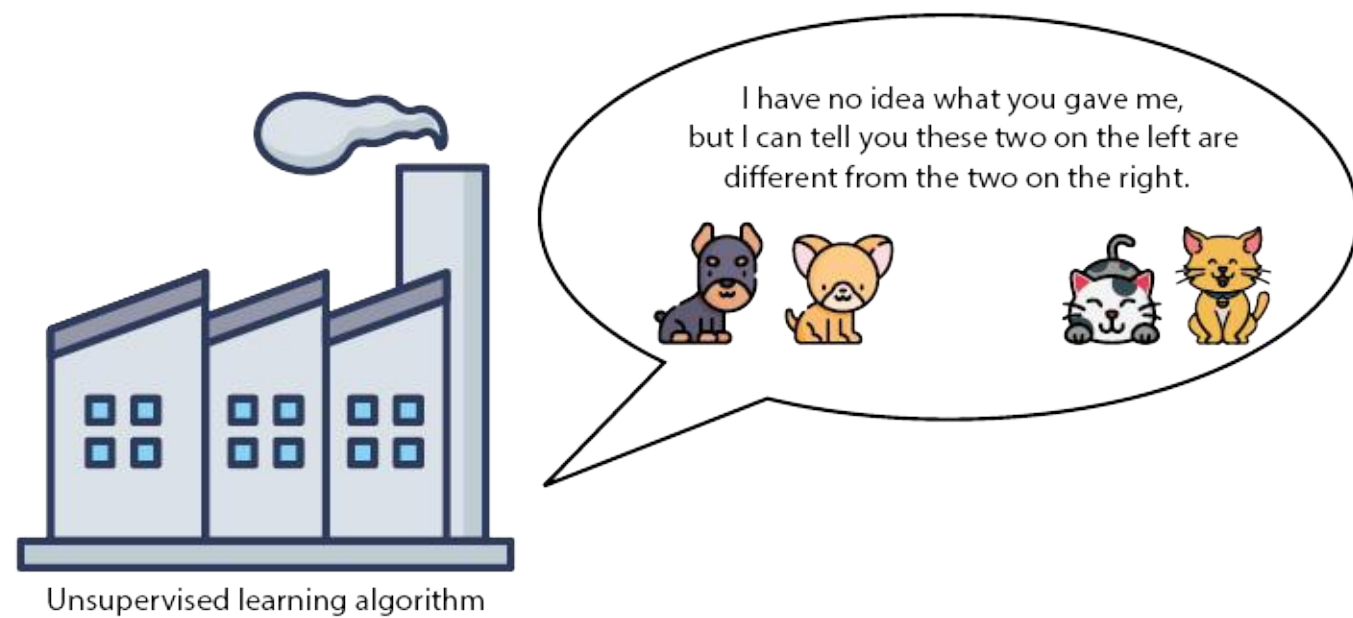
Phần lớn cuốn sách này (chương 5, 6, 8, 9, 10, 11 và 12) đề cập đến các mô hình phân loại. Trong các chương này, chúng ta tìm hiểu về perceptron (chương 5), bộ phân loại logistic (chương 6), thuật toán Bayes ngây thơ (chương 8), cây quyết định (chương 9), mạng lưới thần kinh (chương 10), máy vector hỗ trợ (chương 11) và các phương pháp tập hợp (chương 12).

Học không giám sát: Nhánh học máy hoạt động với dữ liệu không được gắn nhãn

Học không giám sát cũng là một loại học máy phổ biến. Nó khác với học có giám sát ở chỗ dữ liệu không được gắn nhãn. Nói cách khác, mục tiêu của mô hình học máy là trích xuất càng nhiều thông tin càng tốt từ tập dữ liệu không có nhãn hoặc mục tiêu để dự đoán.

Tập dữ liệu đó có thể là gì và chúng ta có thể làm gì với nó? Về nguyên tắc, chúng tôi có thể làm ít hơn một chút so với những gì chúng tôi có thể làm với tập dữ liệu được gắn nhãn vì chúng tôi không có nhãn để dự đoán. Tuy nhiên, chúng ta vẫn có thể trích xuất nhiều thông tin từ tập dữ liệu chưa được gắn nhãn. Ví dụ: hãy quay lại ví dụ về chó và mèo ở tập dữ liệu ngoài cùng bên phải trong hình 2.1. Tập dữ liệu này bao gồm hình ảnh của mèo và chó nhưng không có nhãn. Do đó, chúng tôi không biết mỗi hình ảnh đại diện cho loại thú cưng nào, vì vậy chúng tôi không thể dự đoán liệu hình ảnh mới tương ứng với một con chó hay một con mèo. Tuy nhiên, chúng ta có thể làm những việc khác, chẳng hạn như xác định xem hai bức ảnh giống nhau hay khác nhau. Đây là điều mà các thuật toán học tập không giám sát thực hiện. Thuật toán học không giám sát có thể nhóm các hình ảnh dựa trên sự giống nhau, thậm chí không cần

biết mỗi nhóm đại diện cho điều gì (hình 2.4). Nếu được thực hiện đúng cách, thuật toán có thể tách hình ảnh con chó khỏi hình ảnh con mèo hoặc thậm chí nhóm từng hình ảnh theo giống!



Hình 2.4 Thuật toán học không giám sát vẫn có thể trích xuất thông tin từ dữ liệu. Ví dụ: nó có thể nhóm các phần tử tương tự lại với nhau.

Trên thực tế, ngay cả khi có nhãn, chúng tôi vẫn có thể sử dụng các kỹ thuật học không giám sát trên dữ liệu của mình để xử lý trước dữ liệu đó và áp dụng các phương pháp học có giám sát hiệu quả hơn.

Các nhánh chính của học tập không giám sát là phân cụm, giảm kích thước và học tổng quát.

thuật toán phân cụm Các thuật toán nhóm dữ liệu thành các cụm dựa trên sự giống nhau

thuật toán giảm kích thước Các thuật toán đơn giản hóa dữ liệu của chúng tôi và mô tả dữ liệu một cách trung thực với ít tính năng hơn

thuật toán tổng hợp Các thuật toán có thể tạo ra các điểm dữ liệu mới giống với dữ liệu hiện có

Trong ba phần sau đây, chúng ta sẽ nghiên cứu chi tiết hơn về ba nhánh này.

Thuật toán phân cụm chia tập dữ liệu thành các nhóm tương tự

As we stated previously, clustering algorithms are those that split the dataset into similar groups. To illustrate this, let’s go back to the two datasets in the section “Supervised learning”—the housing dataset and the spam email dataset—but imagine that they have no labels. This means that the housing dataset has no prices, and the email dataset has no information on the emails being spam or ham.

Let’s begin with the housing dataset. What can we do with this dataset? Here is an idea: we could somehow group the houses by similarity. For example, we could group them by location, price, size, or a combination of these factors. This process is called *clustering*. Clustering is a branch of unsupervised machine learning that consists of the tasks that group the elements in our dataset into clusters where all the data points are similar.

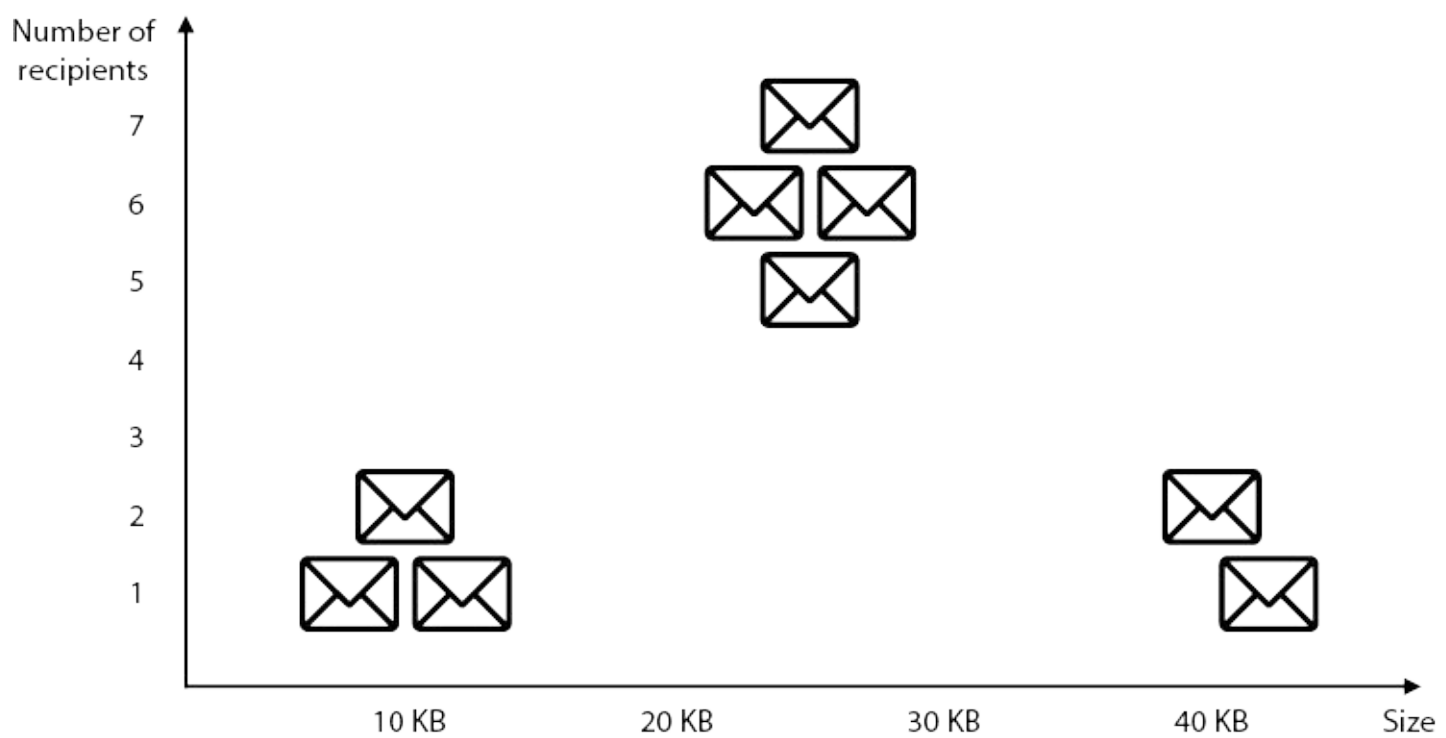
Now let’s look at the second example, the dataset of emails. Because the dataset is unlabeled, we don’t know whether each email is spam or ham. However, we can still apply some clustering to our dataset. A clustering algorithm splits our images into a few different groups based on different features of the email. These features could be the words in the message, the sender, the number and size of the attachments, or the types of links inside the email. After clustering the dataset, a human (or a combination of a human and a supervised learning algorithm) could label these clusters by categories such as “Personal,” “Social,” and “Promotions.”

As an example, let’s look at the dataset in table 2.1, which contains nine emails that we would like to cluster. The features of the dataset are the size of the email and the number of recipients.

Table 2.1 A table of emails with their size and number of recipients

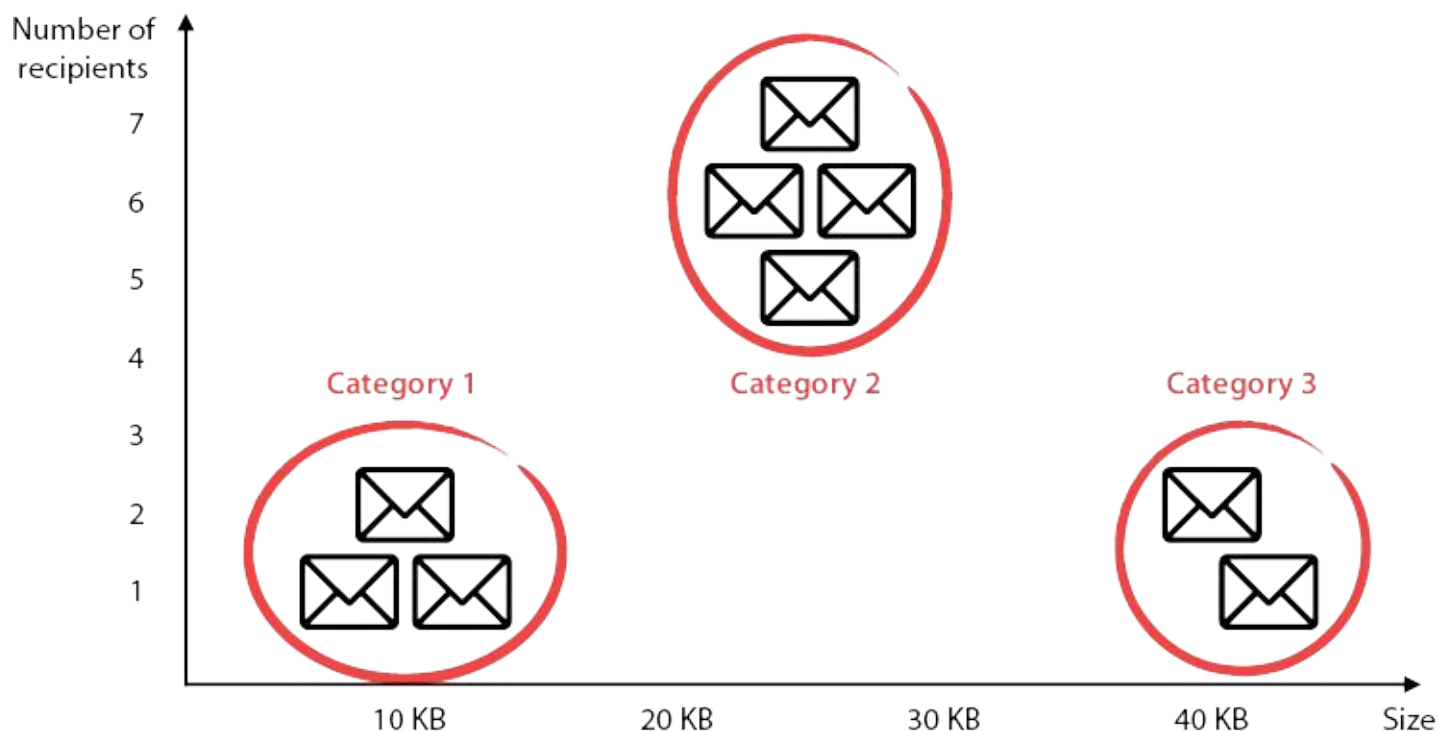
Email	Size	Recipients
1	8	1
2	12	1
3	43	1
4	10	2
5	40	2
6	25	5
7	23	6
8	28	6
9	26	7

Bằng mắt thường, có vẻ như chúng ta có thể nhóm các email theo số lượng người nhận.Điều này sẽ dẫn đến hai cụm: một cụm có email có hai người nhận trở xuống và một cụm có email có năm người nhận trở lên.Chúng ta cũng có thể thử nhóm chúng thành ba nhóm theo quy mô.Nhưng bạn có thể tưởng tượng rằng khi cái bàn ngày càng lớn hơn, việc quan sát các nhóm ngày càng khó khăn hơn.Điều gì sẽ xảy ra nếu chúng ta vẽ dữ liệu?Hãy vẽ các email dưới dạng biểu đồ, trong đó trục hoành ghi lại kích thước và trục tung ghi lại số lượng người nhận.Điều này cho chúng ta đồ thị trong hình 2.5.



Hình 2.5 Sơ đồ tập dữ liệu email.Trục ngang tương ứng với kích thước của email và trục tung tương ứng với số lượng người nhận.Chúng ta có thể thấy ba cụm được xác định rõ ràng trong tập dữ liệu này.

Trong hình 2.5 chúng ta có thể thấy ba cụm được xác định rõ ràng, được đánh dấu trong hình 2.6.



Hình 2.6 Chúng ta có thể phân cụm email thành ba loại dựa trên quy mô và số lượng người nhận.

Bước cuối cùng này chính là nội dung của việc phân cụm. Tất nhiên, đối với con người chúng ta, thật dễ dàng để mắt đến ba nhóm một khi chúng ta nắm được cốt truyện. Nhưng đối với một chiếc máy tính, nhiệm vụ này không hề dễ dàng. Hơn nữa, hãy tưởng tượng nếu dữ liệu của chúng ta chứa hàng triệu điểm, với hàng trăm hoặc hàng nghìn tính năng. Với nhiều hơn ba đặc điểm, con người không thể nhìn thấy các cụm vì chúng sẽ có kích thước mà chúng ta không thể hình dung được. May mắn thay, máy tính có thể thực hiện kiểu phân cụm này cho các tập dữ liệu khổng lồ có nhiều hàng và cột.

Các ứng dụng khác của phân cụm như sau:

- **Phân khúc thị trường:** chia khách hàng thành các nhóm dựa trên nhân khẩu học và hành vi mua hàng trước đó để tạo ra các chiến lược tiếp thị khác nhau cho các nhóm
- **Di truyền:** phân nhóm các loài thành các nhóm dựa trên sự giống nhau về gen
- **Hình ảnh y tế:** chia hình ảnh thành các phần khác nhau để nghiên cứu các loại mô khác nhau
- **Video đề xuất:** chia người dùng thành các nhóm dựa trên nhân khẩu học và các video trước đó đã xem và sử dụng tính năng này để giới thiệu cho người dùng những video mà người dùng khác trong nhóm của họ đã xem
-

THÊM VỀ MÔ HÌNH HỌC TẬP KHÔNG GIÁM SÁT

Trong phần còn lại của cuốn sách này, chúng tôi không đề cập đến việc học không giám sát. Tuy nhiên, tôi đặc biệt khuyến khích bạn tự mình nghiên cứu nó. Dưới đây là một số thuật toán phân cụm quan trọng nhất hiện có. Phụ lục C liệt kê thêm một số thuật toán khác (bao gồm một số video của tôi) nơi bạn có thể tìm hiểu chi tiết về các thuật toán này.

- **K-có nghĩa là phân cụm:** thuật toán này nhóm các điểm bằng cách chọn một số khối tâm ngẫu nhiên và di chuyển chúng ngày càng gần các điểm cho đến khi chúng ở đúng điểm.
- **Phân cụm theo cấp bậc:** thuật toán này bắt đầu bằng cách nhóm các điểm gần nhất lại với nhau và tiếp tục theo cách này cho đến khi chúng ta có một số nhóm được xác định rõ ràng.
- **Phân cụm không gian dựa trên mật độ (DBSCAN):** thuật toán này bắt đầu nhóm các điểm lại với nhau ở những nơi có mật độ cao, đồng thời gắn nhãn các điểm bị cô lập là nhiễu.
- **Mô hình hỗn hợp Gaussian:** thuật toán này không gán điểm cho một cụm mà thay vào đó gán các phân số của điểm cho từng cụm hiện có. Ví dụ: nếu có ba cụm A, B và C thì thuật toán có thể xác định rằng 60% điểm cụ thể thuộc nhóm A, 25% thuộc nhóm B và 15% thuộc nhóm C.

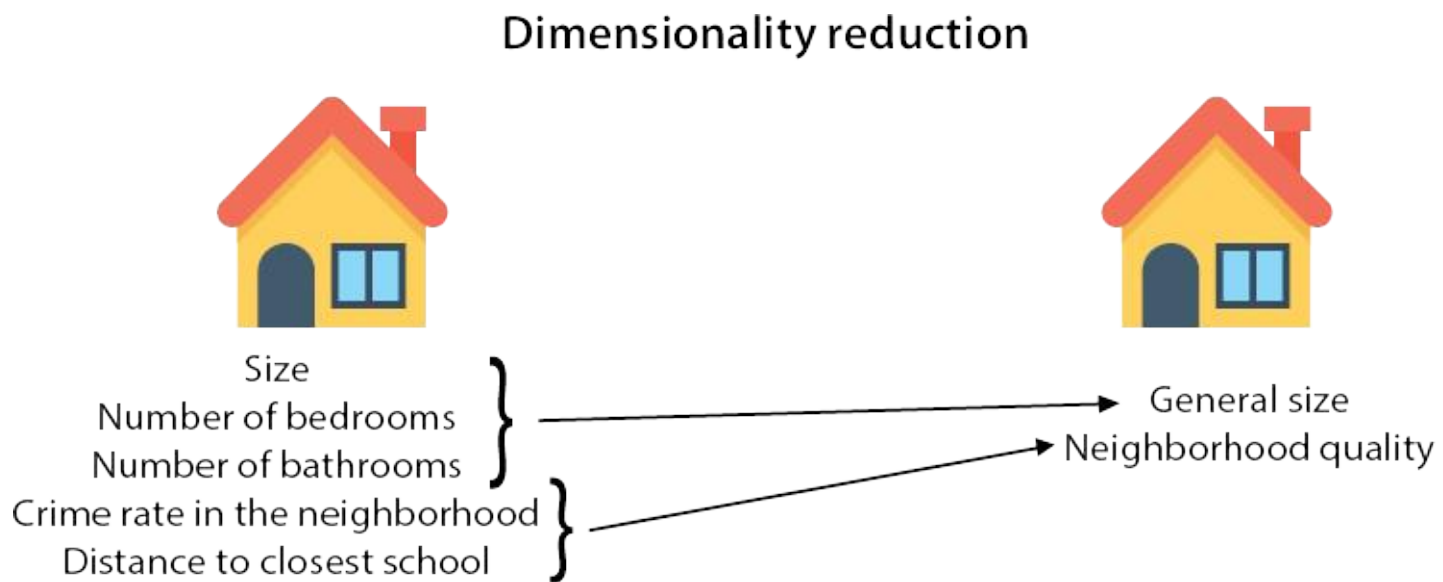
Giảm kích thước giúp đơn giản hóa dữ liệu mà không mất quá nhiều thông tin

Giảm kích thước là một bước tiền xử lý hữu ích mà chúng ta có thể áp dụng để đơn giản hóa đáng kể dữ liệu của mình trước khi áp dụng các kỹ thuật khác. Ví dụ: hãy quay lại tập dữ liệu nhà ở. Hãy tưởng tượng rằng các tính năng như sau:

- Quy mô Số phòng ngủ Số phòng tắm
- Tỷ lệ tội phạm trong khu vực lân cận
- Khoảng cách đến trường học gần nhất
-
-

Tập dữ liệu này có năm cột dữ liệu. Điều gì sẽ xảy ra nếu chúng ta muốn biến tập dữ liệu thành một tập dữ liệu đơn giản hơn với ít cột hơn mà không mất nhiều thông tin? Hãy làm điều này bằng cách sử dụng ý thức chung. Hãy xem xét kỹ hơn năm tính năng. Bạn có thể tìm ra cách nào để đơn giản hóa chúng—có thể nhóm chúng thành một số danh mục nhỏ hơn và tổng quát hơn không?

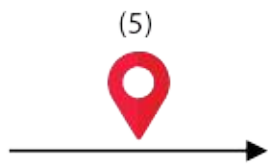
Sau khi xem xét kỹ, chúng ta có thể thấy ba đặc điểm đầu tiên giống nhau vì chúng đều liên quan đến kích thước của ngôi nhà. Tương tự, đặc điểm thứ tư và thứ năm tương tự nhau vì chúng liên quan đến chất lượng của khu vực lân cận. Chúng ta có thể cô đọng ba đặc điểm đầu tiên thành đặc điểm “kích thước” lớn, còn đặc điểm thứ tư và thứ năm thành đặc điểm lớn “chất lượng vùng lân cận”. Làm thế nào để chúng tôi cô đọng các tính năng kích thước? Chúng ta có thể quên đi số phòng và phòng ngủ mà chỉ xem xét kích thước, chúng ta có thể thêm số lượng phòng ngủ và phòng tắm hoặc có thể sử dụng một số kết hợp khác của ba đặc điểm này. Chúng ta cũng có thể cô đọng các đặc điểm chất lượng khu vực theo những cách tương tự. Các thuật toán giảm kích thước sẽ tìm ra những cách tốt để cô đọng các tính năng này, mất ít thông tin nhất có thể và giữ cho dữ liệu của chúng ta nguyên vẹn nhất có thể đồng thời quản lý để đơn giản hóa dữ liệu để xử lý và lưu trữ dễ dàng hơn (hình 2.7).



Hình 2.7 Các thuật toán giảm kích thước giúp chúng ta đơn giản hóa dữ liệu. Ở bên trái, chúng tôi có tập dữ liệu về nhà ở với nhiều tính năng. Chúng ta có thể sử dụng tính năng giảm kích thước để giảm số lượng tính năng trong tập dữ liệu mà không mất nhiều thông tin và thu được tập dữ liệu ở bên phải.

Tại sao nó được gọi là giảm kích thước nếu tất cả những gì chúng ta đang làm là giảm số lượng cột trong dữ liệu của mình? Từ ưa thích để chỉ số lượng cột trong tập dữ liệu là *thứ nguyên*. Hãy nghĩ về điều này: nếu dữ liệu của chúng ta có một cột thì mỗi điểm dữ liệu là một số. Một tập hợp các số có thể được vẽ dưới dạng tập hợp các điểm trên một đường thẳng có chính xác một chiều. Nếu dữ liệu của chúng ta có hai cột thì mỗi điểm dữ liệu được tạo thành bởi một cặp số. Chúng ta có thể tưởng tượng một tập hợp các cặp số là tập hợp các điểm trong một thành phố, trong đó số đầu tiên là số đường và số thứ hai là đại lộ. Địa chỉ trên bản đồ có hai chiều vì chúng nằm trong một mặt phẳng. Điều gì xảy ra khi dữ liệu của chúng tôi có ba

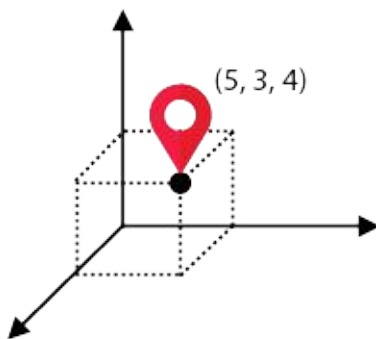
cột? Trong trường hợp này, mỗi điểm dữ liệu được hình thành bởi ba số. Chúng ta có thể tưởng tượng rằng nếu mọi địa chỉ trong thành phố của chúng ta là một tòa nhà thì số thứ nhất và số thứ hai là đường và đại lộ, còn số thứ ba là tầng trong tòa nhà. Điều này trông giống một thành phố ba chiều hơn. Chúng ta có thể tiếp tục đi. Còn bốn số thì sao? Chà, bây giờ chúng ta thực sự không thể hình dung được nó, nhưng nếu có thể, tập hợp các điểm này sẽ trông giống như các địa điểm trong một thành phố bốn chiều, v.v. Cách tốt nhất để hình dung một thành phố bốn chiều là tưởng tượng một cái bàn có bốn cột. Thế còn một thành phố 100 chiều thì sao? Đây sẽ là một bảng có 100 cột, trong đó mỗi người có một địa chỉ gồm 100 số. Hình ảnh tinh thần mà chúng ta có thể có khi nghĩ về những chiều cao hơn được thể hiện trong hình 2.8. Do đó, khi chúng tôi chuyển từ năm chiều xuống còn hai chiều, chúng tôi đã biến thành phố năm chiều của mình thành thành phố hai chiều. Đây là lý do tại sao nó được gọi là giảm kích thước.



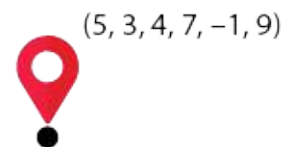
One dimension
Number on a street



Two dimensions
Address in a city (street, avenue)



Three dimensions
Address in a building (street, avenue, floor)

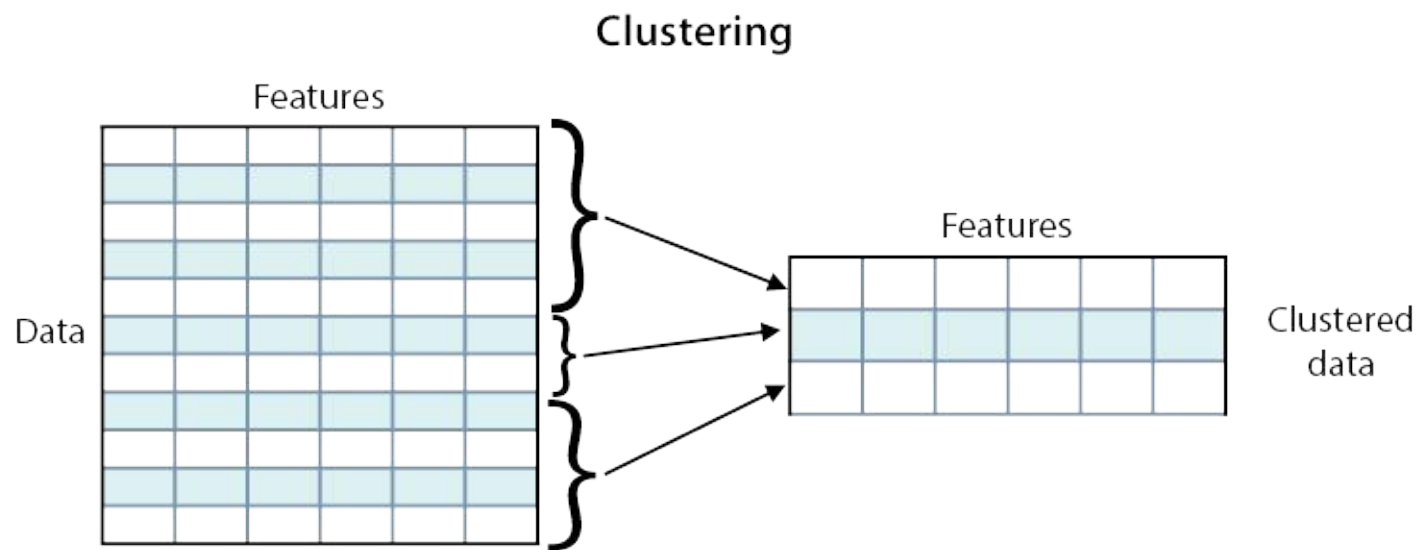


More dimensions...
Address in a high-dimensional
parallel universe

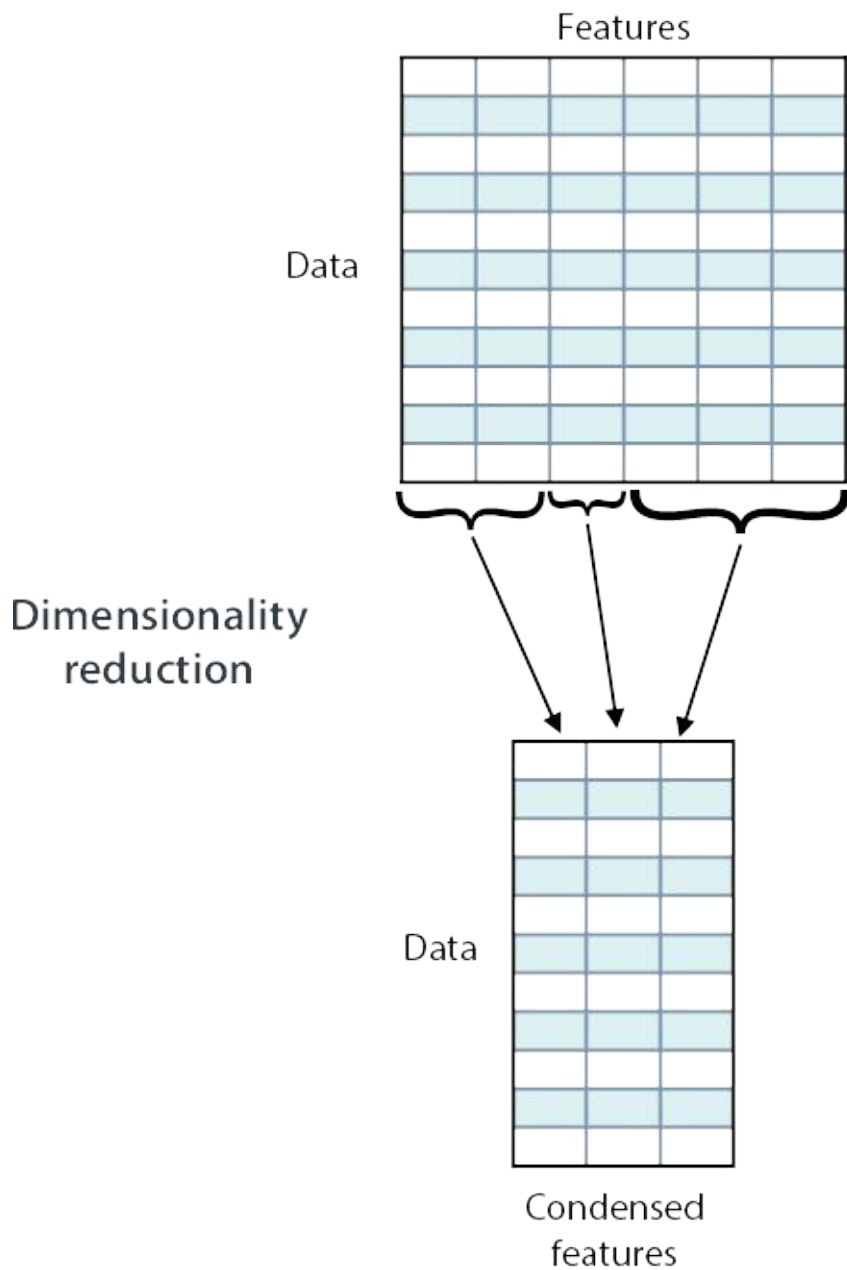
Hình 2.8 Cách tưởng tượng không gian nhiều chiều hơn: Một chiều giống như một con phố, trong đó mỗi ngôi nhà chỉ có một số. Hai chiều giống như một thành phố phẳng, trong đó mỗi địa chỉ có hai số, đường và đại lộ. Ba chiều giống như một thành phố với các tòa nhà, trong đó mỗi địa chỉ có ba số: đường, đại lộ và tầng. Bốn chiều giống như một địa điểm tưởng tượng trong đó mỗi địa chỉ có bốn số. Chúng ta có thể tưởng tượng các chiều cao hơn như một thành phố tưởng tượng khác trong đó các địa chỉ có nhiều tọa độ mà chúng ta cần.

Các cách khác để đơn giản hóa dữ liệu của chúng tôi: Phân tích nhân tử ma trận và phân tách giá trị số ít

Có vẻ như việc phân cụm và giảm kích thước không có gì giống nhau, nhưng trên thực tế, chúng không quá khác nhau. Nếu chúng ta có một bảng chứa đầy dữ liệu, mỗi hàng tương ứng với một điểm dữ liệu và mỗi cột tương ứng với một đối tượng địa lý. Do đó, chúng ta có thể sử dụng phân cụm để giảm số lượng hàng trong tập dữ liệu và giảm kích thước để giảm số lượng cột, như minh họa trong hình 2.9 và 2.10.



Hình 2.9 Phân cụm có thể được sử dụng để đơn giản hóa dữ liệu của chúng tôi bằng cách giảm số lượng hàng trong tập dữ liệu của chúng tôi bằng cách nhóm nhiều hàng thành một.



Hình 2.10 Giảm kích thước có thể được sử dụng để đơn giản hóa dữ liệu của chúng tôi bằng cách giảm số lượng cột trong tập dữ liệu của chúng tôi.

Có thể bạn đang thắc mắc, có cách nào để chúng ta có thể giảm cả hàng và cột cùng một lúc không? Và câu trả lời là có! Hai cách phổ biến mà chúng ta có thể thực hiện là *phân tích nhân tử ma trận* và *phân tách giá trị số ít*. Hai thuật toán này thể hiện một ma trận dữ liệu lớn thành tích của các ma trận nhỏ hơn.

Những nơi như Netflix sử dụng rộng rãi hệ số ma trận để tạo đề xuất. Hãy tưởng tượng một bảng lớn trong đó mỗi hàng tương ứng với một người dùng, mỗi cột tương ứng với một bộ phim và mỗi mục trong ma trận là xếp hạng mà người dùng đã đưa ra cho bộ phim đó. Với hệ số hóa ma trận, người ta có thể trích xuất một số tính năng nhất định, chẳng hạn như loại phim, diễn viên xuất hiện trong phim, v.v., đồng thời có thể dự đoán xếp hạng mà người dùng đưa ra cho phim, dựa trên các tính năng này.

Phân rã giá trị số ít được sử dụng trong nén hình ảnh. Ví dụ: một hình ảnh đen trắng có thể được xem dưới dạng một bảng dữ liệu lớn, trong đó mỗi mục chứa cường độ của pixel tương ứng. Phân tách giá trị số ít sử dụng các kỹ thuật đại số tuyến tính để đơn giản hóa bảng dữ liệu này, do đó cho phép chúng ta đơn giản hóa hình ảnh và lưu trữ phiên bản đơn giản hơn của nó với ít mục nhập hơn.

Học máy sáng tạo

Sáng tạo học máy là một trong những lĩnh vực đáng kinh ngạc nhất của học máy. Nếu bạn đã từng thấy những khuôn mặt, hình ảnh hoặc video siêu thực do máy tính tạo ra thì bạn đã thấy hoạt động của máy học tổng quát đang hoạt động.

Lĩnh vực học tập tổng quát bao gồm các mô hình, với một tập dữ liệu, có thể xuất ra các điểm dữ liệu mới trông giống như các mẫu từ tập dữ liệu gốc đó. **Các thuật toán này buộc phải tìm hiểu dữ liệu trông như thế nào để tạo ra các điểm dữ liệu tương tự.** Ví dụ: nếu tập dữ liệu chứa hình ảnh khuôn mặt thì thuật toán sẽ tạo ra khuôn mặt trông giống thật. Các thuật toán tổng quát đã có thể tạo ra những hình ảnh, bức vẽ cực kỳ chân thực, v.v. Họ cũng đã tạo ra video, âm nhạc, truyện, thơ và nhiều điều tuyệt vời khác. Thuật toán tổng quát phổ biến nhất là mạng đối thủ tổng quát (GAN), được phát triển bởi Ian Goodfellow và các đồng tác giả của ông. Các thuật toán tổng hợp hữu ích và phổ biến khác là bộ mã hóa tự động biến thiên do Kingma và Welling phát triển và máy Boltzmann (RBM) bị hạn chế do Geoffrey Hinton phát triển.

Như bạn có thể tưởng tượng, học tập sáng tạo khá khó khăn. Đối với con người, việc xác định xem một hình ảnh có vẽ một con chó hay không sẽ dễ dàng hơn nhiều so với việc vẽ một con chó. Nhiệm vụ này cũng khó khăn đối với máy tính. Do đó, các thuật toán trong học tập tổng quát rất phức tạp và cần rất nhiều dữ liệu cũng như sức mạnh tính toán để chúng hoạt động tốt. Bởi vì cuốn sách này nói về học tập có giám sát nên chúng tôi sẽ không trình bày chi tiết về học tập tổng quát, nhưng trong chương 10, chúng ta sẽ hiểu được cách hoạt động của một số thuật toán tổng quát này, bởi vì chúng có xu hướng sử dụng mạng lưới thần kinh. Phụ lục C chứa các đề xuất về các nguồn tài nguyên, bao gồm cả video của tác giả, nếu bạn muốn khám phá thêm chủ đề này.

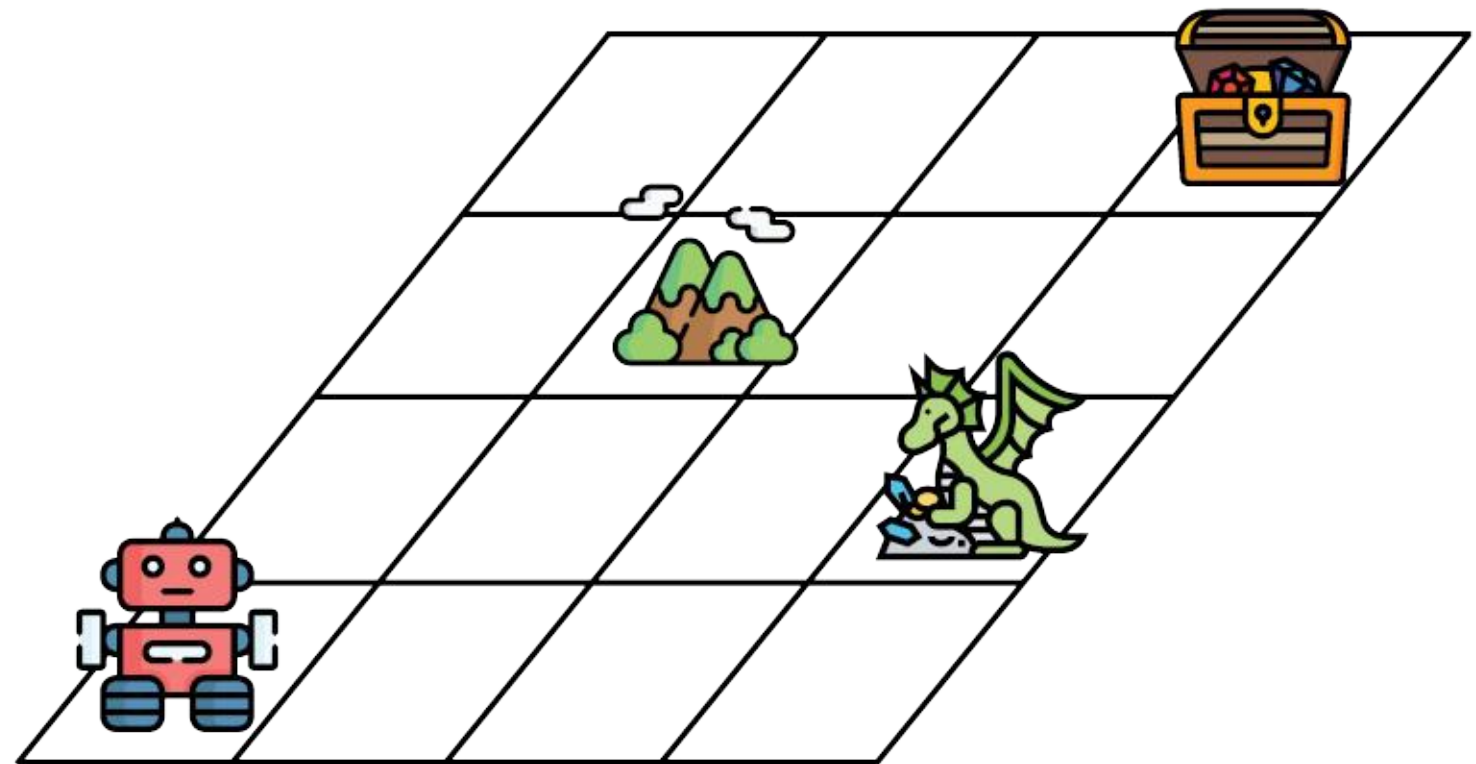
Học tăng cường là gì?

Học tăng cường là một loại học máy khác, trong đó không có dữ liệu nào được cung cấp và chúng ta phải yêu cầu máy tính thực hiện một nhiệm vụ. Thay vì dữ liệu, mô hình

nhận được một môi trường và một tác nhân có nhiệm vụ điều hướng trong môi trường này. Tác nhân có một mục tiêu hoặc một tập hợp các mục tiêu. Môi trường có những phần thưởng và hình phạt hướng dẫn tác nhân đưa ra những quyết định đúng đắn để đạt được mục tiêu. Tất cả điều này nghe có vẻ hơi trừu tượng, nhưng hãy xem một ví dụ.

VÍ DỤ: THẾ GIỚI LƯỚI

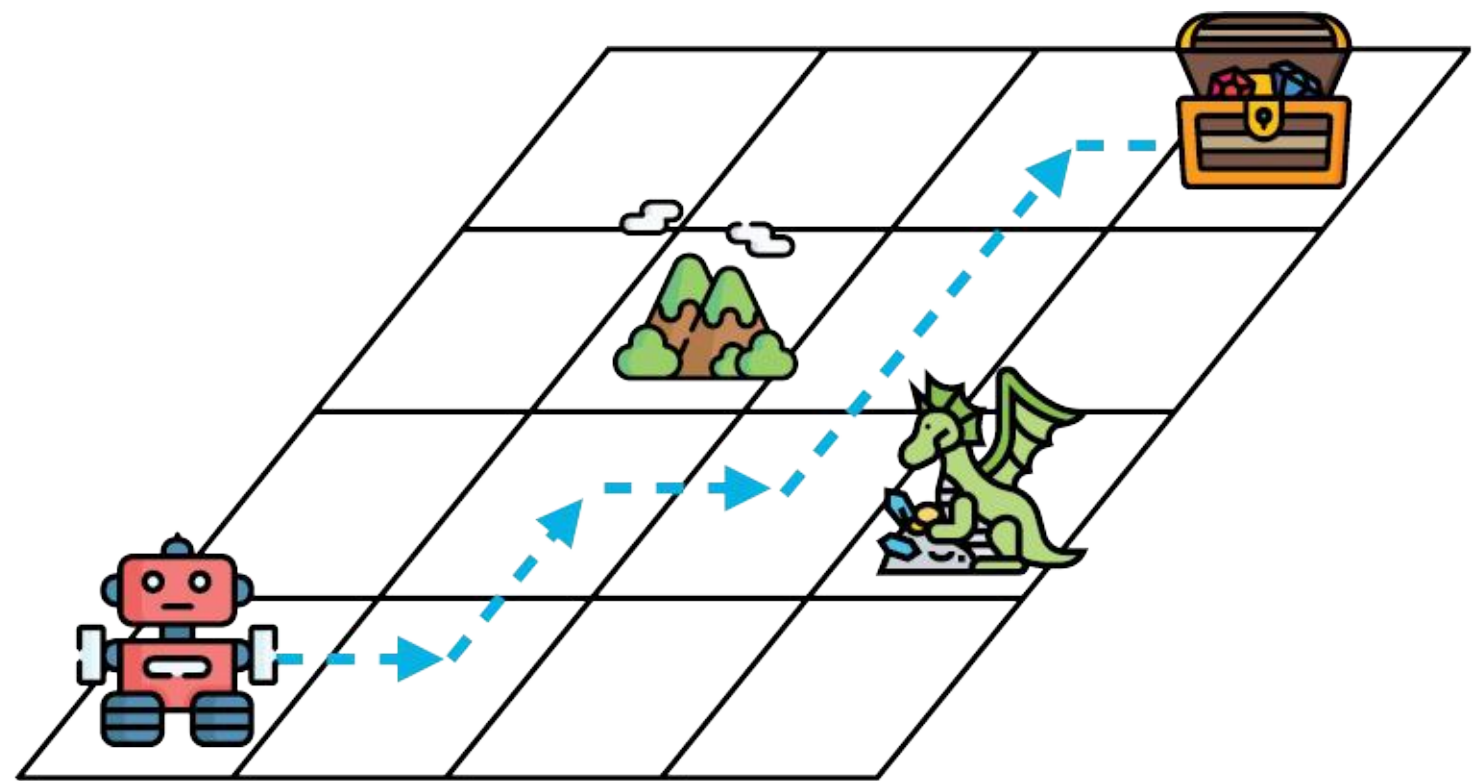
Trong hình 2.11, chúng ta thấy một thế giới lưới có robot ở góc dưới bên trái. Đó là đại lý của chúng tôi. Mục tiêu là đến được rương kho báu ở phía trên bên phải của lưới. Trong lưới chúng ta cũng có thể nhìn thấy một ngọn núi, nghĩa là chúng ta không thể đi qua ô vuông đó, vì robot không thể leo núi. Chúng ta cũng thấy một con rồng sẽ tấn công robot nếu robot dám đáp xuống ô vuông của nó, điều đó có nghĩa là một phần mục tiêu của chúng ta là không đáp xuống đó. Đây là trò chơi. Và để cung cấp cho robot thông tin về cách tiến hành, chúng tôi theo dõi điểm số. Điểm số bắt đầu từ số 0. Nếu robot đến được rương kho báu thì chúng ta được 100 điểm. Nếu robot chạm tới rồng, chúng ta mất 50 điểm. Và để đảm bảo rô-bốt của chúng ta di chuyển nhanh chóng, chúng ta có thể nói rằng cứ mỗi bước rô-bốt thực hiện, chúng ta sẽ mất 1 điểm, vì rô-bốt mất năng lượng khi đi.



Hình 2.11 Một thế giới lưới trong đó tác nhân của chúng ta là một robot. Mục tiêu của robot là tìm rương kho báu, đồng thời tránh con rồng. Ngọn núi tượng trưng cho một nơi mà robot không thể đi qua.

Cách huấn luyện thuật toán này rất sơ bộ như sau: Robot bắt đầu đi xung quanh, ghi lại điểm số của nó và ghi nhớ những bước đã thực hiện nó. Sau một thời gian có thể gặp rồng, mất nhiều điểm. Vì vậy, nó học cách liên kết hình vuông rồng và các hình vuông gần nó với điểm thấp. Tại một thời điểm nào đó, nó cũng có thể chạm vào rương kho báu và nó học cách bắt đầu liên kết hình vuông đó và các hình vuông gần đó với điểm cao. Sau khi chơi trò chơi này một thời gian dài, robot sẽ hiểu rõ mức độ tốt của mỗi ô vuông và nó có thể đi theo con đường đi theo các ô vuông đến tận rương kho báu. Hình 2.12 cho thấy

một con đường khả thi, mặc dù con đường này không lý tưởng, vì nó đi quá gần con rồng. Bạn có thể nghĩ ra một cái tốt hơn?



Hình 2. 12 Đây là con đường mà robot có thể đi để tìm rương kho báu

Tất nhiên, đây là lời giải thích rất ngắn gọn và còn rất nhiều điều nữa về việc học tăng cường. Phụ lục C đề xuất một số tài nguyên để nghiên cứu sâu hơn, bao gồm video học tăng cường sâu.

Học tăng cường có nhiều ứng dụng tiên tiến, bao gồm:

- **Trò chơi:** những tiến bộ gần đây trong việc dạy máy tính cách giành chiến thắng trong các trò chơi, chẳng hạn như cờ vây hoặc cờ vua, sử dụng phương pháp học tăng cường. Ngoài ra, các đặc vụ đã được dạy để giành chiến thắng trong các trò chơi Atari như *Breakout* hoặc *Super Mario*.

- **Robot:** học tăng cường được sử dụng rộng rãi để giúp robot thực hiện các nhiệm vụ như nhặt hộp, dọn phòng hoặc thậm chí khiêu vũ!
- **Ô tô tự lái:** kỹ thuật học tăng cường được sử dụng để giúp ô tô thực hiện nhiều nhiệm vụ như lập kế hoạch đường đi hoặc ứng xử trong các môi trường cụ thể.

Tóm tắt

- Có một số loại học máy tồn tại, bao gồm học có giám sát, học không giám sát và học tăng cường. Dữ liệu có thể được dán nhãn hoặc không được gắn nhãn. Dữ liệu được gắn nhãn chứa một tính năng hoặc nhãn đặc biệt mà chúng tôi mong muốn dự đoán. Dữ liệu chưa được gắn nhãn không có tính năng này. Học có giám sát được sử dụng trên dữ liệu được dán nhãn và bao gồm việc xây dựng các mô hình dự đoán nhãn cho dữ liệu chưa được nhìn thấy. Học không giám sát được sử dụng trên dữ liệu không được gắn nhãn và bao gồm các thuật toán giúp đơn giản hóa dữ liệu của chúng tôi mà không làm mất nhiều thông tin. Học không giám sát thường được sử dụng như một bước tiền xử lý. Hai loại thuật toán học có giám sát phổ biến được gọi là hồi quy và phân loại. Các mô hình hồi quy là những mô hình trong đó câu trả lời là bất kỳ số nào. Các mô hình phân loại là những mô hình trong đó câu trả lời thuộc về một loại hoặc một lớp. Hai loại thuật toán học không giám sát phổ biến là phân cụm và giảm kích thước. Phân cụm được sử dụng để nhóm dữ liệu thành các cụm tương tự nhằm trích xuất thông tin hoặc giúp xử lý dễ dàng hơn. Giảm kích thước là một cách để đơn giản hóa dữ liệu của chúng tôi bằng cách kết hợp một số tính năng tương tự nhất định và mất ít thông tin nhất có thể. Phân tích nhân tử ma trận và phân tách giá trị số ít là các thuật toán khác có thể đơn giản hóa dữ liệu của chúng tôi bằng cách giảm cả số hàng và số cột. Học máy sáng tạo là một loại học tập không giám sát sáng tạo, bao gồm việc tạo ra dữ liệu tương tự với tập dữ liệu của chúng tôi. Các mô hình sáng tạo có thể vẽ ra những khuôn mặt hiện thực, sáng tác nhạc và làm thơ. Học tăng cường là một loại học máy trong đó tác nhân phải điều hướng một môi trường và đạt được mục tiêu. Nó được sử dụng rộng rãi trong nhiều ứng dụng tiên tiến.

○

○

•

•

Bài tập

BÀI TẬP 2.1

Đối với mỗi tình huống sau đây, hãy nêu rõ đó là ví dụ về học tập có giám sát hay không giám sát. Giải thích câu trả lời của bạn. Trong trường hợp không rõ ràng, hãy chọn một và giải thích lý do tại sao bạn chọn nó.

Một. Hệ thống giới thiệu trên mạng xã hội giới thiệu bạn bè tiềm năng cho người dùng b. Một hệ thống trong một trang tin tức chia tin tức thành các chủ đề c. Tính năng tự động hoàn thành câu của Google d. Hệ thống khuyến nghị của nhà bán lẻ trực tuyến đề xuất cho người dùng những gì nên mua dựa trên lịch sử mua hàng trước đây của họ e. Một hệ thống trong công ty thể tin dụng có chức năng ghi lại các giao dịch gian lận

BÀI TẬP 2.2

Đối với mỗi ứng dụng sau đây của học máy, bạn sẽ sử dụng hồi quy hoặc phân loại để giải quyết nó? Giải thích câu trả lời của bạn. Trong trường hợp không rõ ràng, hãy chọn một và giải thích lý do tại sao bạn chọn nó.

Một. Một cửa hàng trực tuyến dự đoán số tiền người dùng sẽ chi tiêu trên trang web của họ b. Trợ lý giọng nói giải mã giọng nói và chuyển nó thành văn bản c. Bán hoặc mua cổ phiếu từ một công ty cụ thể d. YouTube đề xuất video cho người dùng

BÀI TẬP 2.3

Nhiệm vụ của bạn là chế tạo một chiếc ô tô tự lái. Đưa ra ít nhất ba ví dụ về các vấn đề về máy học mà bạn sẽ phải giải quyết để xây dựng nó. Trong mỗi ví dụ, hãy giải thích xem bạn đang sử dụng phương pháp học có giám sát/không giám sát và nếu được giám sát thì bạn đang sử dụng phương pháp hồi quy hay phân loại. Nếu bạn đang sử dụng các loại máy học khác, hãy giải thích loại nào và tại sao.

3 Về một đường gần các điểm của chúng ta: Hồi quy tuyến tính

Trong chương này

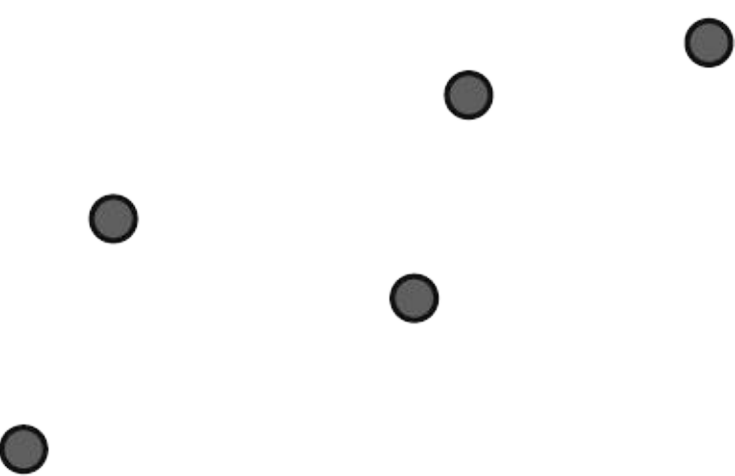
- Hồi quy tuyến tính khớp một đường thông qua một tập hợp các điểm dữ liệu mã hóa thuật toán hồi
- quy tuyến tính trong Python bằng cách sử dụng Turi Create để xây dựng mô hình hồi quy tuyến
- tính nhằm dự đoán giá nhà ở trong tập dữ liệu thực tế **Hồi quy đa thức khớp đường cong phức tạp**
- **hơn với dữ liệu phi tuyến** thảo luận về các ví dụ về hồi quy tuyến tính trong thế giới thực, chẳng
- hạn như các ứng dụng y tế và hệ thống đề xuất là gì

-
-
-



Trong chương này, chúng ta sẽ tìm hiểu về hồi quy tuyến tính. Hồi quy tuyến tính là một phương pháp mạnh mẽ và được sử dụng rộng rãi để ước tính các giá trị, chẳng hạn như giá một ngôi nhà, giá trị của một cổ phiếu nhất định, tuổi thọ của một cá nhân hoặc lượng thời gian người dùng sẽ xem video hoặc dành cho trang web. Bạn có thể đã từng thấy hồi quy tuyến tính trước đây dưới dạng vô số công thức phức tạp bao gồm đạo hàm, hệ phương trình và định thức. Tuy nhiên, chúng ta cũng có thể thấy hồi quy tuyến tính theo cách đồ họa hơn và ít công thức hơn. Trong chương này, để hiểu hồi quy tuyến tính, tất cả những gì bạn cần là khả năng hình dung các điểm và đường chuyển động xung quanh.

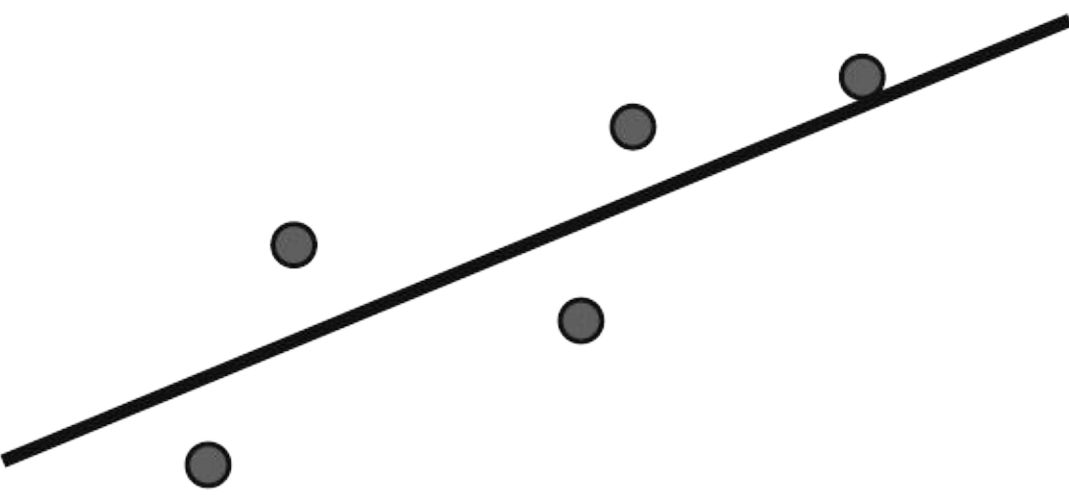
Giả sử chúng ta có một số điểm gần giống như chúng đang tạo thành một đường thẳng, như trong hình 3.1.



Hình 3.1 Một số điểm gần giống như đang tạo thành một đường thẳng

Mục tiêu của hồi quy tuyến tính là vẽ đường đi càng gần các điểm này càng tốt. Bạn sẽ vẽ đường nào đi gần những điểm đó? Thế còn cái được minh họa trong hình 3.2 thì sao?

Hãy coi các điểm như những ngôi nhà trong một thị trấn và mục tiêu của chúng ta là xây dựng một con đường đi qua thị trấn. Chúng tôi muốn đường đi càng gần các điểm càng tốt vì cư dân của thị trấn đều muốn sống gần đường và mục tiêu của chúng tôi là làm hài lòng họ nhiều nhất có thể.



Hình 3.2 Đường đi gần các điểm

Chúng ta cũng có thể tưởng tượng các điểm giống như những nam châm được gắn chặt vào sàn (nên chúng không thể di chuyển). Bây giờ hãy tưởng tượng ném một thanh kim loại thẳng lên trên chúng. Thanh sẽ di chuyển xung quanh, nhưng do nam châm kéo nó nên cuối cùng nó sẽ ở vị trí cân bằng, càng gần tất cả các điểm càng tốt.

Tất nhiên, điều này có thể dẫn đến rất nhiều sự mơ hồ. Chúng ta muốn một con đường đi gần đến tất cả các ngôi nhà, hay có thể thực sự gần một vài ngôi nhà và xa hơn một chút so với những ngôi nhà khác? Một số câu hỏi phát sinh sau:

- Chúng ta muốn nói gì khi nói “các điểm gần giống như đang tạo thành một đường”? Chúng ta
- có ý nghĩa gì khi nói “một đường đi rất gần các điểm”? Làm thế nào để chúng ta tìm thấy một
- dòng như vậy? Tại sao điều này lại hữu ích trong thế giới thực? Tại sao máy này lại học?
-
-

Trong chương này, chúng tôi trả lời tất cả những câu hỏi này và xây dựng mô hình hồi quy tuyến tính để dự đoán giá nhà đất trong một tập dữ liệu thực.

Bạn có thể tìm thấy tất cả mã cho chương này trong kho GitHub sau:

https://github.com/luisguiserrano/manning/tree/master/Chapter_3_Linear_Regressio

Vấn đề: Chúng ta cần dự đoán giá nhà

Giả sử chúng tôi là đại lý bất động sản chịu trách nhiệm bán một căn nhà mới. Chúng tôi không biết giá và chúng tôi muốn suy ra giá đó bằng cách so sánh nó với những ngôi nhà khác. Chúng tôi xem xét các đặc điểm của ngôi nhà có thể ảnh hưởng đến giá cả, chẳng hạn như kích thước, số phòng, vị trí, tỷ lệ tội phạm, chất lượng trường học và khoảng cách đến khu thương mại. Cuối cùng, chúng ta muốn có một công thức cho tất cả những đặc điểm này để cho chúng ta biết giá căn nhà, hoặc ít nhất là ước tính chính xác cho nó.

Giải pháp: Xây dựng mô hình hồi quy giá nhà đất

Hãy lấy một ví dụ đơn giản nhất có thể. Chúng tôi chỉ xem xét một trong các đặc điểm - số lượng phòng. Nhà của chúng tôi có bốn phòng, gần đó có sáu ngôi nhà, lần lượt có một, hai, ba, năm, sáu và bảy phòng. Giá của chúng được thể hiện trong bảng 3.1.

Bảng 3.1 Bảng các nhà có số phòng và giá. Nhà 4 là nhà mà chúng tôi đang cố gắng suy ra giá.

Number of rooms	Price
1	150
2	200
3	250
4	?
5	350
6	400
7	450

What price would you give to house 4, just based on the information on this table? If you said \$300, then we made the same guess. You probably saw a pattern and used it to infer the price of the house. What you did in your head was linear regression. Let’s study this pattern more. You may have noticed that each time you add a room, \$50 is added to the price of the house. More specifically, we can think of the price of a house as a combination of two things: a base price of \$100, and an extra charge of \$50 for each of the rooms. This can be summarized in a simple formula:

$$\text{Price} = 100 + 50(\text{Number of rooms})$$

What we did here is come up with a model represented by a formula that gives us a *prediction* of the price of the house, based on the *feature*, which is the number of rooms. The price per room is called the *weight* of that corresponding feature, and the base price is called the *bias* of the model. These are all important concepts in machine learning. We learned some of them in chapter 1 and 2, but let’s refresh our memory by defining them from the perspective of this problem.

features The features of a data point are those properties that we use to make our prediction. In this case, the features are the number of rooms in the house, the crime rate, the age of the house, the size, and so on. For our case, we’ve decided on one feature: the number of rooms in the house.

labels This is the target that we try to predict from the features. In this case, the label is the price of the house.

model A machine learning model is a rule, or a formula, which predicts a label from the features. In this case, the model is the equation we found for the price.

prediction The prediction is the output of the model. If the model says, “I think

the house with four rooms is going to cost \$300,” then the prediction is 300.

weights In the formula corresponding to the model, each feature is multiplied by a corresponding factor. These factors are the weights. In the previous formula, the only feature is the number of rooms, and its corresponding weight is 50.

bias As you can see, the formula corresponding to the model has a constant that is not attached to any of the features. This constant is called the bias. In this model, the bias is 100, and it corresponds to the base price of a house.

Now the question is, how did we come up with this formula? Or more specifically, how do we get the computer to come up with this weight and bias? To illustrate this, let’s look at a slightly more complicated example. And because this is a machine learning problem, we will approach it using the remember-formulate-predict framework that we learned in chapter 2. More specifically, we’ll *remember* the prices of other houses, *formulate* a model for the price, and use this model to *predict* the price of a new house.

The remember step: Looking at the prices of existing houses

To see the process more clearly, let’s look at a slightly more complicated dataset, such as the one in table 3.2.

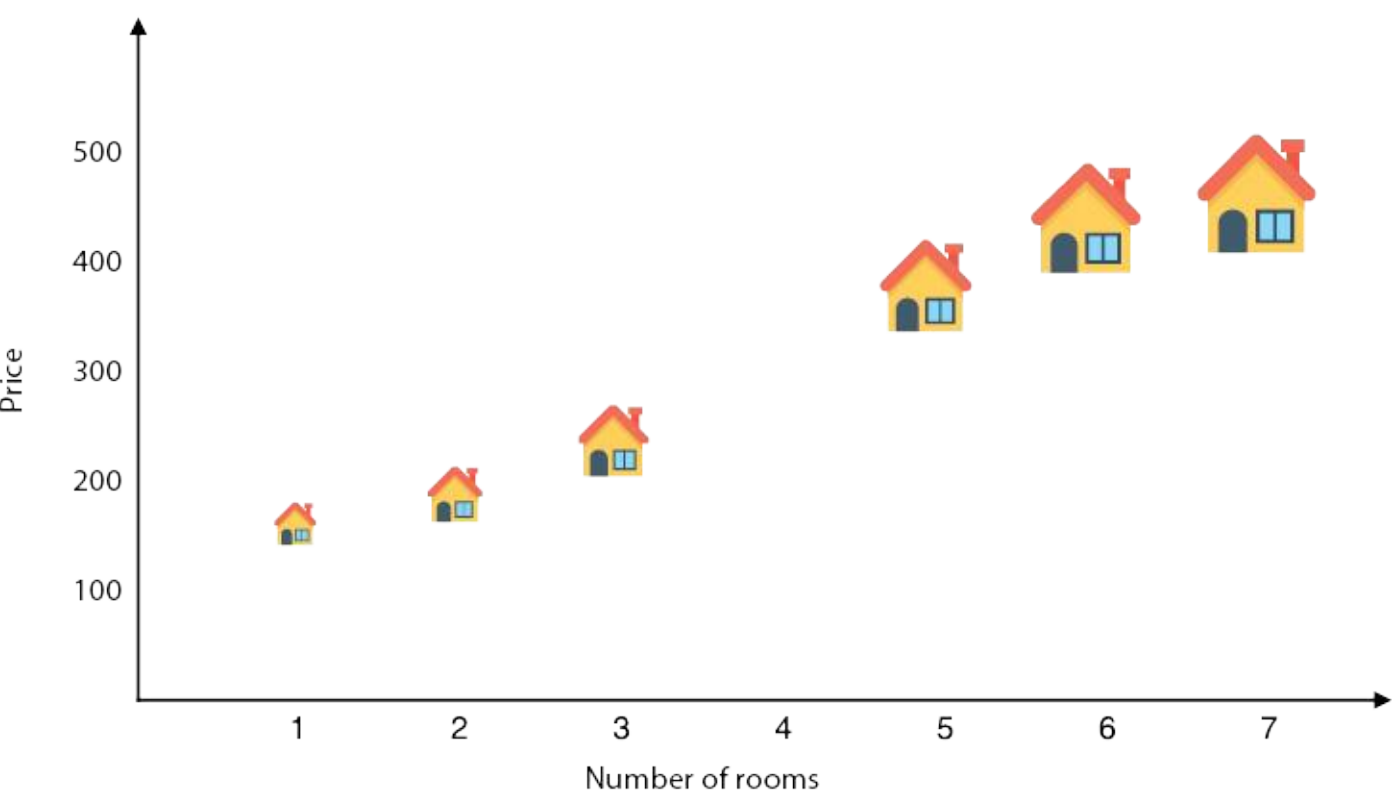
Table 3.2 A slightly more complicated dataset of houses with their number of rooms and their price

Number of rooms	Price
1	155
2	197
3	244
4	?
5	356
6	407
7	448

This dataset is similar to the previous one, except now the prices don’t follow a nice pattern, where each price is \$50 more than the previous one. However, it’s not that far from the original dataset, so we can expect that a similar pattern

nên xấp xỉ tốt những giá trị này.

Thông thường, điều đầu tiên chúng ta làm khi nhận được một tập dữ liệu mới là vẽ biểu đồ cho nó. Trong hình 3.3, chúng ta có thể thấy đồ thị các điểm trong hệ tọa độ trong đó trục hoành biểu thị số phòng và trục tung biểu thị giá căn nhà.



Hình 3.3 Sơ đồ tập dữ liệu trong bảng 3.2. Trục hoành biểu thị số phòng và trục tung biểu thị giá căn nhà.

Bước xây dựng: Xây dựng quy tắc ước tính giá nhà

Tập dữ liệu trong bảng 3.2 đủ gần với tập dữ liệu trong bảng 3.1, vì vậy hiện tại, chúng tôi có thể cảm thấy an toàn khi sử dụng cùng một công thức tính giá. Sự khác biệt duy nhất là bây giờ giá cả không chính xác như công thức nêu và chúng tôi có một lỗi nhỏ. Chúng ta có thể viết phương trình như sau:

$$\text{Giá} = 100 + 50(\text{Số phòng}) + (\text{Lỗi nhỏ})$$

Nếu chúng ta muốn dự đoán giá, chúng ta có thể sử dụng phương trình này. Mặc dù chúng tôi không chắc mình sẽ nhận được giá trị thực tế nhưng chúng tôi biết rằng chúng tôi có khả năng đạt được giá trị đó. Bây giờ câu hỏi là, làm thế nào chúng ta tìm được phương trình này? Và quan trọng nhất, làm thế nào để

máy tính tìm được phương trình này?

Chúng ta hãy quay lại cốt truyện và xem phương trình có ý nghĩa gì ở đó. Điều gì sẽ xảy ra nếu chúng ta xét tất cả các điểm có tọa độ dọc (y) bằng 100 cộng 50 lần tọa độ ngang (x)? Tập hợp các điểm này tạo thành một đường thẳng có độ dốc 50 và y -chặn 100. Trước khi chúng ta giải thích câu lệnh trước, đây là các định nghĩa về độ dốc, y -chặn và phương trình của một đường thẳng. Chúng tôi đi sâu vào những vấn đề này một cách chi tiết hơn trong phần “Đo và chạm trên dốc và y -chặn”.

độ dốc Độ dốc của một đường là thước đo độ dốc của nó. Nó được tính bằng cách chia mức tăng trong suốt thời gian chạy (tức là, nó tăng lên bao nhiêu đơn vị, chia cho bao nhiêu đơn vị nó tăng lên ở bên phải). Tỷ lệ này không đổi trên toàn bộ dòng. Trong mô hình học máy, đây là trọng số của tính năng tương ứng và nó cho chúng ta biết chúng ta mong đợi giá trị của nhãn sẽ tăng lên bao nhiêu khi chúng ta tăng giá trị của tính năng đó lên một đơn vị. Nếu đường thẳng nằm ngang thì độ dốc bằng 0 và nếu đường đi xuống thì độ dốc âm.

y -điểm giao nhau Giao điểm y của một đường thẳng là độ cao mà tại đó đường thẳng đó đi qua trục thẳng đứng (y -). Trong mô hình học máy, đó là độ lệch và cho chúng ta biết nhãn sẽ là gì trong một điểm dữ liệu nơi tất cả các tính năng đều chính xác bằng 0.

phương trình tuyến tính Đây là phương trình của một đường thẳng. Nó được cho bởi hai tham số: độ dốc và phần chặn y . Nếu độ dốc là m và giao điểm y là b thì phương trình của đường thẳng là $y = mx + b$ và đường thẳng được tạo thành bởi tất cả các điểm (x, y) thỏa mãn phương trình. Trong mô hình học máy, x là giá trị của đối tượng và y là dự đoán cho nhãn. Trọng số và độ lệch của mô hình lần lượt là m và b .

Bây giờ chúng ta có thể phân tích phương trình. Khi chúng ta nói rằng độ dốc của đường này là 50—điều này có nghĩa là mỗi lần chúng ta thêm một phòng vào ngôi nhà, chúng ta ước tính giá căn nhà sẽ tăng thêm 50 đô la. Khi chúng ta nói rằng giao điểm y của đường này là 100, điều này có nghĩa là ước tính giá của một ngôi nhà (giả định) không có phòng sẽ là giá cơ bản là 100 USD. Đường này được vẽ trong hình 3.4.



Hình 3.4 Mô hình mà chúng tôi xây dựng là đường càng gần tất cả các ngôi nhà càng tốt.

Bây giờ, trong số tất cả các đường có thể (mỗi đường có phương trình riêng), tại sao chúng ta lại chọn đường này? Bởi vì cái đó đi gần đến điểm. Có thể có một cái tốt hơn, nhưng ít nhất chúng ta biết cái này là tốt, trái ngược với cái không đi đến đâu gần điểm. Bây giờ chúng ta quay lại vấn đề ban đầu, trong đó chúng ta có một dãy nhà và chúng ta muốn xây một con đường càng gần chúng càng tốt.

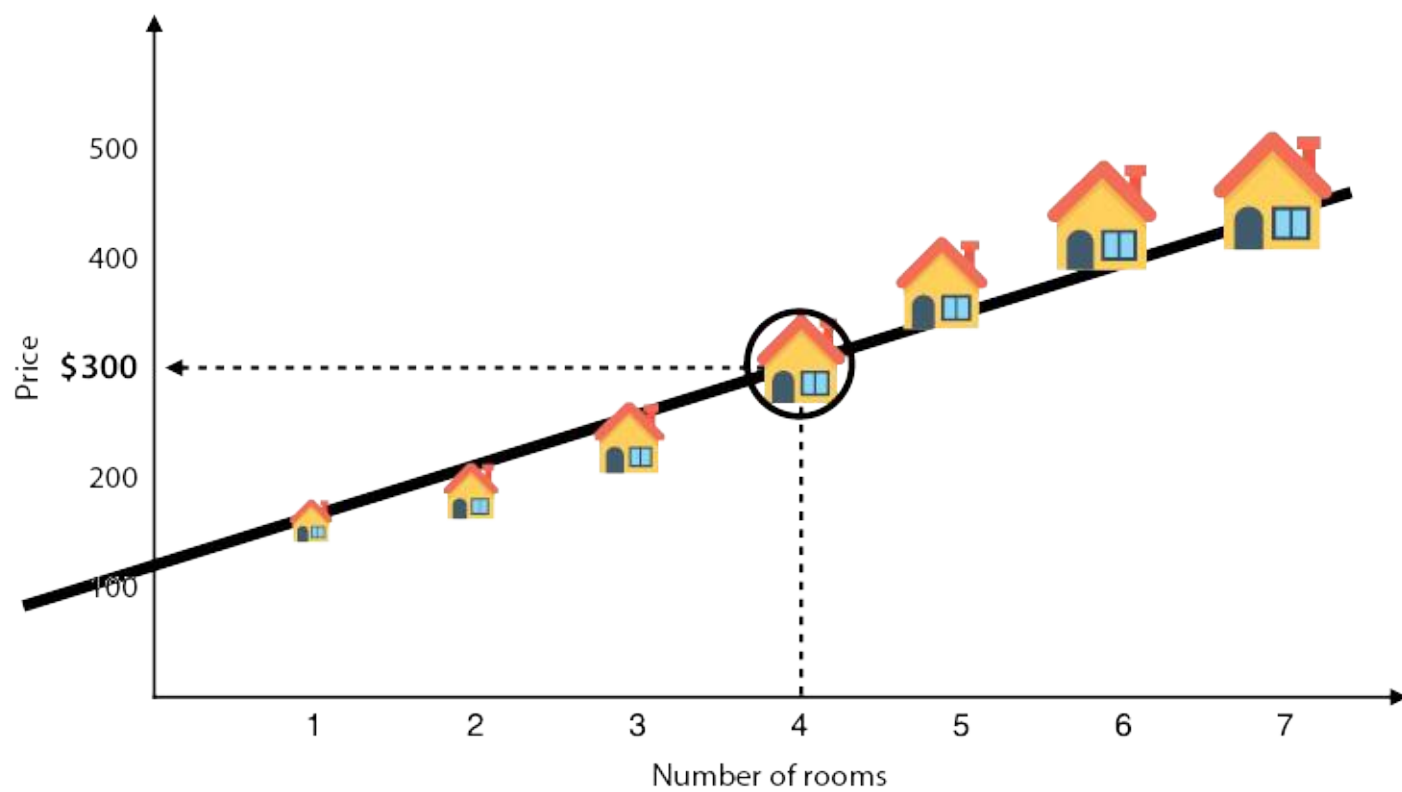
Làm thế nào để chúng ta tìm thấy dòng này? Chúng ta sẽ xem xét điều này sau trong chương này. Nhưng bây giờ, giả sử chúng ta có một quả cầu pha lê, với một loạt điểm, sẽ tìm đường đi gần chúng nhất.

Bước dự đoán: Chúng ta làm gì khi có một ngôi nhà mới xuất hiện trên thị trường?

Bây giờ, hãy sử dụng mô hình của chúng tôi để dự đoán giá của ngôi nhà có bốn phòng. Đối với điều này, chúng tôi cần số bốn làm tính năng trong công thức của chúng tôi để có được kết quả sau:

$$\text{Price} = 100 + 50 \cdot 4 = 300$$

Do đó, mô hình của chúng tôi dự đoán rằng ngôi nhà có giá 300 USD. Điều này cũng có thể được nhìn thấy bằng đồ họa bằng cách sử dụng đường thẳng, như minh họa trong hình 3.5.



Hình 3.5 Nhiệm vụ của chúng ta bây giờ là dự đoán giá của ngôi nhà có 4 phòng. Sử dụng mô hình (đường thẳng), chúng ta suy ra rằng giá dự đoán của ngôi nhà này là 300 USD.

Nếu chúng ta có nhiều biến hơn thì sao? Hồi quy tuyến tính đa biến

Trong các phần trước, chúng ta đã tìm hiểu về mô hình dự đoán giá một ngôi nhà dựa trên một đặc điểm—số phòng. Chúng ta có thể tưởng tượng ra nhiều đặc điểm khác có thể giúp chúng ta dự đoán giá một ngôi nhà, chẳng hạn như quy mô, chất lượng của các trường học trong khu vực lân cận và tuổi của ngôi nhà. Mô hình hồi quy tuyến tính của chúng ta có thể điều chỉnh các biến khác này không? Tuyệt đối. Khi đặc điểm duy nhất là số lượng phòng, mô hình của chúng tôi sẽ dự đoán giá bằng tổng của đặc điểm đó nhân với trọng số tương ứng của chúng, cộng với độ lệch. Nếu chúng ta có nhiều tính năng hơn, tất cả những gì chúng ta cần làm là nhân chúng với trọng số tương ứng và cộng chúng vào giá dự đoán. Do đó, mô hình giá một ngôi nhà có thể trông như thế này:

$$\text{Giá} = 30(\text{số phòng}) + 1.5(\text{kích thước}) + 10(\text{chất lượng trường học}) - 2(\text{tuổi nhà}) + 50$$

Trong phương trình này, tại sao tất cả các trọng số đều dương, ngoại trừ trọng số tương ứng với tuổi của ngôi nhà? Lý do là ba đặc điểm còn lại (số phòng, quy mô và chất lượng của trường học) có mối tương quan tích cực với

giá của ngôi nhà. Nói cách khác, vì những ngôi nhà lớn hơn và có vị trí tốt có giá cao hơn nên đặc điểm này càng cao thì chúng ta kỳ vọng giá căn nhà sẽ càng cao. Tuy nhiên, vì chúng ta tưởng tượng rằng những ngôi nhà cũ có xu hướng rẻ hơn nên đặc điểm tuổi tác *tương quan nghịch* với giá căn nhà.

Điều gì xảy ra nếu trọng số của một tính năng bằng 0? Điều này xảy ra khi một tính năng không liên quan đến giá cả. Ví dụ: hãy tưởng tượng một tính năng đo số lượng hàng xóm có họ bắt đầu bằng chữ A. Đặc điểm này hầu như không liên quan đến giá căn nhà, vì vậy chúng ta kỳ vọng rằng trong một mô hình hợp lý, trọng số tương ứng với đặc điểm này hoặc bằng 0 hoặc gần bằng nó.

Theo cách tương tự, nếu một đặc điểm có trọng số rất cao (dù âm hay dương), chúng tôi giải thích điều này là mô hình cho chúng ta biết rằng đặc điểm đó quan trọng trong việc xác định giá căn nhà. Trong mô hình trước, có vẻ như số lượng phòng là một đặc điểm quan trọng vì trọng lượng của nó là lớn nhất (về giá trị tuyệt đối).

Trong phần có tên “Giảm kích thước giúp đơn giản hóa dữ liệu mà không mất quá nhiều thông tin” ở chương 2, chúng tôi đã liên hệ số cột trong tập dữ liệu với thứ nguyên mà tập dữ liệu tồn tại. Do đó, tập dữ liệu có hai cột có thể được biểu diễn dưới dạng tập hợp các điểm trong mặt phẳng và tập dữ liệu có ba cột có thể được biểu diễn dưới dạng tập hợp các điểm trong không gian ba chiều. Trong tập dữ liệu như vậy, mô hình hồi quy tuyến tính không tương ứng với một đường thẳng mà tương ứng với một mặt phẳng đi càng gần các điểm càng tốt. Hãy tưởng tượng có nhiều con ruồi bay quanh phòng ở tư thế đứng yên, và nhiệm vụ của chúng ta là cố gắng đưa một tấm bìa cứng khổng lồ đến gần tất cả những con ruồi nhất có thể. Đây là hồi quy tuyến tính đa biến với ba biến. Vấn đề trở nên khó hình dung đối với các tập dữ liệu có nhiều cột hơn, nhưng chúng ta luôn có thể tưởng tượng ra một phương trình tuyến tính có nhiều biến.

Trong chương này, chúng ta chủ yếu đề cập đến việc huấn luyện các mô hình hồi quy tuyến tính chỉ với một tính năng, nhưng quy trình tương tự với nhiều tính năng hơn. Tôi khuyến khích bạn đọc về nó trong khi vẫn ghi nhớ sự thật này trong đầu và tưởng tượng cách bạn khái quát hóa từng tuyên bố tiếp theo của chúng tôi cho một trường hợp có một số đặc điểm.

Một số câu hỏi phát sinh và một số câu trả lời nhanh

Được rồi, đầu bạn có thể đang vang lên rất nhiều câu hỏi. Hãy giải quyết một số (hy vọng là tất cả) trong số đó!

1. Điều gì xảy ra nếu mô hình mắc lỗi? 2. Làm thế nào bạn nghĩ ra công thức dự đoán giá? Và chúng ta sẽ làm gì nếu thay vì sáu ngôi nhà, chúng ta có hàng nghìn ngôi nhà? 3. Giả sử chúng tôi đã xây dựng mô hình dự đoán này và sau đó những ngôi nhà mới bắt đầu xuất hiện trên thị trường. Có cách nào để cập nhật mô hình với thông tin mới không?

ch này apter trả lời tất cả những câu hỏi này, nhưng đây là một số câu trả lời nhanh :

1. Điều gì xảy ra nếu mô hình mắc lỗi?

Mô hình đang ước tính giá của một ngôi nhà, vì vậy chúng tôi cho rằng nó sẽ thường xuyên mắc một sai sót nhỏ vì rất khó để đưa ra mức giá chính xác. Quá trình đào tạo bao gồm việc tìm ra mô hình mắc lỗi nhỏ nhất tại các điểm của chúng tôi.

2. Làm thế nào bạn nghĩ ra được công thức dự đoán giá? Và chúng ta sẽ làm gì nếu thay vì sáu ngôi nhà, chúng ta có hàng nghìn ngôi nhà?

Vâng, đây là câu hỏi chính mà chúng tôi sẽ giải quyết trong chương này! Khi chúng ta có sáu ngôi nhà, việc vẽ một đường gần chúng rất đơn giản, nhưng nếu chúng ta có hàng nghìn ngôi nhà thì nhiệm vụ này trở nên khó khăn. Những gì chúng ta làm trong chương này là đưa ra một thuật toán hoặc một quy trình để máy tính tìm ra một đường thẳng phù hợp.

3. Giả sử chúng ta đã xây dựng mô hình dự đoán này và sau đó những ngôi nhà mới bắt đầu xuất hiện trên thị trường. Có cách nào để cập nhật mô hình với thông tin mới không?

Tuyệt đối! Chúng tôi sẽ xây dựng mô hình sao cho có thể dễ dàng cập nhật nếu dữ liệu mới xuất hiện. Đây luôn là điều cần tìm kiếm trong học máy. Nếu chúng ta xây dựng mô hình của mình theo cách mà chúng ta cần tính toán lại toàn bộ mô hình mỗi khi có dữ liệu mới thì điều đó sẽ không hữu ích lắm.

Làm cách nào để máy tính vẽ được đường này: Thuật toán hồi quy tuyến tính

Bây giờ chúng ta đi đến câu hỏi chính của chương này: làm thế nào để máy tính vẽ được một đường đi thật gần các điểm? Cách chúng tôi làm điều này cũng giống như cách chúng tôi làm nhiều việc trong học máy: từng bước một. Bắt đầu một cách ngẫu nhiên

và tìm ra cách cải thiện đường này *một chút* bằng cách di chuyển nó đến gần các điểm hơn. Lặp lại quá trình này nhiều lần và voilà, chúng ta có dòng mong muốn. Quá trình này được gọi là thuật toán hồi quy tuyến tính.

Quy trình này nghe có vẻ ngớ ngẩn nhưng nó thực sự hiệu quả. Bắt đầu với một dòng ngẫu nhiên. Chọn một điểm ngẫu nhiên trong tập dữ liệu và di chuyển đường này đến gần điểm đó hơn một chút. Lặp lại quá trình này nhiều lần, luôn chọn một điểm ngẫu nhiên trong tập dữ liệu. Mã giả cho thuật toán hồi quy tuyến tính, được xem theo kiểu hình học này, như sau. Minh họa được thể hiện trên hình 3.6.

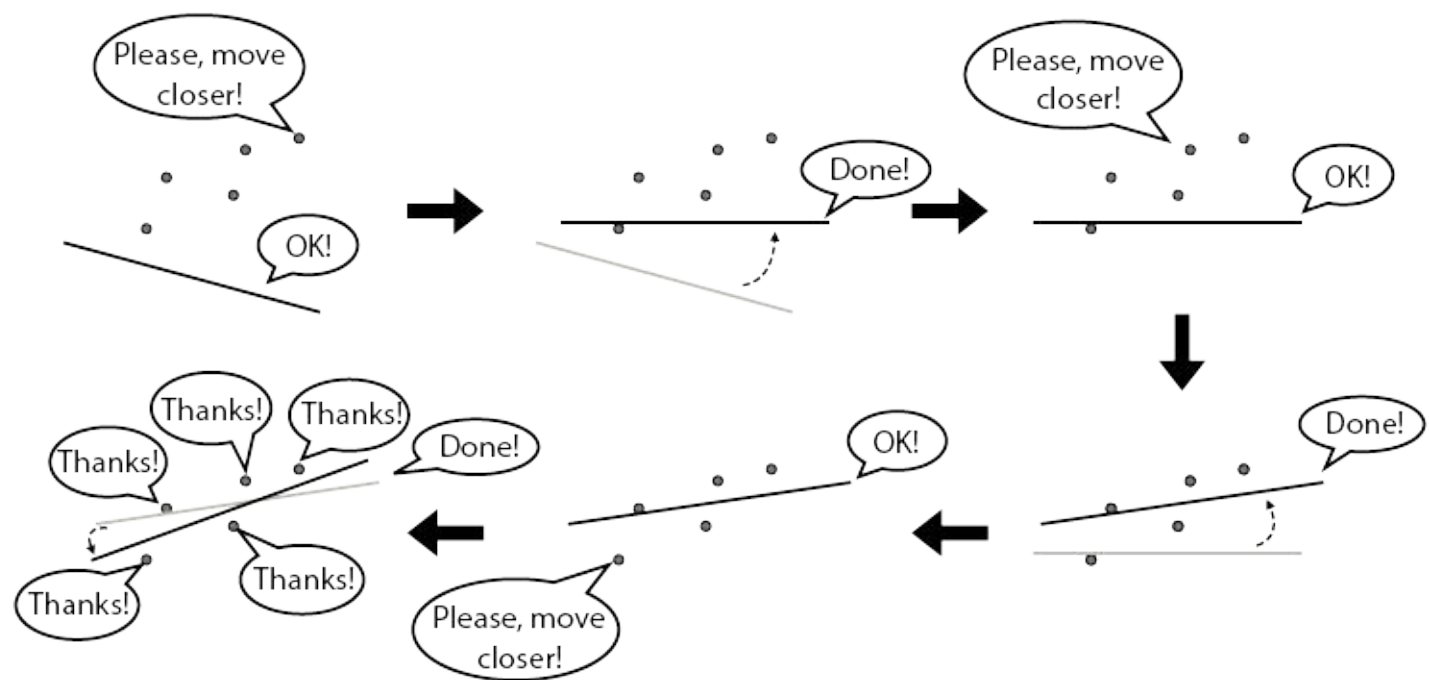
Mã giả cho thuật toán hồi quy tuyến tính (Hình học)

Đầu vào: Tập dữ liệu các điểm trên mặt phẳng

Đầu ra: Một đường đi gần các điểm

Thủ tục:

- Chọn một dòng ngẫu nhiên. Lặp lại nhiều lần: Chọn
- một điểm dữ liệu ngẫu nhiên. Di chuyển đường thẳng đến gần điểm đó hơn một chút. **Trả lại** đường thẳng bạn đã thu được.
-



Hình 3.6 Minh họa thuật toán hồi quy tuyến tính. Chúng tôi bắt đầu ở trên cùng bên trái bằng một dòng ngẫu nhiên và kết thúc ở phía dưới bên trái bằng một dòng phù hợp với tập dữ liệu. Ở mỗi giai đoạn, có hai điều xảy ra: (1) chúng ta chọn một điểm ngẫu nhiên và (2) điểm đó yêu cầu đường thẳng di chuyển đến gần nó hơn. Sau nhiều lần lặp lại, đường này sẽ ở vị trí tốt. Hình này chỉ có ba lần lặp nhằm mục đích minh họa, nhưng trong thực tế, cần nhiều lần lặp hơn nữa.

Đó là quan điểm cấp cao. Để nghiên cứu quá trình chi tiết hơn, chúng ta cần đi sâu vào các chi tiết toán học. Hãy bắt đầu bằng cách xác định một số biến.

- p : Giá nhà trong tập dữ liệu : Giá dự đoán của
- $\hat{p}$: Giá nhà dự đoán
- r : Số phòng m : Giá mỗi phòng b : Giá cơ sở cho một căn nhà
-
-

Tại sao mũ lại vượt quá giá dự đoán, ? Xuyên suốt cuốn sách này, chiếc mũ chỉ ra rằng đây chính là biến số mà mô hình của chúng tôi đang dự đoán. Bằng cách đó, chúng ta có thể biết giá thực tế của một ngôi nhà trong tập dữ liệu từ giá dự đoán của nó.

Do đó, phương trình của mô hình hồi quy tuyến tính dự đoán giá là giá cơ sở cộng với giá mỗi phòng nhân với số phòng là

$$\hat{p} = mr + b.$$

P : giá nhà trong tập dữ liệu
 $\hat{P}$: giá nhà dự đoán
 r : số phòng
 m : giá mỗi phòng
 b : giá cơ sở cho 1 căn nhà

Đây là một cách nói mang tính công thức

Giá dự đoán = (Giá mỗi phòng)(Số phòng) + Giá cơ bản của căn nhà.

Để có ý tưởng về thuật toán hồi quy tuyến tính, hãy tưởng tượng rằng chúng ta có một mô hình trong đó giá mỗi phòng là 40 USD và giá cơ bản của căn nhà là 50 USD. Mô hình này dự đoán giá nhà bằng công thức sau:

$$\hat{p} = 40 \cdot r + 50$$

Để minh họa thuật toán hồi quy tuyến tính, hãy tưởng tượng rằng trong tập dữ liệu của chúng tôi, chúng tôi có một ngôi nhà có hai phòng có giá 150 USD. Mô hình này dự đoán giá căn nhà là $50 + 40 \cdot 2 = 130$. Đó là một dự đoán không tồi nhưng nó thấp hơn giá căn nhà. Làm thế nào chúng ta có thể cải thiện mô hình? Có vẻ như sai lầm của người mẫu là cho rằng căn nhà quá rẻ. Có thể mô hình đó có giá cơ bản thấp hoặc có thể nó có giá mỗi phòng thấp hoặc có thể cả hai. Nếu chúng ta tăng cả hai lên một lượng nhỏ, chúng ta có thể có được ước tính tốt hơn. Hãy tăng giá mỗi phòng thêm 0,5 USD và giá cơ bản thêm 1 USD. (Tôi chọn những con số này một cách ngẫu nhiên). Phương trình mới như sau:

$$\hat{p} = 40.5 \cdot r + 51$$

Giá dự đoán mới cho ngôi nhà là $40.5 \cdot r + 51 = 132$. Vì \$132 gần với \$150 nên mô hình mới của chúng tôi đưa ra dự đoán tốt hơn cho ngôi nhà này. Vì vậy, đây là mô hình tốt hơn cho điểm dữ liệu đó. Chúng tôi không biết liệu đây có phải là mô hình tốt hơn cho các điểm dữ liệu khác hay không, nhưng hiện tại, đừng lo lắng về điều đó. Ý tưởng của thuật toán hồi quy tuyến tính là lặp lại quá trình trước đó nhiều lần. Mã giả của thuật toán hồi quy tuyến tính như sau:

PSEUDOCODE CHO Thuật toán hồi quy tuyến tính

Đầu vào: Tập dữ liệu về điểm

Đầu ra: Mô hình hồi quy tuyến tính phù hợp với tập dữ liệu đó

Thủ tục:

- Chọn một mô hình có trọng số ngẫu nhiên và độ lệch ngẫu nhiên.

- Lặp lại nhiều lần: Chọn một điểm dữ liệu ngẫu nhiên. Điều chỉnh một chút trọng số và độ lệch để cải thiện dự đoán cho điểm dữ liệu cụ thể đó. **Trả về** mô hình bạn đã thu được.
-
-

Bạn có thể có một số câu hỏi, chẳng hạn như sau:

- Tôi nên điều chỉnh trọng lượng bao nhiêu? Tôi nên lặp lại thuật toán bao nhiêu lần? Nói cách
- khác, làm sao tôi biết khi nào tôi đã hoàn thành? Làm sao tôi biết thuật toán này hoạt động?
-

Chúng tôi trả lời tất cả những câu hỏi này trong chương này. Trong phần “Thủ thuật bình phương” và “Thủ thuật tuyệt đối”, chúng ta tìm hiểu một số thủ thuật thú vị để tìm giá trị tốt để điều chỉnh trọng số. Trong phần “Lỗi tuyệt đối” và “Lỗi bình phương”, chúng ta thấy hàm lỗi sẽ giúp chúng ta quyết định khi nào nên dừng thuật toán. Và cuối cùng, trong phần “Giảm dần độ dốc”, chúng tôi đề cập đến một phương pháp mạnh mẽ được gọi là giảm dần độ dốc, giải thích lý do tại sao thuật toán này hoạt động. Nhưng trước tiên, hãy bắt đầu bằng cách di chuyển các đường trong mặt phẳng.

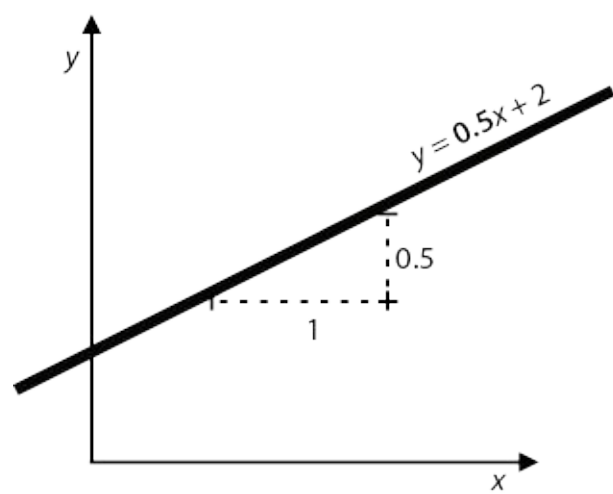
Đường va chạm trên dốc và điểm giao cắt y

Trong phần “Bước lập công thức”, chúng ta đã nói về phương trình của đường thẳng. Trong phần này, chúng ta tìm hiểu cách vận dụng phương trình này để di chuyển đường thẳng của chúng ta. Hãy nhớ rằng phương trình của một đường thẳng có hai thành phần sau:

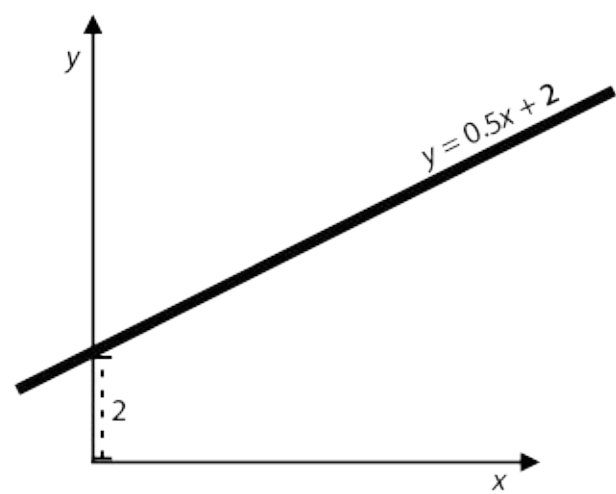
- Độ dốc Điểm chặn
- y

Độ dốc cho chúng ta biết độ dốc của đường này và giao điểm y cho chúng ta biết đường đó nằm ở đâu. Độ dốc được định nghĩa là độ cao chia cho đường chạy và điểm chặn y cho chúng ta biết nơi đường đi qua trục y (trục tung). Trong hình 3.7, chúng ta có thể thấy cả hai trong một ví dụ. Dòng này có phương trình sau:

$$y = 0.5x + 2$$



Slope = 0.5



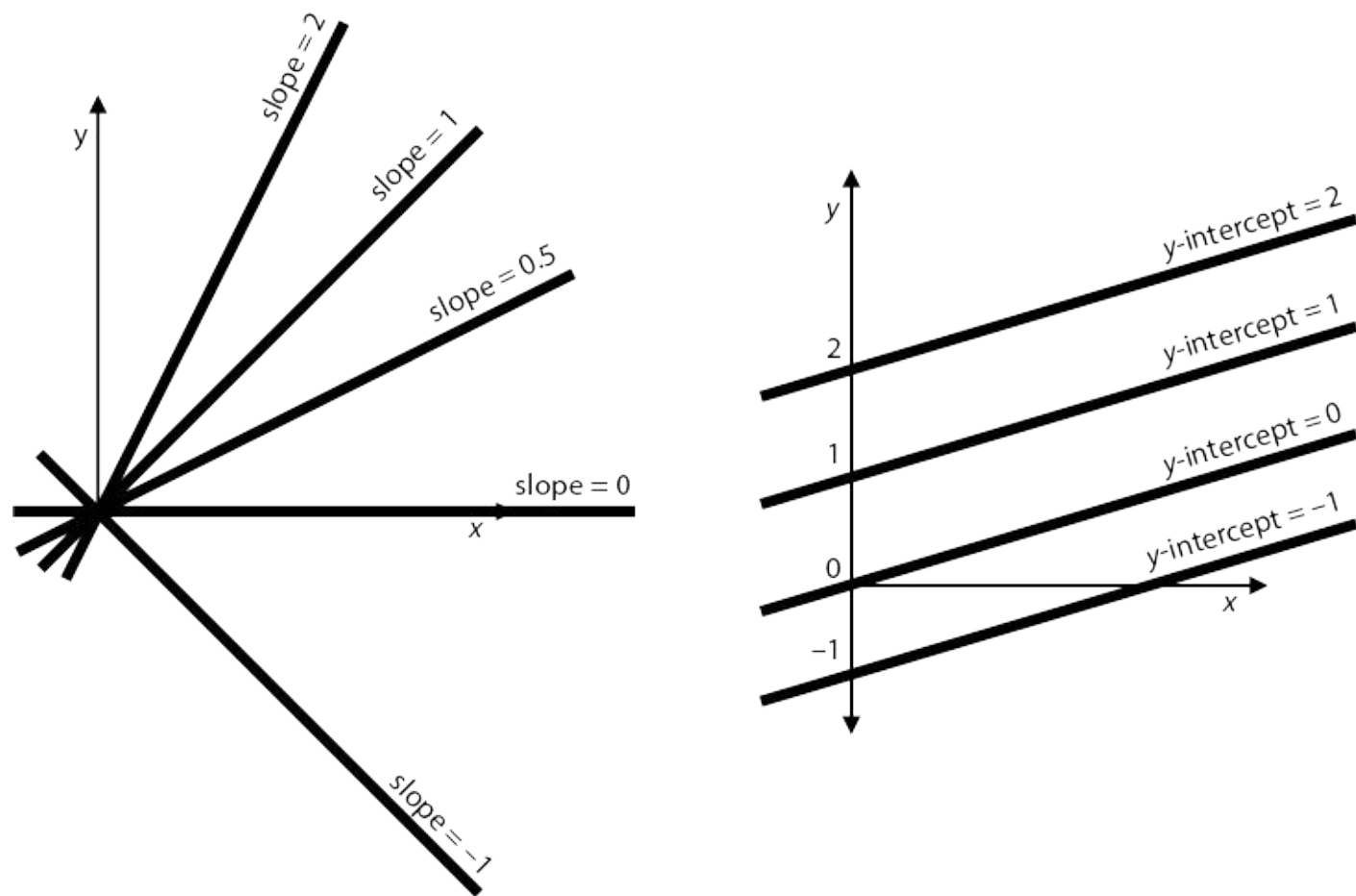
y-intercept = 2

Hình 3.7 Đường thẳng có phương trình $y = 0,5x + 2$ có độ dốc 0,5 (trái) và y-chặn 2 (phải).

Phương trình này có ý nghĩa gì? Nghĩa là độ dốc là 0,5 và phần chặn y là 2.

Khi chúng ta nói hệ số góc là 0,5, điều đó có nghĩa là khi chúng ta đi dọc theo đường này, với mỗi đơn vị chúng ta di chuyển sang phải, chúng ta đang di chuyển lên trên 0,5 đơn vị. Độ dốc có thể bằng 0 nếu chúng ta không di chuyển lên hoặc âm nếu chúng ta di chuyển xuống. Một đường thẳng đứng có độ dốc không xác định, nhưng may mắn thay, những đường thẳng này không có xu hướng hiển thị trong hồi quy tuyến tính. Nhiều đường có thể có cùng độ dốc. Nếu tôi vẽ bất kỳ đường thẳng nào song song với đường trong hình 3.7, đường này cũng sẽ tăng 0,5 đơn vị cho mỗi đơn vị nó di chuyển sang phải. Đây là nơi mà giao điểm y xuất hiện. Giao điểm y cho chúng ta biết nơi đường cắt trục y. Đường này cắt trục x ở độ cao 2 và đó là giao điểm y.

Nói cách khác, độ dốc của đường thẳng cho chúng ta biết *hướng* mà đường thẳng đang hướng tới, và giao điểm y cho chúng ta biết *vị trí* của đường thẳng. Lưu ý rằng bằng cách xác định độ dốc và giao điểm y, đường thẳng được xác định hoàn toàn. Trong hình 3.8, chúng ta có thể thấy các đường khác nhau có cùng đoạn chặn y và các đường khác nhau có cùng độ dốc.



Hình 3.8 Một số ví dụ về độ dốc và điểm chặn y . Ở bên trái, chúng ta thấy một số đường có cùng điểm giao cắt và độ dốc khác nhau. Lưu ý rằng độ dốc càng cao thì đường càng dốc. Ở bên phải, chúng ta thấy một số đường có cùng độ dốc và các giao điểm y khác nhau. Lưu ý rằng điểm chặn y càng cao thì đường này càng nằm ở vị trí cao hơn.

Trong ví dụ về nhà ở hiện tại của chúng ta, độ dốc biểu thị giá mỗi phòng và điểm chặn y biểu thị giá cơ bản của một ngôi nhà. Hãy ghi nhớ điều này và khi chúng ta thao tác các đường, hãy nghĩ xem điều này đang tác động gì đến mô hình giá nhà đất của chúng ta.

Từ định nghĩa về độ dốc và giao điểm y , chúng ta có thể suy ra như sau g:

Thay đổi độ dốc:

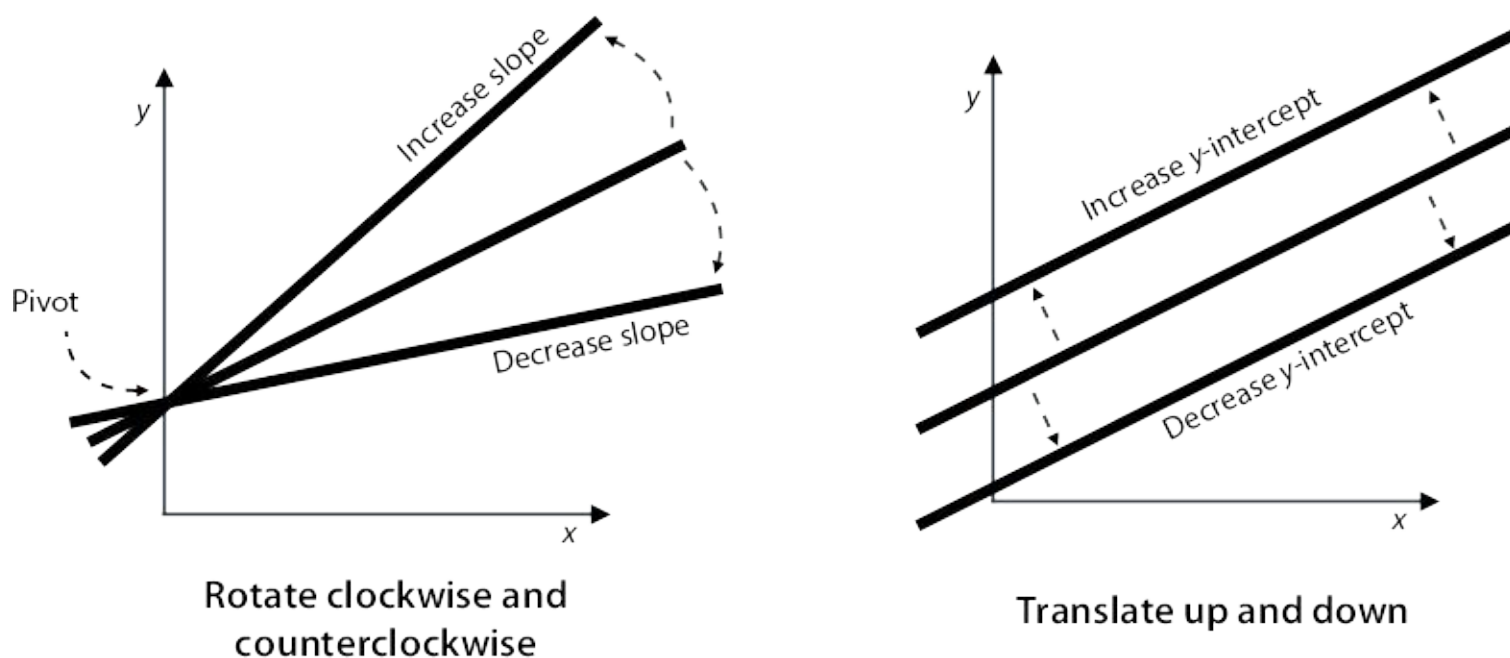
- Nếu chúng ta tăng độ dốc của một đường thì đường đó sẽ quay ngược chiều kim đồng hồ. Nếu chúng ta giảm độ dốc của một đường thì đường đó sẽ quay theo chiều kim đồng hồ.

Các phép quay này nằm trên trục quay như trong hình 3.9, cụ thể là giao điểm của đường thẳng và trục y .

Thay đổi y -chặn:

- Nếu chúng ta tăng phần chặn y của một dòng, thì dòng đó sẽ được dịch lên trên. Nếu
- chúng ta giảm phần chặn y của một dòng, thì dòng đó sẽ được dịch xuống dưới.

Hình 3.9 minh họa các phép quay và tịnh tiến này, chúng sẽ rất hữu ích khi chúng ta muốn điều chỉnh các mô hình hồi quy tuyến tính của mình.



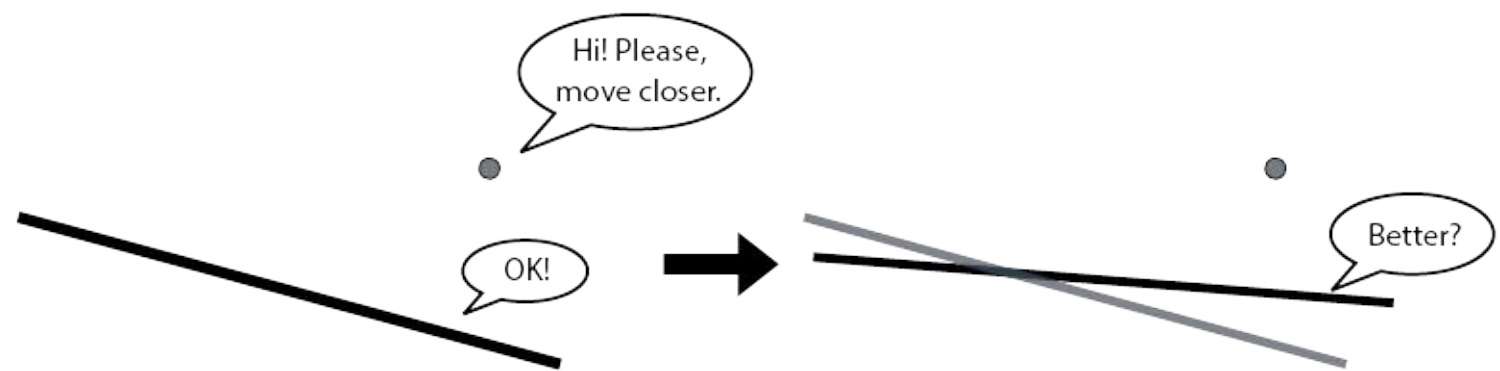
Hình 3.9 Bên trái: Việc tăng độ dốc sẽ xoay đường ngược chiều kim đồng hồ, trong khi việc giảm độ dốc sẽ xoay đường thẳng theo chiều kim đồng hồ. Phải: Tăng điểm chặn y sẽ dịch đường thẳng lên trên, trong khi giảm điểm chặn y sẽ dịch đường thẳng xuống dưới.

Như đã giải thích trước đó, nói chung, phương trình của một đường thẳng được viết là $y = mx + b$, trong đó x và y tương ứng với tọa độ ngang và dọc, m tương ứng với độ dốc và b tương ứng với y -chặn. Trong suốt chương này, để phù hợp với ký hiệu, chúng ta sẽ viết phương trình là $y = mr + b$, tương ứng với giá dự đoán, r tương ứng với số phòng, m (độ dốc) tương ứng với giá mỗi phòng và b (độ dốc) y -chặn) vào giá cơ sở của căn nhà.

Một thủ thuật đơn giản để di chuyển một đường thẳng đến gần một tập hợp các điểm, mỗi lần một điểm

Hãy nhớ lại rằng thuật toán hồi quy tuyến tính bao gồm việc lặp lại một bước trong đó chúng ta di chuyển một đường thẳng đến gần một điểm hơn. Chúng ta có thể làm điều này bằng cách sử dụng phép quay và bản dịch. Trong phần này, chúng ta học một thủ thuật được gọi là *thủ thuật đơn giản*, bao gồm việc xoay nhẹ và dịch đường thẳng theo hướng của điểm để di chuyển nó lại gần hơn

(hình 3.10).



Hình 3.10 Mục tiêu của chúng ta là xoay và dịch đường thẳng một chút để đến gần điểm hơn.

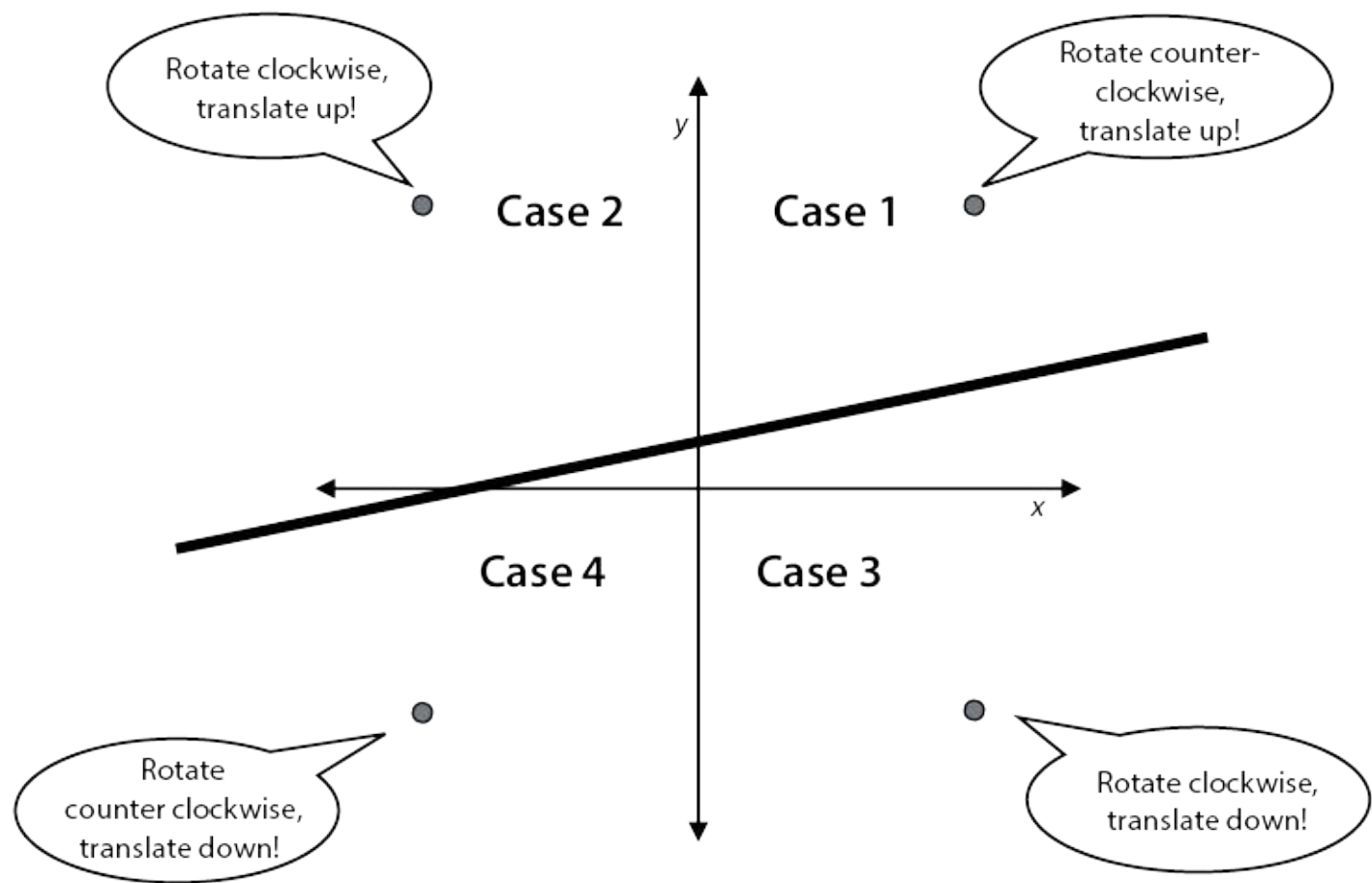
Bí quyết để di chuyển đường thẳng về phía một điểm một cách chính xác là xác định điểm đó nằm ở đâu so với đường thẳng. Nếu điểm ở trên dòng thì dịch dòng lên, nếu điểm ở dưới thì dịch xuống. Xoay khó hơn một chút, nhưng vì trục xoay là điểm giao nhau của đường thẳng và trục y nên chúng ta có thể thấy rằng nếu điểm nằm phía trên đường thẳng và ở bên phải trục y hoặc bên dưới đường thẳng và ở bên trái của trục y thì chúng ta cần xoay đường thẳng ngược chiều kim đồng hồ. Trong hai trường hợp còn lại, chúng ta cần xoay đường thẳng theo chiều kim đồng hồ. Những điều này được tóm tắt trong bốn trường hợp sau, được minh họa trong hình 3.11:

Trường hợp 1: Nếu điểm nằm phía trên đường thẳng và bên phải trục y thì ta xoay đường thẳng ngược chiều kim đồng hồ và dịch nó lên trên.

Trường hợp 2: Nếu điểm nằm phía trên đường thẳng và bên trái trục y thì ta xoay đường thẳng theo chiều kim đồng hồ và dịch nó lên trên.

Trường hợp 3: Nếu điểm nằm dưới đường thẳng và bên phải trục y thì ta xoay đường thẳng theo chiều kim đồng hồ và dịch nó xuống dưới.

Trường hợp 4: Nếu điểm nằm dưới đường thẳng và ở bên trái trục y thì chúng ta xoay đường thẳng ngược chiều kim đồng hồ và dịch nó xuống dưới.



Hình 3.11 Bốn trường hợp. Trong mỗi cách này, chúng ta phải xoay đường thẳng và dịch nó theo một cách khác để di chuyển đường thẳng đó đến gần điểm tương ứng hơn.

Bây giờ chúng ta có bốn trường hợp này, chúng ta có thể viết mã giả cho thủ thuật đơn giản. Nhưng trước tiên, hãy làm rõ một số ký hiệu. Trong phần này chúng ta đã nói về các đường có phương trình $y = mx + b$, trong đó m là độ dốc và b là giao điểm y . Trong ví dụ về nhà ở, chúng tôi đã sử dụng ký hiệu tương tự sau:

- Điểm có tọa độ (r, p) tương ứng với ngôi nhà có r phòng và giá p . Độ dốc m tương ứng với giá phòng. Điểm y -chặn b tương ứng với giá cơ bản của căn nhà. Dự đoán $= mr + b$ tương ứng với giá dự đoán của căn nhà.
-
- $\hat{p}$

Mã giả cho thủ thuật đơn giản

Đầu vào:

- Một đường thẳng có độ dốc m , y -chặn b và phương trình $= \hat{m}r + b$
- Một điểm có tọa độ (r, p)

Đầu ra:

- Đường thẳng có phương trình $y = m'x + b$ gần điểm hơn

Thủ tục:

Chọn hai số ngẫu nhiên rất nhỏ và gọi chúng là η_1 và η_2 (chữ cái Hy Lạp *eta*).

Trường hợp 1: Nếu điểm nằm phía trên đường thẳng và bên phải trục y thì ta xoay đường thẳng ngược chiều kim đồng hồ và dịch nó lên trên:

- Thêm vào η_1 đến độ dốc m . Lấy $m' = m + \eta_1$. Thêm vào
- η_2 tới điểm chặn y . Lấy được $b' = b + \eta_2$.

Trường hợp 2: Nếu điểm nằm phía trên đường thẳng và bên trái trục y thì ta xoay đường thẳng theo chiều kim đồng hồ và dịch nó lên trên:

- Trừ η_1 khỏi độ dốc m . Lấy được $m' = m - \eta_1$.
- Thêm η_2 vào y -chặn b . Lấy được $b' = b + \eta_2$.

Trường hợp 3: Nếu điểm nằm dưới đường thẳng và bên phải trục y thì ta xoay đường thẳng theo chiều kim đồng hồ và dịch nó xuống:

- Trừ η_1 từ độ dốc m . Lấy được $m' = m - \eta_1$.
- Trừ η_2 từ y -chặn b . Lấy được $b' = b - \eta_2$.

Trường hợp 4: Nếu điểm nằm dưới đường thẳng và bên trái trục y thì ta xoay đường thẳng ngược chiều kim đồng hồ và dịch nó xuống:

- Thêm η_1 vào độ dốc m . Lấy được $m' = m + \eta_1$.
- Trừ η_2 khỏi y -chặn b . Lấy được $b' = b - \eta_2$.

Trở về: Đường thẳng có phương trình $y = m'x + b'$.

Lưu ý rằng trong ví dụ của chúng tôi, việc cộng hoặc trừ một số nhỏ vào độ dốc có nghĩa là tăng hoặc giảm giá mỗi phòng. Tương tự, cộng hoặc trừ một số nhỏ vào phần chặn y có nghĩa là tăng hoặc giảm giá cơ bản của căn nhà. Hơn nữa, vì tọa độ x là số phòng nên con số này không bao giờ âm. Vì vậy, chỉ có trường hợp 1 và 3 quan trọng trong ví dụ của chúng tôi, điều đó có nghĩa là chúng tôi có thể tóm tắt thủ thuật đơn giản trong ngôn ngữ thông tục như sau:

THỦ THUẬT ĐƠN GIẢN

- Nếu mô hình đưa ra cho chúng ta giá căn nhà thấp hơn giá thực tế, hãy thêm một lượng nhỏ ngẫu nhiên vào giá mỗi phòng và giá cơ sở của căn nhà. Nếu mô hình đưa ra cho chúng ta giá căn nhà cao hơn giá thực tế, hãy trừ một khoản nhỏ ngẫu nhiên khỏi giá mỗi phòng và giá cơ sở của căn nhà.
-

Thủ thuật này đạt được một số thành công trong thực tế, nhưng nó không phải là cách tốt nhất để di chuyển đường. Một số câu hỏi có thể nảy sinh, chẳng hạn như sau:

- Chúng ta có thể chọn các giá trị tốt hơn cho η_1 và η_2 không? Chúng ta có thể chia bốn trường hợp thành hai, hoặc có lẽ là một?

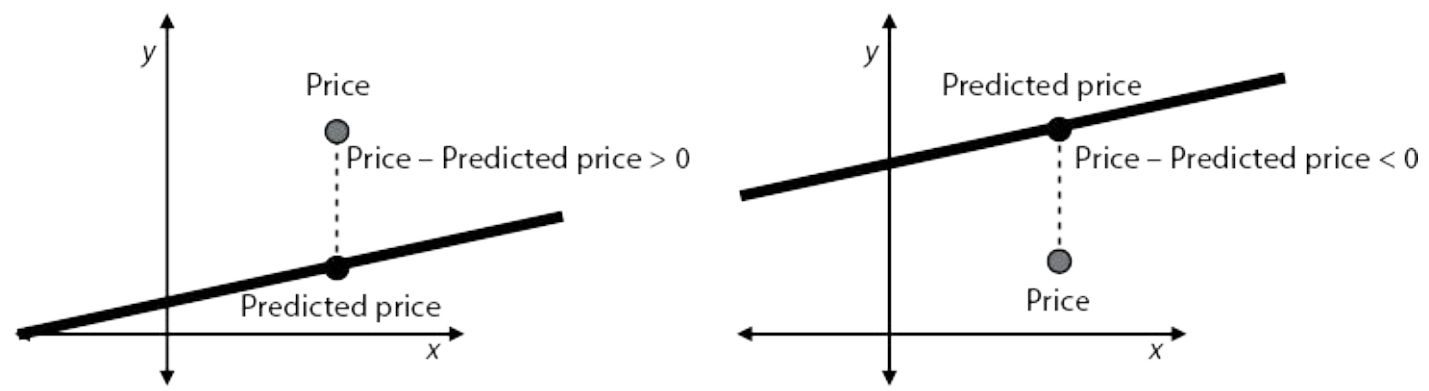
Câu trả lời cho cả hai câu hỏi là có và chúng ta sẽ xem cách thực hiện trong hai phần sau.

Thủ thuật hình vuông: Một cách thông minh hơn nhiều để di chuyển đường thẳng của chúng ta đến gần một trong các điểm

Trong phần này, tôi sẽ chỉ cho bạn một cách hiệu quả để di chuyển một đường thẳng đến gần một điểm hơn. Tôi gọi đây là *thủ thuật hình vuông*. Hãy nhớ lại thủ thuật đơn giản bao gồm bốn trường hợp dựa trên vị trí của điểm so với đường thẳng. Thủ thuật bình phương sẽ đưa bốn trường hợp này xuống còn một bằng cách tìm các giá trị có dấu đúng (+ hoặc -) để cộng vào hệ số góc và giao điểm y để đường thẳng luôn di chuyển đến gần điểm hơn.

Chúng ta bắt đầu với phần chặn y . Hãy chú ý hai nhận xét sau:

- **Quan sát 1:** Trong thủ thuật đơn giản, khi điểm nằm phía trên đường thẳng, chúng ta thêm một lượng nhỏ vào phần chặn y . Khi nó ở dưới đường này, chúng ta trừ đi một lượng nhỏ. **Quan sát 2:** Nếu một điểm nằm phía trên đường này thì giá trị (chênh lệch giữa giá và giá dự đoán) là dương. Nếu nó ở dưới dòng, giá trị này là âm. Quan sát này được minh họa trong hình 3.12.
-



Hình 3.12 Bên trái: Khi điểm nằm phía trên đường này, giá lớn hơn giá dự đoán nên chênh lệch là dương. Đúng: Khi điểm nằm dưới đường này, giá nhỏ hơn giá dự đoán nên chênh lệch là âm.

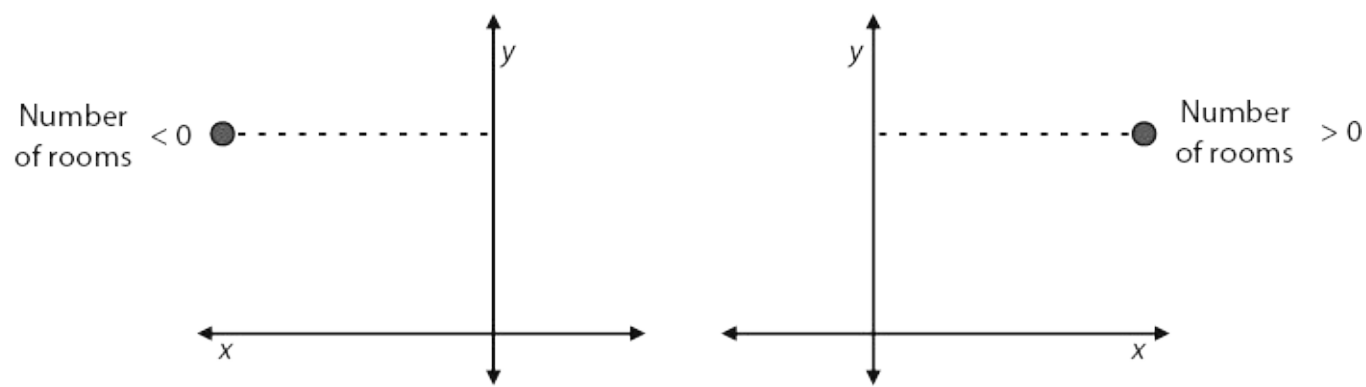
Kết hợp quan sát 1 và quan sát 2, chúng ta kết luận rằng nếu chúng ta cộng hiệu vào giao điểm y , thì đường thẳng sẽ luôn di chuyển về phía điểm, bởi vì giá trị này là dương khi điểm ở trên đường và âm khi điểm ở dưới đường. Tuy nhiên, trong học máy, chúng ta luôn muốn thực hiện từng bước nhỏ. Để giúp chúng tôi thực hiện điều này, chúng tôi giới thiệu một khái niệm quan trọng trong học máy: **tốc độ học**.

tỷ lệ học tập Một con số rất nhỏ mà chúng tôi chọn trước khi đào tạo mô hình của mình. Con số này giúp chúng tôi đảm bảo mô hình của chúng tôi thay đổi với số lượng rất nhỏ bằng cách đào tạo. Trong cuốn sách này, tốc độ học tập sẽ được ký hiệu là η , chữ cái Hy Lạp *eta*.

Vì tốc độ học nhỏ nên giá trị η (). Đây là giá trị chúng ta thêm vào phần chặn y để di chuyển đường thẳng theo hướng của điểm.

Giá trị chúng ta cần thêm vào độ dốc cũng tương tự nhưng phức tạp hơn một chút. Hãy chú ý hai nhận xét sau:

- **Quan sát 3:** Trong thủ thuật đơn giản, khi điểm nằm trong kịch bản 1 hoặc 4 (phía trên đường thẳng và bên phải trục tung, hoặc bên dưới đường thẳng và bên trái trục tung), chúng ta xoay đường thẳng ngược chiều kim đồng hồ. Ngược lại (kịch bản 2 hoặc 3), chúng ta xoay theo chiều kim đồng hồ.
- **Quan sát 4:** Nếu một điểm (r, p) nằm bên phải trục tung thì r là dương. Nếu điểm ở bên trái trục tung thì r là âm. Quan sát này được minh họa trong hình 3.13. Lưu ý rằng trong ví dụ này, r sẽ không bao giờ âm vì đó là số phòng. Tuy nhiên, trong một ví dụ chung, một tính năng có thể âm.



Hình 3.13 Bên trái: Khi điểm nằm bên trái trục y thì số phòng là số âm. Phải: Khi điểm nằm bên phải trục y thì số phòng là số dương.

Xét giá trị $r()$. Giá trị này là dương khi cả r và đều dương hoặc cả hai đều âm. Đây chính xác là những gì xảy ra trong kịch bản 1 và 4. Tương tự, $r()$ âm trong kịch bản 2 và 3. Do đó, theo quan sát 4, đây là đại lượng mà chúng ta cần thêm vào hệ số góc. Chúng ta muốn giá trị này nhỏ nên một lần nữa, chúng ta nhân nó với tốc độ học và kết luận rằng việc thêm $\eta r()$ vào độ dốc sẽ luôn di chuyển đường thẳng theo hướng của điểm.

$p - \hat{p}$

Bây giờ chúng ta có thể viết mã giả cho thủ thuật bình phương như sau:

Mã giả cho thủ thuật hình vuông

Đầu vào:

- Một đường thẳng có độ dốc m , y-chặn b và phương trình $\hat{m}r + b$
- Một điểm có tọa độ (r, p)
- Một giá trị dương nhỏ η (tốc độ học)
-

Đầu ra:

- Một đường thẳng có phương trình $= m'r + b'$ gần điểm hơn

Thủ tục:

- Thêm $\eta r(p)$ vào độ dốc m . Lấy $m' = m + \eta r()$ (điều này sẽ xoay vòng). Thêm $\eta()$ vào y-chặn
- b . Lấy $b' = b + \eta()$ (điều này dịch dòng này).

Trả về: Đường thẳng có phương trình $= m'r + b'$

Bây giờ chúng ta đã sẵn sàng mã hóa thuật toán này bằng Python! Mã cho phần này

sau:

- **Sổ tay:** Coding_Tuyển_regression.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_3_Linear_Re

Và đây là mã cho thủ thuật hình vuông:

```
def square_trick(base_price, price_per_room, num_rooms, price, learning_rate):  
    predicted_price = base_price + price_per_room*num_rooms ❶  
    base_price += learning_rate*(price-predicted_price) ❷  
    price_per_room += learning_rate*num_rooms*(price-predicted_price) ❸  
    return price_per_room, base_price
```

☐ Tính toán dự đoán ☐ Dịch
dòng ☐ Xoay dòng

Thủ thuật tuyệt đối: Một thủ thuật hữu ích khác để di chuyển đường thẳng đến gần các điểm hơn

Thủ thuật hình vuông có hiệu quả, nhưng một thủ thuật hữu ích khác, mà chúng tôi gọi là *thủ thuật tuyệt đối*, là trung gian giữa thủ thuật đơn giản và thủ thuật hình vuông. Trong thủ thuật bình phương, chúng ta đã sử dụng hai đại lượng (giá – giá dự đoán) và r (số phòng), để giúp chúng ta rút bốn trường hợp xuống còn một. Trong thủ thuật tuyệt đối, chúng ta chỉ sử dụng r để giúp giảm bốn trường hợp xuống còn hai. Nói cách khác, đây là thủ thuật tuyệt đối:

Mã giả cho thủ thuật tuyệt đối

Đầu vào:

- Một đường thẳng có độ dốc m , y-chặn b và phương trình $= \hat{m}r + b$
- Một điểm có tọa độ (r, p)
- Một giá trị dương nhỏ η (tốc độ học)
-

Đầu ra:

- Một đường thẳng có phương trình $= m'r + b'$ gần điểm hơn

Thủ tục:

Trường hợp 1: Nếu điểm nằm phía trên đường thẳng (tức là nếu $p > \hat{p}$):

- Thêm ηr vào độ dốc m . Lấy $m' = m + \eta r$ (thao tác này sẽ xoay đường ngược chiều kim đồng hồ nếu điểm nằm ở bên phải trục y và theo chiều kim đồng hồ nếu nó ở bên trái trục y). Thêm η vào y -chặn b . Lấy $b' = b + \eta$ (điều này có nghĩa là xếp hàng).
-

Trường hợp 2: Nếu điểm nằm dưới đường thẳng (tức là nếu $p < \hat{p}$):

- Trừ ηr khỏi độ dốc m . Lấy $m' = m - \eta r$ (thao tác này sẽ xoay đường thẳng theo chiều kim đồng hồ nếu điểm nằm ở bên phải trục y và ngược chiều kim đồng hồ nếu nó ở bên trái trục y). Trừ η khỏi y -chặn b . Lấy $b' = b - \eta$ (điều này dịch dòng xuống).
-

Trả về: Đường thẳng có phương trình $\hat{p} = m'r + b'$

Đây là mã cho thủ thuật tuyệt đối:

```
def absolute_trick(base_price, price_per_room, num_rooms, price, learning_rate):
    predicted_price = base_price + price_per_room*num_rooms
    if price > predicted_price:
        price_per_room += learning_rate*num_rooms
        base_price += learning_rate
    else:
        price_per_room -= learning_rate*num_rooms
        base_price -= learning_rate
    return price_per_room, base_price
```

Tôi khuyến khích bạn xác minh rằng số lượng được thêm vào mỗi trọng lượng thực sự có dấu đúng, như chúng ta đã làm với phép tính bình phương.

Thuật toán hồi quy tuyến tính: Lập lại thủ thuật tuyệt đối hoặc bình phương nhiều lần để di chuyển đường thẳng đến gần các điểm hơn

Bây giờ chúng ta đã hoàn thành tất cả công việc khó khăn, chúng ta đã sẵn sàng phát triển thuật toán hồi quy tuyến tính! Thuật toán này lấy đầu vào là một loạt các điểm và trả về một dòng phù hợp với chúng. Thuật toán này bao gồm việc bắt đầu với các giá trị ngẫu nhiên cho độ dốc và phần chặn y của chúng tôi, sau đó lặp lại quy trình cập nhật chúng nhiều lần bằng cách sử dụng thủ thuật tuyệt đối hoặc thủ thuật bình phương. Đây là mã giả:

PSEUDOCODE CHO Thuật toán hồi quy tuyến tính

Đầu vào:

- Tập dữ liệu các ngôi nhà có số phòng và giá

Đầu ra:

- Trọng lượng mô hình: giá mỗi phòng và giá cơ sở

Thủ tục:

- Bắt đầu với các giá trị ngẫu nhiên cho độ dốc và phần chặn y . Lặp lại nhiều lần: Chọn một điểm dữ liệu ngẫu nhiên. Cập nhật độ dốc và phần chặn y bằng cách sử dụng thủ thuật tuyệt đối hoặc bình phương.
-

Mỗi lần lặp của vòng lặp được gọi là một *epoch* và chúng tôi đặt số này ở đầu thuật toán của mình. Thủ thuật đơn giản này chủ yếu được sử dụng để minh họa, nhưng như đã đề cập trước đó, nó không hoạt động hiệu quả lắm. Trong cuộc sống thực, chúng ta sử dụng thủ thuật tuyệt đối hoặc thủ thuật bình phương, cách này hiệu quả hơn rất nhiều. Trên thực tế, mặc dù cả hai đều được sử dụng phổ biến nhưng thủ thuật bình phương lại phổ biến hơn. Do đó, chúng tôi sẽ sử dụng thuật toán đó cho thuật toán của mình nhưng bạn có thể thoải mái sử dụng thủ thuật tuyệt đối nếu muốn.

Đây là mã cho thuật toán hồi quy tuyến tính. Lưu ý rằng chúng tôi đã sử dụng gói Python để tạo số ngẫu nhiên cho các giá trị ban đầu (độ dốc

và y -intercept) và để chọn các điểm bên trong vòng lặp:

```
import random ❶
def linear_regression(features, labels, learning_rate=0.01, epochs = 1000):
    price_per_room = random.random()
    base_price = random.random() ❷
    for epoch in range(epochs): ❸
        i = random.randint(0, len(features)-1) ❹
        num_rooms = features[i]
        price = labels[i]
        price_per_room, base_price = square_trick(base_price,
                                                    price_per_room,
                                                    num_rooms,
                                                    price,
                                                    learning_rate=learning_rate) ❺
    return price_per_room, base_price
```

- ❶ Imports the random package to generate (pseudo) random numbers
- ❷ Generates random values for the slope and the y -intercept
- ❸ Repeats the update step many times
- ❹ Picks a random point on our dataset
- ❺ Applies the square trick to move the line closer to our point

Bước tiếp theo là chạy thuật toán này để xây dựng mô hình phù hợp với tập dữ liệu của chúng tôi.

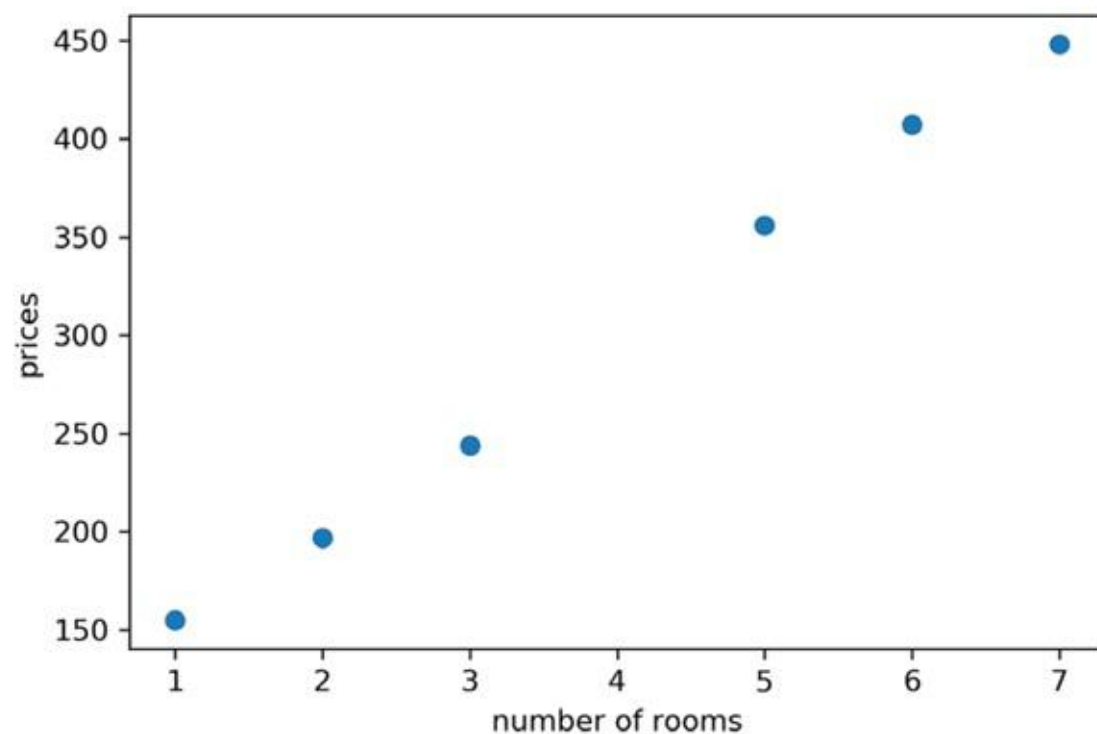
Đang tải dữ liệu của chúng tôi và vẽ biểu đồ

Trong suốt chương này, chúng ta tải và tạo biểu đồ dữ liệu và mô hình bằng Matplotlib và NumPy, hai gói Python rất hữu ích. Chúng tôi sử dụng NumPy để lưu trữ mảng và thực hiện các phép toán, trong khi chúng tôi sử dụng Matplotlib cho các biểu đồ.

Việc đầu tiên chúng ta làm là mã hóa các đặc điểm và nhãn của tập dữ liệu trong bảng 3.2 dưới dạng mảng NumPy như sau:

```
import numpy as np
features = np.array([1, 2, 3, 5, 6, 7])
labels = np.array([155, 197, 244, 356, 407, 448])
```

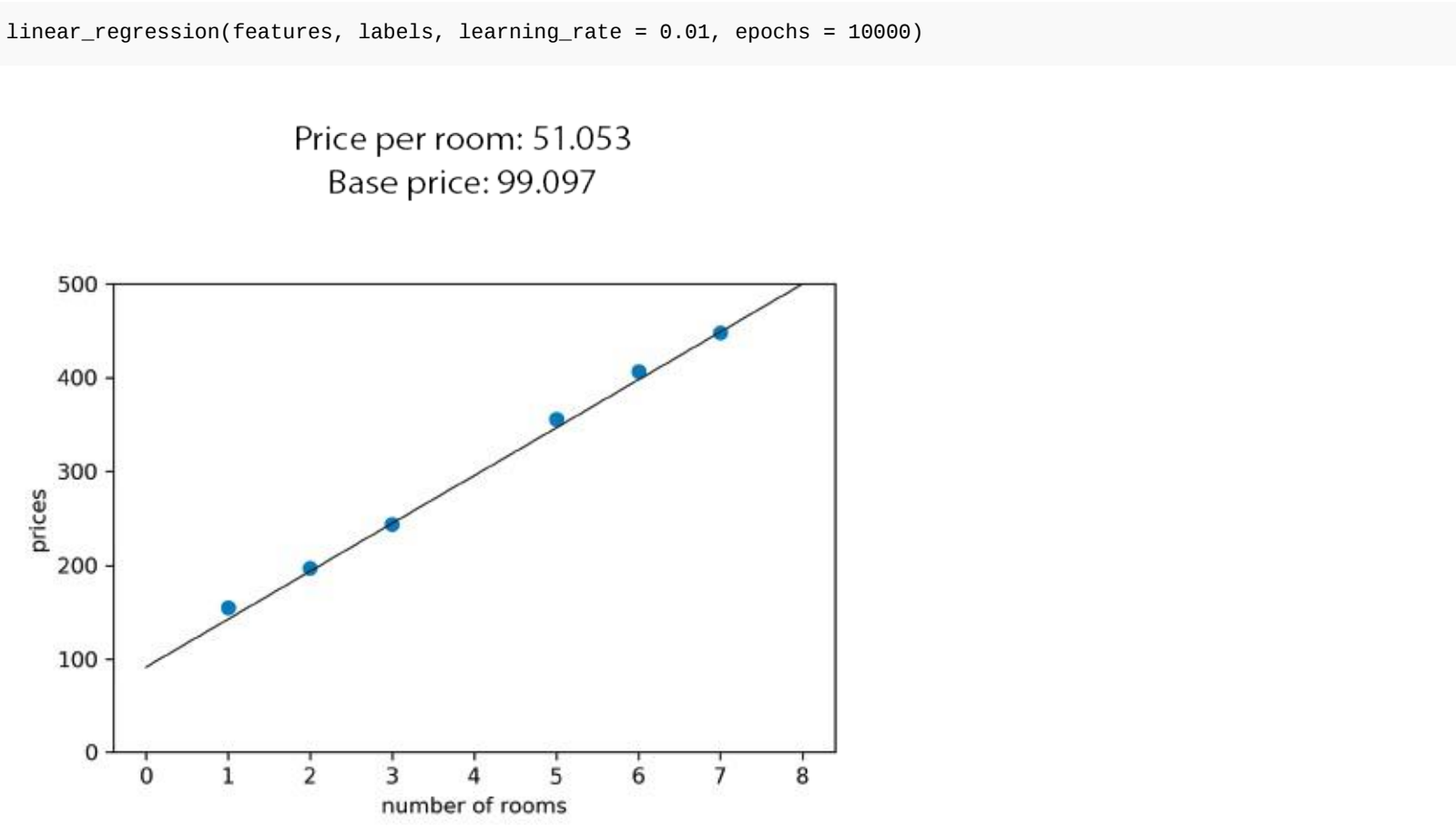
Tiếp theo chúng ta vẽ biểu đồ tập dữ liệu. Trong kho lưu trữ, chúng tôi có một số hàm để vẽ mã trong tệp `utils.py`, mời bạn xem qua. Sơ đồ của tập dữ liệu được thể hiện trong hình 3.14. Lưu ý rằng các điểm có vẻ gần như tạo thành một đường thẳng.



Hình 3.14 Sơ đồ các điểm trong bảng 3.2

Sử dụng thuật toán hồi quy tuyến tính trong tập dữ liệu của chúng tôi

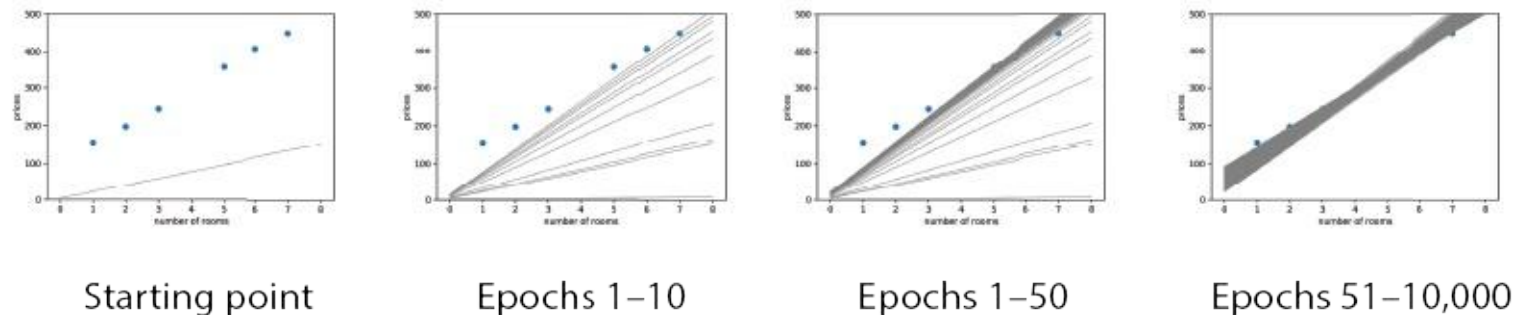
Bây giờ, hãy áp dụng thuật toán để khớp một đường thẳng với các điểm này. Dòng mã sau đây chạy thuật toán với các tính năng, nhãn, tốc độ học bằng 0,01 và số kỷ nguyên bằng 10.000. Kết quả được đồ thị như hình 3.15.



Hình 3.15 Đồ thị các điểm trong bảng 3.2 và đường thẳng ta thu được bằng thuật toán hồi quy tuyến tính

Hình 3.15 cho thấy đường trong đó giá (làm tròn) mỗi phòng là 51,05 USD và giá cơ bản là 99,10 USD. Con số này không khác xa mức 50 đô la và 100 đô la mà chúng tôi đã đề cập trước đó trong chương này.

Để hình dung quá trình, chúng ta hãy nhìn vào sự tiến triển thêm một chút. Trong hình 3.16, bạn có thể thấy một số đường trung gian. Lưu ý rằng dòng bắt đầu cách xa các điểm. Khi thuật toán tiến triển, nó sẽ di chuyển chậm rãi để ngày càng phù hợp hơn. Lưu ý rằng lúc đầu (trong 10 kỷ nguyên đầu tiên), đường thẳng sẽ di chuyển nhanh chóng về phía một giải pháp tốt. Sau kỷ nguyên 50, đường này vẫn tốt, nhưng nó vẫn chưa khớp hoàn toàn với các điểm. Nếu chúng ta để nó chạy trong suốt 10.000 kỷ nguyên, chúng ta sẽ có được sự phù hợp tuyệt vời.



Hình 3.16 Vẽ một số đường trong thuật toán của chúng tôi khi chúng tôi tiếp cận một giải pháp tốt hơn. Đồ họa đầu tiên hiển thị điểm bắt đầu. Đồ họa thứ hai hiển thị 10 kỷ nguyên đầu tiên của thuật toán hồi quy tuyến tính. Lưu ý cách đường đang di chuyển gần hơn để khớp các điểm. Đồ họa thứ ba hiển thị 50 kỷ nguyên đầu tiên. Đồ họa thứ tư hiển thị các kỷ nguyên từ 51 đến 10.000 (kỷ nguyên cuối cùng).

Sử dụng mô hình để đưa ra dự đoán

Bây giờ chúng ta đã có mô hình hồi quy tuyến tính rõ ràng, chúng ta có thể sử dụng nó để đưa ra dự đoán! Hãy nhớ lại từ đầu chương rằng mục tiêu của chúng ta là dự đoán giá của một ngôi nhà có bốn phòng. Ở phần trước, chúng ta đã chạy thuật toán và thu được độ dốc (giá mỗi phòng) là 51,05 và điểm chặn y (giá cơ bản của căn nhà) là 99,10. Do đó, phương trình như sau:

$$\hat{p} = 51.05r + 99.10$$

Dự đoán mà mô hình đưa ra cho ngôi nhà có $r = 4$ phòng là

$$\hat{p} = 51.05 \cdot 4 + 99.10 = 303.30.$$

Lưu ý rằng \$303,30 không quá xa so với \$300 mà chúng tôi đã để mắt tới ở đầu chương!

Theg thuật toán hồi quy tuyến tính chung (tùy chọn))

Phần này là tùy chọn vì nó tập trung chủ yếu vào các chi tiết toán học của thuật toán trừu tượng hơn được sử dụng cho tập dữ liệu chung. Tuy nhiên, tôi khuyến khích bạn đọc nó để làm quen với các ký hiệu được sử dụng trong hầu hết các tài liệu về học máy.

Trong các phần trước, chúng tôi đã phác thảo thuật toán hồi quy tuyến tính cho tập dữ liệu của mình chỉ với một tính năng. Nhưng như bạn có thể tưởng tượng, trong đời thực, chúng ta sẽ làm việc với các tập dữ liệu có nhiều tính năng. Đối với điều này, chúng ta cần một thuật toán chung.

Tin tốt là thuật toán chung không khác lắm so với thuật toán cụ thể mà chúng ta đã học trong chương này. Sự khác biệt duy nhất là mỗi tính năng được cập nhật giống như cách cập nhật độ dốc. Trong ví dụ về nhà ở, chúng ta có một độ dốc và một đoạn chặn y . Trong trường hợp tổng quát, hãy nghĩ đến nhiều con dốc và vẫn có một điểm chặn y .

Trường hợp chung sẽ bao gồm một tập dữ liệu gồm m điểm và n tính năng. Do đó, mô hình có m trọng số (coi chúng là sự khái quát hóa của độ dốc) và một độ lệch. Ký hiệu như sau:

- Các điểm dữ liệu là $x^{(1)}, x^{(2)}, \dots, x^{(m)}$. Mỗi điểm có dạng $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$.
- Các nhãn tương ứng là $y_1, y_2, \dots, y_m$. Các trọng số của mô hình là $w_1, w_2, \dots, w_n$.
- Độ lệch của mô hình là b .

Mã giả cho thủ thuật hình vuông tổng quát

Đầu vào:

- Mô hình có phương trình $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
- Một điểm có tọa độ (x, y) Giá trị dương nhỏ η (tốc độ học)

Đầu ra:

- Mô hình có phương trình $\hat{y} = w_1'x_1 + w_2'x_2 + \dots + w_n'x_n + b'$ gần với điểm

Thủ tục:

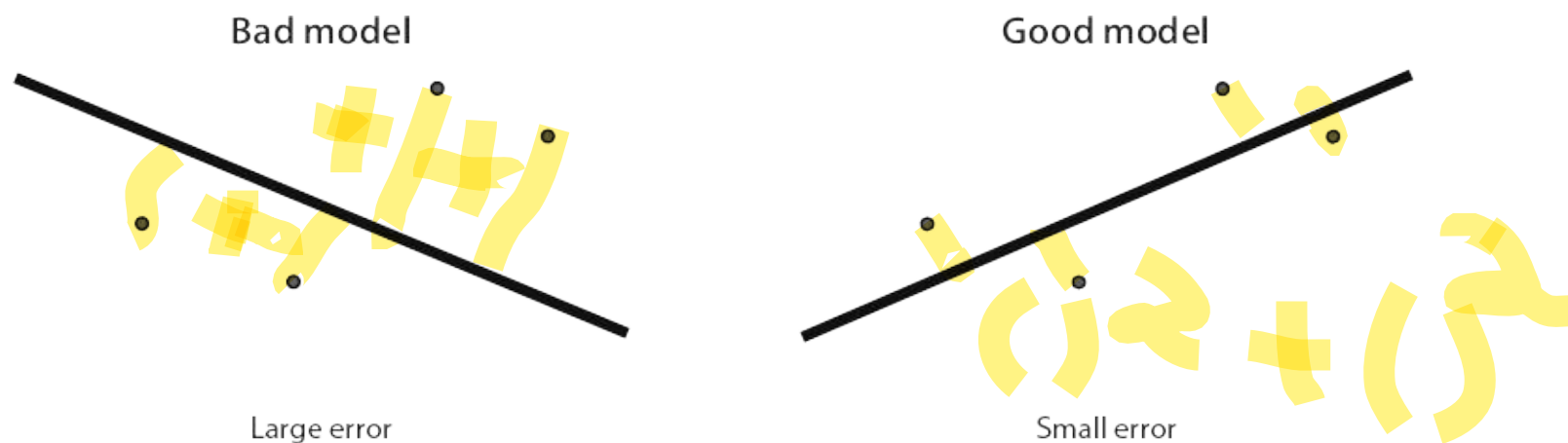
- Thêm $\eta(y - \hat{y})$ vào y -chặn b . Lấy được $b' = b + \eta(y - \hat{y})$. Đối với $i = 1, 2, \dots, n$: Thêm $\eta x(y - \hat{y})$ vào trọng số w_i . Lấy được $w_i' = w_i + \eta r(y - \hat{y})$.

Trả về: Mô hình có phương trình $\hat{y} = w_1'x_1 + w_2'x_2 + \dots + w_n'x_n + b'$

Mã giả của thuật toán hồi quy tuyến tính tổng quát giống với mã trong phần “Thuật toán hồi quy tuyến tính”, vì nó bao gồm việc lặp lại thủ thuật bình phương tổng quát, vì vậy chúng ta sẽ bỏ qua nó.

Chúng tôi đo lường kết quả của mình bằng cách nào? Hàm lỗi

Trong các phần trước, chúng tôi đã phát triển một cách tiếp cận trực tiếp để tìm ra đường phù hợp nhất. Tuy nhiên, nhiều khi sử dụng cách tiếp cận trực tiếp sẽ khó giải quyết được các vấn đề trong học máy. Một cách gián tiếp hơn nhưng máy móc hơn để thực hiện việc này là sử dụng hàm lỗi. Hàm lỗi là một thước đo cho chúng ta biết mô hình của chúng ta đang hoạt động như thế nào. Ví dụ, hãy xem hai mô hình trong hình 3.17. Cái bên trái là mô hình xấu, trong khi cái bên phải là mô hình tốt. Hàm lỗi đo lường điều này bằng cách gán một giá trị lớn cho mô hình xấu ở bên trái và một giá trị nhỏ cho mô hình tốt ở bên phải. Hàm lỗi đôi khi còn được gọi là hàm mất mát hoặc hàm chi phí trong tài liệu. Trong cuốn sách này, chúng tôi gọi chúng là các hàm lỗi ngoại trừ một số trường hợp đặc biệt mà tên được sử dụng phổ biến hơn có yêu cầu khác.



Hình 3.17 Hai mô hình, một mô hình xấu (ở bên trái) và một mô hình tốt (ở bên phải). Cái xấu được gán một lỗi lớn, còn cái tốt được gán một lỗi nhỏ.

Bây giờ câu hỏi là làm thế nào để xác định hàm lỗi tốt cho mô hình hồi quy tuyến tính? Chúng tôi có hai cách phổ biến để thực hiện việc này được gọi là *lỗi tuyệt đối* và *lỗi bình phương*. Nói tóm lại, sai số tuyệt đối là tổng khoảng cách theo chiều dọc từ đường thẳng đến các điểm trong tập dữ liệu và sai số bình phương là tổng bình phương của các khoảng cách này.

Trong một số phần tiếp theo, chúng ta sẽ tìm hiểu chi tiết hơn về hai hàm lỗi này. Sau đó, chúng ta xem cách giảm chúng bằng phương pháp gọi là giảm độ dốc. Cuối cùng, chúng tôi vẽ một trong những thứ này

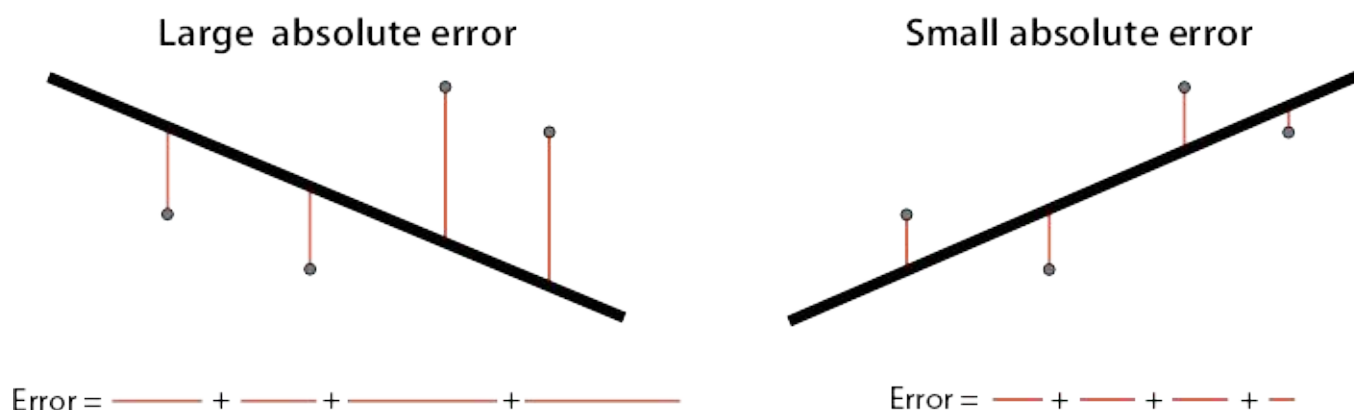
hàm lỗi trong ví dụ hiện tại của chúng tôi và xem tốc độ giảm độ dốc

phương pháp giúp chúng ta giảm bớt nó.

Lỗi tuyệt đối: Một thước đo cho chúng tôi biết mô hình của chúng tôi tốt đến mức nào bằng cách cộng khoảng cách

Trong phần này, chúng ta xem xét **sai số tuyệt đối**, đây là **thước đo cho chúng ta biết mô hình của chúng ta tốt đến mức nào**. Sai số tuyệt đối là **tổng khoảng cách giữa các điểm dữ liệu và đường thẳng**. Tại sao gọi là sai số tuyệt đối? Để tính toán từng khoảng cách, chúng tôi **lấy chênh lệch giữa nhân và nhân được dự đoán**. Sự khác biệt này có thể dương hoặc âm tùy thuộc vào điểm nằm trên hay dưới đường. Để **biến hiệu này thành một số luôn dương**, chúng ta lấy giá trị tuyệt đối của nó.

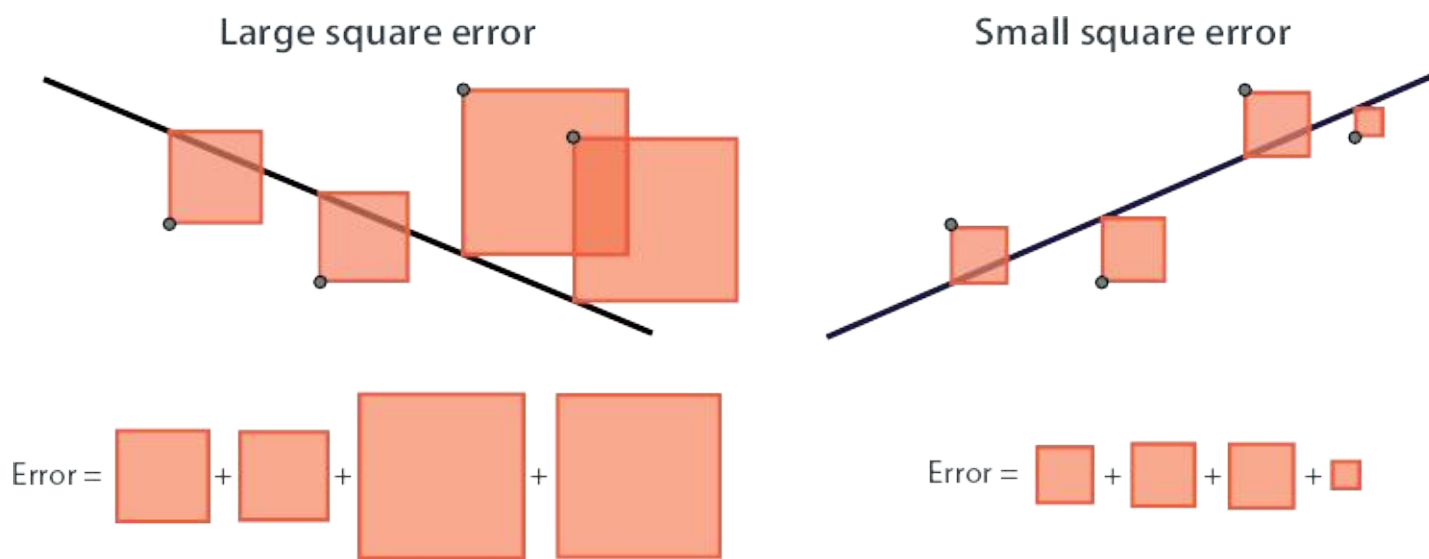
Theo định nghĩa, một mô hình hồi quy tuyến tính tốt là mô hình có đường thẳng gần các điểm. **đóng** có nghĩa là gì trong trường hợp này? Đây là một câu hỏi chủ quan, bởi vì một đường gần một số điểm có thể cách xa các điểm khác. Trong trường hợp đó, liệu chúng ta có nên chọn một đường rất gần với một số điểm và cách xa một số điểm khác không? Hay chúng ta cố gắng chọn một điểm gần với tất cả các điểm? Lỗi tuyệt đối giúp chúng tôi đưa ra quyết định này. **Đường chúng ta chọn là đường có sai số tuyệt đối nhỏ nhất, tức là đường mà tổng khoảng cách theo chiều dọc từ mỗi điểm đến đường thẳng là nhỏ nhất**. Trong hình 3.18, bạn có thể thấy hai đường thẳng và sai số tuyệt đối của chúng được minh họa dưới dạng tổng của các đoạn thẳng đứng. **Dòng bên trái có sai số tuyệt đối lớn, trong khi dòng bên phải có sai số tuyệt đối nhỏ**. Vì vậy, giữa hai cái này, chúng ta sẽ chọn cái bên phải.



Hình 3.18 Sai số tuyệt đối là tổng khoảng cách theo phương thẳng đứng từ các điểm đến đường thẳng. Lưu ý rằng sai số tuyệt đối lớn đối với mô hình xấu ở bên trái và nhỏ đối với mô hình tốt ở bên phải.

Lỗi bình phương: Một thước đo cho chúng ta biết mô hình của chúng ta tốt đến mức nào bằng cách cộng bình phương khoảng cách

Sai số bình phương rất giống với sai số tuyệt đối, ngoại trừ việc thay vì lấy giá trị tuyệt đối của chênh lệch giữa nhãn và nhãn được dự đoán, chúng ta lấy bình phương. Điều này luôn biến số thành số dương, vì bình phương của một số luôn làm cho nó dương. Quá trình này được minh họa trong hình 3.19, trong đó sai số bình phương được minh họa bằng tổng diện tích các bình phương có chiều dài từ các điểm đến đường thẳng. Bạn có thể thấy mô hình xấu ở bên trái có sai số bình phương lớn, trong khi mô hình tốt ở bên phải có sai số bình phương nhỏ.



Hình 3.19 Sai số bình phương là tổng bình phương của khoảng cách theo chiều dọc từ các điểm đến đường thẳng. Lưu ý rằng sai số bình phương lớn đối với mô hình xấu ở bên trái và nhỏ đối với mô hình tốt ở bên phải.

Như đã đề cập trước đó, sai số bình phương được sử dụng phổ biến hơn trong thực tế so với sai số tuyệt đối. Tại sao? Hình vuông có đạo hàm đẹp hơn nhiều so với giá trị tuyệt đối, điều này rất hữu ích trong quá trình đào tạo.

Lỗi bình phương trung bình tuyệt đối và (gốc) trung bình phổ biến hơn trong đời thực

Trong suốt chương này, chúng tôi sử dụng sai số tuyệt đối và sai số bình phương nhằm mục đích minh họa. Tuy nhiên, trong thực tế, sai số tuyệt đối trung bình và sai số bình phương trung bình được sử dụng phổ biến hơn nhiều. Chúng được xác định theo cách tương tự, ngoại trừ thay vì tính tổng, chúng tôi tính trung bình. Vì vậy, sai số tuyệt đối trung bình

là trung bình của các khoảng cách theo chiều dọc từ các điểm đến đường thẳng và sai số bình phương trung bình là trung bình của các bình phương có cùng khoảng cách này. Tại sao chúng phổ biến hơn? Hãy tưởng tượng nếu chúng ta muốn so sánh lỗi hoặc một mô hình sử dụng hai tập dữ liệu, một tập dữ liệu có 10 điểm và một tập dữ liệu có 1 triệu điểm. Nếu lỗi là tổng của các số lượng, mỗi số một cho mỗi điểm, thì lỗi có thể cao hơn nhiều trên tập dữ liệu 1 triệu điểm, vì chúng ta đang cộng nhiều số hơn. Nếu muốn so sánh chúng một cách chính xác, thay vào đó, chúng tôi sử dụng giá trị trung bình trong tính toán sai số để có được số đo khoảng cách từ mỗi điểm trung bình.

Với mục đích minh họa, một lỗi khác thường được sử dụng là *lỗi bình phương trung bình gốc* hoặc gọi tắt là *RMSE*. Như tên ngụ ý, điều này được định nghĩa là gốc của sai số bình phương trung bình. Nó được sử dụng để so khớp các đơn vị trong bài toán và cũng để cho chúng ta ý tưởng tốt hơn về mức độ lỗi mà mô hình mắc phải trong dự đoán. Làm sao vậy? Hãy tưởng tượng kịch bản sau: nếu chúng ta đang cố gắng dự đoán giá nhà thì đơn vị của giá và giá dự đoán là đô la (\$). Đơn vị của sai số bình phương và sai số bình phương trung bình là bình phương đô la, đây không phải là đơn vị phổ biến. Nếu chúng ta lấy căn bậc hai, thì chúng ta không chỉ có được đơn vị chính xác mà còn có được ý tưởng chính xác hơn về số tiền mà mô hình đã giảm cho mỗi ngôi nhà. Giả sử, nếu sai số bình phương trung bình gốc là 10.000 đô la thì chúng ta có thể mong đợi mô hình mắc sai số khoảng 10.000 đô la cho bất kỳ dự đoán nào chúng ta đưa ra.

Giảm dần độ dốc: Cách giảm hàm lỗi bằng cách từ từ đi xuống từ núi

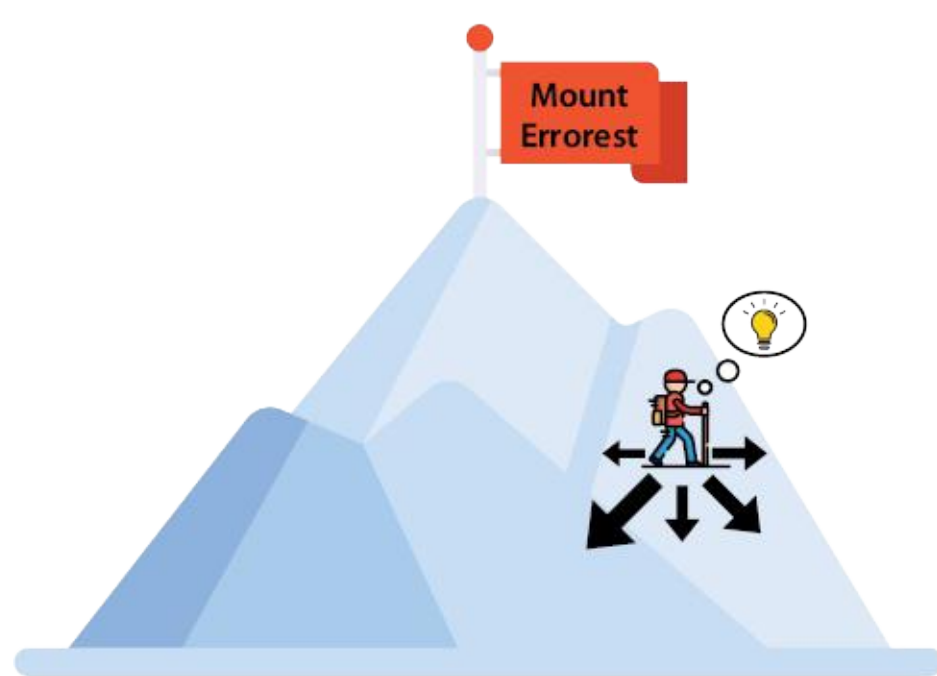
Trong phần này, tôi sẽ chỉ cho bạn cách giảm bất kỳ lỗi nào trước đó bằng phương pháp tương tự như phương pháp chúng ta sử dụng để từ từ đi xuống từ núi. Quá trình này sử dụng các công cụ phái sinh, nhưng đây là tin tuyệt vời: bạn không cần các công cụ phái sinh để hiểu nó. Chúng tôi đã sử dụng chúng trong quá trình đào tạo ở các phần “Thuật toán bình phương” và “Thuật toán tuyệt đối” trước đó. Mỗi khi chúng ta “di chuyển một lượng nhỏ theo hướng này”, chúng ta đang tính toán trong nền đạo hàm của hàm sai số và sử dụng nó để cung cấp cho chúng ta hướng di chuyển đường của mình. Nếu bạn yêu thích phép tính và muốn xem toàn bộ nguồn gốc của thuật toán này bằng cách sử dụng đạo hàm và gradient, hãy xem phụ lục B.

Hãy lùi lại một bước và nhìn vào hồi quy tuyến tính từ xa. Chúng ta muốn làm gì? Chúng tôi muốn tìm dòng phù hợp nhất với dữ liệu của chúng tôi. Chúng tôi có một số liệu được gọi là hàm lỗi, cho chúng tôi biết khoảng cách từ một dòng đến dữ liệu. Vì vậy, nếu chúng ta có thể giảm con số này càng nhiều càng tốt thì chúng ta sẽ tìm được dòng phù hợp nhất.

Quá trình này, phổ biến trong nhiều lĩnh vực toán học, được gọi là *hàm số cực tiểu hóa*, tức là tìm giá trị nhỏ nhất có thể mà một hàm có thể trả về. Đây là lúc độ dốc giảm dần xuất hiện: đó là một cách tuyệt vời để giảm thiểu một hàm.

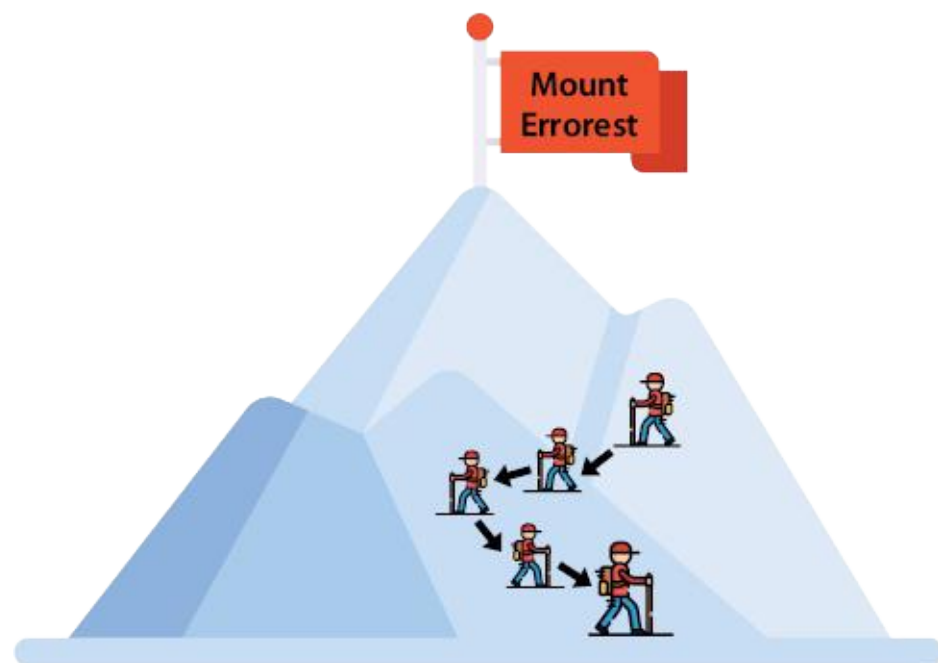
Trong trường hợp này, hàm chúng tôi đang cố gắng giảm thiểu là sai số (tuyệt đối hoặc bình phương) của mô hình của chúng tôi. Một lưu ý nhỏ là độ dốc giảm dần không phải lúc nào cũng tìm thấy giá trị tối thiểu chính xác của hàm, nhưng nó có thể tìm thấy giá trị nào đó rất gần với giá trị đó. Tin tốt là trên thực tế, việc giảm độ dốc diễn ra nhanh chóng và hiệu quả trong việc tìm ra các điểm có hàm số thấp.

Quá trình giảm độ dốc hoạt động như thế nào? Độ dốc giảm dần tương đương với việc đi xuống từ một ngọn núi. Giả sử chúng ta thấy mình đang ở trên đỉnh một ngọn núi cao tên là Mount Errorest. Chúng tôi muốn đi xuống nhưng trời có nhiều sương mù và chúng tôi chỉ có thể nhìn thấy cách đó khoảng một mét. Chúng ta làm gì? Một phương pháp tốt là nhìn xung quanh và tìm ra hướng nào chúng ta có thể thực hiện một bước, theo cách mà chúng ta đi xuống nhiều nhất. Quá trình này được minh họa trong hình 3.20.



Hình 3.20 Chúng tôi đang ở trên đỉnh Mount Errorest và muốn xuống đáy nhưng chúng tôi không thể nhìn xa được. Một cách để đi xuống là xem xét tất cả các hướng mà chúng ta có thể bước một bước và tìm ra hướng nào giúp chúng ta đi xuống nhiều nhất. Sau đó, chúng tôi lại một bước gần hơn đến đáy.

Khi tìm thấy hướng này, chúng ta thực hiện một bước nhỏ và vì bước đó được thực hiện theo hướng đi xuống lớn nhất nên rất có thể chúng ta đã đi xuống một lượng nhỏ. Tất cả những gì chúng ta phải làm là lặp lại quá trình này nhiều lần cho đến khi chúng ta (hy vọng) chạm đến đáy. Quá trình này được minh họa trong hình 3.21.



Hình 3.21 Cách để xuống núi là bước một bước nhỏ theo hướng khiến chúng ta phải đi xuống nhiều nhất và tiếp tục thực hiện việc này trong một thời gian dài.

Tại sao tôi lại nói *hy vọng*? Vâng, quá trình này có nhiều lưu ý. Chúng ta có thể chạm tới đáy, hoặc cũng có thể đến một thung lũng và rồi chúng ta không còn nơi nào khác để di chuyển. Chúng tôi sẽ không giải quyết vấn đề đó ngay bây giờ nhưng chúng tôi có một số kỹ thuật để giảm khả năng điều này xảy ra. Trong phụ lục B, “Sử dụng phương pháp giảm độ dốc để huấn luyện mạng lưới thần kinh”, một số kỹ thuật này đã được trình bày.

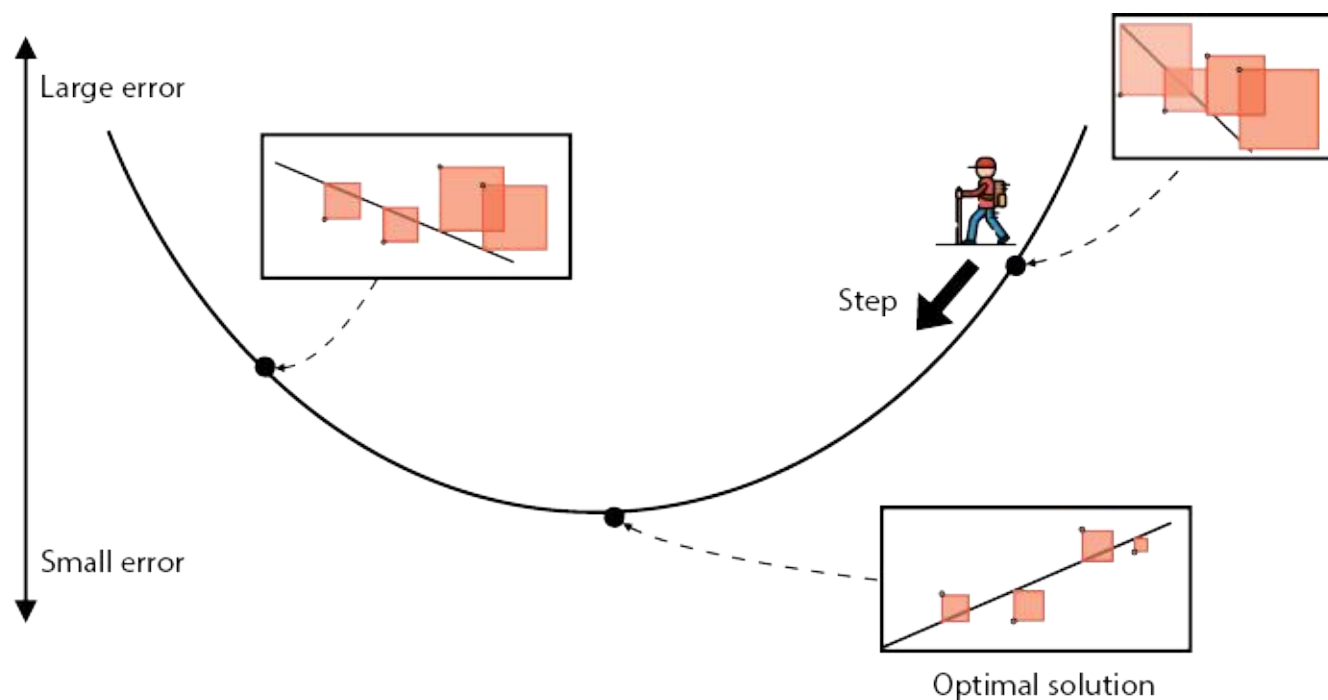
Rất nhiều phép toán ở đây mà chúng ta đang tìm hiểu sâu hơn sẽ được giải thích chi tiết hơn trong phụ lục B. Nhưng những gì chúng ta đã làm trong chương này chính xác là giảm độ dốc. Làm sao vậy? Giảm dần độ dốc hoạt động như sau:

1. Bắt đầu ở đâu đó trên núi. 2. Tìm hướng tốt nhất để thực hiện một bước nhỏ. 3. Thực hiện bước nhỏ này. 4. Lặp lại bước 2 và 3 nhiều lần.

Điều này có thể trông quen thuộc, bởi vì trong phần “Thuật toán hồi quy tuyến tính”, sau khi xác định các thủ thuật tuyệt đối và bình phương, chúng tôi đã xác định thuật toán hồi quy tuyến tính theo cách sau:

1. Bắt đầu với bất kỳ dòng nào. 2. Tìm hướng tốt nhất để di chuyển đường thẳng của chúng ta một chút, sử dụng thủ thuật tuyệt đối hoặc thủ thuật bình phương. 3. Di chuyển đường thẳng một chút theo hướng này.

4. Lặp lại bước 2 và 3 nhiều lần.



Hình 3.22 Mỗi điểm trên ngọn núi này tương ứng với một mô hình khác nhau. Những điểm bên dưới là những mô hình tốt với sai số nhỏ, còn những điểm bên trên là những mô hình xấu với sai số lớn. Mục tiêu là đi xuống từ ngọn núi này. Cách đi xuống là bắt đầu từ đâu đó và liên tục thực hiện một bước khiến chúng ta đi xuống. Độ dốc sẽ giúp chúng ta quyết định hướng nào sẽ thực hiện một bước giúp chúng ta đi xuống nhiều nhất.

Bức tranh tinh thần về điều này được minh họa trong hình 3.22. Điểm khác biệt duy nhất là hàm lỗi này trông không giống một ngọn núi mà giống một thung lũng hơn, và mục tiêu của chúng ta là đi xuống điểm thấp nhất. Mỗi điểm trong thung lũng này tương ứng với một số mô hình (đường) cố gắng khớp với dữ liệu của chúng tôi. Độ cao của điểm là sai số do mô hình đó đưa ra. Như vậy, mô hình xấu ở trên và mô hình tốt ở dưới cùng. Chúng tôi đang cố gắng xuống thấp nhất có thể. Mỗi bước sẽ đưa chúng ta từ một mô hình này đến một mô hình tốt hơn một chút. Nếu chúng ta thực hiện một bước như vậy nhiều lần, cuối cùng chúng ta sẽ có được mô hình tốt nhất (hoặc ít nhất là một mô hình khá tốt!).

Vẽ hàm lỗi và biết khi nào nên dừng chạy thuật toán

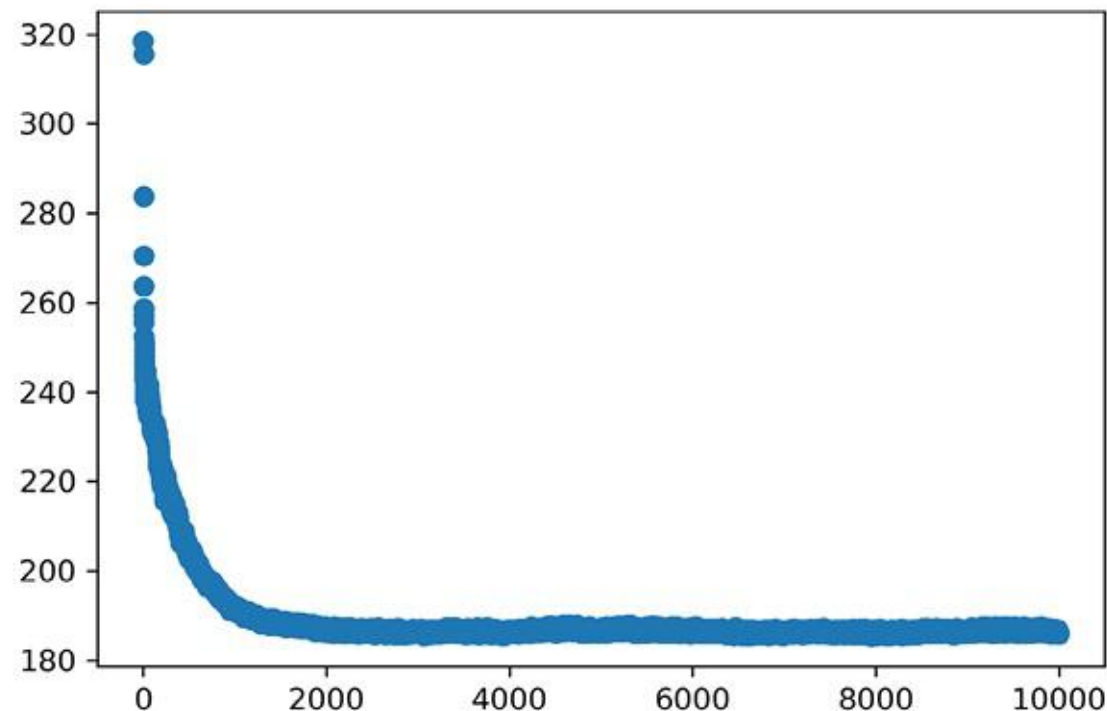
Trong phần này, chúng ta thấy sơ đồ hàm lỗi cho quá trình đào tạo mà chúng tôi đã thực hiện trước đó trong phần “Sử dụng thuật toán hồi quy tuyến tính trong tập dữ liệu của chúng tôi”. Cốt truyện này cung cấp cho chúng ta thông tin hữu ích về việc đào tạo mô hình này. trong

kho lưu trữ, chúng tôi cũng đã vẽ đồ thị **hàm lỗi bình phương trung bình gốc (RMSE)** được xác định trong phần “Lỗi bình phương trung bình tuyệt đối và (gốc) trung bình...”.Mã để tính RMSE như sau:

```
def rmse(labels, predictions):  
    n = len(labels)  
    differences = np.subtract(labels, predictions)  
    return np.sqrt(1.0/n * (np.dot(differences, differences)))
```

tích vô hướng Để mã hóa hàm RMSE, chúng ta đã sử dụng tích vô hướng, đây là một cách dễ dàng để viết tổng các tích của các số hạng tương ứng trong hai vector. Ví dụ: tích vô hướng của vector (1,2,3) và (4,5,6) là $1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$. Nếu tính tích vô hướng của một vector và chính nó, chúng ta thu được tổng bình phương của các phần tử.

Đồ thị lỗi của chúng tôi được thể hiện trên hình 3.23. Lưu ý rằng nó nhanh chóng giảm đi sau khoảng 1.000 lần lặp và không thay đổi nhiều sau đó. Biểu đồ này cung cấp cho chúng ta thông tin hữu ích: nó cho chúng ta biết rằng đối với mô hình này, chúng ta có thể chạy thuật toán huấn luyện chỉ với 1.000 hoặc 2.000 lần lặp thay vì 10.000 mà vẫn nhận được kết quả tương tự.



Hình 3.23 Biểu đồ sai số bình phương trung bình gốc cho ví dụ đang chạy của chúng ta. **Lưu ý cách thuật toán đã thành công trong việc giảm lỗi này sau hơn 1.000 lần lặp.** Điều này có nghĩa là chúng ta không cần tiếp tục chạy thuật toán này trong 10.000 lần lặp vì có khoảng 2.000 lần lặp trong số đó thực hiện công việc.

Nói chung, hàm lỗi cho chúng ta thông tin tốt để quyết định khi nào nên dừng

chạy thuật toán. Thông thường, quyết định này dựa trên thời gian và sức mạnh tính toán mà chúng ta có. Tuy nhiên, các điểm chuẩn hữu ích khác thường được sử dụng trong thực tế, chẳng hạn như sau:

- Khi hàm mất mát đạt đến một giá trị nhất định mà chúng ta đã xác định trước
- Khi hàm mất mát không giảm một lượng đáng kể trong một số kỷ nguyên

Chúng ta tập luyện từng điểm một hay nhiều điểm? Giảm dần độ dốc ngẫu nhiên và hàng loạt

Trong phần “Làm thế nào để máy tính vẽ được đường này”, chúng tôi đã huấn luyện mô hình hồi quy tuyến tính bằng cách lặp lại một bước nhiều lần. Bước này bao gồm việc chọn một điểm và di chuyển đường thẳng về điểm đó. Trong phần “Cách chúng tôi đo lường kết quả của mình”, chúng tôi đã huấn luyện mô hình hồi quy tuyến tính bằng cách tính toán sai số (tuyệt đối hoặc bình phương) và giảm sai số đó bằng cách sử dụng phương pháp giảm độ dốc. Tuy nhiên, lỗi này được tính toán trên toàn bộ tập dữ liệu chứ không phải trên một điểm tại một thời điểm. Tại sao lại thế này?

Thực tế là chúng ta có thể huấn luyện các mô hình bằng cách lặp lại từng điểm một hoặc trên toàn bộ tập dữ liệu. Tuy nhiên, khi tập dữ liệu rất lớn, cả hai lựa chọn đều có thể tốn kém. Chúng ta có thể thực hiện một phương pháp hữu ích được gọi là *học theo đợt nhỏ*, bao gồm việc chia dữ liệu của chúng ta thành nhiều đợt nhỏ. Trong mỗi lần lặp của thuật toán hồi quy tuyến tính, chúng tôi chọn một trong các lô nhỏ và tiến hành điều chỉnh trọng số của mô hình để giảm lỗi trong lô nhỏ đó. Quyết định sử dụng một điểm, một nhóm điểm nhỏ hoặc toàn bộ tập dữ liệu trên mỗi lần lặp sẽ dẫn đến ba loại thuật toán giảm độ dốc chung. Khi chúng tôi sử dụng một điểm tại một thời điểm, nó được gọi là *giảm độ dốc ngẫu nhiên*. Khi chúng tôi sử dụng một lô nhỏ, nó được gọi là *giảm độ dốc của lô nhỏ*. Khi chúng tôi sử dụng toàn bộ tập dữ liệu, nó được gọi là *giảm độ dốc hàng loạt*. Quá trình này được minh họa chi tiết hơn trong phụ lục B, “Sử dụng phương pháp giảm độ dốc để huấn luyện mô hình”.

Ứng dụng thực tế: Sử dụng Turi Create để dự đoán giá nhà ở Ấn Độ

Trong phần này, tôi cho bạn xem một ứng dụng thực tế. Chúng tôi sẽ sử dụng hồi quy tuyến tính để dự đoán giá nhà đất ở Hyderabad, Ấn Độ. Tập dữ liệu chúng tôi sử dụng đến từ Kaggle, một trang web phổ biến dành cho các cuộc thi học máy. Mã cho phần này

- follows:
- **Notebook:** House_price_predictions.ipynb
 - https://github.com/luisguiserrano/manning/blob/master/Chapter_3_Linear_Regression/House_price_predictions.ipynb
 - **Dataset:** Hyderabad.csv

This dataset has 6,207 rows (one per house) and 39 columns (features). As you can imagine, we won't code the algorithm by hand. Instead, we use Turi Create, a popular and useful package in which many machine learning algorithms are implemented. The main object to store data in Turi Create is the SFrame. We start by downloading the data into an SFrame, using the following command:

```
data = tc.SFrame('Hyderabad.csv')
```

The table is too big, but you can see the first few rows and columns in table 3.3.

Table 3.3 The first five rows and seven columns of the Hyderabad housing prices dataset

Price	Area	No. of Bedrooms	Resale	MaintenanceStaff	Gymnasium	SwimmingPool
30000000	3340	4	0	1	1	1
7888000	1045	2	0	0	1	1
4866000	1179	2	0	0	1	1
8358000	1675	3	0	0	0	0
6845000	1670	3	0	1	1	1

Training a linear regression model in Turi Create takes only one line of code. We use the function `create` from the package `linear_regression`. In this function, we only need to specify the target (label), which is `Price`, as follows:

```
model = tc.linear_regression.create(data, target='Price')
```

It may take a few moments to train, but after it trains, it outputs some information. One of the fields it outputs is the root mean square error. For this model, the RMSE is in the order of 3,000,000. This is a large RMSE, but it doesn't mean the model makes bad predictions. It may mean that the dataset has many outliers. As you can imagine, the price of a house may depend on many other features that are not in the dataset.

Chúng ta có thể sử dụng mô hình để dự đoán giá căn nhà có diện tích 1.000 m và 3 phòng ngủ như sau:

```
house = tc.SFrame({'Area': [1000], 'No. of Bedrooms':[3]})
model.predict(house)
Output: 2594841
```

Mô hình cho ra giá của một ngôi nhà cỡ 1.000 và ba phòng ngủ là 2.594.841.

Chúng ta cũng có thể huấn luyện một mô hình với ít tính năng hơn. Các tính năng cho phép chúng ta nhập các tính năng chúng tôi muốn sử dụng như một mảng. Dòng mã sau đây huấn luyện một mô hình có tên sử dụng khu vực để dự đoán giá:

```
simple_model = tc.linear_regression.create(data, features=['Area'], target='Price')
```

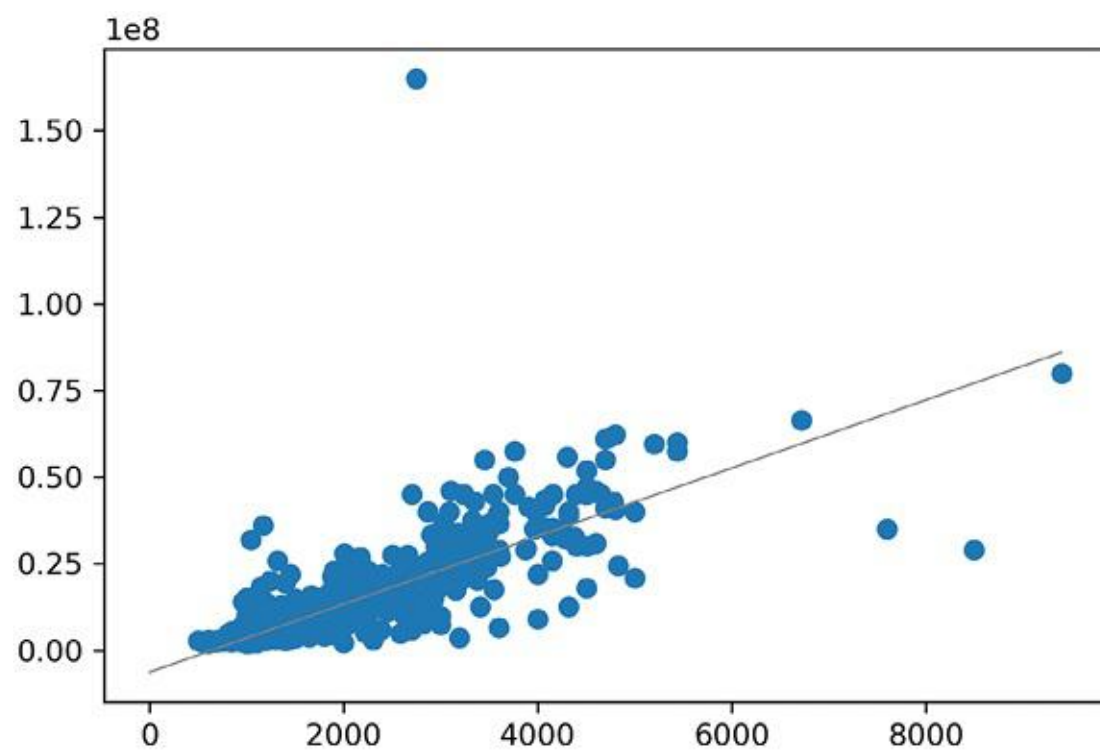
Chúng ta có thể khám phá trọng số của mô hình này bằng dòng mã sau:

```
simple_model.coefficients
```

Đầu ra cho chúng ta các trọng số sau:

- Độ dốc: 9664,97y-điểm chặn:
- −6.105.981,01

Điểm chặn là độ lệch và hệ số cho diện tích là độ dốc của đường khi chúng ta vẽ diện tích và giá. Đồ thị các điểm với mô hình tương ứng được thể hiện trên hình 3.24.



Hình 3.24 Bộ dữ liệu giá nhà ở Hyderabad được giới hạn theo diện tích và giá cả. Đường này là mô hình mà chúng tôi đã thu được chỉ sử dụng tính năng khu vực để dự đoán giá.

Chúng tôi có thể làm được nhiều điều hơn nữa trong tập dữ liệu này và tôi mời bạn tiếp tục khám phá. Ví dụ: cố gắng khám phá những đặc điểm nào quan trọng hơn những đặc điểm khác bằng cách xem xét trọng số của mô hình. Tôi khuyến khích bạn xem tài liệu Turi Create (<https://apple.github.io/turicreate/docs/api/>) để biết các chức năng và thủ thuật khác mà bạn có thể thực hiện để cải thiện mô hình này.

Nếu dữ liệu không nằm trên một dòng thì sao? Hồi quy đa thức

Trong các phần trước, chúng ta đã học cách tìm đường phù hợp nhất cho dữ liệu của mình, giả sử dữ liệu của chúng ta gần giống với một đường. Nhưng điều gì sẽ xảy ra nếu dữ liệu của chúng ta không giống một dòng? Trong phần này, chúng ta tìm hiểu một phần mở rộng mạnh mẽ cho hồi quy tuyến tính được gọi là *hồi quy đa thức*, giúp chúng ta giải quyết các trường hợp trong đó dữ liệu phức tạp hơn.

Một loại hàm cong đặc biệt: Đa thức

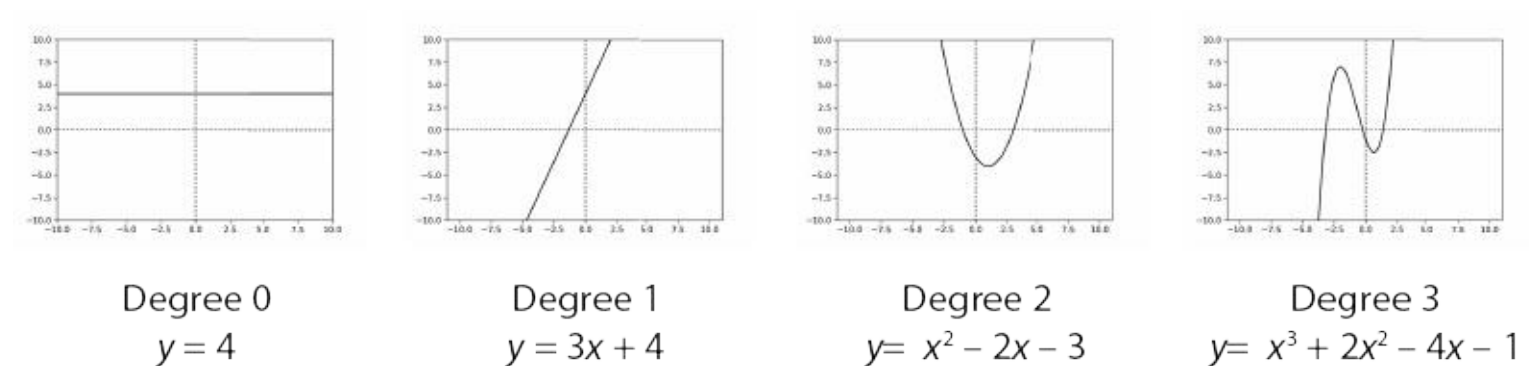
Để tìm hiểu hồi quy đa thức, trước tiên chúng ta cần tìm hiểu đa thức là gì. *Đa thức* là một lớp hàm hữu ích khi lập mô hình dữ liệu phi tuyến.

Chúng ta đã thấy đa thức, bởi vì mỗi dòng là đa thức bậc 1. Parabol là ví dụ về đa thức bậc 2. Về mặt hình thức, đa thức là một hàm trong một biến có thể được biểu thị dưới dạng tổng bội số lũy thừa của biến này. Các lũy thừa của biến x là $1, x, x^2, x^3, \dots$. Lưu ý rằng hai cái đầu tiên là $x^0 = 1$ và $x^1 = x$. Vì vậy, sau đây là ví dụ về đa thức:

- $y = 4$
- $y = 3x + 2$
- $y = x^2 - 2x + 5$
- $y = 2x^3 + 8x^2 - 40$

Chúng ta định nghĩa *bậc* của đa thức là số mũ có lũy thừa cao nhất trong biểu thức của đa thức. Ví dụ: đa thức $y = 2x^3 + 8x^2 - 40$ có bậc 3, vì 3 là số mũ cao nhất mà biến x được nâng lên. Lưu ý rằng trong ví dụ này, các đa thức có bậc 0, 1, 2 và 3. Đa thức bậc 0 luôn là một hằng số và đa thức bậc 1 là một phương trình tuyến tính giống như các phương trình chúng ta đã thấy trước đây trong chương này.

Đồ thị của đa thức trông rất giống một đường cong dao động nhiều lần. Số lần nó dao động có liên quan đến bậc của đa thức. Nếu đa thức có bậc d thì đồ thị của đa thức đó là đường cong dao động tối đa $d - 1$ lần (với $d > 1$). Trong hình 3.25 chúng ta có thể thấy đồ thị của một số ví dụ về đa thức.



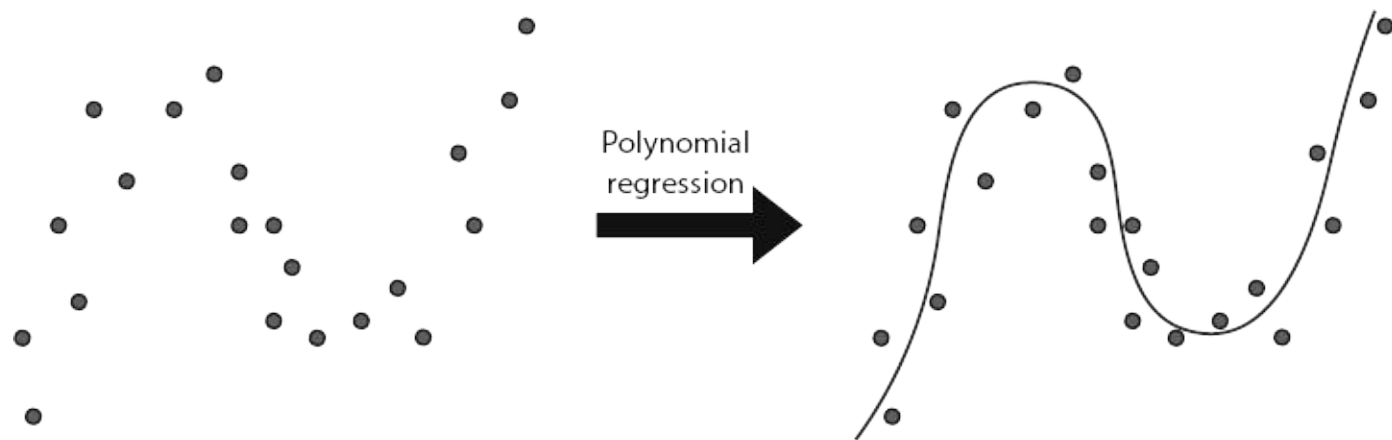
Hình 3.25 Đa thức là các hàm giúp chúng ta mô hình hóa dữ liệu tốt hơn. Dưới đây là đồ thị của bốn đa thức bậc 0 đến 3. Lưu ý rằng đa thức bậc 0 là một đường nằm ngang, đa thức bậc 1 là đường thẳng bất kỳ, đa thức bậc 2 là một parabol và đa thức bậc 3 là một đường cong dao động hai lần.

Từ đồ thị, lưu ý rằng đa thức bậc 0 là các đường thẳng. Đa thức bậc 1 là các đường có hệ số góc khác 0. Đa thức bậc 2 là

bậc hai (parabol). Đa thức bậc 3 trông giống như một đường cong dao động hai lần (mặc dù chúng có thể dao động ít lần hơn). Đồ thị của đa thức bậc 100 sẽ như thế nào? Ví dụ: cốt truyện của $y = x^{100} - 8x^{62} + 73x^{27} - 4x + 38$? Chúng ta phải vẽ đồ thị để tìm ra, nhưng chắc chắn, chúng ta biết rằng đó là một đường cong dao động tối đa 99 lần.

Dữ liệu phi tuyến? Không vấn đề gì: Hãy thử điều chỉnh một đường cong đa thức cho nó

Trong phần này, chúng ta sẽ thấy điều gì sẽ xảy ra nếu dữ liệu của chúng ta không tuyến tính (tức là trông không giống như một đường thẳng) và chúng ta muốn khớp một đường cong đa thức với nó. Giả sử dữ liệu của chúng ta trông giống như bên trái của hình 3.26. Cho dù chúng tôi có cố gắng thế nào đi chăng nữa, chúng tôi thực sự không thể tìm thấy một dòng tốt phù hợp với dữ liệu này. Không có gì! Nếu chúng ta quyết định khớp một đa thức bậc 3 (còn gọi là bậc ba), thì chúng ta sẽ có được đường cong hiển thị ở bên phải của hình 3.26, đường cong này phù hợp hơn nhiều với dữ liệu.



Hình 3.26 Hồi quy đa thức rất hữu ích khi mô hình hóa dữ liệu phi tuyến. Nếu dữ liệu của chúng ta trông giống như phần bên trái của hình thì sẽ khó tìm được đường thẳng phù hợp với nó. Tuy nhiên, một đường cong sẽ phù hợp với dữ liệu, như bạn có thể thấy ở phần bên phải của hình. Hồi quy đa thức giúp chúng ta tìm thấy đường cong này.

Quá trình huấn luyện mô hình hồi quy đa thức tương tự như quá trình huấn luyện mô hình hồi quy tuyến tính. Sự khác biệt duy nhất là chúng ta cần thêm nhiều cột hơn vào tập dữ liệu của mình trước khi áp dụng hồi quy tuyến tính. Ví dụ: nếu chúng ta quyết định khớp đa thức bậc 3 với dữ liệu trong hình 3.26, chúng ta cần thêm hai cột: một cột tương ứng với bình phương của đối tượng địa lý và một cột tương ứng với lập phương của đối tượng địa lý. Nếu bạn muốn nghiên cứu vấn đề này chi tiết hơn, vui lòng xem phần “Hồi quy, kiểm tra và chính quy hóa đa thức với Turi Create” trong chương 4, trong đó chúng ta tìm hiểu một ví dụ về đa thức

hồi quy trong tập dữ liệu parabol.

Một lưu ý nhỏ khi huấn luyện mô hình hồi quy đa thức là chúng ta phải quyết định mức độ của đa thức trước quá trình huấn luyện. Làm thế nào để chúng ta quyết định mức độ này? Chúng ta muốn một đường thẳng (độ 1), một parabol (độ 2), một khối (độ 3) hay một đường cong nào đó có độ 50? Câu hỏi này rất quan trọng và chúng ta sẽ giải quyết nó trong chương 4, khi chúng ta tìm hiểu về trang bị quá mức, thiếu trang bị và chính quy hóa!

Tham số và siêu tham số

Tham số và siêu tham số là một số khái niệm quan trọng nhất trong học máy và trong phần này, chúng ta tìm hiểu chúng là gì cũng như cách phân biệt chúng.

Như chúng ta đã thấy trong chương này, các mô hình hồi quy được xác định bởi trọng số và độ lệch của chúng—*các tham số* của mô hình. Tuy nhiên, chúng ta có thể điều chỉnh nhiều nút khác trước khi huấn luyện một mô hình, chẳng hạn như tốc độ học tập, số kỷ nguyên, mức độ (nếu xem xét mô hình hồi quy đa thức) và nhiều yếu tố khác. Chúng được gọi là *siêu tham số*.

Mỗi mô hình học máy mà chúng ta học trong cuốn sách này đều có một số tham số và siêu tham số được xác định rõ ràng. Chúng có xu hướng dễ bị nhầm lẫn, do đó, nguyên tắc chung để phân biệt chúng như sau:

- Bất kỳ số lượng nào bạn đặt *trước* quá trình đào tạo đều là siêu tham số. Bất kỳ đại lượng nào mà
- mô hình tạo hoặc *sửa đổi trong* quá trình đào tạo đều là một tham số.

Ứng dụng hồi quy

Tác động của học máy không chỉ được đo lường bằng sức mạnh của thuật toán mà còn bằng phạm vi ứng dụng hữu ích mà nó có. Trong phần này, chúng ta thấy một số ứng dụng của hồi quy tuyến tính trong đời sống thực. Trong mỗi ví dụ, chúng tôi phác thảo vấn đề, tìm hiểu một số tính năng để giải quyết nó và sau đó để hồi quy tuyến tính phát huy tác dụng kỳ diệu của nó.

Hệ thống khuyến nghị

Học máy được sử dụng rộng rãi để tạo ra các đề xuất hữu ích trong một số ứng dụng nổi tiếng nhất, bao gồm YouTube, Netflix, Facebook, Spotify và Amazon. Hồi quy đóng một vai trò quan trọng trong hầu hết các hệ thống tư vấn này. Vì hồi quy dự đoán một đại lượng nên tất cả những gì chúng ta phải làm để tạo ra đề xuất tốt là tìm ra đại lượng nào thể hiện sự tương tác hoặc mức độ hài lòng của người dùng tốt nhất. Sau đây là một số ví dụ cụ thể hơn về điều này.

Đề xuất video và âm nhạc

Một trong những cách được sử dụng để tạo đề xuất video và âm nhạc là dự đoán lượng thời gian người dùng sẽ xem video hoặc nghe một bài hát. Để làm được điều này, chúng ta có thể tạo mô hình hồi quy tuyến tính trong đó nhãn trên dữ liệu là số phút mỗi người dùng xem mỗi bài hát. Các đặc điểm này có thể là thông tin nhân khẩu học của người dùng, chẳng hạn như độ tuổi, vị trí và nghề nghiệp của họ, nhưng chúng cũng có thể là hành vi, chẳng hạn như các video hoặc bài hát khác mà họ đã nhấp vào hoặc tương tác.

Đề xuất sản phẩm

Các cửa hàng và trang web thương mại điện tử cũng sử dụng hồi quy tuyến tính để dự đoán doanh số bán hàng của họ. Một cách để làm điều này là dự đoán số tiền khách hàng sẽ chi tiêu tại cửa hàng. Chúng ta có thể làm điều này bằng cách sử dụng hồi quy tuyến tính. Nhãn cần dự đoán có thể là số tiền người dùng đã chi tiêu và các tính năng có thể là nhân khẩu học và hành vi, theo cách tương tự như đề xuất video và âm nhạc.

Chăm sóc sức khỏe

Hồi quy có rất nhiều ứng dụng trong chăm sóc sức khỏe. Tùy thuộc vào vấn đề chúng ta muốn giải quyết, việc dự đoán nhãn phù hợp chính là chìa khóa. Dưới đây là một vài ví dụ:

- Dự đoán tuổi thọ của bệnh nhân, dựa trên tình trạng sức khỏe hiện tại của họ
- Dự đoán thời gian nằm viện, dựa trên các triệu chứng hiện tại

Tóm tắt

- Hồi quy là một phần quan trọng của học máy. Nó bao gồm việc đào tạo một thuật toán với dữ liệu được gắn nhãn và sử dụng nó để đưa ra dự đoán về dữ liệu (không được gắn nhãn) trong tương lai. Dữ liệu được gắn nhãn là dữ liệu đi kèm với nhãn, trong trường hợp hồi quy là các số. Ví dụ, những con số có thể là giá nhà. Trong tập dữ liệu, các tính năng là các thuộc tính mà chúng tôi sử dụng để dự đoán nhãn. Ví dụ: nếu chúng ta muốn dự đoán giá nhà đất, các đặc điểm là bất cứ thứ gì mô tả ngôi nhà và có thể xác định giá, chẳng hạn như quy mô, số phòng, chất lượng trường học, tỷ lệ tội phạm, tuổi của ngôi nhà và khoảng cách đến đường cao tốc. Phương pháp hồi quy tuyến tính để dự đoán bao gồm việc gán trọng số cho từng đặc điểm và cộng các trọng số tương ứng nhân với các đặc điểm, cộng với độ lệch. Về mặt đồ họa, chúng ta có thể thấy thuật toán hồi quy tuyến tính đang cố gắng vượt qua một đường càng gần một tập hợp các điểm càng tốt. Cách thức hoạt động của thuật toán hồi quy tuyến tính là bắt đầu bằng một đường ngẫu nhiên và sau đó từ từ di chuyển nó đến gần từng điểm bị phân loại sai để cố gắng phân loại chúng một cách chính xác. Hồi quy đa thức là dạng tổng quát của hồi quy tuyến tính, trong đó chúng ta sử dụng đường cong thay vì đường để mô hình hóa dữ liệu của mình. Điều này đặc biệt hữu ích khi tập dữ liệu của chúng tôi là phi tuyến tính. Hồi quy có nhiều ứng dụng, bao gồm hệ thống khuyến nghị, thương mại điện tử và chăm sóc sức khỏe.
-
-
-

Bài tập

BÀI TẬP 3.1

Một trang web đã đào tạo mô hình hồi quy tuyến tính để dự đoán số phút mà người dùng sẽ dành cho trang web. Công thức họ thu được là

$$\hat{t} = 0.8d + 0.5m + 0.5y + 0.2a + 1.5$$

trong đó $\hat{t}$ là thời gian dự đoán tính bằng phút và d , m , y và a là các biến chỉ báo (cụ thể là chúng chỉ lấy giá trị 0 hoặc 1) được xác định như sau:

- d là biến cho biết người dùng có sử dụng máy tính để bàn hay không. m là
- biến cho biết người dùng có sử dụng thiết bị di động hay không.

- y là biến cho biết người dùng có còn trẻ (dưới 21 tuổi) hay không. a là biến cho biết người dùng có phải là người lớn (21 tuổi trở lên) hay không.

Ví dụ: Nếu người dùng 30 tuổi và sử dụng máy tính để bàn thì $d = 1$, $m = 0$, $y = 0$ và $a = 1$.

Nếu một người dùng 45 tuổi xem trang web từ điện thoại của họ, họ sẽ dành bao nhiêu thời gian trên trang web?

BÀI TẬP 3.2

Hãy tưởng tượng rằng chúng tôi đã đào tạo mô hình hồi quy tuyến tính trong tập dữ liệu y tế.Mô hình dự đoán tuổi thọ dự kiến của bệnh nhân.Đối với mỗi tính năng trong tập dữ liệu của chúng tôi, mô hình sẽ chỉ định trọng số.

a) Đối với các đại lượng sau đây, hãy cho biết trọng lượng gắn liền với đại lượng này là số dương, số âm hay bằng 0.Lưu ý: nếu bạn tin rằng trọng số là một con số rất nhỏ, dù dương hay âm, bạn có thể nói số không.

- 1. Số giờ tập thể dục của bệnh nhân mỗi tuần
- 2. Số điều thuốc bệnh nhân hút mỗi tuần
- 3. Số thành viên trong gia đình có vấn đề về tim
- 4. Số anh chị em của bệnh nhân
- 5. Bệnh nhân có nhập viện hay không

b) Mô hình cũng có độ lệch.Bạn nghĩ độ lệch là dương, âm hay bằng 0?

BÀI TẬP 3.3

Sau đây là tập dữ liệu về các ngôi nhà có kích thước (tính bằng feet vuông) và giá (tính bằng đô la).

	(Các) kích thước	Giải thưởng (p)
Nhà 1	100	200
Nhà 2	200	475
Nhà 3	200	400
Nhà 4	250	520
Nhà 5	325	735

Giả sử chúng ta đã huấn luyện mô hình trong đó dự đoán giá căn nhà dựa trên kích thước như sau:

$$\hat{p} = 2s + 50$$

Một. Tính toán các dự đoán mà mô hình này đưa ra trên tập dữ liệu. b. Tính sai số tuyệt đối trung bình của mô hình này. c. Tính sai số bình phương trung bình gốc của mô hình này.

BÀI TẬP 3.4

Mục tiêu của chúng ta là di chuyển đường thẳng có phương trình $\hat{y} = 2x + 3$ đến gần điểm $(x, y) = (5, 15)$ bằng cách sử dụng các thủ thuật mà chúng ta đã học trong chương này. Đối với hai bài toán sau, sử dụng tốc độ học $\eta = 0,01$.

Một. Áp dụng thủ thuật tuyệt đối để sửa đổi dòng trên để gần điểm hơn. b. Áp dụng thủ thuật vuông để sửa đổi dòng trên để gần điểm hơn.

4 Tối ưu hóa việc đào tạo

quy trình: Trang bị thiếu, trang bị quá mức, thử nghiệm và chuẩn hóa

underfitting, overfitting, testing, regularization

Trong chương này

- điều gì là thiếu và quá phù hợp một số giải pháp để tránh quá phù hợp: thử nghiệm, biểu đồ độ
- phức tạp của mô hình và chính quy hóa tính toán độ phức tạp của mô hình bằng cách sử dụng các chỉ tiêu L1 và L2 chọn mô hình tốt nhất về hiệu suất và độ phức tạp
-
-



Chương này khác với hầu hết các chương trong cuốn sách này vì nó không chứa một thuật toán học máy cụ thể. Thay vào đó, nó mô tả một số vấn đề tiềm ẩn mà các mô hình học máy có thể gặp phải và những cách thực tế hiệu quả để giải quyết chúng.

Hãy tưởng tượng rằng bạn đã học được một số thuật toán học máy tuyệt vời và bạn đã sẵn sàng áp dụng chúng. Bạn đi làm nhà khoa học dữ liệu và nhiệm vụ đầu tiên của bạn là xây dựng mô hình học máy cho tập dữ liệu về khách hàng. Bạn xây dựng nó và đưa nó vào sản xuất. Tuy nhiên, mọi thứ đều không ổn và mô hình không thực hiện tốt công việc đưa ra dự đoán. Chuyện gì đã xảy ra thế?

Hóa ra câu chuyện này khá phổ biến vì nhiều thứ có thể xảy ra sai sót với các mô hình của chúng ta. May mắn thay, chúng tôi có một số kỹ thuật để cải thiện chúng. Trong chương này,

Tôi sẽ chỉ cho bạn hai vấn đề thường xảy ra khi đào tạo mô hình: trang bị thiếu và trang bị quá mức. Sau đó, tôi sẽ chỉ cho bạn một số giải pháp để tránh trang bị thiếu và quá khớp cho các mô hình của chúng tôi: kiểm tra và xác thực, biểu đồ độ phức tạp của mô hình và chính quy hóa.

Hãy giải thích việc trang bị thiếu và trang bị quá mức bằng phép tương tự sau. Giả sử chúng ta phải học để kiểm tra. Một số điều có thể xảy ra **sai sót trong quá trình học tập của chúng tôi**. Có lẽ chúng ta đã **học chưa đủ**. Không có cách nào để khắc phục điều đó và chúng tôi có thể sẽ **hoạt động kém** trong thử nghiệm của mình. Điều gì sẽ xảy ra nếu chúng ta **học rất nhiều nhưng lại sai cách**. Ví dụ, thay vì tập trung vào việc học, chúng tôi quyết định học thuộc lòng toàn bộ sách giáo khoa từng chữ một. Chúng ta sẽ làm tốt bài kiểm tra của mình chứ? Có khả năng là chúng ta sẽ không làm vậy, bởi vì chúng ta chỉ **đơn giản là ghi nhớ mọi thứ mà không cần học**. Tất nhiên, **lựa chọn tốt nhất sẽ là ôn thi đúng cách và theo cách cho phép chúng ta trả lời những câu hỏi mới mà chúng ta chưa từng thấy trước đây** về chủ đề này.

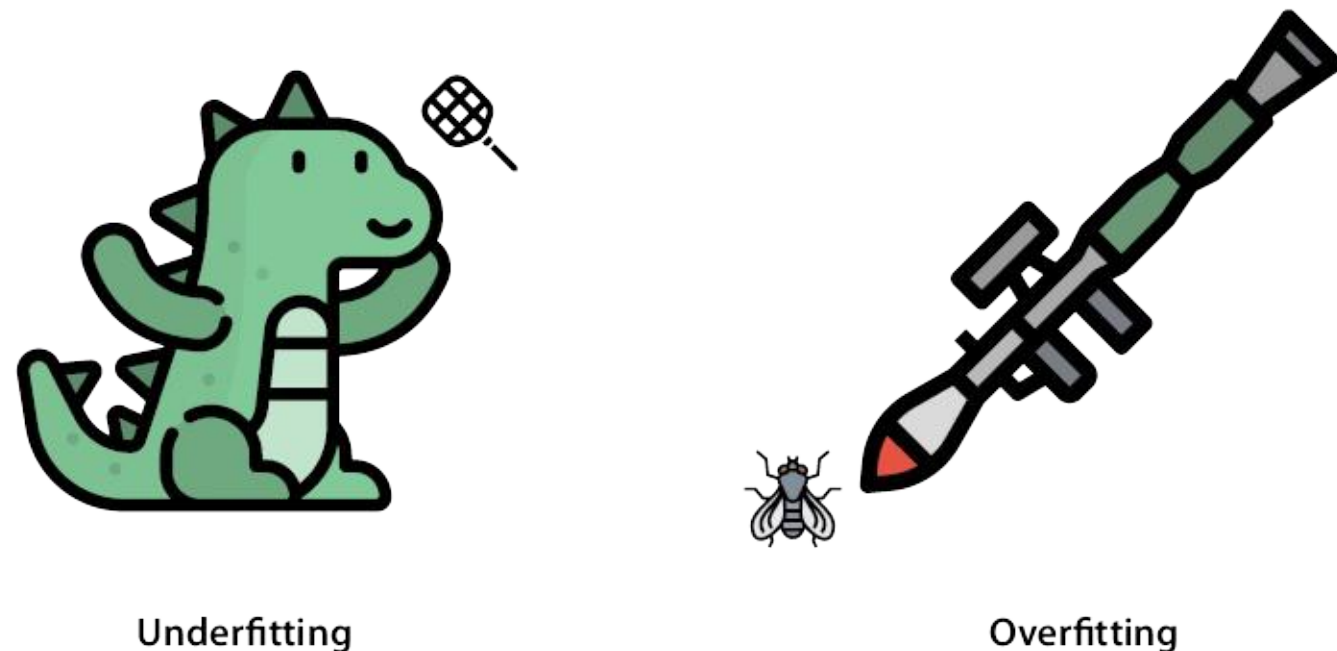
Trong học máy, *không trang bị đầy đủ* có vẻ giống như chưa học đủ để chuẩn bị cho kỳ thi. Nó xảy ra khi chúng ta cố gắng huấn luyện một mô hình quá đơn giản và nó không thể học được dữ liệu. *Trang bị quá mức* trông giống như học thuộc lòng toàn bộ sách giáo khoa thay vì ôn thi. Nó xảy ra khi chúng ta cố gắng huấn luyện một mô hình quá phức tạp và nó ghi nhớ dữ liệu thay vì học tốt. Một người mẫu tốt, không vừa cũng không quá vừa, là một người có vẻ như đã học tập tốt cho kỳ thi. Điều này tương ứng với một mô hình tốt có khả năng tìm hiểu dữ liệu đúng cách và có thể đưa ra dự đoán tốt về dữ liệu mới mà nó chưa thấy.

Một cách khác để nghĩ về việc trang bị thiếu và trang bị quá mức là khi chúng ta có một nhiệm vụ trong tay. Chúng ta có thể phạm hai sai lầm. Chúng ta có thể đơn giản hóa vấn đề và đưa ra giải pháp quá đơn giản. Chúng ta cũng có thể phức tạp hóa vấn đề quá mức và đưa ra giải pháp quá phức tạp.

Hãy tưởng tượng nếu nhiệm vụ của chúng ta là tiêu diệt Godzilla, như minh họa trong hình 4.1, và chúng ta ra trận mà không được trang bị gì ngoài một cái vĩ đập ruồi. Đó là một ví dụ về *sự đơn giản hóa quá mức*. Cách tiếp cận này sẽ không suôn sẻ với chúng tôi vì chúng tôi đã đánh giá thấp vấn đề và không chuẩn bị trước. Điều này chưa phù hợp: tập dữ liệu của chúng tôi rất phức tạp và chúng tôi đi đến mô hình hóa nó mà không có gì ngoài một mô hình đơn giản. Mô hình sẽ không thể nắm bắt được sự phức tạp của tập dữ liệu.

Ngược lại, nếu nhiệm vụ của chúng ta là giết một con ruồi nhỏ và chúng ta sử dụng bazooka để thực hiện công việc đó thì đây là một ví dụ về *sự phức tạp quá mức*. Đúng, chúng ta có thể giết chết con ruồi, nhưng chúng ta cũng sẽ phá hủy mọi thứ trong tầm tay và khiến bản thân gặp nguy hiểm. Chúng tôi đã đánh giá quá cao vấn đề và giải pháp của chúng tôi không tốt. Điều này quá phù hợp: dữ liệu của chúng tôi rất đơn giản, nhưng

chúng tôi cố gắng điều chỉnh nó cho phù hợp với một mô hình quá phức tạp. Mô hình sẽ có thể khớp với dữ liệu của chúng tôi nhưng nó sẽ ghi nhớ thay vì học dữ liệu đó. Lần đầu tiên tôi học cách trang bị quá mức, phản ứng của tôi là "Chà, không sao cả. Nếu tôi sử dụng một mô hình quá phức tạp, tôi vẫn có thể lập mô hình dữ liệu của mình, phải không?" Đúng, nhưng vấn đề thực sự của việc trang bị quá mức là cố gắng làm cho mô hình đưa ra dự đoán về dữ liệu không nhìn thấy được. Những dự đoán có thể sẽ trở nên khủng khiếp, như chúng ta sẽ thấy ở phần sau của chương này.



Hình 4.1 Trang bị thiếu và trang bị quá mức là hai vấn đề có thể xảy ra khi huấn luyện mô hình học máy của chúng ta. Bên trái: Việc trang bị không phù hợp xảy ra khi chúng ta đơn giản hóa quá mức vấn đề trước mắt và cố gắng giải quyết nó bằng một giải pháp đơn giản, chẳng hạn như cố gắng giết Godzilla bằng vĩ đập ruồi. Đúng: Việc trang bị quá mức xảy ra khi chúng ta phức tạp hóa quá mức giải pháp của một vấn đề và cố gắng giải quyết nó bằng một giải pháp cực kỳ phức tạp, chẳng hạn như cố gắng giết một con ruồi bằng bazooka.

Như chúng ta đã thấy trong phần “Tham số và siêu tham số” ở chương 3, mọi mô hình học máy đều có siêu tham số, là các nút mà chúng ta vặn và xoay trước khi huấn luyện mô hình. Việc thiết lập các siêu tham số phù hợp cho mô hình của chúng tôi là cực kỳ quan trọng. Nếu chúng ta đặt sai một số trong số chúng, chúng ta sẽ dễ bị thiếu hoặc thừa. Các kỹ thuật mà chúng tôi đề cập trong chương này rất hữu ích để giúp chúng tôi điều chỉnh các siêu tham số một cách chính xác.

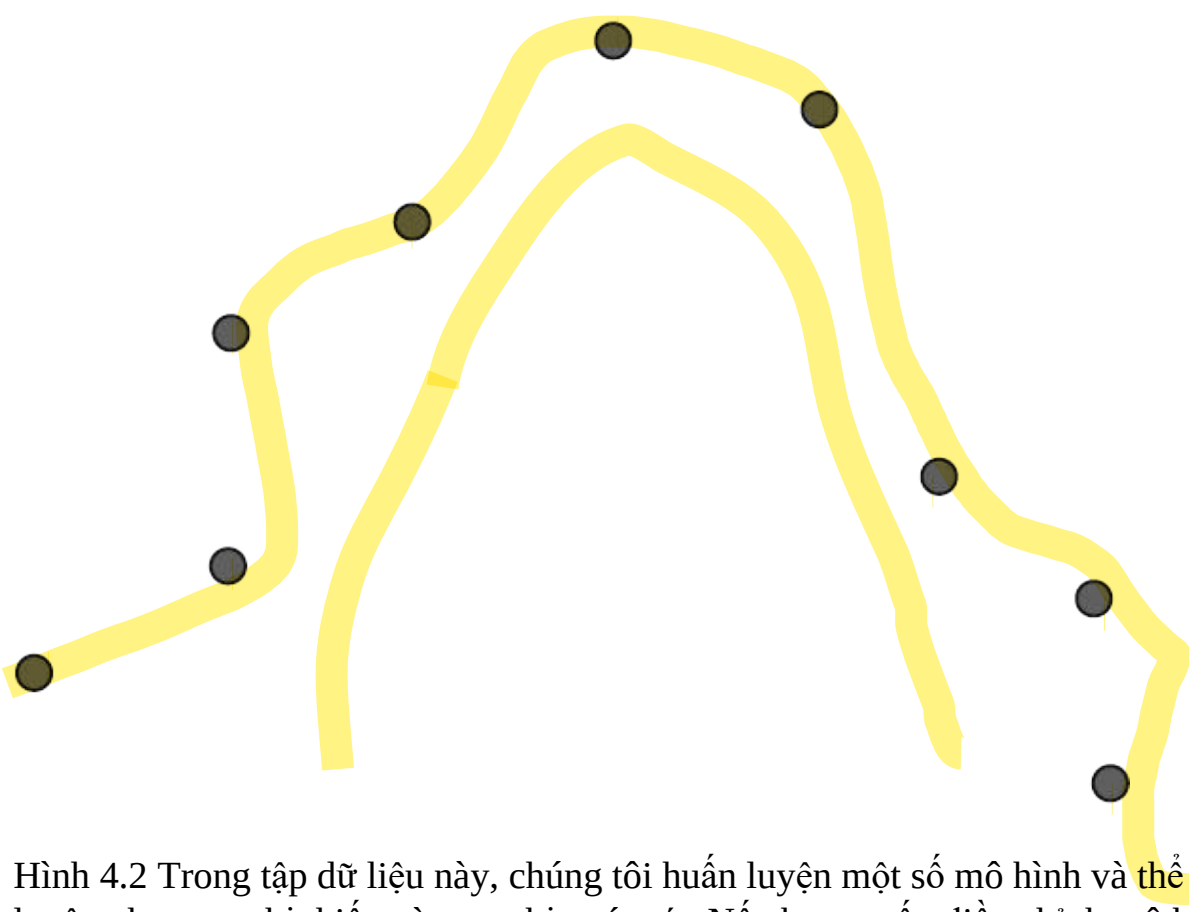
Để làm cho các khái niệm này rõ ràng hơn, chúng ta sẽ xem xét một ví dụ với tập dữ liệu và một số mô hình khác nhau được tạo bằng cách thay đổi một siêu tham số cụ thể: bậc của đa thức.

Bạn có thể tìm tất cả mã cho chương này trong kho GitHub sau:

Ví dụ về trang bị thiếu và trang bị quá mức sử dụng hồi quy đa thức

Trong phần này, chúng ta thấy một ví dụ về trang bị quá mức và thiếu trang bị trong cùng một tập dữ liệu. Xem kỹ tập dữ liệu trong hình 4.2 và cố gắng điều chỉnh mô hình hồi quy đa thức

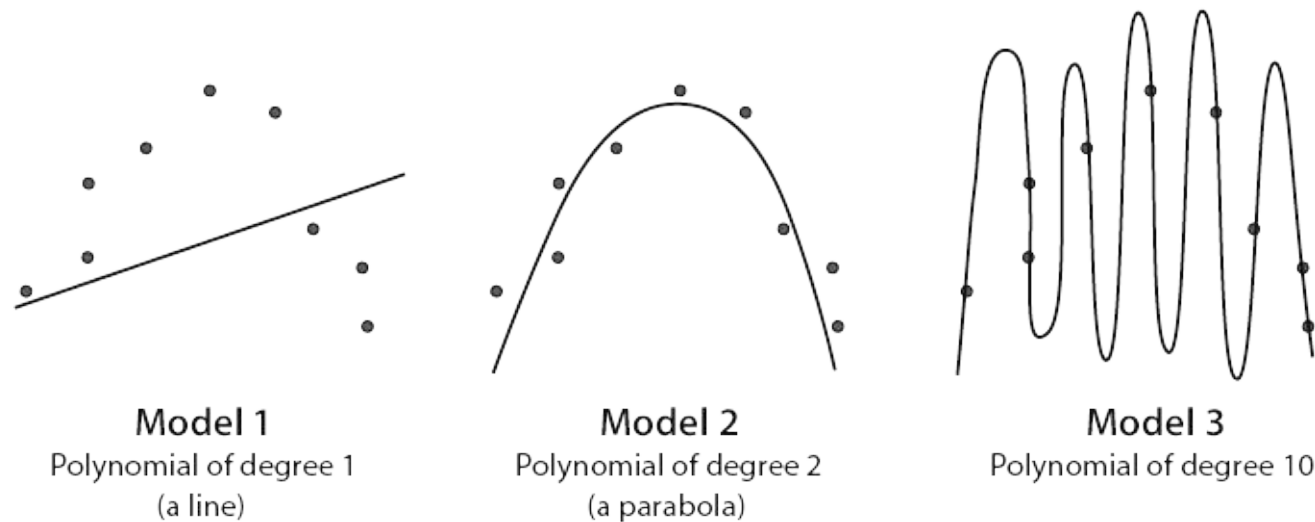
(xem trong phần “Nếu dữ liệu không nằm trên một dòng thì sao?” ở chương 3). Hãy nghĩ xem loại đa thức nào sẽ phù hợp với tập dữ liệu này. Nó sẽ là một đường thẳng, một parabol, một khối hay có lẽ là một đa thức bậc 100? Hãy nhớ lại rằng bậc của đa thức là số mũ cao nhất hiện có. Ví dụ: đa thức $2x^{14} + 9x^6 - 3x + 2$ có bậc 14.



Hình 4.2 Trong tập dữ liệu này, chúng tôi huấn luyện một số mô hình và thể hiện các vấn đề huấn luyện như trang bị thiếu và trang bị quá mức. Nếu bạn muốn điều chỉnh mô hình hồi quy đa thức cho tập dữ liệu này, bạn sẽ sử dụng loại đa thức nào: đường thẳng, parabol hay loại nào khác?

Tôi nghĩ tập dữ liệu đó trông rất giống một hình parabol mở xuống (khuôn mặt buồn). Đây là đa thức bậc 2. Tuy nhiên, chúng ta là con người và chúng ta đã quan sát nó. Một

máy tính không thể làm được điều đó. Một máy tính cần thử nhiều giá trị cho bậc của đa thức và bằng cách nào đó chọn ra giá trị tốt nhất. Giả sử máy tính sẽ cố gắng khớp các đa thức bậc 1, 2 và 10. Khi chúng ta ghép các đa thức bậc 1 (một đường), 2 (bậc hai) và 10 (một đường cong dao động tối đa chín lần) vào tập dữ liệu này, chúng ta thu được kết quả như trong hình 4.3.



Hình 4.3 Ghép ba mô hình vào cùng một tập dữ liệu. Mô hình 1 là đa thức bậc 1, là một đường thẳng. Mô hình 2 là đa thức bậc 2 hoặc bậc hai. Mô hình 3 là đa thức **bậc 10**. Mô hình nào phù hợp nhất?

Trong hình 4.3, chúng ta thấy ba mô hình, mô hình 1, mô hình 2 và mô hình 3. Lưu ý rằng **mô hình 1 quá đơn giản vì nó là một đường cố gắng khớp một tập dữ liệu bậc hai**. Không có cách nào chúng tôi có thể tìm thấy một dòng tốt để phù hợp với tập dữ liệu này, bởi vì tập dữ liệu đơn giản là trông không giống một dòng. Vì vậy, mô hình 1 là một ví dụ rõ ràng về việc trang bị chưa đầy đủ. Ngược lại, Mô hình 2 khá phù hợp với dữ liệu. Mô hình này không vừa vặn cũng không vừa vặn. **Mô hình 3 cực kỳ phù hợp với dữ liệu, nhưng nó hoàn toàn sai mục đích. Dữ liệu trông giống như một parabol có một chút nhiễu và mô hình vẽ ra một đa thức rất phức tạp bậc 10 có thể đi qua từng điểm nhưng không nắm bắt được bản chất của dữ liệu. Mô hình 3 là một ví dụ rõ ràng về việc trang bị quá mức.**

Để tóm tắt lý do trước đó, đây là một quan sát mà chúng tôi sử dụng trong suốt chương này và trong nhiều phần khác của cuốn sách này: các mô hình rất đơn giản có xu hướng không phù hợp. Các mô hình rất phức tạp có xu hướng phù hợp quá mức. **Mục tiêu là tìm ra một mô hình không quá đơn giản cũng không quá phức tạp và nắm bắt tốt bản chất dữ liệu của chúng ta.**

Chúng ta sắp đến phần thử thách. Là con người, chúng ta biết rằng mô hình 2 phù hợp nhất. Nhưng máy tính nhìn thấy gì? **Máy tính chỉ có thể**

tính toán hàm sai số.Như bạn có thể nhớ lại ở chương 3, chúng ta đã định nghĩa hai hàm sai số: sai số tuyệt đối và sai số bình phương.Để rõ ràng về mặt trực quan, trong ví dụ này, chúng tôi sẽ sử dụng sai số tuyệt đối, là giá trị trung bình của tổng các giá trị tuyệt đối của khoảng cách từ các điểm đến đường cong, mặc dù các đối số tương tự sẽ áp dụng với sai số bình phương.**Đối với mô hình 1, các điểm nằm xa mô hình nên sai số này lớn.**Đối với mô hình 2, các khoảng **cách này nhỏ nên sai số nhỏ.**Tuy nhiên, đối **với mô hình 3, khoảng cách bằng 0 vì các điểm đều nằm trên đường cong thực tế!**Điều này có nghĩa là máy tính sẽ cho rằng mô hình hoàn hảo là mô hình 3. Điều này không tốt.Chúng ta cần một cách để nói với máy tính rằng mô hình tốt nhất là mô hình 2 và mô hình 3 là quá phù hợp.Làm thế nào chúng ta có thể làm điều này?Tôi khuyến khích bạn đặt cuốn sách này xuống trong vài phút và tự mình nghĩ ra một số ý tưởng vì có một số giải pháp cho vấn đề này.

Làm cách nào để máy tính chọn đúng model? Bằng cách thử nghiệm

Một cách để xác định xem một mô hình có phù hợp hay không là thử nghiệm nó và đó là những gì chúng tôi làm trong phần này. Việc kiểm tra một mô hình bao gồm việc **chọn một tập hợp nhỏ các điểm trong tập dữ liệu** và chọn sử dụng chúng **không phải để huấn luyện mô hình mà để kiểm tra hiệu suất của mô hình.**Tập hợp điểm này được gọi là **tập kiểm tra.**Tập điểm còn lại (phần lớn) mà chúng tôi sử dụng để huấn luyện mô hình, được gọi là **tập huấn luyện.**Sau khi **đào tạo mô hình trên tập huấn luyện,** chúng tôi **sử dụng tập kiểm tra để đánh giá mô hình.**Bằng cách này, chúng tôi đảm bảo rằng mô hình có khả năng khái quát hóa tốt dữ liệu không nhìn thấy, trái ngược với việc ghi nhớ tập huấn luyện.Quay trở lại với bài kiểm tra tương tự, hãy tưởng tượng việc đào tạo và kiểm tra theo cách này.Giả sử cuốn sách chúng ta đang học trong kỳ thi có 100 câu hỏi ở cuối.Chúng tôi chọn 80 người trong số họ để đào tạo, nghĩa là chúng tôi nghiên cứu

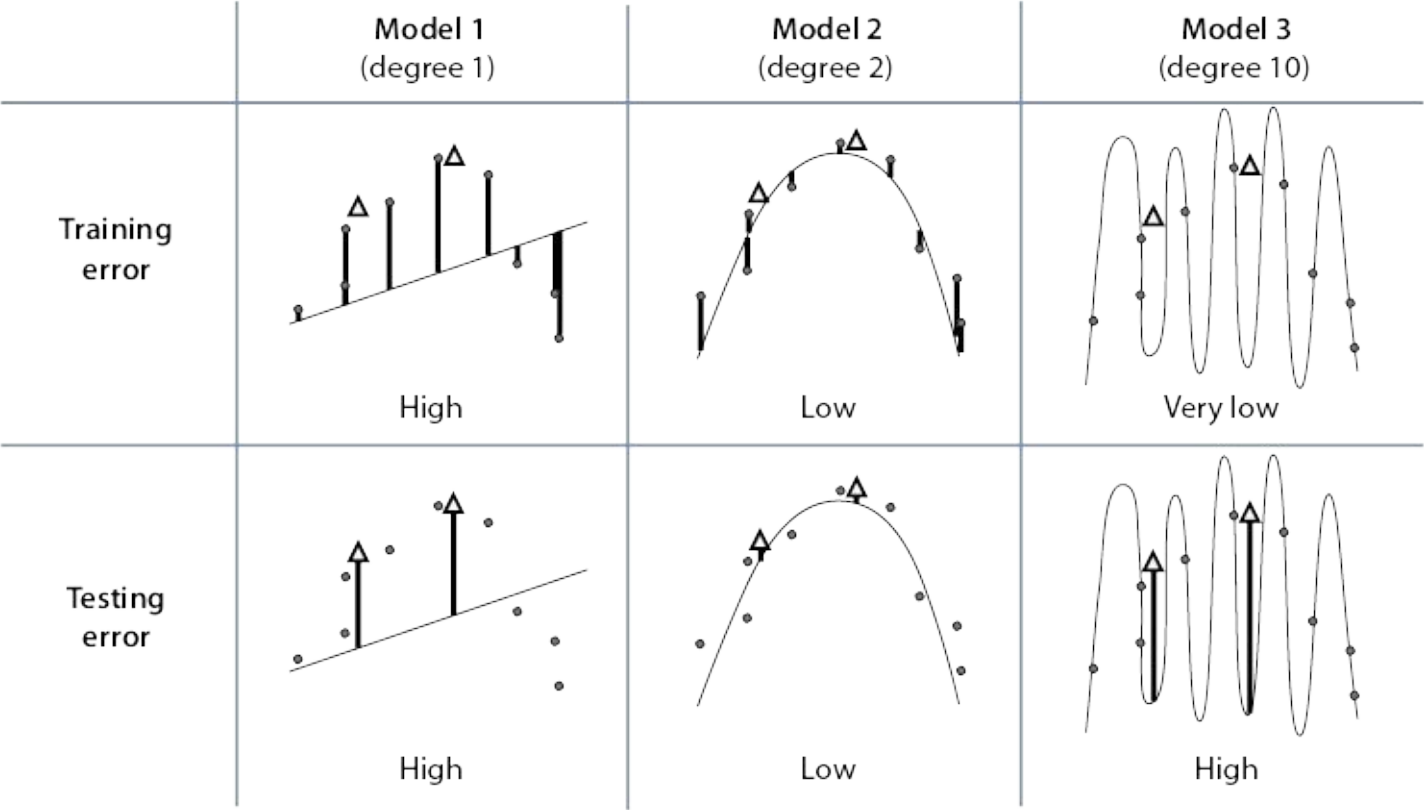
chúng một cách cẩn thận, tra cứu câu trả lời và tìm hiểu chúng.Sau đó, chúng tôi sử dụng 20 câu hỏi còn lại để tự kiểm tra—chúng tôi cố gắng trả lời chúng mà không cần nhìn vào sách, giống như trong một kỳ thi.

Bây giờ hãy xem phương pháp này trông như thế nào với tập dữ liệu và mô hình của chúng tôi.Lưu ý rằng vấn đề thực sự với mô hình 3 không phải là nó không phù hợp với dữ liệu;đó là nó không khái quát hóa tốt dữ liệu mới.Nói cách khác, nếu bạn đã huấn luyện mô hình 3 trên tập dữ liệu đó và một số điểm mới xuất hiện, bạn có tin tưởng mô hình sẽ đưa ra dự đoán tốt với những điểm mới này không?Có lẽ là không, bởi vì mô hình chỉ ghi nhớ toàn bộ tập dữ liệu mà không nắm bắt được bản chất của nó.Trong trường hợp này,

Bản chất của tập dữ liệu là nó trông giống như một parabol mở xuống dưới.

Trong hình 4.4, chúng tôi đã vẽ hai hình tam giác màu trắng trong tập dữ liệu của mình, đại diện cho tập thử nghiệm. Tập huấn luyện tương ứng với các vòng tròn màu đen. Bây giờ, hãy xem xét chi tiết con số này và xem ba mô hình này hoạt động như thế nào với cả tập huấn luyện và tập thử nghiệm của chúng tôi. Nói cách khác, hãy kiểm tra lỗi mà mô hình tạo ra trong cả hai tập dữ liệu. Chúng tôi sẽ gọi hai lỗi này là *lỗi đào tạo* và *lỗi kiểm tra*.

Hàng trên cùng trong hình 4.4 tương ứng với tập huấn luyện và hàng dưới cùng tương ứng với tập kiểm tra. Để minh họa lỗi, chúng tôi đã vẽ các đường thẳng đứng từ điểm đến mô hình. Sai số tuyệt đối trung bình chính xác là giá trị trung bình của độ dài của những đường này. Nhìn vào hàng trên cùng, chúng ta có thể thấy mô hình 1 có lỗi huấn luyện lớn, mô hình 2 có lỗi huấn luyện nhỏ và mô hình 3 có lỗi huấn luyện rất nhỏ (thực tế là bằng 0). Như vậy, mô hình 3 thực hiện công việc tốt nhất trên tập huấn luyện.



Hình 4.4 Chúng ta có thể sử dụng bảng này để quyết định mức độ phức tạp mà chúng ta muốn mô hình của mình. Các cột biểu thị ba mô hình cấp 1, 2 và 10. Các cột biểu thị lỗi huấn luyện và lỗi kiểm tra. Các vòng tròn đặc là tập huấn luyện và các hình tam giác màu trắng là tập kiểm tra. Các sai số tại mỗi điểm có thể được coi là các đường thẳng đứng từ điểm đó đến đường cong. Sai số của mỗi mô hình là sai số tuyệt đối trung bình được tính bằng trung bình của các chiều dài dọc này. Lưu ý rằng lỗi huấn luyện giảm dần khi độ phức tạp của mô hình tăng lên. Tuy nhiên, lỗi kiểm tra giảm dần và sau đó quay trở lại khi độ phức tạp tăng lên. Từ bảng này, chúng tôi kết luận rằng trong số ba mô hình này, mô hình tốt nhất là mô hình 2, vì nó cho chúng tôi sai số kiểm tra thấp.

Tuy nhiên, khi chúng tôi đến bộ thử nghiệm, mọi thứ sẽ thay đổi. Mô hình 1 vẫn còn lỗi kiểm tra lớn, nghĩa là đây đơn giản là một mô hình tồi, hoạt động kém với tập huấn luyện và tập kiểm tra: nó không phù hợp. Mô hình 2 có một lỗi kiểm tra nhỏ, điều đó có nghĩa là nó là một mô hình tốt vì nó phù hợp tốt với cả tập huấn luyện và tập kiểm tra. Tuy nhiên, Mô hình 3 tạo ra lỗi kiểm tra lớn. Bởi vì nó đã thực hiện công việc rất tệ khi khớp với tập kiểm tra nhưng lại làm rất tốt việc khớp với tập huấn luyện, chúng tôi kết luận rằng mô hình 3 phù hợp quá mức.

Hãy tóm tắt những gì chúng ta đã học được cho đến nay.

Người mẫu có thể

- Underfit: sử dụng mô hình quá đơn giản so với tập dữ liệu của chúng tôi.

- Phù hợp với dữ liệu: sử dụng mô hình có độ phức tạp phù hợp cho tập dữ liệu của chúng tôi. Overfit: sử dụng một mô hình quá phức tạp đối với tập dữ liệu của chúng tôi.

-

Trong tập huấn luyện

- Mô hình không phù hợp sẽ hoạt động kém (lỗi huấn luyện lớn). Mô hình tốt
- sẽ hoạt động tốt (lỗi huấn luyện nhỏ). Mô hình overfit sẽ hoạt động rất tốt
- (lỗi huấn luyện rất nhỏ).

Trong bộ thử nghiệm

- Mô hình không phù hợp sẽ hoạt động kém (lỗi kiểm tra lớn). Mô
- hình tốt sẽ hoạt động tốt (lỗi kiểm tra nhỏ). Mô hình overfit sẽ hoạt
- động kém (lỗi kiểm tra lớn).

Vì vậy, cách để biết liệu một mô hình phù hợp, quá phù hợp hay tốt là xem xét các lỗi huấn luyện và kiểm tra. Nếu cả hai lỗi đều cao thì nó không phù hợp. Nếu cả hai sai số đều thấp thì đó là mô hình tốt. Nếu lỗi huấn luyện thấp và lỗi kiểm tra cao thì nó quá khớp.

Chúng ta chọn bộ thử nghiệm như thế nào và nó phải lớn bao nhiêu?

Đây là một câu hỏi. Tôi đã lấy hai điểm mới đó từ đâu? Nếu chúng ta đang đào tạo một mô hình trong quá trình sản xuất trong đó dữ liệu luôn được luân chuyển thì chúng ta có thể chọn một số điểm mới làm dữ liệu thử nghiệm. Nhưng điều gì sẽ xảy ra nếu chúng ta không có cách nào để nhận được điểm mới và tất cả những gì chúng ta có chỉ là tập dữ liệu ban đầu gồm 10 điểm? Khi điều này xảy ra, chúng tôi chỉ hy sinh một số dữ liệu của mình và sử dụng nó làm bộ thử nghiệm. Bao nhiêu dữ liệu? Điều đó phụ thuộc vào lượng dữ liệu chúng tôi có và mức độ chúng tôi muốn mô hình hoạt động tốt như thế nào, nhưng trên thực tế, bất kỳ giá trị nào từ 10% đến 20% dường như đều hoạt động tốt.

Chúng tôi có thể sử dụng dữ liệu thử nghiệm của mình để huấn luyện mô hình không? Không.

Trong học máy, chúng ta luôn cần tuân theo một quy tắc quan trọng: khi chia dữ liệu thành đào tạo và kiểm tra, chúng ta nên sử dụng dữ liệu đào tạo để đào tạo mô hình và hoàn toàn không có lý do gì chúng ta không nên chạm vào dữ liệu kiểm tra trong khi đào tạo mô hình hoặc đưa ra quyết định về siêu tham số của mô hình. Thất bại trong việc

làm như vậy có thể dẫn đến việc trang bị quá mức, thậm chí

nếu nó không được con người chú ý. Trong nhiều cuộc thi học máy, các đội đã gửi những mô hình mà họ cho là tuyệt vời nhưng lại thất bại thảm hại khi thử nghiệm trên một tập dữ liệu bí mật. Điều này có thể là do các nhà khoa học dữ liệu đào tạo các mô hình bằng cách nào đó (có thể vô tình) đã sử dụng dữ liệu thử nghiệm để đào tạo chúng. Trên thực tế, quy tắc này rất quan trọng nên chúng tôi sẽ coi nó là quy tắc vàng của cuốn sách này.

quy tắc vàng Bạn không bao giờ được sử dụng dữ liệu thử nghiệm của mình để đào tạo.

Hiện tại, có vẻ như đây là một quy tắc dễ thực hiện, nhưng như chúng ta sẽ thấy, quy tắc đó rất dễ vô tình bị phá vỡ.

Trên thực tế, chúng ta đã vi phạm nguyên tắc vàng trong chương này. Bạn có thể cho biết ở đâu? Tôi khuyến khích bạn quay lại và tìm nơi chúng tôi đã phá vỡ nó. Chúng ta sẽ xem ở đâu trong phần tiếp theo.

Chúng ta đã vi phạm quy tắc vàng ở đâu và làm cách nào để khắc phục nó? Bộ xác thực

Trong phần này, chúng ta sẽ thấy mình đã vi phạm quy tắc vàng ở đâu và học một kỹ thuật gọi là xác nhận, kỹ thuật này sẽ giúp ích cho chúng ta.

Chúng tôi đã phá vỡ quy tắc vàng trong phần “Làm cách nào để máy tính chọn đúng kiểu máy”. Hãy nhớ lại rằng chúng ta có ba mô hình hồi quy đa thức: một ở mức 1, một ở mức 2 và một ở mức 10, và chúng tôi không biết nên chọn mô hình nào. Chúng tôi đã sử dụng dữ liệu đào tạo của mình để đào tạo ba mô hình và sau đó chúng tôi sử dụng dữ liệu thử nghiệm để quyết định nên chọn mô hình nào. Chúng tôi không được phép sử dụng dữ liệu thử nghiệm để huấn luyện mô hình của mình hoặc đưa ra bất kỳ quyết định nào về mô hình hoặc các siêu tham số của nó. Một khi chúng tôi làm điều này, chúng tôi có khả năng bị trang bị quá mức! Chúng tôi có khả năng phù hợp quá mức mỗi khi xây dựng một mô hình phục vụ quá nhiều cho tập dữ liệu của mình.

Chúng ta có thể làm gì? Giải pháp rất đơn giản: chúng tôi chia nhỏ tập dữ liệu của mình hơn nữa. Chúng tôi giới thiệu một bộ mới, **bộ xác thực**, sau đó chúng tôi sử dụng để đưa ra quyết định về tập dữ liệu của mình. Tóm lại, chúng tôi chia tập dữ liệu của mình thành ba bộ sau:

- **Bộ huấn luyện:** để đào tạo tất cả các mô hình của chúng tôi
- **Bộ xác thực:** để đưa ra quyết định sử dụng mô hình nào
- **Bộ kiểm tra:** để kiểm tra xem mô hình của chúng tôi đã hoạt động tốt như thế nào

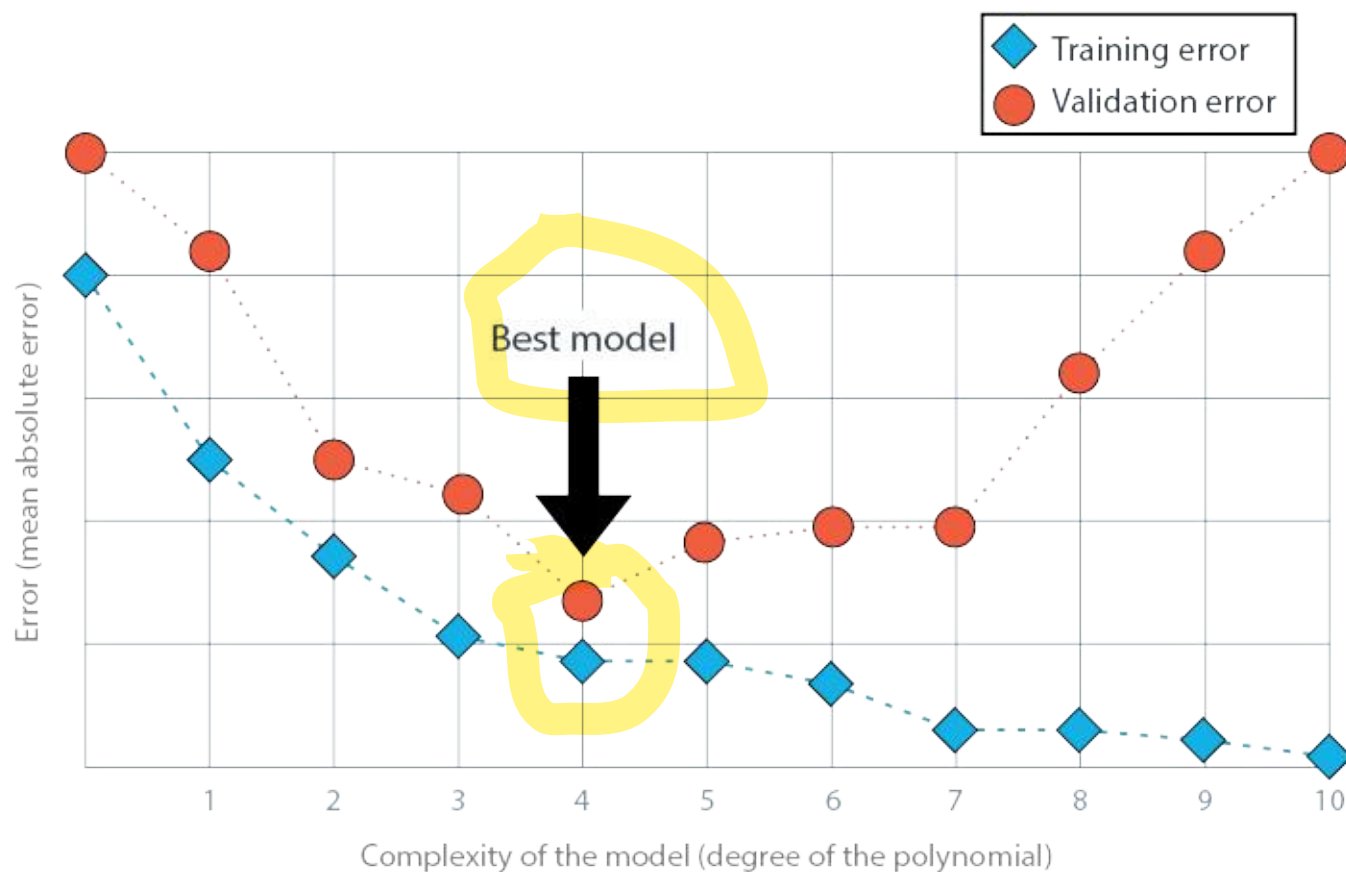
Do đó, trong ví dụ của chúng tôi, chúng tôi sẽ có thêm hai điểm để sử dụng để xác thực và việc xem xét lỗi xác thực sẽ giúp chúng tôi quyết định rằng mô hình tốt nhất để sử dụng là mô hình 2. Chúng tôi nên sử dụng bộ thử nghiệm ở cuối để xem mô hình của chúng tôi đã hoạt động tốt như thế nào. Nếu mô hình không tốt, chúng ta nên vứt bỏ mọi thứ và làm lại từ đầu.

Về quy mô của các bộ kiểm tra và xác nhận, người ta thường sử dụng tỷ lệ phân chia 60-20-20 hoặc 80-10-10—nói cách khác, đào tạo 60%, xác thực 20%, kiểm tra 20% hoặc đào tạo 80%, xác thực 10%, kiểm tra 10%. Những con số này là tùy ý, nhưng chúng có xu hướng hoạt động tốt, vì chúng để lại phần lớn dữ liệu cho việc huấn luyện nhưng vẫn cho phép chúng ta kiểm tra mô hình trong một tập hợp đủ lớn.

Một cách bằng số để quyết định mức độ phức tạp của mô hình của chúng tôi: Biểu đồ độ phức tạp của mô hình

Trong các phần trước, chúng ta đã học cách sử dụng bộ xác thực để giúp chúng ta quyết định mô hình nào là tốt nhất trong số ba mô hình khác nhau. Trong phần này, chúng ta tìm hiểu về một biểu đồ có tên là *biểu đồ độ phức tạp của mô hình*, biểu đồ này giúp chúng ta quyết định trong số nhiều mô hình khác. Hãy tưởng tượng rằng chúng ta có một tập dữ liệu khác và phức tạp hơn nhiều và chúng ta đang cố gắng xây dựng một mô hình hồi quy đa thức để phù hợp với nó. Chúng tôi muốn quyết định mức độ của mô hình của chúng tôi trong số các số từ 0 đến 10 (bao gồm). Như chúng ta đã thấy trong phần trước, cách quyết định sử dụng mô hình nào là chọn mô hình có lỗi xác thực nhỏ nhất.

Tuy nhiên, việc vẽ sơ đồ các lỗi huấn luyện và kiểm tra có thể cung cấp cho chúng ta một số thông tin có giá trị và giúp chúng ta kiểm tra các xu hướng. Trong hình 4.5, bạn có thể thấy một biểu đồ trong đó trục hoành biểu thị bậc của đa thức trong mô hình và trục tung biểu thị giá trị sai số. Các viên kim cương biểu thị lỗi huấn luyện và các vòng tròn biểu thị lỗi xác thực. Đây là biểu đồ độ phức tạp của mô hình.



Hình 4.5 Biểu đồ độ phức tạp của mô hình là một công cụ hiệu quả giúp chúng ta xác định độ phức tạp lý tưởng của một mô hình để tránh khớp quá mức và thiếu khớp. Trong biểu đồ độ phức tạp của mô hình này, trục hoành biểu thị mức độ của một số mô hình hồi quy đa thức, từ 0 đến 10 (tức là độ phức tạp của mô hình). Trục tung biểu thị sai số, trong trường hợp này được biểu thị bằng sai số tuyệt đối trung bình. Lưu ý rằng sai số huấn luyện bắt đầu lớn và giảm dần khi chúng ta di chuyển sang phải. Điều này là do mô hình của chúng tôi càng phức tạp thì nó càng phù hợp với dữ liệu huấn luyện hơn. Tuy nhiên, lỗi xác thực bắt đầu lớn, sau đó giảm dần rồi lại tăng—các mô hình rất đơn giản không thể phù hợp tốt với dữ liệu của chúng tôi (chúng không phù hợp), trong khi các mô hình rất phức tạp phù hợp với dữ liệu đào tạo của chúng tôi chứ không phải dữ liệu xác thực của chúng tôi vì chúng quá phù hợp. Một điểm thú vị ở giữa là mô hình của chúng tôi không phù hợp hoặc quá mức và chúng tôi có thể tìm thấy nó bằng cách sử dụng biểu đồ độ phức tạp của mô hình.

Lưu ý rằng trong biểu đồ độ phức tạp của mô hình trong Hình 4.5, giá trị thấp nhất của lỗi xác thực xảy ra ở cấp độ 4, có nghĩa là đối với tập dữ liệu này, mô hình phù hợp nhất (trong số những mô hình chúng ta đang xem xét) là mô hình hồi quy đa thức cấp độ 4. Nhìn vào bên trái của biểu đồ, chúng ta có thể thấy rằng khi bậc của đa thức nhỏ thì cả lỗi huấn luyện và lỗi xác thực đều lớn, điều này ngụ ý rằng các mô hình không phù hợp. Nhìn vào bên phải biểu đồ, chúng ta có thể thấy rằng lỗi huấn luyện ngày càng nhỏ hơn, nhưng lỗi xác thực ngày càng lớn hơn, điều này ngụ ý rằng các mô hình quá phù hợp. Điểm tốt nhất xảy ra vào khoảng 4 giờ, đó là mô hình chúng tôi chọn.

Một lợi ích của biểu đồ độ phức tạp của mô hình là cho dù tập dữ liệu của chúng ta lớn đến đâu hay chúng ta thử bao nhiêu mô hình khác nhau thì nó luôn trông giống như hai đường cong: một đường cong luôn đi xuống (lỗi huấn luyện) và một đường cong đi xuống rồi sao lưu (lỗi xác thực). Tất nhiên, trong một tập dữ liệu lớn và phức tạp, những đường cong này có thể dao động và hành vi có thể khó phát hiện hơn. Tuy nhiên, biểu đồ độ phức tạp của mô hình luôn là công cụ hữu ích để các nhà khoa học dữ liệu tìm ra vị trí phù hợp trong biểu đồ này và quyết định mức độ phức tạp của mô hình của họ để tránh cả tình trạng thiếu và quá khớp.

Tại sao chúng ta cần một biểu đồ như vậy nếu tất cả những gì chúng ta cần làm là chọn mô hình có sai số xác thực thấp nhất? Phương pháp này đúng về mặt lý thuyết, nhưng trên thực tế, với tư cách là một nhà khoa học dữ liệu, bạn có thể hiểu rõ hơn nhiều về vấn đề bạn đang giải quyết, các ràng buộc và điểm chuẩn. Ví dụ: nếu bạn thấy rằng mô hình có lỗi xác thực nhỏ nhất vẫn khá phức tạp và có một mô hình đơn giản hơn nhiều nhưng chỉ có lỗi xác thực cao hơn một chút, thì bạn có thể có xu hướng chọn mô hình đó hơn. Một nhà khoa học dữ liệu giỏi là người có thể kết hợp các công cụ lý thuyết này với kiến thức của họ về trường hợp sử dụng để xây dựng các mô hình tốt nhất và hiệu quả nhất.

Một cách khác để tránh trang bị quá mức: Chính quy hóa

Trong phần này, chúng ta thảo luận về một kỹ thuật hữu ích khác để tránh trang bị quá mức trong các mô hình không yêu cầu bộ thử nghiệm của chúng tôi: **chính quy hóa**. Quá trình chính quy hóa dựa trên quan sát tương tự mà chúng tôi đã thực hiện trong phần “Ví dụ về khớp quá mức và quá khớp bằng cách sử dụng hồi quy đa thức”, trong đó chúng tôi kết luận rằng các mô hình đơn giản có xu hướng không phù hợp và các mô hình phức tạp có xu hướng phù hợp quá mức. Tuy nhiên, trong các phương pháp trước, chúng tôi đã thử nghiệm một số mô hình và chọn ra mô hình có hiệu suất cân bằng tốt nhất và độ phức tạp. Ngược lại, khi sử dụng chính quy hóa, chúng ta không cần đào tạo nhiều mô hình. Chúng tôi chỉ đào tạo mô hình một lần, nhưng trong quá trình đào tạo, chúng tôi cố gắng không chỉ cải thiện hiệu suất của mô hình mà còn giảm độ phức tạp của mô hình. Chìa khóa để làm điều này là đo lường cả hiệu suất và độ phức tạp cùng một lúc.

Trước khi đi vào chi tiết, chúng ta hãy thảo luận về một sự tương tự khi suy nghĩ về việc đo lường hiệu suất và độ phức tạp của các mô hình. Hãy tưởng tượng chúng ta có ba ngôi nhà và tất cả chúng đều có cùng một vấn đề – mái nhà bị dột (hình 4.6). Ba người thợ mái đến và mỗi người sửa một ngôi nhà. Người thợ mái đầu tiên đã sử dụng một

bằng, cái thứ hai dùng ván lợp mái, và cái thứ ba dùng titan. Theo trực giác của chúng tôi, có vẻ như cách tốt nhất là Roofer 2, bởi vì Roofer 1 đã đơn giản hóa vấn đề quá mức (thiếu trang bị) và Roofer 3 đã phức tạp hóa vấn đề quá mức (trang bị quá mức).



Problem: Broken roof



Roofer 1

Solution: Bandage
(Underfitting)



Roofer 2

Solution: Shingles
(Correct)



Roofer 3

Solution: Titanium
(Overfitting)

Hình 4.6 Sự tương tự giữa trang bị thiếu và trang bị quá mức. Vấn đề của chúng tôi bao gồm một mái nhà bị hỏng. Chúng tôi có ba thợ lợp mái có thể sửa được nó. Roofer 1 đi kèm với băng quấn, Roofer 2 đi kèm với ván lợp mái và Roofer 3 đi kèm với khối titan. Roofer 1 đã đơn giản hóa vấn đề quá mức nên chúng thể hiện sự không phù hợp. Roofer 2 đã sử dụng một giải pháp tốt. Roofer 3 đã làm phức tạp hóa giải pháp quá mức nên chúng đại diện cho việc trang bị quá mức.

Tuy nhiên, chúng ta cần đưa ra quyết định bằng cách sử dụng các con số, vì vậy hãy thực hiện một số phép đo. Cách để đo lường hiệu suất của những người lợp mái là lượng nước rò rỉ sau khi họ sửa mái nhà. Họ đã có số điểm sau:

Hiệu suất (tính bằng mL nước bị rò rỉ)

Mái lợp 1: 1000 mL nước

Lợp 2: 1 mL nước

Mái lợp 3: 0 mL nước

Có vẻ như thợ lợp mái số 1 đã có màn trình diễn tệ hại vì mái nhà vẫn bị dột. Tuy nhiên, giữa thợ lợp mái 2 và thợ lợp mái 3, chúng ta chọn cái nào? Có lẽ

Roofer 3, ai có màn trình diễn tốt hơn?Thước đo hiệu suất không đủ tốt;nó loại bỏ chính xác Roofer 1 khỏi phương trình, nhưng nó lại bảo chúng ta chọn Roofer 3 thay vì Roofer 2 một cách sai lầm. Chúng ta cần thước đo độ phức tạp của chúng để giúp chúng ta đưa ra quyết định đúng đắn.Một thước đo tốt về độ phức tạp của họ là số tiền họ tính cho chúng tôi để sửa mái nhà, tính bằng đô la.Giá như sau:

Độ phức tạp (về giá)

Thợ lợp mái 1: 1\$

Thợ lợp mái 2: 100\$

Thợ lợp mái 3: 100.000 USD

Bây giờ chúng ta có thể nói rằng Roofer 2 tốt hơn Roofer 3, bởi vì chúng có cùng hiệu suất, nhưng Roofer 2 tính phí ít hơn.Tuy nhiên, máy lợp mái số 1 là loại rẻ nhất— tại sao chúng ta không chọn loại này?Có vẻ như điều chúng ta cần là kết hợp các thước đo hiệu suất và độ phức tạp.Chúng ta có thể cộng lượng nước mái nhà bị dột và giá cả để có được kết quả như sau:

Độ phức tạp của hiệu năng+

Thợ lợp mái 1: 1001

Thợ lợp mái 2: 101

Thợ lợp mái 3: 100.000

Bây giờ rõ ràng rằng Roofer 2 là tốt nhất, có nghĩa là việc tối ưu hóa hiệu suất và độ phức tạp đồng thời mang lại kết quả tốt mà cũng đơn giản nhất có thể.Đây chính là nội dung của việc chính quy hóa: đo lường hiệu suất và độ phức tạp bằng hai hàm lỗi khác nhau và thêm chúng để có được hàm lỗi mạnh mẽ hơn.Hàm lỗi mới này đảm bảo rằng mô hình của chúng tôi hoạt động tốt và không phức tạp lắm.Trong các phần sau, chúng ta sẽ tìm hiểu chi tiết hơn về cách xác định hai hàm lỗi này.Nhưng trước đó, hãy xem một ví dụ về trang bị quá mức khác.

Một ví dụ khác về trang bị quá mức: Đề xuất phim

Trong phần này, chúng ta tìm hiểu một cách tinh vi hơn trong đó một mô hình có thể quá khớp—lần này

không liên quan đến bậc của đa thức mà liên quan đến số lượng đặc trưng và độ lớn của các hệ số. Hãy tưởng tượng rằng chúng ta có một trang web phát trực tuyến phim và chúng ta đang cố gắng xây dựng một hệ thống giới thiệu. Để đơn giản, hãy tưởng tượng chúng ta chỉ có 10 phim: $M_1, M_2, \dots, M_{10}$. Một bộ phim mới, M_{11} , sắp ra mắt và chúng tôi muốn xây dựng mô hình hồi quy tuyến tính để đề xuất phim 11 dựa trên 10 phim trước đó. Chúng tôi có tập dữ liệu gồm 100 người dùng. Đối với mỗi người dùng, chúng tôi có 10 tính năng, đó là thời gian (tính bằng giây) mà người dùng đã xem từng bộ phim trong số 10 bộ phim gốc. Nếu người dùng chưa xem phim thì số tiền này là 0. Nhãn cho mỗi người dùng là lượng thời gian người dùng đã xem phim 11. Chúng tôi muốn xây dựng một mô hình phù hợp với tập dữ liệu này. Cho rằng mô hình là mô hình hồi quy tuyến tính, phương trình cho thời gian dự đoán mà người dùng sẽ xem phim 11 là tuyến tính và nó sẽ có dạng như sau:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 + w_7x_7 + w_8x_8 + w_9x_9 + w_{10}x_{10} + b,$$

Ở đâu

- $\hat{y}$ là khoảng thời gian mà mô hình dự đoán người dùng sẽ xem phim 11, x_i là khoảng thời gian người dùng đã xem phim i , đối với $i = 1, 2, \dots, 10$,
- w_i là trọng số liên quan đến phim i và b là độ lệch.
-

Bây giờ hãy kiểm tra trực giác của chúng ta. Trong số hai mô hình sau đây (được đưa ra bởi phương trình của chúng), mô hình nào (hoặc những mô hình) có vẻ như quá phù hợp?

Mẫu 1: $\hat{y} = 2x_3 + 1,4x_7 - 0,5x_9 + 4$

Mẫu 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$

Nếu bạn nghĩ giống tôi, mô hình 2 có vẻ hơi phức tạp và có thể là mô hình quá phù hợp. Trực giác ở đây là khó có khả năng thời gian người dùng xem phim 2 cần được nhân với -103 rồi cộng với các số khác để có được dự đoán. Điều này có thể rất phù hợp với dữ liệu, nhưng có vẻ như nó đang ghi nhớ dữ liệu thay vì học nó.

Ngược lại, Mô hình 1 trông đơn giản hơn nhiều và nó cung cấp cho chúng ta một số thông tin thú vị. Từ thực tế là hầu hết các hệ số đều bằng 0, ngoại trừ các hệ số đối với phim 3, 7 và 9, nó cho chúng ta biết rằng chỉ có ba bộ phim có liên quan đến phim

11 là ba bộ phim đó. Hơn nữa, từ thực tế là hệ số của phim 3 và 7 là dương, mô hình cho chúng ta biết rằng nếu người dùng đã xem phim 3 hoặc phim 7 thì có khả năng họ sẽ xem phim 11. Vì hệ số của phim 9 là âm nên nếu người dùng đã xem phim 9, họ sẽ không xem phim 11.

Mục tiêu của chúng tôi là có một mô hình giống như mô hình 1 và tránh các mô hình như mô hình 2. Nhưng thật không may, nếu mô hình 2 tạo ra sai số nhỏ hơn mô hình 1 thì việc chạy thuật toán hồi quy tuyến tính sẽ chọn mô hình 2 thay thế. Chúng ta có thể làm gì? Đây là nơi chính quy hóa có thể giải cứu. Điều đầu tiên chúng ta cần là một thước đo cho chúng ta biết rằng mô hình 2 phức tạp hơn nhiều so với mô hình 1.

Đo lường mức độ phức tạp của một mô hình: chỉ tiêu $L1$ và $L2$

Trong phần này, chúng ta tìm hiểu hai cách để đo lường độ phức tạp của một mô hình. Nhưng trước đó, hãy xem xét mô hình 1 và 2 từ phần trước và cố gắng đưa ra một số công thức thấp cho mô hình 1 và cao cho mô hình 2.

Lưu ý rằng một mô hình có nhiều hệ số hơn hoặc hệ số có giá trị cao hơn có xu hướng phức tạp hơn. Do đó, bất kỳ công thức nào phù hợp với điều này sẽ hoạt động, chẳng hạn như sau:

- Tổng các giá trị tuyệt đối của các hệ số Tổng bình phương của
- các hệ số

Cái đầu tiên được gọi là *chuẩn $L1$* và cái thứ hai được gọi là *chuẩn $L2$* . Chúng xuất phát từ một lý thuyết tổng quát hơn về không gian L^p , được đặt theo tên nhà toán học người Pháp Henri Lebesgue. Chúng tôi sử dụng các giá trị tuyệt đối và bình phương để loại bỏ các hệ số âm; nếu không, các số âm lớn sẽ bị triệt tiêu bằng các số dương lớn và chúng ta có thể thu được một giá trị nhỏ cho một mô hình rất phức tạp.

Nhưng trước khi chúng ta bắt đầu tính toán các định mức, có một vấn đề kỹ thuật nhỏ: độ lệch trong các mô hình không được bao gồm trong định mức $L1$ và $L2$. Tại sao? Chà, độ lệch trong mô hình chính xác là số giây mà chúng tôi mong đợi người dùng sẽ xem phim 11 nếu họ chưa xem bất kỳ phim nào trong số 10 phim trước đó. Con số này không liên quan đến độ phức tạp của mô hình; do đó, chúng tôi để nó một mình. Việc tính toán định mức $L1$ cho mô hình 1 và 2 như sau.

Hãy nhớ lại rằng các phương trình cho các mô hình như sau:

Mẫu 1: $\hat{y} = 2x_3 + 1,4x_7 - 0,5x_7 + 8$

Mẫu 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$

Định mức L1:

- **Mẫu 1:** $|2| + |1.4| + |-0.5|= 3.9$ **Mẫu 2:** $|22| + |-103| + |-14| + |109| + |-93| + |203| + |87| + |-55| + |378| + |-25| = 1,089$

Định mức L2:

- **Mẫu 1:** $2^2 + 1.4^2 + (-0.5)^2 = 6.21$ **Mẫu 2:** $22^2 + (-103)^2 + (-14)^2 + 109^2 + (-93)^2 + 203^2 + 87^2 + (-55)^2 + 378^2 + (-25)^2 = 227,131$

Đúng như dự đoán, cả định mức L1 và L2 của mô hình 2 đều lớn hơn nhiều so với định mức tương ứng của mô hình 1.

Định mức L1 và L2 cũng có thể được tính trên các đa thức bằng cách lấy tổng các giá trị tuyệt đối hoặc tổng bình phương của các hệ số, ngoại trừ hệ số không đổi.Hãy quay lại ví dụ ở đầu chương này, trong đó ba mô hình của chúng ta là đa thức bậc 1 (đường thẳng), bậc 2 (parabol) và bậc 10 (đường cong dao động 9 lần).Hãy tưởng tượng rằng công thức của họ như sau:

- **Mẫu 1:** $\hat{y} = 2x + 3$ **Mẫu 2:** $\hat{y} = -x^2 + 6x - 2$ **Mẫu 3:** $\hat{y} = x^9 + 4x^8 - 9x^7 + 3x^6 - 14x^5 - 2x^4 - 9x^3 + x^2 + 6x + 10$
-

Định mức L1 và L2 được tính như sau:

Định mức L1:

- **Model 1:** $|2| = 2$
- **Model 2:** $|-1| + |6| = 7$
- **Model 3:** $|1| + |4| + |-9| + |3| + |-14| + |-2| + |-9| + |1| + |6| = 49$

Định mức L2:

- **Mẫu 1:** $2^2 = 2$

- **Mẫu 2:** $(-1)^2 + 6^2 = 37$ **Mẫu 3:** $1^2 + 4^2 + (-9)^2 + 3^2 + (-14)^2 + (-2)^2 + (-9)^2 + 1^2$
- $+ 6^2 = 425$

Bây giờ chúng ta đã được trang bị hai cách để đo lường độ phức tạp của mô hình, hãy bắt tay vào quá trình đào tạo.

Sửa đổi hàm lỗi để giải bài toán: Hồi quy Lasso và hồi quy sườn

Bây giờ chúng ta đã thực hiện hầu hết các công việc nặng nhọc, chúng ta sẽ huấn luyện mô hình hồi quy tuyến tính bằng cách sử dụng chính quy hóa. Chúng tôi có hai thước đo cho mô hình của mình: thước đo hiệu suất (hàm lỗi) và thước đo độ phức tạp (chỉ tiêu L1 hoặc L2).

Hãy nhớ lại rằng trong sự tương tự của thợ lợp mái, mục tiêu của chúng tôi là tìm ra thợ lợp mái có chất lượng tốt và độ phức tạp thấp. Chúng tôi đã làm điều này bằng cách giảm thiểu tổng của hai số: thước đo chất lượng và thước đo độ phức tạp. Chính quy hóa bao gồm việc áp dụng nguyên tắc tương tự cho mô hình học máy của chúng tôi. Đối với điều này, chúng ta có hai đại lượng: sai số hồi quy và thuật ngữ chính quy hóa.

Lỗi hồi quy Thước đo chất lượng của mô hình. Trong trường hợp này, nó có thể là sai số tuyệt đối hoặc sai số bình phương mà chúng ta đã học ở chương 3.

thuật ngữ chính quy hóa Thước đo độ phức tạp của mô hình. Nó có thể là chuẩn L1 hoặc L2 của mô hình.

Đại lượng mà chúng ta muốn giảm thiểu để tìm ra một mô hình tốt và không quá phức tạp là sai số đã sửa đổi, được định nghĩa là tổng của cả hai, như minh họa bên dưới:

$$\text{Lỗi} = \text{Lỗi hồi quy} + \text{Thuật ngữ chính quy hóa}$$

Việc chính quy hóa phổ biến đến mức bản thân các mô hình cũng có những tên gọi khác nhau dựa trên tiêu chuẩn nào được sử dụng. Nếu chúng ta huấn luyện mô hình hồi quy bằng cách sử dụng chuẩn L1 thì mô hình đó được gọi là *hồi quy Lasso*. Lasso là viết tắt của “toán tử lựa chọn và co rút tuyệt đối nhỏ nhất”. Hàm lỗi như sau:

$$\text{Lỗi hồi quy Lasso} = \text{Lỗi hồi quy} + \text{Chỉ tiêu L1}$$

Thay vào đó, nếu chúng ta huấn luyện mô hình bằng cách sử dụng chuẩn L2 thì nó được gọi là *hồi quy sườn*. Cái tên *sườn* xuất phát từ hình dạng của hàm lỗi, bởi vì việc thêm thuật ngữ chuẩn L2 vào hàm lỗi hồi quy sẽ biến một góc nhọn thành một góc nhẵn

thung lũng khi chúng tôi vẽ nó. Hàm lỗi như sau:

$$\text{Sai số hồi quy sùron} = \text{Sai số hồi quy} + \text{chuẩn L2}$$

Cả hồi quy Lasso và Ridge đều hoạt động tốt trong thực tế. Quyết định sử dụng cái nào tùy thuộc vào một số tùy chọn mà chúng ta sẽ tìm hiểu trong các phần sắp tới. Nhưng trước khi làm được điều đó, chúng ta cần tìm ra một số chi tiết để đảm bảo các mô hình chính quy của chúng ta hoạt động tốt.

Điều chỉnh mức độ hiệu suất và độ phức tạp trong mô hình của chúng tôi: Tham số chính quy

Bởi vì quá trình đào tạo mô hình liên quan đến việc giảm hàm chi phí càng nhiều càng tốt, về nguyên tắc, một mô hình được đào tạo với sự chính quy hóa phải có hiệu suất cao và độ phức tạp thấp. Tuy nhiên, có một số vấn đề giằng co—cố gắng làm cho mô hình hoạt động tốt hơn có thể khiến mô hình trở nên phức tạp hơn, trong khi cố gắng giảm độ phức tạp của mô hình có thể khiến mô hình hoạt động kém hơn. May mắn thay, hầu hết các kỹ thuật học máy đều đi kèm với các nút bấm (siêu tham số) để nhà khoa học dữ liệu xoay chuyển và xây dựng các mô hình tốt nhất có thể và việc chính quy hóa không phải là một ngoại lệ. Trong phần này, chúng ta xem cách sử dụng siêu tham số để điều chỉnh giữa hiệu suất và độ phức tạp.

Siêu tham số này được gọi là *tham số chính quy hóa* và mục tiêu của nó là xác định xem quy trình đào tạo mô hình nên nhấn mạnh vào hiệu suất hay tính đơn giản. Tham số chính quy hóa được ký hiệu là λ , chữ cái Hy Lạp *lambda*. Chúng tôi nhân số hạng chính quy với λ , thêm nó vào lỗi hồi quy và sử dụng kết quả đó để huấn luyện mô hình của mình. Lỗi mới trở thành như sau:

$$\text{Lỗi} = \text{Lỗi hồi quy} + \lambda \text{ Thuật ngữ chính quy hóa}$$

Việc chọn giá trị 0 cho λ sẽ hủy bỏ thuật ngữ chính quy hóa và do đó chúng ta thu được mô hình hồi quy tương tự như chúng ta đã có trong chương 3. Việc chọn một giá trị lớn cho λ sẽ dẫn đến một mô hình đơn giản, có thể ở mức độ thấp, có thể không phù hợp lắm với tập dữ liệu của chúng ta. Điều quan trọng là chọn một giá trị phù hợp cho λ và đối với điều này, xác thực là một kỹ thuật hữu ích. Người ta thường chọn lũy thừa của 10, chẳng hạn như 10, 1, 0,1, 0,01, nhưng lựa chọn này hơi tùy tiện. Trong số này, chúng tôi chọn mô hình giúp mô hình của chúng tôi hoạt động tốt nhất trong bộ xác thực của chúng tôi.

Ảnh hưởng của việc chính quy hóa L1 và L2 đến các hệ số của mô hình

Trong phần này, chúng ta thấy những khác biệt quan trọng giữa chính quy hóa L1 và L2 và có một số ý tưởng về việc nên sử dụng cái nào trong các tình huống khác nhau. Thoạt nhìn, chúng trông giống nhau, nhưng tác động của chúng đối với các hệ số rất thú vị và tùy thuộc vào loại mô hình mà chúng ta muốn, việc quyết định giữa việc sử dụng chính quy L1 và L2 có thể rất quan trọng.

Hãy quay lại ví dụ đề xuất phim của chúng tôi, trong đó chúng tôi đang xây dựng mô hình hồi quy để dự đoán lượng thời gian (tính bằng giây) mà người dùng sẽ xem một bộ phim, trong khoảng thời gian mà cùng một người dùng đó đã xem 10 bộ phim khác nhau. Hãy tưởng tượng rằng chúng ta đã huấn luyện mô hình và phương trình chúng ta nhận được như sau:

$$\textbf{Model: } \hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$$

Nếu chúng ta thêm chính quy hóa và huấn luyện lại mô hình, chúng ta sẽ có một mô hình đơn giản hơn. Hai thuộc tính sau đây có thể được biểu diễn bằng toán học:

- Nếu chúng tôi sử dụng chính quy hóa L1 (hồi quy Lasso), bạn sẽ có được một mô hình có ít hệ số hơn. Nói cách khác, chính quy hóa L1 biến một số hệ số thành 0. Do đó, chúng ta có thể thu được một phương trình như $\hat{y} = 2x_3 + 1.4x_7 - 0.5x_9 + 8$.
- Nếu chúng ta sử dụng chính quy hóa L2 (hồi quy sùron), chúng ta sẽ thu được một mô hình có hệ số nhỏ hơn. Nói cách khác, chính quy hóa L2 thu nhỏ tất cả các hệ số nhưng hiếm khi biến chúng về 0. Do đó, chúng ta có thể thu được một phương trình như $\hat{y} = 0.2x_1 - 0.8x_2 - 1.1x_3 + 2.4x_4 - 0.03x_5 + 1.02x_6 + 3.1x_7 - 2x_8 + 2.9x_9 - 0.04x_{10} + 8$.

Do đó, tùy thuộc vào loại phương trình mà chúng ta muốn nhận, chúng ta có thể quyết định giữa việc sử dụng chính quy L1 và L2.

Một nguyên tắc nhỏ cần sử dụng khi quyết định xem chúng ta muốn sử dụng chính quy L1 hay L2 như sau: nếu chúng ta có quá nhiều tính năng và muốn loại bỏ hầu hết chúng, thì chính quy L1 là hoàn hảo cho điều đó. Nếu chúng ta chỉ có một vài tính năng và tin rằng tất cả chúng đều phù hợp thì việc chính quy hóa L2 là thứ chúng ta cần vì nó sẽ không loại bỏ các tính năng hữu ích của chúng ta.

Một ví dụ về một vấn đề trong đó chúng tôi có nhiều tính năng và việc chính quy hóa L1 có thể giúp chúng tôi là hệ thống đề xuất phim mà chúng tôi đã nghiên cứu trong phần “Một ví dụ khác về trang bị quá mức: Đề xuất phim”. Trong mô hình này, mỗi đặc điểm tương ứng với một trong các bộ phim và mục tiêu của chúng tôi là tìm một vài bộ phim có liên quan đến bộ phim mà chúng tôi quan tâm. Vì vậy, chúng tôi cần một mô hình mà hầu hết các hệ số đều bằng 0, ngoại trừ một vài trong số chúng.

Một ví dụ mà chúng ta nên sử dụng chính quy hóa L2 là ví dụ về đa thức ở đầu phần “Ví dụ về việc điều chỉnh thiếu bằng cách sử dụng hồi quy đa thức”. Đối với mô hình này, chúng tôi chỉ có một tính năng: x . Việc chính quy hóa L2 sẽ mang lại cho chúng ta một mô hình đa thức tốt với các hệ số nhỏ, không dao động nhiều và do đó ít có xu hướng bị quá khớp. Trong phần “Hồi quy đa thức, kiểm tra và chính quy hóa với Turi Create”, chúng ta sẽ thấy một ví dụ đa thức mà chính quy L2 là ví dụ phù hợp để sử dụng.

Các tài nguyên tương ứng với chương này (phụ lục C) chỉ ra một số chỗ mà bạn có thể tìm hiểu sâu hơn về lý do toán học tại sao chính quy hóa L1 biến các hệ số thành 0, trong khi chính quy hóa L2 biến chúng thành các số nhỏ. Trong phần tiếp theo, chúng ta sẽ học cách có được trực giác về nó.

Một cách trực quan để xem tính chính quy

Trong phần này, chúng ta tìm hiểu các tiêu chuẩn L1 và L2 khác nhau như thế nào trong cách chúng xử lý sự phức tạp. Phần này chủ yếu mang tính trực quan và được phát triển dưới dạng ví dụ, nhưng nếu bạn muốn xem toán học hình thức đằng sau chúng, vui lòng xem phụ lục B, “Sử dụng độ dốc giảm dần để chính quy hóa”.

Khi cố gắng hiểu cách thức hoạt động của mô hình học máy, chúng ta nên nhìn xa hơn chức năng lỗi. Một hàm lỗi cho biết: “Đây là lỗi và nếu bạn giảm bớt nó, bạn sẽ có được một mô hình tốt”. Nhưng điều đó cũng giống như nói: “Bí quyết thành công trong cuộc sống là càng ít mắc sai lầm càng tốt”. Chẳng phải một thông điệp tích cực sẽ tốt hơn, chẳng hạn như “Đây là những điều bạn có thể làm để cải thiện cuộc sống của mình” thay vì “Đây là những điều bạn nên tránh”? Chúng ta hãy xem sự chính quy hóa theo cách này.

Trong chương 3, chúng ta đã học về thủ thuật tuyệt đối và bình phương, giúp chúng ta có cái nhìn rõ ràng hơn về hồi quy. Ở mỗi giai đoạn trong quá trình huấn luyện, chúng ta chỉ cần chọn một điểm (hoặc một vài điểm) và di chuyển đường thẳng đến gần các điểm đó hơn. Lặp lại quá trình này nhiều lần cuối cùng sẽ mang lại một đường nét phù hợp. Chúng ta có thể cụ thể hơn và lặp lại cách chúng ta đã xác định thuật toán hồi quy tuyến tính trong chương 3.

PSEUDOCODE CHO Thuật toán hồi quy tuyến tính

Đầu vào: Tập dữ liệu về điểm

Đầu ra: Mô hình hồi quy tuyến tính phù hợp với tập dữ liệu đó

Thủ tục:

- Chọn một mô hình có trọng số ngẫu nhiên và độ lệch ngẫu nhiên. Lặp lại nhiều lần: Chọn một
- điểm dữ liệu ngẫu nhiên. Điều chỉnh một chút trọng số và độ lệch để cải thiện dự đoán cho điểm dữ liệu đó. Thưởng thức mô hình của bạn!

◦

•

Có thể sử dụng lý do tương tự để hiểu chính quy?Vâng, chúng tôi có thể.

Để đơn giản hóa mọi thứ, giả sử chúng ta đang trong quá trình đào tạo và chúng ta muốn làm cho mô hình trở nên đơn giản hơn.Chúng ta có thể làm điều này bằng cách giảm các hệ số.Để đơn giản, giả sử mô hình của chúng ta có ba hệ số: 3, 10 và 18. Chúng ta có thể thực hiện một bước nhỏ để giảm ba hệ số này xuống một lượng nhỏ không?Tất nhiên là chúng tôi có thể, và đây là hai phương pháp để làm điều đó.Cả hai đều yêu cầu một số nhỏ, λ , hiện tại chúng tôi sẽ đặt thành 0,01.

Phương pháp 1: Trừ λ khỏi mỗi tham số dương và thêm λ vào mỗi tham số âm.Nếu chúng bằng 0, hãy để chúng yên.

Cách 2: Nhân tất cả với $1 - \lambda$.Lưu ý rằng số này gần bằng 1 vì λ nhỏ.

Sử dụng phương pháp 1, chúng ta nhận được các số 2,99, 9,99 và 17,99.

Sử dụng phương pháp 2, chúng ta nhận được các số 2,97, 9,9 và 17,82.

Trong trường hợp này, λ hoạt động rất giống tốc độ học tập.Trên thực tế, nó có liên quan chặt chẽ đến tốc độ chính quy hóa (xem “Sử dụng độ dốc giảm dần để chính quy hóa” trong phụ lục B để biết chi tiết).Lưu ý rằng trong cả hai phương pháp, chúng tôi đều thu nhỏ kích thước của các hệ số.Bây giờ, tất cả những gì chúng ta phải làm là liên tục thu nhỏ các hệ số ở mọi giai đoạn của thuật toán.Nói cách khác, đây là cách chúng tôi đào tạo mô hình bây giờ:

Đầu vào: Tập dữ liệu về điểm

Đầu ra: Mô hình hồi quy tuyến tính phù hợp với tập dữ liệu đó

Thủ tục:

- Chọn một mô hình có trọng số ngẫu nhiên và độ lệch ngẫu nhiên. Lặp lại nhiều lần: Chọn
- một điểm dữ liệu ngẫu nhiên. Điều chỉnh một chút trọng số và độ lệch để cải thiện dự đoán cho điểm dữ liệu cụ thể đó. **Thu nhỏ một chút các hệ số bằng phương pháp 1 hoặc phương pháp 2.**

○

- Thường thức mô hình của bạn!

Nếu chúng tôi sử dụng phương pháp 1, chúng tôi đang huấn luyện mô hình với chính quy hóa L1 hoặc hồi quy lasso. Nếu chúng tôi sử dụng phương pháp 2, chúng tôi đang huấn luyện nó bằng chính quy hóa L2 hoặc hồi quy sườn. Có một cơ sở toán học biện minh cho điều này, được mô tả trong phụ lục B, “Sử dụng phương pháp giảm độ dốc để chính quy hóa”.

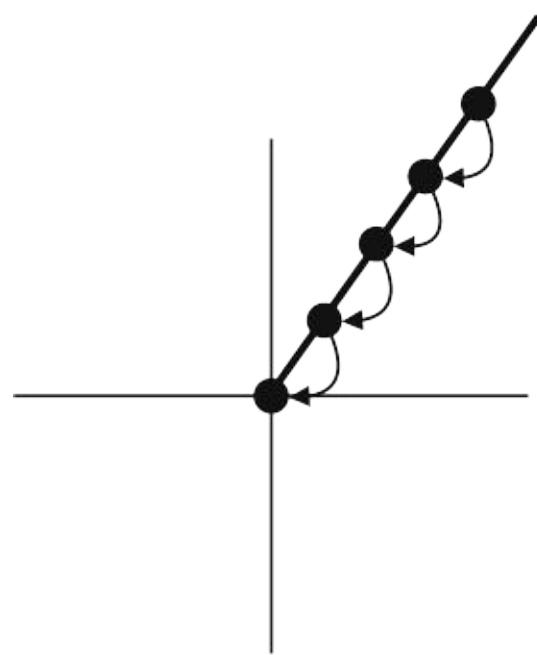
Trong phần trước, chúng ta đã biết rằng chính quy hóa L1 có xu hướng biến nhiều hệ số thành 0, trong khi chính quy hóa L2 có xu hướng giảm chúng nhưng không biến chúng thành 0. Hiện tượng này bây giờ đã dễ nhận thấy hơn. Giả sử hệ số của chúng tôi là 2, với tham số chính quy là $\lambda = 0,01$. Hãy lưu ý điều gì sẽ xảy ra nếu chúng ta sử dụng phương pháp 1 để thu nhỏ hệ số và lặp lại quá trình này 200 lần. Chúng ta nhận được chuỗi giá trị sau:

$$2 \rightarrow 1.99 \rightarrow 1.98 \rightarrow \dots \rightarrow 0.02 \rightarrow 0.01 \rightarrow 0$$

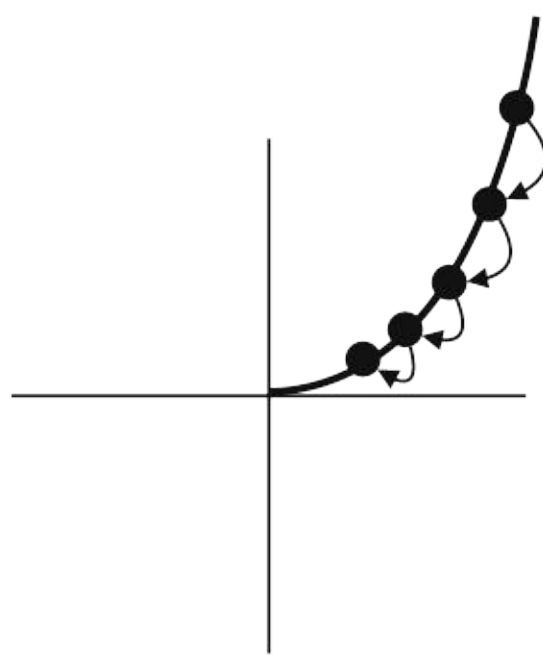
Sau 200 kỷ nguyên đào tạo của chúng tôi, hệ số trở thành 0 và nó không bao giờ thay đổi nữa. Bây giờ hãy xem điều gì sẽ xảy ra nếu chúng ta áp dụng lại phương pháp 2, 200 lần và với cùng tốc độ học là $\eta = 0,01$. Chúng ta nhận được chuỗi giá trị sau:

$$2 \rightarrow 1.98 \rightarrow 1.9602 \rightarrow \dots \rightarrow 0.2734 \rightarrow 0.2707 \rightarrow 0.2680$$

Lưu ý rằng hệ số giảm đáng kể nhưng không trở thành 0. Trên thực tế, cho dù chúng ta có chạy bao nhiêu kỷ nguyên thì hệ số cũng sẽ không bao giờ bằng 0. Điều này là do khi chúng ta nhân một số không âm với 0,99 nhiều lần thì số đó sẽ không bao giờ bằng 0. Điều này được minh họa trong hình 4.7.



L1 regularization



L2 regularization

Hình 4.7 Cả L1 và L2 đều thu nhỏ kích thước của hệ số. Chính quy hóa L1 (trái) thực hiện việc này nhanh hơn nhiều vì nó trừ đi một lượng cố định, do đó cuối cùng nó có khả năng trở thành số 0. Việc chính quy hóa L2 mất nhiều thời gian hơn vì nó nhân hệ số với một hệ số nhỏ, do đó nó không bao giờ đạt đến 0.

Hồi quy đa thức, kiểm tra và chuẩn hóa bằng Turi Create

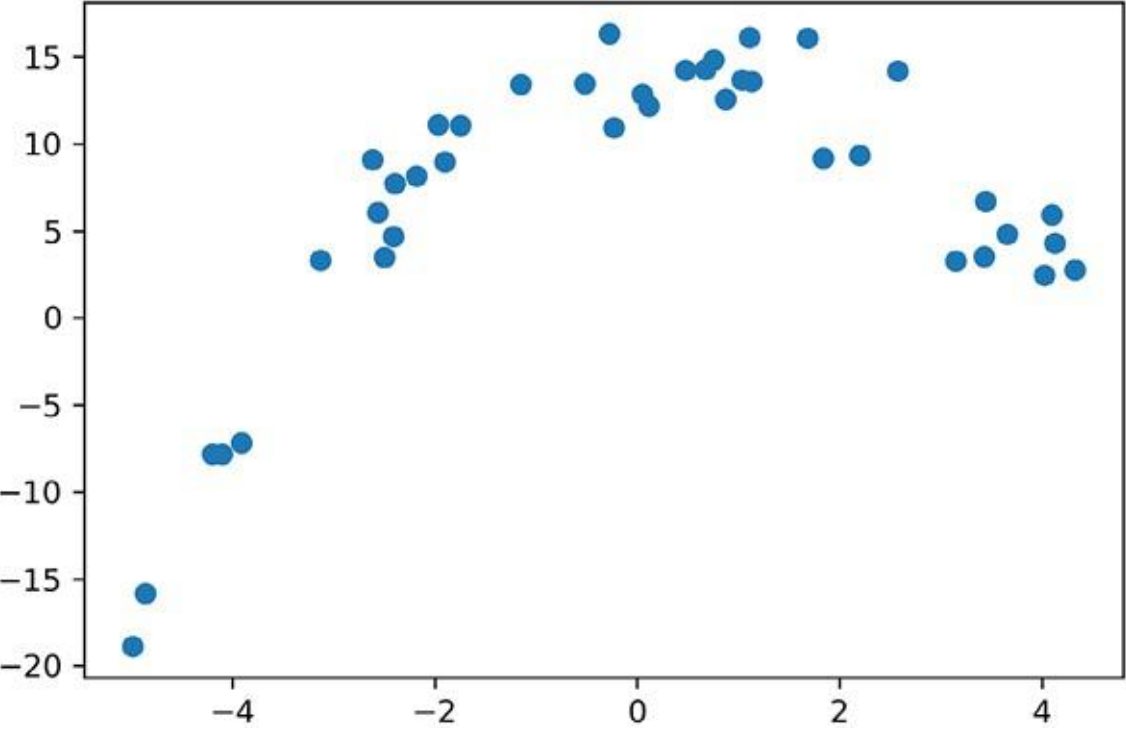
Trong phần này, chúng ta thấy một ví dụ về hồi quy đa thức với tính năng chính quy hóa trong Turi Create. Đây là mã cho phần này:

- **Sổ tay:**

- https://github.com/luisguiserrano/manning/blob/master/Chapter_4_Testing_Overfitting_Underf

Chúng tôi bắt đầu với tập dữ liệu của mình, được minh họa trong hình 4.8. Chúng ta có thể thấy đường cong phù hợp nhất với dữ liệu này là một parabol mở xuống (khuôn mặt buồn). Do đó, đây không phải là vấn đề chúng ta có thể giải quyết bằng hồi quy tuyến tính - chúng ta phải sử dụng hồi quy đa thức. Tập dữ liệu được lưu trữ trong SFrame có tên `data`, và một vài hàng đầu tiên

được thể hiện ở bảng 4.1.



Hình 4.8 Bộ dữ liệu.Lưu ý rằng hình dạng của nó là một parabol mở xuống dưới, do đó sử dụng hồi quy tuyến tính sẽ không hiệu quả.Chúng tôi sẽ sử dụng hồi quy đa thức để phù hợp với tập dữ liệu này và chúng tôi sẽ sử dụng chính quy hóa để điều chỉnh mô hình của mình.

Bảng 4.1 Bốn hàng đầu tiên của tập dữ liệu của chúng tôi

x	y
3.4442185152504816	6.685961311021467
-2.4108324970703663	4.690236225597948
0.11274721368608542	12.205789026637378
-1.9668727392107255	11.133217991032268

Cách thực hiện hồi quy đa thức trong Turi Create là thêm nhiều cột vào tập dữ liệu của chúng tôi, tương ứng với các lũy thừa của tính năng chính và áp dụng hồi quy tuyến tính cho tập dữ liệu mở rộng này.Ví dụ: nếu đặc điểm chính là x thì chúng tôi sẽ thêm các cột có giá trị x^2 , x^3 , x^4 , v.v.Do đó, mô hình của chúng tôi đang tìm kiếm các tổ hợp tuyến tính của lũy thừa của x, chính là các đa thức trong x.Nếu SFrame chứa dữ liệu của chúng tôi được gọi, chúng tôi sử dụng đoạn mã sau để thêm

data

các cột có lũy thừa lên tới x^{199} .Một số hàng và cột đầu tiên của tập dữ liệu kết quả xuất hiện trong bảng 4.2.

```
for i in range(2,200):
    string = 'x^'+str(i)
    data[string] = data['x'].apply(lambda x:x**i)
```

Bảng 4.2 Bốn hàng trên cùng và năm cột ngoài cùng bên trái của tập dữ liệu của chúng tôi. Cột có nhãn x^k tương ứng với biến x , cho $k = 2, 3$ và 4 . Tập dữ liệu có 200 cột.

x	y	x^2	x^3	x^4
3.445	6.686	11.863	40.858	140.722
-2.411	4.690	5.812	-14.012	33.781
0.113	12.206	0.013	0.001	0.000
-1.967	11.133	3.869	-7.609	14.966

Bây giờ, chúng tôi áp dụng hồi quy tuyến tính cho tập dữ liệu lớn với 200 cột này. Lưu ý rằng mô hình hồi quy tuyến tính trong tập dữ liệu này trông giống như sự kết hợp tuyến tính của các biến trong các cột. Nhưng vì mỗi cột tương ứng với một đơn thức nên mô hình thu được trông giống như một đa thức trên biến x .

Trước khi đào tạo bất kỳ mô hình nào, chúng tôi cần chia dữ liệu thành các tập dữ liệu huấn luyện và kiểm tra, sử dụng dòng mã sau:

```
train, test = data.random_split(.8)
```

Bây giờ tập dữ liệu của chúng tôi được chia thành hai tập dữ liệu, tập huấn luyện có tên *train* và tập kiểm tra có tên *test*. Trong kho lưu trữ, một hạt giống ngẫu nhiên được chỉ định nên chúng tôi luôn nhận được kết quả giống nhau, mặc dù điều này không cần thiết trong thực tế.

Cách sử dụng tính chính quy trong Turi Create rất đơn giản: tất cả những gì chúng ta cần làm là chỉ định các tham số và trong phương thức khi huấn luyện `create` mô hình. Hình phạt này chính xác là tham số chính quy hóa mà chúng tôi đã giới thiệu trong phần “Điều chỉnh mức độ hiệu suất và độ phức tạp trong mô hình của chúng tôi”. Hình phạt bằng 0 có nghĩa là chúng tôi không sử dụng chính quy. Vì vậy, chúng ta sẽ huấn luyện ba mô hình khác nhau với các tham số sau:

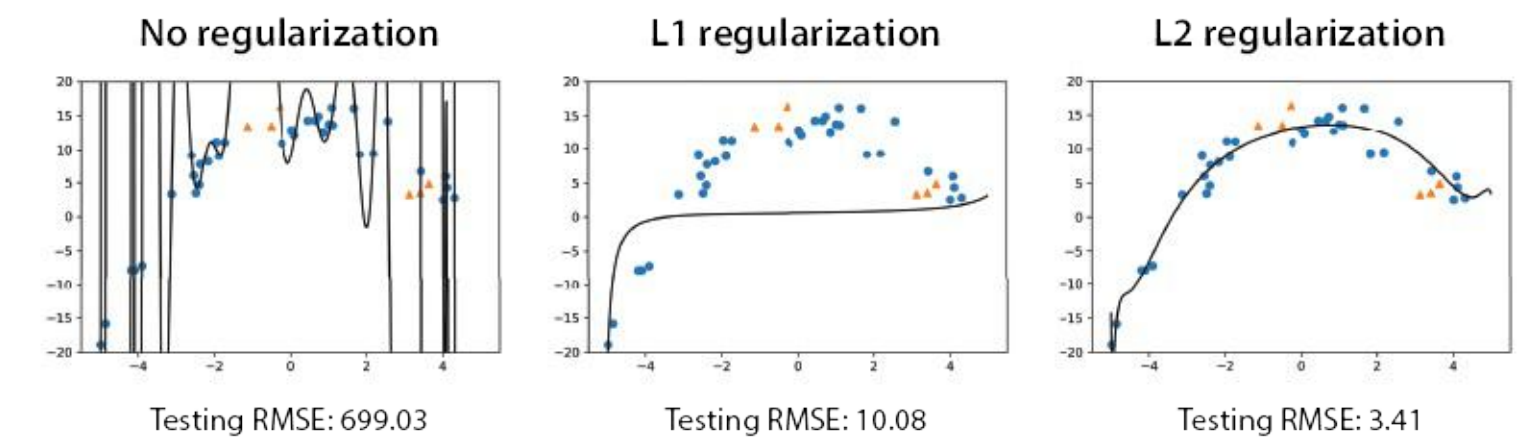
- Không có mô hình chính quy:
 - `l1_penalty=0`
 - `l2_penalty=0`
- Mô hình chính quy hóa L1:
 - `l1_penalty=0.1`
 - `l2_penalty=0`

- Mô hình chính quy hóa L2:
 - `l1_penalty=0`
 - `l2_penalty=0.1`

Chúng tôi đào tạo các mô hình với ba dòng mã sau:

```
model_no_reg = tc.linear_regression.create(train, target='y', l1_penalty=0.0, l2_penalty=0.0)
model_L1_reg = tc.linear_regression.create(train, target='y', l1_penalty=0.1, l2_penalty=0.0)
model_L2_reg = tc.linear_regression.create(train, target='y', l1_penalty=0.0, l2_penalty=0.1)
```

Mô hình đầu tiên không sử dụng chính quy hóa, mô hình thứ hai sử dụng chính quy L1 với tham số 0,1 và mô hình thứ ba sử dụng chính quy L2 với tham số 0,1. Đồ thị của các hàm kết quả được thể hiện trong hình 4.9. Lưu ý rằng trong hình này, các điểm trong tập huấn luyện là các hình tròn và các điểm trong tập kiểm tra là các hình tam giác.



Hình 4.9 Ba mô hình hồi quy đa thức cho tập dữ liệu của chúng tôi. Mô hình bên trái không có chính quy hóa, mô hình ở giữa có chính quy L1 với tham số 0,1 và mô hình bên phải có chính quy L2 với tham số 0,1.

Lưu ý rằng mô hình không có sự chính quy hóa rất phù hợp với các điểm đào tạo, nhưng nó hỗn loạn và không phù hợp lắm với các điểm kiểm tra. Mô hình có chuẩn hóa L1 hoạt động tốt với cả tập huấn luyện và tập kiểm tra. Nhưng mô hình có tính chính quy L2 thực hiện công việc tuyệt vời với cả tập huấn luyện và tập kiểm tra và dường như cũng là mô hình thực sự nắm bắt được hình dạng của dữ liệu.

Cũng lưu ý rằng đối với ba mô hình, đường cong ranh giới hơi khác nhau ở các điểm cuối. Điều này hoàn toàn dễ hiểu, vì các điểm cuối có ít dữ liệu hơn và việc mô hình không biết phải làm gì khi không có dữ liệu là điều đương nhiên. Chúng ta phải luôn đánh giá các mô hình bằng cách chúng hoạt động tốt như thế nào trong các ranh giới

tập dữ liệu của chúng tôi và chúng tôi không bao giờ nên mong đợi một mô hình hoạt động tốt bên ngoài các ranh giới đó.Ngay cả con người chúng ta cũng có thể không đưa ra được những dự đoán tốt bên ngoài ranh giới của mô hình.Ví dụ: bạn nghĩ đường cong này sẽ trông như thế nào bên ngoài tập dữ liệu?Liệu nó có tiếp tục như một parabol mở xuống dưới không? Liệu nó có dao động mãi mãi giống như hàm sin không?Nếu chúng ta không biết điều này thì chúng ta không nên mong đợi mô hình biết điều đó.Vì vậy, hãy cố gắng bỏ qua hành vi lạ ở các điểm cuối trong hình 4.9 và tập trung vào hành vi của mô hình bên trong khoảng nơi dữ liệu được đặt.

Để tìm ra lỗi kiểm tra, chúng tôi sử dụng dòng mã sau, với tên tương ứng của mô hình.Dòng mã này trả về sai số tối đa và sai số bình phương trung bình gốc (RMSE).

```
model.predict(test)
```

RMSE thử nghiệm cho các mô hình như sau:

- Mô hình không chính quy: 699.03
- có L1 chính quy hóa: Mô hình
- 10.08 với L2 chính quy hóa: 3,41

Mô hình không được chính quy hóa có RMSE thực sự lớn!Trong số hai mô hình còn lại, mô hình có chính quy L2 hoạt động tốt hơn nhiều.Dưới đây là hai câu hỏi để suy nghĩ:

1. Tại sao mô hình có chính quy L2 hoạt động tốt hơn mô hình có chính quy L1? 2. Tại sao mô hình có chính quy L1 trông phẳng, trong khi mô hình có chính quy L2 lại có thể nắm bắt được hình dạng của dữ liệu?

Hai câu hỏi có câu trả lời giống nhau và để tìm ra câu trả lời đó, chúng ta có thể xem xét hệ số của các đa thức.Chúng có thể được lấy bằng dòng mã sau:

```
model.coefficients
```

Mỗi đa thức có 200 hệ số, vì vậy chúng tôi sẽ không hiển thị tất cả chúng ở đây, nhưng trong bảng 4.3, bạn có thể thấy năm hệ số đầu tiên của ba mô hình.Bạn nhận thấy điều gì?

Bảng 4.3 Năm hệ số đầu tiên của đa thức trong ba đa thức của chúng ta

mô hình. Lưu ý rằng mô hình không chính quy hóa có hệ số lớn, mô hình chính quy hóa L1 có hệ số rất gần 0 và mô hình có chính quy hóa L2 có hệ số nhỏ.

Hệ số	model_no_reg	model_L1_reg	model_L2_reg
$x^0 = 1$	8.41	0.57	13.24
x^1	15.87	0.07	0.87
x^2	108.87	-0.004	-0.52
x^3	-212.89	0.0002	0.006
x^4	-97.13	-0.0002	-0.02

Để diễn giải bảng 4.3, chúng ta thấy các dự đoán cho ba mô hình đều là đa thức bậc 200. Các số hạng đầu tiên trông như sau:

- Mô hình không chính quy: $\hat{y} = 8,41 + 15,87x + 108,87x^2 - 212,89x^3 - 97,13x^4 + \dots$ Mô hình có chính quy L1: $\hat{y} = 0,57 + 0,07x - 0,004x^2 + 0,0002x^3 - 0,0002x^4$
- $+ \dots$ Mô hình có chính quy hóa L2: $\hat{y} = 13,24 + 0,87x - 0,52x^2 + 0,006x^3 - 0,02x^4 + \dots$
-

Từ các đa thức này, ta thấy:

- Đối với mô hình không có sự chính quy hóa, tất cả các hệ số đều lớn.Điều này có nghĩa là đa thức hỗn loạn và không tốt cho việc đưa ra dự đoán. Đối với mô hình có chính quy hóa L1, tất cả các hệ số, ngoại trừ hằng số (hệ số đầu tiên), đều rất nhỏ—gần như bằng 0. Điều này có nghĩa là đối với các giá trị gần bằng 0, đa thức trông rất giống đường ngang có phương trình $\hat{y} = 0,57$.Mô hình này tốt hơn mô hình trước nhưng vẫn không tốt cho việc đưa ra dự đoán. Đối với mô hình có chính quy hóa L2, các hệ số sẽ nhỏ hơn khi mức độ tăng lên nhưng vẫn không quá nhỏ.Điều này cho chúng ta một đa thức phù hợp để đưa ra dự đoán.
-

Tóm tắt

- Khi nói đến mô hình đào tạo, có rất nhiều vấn đề nảy sinh.Hai vấn đề thường xuyên xảy ra là trang bị thiếu và trang bị quá mức.

- Underfitting xảy ra khi chúng ta sử dụng một mô hình rất đơn giản để phù hợp với tập dữ liệu của mình. Quá khớp xảy ra khi chúng ta sử dụng một mô hình quá phức tạp để phù hợp với tập dữ liệu của mình. Một cách hiệu quả để phân biệt giữa trạng bị thừa và trạng bị thiếu là sử dụng tập dữ liệu thử nghiệm. Để kiểm tra một mô hình, chúng tôi chia dữ liệu thành hai tập: tập huấn luyện và tập kiểm tra

- Tập huấn luyện được sử dụng để huấn luyện mô hình và tập kiểm tra được sử dụng để đánh giá mô hình. Nguyên tắc vàng của học máy là không bao giờ sử dụng dữ liệu thử nghiệm của chúng tôi để đào tạo hoặc đưa ra quyết định trong các mô hình của chúng tôi. Bộ xác thực là một phần khác của tập dữ liệu mà chúng tôi sử dụng để đưa ra quyết định về các siêu tham số trong mô hình của mình. Một mô hình không phù hợp sẽ hoạt động kém trong tập huấn luyện và tập xác thực. Một mô hình quá phù hợp sẽ hoạt động tốt trong tập huấn luyện nhưng kém trong tập xác thực. Một mô hình tốt sẽ hoạt động tốt trên cả tập huấn luyện và tập xác nhận. Biểu đồ độ phức tạp của mô hình được sử dụng để xác định độ phức tạp chính xác của một mô hình, sao cho nó không phù hợp quá mức hoặc quá mức. Chính quy hóa là một kỹ thuật rất quan trọng để giảm tình trạng trạng bị quá mức trong các mô hình học máy. Nó bao gồm việc thêm thước đo độ phức tạp (thuật ngữ chính quy hóa) vào hàm lỗi trong quá trình đào tạo. Các chỉ tiêu L1 và L2 là hai thước đo độ phức tạp phổ biến nhất được sử dụng trong chính quy hóa. Việc sử dụng định mức L1 dẫn đến chính quy hóa L1 hoặc hồi quy lasso. Việc sử dụng định mức L2 dẫn đến việc chính quy hóa L2 hoặc hồi quy sườn. Nên chính quy hóa L1 khi tập dữ liệu của chúng tôi có nhiều tính năng và chúng tôi muốn biến nhiều tính năng trong số đó thành 0. Nên chuẩn hóa L2 khi tập dữ liệu của chúng tôi có ít tính năng và chúng tôi muốn làm cho chúng nhỏ đi nhưng không phải bằng không.

-
-
-

Bài tập

BÀI TẬP 4.1

Chúng tôi đã đào tạo bốn mô hình trong cùng một tập dữ liệu với các siêu tham số khác nhau. Trong bảng sau, chúng tôi đã ghi lại các lỗi huấn luyện và kiểm tra cho từng mô hình.

Model	Training error	Testing error
1	0.1	1.8
2	0.4	1.2
3	0.6	0.8
4	1.9	2.3

- Which model would you select for this dataset?
- Which model looks like it’s underfitting the data?
- Which model looks like it’s overfitting the data?

EXERCISE 4.2

We are given the following dataset:

x	y
1	2
2	2.5
3	6
4	14.5
5	34

We train the polynomial regression model that predicts the value of y as $\hat{y}$, where

$$\hat{y} = 2x^2 - 5x + 4.$$

If the regularization parameter is $\lambda = 0.1$ and the error function we’ve used to train this dataset is the mean absolute value (MAE), determine the following:

- The lasso regression error of our model (using the L1 norm)
- The ridge regression error of our model (using the L2 norm)

5 Dừng đường kẻ để chia điểm: Thuật toán perceptron

Trong chương này

- phân tích cảm xúc phân loại là gì: làm thế nào để biết một câu là vui hay buồn bằng cách sử dụng máy học cách vẽ một đường phân cách các điểm có hai màu perceptron là gì và làm cách nào để huấn luyện nó mã hóa thuật toán perceptron trong Python và Turi Tạo

-
-
-



Trong chương này, chúng ta tìm hiểu một nhánh của máy học được gọi là *phân loại*. Các mô hình phân loại tương tự như các mô hình hồi quy, trong đó mục đích của chúng là dự đoán nhãn của tập dữ liệu dựa trên các đặc điểm. Sự khác biệt là các mô hình hồi quy nhằm mục đích dự đoán một con số, trong khi các mô hình phân loại nhằm mục đích dự đoán một trạng thái hoặc một danh mục. Các mô hình phân loại thường được gọi là *bộ phân loại* và chúng tôi sẽ sử dụng các thuật ngữ này thay thế cho nhau. Nhiều bộ phân loại dự đoán một trong hai trạng thái có thể (thường là có/không), mặc dù có thể xây dựng các bộ phân loại dự đoán số lượng trạng thái có thể cao hơn. Sau đây là những ví dụ phổ biến về phân loại:

- Mô hình đề xuất dự đoán liệu người dùng có xem một bộ phim nhất định hay không. Mô hình email dự đoán liệu email đó có phải là thư rác hay không

-

- Mô hình y tế dự đoán liệu bệnh nhân ốm hay khỏe mạnh Mô hình nhận dạng hình ảnh dự
- đoán liệu hình ảnh có chứa ô tô, chim, mèo hay chó Mô hình nhận dạng giọng nói dự đoán liệu người dùng có nói một lệnh cụ thể hay không
-

Phân loại là một lĩnh vực phổ biến trong học máy và phần lớn các chương trong cuốn sách này (chương 5, 6, 8, 9, 10, 11 và 12) nói về các mô hình phân loại khác nhau. Trong chương này, chúng ta tìm hiểu mô hình *perceptron*, còn được gọi là *bộ phân loại perceptron* hoặc đơn giản là *perceptron*. Perceptron tương tự như mô hình hồi quy tuyến tính, trong đó nó sử dụng sự kết hợp tuyến tính của các đặc điểm để đưa ra dự đoán và là khối xây dựng của mạng lưới thần kinh (chúng ta sẽ tìm hiểu trong chương 10).

Hơn nữa, quá trình huấn luyện một perceptron tương tự như quá trình huấn luyện mô hình hồi quy tuyến tính. Giống như chúng ta đã làm trong chương 3 với thuật toán hồi quy tuyến tính, chúng ta phát triển thuật toán perceptron theo hai cách: sử dụng một thủ thuật mà chúng ta có thể lặp lại nhiều lần và xác định hàm lỗi mà chúng ta có thể giảm thiểu bằng cách sử dụng phương pháp giảm độ dốc.

Ví dụ chính về mô hình phân loại mà chúng ta học trong chương này là *phân tích cảm tính*. Trong phân tích cảm xúc, mục tiêu của mô hình là dự đoán cảm xúc của một câu. Nói cách khác, mô hình dự đoán câu đó vui hay buồn. Ví dụ: một mô hình phân tích tình cảm tốt có thể dự đoán rằng câu “Tôi cảm thấy tuyệt vời!” là một câu vui, và đó là câu “Thật là một ngày khủng khiếp!” là một câu buồn.

tình cảm phân tích được sử dụng trong nhiều ứng dụng thực tế, chẳng hạn như sau :

- Khi một công ty phân tích các cuộc trò chuyện giữa khách hàng và bộ phận hỗ trợ kỹ thuật, để đánh giá chất lượng của cuộc trò chuyện
- Khi phân tích sắc thái hiện diện kỹ thuật số của một thương hiệu, chẳng hạn như các bình luận trên mạng xã hội hoặc các bài đánh giá liên quan đến sản phẩm của công ty đó
- Khi một nền tảng xã hội như Twitter phân tích tâm trạng chung của một nhóm người nhất định sau một sự kiện
- Khi một nhà đầu tư sử dụng tình cảm của công chúng đối với một công ty để dự đoán giá cổ phiếu của công ty đó
-

Làm thế nào chúng ta có thể xây dựng một bộ phân loại phân tích tình cảm? Nói cách khác, làm thế nào chúng ta có thể xây dựng một mô hình học máy lấy một câu làm đầu vào và làm đầu ra cho chúng ta biết câu đó vui hay buồn. Tất nhiên, mô hình này có thể mắc lỗi, nhưng ý tưởng là xây dựng nó theo cách sao cho ít mắc lỗi nhất có thể.

khả thi. Hãy đặt cuốn sách xuống trong vài phút và nghĩ xem chúng ta sẽ xây dựng loại mô hình này như thế nào.

Đây là một ý tưởng. Các câu vui có xu hướng chứa các từ vui, chẳng hạn như *tuyệt vời*, *hạnh phúc* hoặc *niềm vui*, trong khi các câu buồn có xu hướng chứa các từ buồn, chẳng hạn như *khủng khiếp*, *buồn* hoặc *tuyệt vọng*. Trình phân loại có thể bao gồm điểm “hạnh phúc” cho mỗi từ trong từ điển. Những từ vui có thể được cho điểm tích cực và những từ buồn có thể được cho điểm tiêu cực. Những từ trung tính như *the* có thể bị cho điểm 0. Khi chúng tôi đưa một câu vào bộ phân loại, bộ phân loại chỉ cần cộng điểm của tất cả các từ trong câu. Nếu kết quả là dương thì bộ phân loại kết luận rằng câu đó là tốt. Nếu kết quả là âm thì bộ phân loại kết luận rằng câu đó buồn. Mục tiêu bây giờ là tìm điểm cho tất cả các từ trong từ điển. Đối với điều này, chúng tôi sử dụng máy học.

Loại mô hình chúng tôi vừa xây dựng được gọi là *mô hình nhận thức*. Trong chương này, chúng ta tìm hiểu định nghĩa chính thức của một perceptron và cách huấn luyện nó bằng cách tìm ra điểm số hoàn hảo cho tất cả các từ để bộ phân loại của chúng ta mắc ít lỗi nhất có thể.

Quá trình huấn luyện một perceptron được gọi là *thuật toán perceptron* và nó không khác mấy so với thuật toán hồi quy tuyến tính mà chúng ta đã học ở chương 3. Đây là ý tưởng về thuật toán perceptron: Để huấn luyện mô hình, trước tiên chúng ta cần một tập dữ liệu chứa nhiều câu cùng với nhãn của chúng (vui/buồn). Chúng tôi bắt đầu xây dựng trình phân loại của mình bằng cách gán điểm ngẫu nhiên cho tất cả các từ. Sau đó, chúng tôi xem lại tất cả các câu trong tập dữ liệu của mình nhiều lần. Đối với mỗi câu, chúng tôi điều chỉnh điểm số một chút để bộ phân loại cải thiện khả năng dự đoán cho câu đó. Làm cách nào để chúng tôi điều chỉnh điểm số? Chúng tôi thực hiện điều đó bằng cách sử dụng một thủ thuật gọi là *thủ thuật nhận thức* mà chúng tôi đã học trong phần “Thủ thuật nhận thức”. Một cách tương đương để huấn luyện các mô hình perceptron là sử dụng hàm lỗi, giống như chúng ta đã làm trong chương 3. Sau đó, chúng ta sử dụng phương pháp giảm độ dốc để giảm thiểu hàm này.

Tuy nhiên, ngôn ngữ rất phức tạp - nó có nhiều sắc thái, hàm ý kép và sự mỉa mai. Chẳng phải chúng ta sẽ mất quá nhiều thông tin nếu giảm một từ thành một điểm đơn giản sao? Câu trả lời là có—chúng tôi thực sự mất rất nhiều thông tin và chúng tôi sẽ không thể tạo bộ phân loại hoàn hảo theo cách này. Tin vui là khi sử dụng phương pháp này, chúng ta vẫn có thể tạo một bộ phân loại chính xác *hầu hết*. Đây là bằng chứng cho thấy phương pháp chúng tôi đang sử dụng không phải lúc nào cũng đúng. Câu “Tôi không buồn, tôi vui” và “Tôi không vui, tôi buồn” có cùng từ ngữ nhưng ý nghĩa hoàn toàn khác nhau. Do đó, bất kể chúng ta cho các từ đó điểm bao nhiêu, hai câu này sẽ đạt được cùng một số điểm và do đó, bộ phân loại sẽ trả về cùng một dự đoán cho chúng. Chúng có các nhãn khác nhau nên bộ phân loại phải

đã phạm sai lầm với một trong số họ.

Giải pháp cho vấn đề này là xây dựng một bộ phân loại có tính đến thứ tự của các từ hoặc thậm chí những thứ khác như dấu câu hoặc thành ngữ. Một số mô hình như *mô hình Markov ẩn* (HMM), *mạng thần kinh tái phát* (RNN) hoặc *mạng bộ nhớ ngắn hạn dài* (LSTM) đã đạt được thành công lớn với dữ liệu tuần tự, nhưng chúng tôi sẽ không đưa chúng vào cuốn sách này. Tuy nhiên, nếu bạn muốn khám phá những mô hình này, trong phụ lục C bạn có thể tìm thấy một số tài liệu tham khảo rất hữu ích cho việc đó.

Bạn có thể tìm thấy tất cả mã cho chương này trong kho GitHub sau:

https://github.com/luisguiserrano/manning/tree/master/Chapter_5_Perceptron_Algor

Vấn đề: Chúng ta đang ở trên một hành tinh xa lạ và chúng ta không biết ngôn ngữ của họ!

Hãy tưởng tượng kịch bản sau: chúng ta là những phi hành gia và vừa hạ cánh xuống một hành tinh xa xôi, nơi sinh sống của một chủng tộc người ngoài hành tinh vô danh. Chúng tôi rất muốn có thể giao tiếp với người ngoài hành tinh, nhưng họ nói một ngôn ngữ lạ mà chúng tôi không hiểu. Chúng tôi nhận thấy người ngoài hành tinh có hai tâm trạng vui và buồn. Bước đầu tiên của chúng ta khi giao tiếp với họ là tìm hiểu xem họ vui hay buồn dựa trên những gì họ nói. Nói cách khác, chúng tôi muốn xây dựng một bộ phân loại phân tích tình cảm.

Chúng tôi kết bạn được với bốn người ngoài hành tinh và bắt đầu quan sát tâm trạng của họ cũng như ghiên cứu những gì họ nói. Chúng ta quan sát thấy hai người trong số họ vui và hai người buồn. Họ cũng cứ lặp đi lặp lại cùng một câu. Ngôn ngữ của họ dường như chỉ có hai từ: *aack* và *beep*. Chúng tôi tạo thành tập dữ liệu sau với câu họ nói và tâm trạng của họ:

Tập dữ liệu:

- Alien 1 Mood: Câu vui: “*Aack, aack, aaack!*” Alien 2: Mood: Câu buồn: “*Bíp bíp!*”
- -
 -

- Alien 3: Tâm trạng: Câu vui: “*Aack beep aack!*”
Alien 4: Tâm trạng: Buồn Câu: “*Aack beep beep beep!*”
- -
 -

Đột nhiên, người ngoài hành tinh thứ năm bước vào và nói: “*Aack beep aack!*” Chúng ta thực sự không thể nói được tâm trạng của người ngoài hành tinh này. Từ những gì chúng ta biết, chúng ta nên dự đoán tâm trạng của người ngoài hành tinh như thế nào (hình 5.1)?

Chúng ta dự đoán rằng người ngoài hành tinh này hạnh phúc vì dù chúng ta không biết ngôn ngữ nhưng từ *aack* dường như xuất hiện nhiều hơn trong các câu vui, trong khi từ *bíp* dường như xuất hiện nhiều hơn trong các câu buồn. Có lẽ *aack* có nghĩa là điều gì đó tích cực, chẳng hạn như “niềm vui” hay “hạnh phúc”, trong khi *bíp* có thể mang nghĩa điều gì đó buồn, chẳng hạn như “tuyệt vọng” hoặc “nỗi buồn”.

Data	Prediction
 <i>Aack aack aack!</i>	Is this alien happy or sad?
 <i>Beep beep!</i>	
 <i>Aack beep aack!</i>	<i>Aack beep aack aack!</i>
 <i>Aack beep beep beep!</i>	

Hình 5.1 Tập dữ liệu của chúng tôi về người ngoài hành tinh. Chúng tôi đã ghi lại tâm trạng của họ (vui hay buồn) và câu nói họ cứ lặp đi lặp lại. Bây giờ người ngoài hành tinh thứ năm bước vào và nói một câu khác. Chúng ta dự đoán người ngoài hành tinh này vui hay buồn?

Quan sát này làm phát sinh phân loại phân tích tình cảm đầu tiên của chúng tôi. Trình phân loại này đưa ra dự đoán theo cách sau: nó đếm số lần xuất hiện của các từ *aack* và *beep*. Nếu số lần xuất hiện của *aack* lớn hơn số lần xuất hiện của *beep* thì bộ phân loại sẽ dự đoán rằng câu đó là tốt. Nếu nhỏ hơn thì

bộ phân loại dự đoán rằng câu đó buồn.Điều gì xảy ra khi cả hai từ xuất hiện cùng số lần?Chúng ta không có cơ sở để nói nên cứ mặc định dự đoán là câu vui.Trong thực tế, những trường hợp đặc biệt này không xảy ra thường xuyên nên chúng sẽ không gây ra vấn đề lớn cho chúng tôi.

Bộ phân loại chúng ta vừa xây dựng là perceptron (còn gọi là bộ phân loại tuyến tính).Chúng ta có thể viết nó dưới dạng điểm số hoặc trọng số theo cách sau:

PHÂN LOẠI PHÂN TÍCH TÌNH CẢM

Cho một câu, gán điểm sau cho các từ:

Điểm:

- *Aack*: 1 điểm
- *Bíp*: -1 điểm

Quy tắc:

Tính điểm của câu bằng cách cộng điểm của tất cả các từ trên đó như sau:

- Nếu điểm là dương hoặc bằng 0 thì dự đoán câu đó là vui. Nếu điểm âm, dự đoán câu đó buồn.

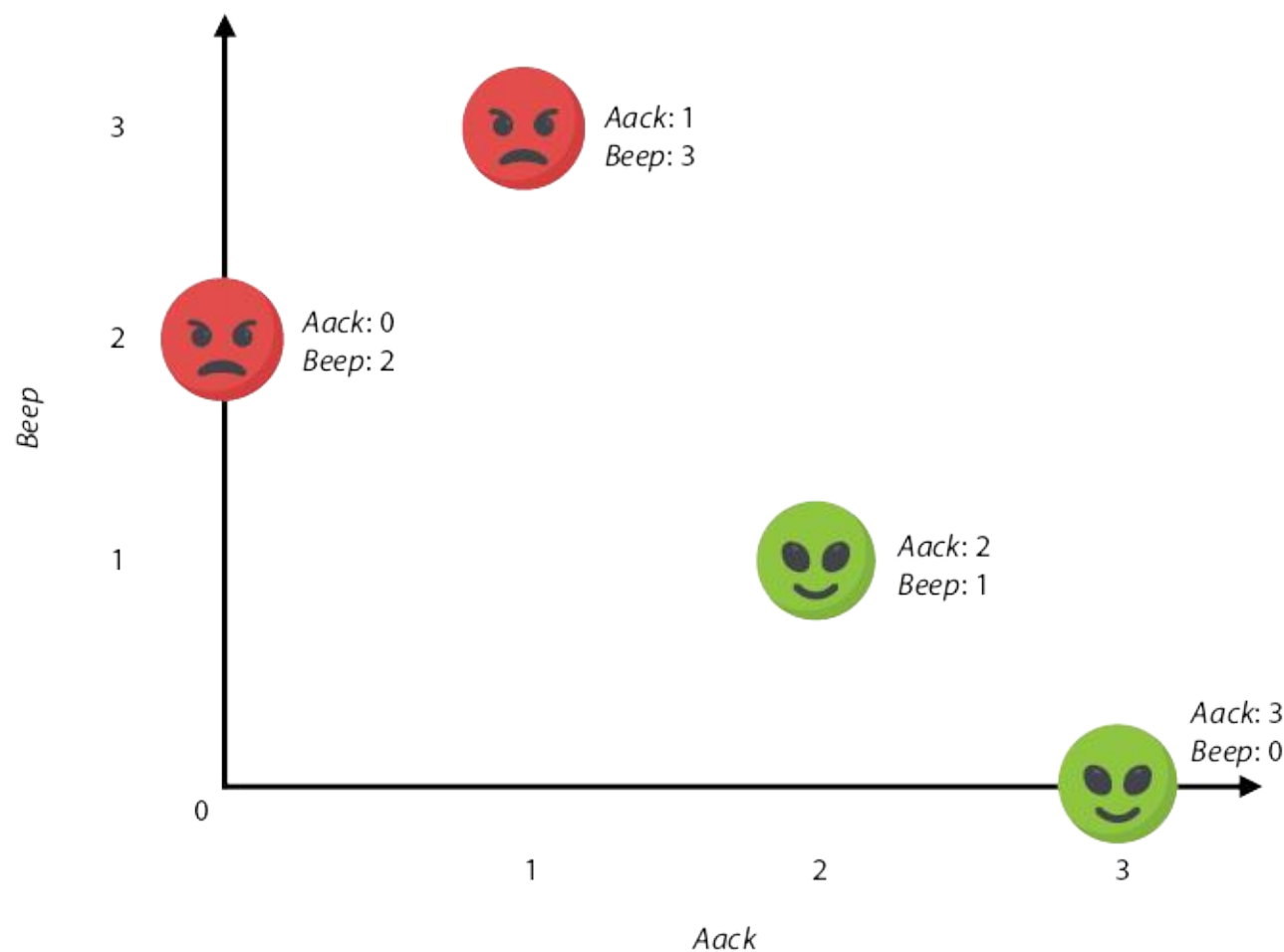
Trong hầu hết các tình huống, việc vẽ biểu đồ dữ liệu của chúng ta rất hữu ích vì đôi khi các mẫu đẹp sẽ hiển thị.Trong bảng 5.1, chúng ta có bốn người ngoài hành tinh, cũng như số lần mỗi người nói các từ *aack* và *beep* cũng như tâm trạng của họ.

Bảng 5.1 Tập dữ liệu của chúng tôi về người ngoài hành tinh, những câu họ nói và tâm trạng của họ. Chúng tôi đã chia mỗi câu thành số lần xuất hiện của các từ aack và beep.

Câu	Ách	Bíp	Tâm trạng
Ú ù ù ù!	3	0	Vui mừng
Bíp bíp!	0	2	Sad
Ách bíp bíp!	2	1	Vui mừng
Aack bíp bíp bíp!	1	3	Sad

Đồ thị gồm có hai trục, trục ngang (x) và trục dọc (y).Ở trục hoành, chúng tôi ghi lại số lần xuất hiện của *aack*, còn ở trục dọc

trực, sự xuất hiện của *tiếng bíp*. Sơ đồ này có thể được nhìn thấy trong hình 5.2.



Hình 5.2 Sơ đồ tập dữ liệu về người ngoài hành tinh. Trong trục hoành, chúng ta biểu thị số lần xuất hiện của từ *aack* và trên trục tung, số lần xuất hiện của từ *bíp*.

Lưu ý rằng trong sơ đồ ở hình 5.2, những người ngoài hành tinh vui vẻ nằm ở phía dưới bên phải, trong khi những người ngoài hành tinh buồn bã nằm ở phía trên bên trái. Điều này là do phía dưới bên phải là khu vực mà các câu có nhiều tiếng *aack* hơn là *tiếng bíp* và khu vực phía trên bên trái thì ngược lại. Trên thực tế, một dòng được tạo bởi tất cả các câu có cùng số lần xuất hiện *aack* và *beep* chia hai vùng này, như minh họa trong hình 5.3. Dòng này có phương trình sau:

$$\#aack = \#bíp$$

Hoặc tương đương, phương trình này:

$$\#aack - \#bíp = 0$$

Trong suốt chương này, chúng ta sẽ sử dụng các biến x với các chỉ số dưới khác nhau để biểu thị số lần xuất hiện của một từ trong câu. Trong trường hợp này, x_{aack} là

số lần từ *aack* xuất hiện và x_{beep} là số lần từ *bíp* xuất hiện.

Sử dụng ký hiệu này, phương trình của bộ phân loại trở thành $x_{aack} - x_{beep} = 0$, hoặc tương đương $x_{aack} = x_{beep}$. Đây là phương trình đường thẳng trong mặt phẳng. Nếu không xuất hiện, hãy nghĩ đến phương trình đường thẳng $y = x$, ngoại trừ thay vì x , chúng ta có x_{aack} , và thay vì y , chúng ta có x_{beep} . Tại sao không sử dụng x và y giống như chúng ta đã làm từ thời trung học phải không? Tôi rất muốn, nhưng tiếc là chúng tôi cần y cho mục đích khác (dự đoán) sau này. Do đó, hãy coi trục x_{aack} là trục hoành và trục x_{beep} là trục tung. Cùng với phương trình này, chúng ta có hai

những khu vực quan trọng mà chúng tôi gọi là *vùng tích cực* và *vùng tiêu cực*. Chúng được định nghĩa như sau:

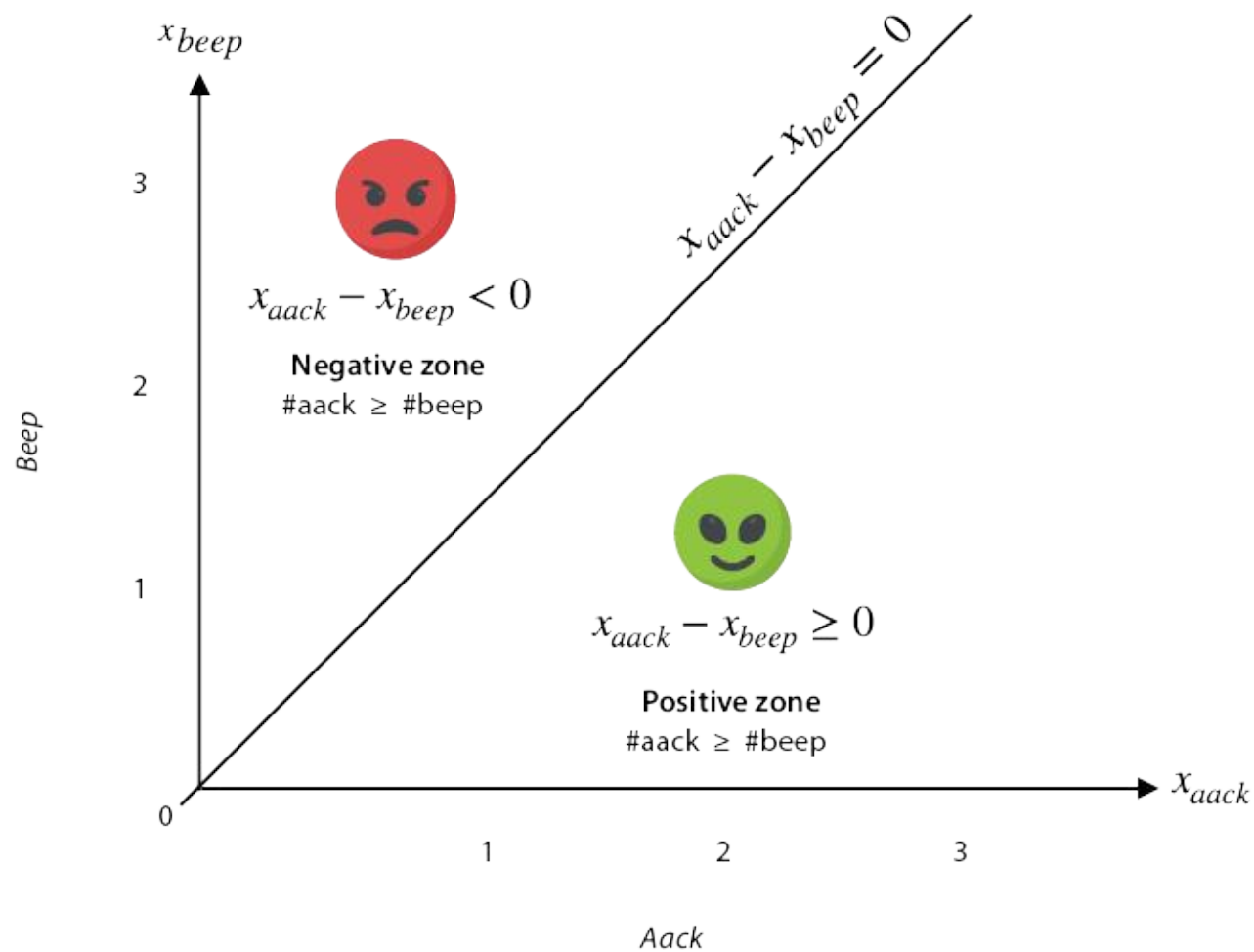
Vùng dương: Diện tích trên mặt phẳng mà $x_{aack} - x_{beep} \geq 0$. Điều này tương ứng đối với những câu trong đó từ *aack* xuất hiện ít nhất bằng số lần từ *bíp*.

Vùng âm: Diện tích trên mặt phẳng mà $x_{aack} - x_{beep} < 0$. Điều này tương ứng đối với những câu trong đó từ *aack* xuất hiện ít lần hơn từ *bíp*.

Bộ phân loại mà chúng tôi tạo ra dự đoán rằng mọi câu trong vùng tích cực đều vui và mọi câu trong vùng tiêu cực đều buồn. Vì vậy, mục tiêu của chúng tôi là tìm ra bộ phân loại có thể đặt càng nhiều câu vui càng tốt vào vùng tích cực và càng nhiều câu buồn càng tốt vào vùng tiêu cực. Đối với ví dụ nhỏ này, trình phân loại của chúng tôi hoàn thành công việc này một cách hoàn hảo. Điều này không phải lúc nào cũng đúng, nhưng thuật toán perceptron sẽ giúp chúng ta tìm ra bộ phân loại thực hiện tốt công việc này.

Trong hình 5.3, chúng ta có thể thấy đường tương ứng với bộ phân loại và vùng dương và âm. Nếu so sánh hình 5.2 và 5.3, bạn có thể thấy bộ phân loại hiện tại là tốt, vì tất cả các câu vui đều nằm trong vùng tích cực và tất cả các câu buồn đều nằm trong vùng tiêu cực.

Bây giờ chúng ta đã xây dựng một bộ phân loại perceptron phân tích cảm xúc đơn giản, hãy xem một ví dụ phức tạp hơn một chút.



Hình 5.3 Bộ phân loại là đường chéo phân chia điểm vui và điểm buồn. Phương trình của đường thẳng này là $x_{aack} = x_{beep}$ (hoặc tương đương $x_{aack} - x_{beep} = 0$), bởi vì đường thẳng tương ứng với tất cả các điểm có tọa độ ngang và dọc bằng nhau. Vùng vui là vùng trong đó số lần xuất hiện của *aack* lớn hơn hoặc bằng số lần xuất hiện của *tiếng bíp* và vùng buồn là vùng có số lần xuất hiện của *aack* ít hơn số lần xuất hiện của *bíp*.

Một hành tinh phức tạp hơn một chút

Trong phần này, chúng ta thấy một ví dụ phức tạp hơn, trong đó giới thiệu một khía cạnh mới của perceptron: độ lệch. Sau khi chúng ta có thể giao tiếp với người ngoài hành tinh trên hành tinh thứ nhất, chúng ta được cử đi làm nhiệm vụ đến hành tinh thứ hai, nơi người ngoài hành tinh có ngôn ngữ phức tạp hơn một chút. Mục tiêu của chúng tôi vẫn giống nhau: tạo ra bộ phân loại phân tích tình cảm bằng ngôn ngữ của họ. Ngôn ngữ ở hành tinh mới có hai từ: *crack* và *doink*. Bộ dữ liệu được thể hiện trong bảng 5.2.

Việc xây dựng bộ phân loại cho tập dữ liệu này có vẻ khó hơn một chút so với tập dữ liệu trước đó. Trước hết, chúng ta nên chấm điểm tích cực hay tiêu cực cho từ *crack* và *doink*? Hãy lấy giấy bút và thử nghĩ ra một bộ phân loại

that can correctly separate the happy and sad sentences in this dataset. Looking at the plot of this dataset in figure 5.4 may be helpful.

Table 5.2 The new dataset of alien words. Again, we’ve recorded each sentence, the number of appearances of each word in that sentence, and the mood of the alien.

Sentence	Crack	Doink	Mood
Crack!	1	0	Sad
Doink doink!	0	2	Sad
Crack doink!	1	1	Sad
Crack doink crack!	2	1	Sad
Doink crack doink doink!	1	3	Happy
Crack doink doink crack!	2	2	Happy
Doink doink crack crack crack!	3	2	Happy
Crack doink doink crack doink!	2	3	Happy



Figure 5.4 The plot of the new dataset of aliens. Notice that the happy ones tend to be above and to the right, and the sad ones below and to the left.

The idea for this classifier is to count the number of words in a sentence. Notice

rằng những câu có một, hai, ba chữ đều buồn, những câu có bốn, năm chữ đều vui. Đó là bộ phân loại! Nó phân loại những câu có ba từ trở xuống là buồn và những câu có bốn từ trở lên là vui. Chúng ta có thể viết lại điều này theo cách toán học hơn.

PHÂN LOẠI PHÂN TÍCH TÌNH CẢM

Cho một câu, gán điểm sau cho các từ:

Điểm:

- *Crack*: một điểm
- *Doink*: một điểm

Quy tắc:

Tính điểm của câu bằng cách cộng điểm của tất cả các từ trong câu đó.

- Nếu số điểm từ bốn trở lên thì dự đoán câu đó là vui. Nếu số điểm từ ba trở
- xuống thì dự đoán câu đó là câu buồn.

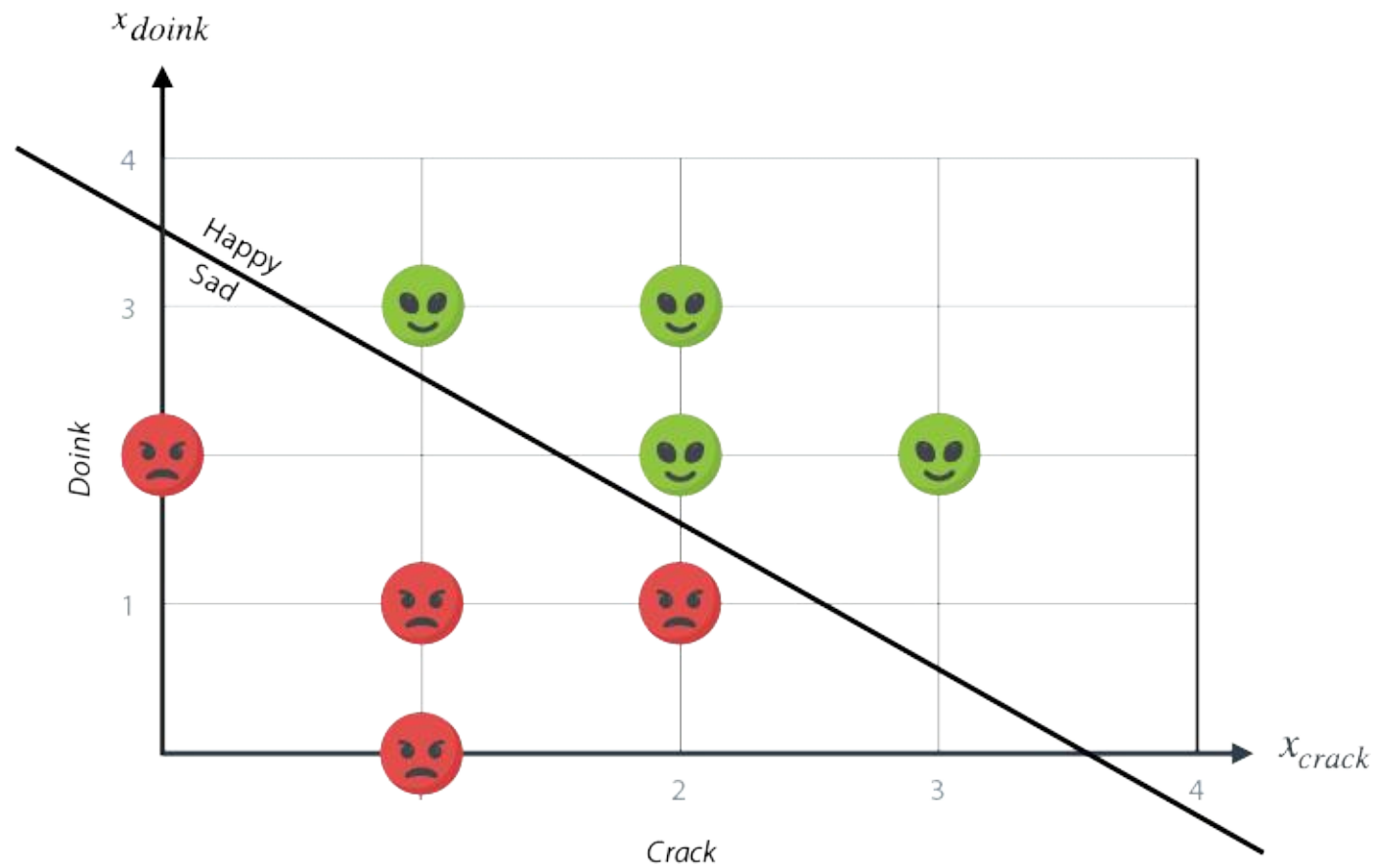
Để đơn giản hơn, hãy thay đổi quy tắc một chút bằng cách sử dụng mức cắt là 3,5.

Quy tắc:

Tính điểm của câu bằng cách cộng điểm của tất cả các từ trong câu đó.

- Nếu số điểm từ 3,5 trở lên dự đoán câu đó vui. Nếu số điểm nhỏ hơn 3,5 dự
- đoán câu đó buồn.

Phân loại này lại tương ứng với một đường thẳng và đường thẳng đó được minh họa trong hình 5.5



Hình 5.5 Bộ phân loại cho tập dữ liệu mới về người ngoài hành tinh. Nó lại là ranh giới chia cắt người ngoài hành tinh vui và buồn.

Trong ví dụ trước, chúng ta đã kết luận rằng từ *aack* là từ vui vẻ và từ *beep* là từ buồn. Điều gì xảy ra trong ví dụ này? Có vẻ như cả hai từ *crack* và *doink* đều vui vì điểm của chúng đều dương. Vậy tại sao câu “*Crack doink*” lại là câu buồn? Nó không có đủ từ. Người ngoài hành tinh trên hành tinh này có một tính cách đặc biệt. Người ngoài hành tinh ít nói thì buồn, còn người nói nhiều thì vui. Cách chúng ta có thể giải thích là người ngoài hành tinh trên hành tinh này vốn dĩ rất buồn, nhưng họ có thể thoát khỏi nỗi buồn bằng cách nói nhiều.

Một yếu tố quan trọng khác trong bộ phân loại này là ngưỡng hoặc ngưỡng là 3,5. Ngưỡng này được bộ phân loại sử dụng để đưa ra dự đoán, vì những câu có điểm cao hơn hoặc bằng ngưỡng được phân loại là vui và những câu có điểm thấp hơn ngưỡng được phân loại là buồn. Tuy nhiên, các ngưỡng không phổ biến và thay vào đó chúng tôi sử dụng khái niệm *khuy nh hướng*. Độ lệch là giá trị âm của ngưỡng và chúng tôi cộng nó vào điểm. Bằng cách này, bộ phân loại có thể tính điểm và trả về dự đoán là vui nếu điểm không âm hoặc buồn nếu nó âm. Thay đổi cuối cùng về ký hiệu là chúng tôi sẽ gọi điểm của các từ là *trọng số*. Phân loại của chúng tôi có thể được thể hiện như sau:

PHÂN LOẠI PHÂN TÍCH TÌNH CẢM

Cho một câu, gán trọng số và độ lệch sau cho các từ:

Trọng lượng:

- *Crack*: một điểm
- *Doink*: một điểm

Thành kiến: $-3,5$ điểm

Quy tắc:

Tính điểm của câu bằng cách cộng trọng số của tất cả các từ trên đó và độ lệch.

- Nếu số điểm lớn hơn hoặc bằng 0 thì dự đoán câu đó là vui. Nếu điểm nhỏ hơn 0 dự đoán câu đó buồn.

phương trình về điểm của bộ phân loại và cả dòng trong hình 5.5 như sau :

$$\#crack + \#doink - 3,5 = 0$$

Lưu ý rằng việc xác định một bộ phân loại perceptron có ngưỡng 3,5 và độ lệch $-3,5$ là như nhau, vì hai phương trình sau là tương đương:

- $\#crack + \#doink \geq 3,5$
- $\#doink - 3,5 \geq 0$

Chúng ta có thể sử dụng ký hiệu tương tự như ở phần trước, trong đó x_{crack} là số lần xuất hiện của từ *crack* và x_{doink} là số lần xuất hiện của từ *doink*. Như vậy phương trình đường thẳng trên hình 3.5 có thể viết là

$$x_{crack} + x_{doink} - 3,5 = 0.$$

Đường này cũng chia mặt phẳng thành các vùng dương và âm, được xác định như sau:

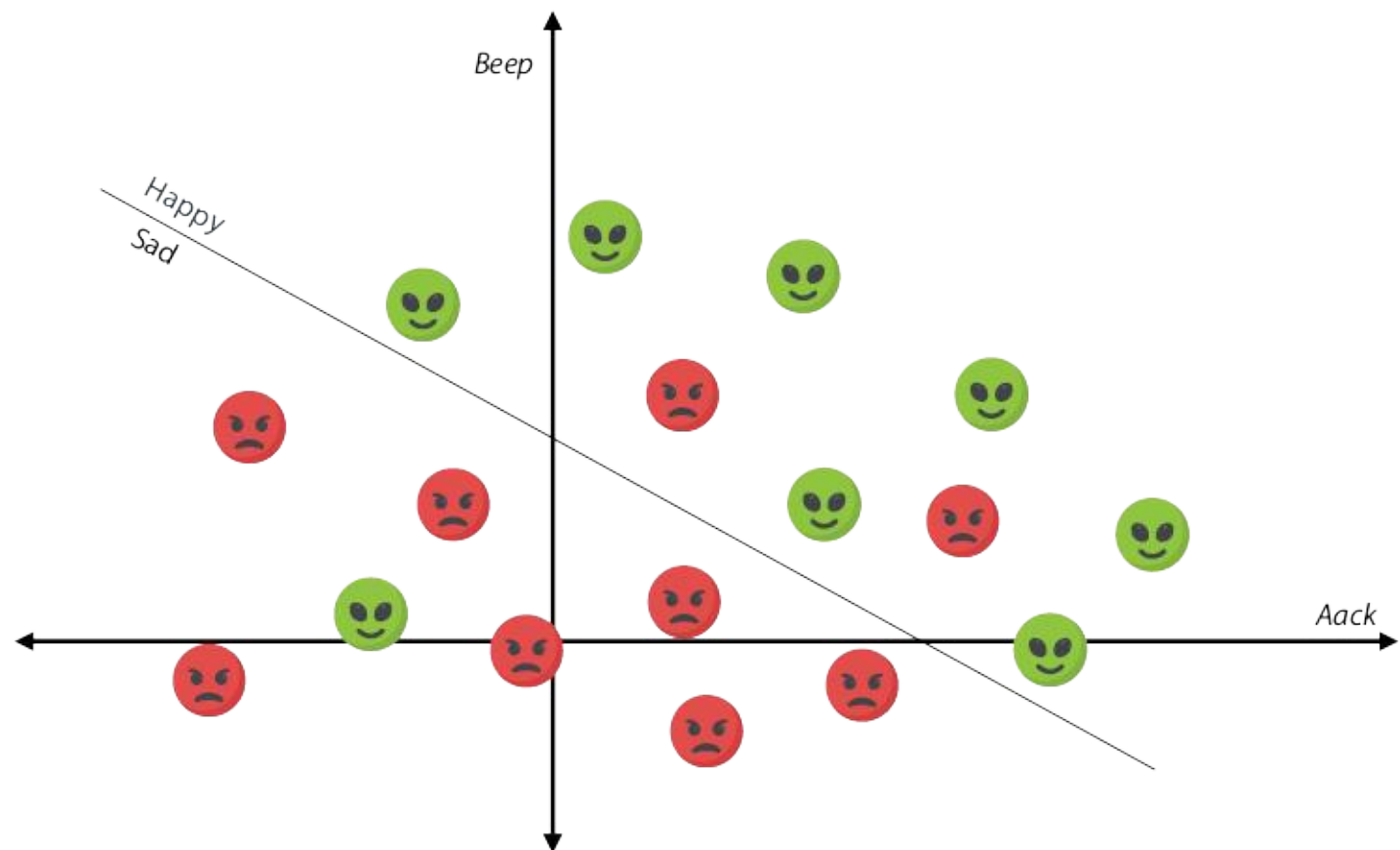
Vùng dương: diện tích trên mặt phẳng mà $x_{crack} + x_{doink} - 3,5 \geq 0$

Vùng âm: diện tích trên mặt phẳng mà $x_{crack} + x_{doink} - 3,5 < 0$

Bộ phân loại của chúng ta có cần phải luôn chính xác không?

Không

Trong hai ví dụ trước, chúng tôi đã xây dựng một bộ phân loại luôn chính xác. Nói cách khác, bộ phân loại đã phân loại hai câu vui là vui và hai câu buồn là buồn. Đây không phải là điều người ta thường thấy trong thực tế, đặc biệt là trong các tập dữ liệu có nhiều điểm. Tuy nhiên, mục tiêu của bộ phân loại là phân loại các điểm tốt nhất có thể. Trong hình 5.6, chúng ta có thể thấy một tập dữ liệu có 17 điểm (tám vui và chín buồn) không thể chia thành hai một cách hoàn hảo chỉ bằng một dòng. Tuy nhiên, dòng trong hình làm rất tốt, chỉ phân loại sai ba điểm.



Hình 5.6 Dòng này phân chia tập dữ liệu tốt. Lưu ý rằng nó chỉ mắc ba lỗi: hai ở vùng vui và một ở vùng buồn.

Một công cụ phân loại tổng quát hơn và một cách hơi khác để xác định các đường

Trong phần này, chúng ta có cái nhìn tổng quát hơn về bộ phân loại perceptron. Trong giây lát, hãy gọi các từ của chúng ta là 1 và 2 và các biến theo dõi giá trị của chúng

lần xuất hiện x_1 và x_2 . Các phương trình của hai phân loại trước đó như sau:

- $x_1 - x_2 = 0$
- $x_1 + x_2 - 3.5 = 0$

Dạng tổng quát của phương trình của bộ phân loại perceptron là $ax_1 + bx_2 + c = 0$, trong đó a là điểm của từ 1, b là điểm của từ 2 và c là điểm thiên vị. Phương trình này tương ứng với một đường chia mặt phẳng thành hai vùng như sau:

Vùng dương: vùng trên mặt phẳng mà $ax_1 + bx_2 + c \geq 0$

Vùng âm: vùng trên mặt phẳng mà $ax_1 + bx_2 + c < 0$

Ví dụ: nếu từ 1 có điểm là 4, từ 2 có điểm $-2,5$ và độ lệch là $1,8$ thì phương trình của phân loại này là

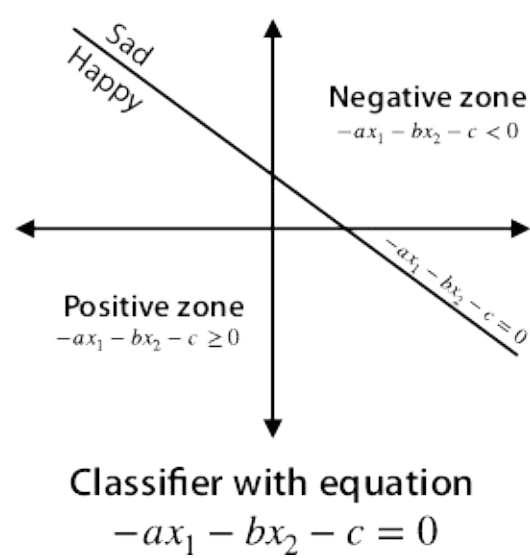
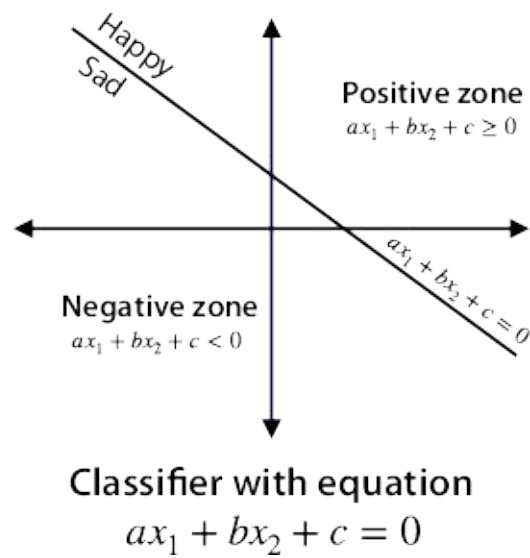
$$4x_1 - 2.5x_2 + 1.8 = 0,$$

và các vùng tích cực và tiêu cực là những vùng có $4x_1 - 2,5x_2 + 1,8 \geq 0$ và $4x_1 - 2,5x_2 + 1,8 < 0$ tương ứng.

ngoài ra: Phương trình đường thẳng và miền trong mặt phẳng Trong chương 3, chúng ta đã xác định đường thẳng bằng phương trình $y = mx + b$ trên mặt phẳng có trục là x và y . Trong chương này, chúng ta định nghĩa chúng bằng phương trình $ax_1 + bx_2 + c = 0$ trên mặt phẳng có các trục là x_1 và x_2 . Chúng khác nhau như thế nào? Cả hai đều là những cách hoàn toàn hợp lệ để

xác định một dòng. Tuy nhiên, trong khi phương trình đầu tiên hữu ích cho các mô hình hồi quy tuyến tính, thì phương trình thứ hai lại hữu ích cho các mô hình perceptron (và nói chung, cho các thuật toán phân loại khác, chẳng hạn như hồi quy logistic, mạng lưới thần kinh và máy vector hỗ trợ, mà chúng ta sẽ thấy lần lượt trong chương 6, 10 và 11). Tại sao phương trình này tốt hơn cho các mô hình perceptron? Một số ưu điểm sau:

- Phương trình $ax_1 + bx_2 + c = 0$ không chỉ xác định đường thẳng mà còn xác định rõ ràng hai vùng dương và âm. Nếu chúng ta muốn có cùng một đường thẳng, ngoại trừ các vùng dương và âm bị đảo ngược, chúng ta sẽ xét phương trình $-ax_1 - bx_2 - c = 0$.
- Sử dụng phương trình $ax_1 + bx_2 + c = 0$, ta vẽ được các đường thẳng đứng, vì phương trình của đường thẳng đứng là $x = c$ hoặc $1x_1 + 0x_2 - c = 0$. Mặc dù đường thẳng đứng không thường xuất hiện trong các mô hình hồi quy tuyến tính, chúng xuất hiện trong các mô hình phân loại.



Hình 5.7 Một bộ phân loại được xác định bởi một đường thẳng có phương trình $ax_1 + bx_2 + c = 0$, a vùng dương và vùng âm. Nếu chúng ta muốn đảo ngược vùng dương và vùng âm, tất cả những gì chúng ta cần làm là phủ nhận trọng số và độ lệch. Ở bên trái, chúng ta có bộ phân loại có phương trình $ax_1 + bx_2 + c = 0$. Ở bên phải, bộ phân loại có các vùng đảo ngược và phương trình $-ax_1 - bx_2 - c = 0$.

Hàm bước và hàm kích hoạt: Một cách cô đọng để nhận dự đoán

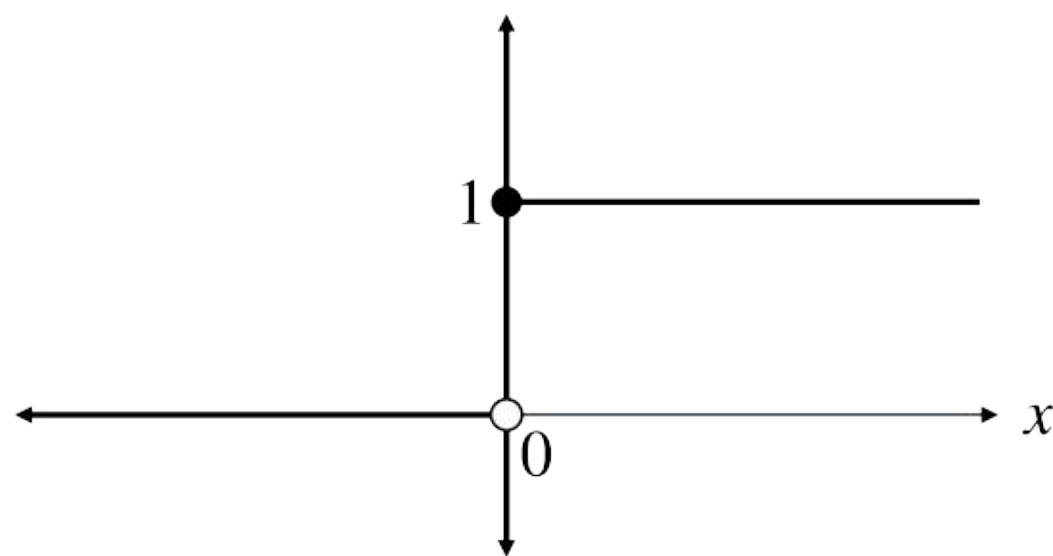
Trong phần này, chúng ta tìm hiểu một phím tắt toán học để có được dự đoán. Tuy nhiên, trước khi tìm hiểu điều này, chúng ta cần biến tất cả dữ liệu của mình thành số. Lưu ý rằng các nhãn trong tập dữ liệu của chúng tôi là “vui” và “buồn”. Chúng tôi ghi lại những giá trị này lần lượt là 1 và 0.

Cả hai bộ phân loại perceptron mà chúng ta xây dựng trong chương này đều được xác định bằng câu lệnh if. Cụ thể, bộ phân loại dự đoán “vui” hay “buồn” dựa trên tổng điểm của câu; nếu điểm này là dương hoặc bằng 0, thì bộ phân loại sẽ dự đoán là “vui vẻ” và nếu nó âm, thì bộ phân loại sẽ dự đoán là “buồn”. Chúng tôi có một cách trực tiếp hơn để biến điểm số thành dự đoán: sử dụng *chức năng bước*.

hàm bước Hàm trả về 1 nếu đầu ra không âm và 0 nếu đầu ra âm. Nói cách khác, nếu đầu vào là x thì

- $bước(x) = 1$ if $x \geq 0$
- $bước(x) = 0$ if $x < 0$

Hình 5.8 thể hiện đồ thị của hàm bước.



Hình 5.8 Hàm bước rất hữu ích trong việc nghiên cứu các mô hình perceptron. Đầu ra của hàm bước là 0 khi đầu vào âm và 1 nếu ngược lại.

Với hàm bước, chúng ta có thể biểu thị đầu ra của bộ phân loại perceptron một cách dễ dàng. Trong tập dữ liệu của chúng tôi, chúng tôi sử dụng biến y để chỉ các nhãn, giống như chúng tôi đã làm trong chương 3. Dự đoán mà mô hình đưa ra cho nhãn được ký hiệu là $\hat{y}$. Đầu ra của mô hình perceptron được viết dưới dạng cô đọng như sau

$$\hat{y} = \text{step}(ax_1 + bx_2 + c).$$

Hàm bước là trường hợp cụ thể của *chức năng kích hoạt*. Hàm kích hoạt là một khái niệm quan trọng trong học máy, đặc biệt là trong học sâu và sẽ xuất hiện lại trong chương 6 và 10. Định nghĩa chính thức về hàm kích hoạt sẽ được trình bày sau, vì toàn bộ sức mạnh của nó được sử dụng để xây dựng mạng lưới thần kinh. Nhưng hiện tại, chúng ta có thể coi hàm kích hoạt là một hàm mà chúng ta có thể sử dụng để biến điểm số thành dự đoán.

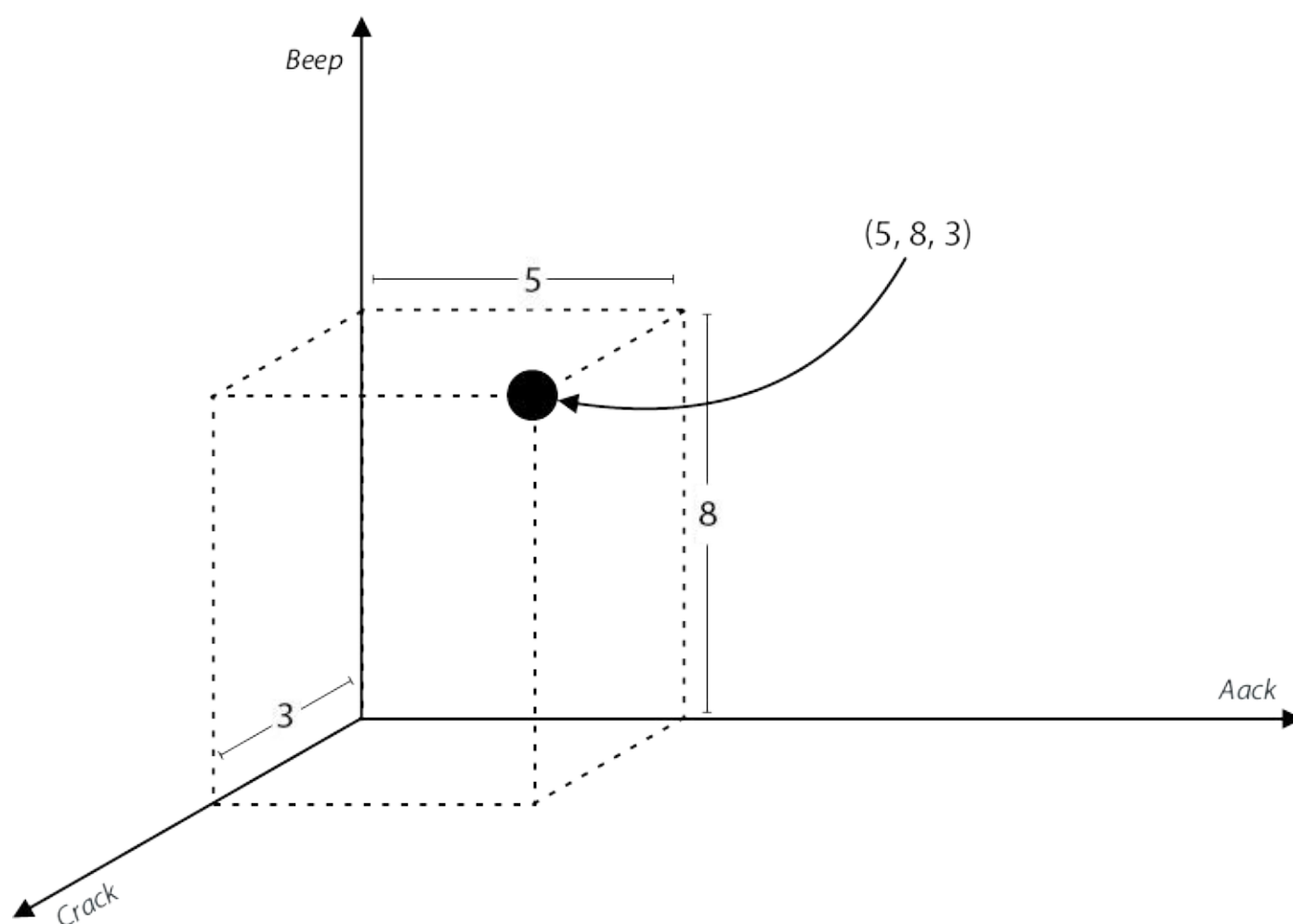
Điều gì xảy ra nếu tôi có nhiều hơn hai từ? Định nghĩa chung về bộ phân loại perceptron

Trong hai ví dụ về người ngoài hành tinh ở đầu phần này, chúng tôi đã xây dựng bộ phân loại perceptron cho các ngôn ngữ có hai từ. Nhưng chúng ta có thể xây dựng các bộ phân loại với số lượng từ tùy thích. Ví dụ: nếu chúng ta có một ngôn ngữ có ba từ, chẳng hạn như *aack*, *beep* và *crack*, trình phân loại sẽ đưa ra dự đoán theo công thức sau:

$$\hat{y} = \text{step}(ax_{aack} + bx_{beep} + cx_{crack} + d),$$

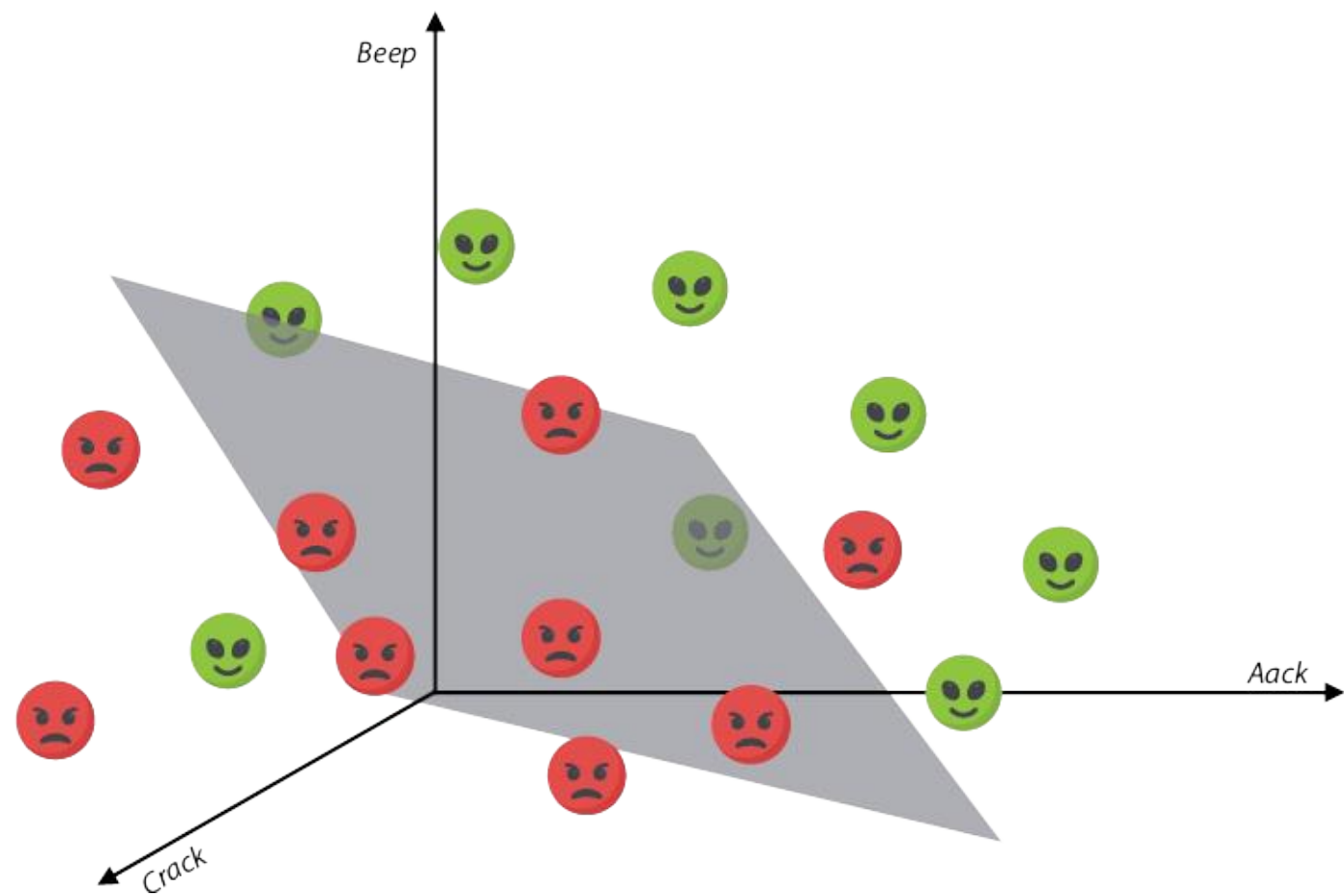
trong đó a , b và c lần lượt là trọng số của các từ *aack*, *beep* và *crack* và d là độ lệch.

Như chúng ta đã thấy, bộ phân loại perceptron phân tích tình cảm cho các ngôn ngữ có hai từ có thể được biểu thị dưới dạng một đường trong mặt phẳng phân chia điểm vui và điểm buồn. Bộ phân loại phân tích cảm xúc cho các ngôn ngữ có ba từ cũng có thể được biểu diễn dưới dạng hình học. Chúng ta có thể tưởng tượng các điểm tồn tại trong không gian ba chiều. Trong trường hợp này, mỗi trục tương ứng với từng từ *aack*, *beep* và *crack* và một câu tương ứng với một điểm trong không gian mà ba tọa độ của nó là số lần xuất hiện của ba từ. Hình 5.9 minh họa một ví dụ trong đó câu chứa *aack* năm lần, *beep* tám lần và *crack* ba lần, tương ứng với điểm có tọa độ $(5, 8, 3)$.



Hình 5.9 Một câu có ba từ có thể được vẽ dưới dạng một điểm trong không gian. Trong trường hợp này, một câu có từ *aack* năm lần, *bíp* tám lần và *crack* ba lần được vẽ tại điểm có tọa độ $(5, 8, 3)$.

Cách để phân tách các điểm này là sử dụng một mặt phẳng. Phương trình với một mặt phẳng chính xác là $ax_{aack} + bx_{beep} + cx_{crack} + d$, và mặt phẳng này được minh họa trong hình 5.10.



Hình 5.10 Một tập dữ liệu gồm các câu có ba từ được biểu thị theo không gian ba chiều. Bộ phân loại được biểu diễn bằng một mặt phẳng chia không gian thành hai vùng.

Chúng tôi có thể xây dựng các bộ phân loại perceptron phân tích tình cảm cho các ngôn ngữ có nhiều từ nhất có thể. Giả sử ngôn ngữ của chúng ta có n từ mà chúng ta gọi là $1, 2, \dots, n$. Tập dữ liệu của chúng tôi bao gồm m câu mà chúng tôi gọi là $x^{(1)}, x^{(2)}, \dots, x^{(m)}$. Mỗi câu $x^{(1)}$

có nhãn y_i , là 1 nếu câu vui và 0 nếu câu buồn. Cách chúng tôi ghi lại từng câu là sử dụng số lần xuất hiện của mỗi n từ. Do đó, mỗi câu tương ứng với một hàng trong tập dữ liệu của chúng tôi và có thể được xem dưới dạng một vector hoặc một n -bộ số $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$, trong đó $x_j^{(i)}$ là

số lần xuất hiện của từ j trong câu thứ i .

Bộ phân loại perceptron bao gồm n trọng số (điểm), một cho mỗi từ n trong ngôn ngữ của chúng ta và một độ lệch. Các trọng số được ký hiệu là w_i và độ lệch b .

Do đó, dự đoán mà bộ phân loại đưa ra cho câu $x^{(i)}$ là

$$\hat{y}_i = \text{step}(w_1 x_1^{(i)} + w_2 x_2^{(i)} + \dots + w_n x_n^{(i)} + b).$$

Theo cách tương tự, các bộ phân loại có hai từ có thể được biểu diễn về mặt hình học dưới dạng một đường cắt mặt phẳng thành hai vùng và các bộ phân loại có ba từ có thể được biểu diễn dưới dạng một mặt phẳng cắt không gian ba chiều.

thành hai vùng, các bộ phân loại có n cũng có thể được biểu diễn dưới dạng hình học. Thật không may, chúng tôi cần n kích thước để xem chúng. Con người chỉ có thể nhìn thấy ba chiều, vì vậy chúng ta có thể phải tưởng tượng một mặt phẳng có chiều $(n-1)$ (gọi là *siêu phẳng*) cắt không gian n chiều thành hai vùng.

Tuy nhiên, việc chúng ta không thể tưởng tượng chúng về mặt hình học không có nghĩa là chúng ta không thể hiểu rõ về cách chúng hoạt động. Hãy tưởng tượng nếu bộ phân loại của chúng tôi được xây dựng trên ngôn ngữ tiếng Anh. Mỗi từ được gán một trọng số. Điều đó tương đương với việc tra từ điển và chấm điểm hạnh phúc cho từng từ. Kết quả có thể trông giống như thế này:

Trọng lượng (điểm):

- A: 0,1 điểm Aardvark: 0,2
- điểm Aargh: −4 điểm
- ...Niềm vui: 10 điểm
- ...Đau khổ: −8,5 điểm
- ...Hợp tử: 0,4 điểm
-
-
-
-
-

Thành kiến:

- −2,3 điểm

Nếu đó là trọng số và độ lệch của bộ phân loại, để dự đoán một câu vui hay buồn, chúng tôi cộng điểm của tất cả các từ trên đó (có lặp lại). Nếu kết quả cao hơn hoặc bằng 2,3 (âm của độ lệch) thì câu đó được dự đoán là hạnh phúc; ngược lại, nó được dự đoán là buồn.

Hơn nữa, ký hiệu này áp dụng cho mọi ví dụ, không chỉ cho phân tích cảm tính. Nếu chúng tôi gặp một vấn đề khác với các điểm dữ liệu, tính năng và nhãn khác nhau, chúng tôi có thể mã hóa vấn đề đó bằng cách sử dụng cùng một biến. Ví dụ: nếu chúng ta có một ứng dụng y tế trong đó chúng ta đang cố gắng dự đoán liệu một bệnh nhân ốm hay khỏe mạnh bằng cách sử dụng n trọng số và độ lệch, chúng ta vẫn có thể gọi các nhãn là y , các đặc tính x_i ,

trọng số w_i và độ lệch b .

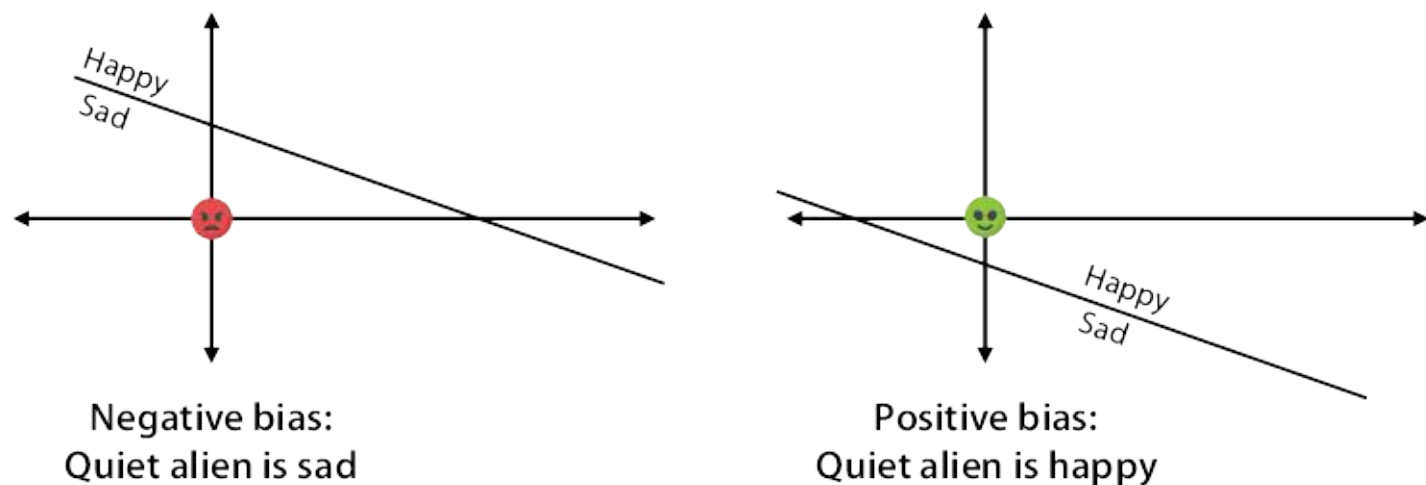
Sự thiên vị, điểm chặn y và tâm trạng vốn có của

a quiet alien

Cho đến nay chúng ta đã có ý tưởng tốt về ý nghĩa của trọng số của bộ phân loại. Những từ mang ý nghĩa tích cực là vui, những từ mang ý nghĩa tiêu cực là buồn. Những từ có trọng lượng rất nhỏ (dù tích cực hay tiêu cực) là những từ trung tính hơn. Tuy nhiên, sự thiên vị có nghĩa là gì?

Trong chương 3, chúng tôi đã xác định rằng độ lệch trong mô hình hồi quy đối với giá nhà là giá cơ sở của một ngôi nhà. Nói cách khác, đó là giá dự đoán của một ngôi nhà giả định không có phòng (studio?). Trong mô hình perceptron, độ lệch có thể được hiểu là điểm của câu trống. Nói cách khác, nếu một người ngoài hành tinh hoàn toàn không nói gì thì người ngoài hành tinh này vui hay buồn? Nếu một câu không có từ nào, điểm của nó chính xác là sai lệch. Vì vậy, nếu thành kiến là tích cực thì người ngoài hành tinh không nói gì sẽ hạnh phúc, và nếu thành kiến là tiêu cực thì người ngoài hành tinh đó sẽ buồn.

Về mặt hình học, sự khác biệt giữa độ lệch dương và độ lệch âm nằm ở vị trí của gốc (điểm có tọa độ $(0,0)$) đối với bộ phân loại. Điều này là do điểm có tọa độ $(0,0)$ tương ứng với câu không có từ. Trong các bộ phân loại có độ lệch dương, điểm gốc nằm ở vùng dương, trong khi ở các bộ phân loại có độ lệch âm, điểm gốc nằm ở vùng âm, như minh họa trong hình 5.11.



Hình 5.11 Bên trái: bộ phân loại có độ lệch âm hoặc *ngưỡng* dương (y - chặn). Điều này có nghĩa là người ngoài hành tinh không nói gì sẽ rơi vào vùng buồn và được xếp vào loại buồn. Phải: Trình phân loại có độ lệch dương hoặc ngưỡng âm. Điều này có nghĩa là người ngoài hành tinh không nói gì sẽ rơi vào vùng hạnh phúc và được xếp vào loại hạnh phúc.

Có thể nghĩ về các tập dữ liệu phân tích tình cảm trong đó độ lệch là dương hoặc

tiêu cực?Thế còn hai ví dụ sau:

Ví dụ 1 (thành kiến tích cực): tập dữ liệu về các đánh giá trực tuyến về sản phẩm

Hãy tưởng tượng một tập dữ liệu trong đó chúng tôi ghi lại tất cả các đánh giá về một sản phẩm cụ thể trên Amazon.Một số trong số chúng là tích cực và một số trong số đó là tiêu cực, tùy theo số lượng sao mà chúng nhận được.Bạn nghĩ điểm số sẽ là bao nhiêu cho một bài đánh giá trống?Theo kinh nghiệm của tôi, những đánh giá tiêu cực có xu hướng chứa nhiều từ ngữ vì khách hàng khó chịu và họ mô tả trải nghiệm tiêu cực của mình. Tuy nhiên, nhiều đánh giá tích cực đều trống rỗng—khách hàng chỉ cần cho điểm tốt mà không cần giải thích lý do tại sao họ thích sản phẩm.Do đó, bộ phân loại này có thể có độ lệch dương.

Ví dụ 2 (thành kiến tiêu cực): tập dữ liệu về các cuộc trò chuyện với bạn bè S

Hãy tưởng tượng rằng chúng ta ghi lại tất cả các cuộc trò chuyện của mình với bạn bè và phân loại chúng thành những cuộc trò chuyện vui hay buồn.Nếu một ngày nọ, chúng ta tình cờ gặp một người bạn và người bạn của chúng ta hoàn toàn không nói gì, chúng ta tưởng tượng rằng họ đang giận chúng ta hoặc họ rất khó chịu.Vì vậy, câu trống được xếp vào loại câu buồn.Điều này có nghĩa là bộ phân loại này có thể có độ lệch âm.

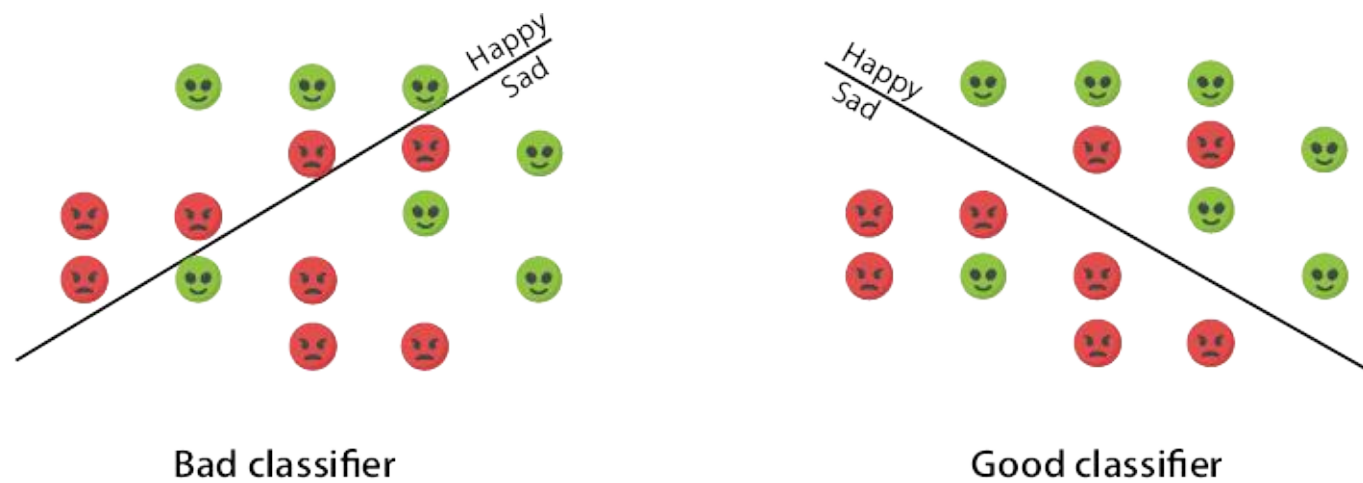
Làm cách nào để xác định liệu một bộ phân loại là tốt hay xấu?Hàm lỗi

Bây giờ chúng ta đã xác định được bộ phân loại perceptron là gì, mục tiêu tiếp theo của chúng ta là hiểu cách huấn luyện nó—nói cách khác, làm cách nào để tìm ra bộ phân loại perceptron phù hợp nhất với dữ liệu của chúng ta?Nhưng trước khi học cách huấn luyện các perceptron, chúng ta cần học một khái niệm quan trọng: cách đánh giá chúng.Cụ thể hơn, trong phần này, chúng ta tìm hiểu một hàm lỗi hữu ích sẽ cho chúng ta biết liệu bộ phân loại perceptron có phù hợp với dữ liệu của chúng ta hay không.Giống như cách mà sai số tuyệt đối và sai số bình phương hoạt động trong hồi quy tuyến tính ở chương 3, hàm sai số mới này sẽ lớn đối với các bộ phân loại không phù hợp với dữ liệu và nhỏ đối với những bộ phân loại phù hợp với dữ liệu.

Ồ w để so sánh các bộ phân loại?Hàm lỗi

Trong phần này, chúng ta tìm hiểu cách xây dựng một hàm lỗi hiệu quả giúp chúng ta xác định mức độ tốt của một bộ phân loại perceptron cụ thể.Đầu tiên, hãy kiểm tra trực giác của chúng ta.Hình 5.12 cho thấy hai bộ phân loại perceptron khác nhau trên cùng một

tập dữ liệu. Các bộ phân loại được biểu diễn dưới dạng một đường có hai cạnh vui và buồn được xác định rõ ràng. Rõ ràng, cái bên trái là bộ phân loại kém, còn cái bên phải là bộ phân loại tốt. Chúng ta có thể đưa ra thước đo xem chúng tốt như thế nào không? Nói cách khác, liệu chúng ta có thể gán một số cho mỗi người trong số họ không, theo cách mà số bên trái được gán số cao và số bên phải là số thấp?



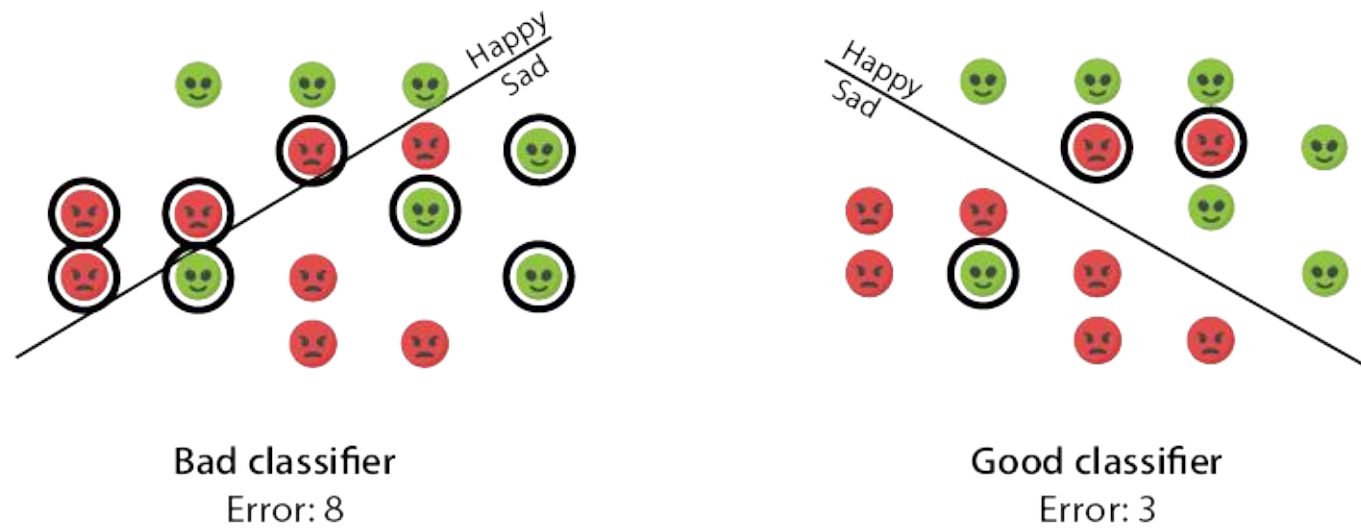
Hình 5.12 Bên trái: một bộ phân loại tồi, nó không thực sự phân chia điểm tốt. Đúng: một bộ phân loại tốt. Chúng ta có thể nghĩ ra một hàm lỗi gán số cao cho bộ phân loại kém và số thấp cho bộ phân loại tốt không?

Tiếp theo, chúng ta thấy các câu trả lời khác nhau cho câu hỏi này, tất cả đều có một số ưu và nhược điểm. Một trong số chúng (spoiler: cái thứ ba) là cái chúng tôi sử dụng để huấn luyện các perceptron.

CHỨC NĂNG LỖI 1: SỐ LỖI

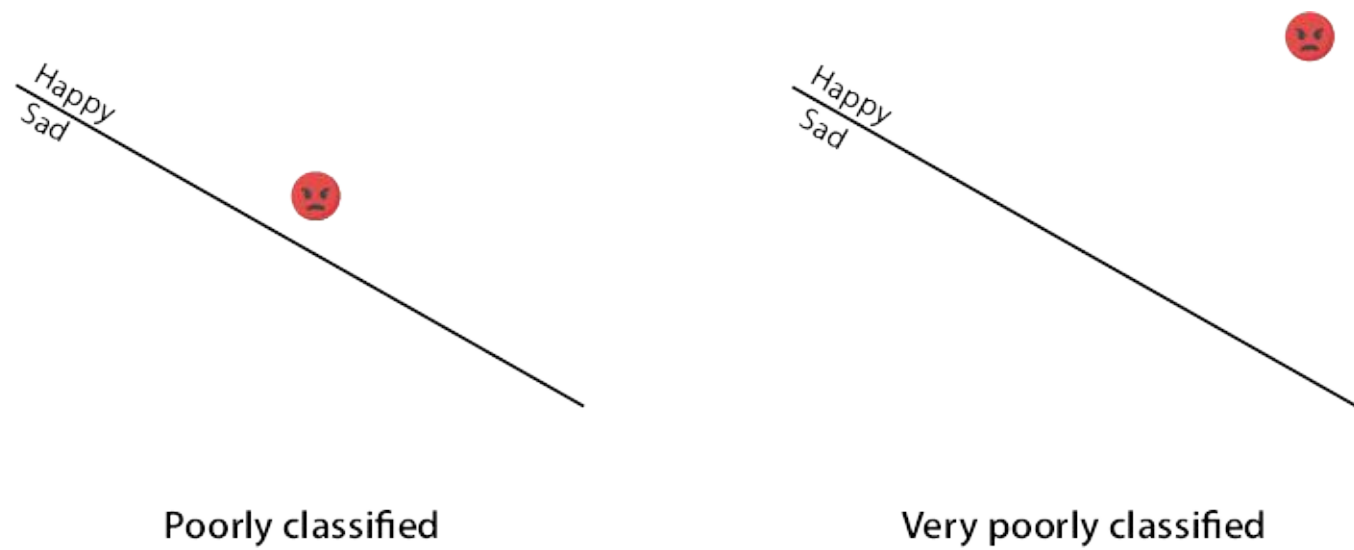
Cách đơn giản nhất để đánh giá một bộ phân loại là đếm số lỗi mà nó mắc phải—nói cách khác là đếm số điểm mà nó phân loại không chính xác.

Trong trường hợp này, bộ phân loại bên trái có lỗi 8, vì nó dự đoán sai bốn điểm vui là buồn và bốn điểm buồn là vui. Bộ phân loại tốt có sai số 3 vì nó dự đoán sai một điểm vui là buồn và hai điểm buồn là vui. Điều này được minh họa trong hình 5.13.



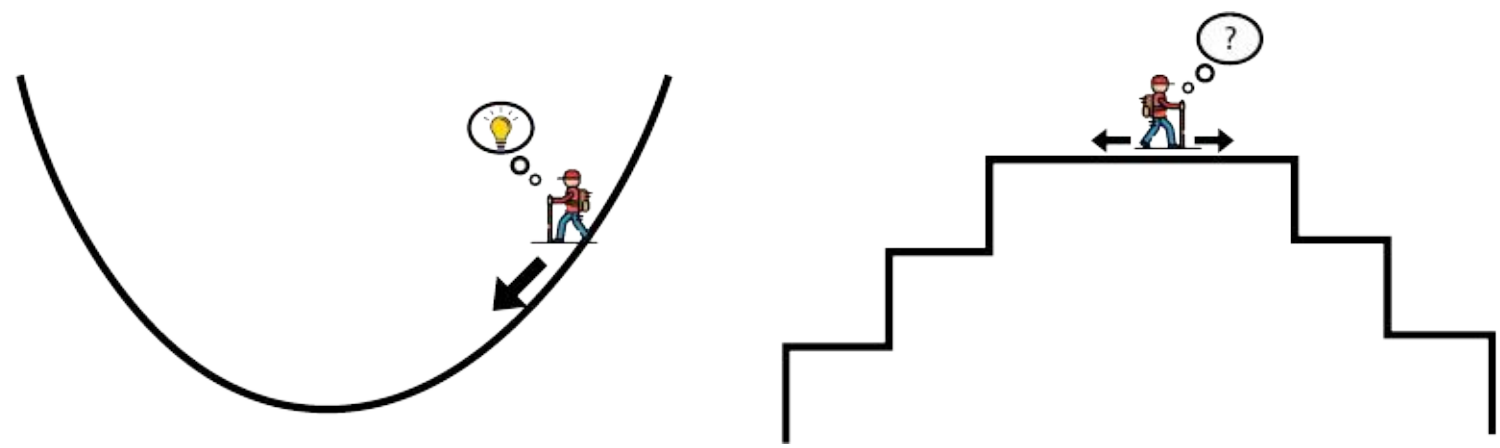
Hình 5.13 Chúng tôi đánh giá hai bộ phân loại bằng cách đếm số điểm mà mỗi bộ phân loại sai. Bộ phân loại bên trái phân loại sai 8 điểm, trong khi bộ phân loại bên phải phân loại sai 3 điểm. Do đó, chúng tôi kết luận rằng bộ phân loại ở bên phải là bộ phân loại tốt hơn cho tập dữ liệu của chúng tôi.

Đây là một hàm báo lỗi tốt nhưng không phải là một hàm báo lỗi lớn. Tại sao? Nó cho chúng ta biết khi nào có lỗi, nhưng nó không đo lường mức độ nghiêm trọng của lỗi. Ví dụ: nếu một câu buồn và bộ phân loại cho điểm đó là 1 thì bộ phân loại đã mắc lỗi. Tuy nhiên, nếu một bộ phân loại khác cho nó điểm 100 thì bộ phân loại này đã mắc một sai lầm lớn hơn nhiều. Cách để thấy điều này về mặt hình học là ở hình 5.14. Trong hình ảnh này, cả hai bộ phân loại đều phân loại sai một điểm buồn khi dự đoán rằng đó là điểm vui. Tuy nhiên, bộ phân loại ở bên trái đã định vị đường gần với điểm, có nghĩa là điểm buồn không quá xa vùng buồn. Ngược lại, bộ phân loại ở bên phải đã xác định được điểm rất xa vùng buồn của nó.



Hình 5.14 Hai bộ phân loại phân loại sai điểm. Tuy nhiên, bộ phân loại bên phải đã mắc một lỗi lớn hơn nhiều so với bộ phân loại bên trái. Điểm bên trái không xa ranh giới, và do đó, nó không xa vùng buồn lắm. Tuy nhiên, điểm bên phải rất xa vùng buồn. Lý tưởng nhất là chúng ta muốn một hàm lỗi gán lỗi cao hơn cho bộ phân loại ở bên phải so với bộ phân loại ở bên trái.

Tại sao chúng ta quan tâm đến việc đo lường mức độ nghiêm trọng của một lỗi? Đếm chúng chưa đủ sao? Hãy nhớ lại những gì chúng ta đã làm ở chương 3 với thuật toán hồi quy tuyến tính. Cụ thể hơn, hãy nhớ lại phần “Giảm dần độ dốc”, trong đó chúng tôi đã sử dụng giảm độ dốc để giảm lỗi này. Cách để giảm sai số là giảm sai số xuống từng lượng nhỏ cho đến khi chúng ta đạt đến điểm có sai số nhỏ. Trong thuật toán hồi quy tuyến tính, chúng tôi đã di chuyển đường này một lượng nhỏ và chọn hướng mà lỗi giảm nhiều nhất. Nếu lỗi của chúng tôi được tính bằng cách đếm số điểm bị phân loại sai thì lỗi này sẽ chỉ lấy giá trị nguyên. Nếu chúng ta lắc lư đường này một chút, sai số có thể không giảm chút nào và chúng ta không biết nên di chuyển theo hướng nào. Mục tiêu của việc giảm độ dốc là cực tiểu hóa hàm bằng cách thực hiện các bước nhỏ theo hướng mà hàm giảm nhiều nhất. Nếu hàm chỉ nhận các giá trị nguyên thì điều này tương đương với việc cố gắng đi xuống từ cầu thang Aztec. Khi chúng ta đang ở một bước phẳng, chúng ta không biết phải thực hiện bước nào vì hàm số không giảm theo bất kỳ hướng nào. Điều này được minh họa trong hình 5.15.



Hình 5.15 Thực hiện giảm độ dốc để giảm thiểu hàm lỗi cũng giống như đi xuống từ núi bằng cách thực hiện các bước nhỏ. Tuy nhiên, để chúng ta làm được điều đó thì hàm sai số không được phẳng (như hàm bên phải), vì trong hàm sai số phẳng, việc thực hiện một bước nhỏ sẽ không làm giảm sai số. Một hàm lỗi tốt giống như hàm bên trái, trong đó chúng ta có thể dễ dàng nhìn thấy hướng mà chúng ta phải sử dụng để thực hiện một bước nhằm giảm nhẹ hàm lỗi.

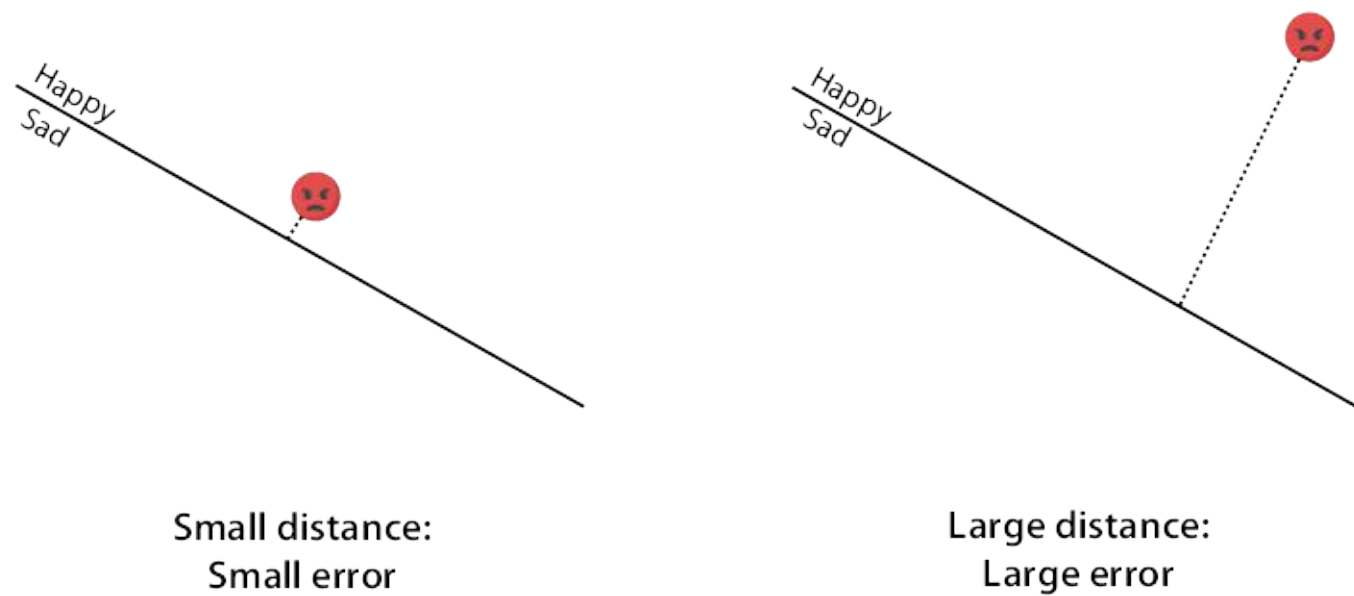
Chúng ta cần một hàm đo lường mức độ sai số và gán sai số cao hơn cho các điểm bị phân loại sai ở xa ranh giới so với các điểm ở gần nó.

CHỨC NĂNG LỖI 2: KHOẢNG CÁCH

Một cách để phân biệt hai bộ phân loại trong hình 5.16 là xem xét khoảng cách vuông góc từ điểm đến đường thẳng. Lưu ý rằng đối với bộ phân loại ở bên trái, khoảng cách này là nhỏ, trong khi đối với bộ phân loại ở bên phải, khoảng cách này là lớn.

Chức năng báo lỗi này hiệu quả hơn nhiều. Chức năng báo lỗi này thực hiện như sau:

- Các điểm được phân loại đúng sẽ tạo ra sai số bằng 0. Các điểm được phân loại sai sẽ tạo ra sai số bằng khoảng cách từ điểm đó đến đường thẳng.



Hình 5.16 Một cách hiệu quả để đo mức độ phân loại sai của một điểm là đo khoảng cách vuông góc từ điểm đến đường thẳng. Đối với bộ phân loại ở bên trái, khoảng cách này là nhỏ, trong khi đối với bộ phân loại ở bên phải thì khoảng cách này là lớn.

Hãy quay lại hai bộ phân loại mà chúng ta đã có ở đầu phần này. Cách chúng tôi tính toán tổng sai số là cộng các sai số tương ứng với tất cả các điểm dữ liệu, như minh họa trong hình 5.17. Điều này có nghĩa là chúng ta chỉ xem xét các điểm bị phân loại sai và cộng khoảng cách vuông góc từ các điểm này vào đường thẳng. Lưu ý rằng bộ phân loại kém có lỗi lớn và bộ phân loại tốt có lỗi nhỏ.



Hình 5.17 Để tính sai số tổng của một bộ phân loại, chúng ta cộng tất cả các sai số, đó là khoảng cách vuông góc từ các điểm bị phân loại sai. Lỗi lớn đối với bộ phân loại ở bên trái và nhỏ đối với bộ phân loại ở bên phải. Vì vậy, chúng tôi kết luận rằng bộ phân loại ở bên phải là tốt hơn.

Đây *gần như* là hàm báo lỗi mà chúng ta sẽ sử dụng. Tại sao chúng ta không sử dụng cái này? Bởi vì khoảng cách từ một điểm đến một đường thẳng là một công thức phức tạp. Nó chứa một căn bậc hai, vì chúng ta tính toán nó bằng định lý Pythagore. Căn bậc hai có các đạo hàm phức tạp, điều này làm tăng thêm độ phức tạp không cần thiết khi chúng ta áp dụng thuật toán giảm độ dốc. Chúng ta không cần phải thực hiện sự phức tạp này, vì thay vào đó, chúng ta có thể tạo một hàm lỗi dễ tính toán hơn nhưng vẫn nắm bắt được bản chất của hàm lỗi: trả về lỗi cho các điểm bị phân loại sai và thay đổi độ lớn dựa trên khoảng cách từ điểm bị phân loại sai đến ranh giới.

CHỨC NĂNG LỖI 3: ĐIỂM

Trong phần này, chúng ta xem cách xây dựng hàm lỗi tiêu chuẩn cho perceptron mà chúng ta gọi là *hàm lỗi perceptron*. Trước tiên, hãy tóm tắt các thuộc tính mà chúng ta muốn có trong hàm lỗi như sau:

- Hàm lỗi của điểm được phân loại đúng là 0. Hàm lỗi của điểm được phân loại sai là số dương.
- Đối với các điểm bị phân loại sai gần biên, hàm sai số nhỏ. Đối với các điểm bị phân loại sai ở xa ranh giới, hàm lỗi rất lớn. Nó được đưa ra bởi một công thức đơn giản.
-
-

Hãy nhớ lại rằng bộ phân loại dự đoán nhãn 1 cho các điểm trong vùng dương và nhãn 0 cho các điểm trong vùng âm. Do đó, điểm bị phân loại sai là điểm có nhãn 0 ở vùng dương hoặc điểm có nhãn 1 ở vùng âm.

Để xây dựng hàm lỗi perceptron, chúng tôi sử dụng điểm số. Cụ thể, chúng tôi sử dụng các thuộc tính sau của điểm:

Thuộc tính của điểm:

1. Các điểm ở ranh giới có điểm 0.
2. Các điểm ở vùng dương có điểm dương.
3. Các điểm trong vùng âm có điểm âm.
4. Các điểm gần ranh giới có điểm có cường độ thấp (tức là điểm dương hoặc điểm âm có giá trị tuyệt đối thấp).
5. Các điểm ở xa ranh giới có điểm có cường độ cao (tức là điểm dương hoặc điểm âm có giá trị tuyệt đối cao).

Đối với một điểm bị phân loại sai, lỗi perceptron muốn gán một giá trị tỷ lệ thuận với khoảng cách của nó tới ranh giới. Do đó, sai số đối với các điểm bị phân loại sai nằm xa ranh giới phải cao và sai số đối với các điểm bị phân loại sai nằm gần ranh giới phải thấp. Nhìn vào tính chất 4 và 5, chúng ta có thể thấy giá trị tuyệt đối của điểm luôn cao đối với các điểm ở xa ranh giới và thấp đối với các điểm gần ranh giới. Do đó, chúng tôi xác định lỗi là giá trị tuyệt đối của điểm đối với các điểm bị phân loại sai.

Cụ thể hơn, hãy xem xét trình phân loại gán trọng số a và b cho các từ *aack* và *beep* và có độ lệch là c . Trình phân loại này thực hiện dự đoán $\hat{y} = \text{bước}(ax_{aack} + bx_{beep} + c)$ cho câu có x_{aack} sự xuất hiện của từ *chắc*

và x_{beep} sự xuất hiện của từ *bíp*. Lỗi perceptron được định nghĩa như sau:

LỖI PECEPTRON CHO MỘT CÂU

- Nếu câu được phân loại đúng thì lỗi bằng 0. Nếu câu bị phân loại sai thì lỗi
- là $|x_{aack} + bx_{beep} + c|$.

Trong trường hợp chung, trong đó ký hiệu được xác định như trong phần “Điều gì xảy ra nếu tôi có nhiều hơn hai từ?”, thì sau đây là định nghĩa về lỗi perceptron:

LỖI PECEPTRON CHO MỘT ĐIỂM (CHUNG)

- Nếu điểm được phân loại đúng thì sai số bằng 0. Nếu điểm bị phân loại sai thì sai số là
- $|w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b|$.

LỖI PERCEPTRON TRUNG BÌNH: MỘT CÁCH TÍNH TOÁN LỖI CỦA TOÀN BỘ DỮ LIỆU

Để tính lỗi perceptron cho toàn bộ tập dữ liệu, chúng tôi lấy mức trung bình của tất cả các lỗi tương ứng với tất cả các điểm. Chúng ta cũng có thể lấy tổng nếu muốn, mặc dù trong chương này chúng ta chọn mức trung bình và gọi nó là *lỗi nhận thức trung bình*.

Để minh họa lỗi perceptron trung bình, hãy xem một ví dụ.

VÍ DỤ

Hãy xem xét tập dữ liệu gồm bốn câu, hai câu có nhãn vui và hai câu có nhãn buồn, được minh họa trong bảng 5.3.

Bảng 5.3 Bộ dữ liệu mới về người ngoài hành tinh. Một lần nữa, chúng tôi đã ghi lại từng câu, số lần xuất hiện của mỗi từ trong câu đó và tâm trạng của người ngoài hành tinh.

Câu	Ách	Bíp	Nhãn (tâm trạng)
Ách	1	0	Sad
Bíp	0	1	Vui mừng
Ack bip bip	1	3	Vui mừng
Ack bip ack aack	3	2	Sad

Chúng tôi sẽ so sánh hai phân loại sau trên tập dữ liệu này:

PHÂN LOẠI 1

Trọng lượng:

- Ách: $a = 1$
- Bíp: $b = 2$

Thành kiến: $c = -4$

Điểm của một câu: $1x_{aack} + 2x_{beep} - 4$

Dự đoán: $\hat{y} = bước(1x_{aack} + 2x_{beep} - 4)$

PHÂN LOẠI 2

Trọng lượng:

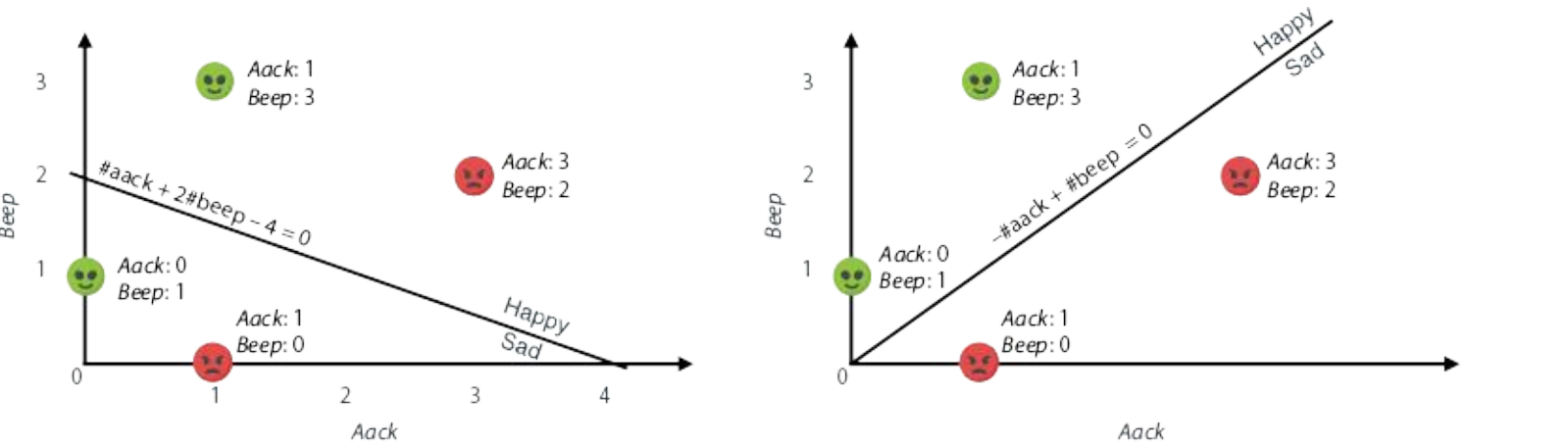
- Aack: $a = -1$
- Bíp: $b = 1$

Thành kiến: $c = 0$

Điểm của câu: $-x_{aack} + x_{beep}$

Dự đoán: $\hat{y} = bước(-x_{aack} + x_{beep})$

Các điểm và phân loại có thể được nhìn thấy trong hình 5.18. Thoạt nhìn, cái nào có vẻ phân loại tốt hơn? Có vẻ như bộ phân loại 2 tốt hơn vì nó phân loại chính xác mọi điểm, trong khi bộ phân loại 1 mắc hai lỗi. Bây giờ hãy tính toán các lỗi và đảm bảo rằng bộ phân loại 1 có lỗi cao hơn bộ phân loại 2.



Hình 5.18 Ở bên trái chúng ta có bộ phân loại 1 và bên phải chúng ta có bộ phân loại 2. Dự đoán cho cả hai phân loại được tính toán trong bảng 5.4.

Bảng 5.4 Tập dữ liệu của chúng tôi gồm bốn câu kèm theo nhãn. Đối với mỗi bộ phân loại, chúng ta có điểm số và dự đoán.

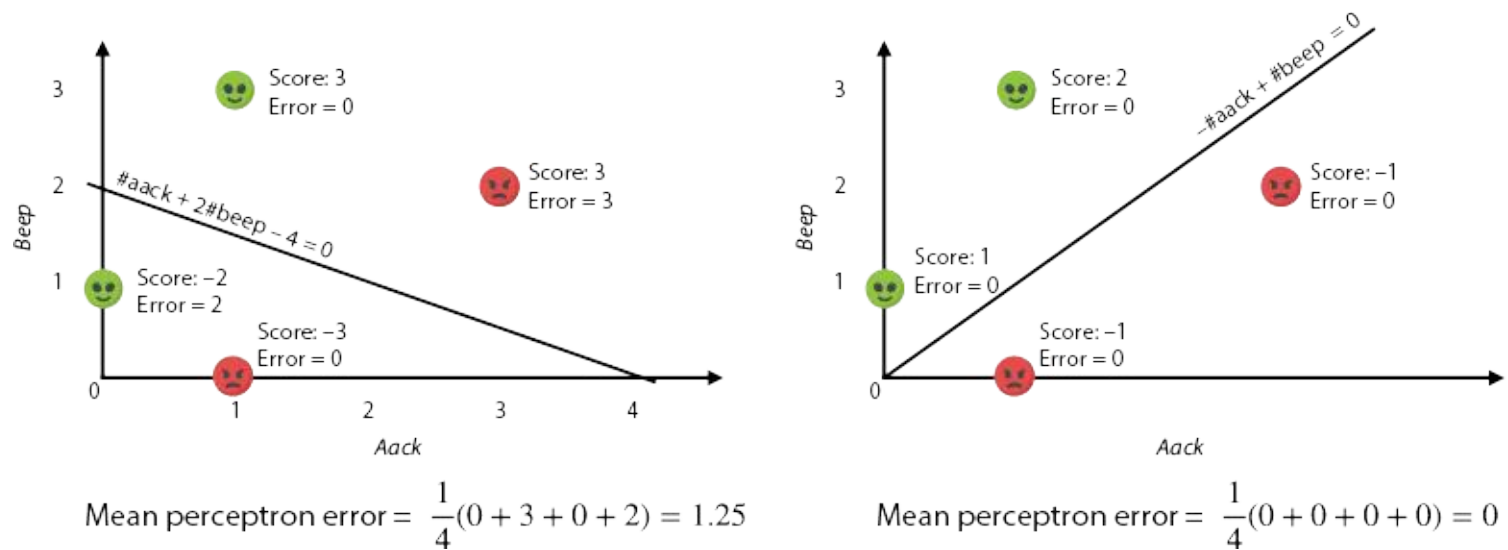
Câu (x_{aack}, x_{beep})	Nhãn y	Điểm phân loại 1 $1x_{aack} + 2x_{beep} - 4$	Dự đoán phân loại 1 $bước(1x_{aack} + 2x_{beep} - 4)$	Lỗi phân loại 1	Điểm phân loại 2 $-x_{aack} + x_{beep}$	Classi predic step(- + 2 x_{beep}
(1,0)	Sad (0)	-3	0 (đúng)	0	-1	0 (correct)
(0,1)	Hạnh Phúc (1)	-2	0 (không chính xác)	2	1	1 (correct)
(1,3)	Hạnh Phúc (1)	3	1 (đúng)	3	2	1 (correct)
(3,2)	Sad (0)	3	1 (không chính xác)	0	-1	0 (correct)
Lỗi nhận thức trung bình				1.25		

Bây giờ hãy tính toán các lỗi. Lưu ý rằng loại 1 chỉ phân loại sai câu 2 và 4. Câu 2 vui nhưng bị phân loại sai thành buồn và câu 4 buồn,

nhưng nó bị phân loại sai thành hạnh phúc. Lỗi ở câu 2 là giá trị tuyệt đối của điểm, hoặc $|-2| = 2$. Lỗi ở câu 4 là giá trị tuyệt đối của điểm, hoặc $|3| = 3$. Hai câu còn lại có lỗi 0, vì được phân loại đúng. Do đó, sai số nhận thức trung bình của phân loại 1 là

$$1/4(0 + 2 + 0 + 3) = 1.25.$$

Bộ phân loại 2 không mắc lỗi—nó phân loại chính xác tất cả các điểm. Do đó, sai số tri giác trung bình của phân loại 2 là 0. Khi đó chúng tôi kết luận rằng phân loại 2 tốt hơn phân loại 1. Tóm tắt các tính toán này được trình bày trong bảng 5.4 và hình 5.19.



Hình 5.19 Bộ phân loại 1 có sai số 1,25, trong khi bộ phân loại 2 có sai số 0. Do đó, chúng tôi kết luận rằng bộ phân loại 2 tốt hơn bộ phân loại 1.

Bây giờ chúng ta đã biết cách so sánh các bộ phân loại, hãy chuyển sang tìm cái tốt nhất trong số chúng, hoặc ít nhất là một cái khá tốt.

Làm thế nào để tìm được bộ phân loại tốt? Thuật toán perceptron

Để xây dựng một trình phân loại perceptron tốt, chúng ta sẽ làm theo cách tiếp cận tương tự như cách chúng ta đã làm với hồi quy tuyến tính trong chương 3. Quá trình này được gọi là *thuật toán perceptron* và nó bao gồm việc bắt đầu với một trình phân loại perceptron ngẫu nhiên và dần dần cải thiện nó cho đến khi chúng ta có một thuật toán tốt. Các bước chính của thuật toán perceptron như sau:

1. Bắt đầu với bộ phân loại perceptron ngẫu nhiên.
2. Cải thiện một chút khả năng phân loại.(Lặp lại nhiều lần).
3. Đo lỗi perceptron để quyết định khi nào dừng chạy vòng lặp.

Chúng tôi bắt đầu bằng cách phát triển bước bên trong vòng lặp, một kỹ thuật được sử dụng để cải thiện một chút trình phân loại perceptron được gọi là *thủ thuật perceptron*. Nó tương tự như thủ thuật bình phương và tuyệt đối mà chúng ta đã học ở phần “Thủ thuật bình phương” và “Thủ thuật tuyệt đối” ở chương 3.

Thủ thuật perceptron: Một cách cải thiện perceptron một chút

Thủ thuật perceptron là một bước nhỏ giúp chúng ta chuyển từ bộ phân loại perceptron sang bộ phân loại perceptron tốt hơn một chút. Tuy nhiên, chúng tôi sẽ bắt đầu bằng cách mô tả một bước ít tham vọng hơn một chút. Giống như chúng ta đã làm trong chương 3, trước tiên chúng ta sẽ tập trung vào một điểm và cố gắng cải thiện bộ phân loại cho điểm đó.

Có hai cách để xem bước perceptron, mặc dù cả hai đều tương đương nhau. Cách đầu tiên là cách hình học, trong đó chúng ta coi bộ phân loại là một đường thẳng.

Mã giả cho thủ thuật PERCEPTRON (Hình học)

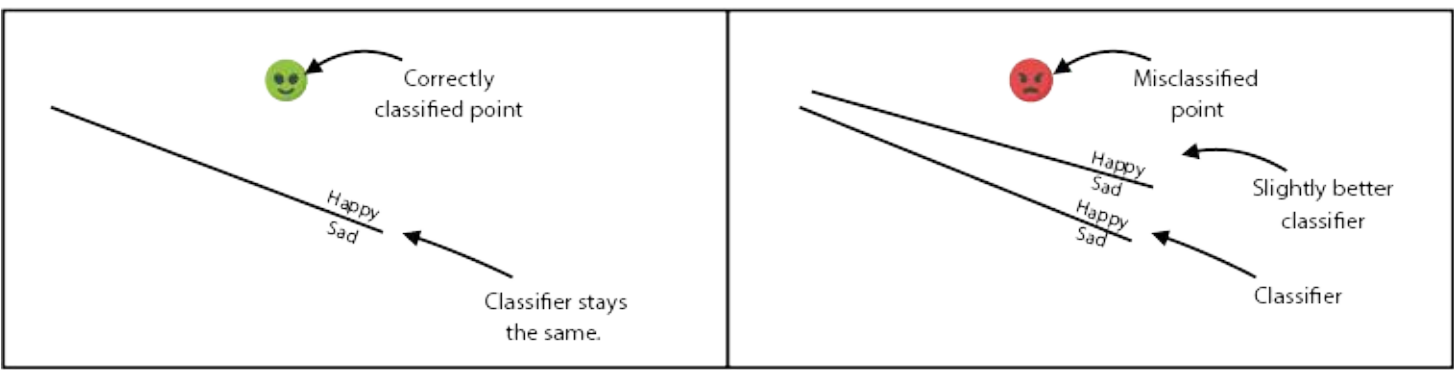
- **Trường hợp 1:** Nếu điểm được phân loại chính xác, hãy giữ nguyên đường thẳng.
- **Trường hợp 2:** Nếu điểm được phân loại không chính xác, hãy di chuyển đường thẳng đến gần điểm hơn một chút.

Tại sao điều này làm việc? Hãy nghĩ về nó. Nếu điểm bị phân loại sai, điều đó có nghĩa là nó nằm ở phía sai của đường thẳng. Di chuyển đường thẳng đến gần nó hơn có thể không đặt nó ở phía bên phải, nhưng ít nhất nó đưa nó đến gần đường thẳng hơn và do đó, gần phía bên phải của đường thẳng hơn. Chúng tôi lặp lại quá trình này nhiều lần, vì vậy có thể tưởng tượng rằng một ngày nào đó chúng tôi sẽ có thể di chuyển đường thẳng qua điểm, từ đó phân loại nó một cách chính xác. Quá trình này được minh họa trong hình 5.20.

Chúng ta cũng có một cách đại số để xem thủ thuật perceptron.

Case 1

Case 2



Hình 5.20 Trường hợp 1 (trái): Một điểm được phân loại chính xác sẽ bảo đường thẳng giữ nguyên vị trí của nó. Trường hợp 2 (phải): Một điểm bị phân loại sai sẽ yêu cầu đường thẳng tiến lại gần điểm đó.

MÃ GIÁ CHO THỦ THUẬT PECEPTRON (đại số)

- **Trường hợp 1:** Nếu điểm được phân loại chính xác, hãy để nguyên bộ phân loại. **Trường hợp 2:**
- Nếu điểm được phân loại không chính xác, điều đó có nghĩa là nó tạo ra lỗi dương. Điều chỉnh trọng số và độ lệch một lượng nhỏ để sai số này giảm đi một chút.

Cách hình học là cách dễ dàng hơn để hình dung thủ thuật này, nhưng cách đại số là cách dễ dàng hơn để phát triển nó, vì vậy chúng ta sẽ xem xét nó theo cách đại số. Đầu tiên, hãy sử dụng trực giác của chúng ta. Hãy tưởng tượng rằng chúng ta có một bộ phân loại cho toàn bộ ngôn ngữ tiếng Anh. Chúng tôi thử phân loại này trên câu “Tôi buồn” và nó dự đoán rằng câu đó là vui. Điều này rõ ràng là sai. Chúng ta có thể đã sai ở đâu? Nếu dự đoán là câu vui thì câu đó phải nhận được điểm tích cực. Câu này không nên nhận điểm tích cực—nó phải nhận điểm âm mới được xếp vào loại câu buồn. Điểm được tính bằng tổng điểm của các từ *I*, *am* và *buồn* cộng với độ chệch. Chúng ta cần giảm điểm này xuống để câu văn buồn hơn một chút. Nếu chúng ta giảm xuống một chút thì điểm vẫn dương. Chúng tôi hy vọng rằng việc chạy quá trình này nhiều lần sẽ một ngày nào đó chuyển điểm thành âm và phân loại chính xác câu của chúng tôi. Cách để giảm điểm là giảm tất cả các phần của nó, cụ thể là trọng số của các từ *Tôi*, *am*, *buồn* và độ lệch. Chúng ta nên giảm chúng bao nhiêu? Chúng ta giảm chúng đi một lượng bằng với tốc độ học mà chúng ta đã học trong phần “Thủ thuật bình phương” ở chương 3.

Tương tự, nếu bộ phân loại của chúng ta phân loại sai câu “Tôi vui” thành câu buồn

thì quy trình của chúng tôi là tăng nhẹ trọng số của các từ *I*, *am* và *hạnh phúc* và độ lệch lên một lượng bằng với tốc độ học tập.

Hãy minh họa điều này bằng một ví dụ số. Trong ví dụ này, chúng tôi sử dụng tốc độ học tập $\eta = 0,01$. Hãy tưởng tượng rằng chúng ta có cùng một bộ phân loại mà chúng ta đã có trong phần trước, cụ thể là bộ phân loại có trọng số và độ lệch như sau. Chúng tôi sẽ gọi nó là bộ phân loại kém vì mục tiêu của chúng tôi là cải thiện nó.

PHÂN LOẠI XẤU

Trọng lượng:

- *Aack*: $a =$
- *1Bíp*: $b = 2$

Thành kiến: $c = -4$

Dự đoán: $\hat{y} = \text{bước}(x_{aack} + 2x_{beep} - 4)$

Câu sau đây bị mô hình phân loại sai và chúng tôi sẽ sử dụng nó để cải thiện trọng số:

Câu 1: “Beep aack aack beep beep beep.”

Nhãn: Buồn (0)

Đối với câu này, số lần xuất hiện của *aack* là $x_{aack} = 2$, và số số lần xuất hiện của *tiếng bíp* là $x_{beep} = 5$. Như vậy, điểm là $1 \cdot x_{aack} + 2 \cdot x_{beep} - 4 = 1 \cdot 2 + 2 \cdot 5 - 4 = 8$, và dự đoán là $\hat{y} = \text{bước}(8) = 1$.

Câu lẽ ra phải có điểm âm mới được xếp vào loại buồn. Tuy nhiên, bộ phân loại đã cho điểm 8, đây là điểm tích cực. Chúng ta cần giảm điểm này. Một cách để giảm nó là trừ tốc độ học với trọng số của *aack* với trọng số của *bíp* và với độ lệch, do đó thu được các trọng số mới mà chúng tôi gọi là $a' = 0,99$, $b' = 1,99$ và độ lệch mới $c' = 4,01$. Tuy nhiên, hãy nghĩ về điều này: từ *bíp* xuất hiện nhiều lần hơn từ *aack*. Theo một cách nào đó, *tiếng bíp* đóng vai trò quan trọng đối với điểm của câu hơn là *aack*. Có lẽ chúng ta nên giảm trọng số của *bíp* nhiều hơn số điểm của *aack*. Hãy giảm trọng số của mỗi từ bằng tốc độ học nhân với số lần từ đó xuất hiện trong câu. Nói cách khác:

- Từ aack xuất hiện hai lần, vì vậy chúng ta sẽ giảm trọng số của nó xuống hai lần tốc độ học, hay 0,02. Ta thu được trọng số mới $a' = 1 - 2 \cdot 0,01 = 0,98$.
- Từ bíp xuất hiện năm lần, vì vậy chúng tôi sẽ giảm trọng số của nó xuống 5 lần tốc độ học, hay 0,05. Chúng ta thu được trọng số mới $b' = 2 - 5 \cdot 0,01 = 1,95$.
- Độ lệch chỉ cộng vào điểm một lần, vì vậy chúng tôi giảm độ lệch theo tỷ lệ học hoặc 0,01. Chúng ta thu được độ lệch mới $c' = -4 - 0,01 = -4,01$.

bỏ qua Thay vì trừ tốc độ học từ mỗi trọng số, chúng tôi trừ tốc độ học nhân với số lần xuất hiện của từ trong câu. Lý do thực sự cho điều này là tính toán. Nói cách khác, khi chúng ta phát triển phương pháp giảm độ dốc, đạo hàm của hàm sai số buộc chúng ta phải thực hiện điều này. Quá trình này được trình bày chi tiết trong phụ lục B, phần “Sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình phân loại”.

Bộ phân loại cải tiến mới như sau:

PHÂN LOẠI ĐƯỢC CẢI TIẾN 1

Trọng lượng:

- Aack: $a' = 0,98$
- Bíp: $b' = 1,95$

Thành kiến: $c' = -4,01$

Dự đoán: $\hat{y} = \text{bước}(0,98x_{\text{aack}} + 1,95x_{\text{beep}} - 4,01)$

Hãy xác minh lỗi của cả hai phân loại. Hãy nhớ lại rằng lỗi là giá trị tuyệt đối của điểm. Do đó, trình phân loại kém tạo ra lỗi $|1 \cdot x_{\text{aack}} + 2 \cdot x_{\text{beep}} - 4| = |1 \cdot 2 + 2 \cdot 5 - 4| = 8$. Bộ phân loại cải tiến tạo ra lỗi $|0,98 \cdot x_{\text{aack}} +$

$1,95 \cdot x_{\text{beep}} - 4,01| = |0,98 \cdot 2 + 1,95 \cdot 5 - 4,01| = 7,7$. Đó là một lỗi nhỏ hơn, vì vậy chúng tôi thực sự đã cải thiện trình phân loại cho điểm đó!

Trường hợp chúng tôi vừa phát triển bao gồm một điểm bị phân loại sai có nhãn âm. Điều gì xảy ra nếu điểm bị phân loại sai có nhãn dương? Quy trình thực hiện tương tự, ngoại trừ việc thay vì trừ đi một lượng từ trọng số, chúng ta cộng nó lại. Hãy quay lại bộ phân loại xấu và xem xét câu sau:

Câu 2: “Aack aack.”

Nhãn: Hạnh phúc

Dự đoán cho câu này là $\hat{y} = \text{step}(x_{aack} + 2x_{beep} - 4) = \text{step}(2 + 2 \cdot 0 - 4)$

$= \text{bước}(-2) = 0$. Vì dự đoán buồn nên câu bị phân loại sai. Điểm của câu này là -2 , và để phân loại câu này là vui, chúng ta cần bộ phân loại cho điểm tích cực. Thủ thuật perceptron sẽ tăng số điểm này lên -2 bằng cách tăng trọng số của các từ và độ lệch như sau:

- Từ *aack* xuất hiện hai lần, vì vậy chúng tôi sẽ tăng trọng số của nó lên gấp đôi tốc độ học hoặc $0,02$. Chúng ta thu được trọng số mới $a' = 1 + 2 \cdot 0,01 = 1,02$. Từ *bíp* xuất hiện 0 lần nên chúng ta sẽ không tăng trọng số của nó vì từ này không liên quan đến câu. Độ lệch chỉ cộng vào điểm một lần, vì vậy chúng tôi tăng độ lệch theo tỷ lệ học tập hoặc $0,01$. Chúng ta thu được độ lệch mới $c' = -4 + 0,01 = -3,99$.
-

Do đó, bộ phân loại cải tiến mới của chúng tôi như sau:

PHÂN LOẠI ĐƯỢC CẢI TIẾN 2

Trọng lượng:

- *Aack*: $a' = 1,02$
- *Bíp*: $b' = 2$

Thành kiến: $c' = -3,99$

Dự đoán: $\hat{y} = \text{bước}(1,02x_{aack} + 2x_{beep} - 3,99)$

Bây giờ hãy xác minh các lỗi. Vì người phân loại kém cho điểm -2 nên lỗi là $|-2| = 2$. Người phân loại kém cho câu $1,02x_{aack} + 2x_{beep} - 3,99 = 1,02 \cdot 2 + 2 \cdot 0 - 3,99 = -1,95$ và sai số $1,95$.

Do đó, trình phân loại cải tiến có lỗi nhỏ hơn ở điểm đó so với trình phân loại kém, đó chính xác là những gì chúng tôi mong đợi.

Hãy tóm tắt hai trường hợp này và lấy mã giả cho thủ thuật perceptron.

MÃ GIẢ CHO THỦ THUẬT PERCEPTRON

Đầu vào:

- Một perceptron có trọng số a, b , và độ lệch c
- Một điểm có tọa độ (x_1, x_2) và nhãn y
- Giá trị dương nhỏ η (tỷ lệ học)

Đầu ra:

- Perceptron với các trọng số mới a', b' và độ lệch c'

Thủ tục:

- Dự đoán mà perceptron đưa ra tại thời điểm đó là $\hat{y} = \text{bước}(ax_1 + bx_2 + c)$. **Trường hợp 1:**
- Nếu $\hat{y} = y$: **Trả về** perceptron gốc có trọng số a', b' và độ lệch c' . **Trường hợp 2:** Nếu $\hat{y} = 1$ và $y = 0$:
- - **Trả về** perceptron với các trọng số và độ lệch sau:
 - $a' = a - \eta x_1$
 - $b' = b - \eta x_2$
 - $c' = c - \eta x_1$
- **Trường hợp 3:** Nếu $\hat{y} = 0$ và $y = 1$:
 - **Trả về** perceptron với các trọng số và độ lệch sau:
 - $a' = a + \eta x_1$
 - $b' = b + \eta x_2$
 - $c' = c + \eta x_1$

Nếu perceptron phân loại chính xác điểm thì perceptron đầu ra giống với perceptron đầu vào và cả hai đều tạo ra lỗi bằng 0. Nếu perceptron phân loại sai điểm, perceptron đầu ra sẽ tạo ra lỗi nhỏ hơn perceptron đầu vào.

Sau đây là một thủ thuật khéo léo để cô đọng mã giả. Lưu ý đại lượng $y - \hat{y}$ là 0, -1, và +1 cho 3 trường hợp trong thủ thuật perceptron. Vì vậy, chúng ta có thể tóm tắt nó như sau:

MÃ GIẢ CHO THỦ THUẬT PERCEPTRON

Đầu vào:

- Một perceptron có trọng số a, b và độ lệch c
- Một điểm có tọa độ (x_1, x_2) và nhãn y
- Giá trị nhỏ η (tốc độ học)

Đầu ra:

- Perceptron với các trọng số mới a' , b' và độ lệch c'

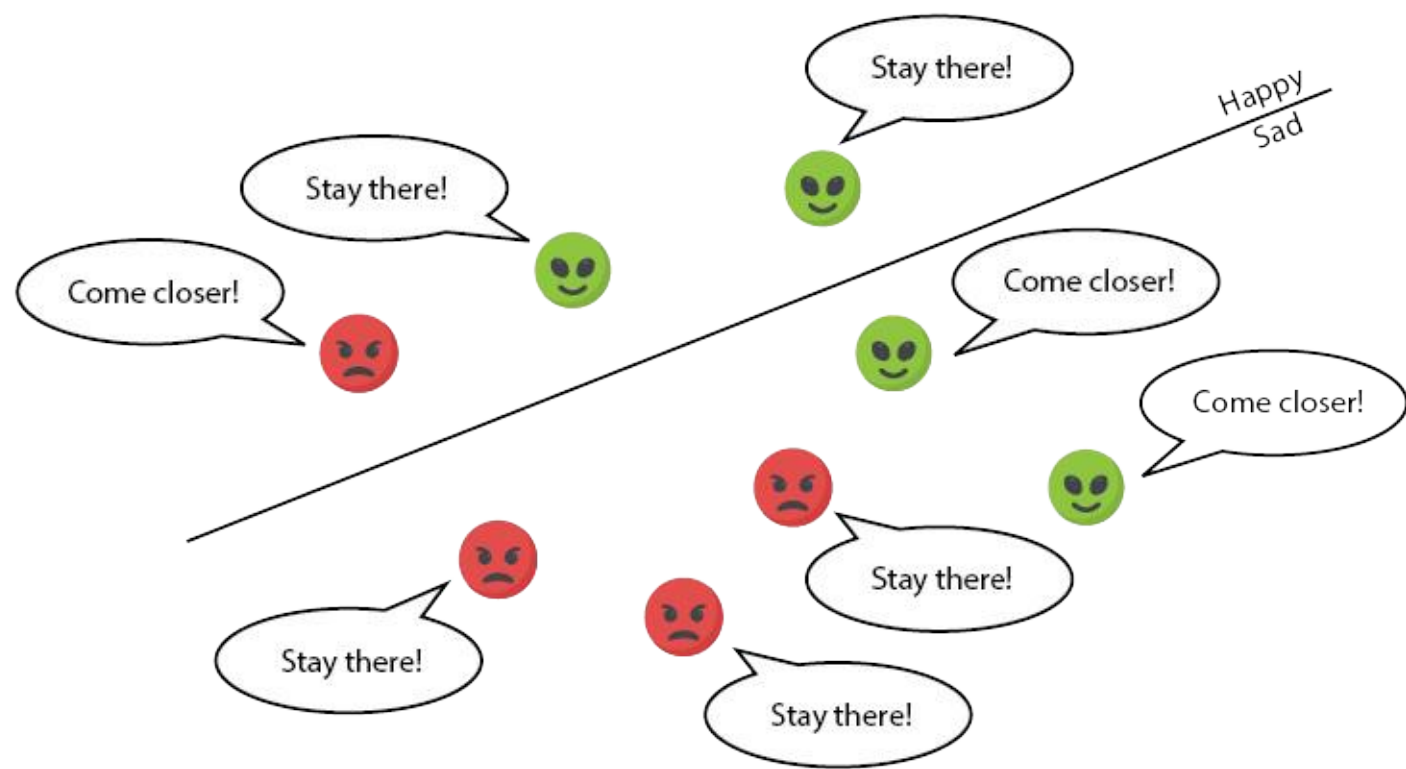
Thủ tục:

- Dự đoán mà perceptron đưa ra tại thời điểm đó là $\hat{y} = \text{bước}(ax_1 + bx_2 + c)$.
- **Trả về** perceptron với các trọng số và độ lệch sau:
 - $a' = a + \eta(y - \hat{y})x_1$
 - $b' = b + \eta(y - \hat{y})x_2$
 - $c' = c + \eta(y - \hat{y})$

Lặp lại thủ thuật perceptron nhiều lần: Thuật toán perceptron

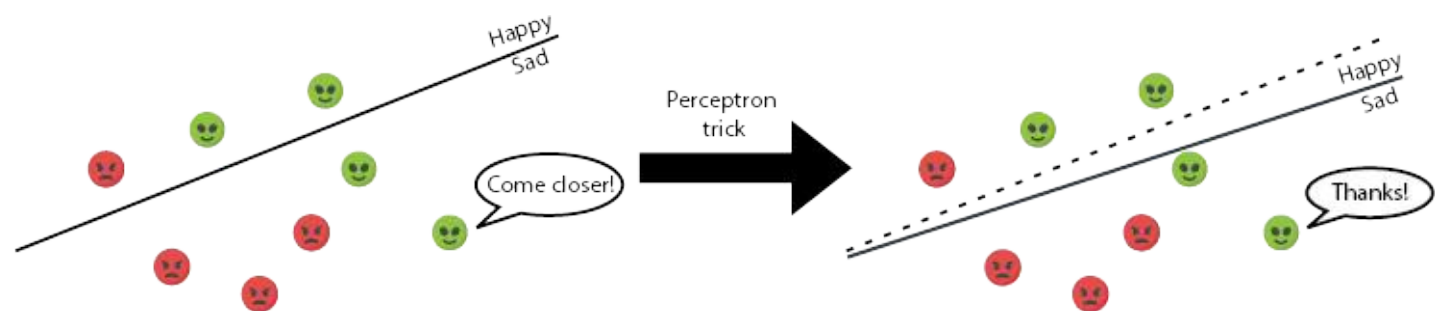
Trong phần này, chúng ta tìm hiểu *thuật toán perceptron*, được sử dụng để huấn luyện bộ phân loại perceptron trên một tập dữ liệu. Hãy nhớ lại rằng thủ thuật perceptron cho phép chúng ta cải thiện một chút perceptron để đưa ra dự đoán tốt hơn về một điểm. Thuật toán perceptron bao gồm việc bắt đầu với một bộ phân loại ngẫu nhiên và liên tục cải tiến nó bằng cách sử dụng thủ thuật perceptron nhiều lần.

Như chúng ta đã thấy trong chương này, chúng ta có thể nghiên cứu bài toán này theo hai cách: về mặt hình học và đại số. Về mặt hình học, tập dữ liệu được cho bởi các điểm trong mặt phẳng được tô màu bằng hai màu và bộ phân loại là một đường cố gắng phân chia các điểm này. Hình 5.21 chứa tập hợp các câu vui và câu buồn giống như những câu chúng ta đã thấy ở đầu chương này. Bước đầu tiên của thuật toán là vẽ một đường ngẫu nhiên. Rõ ràng là dòng trong hình 5.21 không đại diện cho một bộ phân loại perceptron tuyệt vời, bởi vì nó không làm tốt công việc phân chia câu vui và câu buồn.



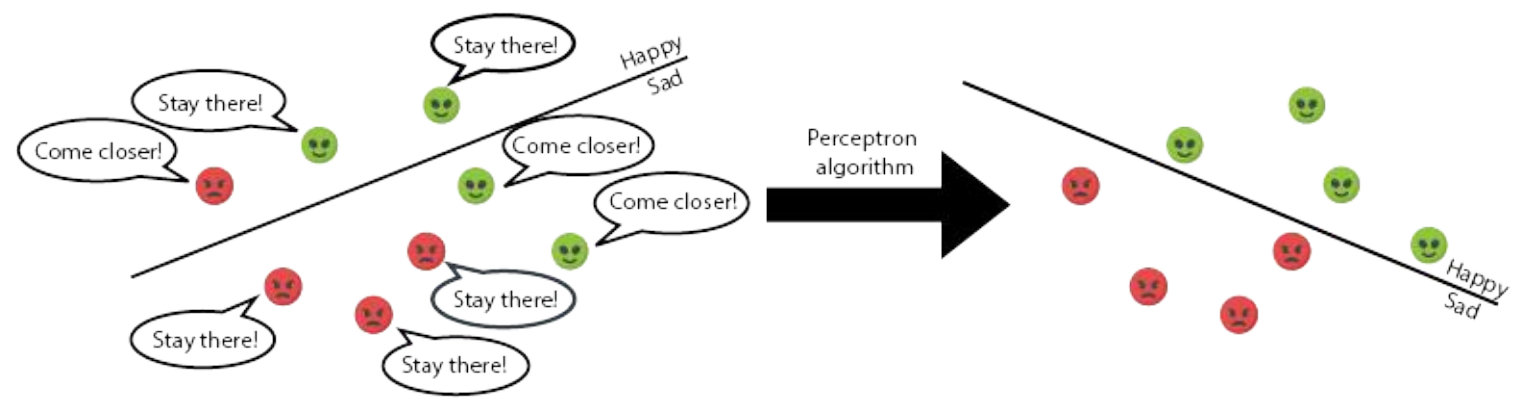
Hình 5.21 Mỗi điểm cho bộ phân loại biết phải làm gì để khiến cuộc sống của chính nó trở nên tốt đẹp hơn. Các điểm được phân loại chính xác sẽ yêu cầu dòng giữ nguyên. Các điểm bị phân loại sai yêu cầu đường di chuyển nhẹ về phía chúng.

Bước tiếp theo trong thuật toán perceptron bao gồm việc chọn một điểm ngẫu nhiên, chẳng hạn như điểm trong hình 5.22. Nếu điểm được phân loại chính xác, đường thẳng sẽ được giữ nguyên. Nếu nó bị phân loại sai thì đường này sẽ được di chuyển đến gần điểm hơn một chút, do đó làm cho đường này phù hợp hơn với điểm đó. Nó có thể trở nên kém phù hợp hơn đối với các điểm khác, nhưng hiện tại điều đó không quan trọng.



Hình 5.22 Nếu chúng ta áp dụng thủ thuật perceptron cho một bộ phân loại và một điểm bị phân loại sai, thì bộ phân loại sẽ di chuyển một chút về phía điểm đó.

Có thể tưởng tượng rằng nếu chúng ta lặp lại quá trình này nhiều lần thì cuối cùng chúng ta sẽ tìm được giải pháp tốt. Quy trình này không phải lúc nào cũng đưa chúng ta đến giải pháp tốt nhất. Nhưng trong thực tế, phương pháp này thường cho lời giải tốt như trên hình 5.23. Chúng tôi gọi đây là *thuật toán perceptron*.



Hình 5.23 Nếu chúng ta áp dụng thủ thuật perceptron nhiều lần, mỗi lần chọn một điểm ngẫu nhiên, chúng ta có thể tưởng tượng rằng chúng ta sẽ có được một bộ phân loại phân loại hầu hết các điểm một cách chính xác.

Số lần chúng tôi chạy thuật toán là số kỷ nguyên. Do đó, thuật toán này có hai siêu tham số: số kỷ nguyên và tốc độ học. Mã giả của thuật toán perceptron như sau:

Mã giả cho thuật toán Perceptron

Đầu vào:

- Tập dữ liệu các điểm, được gắn nhãn 1 và 0
- Một số kỷ nguyên, n Tỷ lệ học tập η
-

Đầu ra:

- Trình phân loại perceptron bao gồm một tập hợp các trọng số và độ lệch phù hợp với tập dữ liệu

Thủ tục:

- Bắt đầu với các giá trị ngẫu nhiên cho trọng số và độ lệch của bộ phân loại perceptron. Lặp lại
- nhiều lần: Chọn một điểm dữ liệu ngẫu nhiên. Cập nhật trọng số và độ lệch bằng thủ thuật perceptron.
 -

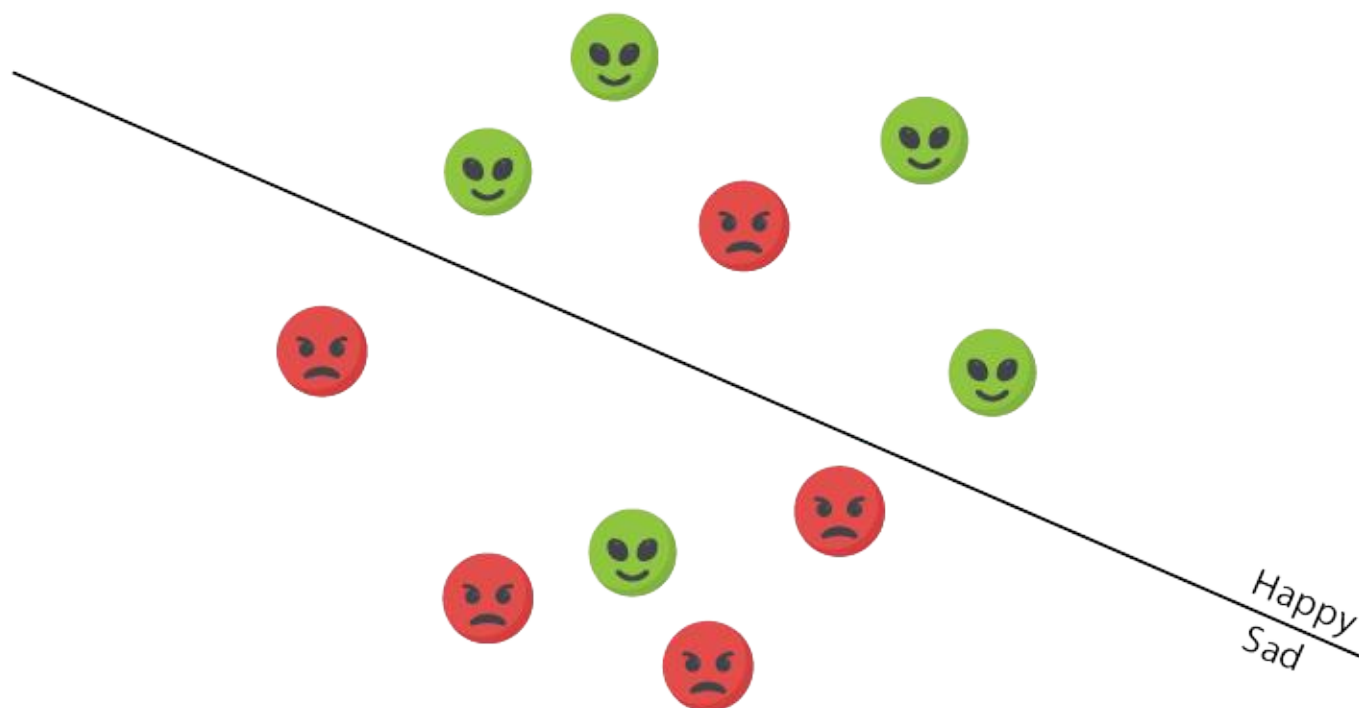
Trả về: Bộ phân loại perceptron có trọng số và độ lệch được cập nhật.

Chúng ta nên chạy vòng lặp trong bao lâu? Nói cách khác, chúng ta nên sử dụng bao nhiêu kỷ nguyên? Một số tiêu chí giúp chúng tôi đưa ra quyết định đó như sau:

- Chạy vòng lặp với số lần cố định, có thể dựa trên khả năng tính toán của chúng tôi hoặc lượng thời gian chúng tôi có. Chạy vòng lặp cho đến khi lỗi thấp hơn ngưỡng nhất định mà chúng tôi đã đặt trước. Chạy vòng lặp cho đến khi lỗi không thay đổi đáng kể trong một số kỷ nguyên nhất định.
-

Thông thường, nếu chúng ta có khả năng tính toán, bạn có thể chạy nó nhiều lần hơn mức cần thiết, bởi vì một khi chúng ta có bộ phân loại perceptron được trang bị tốt, nó có xu hướng không thay đổi nhiều. Trong phần “Mã hóa thuật toán perceptron”, chúng tôi mã hóa thuật toán perceptron và phân tích nó bằng cách đo lỗi trong từng bước, nhờ đó, chúng tôi sẽ biết rõ hơn về thời điểm nên ngừng chạy thuật toán đó.

Lưu ý rằng đối với một số trường hợp, chẳng hạn như trường hợp được hiển thị trong hình 5.24, không thể tìm thấy đường phân cách giữa hai lớp trong tập dữ liệu. Không sao cả: mục tiêu là tìm một đường phân tách tập dữ liệu với ít lỗi nhất có thể (như lỗi trong hình) và thuật toán perceptron làm tốt việc này.



Hình 5.24 Một tập dữ liệu có hai lớp không thể phân tách bằng một dòng. Thuật toán perceptron sau đó được huấn luyện để tìm ra đường phân cách chúng tốt nhất có thể.

Giảm dần độ dốc

Bạn có thể nhận thấy rằng quy trình đào tạo mô hình này trông rất quen thuộc. Trong thực tế,

nó tương tự như những gì chúng ta đã làm trong chương 3 với hồi quy tuyến tính. Hãy nhớ lại rằng mục đích của hồi quy tuyến tính là làm cho một đường thẳng càng gần một tập hợp các điểm càng tốt. Trong chương 3, chúng ta đã huấn luyện mô hình hồi quy tuyến tính của mình bằng cách bắt đầu với một đường ngẫu nhiên và thực hiện các bước nhỏ để ngày càng tiến gần hơn đến các điểm. Sau đó, chúng tôi sử dụng phép tương tự khi đi xuống từ một ngọn núi (Mount Error) bằng cách bước những bước nhỏ về phía đáy. Độ cao tại mỗi điểm trên núi là hàm lỗi perceptron trung bình, mà chúng tôi đã định nghĩa là sai số tuyệt đối hoặc sai số bình phương. Vì vậy, việc đi xuống từ núi tương đương với việc giảm thiểu sai số, tương đương với việc tìm ra đường phù hợp nhất. Chúng tôi gọi quá trình này là giảm độ dốc, bởi vì độ dốc chính xác là vectơ chỉ theo hướng tăng trưởng lớn nhất (vì vậy các điểm âm của nó theo hướng giảm lớn nhất) và thực hiện một bước theo hướng này sẽ khiến chúng ta đi xuống nhiều nhất.

Trong chương này, điều tương tự cũng xảy ra. Vấn đề của chúng tôi hơi khác một chút vì chúng tôi không muốn làm cho một đường thẳng càng gần một tập hợp điểm càng tốt. Thay vào đó, chúng ta muốn vẽ một đường phân cách hai tập hợp điểm theo cách tốt nhất có thể. Thuật toán perceptron là quá trình bắt đầu bằng một dòng ngẫu nhiên và nó di chuyển từ từ từng bước để xây dựng một dải phân cách tốt hơn. Sự tương tự với việc đi xuống từ một ngọn núi cũng có tác dụng ở đây. Điểm khác biệt duy nhất là ở ngọn núi này, độ cao tại mỗi điểm là sai số nhận thức trung bình mà chúng ta đã học trong phần “Làm thế nào để so sánh các bộ phân loại? Hàm lỗi”.

Giảm độ dốc ngẫu nhiên và hàng loạt

Cách chúng tôi phát triển thuật toán perceptron trong phần này là liên tục lấy từng điểm một và điều chỉnh perceptron (đường) để phù hợp hơn với điểm đó. Đây được gọi là một kỷ nguyên. Tuy nhiên, giống như chúng ta đã làm với hồi quy tuyến tính trong phần “Chúng ta huấn luyện sử dụng một điểm tại một thời điểm hay nhiều điểm?” trong chương 3, cách tiếp cận tốt hơn là lấy nhiều điểm cùng một lúc và điều chỉnh perceptron để phù hợp hơn với các điểm đó trong một bước. Trường hợp cực đoan là lấy tất cả các điểm trong tập hợp cùng một lúc và điều chỉnh perceptron để phù hợp hơn với tất cả chúng trong một bước. Ở phần “Chúng ta tập luyện từng điểm một hay nhiều điểm?” trong chương 3, chúng tôi gọi những phương pháp này là *ngẫu nhiên*, *lô nhỏ* và *giảm độ dốc hàng loạt*. Trong phần này, chúng tôi sử dụng thuật toán perceptron chính thức bằng cách sử dụng phương pháp giảm độ dốc theo lô nhỏ. Các chi tiết toán học xuất hiện trong phụ lục B, phần “Sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình phân loại”, trong đó thuật toán perceptron được mô tả một cách tổng quát đầy đủ bằng cách sử dụng phương pháp giảm độ dốc theo lô nhỏ.

Coding the perceptron algorithm

Now that we have developed the perceptron algorithm for our sentiment analysis application, in this section we write the code for it. First we'll write the code from scratch to fit our original dataset, and then we'll use Turi Create. In real life, we always use a package and have little need to code our own algorithms. However, it's good to code some of the algorithms at least once—think of it as doing long division. Although we usually don't do long division without using a calculator, it's good we had to in high school, because now when we do it using a calculator, we know what's happening in the background. The code for this section follows, and the dataset we use is shown in table 5.5:

- **Notebook:** Coding_perceptron_algorithm.ipynb
 - https://github.com/luisguiserrano/manning/blob/master/Chapter_5_Perceptron

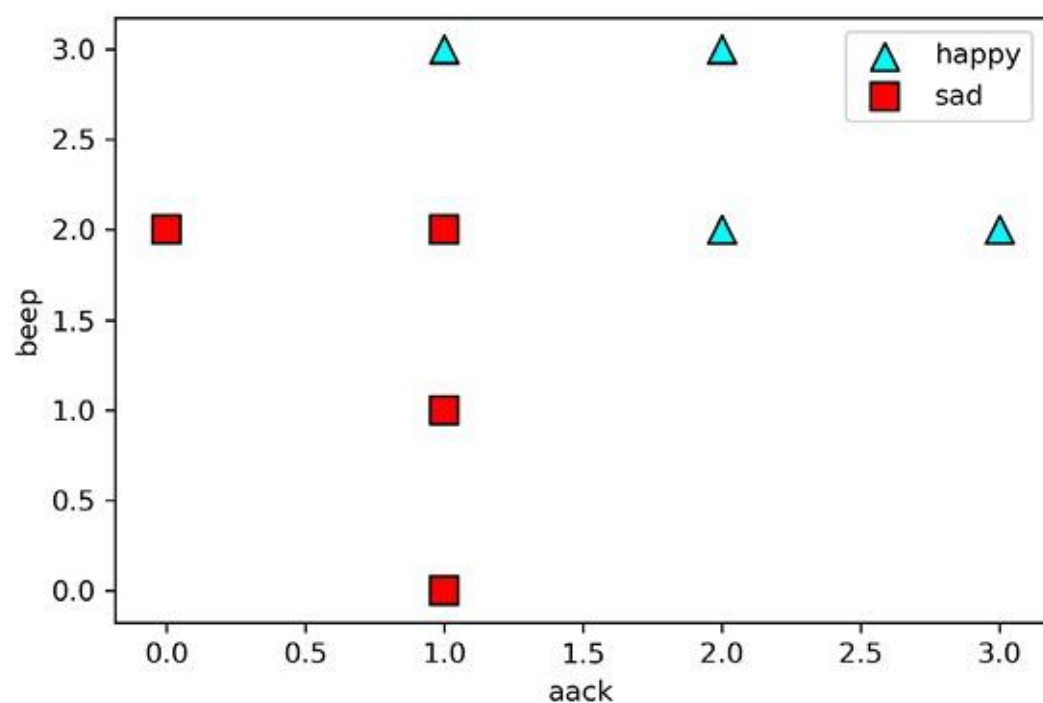
Table 5.5 A dataset of aliens, the times they said each of the words, and their mood.

Aack	Beep	Happy/Sad
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

Let's begin by defining our dataset as a NumPy array. The features correspond to two numbers corresponding to the appearances of *aack* and *beep*. The labels are 1 for the happy sentences and 0 for the sad ones.

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```

This gives us the plot in figure 5.25. In this figure, the happy sentences are triangles, and the sad ones are squares.



Hình 5.25 Sơ đồ của tập dữ liệu của chúng tôi. Hình tam giác là người ngoài hành tinh vui vẻ, còn hình vuông là người ngoài hành tinh buồn bã.

Mã hóa thủ thuật perceptron

Trong phần này, chúng tôi viết mã thủ thuật perceptron. Chúng ta sẽ mã hóa nó bằng cách sử dụng phương pháp giảm độ dốc ngẫu nhiên (mỗi lần một điểm), nhưng chúng ta cũng có thể mã hóa nó bằng cách sử dụng phương pháp giảm độ dốc theo lô hoặc theo đợt. Chúng tôi bắt đầu bằng cách mã hóa hàm điểm và dự đoán. Cả hai hàm đều nhận được cùng một đầu vào, đó là trọng số của mô hình, độ lệch và các đặc điểm của một điểm dữ liệu. Hàm điểm trả về điểm mà mô hình đưa ra cho điểm dữ liệu đó và hàm dự đoán trả về 1 nếu điểm lớn hơn hoặc bằng 0 và 0 nếu điểm nhỏ hơn 0. Đối với hàm này, chúng ta sử dụng tích số chấm được xác định trong phần “Vẽ hàm lỗi và biết khi nào nên dừng chạy thuật toán” ở chương 3.

```
def score(weights, bias, features):
    return np.dot(features, weights) + bias ❶
```

❶ Tính tích số chấm giữa trọng số và đặc điểm, cộng độ lệch và áp dụng hàm bước

Để viết hàm dự đoán, trước tiên chúng ta viết hàm bước. Dự đoán là hàm bước của điểm số.

```
def step(x):
    if x >= 0:
        return 1
    else:
```

```
return 0
```

```
def prediction(weights, bias, features):  
    return step(score(weights, bias, features)) ❶
```

❑ Xem điểm và nếu nó dương hoặc bằng 0, trả về 1; nếu nó âm, trả về 0

Tiếp theo, chúng ta mã hóa hàm lỗi cho một điểm. Hãy nhớ lại rằng sai số bằng 0 nếu điểm được phân loại chính xác và giá trị tuyệt đối của điểm nếu điểm đó được phân loại sai. Hàm này lấy trọng số và độ lệch của mô hình cũng như các đặc điểm và nhãn của điểm dữ liệu làm đầu vào.

```
def error(weights, bias, features, label):  
    pred = prediction(weights, bias, features)  
    if pred == label: ❶  
        return 0  
    else: ❷  
        return np.abs(score(weights, bias, features))
```

❑ Nếu dự đoán bằng nhãn thì điểm đó được phân loại tốt, nghĩa là sai số bằng 0. ❑ Nếu dự đoán khác với nhãn thì điểm đó bị phân loại sai, nghĩa là sai số bằng giá trị tuyệt đối của điểm.

Bây giờ chúng ta viết một hàm cho lỗi perceptron trung bình. Hàm này tính toán mức trung bình của các lỗi của tất cả các điểm trong tập dữ liệu của chúng tôi.

```
def mean_perceptron_error(weights, bias, features, labels):  
    total_error = 0  
    for i in range(len(features)): ❶  
        total_error += error(weights, bias, features[i], labels[i])  
    return total_error/len(features) ❷
```

❑ Lặp lại dữ liệu của chúng tôi và đối với mỗi điểm, thêm lỗi tại điểm đó, sau đó trả về lỗi này ❑ Tổng số lỗi được chia cho số điểm để có được lỗi perceptron trung bình.

Bây giờ chúng ta đã có hàm lỗi, chúng ta có thể tiếp tục viết mã thủ thuật perceptron. Chúng ta sẽ mã hóa phiên bản thu gọn của thuật toán được tìm thấy ở cuối phần “Thủ thuật perceptron”. Tuy nhiên, trong sổ ghi chép, bạn có thể thấy nó được mã hóa theo cả hai cách, cách đầu tiên sử dụng câu lệnh để kiểm tra xem điểm đó có phải là

if

được phân loại tốt.

```
def perceptron_trick(weights, bias, features, label, learning_rate = 0.01):  
    pred = prediction(weights, bias, features)  
    for i in range(len(weights)):  
        weights[i] += (label-pred)*features[i]*learning_rate ❑  
    bias += (label-pred)*learning_rate    return weights, bias
```

❑ Cập nhật trọng số và độ lệch bằng thủ thuật perceptron

Mã hóa thuật toán perceptron

Bây giờ chúng ta đã có thủ thuật perceptron, chúng ta có thể mã hóa thuật toán perceptron. Hãy nhớ lại rằng thuật toán perceptron bao gồm việc bắt đầu bằng một bộ phân loại perceptron ngẫu nhiên và lặp lại thủ thuật perceptron nhiều lần (nhiều lần bằng số kỷ nguyên).Để theo dõi hiệu suất của thuật toán, chúng tôi cũng sẽ theo dõi lỗi nhận thức trung bình ở mỗi kỷ nguyên.Là đầu vào, chúng tôi có dữ liệu (tính năng và nhãn), tốc độ học tập mà chúng tôi mặc định là 0,01 và số lượng kỷ nguyên mà chúng tôi mặc định là 200. Mã cho thuật toán perceptron như sau:

```
def perceptron_algorithm(features, labels, learning_rate = 0.01, epochs = 200):
    weights = [1.0 for i in range(len(features[0]))] ❶
    bias = 0.0
    errors = [] ❷
    for epoch in range(epochs): ❸
        error = mean_perceptron_error(weights, bias, features, labels) ❹
        errors.append(error)
        i = random.randint(0, len(features)-1) ❺
        weights, bias = perceptron_trick(weights, bias, features[i], labels[i])❻
    return weights, bias, errors
```

❑ Khởi tạo trọng số thành 1 và độ lệch thành 0. Bạn có thể thoải mái khởi tạo chúng thành các số ngẫu nhiên nhỏ nếu bạn muốn. ❑ Một mảng để lưu trữ các lỗi ❑ Lặp lại quy trình nhiều lần theo số kỷ nguyên ❑ Tính toán sai số trung bình của perceptron và lưu trữ nó ❑ Chọn một điểm ngẫu nhiên trong tập dữ liệu của chúng ta ❑ Áp dụng thuật toán perceptron để cập nhật trọng số và độ lệch của mô hình của chúng ta dựa trên điểm đó

Bây giờ hãy chạy thuật toán trên tập dữ liệu của chúng tôi!

```
perceptron_algorithm(features, labels)
Output: ([0.6299999999999997, 0.17999999999999938], -1.0400000000000007)
```

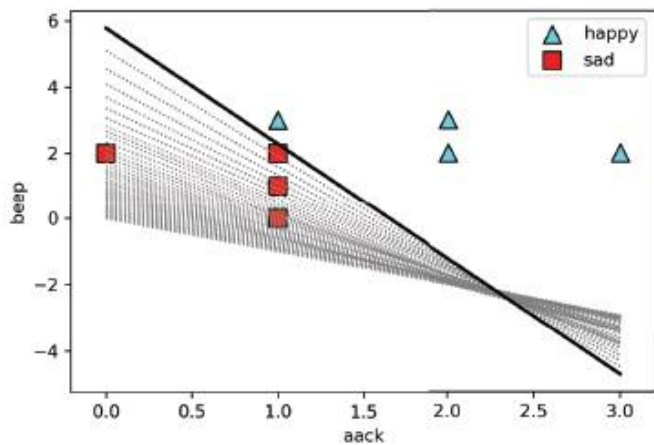
Kết quả đầu ra cho thấy trọng số và độ lệch mà chúng tôi thu được như sau:

- Trọng lượng của *aack*:
- 0,63 Trọng lượng của *bíp*:
- 0,18 Độ lệch: −1,04

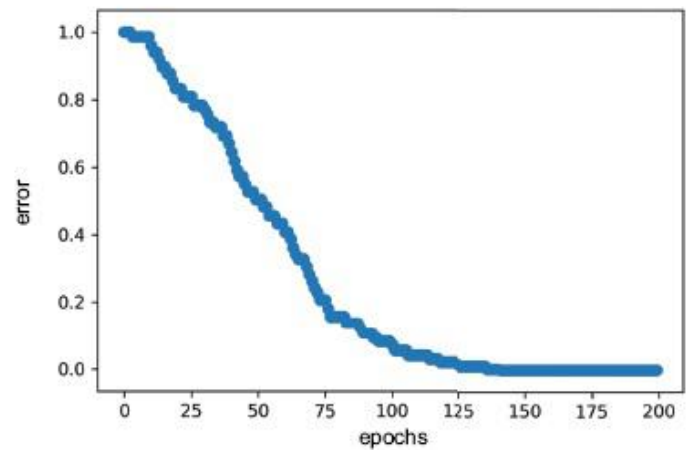
Chúng ta có thể có một câu trả lời khác vì tính ngẫu nhiên trong việc lựa chọn các điểm bên trong thuật toán.Để mã trong kho lưu trữ luôn trả về cùng một câu trả lời, hạt giống ngẫu nhiên được đặt thành 0.

chức năng. Đường tương ứng với perceptron thu được là đường dày, phân loại chính xác mọi điểm. Các đường mảnh hơn là các đường tương ứng với các perceptron thu được sau mỗi 200 kỷ nguyên. Lưu ý tại mỗi thời điểm, đường thẳng trở nên phù hợp hơn với các điểm. Lỗi giảm (hầu hết) khi chúng tôi tăng số lượng kỷ nguyên, cho đến khi nó đạt đến 0 vào khoảng kỷ nguyên 140, nghĩa là mọi điểm đều được phân loại chính xác.

Classifier



Error



Hình 5.26 Bên trái: Sơ đồ phân loại kết quả của chúng tôi. Lưu ý rằng nó phân loại chính xác từng điểm. Đúng: Sơ đồ lỗi. Lưu ý rằng chúng ta chạy thuật toán perceptron càng nhiều kỷ nguyên thì lỗi càng thấp.

Đó là mã cho thuật toán perceptron! Như tôi đã đề cập trước đây, trong thực tế, chúng tôi thường không viết mã thuật toán bằng tay mà sử dụng một gói, chẳng hạn như Turi Create hoặc Scikit-Learn. Đây là những gì chúng tôi đề cập trong phần tiếp theo.

Mã hóa thuật toán perceptron bằng Turi Create

Trong phần này, chúng ta tìm hiểu cách viết mã thuật toán perceptron trong Turi Create. Mã nằm trong cùng số ghi chép với bài tập trước. Nhiệm vụ đầu tiên của chúng tôi là nhập Turi Tạo và tạo SFrame với dữ liệu của chúng tôi từ từ điển như sau:

```
import turicreate as tc

datadict = {'aack': features[:,0], 'beep': features[:,1], 'prediction': labels}
data = tc.SFrame(datadict)
```

Tiếp theo, chúng ta tạo và huấn luyện bộ phân loại perceptron bằng cách sử dụng đối tượng và phương thức như minh họa trong đoạn mã tiếp theo. Đầu vào là tập dữ liệu và tên cột chứa nhãn

(mục tiêu).

```
perceptron = tc.logistic_classifier.create(data, target='prediction')
```

Đầu ra:

Iteration	Passes	Elapsed Time	Training Accuracy
1	2	1.003120	1.000000
2	3	1.004235	1.000000
3	4	1.004840	1.000000
4	5	1.005574	1.000000

SUCCESS: Optimal solution found.

Lưu ý rằng thuật toán perceptron đã chạy trong bốn kỷ nguyên và trong kỷ nguyên cuối cùng (trên thực tế là trong tất cả các kỷ nguyên đó), nó có độ chính xác huấn luyện là 1. Điều này có nghĩa là mọi điểm trong tập dữ liệu đều được phân loại chính xác.

Cuối cùng, chúng ta có thể xem trọng số và độ lệch của mô hình bằng cách sử dụng lệnh sau:

```
perceptron.coefficients
```

Đầu ra của hàm này hiển thị các trọng số và độ lệch sau cho perceptron kết quả:

- Trọng lượng của *aack*:
- 2,70 Trọng lượng của *bíp*:
- 2,46 Độ lệch: -8,96

Đây là những kết quả khác với kết quả chúng tôi thu được bằng tay, nhưng cả hai perceptron đều hoạt động tốt trong tập dữ liệu.

Ứng dụng của thuật toán perceptron

Thuật toán perceptron có nhiều ứng dụng trong đời sống.Hầu như mỗi khi chúng ta cần trả lời một câu hỏi có hoặc không, trong đó câu trả lời được dự đoán từ dữ liệu trước đó, thuật toán perceptron có thể giúp chúng ta.Dưới đây là một số ví dụ về ứng dụng thực tế của thuật toán perceptron.

Bộ lọc email rác

Tương tự như cách chúng ta dự đoán một câu vui hay buồn dựa trên các từ trong câu, chúng ta có thể dự đoán một email là thư rác hay không phải thư rác dựa trên các từ trong email. Chúng ta cũng có thể sử dụng các tính năng khác, chẳng hạn như sau:

- Độ dài của email Kích thước của tệp đính kèm Số
- lượng người gửi Liệu có bất kỳ liên hệ nào của
- chúng tôi là người gửi hay không
-

Hiện tại, thuật toán perceptron (và các đối tác tiên tiến hơn của nó, hồi quy logistic và mạng thần kinh) và các mô hình phân loại khác được hầu hết các nhà cung cấp email lớn nhất sử dụng như một phần của quy trình phân loại thư rác, với kết quả tuyệt vời.

Chúng ta cũng có thể phân loại email bằng các thuật toán phân loại như thuật toán perceptron. Việc phân loại email thành cá nhân, đăng ký và quảng cáo cũng là một vấn đề tương tự. Ngay cả việc đưa ra các phản hồi tiềm năng cho một email cũng là một vấn đề về phân loại, ngoại trừ việc hiện nay các nhãn mà chúng tôi sử dụng là các phản hồi cho email.

Hệ thống khuyến nghị

Trong nhiều hệ thống đề xuất, việc giới thiệu video, phim, bài hát hoặc sản phẩm cho người dùng sẽ chỉ là câu trả lời có hoặc không. Trong những trường hợp này, câu hỏi có thể là một trong những câu sau:

- Người dùng có nhấp vào video/phim mà chúng tôi đề xuất không? Người dùng
- có xem toàn bộ video/phim mà chúng tôi đề xuất không? Người dùng có nghe bài
- hát chúng tôi giới thiệu không? Liệu người dùng có mua sản phẩm chúng tôi giới
- thiệu không?

Các tính năng có thể là bất kỳ thứ gì, từ nhân khẩu học (tuổi, giới tính, vị trí của người dùng), đến hành vi (người dùng đã xem video gì, họ đã nghe bài hát nào, họ đã mua sản phẩm gì?). Bạn có thể tưởng tượng rằng vector người dùng sẽ dài! Để làm được điều này, cần có sức mạnh tính toán lớn và việc triển khai các thuật toán rất thông minh.

Các công ty như Netflix, YouTube và Amazon, cùng nhiều công ty khác, sử dụng thuật toán perceptron hoặc các mô hình phân loại tương tự, nâng cao hơn trong hệ thống đề xuất của họ.

Chăm sóc sức khỏe

Nhiều mô hình y tế cũng sử dụng các thuật toán phân loại như thuật toán perceptron để trả lời các câu hỏi như sau:

- Bệnh nhân có mắc một căn bệnh cụ thể nào không?
- Liệu một phương pháp điều trị nhất định có hiệu quả với bệnh nhân không?

Các đặc điểm của những mô hình này thường là các triệu chứng mà bệnh nhân đang mắc phải và tiền sử bệnh của họ. Đối với các loại thuật toán này, người ta cần mức hiệu suất rất cao. Việc đề xuất phương pháp điều trị sai cho bệnh nhân còn nghiêm trọng hơn nhiều so với việc giới thiệu một video mà người dùng không xem. Đối với loại phân tích này, hãy tham khảo chương 7, nơi chúng tôi nói về độ chính xác và các cách khác để đánh giá các mô hình phân loại.

Thị giác máy tính

Các thuật toán phân loại như thuật toán perceptron được sử dụng rộng rãi trong thị giác máy tính, cụ thể hơn là trong nhận dạng hình ảnh. Hãy tưởng tượng rằng chúng ta có một bức ảnh và chúng ta muốn dạy cho máy tính biết liệu bức ảnh đó có chứa một con chó hay không. Đây là mô hình phân loại trong đó các đặc điểm là các pixel của hình ảnh.

Thuật toán perceptron có hiệu suất tốt trong các bộ dữ liệu hình ảnh được quản lý như MNIST, đây là bộ dữ liệu gồm các chữ số viết tay. Tuy nhiên, đối với những hình ảnh phức tạp hơn, nó không hoạt động tốt lắm. Đối với những điều này, người ta sử dụng các mô hình bao gồm sự kết hợp của nhiều perceptron. Những mô hình này được gọi một cách khéo léo là perceptron đa lớp, hay mạng lưới thần kinh, và chúng ta sẽ tìm hiểu chi tiết về chúng trong chương 10.

Tóm tắt

- Phân loại là một phần quan trọng của học máy. Nó tương tự như hồi quy ở chỗ nó bao gồm việc huấn luyện một thuật toán với dữ liệu được gắn nhãn và sử dụng nó để đưa ra dự đoán về dữ liệu (không được gắn nhãn) trong tương lai. Sự khác biệt so với hồi quy là trong phân loại, các dự đoán là các danh mục, chẳng hạn như có/không, spam/ham, v.v.
- Trình phân loại Perceptron hoạt động bằng cách gán trọng số cho từng đặc điểm và độ lệch. Điểm của một điểm dữ liệu được tính bằng tổng của các trọng số và đặc điểm, cộng với độ lệch. Nếu điểm lớn hơn hoặc bằng 0, bộ phân loại sẽ dự đoán là có. Ngược lại, nó dự đoán là không.
- Để phân tích tình cảm, một perceptron bao gồm điểm cho mỗi từ trong từ điển, cùng với độ lệch. Những lời nói vui vẻ thường được điểm tích cực và những lời buồn thường được điểm âm. Những từ trung tính như *the* có thể sẽ có số điểm gần bằng 0.
- Sự thiên vị giúp chúng ta quyết định xem câu trống đó là vui hay buồn. Nếu thiên vị là tích cực thì câu trống là vui, còn nếu nó là tiêu cực thì câu trống là buồn.
- Về mặt đồ họa, chúng ta có thể thấy perceptron là một đường cố gắng phân tách hai loại điểm, có thể được coi là các điểm có hai màu khác nhau. Ở các chiều cao hơn, perceptron là một điểm phân tách siêu phẳng.
- Thuật toán perceptron hoạt động bằng cách bắt đầu bằng một đường ngẫu nhiên và sau đó di chuyển từ từ để phân tách các điểm rõ ràng. Trong mỗi lần lặp, nó chọn một điểm ngẫu nhiên. Nếu điểm được phân loại chính xác, đường sẽ không di chuyển. Nếu nó bị phân loại sai thì đường thẳng sẽ di chuyển đến gần điểm hơn một chút để vượt qua nó và phân loại nó một cách chính xác.
- Thuật toán perceptron có nhiều ứng dụng, bao gồm phát hiện email spam, hệ thống đề xuất, thương mại điện tử và chăm sóc sức khỏe.

Bài tập

BÀI TẬP 5.1

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán (D)
Bệnh nhân 1		X	X	X	Bệnh
Bệnh nhân 2	X	X		X	Bệnh
Bệnh nhân 3	X		X	X	Bệnh
Bệnh nhân 4	X	X	X		Bệnh
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

Xây dựng mô hình perceptron để phân loại tập dữ liệu này.

gợi ý Bạn có thể sử dụng thuật toán perceptron, nhưng bạn có thể nhắm đến một mô hình perceptron tốt hoạt động được.

BÀI TẬP 5.2

Xét mô hình perceptron gán cho điểm (x_1, x_2) dự đoán $\hat{y} =$
bước $(2x_1 + 3x_2 - 4)$.Mô hình này có dạng đường biên với phương trình $2x_1 + 3x_2 - 4$
 $=0$. Ta có điểm $p =(1, 1)$ có nhãn 0.

Một. Xác minh rằng điểm p bị mô hình phân loại sai. b. Tính sai số perceptron mà mô hình tạo ra tại điểm p . c. Sử dụng thủ thuật perceptron để thu được mô hình mới vẫn phân loại sai p nhưng tạo ra lỗi nhỏ hơn.Bạn có thể sử dụng $\eta = 0,01$ làm tốc độ học tập.

- d. Find the prediction given by the new model at the point p , and verify that the perceptron error obtained is smaller than the original.

EXERCISE 5.3

Perceptrons are particularly useful for building logical gates such as AND and OR.

- a. Build a perceptron that models the AND gate. In other words, build a perceptron to fit the following dataset (where x_1 , x_2 are the features and y is the label):

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- b. Similarly, build a perceptron that models the OR gate, given by the following dataset:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

- c. Show that there is no perceptron that models the XOR gate, given by the following dataset:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

6 Một cách tiếp cận liên tục để phân chia các điểm: Phân loại logistic

Trong chương này

- sự khác biệt giữa phép gán cứng và phép gán mềm trong mô hình phân loại hàm sigmoid, hàm kích hoạt liên tục Perceptron rời rạc so với perceptron liên tục, còn được gọi là bộ phân loại logistic
- thuật toán hồi quy logistic để phân loại mã hóa dữ liệu thuật toán hồi quy logistic trong Python sử dụng bộ phân loại logistic trong Turi Tạo để phân tích cảm tính của các bài đánh giá phim bằng cách sử dụng hàm softmax để xây dựng bộ phân loại cho nhiều hơn hai lớp
-
-
-
-



Ở chương trước, chúng ta đã xây dựng một bộ phân loại để xác định xem một câu là vui hay buồn. Nhưng như chúng ta có thể tưởng tượng, một số câu hạnh phúc hơn những câu khác. Ví dụ: câu “Tôi ổn” và câu “Hôm nay là ngày tuyệt vời nhất trong cuộc đời tôi!” cả hai đều hạnh phúc, tuy nhiên người thứ hai lại hạnh phúc hơn người thứ nhất nhiều. Sẽ thật tuyệt nếu có một bộ phân loại không chỉ dự đoán các câu vui hay buồn mà còn đưa ra đánh giá về mức độ hạnh phúc của các câu—chẳng hạn, một bộ phân loại cho chúng ta biết rằng câu đầu tiên là 60% hạnh phúc và câu thứ hai là 95% hạnh phúc? Trong chương này, chúng tôi định nghĩa *bộ phân loại logistic* để thực hiện chính xác điều đó. Bộ phân loại này gán điểm từ 0 đến 1 cho mỗi câu, theo cách mà

câu nào càng vui thì điểm nhận được càng cao.

Tóm lại, bộ phân loại logistic là một loại mô hình hoạt động giống như bộ phân loại perceptron, ngoại trừ thay vì trả về câu trả lời có hoặc không, nó trả về một số từ 0 đến 1. Trong trường hợp này, mục tiêu là gán điểm gần 0 cho những câu buồn nhất, điểm gần 1 cho những câu vui vẻ nhất và điểm gần 0,5 cho những câu trung lập. Ngưỡng 0,5 này là phổ biến trong thực tế, mặc dù tùy ý. Trong chương 7, chúng ta sẽ xem cách điều chỉnh nó để tối ưu hóa mô hình của mình, nhưng đối với chương này, chúng ta sử dụng 0,5.

Chương này dựa trên chương 5 vì các thuật toán chúng tôi phát triển ở đây tương tự nhau, ngoại trừ một số khác biệt về mặt kỹ thuật. Đảm bảo rằng bạn hiểu rõ chương 5 sẽ giúp bạn hiểu được nội dung trong chương này. Trong chương 5, chúng tôi đã mô tả thuật toán perceptron bằng cách sử dụng hàm lỗi cho chúng ta biết trình phân loại perceptron hoạt động tốt như thế nào và một bước lặp để biến trình phân loại thành trình phân loại tốt hơn một chút. Trong chương này, chúng ta tìm hiểu thuật toán hồi quy logistic, hoạt động theo cách tương tự. Sự khác biệt chính sau:

- Hàm bước được thay thế bằng hàm kích hoạt mới, trả về các giá trị từ 0 đến 1. Hàm lỗi perceptron được thay thế bằng hàm lỗi mới, dựa trên tính toán xác suất. Thủ thuật perceptron được thay thế bằng thủ thuật mới, giúp cải thiện bộ phân loại dựa trên hàm lỗi mới này.
-

ngoài ra Trong chương này chúng ta thực hiện rất nhiều phép tính số. Nếu bạn làm theo các phương trình, bạn có thể thấy rằng các phép tính của bạn khác với những tính toán trong sách một chút. Cuốn sách làm tròn các số ở cuối phương trình chứ không phải ở giữa các bước. Tuy nhiên, điều này sẽ có rất ít ảnh hưởng đến kết quả cuối cùng.

Ở cuối chương này, chúng tôi áp dụng kiến thức của mình vào tập dữ liệu thực tế về các bài đánh giá phim trên trang web nổi tiếng IMDB (www.imdb.com). Chúng tôi sử dụng bộ phân loại logistic để dự đoán xem các bài đánh giá phim là tích cực hay tiêu cực.

Mã cho chương này có sẵn trong kho GitHub sau:

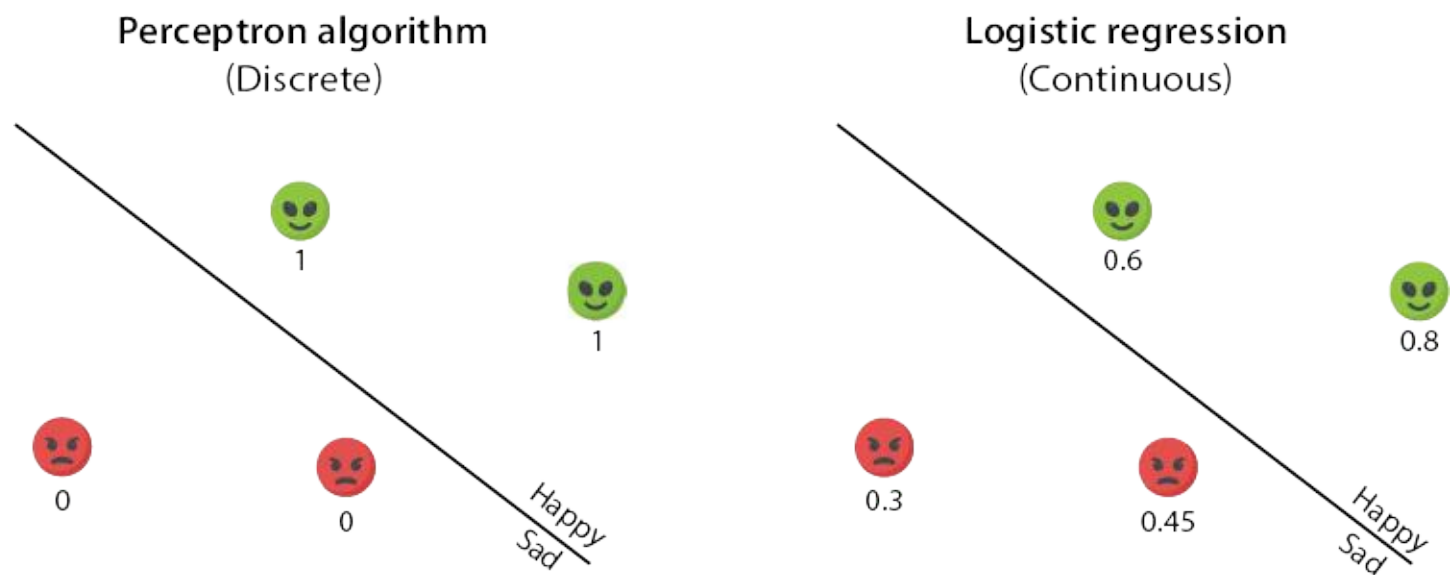
https://github.com/luisguiserrano/manning/tree/master/Chapter_6_Logistic_Regressi

Bộ phân loại logistic: Phiên bản liên tục của

bộ phân loại nhận thức

Trong chương 5, chúng ta đã đề cập đến perceptron, một loại bộ phân loại sử dụng các đặc điểm của dữ liệu của chúng ta để đưa ra dự đoán. Dự đoán có thể là 1 hoặc 0. Đây được gọi là *perceptron rời rạc*, vì nó trả về câu trả lời từ một tập hợp rời rạc (tập hợp chứa 0 và 1). Trong chương này, chúng ta tìm hiểu các *perceptron liên tục*, trả về câu trả lời có thể là số bất kỳ trong khoảng từ 0 đến 1. Tên phổ biến hơn cho các perceptron liên tục là *bộ phân loại logistic*. Đầu ra của bộ phân loại logistic có thể được hiểu là một điểm số và mục tiêu của bộ phân loại logistic là gán điểm càng gần nhãn của các điểm càng tốt—các điểm có nhãn 0 sẽ có điểm gần bằng 0 và các điểm có nhãn 1 sẽ có điểm gần bằng 1.

Chúng ta có thể hình dung các perceptron liên tục tương tự như các perceptron rời rạc: bằng một đường thẳng (hoặc mặt phẳng nhiều chiều) ngăn cách hai lớp dữ liệu. Sự khác biệt duy nhất là perceptron rời rạc dự đoán rằng mọi thứ ở một bên của đường thẳng có nhãn 1 và phía bên kia có nhãn 0, trong khi perceptron liên tục gán giá trị từ 0 đến 1 cho tất cả các điểm dựa trên vị trí của chúng đối với đường thẳng. Mỗi điểm trên đường có giá trị 0,5. Giá trị này có nghĩa là mô hình không thể quyết định câu đó là vui hay buồn. Ví dụ: trong ví dụ phân tích tình cảm đang diễn ra, câu “Hôm nay là thứ Ba” không vui cũng không buồn nên mô hình sẽ cho điểm gần 0,5. Các điểm trong vùng dương có điểm lớn hơn 0,5, trong đó các điểm thậm chí ở xa đường 0,5 theo hướng dương sẽ nhận giá trị gần hơn với 1. Các điểm trong vùng âm có điểm nhỏ hơn 0,5, trong đó, một lần nữa, các điểm ở xa đường 0,5 có giá trị gần 0 hơn. Không có điểm nào nhận giá trị 1 hoặc 0 (trừ khi chúng ta xem xét các điểm ở vô cực), như trong hình 6.1.



Hình 6.1 Bên trái: Thuật toán perceptron huấn luyện một perceptron rời rạc, trong đó các dự đoán là 0 (vui) và 1 (buồn).Đúng: Thuật toán hồi quy logistic huấn luyện một perceptron liên tục, trong đó các dự đoán là các số từ 0 đến 1 biểu thị mức độ hạnh phúc được dự đoán.

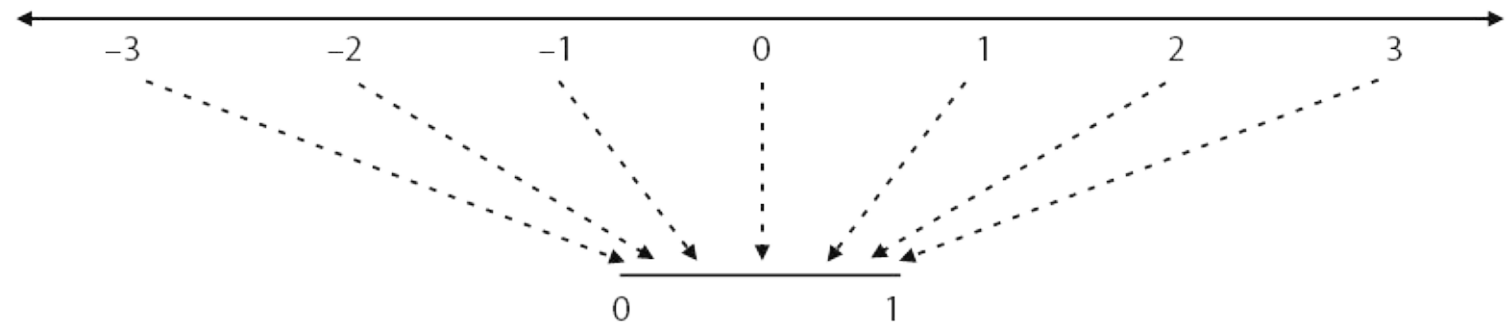
Tại sao chúng tôi gọi đây là *phân loại* thay vì *hồi quy*, vì bộ phân loại logistic không xuất ra một trạng thái mà là một con số? Lý do là sau khi chấm điểm, chúng ta có thể phân thành hai loại, đó là điểm có điểm từ 0,5 trở lên và điểm có điểm thấp hơn 0,5. Về mặt đồ họa, hai lớp được phân tách bằng đường ranh giới, giống như bộ phân loại perceptron. Tuy nhiên, thuật toán chúng tôi sử dụng để huấn luyện bộ phân loại logistic được gọi là *thuật toán hồi quy logistic*. Ký hiệu này hơi đặc biệt một chút, nhưng chúng tôi sẽ giữ nguyên như vậy để phù hợp với tài liệu.

Phương pháp xác suất để phân loại: Hàm sigmoid

Làm cách nào để sửa đổi một chút các mô hình perceptron từ phần trước để tính điểm cho mỗi câu, thay vì chỉ đơn giản là “vui” hay “buồn”? Hãy nhớ lại cách chúng ta đưa ra dự đoán trong các mô hình perceptron. Chúng tôi cho điểm từng câu bằng cách cho điểm riêng từng từ và cộng điểm, cộng với độ lệch. Nếu điểm dương, chúng tôi dự đoán câu đó là câu vui, còn nếu điểm âm, chúng tôi dự đoán câu đó là câu buồn. Nói cách khác, những gì chúng tôi đã làm là áp dụng hàm bước cho điểm số. Hàm bước trả về 1 nếu điểm không âm và 0 nếu điểm âm.

Bây giờ chúng ta làm điều gì đó tương tự. Chúng ta lấy một hàm nhận điểm làm

nhập và xuất một số từ 0 đến 1. Số này gần bằng 1 nếu điểm là dương và gần bằng 0 nếu điểm là âm. Nếu điểm bằng 0 thì đầu ra là 0,5. Hãy tưởng tượng nếu bạn có thể lấy toàn bộ trục số và chia nó thành khoảng từ 0 đến 1. Nó sẽ trông giống như hàm số trong hình 6.2.



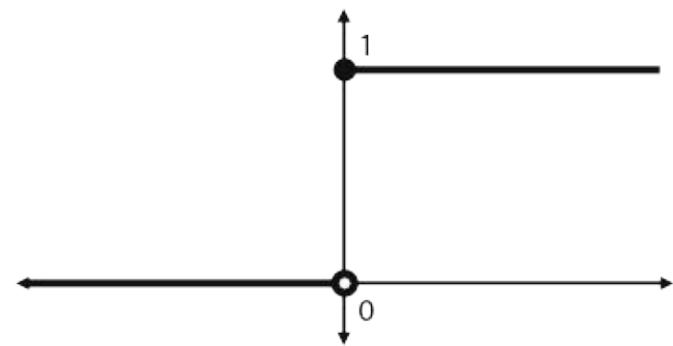
Hình 6.2 Hàm sigmoid gửi toàn bộ dãy số tới khoảng (0,1).

Nhiều hàm có thể giúp chúng ta ở đây và trong trường hợp này, chúng ta sử dụng một hàm gọi là *sigmoid*, được biểu thị bằng chữ cái Hy Lạp *sigma* (σ). Công thức của sigmoid như sau:

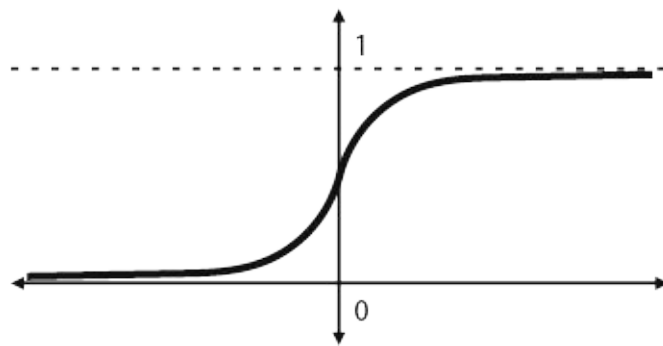
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Điều thực sự quan trọng ở đây không phải là công thức mà là chức năng của hàm, tức là biến dòng số thực thành khoảng (0,1). Trong hình 6.3, chúng ta có thể thấy sự so sánh giữa đồ thị bước và hàm sigmoid.

Step function
(discrete)



Sigmoid function
(continuous)



Hình 6.3 Bên trái: Hàm bước được sử dụng để xây dựng các perceptron rời rạc. Nó xuất ra giá trị 0 cho bất kỳ đầu vào âm nào và giá trị 1 cho bất kỳ đầu vào nào dương hoặc bằng 0. Nó có điểm gián đoạn ở mức 0. Phải: Hàm sigmoid được sử dụng để xây dựng các perceptron liên tục. Nó xuất ra các giá trị nhỏ hơn 0,5 cho đầu vào âm và giá trị lớn hơn 0,5 cho đầu vào dương. Ở mức 0, nó xuất ra 0,5. Nó liên tục và khác biệt ở mọi nơi.

Nói chung, hàm sigmoid tốt hơn hàm bước vì một số lý do. Việc có những dự đoán liên tục cho chúng ta nhiều thông tin hơn những dự đoán rời rạc. Ngoài ra, khi chúng ta đi sâu vào giải tích, hàm sigmoid có đạo hàm đẹp hơn rất nhiều so với hàm bậc thang. Hàm bước có đạo hàm bằng 0, ngoại trừ gốc tọa độ, nơi nó không được xác định. Trong bảng 6.1, chúng ta tính toán một số giá trị của hàm sigmoid để đảm bảo hàm này thực hiện những gì chúng ta mong muốn.

Bảng 6.1 Một số đầu vào và đầu ra theo hàm sigmoid. Lưu ý rằng đối với đầu vào âm lớn thì đầu ra gần bằng 0, trong khi đối với đầu vào dương lớn thì đầu ra gần bằng 1. Đối với đầu vào 0 thì đầu ra là 0,5.

x	$\sigma(x)$
-5	0.007
-1	0.269
0	0.5
1	0.731
5	0.993

Dự đoán của bộ phân loại logistic có được bằng cách áp dụng hàm sigmoid cho điểm và nó trả về một số từ 0 đến 1, như đã đề cập

trước đó, có thể được hiểu trong ví dụ của chúng tôi là xác suất mà câu đó hài lòng.

Trong chương 5, chúng ta đã định nghĩa một hàm lỗi cho một perceptron, được gọi là lỗi perceptron.Chúng tôi đã sử dụng lỗi perceptron này để xây dựng lặp lại một bộ phân loại perceptron.Trong chương này, chúng ta thực hiện theo quy trình tương tự.Lỗi của một perceptron liên tục hơi khác so với lỗi của một bộ dự đoán rời rạc, nhưng chúng vẫn có những điểm tương đồng.

Bộ dữ liệu và dự đoán

Trong chương này, chúng tôi sử dụng trường hợp sử dụng tương tự như trong chương 5, trong đó chúng tôi có tập dữ liệu các câu bằng ngôn ngữ ngoài hành tinh với nhãn “vui” và “buồn”, ký hiệu lần lượt là 1 và 0.Tập dữ liệu cho chương này hơi khác so với chương 5 và được trình bày trong bảng 6.2.

Bảng 6.2 Tập dữ liệu các câu có nhãn vui/buồn. Tọa độ là số lần xuất hiện của từ aack và beep trong câu.

	Từ	Tọa độ (#aack, #beep)	Nhãn
Câu 1	Ack bíp ack aack.	(3,2)	Sad (0)
Câu 2	Bíp bíp.	(1,2)	Hạnh Phúc (1)
Câu 3	Bíp!	(0,1)	Hạnh Phúc (1)
Câu 4	Ách ôi.	(2,0)	Sad (0)

Mô hình chúng tôi sử dụng có trọng số và độ lệch sau:

Bộ phân loại logistic 1

- Trọng lượng của Aack: $a =$
- 1 Trọng lượng của Beep: b
- = 2 Độ lệch: $c = -4$

Chúng tôi sử dụng ký hiệu tương tự như trong chương 5, trong đó các biến x_{aack} và x_{beep} lần lượt theo dõi sự xuất hiện của *aack* và *beep*.Bộ phân loại perceptron sẽ dự đoán theo công thức $\hat{y} = \text{bước}(ax_{aack} + bx_{beep} + c)$, nhưng bởi vì

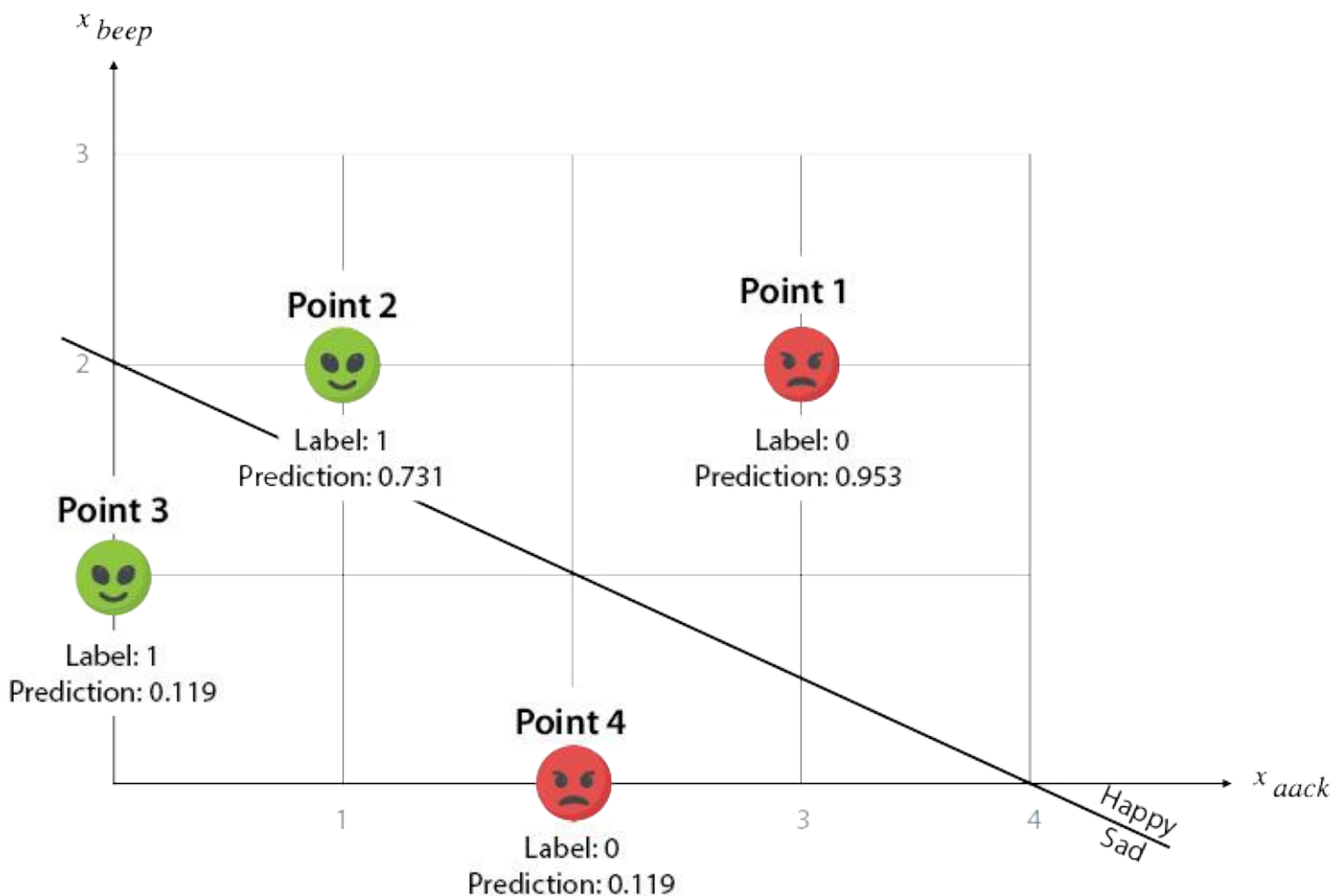
đây là một bộ phân loại logistic, nó sử dụng hàm sigmoid thay vì hàm bước. Như vậy dự đoán của nó là $\hat{y} = \sigma(ax_{aack} + bx_{beep} + c)$. Trong trường hợp này, dự đoán như sau:

Dự đoán: $\hat{y} = \sigma(1 \cdot x_{aack} + 2 \cdot x_{beep} - 4)$

Do đó, trình phân loại đưa ra các dự đoán sau trên tập dữ liệu của chúng tôi:

- **Câu 1:** $\hat{y} = \sigma(3 + 2 \cdot 2 - 4) = \sigma(3) = 0,953$. **Câu 2:** $\hat{y} = \sigma(1 + 2 \cdot 2 - 4) = \sigma(1) = 0,731$. **Câu 3:** $\hat{y} = \sigma(0 + 2 \cdot 1 - 4) = \sigma(-2) = 0.119$. **Câu 4:** $\hat{y} = \sigma(2 + 2 \cdot 0 - 4) = \sigma(-2) = 0.119$.
-

Ranh giới giữa lớp “vui” và “buồn” là đường thẳng có phương trình $x_{aack} + 2x_{beep} - 4 = 0$, minh họa trên hình 6.4.



Hình 6.4 Sơ đồ của tập dữ liệu trong bảng 6.2 với các dự đoán. Lưu ý rằng điểm 2 và 4 được phân loại chính xác nhưng điểm 1 và 3 bị phân loại sai.

Đường này chia mặt phẳng thành các vùng dương (vui) và âm (buồn). Vùng dương được hình thành bởi các điểm có dự đoán cao hơn hoặc bằng 0,5 và vùng âm được hình thành bởi các điểm có dự đoán nhỏ hơn 0,5.

Các hàm lỗi: Tuyệt đối, bình phương và mất log

Trong phần này, chúng tôi xây dựng ba hàm lỗi cho bộ phân loại logistic. Bạn muốn một hàm báo lỗi tốt có những thuộc tính nào? Một số ví dụ sau:

- Nếu một điểm được phân loại chính xác thì sai số là một con số nhỏ. Nếu một điểm được phân loại không chính xác thì sai số là một con số lớn. Lỗi của bộ phân loại đối với một tập hợp điểm là tổng (hoặc trung bình) của các lỗi tại tất cả các điểm.

Nhiều hàm thỏa mãn các tính chất này và chúng ta sẽ thấy ba trong số chúng; sai số tuyệt đối, sai số bình phương và mất log. Trong bảng 6.3, chúng ta có các nhãn và dự đoán cho 4 điểm tương ứng với các câu trong tập dữ liệu của chúng ta với các đặc điểm sau:

- Các điểm trên đường thẳng được dự đoán là 0,5. Các điểm nằm trong vùng dương được đưa ra dự đoán cao hơn 0,5 và điểm càng xa đường theo hướng đó thì dự đoán của nó càng gần 1. Các điểm nằm trong vùng âm được đưa ra dự đoán thấp hơn 0,5 và điểm càng xa đường theo hướng đó thì dự đoán của nó càng gần 0.
-

Bảng 6.3 Bốn điểm – hai điểm vui và hai điểm buồn với dự đoán của mình – như minh họa trong hình 6.4. Lưu ý rằng điểm 1 và 4 được phân loại chính xác, nhưng điểm 2 và 3 thì không. Một hàm lỗi tốt sẽ gán các lỗi nhỏ cho các điểm được phân loại đúng và lỗi lớn cho các điểm được phân loại kém.

Điểm	Nhãn	Dự đoán	Lỗi?
1	0 (Sad)	0.953	Nên lớn
2	1 (Hạnh phúc)	0.731	Nên nhỏ
3	1 (Hạnh phúc)	0.119	Nên lớn
4	0 (Sad)	0.119	Nên nhỏ

Lưu ý rằng trong bảng 6.3, điểm 2 và 4 nhận được dự đoán gần với nhãn nên sẽ có sai số nhỏ. Ngược lại, điểm 1 và 3 nhận được dự đoán cách xa nhãn nên có sai số lớn. Ba hàm lỗi có thuộc tính cụ thể này như sau:

CHỨC NĂNG LỖI 1: LỖI TUYỆT ĐỐI

lỗi tuyệt đối tương tự như *lỗi tuyệt đối* mà chúng tôi đã xác định cho *hội quy tuyến tính* ở chương 3. Đó là giá trị tuyệt đối của chênh lệch giữa dự đoán và nhãn. Như chúng ta có thể thấy, nó lớn khi dự đoán ở xa nhãn và nhỏ khi chúng ở gần.

CHỨC NĂNG LỖI 2: LỖI VUÔNG

Một lần nữa, giống như trong *hội quy tuyến tính*, chúng ta cũng có *sai số bình phương*. Đây là bình phương của sự khác biệt giữa dự đoán và nhãn và nó hoạt động với lý do tương tự như *lỗi tuyệt đối*.

Trước khi tiếp tục, hãy tính sai số tuyệt đối và sai số bình phương cho các điểm trong bảng 6.4. Lưu ý rằng điểm 2 và 4 (được phân loại đúng) có sai số nhỏ, còn điểm 1 và 3 (được phân loại sai) có sai số lớn hơn.

Bảng 6.4 Chúng tôi đã đính kèm sai số tuyệt đối và sai số bình phương cho các điểm trong bảng 6.3. Lưu ý rằng như chúng tôi mong muốn, điểm 2 và 4 có lỗi nhỏ, còn điểm 1 và 3 có lỗi lớn hơn.

Điểm	Nhãn	Nhãn dự đoán	Lỗi tuyệt đối	Lỗi hình vuông
1	0 (Sad)	0.953	0.953	0.908
2	1 (Hạnh phúc)	0.731	0.269	0.072
3	1 (Hạnh phúc)	0.119	0.881	0.776
4	0 (Sad)	0.119	0.119	0.014

Sai số tuyệt đối và sai số bình phương có thể khiến bạn nhớ đến các hàm sai số được sử dụng trong *hội quy*. Tuy nhiên, trong phân loại, chúng không được sử dụng rộng rãi. Phổ biến nhất là cái tiếp theo chúng ta thấy. Tại sao nó phổ biến hơn? Toán học (đạo hàm) hoạt động tốt hơn nhiều với hàm tiếp theo. Ngoài ra, những lỗi này đều khá nhỏ. Trên thực tế, chúng đều nhỏ hơn 1, cho dù điểm được phân loại kém đến đâu. Lý do là hiệu (hoặc bình phương của hiệu) giữa hai số nằm trong khoảng từ 0 đến 1 nhiều nhất là 1. Để huấn luyện mô hình đúng cách, chúng ta cần các hàm lỗi nhận giá trị lớn hơn thế. Rất may, chức năng báo lỗi thứ ba có thể giúp chúng ta làm điều đó.

CHỨC NĂNG LỖI 3: MẤT ĐĂNG NHẬP

log loss là hàm lỗi được sử dụng rộng rãi nhất cho các perceptron liên tục. Hầu hết các hàm báo lỗi trong cuốn sách này đều có từ *error* trong tên, nhưng hàm này lại có từ *loss* trong tên của nó. Phần *log* trong tên xuất phát từ logarit tự nhiên mà chúng tôi sử dụng trong công thức. Tuy nhiên, bản chất thực sự của việc mất log là xác suất.

Đầu ra của một perceptron liên tục là các số từ 0 đến 1, vì vậy chúng có thể được coi là xác suất. Mô hình gán một xác suất cho mọi điểm dữ liệu và đó là xác suất mà điểm đó hài lòng. Từ đó suy ra xác suất điểm đó buồn bằng 1 trừ xác suất điểm vui. Ví dụ: nếu dự đoán là 0,75, điều đó có nghĩa là mô hình tin rằng điểm đó vui với xác suất là 0,75 và buồn với xác suất là 0,25.

Bây giờ, đây là quan sát chính. Mục tiêu của mô hình là gán xác suất cao cho các điểm vui (những điểm có nhãn 1) và xác suất thấp cho các điểm buồn (những điểm có nhãn 0). Lưu ý rằng xác suất để một điểm buồn bằng 1 trừ đi xác suất điểm đó vui. Vì vậy, đối với mỗi điểm, hãy tính xác suất mà mô hình đưa ra cho nhãn của nó. Đối với các điểm trong tập dữ liệu của chúng tôi, xác suất tương ứng như sau:

- **Điểm 1:**
 - Nhãn = 0 (buồn) Dự đoán (xác suất hạnh phúc) = 0,953
 - Xác suất là nhãn của nó: $1 - 0,953 = \mathbf{0,047}$
 -
- **Điểm 2:**
 - Nhãn = 1 (hạnh phúc) Dự đoán (xác suất hạnh phúc) = 0,731
 - Xác suất là nhãn của nó: **0,731**
 -
- **Điểm 3:**
 - Nhãn = 1 (hạnh phúc) Dự đoán (xác suất hạnh phúc) = 0.119
 - Xác suất là nhãn của nó: **0.119**
 -
- **Điểm 4:**
 - Nhãn = 0 (buồn) Dự đoán (xác suất hạnh phúc) = 0,119
 - Xác suất là nhãn của nó: $1 - 0,119 = \mathbf{0,881}$
 -

Lưu ý rằng điểm 2 và 4 là những điểm được phân loại tốt và mô hình gán xác suất cao rằng chúng là nhãn riêng của chúng. Ngược lại, điểm 1 và 3 được phân loại kém và mô hình cho rằng chúng có xác suất thấp

nhãn hiệu riêng.

Bộ phân loại logistic, trái ngược với bộ phân loại perceptron, không đưa ra câu trả lời chắc chắn. Bộ phân loại perceptron sẽ nói: “Tôi chắc chắn 100% rằng điểm này là vui”, trong khi bộ phân loại logistic nói: “Quan điểm của bạn có 73% xác suất là vui và 27% là buồn”. Mặc dù mục tiêu của bộ phân loại perceptron là chính xác nhiều lần nhất có thể, nhưng mục tiêu của bộ phân loại logistic là gán cho mỗi điểm xác suất có nhãn chính xác cao nhất có thể. Trình phân loại này gán các xác suất 0,047, 0,731, 0,119 và 0,881 cho bốn nhãn. Lý tưởng nhất là chúng tôi muốn những con số này cao hơn. Làm thế nào để chúng ta đo lường bốn con số này? Một cách sẽ là thêm chúng hoặc tính trung bình chúng. Nhưng vì chúng là xác suất nên cách tiếp cận tự nhiên là nhân chúng lên. Khi các sự kiện là độc lập, xác suất chúng xảy ra đồng thời là tích của xác suất của chúng. Nếu chúng ta giả sử rằng bốn dự đoán là độc lập thì xác suất mà mô hình này gán cho các nhãn “buồn, vui, vui, buồn” là tích của bốn số đó là $0,047 \cdot 0,731 \cdot 0,119 \cdot 0,881 = 0,004$. Đây là một xác suất rất nhỏ. Chúng tôi hy vọng rằng một mô hình phù hợp với tập dữ liệu này tốt hơn sẽ mang lại xác suất cao hơn.

Xác suất mà chúng ta vừa tính toán có vẻ như là một thước đo tốt cho mô hình của chúng ta, nhưng nó có một số vấn đề. Ví dụ, nó là tích của nhiều số nhỏ. Sản phẩm có nhiều số lượng nhỏ có xu hướng rất nhỏ. Hãy tưởng tượng nếu tập dữ liệu của chúng tôi có một triệu điểm. Xác suất sẽ là tích của một triệu số, tất cả đều nằm trong khoảng từ 0 đến 1. Con số này có thể nhỏ đến mức máy tính không thể biểu diễn được nó. Ngoài ra, việc thao tác một sản phẩm gồm một triệu con số là vô cùng khó khăn. Có cách nào để chúng ta có thể biến nó thành thứ gì đó dễ thao tác hơn, chẳng hạn như một phép tính không?

Thật may mắn cho chúng ta, chúng ta có một cách thuận tiện để biến tích thành tổng—sử dụng logarit. Trong toàn bộ cuốn sách này, tất cả những gì chúng ta cần biết về logarit là nó biến tích thành tổng. Cụ thể hơn, logarit của tích hai số là tổng logarit của các số, như minh họa dưới đây:

$$\ln(a \cdot b) = \ln(a) + \ln(b)$$

Chúng ta có thể sử dụng logarit trong cơ số 2, 10 hoặc e. Trong chương này, chúng tôi sử dụng logarit tự nhiên, dựa trên cơ số e. Tuy nhiên, kết quả tương tự có thể thu được nếu chúng ta sử dụng logarit trong bất kỳ cơ số nào khác.

Nếu chúng ta áp dụng logarit tự nhiên cho tích xác suất, chúng ta thu được

$$\ln(0.047 \cdot 0.731 \cdot 0.119 \cdot 0.881) = \ln(0.047) + \ln(0.731) + \ln(0.119) + \ln(0.881) = -5.616.$$

Một chi tiết nhỏ.Lưu ý rằng kết quả là một số âm.Trên thực tế, điều này luôn đúng vì logarit của một số từ 0 đến 1 luôn âm.Vì vậy, nếu chúng ta lấy logarit âm của tích xác suất thì nó luôn là số dương.

Mất log được định nghĩa là logarit âm của tích các xác suất, cũng là tổng logarit âm của xác suất.Hơn nữa, mỗi lệnh triệu tập là mất nhật ký tại thời điểm đó.Trong bảng 6.5, bạn có thể xem cách tính log loss cho từng điểm.Bằng cách cộng tổng tổn thất log của tất cả các điểm, chúng ta có được tổng tổn thất log là 5,616.

Bảng 6.5 Tính toán tổn thất log cho các điểm trong tập dữ liệu của chúng tôi. Lưu ý rằng các điểm được phân loại tốt (2 và 4) có log bị mất nhỏ, trong khi các điểm được phân loại kém (1 và 3) có log bị mất lớn.

Điểm	Nhãn	Nhãn dự đoán	Xác suất là nhãn của nó	Mất nhật ký
1	0 (Sad)	0.953	0.047	$-\ln(0.047) = 3.049$
2	1(Hạnh phúc)	0.731	0.731	$-\ln(0.731) = 0.313$
3	1(Hạnh phúc)	0.119	0.119	$-\ln(0.119) = 2.127$
4	0 (Sad)	0.119	0.881	$-\ln(0.881) = 0.127$

Lưu ý rằng, thực tế, các điểm được phân loại tốt (2 và 4) có log bị mất nhỏ và các điểm được phân loại kém có log bị mất lớn.Lý do là vì nếu số x gần bằng 0 thì $-\ln(x)$ là số lớn, nhưng nếu x gần bằng 1 thì $-\ln(x)$ là số nhỏ.

Tóm lại, các bước tính toán tổn thất nhật ký như sau:

- Đối với mỗi điểm, chúng tôi tính toán xác suất mà bộ phân loại đưa ra nhãn của nó. Đối với những điểm hạnh phúc, xác suất này chính là điểm số. Đối với điểm buồn, xác suất này là 1 trừ điểm. Chúng tôi nhân tất cả các xác suất này để có được tổng xác suất mà bộ phân loại đã đưa ra cho các nhãn này.

- Chúng tôi áp dụng logarit tự nhiên cho tổng xác suất đó. Logarit của tích là tổng logarit của các thừa số, vì vậy chúng ta thu được tổng logarit, mỗi logarit cho mỗi điểm. Chúng ta nhận thấy rằng tất cả các số hạng đều âm, vì logarit của một số nhỏ hơn 1 là số âm. Do đó, chúng ta nhân mọi thứ với -1 để có tổng các số dương. Số tiền này là log loss của chúng tôi.

•

Việc mất nhật ký có liên quan chặt chẽ đến khái niệm *entropy chéo*, đây là một cách để đo lường mức độ tương tự giữa hai phân bố xác suất. Thông tin chi tiết hơn về entropy chéo có sẵn trong tài liệu tham khảo ở phụ lục C.

CÔNG THỨC MẤT LOG

Việc mất nhật ký cho một điểm có thể được cô đọng thành một công thức hay. Hãy nhớ lại rằng log loss là logarit âm của xác suất điểm đó là nhãn của nó (vui hoặc buồn). Dự đoán mà mô hình đưa ra cho mỗi điểm là $\hat{y}$ và đó là xác suất để điểm đó hài lòng. Như vậy, xác suất để điểm buồn theo mô hình là $1 - \hat{y}$. Vì vậy, chúng ta có thể viết log loss như sau:

- Nếu nhãn là 0: $\log \text{loss} = -\ln(1 - \hat{y})$
- Nếu nhãn là 1: $\log \text{loss} = -\ln(\hat{y})$

Vì nhãn là y nên câu lệnh trước có thể được cô đọng thành công thức sau:

$$\log \text{loss} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

Công thức trước có hiệu quả vì nếu nhãn bằng 0 thì triệu hồi đầu tiên là 0 và nếu nhãn là 1 thì triệu hồi thứ hai là 0. Chúng tôi sử dụng thuật ngữ *mất nhật ký* khi đề cập đến mất nhật ký của một điểm hoặc của toàn bộ tập dữ liệu. Mất nhật ký của tập dữ liệu là tổng tổn thất nhật ký tại mọi điểm.

So sánh các bộ phân loại bằng cách sử dụng log loss

Bây giờ chúng ta đã giải quyết xong hàm lỗi cho các bộ phân loại logistic, mất log, chúng ta có thể sử dụng nó để so sánh hai bộ phân loại. Hãy nhớ lại rằng bộ phân loại mà chúng ta đang sử dụng trong chương này được xác định bởi các trọng số và độ lệch sau:

Bộ phân loại logistic 1

- Trọng lượng của *Aack*: $a =$
- 1 Trọng lượng của *Beep*: b
- $= 2$ Độ lệch: $c = -4$

Trong phần này, chúng tôi so sánh nó với bộ phân loại logistic sau:

Bộ phân loại logistic 2

- Trọng lượng của *Aack*: $a =$
- -1 Trọng lượng của *Beep*: b
- $= 1$ Độ lệch: $c = 0$

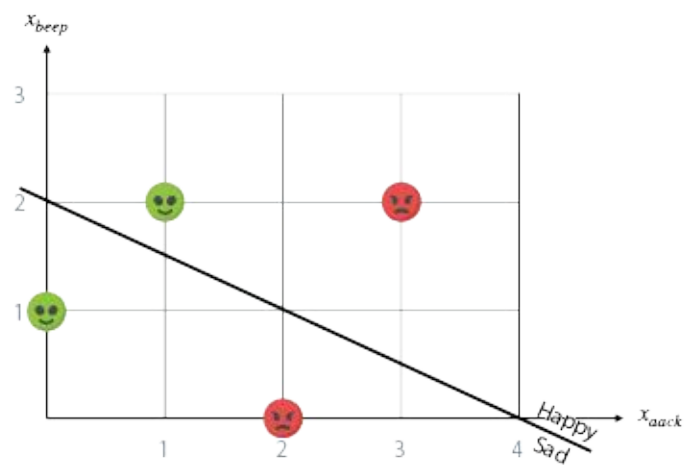
Các dự đoán mà mỗi bộ phân loại đưa ra như sau:

- **Bộ phân loại 1:** $\hat{y} = \sigma(x_{aack} + 2x_{beep} - 4)$
- **phân loại 2:** $\hat{y} = \sigma(-x_{aack} + x_{beep})$

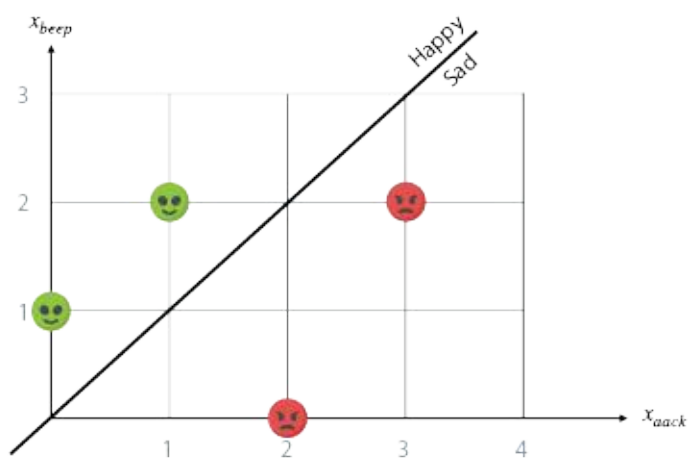
Dự đoán của cả hai phân loại được ghi lại trong bảng 6.6, và biểu đồ của tập dữ liệu và hai đường ranh giới được thể hiện trong hình 6.5.

Bảng 6.6 Tính toán tổn thất log cho các điểm trong tập dữ liệu của chúng tôi. Lưu ý rằng các dự đoán do phân loại 2 đưa ra gần với nhãn của điểm hơn nhiều so với dự đoán của phân loại 1. Do đó, phân loại 2 là phân loại tốt hơn.

Điểm	Nhãn	Dự đoán phân loại 1	Dự đoán phân loại 2
1	0 (Sad)	0.953	0.269
2	1 (Hạnh phúc)	0.731	0.731
3	1 (Hạnh phúc)	0.119	0.731
4	0 (Sad)	0.881	0.119



Classifier 1



Classifier 2

Hình 6.5 Bên trái: Một bộ phân loại tồi mắc hai lỗi.Đúng: Một bộ phân loại tốt có thể phân loại chính xác cả bốn điểm.

Từ kết quả trong bảng 6.6 và hình 6.5, rõ ràng phân loại 2 tốt hơn nhiều so với phân loại 1. Ví dụ, trong hình 6.5, chúng ta có thể thấy rằng phân loại 2 đã định vị chính xác hai câu vui ở vùng tích cực và hai câu buồn ở vùng tiêu cực.Tiếp theo, chúng tôi so sánh tổn thất nhật ký.Hãy nhớ lại rằng log loss của phân loại 1 là 5,616.Chúng ta sẽ thu được log loss nhỏ hơn cho bộ phân loại 2, vì đây là bộ phân loại tốt hơn.

Theo công thức $\log \text{loss} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, log loss của phân loại 2 tại mỗi điểm trong tập dữ liệu của chúng tôi như sau:

- **Point 1:** $y = 0, \hat{y} = 0.269$:
 $\log \text{loss} = \ln(1 - 0.269) = 0.313$
- **Point 2:** $y = 1, \hat{y} = 0.73$:
 $\log \text{loss} = \ln(0.721) = 0.313$
- **Point 3:** $y = 1, \hat{y} = 0.73$:
 $\log \text{loss} = \ln(0.269) = 0.313$
- **Point 4:** $y = 0, \hat{y} = 0.119$:
 $\log \text{loss} = \ln(1 - 0.119) = 0.127$

o

Tổng tổn thất nhật ký cho tập dữ liệu là tổng của bốn dữ liệu này, là 1,067.Lưu ý rằng giá trị này nhỏ hơn nhiều so với 5,616, xác nhận rằng bộ phân loại 2 thực sự tốt hơn nhiều so với bộ phân loại 1.

Làm thế nào để tìm được bộ phân loại logistic tốt?Cái

thuật toán hồi quy logistic

Trong phần này, chúng ta tìm hiểu cách huấn luyện một bộ phân loại logistic. Quá trình này tương tự như quá trình huấn luyện mô hình hồi quy tuyến tính hoặc bộ phân loại perceptron và bao gồm các bước sau:

- Bắt đầu với một bộ phân loại logistic ngẫu nhiên. Lặp lại nhiều lần: Cải thiện
- một chút khả năng phân loại. Đo mức độ mất nhật ký để quyết định thời điểm dừng chạy vòng lặp.
-

Chìa khóa của thuật toán là bước bên trong vòng lặp, bao gồm việc cải thiện một chút bộ phân loại logistic. Bước này sử dụng một thủ thuật gọi là *thủ thuật hậu cần*. Thủ thuật logistic tương tự như thủ thuật perceptron, như chúng ta sẽ thấy trong phần tiếp theo.

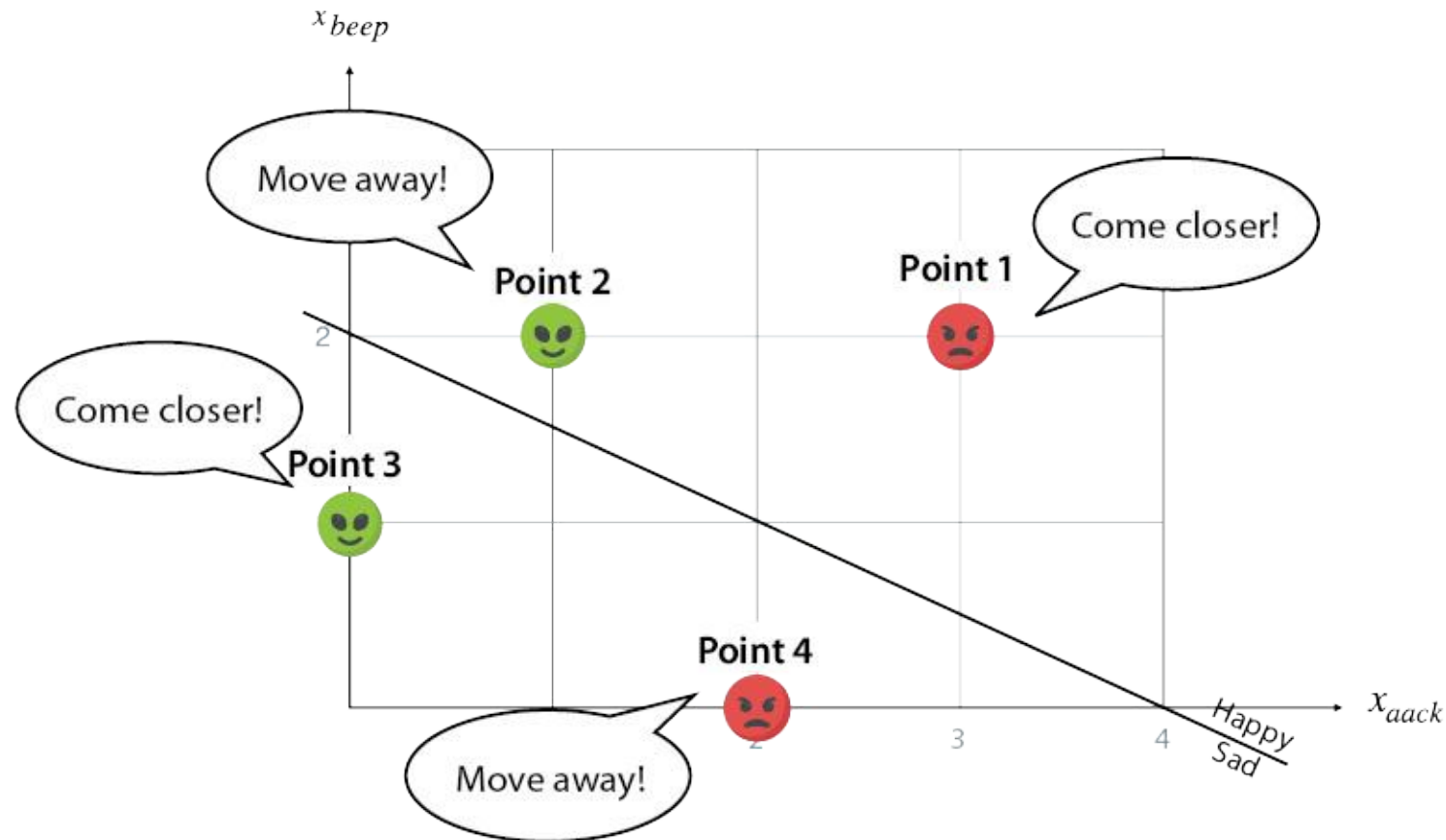
Thủ thuật logistic: Một cách cải thiện một chút perceptron liên tục

Hãy nhớ lại ở chương 5 rằng thủ thuật perceptron bao gồm việc bắt đầu bằng một bộ phân loại ngẫu nhiên, liên tục chọn một điểm ngẫu nhiên và áp dụng thủ thuật perceptron. Nó có hai trường hợp sau:

- **Trường hợp 1:** Nếu điểm được phân loại chính xác, hãy giữ nguyên đường thẳng. **Trường hợp**
- **2:** Nếu điểm được phân loại không chính xác, hãy di chuyển đường thẳng đến gần điểm hơn một chút.

Thủ thuật logistic (minh họa trong hình 6.6) tương tự như thủ thuật perceptron. Điều duy nhất thay đổi là khi điểm được phân loại tốt, chúng ta di chuyển đường *ra xa* khỏi điểm. Nó có hai trường hợp sau:

- **Trường hợp 1:** Nếu điểm được phân loại chính xác, hãy di chuyển đường thẳng ra xa điểm một chút. **Trường hợp 2:** Nếu điểm được phân loại không chính xác, hãy di chuyển đường thẳng đến gần điểm hơn một chút.
-



Hình 6.6 Trong thuật toán hồi quy logistic, mọi điểm đều có tiếng nói. Các điểm được phân loại chính xác sẽ yêu cầu đường đi chuyển ra xa hơn, sâu hơn trong vùng chính xác. Những điểm được phân loại không chính xác sẽ yêu cầu đường đến gần hơn với hy vọng một ngày nào đó sẽ ở phía bên phải của đường.

Tại sao chúng ta di chuyển đường ra khỏi điểm được phân loại chính xác? Nếu điểm được phân loại tốt, điều đó có nghĩa là nó nằm trong vùng chính xác đối với đường thẳng. Nếu chúng ta di chuyển đường ra xa hơn, chúng ta sẽ di chuyển điểm sâu hơn vào vùng chính xác. Bởi vì dự đoán dựa trên khoảng cách từ điểm đến đường biên, đối với các điểm trong vùng dương (hạnh phúc), dự đoán sẽ tăng nếu điểm ở xa đường biên hơn. Tương tự, đối với các điểm trong vùng âm (buồn), dự đoán sẽ giảm nếu điểm ở xa đường thẳng hơn. Do đó, nếu nhãn của điểm là 1, chúng tôi sẽ tăng dự đoán (làm cho nó thậm chí gần hơn với 1) và nếu nhãn của điểm là 0, chúng tôi sẽ giảm dự đoán (làm cho nó thậm chí còn gần hơn với 0).

Ví dụ: hãy xem phân loại 1 và câu đầu tiên trong tập dữ liệu của chúng tôi. Hãy nhớ lại rằng bộ phân loại có trọng số $a = 1$, $b = 2$ và độ lệch $c = -4$. Câu tương ứng với một điểm tọa độ $(x_{aack}, x_{beep}) = (3, 2)$, và gán nhãn $y = 0$. Dự đoán chúng ta

thu được cho điểm này là $\hat{y} = \sigma(3 + 2 \cdot 2 - 4) = \sigma(3) = 0,953$. Dự đoán khá xa nhãn nên sai số cao: thực tế ở bảng 6.5 chúng tôi tính ra là 3,049. Lỗi mà bộ phân loại này mắc phải là cho rằng câu này hạnh phúc hơn nó vốn có. Vì vậy, để điều chỉnh các trọng số để đảm bảo rằng bộ phân loại giảm

dự đoán cho câu này, chúng ta nên giảm mạnh các trọng số a , b và độ lệch c .

Sử dụng logic tương tự, chúng ta có thể phân tích cách điều chỉnh trọng số để cải thiện việc phân loại cho các điểm khác. Đối với câu thứ hai trong tập dữ liệu, nhãn là $y = 1$ và dự đoán là $\hat{y} = 0,731$. Đây là một dự đoán tốt, nhưng nếu muốn cải thiện nó, chúng ta nên tăng nhẹ trọng số và độ lệch. Đối với câu thứ ba, vì nhãn là $y = 1$ và dự đoán là $\hat{y} = 0,119$, nên chúng ta nên tăng mạnh trọng số và độ lệch. Cuối cùng, đối với câu thứ tư, nhãn là $y = 0$ và dự đoán là $\hat{y} = 0,119$, vì vậy chúng ta nên giảm nhẹ trọng số và độ lệch. Những điều này được tóm tắt trong bảng 6.7.

Bảng 6.7 Tính toán tổn thất log cho các điểm trong tập dữ liệu của chúng tôi. Lưu ý rằng các điểm được phân loại tốt (2 và 4) có log bị mất nhỏ, trong khi các điểm được phân loại kém (1 và 3) có log bị mất lớn.

Điểm	Nhãn y	Dự đoán phân loại $\hat{y}$	Cách điều chỉnh trọng số a , b và độ lệch c	$y - \hat{y}$
1	0	0.953	Giảm một lượng lớn	-0.953
2	1	0.731	Tăng thêm một lượng nhỏ	0.269
3	1	0.119	Tăng một lượng lớn	0.881
4	0	0.119	Giảm một lượng nhỏ	-0.119

Những quan sát sau đây có thể giúp chúng tôi tìm ra lượng hoàn hảo mà chúng tôi muốn thêm vào trọng số và độ lệch để cải thiện dự đoán:

- **Quan sát 1:** cột cuối cùng của bảng 6.7 có giá trị nhãn trừ đi dự đoán. Hãy chú ý những điểm tương đồng giữa hai cột ngoài cùng bên phải trong bảng này. Điều này gợi ý rằng số tiền chúng ta nên cập nhật trọng số và độ lệch phải là bội số của $y - \hat{y}$.
- **Quan sát 2:** hãy tưởng tượng một câu trong đó từ *aack* xuất hiện 10 lần và *bíp* xuất hiện một lần. Nếu chúng ta cộng (hoặc trừ) một giá trị cho trọng số của hai từ này, sẽ hợp lý khi nghĩ rằng trọng số của *aack* nên được cập nhật với số lượng lớn hơn, vì từ này quan trọng hơn đối với điểm tổng thể của câu. Do đó, số tiền chúng tôi nên cập nhật trọng số của *aack* sẽ được nhân với x_{aack} và số tiền chúng tôi nên cập nhật trọng số của *beep* sẽ được nhân với x_{beep} .

- **Quan sát 3:** số lượng chúng tôi cập nhật trọng số và độ lệch cũng phải được nhân với tỷ lệ học η vì chúng tôi muốn đảm bảo rằng con số này nhỏ.

Kết hợp ba quan sát lại với nhau, chúng tôi kết luận rằng sau đây là một tập hợp các trọng số được cập nhật phù hợp:

- $a' = a + \eta(y - \hat{y})x_1$
- $b' = b + \eta(y - \hat{y})x_2$
- $c' = c + \eta(y - \hat{y})$

Do đó, mã giả cho thủ thuật logistic sẽ theo sau. Hãy chú ý rằng nó giống với mã giả cho thủ thuật perceptron mà chúng ta đã học ở cuối phần “Thủ thuật perceptron” trong chương 5.

MÃ GIẢ CHO THỦ THUẬT LOGISTICS

Đầu vào:

- Bộ phân loại logistic có trọng số a , b , và độ lệch c
- Một điểm có tọa độ (x_1, x_2) và nhãn y
- Giá trị nhỏ η (tốc độ học)

Đầu ra:

- Một perceptron có trọng số mới a' , b' , và độ lệch c' ít nhất cũng tốt bằng đầu vào

Thủ tục:

- Dự đoán mà perceptron đưa ra tại thời điểm đó là $\hat{y} = \sigma(ax_1 + bx_2 + c)$.

Trở về:

- Perceptron có trọng số và độ lệch sau:
 - $a' = a + \eta(y - \hat{y})x_1$
 - $b' = b + \eta(y - \hat{y})x_2$
 - $c' = c + \eta(y - \hat{y})$

Cách chúng tôi cập nhật trọng số và độ lệch trong thủ thuật logistic không phải là ngẫu nhiên. Nó xuất phát từ việc áp dụng thuật toán giảm độ dốc để giảm mất mát nhật ký các

chi tiết toán học được mô tả trong phụ lục B, phần “Sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình phân loại”.

Để xác minh rằng thủ thuật logistic có hiệu quả trong trường hợp của chúng ta hay không, hãy áp dụng nó cho tập dữ liệu hiện tại. Trên thực tế, chúng tôi sẽ áp dụng thủ thuật này cho từng điểm trong số bốn điểm riêng biệt để xem mỗi điểm trong số chúng sẽ sửa đổi trọng số và độ lệch của mô hình đến mức nào. Cuối cùng, chúng tôi sẽ so sánh tình trạng mất nhật ký tại thời điểm đó trước và sau khi cập nhật và xác minh rằng lượng mất nhật ký đó thực sự đã giảm. Đối với các tính toán sau, chúng tôi sử dụng tốc độ học $\eta = 0,05$.

CẬP NHẬT PHÂN LOẠI SỬ DỤNG TỪNG CÂU

Sử dụng câu đầu tiên:

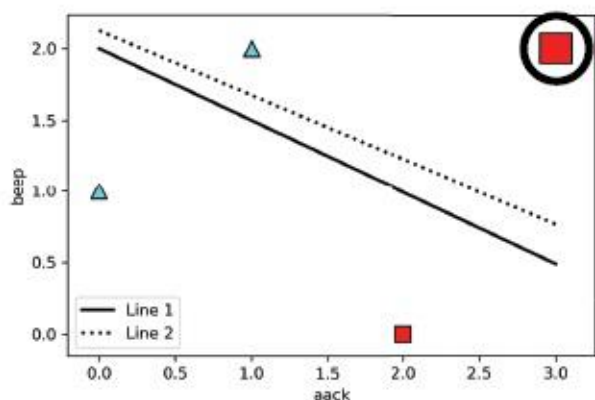
- Trọng số và độ lệch ban đầu: $a = 1, b = 2, c = -4$ Nhãn: $y = 0$ Dự đoán: 0,953 Mất nhật ký ban đầu: $-0 \cdot \ln(0,953) - 1 \ln(1 - 0,953) = 3,049$ Tọa độ của điểm: $x_{aack} = 3, x_{beep} = 2$
-
-
- Tỷ lệ học: $\eta = 0,01$ Trọng số và độ lệch được cập nhật: $a' = 1 + 0,05 \cdot (0 - 0,953) \cdot 3 = 0,857$
 $b' = 2 + 0,05 \cdot (0 - 0,953) \cdot 2 = 1,905$ $c' = -4 + 0,05 \cdot (0 - 0,953) = -4.048$ Cập nhật dự đoán: $\hat{y} = \sigma(0,857 \cdot 3 + 1,905 \cdot 2 - 4,048 = 0,912$. (Lưu ý rằng dự đoán đã giảm nên hiện tại nó gần với nhãn 0 hơn). Mất log cuối cùng: $-0 \cdot \ln(0,912) - 1 \ln(1 - 0,912) = 2.426$ (Lưu ý rằng lỗi giảm từ 3.049 xuống 2.426).
-
-

Các tính toán cho ba điểm còn lại được trình bày trong bảng 6.8. Lưu ý rằng trong bảng, dự đoán được cập nhật luôn gần nhãn hơn dự đoán ban đầu và tổn thất nhật ký cuối cùng luôn nhỏ hơn tổn thất nhật ký ban đầu. Điều này có nghĩa là bất kể chúng tôi sử dụng điểm nào cho thủ thuật logistic, chúng tôi sẽ cải thiện mô hình cho điểm đó và giảm tổn thất log cuối cùng.

Bảng 6.8 Tính toán dự đoán, mất log, trọng số cập nhật và dự đoán cập nhật cho tất cả các điểm.

Điểm	Tọa độ	Nhãn	Dự đoán ban đầu	Mất nhật ký ban đầu	Trọng lượng được cập nhật:	Dự đoán đã cập nhật	Mất nhật ký cuối cùng
1	(3,2)	0	0.953	3.049	$a' = 0.857$ $b' = 1.905$ $c' = -4.048$	0.912	2.426
2	(1,2)	1	0.731	0.313	$a' = 1.013$ $b' = 2.027$ $c' = -3.987$	0.747	0.292
3	(0,1)	1	0.119	2.127	$a' = 1$ $b' = 2.044$ $c' = -3.956$	0.129	2.050
4	(2,0)	0	0.119	0.127	$a' = 0.988$ $b' = 2$ $c' = -4.006$	0.127	0.123

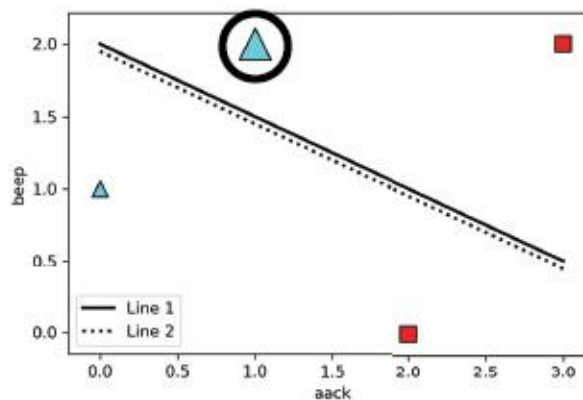
Ở phần đầu của phần này, chúng ta đã thảo luận rằng thủ thuật logistic cũng có thể được hình dung về mặt hình học như việc di chuyển đường biên đối với một điểm. Cụ thể hơn, đường được di chuyển đến gần điểm hơn nếu điểm được phân loại sai và xa điểm hơn nếu điểm được phân loại chính xác.Chúng ta có thể xác minh điều này bằng cách vẽ biểu đồ phân loại ban đầu và phân loại đã sửa đổi trong bốn trường hợp trong bảng 6.8.Trong hình 6.7, bạn có thể thấy bốn ô.Trong mỗi đường đó, đường liền nét là bộ phân loại ban đầu và đường chấm chấm là bộ phân loại thu được bằng cách áp dụng thủ thuật logistic, sử dụng điểm được đánh dấu.Lưu ý rằng các điểm 2 và 4, được phân loại chính xác, sẽ đẩy đường thẳng ra xa, trong khi các điểm 1 và 3, được phân loại sai, sẽ di chuyển đường thẳng đến gần chúng hơn.



Point: (3,2) (misclassified)

$$\text{Line 1: } 1x_{aack} + 2x_{beep} - 4 = 0$$

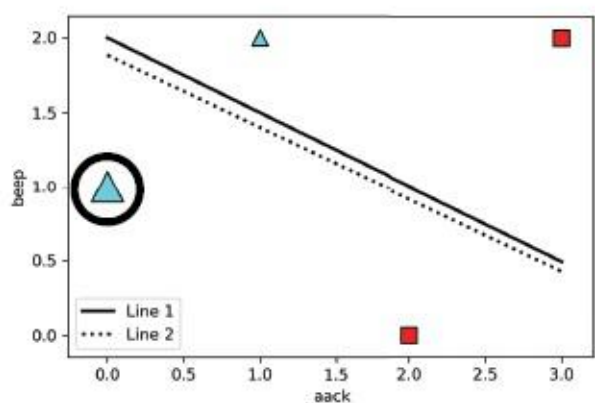
$$\text{Line 2: } 0.857x_{aack} + 1.905x_{beep} - 4.048 = 0$$



Point: (1,2) (correctly classified)

$$\text{Line 1: } 1x_{aack} + 2x_{beep} - 4 = 0$$

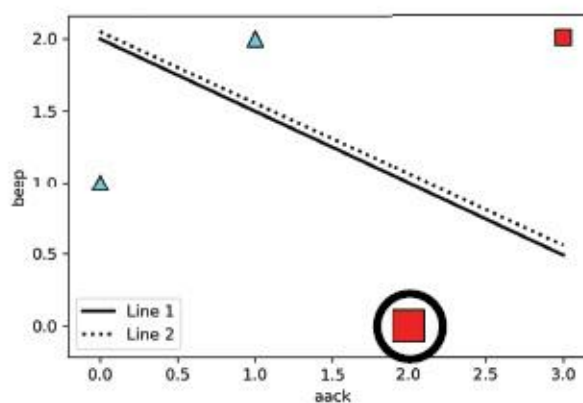
$$\text{Line 2: } 1.013x_{aack} + 2.027x_{beep} - 3.987 = 0$$



Point: (0,1) (misclassified)

$$\text{Line 1: } 1x_{aack} + 2x_{beep} - 4 = 0$$

$$\text{Line 2: } 1x_{aack} + 2.044x_{beep} - 3.956 = 0$$



Point: (2,0) (correctly classified)

$$\text{Line 1: } 1x_{aack} + 2x_{beep} - 4 = 0$$

$$\text{Line 2: } 0.988x_{aack} + 2x_{beep} - 4.006 = 0$$

Hình 6.7 Thủ thuật logistic được áp dụng cho từng điểm trong số bốn điểm dữ liệu. Lưu ý rằng đối với các điểm được phân loại chính xác, đường sẽ di chuyển ra xa điểm, trong khi đối với các điểm được phân loại sai, đường sẽ di chuyển đến gần điểm hơn.

Lặp lại thủ thuật logistic nhiều lần: Thuật toán hồi quy logistic

Thuật toán hồi quy logistic là những gì chúng tôi sử dụng để huấn luyện bộ phân loại logistic. Tương tự như cách thuật toán perceptron bao gồm việc lặp lại thủ thuật perceptron nhiều lần, thuật toán hồi quy logistic bao gồm việc lặp lại thủ thuật logistic nhiều lần. Mã giả sau:

MÃ GIẢ CHO SỰ PHẢI HỒI LOGISTICS

thuật toán

Đầu vào:

- Tập dữ liệu các điểm, được gán nhãn 1 và 0
- Một số kỹ nguyên, n Tỷ lệ học tập η
-

Đầu ra:

- Một bộ phân loại logistic bao gồm một tập hợp các trọng số và độ lệch, phù hợp với tập dữ liệu

Thủ tục:

- Bắt đầu với các giá trị ngẫu nhiên cho trọng số và độ lệch của bộ phân loại logistic. Lặp lại
- nhiều lần: Chọn một điểm dữ liệu ngẫu nhiên. Cập nhật trọng số và độ lệch bằng thủ thuật logistic.
 -

Trở về:

- Bộ phân loại perceptron với trọng số và độ lệch được cập nhật

Như chúng ta đã thấy trước đây, mỗi lần lặp lại thủ thuật logistic sẽ di chuyển đường thẳng đến gần điểm được phân loại sai hoặc ra xa điểm được phân loại chính xác.

Giảm độ dốc ngẫu nhiên, từng đợt nhỏ và theo đợt

Thuật toán hồi quy logistic, cùng với hồi quy tuyến tính và perceptron, là một thuật toán khác dựa trên phương pháp giảm độ dốc. Nếu chúng ta sử dụng phương pháp giảm độ dốc để giảm mất mát nhật ký thì bước giảm độ dốc sẽ trở thành thủ thuật logistic.

Thuật toán hồi quy logistic tổng quát không chỉ hoạt động với các tập dữ liệu có hai tính năng mà còn hoạt động với các tập dữ liệu có nhiều tính năng như chúng ta muốn. Trong trường hợp này, giống như thuật toán perceptron, ranh giới sẽ không giống như một đường thẳng mà trông giống như một siêu phẳng có chiều cao hơn phân chia các điểm trong không gian có chiều cao hơn. Tuy nhiên, chúng ta không cần hình dung không gian có chiều cao hơn này; chúng ta chỉ cần xây dựng một bộ phân loại hồi quy logistic với số lượng trọng số tương ứng với các đặc điểm trong dữ liệu của chúng ta. Thủ thuật logistic và thuật toán logistic cập nhật các trọng số theo cách tương tự

way to what we did in the previous sections.

Just like with the previous algorithms we learned, in practice, we don't update the model by picking one point at a time. Instead, we use mini-batch gradient descent—we take a batch of points and update the model to fit those points better. For the fully general logistic regression algorithm and a thorough mathematical derivation of the logistic trick using gradient descent, please refer to appendix B, section “Using gradient descent to train classification models.”

Coding the logistic regression algorithm

In this section, we see how to code the logistic regression algorithm by hand. The code for this section follows:

- **Notebook:** Coding_logistic_regression.ipynb
 - https://github.com/luisguiserrano/manning/blob/master/Chapter_6_Logistic_I

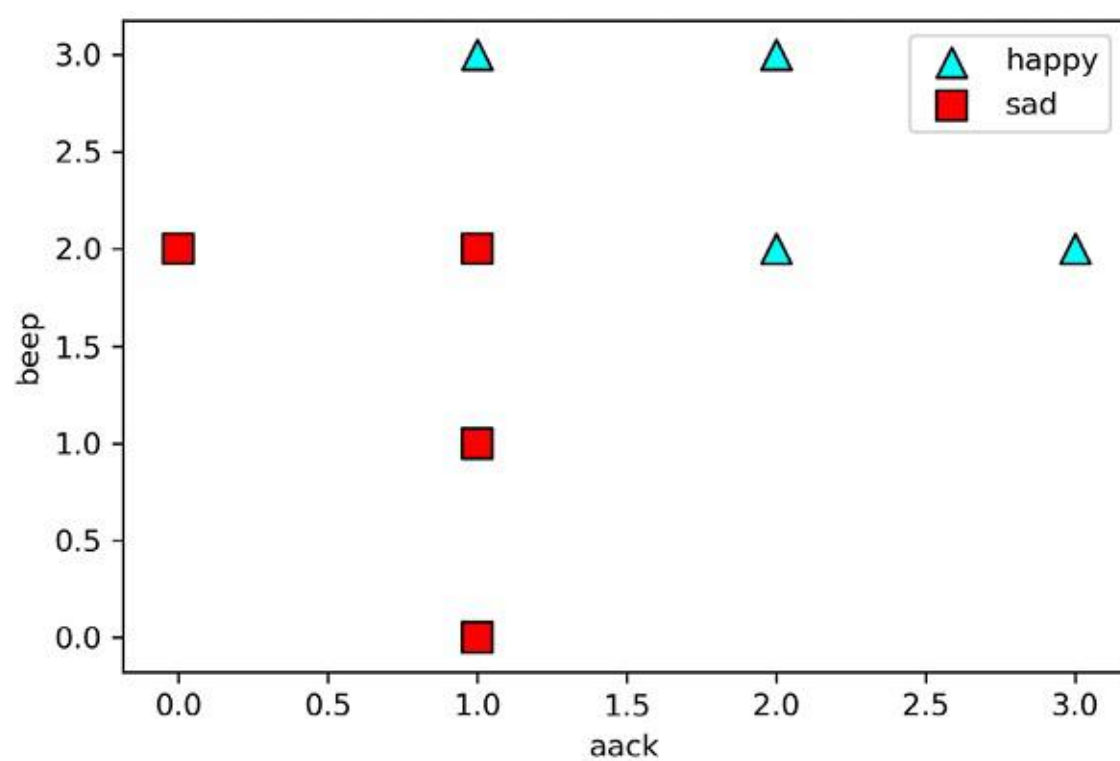
We'll test our code in the same dataset that we used in chapter 5. The dataset is shown in table 6.9.

Table 6.9 The dataset that we will fit with a logistic classifier

<i>Aack</i> x_1	<i>Beep</i> x_2	Label y
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

The code for loading our small dataset follows, and the plot of the dataset is shown in figure 6.8:

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```



Hình 6.8 Sơ đồ của tập dữ liệu của chúng tôi, trong đó các câu vui được biểu thị bằng hình tam giác và các câu buồn được biểu thị bằng hình vuông.

Mã hóa thuật toán hồi quy logistic bằng tay

Trong phần này, chúng ta xem cách viết mã thủ thuật logistic và thuật toán hồi quy logistic bằng tay. Tổng quát hơn, chúng tôi sẽ mã hóa thuật toán hồi quy logistic cho tập dữ liệu có trọng số n . Ký hiệu chúng tôi sử dụng như sau:

- Đặc điểm: $x_1, x_2, \dots, x_n$
- Nhãn: y Trọng số: $w_1, w_2, \dots$
- w_n Độ lệch: b
-

Điểm cho một câu cụ thể là sigmoid của tổng trọng số của mỗi từ (w_i) nhân với số lần xuất hiện (x_i), cộng với độ lệch (b). Chú ý rằng

chúng tôi sử dụng ký hiệu tổng hợp cho

$$\sum_{i=1}^n a_i = a_1 + a_2 + \dots + a_n.$$

- Prediction: $\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b) = \sigma(\sum_{i=1}^n w_i x_i + b)$.

Đối với vấn đề hiện tại, chúng ta sẽ gọi x_{aack} và x_{beep} lần lượt là x_1 và x_2 . Của họ trọng số tương ứng là w_1 và w_2 , và độ lệch là b .

Chúng tôi bắt đầu bằng cách mã hóa hàm sigmoid, điểm số và dự đoán.Hãy nhớ lại rằng công thức của hàm sigmoid là

$$\sigma(x)=\frac{1}{1+e^{-x}}$$

```
def sigmoid(x):
    return np.exp(x)/(1+np.exp(x))
```

Đối với hàm tính điểm, chúng tôi sử dụng tích số chấm giữa các đặc điểm và trọng số.Hãy nhớ lại rằng tích vô hướng giữa các vectơ $(x_1, x_2, \dots, x_n)$ và $(w_1, w_2, \dots, w_n)$ là $w_1 x_1 + w_2 x_2 + \dots + w_n x_n$.

```
def score(weights, bias, features):
    return np.dot(weights, features) + bias
```

Cuối cùng, hãy nhớ lại rằng dự đoán là hàm kích hoạt sigmoid được áp dụng cho điểm số.

```
def prediction(weights, bias, features):
    return sigmoid(score(weights, bias, features))
```

Bây giờ chúng ta đã có dự đoán, chúng ta có thể tiến hành mất nhật ký.Hãy nhớ lại rằng công thức tính log bị mất là

$$log\ loss = -y\ln(\hat{y}) - (1 - y)\ln(1 - \hat{y}).$$

Hãy mã hóa công thức đó như sau:

```
def log_loss(weights, bias, features, label):
    pred = prediction(weights, bias, features)
    return -label*np.log(pred) - (1-label)*np.log(1-pred)
```

Chúng tôi cần mất nhật ký trên toàn bộ tập dữ liệu, vì vậy chúng tôi có thể thêm tất cả các điểm dữ liệu như được hiển thị ở đây:

```
def total_log_loss(weights, bias, features, labels):
    total_error = 0
    for i in range(len(features)):
        total_error += log_loss(weights, bias, features[i], labels[i])
    return total_error
```

Bây giờ chúng ta đã sẵn sàng viết mã thủ thuật hồi quy logistic và thuật toán hồi quy logistic.Trong trường hợp có nhiều hơn hai biến, hãy nhớ lại rằng bước hồi quy logistic cho trọng số thứ i là công thức sau, trong đó η là tốc độ học:

- $w_i \rightarrow w_i + \eta(y - \hat{y})x_i$ cho $i = 1, 2, \dots, n$
- $b \rightarrow b + \eta(y - \hat{y})$ cho $i = 1, 2, \dots, n$.

```
def logistic_trick(weights, bias, features, label, learning_rate = 0.01):
    pred = prediction(weights, bias, features)
    for i in range(len(weights)):
        weights[i] += (label-pred)*features[i]*learning_rate
        bias += (label-pred)*learning_rate
    return weights, bias

def logistic_regression_algorithm(features, labels, learning_rate = 0.01, epochs = 1000):
    utils.plot_points(features, labels)
    weights = [1.0 for i in range(len(features[0]))]
    bias = 0.0
    errors = []
    for i in range(epochs):
        errors.append(total_log_loss(weights, bias, features, labels))
        j = random.randint(0, len(features)-1)
        weights, bias = logistic_trick(weights, bias, features[j], labels[j])
    return weights, bias
```

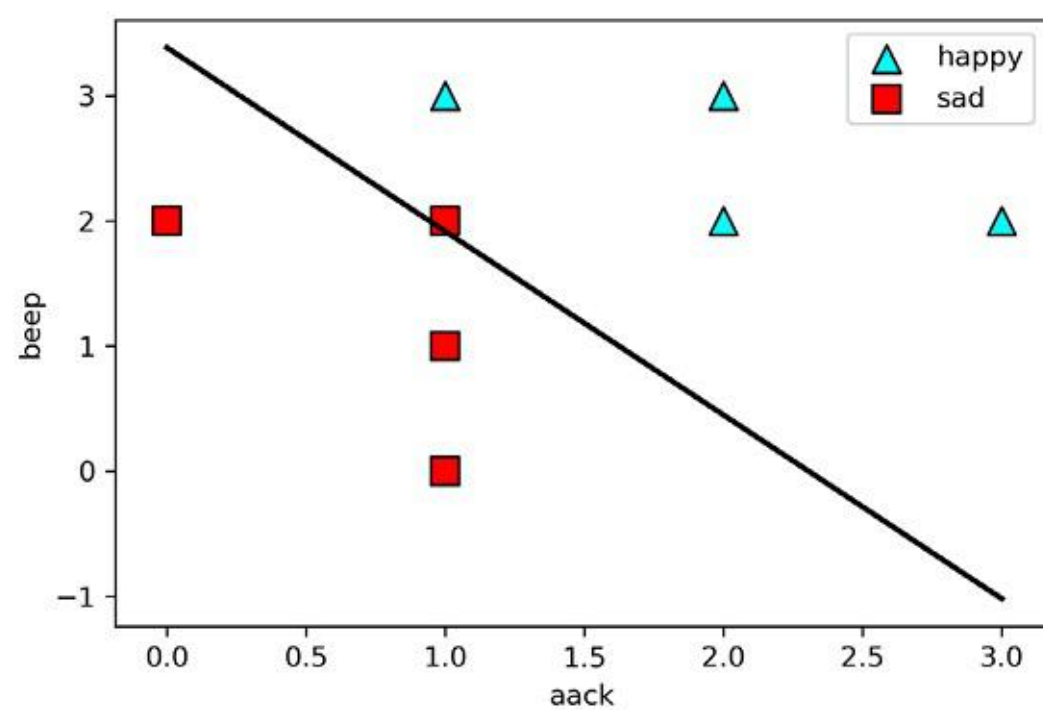
Bây giờ chúng ta có thể chạy thuật toán hồi quy logistic để xây dựng bộ phân loại logistic phù hợp với tập dữ liệu của mình như sau:

```
logistic_regression_algorithm(features, labels)
([0.46999999999999953, 0.09999999999999937], -0.6800000000000004)
```

Bộ phân loại mà chúng tôi thu được có các trọng số và độ lệch sau:

- $w_1 = 0.47$
- $w_2 = 0.10$
- $b = -0.68$

Sơ đồ của bộ phân loại (cùng với sơ đồ của các bộ phân loại trước đó ở mỗi kỷ nguyên) được mô tả trong hình 6.9.

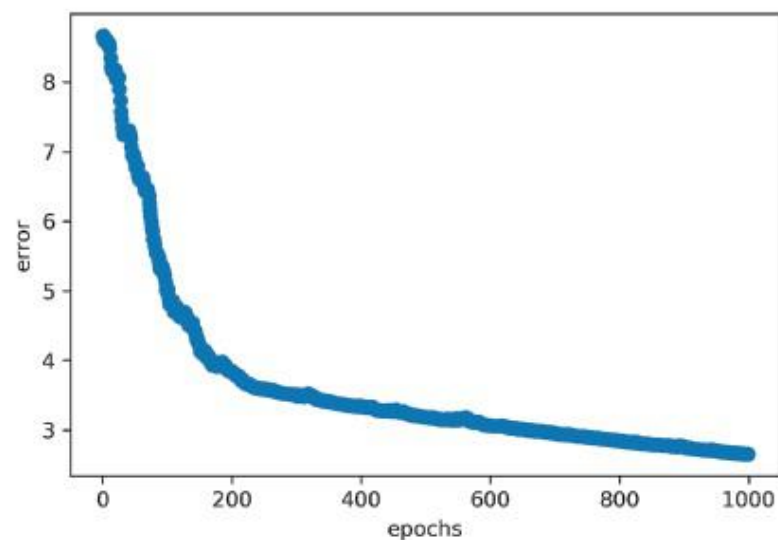
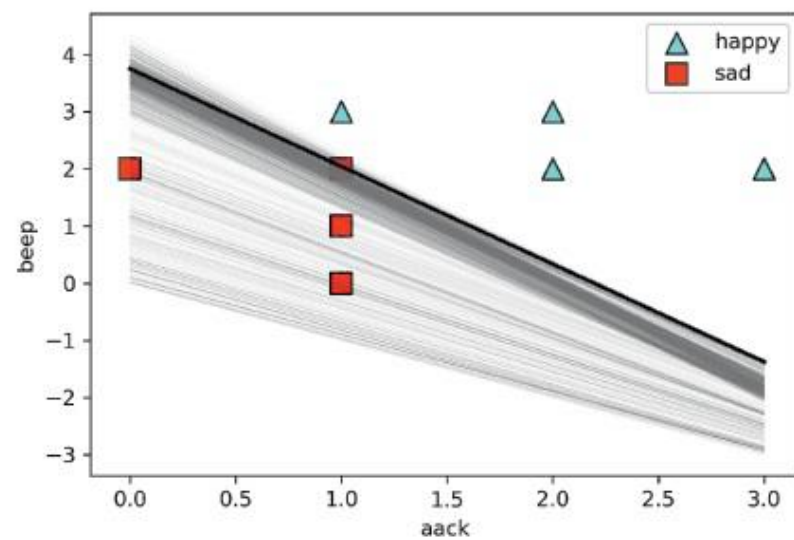


Hình 6.9 Ranh giới của bộ phân loại logistic thu được

Trong hình 6.10, chúng ta có thể thấy biểu đồ của các bộ phân loại tương ứng với tất cả các kỷ nguyên (trái) và biểu đồ mất log (phải). Trên biểu đồ của các phân loại trung gian, phân loại cuối cùng tương ứng với vạch tối. Lưu ý từ biểu đồ mất nhật ký rằng khi chúng tôi chạy thuật toán trong nhiều kỷ nguyên hơn, tổn thất nhật ký sẽ giảm đáng kể, đó chính xác là điều chúng tôi mong muốn. Hơn nữa, log loss không bao giờ bằng 0, mặc dù tất cả các điểm đều được phân loại chính xác. Điều này là do đối với bất kỳ điểm nào, dù được phân loại tốt đến đâu, log loss không bao giờ bằng không. Đối chiếu điều này với hình 5.26 trong chương 5, trong đó tổn thất perceptron thực sự đạt đến giá trị bằng 0 khi mọi điểm được phân loại chính xác.

Classifier

Error



Hình 6.10 Bên trái: Sơ đồ tất cả các bước trung gian của thuật toán hồi quy logistic. Lưu ý rằng chúng ta bắt đầu với một bộ phân loại kém và từ từ chuyển sang một bộ phân loại tốt (đường kẻ dày). Đúng: Sơ đồ lỗi. Lưu ý rằng chúng ta chạy thuật toán hồi quy logistic càng nhiều kỷ nguyên thì lỗi càng thấp.

Ứng dụng thực tế: Phân loại đánh giá IMDB bằng Turi Create

Trong phần này, chúng ta thấy ứng dụng thực tế của bộ phân loại logistic trong phân tích tình cảm. Chúng tôi sử dụng Turi Create để xây dựng mô hình phân tích các bài đánh giá phim trên trang IMDB phổ biến. Mã cho phần này như sau:

- **Sổ tay:** Sentiment_analysis_IMDB.ipynb
https://github.com/luiguiserrano/manning/blob/master/Chapter_6_Logistic_
- **Tập dữ liệu:** IMDB_Dataset.csv

Đầu tiên, chúng tôi nhập Turi Create, tải xuống tập dữ liệu và chuyển đổi nó thành SFrame mà chúng tôi gọi là `movies`, như sau:

```
import turicreate as tc
movies = tc.SFrame('IMDB Dataset.csv')
```

Năm hàng đầu tiên của tập dữ liệu xuất hiện trong bảng 6.10.

Bảng 6.10 Năm hàng đầu tiên của tập dữ liệu IMDB. Cột Đánh giá có nội dung đánh giá, còn cột Cảm tính có cảm nhận của đánh giá, có thể tích cực hoặc tiêu cực.

Đánh giá	Cảm giác
Một trong những người đánh giá khác đã đề cập...	Tích cực
Một sản phẩm nhỏ tuyệt vời...	Tích cực
Tôi nghĩ đây là một ngày tuyệt vời để dành...	Tích cực
Về cơ bản, có một gia đình có một chút...	Tiêu cực
“Tình yêu thời tiền” của Petter Mattei là một...	Tích cực

Tập dữ liệu có hai cột, một cột chứa đánh giá dưới dạng chuỗi và một cột chứa cảm tính, dưới dạng tích cực hoặc tiêu cực.Đầu tiên, chúng ta cần xử lý chuỗi, vì mỗi từ cần phải có một tính năng khác nhau.Hàm tích hợp Turi Create trong gói rất hữu ích cho tác vụ này vì nó biến một câu thành một từ điển có số lượng từ.Ví dụ, câu “to be or not to be” được đưa vào từ điển {‘to’:2, ‘be’:2, ‘or’:1, ‘not’:1}.Chúng ta thêm một cột mới tên là chứa từ điển này như sau:

```
words

movies['words'] = tc.text_analytics.count_words(movies['review'])
```

Một số hàng đầu tiên của tập dữ liệu của chúng tôi với cột mới được hiển thị trong bảng 6.11.

Bảng 6.11 Cột Từ là một từ điển trong đó mỗi từ trong phần đánh giá được ghi lại cùng với số lần xuất hiện của nó. Đây là cột đặc điểm dành cho bộ phân loại logistic của chúng tôi.

Đánh giá	Cảm giác	Từ
Một trong những người đánh giá khác đã đề cập...	Tích cực	{‘if’: 1.0, ‘đang xem’: 1.0, ‘thoải mái’: 1.0, ...
Một sản phẩm nhỏ tuyệt vời...	Tích cực	{‘xong’: 1.0, ‘mọi’: 1.0, ‘trang trí’: 1.0, ...
Tôi nghĩ đây là một ngày tuyệt vời để dành...	Tích cực	{‘thấy’: 1.0, ‘go’: 1.0, ‘tuyệt vời’: 1.0, ‘siêu ...
Về cơ bản, có một gia đình có một chút...	Tiêu cực	{‘chúng’: 1.0, ‘bỏ qua’: 1.0, ‘hộp thoại’: 1.0, ...
Tình yêu trong thời gian của tiền của Peter Mattei là một...	Tích cực	{‘work’: 1.0, ‘his’: 1.0, ‘for’: 1.0, ‘lo lắng’: ...

Chúng tôi đã sẵn sàng đào tạo mô hình của mình!Để làm điều này, chúng tôi sử dụng hàm trong logistic_classifiergói, trong đó chúng tôi chỉ định mục tiêu (nhãn) là cột và các tính năng là sentimentcột.Lưu ý rằng mục tiêu wordsđược biểu thị dưới dạng một chuỗi có tên cột chứa nhãn, nhưng các tính năng được biểu thị dưới dạng một chuỗi các chuỗi có tên của các cột

chứa từng tính năng (trong trường hợp chúng ta cần chỉ định một số cột), như được hiển thị ở đây:

```
model = tc.logistic_classifier.create(movies, features=['words'], target='sentiment')
```

Bây giờ chúng ta đã huấn luyện mô hình của mình, chúng ta có thể xem xét trọng số của các từ bằng lệnh `model.feature_weights`. Bảng chúng tôi thu được có một số cột, nhưng những cái chúng ta quan tâm là `words`, `weights` và `abs_weights`.

Năm người đứng đầu theo sau:

- (chặn): 0,065 nếu:
- −0,018 xem: 0,089
- thoải mái: 0,517 trở
- thành: 0,106
-

Cái đầu tiên, được gọi là đánh chặn, là độ lệch. Bởi vì thiên kiến của mô hình là tích cực nên đánh giá trống rỗng là tích cực, như chúng ta đã học ở chương 5, trong phần “Thành kiến, sự chận y và tâm trạng vốn có của một người ngoài hành tinh trầm lắng”. Điều này hợp lý vì những người dùng đánh giá phim tiêu cực có xu hướng để lại đánh giá, trong khi nhiều người dùng đánh giá phim tích cực lại không để lại bất kỳ đánh giá nào. Các từ khác là trung lập nên trọng số của chúng không có nhiều ý nghĩa, nhưng hãy khám phá trọng số của một số từ, chẳng hạn như *tuyệt vời*, *khủng khiếp* và *the*, như minh họa tiếp theo:

- tuyệt vời: 1,043
- khủng khiếp: −1,075
- the: 0,0005

Như chúng ta thấy, trọng số của từ *tuyệt vời* là dương, trọng lượng của từ *kinh khủng* là âm và trọng lượng của từ *the* là nhỏ. Điều này có ý nghĩa: *tuyệt vời* là một từ tích cực, *kinh khủng* là một từ tiêu cực và *the* là một từ trung tính.

Bước cuối cùng, hãy tìm những đánh giá tích cực và tiêu cực nhất. Để làm được điều này, chúng tôi sử dụng mô hình để đưa ra dự đoán cho tất cả các bộ phim. Những dự đoán này sẽ được lưu trữ trong một cột mới có tên `predictions`, sử dụng lệnh sau:

```
movies['prediction'] = model.predict(movies, output_type='probability')
```

Hãy cùng tìm ra những bộ phim tích cực nhất và tiêu cực nhất theo mô hình. Chúng tôi thực hiện việc này bằng cách sắp xếp mảng như sau:

Đánh giá tích cực nhất:

```
movies.sort('predictions')[-1]
```

Đầu ra: “Đôi với tôi, có vẻ như nhiều người không biết rằng *Blade* thực chất là một bộ phim siêu anh hùng sánh ngang với *X-Men*...”

Đánh giá tiêu cực nhất:

```
movies.sort('predictions')[0]
```

Đầu ra: “Thậm chí còn buồn tẻ hơn, nếu có thể, so với bản gốc...”

Chúng tôi có thể làm nhiều hơn nữa để cải thiện mô hình này. Ví dụ: một số kỹ thuật xử lý văn bản, chẳng hạn như xóa dấu câu và viết hoa hoặc xóa các từ dừng (chẳng hạn như *the*, *và*, *of*, *it*), có xu hướng mang lại cho chúng tôi kết quả tốt hơn. Nhưng thật tuyệt khi thấy rằng chỉ với một vài dòng mã, chúng ta có thể xây dựng bộ phân loại phân tích cảm xúc của riêng mình!

Phân loại thành nhiều lớp: Hàm softmax

Cho đến nay chúng ta đã thấy các perceptron liên tục phân loại hai lớp, vui và buồn. Nhưng nếu chúng ta có nhiều lớp hơn thì sao? Ở cuối chương 5, chúng ta đã thảo luận rằng việc phân loại giữa nhiều hơn hai lớp là khó đối với một perceptron rời rạc. Tuy nhiên, điều này có thể dễ dàng thực hiện được với bộ phân loại logistic.

Hãy tưởng tượng một tập dữ liệu hình ảnh có ba nhãn: “chó”, “mèo” và “chim”. Cách xây dựng bộ phân loại dự đoán một trong ba nhãn này cho mỗi hình ảnh là xây dựng ba bộ phân loại, một bộ phân loại cho mỗi nhãn. Khi có một hình ảnh mới xuất hiện, chúng tôi sẽ đánh giá nó bằng từng bộ phân loại trong số ba bộ phân loại. Bộ phân loại tương ứng với mỗi con vật trả về xác suất hình ảnh đó là con vật tương ứng. Sau đó, chúng tôi phân loại hình ảnh là động vật từ bộ phân loại có xác suất cao nhất.

Tuy nhiên, đây không phải là cách lý tưởng để thực hiện vì bộ phân loại này trả về một câu trả lời riêng biệt, chẳng hạn như “chó”, “mèo” hoặc “chim”. Điều gì sẽ xảy ra nếu chúng ta muốn một bộ phân loại trả về xác suất của ba con vật? Giả sử, câu trả lời có thể ở dạng “10% chó, 85% mèo và 5% chim”. Cách chúng tôi làm điều này là sử dụng softmax

chức năng.

Hàm softmax hoạt động như sau: nhớ lại rằng bộ phân loại logistic đưa ra dự đoán bằng cách sử dụng quy trình hai bước—đầu tiên, nó tính điểm, sau đó áp dụng hàm sigmoid cho điểm này. Hãy quên hàm sigmoid đi và thay vào đó hãy xuất ra điểm số. Bây giờ hãy tưởng tượng rằng ba bộ phân loại trả về các điểm sau:

- Phân loại chó: 3
- Phân loại mèo: 2
- Phân loại chim: -1

Làm thế nào để chúng ta biến những điểm số này thành xác suất? Chà, đây là một ý tưởng: chúng ta có thể bình thường hóa. Điều này có nghĩa là chia tất cả những số này cho tổng của chúng, tức là năm, để chúng cộng lại thành một. Khi làm điều này, chúng ta nhận được xác suất là 3/5 đối với chó, 2/5 đối với mèo và -1/5 đối với chim. Điều này có hiệu quả nhưng không lý tưởng vì xác suất hình ảnh là một con chim là một số âm. Xác suất phải luôn dương, vì vậy chúng ta cần thử điều gì đó khác biệt.

Cái chúng ta cần là một hàm số luôn dương và luôn tăng. Hàm số mũ hoạt động tốt cho việc này. Bất kỳ hàm mũ nào, chẳng hạn như 2^x , 3^x hoặc 10^x , đều có thể thực hiện được. Theo mặc định, chúng tôi sử dụng hàm e^x , hàm này có các đặc tính toán học tuyệt vời (ví dụ: đạo hàm của e^x cũng là e^x). Chúng tôi áp dụng hàm này cho điểm số để nhận được các giá trị sau:

- Phân loại chó: $e^3 = 20.085$
- Bộ phân loại mèo: $e^2 = 7.389$
- Phân loại chim: $e^{-1} = 0.368$

Bây giờ, chúng tôi làm những gì chúng tôi đã làm trước đây—chúng tôi chuẩn hóa hoặc chia cho tổng của các số này để chúng cộng thành một. Tổng là $20,085 + 7,389 + 0,368 = 27,842$ nên ta có:

- Xác suất của con chó: $20,085/27,842 = 0,721$ Xác
- suất của con mèo: $7,389/27,842 = 0,265$ Xác suất
- của con chim: $0,368/27,842 = 0,013$

Đây là ba xác suất được đưa ra bởi ba phân loại của chúng tôi. Hàm chúng tôi đã sử dụng là softmax và phiên bản chung như sau: nếu chúng tôi có các bộ phân loại n đưa ra n điểm $a_1, a_2, \dots, a_n$, thì xác suất thu được là $p_1, p_2, \dots, p_n$, ở đâu

$$p_i = \frac{e^{a_i}}{e^{a_1} + e^{a_2} + \dots + e^{a_n}}.$$

Công thức này được gọi là hàm softmax.

Điều gì sẽ xảy ra nếu chúng ta chỉ sử dụng hàm softmax cho hai lớp? Chúng ta thu được hàm sigmoid. Tại sao không thuyết phục bản thân về điều này như một bài tập?

Tóm tắt

- Perceptron liên tục, hay bộ phân loại logistic, tương tự như bộ phân loại perceptron, ngoại trừ việc thay vì đưa ra dự đoán rời rạc như 0 hoặc 1, chúng dự đoán bất kỳ số nào trong khoảng từ 0 đến 1.
- Các bộ phân loại logistic hữu ích hơn các perceptron rời rạc vì chúng cung cấp cho chúng ta nhiều thông tin hơn. Ngoài việc cho chúng ta biết lớp nào mà bộ phân loại dự đoán, chúng còn cho chúng ta một xác suất. Một bộ phân loại logistic tốt sẽ chỉ định xác suất thấp cho các điểm có nhãn 0 và xác suất cao cho các điểm có nhãn 1.
- Mật nhật ký là một hàm lỗi đối với các bộ phân loại hậu cần. Nó được tính riêng cho từng điểm dưới dạng âm của logarit tự nhiên của xác suất mà bộ phân loại gán cho nhãn của nó.
- Tổng tổn thất nhật ký của bộ phân loại trên tập dữ liệu là tổng tổn thất nhật ký tại mọi điểm.
- Thủ thuật logistic lấy một điểm dữ liệu được gán nhãn và một đường ranh giới. Nếu điểm được phân loại không chính xác, đường sẽ được di chuyển đến gần điểm hơn và nếu được phân loại chính xác, đường sẽ được di chuyển ra xa điểm hơn. Điều này hữu ích hơn thủ thuật perceptron vì thủ thuật perceptron không di chuyển đường thẳng nếu điểm được phân loại chính xác.
- Thuật toán hồi quy logistic được sử dụng để khớp bộ phân loại logistic với tập dữ liệu được gán nhãn. Nó bao gồm việc bắt đầu với một bộ phân loại logistic có trọng số ngẫu nhiên và liên tục chọn một điểm ngẫu nhiên và áp dụng thủ thuật logistic để có được bộ phân loại tốt hơn một chút.
- Khi có một số lớp cần dự đoán, chúng ta có thể xây dựng một số bộ phân loại tuyến tính và kết hợp chúng bằng hàm softmax.

Bài tập

BÀI TẬP 6.1

Một nha sĩ đã đào tạo một bộ phân loại logistic trên tập dữ liệu của bệnh nhân để dự đoán xem họ có bị sâu răng hay không. Mô hình đã xác định được xác suất để một bệnh nhân bị sâu răng là

$$\sigma(d + 0.5c - 0.8),$$

Ở đây

- d là biến cho biết trước đây bệnh nhân có bị sâu răng nữa hay không và c là biến cho biết bệnh nhân có ăn kẹo hay không.
-

Ví dụ: nếu một bệnh nhân ăn kẹo thì $c = 1$, còn nếu không thì $c = 0$. Xác suất để một bệnh nhân ăn kẹo và đã được điều trị răng sâu năm ngoái sẽ bị sâu răng hôm nay là bao nhiêu?

BÀI TẬP 6.2

Hãy xem xét trình phân loại logistic gán cho điểm (x_1, x_2) dự đoán $\hat{y} = \sigma(2x_1 + 3x_2 - 4)$, và điểm $p = (1, 1)$ có nhãn 0.

Một. Tính toán dự đoán $\hat{y}$ mà mô hình đưa ra cho điểm p . b. Tính log loss mà mô hình tạo ra tại điểm p . c. Sử dụng thủ thuật logistic để có được mô hình mới tạo ra tổn thất log nhỏ hơn. Bạn có thể sử dụng $\eta = 0,1$ làm tốc độ học tập. d. Tìm dự đoán mà mô hình mới đưa ra tại điểm p và xác minh rằng log loss thu được nhỏ hơn ban đầu.

BÀI TẬP 6.3

Sử dụng mô hình trong phát biểu của bài tập 6.2, hãy tìm một điểm mà tại đó dự đoán là 0,8.

gợi ý Trước tiên hãy tìm điểm sẽ đưa ra dự đoán là 0,8 và nhớ lại rằng dự đoán đó là $\hat{y} = \sigma(\text{điểm})$.

7 Bạn đo lường các mô hình phân loại như thế nào? Độ chính xác và những người bạn của nó

Trong chương này

- các loại lỗi mà mô hình có thể mắc phải: dương tính giả và âm tính giả đưa các lỗi này vào bảng:
- ma trận nhầm lẫn độ chính xác, thu hồi, độ chính xác, điểm F, độ nhạy và độ đặc hiệu là gì và
- chúng được sử dụng như thế nào để đánh giá mô hình Đường cong ROC là gì và nó theo dõi độ nhạy và độ đặc hiệu cùng một lúc như thế nào

•



Chương này hơi khác so với hai chương trước - nó không tập trung vào việc xây dựng các mô hình phân loại; thay vào đó, nó tập trung vào việc đánh giá chúng. Đối với một chuyên gia về máy học, việc có thể đánh giá hiệu suất của các mô hình khác nhau cũng quan trọng như khả năng đào tạo chúng. Chúng tôi hiếm khi đào tạo một mô hình duy nhất trên một tập dữ liệu; chúng tôi đào tạo một số mô hình khác nhau và chọn mô hình hoạt động tốt nhất. Chúng ta cũng cần đảm bảo các mẫu mã có chất lượng tốt trước khi đưa vào sản xuất. Chất lượng của một mô hình không phải lúc nào cũng dễ đo lường và trong chương này, chúng ta tìm hiểu một số kỹ thuật để đánh giá các mô hình phân loại của mình. Trong chương 4, chúng ta đã học cách đánh giá các mô hình hồi quy, vì vậy chúng ta có thể coi chương này là chương tương tự nhưng có các mô hình phân loại.

Cách đơn giản nhất để đo lường hiệu suất của mô hình phân loại là tính toán độ chính xác của nó. Tuy nhiên, chúng ta sẽ thấy rằng độ chính xác không thể hiện được toàn bộ

hình ảnh, bởi vì một số mô hình thể hiện độ chính xác cao nhưng dù sao cũng không phải là mô hình tốt. Để khắc phục điều này, chúng tôi sẽ xác định một số số liệu hữu ích, chẳng hạn như độ chính xác và khả năng thu hồi. Sau đó, chúng tôi sẽ kết hợp chúng thành một số liệu mới mạnh mẽ hơn được gọi là điểm F. Các số liệu này được các nhà khoa học dữ liệu sử dụng rộng rãi để đánh giá mô hình của họ. Tuy nhiên, trong các ngành khác, chẳng hạn như y học, các số liệu tương tự khác cũng được sử dụng, chẳng hạn như độ nhạy và độ đặc hiệu. Sử dụng hai số liệu cuối cùng này, chúng ta sẽ có thể xây dựng một đường cong được gọi là đường cong đặc tính vận hành máy thu (ROC). Đường cong ROC là một biểu đồ đơn giản cung cấp cho chúng ta những hiểu biết sâu sắc về các mô hình của mình.

Độ chính xác: Mô hình của tôi có thường xuyên đúng không?

Trong phần này, chúng ta thảo luận về độ chính xác, thước đo đơn giản và phổ biến nhất của các mô hình phân loại. Độ chính xác của mô hình là tỷ lệ phần trăm số lần mô hình đúng. Nói cách khác, đó là tỷ lệ giữa số lượng điểm dữ liệu được dự đoán chính xác và tổng số điểm dữ liệu. Ví dụ: nếu chúng ta đánh giá một mô hình trên tập dữ liệu thử nghiệm gồm 1.000 mẫu và mô hình đó dự đoán đúng nhãn của mẫu 875 lần thì mô hình này có độ chính xác là $875/1000=0,875$, hay 87,5%.

Độ chính xác là cách phổ biến nhất để đánh giá mô hình phân loại và chúng ta nên luôn sử dụng nó. Tuy nhiên, đôi khi độ chính xác không mô tả đầy đủ hiệu suất của mô hình, như chúng ta sẽ thấy ngay sau đây. Hãy bắt đầu bằng cách xem xét hai ví dụ mà chúng ta sẽ nghiên cứu trong suốt chương này.

Hai ví dụ về mô hình: Virus Corona và email spam

Trong chương này, chúng tôi sử dụng số liệu của mình để đánh giá một số mô hình trên hai bộ dữ liệu. Tập dữ liệu đầu tiên là tập dữ liệu y tế về bệnh nhân, trong đó một số người trong số họ đã được chẩn đoán mắc bệnh coronavirus. Tập dữ liệu thứ hai là tập dữ liệu gồm các email đã bị gắn nhãn là thư rác hoặc không phải thư rác. Như chúng ta đã học ở chương 1, *spam* là thuật ngữ dùng cho email rác và *ham* là thuật ngữ dùng cho email không phải là thư rác. Trong chương 8, chúng ta sẽ nghiên cứu tập dữ liệu như thế này chi tiết hơn nhiều khi chúng ta tìm hiểu thuật toán Bayes đơn giản. Trong chương này, chúng ta không xây dựng mô hình. Thay vào đó, chúng tôi sử dụng các mô hình làm hộp đen và đánh giá chúng dựa trên số lượng điểm dữ liệu mà chúng dự đoán đúng hoặc sai. Cả hai bộ dữ liệu đều hoàn toàn là tưởng tượng.

TẬP DỮ LIỆU Y TẾ: TẬP ĐOÀN BỆNH NHÂN ĐƯỢC CHẨN ĐOÁN BỊ CORONAVIRUS

Tập dữ liệu đầu tiên của chúng tôi là tập dữ liệu y tế với 1.000 bệnh nhân. Trong số đó, 10 người được chẩn đoán mắc virus Corona và 990 người còn lại được chẩn đoán là khỏe mạnh. Do đó, các nhãn trong tập dữ liệu này là “ốm” hoặc “khỏe mạnh”, tương ứng với chẩn đoán. Mục tiêu của mô hình là dự đoán chẩn đoán dựa trên đặc điểm của từng bệnh nhân.

TẬP DỮ LIỆU EMAIL: TẬP HỢP CÁC EMAIL ĐƯỢC GHI NHÃN THƯ RÁC HOẶC HAM

Tập dữ liệu thứ hai của chúng tôi là tập dữ liệu gồm 100 email. Trong số đó, 40 là thư rác và 60 còn lại là ham. Các nhãn trong tập dữ liệu này là “thư rác” và “ham” và mục tiêu của mô hình là dự đoán nhãn dựa trên các tính năng của email.

Một mô hình siêu hiệu quả nhưng lại siêu vô dụng

Độ chính xác là một thước đo rất hữu ích nhưng liệu nó có vẽ nên bức tranh toàn cảnh về mô hình không? Không, và chúng tôi sẽ minh họa điều này bằng một ví dụ. Hiện tại, hãy tập trung vào bộ dữ liệu coronavirus. Chúng ta sẽ quay lại tập dữ liệu email trong phần tiếp theo.

Giả sử một nhà khoa học dữ liệu nói với chúng ta như sau: “Tôi đã phát triển một xét nghiệm tìm vi-rút Corona, chạy trong 10 giây, không yêu cầu bất kỳ xét nghiệm nào và có độ chính xác là 99%!” Chúng ta nên vui mừng hay hoài nghi? Có lẽ chúng tôi sẽ hoài nghi. Tại sao? Chúng ta sẽ sớm thấy rằng việc tính toán độ chính xác của mô hình đôi khi là chưa đủ. Mô hình của chúng tôi có thể có độ chính xác 99% nhưng hoàn toàn vô dụng.

Chúng ta có thể nghĩ ra một mô hình hoàn toàn vô dụng có thể dự đoán virus Corona trong tập dữ liệu của mình nhưng vẫn chính xác đến 99% không? Hãy nhớ lại rằng tập dữ liệu của chúng tôi bao gồm 1.000 bệnh nhân và trong số đó, 10 người mắc bệnh coronavirus. Vui lòng đặt cuốn sách này xuống một lúc và nghĩ cách xây dựng một mô hình phát hiện coronavirus và điều đó đúng 99% đối với tập dữ liệu này.

Nó có thể là một mô hình như thế này: chỉ cần chẩn đoán mọi bệnh nhân đều khỏe mạnh. Đó là một mô hình đơn giản nhưng vẫn là một mô hình; đó là mô hình dự đoán mọi thứ dưới dạng một lớp.

Độ chính xác của mô hình này là gì?Chà, trong số 1.000 lần thử, nó sai 10 lần và đúng 990 lần.Điều này mang lại độ chính xác 99%, giống như chúng tôi đã hứa. Tuy nhiên, mô hình này tương đương với việc nói với mọi người rằng họ vẫn khỏe mạnh giữa đại dịch toàn cầu, thật kinh khủng!

Vậy vấn đề với mô hình của chúng ta là gì?Vấn đề là các lỗi không được tạo ra như nhau và một số lỗi gây ra hậu quả nghiêm trọng hơn nhiều so với những lỗi khác, như chúng ta sẽ thấy trong phần tiếp theo.

Làm cách nào để khắc phục vấn đề về độ chính xác?Xác định các loại lỗi khác nhau và cách đo lường chúng

Ở phần trước, chúng ta đã xây dựng một mô hình vô dụng nhưng có độ chính xác cao.Trong phần này, chúng ta nghiên cứu xem điều gì đã xảy ra.Cụ thể là, chúng tôi nghiên cứu vấn đề xảy ra với việc tính toán độ chính xác trong mô hình đó và chúng tôi giới thiệu một số số liệu hơi khác một chút sẽ giúp chúng tôi đánh giá tốt hơn về mô hình này.

Điều đầu tiên chúng ta cần nghiên cứu là các loại lỗi.Trong phần tiếp theo, chúng ta thấy rằng một số lỗi nghiêm trọng hơn những lỗi khác.Sau đó trong phần “Lưu trữ chính xác và không chính xác

các điểm được phân loại trong bảng” đến “Thu hồi, độ chính xác hoặc điểm F”, chúng tôi tìm hiểu các số liệu khác nhau được trang bị nhiều hơn để phát hiện những lỗi nghiêm trọng này hơn là độ chính xác.

Dương tính giả và âm tính giả: Cái nào tệ hơn?

Trong nhiều trường hợp, tổng số lỗi không cho chúng ta biết mọi thứ về hiệu suất của mô hình và chúng ta cần tìm hiểu sâu hơn và xác định một số loại lỗi nhất định theo nhiều cách khác nhau.Trong phần này, chúng ta thấy có hai loại lỗi.Hai loại lỗi mà mô hình virus Corona có thể mắc phải là gì?Nó có thể chẩn đoán một người khỏe mạnh là bị bệnh hoặc một người bệnh là khỏe mạnh.Trong mô hình của chúng tôi, theo quy ước, chúng tôi dán nhãn các bệnh nhân bị bệnh là dương tính.Hai loại lỗi được gọi là dương tính giả và âm tính giả, như sau:

- **Dương tính giả:** một người khỏe mạnh bị chẩn đoán nhầm là bị bệnh

- **Âm tính giả:** người bệnh được chẩn đoán nhầm là khỏe mạnh

Trong cài đặt chung, dương tính giả là điểm dữ liệu có nhãn âm nhưng mô hình phân loại sai nó thành dương. Âm tính giả là điểm dữ liệu có nhãn dương nhưng mô hình đã phân loại sai nó thành âm.Đương nhiên, những trường hợp được chẩn đoán chính xác cũng có tên như sau:

- **Dương tính thực sự:** một người bệnh được chẩn đoán là ốm **Âm tính thực**
- **sự:** một người khỏe mạnh được chẩn đoán là khỏe mạnh

Trong cài đặt chung, điểm dương thực sự là điểm dữ liệu có nhãn dương được phân loại chính xác là dương và điểm âm thực là điểm có nhãn âm được phân loại chính xác là âm.

Bây giờ, hãy xem tập dữ liệu email.Giả sử chúng ta có một mô hình dự đoán xem mỗi email là thư rác hay vi phạm.Chúng tôi coi những điều tích cực là các email spam.Do đó, hai loại lỗi của chúng tôi như sau:

- **Dương tính giả:** một email ham được phân loại không chính xác là thư rác **Âm**
- **tính giả:** một email spam được phân loại không chính xác là ham

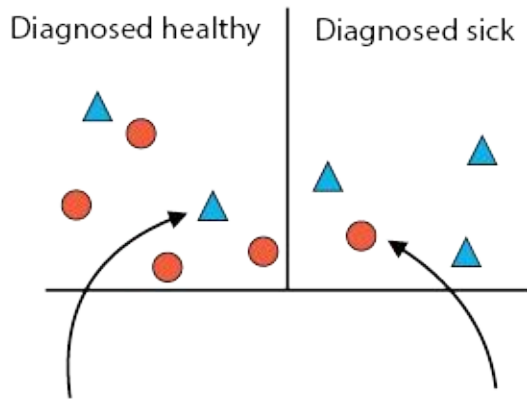
Và các email được phân loại chính xác như sau:

- **Tích cực thực sự:** một email spam được phân loại chính xác là thư rác **Tiêu cực**
- **thực sự:** một email ham được phân loại chính xác là ham

Hình 7.1 biểu diễn đồ họa của các mô hình, trong đó đường thẳng đứng là ranh giới, vùng bên trái đường là vùng âm và vùng bên phải là vùng dương.Các hình tam giác là những điểm có nhãn dương và các hình tròn là những điểm có nhãn âm.Bốn đại lượng được xác định ở trên như sau:

- Tam giác ở bên phải đường thẳng: dương tính thật Tam
- giác ở bên trái đường thẳng: âm tính giả Vòng tròn ở
- bên phải đường thẳng: dương tính giả Vòng tròn ở bên
- trái đường thẳng: âm tính thật

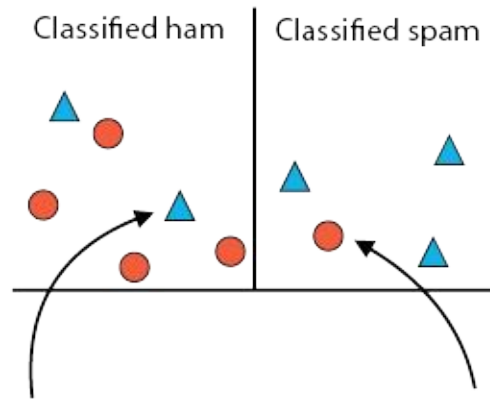
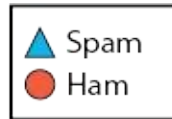
Coronavirus model



False negative:
Sick person
not treated

False positive:
Healthy person
sent for more tests

Spam model



False negative:
Spam email
sent to inbox

False positive:
Ham email
sent to spam folder

Hình 7.1 Ví dụ về hai mô hình được sử dụng rộng rãi trong đời sống thực và chúng ta sẽ sử dụng trong suốt chương này. Ở bên trái, một mô hình virus corona trong đó mọi người được chẩn đoán là khỏe mạnh hoặc bị bệnh. Ở bên phải, mô hình phát hiện thư rác trong đó các email được phân loại là thư rác hoặc vi phạm. Đối với mỗi mô hình, chúng tôi đã nêu bật một số lỗi của chúng và tách chúng thành lỗi dương tính giả và âm tính giả.

Lưu ý rằng cả hai mô hình trong hình 7.1 đều tạo ra các đại lượng sau:

- Ba kết quả dương tính
- đúng Bốn kết quả âm tính
- đúng Một kết quả dương
- tính giả Hai kết quả âm
- tính giả

Để thấy sự khác biệt giữa mô hình coronavirus và mô hình thư rác, chúng ta cần phân tích mô hình nào tệ hơn giữa kết quả dương tính giả và âm tính giả. Hãy làm điều này cho từng mô hình riêng biệt.

PHÂN TÍCH MỨC TÍCH CỰC SAI VÀ TIÊU CỰC SAI TRONG MÔ HÌNH CORONAVIRUS

Hãy dừng lại và suy nghĩ. Trong mô hình coronavirus, lỗi nào nghe có vẻ là sai lầm tồi tệ hơn: dương tính giả hay âm tính giả? Nói cách khác, điều gì tệ hơn: chẩn đoán sai một bệnh nhân khỏe mạnh là bị bệnh, hay một bệnh nhân bị bệnh là khỏe mạnh? Giả sử rằng khi chúng tôi chẩn đoán một bệnh nhân là khỏe mạnh, chúng tôi sẽ gửi họ về nhà mà không cần điều trị và khi chúng tôi chẩn đoán một bệnh nhân bị bệnh, chúng tôi sẽ gửi họ đi khám thêm.

các bài kiểm tra. Việc chẩn đoán sai một người khỏe mạnh có thể gây phiền toái nhỏ, vì điều đó có nghĩa là người khỏe mạnh sẽ phải ở lại để xét nghiệm thêm. Tuy nhiên, chẩn đoán sai người bệnh có nghĩa là người bệnh sẽ không được điều trị, tình trạng của họ có thể trở nên tồi tệ hơn và có khả năng lây nhiễm cho nhiều người khác. Do đó, **trong mô hình coronavirus, kết quả âm tính giả còn tệ hơn nhiều so với kết quả dương tính giả.**

PHÂN TÍCH MỤC TIÊU CỰC SAI TRONG MÔ HÌNH EMAIL THƯ RÁC

Bây giờ chúng ta sẽ thực hiện phân tích tương tự trên mô hình thư rác. Trong trường hợp này, giả sử rằng nếu trình phân loại thư rác của chúng tôi phân loại một email là thư rác thì email này sẽ tự động bị xóa. Nếu nó phân loại nó là ham thì email sẽ được gửi đến hộp thư đến của chúng tôi. Điều nào nghe có vẻ là một sai lầm tồi tệ hơn: dương tính giả hay âm tính giả? Nói cách khác, điều gì tệ hơn, phân loại sai một email ham là thư rác và xóa nó, hoặc phân loại sai một email spam là ham và gửi nó vào hộp thư đến? Tôi nghĩ chúng ta có thể đồng ý rằng việc xóa một email tốt còn tệ hơn nhiều so với việc gửi một email spam vào hộp thư đến. Email spam thỉnh thoảng xuất hiện trong hộp thư đến của chúng tôi có thể gây khó chịu, nhưng một email bị xóa có thể là một thảm họa hoàn toàn! Hãy tưởng tượng nỗi buồn mà chúng ta sẽ cảm thấy trong lòng nếu bà của chúng ta gửi cho chúng ta một email tử tế nói rằng bà đã nướng bánh quy và bộ lọc của chúng tôi đã xóa nó. Do đó, **trong mô hình email spam, kết quả dương tính giả còn tệ hơn nhiều so với kết quả âm tính giả.**

Đây là điểm khác biệt giữa hai mô hình. Trong mô hình coronavirus, kết quả âm tính giả còn tệ hơn, trong khi ở mô hình email spam, kết quả dương tính giả còn tệ hơn. Vấn đề khi đo lường độ chính xác trong bất kỳ mô hình nào trong hai mô hình này là độ chính xác coi cả hai loại lỗi đều nghiêm trọng như nhau và không phân biệt chúng.

Trong phần “Mô hình siêu hiệu quả nhưng siêu vô dụng”, chúng tôi có một ví dụ về mô hình chẩn đoán mọi bệnh nhân đều khỏe mạnh. Mô hình này chỉ mắc 10 lỗi trong số 1.000 bệnh nhân. Tuy nhiên, cả 10 trường hợp đó đều là âm tính giả, điều này thật khủng khiếp. Thay vào đó, nếu 10 kết quả đó là dương tính giả thì mô hình sẽ tốt hơn nhiều.

Trong các phần sau, chúng tôi sẽ đưa ra hai số liệu mới, tương tự như độ chính xác. Số liệu đầu tiên giúp chúng tôi xử lý các mô hình trong đó kết quả âm tính giả tệ hơn và số liệu thứ hai giúp chúng tôi xử lý các mô hình trong đó kết quả dương tính giả tệ hơn.

Lưu trữ các điểm được phân loại đúng và sai trong bảng: Ma trận nhầm lẫn

Trong tiêu mục trước, chúng ta đã tìm hiểu về dương tính giả, âm tính giả, dương tính thật và âm tính thật. Để theo dõi bốn thực thể này, chúng tôi đặt chúng cùng nhau trong một bảng được đặt tên phù hợp là *ma trận nhầm lẫn*. Đối với các mô hình phân loại nhị phân (mô hình dự đoán hai lớp), ma trận nhầm lẫn có hai hàng và hai cột. Trong các hàng, chúng tôi viết các nhãn thực sự (trong ví dụ y tế, đây là tình trạng của một người, ốm hay khỏe mạnh) và trong các cột, chúng tôi viết các nhãn dự đoán (chẩn đoán một người, ốm hay khỏe mạnh). Ma trận nhầm lẫn chung được minh họa trong bảng 7.1 và ma trận cụ thể cho các ví dụ về mô hình trong hai bộ dữ liệu này được trình bày trong bảng 7.2 đến 7.5. Đây được gọi là ma trận nhầm lẫn vì nó giúp dễ dàng nhận biết liệu mô hình có nhầm lẫn giữa hai lớp hay không, đó là lớp tích cực (bệnh) và lớp tiêu cực (khỏe mạnh).

Bảng 7.1 Ma trận nhầm lẫn giúp chúng ta đếm số lần mỗi lớp được dự đoán đúng và bao nhiêu lần mỗi lớp bị nhầm lẫn với một lớp khác. Trong ma trận này, các hàng biểu thị nhãn và các cột biểu thị dự đoán. Các phần tử trong đường chéo được phân loại chính xác, còn các phần tử ngoài đường chéo thì không.

Tình trạng của người đó	Dự đoán dương tính	Dự đoán âm tính
Tích cực	Số lượng dương tính thực sự	Số lượng âm tính giả
Tiêu cực	Số lượng dương tính giả	Số lượng âm tính thực sự

Đối với mô hình hiện tại của chúng tôi (mô hình chẩn đoán mọi bệnh nhân đều khỏe mạnh), mà từ giờ trở đi chúng tôi gọi là mô hình coronavirus 1, ma trận nhầm lẫn được minh họa trong bảng 7.2.

Bảng 7.2 Ma trận nhầm lẫn của mô hình vi-rút Corona giúp chúng tôi đào sâu vào mô hình của mình và phân biệt hai loại lỗi. Mô hình này mắc 10 lỗi âm tính giả (một người bệnh được chẩn đoán khỏe mạnh) và không có lỗi dương tính giả nào (một người khỏe mạnh được chẩn đoán ốm). Lưu ý rằng mô hình tạo ra quá nhiều âm tính giả, đây là loại lỗi nặng nhất trong trường hợp này, điều này hàm ý rằng mô hình này không tốt lắm.

Mô hình vi-rút Corona 1	Được chẩn đoán ốm (dự đoán dương tính)	Được chẩn đoán khỏe mạnh (dự đoán âm tính)
Bệnh (dương tính)	0 (số lượng dương tính thật)	10 (số âm tính giả)
Khỏe mạnh (tiêu cực)	0 (số lượng dương tính giả)	990 (số âm tính thật)

Đối với các bài toán có nhiều lớp hơn, chúng ta có ma trận nhầm lẫn lớn hơn. Ví dụ, nếu

mô hình của chúng tôi phân loại hình ảnh thành lợn đất, chim, mèo và chó, sau đó ma trận nhầm lẫn của chúng tôi là ma trận 4 x 4, trong đó dọc theo các hàng chúng tôi có nhãn thực (loại động vật) và dọc theo các cột chúng tôi có nhãn dự đoán (loại động vật mà mô hình dự đoán). Ma trận nhầm lẫn này còn có đặc tính là những điểm được phân loại đúng sẽ được tính theo đường chéo, còn những điểm được phân loại sai sẽ được tính theo đường chéo.

Nhớ lại: Trong số các ví dụ tích cực, chúng ta đã phân loại đúng bao nhiêu ví dụ?

Bây giờ chúng ta đã biết hai loại lỗi, trong phần này, chúng ta tìm hiểu một số liệu sẽ mang lại cho mô hình vi-rút Corona 1 điểm thấp hơn nhiều. Chúng tôi đã xác định rằng vấn đề với mô hình này là nó cho chúng ta quá nhiều kết quả âm tính sai, cụ thể là nó chẩn đoán có quá nhiều người bệnh là những người khỏe mạnh.

Trong giây lát, hãy giả sử rằng chúng ta không bận tâm đến những kết quả dương tính giả. Giả sử rằng nếu mô hình chẩn đoán một người khỏe mạnh bị bệnh thì người đó có thể cần phải làm xét nghiệm bổ sung hoặc cách ly lâu hơn một chút, nhưng điều này không có vấn đề gì cả. Đương nhiên, đây không phải là trường hợp; dương tính giả cũng đắt, nhưng bây giờ, hãy giả vờ như không phải vậy. Trong trường hợp này, chúng tôi cần một số liệu thay thế độ chính xác và đặt tầm quan trọng vào việc tìm kiếm các trường hợp tích cực và ít quan tâm đến việc phân loại nhầm các trường hợp tiêu cực.

Để tìm số liệu này, chúng ta cần đánh giá mục tiêu của chúng ta là gì. Nếu chúng ta muốn chữa khỏi virus Corona, thì điều chúng ta thực sự mong muốn là: trong số tất cả những người bệnh trên thế giới, chúng ta muốn tìm ra tất cả họ. Sẽ không có vấn đề gì nếu chúng ta vô tình tìm thấy những người khác không bị bệnh, miễn là chúng ta tìm thấy tất cả những người bị bệnh. Đây là chìa khóa. Số liệu mới này, được gọi là *thu hồi*, đo lường chính xác rằng: trong số những người bị bệnh, có bao nhiêu người được mô hình của chúng tôi chẩn đoán chính xác?

Trong biệt ngữ tổng quát hơn, việc thu hồi tìm thấy tỷ lệ dự đoán đúng trong số các điểm dữ liệu có nhãn dương. Đây là số lượng dương tính thực sự, chia cho tổng số lượng dương tính. Mô hình vi-rút Corona 1 có tổng cộng 0 trường hợp dương tính thực sự trong số 10 trường hợp dương tính, do đó tỷ lệ thu hồi của nó là $0/10 = 0$. Một cách khác để diễn đạt là lấy số trường hợp dương tính thực chia cho tổng số trường hợp dương tính thật và âm tính giả, như được hiển thị ở đây:

$$\text{Recall} = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}}$$

Ngược lại, giả sử chúng ta có mô hình thứ hai gọi là mô hình coronavirus 2. Ma trận nhầm lẫn của mô hình này được hiển thị trong bảng 7.3. Mô hình thứ hai này mắc nhiều lỗi hơn mô hình thứ nhất—nó mắc tổng cộng 50 lỗi so với chỉ 10. Độ chính xác của mô hình thứ hai là $950/1000 = 0,95$, hay 95%. Về độ chính xác, mô hình thứ hai không tốt bằng mô hình thứ nhất.

Tuy nhiên, mô hình thứ hai đã chẩn đoán chính xác 8 trong số 10 người bị bệnh và 942 trong số 1.000 người. Nói cách khác, nó có hai kết quả âm tính giả và 48 kết quả dương tính giả.

Bảng 7.3 Ma trận nhầm lẫn của mô hình virus Corona thứ hai của chúng tôi

Mô hình vi-rút Corona 2	Được chẩn đoán bị bệnh	Được chẩn đoán khỏe mạnh
Đau ốm	8 (dương tính thực sự)	2 (âm tính giả)
Khỏe mạnh	48 (dương tính giả)	942 (âm tính thật)

Việc thu hồi mô hình này là lấy số ca dương tính thực sự (tám người bệnh được chẩn đoán chính xác) chia cho tổng số ca dương tính (10 người bệnh), là $8/10=0,8$, hay 80%. Về khả năng thu hồi, mẫu thứ hai tốt hơn nhiều. Hãy tóm tắt các tính toán này cho rõ ràng như sau:

Mô hình vi-rút Corona 1:

Kết quả dương tính thực sự (bệnh nhân được chẩn đoán mắc bệnh và được gửi đi xét nghiệm thêm) = 0

Âm tính giả (bệnh nhân được chẩn đoán khỏe mạnh và cho về nhà) = 10

Nhớ lại = $0/10=0\%$

Mô hình vi-rút Corona 2:

True dương tính (bệnh nhân được chẩn đoán mắc bệnh và được gửi đi xét nghiệm thêm) = 8

Âm tính giả (bệnh nhân được chẩn đoán khỏe mạnh và cho về nhà) = 2

Nhớ lại = $10/8= 80\%$

Các mô hình như mô hình vi-rút Corona, trong đó kết quả âm tính giả đắt hơn nhiều so với kết quả dương tính giả, là *mô hình có khả năng thu hồi cao*.

Bây giờ chúng ta đã có số liệu tốt hơn, liệu chúng ta có thể đánh lừa số liệu này giống như cách chúng ta đánh lừa độ chính xác không? Nói cách khác, liệu chúng ta có thể xây dựng một mô hình có khả năng thu hồi toàn bộ không? Tốt,

hãy sẵn sàng cho một điều bất ngờ, bởi vì chúng ta có thể. Nếu chúng tôi xây dựng một mô hình chẩn đoán mọi bệnh nhân đều bị bệnh thì mô hình này có tỷ lệ thu hồi 100%. Tuy nhiên, mô hình này cũng tệ vì mặc dù nó không có âm tính giả nhưng lại có quá nhiều âm tính giả để biến nó thành một mô hình tốt. Có vẻ như chúng ta vẫn cần thêm số liệu để có thể đánh giá đúng mô hình của mình.

Độ chính xác: Trong số các ví dụ chúng tôi đã phân loại là tích cực, chúng tôi đã phân loại chính xác bao nhiêu ví dụ?

Trong phần trước, chúng ta đã tìm hiểu về khả năng thu hồi, một thước đo đo lường mức độ hoạt động của mô hình của chúng tôi đối với các kết quả âm tính giả. Số liệu đó hoạt động tốt đối với mô hình vi-rút Corona — chúng tôi đã thấy rằng mô hình này không thể có quá nhiều kết quả âm tính giả. Trong phần này, chúng ta tìm hiểu về một chỉ số tương tự, *độ chính xác*, đo lường mức độ hoạt động của mô hình của chúng tôi với các kết quả dương tính giả. Chúng tôi sẽ sử dụng số liệu này để đánh giá mô hình email spam vì mô hình này không thể có quá nhiều kết quả dương tính giả.

Giống như chúng ta đã làm với việc thu hồi, để đưa ra một thước đo, trước tiên chúng ta cần xác định mục tiêu của mình. Chúng tôi muốn một bộ lọc thư rác không xóa bất kỳ email vi phạm nào. Nếu thay vì xóa email, nó sẽ gửi chúng vào hộp thư rác. Sau đó, chúng ta cần xem xét hộp thư rác đó và hy vọng rằng chúng ta không nhìn thấy một email ham nào. Do đó, số liệu của chúng tôi phải đo lường chính xác rằng: Trong số các email trong hộp thư thư rác, có bao nhiêu email thực sự là thư rác? Nói cách khác, trong số những email được dự đoán là thư rác thì có bao nhiêu trong số đó thực sự là thư rác? Đây là số liệu của chúng tôi và chúng tôi gọi nó là *độ chính xác*.

Chính thức hơn, độ chính xác chỉ xem xét các điểm dữ liệu đã được gắn nhãn dương tính và trong số đó có bao nhiêu điểm là dương tính thực sự. Bởi vì các điểm dữ liệu được dự đoán là dương tính là sự kết hợp giữa dương tính thực và dương tính giả nên công thức như sau:

$$\text{Precision} = \frac{\text{True positives}}{\text{True positives} + \text{False positives}}$$

Hãy nhớ rằng trong tập dữ liệu 100 email của chúng tôi, 40 email là thư rác và 60 email là thư rác. Giả sử chúng tôi đã đào tạo hai mô hình sau đây được gọi là mô hình thư rác 1 và mô hình thư rác 2. Ma trận nhầm lẫn của chúng được hiển thị trong bảng 7.4 và 7.5.

Bảng 7.4 Ma trận nhầm lẫn của mô hình thư rác đầu tiên của chúng tôi

Mô hình thư rác 1	Dự đoán thư rác	Dự đoán ham
Thư rác	30 (dương tính thực sự)	10 (âm tính giả)
Ham	5 (dương tính giả)	55 (âm tính thật)

Bảng 7.5 Ma trận nhầm lẫn của mô hình thư rác thứ hai của chúng tôi

Mô hình thư rác 2	Dự đoán thư rác	Dự đoán ham
Thư rác	35 (dương tính thực sự)	5 (âm tính giả)
Ham	10 (dương tính giả)	50 (âm tính thật)

Về độ chính xác, có vẻ như cả hai mô hình đều tốt như nhau—cả hai đều đưa ra dự đoán chính xác với tỷ lệ 85% (85 đúng trong số 100 email).Tuy nhiên, thoát nhìn, có vẻ như mô hình thứ nhất tốt hơn mô hình thứ hai, vì mô hình đầu tiên chỉ xóa 5 email ham và mô hình thứ hai xóa 10 email trong số đó.Bây giờ hãy tính độ chính xác như sau:

Mô hình thư rác 1:

- Kết quả tích cực đúng (đã xóa email rác) = 30 Kết
- quả sai (đã xóa email ham) = 5 Độ chính xác =
- $30/35 = 85,7\%$

Mô hình thư rác 2:

- Kết quả tích cực thực sự (đã xóa email rác) = 35
- Kết quả tích cực sai (đã xóa email ham) = 10 Độ
- chính xác = $35/45 = 77,7\%$

Đúng như chúng tôi nghĩ: độ chính xác mang lại điểm số cao hơn cho mô hình đầu tiên so với mô hình thứ hai.Chúng tôi kết luận rằng các mô hình như mô hình spam, trong đó kết quả dương tính giả đắt hơn nhiều so với kết quả âm tính giả, là *mô hình có độ chính xác cao*.Và tại sao mô hình đầu tiên lại tốt hơn mô hình thứ hai?Mô hình thứ hai đã xóa 10 email tốt (ham), nhưng mô hình đầu tiên chỉ xóa 5 trong số đó.Mô hình thứ hai có thể đã dọn sạch nhiều thư rác hơn mô hình đầu tiên, nhưng điều đó không bù đắp được cho 5 email vi phạm mà nó đã xóa.

Bây giờ, giống như cách chúng ta đánh lừa độ chính xác và thu hồi, chúng ta cũng có thể đánh lừa độ chính xác. Hãy xem xét bộ lọc thư rác sau: bộ lọc thư rác không bao giờ phát hiện bất kỳ thư rác nào. Độ chính xác của mô hình này là gì?Điều này phức tạp vì không có email spam nào bị xóa (không có email dương tính đúng) và không có email ham nào bị xóa (không có email sai).

tích cực).Chúng tôi sẽ không cố gắng chia số 0 cho số 0 vì cuốn sách này sẽ bốc cháy, nhưng theo quy ước, một mô hình không mắc lỗi dương tính giả có độ chính xác 100%.Tuy nhiên, tất nhiên, một bộ lọc thư rác không làm gì thì không phải là một bộ lọc thư rác tốt.

Điều này cho thấy rằng dù số liệu của chúng tôi tốt đến đâu thì chúng vẫn luôn có thể bị đánh lừa.Điều đó không có nghĩa là chúng không hoạt động.Độ chính xác, chính xác và thu hồi là những công cụ hữu ích trong hộp công cụ của nhà khoa học dữ liệu.Chúng ta có quyền quyết định xem lỗi nào tốt cho mô hình của mình, bằng cách quyết định lỗi nào đắt hơn các lỗi khác. Hãy luôn cẩn thận để không rơi vào cái bẫy nghĩ rằng một mô hình là tốt trước khi đánh giá nó bằng các số liệu khác nhau.

Kết hợp khả năng thu hồi và độ chính xác như một cách để tối ưu hóa cả hai: Điểm F

Trong phần này, chúng ta thảo luận về điểm F, một thước đo kết hợp cả khả năng thu hồi và độ chính xác.Trong các phần trước, chúng ta đã thấy hai ví dụ, mô hình vi-rút corona và mô hình thư rác, trong đó kết quả âm tính giả hoặc dương tính giả quan trọng hơn.Tuy nhiên, trong cuộc sống thực, cả hai đều quan trọng, ngay cả khi chúng quan trọng ở những mức độ khác nhau.Ví dụ: chúng ta có thể muốn một mô hình không chẩn đoán sai bất kỳ người bệnh nào nhưng cũng không chẩn đoán sai quá nhiều người khỏe mạnh, bởi vì việc chẩn đoán sai một người khỏe mạnh có thể bao gồm các xét nghiệm không cần thiết và gây đau đớn, hoặc thậm chí là một cuộc phẫu thuật không cần thiết, có thể ảnh hưởng tiêu cực đến sức khỏe của họ.Theo cách tương tự, chúng ta có thể muốn một mô hình không xóa bất kỳ email hay nào của chúng ta.Nhưng để là một bộ lọc thư rác tốt, nó vẫn cần bắt được nhiều thư rác; nếu không thì nó vô dụng.Điểm F có tham số β đi kèm nên thuật ngữ phổ biến hơn là điểm F_β -score.Khi $\beta=1$ thì được gọi là điểm F_1 .

Tính điểm F

Mục tiêu của chúng tôi là tìm ra một số liệu cung cấp cho chúng tôi một số con số giữa mức độ thu hồi và độ chính xác.Điều đầu tiên bạn nghĩ đến là mức trung bình giữa khả năng thu hồi và độ chính xác.Điều này có hiệu quả không?Đúng vậy, nhưng đó không phải là thứ chúng tôi chọn, vì một lý do cơ bản.Một mô hình tốt là một mô hình có khả năng thu hồi tốt và độ chính xác tốt. Ví dụ: nếu một mô hình có tỷ lệ thu hồi là 50% và độ chính xác là 100% thì mức trung bình là 75%.Đây là điểm tốt nhưng mẫu mã có thể không được, vì tỷ lệ thu hồi 50% là không tốt lắm.Chúng ta cần một số liệu hoạt động giống như số liệu trung bình nhưng gần với giá trị tối thiểu của cả hai.

Đại lượng giống như trung bình của hai số được gọi là *trung bình điều hòa*. Trong khi trung bình cộng của hai số a và b là $(a + b)/2$ thì trung bình điều hòa của chúng là $2ab/(a + b)$. Giá trị trung bình điều hòa có đặc tính này: nó luôn nhỏ hơn hoặc bằng giá trị trung bình. Nếu các số a và b bằng nhau, người ta có thể nhanh chóng kiểm tra xem trung bình điều hòa của chúng có bằng cả hai số đó không, giống như trung bình cộng. Nhưng trong các trường hợp khác, giá trị trung bình điều hòa nhỏ hơn. Hãy xem một ví dụ: Nếu $a = 1$ và $b = 9$,

trung bình của họ là 5. Giá trị trung bình hài hòa là $\frac{2 \cdot 1 \cdot 9}{1 + 9} = 1.8$

Điểm F_1 được định nghĩa là giá trị trung bình hài hòa giữa độ chính xác và triệu hồi như sau:

$$F_1 = \frac{2PR}{P + R}$$

Nếu cả hai số đều cao thì điểm F_1 cao. Tuy nhiên, nếu một trong số chúng thấp thì điểm F_1 sẽ thấp. Mục đích của điểm F_1 là để đo lường xem cả khả năng thu hồi và độ chính xác có cao hay không và để cảnh báo khi một trong hai điểm này thấp.

Tính F_B -ĐIỂM

Trong tiểu mục trước, chúng ta đã tìm hiểu về điểm F_1 , một điểm số kết hợp khả năng thu hồi và độ chính xác nhằm mục đích đánh giá một mô hình. Tuy nhiên, đôi khi chúng ta muốn thu hồi nhiều hơn độ chính xác hoặc ngược lại. Vì vậy, khi kết hợp hai điểm số, chúng ta có thể muốn cho một trong hai điểm đó có trọng số hơn. Điều này có nghĩa là đôi khi chúng ta có thể muốn một mô hình quan tâm đến cả kết quả dương tính giả và âm tính giả nhưng lại gán nhiều trọng số hơn cho một trong số chúng. Ví dụ: mô hình coronavirus quan tâm nhiều hơn đến âm tính giả, bởi vì mạng sống của mọi người có thể phụ thuộc vào việc xác định chính xác loại vi rút, nhưng nó vẫn không muốn tạo ra quá nhiều kết quả dương tính giả, bởi vì chúng ta có thể không muốn tốn quá nhiều nguồn lực để kiểm tra lại những người khỏe mạnh. Mô hình thư rác quan tâm nhiều hơn đến các thông báo sai, bởi vì chúng ta thực sự không muốn xóa những email tốt nhưng vẫn không muốn tạo ra quá nhiều thông báo sai, vì chúng ta không muốn hộp thư đến của mình chứa đầy tin nhắn rác.

Đây là lúc điểm F_β phát huy tác dụng. Công thức tính điểm F_β ban đầu có thể trông phức tạp nhưng khi chúng ta xem xét kỹ thì nó thực hiện chính xác những gì chúng ta mong muốn. Điểm F_β sử dụng một tham số có tên là β (chữ cái Hy Lạp beta), có thể lấy bất kỳ

giá trị dương. Mục đích của β là hoạt động như một mặt số mà chúng ta xoay để nhấn mạnh độ chính xác hoặc khả năng thu hồi. Cụ thể hơn, nếu chúng ta trượt nút xoay β về 0, chúng ta sẽ có được độ chính xác hoàn toàn; nếu chúng ta

trượt nó đến vô cùng, chúng ta sẽ có được sự thu hồi hoàn toàn. Nói chung, giá trị β càng thấp thì chúng ta càng nhấn mạnh đến độ chính xác và giá trị β càng cao thì chúng ta càng nhấn mạnh vào việc thu hồi.

Đây là nơi F_β -điểm được xác định như sau (trong đó độ chính xác là P và khả năng thu hồi là R):

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

Hãy phân tích công thức này một cách cẩn thận bằng cách xem xét một số giá trị của β .

Trường hợp 1: $\beta = 1$

Khi β bằng 1, điểm F_β trở thành như sau:

$$F_1 = \frac{(1 + 1^2)PR}{1^2 P + R}$$

Điểm này giống với điểm F_1 coi khả năng thu hồi và độ chính xác là như nhau.

Trường hợp 2: $\beta = 10$

Khi β bằng 10, điểm F_β trở thành như sau:

$$F_{10} = \frac{(1 + 10^2)PR}{10^2 P + R}$$

Điều này có thể được viết là

$$\frac{101PR}{100P + R}.$$

Điều này tương tự với điểm F_1 , ngoại trừ việc lưu ý rằng nó mang lại tầm quan trọng nhiều hơn cho R hơn là P . Để thấy điều này, hãy lưu ý rằng giới hạn β có xu hướng ∞ của F_β -điểm là R .

Do đó, khi chúng tôi muốn có điểm giữa mức thu hồi và độ chính xác để mang lại nhiều trọng số hơn cho việc thu hồi, chúng tôi chọn giá trị β lớn hơn 1. Giá trị càng lớn thì chúng tôi càng chú trọng đến việc thu hồi và ít chú trọng đến độ chính xác hơn.

Trường hợp 3: $\beta = 0,1$

Khi β bằng 0,1, điểm F_β trở thành như sau:

$$F_{0.1} = \frac{(1 + 0.1^2)PR}{0.1^2P + R}$$

Cũng giống như trước đây, chúng ta có thể viết điều này như

$$\frac{1.01PR}{0.01P + R}.$$

Điều này tương tự như công thức trong trường hợp 2, ngoại trừ công thức này mang lại cho P tầm quan trọng hơn rất nhiều. Do đó, khi chúng tôi muốn có điểm giữa mức thu hồi và độ chính xác để chú trọng nhiều hơn đến độ chính xác, chúng tôi chọn giá trị β nhỏ hơn 1. Giá trị càng nhỏ thì chúng tôi càng chú trọng đến độ chính xác và ít thu hồi hơn. Trong các giới hạn, chúng tôi nói rằng giá trị $\beta = 0$ mang lại cho chúng tôi độ chính xác và giá trị $\beta = \infty$ mang lại cho chúng tôi khả năng thu hồi.

Thu hồi, độ chính xác hoặc điểm F: Chúng ta nên sử dụng điểm nào?

Bây giờ, làm thế nào để chúng ta áp dụng khả năng thu hồi và độ chính xác vào thực tế? Khi chúng ta có một mô hình, đó là mô hình có độ thu hồi cao hay độ chính xác cao? Chúng ta có sử dụng điểm F không? Nếu vậy, chúng ta nên chọn giá trị nào của β ? Câu trả lời cho những câu hỏi này tùy thuộc vào chúng ta, những nhà khoa học dữ liệu. Điều quan trọng là chúng ta phải biết rõ vấn đề mà chúng ta đang cố gắng giải quyết để quyết định lỗi nào, giữa dương tính giả và âm tính giả, đắt hơn.

Trong hai ví dụ trước, chúng ta có thể thấy rằng do mô hình vi-rút Corona cần tập trung nhiều vào việc thu hồi hơn là độ chính xác, nên chúng ta nên chọn một giá trị lớn β , chẳng hạn như 2. Ngược lại, mô hình thư rác cần tập trung nhiều hơn vào độ chính xác hơn là thu hồi, vì vậy chúng ta nên chọn một giá trị nhỏ β , chẳng hạn như 0,5. Để thực hành thêm việc phân tích các mô hình và ước tính những giá trị nào của β sẽ sử dụng, hãy xem bài tập 7.4 ở cuối chương.

Một công cụ hữu ích để đánh giá mô hình của chúng tôi: Đường cong đặc tính vận hành máy thu (ROC)

Trong phần “Làm thế nào để khắc phục vấn đề về độ chính xác?”, chúng ta đã học cách đánh giá một

mô hình bằng cách sử dụng các số liệu như độ chính xác, độ thu hồi và điểm F. Chúng tôi cũng biết rằng một trong những thách thức chính của việc đánh giá một mô hình nằm ở chỗ tồn tại nhiều loại lỗi và các loại lỗi khác nhau có mức độ quan trọng khác nhau. Chúng tôi đã học được hai loại lỗi: dương tính giả và âm tính giả. Trong một số mô hình, kết quả âm tính giả đắt hơn nhiều so với kết quả dương tính giả và ở một số mô hình thì ngược lại.

Trong phần này, tôi dạy bạn một kỹ thuật hữu ích để đánh giá một mô hình dựa trên hiệu suất của nó đối với các kết quả dương tính giả và âm tính giả cùng một lúc. Hơn nữa, phương pháp này có một tính năng quan trọng: một vòng quay cho phép chúng ta chuyển đổi dần dần giữa mô hình hoạt động tốt với kết quả dương tính giả và mô hình hoạt động tốt với kết quả âm tính giả. Kỹ thuật này dựa trên một đường cong được gọi là *đường cong đặc tính vận hành máy thu (ROC)*.

Trước khi tìm hiểu đường cong ROC, chúng ta cần giới thiệu hai số liệu mới gọi là độ đặc hiệu và độ nhạy. Trên thực tế, chỉ có một trong số đó là mới. Cái còn lại, chúng ta đã thấy trước đây.

Độ nhạy và độ đặc hiệu: Hai cách mới để đánh giá mô hình của chúng tôi

Trong phần “Làm cách nào để khắc phục vấn đề về độ chính xác?”, chúng tôi đã xác định mức thu hồi và độ chính xác là số liệu của mình và nhận thấy rằng chúng là những công cụ hữu ích để đo lường mô hình của chúng tôi về cả kết quả âm tính giả và kết quả dương tính giả. Tuy nhiên, trong phần này, chúng tôi sử dụng hai số liệu khác nhau nhưng rất giống nhau: *độ nhạy* và *độ đặc hiệu*. Những cái này có cách sử dụng tương tự như những cái trước, nhưng chúng hữu ích hơn cho chúng ta khi phải xây dựng đường cong ROC. Hơn nữa, mặc dù độ chính xác và thu hồi được các nhà khoa học dữ liệu sử dụng rộng rãi hơn, nhưng độ nhạy và độ đặc hiệu lại phổ biến hơn trong lĩnh vực y tế. Độ nhạy và độ đặc hiệu được xác định như sau:

Độ nhạy (tỷ lệ dương tính thực): khả năng của xét nghiệm trong việc xác định các điểm được dán nhãn dương tính. Đây là tỷ lệ giữa số lượng dương tính thực sự và tổng số lượng dương tính. (Lưu ý: điều này giống như thu hồi).

$$\text{Sensitivity} = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}}$$

Độ đặc hiệu (tỷ lệ âm tính thực): khả năng của xét nghiệm trong việc xác định các điểm được dán nhãn âm tính. Đây là tỷ lệ giữa số lượng âm tính thực sự

và tổng số âm bản.

$$\text{Specificity} = \frac{\text{True negatives}}{\text{True negatives} + \text{False positives}}$$

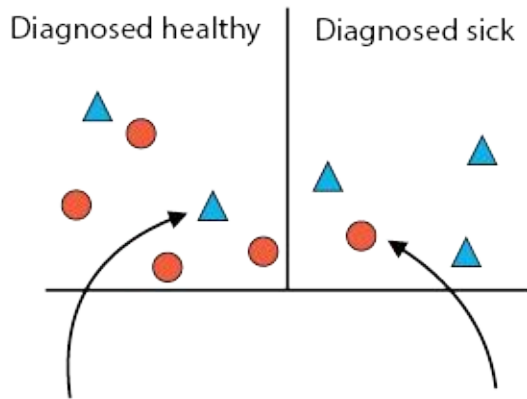
Như tôi đã đề cập, độ nhạy cũng giống như khả năng thu hồi. Tuy nhiên, tính đặc hiệu không giống như độ chính xác (mỗi danh pháp phổ biến trong các ngành khác nhau và vì lý do đó, chúng tôi sử dụng cả hai ở đây). Chúng ta thấy điều này chi tiết hơn trong phần “Thu hồi là độ nhạy, nhưng độ chính xác và độ đặc hiệu thì khác nhau”.

Trong mô hình coronavirus, độ nhạy là tỷ lệ người bệnh mà mô hình đã chẩn đoán chính xác, trong số tất cả những người bị bệnh. Độ đặc hiệu là tỷ lệ người khỏe mạnh mà mô hình đã chẩn đoán đúng trong số những người khỏe mạnh. Chúng tôi quan tâm hơn đến việc chẩn đoán chính xác người bệnh nên chúng tôi cần mô hình vi-rút Corona có *độ nhạy cao*.

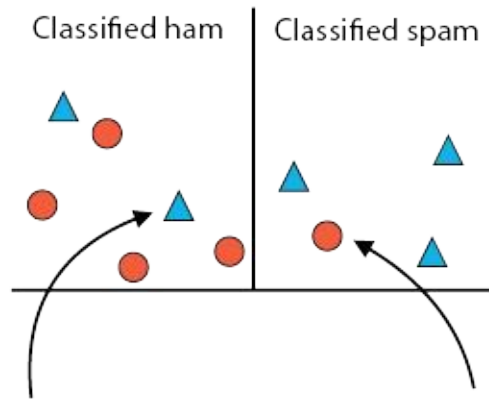
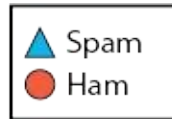
Trong mô hình phát hiện thư rác, độ nhạy là tỷ lệ thư rác mà chúng tôi đã xóa chính xác, trong số tất cả các thư rác. Độ đặc hiệu nằm ở tỷ lệ số email ham chúng tôi gửi chính xác vào hộp thư đến, trong số tất cả các email ham. Vì chúng tôi quan tâm nhiều hơn đến việc phát hiện chính xác các email ham nên chúng tôi cần mô hình phát hiện thư rác có *độ đặc hiệu cao*.

Để làm rõ các khái niệm trước đó, chúng ta hãy xem chúng trong ví dụ đồ họa mà chúng ta đang thực hiện. Cụ thể, hãy tính độ đặc hiệu và độ nhạy cho hai mô hình của chúng ta trong hình 7.2 (giống như hình 7.1).

Coronavirus model



Spam model



Hình 7.2 Ở bên trái, mô hình vi-rút Corona trong đó mọi người được chẩn đoán là khỏe mạnh hoặc bị bệnh; ở bên phải, mô hình phát hiện thư rác trong đó các email được phân loại là thư rác hoặc có nội dung xấu

Như we đã thấy trước đó, hai mẫu này sản xuất số lượng như sau :

- Ba kết quả dương tính
- đúng Bốn kết quả âm tính
- đúng Một kết quả dương
- tính giả Hai kết quả âm
- tính giả

Bây giờ hãy tính độ đặc hiệu và độ nhạy của các mô hình này.

Tính ĐẶC BIỆT

Trong trường hợp này, chúng ta tính toán độ nhạy như sau: trong số các điểm tích cực, mô hình đã phân loại đúng bao nhiêu điểm? Điều này tương đương với việc hỏi: trong số các hình tam giác, có bao nhiêu hình nằm ở bên phải đường thẳng? Có năm hình tam giác và mô hình đã phân loại ba hình tam giác đó một cách chính xác ở bên phải của đường thẳng, do đó độ nhạy là $3/5$, bằng $0,6$ hoặc 60% .

Tính độ nhạy

Ta tính độ đặc hiệu như sau: trong số các điểm tiêu cực, mô hình đã phân loại đúng bao nhiêu điểm? Điều này tương đương với việc hỏi: trong số các vòng tròn có bao nhiêu

nằm ở bên trái của dòng? Có năm vòng tròn và mô hình đã phân loại chính xác bốn vòng tròn trong số đó ở bên trái của đường thẳng, do đó độ đặc hiệu là $4/5$, bằng 0,8 hoặc 80%.

Đường cong đặc tính vận hành máy thu (ROC): Một cách tối ưu hóa độ nhạy và độ đặc hiệu trong mô hình

Trong phần này, chúng ta xem cách vẽ đường cong đặc tính vận hành máy thu (ROC), đường cong này sẽ cung cấp cho chúng ta nhiều thông tin về mô hình. Tóm lại, những gì chúng ta sẽ làm là sửa đổi mô hình một cách từ từ và ghi lại độ nhạy cũng như độ đặc hiệu của mô hình ở mỗi bước thời gian.

Giả định đầu tiên và duy nhất mà chúng ta cần đưa ra về mô hình của mình là nó trả về dự đoán dưới dạng một giá trị liên tục, cụ thể là dưới dạng xác suất. Điều này đúng với các mô hình như bộ phân loại logistic, trong đó dự đoán không phải là một lớp, chẳng hạn như dương/âm, mà là giá trị từ 0 đến 1, chẳng hạn như 0,7. Những gì chúng ta thường làm với giá trị này là chọn một ngưỡng, chẳng hạn như 0,5 và phân loại mọi điểm nhận được dự đoán cao hơn hoặc bằng ngưỡng là dương và mọi điểm khác là âm. Tuy nhiên, ngưỡng này có thể là bất kỳ giá trị nào—không nhất thiết phải là 0,5. Quy trình của chúng tôi bao gồm việc thay đổi ngưỡng này từ 0 đến 1 và ghi lại độ nhạy và độ đặc hiệu của mô hình ở mỗi giá trị ngưỡng.

Hãy xem một ví dụ. Chúng tôi tính toán độ nhạy và độ đặc hiệu cho ba ngưỡng khác nhau: 0,2, 0,5 và 0,8. Trong hình 7.3, chúng ta có thể thấy có bao nhiêu điểm ở bên trái và bên phải của đường cho mỗi ngưỡng này. Hãy nghiên cứu chúng một cách chi tiết. Hãy nhớ rằng độ nhạy là tỷ lệ giữa kết quả dương tính thực sự trên tất cả các kết quả dương tính và độ đặc hiệu là tỷ lệ giữa kết quả âm tính thực sự trên tất cả kết quả âm tính. Cũng nên nhớ rằng đối với mỗi một trong số này, có tổng cộng năm điểm tích cực và năm điểm tiêu cực.

Ngưỡng = 0,2

Số lượng dương tính thực sự: 4

Độ nhạy: $4/5$

Số âm tính thật: 3

Tính cụ thể: $\frac{3}{5}$

Ngưỡng = 0,5

Số lượng dương tính thực sự: 3

Độ nhạy: $\frac{3}{5}$

Số âm tính thật: 4

Tính cụ thể: $\frac{4}{5}$

Ngưỡng = 0,2

Số lượng dương tính thực sự: 2

Độ nhạy: $\frac{2}{5}$

Số âm tính thật: 5

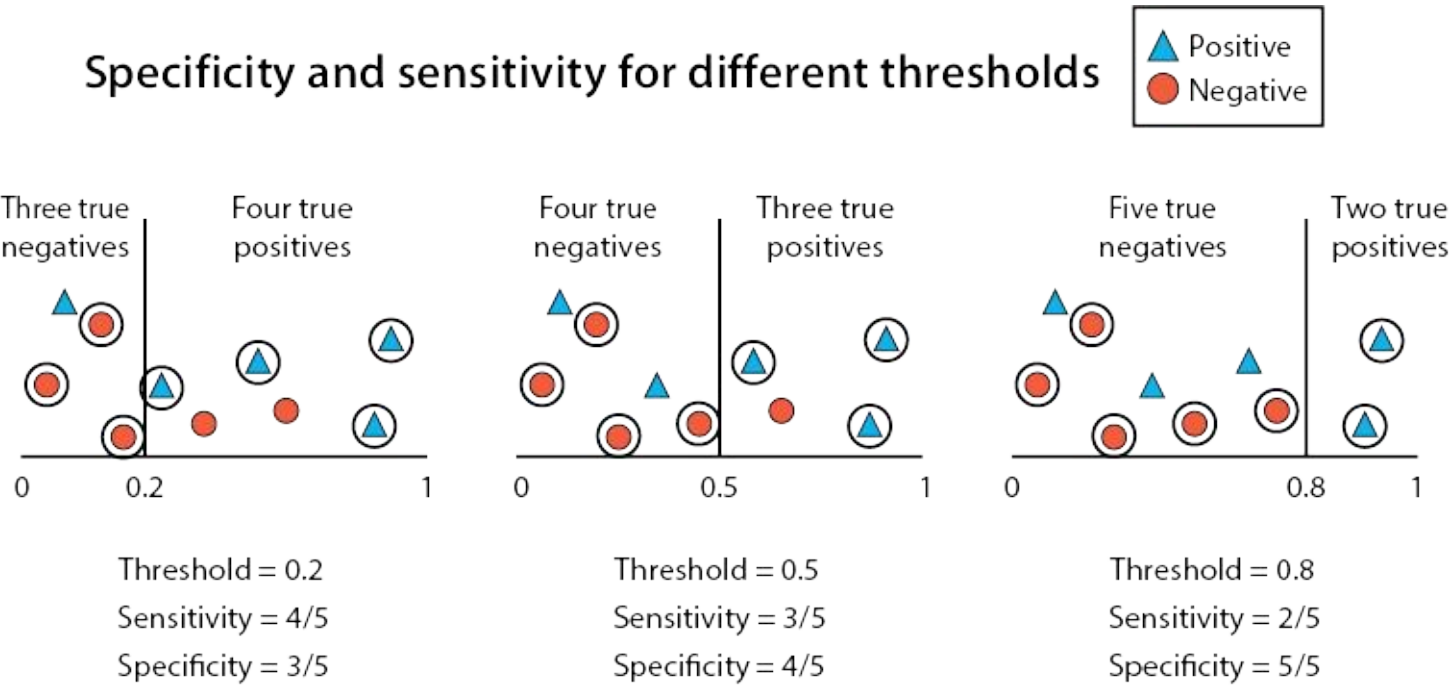
Độ đặc hiệu: $\frac{5}{5} = 1$

Lưu ý rằng ngưỡng thấp dẫn đến nhiều dự đoán tích cực. Do đó, chúng ta sẽ có ít kết quả âm tính giả, ngụ ý điểm độ nhạy cao và nhiều kết quả dương tính giả, ngụ ý điểm độ đặc hiệu thấp. Tương tự, ngưỡng cao hàm ý điểm nhạy thấp và điểm đặc hiệu cao. Khi chúng ta chuyển ngưỡng từ thấp lên cao, độ nhạy giảm và độ đặc hiệu tăng. Đây là điểm quan trọng mà chúng ta sẽ đề cập ở phần sau của chương này, khi chúng ta quyết định ngưỡng tốt nhất cho mô hình của mình.

Bây giờ chúng ta đã sẵn sàng xây dựng đường cong ROC. Đầu tiên, chúng tôi xem xét ngưỡng bằng 0 và tăng dần giá trị của ngưỡng này theo từng khoảng thời gian nhỏ cho đến khi đạt đến 1. Đối với mỗi lần tăng ngưỡng, chúng tôi sẽ vượt qua chính xác một điểm. Giá trị của các ngưỡng không quan trọng—điều quan trọng là ở mỗi bước, chúng ta vượt qua chính xác một điểm (điều này có thể thực hiện được vì tất cả các điểm đều cho chúng ta những điểm số khác nhau, nhưng nói chung đó không phải là một yêu cầu). Vì vậy, chúng ta sẽ gọi các bước là 0, 1, 2,..., 10. Trong đầu, bạn hãy tưởng tượng đường thẳng đứng trong hình 7.3 bắt đầu từ 0 và di chuyển chậm từ trái sang phải, quét từng điểm một cho đến khi đạt 1. Các bước này được ghi lại trong bảng 7.6, cùng với số lượng điểm dương và âm thực, độ nhạy và độ đặc hiệu ở mỗi bước.

Một điều cần lưu ý là ở bước đầu tiên (bước 0), dòng này ở ngưỡng 0. Điều này

có nghĩa là mọi điểm được mô hình phân loại là tích cực. Tất cả các điểm tích cực cũng được phân loại là tích cực, vì vậy mọi điểm tích cực đều là điểm tích cực thực sự. Điều này có nghĩa là tại dấu thời gian 0, độ nhạy là $5/5 = 1$. Nhưng vì mọi điểm âm đều được phân loại là dương nên không có điểm dương thực sự nào nên độ đặc hiệu là $0/5 = 0$. Tương tự, ở bước cuối cùng (bước 10), ngưỡng là 1 và chúng ta có thể kiểm tra điều đó vì mọi điểm đều được phân loại là âm nên độ nhạy bây giờ là 0 và độ đặc hiệu là 1. Để rõ ràng, ba mô hình trong hình 7.3 được đánh dấu trong bảng 7.6 lần lượt là các dấu thời gian 4, 6 và 8.



Hình 7.3 Ảnh hưởng của việc di chuyển ngưỡng đến độ nhạy và độ đặc hiệu. Ở bên trái, chúng ta có một mô hình có ngưỡng thấp; ở giữa, chúng ta có một cái có ngưỡng trung bình; và ở bên phải, chúng ta có một cái có ngưỡng cao. Đối với mỗi mô hình, có năm điểm tích cực và năm điểm tiêu cực. Mỗi mô hình được thể hiện bằng đường thẳng đứng. Mô hình dự đoán rằng các điểm ở bên phải đường thẳng là dương và các điểm ở bên trái là âm. Đối với mỗi mô hình, chúng tôi đã đếm số lượng điểm tích cực và tiêu cực thực sự, nghĩa là số điểm tích cực và tiêu cực đã được dự đoán chính xác. Chúng tôi đã sử dụng chúng để tính toán độ nhạy và độ đặc hiệu. Lưu ý rằng khi chúng ta tăng ngưỡng (tức là khi chúng ta di chuyển đường thẳng đứng từ trái sang phải), độ nhạy sẽ giảm và độ đặc hiệu sẽ tăng.

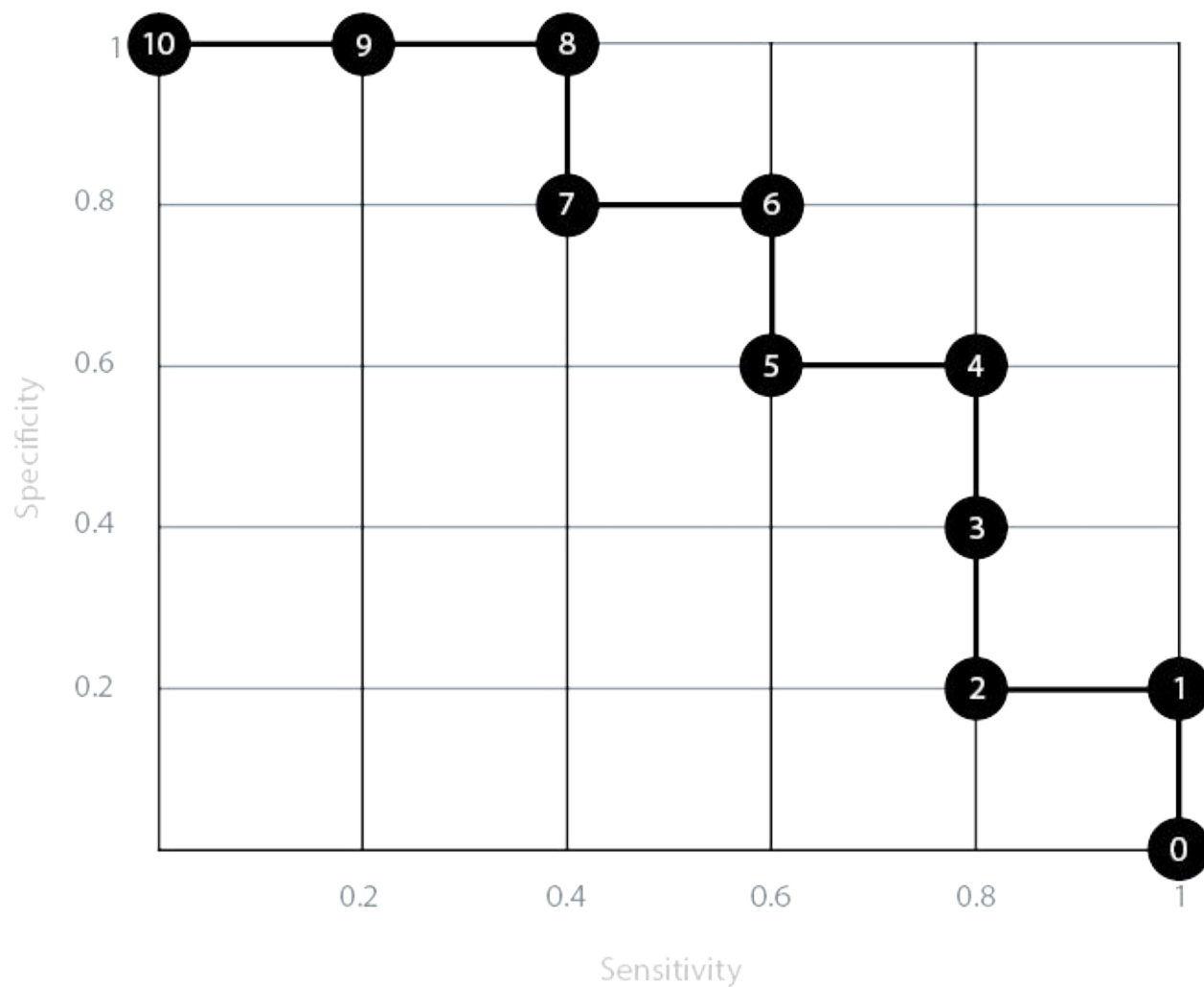
Bảng 7.6 Tất cả các bước thời gian trong quá trình tăng ngưỡng của chúng tôi, đây là một bước quan trọng trong việc xây dựng đường cong ROC của chúng tôi. Ở mỗi dấu thời gian, chúng tôi ghi lại số lượng kết quả dương tính thực sự và âm tính thực sự. Sau đó chúng tôi tính toán độ đặc hiệu của mô hình bằng cách chia số giá trị đúng

positives by the total number of positives. As a final step, we calculate the specificity by dividing the number of true negatives by the total number of negatives.

Step	True positives	Sensitivity	True negatives	Specificity
0	5	1	0	0
1	5	1	1	0.2
2	4	0.8	1	0.2
3	4	0.8	2	0.4
4	4	0.8	3	0.6
5	3	0.6	3	0.6
6	3	0.6	4	0.8
7	2	0.4	4	0.8
8	2	0.4	5	1
9	1	0.2	5	1
10	0	0	5	1

As a last step, we plot the sensitivity and specificity values. This is the ROC curve, which we see in figure 7.4. In this figure, each of the black points corresponds to a timestep (indicated inside the point), the horizontal coordinate corresponds to the sensitivity, and the vertical coordinate to the specificity.

ROC curve



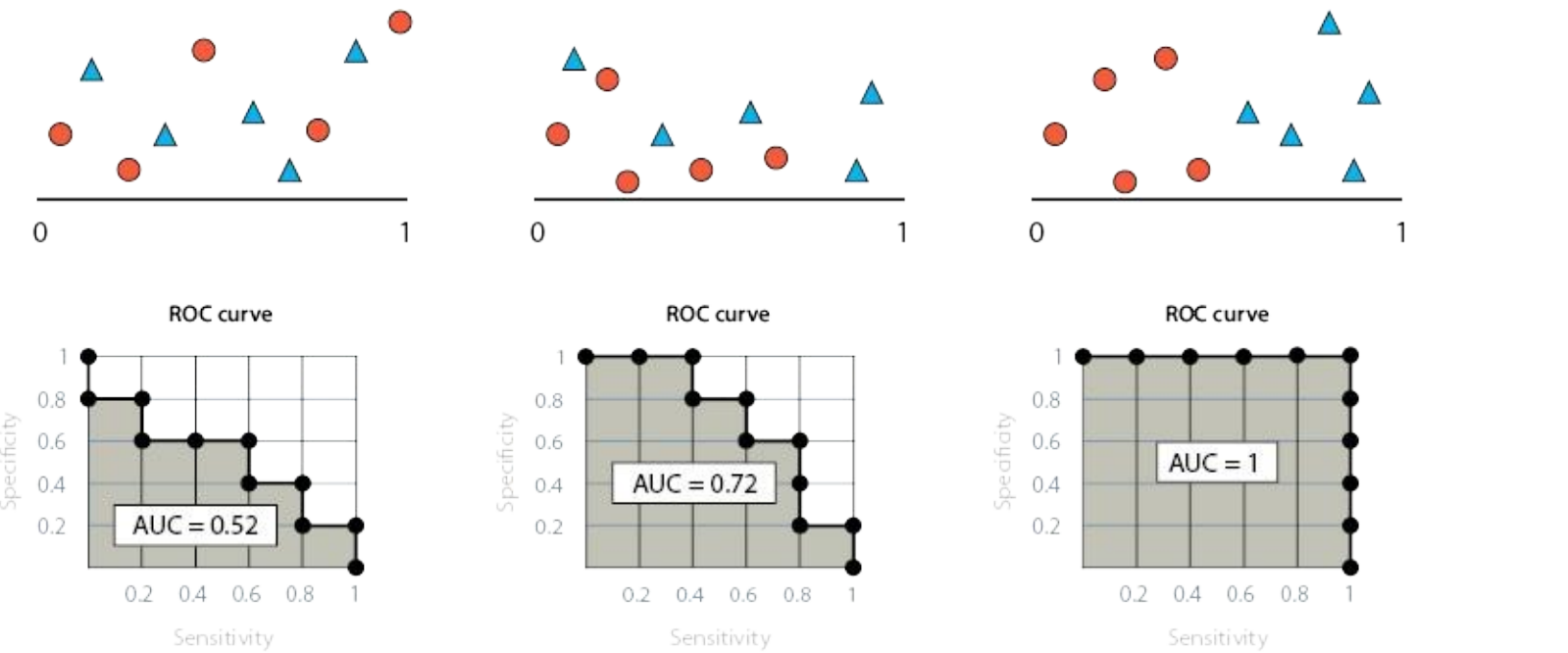
Hình 7.4 Ở đây chúng ta có thể thấy đường cong ROC tương ứng với ví dụ đang diễn ra, nó cung cấp cho chúng ta rất nhiều thông tin về mô hình của chúng ta. Các dấu chấm được đánh dấu tương ứng với dấu thời gian thu được bằng cách di chuyển ngưỡng của chúng tôi từ 0 lên 1 và mỗi dấu chấm được gắn nhãn theo dấu thời gian. Trên trục hoành, chúng tôi ghi lại độ nhạy của mô hình tại mỗi dấu thời gian và trên trục tung, chúng tôi ghi lại độ đặc hiệu.

Một thước đo cho chúng tôi biết mô hình của chúng tôi tốt đến mức nào: AUC (diện tích dưới đường cong)

Như chúng ta đã thấy trước đây trong cuốn sách này, việc đánh giá một mô hình học máy là một nhiệm vụ cực kỳ quan trọng và trong phần này, chúng ta thảo luận cách sử dụng đường cong ROC để đánh giá một mô hình. Đối với điều này, chúng tôi đã hoàn thành tất cả công việc—tất cả những gì còn lại là tính diện tích dưới đường cong hoặc AUC. Ở đầu hình 7.5, chúng ta có thể thấy ba mô hình, trong đó dự đoán được đưa ra bởi trục hoành (từ 0 đến 1). Ở phía dưới, bạn có thể thấy ba đường cong ROC tương ứng. Mỗi hình vuông có kích thước 0,2 nhân 0,2. Số ô vuông dưới mỗi đường cong là 13,

18 và 25, tương ứng với các diện tích dưới đường cong 0,52, 0,72 và 1.

Lưu ý rằng giá trị tốt nhất mà một mô hình có thể làm là AUC bằng 1, tương ứng với mô hình bên phải. Điều tồi tệ nhất mà một mô hình có thể làm là AUC bằng 0,5, vì điều này có nghĩa là mô hình đó tốt như việc đoán ngẫu nhiên. Điều này tương ứng với mô hình bên trái. Mô hình ở giữa là mô hình ban đầu của chúng tôi, với AUC là 0,72.



Hình 7.5 Trong hình này, chúng ta có thể thấy rằng AUC, hay diện tích dưới đường cong, là một thước đo tốt để xác định mức độ tốt của một mô hình. AUC càng cao thì mô hình càng tốt. Ở bên trái, chúng ta có một mô hình xấu với AUC là 0,52. Ở giữa, chúng ta có một mô hình tốt với AUC là 0,72. Ở bên phải, chúng ta có một mô hình tuyệt vời với AUC là 1.

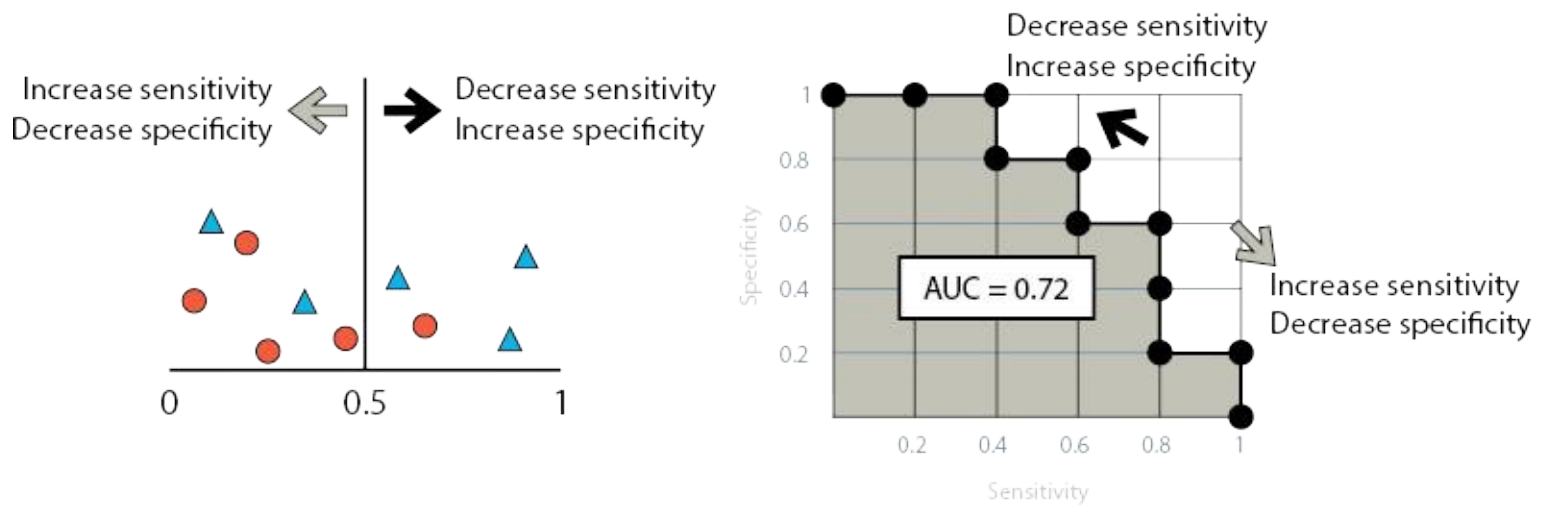
Thế còn một mô hình có AUC bằng 0 thì sao? Chà, điều này thật khó khăn. Một mô hình có AUC bằng 0 sẽ tương ứng với một mô hình phân loại sai mọi điểm. Đây có phải là một mô hình xấu? Đó thực sự là một mô hình rất tốt, bởi vì tất cả những gì chúng ta phải làm để khắc phục nó là lật ngược tất cả các dự đoán tích cực và tiêu cực để có được một mô hình hoàn hảo. Nó có tác dụng tương tự như việc một người nói dối mỗi khi họ nhận được câu hỏi đúng hay sai. Tất cả những gì chúng ta phải làm để khiến họ nói sự thật là lật ngược tất cả các câu trả lời của họ. Điều này có nghĩa là điều tồi tệ nhất mà chúng ta có thể có trong mô hình phân loại nhị phân là AUC là 0,5, vì con số này tương ứng với một người nói dối 50% thời gian. Họ không cung cấp cho chúng tôi thông tin vì chúng tôi không bao giờ biết họ nói thật hay nói dối! Ngẫu nhiên, nếu chúng ta có một mô hình có AUC nhỏ hơn 0,5, chúng ta có thể lật ngược các dự đoán tích cực và tiêu cực và thu được một mô hình có AUC lớn hơn 0,5.

Cách đưa ra quyết định bằng đường cong ROC

ROC là một đồ họa mạnh mẽ cung cấp cho chúng ta nhiều thông tin về mô hình của chúng ta. Trong phần này, chúng ta tìm hiểu cách sử dụng nó để cải thiện mô hình của mình. Nói tóm lại, chúng tôi sử dụng ROC để điều chỉnh ngưỡng trong một mô hình và áp dụng nó để chọn mô hình tốt nhất cho trường hợp sử dụng của chúng tôi.

Ở đầu chương này, chúng tôi đã giới thiệu hai mô hình, mô hình virus Corona và mô hình phát hiện thư rác. Những mô hình này rất khác nhau vì như chúng ta đã thấy, mô hình virus Corona yêu cầu độ nhạy cao, trong khi mô hình phát hiện thư rác yêu cầu độ đặc hiệu cao. Mỗi mô hình đều yêu cầu độ nhạy và độ đặc hiệu nhất định dựa trên vấn đề chúng ta phải giải quyết. Giả sử chúng ta đang ở trong tình huống sau: chúng ta đang đào tạo một mô hình được cho là có độ nhạy cao và chúng ta nhận được một mô hình có độ nhạy thấp và độ đặc hiệu cao. Có cách nào để chúng ta có thể đánh đổi một số tính đặc hiệu và đạt được độ nhạy nào đó không?

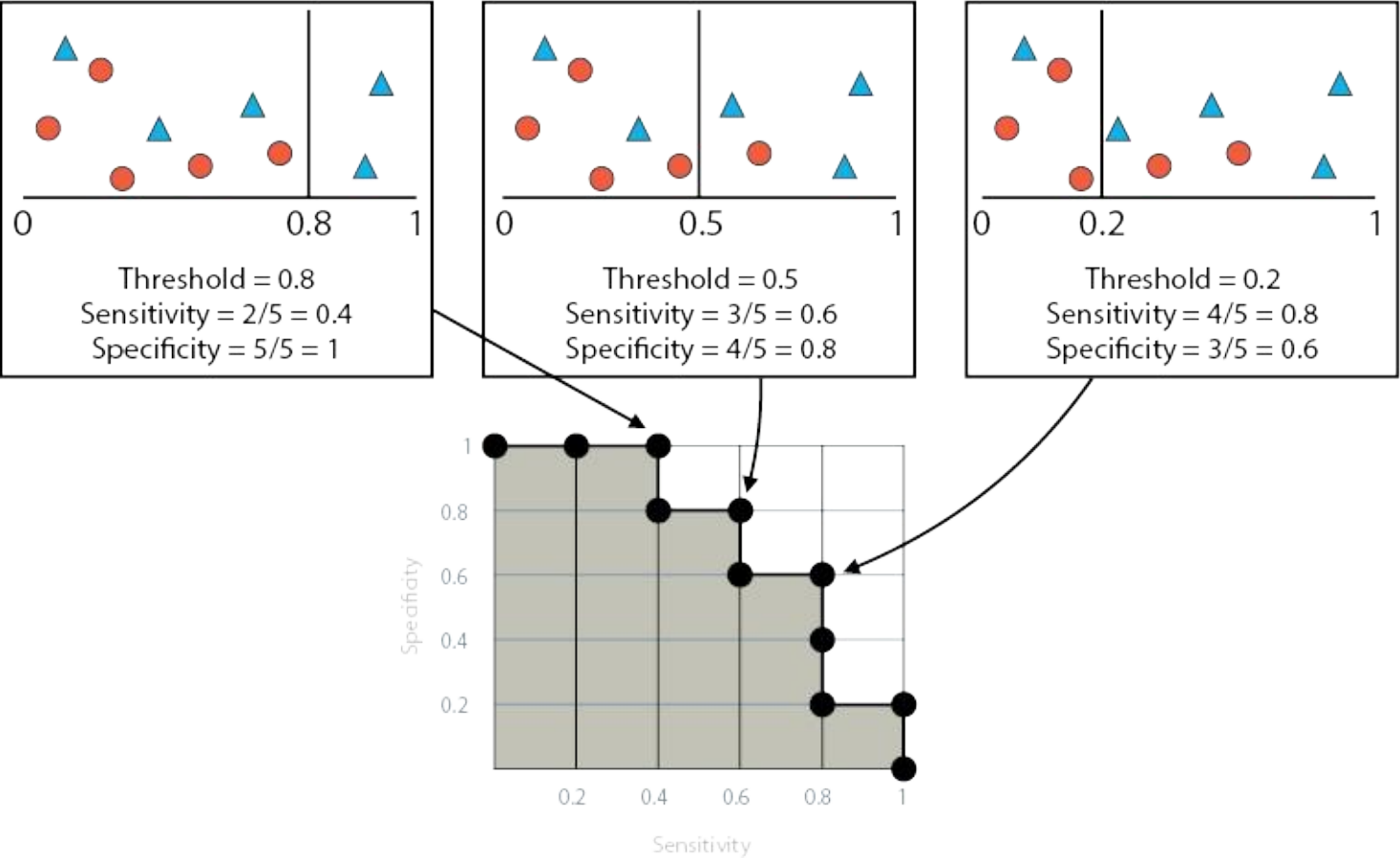
Câu trả lời là có! Chúng ta có thể cân bằng độ đặc hiệu và độ nhạy bằng cách di chuyển ngưỡng. Hãy nhớ lại rằng khi lần đầu tiên xác định đường cong ROC, chúng tôi nhận thấy rằng ngưỡng càng thấp thì độ nhạy và độ đặc hiệu càng cao trong mô hình và ngưỡng càng cao thì độ nhạy và độ đặc hiệu càng cao trong mô hình. Khi đường thẳng đứng tương ứng với ngưỡng nằm ở bên trái, mọi điểm được dự đoán là dương, do đó tất cả các điểm dương đều là dương thực, trong khi khi đường thẳng đứng ở ngay bên phải, mọi điểm được dự đoán là âm, vì vậy tất cả các điểm âm đều là âm thực. Khi chúng ta di chuyển đường này sang bên phải, chúng ta sẽ mất đi một số kết quả dương tính thực sự và thu được một số kết quả âm tính thực sự, do đó độ nhạy giảm và độ đặc hiệu tăng lên. Lưu ý rằng khi ngưỡng di chuyển từ 0 đến 1, chúng ta di chuyển lên trên và sang trái trong đường cong ROC, như minh họa ở hình 7.6.



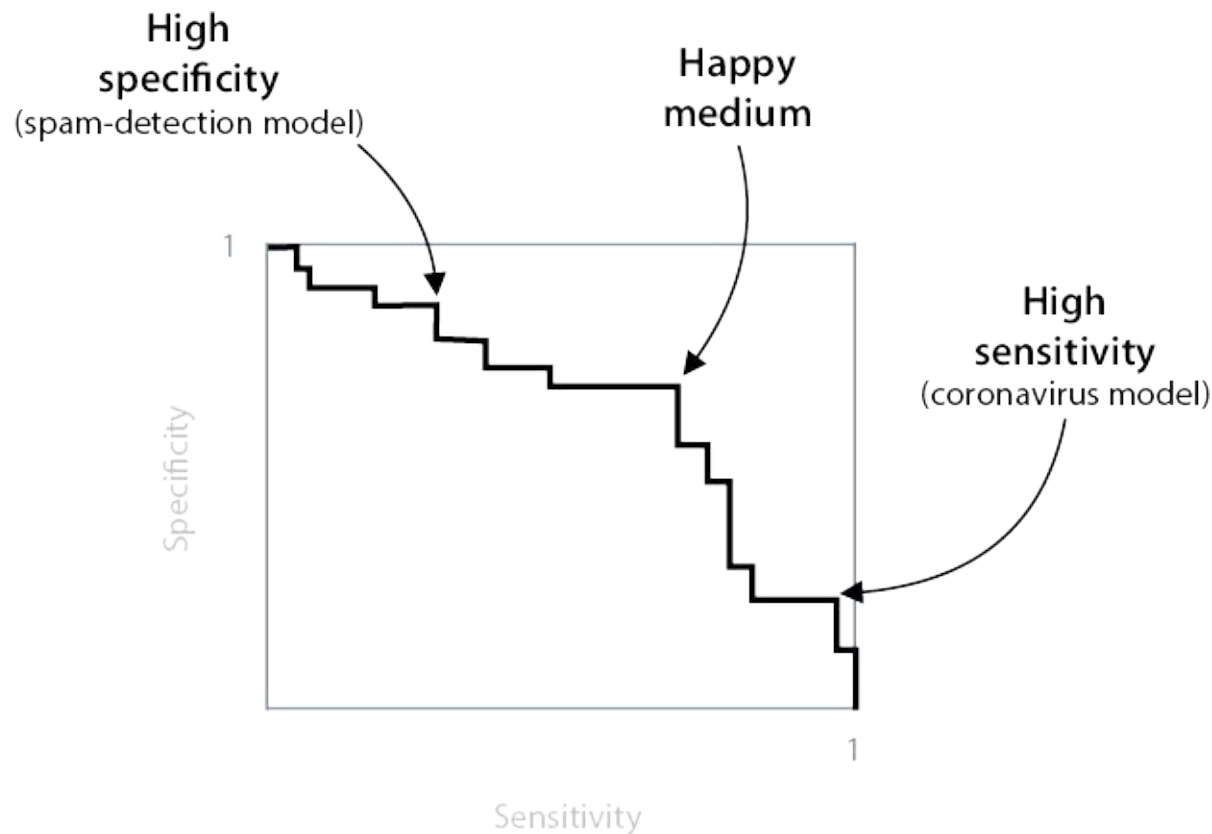
Hình 7.6 Ngưỡng của mô hình liên quan nhiều đến độ nhạy và độ đặc hiệu, và mối quan hệ này sẽ giúp chúng ta chọn ngưỡng hoàn hảo cho mô hình của mình. Ở bên trái, chúng ta có mô hình của mình và ở bên phải là đường cong ROC tương ứng. Khi chúng tôi tăng hoặc giảm ngưỡng, chúng tôi sẽ thay đổi độ nhạy và độ đặc hiệu của mô hình và sự thay đổi này được minh họa bằng cách di chuyển trên đường cong ROC.

Tại sao điều này xảy ra? Ngưỡng cho chúng ta biết nơi chúng ta vẽ đường phân loại một điểm. Ví dụ: trong mô hình vi-rút Corona, ngưỡng này cho chúng tôi biết chúng tôi sẽ vạch ra ranh giới ở đâu đối với một người được gửi đi xét nghiệm thêm hoặc được đưa về nhà. Mô hình có ngưỡng thấp là mô hình gửi mọi người đi xét nghiệm bổ sung nếu họ có nhiều triệu chứng nhẹ. Một mô hình có ngưỡng cao là mô hình cần những người có các triệu chứng rõ ràng để gửi họ đi xét nghiệm nhiều hơn. Bởi vì chúng tôi muốn bắt tất cả những người bị bệnh nên chúng tôi muốn có ngưỡng thấp cho mô hình này, nghĩa là chúng tôi muốn một mô hình có độ nhạy cao. Để rõ ràng hơn, trong hình 7.7, chúng ta có thể thấy ba ngưỡng được sử dụng trước đó, cũng như các điểm mà chúng tương ứng trên đường cong.

Nếu muốn mô hình của mình có độ nhạy cao, chúng ta chỉ cần đẩy ngưỡng sang trái (tức là giảm nó) cho đến khi đạt đến một điểm trên đường cong có độ nhạy như mong muốn. Lưu ý rằng mô hình có thể mất đi một số tính đặc hiệu và đó là cái giá chúng ta phải trả. Ngược lại, nếu chúng ta muốn độ đặc hiệu cao hơn, chúng ta đẩy ngưỡng sang bên phải (tức là tăng nó) cho đến khi chúng ta đạt đến một điểm trên đường cong có độ đặc hiệu cao như chúng ta mong muốn. Một lần nữa, chúng ta mất đi một số độ nhạy trong quá trình này. Đường cong cho chúng ta biết chính xác chúng ta được và mất bao nhiêu. Vì vậy, với tư cách là nhà khoa học dữ liệu, đây là một công cụ tuyệt vời giúp chúng ta quyết định ngưỡng tốt nhất cho mô hình của mình. Trong hình 7.8, chúng ta có thể thấy một ví dụ tổng quát hơn với tập dữ liệu lớn hơn.



Hình 7.7 Sự song song giữa ngưỡng của mô hình và ROC của nó. Mô hình bên trái có ngưỡng cao, độ nhạy thấp và độ đặc hiệu cao. Mô hình ở giữa có các giá trị trung bình về ngưỡng, độ nhạy và độ đặc hiệu. Mô hình bên phải có ngưỡng thấp, độ nhạy cao và độ đặc hiệu thấp.



Hình 7.8 Trong kịch bản tổng quát hơn này, chúng ta có thể thấy đường cong ROC và ba điểm trên đó tương ứng với ba ngưỡng khác nhau. Nếu chúng ta muốn chọn một ngưỡng mang lại cho chúng ta độ đặc hiệu cao, chúng ta sẽ chọn ngưỡng ở bên trái. Đối với mô hình có độ nhạy cao, chúng tôi chọn mô hình bên phải. Nếu chúng ta muốn một mô hình có cả độ nhạy và độ đặc hiệu tốt, chúng ta sẽ chọn mô hình ở giữa.

Nếu chúng ta cần một mô hình có độ nhạy cao, chẳng hạn như mô hình virus Corona, chúng ta sẽ chọn điểm bên phải. Nếu chúng ta cần một mô hình có độ đặc hiệu cao, chẳng hạn như mô hình phát hiện thư rác, chúng ta có thể chọn điểm bên trái. Tuy nhiên, nếu chúng ta muốn độ nhạy và độ đặc hiệu tương đối cao, chúng ta có thể chọn điểm ở giữa. Trách nhiệm của chúng tôi với tư cách là nhà khoa học dữ liệu là phải hiểu rõ vấn đề để đưa ra quyết định đúng đắn.

Thu hồi là độ nhạy, nhưng độ chính xác và độ đặc hiệu thì khác nhau

Tại thời điểm này, bạn có thể tự hỏi làm thế nào chúng ta có thể nhớ tất cả các thuật ngữ này ngay lập tức. Câu trả lời là, họ khó mà không bị nhầm lẫn. Hầu hết các nhà khoa học dữ liệu (bao gồm cả tác giả) thường cần tra cứu nhanh chóng trên Wikipedia để đảm bảo rằng chúng không gây nhầm lẫn. Chúng ta có thể sử dụng phương pháp ghi nhớ để giúp chúng ta nhớ cái nào là cái nào.

Ví dụ, khi chúng ta nghĩ đến việc thu hồi, hãy nghĩ đến một công ty ô tô đã sản xuất một chiếc ô tô có lỗi thiết kế nghiêm trọng. Họ cần tìm ra tất cả những chiếc xe bị lỗi và *thu hồi* chúng. Nếu họ vô tình nhận được nhiều xe không bị lỗi, họ chỉ cần trả lại. Tuy nhiên, không tìm được một trong những chiếc xe bị lỗi sẽ là điều khủng khiếp. Vì vậy, việc thu hồi quan tâm đến việc tìm ra tất cả các ví dụ được gắn nhãn tích cực. Điều này thể hiện một mô hình có *khả năng thu hồi* cao.

Mặt khác, nếu chúng ta làm việc cho công ty ô tô này và chúng ta đã hơi quá nhiệt tình và bắt đầu thu hồi *tất cả* xe, sếp của chúng ta có thể đến và nói: "Này, bạn gửi quá nhiều xe để sửa và chúng tôi sắp hết nguồn lực. Bạn có thể chọn lọc hơn và gửi cho tôi *chính xác* những chiếc bị lỗi không?" Sau đó, chúng ta cần tăng độ chính xác cho mô hình và cố gắng chỉ tìm những cái bị lỗi, ngay cả khi chúng ta vô tình bỏ sót một số cái bị lỗi (hy vọng là không!). Điều này thể hiện một mô hình có *độ chính xác* cao.

Khi nói đến độ đặc hiệu và độ nhạy, hãy nghĩ đến cảm biến động đất phát ra tiếng bíp mỗi khi có động đất. Cảm biến này cực kỳ *nhạy cảm*. Nếu một con bướm hắt hơi ở nhà bên cạnh, cảm biến sẽ phát ra tiếng bíp. Cảm biến này chắc chắn sẽ ghi lại tất cả các trận động đất, nhưng nó cũng sẽ ghi lại nhiều thứ khác không phải là trận động đất. Điều này thể hiện một mô hình có *độ nhạy* cao.

Bây giờ, hãy tưởng tượng rằng cảm biến này có một mặt số và chúng ta giảm độ nhạy của nó xuống hết mức. Bây giờ cảm biến sẽ chỉ phát ra tiếng bíp khi có nhiều chuyển động. Khi cảm biến đó phát ra tiếng bíp, chúng tôi *biết* đó là một trận động đất. Vấn đề là nó có thể bỏ sót một số trận động đất nhỏ hoặc trung bình. Nói cách khác, cảm biến này rất *đặc hiệu* đối với động đất nên rất có thể nó sẽ không phát ra tiếng bíp với bất kỳ thứ gì khác. Điều này thể hiện một mô hình có *tính đặc hiệu* cao.

Nếu quay lại và đọc bốn đoạn trước, chúng ta có thể nhận thấy hai điều sau:

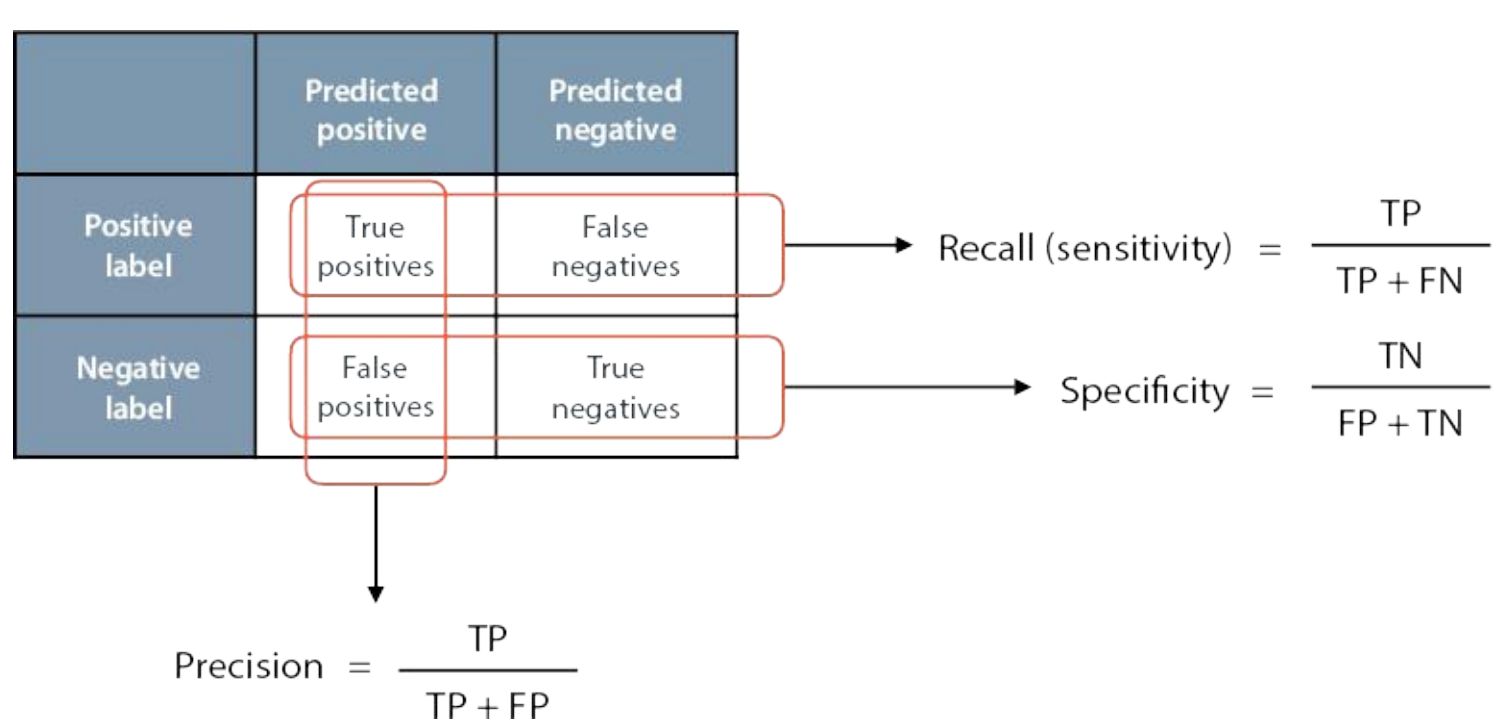
- Khả năng thu hồi và độ nhạy rất giống nhau. Độ
- chính xác và độ đặc hiệu rất giống nhau.

Ít nhất, việc thu hồi và độ nhạy có cùng một mục đích, đó là đo lường xem có bao nhiêu kết quả âm tính giả. Tương tự, độ chính xác và độ đặc hiệu cũng có cùng một mục đích, đó là đo lường xem có bao nhiêu kết quả dương tính giả.

Hóa ra khả năng thu hồi và độ nhạy *chính xác* là như nhau. Tuy nhiên, độ chính xác và độ đặc hiệu không giống nhau. Mặc dù chúng không đo lường giống nhau nhưng cả hai đều trừng phạt những mô hình có số lượng kết quả dương tính giả cao. Làm cách nào để

nhớ tất cả các số liệu này? Phương pháp phỏng đoán bằng đồ họa có thể giúp chúng ta ghi nhớ khả năng thu hồi, độ chính xác, độ nhạy và độ đặc hiệu. Trong hình 7.9, chúng ta thấy một ma trận nhầm lẫn với bốn đại lượng: dương tính đúng, âm tính thực, dương tính giả và âm tính giả. Nếu tập trung vào hàng trên cùng (các ví dụ được gắn nhãn tích cực), chúng ta có thể tính toán khả năng thu hồi bằng cách chia số ở cột bên trái cho tổng các số ở cả hai cột. Nếu tập trung vào cột ngoài cùng bên trái (các ví dụ được dự đoán là dương), chúng ta có thể tính toán độ chính xác bằng cách chia số ở hàng trên cùng cho tổng các số ở cả hai hàng. Nếu chúng ta tập trung vào hàng dưới cùng (các ví dụ được gắn nhãn tiêu cực), chúng ta có thể tính toán độ đặc hiệu bằng cách chia số ở cột bên trái cho tổng các số trên cả hai cột. Nói cách khác

- Thu hồi và độ nhạy tương ứng với hàng trên cùng. Độ chính
- xác tương ứng với cột bên trái. Độ đặc hiệu tương ứng với
- hàng dưới cùng.



Hình 7.9 Hàng trên cùng của ma trận nhầm lẫn cho chúng ta khả năng thu hồi và độ nhạy: tỷ lệ giữa số kết quả dương tính thật và tổng số kết quả dương tính thật và âm tính giả. Cột ngoài cùng bên trái cho chúng ta độ chính xác: tỷ lệ giữa số lượng kết quả dương tính thật và tổng số kết quả dương tính thật và dương tính giả. Hàng dưới cùng cho chúng ta biết độ đặc hiệu: tỷ lệ giữa số lượng dương tính giả và tổng số lượng dương tính giả và âm tính thực.

Tóm lại, các đại lượng này như sau trong cả hai mô hình của chúng tôi:

Mẫu y tế:

- **Ghi nhớ và độ nhạy:** trong số những người bị bệnh (dương tính), có bao nhiêu người được chẩn đoán chính xác là bị bệnh? **Độ chính xác:** trong số những người được chẩn đoán là bị bệnh, có bao nhiêu người thực sự bị bệnh? **Tính đặc hiệu:** trong số những người khỏe mạnh (âm tính), có bao nhiêu người được chẩn đoán chính xác là khỏe mạnh?

Mẫu email:

- **Gợi nhớ và nhạy cảm:** trong số các email spam (tích cực), có bao nhiêu email đã bị xóa chính xác?
- **Độ chính xác:** trong số các email đã xóa, có bao nhiêu email thực sự là thư rác?
- **Tính cụ thể:** trong số các email ham (phủ định), có bao nhiêu email được gửi đúng vào hộp thư đến?

Tóm tắt

- Khả năng đánh giá một mô hình cũng quan trọng như khả năng đào tạo một mô hình.
- Chúng ta có thể sử dụng một số số liệu quan trọng để đánh giá một mô hình. Những cái chúng ta đã học trong chương này là độ chính xác, khả năng thu hồi, độ chính xác, điểm F, độ đặc hiệu và độ nhạy.
- Độ chính xác tính toán tỷ lệ giữa dự đoán đúng và tổng dự đoán. Nó rất hữu ích nhưng có thể thất bại trong một số trường hợp nhất định, đặc biệt khi nhãn dương và âm không cân bằng.
- Lỗi được chia thành hai loại: dương tính giả và âm tính giả.
 - Kết quả dương tính giả là điểm được gắn nhãn tiêu cực mà mô hình dự đoán không chính xác là điểm tích cực. Âm tính giả là một điểm được gắn nhãn dương mà mô hình dự đoán không chính xác là âm.
- Đối với một số mô hình, kết quả âm tính giả và dương tính giả có mức độ quan trọng khác nhau.

- Khả năng thu hồi và độ chính xác là những số liệu hữu ích để đánh giá các mô hình, đặc biệt khi các mô hình đưa ra kết quả âm tính giả và dương tính giả có mức độ quan trọng khác nhau.
 - Việc thu hồi đo lường số lượng điểm tích cực được mô hình dự đoán chính xác. Khả năng thu hồi thấp khi mô hình tạo ra nhiều âm tính giả. Vì lý do này, việc thu hồi là một thước đo hữu ích trong các mô hình mà chúng tôi không muốn có nhiều kết quả âm tính giả, chẳng hạn như các mô hình chẩn đoán y tế.
 - Độ chính xác đo lường có bao nhiêu điểm mà mô hình dự đoán là tích cực thực sự là tích cực. Độ chính xác thấp khi mô hình tạo ra nhiều kết quả dương tính giả. Vì lý do này, độ chính xác là thước đo hữu ích trong các mô hình mà chúng tôi không muốn có nhiều thông tin sai lệch, chẳng hạn như mô hình email spam.
- $F1$ điểm là thước đo hữu ích kết hợp khả năng thu hồi và độ chính xác. Nó trả về một giá trị ở giữa thu hồi và độ chính xác nhưng gần với mức nhỏ hơn trong hai.
- Điểm F_β là một biến thể của điểm F_1 , trong đó người ta có thể điều chỉnh tham số β để mang lại độ chính xác hoặc mức thu hồi có tầm quan trọng cao hơn. Giá trị β cao hơn mang lại tầm quan trọng cao hơn cho việc thu hồi và giá trị thấp hơn của β mang lại tầm quan trọng cao hơn cho độ chính xác. F_β -điểm đặc biệt hữu ích để đánh giá các mô hình trong đó độ chính xác hoặc việc thu hồi quan trọng hơn chỉ số kia nhưng chúng tôi vẫn quan tâm đến cả hai chỉ số.
- Độ nhạy và độ đặc hiệu là hai số liệu hữu ích giúp chúng ta đánh giá mô hình. Chúng được sử dụng nhiều trong lĩnh vực y tế.
 - Độ nhạy hoặc tỷ lệ dương thực sự đo lường số lượng điểm dương được mô hình dự đoán chính xác. Độ nhạy thấp khi mô hình tạo ra nhiều âm tính giả. Vì lý do này, độ nhạy là một thước đo hữu ích để sử dụng trong các mô hình y tế mà chúng tôi không muốn vô tình để lại nhiều bệnh nhân khỏe mạnh mà không được điều trị.
 - Độ đặc hiệu, hay tỷ lệ âm thực, đo lường số lượng điểm âm được mô hình dự đoán chính xác. Độ đặc hiệu thấp khi mô hình tạo ra nhiều kết quả dương tính giả. Vì lý do này, độ đặc hiệu là thước đo hữu ích trong các mô hình y tế mà chúng ta không muốn vô tình điều trị hoặc thực hiện các xét nghiệm xâm lấn sâu hơn trên những bệnh nhân khỏe mạnh.
- Việc thu hồi và độ nhạy là như nhau. Tuy nhiên, độ chính xác và độ đặc hiệu không giống nhau. Độ chính xác đảm bảo rằng hầu hết các kết quả dương tính được dự đoán là thực sự dương tính và kiểm tra độ đặc hiệu để đảm bảo rằng hầu hết các kết quả âm tính thực sự đã được phát hiện.
- Khi chúng tôi tăng ngưỡng trong một mô hình, chúng tôi sẽ giảm độ nhạy và tăng độ đặc hiệu của nó.

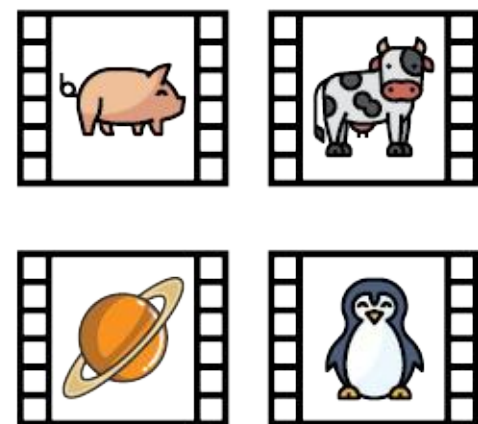
- ROC, hay đường cong đặc tính vận hành máy thu, là một biểu đồ hữu ích giúp chúng ta theo dõi độ nhạy và độ đặc hiệu của mô hình đối với từng giá trị khác nhau của ngưỡng. ROC cũng giúp chúng tôi xác định mức độ tốt của mô hình bằng cách sử dụng diện tích dưới đường cong hoặc AUC. AUC càng gần 1 thì mô hình càng tốt. AUC càng gần 0,5 thì mô hình càng tệ. Bằng cách xem xét đường cong ROC, chúng tôi có thể đưa ra quyết định về ngưỡng sử dụng để mang lại cho chúng tôi các giá trị tốt cho cả độ nhạy và độ đặc hiệu, tùy thuộc vào mức độ mong đợi của từng mô hình. Điều này làm cho đường cong ROC trở thành một trong những cách phổ biến và hữu ích nhất để đánh giá và cải thiện mô hình.

Bài tập

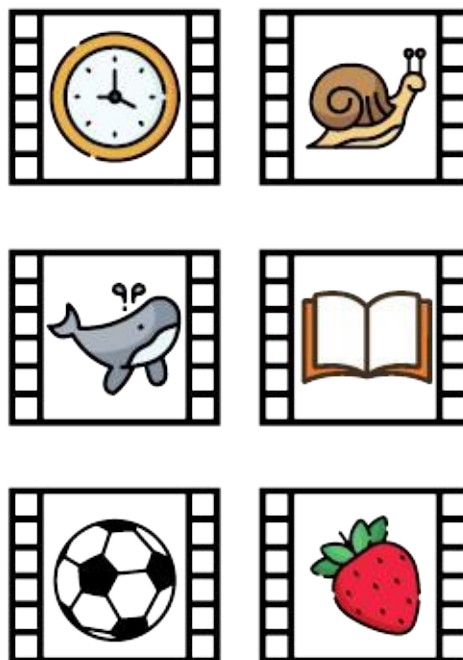
BÀI TẬP 7.1

Một trang web video đã xác nhận rằng một người dùng cụ thể thích video về động vật và hoàn toàn không thích gì khác. Trong hình tiếp theo, chúng ta có thể thấy các đề xuất mà người dùng này nhận được khi đăng nhập vào trang web.

Recommended



Not recommended



Nếu điều này là tất cả dữ liệu chúng ta có trên mô hình, hãy trả lời các câu hỏi sau:

Một. Độ chính xác của mô hình là gì?

b. Việc thu hồi mô hình là gì? c. Độ chính xác của mô hình là gì? d. Điểm F_1 của mô hình là bao nhiêu?

đ. Bạn có cho rằng đây là một mô hình khuyến nghị tốt không?

BÀI TẬP 7.2

Tìm độ nhạy và độ đặc hiệu của mô hình y tế với ma trận nhầm lẫn sau:

	Dự đoán bị bệnh	Dự đoán là khỏe mạnh
Bệnh	120	22
Khỏe mạnh	63	795

BÀI TẬP 7.3

Đối với các mô hình sau, hãy xác định lỗi nào nghiêm trọng hơn, dương tính giả hay âm tính giả. Dựa vào đó, xác định chỉ số nào trong hai chỉ số, độ chính xác hay mức thu hồi, chúng ta nên nhấn mạnh khi đánh giá từng mô hình.

1. Hệ thống đề xuất phim dự đoán liệu người dùng có xem phim hay không
2. Mô hình phát hiện hình ảnh được sử dụng trong ô tô tự lái để phát hiện xem hình ảnh có chứa người đi bộ hay không
3. Trợ lý giọng nói tại nhà dự đoán liệu người dùng có ra lệnh hay không

BÀI TẬP 7.4

Chúng ta được cung cấp các mô hình sau:

1. Mô hình ô tô tự lái phát hiện người đi bộ dựa trên hình ảnh từ camera của ô tô
2. Mô hình y tế chẩn đoán căn bệnh chết người dựa trên các triệu chứng của bệnh nhân
3. Hệ thống đề xuất phim dựa trên các bộ phim đã xem trước đó của người dùng

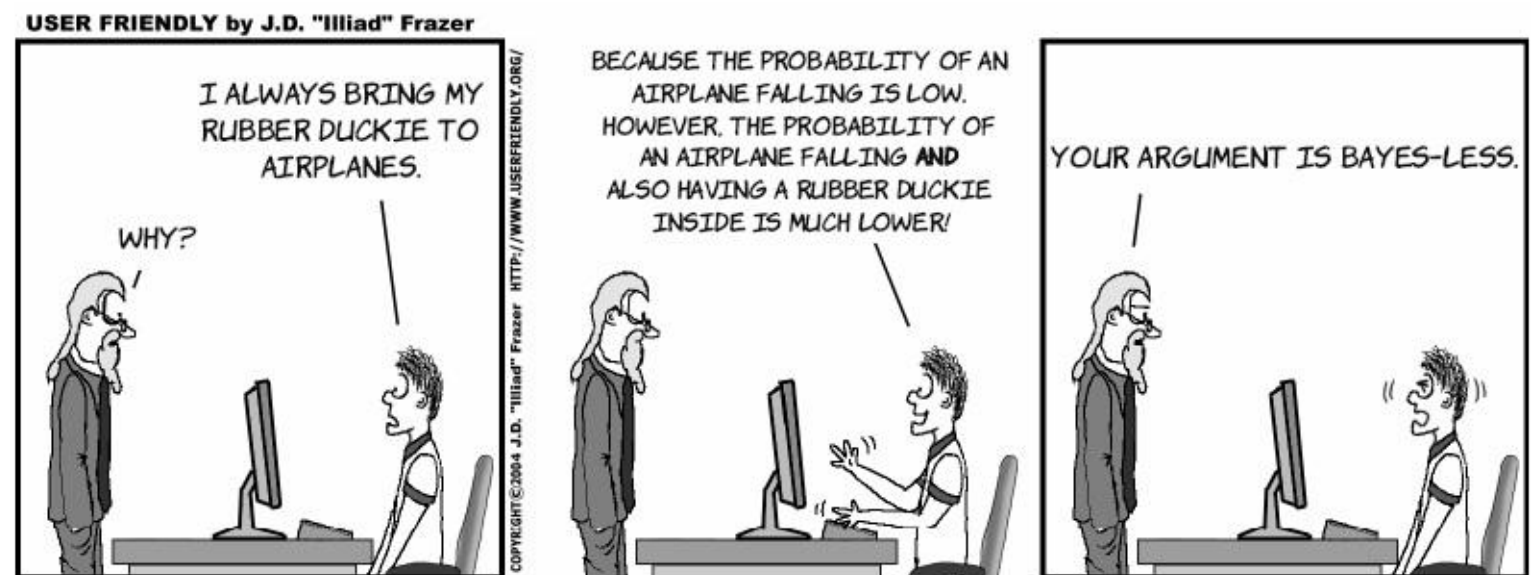
4. Trợ lý giọng nói xác định xem người dùng có cần hỗ trợ khi ra lệnh bằng giọng nói hay không 5.
Mô hình phát hiện thư rác xác định xem email có phải là thư rác hay không dựa trên các từ trong email

Chúng tôi được giao nhiệm vụ đánh giá các mô hình này bằng cách sử dụng điểm F_β . Tuy nhiên, chúng tôi chưa được cung cấp các giá trị của β để sử dụng. Bạn sẽ sử dụng giá trị nào của β để đánh giá từng mô hình?

8 Sử dụng xác suất ở mức tối đa: Mô hình Bayes ngây thơ

Trong chương này

- Các sự kiện độc lập và phụ thuộc vào định lý Bayes là gì xác suất trước và sau tính toán xác suất có điều kiện dựa trên các sự kiện sử dụng mô hình Bayes ngây thơ để dự đoán xem một email là thư rác hay vi phạm, dựa trên các từ trong email mã hóa thuật toán Bayes ngây thơ trong Python
-
-



Naive Bayes là một mô hình học máy quan trọng được sử dụng để phân loại. Mô hình Bayes ngây thơ là một mô hình xác suất thuần túy, có nghĩa là dự đoán là một số từ 0 đến 1, biểu thị xác suất nhận là dương. Thành phần chính của mô hình Bayes ngây thơ là định lý Bayes.

Định lý Bayes đóng vai trò cơ bản trong xác suất và thống kê vì nó giúp tính toán xác suất. Nó dựa trên tiền đề rằng chúng ta càng thu thập được nhiều thông tin về một sự kiện thì ước tính xác suất mà chúng ta có thể đưa ra càng chính xác. Ví dụ: giả sử chúng ta muốn tìm xác suất để hôm nay trời có tuyết. Nếu chúng ta không có thông tin về nơi chúng ta đang ở và thời điểm nào trong năm, chúng ta chỉ có thể đưa ra một ước tính mơ hồ. Tuy nhiên, nếu được cung cấp thông tin, chúng ta có thể ước tính xác suất tốt hơn. Hãy tưởng tượng rằng tôi nói với bạn rằng tôi đang nghĩ đến một

loại động vật, và tôi muốn bạn đoán nó. Xác suất để con vật tôi đang nghĩ tới là một con chó là bao nhiêu? Cho rằng bạn không biết bất kỳ thông tin nào, xác suất là khá nhỏ. Tuy nhiên, nếu tôi nói với bạn rằng con vật mà tôi đang nghĩ đến là thú cưng trong nhà thì khả năng xảy ra sẽ tăng lên khá nhiều. Tuy nhiên, nếu bây giờ tôi nói với bạn rằng con vật mà tôi đang nghĩ đến có cánh thì xác suất bây giờ là bằng không. Mỗi lần tôi cho bạn biết một thông tin mới, ước tính của bạn về xác suất đó là một con chó sẽ ngày càng chính xác hơn. Định lý Bayes là một cách hình thức hóa loại logic này và đưa nó vào các công thức.

Cụ thể hơn, định lý Bayes trả lời câu hỏi “Xác suất của Y khi x xảy ra là bao nhiêu?” được gọi là *xác suất có điều kiện*. Như bạn có thể tưởng tượng, việc trả lời loại câu hỏi này rất hữu ích trong học máy, bởi vì nếu chúng ta có thể trả lời câu hỏi “Xác suất *nhãn là dương tính* với *các tính năng* là bao nhiêu?” chúng tôi có một mô hình phân loại. Ví dụ: chúng ta có thể xây dựng mô hình phân tích tình cảm (giống như chúng ta đã làm trong chương 6) bằng cách trả lời câu hỏi “Xác suất *câu này hài lòng* với *các từ trong đó* là bao nhiêu?” Tuy nhiên, khi chúng ta có quá nhiều đặc trưng (trong trường hợp này là từ), việc tính xác suất sử dụng định lý Bayes trở nên rất phức tạp. Đây là lúc thuật toán Bayes ngây thơ đến giải cứu chúng ta. Thuật toán Bayes ngây thơ sử dụng sự đơn giản hóa khéo léo phép tính này để giúp chúng tôi xây dựng mô hình phân loại mong muốn của mình, được gọi là *mô hình Bayes ngây thơ*. Nó được gọi là *ngây thơ* Bayes vì để đơn giản hóa việc tính toán, chúng tôi đưa ra một giả định hơi ngây thơ nhưng không nhất thiết đúng. Tuy nhiên, giả định này giúp chúng ta đưa ra ước tính tốt về xác suất.

Trong chương này, chúng ta thấy định lý Bayes được sử dụng với một số ví dụ thực tế. Chúng ta bắt đầu bằng việc nghiên cứu một ví dụ y học thú vị và hơi bất ngờ. Sau đó, chúng tôi đi sâu vào mô hình Bayes ngây thơ bằng cách áp dụng nó vào một vấn đề phổ biến trong học máy: phân loại thư rác. Chúng tôi hoàn thiện bằng cách mã hóa thuật toán bằng Python và sử dụng thuật toán đó để đưa ra dự đoán trong tập dữ liệu email spam thực sự.

Tất cả mã cho chương này đều có sẵn tại kho lưu trữ GitHub này:
https://github.com/luisguiserrano/manning/tree/master/Chapter_8_Naive_Bayes.

Ốm hay khỏe? Một câu chuyện với định lý Bayes là người hùng

Hãy xem xét kịch bản sau đây. Người bạn (hơi nghi ngờ) của bạn gọi cho bạn và

cuộc trò chuyện sau đây diễn ra:

Bạn: Xin chào!

Bạn: Xin chào.Tôi có một số tin khủng khiếp!

Bạn:Ồ không, cái gì vậy?

Bạn: Tôi nghe nói về căn bệnh hiếm gặp và khủng khiếp này, tôi đã đến gặp bác sĩ để xét nghiệm. Bác sĩ cho biết cô sẽ thực hiện một cuộc kiểm tra rất chính xác. Rồi hôm nay, cô ấy gọi cho tôi và nói với tôi rằng tôi có kết quả xét nghiệm dương tính! Chắc là tôi có bệnh rồi!

Ồ không!Bạn nên nói gì với bạn của bạn?Trước hết, chúng ta hãy giúp anh ấy bình tĩnh lại và cố gắng tìm hiểu xem liệu anh ấy có khả năng mắc bệnh hay không.

Bạn: Đầu tiên, hãy bình tĩnh lại. Sai lầm xảy ra trong y học. Hãy thử xem khả năng bạn thực sự mắc bệnh là bao nhiêu.Bác sĩ nói kết quả xét nghiệm chính xác đến mức nào?

Bạn: Cô ấy nói chính xác tới 99%. Điều đó có nghĩa là tôi có 99% khả năng mắc bệnh!

Bạn: Đợi đã, chúng ta hãy xem *tất cả* các con số. Khả năng mắc bệnh là bao nhiêu, bất kể xét nghiệm?Có bao nhiêu người mắc bệnh?

Bạn: Tôi đọc trên mạng thì thấy trung bình cứ 10.000 người thì có một người mắc bệnh.

Bạn: Được rồi, để tôi lấy một mảnh giấy (*tạm dùng cho bạn*).

Hãy dừng lại để làm một bài kiểm tra.

câu đố Bạn nghĩ xác suất mà bạn của bạn mắc bệnh nằm trong khoảng nào khi cho rằng anh ấy đã có kết quả xét nghiệm dương tính?

1. 0–20%
2. 20–40%
3. 40–60%
4. 60–80%
5. 80–100%

Hãy tính xác suất này. Tóm lại, chúng tôi có hai thông tin sau:

- Bài kiểm tra đúng 99%. Nói chính xác hơn (chúng tôi đã kiểm tra với bác sĩ để xác nhận điều này), trung bình cứ 100 người khỏe mạnh thì xét nghiệm chẩn đoán chính xác 99 người trong số họ và cứ 100 người bị bệnh thì xét nghiệm chẩn đoán chính xác 99 người trong số họ. Do đó, cả người khỏe và người bệnh, xét nghiệm đều có độ chính xác là 99%. Trung bình cứ 10.000 người thì có 1 người mắc bệnh.

-

Chúng ta hãy thực hiện một số tính toán sơ bộ để xem xác suất sẽ là bao nhiêu. Những điều này được tóm tắt trong ma trận nhầm lẫn được hiển thị trong hình 8.1. Để tham khảo, chúng ta có thể chọn ngẫu nhiên một nhóm gồm một triệu người. Trung bình cứ 10.000 người thì có một người bị bệnh, vì vậy chúng tôi dự đoán 100 người trong số này mắc bệnh và 999.900 người khỏe mạnh.

Đầu tiên, hãy chạy thử nghiệm trên 100 người bị bệnh. Vì xét nghiệm có độ chính xác 99% nên chúng tôi kỳ vọng 99 trong số 100 người này được chẩn đoán chính xác là bị bệnh—tức là 99 người bị bệnh có kết quả xét nghiệm dương tính.

Bây giờ, hãy chạy thử nghiệm trên 999.900 người khỏe mạnh. Xét nghiệm có tỷ lệ mắc lỗi 1%, vì vậy chúng tôi dự đoán 1% trong số 999.900 người khỏe mạnh này sẽ bị chẩn đoán nhầm là bị bệnh. Đó là 9.999 người khỏe mạnh có kết quả xét nghiệm dương tính.

Điều này có nghĩa là tổng số người có kết quả xét nghiệm dương tính là $99 + 9.999 = 10.098$. Trong số này, chỉ có 99 người bị bệnh. Do đó, xác suất để bạn của bạn bị bệnh, với điều kiện là anh ấy có kết quả xét nghiệm dương tính, là $99/10.098 = 0,0098$, hay 0,98%. Đó là ít hơn 1%! Vì vậy, chúng ta có thể quay lại với người bạn của mình.

Bạn: Đừng lo lắng, dựa trên những con số bạn đưa cho tôi, xác suất bạn mắc bệnh vì kết quả xét nghiệm dương tính là dưới 1%!

Bạn: Ôi Chúa ơi, thật sao? Thật là nhẹ nhõm, cảm ơn bạn!

Bạn: Đừng cảm ơn tôi, hãy cảm ơn môn toán (*nháy mắt*).

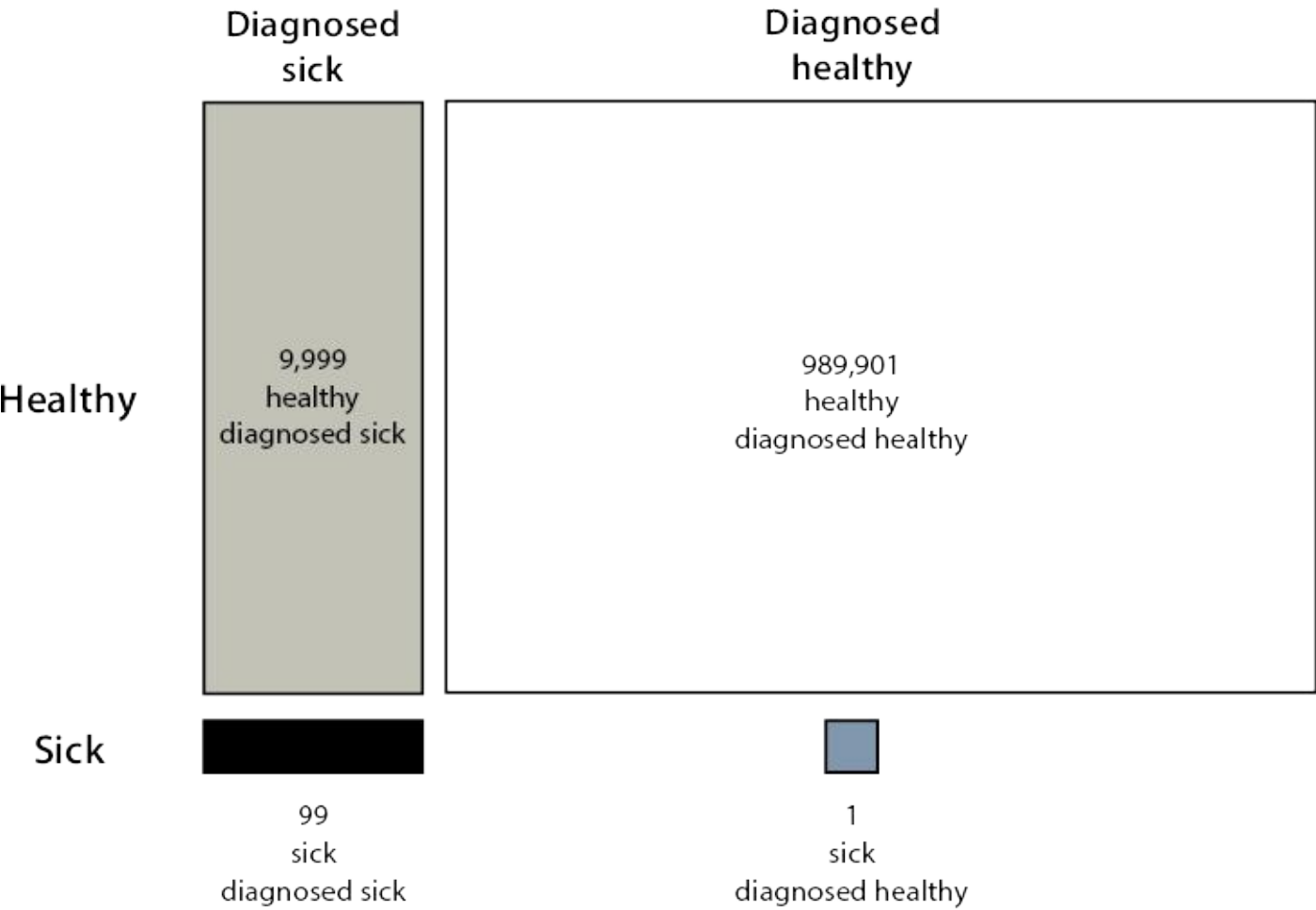
Hãy tóm tắt tính toán của chúng tôi. Đây là sự thật của chúng tôi:

- **Sự thật 1:** Cứ 10.000 người thì có một người mắc bệnh. **Sự thật 2:** Cứ 100 người bệnh làm xét nghiệm thì có 99 người xét nghiệm dương tính và một người xét nghiệm âm tính.

- Sự thật 3:** Cứ 100 người khỏe mạnh làm xét nghiệm thì có 99 người xét nghiệm âm tính và một người xét nghiệm dương tính.

Chúng tôi chọn một mẫu dân số gồm một triệu người, được chia nhỏ trong hình 8.1 như sau:

- Theo thực tế 1, chúng tôi dự đoán có 100 người trong nhóm mẫu của chúng tôi mắc bệnh và 999.900 người khỏe mạnh. Theo sự thật thứ 2, trong số 100 người bị bệnh, 99 người có kết quả xét nghiệm dương tính và một người có kết quả xét nghiệm âm tính. Theo thực tế thứ 3, trong số 999.900 người khỏe mạnh, có 9.999 người có kết quả xét nghiệm dương tính và 989.901 người có kết quả xét nghiệm âm tính

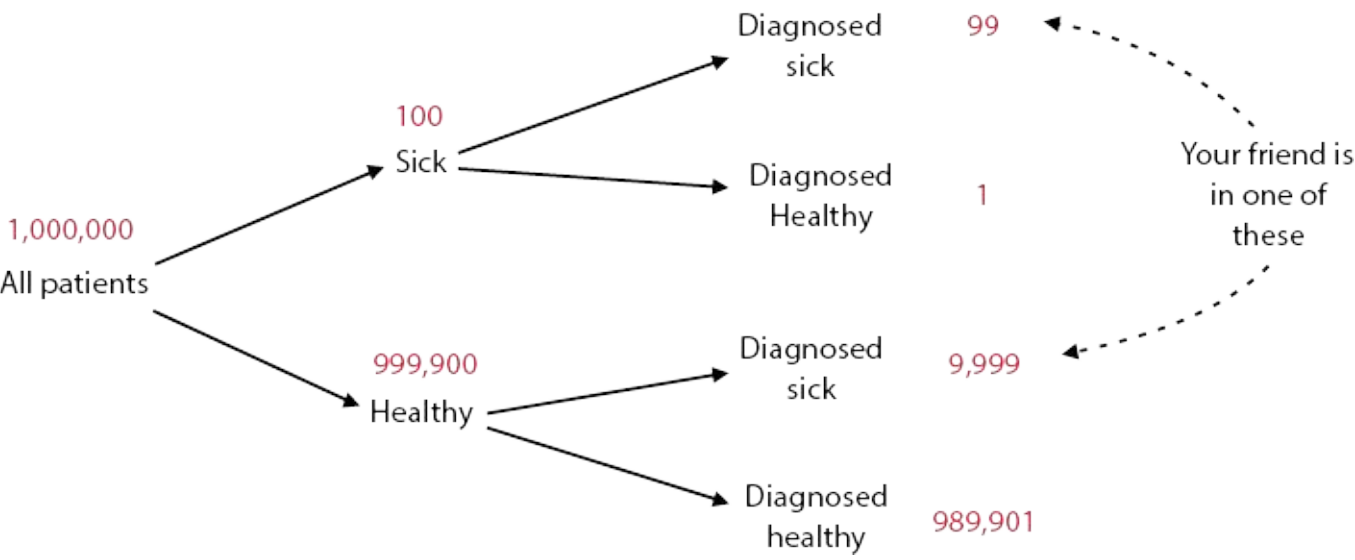


Hình 8.1 Trong số 1.000.000 bệnh nhân của chúng tôi, chỉ có 100 người bị bệnh (hàng dưới cùng). Trong số 10.098 người được chẩn đoán mắc bệnh (cột bên trái), chỉ có 99 người trong số họ thực sự bị bệnh. 9.999 người còn lại khỏe mạnh nhưng bị chẩn đoán nhầm là bị bệnh. Do đó, nếu người bạn của chúng ta được chẩn đoán là bị bệnh, anh ấy có cơ hội nằm trong số 9.999 người khỏe mạnh (trên cùng bên trái) cao hơn nhiều so với nằm trong số 99 người bị bệnh (dưới cùng bên trái).

cột có 9.999 người khỏe mạnh bị chẩn đoán nhầm là bị bệnh và 99 người bị bệnh được chẩn đoán chính xác.Xác suất bạn của bạn bị bệnh là $99/9,999=0,0089$ tức là nhỏ hơn 1%.

Điều này hơi ngạc nhiên, nếu bài kiểm tra đúng 99% thì tại sao nó lại sai đến vậy?Chà, bài kiểm tra cũng không tệ nếu chỉ sai 1% số lần.Nhưng vì cứ 10.000 người thì có một người mắc bệnh, điều đó có nghĩa là có 0,01% số người mắc bệnh.Điều gì có nhiều khả năng nằm trong số 1% dân số bị chẩn đoán sai hoặc nằm trong số 0,01% dân số bị bệnh?Nhóm 1%, mặc dù là một nhóm nhỏ nhưng lớn hơn nhiều so với nhóm 0,01%.Bài kiểm tra có vấn đề;nó có tỷ lệ lỗi lớn hơn nhiều so với tỷ lệ bị bệnh.Chúng tôi gặp vấn đề tương tự trong phần “Hai ví dụ về mô hình: Vi-rút Corona và email spam” ở chương 7—chúng tôi không thể dựa vào độ chính xác để đo lường mô hình này.

Một cách để xem xét điều này là sử dụng sơ đồ dạng cây.Trong sơ đồ của chúng tôi, chúng tôi bắt đầu với gốc ở bên trái, phân nhánh thành hai khả năng: bạn của bạn bị ốm hoặc khỏe mạnh.Mỗi khả năng trong số hai khả năng này đều chia thành hai khả năng nữa: bạn của bạn được chẩn đoán là khỏe mạnh hoặc được chẩn đoán là bị bệnh.Cây được minh họa trong hình 8.2 cùng với số lượng bệnh nhân ở mỗi nhánh.



Hình 8.2 Cây khả năng.Mỗi bệnh nhân có thể bị bệnh hoặc khỏe mạnh.Đối với mỗi khả năng, bệnh nhân có thể được chẩn đoán là ốm yếu hoặc khỏe mạnh, điều này cho chúng ta bốn khả năng.Chúng tôi bắt đầu với một triệu bệnh nhân: 100 người trong số họ bị bệnh và 999.900 người còn lại khỏe mạnh.Trong số 100 người bị bệnh, một người bị chẩn đoán nhầm là khỏe mạnh và 99 người còn lại được chẩn đoán chính xác là bị bệnh.Trong số 999.900 bệnh nhân khỏe mạnh, 9.999 người bị chẩn đoán nhầm là bị bệnh và 989.901 người còn lại được chẩn đoán chính xác là khỏe mạnh.

Từ hình 8.2, một lần nữa chúng ta có thể thấy rằng xác suất bạn của bạn bị bệnh khi anh ấy có kết quả xét nghiệm dương tính là $99/99 + 9,999 = 0,0098$, với điều kiện là anh ấy chỉ có thể thuộc nhóm thứ nhất và thứ ba ở bên phải.

Mở đầu cho định lý Bayes: Sự trước, sự kiện và sự sau

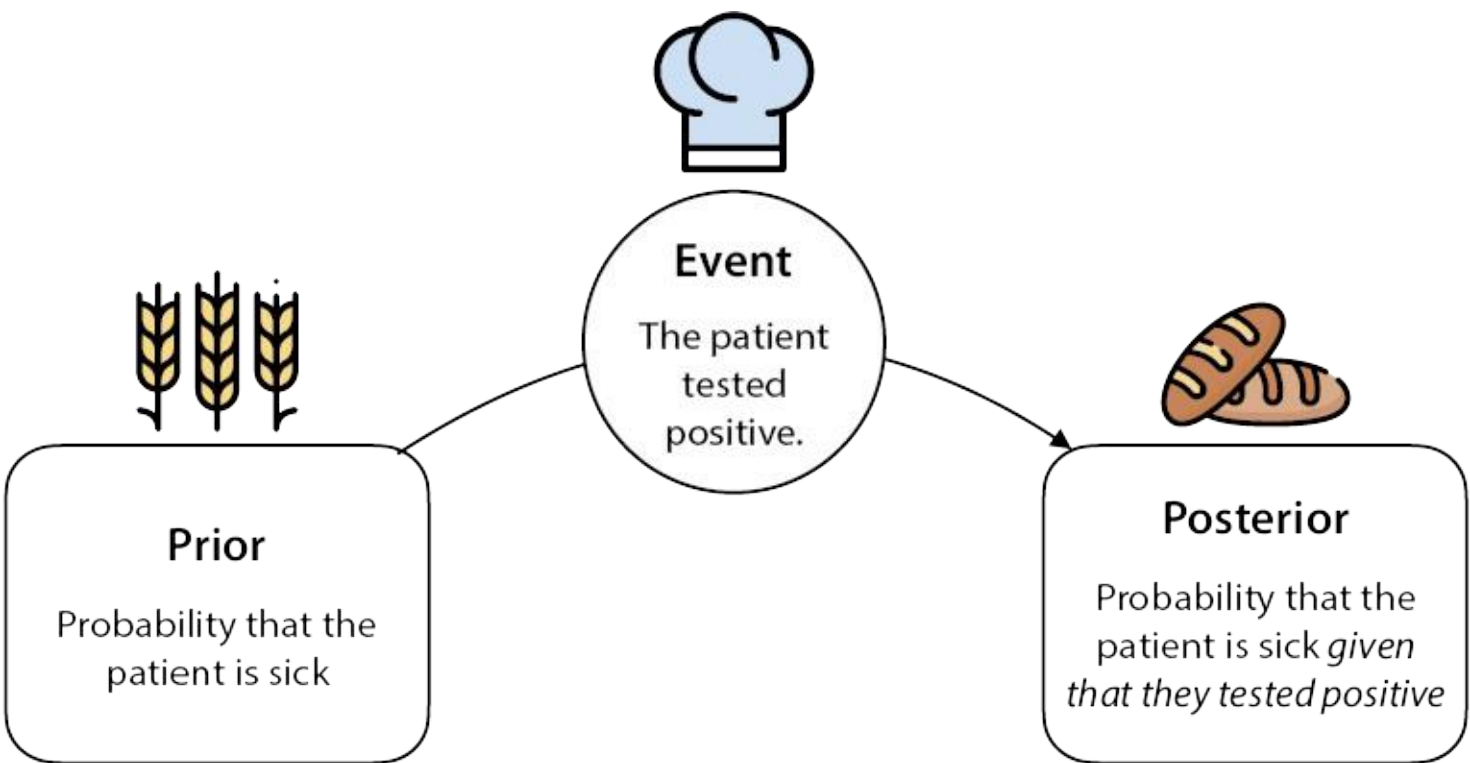
Bây giờ chúng ta có tất cả các công cụ để phát biểu định lý Bayes. Mục tiêu chính của định lý Bayes là tính xác suất. Lúc đầu, không có thông tin trong tay, chúng tôi chỉ có thể tính toán xác suất ban đầu mà chúng tôi gọi là *trước*. Sau đó, một sự kiện xảy ra cung cấp cho chúng ta thông tin. Sau thông tin này, chúng ta có ước tính tốt hơn nhiều về xác suất mà chúng ta muốn tính toán. Chúng tôi gọi đây là ước tính tốt hơn *hậu nghiệm*. Trước, sự kiện và sau được minh họa trong hình 8.3.

trước Xác suất ban đầu

sự kiện Điều gì đó xảy ra, cung cấp cho chúng ta thông tin

sau Xác suất cuối cùng (và chính xác hơn) mà chúng tôi tính toán bằng cách sử dụng xác suất trước đó và sự kiện

Một ví dụ sau đây. Hãy tưởng tượng rằng chúng ta muốn tìm ra xác suất hôm nay trời sẽ mưa. Nếu không biết gì, chúng ta chỉ có thể đưa ra ước tính sơ bộ về xác suất, tức là xác suất trước đó. Nếu chúng ta nhìn xung quanh và phát hiện ra rằng chúng ta đang ở trong rừng mưa Amazon (sự kiện), thì chúng ta có thể đưa ra ước tính chính xác hơn nhiều. Thực tế, nếu chúng ta ở trong rừng mưa Amazon thì có lẽ hôm nay trời sẽ mưa. Ước tính mới này là hậu thế.



Hình 8.3 Phần trước, sự kiện và phần sau. Ưu tiên là xác suất “thô”, tức là xác suất chúng ta tính toán khi chúng ta biết rất ít. Sự kiện là thông tin mà chúng ta thu được, thông tin này sẽ giúp chúng ta tinh chỉnh việc tính toán xác suất. Phần sau là xác suất “nấu chín” hoặc xác suất chính xác hơn nhiều mà chúng ta tính toán khi có nhiều thông tin hơn.

Trong ví dụ y tế đang diễn ra, chúng ta cần tính xác suất một bệnh nhân bị bệnh. Sự việc trước, sự kiện và sự việc sau như sau:

- **Trước:** Ban đầu, xác suất này là $1/10.000$, vì chúng tôi không có thông tin nào khác ngoài việc cứ 10.000 bệnh nhân thì có một người mắc bệnh. Tỷ lệ $1/10.000$, hay $0,0001$ này là tỷ lệ trước đó. **Sự kiện:** Đột nhiên, thông tin mới được đưa ra ánh sáng. Trong trường hợp này, bệnh nhân đã làm xét nghiệm và cho kết quả dương tính. **Hậu nghiệm:** Sau khi cho kết quả dương tính, chúng tôi tính lại xác suất bệnh nhân mắc bệnh và kết quả là $0,0098$. Đây là hậu thế.
-

Định lý Bayes là một trong những nền tảng quan trọng nhất của xác suất và học máy. Điều quan trọng là một số lĩnh vực được đặt theo tên của nó, chẳng hạn như *Học Bayesian*, *Thống kê Bayesian* và *Phân tích Bayesian*. Trong chương này, chúng ta tìm hiểu định lý Bayes và một mô hình phân loại quan trọng bắt nguồn từ nó: mô hình Bayes ngây thơ. Tóm lại, mô hình Bayes ngây thơ thực hiện điều mà hầu hết các mô hình phân loại đều làm, đó là dự đoán một nhãn từ một tập hợp các

đặc trưng. Mô hình trả về câu trả lời dưới dạng xác suất, đó là

tính toán bằng định lý Bayes.

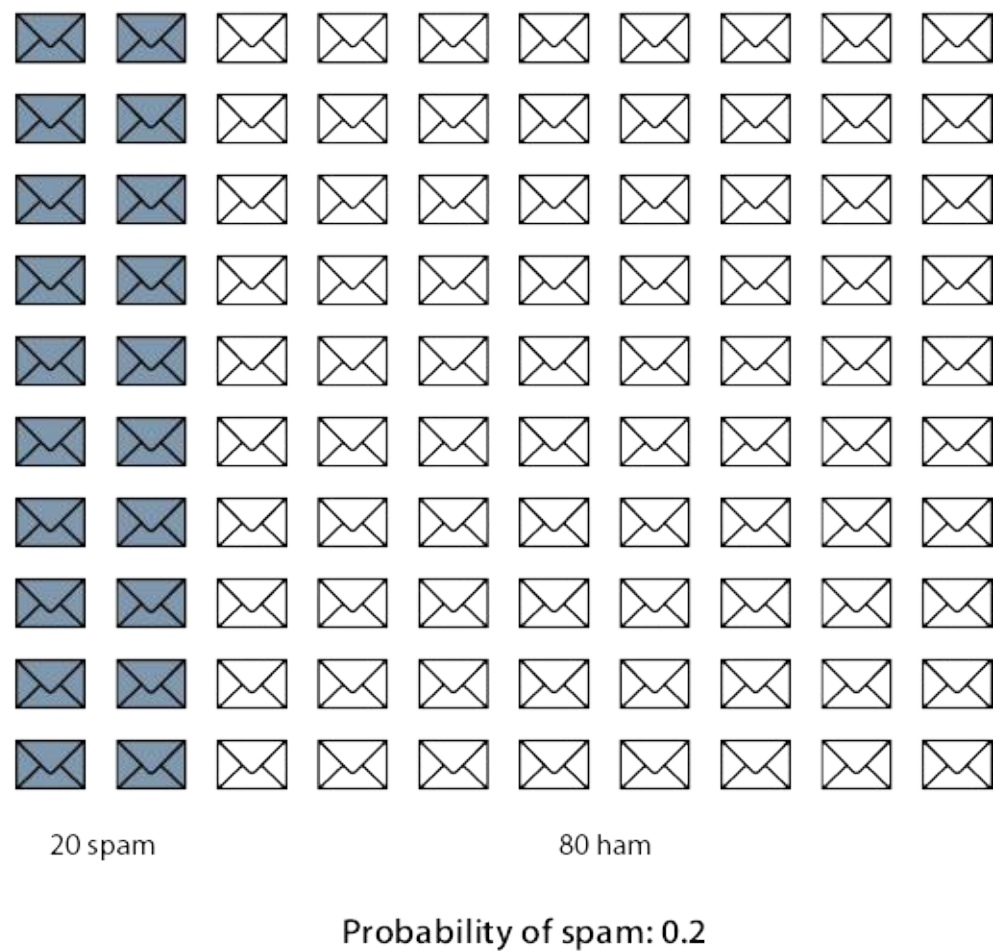
Trường hợp sử dụng: Mô hình phát hiện thư rác

Trường hợp sử dụng mà chúng tôi nghiên cứu trong chương này là mô hình phát hiện thư rác. Mô hình này giúp chúng tôi tách thư rác khỏi email ham. Như chúng ta đã thảo luận trong chương 1 và 7, spam là tên được đặt cho email rác và ham là tên được đặt cho email không phải là email rác.

Mô hình Bayes ngây thơ đưa ra xác suất một email là thư rác hoặc có nội dung xấu. Bằng cách đó, chúng tôi có thể gửi những email có khả năng bị spam cao nhất trực tiếp vào thư mục thư rác và giữ phần còn lại trong hộp thư đến của chúng tôi. Xác suất này sẽ phụ thuộc vào các tính năng của email, chẳng hạn như từ ngữ, người gửi, kích thước, v.v. Đối với chương này, chúng tôi chỉ coi các từ là đặc điểm. Ví dụ này không khác mấy so với ví dụ phân tích cảm tính mà chúng ta đã nghiên cứu ở chương 5 và 6. Điểm mấu chốt của bộ phân loại phân tích cảm xúc này là mỗi từ có xác suất xuất hiện nhất định trong email spam. Ví dụ: từ *xổ số* có nhiều khả năng xuất hiện trong email spam hơn từ *cuộc họp*. Xác suất này là cơ sở tính toán của chúng tôi.

Tìm trước: Xác suất email bất kỳ là thư rác

Xác suất một email là thư rác là bao nhiêu? Đó là một câu hỏi khó, nhưng chúng ta hãy thử đưa ra một ước tính sơ bộ mà chúng ta gọi là ước tính trước. Chúng tôi nhìn vào hộp thư đến hiện tại của mình và đếm xem có bao nhiêu email là thư rác và vi phạm. Hãy tưởng tượng rằng trong 100 email, có 20 email là thư rác và 80 email là thư rác. Như vậy, 20% số email là thư rác. Nếu chúng tôi muốn đưa ra một ước tính hợp lý, chúng tôi có thể nói rằng *theo hiểu biết tốt nhất của chúng tôi*, xác suất một email mới là thư rác là 0,2. Đây là xác suất trước. Phép tính được minh họa trong hình 8.4, trong đó các email spam có màu xám đen và các email ham có màu trắng.



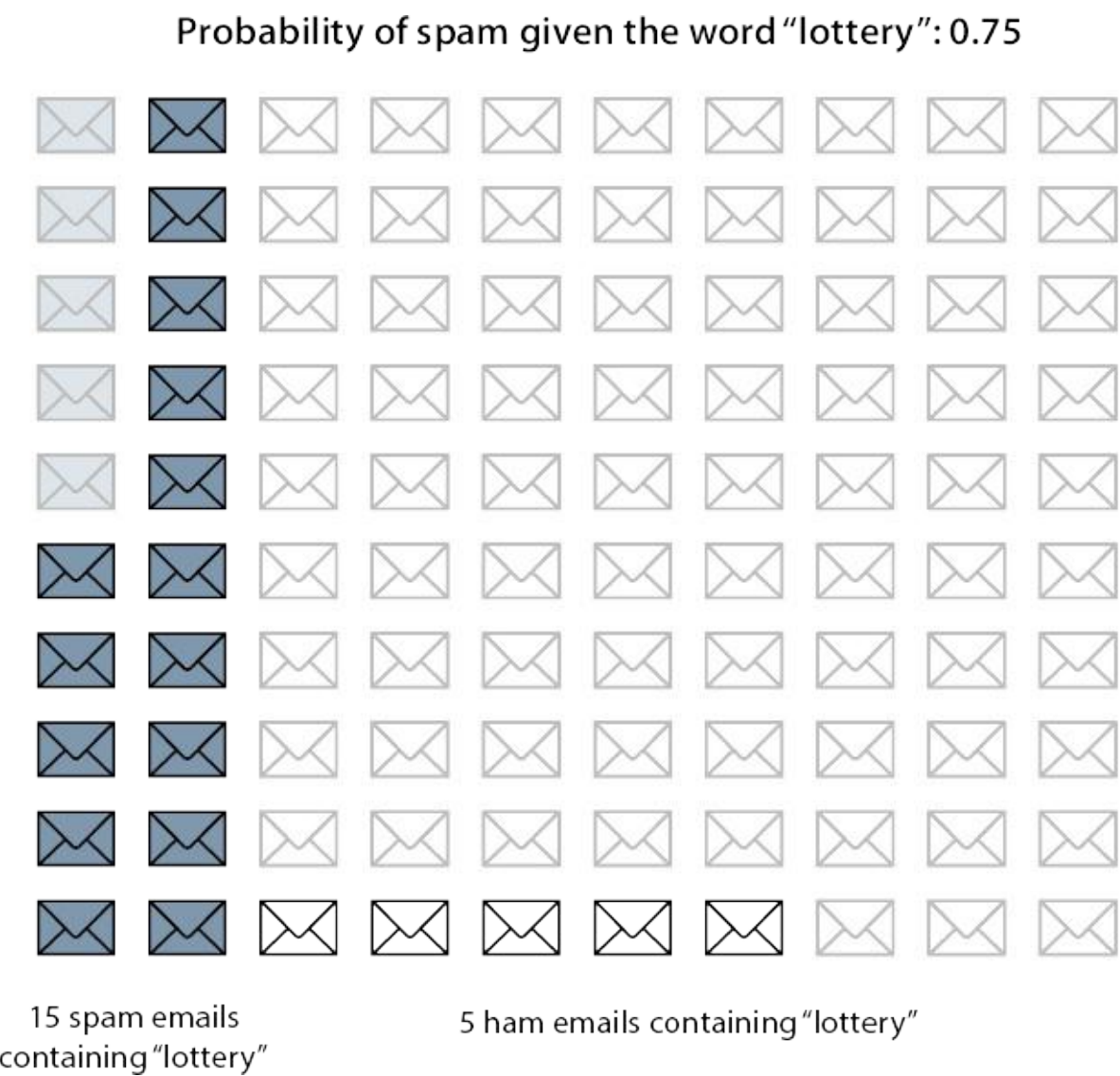
Hình 8.4 Chúng tôi có tập dữ liệu gồm 100 email, trong đó có 20 email là thư rác. Ước tính xác suất một email là thư rác là 0,2. Đây là xác suất trước.

Tìm hậu tố: Xác suất một email là thư rác khi biết rằng trong đó có chứa một từ cụ thể

Tất nhiên, không phải tất cả các email đều được tạo ra như nhau. Chúng tôi muốn đưa ra một phỏng đoán có cơ sở hơn về xác suất bằng cách sử dụng các thuộc tính của email. Chúng ta có thể sử dụng nhiều thuộc tính, chẳng hạn như người gửi, kích thước hoặc từ trong email. Đối với ứng dụng này, chúng tôi chỉ sử dụng các từ trong email. Tuy nhiên, tôi khuyến khích bạn xem qua ví dụ để suy nghĩ về cách sử dụng điều này với các thuộc tính khác.

Giả sử chúng ta tìm thấy một từ cụ thể, chẳng hạn như từ *xổ số*, từ này có xu hướng xuất hiện thường xuyên hơn trong các email spam hơn là trong các email ham. Từ đó tượng trưng cho sự kiện của chúng tôi. Trong số các email spam, từ *xổ số* xuất hiện trong 15 email trong khi nó chỉ xuất hiện trong 5 email ham muốn. Vì vậy, trong số 20 email có chứa từ *xổ số* thì có 15 email là thư rác, 5 email là ham. Như vậy, xác suất một email có chứa từ *xổ số* là thư rác chính xác là $20/15 =$

0,75.Đó là xác suất sau.Quá trình tính xác suất này được minh họa trên hình 8.5.



Hình 8.5 Chúng tôi đã xóa (chuyển sang màu xám) những email không chứa từ xổ số.Đột nhiên, xác suất của chúng tôi thay đổi.Trong số email chứa từ *lottery* có 15 email spam và 5 email ham, nên xác suất một email chứa từ *lottery* là spam là $15/20= 0,75$.

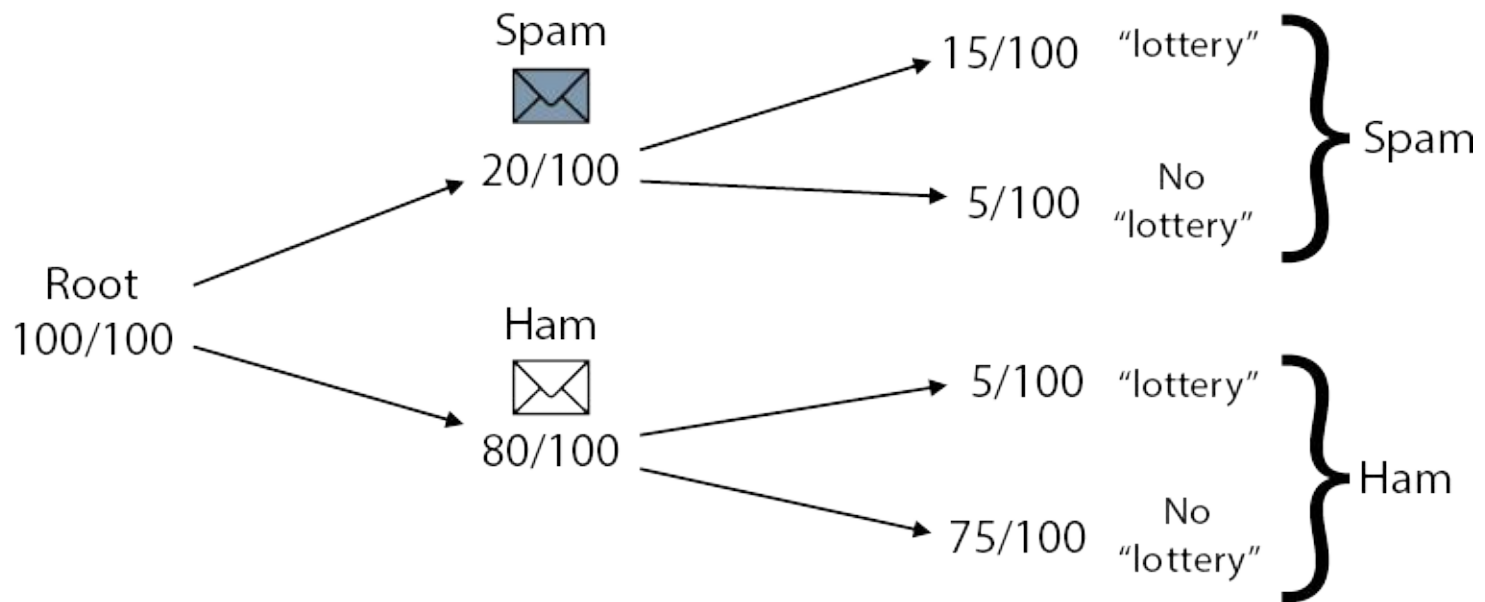
Vậy là chúng tôi đã tính được xác suất một email là thư rác vì nó có chứa từ xổ số.Để tóm tắt:

- Xác suất **trước** là 0,2.Đây là xác suất email đó là thư rác, không biết gì về email đó. **Sự kiện** là email có chứa từ xổ số.Điều này giúp chúng tôi ước tính xác suất tốt hơn. Xác suất **sau** là
- 0,75.Đây là xác suất email đó là thư rác, vì trong đó có chứa từ xổ số.
-

Trong ví dụ này, chúng tôi đã tính xác suất bằng cách đếm email và chia. Điều này chủ yếu được thực hiện cho mục đích sư phạm, nhưng trong thực tế, chúng ta có thể sử dụng một phím tắt để tính xác suất này bằng công thức. Công thức này được gọi là định lý Bayes, và chúng ta sẽ xem nó tiếp theo.

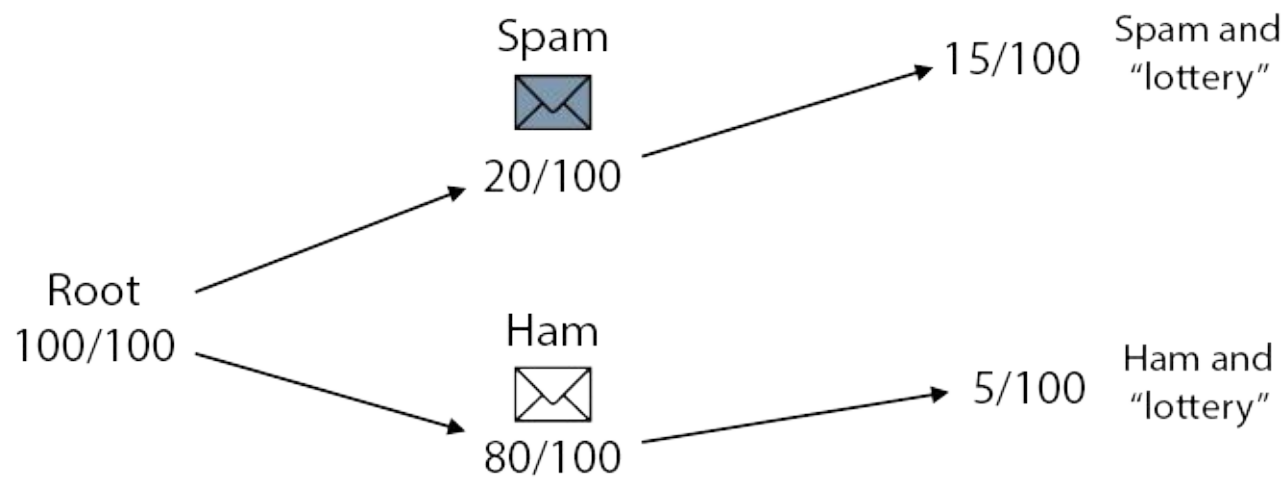
Chuyện gì vừa xảy ra vậy? Biến tỷ lệ thành xác suất

Một cách để hình dung ví dụ trước là sử dụng một cây có cả bốn khả năng, giống như chúng ta đã làm với ví dụ y tế trong hình 8.2. Có khả năng email đó là spam hoặc ham và có chứa từ *xổ số* hay không. Chúng tôi vẽ nó theo cách sau: chúng tôi bắt đầu với phần gốc, chia thành hai nhánh. Nhánh trên tương ứng với thư rác và nhánh dưới tương ứng với ham. Mỗi nhánh sẽ chia thành hai nhánh nữa, đó là khi email chứa từ *xổ số* và khi thì không. Cây được minh họa trong hình 8.6. Lưu ý rằng trong cây này, chúng tôi cũng đã chỉ ra có bao nhiêu email trong tổng số 100 email thuộc về mỗi nhóm cụ thể.



Hình 8.6 Cây khả năng. Gốc chia làm 2 nhánh: spam và ham. Sau đó, mỗi nhánh này sẽ chia thành hai nhánh: khi email chứa từ *xổ số* và khi không.

Khi chúng tôi có cây này và chúng tôi muốn tính xác suất một email là thư rác vì nó chứa từ *xổ số*, chúng tôi chỉ cần xóa tất cả các nhánh trong đó email không chứa từ *xổ số*. Điều này được minh họa trong hình 8.7.



Hình 8.7 Từ cây trước, chúng tôi đã loại bỏ hai nhánh trong đó các email không chứa từ xổ số. Trong số 100 email ban đầu, chúng tôi còn lại 20 email chứa xổ số. Vì trong 20 email này, 15 email là thư rác nên chúng tôi kết luận rằng xác suất một email là thư rác vì nó chứa từ xổ số là 0,75.

Bây giờ, chúng tôi có 20 email, trong đó có 15 email là spam và 5 email là ham. Như vậy, xác suất một email là thư rác vì nó có chứa từ xổ số là .

$$\frac{15}{20} = 0.75$$

Nhưng chúng ta đã làm điều đó rồi, vậy lợi ích của sơ đồ là gì? Ngoài việc làm cho mọi việc đơn giản hơn, lợi ích là thông thường, thông tin chúng tôi có được dựa trên xác suất chứ không dựa trên số lượng email. Nhiều khi chúng tôi không biết có bao nhiêu email là thư rác hoặc rác. Tất cả những gì chúng tôi biết là như sau:

- Xác suất để một email là thư rác là . $\frac{1}{5} = 0.2$
- Xác suất để một email spam chứa từ xổ số là . $\frac{3}{4} = 0.75$
- Xác suất để một email ham chứa từ xổ số là . **Câu hỏi:** Xác suất một email chứa từ xổ số là thư rác là bao nhiêu? $\frac{1}{7} = 0.0625$

Trước tiên, hãy kiểm tra xem thông tin này đã đủ chưa. Chúng ta có biết xác suất để một email có bị ham không? Chà, chúng tôi biết rằng xác suất đó là thư rác là . *chỉ* $\frac{1}{5} = 0.2$

khả năng khác là email bị lỗi nên nó phải là phần bổ sung hoặc . Đây là một quy tắc quan trọng – quy tắc xác suất bổ sung. $\frac{4}{5} = 0.8$

quy tắc xác suất bổ sung Đối với biến cố E , phần bù của biến cố E , ký hiệu là E^c , là biến cố nghịch với E . Xác suất của E^c bằng 1 trừ xác suất của E , cụ thể là:

$$P(E^c) = 1 - P(E)$$

Vì vậy, chúng tôi có những điều sau đây:

- $P(spam) = \frac{1}{5} = 0.2$: xác suất một email là thư rác
- $P(ham) = \frac{4}{5} = 0.8$: xác suất một email bị hỏng

Bây giờ chúng ta hãy xem các thông tin khác. Xác suất chứa một email spam

từ *xổ số* là $\frac{3}{4} = 0.75$. Điều này có thể được hiểu là xác suất một email chứa từ *xổ số* cho rằng đó là thư rác, là 0,75. Đây là xác suất có điều kiện, trong đó điều kiện là email đó là thư rác. Chúng ta biểu thị tình trạng bằng một thanh dọc nên có thể viết là $P('lottery'|spam)$. Phần bổ sung của điều này là $P(\text{không có 'xổ số'}|thư rác)$, cụ thể là xác suất một email spam không chứa từ *xổ số*. Xác suất này là $1 - P('xổ số'|thư rác)$. Bằng cách này, chúng ta có thể tính các xác suất khác như sau:

- $P('lottery'|spam) = \frac{3}{4} = 0.75$: xác suất một email spam có chứa từ *xổ số*. : xác suất một email spam không chứa từ *xổ số*. : xác suất một email ham chứa từ *xổ số*. : xác suất một email ham không chứa từ *xổ số*.
- $P(\text{no 'lottery'}|spam) = \frac{1}{4} = 0.25$
- $P('lottery'|ham) = \frac{1}{16} = 0.0625$
- $P(\text{no 'lottery'}|ham) = \frac{15}{16} = 0.9375$

Việc tiếp theo chúng ta làm là tìm xác suất để hai sự kiện xảy ra *cùng một lúc*. Cụ thể hơn, chúng tôi muốn có bốn xác suất sau:

- Xác suất một email là spam và chứa từ *lottery*
- Xác suất một email là spam và không chứa từ *lottery*
- Xác suất một email là ham và chứa từ *lottery*
- Xác suất một email là ham và không chứa từ *lottery*

Những sự kiện này được gọi là *điểm giao* của các sự kiện và được ký hiệu bằng ký hiệu $\cap$. Vì vậy, chúng ta cần tìm các xác suất sau:

- $P('xổ số' \cap \text{thư rác})$

- $P(\text{không có 'xổ số'} \cap \text{spam})$
- $P(\text{'xổ số'} \cap \text{ham})$
- $P(\text{'xổ số'} \cap \text{giảm bông})$

Hãy nhìn vào một số con số. Chúng tôi biết rằng, hoặc $\frac{1}{20}$ trên 100 email là thư rác.

Trong số 20 cái đó có $\frac{3}{4}$ chứa từ xổ số. Cuối cùng, chúng tôi nhân chúng

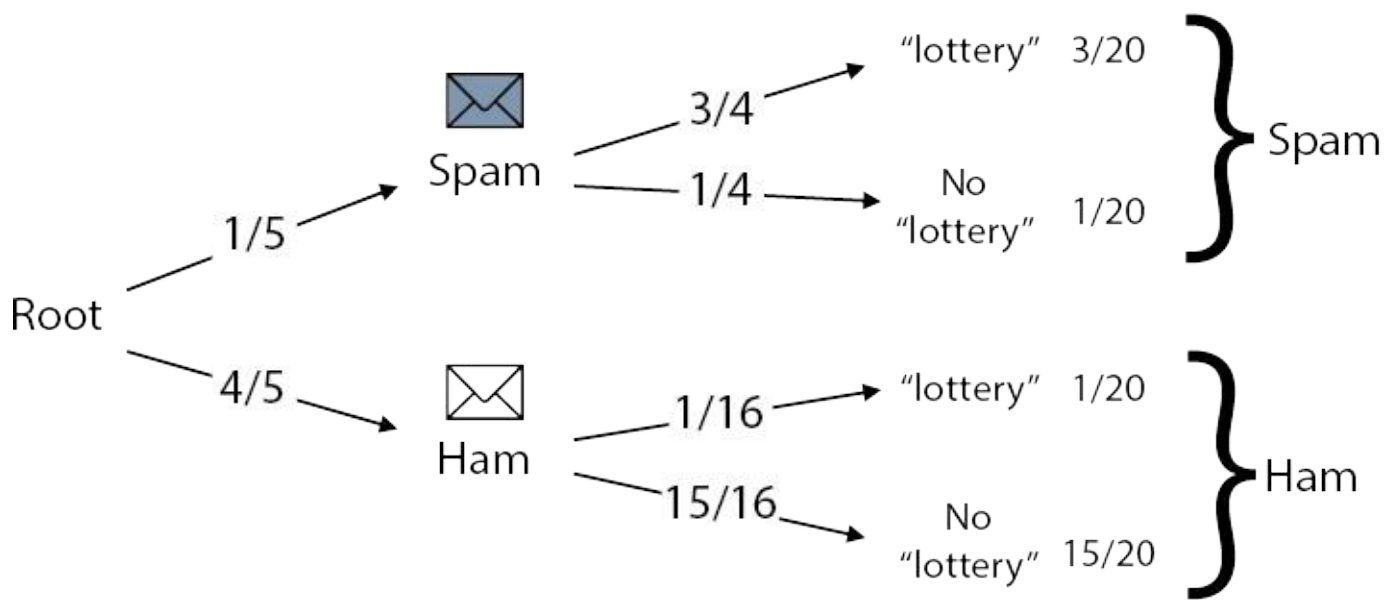
hai con số, thời gian $\frac{1}{5}$, để có được $\frac{3}{4}$, tương đương với $\frac{3}{20}$, tỷ lệ email là thư rác và chứa từ xổ số. Những gì chúng tôi đã làm như sau: chúng tôi nhân xác suất một email là thư rác với xác suất một email spam chứa từ xổ số để có được xác suất một email là thư rác và chứa từ xổ số. Xác suất một email spam chứa từ xổ số chính xác là xác suất có điều kiện hoặc xác suất một email chứa từ xổ số vì đó là một email spam. Điều này dẫn đến quy tắc nhân xác suất.

Quy tắc tích xác suất Đối với các sự kiện E và F , xác suất giao nhau của chúng là tích của xác suất có điều kiện của F cho E , nhân với xác suất của F , cụ thể là $P(E \cap F) = P(E|F) \cap P(F)$.

Bây giờ chúng ta có thể tính toán các xác suất này như sau:

- $P(\text{'lottery'} \cap \text{spam}) = P(\text{'lottery'} | \text{spam}) \cap P(\text{spam}) = \frac{3}{4} \cdot \frac{1}{5} = \frac{3}{20} = 0.15$
- $P(\text{no 'lottery'} \cap \text{spam}) = P(\text{no 'lottery'} | \text{spam}) \cap P(\text{spam}) = \frac{1}{4} \cdot \frac{1}{5} = \frac{1}{20} = 0.05$
- $P(\text{no 'lottery'} \cap \text{ham}) = P(\text{no 'lottery'} | \text{ham}) \cap P(\text{ham}) = \frac{1}{16} \cdot \frac{4}{5} = \frac{1}{20} = 0.05$
- $P(\text{no 'lottery'} \cap \text{ham}) = P(\text{no 'lottery'} | \text{ham}) \cdot P(\text{ham}) = \frac{15}{16} \cdot \frac{4}{5} = \frac{15}{20} = 0.75$

Những xác suất này được tóm tắt trong hình 8.8. Lưu ý rằng tích của các xác suất ở các cạnh là xác suất ở bên phải. Hơn nữa, hãy lưu ý rằng tổng của cả bốn xác suất này là một, vì chúng bao gồm tất cả các tình huống có thể xảy ra.

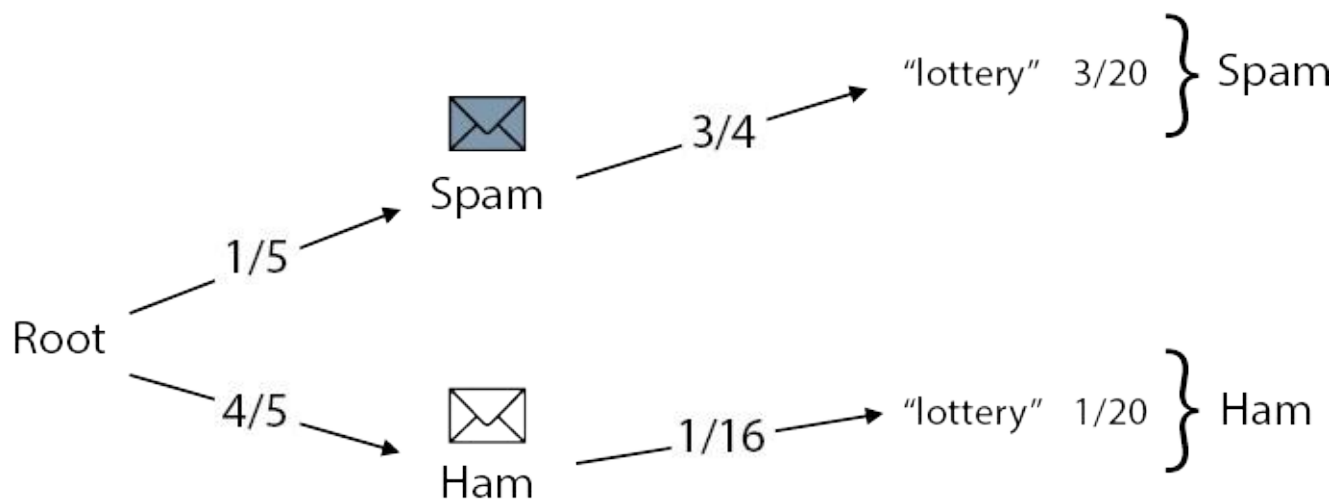


Hình 8.8 Cây tương tự từ hình 8.6, nhưng bây giờ có xác suất. Từ gốc, hai nhánh xuất hiện, một nhánh dành cho email spam và một nhánh dành cho email ham. Trong mỗi cái, chúng tôi ghi lại xác suất tương ứng. Mỗi nhánh lại chia thành hai lá: một lá dành cho các email có chứa từ *xổ số* và một lá dành cho các email không chứa từ đó. Ở mỗi nhánh chúng tôi ghi lại xác suất tương ứng. Lưu ý rằng tích của các xác suất này là xác suất ở bên phải của mỗi lá. Ví dụ: đối với lá trên cùng là $1/5 \cdot 3/4 = 3/20 = 0,15$.

Chúng tôi gần như đã hoàn tất. Chúng tôi muốn tìm $P(\text{spam} | \text{'lottery'})$, đó là xác suất một email là spam vì trong đó có chứa từ *lottery*. Trong số 4 sự kiện chúng ta vừa nghiên cứu, chỉ có 2 sự kiện có từ *xổ số* xuất hiện. Vì vậy, chúng ta chỉ cần xem xét những điều đó, cụ thể là:

- $P(\text{'lottery'} \cap \text{spam}) = \frac{3}{20} = 0.15$
- $P(\text{'lottery'} \cap \text{ham}) = \frac{1}{20} = 0.05$

Nói cách khác, chúng ta chỉ cần xem xét hai nhánh được hiển thị trong hình 8.9— nhánh thứ nhất và nhánh thứ ba, cụ thể là những nhánh trong đó email có chứa từ *xổ số*.



Hình 8.9 Từ cây trong hình 8.8, chúng tôi đã loại bỏ hai nhánh trong đó các email không chứa từ xổ số.

Đầu tiên là xác suất một email là thư rác và thứ hai là xác suất email đó bị hỏng. Hai xác suất này không cộng lại thành một. Tuy nhiên, vì chúng ta hiện đang sống trong một thế giới mà email có chứa từ xổ số, nên hai trường hợp này là những tình huống duy nhất có thể xảy ra. Do đó, xác suất của chúng phải cộng bằng 1. Hơn nữa, chúng vẫn phải có cùng tỷ lệ tương đối với nhau. Cách khắc phục điều này là chuẩn hóa—tìm hai số

có cùng tỷ lệ tương đối với nhau nhưng lại cộng lại thành một. Cách để tìm những số này là chia cả hai cho tổng. Trong trường hợp này,

các số trở thành $\frac{3/20}{3/20 + 1/20} = \frac{3}{4}$ và $\frac{1/20}{3/20 + 1/20} = \frac{1}{4}$. Chúng đơn giản hóa thành $\frac{3}{4}$ và $\frac{1}{4}$, là các xác suất mong muốn. Vì vậy, chúng tôi kết luận như sau:

- $P(\text{spam} \mid \text{'lottery'}) = \frac{3}{4} = 0.75$
- $P(\text{ham} \mid \text{'lottery'}) = \frac{1}{4} = 0.25$

Đây chính xác là những gì chúng tôi đã tìm ra khi đếm số email. Để tóm tắt thông tin này, chúng ta cần một công thức. Chúng tôi có hai xác suất: xác suất một email là thư rác và chứa từ xổ số và xác suất một email là thư rác và không chứa từ xổ số. Để thêm chúng vào một, chúng tôi đã chuẩn hóa chúng. Điều này cũng tương tự như việc chia từng phần cho tổng của chúng. Về mặt toán học, chúng tôi đã làm như sau:

$$P(\text{spam} \mid \text{'lottery'}) = \frac{P(\text{'lottery'} \cap \text{spam})}{P(\text{'lottery'} \cap \text{spam}) + P(\text{'lottery'} \cap \text{ham})}$$

Nếu chúng ta nhớ hai xác suất này là gì, bằng cách sử dụng quy tắc tích, chúng ta sẽ nhận được kết quả sau:

$$P(\text{spam} | \text{'lottery'}) = \frac{P(\text{'lottery'} | \text{spam}) \cdot P(\text{spam})}{P(\text{'lottery'} | \text{spam}) \cdot P(\text{spam}) + P(\text{'lottery'} | \text{ham}) \cdot P(\text{ham})}$$

Để xác minh, chúng tôi cầm số để có được:

$$P(\text{spam} | \text{'lottery'}) = \frac{\frac{1}{5} \cdot \frac{3}{4}}{\frac{1}{5} \cdot \frac{3}{4} + \frac{4}{5} \cdot \frac{1}{16}} = \frac{\frac{3}{20}}{\frac{3}{20} + \frac{1}{20}} = \frac{\frac{3}{20}}{\frac{4}{20}} = \frac{3}{4} = 0.75$$

Đây là công thức của định lý Bayes! Chính thức hơn:

Định lý Bayes Đối với biến cố E và F,

$$P(E | F) = \frac{P(F | E) \cdot P(E)}{P(F)}$$

Vì sự kiện F có thể chia thành hai sự kiện rời nhau $F|E$ và $F|E^c$ nên

$$P(E | F) = \frac{P(F | E) \cdot P(E)}{P(F | E) \cdot P(E) + P(F | E^c) \cdot P(E^c)}$$

Cái gì vậy về hai từ? Thuật toán Bayes ngây thơ

Trong phần trước, chúng tôi đã tính xác suất một email là thư rác nếu email đó chứa từ khóa *xổ số*. Tuy nhiên, từ điển chứa nhiều từ hơn và chúng tôi muốn tính xác suất một email là thư rác nếu nó chứa nhiều từ. Như bạn có thể tưởng tượng, việc tính toán trở nên phức tạp hơn, nhưng trong phần này, chúng ta tìm hiểu một thủ thuật giúp chúng ta ước tính xác suất này.

Nói chung, thủ thuật này giúp chúng ta tính toán xác suất hậu nghiệm dựa trên hai sự kiện thay vì một (và nó dễ dàng khái quát hóa thành nhiều hơn hai sự kiện). Nó dựa trên tiên đề rằng khi các sự kiện là độc lập thì xác suất cả hai sự kiện xảy ra cùng lúc là tích của xác suất của chúng. Các sự kiện không phải lúc nào cũng độc lập, nhưng việc giả sử chúng đôi khi giúp chúng ta đưa ra những ước tính gần đúng. Ví dụ, hãy tưởng tượng tình huống sau: có một hòn đảo

với 1.000 người. Một nửa số cư dân (500) là phụ nữ và 1/10 cư dân (100) có mắt nâu. Bạn nghĩ có bao nhiêu cư dân là phụ nữ có mắt nâu? Nếu tất cả những gì chúng tôi biết chỉ là thông tin này, chúng tôi không thể tìm ra trừ khi chúng tôi trực tiếp đếm chúng. Tuy nhiên, nếu chúng ta giả định rằng giới tính và màu mắt là độc lập thì chúng ta có thể ước tính rằng một nửa dân số

bao gồm những phụ nữ có đôi mắt nâu. Đó là của dân số $\frac{1}{2} \cdot \frac{1}{10} = \frac{1}{20}$ và tổng dân số là 1.000 người nên ước tính của chúng tôi về số phụ nữ mắc bệnh

mắt nâu là người. $1000 \cdot \frac{1}{20} = 50$ Có thể chúng tôi đi đến hòn đảo và phát hiện ra rằng không phải như vậy, nhưng *theo theo hiểu biết tốt nhất của chúng tôi* thì 50 là một ước tính tốt. Người ta có thể nói rằng giả định của chúng tôi về sự độc lập về giới tính và màu mắt là *ngây thơ*, và có thể đúng như vậy, nhưng đó là ước tính tốt nhất mà chúng tôi có thể đưa ra dựa trên thông tin chúng tôi có.

Quy tắc chúng tôi sử dụng trong ví dụ trước là quy tắc tích cho xác suất độc lập, quy tắc này nêu rõ như sau:

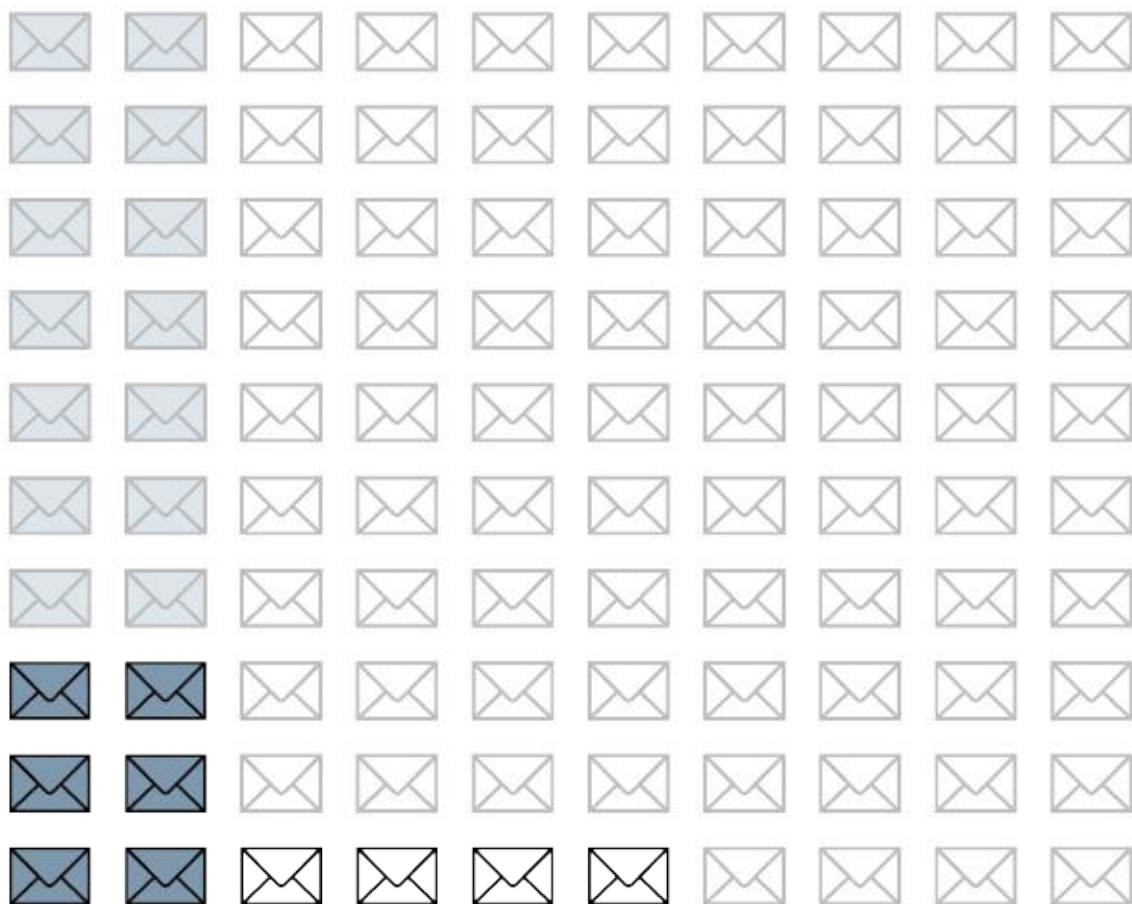
quy tắc tích cho xác suất độc lập Nếu hai sự kiện E và F độc lập, tức là sự xuất hiện của một sự kiện không ảnh hưởng đến sự xuất hiện của sự kiện kia theo bất kỳ cách nào, thì xác suất cả hai xảy ra (giao điểm của các sự kiện) là tích của xác suất của từng sự kiện. Nói cách khác,

$$P(E \cap F) = P(E) \cdot P(F).$$

Quay lại ví dụ về email. Sau khi chúng tôi tìm ra xác suất một email là thư rác vì nó chứa từ *xổ số*, chúng tôi nhận thấy rằng một từ khác, *bán*, cũng có xu hướng xuất hiện nhiều trong email spam. Chúng tôi muốn tìm ra xác suất một email là thư rác vì nó chứa cả *xổ số* và *bán hàng*. Chúng tôi bắt đầu bằng cách đếm xem có bao nhiêu email spam và ham chứa từ *bán hàng* và thấy rằng nó xuất hiện ở 6 trong số 20 email spam và 4 trong số 80 email ham. Do đó, các xác suất như sau (minh họa trong hình 8.10):

- $P(\text{'sale'} \mid \text{spam}) = \frac{6}{20} = 0.3$
- $P(\text{'sale'} \mid \text{ham}) = \frac{4}{80} = 0.05$

Probability of spam given the word "sale": 0.6



6 spam emails
containing "sale"

4 ham emails containing "sale"

Hình 8.10 Trong phép tính tương tự như đối với từ *xổ số*, chúng ta xem xét các email có chứa từ *bán*. Trong số những email (không có màu xám) này, có sáu email spam và bốn email có nội dung xấu.

Người ta có thể sử dụng lại định lý Bayes để kết luận rằng xác suất một email là thư rác vì nó chứa từ *bán* là 0,6 và tôi khuyến khích bạn tự mình thực hiện các phép tính. Tuy nhiên, câu hỏi quan trọng hơn là: xác suất một email là thư rác là bao nhiêu khi nó chứa các từ *xổ số* và *bán* cùng một lúc? Trước khi thực hiện việc này, hãy tìm xác suất để một email chứa các từ *xổ số* và *bán hàng* nếu đó là thư rác. Điều này thật dễ dàng: chúng tôi xem qua tất cả các email của mình và tìm xem có bao nhiêu email spam có các từ *xổ số* và *bán hàng*.

Tuy nhiên, chúng tôi có thể gặp phải vấn đề là không có email nào có từ *xổ số* và *bán hàng*. Chúng tôi chỉ có 100 email và khi chúng tôi cố gắng tìm hai từ trên đó, chúng tôi có thể không có đủ để ước tính xác suất chính xác. Chúng ta có thể làm gì? Một giải pháp khả thi là thu thập thêm dữ liệu cho đến khi chúng tôi có nhiều email đến mức có khả năng một số trong số đó sẽ xuất hiện hai từ này.

Tuy nhiên, trường hợp có thể xảy ra là chúng tôi không thể thu thập thêm dữ liệu nào nữa nên chúng tôi phải làm việc với những gì mình có. Đây là nơi giả định ngây thơ sẽ giúp chúng ta.

Hãy thử ước tính xác suất này giống như cách chúng tôi ước tính số phụ nữ có mắt nâu trên đảo ở đầu phần này. Chúng tôi biết rằng xác suất từ *xổ số* xuất hiện trong email spam là 0,75, tính từ phần trước. Từ phần trước của phần này, xác suất *bán hàng* xuất hiện trong email spam là 0,6. Như vậy, nếu chúng ta ngây thơ cho rằng sự xuất hiện của hai từ này là độc lập thì xác suất cả hai từ này xuất hiện trong một email spam là $0,75 \cdot 0,3 = 0,225$. Theo cách tương tự, vì chúng tôi đã tính toán rằng xác suất của một email ham chứa từ *xổ số* là 0,0625 và chứa từ *bán* là 0,05, thì xác suất của một email ham chứa cả hai là $0,0625 \cdot 0,05 = 0,003125$. Nói cách khác, chúng tôi đã thực hiện các ước tính sau:

- $P('xổ số', 'bán hàng'|thư rác) = P('xổ số'|thư rác) P('bán'|thư rác) = 0,75 \cdot 0,3 = 0,225$
 $P('xổ số', 'giảm giá'|ham) = P('xổ số'|ham) P('sale'|ham) = 0,0625 \cdot 0,05 = 0,003125$
-

Giả định ngây thơ mà chúng tôi đã đưa ra như sau:

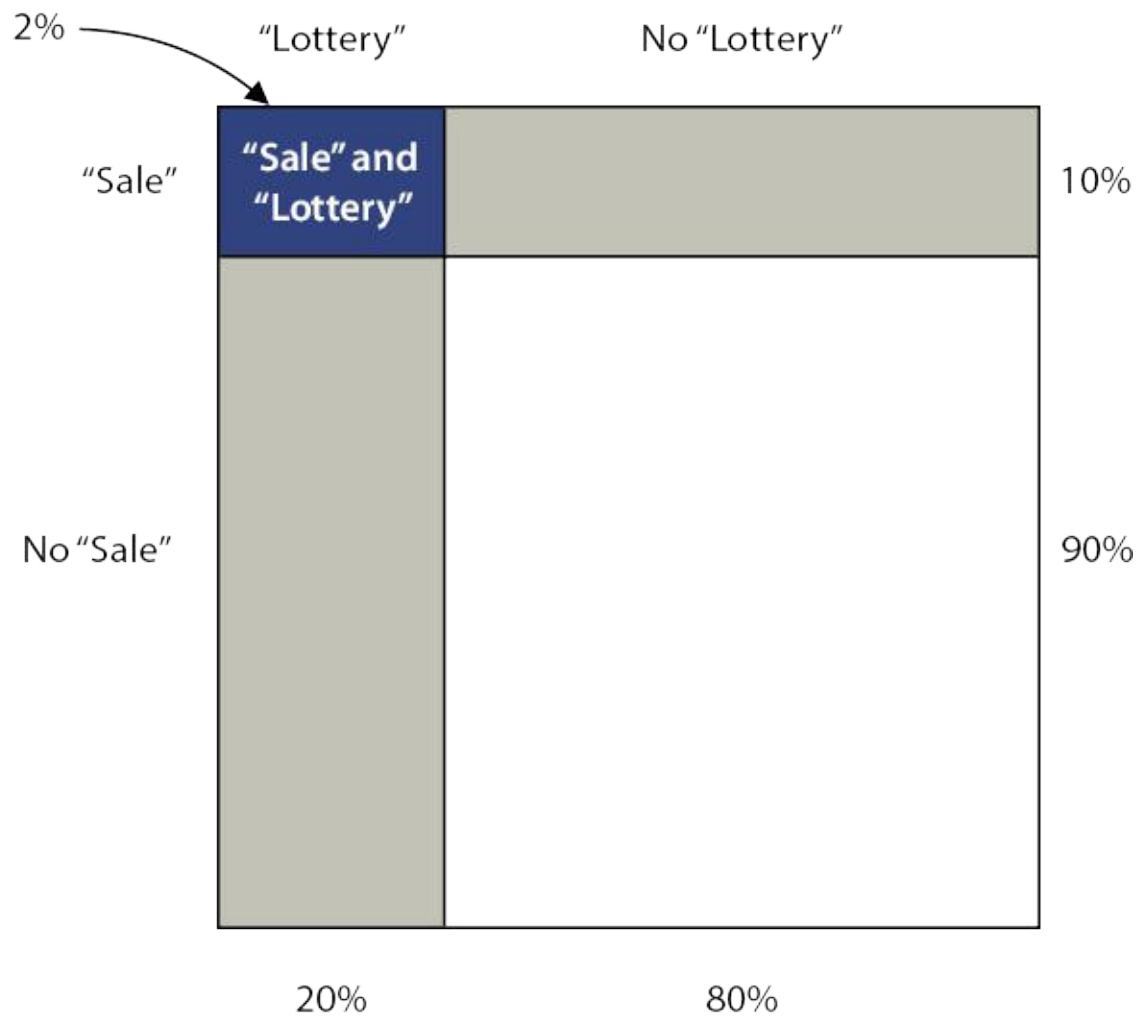
giả định ngây thơ Các từ xuất hiện trong email hoàn toàn độc lập với nhau. Nói cách khác, sự xuất hiện của một từ cụ thể trong email không hề ảnh hưởng đến sự xuất hiện của một từ khác.

Rất có thể, giả định ngây thơ này là không đúng sự thật. Sự xuất hiện của một từ đôi khi có thể ảnh hưởng lớn đến sự xuất hiện của một từ khác. Ví dụ: nếu một email có chứa từ *muối* thì từ *hạt tiêu* có nhiều khả năng xuất hiện trong email này hơn, vì nhiều khi chúng đi cùng nhau. Đây là lý do tại sao giả định của chúng tôi là ngây thơ. Tuy nhiên, hóa ra giả định này hoạt động tốt trong thực tế và nó đơn giản hóa toán học của chúng ta rất nhiều. Nó được gọi là quy tắc tích của xác suất và được minh họa trong hình 8.11.

Bây giờ chúng tôi đã ước tính xác suất, chúng tôi tiến hành tìm số lượng email spam và email ham dự kiến có chứa các từ *xổ số* và *bán hàng*.

- Vì có 20 email spam và xác suất một email spam chứa cả hai từ là 0,45 nên số email spam chứa cả hai từ dự kiến là $20 \cdot 0,225 = 4,5$.

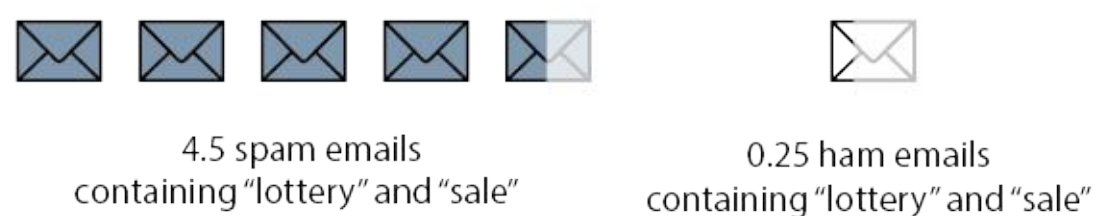
- Tương tự, có 80 email ham và xác suất một email ham chứa cả hai từ là 0,00325, do đó số email ham chứa cả hai từ dự kiến là $80 \cdot 0,00325 = 0,25$.



Hình 8.11 Giả sử 20% email chứa từ *xổ số* và 10% email chứa từ *bán*. Chúng tôi đưa ra giả định ngây thơ rằng hai từ này độc lập với nhau. Theo giả định này, tỷ lệ phần trăm email chứa cả hai từ có thể được ước tính là 2%, cụ thể là tích của 20% và 10%.

Các tính toán trước đây ngụ ý rằng nếu chúng tôi giới hạn tập dữ liệu của mình chỉ ở những email chứa cả từ *xổ số* và *bán hàng*, thì chúng tôi dự đoán 4,5 trong số đó là thư rác và 0,25 là thư rác. Vì vậy, nếu chúng ta chọn ngẫu nhiên một trong số đó, xác suất để chúng ta chọn một thư là thư rác là bao nhiêu? Điều này có vẻ khó hơn với số không nguyên so với số nguyên, nhưng nếu chúng ta nhìn vào hình 8.12, điều này có thể rõ ràng hơn. Chúng tôi có 4,5 email spam và 0,25 email ham (chính xác là 1/4 số email). Nếu chúng ta ném phi tiêu và nó rơi vào một trong các email, xác suất nó rơi vào một email spam là bao nhiêu? Chà, tổng số email (hoặc tổng diện tích, nếu bạn muốn tưởng tượng theo cách đó) là $4,5 + 0,25 = 4,75$. Vì 4,5 là thư rác nên xác suất phi tiêu rơi trúng thư rác là $4,5/4,75 = 0,9474$. Điều này có nghĩa là một email có

các từ *xổ số* và *bán hàng* có khả năng là thư rác là 94,74%.Đó là khá cao!



Hình 8.12 Chúng tôi có 4,5 email spam và 0,25 email ham.Chúng tôi ném phi tiêu và nó trúng một trong các email.Xác suất nó trúng một email spam là bao nhiêu?Câu trả lời là 94,74%.

Những gì chúng tôi đã làm ở đây, sử dụng xác suất, là sử dụng định lý Bayes, ngoại trừ các sự kiện

- $E = \text{xổ số} \cap \text{bán}$ $F =$
- *thư rác*

để có được công thức

$$P(\text{spam} | \text{lottery} \cap \text{sale}) = \frac{P(\text{lottery} \cap \text{sale} | \text{spam}) \cdot P(\text{spam})}{P(\text{lottery} \cap \text{sale} | \text{spam}) \cdot P(\text{spam}) + P(\text{lottery} \cap \text{sale} | \text{ham}) \cdot P(\text{ham})}$$

Sau đó, chúng tôi (ngây thơ) giả định rằng sự xuất hiện của các từ *xổ số* và *bán hàng* là độc lập với các email spam (và ham), để có được hai công thức sau:

$$P(\text{lottery} \cap \text{sale} | \text{spam}) = P(\text{lottery} | \text{spam}) \cdot P(\text{sale} | \text{spam})$$

$$P(\text{lottery} \cap \text{sale} | \text{ham}) = P(\text{lottery} | \text{ham}) \cdot P(\text{sale} | \text{ham})$$

Cắm chúng vào công thức trước, chúng ta nhận được

$$\begin{aligned} & P(\text{spam} | \text{lottery} \cap \text{sale}) \\ &= \frac{P(\text{lottery} | \text{spam}) \cdot P(\text{sale} | \text{spam}) \cdot P(\text{spam})}{P(\text{lottery} | \text{spam}) \cdot P(\text{sale} | \text{spam}) \cdot P(\text{spam}) + P(\text{lottery} | \text{ham}) \cdot P(\text{sale} | \text{ham}) \cdot P(\text{ham})} \end{aligned}$$

Cuối cùng, cắm vào các giá trị sau:

- $P(\text{lottery} | \text{spam}) = \frac{3}{4}$
- $P(\text{sale} | \text{spam}) = \frac{3}{10}$
- $P(\text{spam}) = \frac{1}{5}$
- $P(\text{lottery} | \text{ham}) = \frac{1}{16}$
- $P(\text{sale} | \text{ham}) = \frac{1}{20}$
- $P(\text{ham}) = \frac{4}{5}$

we get

$$P(\text{spam} | \text{lottery} \cap \text{sale}) = \frac{\frac{3}{4} \cdot \frac{3}{10} \cdot \frac{1}{5}}{\frac{3}{4} \cdot \frac{3}{10} \cdot \frac{1}{5} + \frac{1}{16} \cdot \frac{1}{20} \cdot \frac{4}{5}} = 0.9474$$

Nhiều hơn hai từ thì sao?

Trong trường hợp chung, email có n từ $x_1, x_2, \dots, x_n$. Định lý Bayes phát biểu rằng xác suất một email là thư rác vì nó chứa các từ $x_1, x_2, \dots, x_n$ là

$$P(\text{spam} | x_1, x_2, \dots, x_n) = \frac{P(x_1, x_2, \dots, x_n | \text{spam}) P(\text{spam})}{P(x_1, x_2, \dots, x_n | \text{spam}) P(\text{spam}) + P(x_1, x_2, \dots, x_n | \text{ham}) P(\text{ham})}$$

Trong phương trình trước, chúng ta đã loại bỏ dấu giao nhau và thay thế bằng dấu phẩy. Giả định ngây thơ là sự xuất hiện của tất cả những từ này là độc lập. Vì thế,

$$P(x_1, x_2, \dots, x_n | \text{spam}) = P(x_1 | \text{spam}) P(x_2 | \text{spam}) \dots P(x_n | \text{spam})$$

Và

$$P(x_1, x_2, \dots, x_n | \text{ham}) = P(x_1 | \text{ham}) P(x_2 | \text{ham}) \dots P(x_n | \text{ham}).$$

Kết hợp ba phương trình cuối cùng, chúng ta có được

$$\begin{aligned} & P(\text{spam} | x_1, x_2, \dots, x_n) \\ &= \frac{P(x_1 | \text{spam}) P(x_2 | \text{spam}) \dots P(x_n | \text{spam}) P(\text{spam})}{P(x_1 | \text{spam}) P(x_2 | \text{spam}) \dots P(x_n | \text{spam}) P(\text{spam}) + P(x_1 | \text{ham}) P(x_2 | \text{ham}) \dots P(x_n | \text{ham}) P(\text{ham})} \end{aligned}$$

Mỗi số lượng ở phía bên phải này có thể dễ dàng ước tính theo tỷ lệ giữa số lượng email. Ví dụ: $P(x_i | spam)$ là tỷ lệ giữa

số lượng email rác có chứa từ x_i và tổng số lượng email rác.

Một ví dụ nhỏ, giả sử email chứa các từ *xổ số*, *bán hàng* và *mẹ*. Chúng tôi kiểm tra từ *mẹ* và nhận thấy rằng nó chỉ xuất hiện ở một trong số 20 email spam và 10 trong số 80 email ham. Do đó, $P('mom' | thư rác) = 1/20$ và $P('mom' | ham) = 1/8$. Sử dụng cùng xác suất cho các từ *xổ số* và *bán hàng* như trong phần trước, chúng ta nhận được kết quả sau:

$$\begin{aligned} & P(spam | 'lottery', 'sale', 'mom') \\ &= \frac{P('lottery' | spam)P('sale' | spam)P('mom' | spam)P(spam)}{P('lottery' | spam)P('sale' | spam)P('mom' | spam)P(spam) + P('lottery' | ham)P('sale' | ham)P('mom' | ham)P(ham)} \\ &= \frac{\frac{3}{4} \cdot \frac{3}{10} \cdot \frac{1}{20} \cdot \frac{1}{5}}{\frac{3}{4} \cdot \frac{3}{10} \cdot \frac{1}{20} \cdot \frac{1}{5} + \frac{1}{16} \cdot \frac{1}{20} \cdot \frac{1}{8} \cdot \frac{4}{5}} = 0.8780 \end{aligned}$$

Lưu ý rằng việc thêm từ *mẹ* vào phương trình đã giảm xác suất spam từ 94,74% xuống 87,80%, điều này có ý nghĩa vì từ này có nhiều khả năng xuất hiện trong email ham hơn là trong email spam.

Xây dựng mô hình phát hiện thư rác bằng dữ liệu thực

Bây giờ chúng ta đã phát triển thuật toán, hãy xắn tay áo lên và viết mã cho thuật toán Bayes ngây thơ. Một số gói như Scikit-Learn có cách triển khai tuyệt vời cho thuật toán này và tôi khuyến khích bạn xem xét chúng. Tuy nhiên, chúng tôi sẽ mã hóa nó bằng tay. Tập dữ liệu chúng tôi sử dụng là từ Kaggle và để có liên kết tải xuống, vui lòng kiểm tra tài nguyên cho chương này trong phụ lục C. Đây là mã cho phần này:

- **Sổ tay:** Coding_naive_Bayes.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_8_Naive_Ba
- **Tập dữ liệu:** email.csv

Trong ví dụ này, chúng tôi sẽ giới thiệu một gói hữu ích để xử lý các tập dữ liệu lớn có tên là Pandas (để tìm hiểu thêm về gói này, vui lòng xem phần “Sử dụng Pandas

để tải tập dữ liệu” ở chương 13). Đối tượng chính được sử dụng để lưu trữ tập dữ liệu trong pandas là DataFrame. Để tải dữ liệu của chúng ta vào Pandas DataFrame, chúng ta sử dụng lệnh sau:

```
import pandas
emails = pandas.read_csv('emails.csv')
```

Trong bảng 8.1, bạn có thể thấy năm hàng đầu tiên của tập dữ liệu.

Tập dữ liệu này có hai cột.Cột đầu tiên là văn bản của email (cùng với dòng chủ đề), ở định dạng chuỗi.Cột thứ hai cho chúng ta biết email đó là thư rác (1) hay ham (0).Đầu tiên chúng ta cần thực hiện một số bước tiền xử lý dữ liệu.

Bảng 8.1 Năm hàng đầu tiên của tập dữ liệu email của chúng tôi.Cột Văn bản hiển thị văn bản trong mỗi email và cột Thư rác hiển thị số 1 nếu email là thư rác và số 0 nếu email đó là thư rác.Lưu ý rằng năm email đầu tiên đều là thư rác.

Văn bản	Thư rác
Chủ đề: một cách tự nhiên không thể cưỡng lại được công ty của bạn...	1
Chủ đề: tay súng giao dịch chúng khoán fanny tôi...	1
Chủ đề: những ngôi nhà mới khó tin được thực hiện dễ dàng ...	1
Chủ đề: In 4 màu yêu cầu đặc biệt thêm...	1
Tiêu đề: không có tiền, lấy đĩa CD phần mềm...	1

Tiền xử lý dữ liệu

Hãy bắt đầu bằng cách chuyển chuỗi văn bản thành danh sách các từ.Chúng tôi thực hiện việc này bằng cách sử dụng hàm sau, hàm này sử dụng hàm để biến tất cả các từ thành chữ thường và chức năng chuyển các từ thành danh sách.Chúng tôi chỉ kiểm tra mỗi từ có xuất hiện trong email hay không, bất kể nó xuất hiện bao nhiêu lần, vì vậy chúng tôi biến nó thành một bộ rồi thành một danh sách.

```
def process_email(text):
    text = text.lower()
    return list(set(text.split()))
```

Bây giờ chúng ta sử dụng hàm để áp dụng thay đổi này cho toàn bộ cột.Chúng tôi gọi cột mới .

```
emails['words']
```

```
emails['words'] = emails['text'].apply(process_email)
```

Th Năm hàng đầu tiên của tập dữ liệu email đã sửa đổi được hiển thị trong bảng 8.2 .

Bảng 8.2 Tập dữ liệu email với một cột mới có tên Từ, chứa danh sách các từ trong email (không lặp lại) và dòng chủ đề

Văn bản	Thư rác	Từ
Chủ đề: một cách tự nhiên không thể cưỡng lại được công ty của bạn...	1	[cho phép bạn, tất cả, làm, nhưng, liệt kê, là, thông tin,...
Chủ đề: tay súng giao dịch chứng khoán fanny tôi...	1	[không, như, duane, giao dịch, libretto, Đợtde...
Chủ đề: những ngôi nhà mới khó tin được thực hiện dễ dàng ...	1	[im, has, \$, take, foward, all, limit, subj...
Chủ đề: In 4 màu yêu cầu đặc biệt thêm...	1	[màu, azusa, pdf, có thể in được, 8102, chủ đề:,...
Tiêu đề: không có tiền, lấy đĩa CD phần mềm...	1	[có được, không, có, tất cả, làm, chủ đề:, được, bởi, tôi...

Tìm kiếm các linh mục

Trước tiên, hãy tìm xác suất một email là thư rác (trước đó).Để làm điều này, chúng tôi tính toán số lượng email là thư rác và chia cho tổng số email.Lưu ý rằng số lượng email là thư rác là tổng số mục trong cột Thư rác.Dòng sau sẽ thực hiện công việc:

```
sum(emails['spam'])/len(emails)
0.2388268156424581
```

Chúng tôi suy luận rằng xác suất trước đó của email là thư rác là khoảng 0,24.Đây là xác suất email bị spam nếu chúng ta không biết gì về email đó. Tương tự như vậy, xác suất trước đó để một email bị lỗi là khoảng 0,76.

Tìm hậu thế theo định lý Bayes

Chúng ta cần tìm xác suất các email spam (và ham) chứa một từ nhất định.Chúng tôi làm điều này cho tất cả các từ cùng một lúc.Hàm sau tạo ra một từ điển có tên model, ghi lại từng từ, cùng với số lần xuất hiện của từ đó trong email spam và trong email ham:

```
model = {}
```

```
for index, email in emails.iterrows():
    for word in email['words']:
        if word not in model:
            model[word] = {'spam': 1, 'ham': 1}
        if word in model:
            if email['spam']:
                model[word]['spam'] += 1
            else:
                model[word]['ham'] += 1
```

Lưu ý rằng số lượng được khởi tạo ở mức 1, vì vậy trên thực tế, chúng tôi đang ghi lại một lần xuất hiện email nữa là thư rác và vi phạm. Chúng tôi sử dụng mẹo nhỏ này để tránh số đếm bằng 0, vì chúng tôi không muốn vô tình chia cho 0. Bây giờ chúng ta hãy kiểm tra một số hàng của từ điển như sau:

```
model['lottery']
{'ham': 1, 'spam': 9}

model['sale']
{'ham': 42, 'spam': 39}
```

Điều này có nghĩa là từ *xổ số* xuất hiện trong 1 email ham và 9 email spam, trong khi từ *bán* xuất hiện trong 42 email ham và 39 email spam. Mặc dù từ điển này không chứa bất kỳ xác suất nào, nhưng chúng có thể được suy ra bằng cách chia mục đầu tiên cho tổng của cả hai mục. Vì vậy, nếu một email có chứa từ

xổ số, xác suất nó là thư rác là $\frac{9}{9+1} = 0.9$, và nếu nó chứa từ *bán hàng*, xác suất nó là thư rác là $\frac{39}{39+42} = 0.48$.

Triển khai thuật toán Bayes ngây thơ

Đầu vào của thuật toán là email. Nó xem xét tất cả các từ trong email và với mỗi từ, nó tính toán xác suất mà một email spam chứa nó và một email ham chứa nó. Những xác suất này được tính toán bằng cách sử dụng từ điển mà chúng ta đã xác định ở phần trước. Sau đó, chúng tôi nhân các xác suất này (giả định ngây thơ) và áp dụng định lý Bayes để tìm xác suất một email là thư rác vì nó có chứa các từ trên email cụ thể này. Mã để đưa ra dự đoán bằng mô hình này như sau:

```
def predict_naive_bayes(email):
    total = len(emails)
    num_spam = sum(emails['spam'])
    num_ham = total - num_spam
    email = email.lower()
    words = set(email.split())
    spams = [1.0]
```

```

hams = [1.0]
for word in words:
    if word in model:
        spams.append(model[word]['spam']/num_spam*total)
        hams.append(model[word]['ham']/num_ham*total)
prod_spams = np.long(np.prod(spams)*num_spam)
prod_hams = np.long(np.prod(hams)*num_ham)
return prod_spams/(prod_spams + prod_hams)

```

□ Tính tổng số email, email spam và email ham □ Xử lý từng email bằng cách biến nó thành danh sách các từ viết thường □ Đối với mỗi từ, tính xác suất có điều kiện để một email chứa từ đó là spam (hoặc ham), theo tỷ lệ □ Nhân tất cả các xác suất trước đó với xác suất trước đó của email là thư rác và gọi đây là prod_spams. Thực hiện quy trình tương tự cho prod_hams. □ Chuẩn hóa hai xác suất này để cộng chúng thành một (sử dụng định lý Bayes) và trả về kết quả

Bạn có thể lưu ý rằng trong đoạn mã trước, chúng tôi đã sử dụng một cách hack nhỏ khác. Mọi xác suất được nhân với tổng số email trong tập dữ liệu. Điều này sẽ không ảnh hưởng đến tính toán của chúng ta vì thừa số này xuất hiện ở tử số và mẫu số. Tuy nhiên, nó đảm bảo rằng tích xác suất của chúng tôi không quá nhỏ để Python xử lý.

Bây giờ chúng ta đã xây dựng xong mô hình, hãy kiểm tra nó bằng cách đưa ra dự đoán trên một số email như sau:

```

predict_naive_bayes('Hi mom how are you')
0.12554358867163865

predict_naive_bayes('meet me at the lobby of the hotel at nine am')
0.00006964603508395

predict_naive_bayes('buy cheap lottery easy money now')
0.9999734722659664

predict_naive_bayes('asdfgh')
0.2388268156424581

```

Nó dường như hoạt động tốt. Những email như 'xin chào mẹ, bạn khỏe không' có xác suất bị spam thấp (khoảng 0,12) và những email như 'mua xổ số giá rẻ để kiếm tiền ngay bây giờ' có xác suất rất cao (trên 0,99) là spam. Lưu ý rằng email cuối cùng, không chứa bất kỳ từ nào trong từ điển, có xác suất là 0,2388, chính xác là email trước đó.

Công việc tiếp theo

Đây là một triển khai nhanh chóng của thuật toán Bayes ngây thơ. Nhưng đối với tập dữ liệu lớn hơn và email lớn hơn, chúng ta nên sử dụng gói. Các gói như Scikit-Learn

cung cấp các triển khai tuyệt vời của thuật toán Bayes đơn giản, với nhiều tham số để sử dụng. Khám phá gói này và các gói khác, đồng thời sử dụng thuật toán Bayes đơn giản trên tất cả các loại tập dữ liệu!

Tóm tắt

- Định lý Bayes là một kỹ thuật được sử dụng rộng rãi trong xác suất, thống kê và học máy. Định lý Bayes bao gồm việc tính xác suất hậu nghiệm, dựa trên xác suất trước đó và một sự kiện. Xác suất trước là phép tính cơ bản của xác suất, với rất ít thông tin. Định lý Bayes sử dụng sự kiện này để đưa ra ước tính tốt hơn nhiều về xác suất được đề cập. Thuật toán Bayes ngây thơ được sử dụng khi người ta muốn kết hợp xác suất trước đó với một số sự kiện. Từ *ngây thơ* xuất phát từ thực tế là chúng ta đang đưa ra một giả định ngây thơ, cụ thể là các sự kiện được đề cập đều độc lập.
-
-
-

Bài tập

BÀI TẬP 8.1

Với mỗi cặp biến cố A và B, hãy xác định xem chúng độc lập hay phụ thuộc. Đối với (a) đến (d), hãy đưa ra sự chứng minh bằng toán học. Đối với (e) và (f) đưa ra lời giải thích bằng lời nói.

Ném ba đồng xu công bằng:

Một. A: Người đầu tiên rơi vào đầu. B: Con thứ ba rơi vào đuôi. b. A: Người đầu tiên rơi vào đầu. B: Có một số lẻ mặt ngửa trong ba lần ném.

Tung hai con xúc xắc:

c. A: Cái đầu tiên hiển thị số 1. B: Cái thứ hai hiển thị số 2.

d. A: Cái đầu tiên hiển thị số 3. B: Cái thứ hai hiển thị giá trị cao hơn cái đầu tiên.

Đối với những điều sau đây, hãy đưa ra lời biện minh bằng lời nói. Giả sử rằng đối với bài toán này, chúng ta đang sống ở một nơi có các mùa.

đ. A: Ngoài trời đang mưa. B: Hôm nay là thứ Hai.

f. A: Ngoài trời đang mưa. B: Bây giờ là tháng Sáu.

BÀI TẬP 8.2

Có một văn phòng nơi chúng tôi phải đến thường xuyên để làm một số giấy tờ. Văn phòng này có hai nhân viên, Aisha và Beto. Chúng tôi biết rằng Aisha làm việc ở đó ba ngày một tuần và Beto làm việc hai ngày còn lại. Tuy nhiên, lịch trình thay đổi hàng tuần nên chúng ta không bao giờ biết Aisha ở đó ba ngày và Beto ở đó hai ngày.

Một. Nếu chúng ta ngẫu nhiên đến văn phòng vào một ngày nào đó thì xác suất để Aisha là nhân viên bán hàng là bao nhiêu?

Chúng tôi nhìn từ bên ngoài và nhận thấy người bán hàng đang mặc một chiếc áo len màu đỏ, mặc dù chúng tôi không thể biết người bán hàng là ai. Chúng tôi đã đến văn phòng đó rất nhiều nên biết rằng Beto có xu hướng mặc đồ màu đỏ thường xuyên hơn Aisha. Trên thực tế, Aisha mặc màu đỏ một trong ba ngày (một phần ba thời gian), và Beto mặc màu đỏ một trong hai ngày (một nửa thời gian).

b. Xác suất để Aisha là nhân viên bán hàng biết rằng hôm nay nhân viên đó mặc đồ màu đỏ là bao nhiêu?

BÀI TẬP 8.3

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán
Bệnh nhân 1		X	X	X	Đau ốm
Bệnh nhân 2	X	X		X	Đau ốm
Bệnh nhân 3	X		X	X	Đau ốm
Bệnh nhân 4	X	X	X		Đau ốm
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

Mục tiêu của bài tập này là xây dựng một mô hình Bayes đơn giản để dự đoán chẩn đoán từ các triệu chứng.Sử dụng thuật toán Bayes ngây thơ để tìm các xác suất sau:

lưu ý Đối với các câu hỏi sau, chúng tôi hoàn toàn không biết các triệu chứng không được đề cập.Ví dụ: nếu chúng ta biết rằng bệnh nhân bị ho nhưng không nói gì về việc họ bị sốt thì điều đó không có nghĩa là bệnh nhân không bị sốt.

Một. Xác suất để một bệnh nhân bị bệnh khi bệnh nhân đó bị ho b. Xác suất để một bệnh nhân bị ốm nếu bệnh nhân đó không mệt c. Xác suất một bệnh nhân bị bệnh khi bệnh nhân đó bị ho và sốt d. Xác suất một bệnh nhân bị bệnh khi bệnh nhân đó ho và sốt nhưng không khó thở

9 Phân chia dữ liệu bằng cách đặt câu hỏi: Cây quyết định

Trong chương này

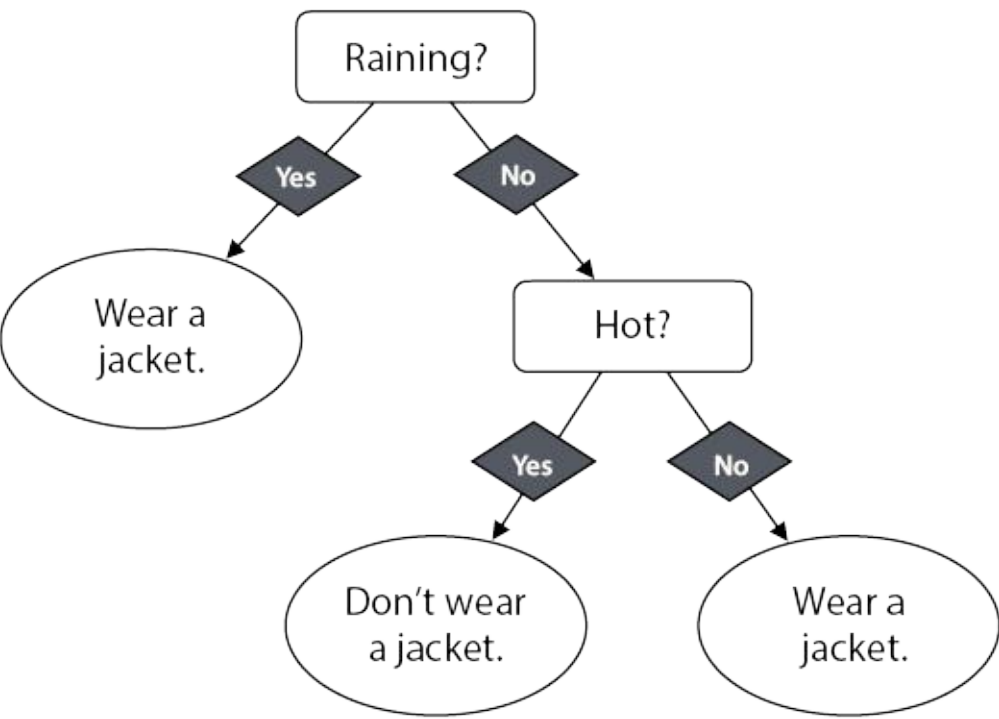
- cây quyết định sử dụng cây quyết định để phân loại và hồi quy là gì xây dựng hệ thống đề xuất
- ứng dụng sử dụng độ chính xác thông tin của người dùng, chỉ số Gini và entropy cũng như vai trò của chúng trong việc xây dựng cây quyết định bằng Scikit-Learn để huấn luyện cây quyết định trên tập dữ liệu tuyến sinh đại học
-



Trong chương này, chúng tôi đề cập đến cây quyết định. Cây quyết định là mô hình phân loại và hồi quy mạnh mẽ, cũng cung cấp cho chúng ta rất nhiều thông tin về tập dữ liệu của chúng ta. Cũng giống như các mô hình trước đây mà chúng ta đã học trong cuốn sách này, cây quyết định được huấn luyện bằng dữ liệu được gắn nhãn, trong đó các nhãn mà chúng ta muốn dự đoán có thể là lớp (để phân loại) hoặc giá trị (để hồi quy). Trong hầu hết chương này, chúng tôi tập trung vào cây quyết định để phân loại, nhưng ở gần cuối chương, chúng tôi mô tả cây quyết định cho hồi quy. Tuy nhiên, cấu trúc và quá trình huấn luyện của cả hai loại cây đều giống nhau. Trong chương này, chúng tôi phát triển một số trường hợp sử dụng, bao gồm hệ thống đề xuất ứng dụng và mô hình dự đoán tuyến sinh tại một trường đại học.

Cây quyết định tuân theo một quy trình trực quan để đưa ra dự đoán—một quy trình rất giống với cách suy luận của con người. Hãy xem xét tình huống sau: chúng ta muốn quyết định xem hôm nay có nên mặc áo khoác hay không. Quá trình ra quyết định trông như thế nào?

Chúng ta có thể nhìn ra ngoài và kiểm tra xem trời có mưa không. Nếu trời mưa thì chúng ta nhất định phải mặc áo khoác. Nếu không, có lẽ chúng tôi sẽ kiểm tra nhiệt độ. Trời nóng thì không mặc áo khoác, trời lạnh thì mặc áo khoác. Trong hình 9.1, chúng ta có thể thấy biểu đồ của quá trình ra quyết định này, trong đó các quyết định được thực hiện bằng cách duyệt cây từ trên xuống dưới.



Hình 9.1 Cây quyết định dùng để quyết định xem chúng ta có muốn mặc áo khoác hay không vào một ngày nhất định. Chúng ta đưa ra quyết định bằng cách duyệt cây xuống và lấy nhánh tương ứng với mỗi câu trả lời đúng.

Quá trình ra quyết định của chúng ta trông giống như một cái cây, ngoại trừ việc nó bị lộn ngược. Cây được hình thành từ các đỉnh, được gọi là *nút* và các cạnh. Ở trên cùng, chúng ta có thể thấy *nút gốc*, từ đó phát ra hai nhánh. Mỗi nút có hai hoặc không có nhánh (cạnh) nào phát ra từ chúng và vì lý do này, chúng tôi gọi nó là *cây nhị phân*. Các nút có hai nhánh phát ra từ chúng được gọi là *nút quyết định* và các nút không có nhánh phát ra từ chúng được gọi là *nút lá* hoặc *lá*. Sự sắp xếp các nút, lá và cạnh này được gọi là cây quyết định. Cây cối là đối tượng tự nhiên trong khoa học máy tính vì máy tính chia mọi quy trình thành một chuỗi các phép toán nhị phân.

Cây quyết định đơn giản nhất có thể, được gọi là *gốc quyết định*, được hình thành bởi một nút quyết định duy nhất (nút gốc) và hai lá. Điều này thể hiện một câu hỏi có hoặc không, dựa vào đó chúng ta sẽ đưa ra quyết định ngay lập tức.

Độ sâu của cây quyết định là số cấp bên dưới nút gốc.

Một cách khác để đo nó là bằng độ dài của đường đi dài nhất từ nút gốc đến lá, trong đó đường đi được đo bằng số cạnh mà nó chứa. Cây trong hình 9.1 có độ sâu là 2. Cây quyết định có độ sâu là 1.

Dưới đây là bản tóm tắt các định nghĩa mà chúng ta đã học được cho đến nay:

cây quyết định Một mô hình học máy dựa trên các câu hỏi có hoặc không và được biểu thị bằng cây nhị phân. Cây có nút gốc, nút quyết định, nút lá và nhánh.

nút gốc Nút trên cùng của cây. Nó chứa câu hỏi có hoặc không đầu tiên. Để thuận tiện, chúng tôi gọi nó là *gốc*.

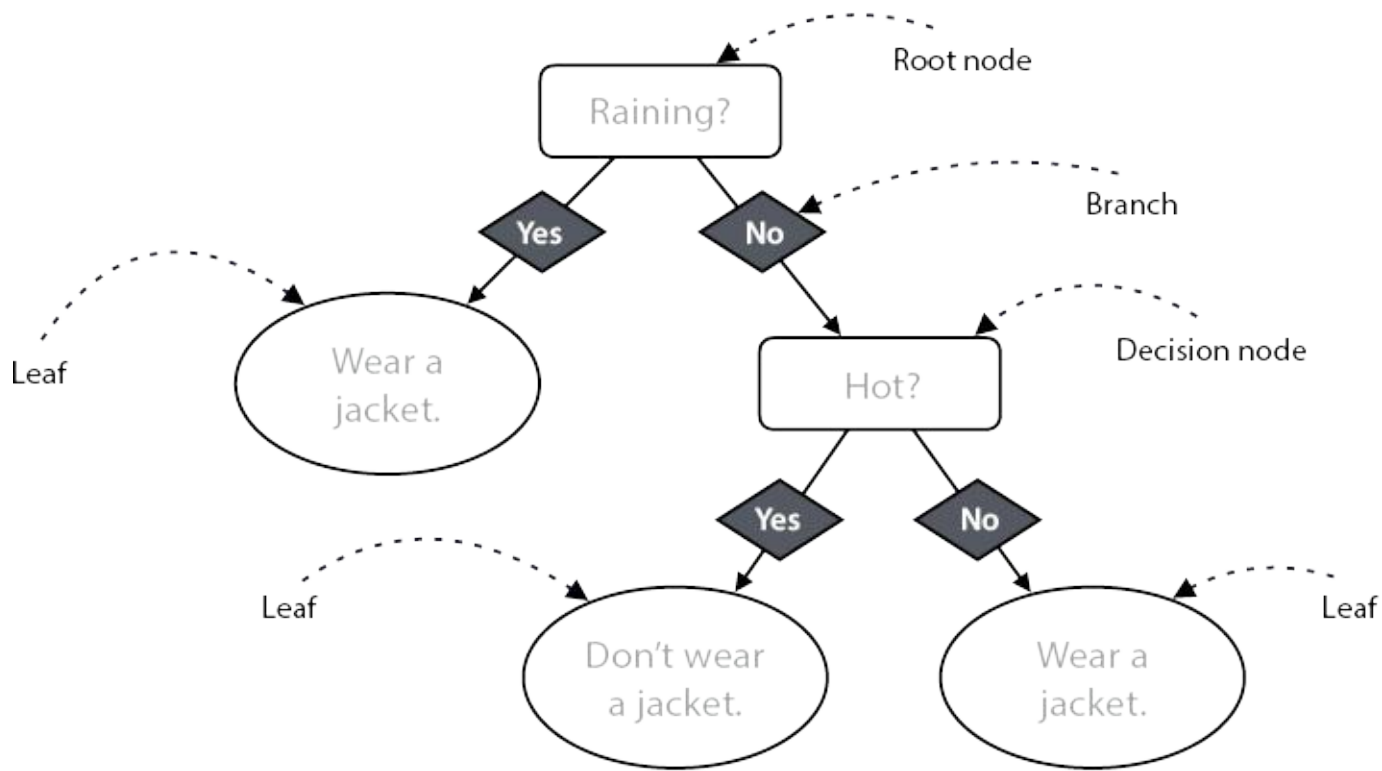
nút quyết định Mỗi câu hỏi có hoặc không trong mô hình của chúng tôi được biểu thị bằng một nút quyết định, với hai nhánh bắt nguồn từ nút đó (một nhánh cho câu trả lời “có” và một nhánh cho câu trả lời “không”).

nút lá Một nút không có nhánh nào phát ra từ nó. Chúng đại diện cho những quyết định mà chúng ta đưa ra sau khi duyệt cây. Để thuận tiện, chúng tôi gọi chúng là *lá*.

nhánh Hai cạnh phát ra từ mỗi nút quyết định, tương ứng với câu trả lời “có” và “không” cho câu hỏi trong nút. Trong chương này, theo quy ước, nhánh bên trái tương ứng với “có” và nhánh bên phải tương ứng với “không”.

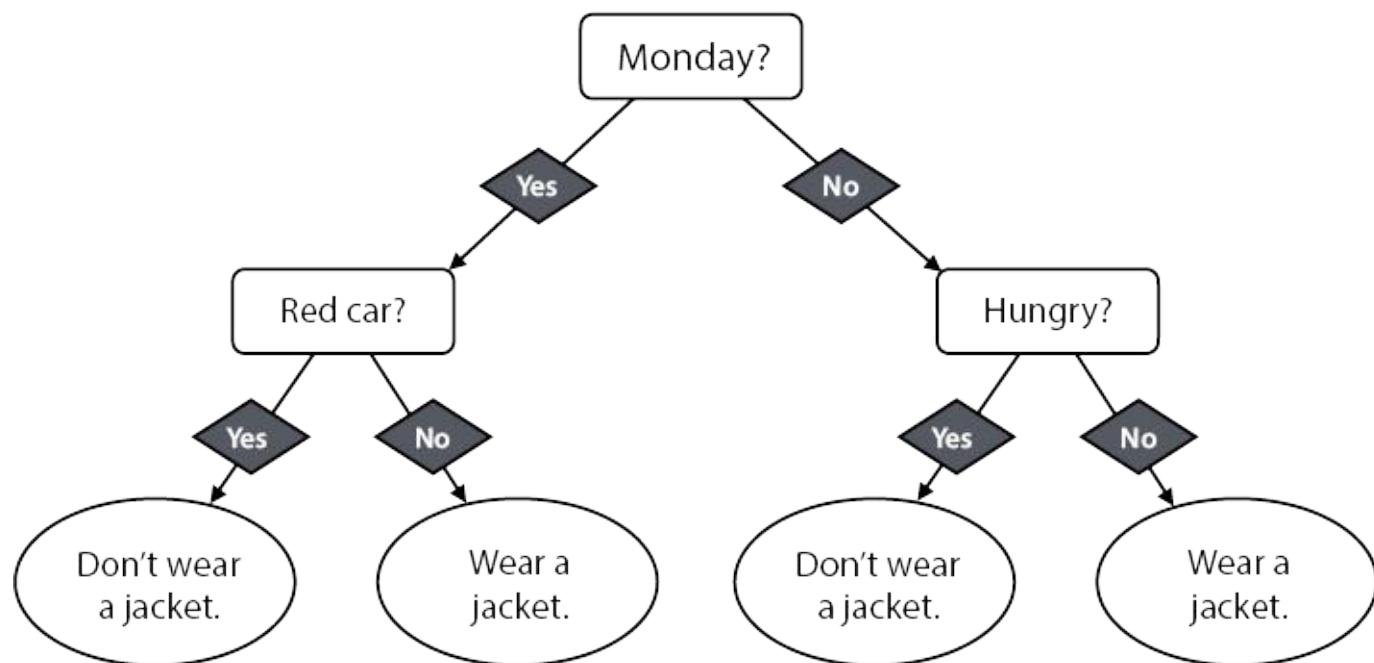
độ sâu Số cấp trong cây quyết định. Ngoài ra, đó là số nhánh trên đường đi dài nhất từ nút gốc đến nút lá.

Trong suốt chương này, các nút được vẽ dưới dạng hình chữ nhật với các cạnh tròn, các câu trả lời ở các nhánh là hình thoi và lá là hình bầu dục. Hình 9.2 cho thấy cây quyết định nhìn tổng thể như thế nào.



Hình 9.2 Cây quyết định thông thường có nút gốc, nút quyết định, nhánh và lá. Lưu ý rằng mỗi nút quyết định đều chứa một câu hỏi có hoặc không. Từ mỗi câu trả lời có thể, một nhánh sẽ xuất hiện, có thể dẫn đến một nút quyết định khác hoặc một lá. Cây này có độ sâu là 2, vì đường đi dài nhất từ lá đến gốc đi qua hai nhánh.

Chúng ta đã xây dựng cái cây này như thế nào? Tại sao đó là những câu hỏi chúng tôi hỏi? Chúng ta cũng có thể kiểm tra xem hôm nay có phải là thứ Hai không, chúng ta nhìn thấy một chiếc ô tô màu đỏ bên ngoài hay chúng ta đang đói và xây dựng cây quyết định sau:



Hình 9.3 Cây quyết định thứ hai (có thể không tốt bằng) mà chúng ta có thể sử dụng để quyết định xem chúng ta có muốn mặc áo khoác vào một ngày nhất định hay không

Chúng ta nghĩ cây nào tốt hơn khi quyết định mặc áo khoác hay không: cây 1 (hình 9.1) hay cây 2 (hình 9.3)?Chà, với tư cách là con người, chúng ta có đủ kinh nghiệm để nhận ra rằng cây 1 tốt hơn nhiều so với cây 2 trong quyết định này.Làm sao máy tính có thể biết được?Máy tính không có kinh nghiệm nhưng chúng có thứ tương tự, đó là dữ liệu.Nếu muốn suy nghĩ giống như một chiếc máy tính, chúng ta có thể xem xét tất cả các cây có thể, thử từng cây trong một khoảng thời gian—chẳng hạn như một năm—và so sánh xem chúng hoạt động tốt như thế nào bằng cách đếm xem chúng ta đã đưa ra quyết định đúng bao nhiêu lần khi sử dụng từng cây.Chúng ta hãy tưởng tượng rằng nếu sử dụng cây 1 thì hầu hết các ngày chúng ta đều đúng, trong khi nếu sử dụng cây 2, chúng ta có thể bị chết cứng vào một ngày lạnh giá mà không có áo khoác hoặc mặc áo khoác vào một ngày cực kỳ nóng.Tất cả những gì máy tính phải làm là xem xét tất cả các cây, thu thập dữ liệu và tìm ra cây nào tốt nhất, phải không?

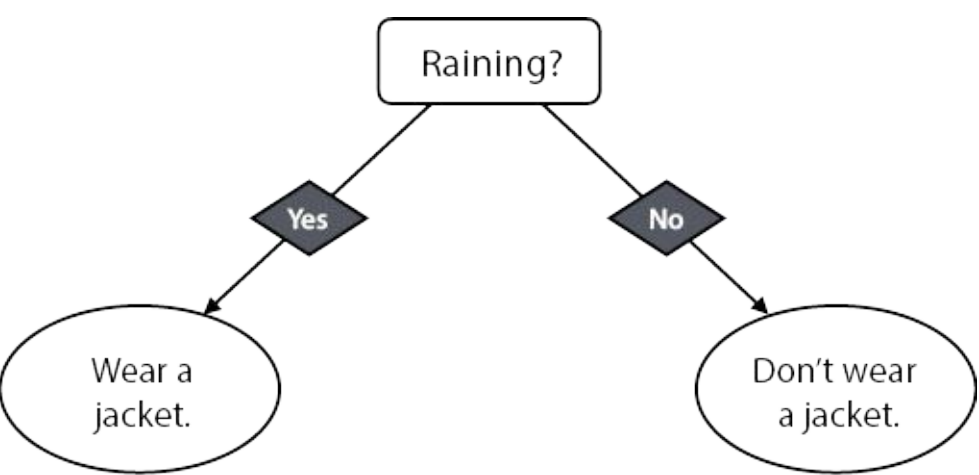
Hầu hết!Thật không may, ngay cả đối với máy tính, việc tìm kiếm trên tất cả các cây có thể để tìm ra cây hiệu quả nhất sẽ mất rất nhiều thời gian.Nhưng may mắn thay, chúng tôi có các thuật toán giúp việc tìm kiếm này nhanh hơn nhiều và do đó, chúng tôi có thể sử dụng cây quyết định cho nhiều ứng dụng tuyệt vời, bao gồm phát hiện thư rác, phân tích cảm xúc và chẩn đoán y tế.Trong chương này, chúng ta sẽ tìm hiểu một thuật toán để xây dựng cây quyết định tốt một cách nhanh chóng.Tóm lại, chúng ta xây dựng cây từng nút một, bắt đầu từ trên cùng.Để chọn đúng câu hỏi tương ứng với mỗi nút, chúng tôi xem xét tất cả các câu hỏi có thể hỏi và chọn câu hỏi đúng với số lần cao nhất.Quá trình diễn ra như sau:

CHỌN CÂU HỎI HAY ĐẦU TIÊN

Chúng ta cần chọn một câu hỏi đầu tiên hay cho phần gốc của cây. Câu hỏi hay sẽ giúp chúng ta quyết định có nên mặc áo khoác vào một ngày nhất định hay không? Ban đầu, nó có thể là bất cứ điều gì. Giả sử chúng ta đưa ra năm ứng cử viên cho câu hỏi đầu tiên:

- 1. Trời có mưa không?
- 2. Bên ngoài có lạnh không?
- 3. Tôi có đói không?
- 4. Bên ngoài có xe màu đỏ không?
- 5. Hôm nay là thứ Hai phải không?

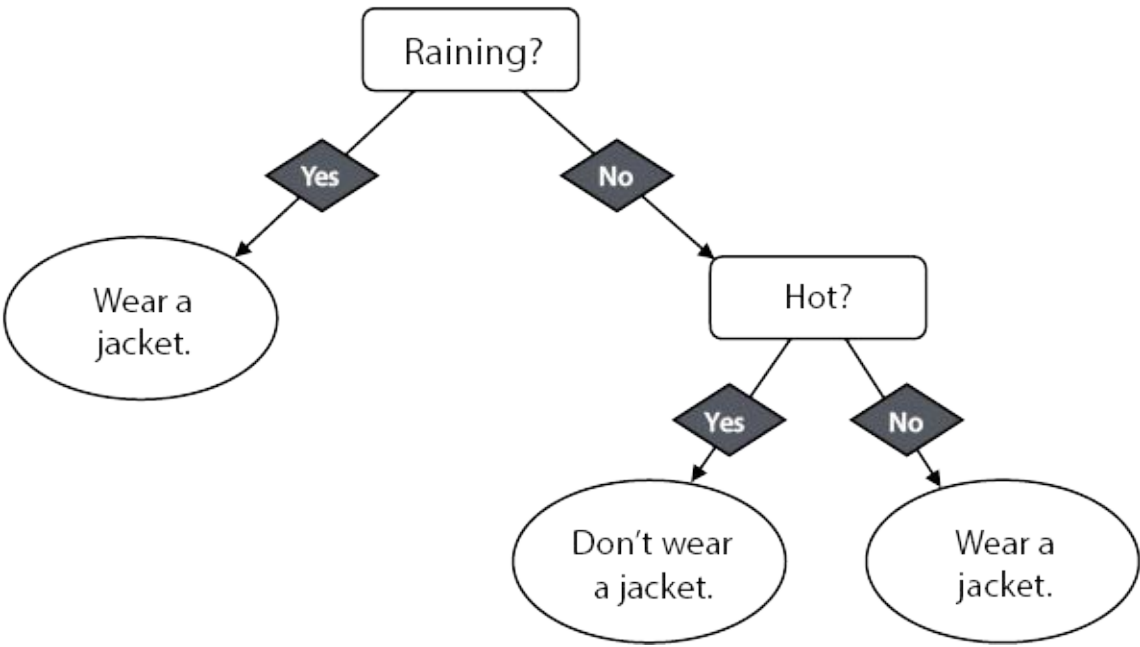
Trong số năm câu hỏi này, câu hỏi nào có vẻ là câu hỏi tốt nhất giúp chúng ta quyết định xem có nên mặc áo khoác hay không? Trực giác của chúng ta mách bảo rằng ba câu hỏi cuối cùng không giúp chúng ta quyết định được. Giả sử rằng theo kinh nghiệm, chúng tôi nhận thấy rằng trong số hai cái đầu tiên, cái đầu tiên hữu ích hơn. Chúng tôi sử dụng câu hỏi đó để bắt đầu xây dựng cây của mình. Cho đến nay, chúng ta có một cây quyết định đơn giản, hay một gốc quyết định, bao gồm câu hỏi duy nhất đó, như được minh họa trong Hình 9.4.



Hình 9.4 Một cây quyết định đơn giản (gốc quyết định) chỉ bao gồm câu hỏi “Trời có mưa không?” Nếu câu trả lời là có thì quyết định của chúng ta là mặc áo khoác.

Chúng ta có thể làm tốt hơn không? Hãy tưởng tượng rằng chúng ta bắt đầu nhận thấy rằng khi trời mưa, việc mặc áo khoác luôn là quyết định đúng đắn. Tuy nhiên, có những ngày trời không mưa và việc không mặc áo khoác không phải là quyết định đúng đắn. Đây chính là lúc câu hỏi số 2 giải cứu chúng ta. Chúng tôi sử dụng câu hỏi đó để giúp chúng tôi theo cách sau: sau khi chúng tôi kiểm tra xem trời không mưa, sau đó chúng tôi kiểm tra

hiệt độ, và nếu trời lạnh, chúng tôi quyết định mặc áo khoác.Điều này biến lá bên trái của cây thành một nút, với hai lá phát ra từ nó, như trong hình 9.5.



Hình 9.5 Một cây quyết định phức tạp hơn một chút so với cây trong hình 9.4, trong đó chúng ta đã chọn một lá và chia nó thành hai lá khác.Đây là cây giống như trong hình 9.1.

Bây giờ chúng ta có cây quyết định.Chúng ta có thể làm tốt hơn không?Có lẽ chúng ta có thể làm được nếu chúng ta thêm nhiều nút và lá vào cây của mình.Nhưng hiện tại, cái này hoạt động rất tốt.Trong ví dụ này, chúng tôi đưa ra quyết định bằng trực giác và kinh nghiệm của mình.Trong chương này, chúng ta tìm hiểu một thuật toán xây dựng những cây này chỉ dựa trên dữ liệu.

Nhiều câu hỏi có thể nảy sinh trong đầu bạn, chẳng hạn như sau:

1. Làm thế nào để bạn quyết định chính xác câu hỏi nào là tốt nhất để hỏi?
2. Liệu quá trình luôn chọn câu hỏi tốt nhất có thể có thực sự giúp chúng ta xây dựng được *cây quyết định* tốt nhất không?
3. Tại sao chúng ta không xây dựng tất cả các cây quyết định có thể và chọn ra cây quyết định tốt nhất từ đó?
4. Chúng ta có viết thuật toán này không?
5. Chúng ta có thể tìm cây quyết định ở đâu trong đời thực?
6. Chúng ta có thể thấy cây quyết định hoạt động như thế nào trong việc phân loại, nhưng chúng hoạt động như thế nào trong hồi quy?

Chương này trả lời tất cả những câu hỏi này, nhưng sau đây là một số câu trả lời nhanh:

1. Làm thế nào để bạn quyết định chính xác câu hỏi nào là tốt nhất để hỏi?

Chúng tôi có một số cách để làm điều này. Cách đơn giản nhất là sử dụng độ chính xác, nghĩa là: câu hỏi nào giúp tôi đúng thường xuyên hơn? Tuy nhiên, trong chương này, chúng ta cũng tìm hiểu các phương pháp khác, chẳng hạn như chỉ số Gini hoặc entropy.

2. Liệu quá trình luôn chọn câu hỏi tốt nhất có thể có thực sự giúp chúng ta xây dựng cây quyết định tốt nhất không?

Trên thực tế, quá trình này không đảm bảo rằng chúng ta sẽ có được cây tốt nhất có thể. Đây là cái mà chúng tôi gọi là *thuật toán tham lam*. Thuật toán tham lam hoạt động như sau: tại mọi thời điểm, thuật toán sẽ đưa ra nước đi tốt nhất có thể. Chúng có xu hướng hoạt động tốt, nhưng không phải lúc nào việc thực hiện bước đi tốt nhất có thể ở mỗi bước thời gian sẽ giúp bạn đạt được kết quả tổng thể tốt nhất. Có thể đôi khi việc đặt một câu hỏi yếu hơn sẽ nhóm dữ liệu của chúng ta theo cách mà cuối cùng chúng ta sẽ có được một cái cây tốt hơn. Tuy nhiên, các thuật toán xây dựng cây quyết định có xu hướng hoạt động rất tốt và nhanh chóng, vì vậy chúng ta sẽ chấp nhận điều này. Hãy xem các thuật toán mà chúng ta thấy trong chương này và cố gắng tìm ra cách cải thiện chúng bằng cách loại bỏ thuộc tính tham lam!

3. Thay vào đó, tại sao chúng ta không xây dựng tất cả các cây quyết định có thể có và chọn ra cây quyết định tốt nhất từ đó?

Số lượng cây quyết định có thể có là rất lớn, đặc biệt nếu tập dữ liệu của chúng tôi có nhiều tính năng. Đi qua tất cả chúng sẽ rất chậm. Ở đây, việc tìm từng nút chỉ yêu cầu tìm kiếm tuyến tính trên các đối tượng địa lý chứ không phải trên tất cả các cây có thể, điều này làm cho việc tìm kiếm nhanh hơn nhiều.

4. Chúng ta có mã hóa thuật toán này không?

Thuật toán này có thể được mã hóa bằng tay. Tuy nhiên, chúng ta sẽ thấy rằng vì nó là đệ quy nên việc mã hóa có thể hơi tẻ nhạt. Do đó, chúng tôi sẽ sử dụng gói hữu ích có tên Scikit-Learn để xây dựng cây quyết định bằng dữ liệu thực.

5. Chúng ta có thể tìm cây quyết định ở đâu trong đời thực?

Ở nhiều nơi! Chúng được sử dụng rộng rãi trong học máy, không chỉ vì chúng hoạt động rất tốt mà còn vì chúng cung cấp cho chúng ta rất nhiều thông tin về dữ liệu của chúng ta. Một số nơi mà cây quyết định được sử dụng là trong các hệ thống khuyến nghị (để giới thiệu video, phim, ứng dụng, sản phẩm nên mua, v.v.), trong phân loại thư rác (để quyết định xem một email có phải là thư rác hay không), trong phân tích tình cảm (để quyết định xem một câu là vui hay buồn) và trong sinh học (để quyết định xem một bệnh nhân có bị bệnh hay không hoặc giúp xác định các hệ thống phân cấp nhất định trong loài hoặc trong các loại bộ gen).

6. Chúng ta có thể thấy cây quyết định hoạt động như thế nào trong việc phân loại, nhưng chúng hoạt động như thế nào trong hồi quy?

Cây quyết định hồi quy trông giống hệt cây quyết định phân loại, ngoại trừ các lá. Trong cây quyết định phân loại, các lá có các lớp như có và không. Trong cây quyết định hồi quy, các lá có giá trị như 4, 8,2 hoặc -199. Dự đoán mà mô hình của chúng tôi đưa ra được đưa ra bởi chiếc lá mà chúng tôi đã đến khi đi ngang qua cây theo hướng đi xuống.

Trường hợp sử dụng đầu tiên mà chúng ta sẽ nghiên cứu trong chương này là một ứng dụng phổ biến trong học máy và một trong những ứng dụng yêu thích của tôi: hệ thống đề xuất.

Mã cho chương này có sẵn trong kho lưu trữ GitHub này:

https://github.com/luisguiserrano/manning/tree/master/Chapter_9_Decision_Trees.

Vấn đề: Chúng tôi cần giới thiệu ứng dụng cho người dùng dựa trên những gì họ có thể tải xuống

Hệ thống khuyến nghị là một trong những ứng dụng phổ biến và thú vị nhất trong học máy. Bạn có bao giờ thắc mắc cách Netflix đề xuất phim, YouTube đoán video bạn có thể xem hoặc Amazon hiển thị cho bạn những sản phẩm mà bạn có thể muốn mua không? Đây là tất cả các ví dụ về hệ thống khuyến nghị. Một cách đơn giản và thú vị để xem các vấn đề đề xuất là xem xét chúng như các vấn đề phân loại. Hãy bắt đầu với một ví dụ dễ hiểu: hệ thống đề xuất ứng dụng của riêng chúng tôi sử dụng cây quyết định.

Giả sử chúng tôi muốn xây dựng một hệ thống đề xuất cho người dùng nên tải xuống ứng dụng nào trong số các tùy chọn sau.Chúng tôi có ba ứng dụng sau trong cửa hàng của mình (hình 9.6):

- **Số lượng nguyên tử:** một ứng dụng đếm số lượng nguyên tử trong cơ thể bạn **Beehive**
- **Finder:** một ứng dụng lập bản đồ vị trí của bạn và tìm thấy những tổ ong gần nhất
- **Check Mate Mate:** một ứng dụng tìm kiếm người chơi cờ vua Úc



Hình 9.6 Ba ứng dụng chúng tôi đề xuất: Atom Count, một ứng dụng để đếm số lượng nguyên tử trong cơ thể bạn;Beehive Finder, một ứng dụng định vị các tổ ong gần vị trí của bạn nhất;và Check Mate Mate, một ứng dụng tìm kiếm người chơi cờ Úc trong khu vực của bạn

Dữ liệu đào tạo là một bảng chứa nền tảng mà người dùng sử dụng (iPhone hoặc Android), độ tuổi của họ và ứng dụng họ đã tải xuống (trong thực tế có nhiều nền tảng hơn, nhưng để đơn giản, chúng tôi sẽ cho rằng đây là hai tùy chọn duy nhất).Bảng của chúng tôi có sáu người, như thể hiện trong bảng 9.1.

Bảng 9.1 Tập dữ liệu về người dùng cửa hàng ứng dụng. Đối với mỗi khách hàng, chúng tôi ghi lại nền tảng, độ tuổi và ứng dụng họ đã tải xuống.

Nền tảng	Age	App
iPhone	15	Số nguyên tử
iPhone	25	Kiểm tra bạn đời
Android	32	Công cụ tìm tổ ong
iPhone	35	Kiểm tra bạn đời
Android	12	Số nguyên tử
Android	14	Số nguyên tử

Với bảng này, bạn sẽ giới thiệu ứng dụng nào cho mỗi khách hàng trong số ba khách hàng sau?

- **Khách hàng 1:** người dùng iPhone 13 tuổi **Khách**
- **hàng 2:** người dùng iPhone 28 tuổi **Khách hàng**
- **3:** người dùng Android 34 tuổi

Những gì chúng ta nên làm như sau:

Khách hàng 1: người dùng iPhone 13 tuổi. To this customer, we should recommend AtomCount, because it seems (looking at the three customers in their teens) that young people tend to download AtomCount.

Khách hàng 2: người dùng iPhone 28 tuổi. Đối với khách hàng này, chúng tôi nên giới thiệu Check Mate Mate, vì nhìn vào hai người dùng iPhone trong tập dữ liệu (25 và 35 tuổi), cả hai đều đã tải xuống Check Mate Mate.

Khách hàng 3: người dùng Android 34 tuổi. Đối với khách hàng này, chúng tôi nên giới thiệu Beehive Finder vì có một người dùng Android trong tập dữ liệu 32 tuổi và họ đã tải xuống Beehive Finder.

Tuy nhiên, đi từng khách hàng một có vẻ là một công việc tẻ nhạt. Tiếp theo, chúng ta sẽ xây dựng cây quyết định để chăm sóc tất cả khách hàng cùng một lúc.

Giải pháp: Xây dựng hệ thống đề xuất ứng dụng

Trong phần này, chúng ta xem cách xây dựng hệ thống đề xuất ứng dụng bằng cách sử dụng cây quyết định. Tóm lại, thuật toán xây dựng cây quyết định như sau:

1. Tìm ra dữ liệu nào hữu ích nhất để quyết định nên đề xuất ứng dụng nào.
2. Tính năng này chia dữ liệu thành hai tập dữ liệu nhỏ hơn.
3. Lặp lại quy trình 1 và 2 cho mỗi tập dữ liệu nhỏ hơn.

Nói cách khác, những gì chúng tôi làm là quyết định tính năng nào trong hai tính năng (nền tảng hoặc độ tuổi) thành công hơn trong việc xác định ứng dụng nào người dùng sẽ tải xuống và chọn ứng dụng này làm gốc của cây quyết định. Sau đó, chúng tôi lặp lại các nhánh, luôn chọn đặc điểm có tính xác định cao nhất cho dữ liệu trong nhánh đó, từ đó xây dựng cây quyết định của chúng tôi.

Bước đầu tiên xây dựng mô hình: Yêu cầu điều tốt nhất

câu hỏi

Bước đầu tiên để xây dựng mô hình của chúng ta là tìm ra đặc điểm hữu ích nhất: nói cách khác, câu hỏi hữu ích nhất để hỏi.Đầu tiên, hãy đơn giản hóa dữ liệu của chúng tôi một chút.Hãy gọi tất cả mọi người dưới 20 tuổi là “Trẻ” và mọi người từ 20 tuổi trở lên là “Người lớn” (đừng lo—chúng tôi sẽ sớm quay lại tập dữ liệu gốc trong phần “Phân chia dữ liệu bằng các tính năng liên tục, chẳng hạn như tuổi”).Tập dữ liệu đã sửa đổi của chúng tôi được hiển thị trong bảng 9.2.

Bảng 9.2 Một phiên bản đơn giản của tập dữ liệu trong bảng 9.1, trong đó cột độ tuổi đã được đơn giản hóa thành hai loại, “trẻ” và “người lớn”

Nền tảng	Age	App
iPhone	Trẻ	Số nguyên tử
iPhone	Người lớn	Kiểm tra bạn đời
Android	Người lớn	Công cụ tìm tổ ong
iPhone	Người lớn	Kiểm tra bạn đời
Android	Trẻ	Số nguyên tử
Android	Trẻ	Số nguyên tử

Nền tảng của cây quyết định là các câu hỏi có dạng “Người dùng có sử dụng iPhone không?”hoặc “Người dùng có trẻ không?”Chúng ta cần một trong những thứ này để sử dụng làm gốc của cây.Chúng ta nên chọn cái nào?Chúng ta nên chọn ứng dụng xác định chính xác nhất ứng dụng họ đã tải xuống.Để quyết định câu hỏi nào tốt hơn ở câu hỏi này, hãy so sánh chúng.

CÂU HỎI ĐẦU TIÊN: NGƯỜI DÙNG CÓ SỬ DỤNG IPHONE HOẶC ANDROID không?

Câu hỏi này chia người dùng thành hai nhóm, người dùng iPhone và người dùng Android. Mỗi nhóm có ba người dùng trong đó.Nhưng chúng tôi cần theo dõi xem mỗi người dùng đã tải xuống ứng dụng nào.Nhìn nhanh vào bảng 9.2 giúp chúng ta nhận thấy những điều sau:

- Trong số những người dùng iPhone, một người đã tải xuống Atom Count và hai người đã tải xuống Check Mate Mate. Trong số những người dùng Android, hai người đã tải xuống Atom Count và một người đã tải xuống Beehive Finder.

Gốc quyết định kết quả được thể hiện trong hình 9.7.

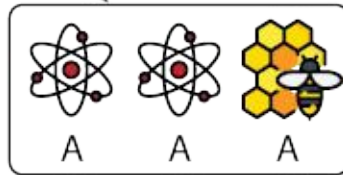
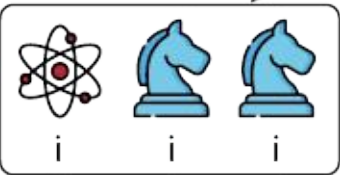
Split by platform

Platform= iPhone?



Yes

No



Hình 9.7 Nếu chúng ta chia người dùng theo nền tảng, chúng ta sẽ có sự phân chia này: người dùng iPhone ở bên trái và người dùng Android ở bên phải. Trong số những người dùng iPhone, một người đã tải xuống Atom Count và hai người đã tải xuống Check Mate Mate. Trong số những người dùng Android, hai người đã tải xuống Atom Count và một người đã tải xuống Beehive Finder.

Bây giờ hãy xem điều gì sẽ xảy ra nếu chúng ta chia chúng theo độ tuổi.

CÂU HỎI THỨ HAI: NGƯỜI DÙNG TRẺ HAY NGƯỜI LỚN?

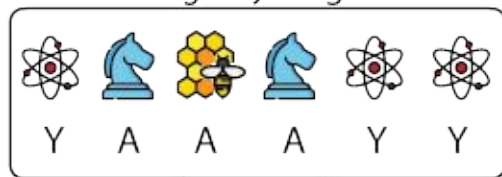
Câu hỏi này chia người dùng thành hai nhóm, người trẻ và người lớn. Một lần nữa, mỗi nhóm có ba người dùng trong đó. Nhìn nhanh vào bảng 9.2 giúp chúng ta nhận thấy mỗi người dùng đã tải xuống những gì, như sau:

- Tất cả người dùng trẻ đều tải xuống Atom Count. Trong số những người dùng trưởng thành, hai người đã tải xuống Atom Count và một người đã tải xuống Beehive Finder.

Gốc quyết định kết quả được thể hiện trong hình 9.8.

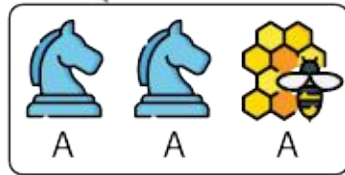
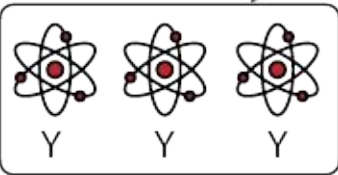
Split by age

Age = young?



Yes

No



Hình 9.8 Nếu chúng ta chia người dùng theo độ tuổi, chúng ta sẽ có sự phân chia này: người trẻ ở bên trái và người lớn ở bên phải. Trong số những người dùng trẻ tuổi, cả ba đều tải xuống Atom Count. Trong số người dùng trưởng thành, một người đã tải xuống Beehive Finder và hai người đã tải xuống Check Mate Mate.

Nhìn vào hình 9.7 và 9.8, cái nào có vẻ được chia tốt hơn? Có vẻ như cái thứ hai (dựa trên độ tuổi) tốt hơn, vì nó cho thấy thực tế là cả ba thanh niên đều đã tải xuống Atom Count. Nhưng chúng tôi cần máy tính để tìm ra rằng độ tuổi là một tính năng tốt hơn, vì vậy chúng tôi sẽ cung cấp cho nó một số con số để so sánh. Trong phần này, chúng ta tìm hiểu ba cách để so sánh hai phần tách này: độ chính xác, tạp chất Gini và entropy. Hãy bắt đầu với điều đầu tiên: độ chính xác.

ĐỘ CHÍNH XÁC: MÔ HÌNH CỦA CHÚNG TÔI ĐÚNG CÓ THƯỜNG XUYỀN?

Chúng ta đã tìm hiểu về độ chính xác ở chương 7 nhưng đây là phần tóm tắt nhỏ. Độ chính xác là tỷ lệ các điểm dữ liệu được phân loại chính xác trên tổng số điểm dữ liệu.

Giả sử rằng chúng tôi chỉ được phép có một câu hỏi và với một câu hỏi đó, chúng tôi phải xác định ứng dụng nào sẽ giới thiệu cho người dùng của mình. Chúng ta có hai cách phân loại sau:

- **Bộ phân loại 1:** đặt câu hỏi “Bạn sử dụng nền tảng nào?” và từ đó xác định nên giới thiệu ứng dụng nào
- **Phân loại 2:** đặt câu hỏi “Bạn bao nhiêu tuổi?” và từ đó xác định ứng dụng nào sẽ được đề xuất

Chúng ta hãy xem xét kỹ hơn các phân loại. Nhận xét quan trọng sau đây: nếu chúng ta phải

đề xuất một ứng dụng bằng cách chỉ hỏi một câu hỏi, cách tốt nhất của chúng tôi là xem xét tất cả những người đã trả lời cùng một câu trả lời và đề xuất ứng dụng phổ biến nhất trong số họ.

Bộ phân loại 1: Bạn sử dụng nền tảng nào?

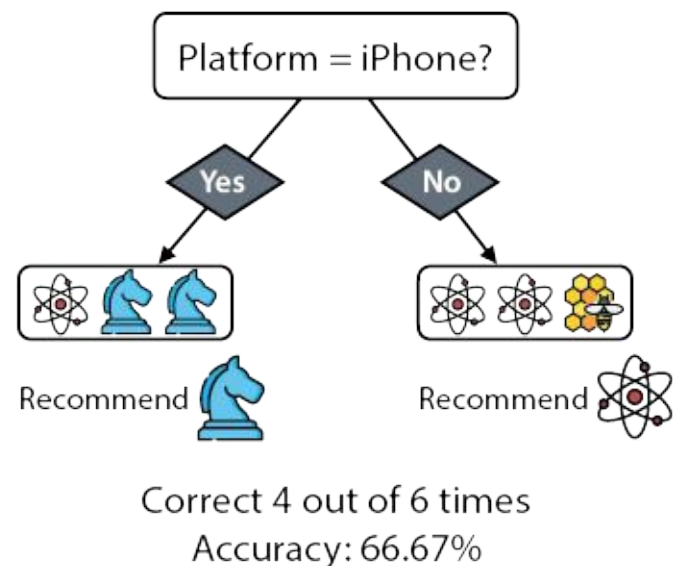
- Nếu câu trả lời là “iPhone” thì chúng tôi nhận thấy rằng trong số những người dùng iPhone, phần lớn đã tải xuống Check Mate Mate. Do đó, chúng tôi khuyên dùng Check Mate Mate cho tất cả người dùng iPhone. Chúng tôi đúng **hai trong số ba lần**. Nếu câu trả lời là “Android”, thì chúng tôi nhận thấy rằng trong số người dùng Android, phần lớn đã tải xuống Atom Count, vì vậy đó là phiên bản mà chúng tôi đề xuất cho tất cả người dùng Android. Chúng tôi đúng **hai trong ba lần**.

Phân loại 2: Bạn bao nhiêu tuổi?

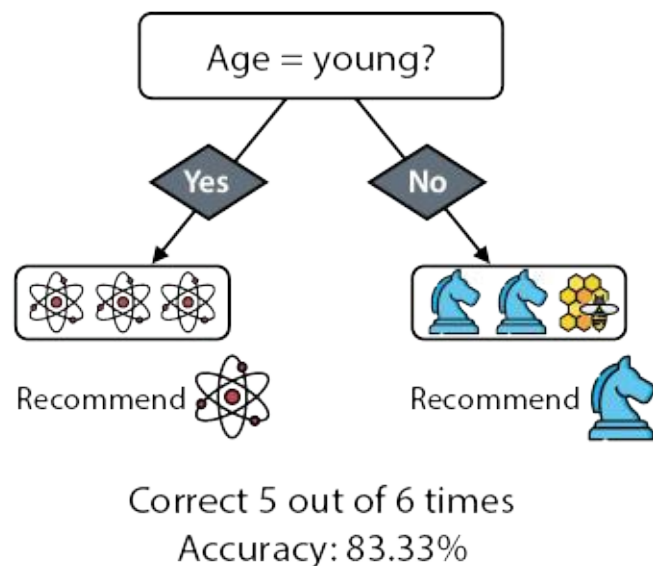
- Nếu câu trả lời là “trẻ” thì chúng tôi nhận thấy rằng tất cả những người trẻ tuổi đã tải xuống Atom Count, vì vậy đó là khuyến nghị mà chúng tôi đưa ra. Chúng tôi đúng **ba lần trong số ba lần**. Nếu câu trả lời là “người lớn”, thì chúng tôi nhận thấy rằng trong số người lớn, phần lớn đã tải xuống Check Mate Mate, vì vậy chúng tôi khuyên bạn nên sử dụng câu trả lời đó. Chúng tôi đúng **hai trong ba lần**.

Lưu ý rằng phân loại 1 đúng **bốn lần trên sáu** và phân loại 2 đúng **năm lần trên sáu**. Do đó, đối với tập dữ liệu này, phân loại 2 tốt hơn. Trong hình 9.9, bạn có thể thấy độ chính xác của hai bộ phân loại. Lưu ý rằng các câu hỏi được diễn đạt lại để chúng có câu trả lời có hoặc không, điều này không làm thay đổi cách phân loại hoặc kết quả.

Classifier 1: By platform



Classifier 2: By age



Hình 9.9 Bộ phân loại 1 sử dụng nền tảng và bộ phân loại 2 sử dụng độ tuổi. Để đưa ra dự đoán ở mỗi lá, mỗi bộ phân loại sẽ chọn nhãn phổ biến nhất trong số các mẫu ở lá đó. Phân loại 1 đúng bốn trên sáu lần và phân loại 2 đúng năm trên sáu lần. Vì vậy, dựa trên độ chính xác, phân loại 2 là tốt hơn.

CHỈ SỐ TẠP CHẤT GINI: TẬP DỮ LIỆU CỦA TÔI ĐA DẠNG NHƯ THẾ NÀO?

Chỉ số tạp chất Gini, hoặc *chỉ số Gini*, là một cách khác để chúng ta có thể so sánh nền tảng và sự phân chia theo độ tuổi. Chỉ số Gini là thước đo mức độ đa dạng trong tập dữ liệu. Nói cách khác, nếu chúng ta có một tập hợp trong đó tất cả các phần tử đều giống nhau thì tập hợp này có chỉ số Gini thấp và nếu tất cả các phần tử khác nhau thì tập hợp đó có chỉ số Gini lớn. Để rõ ràng, hãy xem xét hai bộ 10 quả bóng màu sau đây (trong đó không thể phân biệt được hai quả bóng cùng màu):

- **Bộ 1:** tám quả bóng đỏ, hai quả bóng xanh
- **Bộ 2:** bốn quả bóng đỏ, ba quả bóng xanh, hai quả bóng vàng, một quả bóng xanh

Bộ 1 trông thuần khiết hơn bộ 2, vì bộ 1 chứa hầu hết các quả bóng màu đỏ và một vài quả bóng màu xanh, trong khi bộ 2 có nhiều màu sắc khác nhau. Tiếp theo, chúng tôi nghĩ ra thước đo tạp chất gán giá trị thấp cho tập 1 và giá trị cao cho tập 2. Thước đo tạp chất này dựa vào xác suất. Hãy xem xét câu hỏi sau:

Nếu chúng ta chọn ngẫu nhiên hai phần tử của tập hợp thì xác suất để chúng có màu khác nhau là bao nhiêu? Hai yếu tố này không cần phải khác biệt; chúng ta được phép chọn cùng một phần tử hai lần.

Đối với set 1, xác suất này thấp vì các quả bóng trong set có màu giống nhau. Đối với set 2, xác suất này cao vì bộ này rất đa dạng và nếu chúng ta chọn hai quả bóng, chúng có khả năng thuộc về

màu sắc khác nhau. Hãy tính các xác suất này. Đầu tiên, hãy lưu ý rằng theo định luật xác suất bổ sung (xem phần “Bài toán vừa xảy ra?” ở chương 8), xác suất để chúng ta chọn được hai quả bóng khác màu là 1 trừ đi xác suất để chúng ta chọn được hai quả bóng cùng màu:

$$P(\text{chọn 2 bi khác màu}) = 1 - P(\text{chọn 2 bi cùng màu})$$

Bây giờ hãy tính xác suất để chọn được 2 quả bóng cùng màu. Hãy xem xét một tập hợp tổng quát, trong đó các quả bóng có n màu. Hãy gọi chúng là màu 1, màu 2, cho đến màu n . Vì hai quả bóng phải có một trong các n màu nên xác suất chọn được hai quả bóng cùng màu là tổng các xác suất để chọn được hai quả bóng có mỗi màu n :

$$P(\text{chọn 2 bi cùng màu}) = P(\text{cả 2 bi cùng màu 1}) + P(\text{cả 2 bi cùng màu 2}) + \dots + P(\text{cả 2 bi cùng màu } n)$$

Những gì chúng tôi sử dụng ở đây là quy tắc tính tổng cho các xác suất khác nhau, quy tắc này nêu rõ như sau:

quy tắc tổng cho các xác suất rời rạc Nếu hai sự kiện E và F rời rạc, tức là chúng không bao giờ xảy ra cùng một lúc, thì xác suất để một trong hai sự kiện xảy ra (sự kết hợp của các sự kiện) là tổng xác suất của từng sự kiện. Nói cách khác,

$$P(E \cup F) = P(E) + P(F)$$

Bây giờ, hãy tính xác suất để hai quả bóng có cùng màu với mỗi màu. Lưu ý rằng chúng ta đang chọn từng quả bóng hoàn toàn độc lập với những quả bóng khác. Do đó, theo quy tắc tích xác suất độc lập (phần “Bài toán vừa xảy ra?” ở chương 8), xác suất để cả hai quả bóng đều có màu 1 là bình phương xác suất để chúng ta chọn được một quả bóng và nó có màu 1. Nói chung, nếu p_i là xác suất để chúng ta chọn được một quả bóng ngẫu nhiên và nó có màu i thì

$$P(\text{cả hai quả bóng đều có màu } i) = p_i^2.$$

Kết hợp tất cả các công thức này lại với nhau (hình 9.10), chúng ta có được điều đó

$$P(\text{picking two balls of different colors}) = 1 - p_1^2 - p_2^2 - \dots - p_n^2.$$

Công thức cuối cùng này là chỉ số Gini của tập hợp.

Gini impurity Index = P(picking two balls of different colors)

$$= 1 - P(\text{picking two balls of the same color})$$

$$= 1 - p_1^2 - p_2^2 - \dots - p_n^2$$

P(Both balls are color 1)

P(Both balls are color 2)

P(Both balls are color n)

Hình 9.10 Tóm tắt cách tính chỉ số tạp chất Gini

Cuối cùng, xác suất để chúng ta chọn được một quả bóng màu ngẫu nhiên i là số quả bóng màu i chia cho tổng số quả bóng. Điều này dẫn tới định nghĩa chính thức về chỉ số Gini.

chỉ số tạp chất gini Trong một tập hợp có các phần tử m và các lớp n , với các phần tử a_i Thuộc lớp i thì chỉ số tạp chất Gini là

$$Gini = 1 - p_1^2 - p_2^2 - \dots - p_n^2,$$

ở đây $p_i = a_i / m$. Điều này có thể được hiểu là xác suất mà nếu chúng ta chọn hai các phần tử ngẫu nhiên ngoài tập hợp thì chúng thuộc các lớp khác nhau.

Bây giờ chúng ta có thể tính chỉ số Gini cho cả hai bộ của mình. Để rõ ràng, cách tính chỉ số Gini cho tập 1 được minh họa trong hình 9.11 (với màu đỏ và xanh được thay thế bằng đen và trắng).

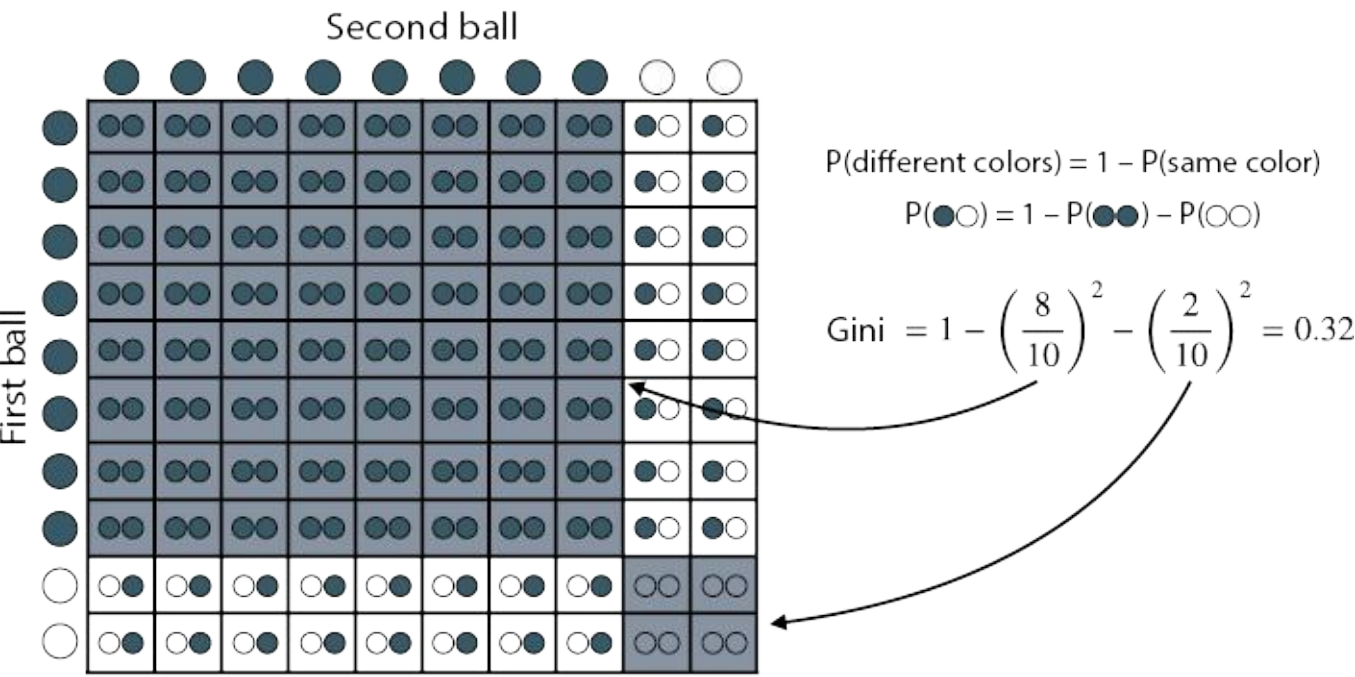
Bộ 1: {red, red, red, red, red, red, red, red, blue, blue} (tám bi đỏ, hai bi xanh)

$$\text{Gini impurity index} = 1 - \left(\frac{8}{10}\right)^2 - \left(\frac{2}{10}\right)^2 = 1 - \frac{68}{100} = 0.32$$

Bộ 2: {đỏ, đỏ, đỏ, đỏ, xanh lam, xanh lam, xanh lam, vàng, vàng, xanh lục}

$$\text{Gini impurity index} = 1 - \left(\frac{4}{10}\right)^2 - \left(\frac{3}{10}\right)^2 - \left(\frac{2}{10}\right)^2 - \left(\frac{1}{10}\right)^2 = 1 - \frac{30}{100} = 0.7$$

Lưu ý rằng, thực tế, chỉ số Gini của tập 1 lớn hơn chỉ số Gini của tập 2.



Hình 9.11 Công thức tính chỉ số Gini cho bộ 8 bi đen và 2 bi trắng. Lưu ý nếu tổng diện tích hình vuông là 1 thì xác suất lấy được 2 bi đen là $0,8^2$, xác suất lấy được 2 bi trắng là $0,2^2$

(hai cái này được biểu thị bằng các ô vuông được tô bóng). Như vậy, xác suất để chọn được hai viên bi khác màu là diện tích còn lại là $1 - 0,8^2 - 0,2^2$
 $= 0,32$. Đó là chỉ số Gini.

Làm cách nào để sử dụng chỉ số Gini để quyết định cách nào trong hai cách phân chia dữ liệu (tuổi hoặc nền tảng) là tốt hơn? Rõ ràng, nếu chúng tôi có thể chia dữ liệu thành hai tập dữ liệu thuần túy hơn thì chúng tôi đã thực hiện phân chia tốt hơn. Vì vậy, hãy tính chỉ số Gini của tập nhãn của mỗi lá. Nhìn vào hình 9.12, đây là nhãn của các lá (trong đó chúng tôi viết tắt từng ứng dụng bằng chữ cái đầu tiên trong tên của ứng dụng):

Bộ phân loại 1 (theo nền tảng):

- Lá trái (iPhone): {A, C, C} Lá phải
- (Android): {A, A, B}

Loại 2 (theo độ tuổi):

- Lá trái (trẻ): {A, A, A} Lá phải
- (trưởng thành): {B, C, C}

Gini trong xúc xắc của các bộ {A, C, C}, {A, A, B} và {B, C, C} đều giống nhau: □

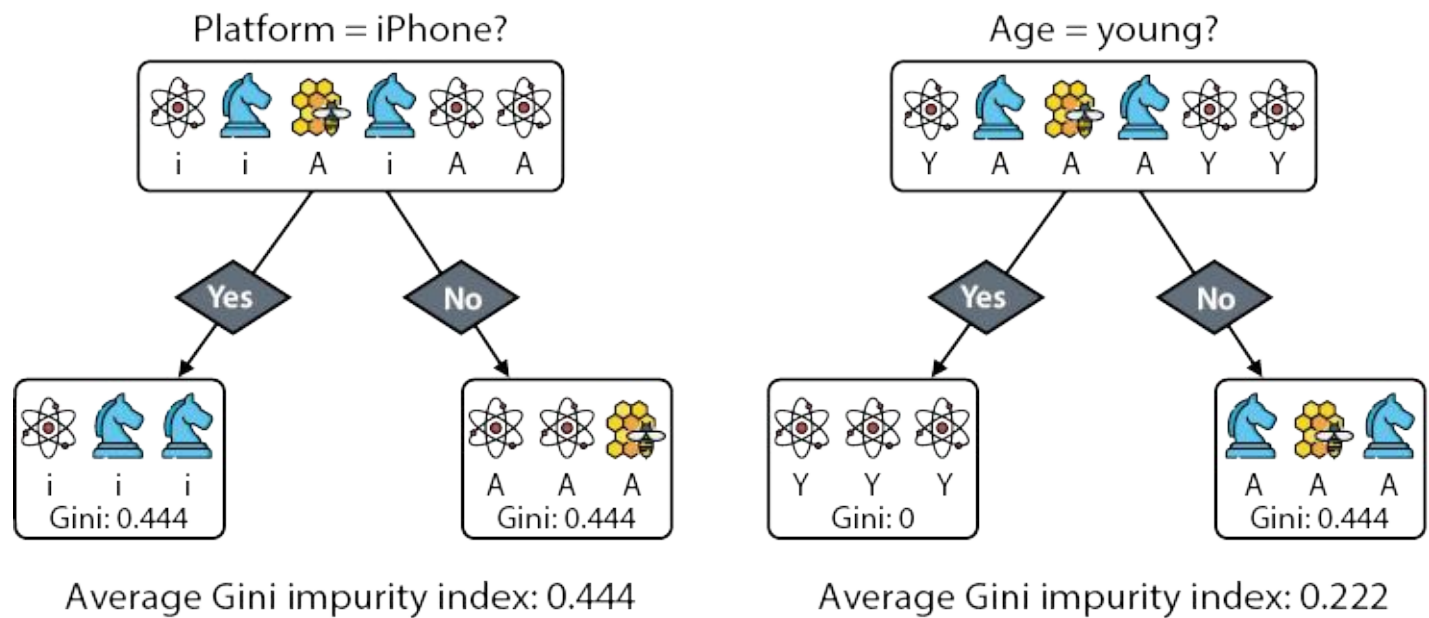
$1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$ Chỉ số Gini của tập {A, A, A} là 0. Nói chung, chỉ số Gini của một tập thuần túy luôn bằng 0. Để đo độ tinh khiết của phép phân tách, chúng ta lấy trung bình chỉ số Gini của hai lá. Vì vậy, chúng ta có các tính toán sau:

Bộ phân loại 1 (theo nền tảng):

$$\text{Average Gini index} = 1/2(0.444 + 0.444) = 0.444$$

Loại 2 (theo độ tuổi):

$$\text{Average Gini index} = 1/2(0.444 + 0) = 0.222$$



Hình 9.12 Hai cách để phân chia tập dữ liệu, theo nền tảng và độ tuổi cũng như tính toán chỉ số Gini của chúng. Lưu ý rằng việc chia tập dữ liệu theo độ tuổi sẽ mang lại cho chúng ta hai tập dữ liệu nhỏ hơn với chỉ số Gini trung bình thấp hơn. Vì vậy, chúng tôi chọn chia tập dữ liệu theo độ tuổi.

Chúng tôi kết luận rằng cách phân chia thứ hai là tốt hơn vì nó có chỉ số Gini trung bình thấp hơn.

ngoài ra Không nên nhầm lẫn chỉ số tạp chất Gini với hệ số Gini. Hệ số Gini được sử dụng trong thống kê để tính toán sự bất bình đẳng về thu nhập hoặc sự giàu có ở các quốc gia. Trong cuốn sách này, bất cứ khi nào chúng ta nói về chỉ số Gini, chúng ta đang đề cập đến chỉ số tạp chất Gini.

VÀO PY: MỘT BIỆN PHÁP KHÁC ĐỂ ĐA DẠNG VỚI

ỨNG DỤNG MẠNH MỀ TRONG LÝ THUYẾT THÔNG TIN

Trong phần này, chúng ta tìm hiểu một thước đo khác về tính đồng nhất trong một tập hợp—entropy của nó—dựa trên khái niệm vật lý về entropy và rất quan trọng trong lý thuyết xác suất và thông tin. Để hiểu entropy, chúng ta xem xét một câu hỏi xác suất hơi lạ. Hãy xem xét hai bộ bóng màu giống như trong phần trước, nhưng hãy coi màu sắc như một bộ có thứ tự.

- **Bộ 1:** {red, red, red, red, red, red, red, red, blue, blue} (tám quả bóng đỏ, hai quả bóng xanh)
- **Bộ 2:** {red, red, red, red, blue, blue, blue, yellow, yellow, green} (bốn quả bóng đỏ, ba quả bóng xanh, hai quả bóng vàng, một quả bóng xanh)

Bây giờ, hãy xem xét tình huống sau: chúng ta đặt 1 quả bóng vào trong một cái túi, và chúng ta bắt đầu nhặt những quả bóng ra khỏi túi này và ngay lập tức trả từng quả bóng mà chúng ta vừa nhặt lại vào túi. Chúng tôi ghi lại màu sắc của những quả bóng chúng tôi đã chọn. Nếu chúng ta làm điều này 10 lần, hãy tưởng tượng rằng chúng ta nhận được trình tự sau:

- Đỏ, đỏ, đỏ, xanh, đỏ, xanh, xanh, đỏ, đỏ, đỏ

Đây là câu hỏi chính xác định entropy:

Xác suất để bằng cách làm theo quy trình được mô tả trong đoạn trước, chúng ta có được trình tự chính xác xác định tập 1, đó là {red, red, red, red, red, red, red, red, red, blue, blue} là bao nhiêu?

Xác suất này không lớn lắm, bởi vì chúng ta phải thực sự may mắn mới có được dãy số này. Hãy tính toán nó. Chúng ta có tám quả bóng màu đỏ và hai quả bóng màu xanh, vì vậy

xác suất để chúng ta lấy được quả bóng màu ⁸đỏ là ⁸~~10~~ và xác suất để chúng ta lấy được quả bóng màu xanh là ²~~10~~. Vì tất cả các lần rút đều độc lập nên xác suất để chúng ta có được dãy số mong muốn là

$$P(r, r, r, r, r, r, r, r, b, b) = \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{8}{10} \cdot \frac{2}{10} \cdot \frac{2}{10}$$
$$= \left(\frac{8}{10}\right)^8 \left(\frac{2}{10}\right)^2 = 0.0067108864.$$

Con số này rất nhỏ nhưng bạn có thể tưởng tượng được xác suất tương ứng của tập 2 không? Đối với set 2, chúng ta chọn những quả bóng từ một túi có bốn quả bóng đỏ, ba quả bóng xanh, hai quả bóng vàng.

quả bóng và một quả bóng màu xanh lá cây và hy vọng đạt được trình tự sau:

- Đỏ, đỏ, đỏ, đỏ, xanh, xanh, xanh, vàng, vàng, xanh lá cây.

Điều này gần như không thể, bởi vì chúng ta có nhiều màu sắc và không có nhiều quả bóng mỗi màu. Xác suất này, được tính theo cách tương tự, là

$$P(r, r, r, r, b, b, b, y, y, g) = \frac{4}{10} \cdot \frac{4}{10} \cdot \frac{4}{10} \cdot \frac{4}{10} \cdot \frac{3}{10} \cdot \frac{3}{10} \cdot \frac{3}{10} \cdot \frac{2}{10} \cdot \frac{2}{10} \cdot \frac{1}{10}$$
$$= \left(\frac{4}{10}\right)^4 \left(\frac{3}{10}\right)^3 \left(\frac{2}{10}\right)^2 \left(\frac{1}{10}\right)^1 = 0.0000027648.$$

Bộ càng đa dạng thì chúng ta càng khó có được trình tự ban đầu bằng cách chọn từng quả bóng một. Ngược lại, bộ thuần khiết nhất, trong đó tất cả các quả bóng có cùng màu, thì dễ dàng có được theo cách này. Ví dụ: nếu bộ ban đầu của chúng ta có 10 quả bóng màu đỏ thì mỗi lần chúng ta chọn ngẫu nhiên một quả bóng sẽ có màu đỏ. Do đó, xác suất để nhận được dãy {đỏ, đỏ, đỏ, đỏ, đỏ, đỏ, đỏ, đỏ, đỏ, đỏ} là 1.

Những con số này rất nhỏ trong hầu hết các trường hợp—và con số này chỉ có 10 phần tử. Hãy tưởng tượng nếu tập dữ liệu của chúng tôi có một triệu phần tử. Chúng ta sẽ phải giải quyết những con số cực kỳ nhỏ. Khi chúng ta phải xử lý các số thực sự nhỏ, sử dụng logarit là phương pháp tốt nhất, vì chúng cung cấp một cách thuận tiện để viết các số nhỏ. Ví dụ: 0,0000000000000001 bằng 10^{-15} , do đó logarit của nó trong cơ số 10 là -15 , đây là một con số đẹp hơn nhiều để làm việc.

Entropy được định nghĩa như sau: chúng ta bắt đầu với xác suất để khôi phục chuỗi ban đầu bằng cách chọn các phần tử trong tập hợp của chúng ta, lần lượt từng phần tử, với sự lặp lại. Sau đó, chúng ta lấy logarit và chia cho tổng số phần tử trong tập hợp. Vì cây quyết định xử lý các quyết định nhị phân nên chúng tôi sẽ sử dụng logarit trong cơ số 2. Lý do chúng tôi lấy logarit âm là vì logarit của các số rất nhỏ đều âm, vì vậy chúng tôi nhân với -1 để biến nó thành số dương. Bởi vì chúng ta lấy giá trị âm, tập hợp càng đa dạng thì entropy càng cao.

Bây giờ chúng ta có thể tính toán entropy của cả hai bộ và mở rộng chúng bằng cách sử dụng hai nhận dạng sau:

- $\log(ab) = \log(a) + \log(b)$
- $\log(a^c) = c \log(a)$

Bộ 1: {red, red, red, red, red, red, red, red, blue, blue} (tám bi đỏ, hai bi xanh)

$$Entropy = -\frac{1}{10} \log_2 = \left[\left(\frac{8}{10} \right)^8 \left(\frac{2}{10} \right)^2 \right] = -\frac{8}{10} \log_2 \left(\frac{8}{10} \right) - \frac{2}{10} \log_2 \left(\frac{2}{10} \right) = 0.722$$

Bộ 2: {đỏ, đỏ, đỏ, đỏ, xanh lam, xanh lam, xanh lam, vàng, vàng, xanh lục}

$$Entropy = -\frac{1}{10} \log_2 \left[\left(\frac{4}{10} \right)^4 \left(\frac{3}{10} \right)^3 \left(\frac{2}{10} \right)^2 \left(\frac{1}{10} \right)^1 \right]$$
$$= -\frac{4}{10} \log_2 \left(\frac{4}{10} \right) - \frac{3}{10} \log_2 \left(\frac{3}{10} \right) - \frac{2}{10} \log_2 \left(\frac{2}{10} \right) - \frac{1}{10} \log_2 \left(\frac{1}{10} \right) = 1.846$$

Lưu ý rằng entropy của tập 2 lớn hơn entropy của tập 1, ngụ ý rằng tập 2 đa dạng hơn tập 1. Sau đây là định nghĩa chính thức về entropy:

entropy Trong một tập hợp có m phần tử và các lớp n , với các phần tử a_i thuộc lớp i , entropy là

$$Entropy = -p_1 \log_2(p_1) - p_2 \log_2(p_2) - \dots - p_n \log_2(p_n),$$

$$\text{where } p_i = \frac{a_i}{m}.$$

Chúng ta có thể sử dụng entropy để quyết định cách nào trong hai cách phân chia dữ liệu (nền tảng hoặc độ tuổi) tốt hơn giống như cách chúng ta đã làm với chỉ mục Gini. Nguyên tắc chung là nếu chúng ta có thể chia dữ liệu thành hai tập dữ liệu với ít entropy kết hợp hơn thì chúng ta đã thực hiện phân chia tốt hơn. Vì vậy, hãy tính entropy của tập nhãn của mỗi lá. Một lần nữa, nhìn vào hình 9.12, đây là nhãn của các lá (trong đó chúng tôi viết tắt từng ứng dụng bằng chữ cái đầu tiên trong tên của nó):

Bộ phân loại 1 (theo nền tảng):

Lá trái: {A, C, C}

Lá phải: {A, A, B}

Loại 2 (theo độ tuổi):

Lá trái: {A, A, A}

Lá phải: {B, C, C}

Entropy của các tập {A, C, C}, {A, A, B} và {B, C, C} đều giống nhau:

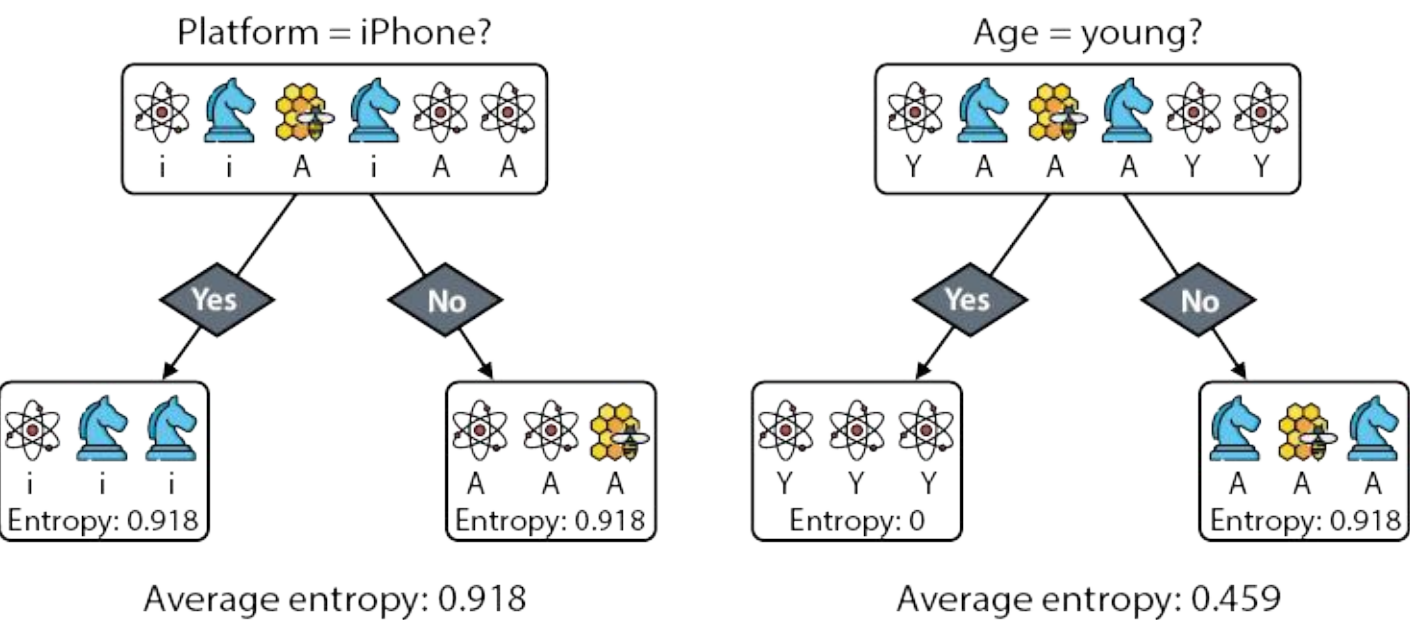
$-\frac{2}{3}\log_2\left(\frac{2}{3}\right)-\frac{1}{3}\log_2\left(\frac{1}{3}\right)$.Entropy của tập {A, A, A} là $-\log\left(\frac{3}{3}\right)=-\log_2(1)=0$. Nói chung, entropy của một tập hợp trong đó tất cả các phần tử giống nhau luôn bằng 0. Để đo độ thuần khiết của phép tách, chúng ta tính trung bình entropy của các tập nhận của hai lá như sau (minh họa trên hình 9.13):

Bộ phân loại 1 (theo nền tảng):

Average entropy = 1/2(0.918 + 0.918) = 0.918

Loại 2 (theo độ tuổi):

Average entropy = 1/2(0.918+0) = 0.459



Hình 9.13 Hai cách để phân chia tập dữ liệu, theo nền tảng và độ tuổi cũng như các phép tính entropy của chúng.Lưu ý rằng việc chia tập dữ liệu theo độ tuổi sẽ mang lại cho chúng ta hai tập dữ liệu nhỏ hơn với entropy trung bình thấp hơn.Do đó, chúng tôi lại chọn chia tập dữ liệu theo độ tuổi.

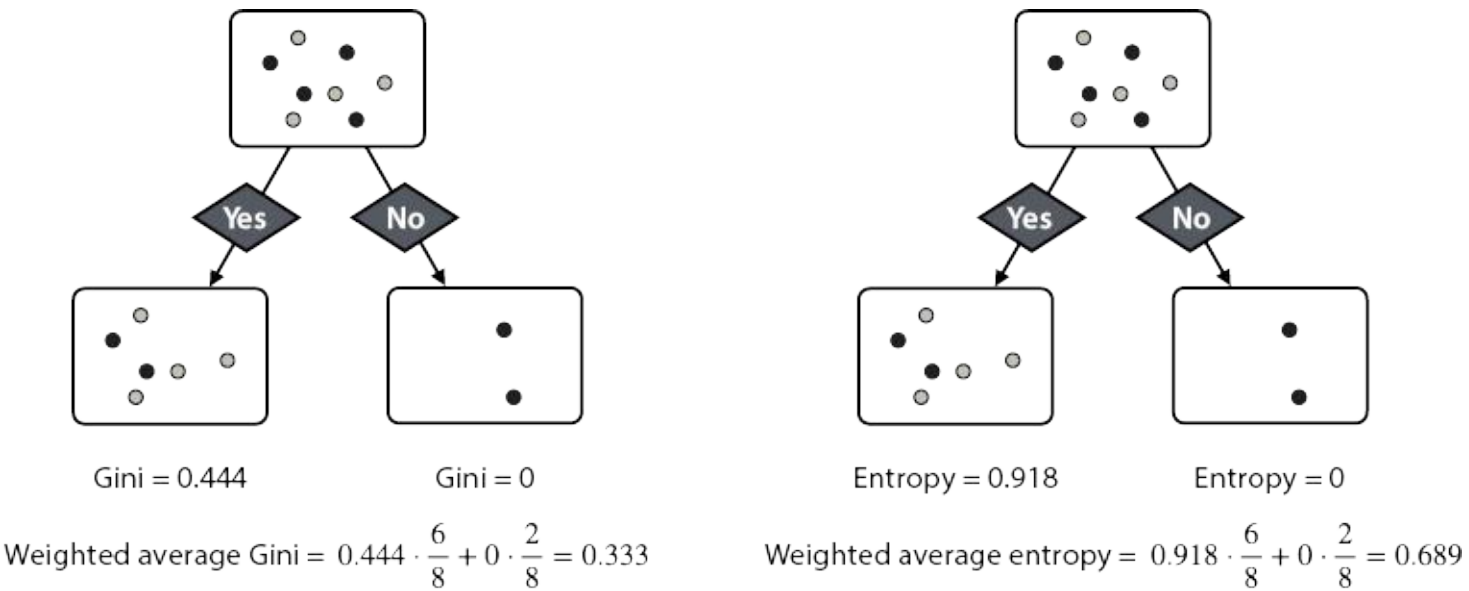
Vì vậy, một lần nữa chúng ta kết luận rằng phép chia thứ hai tốt hơn vì nó có entropy trung bình thấp hơn.

Entropy là một khái niệm cực kỳ quan trọng trong xác suất và thống kê, bởi vì nó có mối liên hệ chặt chẽ với lý thuyết thông tin, chủ yếu nhờ vào công trình của Claude Shannon.Trên thực tế, một khái niệm quan trọng được gọi là *thu thập thông tin* chính xác là

sự thay đổi entropy. Để tìm hiểu thêm về chủ đề này, vui lòng xem phụ lục C để có video và bài đăng blog trình bày chi tiết hơn về chủ đề này.

LỚP CÓ KÍCH CỠ KHÁC NHAU? KHÔNG CÓ VẤN ĐỀ: CHÚNG TÔI CÓ THỂ LẤY TRUNG BÌNH CÓ TRỌNG LƯỢNG

Trong các phần trước, chúng ta đã học cách thực hiện phép chia tốt nhất có thể bằng cách giảm thiểu chỉ số tạp chất Gini hoặc entropy trung bình. Tuy nhiên, hãy tưởng tượng rằng bạn có một tập dữ liệu có tám điểm dữ liệu (khi đào tạo cây quyết định, chúng tôi cũng gọi là mẫu) và bạn chia nó thành hai tập dữ liệu có kích thước sáu và hai. Như bạn có thể tưởng tượng, tập dữ liệu lớn hơn sẽ được tính nhiều hơn trong các phép tính chỉ số tạp chất Gini hoặc entropy. Do đó, thay vì xem xét mức trung bình, chúng tôi xem xét mức trung bình có trọng số, trong đó tại mỗi lá, chúng tôi ấn định tỷ lệ điểm tương ứng với lá đó. Vì vậy, trong trường hợp này, chúng tôi sẽ cân chỉ số tạp chất Gini (hoặc entropy) đầu tiên là 6/8 và chỉ số thứ hai là 2/8. Hình 9.14 cho thấy một ví dụ về chỉ số tạp chất Gini trung bình có trọng số và entropy trung bình có trọng số cho một phân tách mẫu.



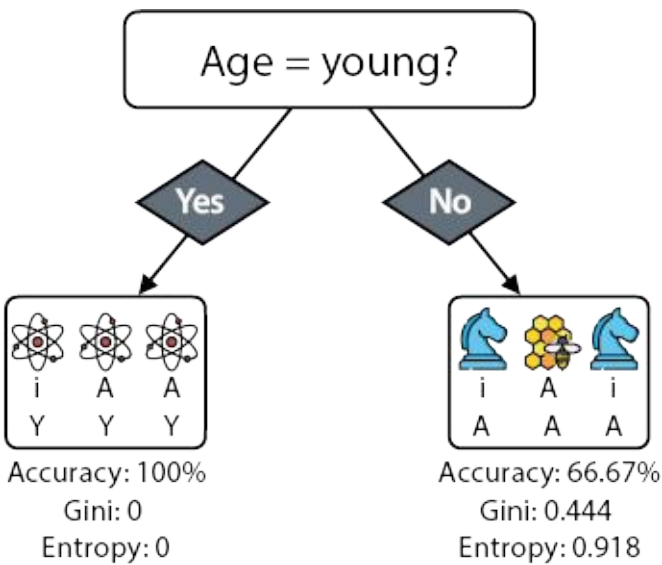
Hình 9.14 Phân chia tập dữ liệu có kích thước tám thành hai tập dữ liệu có kích thước sáu và hai. Để tính chỉ số Gini trung bình và entropy trung bình, chúng tôi tính chỉ số của tập dữ liệu bên trái là 6/8 và chỉ số của tập dữ liệu bên phải là 2/8. Điều này dẫn đến chỉ số Gini có trọng số là 0,333 và entropy có trọng số là 0,689.

Bây giờ chúng ta đã học được ba cách (độ chính xác, chỉ số Gini và entropy) để chọn phân tách tốt nhất, tất cả những gì chúng ta cần làm là lặp lại quá trình này nhiều lần để xây dựng cây quyết định! Điều này được trình bày chi tiết trong phần tiếp theo.

Bước thứ hai để xây dựng mô hình: Lặp lại

Trong phần trước, chúng ta đã tìm hiểu cách phân chia dữ liệu theo cách tốt nhất có thể bằng cách sử dụng một trong các tính năng. Đó là phần lớn quá trình huấn luyện của cây quyết định. Tất cả những gì còn lại để hoàn thành việc xây dựng cây quyết định của chúng ta là lặp lại bước này nhiều lần. Trong phần này chúng ta học cách làm điều này.

Sử dụng ba phương pháp, độ chính xác, chỉ số Gini và entropy, chúng tôi đã quyết định rằng cách phân chia tốt nhất được thực hiện bằng cách sử dụng tính năng “tuổi”. Khi chúng tôi thực hiện việc phân chia này, tập dữ liệu của chúng tôi sẽ được chia thành hai tập dữ liệu. Việc chia thành hai bộ dữ liệu này với độ chính xác, chỉ số Gini và entropy được minh họa trong hình 9.15.

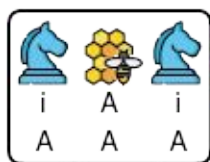


Hình 9.15 Khi chia tập dữ liệu theo độ tuổi, chúng tôi nhận được hai tập dữ liệu. Người dùng bên trái có ba người dùng đã tải xuống Atom Count và người dùng bên phải có một người dùng đã tải xuống Beehive Count và hai người dùng đã tải xuống Check Mate Mate.

Lưu ý rằng tập dữ liệu ở bên trái là thuần túy—tất cả các nhãn đều giống nhau, độ chính xác của nó là 100%, chỉ số Gini và entropy của nó đều bằng 0. Chúng tôi không thể làm gì hơn để phân tách tập dữ liệu này hoặc cải thiện việc phân loại. Do đó, nút này trở thành nút lá và khi chúng ta đến lá đó, chúng ta trả về dự đoán “Số lượng nguyên tử”.

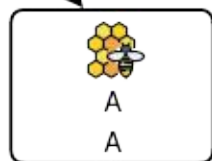
Tập dữ liệu ở bên phải vẫn có thể được chia vì nó có hai nhãn: “Đếm tổ ong” và “Kiểm tra Mate Mate”. Chúng tôi đã sử dụng tính năng độ tuổi, vì vậy hãy thử sử dụng tính năng nền tảng. Hóa ra chúng ta rất may mắn vì người dùng Android đã tải xuống Beehive Count và hai người dùng iPhone đã tải xuống Check Mate Mate. Do đó, chúng ta có thể tách lá này bằng cách sử dụng tính năng nền tảng và thu được nút quyết định như trong hình 9.16.

Platform = iPhone?



Yes

No



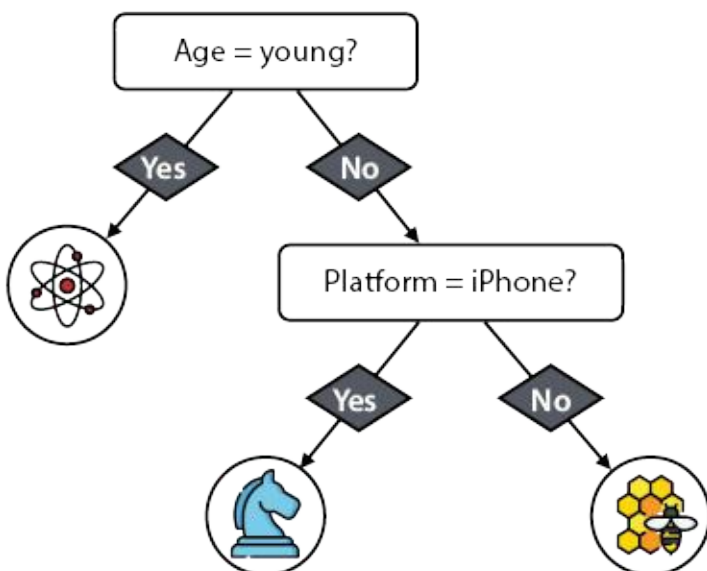
Accuracy: 100%
Gini: 0
Entropy: 0

Accuracy: 100%
Gini: 0
Entropy: 0

Hình 9.16 Chúng ta có thể tách lá bên phải của cây trong hình 9.15 bằng cách sử dụng nền tảng và thu được hai bộ dữ liệu thuần túy. Mỗi một trong số chúng có độ chính xác 100% và chỉ số Gini và entropy bằng 0.

Sau lần chia tách này, chúng tôi đã xong việc vì chúng tôi không thể cải thiện sự chia tách của mình thêm nữa. Vì vậy, chúng ta thu được cây trong hình 9.17.

Final decision tree



Hình 9.17 Cây quyết định thu được có hai nút và ba lá. Cây này dự đoán chính xác mọi điểm trong tập dữ liệu gốc.

Đây là phần cuối của quá trình và chúng tôi đã xây dựng một cây quyết định để phân loại toàn bộ tập dữ liệu của mình. Chúng ta gần như có tất cả mã giả cho thuật toán, ngoại trừ một số chi tiết cuối cùng mà chúng ta sẽ thấy trong phần tiếp theo.

Bước cuối cùng: Khi nào nên dừng xây dựng cây và các siêu tham số khác

Trong phần trước, chúng tôi đã xây dựng cây quyết định bằng cách phân tách đệ quy tập dữ liệu của mình. Mỗi lần phân chia được thực hiện bằng cách chọn tính năng tốt nhất để phân chia. Tính năng này được tìm thấy bằng cách sử dụng bất kỳ số liệu nào sau đây: độ chính xác, chỉ số Gini hoặc entropy. Chúng tôi kết thúc khi phân tập dữ liệu tương ứng với từng nút lá là thuần túy—nói cách khác, khi tất cả các mẫu trên đó có cùng một nhãn.

Nhiều vấn đề có thể phát sinh trong quá trình này. Ví dụ: nếu tiếp tục phân tách dữ liệu quá lâu, chúng ta có thể gặp phải tình huống cực đoan là mỗi lá chứa rất ít mẫu, điều này có thể dẫn đến tình trạng khớp quá mức nghiêm trọng. Cách để ngăn chặn điều này là đưa ra điều kiện dừng. Điều kiện này có thể là bất kỳ điều nào sau đây:

1. Không tách nút nếu thay đổi về độ chính xác, chỉ số Gini hoặc entropy dưới ngưỡng nào đó.
2. Không chia nút nếu nó có ít hơn một số lượng mẫu nhất định.
3. Chỉ tách một nút nếu cả hai lá kết quả đều chứa ít nhất một số lượng mẫu nhất định.
4. Dừng xây dựng cây sau khi bạn đạt đến độ sâu nhất định.

Tất cả các điều kiện dừng này đều yêu cầu siêu tham số. Cụ thể hơn, đây là các siêu tham số tương ứng với bốn điều kiện trước đó:

1. Lượng thay đổi tối thiểu về độ chính xác (hoặc chỉ số Gini hoặc entropy)
2. Số lượng mẫu tối thiểu mà một nút phải có để phân tách nó
3. Số lượng mẫu tối thiểu được phép trong một nút lá
4. Độ sâu tối đa của cây

Cách chúng tôi chọn các siêu tham số này là theo kinh nghiệm hoặc bằng cách thực hiện tìm kiếm toàn diện trong đó chúng tôi tìm kiếm các kết hợp siêu tham số khác nhau và chọn một tổ hợp siêu tham số hoạt động tốt nhất trong bộ xác thực của chúng tôi. Quá trình này được gọi là *tìm kiếm lưới* và chúng ta sẽ nghiên cứu nó chi tiết hơn trong phần "Điều chỉnh siêu tham số để tìm ra mô hình tốt nhất: Tìm kiếm lưới" ở chương 13.

Thuật toán cây quyết định: Cách xây dựng cây quyết định và đưa ra dự đoán với nó

Cuối cùng, bây giờ chúng ta đã sẵn sàng nêu mã giả cho thuật toán cây quyết định, thuật toán này cho phép chúng ta huấn luyện cây quyết định để phù hợp với tập dữ liệu.

Mã giả cho thuật toán cây quyết định

Đầu vào:

- Tập dữ liệu huấn luyện gồm các mẫu có nhãn liên kết của chúng Một số
- liệu để phân chia dữ liệu (độ chính xác, chỉ số Gini hoặc entropy) Một
- (hoặc nhiều) điều kiện dừng

Đầu ra:

- Cây quyết định phù hợp với tập dữ liệu

Thủ tục:

- Thêm nút gốc và liên kết nó với toàn bộ tập dữ liệu. Nút này có cấp 0 .
Gọi nó là nút lá. Lặp lại cho đến khi các điều kiện dừng được đáp ứng ở mọi nút lá: Chọn một
- trong các nút lá ở mức cao nhất. Xem qua tất cả các tính năng và chọn tính năng phân chia các mẫu tương ứng với nút đó theo cách tối ưu, theo số liệu đã chọn. Liên kết tính năng đó với nút. Tính năng này chia tập dữ liệu thành hai nhánh. Tạo hai nút lá mới, một nút cho mỗi nhánh và liên kết các mẫu tương ứng với mỗi nút. Nếu các điều kiện dừng cho phép phân chia, hãy biến nút đó thành nút quyết định và thêm hai nút lá mới bên dưới nó. Nếu cấp của nút là i thì hai nút lá mới ở cấp $i + 1$. Nếu điều kiện dừng không cho phép phân chia thì nút đó sẽ trở thành nút lá. Với nút lá này, hãy liên kết nhãn phổ biến nhất trong số các mẫu của nó. Nhãn đó chính là dự đoán ở chiếc lá.
 -
 -

Trở về:

- Cây quyết định thu được.

Để đưa ra dự đoán bằng cách sử dụng cây này, chúng ta chỉ cần duyệt qua nó, sử dụng các quy tắc sau:

- Đi ngang qua cây hướng xuống dưới. Tại mỗi nút, hãy tiếp tục theo hướng được chỉ định bởi đối tượng địa lý. Khi đến một lá, dự đoán chính là nhãn gắn với lá đó (phổ biến nhất trong số các mẫu gắn với lá đó trong quá trình huấn luyện).

Đây là cách chúng tôi đưa ra dự đoán bằng cách sử dụng cây quyết định đề xuất ứng dụng mà chúng tôi đã xây dựng trước đó. Khi có người dùng mới đến, chúng tôi sẽ kiểm tra độ tuổi cũng như nền tảng của họ và thực hiện các hành động sau:

- Nếu người dùng còn trẻ thì chúng tôi khuyên họ nên sử dụng Atom Count. Nếu
- người dùng là người lớn thì chúng tôi sẽ kiểm tra nền tảng của họ. Nếu nền tảng là Android thì chúng tôi khuyên dùng Beehive Count. Nếu nền tảng là iPhone thì chúng tôi khuyên bạn nên kiểm tra Mate Mate.

sang một bên Tài liệu này chứa các thuật ngữ như *Gini Gain* và *Information Gain* khi đào tạo cây quyết định. Mức tăng Gini là sự khác biệt giữa chỉ số tạp chất Gini có trọng số của lá và chỉ số tạp chất Gini (entropy) của nút quyết định mà chúng ta đang phân tách. Theo cách tương tự, thông tin thu được là sự khác biệt giữa entropy có trọng số của lá và entropy của gốc. Cách phổ biến hơn để đào tạo cây quyết định là tối đa hóa mức tăng Gini hoặc mức tăng thông tin. Tuy nhiên, trong chương này, chúng ta huấn luyện cây quyết định bằng cách giảm thiểu chỉ số Gini có trọng số hoặc entropy có trọng số. Quá trình huấn luyện hoàn toàn giống nhau, vì chỉ số tạp chất Gini (entropy) của nút quyết định là không đổi trong suốt quá trình phân tách nút quyết định cụ thể đó.

Ngoài những câu hỏi như có/không

Trong phần “Giải pháp: Xây dựng hệ thống đề xuất ứng dụng”, chúng tôi đã học cách xây dựng cây quyết định cho một trường hợp rất cụ thể trong đó mọi tính năng đều có tính phân loại và nhị phân (có nghĩa là nó chỉ có hai lớp, chẳng hạn như nền tảng của người dùng). Tuy nhiên, gần như cùng một thuật toán hoạt động để xây dựng cây quyết định với các đặc điểm phân loại với nhiều lớp hơn (chẳng hạn như chó/mèo/chim) và thậm chí với các đặc điểm số (chẳng hạn như tuổi hoặc thu nhập trung bình). Bước chính để sửa đổi là bước chúng tôi chia tập dữ liệu và trong phần này, chúng tôi sẽ chỉ cho bạn cách thực hiện.

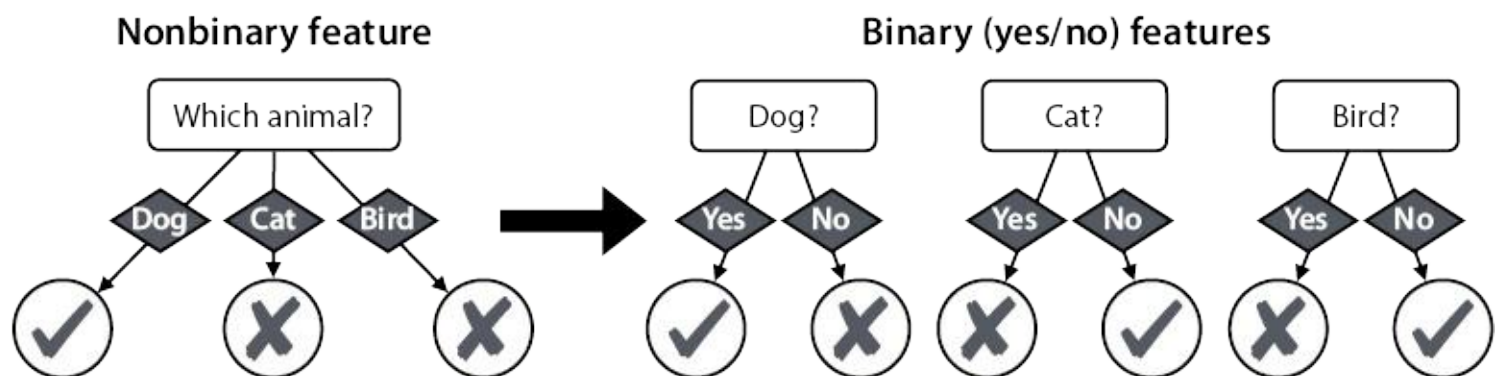
Chia dữ liệu bằng cách sử dụng phân loại phi nhị phân

các đặc điểm, chẳng hạn như chó/mèo/chim

Hãy nhớ lại rằng khi chúng ta muốn phân tách một tập dữ liệu dựa trên một đối tượng địa lý nhị phân, chúng ta chỉ cần hỏi một câu hỏi có hoặc không có dạng, “Đặc điểm đó có phải là X không?” Ví dụ: khi tính năng là nền tảng, câu hỏi cần đặt ra là “Người dùng có phải là người dùng iPhone không?” Nếu chúng tôi có một tính năng có nhiều hơn hai lớp, chúng tôi chỉ cần đặt một số câu hỏi. Ví dụ: nếu đầu vào là một con vật có thể là chó, mèo hoặc chim thì chúng tôi sẽ hỏi những câu hỏi sau:

- Con vật đó có phải là chó không?
- Con vật đó có phải là mèo không?
- Con vật đó có phải là một con chim không?

Cho dù một đối tượng có bao nhiêu lớp, chúng ta có thể chia nó thành nhiều câu hỏi nhị phân (hình 9.18).



Hình 9.18 Khi chúng tôi có một đối tượng địa lý không nhị phân, chẳng hạn như một đối tượng có ba hoặc nhiều danh mục có thể, thay vào đó, chúng tôi biến nó thành một số đối tượng địa lý nhị phân (có hoặc không), một đối tượng cho mỗi danh mục. Ví dụ: nếu đối tượng địa lý là một con chó thì câu trả lời cho ba câu hỏi “Đây có phải là một con chó không?”, “Có phải là một con mèo không?” và “Có phải là một con chim không?” là “có”, “không” và “không”.

Mỗi câu hỏi phân chia dữ liệu theo một cách khác nhau. Để tìm ra câu hỏi nào trong ba câu hỏi mang lại cho chúng ta sự phân chia tốt nhất, chúng ta sử dụng các phương pháp tương tự như trong phần “Bước đầu tiên để xây dựng mô hình”: độ chính xác, chỉ số Gini hoặc entropy. Quá trình biến một đối tượng phân loại không nhị phân thành một số đối tượng nhị phân được gọi là *mã hóa một lần*. Trong phần “Biến dữ liệu phân loại thành dữ liệu số” ở chương 13, chúng ta thấy nó được sử dụng trong tập dữ liệu thực.

Chia dữ liệu bằng các tính năng liên tục, chẳng hạn như

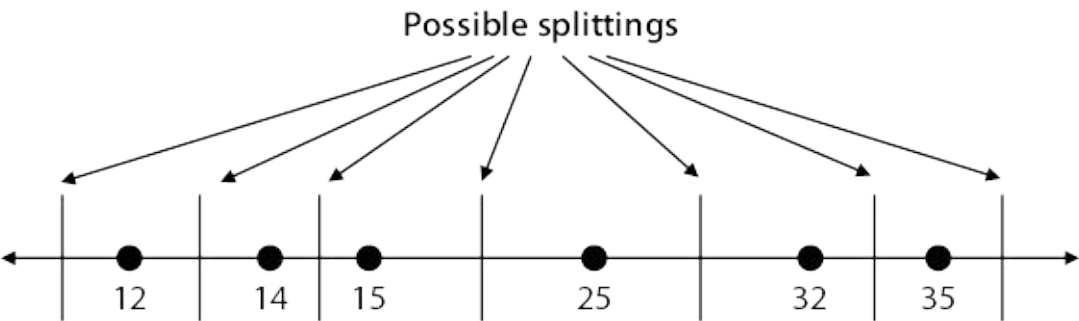
theo độ tuổi

Hãy nhớ lại rằng trước khi chúng tôi đơn giản hóa tập dữ liệu của mình, tính năng “tuổi” chứa các số. Hãy quay lại bảng ban đầu của chúng ta và xây dựng cây quyết định ở đó (bảng 9.3).

Bảng 9.3 Tập dữ liệu đề xuất ứng dụng ban đầu của chúng tôi với nền tảng và độ tuổi (bằng số) của người dùng.Tương tự như bảng 9.1.

Nền tảng	Age	App
iPhone	15	Số nguyên tử
iPhone	25	Kiểm tra bạn đời
Android	32	Công cụ tìm tổ ong
iPhone	35	Kiểm tra bạn đời
Android	12	Số nguyên tử
Android	14	Số nguyên tử

Ý tưởng là biến cột Tuổi thành một số câu hỏi có dạng “Người dùng có trẻ hơn X không?” hoặc “Người dùng có lớn tuổi hơn X không?” Có vẻ như chúng ta có vô số câu hỏi để hỏi vì có vô số con số, nhưng hãy lưu ý rằng nhiều câu hỏi trong số này phân chia dữ liệu theo cùng một cách. Ví dụ: hỏi "Người dùng có dưới 20 tuổi không?" và “Người dùng có trẻ hơn 21 tuổi không” mang lại cho chúng tôi sự phân chia tương tự. Trong thực tế, chỉ có bảy cách chia có thể xảy ra, như được minh họa trong hình 9.19.



Hình 9.19 Đồ họa về bảy cách có thể phân chia người dùng theo độ tuổi.Lưu ý rằng việc chúng ta đặt điểm cắt ở đâu không quan trọng, miễn là chúng nằm giữa các độ tuổi liên tiếp (ngoại trừ điểm cắt đầu tiên và điểm cắt cuối cùng).

Theo quy ước, chúng ta sẽ chọn trung điểm giữa các độ tuổi liên tiếp làm độ tuổi phân chia.Đối với các điểm cuối, chúng ta có thể chọn bất kỳ giá trị ngẫu nhiên nào nằm ngoài khoảng.Vì vậy, chúng ta có bảy câu hỏi có thể chia dữ liệu thành hai bộ, như trong bảng 9.4.Trong bảng này, chúng tôi cũng đã tính toán độ chính xác, chỉ số tạp chất Gini và entropy của từng phân tách.

Lưu ý rằng câu hỏi thứ tư (“Người dùng có trẻ hơn 20 không?”) cho độ chính xác cao nhất, chỉ số Gini có trọng số thấp nhất và entropy có trọng số thấp nhất và do đó, đây là cách phân chia tốt nhất có thể được thực hiện bằng cách sử dụng đặc điểm “tuổi”.

Bảng 9.4 Chúng ta có thể chọn bảy câu hỏi khả thi, mỗi câu hỏi có cách phân chia tương ứng. Trong tập đầu tiên, chúng tôi xếp những người dùng trẻ hơn ngưỡng và trong tập thứ hai, những người lớn hơn ngưỡng.

Câu hỏi	Bộ đầu tiên (có)	Bộ thứ hai (không)	Nhãn	Độ chính xác có trọng số	Chỉ số tạp chất Gini có trọng số	Entropy có trọng số
Người dùng có trẻ hơn 7 tuổi không?	trống	12, 14, 15, 25, 32, 35	{}, {A,A,A,C,B,C}	3/6	0.611	1.459
Người dùng có dưới 13 tuổi không?	12	14, 15, 25, 32, 35	{A}, {A,A,C,B,C}	3/6	0.533	1.268
Người dùng có trẻ hơn 14,5 không?	12, 14	15, 25, 32, 35	{A,A} {A,C,B,C}	4/6	0.417	1.0
Người dùng có dưới 20 tuổi không?	12, 14, 15	25, 32, 35	{A,A,A}, {C,B,C}	5/6	0.222	0.459
Người dùng có trẻ hơn 28,5 không?	12, 14, 15, 25	32, 35	{A,A,A,C}, {B,C}	4/6	0.416	0.874
Người dùng có trẻ hơn 33,5 không?	12, 14, 15, 25, 32	35	{A,A,A,C,B}, {C}	4/6	0.467	1.145
Người dùng có trẻ hơn 100 không?	12, 14, 15, 25, 32, 35	trống	{A,A,A,C,B,C}, {}	3/6	0.611	1/459

Thực hiện các phép tính trong bảng và xác minh rằng bạn nhận được câu trả lời tương tự. Toàn bộ cách tính các chỉ số Gini này có trong sổ ghi chép sau:
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Ddecision_Trees/G

Để rõ ràng, chúng ta hãy thực hiện các phép tính về độ chính xác, chỉ số tạp chất Gini có trọng số và entropy có trọng số cho câu hỏi thứ ba.Lưu ý rằng câu hỏi này chia dữ liệu thành hai bộ sau:

- **Bộ 1** (nhỏ hơn 14,5) Độ tuổi: 12, 14
Nhãn: {A, A}
- **Bộ 2** (14,5 trở lên): Độ tuổi: 15, 25, 32, 25
Nhãn: {A, C, B, C}
-
-

Tính toán chính xác

Nhãn phổ biến nhất trong bộ 1 là “A” và trong bộ 2 là “C”, vì vậy đây là những dự đoán mà chúng tôi sẽ đưa ra cho từng lá tương ứng. Trong tập 1, mọi phần tử đều được dự đoán chính xác và trong tập 2, chỉ có hai phần tử được dự đoán chính xác. Do đó, gốc quyết định này đúng ở 4 trên 6 điểm dữ liệu, với độ chính xác là $4/6 = 0,667$.

Đối với hai phép tính tiếp theo, hãy chú ý những điều sau:

- Tập 1 là thuần túy (tất cả các nhãn của nó đều giống nhau), do đó chỉ số tạp chất Gini và entropy của nó đều bằng 0. Trong tập 2, tỷ lệ của các phần tử có nhãn “A”, “B” và “C” lần lượt là 1/4, 1/4 và 2/4 = 1/2.

Tính chỉ số tạp chất GINI trọng lượng

Chỉ số tạp chất Gini của tập {A, A} là 0.

$$1 - \left(\frac{1}{4}\right)^2 - \left(\frac{1}{4}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.625$$

Chỉ số tạp chất Gini của tập {A, C, B, C} là

$$\frac{2}{6} \cdot 0 + \frac{4}{6} \cdot 0.625 = 0.417$$

Giá trị trung bình có trọng số của hai chỉ số tạp chất Gini là

Tính toán chính xác

Entropy của tập {A, A} là 0.

$$-\frac{1}{4} \log_2 \left(\frac{1}{4}\right) - \frac{1}{4} \log_2 \left(\frac{1}{4}\right) - \frac{1}{2} \log_2 \left(\frac{1}{2}\right) = 1.5$$

Entropy của tập {A, C, B, C} là

$$\frac{2}{6} \cdot 0 + \frac{4}{6} \cdot 1.5 = 1.0$$

Trung bình có trọng số của hai entropy là

Một đối tượng số trở thành một chuỗi câu hỏi có hoặc không, có thể đo lường và so sánh với các câu hỏi có hoặc không khác đến từ các đối tượng địa lý khác, để chọn câu hỏi tốt nhất cho nút quyết định đó.

ngoài ra Mô hình đề xuất ứng dụng này rất nhỏ nên chúng tôi có thể thực hiện tất cả bằng tay. Tuy nhiên, để xem nó ở dạng mã, vui lòng kiểm tra sổ ghi chép này:

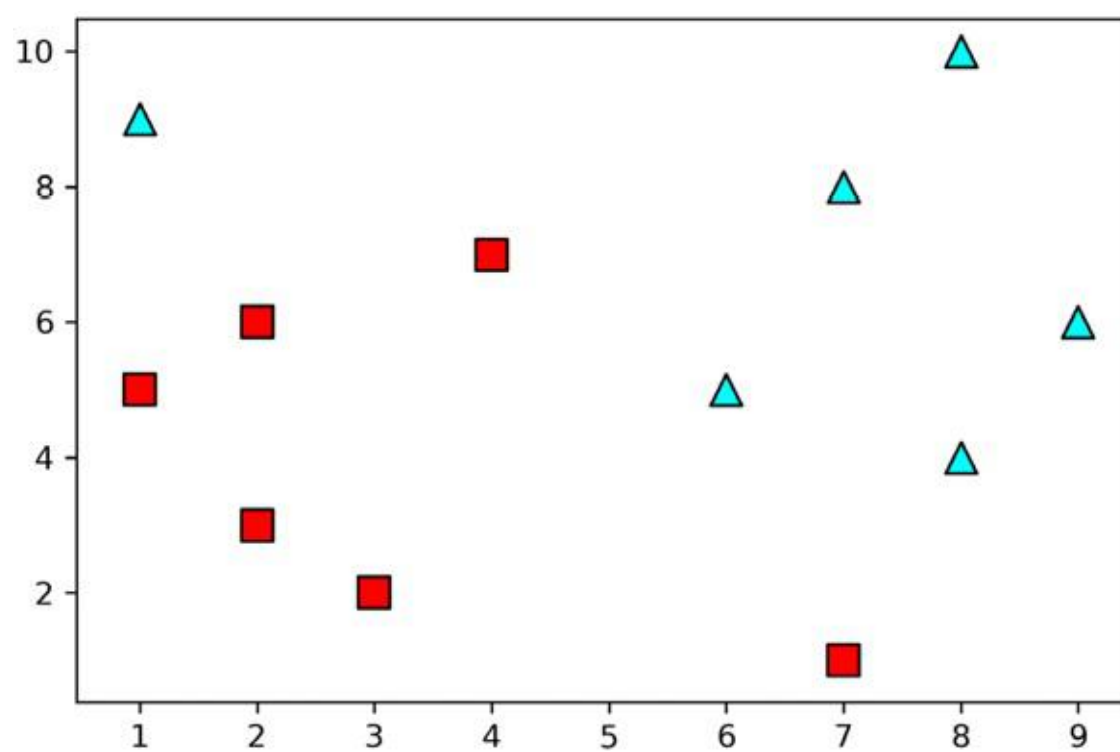
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/A

Sổ ghi chép sử dụng gói Scikit-Learn mà chúng tôi giới thiệu chi tiết hơn trong phần “Sử dụng Scikit-Learn để xây dựng cây quyết định”.

Ranh giới đồ họa của cây quyết định

Trong phần này, tôi chỉ cho bạn hai điều: cách xây dựng cây quyết định về mặt hình học (theo hai chiều) và cách viết mã cây quyết định trong gói học máy phổ biến Scikit-Learn.

Hãy nhớ lại rằng trong các mô hình phân loại, chẳng hạn như perceptron (chương 5) hoặc bộ phân loại logistic (chương 6), chúng ta đã vẽ ranh giới của mô hình phân tách các điểm bằng nhãn 0 và 1, và hóa ra nó là một đường thẳng. Ranh giới của cây quyết định cũng đẹp và khi dữ liệu là hai chiều, nó được hình thành bởi sự kết hợp của các đường dọc và ngang. Trong phần này, chúng tôi minh họa điều này bằng một ví dụ. Hãy xem xét tập dữ liệu trong hình 9.20, trong đó các điểm có nhãn 1 là hình tam giác và các điểm có nhãn 0 là hình vuông. Trục ngang và trục dọc lần lượt được gọi là x_0 và x_1 .



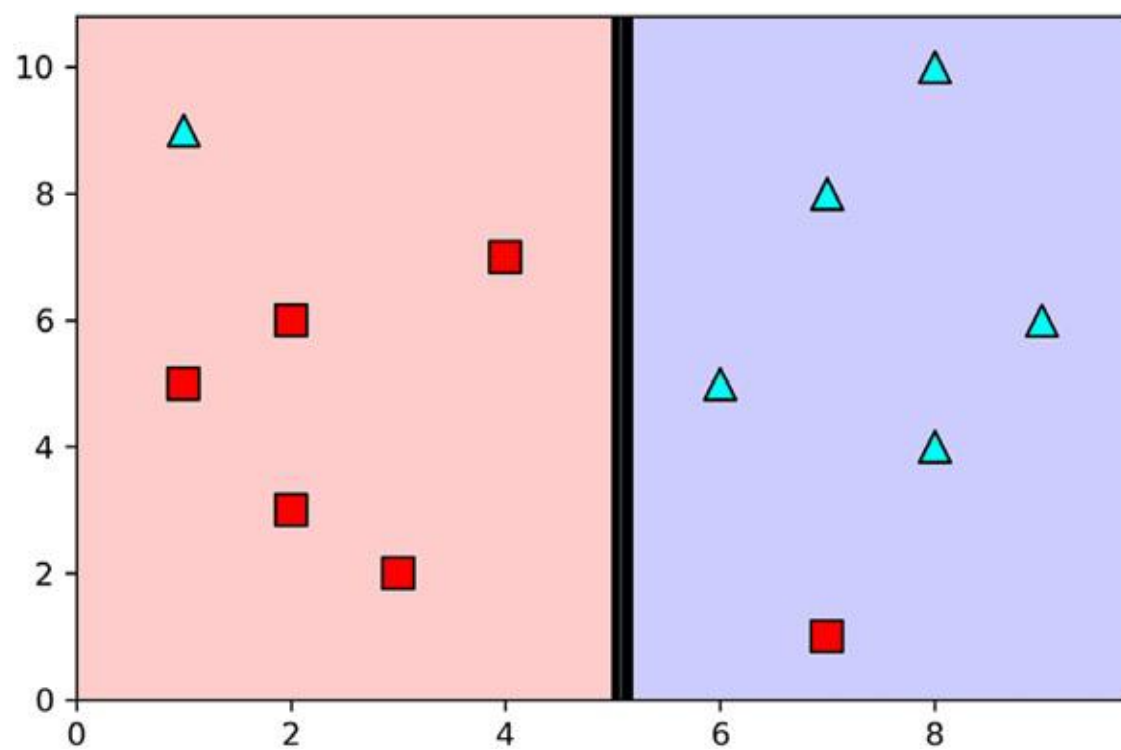
Hình 9.20 Một tập dữ liệu có hai đặc điểm (x_0 và x_1) và hai nhãn (tam giác và Square) trong đó chúng ta sẽ huấn luyện cây quyết định

Nếu bạn phải chia tập dữ liệu này chỉ bằng một dòng ngang hoặc dọc, bạn sẽ chọn dòng nào? Có thể có nhiều dòng khác nhau, tùy theo tiêu chí bạn sẽ sử dụng để đo lường hiệu quả của giải pháp. Hãy tiếp tục và chọn một đường thẳng đứng tại $x_0 = 5$. Điều này để lại hầu hết các hình tam giác ở bên phải của nó và chủ yếu là hình vuông

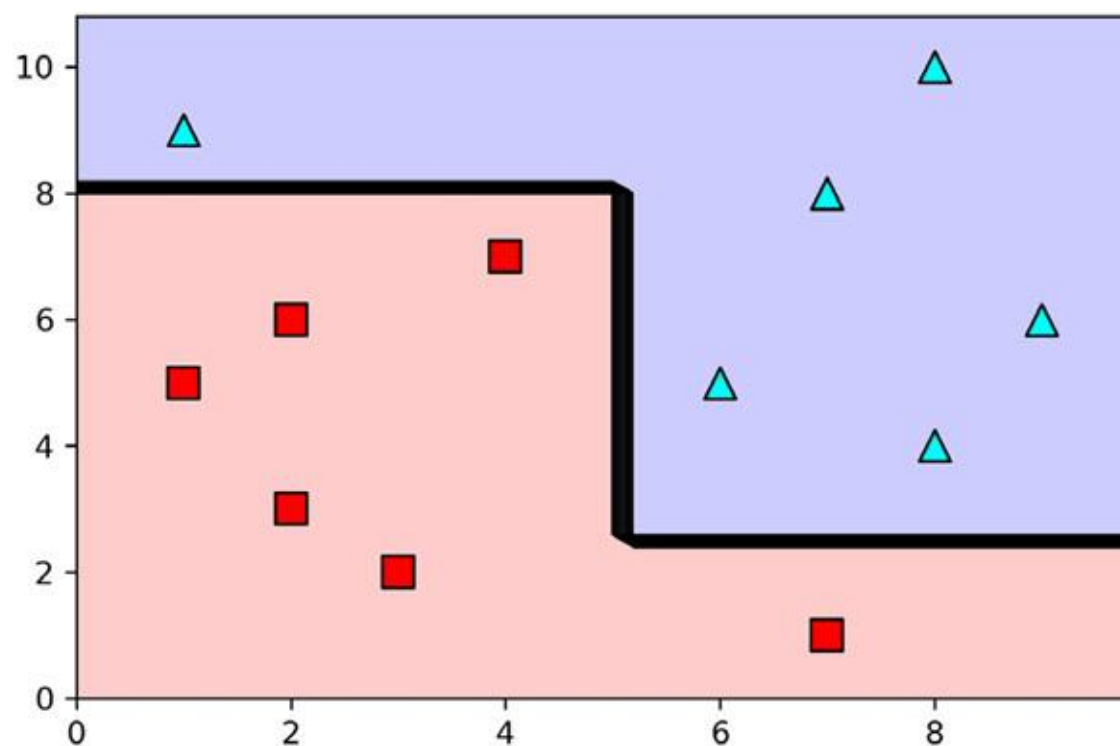
bên trái của nó, ngoại trừ hai điểm bị phân loại sai, một hình vuông và một hình tam giác (hình 9.21). Hãy thử kiểm tra tất cả các đường dọc và ngang có thể có khác, so sánh chúng bằng số liệu yêu thích của bạn (độ chính xác, chỉ số Gini và entropy) và xác minh rằng đây là đường phân chia điểm tốt nhất.

Bây giờ chúng ta hãy xem xét từng nửa riêng biệt. Lần này dễ dàng nhận thấy hai đường ngang tại $x_1 = 8$ và $x_1 = 2.5$ sẽ thực hiện công việc ở bên trái và bên phải,

tương ứng. Những dòng này chia hoàn toàn tập dữ liệu thành hình vuông và hình tam giác. Hình 9.22 minh họa kết quả.



Hình 9.21 Nếu chúng ta chỉ phải sử dụng một đường dọc hoặc ngang để phân loại tập dữ liệu này theo cách tốt nhất có thể, chúng ta sẽ sử dụng đường nào? Dựa trên độ chính xác, trình phân loại tốt nhất là đường thẳng đứng tại $x_0 = 5$, trong đó chúng tôi phân loại mọi thứ ở bên phải là hình tam giác và mọi thứ ở bên trái là hình vuông. Trình phân loại đơn giản này phân loại chính xác 8 trên 10 điểm, với độ chính xác là 0,8.



Hình 9.22 Bộ phân loại trong hình 9.21 để lại cho chúng ta hai bộ dữ liệu, mỗi bộ ở hai bên của đường thẳng đứng. Nếu chúng ta phải phân loại từng đường thẳng, lại sử dụng một đường dọc hoặc đường ngang, chúng ta sẽ chọn đường nào? Lựa chọn tốt nhất là các đường ngang tại $x_1 = 8$ và $x_1 = 2.5$, như hình minh họa.

Những gì chúng tôi đã làm ở đây là xây dựng một cây quyết định. Ở mọi giai đoạn, chúng tôi chọn từng tính năng trong số hai tính năng (x_0 và x_1) và chọn ngưỡng phân chia dữ liệu của chúng tôi tốt nhất. TRONG thực tế, trong tiêu mục tiếp theo, chúng tôi sử dụng Scikit-Learn để xây dựng cây quyết định tương tự trên tập dữ liệu này.

Sử dụng Scikit-Learn để xây dựng cây quyết định

Trong phần này, chúng ta tìm hiểu cách sử dụng gói máy học phổ biến có tên Scikit-Learn (viết tắt sklearn) để xây dựng cây quyết định. Mã cho phần này như sau:

- **Sổ tay:** Graphical_example.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_

Chúng tôi bắt đầu bằng cách tải tập dữ liệu dưới dạng Pandas DataFrame được gọi là `dataset` (được giới thiệu ở chương 8), với các dòng mã sau:

```
import pandas as pd
dataset = pd.DataFrame({
    'x_0': [7, 3, 2, 1, 2, 4, 1, 8, 6, 7, 8, 9],
    'x_1': [1, 2, 3, 5, 6, 7, 9, 10, 5, 8, 4, 6],
```

```
'y': [0,0,0,0,0,0,1,1,1,1,1,1]})
```

Bây giờ chúng tôi tách các tính năng khỏi nhãn như được hiển thị ở đây:

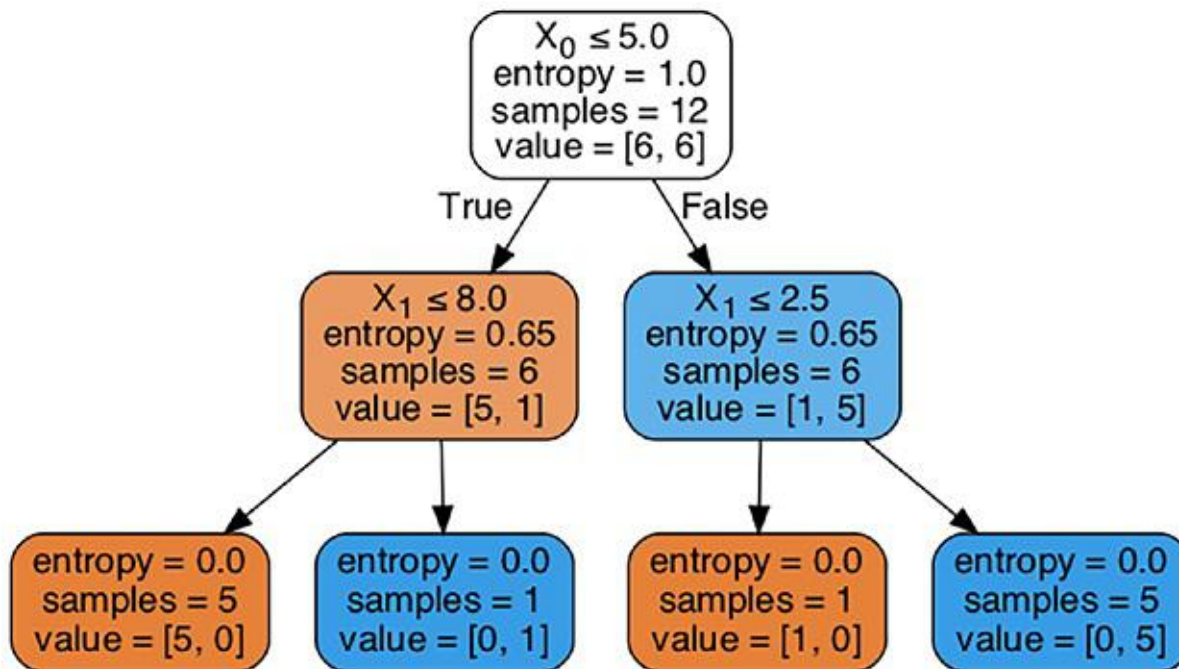
```
features = dataset[['x_0', 'x_1']]
labels = dataset['y']
```

Để xây dựng cây quyết định, chúng ta tạo một đối tượng và sử dụng hàm `fit` sau:

```
fit
```

```
decision_tree = DecisionTreeClassifier()
decision_tree.fit(features, labels)
```

Chúng ta thu được đồ thị của cây như trong hình 9.23 bằng cách sử dụng `display_tree` hoạt động trong tệp `utils.py`.



Hình 9.23 Cây quyết định kết quả có độ sâu 2 tương ứng với ranh giới trong hình 9.22. Nó có ba nút và bốn lá.

Lưu ý rằng cây trong hình 9.23 tương ứng chính xác với ranh giới trong hình 9.22. Nút gốc tương ứng với đường thẳng đứng đầu tiên tại $x_0 = 5$, có các điểm

ở mỗi bên của đường tương ứng với hai nhánh. Hai đường ngang tại $x_1 = 8.0$ và $x_1 = 2.5$ ở nửa bên trái và bên phải của ô tương ứng với hai nhánh. Hơn nữa, tại mỗi nút, chúng tôi có thông tin sau:

- **Gini:** chỉ số tạp chất Gini của các nhãn tại nút đó
- **Mẫu:** số lượng điểm dữ liệu (mẫu) tương ứng với nút đó

- **Giá trị:** số lượng điểm dữ liệu của mỗi nhãn trong số hai nhãn tại nút đó

Như bạn có thể thấy, cây này đã được huấn luyện bằng chỉ mục Gini, đây là chỉ mục mặc định trong Scikit-Learn. Để huấn luyện nó bằng entropy, chúng ta có thể chỉ định nó khi xây dựng đối tượng, như sau:

```
decision_tree = DecisionTreeClassifier(criterion='entropy')
```

Chúng ta có thể chỉ định nhiều tiêu chí tham số hơn khi huấn luyện cây, chúng ta sẽ thấy điều này trong phần tiếp theo với một ví dụ lớn hơn nhiều.

Ứng dụng thực tế: Lập mô hình tuyển sinh với Scikit-Learn

Trong phần này, chúng tôi sử dụng cây quyết định để xây dựng mô hình dự đoán tuyển sinh vào các trường sau đại học. Có thể tìm thấy tập dữ liệu trong Kaggle (xem phụ lục C để biết liên kết). Như trong phần “Ranh giới đồ họa của cây quyết định”, chúng ta sẽ sử dụng Scikit-Learn để huấn luyện cây quyết định và Pandas để xử lý tập dữ liệu. Mã cho phần này như sau:

- **Sổ tay:** University_admissions.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_
- **Tập dữ liệu:** Entry_Predict.csv

Bộ dữ liệu có các tính năng sau:

- **Điểm GRE:** một số trên 340 **Điểm TOEFL:** một số trên 120 **Xếp hạng đại học:** một số từ 1 đến 5 **Tuyên bố về mục đích mạnh mẽ (SOP):** một số từ 1 đến 5 **Điểm trung bình đại học (CGPA):** một số từ 1 đến 10 **Thư giới thiệu sức mạnh (LOR):** số từ 1 đến 5 **Kinh nghiệm nghiên cứu:** Biến Boolean (0 hoặc 1)
-
-
-

Các nhãn trên tập dữ liệu là cơ hội được nhận, là một số từ 0 đến 1. Để có nhãn nhị phân, chúng tôi sẽ coi mọi học sinh có cơ hội 0,75 trở lên là “được nhận” và bất kỳ học sinh nào khác là “không được nhận”.

Mã để tải tập dữ liệu vào Pandas DataFrame và thực hiện việc này

preprocessing step is shown next:

```
import pandas as pd
data = pd.read_csv('Admission_Predict.csv', index_col=0)
data['Admitted'] = data['Chance of Admit'] >= 0.75
data = data.drop(['Chance of Admit'], axis=1)
```

Một số hàng đầu tiên của tập dữ liệu kết quả được hiển thị trong bảng 9.5.

Bảng 9.5 Một tập dữ liệu với 400 sinh viên và điểm số của họ trong các bài kiểm tra tiêu chuẩn, điểm số, xếp hạng đại học, thư giới thiệu, tuyên bố mục đích và thông tin về cơ hội được nhận vào trường sau đại học

Điểm GRE	Điểm TOEFL	Xếp hạng đại học	SOP	LOR	CGPA	Nghiên cứu	Đã thừa nhận
337	118	4	4.5	4.5	9.65	1	ĐÚNG VẬY
324	107	4	4.0	4.5	8.87	1	ĐÚNG VẬY
316	104	3	3.0	3.5	8.00	1	SAI
322	110	3	3.5	2.5	8.67	1	ĐÚNG VẬY
314	103	2	2.0	3.0	8.21	0	SAI

Như chúng ta đã thấy trong phần “Ranh giới đồ họa của cây quyết định”, Scikit-Learn yêu cầu chúng ta nhập các tính năng và nhãn riêng biệt.Chúng tôi sẽ xây dựng một Khung dữ liệu Pandas có tên chứa tất cả các cột ngoại trừ **Đã nhận** **features** cột và một chuỗi Pandas được gọi chỉ chứa cột **Đã nhận** **labels**.

Mã sau:

```
features = data.drop(['Admitted'], axis=1)
labels = data['Admitted']
```

Bây giờ chúng ta tạo một đối tượng (mà chúng ta gọi là `dt`) và sử dụng `DecisionTreeClassifier` phương pháp.Chúng ta sẽ huấn luyện nó bằng chỉ số Gini, như minh họa tiếp theo, vì vậy không cần thiết để chỉ định siêu tham số, nhưng hãy tiếp tục và huấn luyện nó với entropy và so sánh kết quả với kết quả chúng tôi nhận được ở đây:

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(features, labels)
```

Để đưa ra dự đoán, chúng ta có thể sử dụng hàm `predict`. Ví dụ, đây là cách chúng tôi đưa ra dự đoán cho năm học sinh đầu tiên:

```
dt.predict(features[0:5])
Output: array([ True,  True, False,  True, False])
```

Tuy nhiên, cây quyết định mà chúng ta vừa huấn luyện quá phù hợp. Một cách để thấy điều này là sử dụng hàm và nhận ra rằng nó đạt 100% trong tập huấn luyện. Trong chương này, chúng ta sẽ không thử nghiệm mô hình mà sẽ thử xây dựng một bộ thử nghiệm và xác minh rằng mô hình này phù hợp quá mức. Một cách khác để thấy sự trang bị quá mức là vẽ sơ đồ cây và chú ý rằng độ sâu của nó là 10 (xem sổ ghi chép). Trong phần tiếp theo, chúng ta tìm hiểu về một số siêu tham số giúp chúng ta ngăn chặn việc trang bị quá mức.

Đặt siêu tham số trong Scikit-Learn

Để ngăn chặn việc trang bị quá mức, chúng ta có thể sử dụng một số siêu tham số mà chúng ta đã học trong phần “Bước cuối cùng: Khi nào nên ngừng xây dựng cây và các siêu tham số khác”, chẳng hạn như sau:

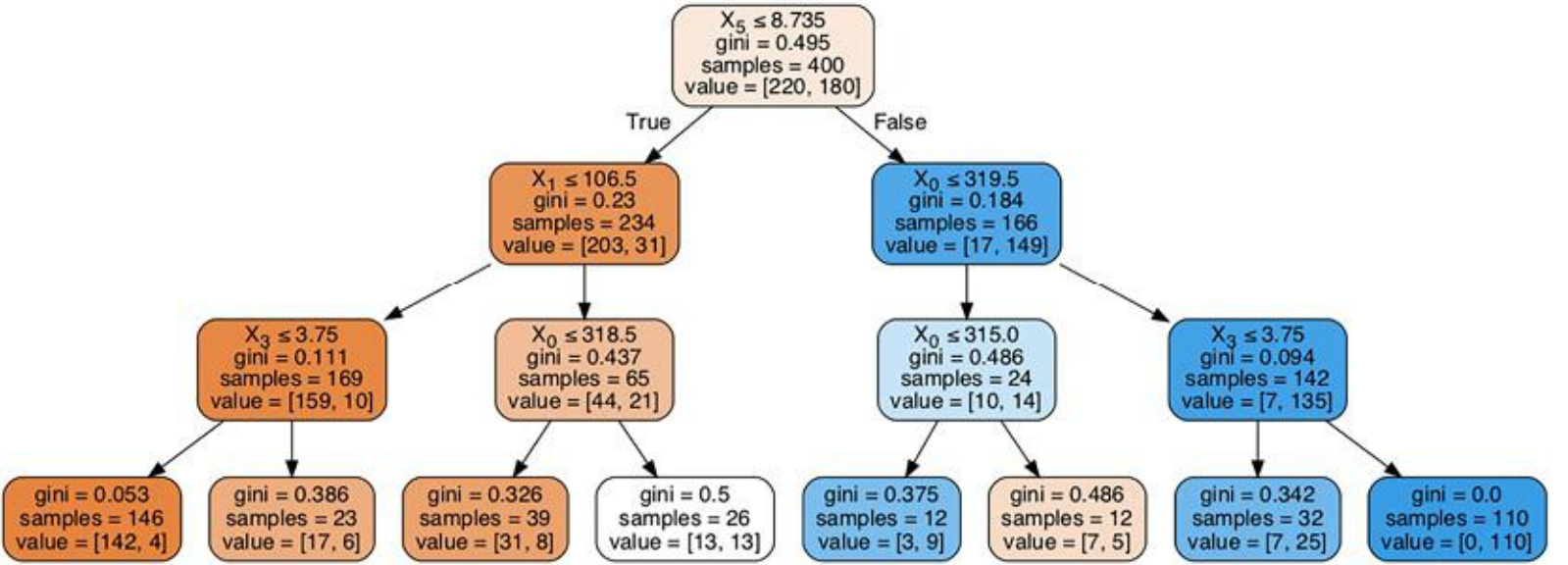
- `max_depth`: độ sâu tối đa cho phép
- `max_features`: số lượng tính năng tối đa được xem xét ở mỗi lần phân chia (hữu ích khi có quá nhiều tính năng và quá trình đào tạo mất quá nhiều thời gian). : độ giảm tạp chất phải cao hơn mức này
- `min_impurity_decrease`
ngưỡng để phân chia một nút.
- `min_impurity_split`: khi tạp chất tại một nút thấp hơn ngưỡng này, nút đó sẽ trở thành một chiếc lá. : số lượng mẫu tối thiểu cần thiết cho một nút lá
- `min_samples_leaf`
Nếu việc phân chia lá có ít hơn số lượng mẫu này thì việc phân tách sẽ không được thực hiện.
: số lượng mẫu tối thiểu cần thiết để phân chia một
- `min_samples_split`
nút.

Chơi xung quanh với các tham số này để tìm ra một mô hình tốt. Chúng tôi sẽ sử dụng như sau :

- `max_depth = 3`
- `min_samples_leaf = 10`
- `min_samples_split = 10`

```
dt_smaller = DecisionTreeClassifier(max_depth=3, min_samples_leaf=10, min_samples_split=10)
dt_smaller.fit(features, labels)
```

Cây kết quả được minh họa trong hình 9.24. Lưu ý rằng trong cây này, tất cả các cạnh bên phải tương ứng với “Sai” và bên trái là “Đúng”.



Hình 9.24 Cây quyết định có độ sâu 3 được đào tạo trong tập dữ liệu tuyển sinh sinh viên

Dự đoán được đưa ra ở mỗi lá là nhãn tương ứng với phần lớn các nút trong lá đó. Trong sổ ghi chép, mỗi nút được gán một màu, từ màu cam đến màu xanh lam. Các nút màu cam là những nút có nhiều điểm có nhãn 0 và các nút màu xanh là những nút có nhãn 1. Lưu ý rằng lá màu trắng, trong đó có cùng số điểm có nhãn 0 và 1. Đối với lá này, mọi dự đoán đều có hiệu suất như nhau. Trong trường hợp này, Scikit-Learn mặc định là lớp đầu tiên trong danh sách, trong trường hợp này là sai.

Để đưa ra dự đoán, chúng ta sử dụng hàm `predict` của đối tượng `DecisionTreeClassifier`. Ví dụ, hãy dự đoán tuyển sinh của một sinh viên với các số sau:

- Điểm GRE: 320 Điểm TOEFL:
- 110 Xếp hạng đại học: 3 SOP:
- 4.0 LOR: 3.5 CGPA: 8.9
- Nghiên cứu: 0 (không có nghiên cứu)
-
-
-

```
dt_smaller.predict([[320, 110, 3, 4.0, 3.5, 8.9, 0]])
Output: array([ True])
```

Cây dự đoán rằng học sinh sẽ được nhận vào học.

Từ cây này, chúng ta có thể suy ra những điều sau về tập dữ liệu của mình:

- Đặc điểm quan trọng nhất là cột thứ sáu (X_5), tương ứng với

CGPA, hoặc các lớp. Điểm giới hạn là 8,735 trên 10. Trên thực tế, hầu hết các dự đoán ở bên phải của nút gốc là “thừa nhận” và ở bên trái là “không thừa nhận”, điều này ngụ ý rằng CGPA là một tính năng rất mạnh. Sau tính năng này, 2 tính năng quan trọng nhất là điểm GRE (X_0) và TOEFL

- điểm (X_1), cả hai đều là bài kiểm tra tiêu chuẩn hóa. Trên thực tế, trong số những học sinh đạt điểm cao, hầu hết đều có khả năng được nhận vào học, trừ khi họ đạt điểm GRE kém, như được tính ở lá thứ sáu từ bên trái trong cây trong hình 9.24. Ngoài điểm số và các bài kiểm tra tiêu chuẩn hóa, tính năng duy nhất khác xuất hiện trong cây là SOP, hay sức mạnh của tuyên bố mục đích. Cái này nằm ở dưới cây và nó không làm thay đổi nhiều dự đoán.

Tuy nhiên, hãy nhớ lại rằng việc xây dựng cây có bản chất tham lam, cụ thể là tại mỗi điểm, nó sẽ chọn đặc điểm trên cùng. Tuy nhiên, điều này không đảm bảo rằng việc lựa chọn các tính năng là tốt nhất. Ví dụ: có thể có sự kết hợp của các đặc điểm rất mạnh nhưng không có đặc điểm nào mạnh riêng lẻ và cây có thể không nhận ra được điều này. Vì vậy, mặc dù chúng tôi đã nhận được một số thông tin về tập dữ liệu nhưng chúng tôi vẫn chưa nên loại bỏ các tính năng không có trong cây. Một thuật toán lựa chọn tính năng tốt, chẳng hạn như chính quy L1, sẽ có ích khi chọn các tính năng trong tập dữ liệu này.

Cây quyết định hồi quy

Trong hầu hết chương này, chúng ta đã sử dụng cây quyết định để phân loại, nhưng như đã đề cập trước đó, cây quyết định cũng là mô hình hồi quy tốt. Trong phần này, chúng ta xem cách xây dựng mô hình hồi quy cây quyết định. Mã cho phần này như sau:

- **Sổ tay:** Regression_decision_tree.ipynb
https://github.com/luiguiserrano/manning/blob/master/Chapter_9_Decision_

Hãy xem xét vấn đề sau: chúng tôi có một ứng dụng và chúng tôi muốn dự đoán mức độ tương tác của người dùng dựa trên số ngày họ sử dụng ứng dụng đó mỗi tuần. Tính năng duy nhất chúng tôi có là độ tuổi của người dùng. Tập dữ liệu được hiển thị trong bảng 9.6 và sơ đồ của nó là trong hình 9.25.

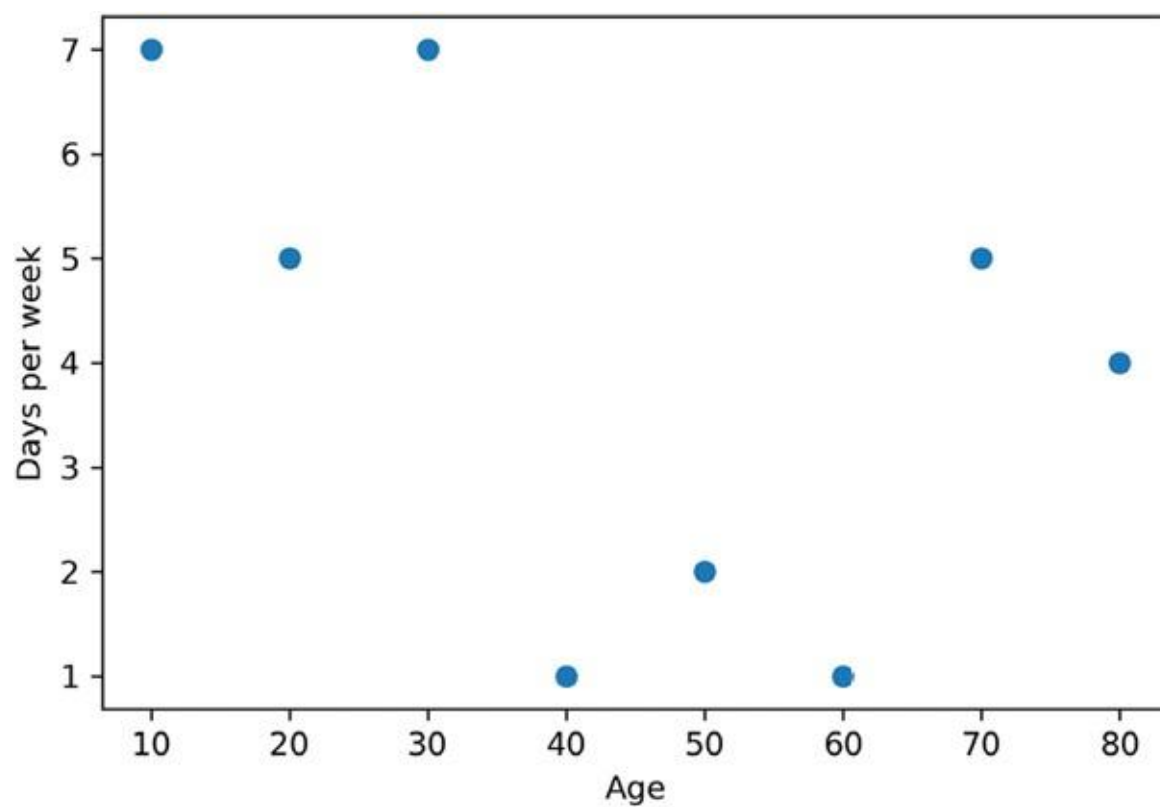
Bảng 9.6 Một tập dữ liệu nhỏ có 8 người dùng, độ tuổi và của họ

engagement with our app. The engagement is measured in the number of days when they opened the app in one week.

Age	Engagement
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

From this dataset, it seems that we have three clusters of users. The young users (ages 10, 20, 30) use the app a lot, the middle-aged users (ages 40, 50, 60) don't use it very much, and the older users (ages 70, 80) use it sometimes. Thus, a prediction like this one would make sense:

- If the user is 34 years old or younger, the engagement is 6 days per week.
- If the user is between 35 and 64, the engagement is 1 day per week.
- If the user is 65 or older, the engagement is 3.5 days per week.



Hình 9.25 Biểu đồ của tập dữ liệu trong bảng 9.6, trong đó trục ngang tương ứng với độ tuổi của người dùng và trục tung tương ứng với số ngày mỗi tuần mà họ tương tác với ứng dụng

Các dự đoán của cây quyết định hồi quy trông tương tự như sau, vì cây quyết định chia người dùng của chúng ta thành các nhóm và dự đoán một giá trị cố định cho mỗi nhóm. Cách phân chia người dùng là sử dụng các tính năng, giống hệt như chúng tôi đã làm đối với các vấn đề phân loại.

Thật may mắn cho chúng tôi, thuật toán được sử dụng để huấn luyện cây quyết định hồi quy rất giống với thuật toán chúng tôi đã sử dụng để huấn luyện cây quyết định phân loại. Điểm khác biệt duy nhất là đối với cây phân loại, chúng tôi sử dụng độ chính xác, chỉ số Gini hoặc entropy và đối với cây hồi quy, chúng tôi sử dụng sai số bình phương trung bình (MSE). Sai số bình phương trung bình nghe có vẻ quen thuộc—chúng tôi đã sử dụng nó để huấn luyện các mô hình hồi quy tuyến tính trong phần “Làm cách nào để đo lường kết quả của chúng tôi? Hàm lỗi” ở chương 3.

Trước khi đi vào thuật toán, chúng ta hãy suy nghĩ về nó một cách khái niệm. Hãy tưởng tượng rằng bạn phải làm cho một đường thẳng càng gần với tập dữ liệu trong hình 9.25 càng tốt. Nhưng có một nhược điểm – đường dây phải nằm ngang. Chúng ta nên đặt đường ngang này ở đâu? Sẽ hợp lý nếu đặt nó ở “giữa” của tập dữ liệu—nói cách khác, ở độ cao bằng mức trung bình của các nhãn, là 4. Đó là một mô hình phân loại rất đơn giản, gán cho mọi điểm cùng một dự đoán là 4.

Bây giờ, chúng ta hãy đi xa hơn một chút. Nếu phải sử dụng hai đoạn ngang thì làm thế nào

chúng ta làm cho chúng càng gần với dữ liệu càng tốt? Chúng ta có thể có một số dự đoán, chẳng hạn như đặt thanh cao cho số điểm ở bên trái là 35 và thanh thấp ở bên phải là 35. Điều đó thể hiện một gốc quyết định đặt câu hỏi, “Bạn có trẻ hơn 35 không?” và đưa ra dự đoán dựa trên cách người dùng trả lời câu hỏi đó.

Điều gì sẽ xảy ra nếu chúng ta có thể chia mỗi đoạn trong số hai đoạn ngang này thành hai đoạn nữa—chúng ta nên đặt chúng ở đâu? Chúng tôi có thể tiếp tục theo dõi quá trình này cho đến khi phân tích được người dùng

thành nhiều nhóm trong đó nhãn của chúng rất giống nhau. Sau đó, chúng tôi dự đoán nhãn trung bình cho tất cả người dùng trong nhóm đó.

Quá trình chúng ta vừa thực hiện là quá trình huấn luyện cây quyết định hồi quy. Bây giờ chúng ta hãy trang trọng hơn. Hãy nhớ lại rằng khi một đối tượng là số, chúng ta xem xét tất cả các cách có thể để phân tách nó. Do đó, các cách khả thi để phân chia đặc điểm độ tuổi đang sử dụng, chẳng hạn như các ngưỡng sau: 15, 25, 35, 45, 55, 65 và 75. Mỗi ngưỡng này cung cấp cho chúng ta hai tập dữ liệu nhỏ hơn mà chúng tôi gọi là tập dữ liệu bên trái và tập dữ liệu bên phải. Bây giờ chúng ta thực hiện các bước sau:

1. Đối với mỗi tập dữ liệu nhỏ hơn, chúng tôi dự đoán giá trị trung bình của các nhãn. 2. Chúng tôi tính sai số bình phương trung bình của dự đoán. 3. Chúng tôi chọn điểm cắt mang lại cho chúng tôi sai số bình phương nhỏ nhất.

Ví dụ: nếu ngưỡng của chúng tôi là 65 thì hai tập dữ liệu như sau:

- **Tập dữ liệu bên trái:** người dùng dưới 65 tuổi. Các nhãn là {7, 5, 7, 1, 2, 1}. **Tập dữ liệu bên phải:** người dùng 65 tuổi trở lên. Các nhãn là {5, 4}.

Đối với mỗi tập dữ liệu, chúng tôi dự đoán mức trung bình của các nhãn là 3,833 cho nhãn bên trái và 4,5 cho nhãn bên phải. Do đó, dự đoán cho sáu người dùng đầu tiên là 3,833 và cho hai người dùng cuối là 4,5. Bây giờ, chúng ta tính MSE như sau:

$$\begin{aligned}MSE &= \frac{1}{8} [(7 - 3.833)^2 + (5 - 3.833)^2 + (7 - 3.833)^2 + (1 - 3.833)^2 + (2 - 3.833)^2 \\ &\quad + (1 - 3.833)^2 + (5 - 4.5)^2 + (4 - 4.5)^2] = 5.167\end{aligned}$$

Trong bảng 9.7, chúng ta có thể thấy các giá trị thu được cho từng điểm cắt có thể có. Các tính toán đầy đủ nằm ở cuối sổ ghi chép cho phần này.

ngắt. Mỗi điểm cắt sẽ chia tập dữ liệu thành hai tập dữ liệu nhỏ hơn và đối với mỗi tập dữ liệu trong số đó, dự đoán được đưa ra bằng mức trung bình của các nhãn. Sai số bình phương trung bình (MSE) được tính bằng giá trị trung bình của bình phương của sự khác biệt giữa nhãn và dự đoán.Lưu ý rằng phép phân tách có MSE nhỏ nhất đạt được với điểm cắt là 35. Điều này mang lại cho chúng ta nút gốc trong cây quyết định.

Ngắt	Nhãn còn lại được đặt	Đặt đúng nhãn	Dự đoán được đặt bên trái	Đặt quyền dự đoán	MSE
0	{}	{7,5,7,1,2,1,5,4}	Không có	4.0	5.25
15	{7}	{5,7,1,2,1,5,4}	7.0	3.571	3.964
25	{7,5}	{7,1,2,1,5,4}	6.0	3.333	3.917
35	{7,5,7}	{1,2,1,5,4}	6.333	2.6	1.983
45	{7,5,7,1}	{2,1,5,4}	5.0	3.0	4.25
55	{7,5,7,1,2}	{1,5,4}	4.4	3.333	4.983
65	{7,5,7,1,2,1}	{5,4}	3.833	4.5	5.167
75	{7,5,7,1,2,1,5}	{4}	4.0	4.0	5.25
100	{7,5,7,1,2,1,5,4}	{}	4.0	không có	5.25

Ngưỡng tốt nhất là ở tuổi 35, vì nó cho chúng ta dự đoán với sai số bình phương trung bình nhỏ nhất.Vì vậy, chúng tôi đã xây dựng nút quyết định đầu tiên trong cây quyết định hồi quy của mình.Các bước tiếp theo là tiếp tục phân chia đệ quy các tập dữ liệu bên trái và bên phải theo cùng một cách.Thay vì làm bằng tay, chúng ta sẽ sử dụng Scikit-Learn như trước.

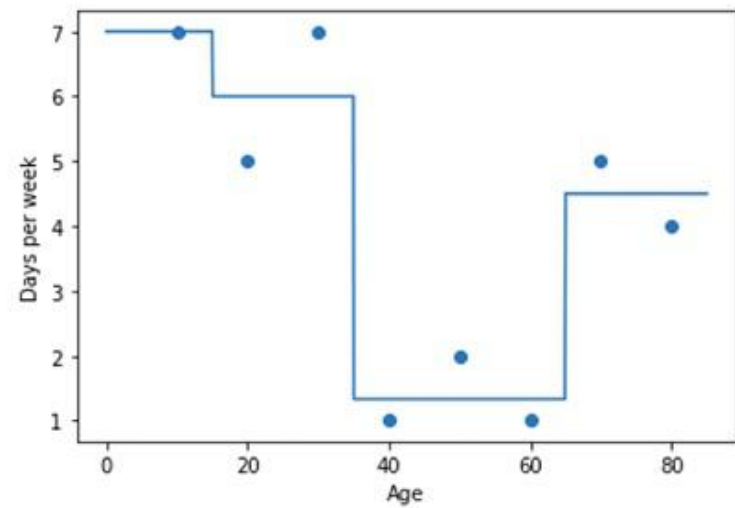
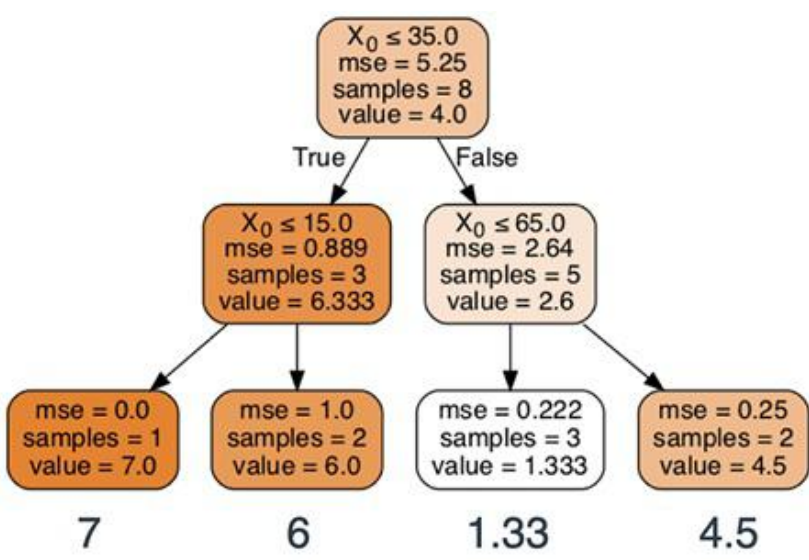
Đầu tiên, chúng tôi xác định các tính năng và nhãn của chúng tôi.Chúng ta có thể sử dụng mảng cho việc này, như được hiển thị tiếp theo:

```
features = [[10],[20],[30],[40],[50],[60],[70],[80]]
labels = [7,5,7,1,2,1,5,4]
```

Bây giờ, chúng ta xây dựng cây quyết định hồi quy có độ sâu tối đa 2 bằng cách sử dụng đối tượng như sau:

```
from sklearn.tree import DecisionTreeRegressor
dt_regressor = DecisionTreeRegressor(max_depth=2)
dt_regressor.fit(features, labels)
```

Cây quyết định kết quả được thể hiện trong hình 9.26.Ngưỡng đầu tiên là ở mức 35, như chúng tôi đã tính toán.Hai điểm cắt tiếp theo là 15 và 65. Ở bên phải hình 9.26, chúng ta cũng có thể thấy các dự đoán cho từng tập hợp con trong số bốn tập con dữ liệu thu được này.



Hình 9.26 Bên trái: Cây quyết định thu được trong Scikit-Learn. Cây này có ba nút quyết định và bốn lá. Phải: Biểu đồ các dự đoán được đưa ra bởi cây quyết định này. Lưu ý rằng các điểm cắt ở độ tuổi 35, 15 và 65, tương ứng với các nút quyết định trong cây. Các dự đoán là 7, 6, 1,33 và 4,5 tương ứng với các lá trên cây.

Ứng dụng

Cây quyết định có nhiều ứng dụng hữu ích trong đời sống. Một tính năng đặc biệt của cây quyết định là ngoài việc dự đoán, chúng còn cung cấp cho chúng ta rất nhiều thông tin về dữ liệu của chúng ta vì chúng sắp xếp dữ liệu theo cấu trúc phân cấp. Nhiều khi, thông tin này có giá trị ngang bằng hoặc thậm chí cao hơn khả năng đưa ra dự đoán. Trong phần này, chúng ta thấy một số ví dụ về cây quyết định được sử dụng trong đời thực trong các lĩnh vực sau:

- Hệ thống khuyến nghị chăm sóc
- sức khỏe

Cây quyết định được sử dụng rộng rãi trong chăm sóc sức khỏe

Cây quyết định được sử dụng rộng rãi trong y học, không chỉ để đưa ra dự đoán mà còn để xác định các đặc điểm mang tính quyết định trong dự đoán. Bạn có thể tưởng tượng rằng trong y học, một chiếc hộp đen ghi “bệnh nhân bị bệnh” hay “bệnh nhân khỏe mạnh” là chưa đủ. Tuy nhiên, cây quyết định chứa rất nhiều thông tin về lý do đưa ra dự đoán. Bệnh nhân có thể bị bệnh dựa trên các triệu chứng, tiền sử bệnh tật của gia đình, thói quen hoặc nhiều yếu tố khác.

Cây quyết định rất hữu ích trong các hệ thống khuyến nghị

Trong các hệ thống khuyến nghị, cây quyết định cũng rất hữu ích. Một trong những vấn đề về hệ thống đề xuất nổi tiếng nhất, giải thưởng Netflix, đã được giải nhờ sự trợ giúp của cây quyết định. Năm 2006, Netflix tổ chức một cuộc thi liên quan đến việc xây dựng hệ thống đề xuất tốt nhất có thể để dự đoán xếp hạng của người dùng đối với phim của họ. Năm 2009, họ đã trao giải thưởng 1.000.000 USD cho người chiến thắng, người đã cải thiện thuật toán Netflix hơn 10%. Cách họ làm điều này là sử dụng cây quyết định tăng cường độ dốc để kết hợp hơn 500 mô hình khác nhau. Các công cụ đề xuất khác sử dụng cây quyết định để nghiên cứu mức độ tương tác của người dùng và tìm ra các đặc điểm nhân khẩu học để xác định mức độ tương tác tốt nhất.

Trong chương 12, chúng ta sẽ tìm hiểu thêm về cây quyết định tăng cường độ dốc và rừng ngẫu nhiên. Hiện tại, cách tốt nhất để hình dung chúng là tập hợp nhiều cây quyết định phối hợp với nhau để đưa ra những dự đoán tốt nhất.

Tóm tắt

- Cây quyết định là mô hình học máy quan trọng, được sử dụng để phân loại và hồi quy. Cách thức hoạt động của cây quyết định là đặt các câu hỏi nhị phân về dữ liệu của chúng ta và đưa ra dự đoán dựa trên câu trả lời cho những câu hỏi đó. Thuật toán xây dựng cây quyết định để phân loại bao gồm việc tìm kiếm đặc điểm trong dữ liệu của chúng tôi để xác định nhãn tốt nhất và lặp lại bước này. Chúng tôi có một số cách để biết liệu một tính năng có xác định nhãn tốt nhất hay không. Ba
- thông số chúng ta đã học trong chương này là độ chính xác, chỉ số tạp chất Gini và entropy. Chỉ số tạp chất Gini đo độ tinh khiết của một bộ. Theo cách đó, một tập hợp trong đó mọi phần tử có cùng nhãn có chỉ số tạp chất Gini là 0. Một tập hợp trong đó mọi phần tử có nhãn khác nhau có nhãn tạp chất Gini gần bằng 1. Entropy là một thước đo khác cho độ tinh khiết của một tập hợp. Một tập hợp trong đó mọi phần tử có cùng nhãn có entropy bằng 0. Một tập hợp trong đó một nửa số phần tử có một nhãn và nửa còn lại có nhãn khác có entropy là 1. Khi xây dựng cây quyết định, sự khác biệt về entropy trước và sau khi phân tách được gọi là độ lợi thông tin.

-

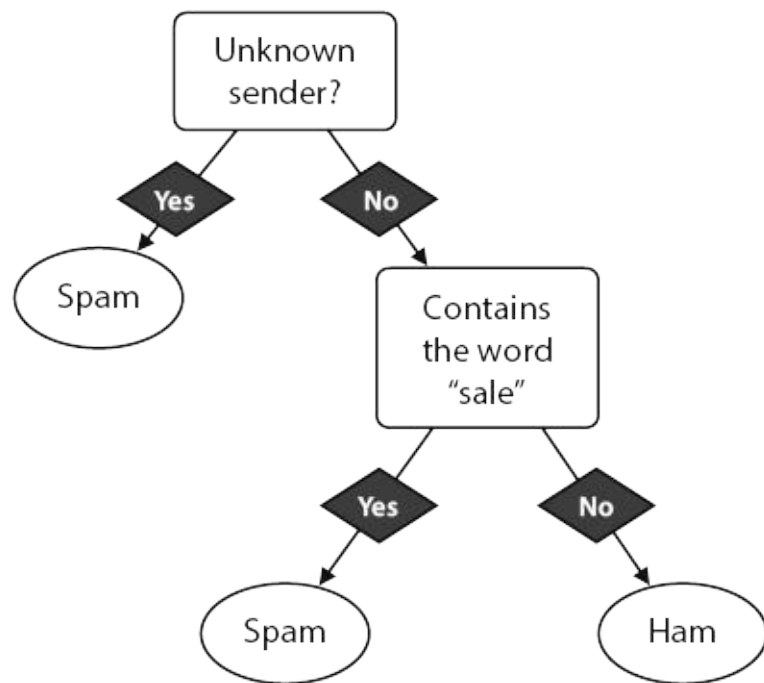
- Thuật toán xây dựng cây quyết định cho hồi quy tương tự như thuật toán được sử dụng để phân loại. Điểm khác biệt duy nhất là chúng tôi sử dụng sai số bình phương trung bình để chọn đặc điểm tốt nhất để phân chia dữ liệu. Trong không gian hai chiều, biểu đồ cây hồi quy trông giống như sự kết hợp của một số đường ngang, trong đó mỗi đường ngang là dự đoán cho các phần tử trong một chiếc lá cụ thể. Các ứng dụng của cây quyết định rất đa dạng, từ các thuật toán khuyến nghị đến các ứng dụng trong y học và sinh học.

•

Bài tập

BÀI TẬP 9.1

Trong mô hình cây quyết định phát hiện thư rác sau đây, hãy xác định xem email từ mẹ bạn có dòng chủ đề “Xin vui lòng đến cửa hàng, đang có đợt giảm giá” có bị phân loại là thư rác hay không.



BÀI TẬP 9.2

Mục tiêu của chúng tôi là xây dựng mô hình cây quyết định để xác định xem các giao dịch thẻ tín dụng có gian lận hay không. Chúng tôi sử dụng tập dữ liệu về các giao dịch thẻ tín dụng bên dưới, với các tính năng sau:

- **Value:** giá trị giao dịch.

- **Nhà cung cấp được phê duyệt:** công ty thẻ tín dụng có danh sách các nhà cung cấp được phê duyệt. Biến này cho biết nhà cung cấp có nằm trong danh sách này hay không.

	Giá trị	Nhà cung cấp được phê duyệt	Lừa đảo
Giao dịch 1	\$100	Không được phê duyệt	Yes
Giao dịch 2	\$100	Tán thành	No
Giao dịch 3	\$10,000	Tán thành	No
Giao dịch 4	\$10,000	Không được phê duyệt	Yes
Giao dịch 5	\$5,000	Tán thành	Yes
Giao dịch 6	\$100	Tán thành	No

Xây dựng nút đầu tiên của cây quyết định với các thông số kỹ thuật sau:

Một. Sử dụng chỉ số tạp chất Gini b. Sử dụng entropy

BÀI TẬP 9.3

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán
Bệnh nhân 1		X	X	X	Đau ốm
Bệnh nhân 2	X	X		X	Đau ốm
Bệnh nhân 3	X		X	X	Đau ốm
Bệnh nhân 4	X	X	X		Đau ốm
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

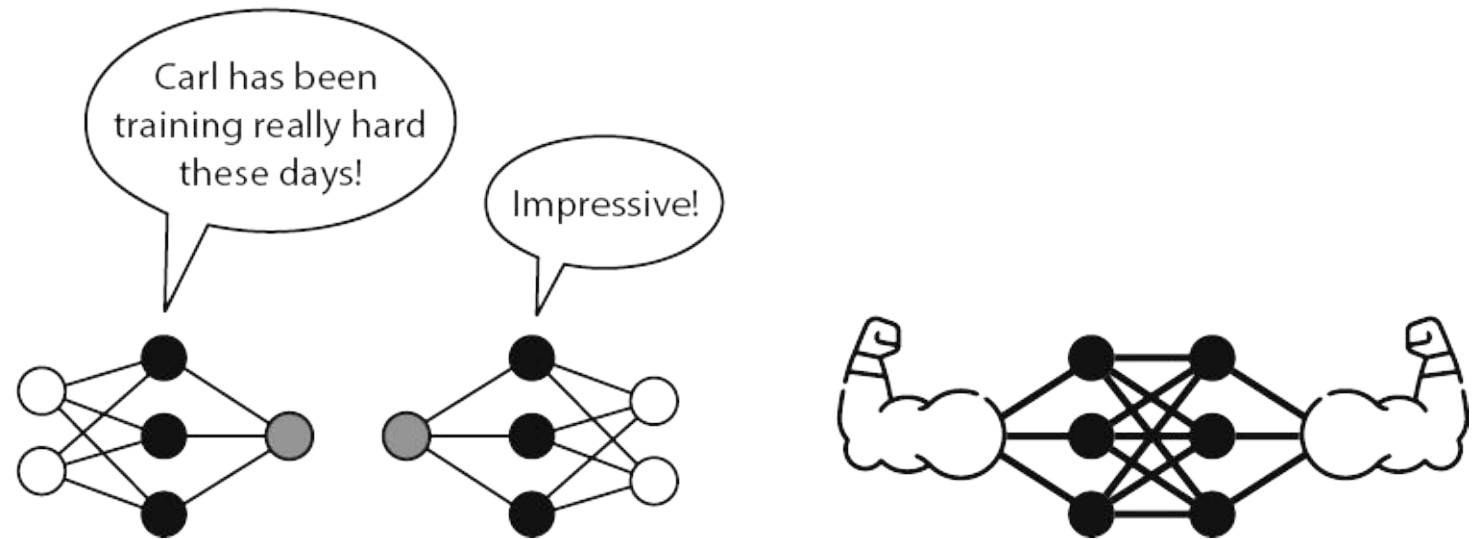
Sử dụng độ chính xác, xây dựng cây quyết định có độ cao 1 (gốc quyết định) để phân loại dữ liệu này.Độ chính xác của bộ phân loại này trên tập dữ liệu là gì?

10 Kết hợp các khối xây dựng để có thêm sức mạnh: Mạng lưới thần kinh

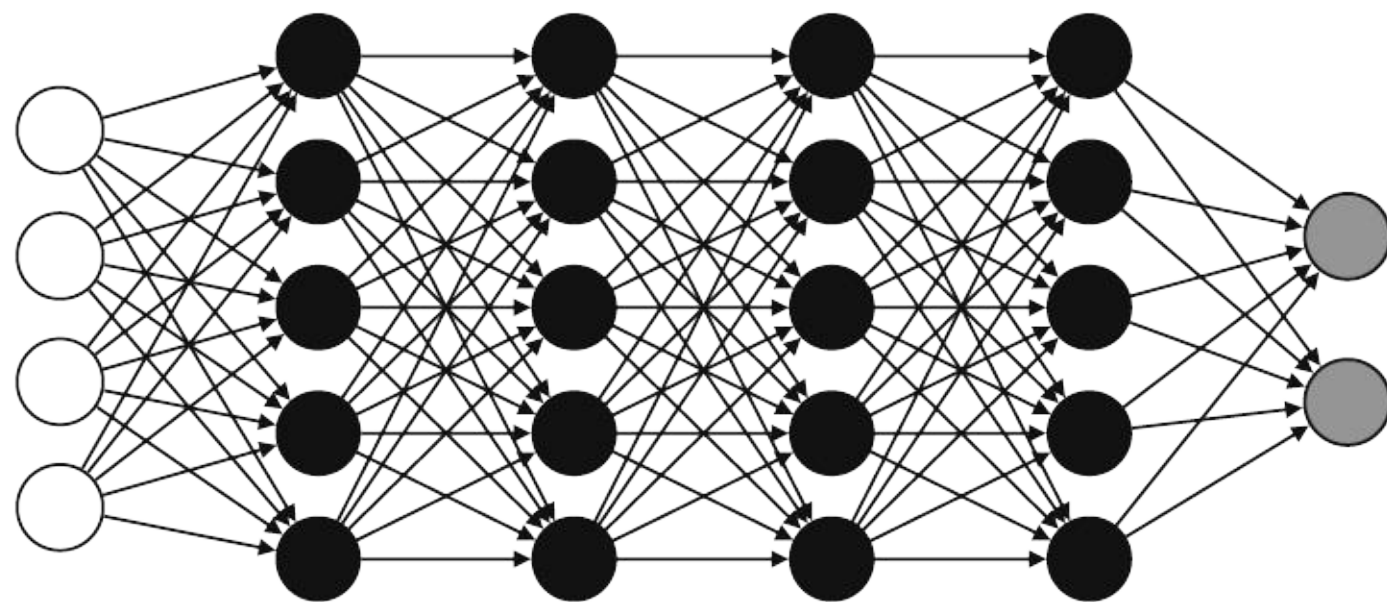
Trong chương này

- Mạng nơ-ron là gì Kiến trúc của mạng nơ-ron: các nút, lớp, độ sâu và chức năng kích hoạt huấn
- luyện mạng nơ-ron bằng cách sử dụng các vấn đề tiềm ẩn về lan truyền ngược trong huấn luyện
- mạng nơ-ron, chẳng hạn như vấn đề độ dốc biến mất và các kỹ thuật trang bị quá mức để cải thiện
- việc huấn luyện mạng nơ-ron, chẳng hạn như chính quy hóa và bỏ học bằng Keras để huấn luyện
- mạng nơ-ron để phân tích tình cảm và phân loại hình ảnh bằng cách sử dụng mạng nơ-ron làm mô
- hình hồi quy

-
-
-



Trong chương này, chúng ta tìm hiểu *mạng lưới thần kinh*, còn được gọi là *các tri giác đa lớp*. Mạng lưới thần kinh là một trong những mô hình học máy phổ biến nhất (nếu không muốn nói là phổ biến nhất). Chúng hữu ích đến mức lĩnh vực này có tên riêng: *deep learning*. Học sâu có nhiều ứng dụng trong các lĩnh vực học máy tiên tiến nhất, bao gồm nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên, y học và xe tự lái. Theo nghĩa rộng của từ này, mạng lưới thần kinh có nghĩa là bắt chước cách thức hoạt động của bộ não con người. Chúng có thể rất phức tạp, như hình 10.1 cho thấy.



Hình 10.1 Mạng lưới thần kinh. Nó có thể trông phức tạp nhưng trong vài trang tiếp theo, chúng ta sẽ làm sáng tỏ hình ảnh này.

Mạng nơ-ron trong hình 10.1 có thể trông đáng sợ với rất nhiều nút, cạnh, v.v. Tuy nhiên, chúng ta có thể hiểu mạng lưới thần kinh theo những cách đơn giản hơn nhiều. Một cách để xem chúng là tập hợp các perceptron (chúng ta đã học ở chương 5 và 6). Tôi muốn xem mạng lưới thần kinh là sự kết hợp của các bộ phân loại tuyến tính tạo ra các bộ phân loại phi tuyến. Ở kích thước thấp, các bộ phân loại tuyến tính sẽ trông giống như các đường thẳng hoặc mặt phẳng và các bộ phân loại phi tuyến sẽ trông giống như các đường cong hoặc bề mặt phức tạp. Trong chương này, chúng ta thảo luận về trực quan đằng sau mạng nơ-ron và chi tiết về cách chúng hoạt động, đồng thời chúng tôi cũng mã hóa mạng nơ-ron và sử dụng chúng cho một số ứng dụng như nhận dạng hình ảnh.

Mạng lưới thần kinh rất hữu ích cho việc phân loại và hồi quy. Trong chương này, chúng ta chủ yếu tập trung vào phân loại mạng lưới thần kinh, nhưng chúng ta cũng tìm hiểu những thay đổi nhỏ cần thiết để chúng hoạt động hồi quy. Đầu tiên, một chút về thuật ngữ. Hãy nhớ lại rằng trong chương 5, chúng ta đã học về perceptron và trong chương 6, chúng ta đã học về bộ phân loại logistic. Chúng ta cũng đã biết rằng chúng được gọi là các perceptron rời rạc và liên tục. Để làm mới bộ nhớ của bạn, đầu ra của perceptron rời rạc là 0 hoặc 1 và đầu ra của perceptron liên tục là bất kỳ số nào trong khoảng $(0,1)$. Để tính toán kết quả đầu ra này, perceptron rời rạc sử dụng hàm bước (phần “Hàm bước và các hàm kích hoạt” ở chương 5) và perceptron liên tục sử dụng hàm sigmoid (phần “Phương pháp xác suất để phân loại: Hàm sigmoid” ở chương 6). Trong chương này, chúng tôi gọi cả hai bộ phân loại là perceptron và khi cần, chúng tôi sẽ xác định xem chúng ta đang nói về perceptron rời rạc hay perceptron liên tục.

The code for this chapter is available in this GitHub repository:
https://github.com/luisguiserrano/manning/tree/master/Chapter_10_Neural_Network

Mạng lưới thần kinh với một ví dụ: Một hành tinh ngoài hành tinh phức tạp hơn

Trong phần này, chúng ta tìm hiểu mạng lưới thần kinh đang sử dụng ví dụ phân tích cảm xúc quen thuộc từ chương 5 và 6. Kịch bản như sau: chúng ta thấy mình đang ở trên một hành tinh xa xôi có người ngoài hành tinh sinh sống. Họ dường như nói một ngôn ngữ được hình thành bởi hai từ *ack* và *beep* và chúng tôi muốn xây dựng một mô hình học máy giúp chúng tôi xác định xem người ngoài hành tinh đang vui hay buồn dựa trên những lời họ nói. Đây gọi là phân tích tình cảm, vì chúng ta cần xây dựng một mô hình để phân tích tình cảm của người ngoài hành tinh. Chúng tôi ghi lại một số người ngoài hành tinh đang nói chuyện và cố gắng xác định bằng các phương tiện khác xem họ vui hay buồn, và chúng tôi đưa ra tập dữ liệu trong bảng 10.1.

Bảng 10.1 Tập dữ liệu của chúng tôi, trong đó mỗi hàng đại diện cho một người ngoài hành tinh. Cột đầu tiên thể hiện câu họ đã thốt ra. Cột thứ hai và thứ ba biểu thị số lần xuất hiện của mỗi từ trong câu. Cột thứ tư thể hiện tâm trạng của người ngoài hành tinh.

Câu	Ách	Bíp	Tâm trạng
“Ách”	1	0	Sad
“Ách ôi”	2	0	Sad
“Bíp”	0	1	Sad
“Bíp bíp”	0	2	Sad
“Ack bíp”	1	1	Vui mừng
“Ack ack beep”	2	1	Vui mừng
“Bíp bíp”	1	2	Vui mừng
“Bíp bíp bíp”	2	2	Vui mừng

Đây có vẻ là một tập dữ liệu đủ đẹp và chúng ta có thể điều chỉnh bộ phân loại cho dữ liệu này. Trước tiên hãy vẽ đồ thị như trong hình 10.2.



Hình 10.2 Sơ đồ của tập dữ liệu trong bảng 10.1. Trục hoành tương ứng với số lần xuất hiện của từ *aack* và trục tung tương ứng với số lần xuất hiện của từ *bíp*. Những khuôn mặt vui vẻ tương ứng với những người ngoài hành tinh vui vẻ, và những khuôn mặt buồn tương ứng với những người ngoài hành tinh buồn bã.

Từ hình 10.2, có vẻ như chúng ta sẽ không thể lắp bộ phân loại tuyến tính vào dữ liệu này. Nói cách khác, việc vẽ một đường phân chia khuôn mặt vui và buồn là điều không thể. Chúng ta có thể làm gì? Chúng ta đã học các trình phân loại khác có thể thực hiện công việc này, chẳng hạn như trình phân loại Bayes đơn giản (chương 8) hoặc cây quyết định (chương 9). Nhưng trong chương này, chúng ta sẽ tập trung vào perceptron. Nếu mục tiêu của chúng ta là tách các điểm trong hình 10.2 và một đường không thực hiện được thì điều gì tốt hơn một đường? Còn những điều sau đây:

1. Hai đường thẳng
2. Một đường cong

Đây là những ví dụ về mạng lưới thần kinh. Hãy bắt đầu bằng cách xem tại sao cái đầu tiên, bộ phân loại sử dụng hai dòng, lại là một mạng lưới thần kinh.

Giải pháp: Nếu một dòng không đủ, hãy sử dụng hai dòng để phân loại tập dữ liệu của bạn

Trong phần này, chúng ta khám phá một trình phân loại sử dụng hai dòng để phân chia tập dữ liệu. Chúng ta có nhiều cách vẽ hai đường để phân chia tập dữ liệu này và một trong những cách này được minh họa trong hình 10.3. Hãy gọi chúng là dòng 1 và dòng 2.



Hình 10.3 Điểm vui và điểm buồn trong tập dữ liệu của chúng tôi không thể chia thành một dòng. Tuy nhiên, việc vẽ hai đường sẽ tách chúng ra rõ ràng—các điểm phía trên cả hai đường có thể được phân loại là vui và các điểm còn lại là buồn. Việc kết hợp các bộ phân loại tuyến tính theo cách này là cơ sở cho mạng lưới thần kinh.

Chúng ta có thể định nghĩa trình phân loại của mình như sau:

PHÂN LOẠI PHÂN TÍCH TÌNH CẢM

Một câu được xếp loại là vui nếu điểm tương ứng của nó nằm trên hai đường như trong hình 10.3. Nếu nó ở dưới ít nhất một trong các dòng thì nó được phân loại là buồn.

Bây giờ, chúng ta hãy thực hiện một số phép toán. Chúng ta có thể nghĩ ra hai phương trình cho những đường này không? Nhiều phương trình có thể hoạt động, nhưng hãy sử dụng hai phương trình sau (trong đó x_a là số lần từ *aack* xuất hiện trong câu và x_b là số lần lần *tiếng bíp* xuất hiện).

- Line 1: $6x_a + 10x_b - 15 = 0$
- Line 2: $10x_a + 6x_b - 15 = 0$

ngoài ra: **Làm thế nào CHÚNG TÔI tìm được các phương trình này?** Lưu ý rằng dòng 1 đi qua các điểm (0, 1.5) và (2.5, 0). Do đó, độ dốc, được định nghĩa là sự thay đổi trong

trục hoành chia cho sự thay đổi của trục tung, chính xác là $\frac{-1.5}{2.5} = -\frac{3}{5}$. Điểm chặn y—cụ thể là độ cao mà tại đó đường thẳng đi qua trục tung—là 1,5.

Do đó, phương trình của đường thẳng này là $x_b = -\frac{3}{5}x_a + 1.5$. Bằng cách giải phương trình này, ta được $6x_a + 10x_b - 15 = 0$. Ta có thể áp dụng cách tiếp cận tương tự để tìm phương trình cho dòng 2.

Do đó, trình phân loại của chúng tôi trở thành như sau:

PHÂN LOẠI PHÂN TÍCH TÌNH CẢM

Họ nghe thấy ce được xếp vào loại hạnh phúc nếu cả hai bất đẳng thức sau đều đúng :

- **Bất đẳng thức 1:** $6x_a + 10x_b - 15 \geq 0$
- **Bất bình đẳng 2:** $10x_a + 6x_b - 15 \geq 0$

Nếu ít nhất một trong số chúng không đạt thì câu đó được xếp vào loại buồn.

Để kiểm tra tính nhất quán, bảng 10.2 chứa các giá trị của mỗi phương trình trong số hai phương trình. Ở bên phải của mỗi phương trình, chúng tôi kiểm tra xem giá trị của phương trình có lớn hơn hoặc bằng 0 hay không. Cột ngoài cùng bên phải sẽ kiểm tra xem cả hai giá trị có lớn hơn hoặc bằng 0 hay không.

Bảng 10.2 Bộ dữ liệu tương tự như trong bảng 10.1 nhưng có một số cột mới. Cột thứ tư và thứ sáu tương ứng với hai dòng của chúng tôi. Cột thứ năm và thứ bảy kiểm tra xem phương trình của từng đường tại mỗi điểm dữ liệu có cho giá trị không âm hay không. Cột cuối cùng kiểm tra xem hai giá trị thu được có phải là không âm hay không.

Câu	Ách	Bíp	Phương trình 1	Phương trình 1 ≥ 0 ?	Phương trình 2	Phương trình 2 ≥ 0 ?	Cả hai phương trình ≥ 0
“Ách”	1	0	-9	No	-5	No	No
“Ách ôi”	2	0	-3	No	5	Yes	No
“Bíp”	0	1	-5	No	-9	No	No
“Bíp bíp”	0	2	5	Yes	3	No	No
“Ack bíp”	1	1	1	Yes	1	Yes	Yes
“Ack ack beep”	1	2	11	Yes	7	Yes	Yes
“Bíp bíp”	2	1	7	Yes	11	Yes	Yes
“Bíp bíp bíp”	2	2	17	Yes	17	Yes	Yes

Lưu ý cột ngoài cùng bên phải trong bảng 10.2 (có/không) trùng với cột ngoài cùng bên phải trong bảng 10.1 (vui/buồn).Điều này có nghĩa là trình phân loại đã quản lý để phân loại tất cả dữ liệu một cách chính xác.

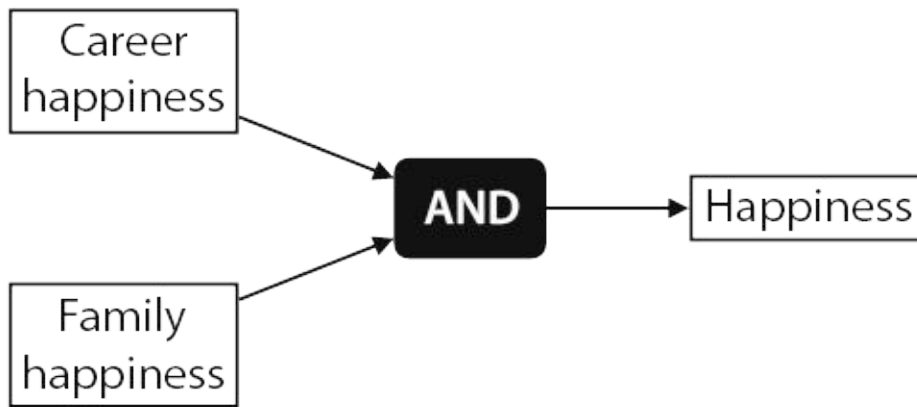
Tại sao lại có hai dòng?Hạnh phúc có phải là tuyến tính không?

Trong chương 5 và 6, chúng tôi đã suy luận được nhiều điều về ngôn ngữ dựa trên các phương trình của các bộ phân loại.Ví dụ: nếu trọng số của từ *aack* là dương thì chúng tôi kết luận rằng đó có thể là một từ vui vẻ.Còn bây giờ thì sao?Chúng ta có thể suy luận bất cứ điều gì về ngôn ngữ trong bộ phân loại này có chứa hai phương trình không?

Cách chúng ta có thể nghĩ ra hai phương trình là có thể trên hành tinh xa lạ, hạnh phúc không phải là một thứ tuyến tính đơn giản mà thay vào đó dựa trên hai thứ.Trong cuộc sống thực, hạnh phúc có thể dựa trên nhiều thứ: nó có thể dựa trên việc có được một sự nghiệp viên mãn kết hợp với cuộc sống gia đình hạnh phúc và đồ ăn đầy đủ trên bàn.Nó có thể dựa trên việc uống cà phê và bánh rán.Trong trường hợp này, giả sử hai khía cạnh của hạnh phúc là sự nghiệp và gia đình.Để người ngoài hành tinh được hạnh phúc, họ cần phải có *cả hai*.

Hóa ra trong trường hợp này, cả hạnh phúc nghề nghiệp và hạnh phúc gia đình đều là những phân loại tuyến tính đơn giản và mỗi phân loại được mô tả bằng một trong hai dòng.Hãy nói

dòng 1 tương ứng với hạnh phúc nghề nghiệp và dòng 2 tương ứng với hạnh phúc gia đình. Vì vậy, chúng ta có thể coi hạnh phúc xa lạ như sơ đồ trong hình 10.4. Trong sơ đồ này, hạnh phúc nghề nghiệp và hạnh phúc gia đình được nối bằng toán tử AND để kiểm tra xem cả hai có đúng hay không. Nếu đúng như vậy thì người ngoài hành tinh sẽ hạnh phúc. Nếu bất kỳ điều nào trong số đó thất bại, người ngoài hành tinh sẽ không vui.



Career happiness **AND** Family happiness $\longrightarrow$ Happiness

Hình 10.4 Bộ phân loại hạnh phúc được hình thành bởi bộ phân loại hạnh phúc nghề nghiệp, bộ phân loại hạnh phúc gia đình và toán tử AND. Nếu cả bộ phân loại hạnh phúc nghề nghiệp và gia đình đều đưa ra kết quả Có thì bộ phân loại hạnh phúc cũng vậy. Nếu bất kỳ trong số chúng đưa ra kết quả là Không thì bộ phân loại hạnh phúc cũng đưa ra kết quả là Không.

Các thước đo hạnh phúc gia đình và nghề nghiệp đều là các cảm quan, bởi vì chúng được cho bởi phương trình đường thẳng. Chúng ta có thể biến toán tử AND này thành một perceptron khác không? Câu trả lời là có và chúng ta sẽ xem cách thực hiện ở phần tiếp theo.

Hình 10.4 bắt đầu trông giống một mạng lưới thần kinh. Chỉ cần thêm một vài bước nữa và thêm một chút phép toán nữa là chúng ta sẽ có được thứ gì đó trông giống hình 10.1 ở đầu chương này.

Kết hợp đầu ra của Perceptron vào Perceptron khác

Hình 10.4 gợi ý về sự kết hợp của các perceptron, trong đó chúng ta cắm đầu ra của hai perceptron làm đầu vào cho perceptron thứ ba. Đây là cách mạng lưới thần kinh được xây dựng và trong phần này, chúng ta sẽ thấy phép toán đằng sau nó.

Trong phần “Hàm bước và các hàm kích hoạt” ở chương 5, chúng ta đã định nghĩa hàm bước, trả về 0 nếu đầu vào âm và 1 nếu đầu vào là

tích cực hoặc bằng không. Lưu ý rằng vì chúng ta đang sử dụng hàm bước nên đây là các perceptron rời rạc. Sử dụng hàm này, chúng ta có thể định nghĩa các thang phân loại hạnh phúc gia đình và nghề nghiệp như sau:

PHÂN LOẠI HẠNH PHÚC NGHỀ NGHIỆP

Trọng lượng:

- A_{ack} : 6
- $Bíp$: 10

Độ lệch: -15

Điểm của một câu: $6x_a + 10x_b - 15$

Dự đoán: $F = \text{bước}(6x_a + 10x_b - 15)$

PHÂN LOẠI HẠNH PHÚC GIA ĐÌNH

Trọng lượng:

- A_{ack} : 10
- $Bíp$: 6

Độ lệch: -15

Điểm của một câu: $10x_a + 6x_b - 15$

Dự đoán: $C = \text{bước}(10x_a + 6x_b - 15)$

Bước tiếp theo là kết nối đầu ra của bộ phân loại hạnh phúc gia đình và sự nghiệp vào một bộ phân loại hạnh phúc mới. Hãy thử xác minh rằng trình phân loại sau hoạt động. Hình 10.5 chứa hai bảng với kết quả đầu ra của bộ phân loại nghề nghiệp và gia đình và bảng thứ ba trong đó hai cột đầu tiên là đầu vào và đầu ra của bộ phân loại nghề nghiệp và gia đình, và cột cuối cùng là đầu ra của bộ phân loại gia đình. Mỗi bảng trong hình 10.5 tương ứng với một perceptron.

Career classifier

Family classifier

Happiness classifier

x_a	x_b	$6x_a + 10x_b - 15$	$C = \text{step}(6x_a + 10x_b - 15)$
1	0	-9	0
2	0	-3	0
0	1	-5	0
0	2	5	1
1	1	1	1
1	2	11	1
2	1	7	1
2	2	17	1

x_a	x_b	$10x_a + 6x_b - 15$	$F = \text{step}(10x_a + 6x_b - 15)$
1	0	-5	0
2	0	5	1
0	1	-9	0
0	2	3	0
1	1	1	1
1	2	7	1
2	1	11	1
2	2	17	1

C	F	$1 \cdot C + 1 \cdot F - 1.5$	$\hat{y} = \text{step}(C + F - 1.5)$
0	0	-1.5	0
0	1	-0.5	0
0	0	-0.5	0
1	0	-0.5	0
1	1	0.5	1
1	1	0.5	1
1	1	0.5	1
1	1	0.5	1

Hình 10.5 Ba bộ phân loại perceptron, một cho hạnh phúc nghề nghiệp, một cho hạnh phúc gia đình và một cho hạnh phúc, kết hợp hai phân loại trước đó. Kết quả đầu ra của nhận thức sự nghiệp và gia đình là đầu vào của nhận thức hạnh phúc.

PHÂN LOẠI HẠNH PHÚC

Trọng lượng:

- Sự nghiệp:
1
- Gia đình:
1

Độ lệch: -1,5

Điểm của một câu: $1 \cdot C + 1 \cdot F - 1,5$

Dự đoán: $\hat{y} = \text{bước}(1 \cdot C + 1 \cdot F - 1.5)$

Sự kết hợp của các phân loại này là một mạng lưới thần kinh. Tiếp theo, chúng ta xem cách làm cho nó trông giống như hình 10.1.

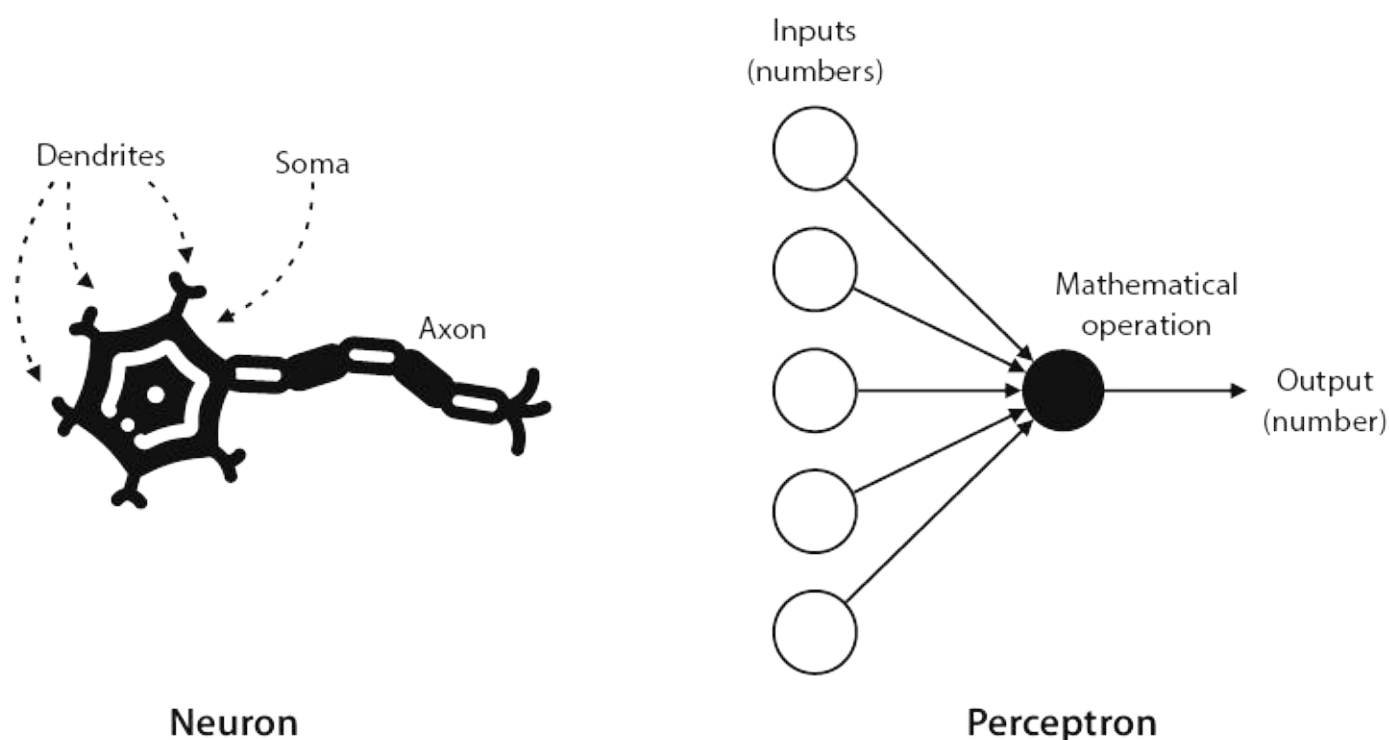
Biểu diễn đồ họa của các perceptron

Trong phần này, tôi chỉ cho bạn cách biểu diễn các perceptron bằng đồ họa, điều này dẫn đến cách biểu diễn đồ họa của mạng nơ-ron. Chúng tôi gọi chúng là thần kinh

mạng vì đơn vị cơ bản của chúng, perceptron, gần giống một nơ-ron.

Một tế bào thần kinh bao gồm ba phần chính: soma, đuôi gai và sợi trục. Theo nghĩa rộng, tế bào thần kinh nhận tín hiệu đến từ các tế bào thần kinh khác thông qua các sợi nhánh, xử lý chúng trong soma và gửi tín hiệu qua sợi trục để các tế bào thần kinh khác nhận được. Hãy so sánh điều này với một perceptron, nhận các số làm đầu vào, áp dụng một phép toán học

hoạt động cho chúng (thường bao gồm một tổng được tạo bằng hàm kích hoạt) và xuất ra một số mới. Quá trình này được minh họa trong hình 10.6.



Hình 10.6 Một perceptron dựa trên một nơ-ron. Bên trái: Một tế bào thần kinh với các thành phần chính: đuôi gai, thân và sợi trục. Tín hiệu đi qua các sợi nhánh, được xử lý trong tế bào thần kinh và được gửi đến các tế bào thần kinh khác thông qua sợi trục. Đúng: Một perceptron. Các nút ở bên trái tương ứng với các đầu vào số, nút ở giữa thực hiện một phép toán và xuất ra một số.

Chính thức hơn, hãy nhớ lại định nghĩa của một perceptron trong chương 5 và 6, trong đó chúng ta có các thực thể sau:

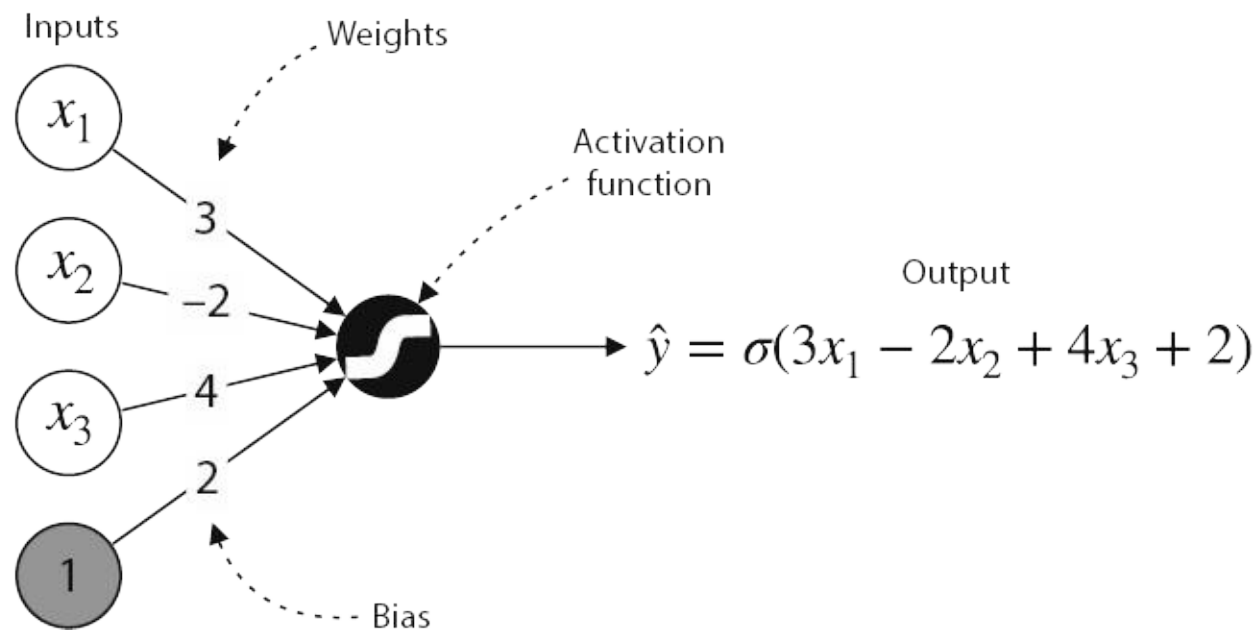
- **Đầu vào:** $x_1, x_2, \dots, x_n$ **Trọng**
- **số:** $w_1, w_2, \dots, w_n$ **Thành**
- **kiến:** b

- **Hàm kích hoạt:** Hàm bước (đối với các perceptron rời rạc) hoặc hàm sigmoid (đối với các perceptron liên tục).(Phần sau của chương này chúng ta tìm hiểu các hàm kích hoạt mới khác.)
- **Dự đoán:** Được xác định theo công thức $\hat{y} = f(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b)$, trong đó f là hàm kích hoạt tương ứng

Cách chúng được đặt trong sơ đồ được minh họa trong hình 10.7. Ở bên trái, chúng ta có các nút đầu vào và ở bên phải, chúng ta có nút đầu ra. Các biến đầu vào đi trên các nút đầu vào. Nút đầu vào cuối cùng không chứa biến nhưng nó chứa giá trị 1. Các trọng số nằm trên các cạnh nối các nút đầu vào với nút đầu ra. Trọng số tương ứng với nút đầu vào cuối cùng là độ lệch. Các phép toán để tính toán dự đoán diễn ra bên trong nút đầu ra và nút này đưa ra dự đoán.

Ví dụ: perceptron được xác định theo phương trình $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$ được minh họa trong hình 10.7. Lưu ý rằng trong perceptron này, các bước sau được thực hiện:

- Các đầu vào được nhân với trọng số tương ứng và cộng lại để thu được $3x_1 - 2x_2 + 4x_3$.
- Độ lệch được thêm vào phương trình trước, để thu được $3x_1 - 2x_2 + 4x_3 + 2$. Hàm kích hoạt
- sigmoid được áp dụng để thu được đầu ra $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$.



Hình 10.7 Một biểu diễn trực quan của một perceptron. Các đầu vào (đặc điểm và độ lệch) xuất hiện dưới dạng các nút ở bên trái, còn trọng số và độ lệch nằm ở các cạnh kết nối các nút đầu vào với nút chính ở giữa. Nút ở giữa lấy sự kết hợp tuyến tính của các trọng số và đầu vào, thêm độ lệch và áp dụng hàm kích hoạt, trong trường hợp này là hàm sigmoid. Kết quả đầu ra là dự đoán được đưa ra bởi công thức $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$.

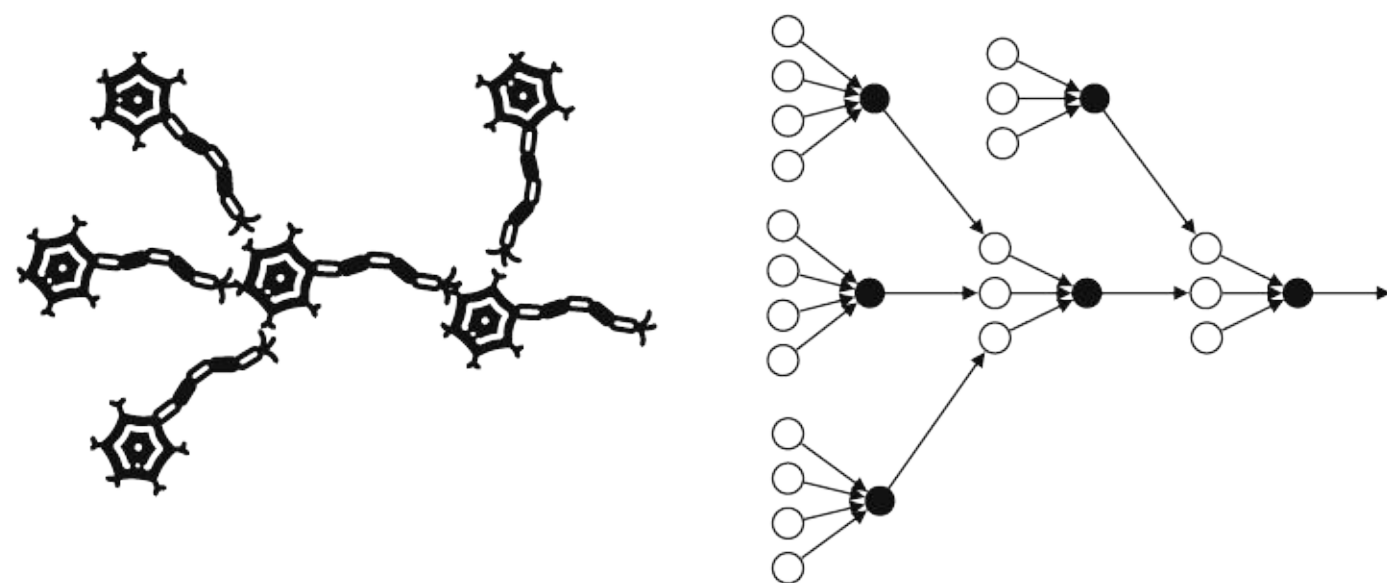
Ví dụ: nếu đầu vào của perceptron này là điểm $(x_1, x_2, x_3) = (1, 3, 1)$, thì đầu ra là $\sigma(3 \cdot 1 - 2 \cdot 3 + 4 \cdot 1 + 2) = \sigma(3) = 0,953$.

Nếu perceptron này được xác định bằng hàm bước thay vì hàm sigmoid, thì đầu ra sẽ là $\text{bước}(3 \cdot 1 - 2 \cdot 3 + 4 \cdot 1 + 2) = \text{bước}(3) = 1$.

Cách trình bày bằng đồ họa này giúp cho việc ghép nối các perceptron trở nên dễ dàng, như chúng ta sẽ thấy trong phần tiếp theo.

Biểu diễn đồ họa của mạng lưới thần kinh

Như chúng ta đã thấy ở phần trước, mạng nơ-ron là sự kết hợp của các perceptron. Cấu trúc này nhằm mô phỏng một cách lỏng lẻo bộ não con người, trong đó đầu ra của một số nơ-ron trở thành đầu vào cho một nơ-ron khác. Theo cách tương tự, trong mạng lưới thần kinh, đầu ra của một số perceptron sẽ trở thành đầu vào của một perceptron khác, như minh họa trong hình 10.8.

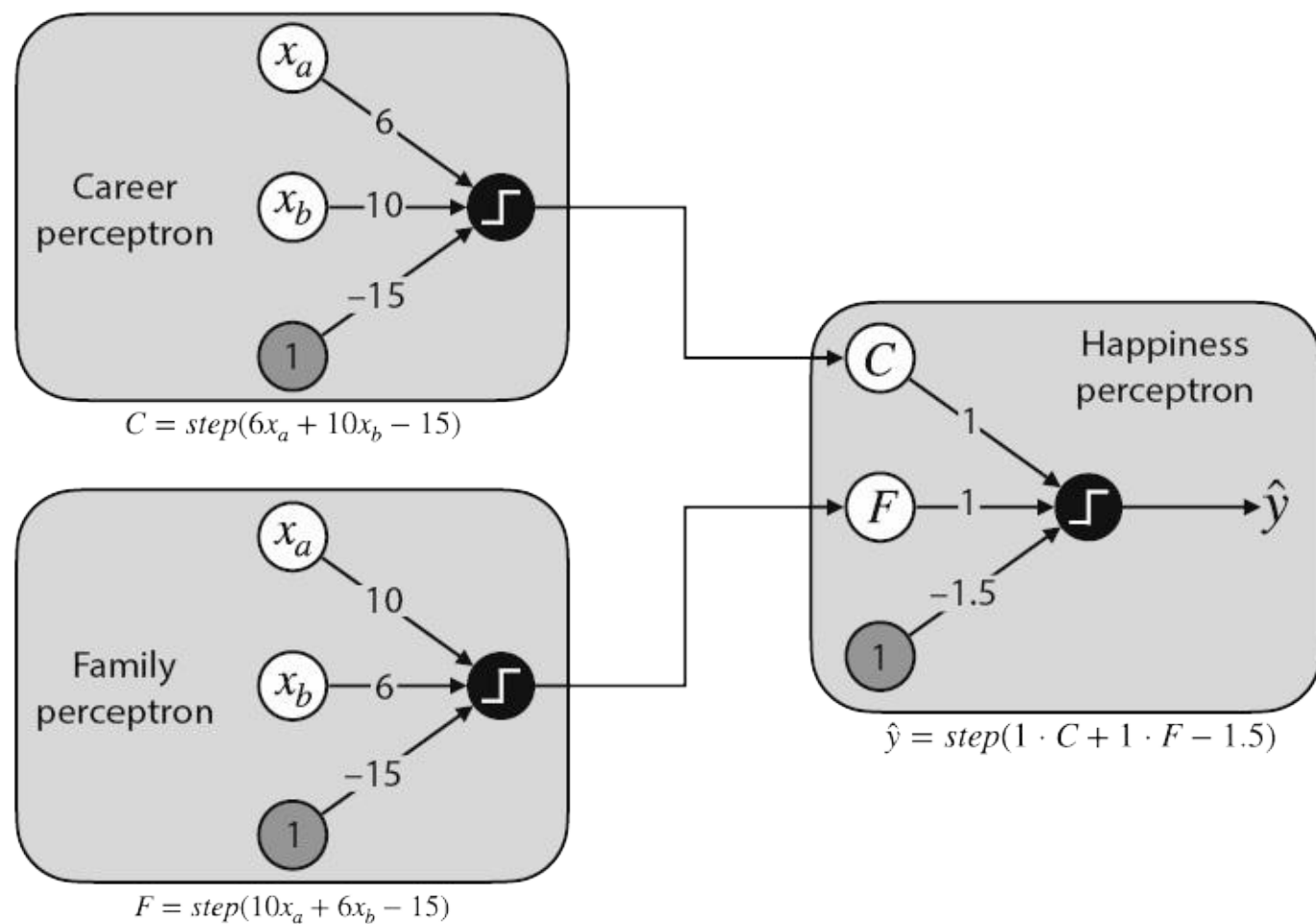


Hình 10.8 Mạng lưới thần kinh có mục đích mô phỏng (một cách lỏng lẻo) cấu trúc của não. Bên trái: Các tế bào thần kinh được kết nối bên trong não theo cách mà đầu ra của một tế bào thần kinh trở thành đầu vào của một tế bào thần kinh khác. Đúng: Các perceptron được kết nối theo cách mà đầu ra của một perceptron trở thành đầu vào của một perceptron khác.

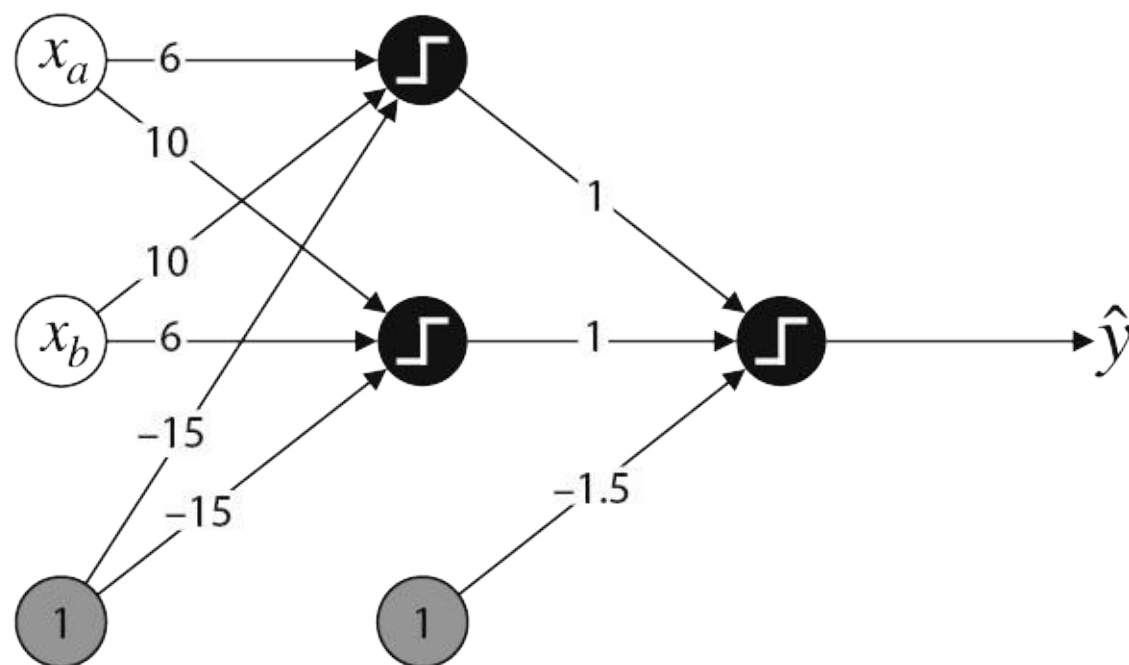
Mạng lưới thần kinh mà chúng ta đã xây dựng ở phần trước, trong đó chúng ta ghép nối Perceptron sự nghiệp và Perceptron gia đình với Perceptron hạnh phúc, được minh họa trong hình 10.9.

Lưu ý rằng trong sơ đồ ở hình 10.9, các đầu vào cho nhận thức nghề nghiệp và gia đình được lặp lại. Một cách viết rõ ràng hơn, trong đó những đầu vào này không bị lặp lại, được minh họa trong hình 10.10.

Lưu ý rằng ba perceptron này sử dụng hàm bước. Chúng tôi làm điều này chỉ vì mục đích giáo dục, bởi vì trong cuộc sống thực, mạng nơ-ron không bao giờ sử dụng hàm bước làm hàm kích hoạt, vì nó khiến chúng tôi không thể sử dụng phương pháp giảm độ dốc (xem thêm điều này trong phần “Đào tạo mạng nơ-ron”). Tuy nhiên, hàm sigmoid được sử dụng rộng rãi trong mạng nơ-ron và trong phần “Các hàm kích hoạt khác nhau”, chúng ta tìm hiểu một số hàm kích hoạt hữu ích khác được sử dụng trong thực tế.



Hình 10.9 Khi chúng ta kết nối đầu ra của cảm biến sự nghiệp và gia đình với cảm biến hạnh phúc, chúng ta sẽ có được một mạng lưới thần kinh. Mạng nơ-ron này sử dụng hàm bước làm hàm kích hoạt.



Hình 10.10 Một phiên bản sạch hơn của sơ đồ trong hình 10.9. Trong sơ đồ này, các đặc điểm x_a và x_b và độ lệch không được lặp lại. Thay vào đó, mỗi nút kết nối với cả hai nút ở bên phải, kết hợp ba bộ phận nhận cảm một cách độc đáo vào cùng một sơ đồ.

Ranh giới của mạng lưới thần kinh

Trong chương 5 và 6, chúng ta đã nghiên cứu ranh giới của các perceptron được biểu thị bằng các đường thẳng. Trong phần này, chúng ta thấy ranh giới của mạng lưới thần kinh trông như thế nào.

Nhớ lại chương 5 và 6 rằng cả perceptron rời rạc và perceptron liên tục (bộ phân loại logistic) đều có ranh giới tuyến tính được đưa ra bởi phương trình tuyến tính xác định chúng. Perceptron rời rạc gán các dự đoán 0 và 1 cho các điểm tùy theo vị trí của chúng. Perceptron liên tục ấn định dự đoán trong khoảng từ 0 đến 1 cho mọi điểm trên mặt phẳng. Các điểm phía trên đường thẳng có dự đoán là 0,5, các điểm ở một bên đường có dự đoán cao hơn 0,5 và các điểm ở phía bên kia đường có dự đoán thấp hơn 0,5. Hình 10.11 minh họa các perceptron rời rạc và liên tục tương ứng với phương trình $10x_a + 6x_b - 15 = 0$.

$$C = \text{step}(10x_a + 6x_b - 15)$$

$$C = \sigma(10x_a + 6x_b - 15) = 0$$



Line with equation $10x_a + 6x_b - 15 = 0$

Hình 10.11 Ranh giới của một perceptron là một đường thẳng. Bên trái: Đối với một perceptron rời rạc, các điểm ở một bên của đường thẳng được dự đoán là 0 và các điểm ở phía bên kia được dự đoán là 1. Bên phải: Đối với một perceptron liên tục, các điểm đều được đưa ra dự đoán trong khoảng (0,1). Trong ví dụ này, các điểm ở bên trái nhận được dự đoán gần bằng 0, các điểm ở bên phải nhận được dự đoán gần bằng 1 và các điểm ở bên phải nhận được dự đoán là 0,5.

Chúng ta cũng có thể hình dung đầu ra của mạng nơ-ron theo cách tương tự. Hãy nhớ lại rằng đầu ra của mạng nơ-ron với chức năng kích hoạt bước như sau:

- Nếu $6x_a + 10x_b - 15 \geq 0$ và $10x_a + 6x_b - 15 \geq 0$ thì đầu ra là 1.
- Nếu không thì đầu ra là 0.

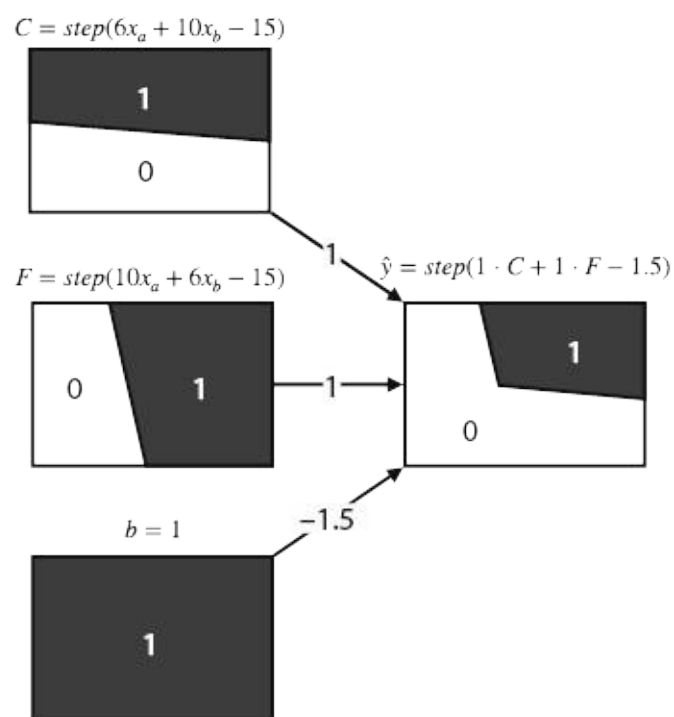
Ranh giới này được minh họa ở phía bên trái của hình 10.12 bằng hai đường thẳng. Lưu ý rằng nó được biểu thị dưới dạng sự kết hợp các ranh giới của hai thụ thể đầu vào và nút thiên vị. Ranh giới thu được bằng hàm kích hoạt bước được tạo bằng các đường đứt nét, trong khi ranh giới thu được bằng hàm kích hoạt sigmoid là một đường cong.

Để nghiên cứu những ranh giới này cẩn thận hơn, hãy kiểm tra sổ ghi chép sau:

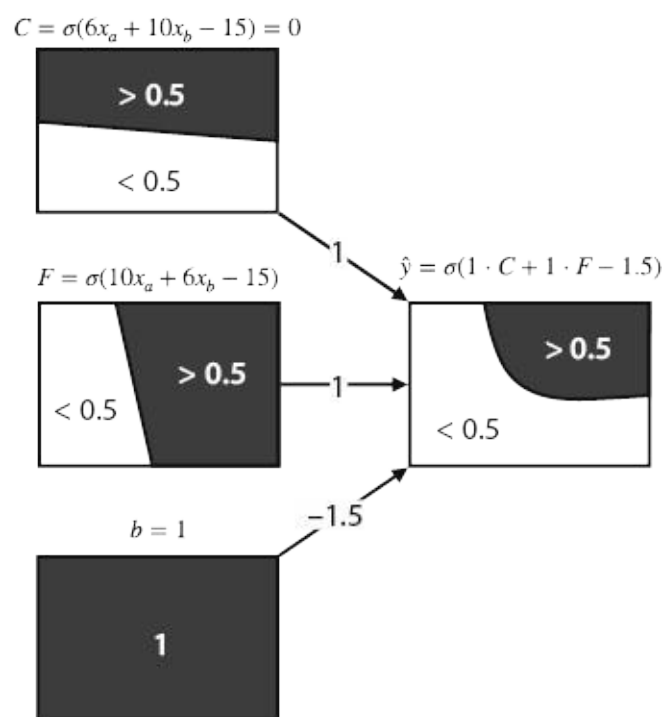
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_Network

Trong sổ ghi chép này, ranh giới của hai đường và hai mạng thần kinh được vẽ bằng các hàm kích hoạt bước và sigmoid, như minh họa trong hình 10.13.

With the step activation function



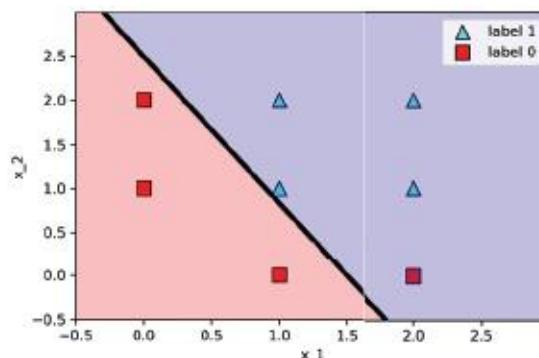
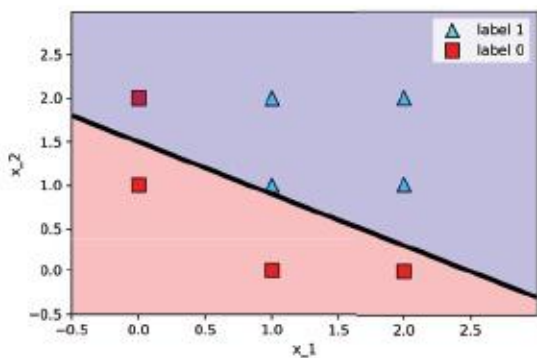
With the sigmoid activation function



Hình 10.12 Để xây dựng mạng nơ-ron, chúng tôi sử dụng đầu ra của hai perceptron và một nút thiên vị (được biểu thị bằng bộ phân loại luôn xuất giá trị 1) cho perceptron thứ ba. Ranh giới của bộ phân loại kết quả là sự kết hợp các ranh giới của các bộ phân loại đầu vào. Ở bên trái, chúng ta thấy ranh giới thu được bằng hàm bước, là một đường đứt nét. Ở bên phải, chúng ta thấy ranh giới thu được bằng hàm sigmoid, là một đường cong.

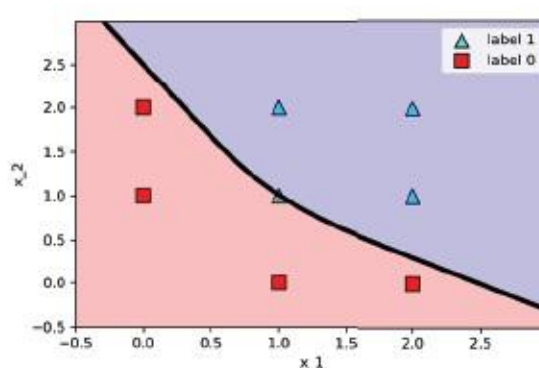
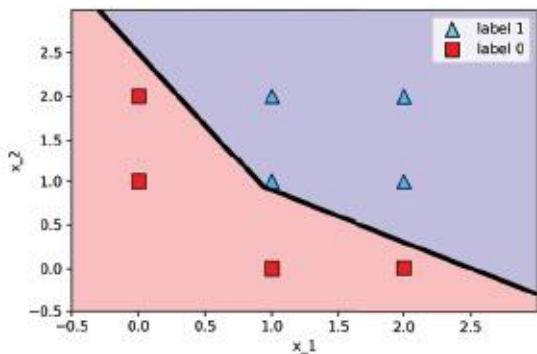
$$C = \text{step}(6x_a + 10x_b - 15)$$

$$F = \text{step}(10x_a + 6x_b - 15)$$



$$\hat{y} = \text{step}(1 \cdot C + 1 \cdot F - 1.5)$$

$$\hat{y} = \sigma(1 \cdot C + 1 \cdot F - 1.5)$$



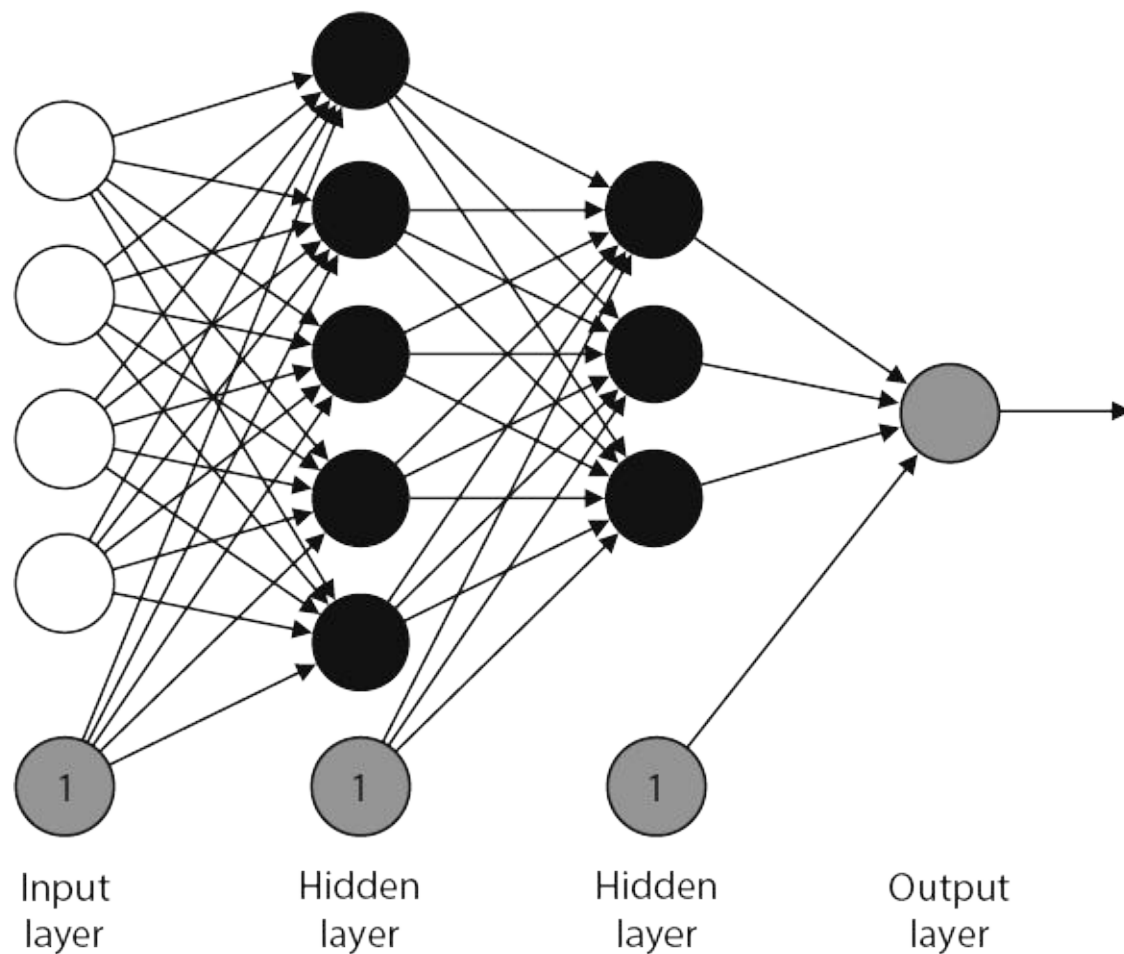
Hình 10.13 Đồ thị ranh giới của các phân loại. Trên cùng: Hai bộ phân loại tuyến tính, bộ phân loại nghề nghiệp (trái) và bộ phân loại gia đình (phải). Dưới cùng: Hai mạng thần kinh, sử dụng hàm bước (trái) và hàm sigmoid (phải).

Lưu ý rằng mạng nơ-ron có chức năng kích hoạt sigmoid thực sự không phù hợp với toàn bộ tập dữ liệu vì nó phân loại sai điểm (1,1), như thể hiện ở phía dưới bên phải của hình 10.13. Hãy thử thay đổi trọng lượng theo cách phù hợp nhất với điểm này. (Xem bài tập 10.3 ở cuối chương).

Kiến trúc chung của mạng nơ-ron được kết nối đầy đủ

Trong các phần trước, chúng ta đã thấy một ví dụ về mạng nơ-ron nhỏ, nhưng trong đời thực, mạng nơ-ron lớn hơn nhiều. Các nút được sắp xếp theo lớp, như minh họa trong hình 10.14. Lớp đầu tiên là lớp đầu vào, lớp cuối cùng là lớp đầu ra và tất cả các lớp ở giữa được gọi là lớp ẩn. Sự sắp xếp các nút và lớp được gọi là *kiến trúc* của mạng lưới thần kinh. Số lớp (không bao gồm lớp đầu vào) được gọi là *độ sâu* của mạng nơ-ron. Mạng nơ-ron trong hình 10.14 có độ sâu bằng 3 và có kiến trúc như sau:

- Lớp đầu vào có kích thước 4
- Lớp ẩn có kích thước 5
- Lớp ẩn có kích thước 3
- Lớp đầu ra có kích thước 1



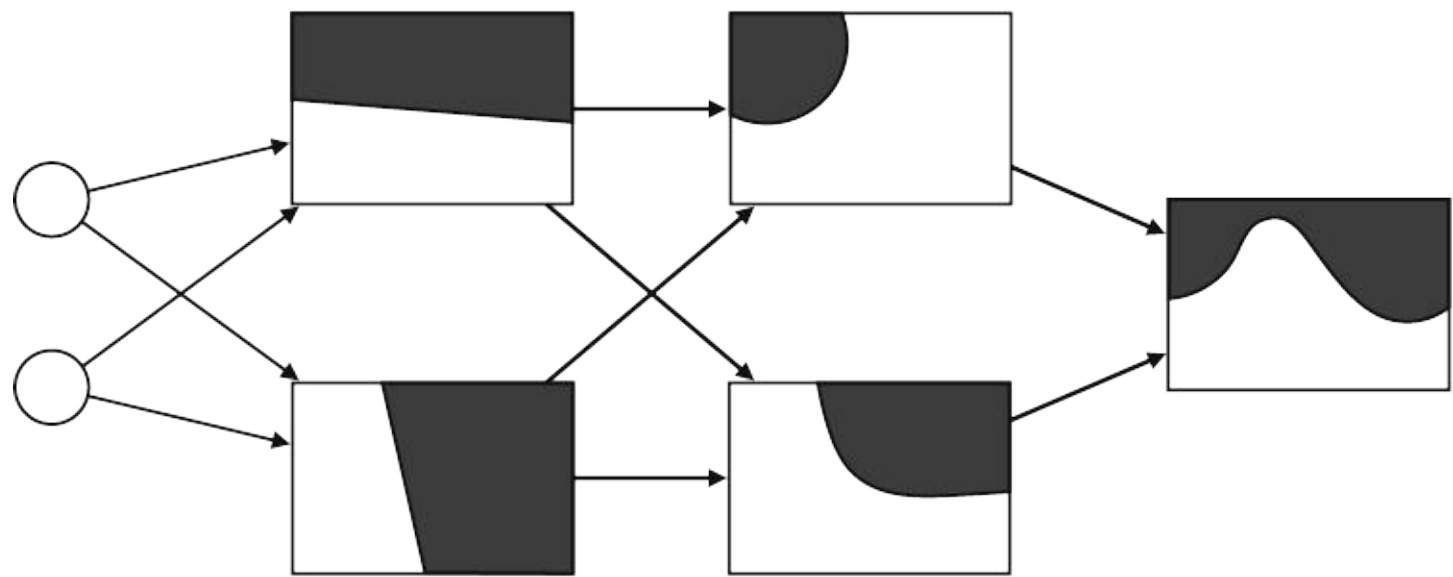
Hình 10.14 Kiến trúc chung của mạng nơ-ron. Các nút được chia thành các lớp, trong đó lớp ngoài cùng bên trái là lớp đầu vào, lớp ngoài cùng bên phải là lớp đầu ra và tất cả các lớp ở giữa là lớp ẩn. Tất cả các nút trong một lớp được kết nối với tất cả các nút (không thiên vị) trong lớp tiếp theo.

Mạng lưới thần kinh thường được vẽ mà không có các nút thiên vị, nhưng người ta cho rằng chúng là một phần của kiến trúc. Tuy nhiên, chúng tôi không tính các nút thiên vị trong kiến trúc. Nói cách khác, kích thước của một lớp là số nút không thiên vị trong lớp đó.

Lưu ý rằng trong mạng nơ-ron trong hình 10.14, mọi nút trong một lớp được kết nối với mọi nút (không thiên vị) ở lớp tiếp theo. Hơn nữa, không có kết nối nào xảy ra giữa các lớp không liên tiếp. Kiến trúc này được gọi là *được kết nối đầy đủ*. Đối với một số ứng dụng, chúng tôi sử dụng các kiến trúc khác nhau trong đó không có tất cả các kết nối ở đó hoặc trong đó một số nút được kết nối giữa các lớp không liên tiếp—

xem phần “Các kiến trúc khác dành cho các phương ngữ phức tạp hơn” để đọc về một số kiến trúc đó. Tuy nhiên, trong chương này, tất cả mạng lưới thần kinh mà chúng ta xây dựng đều được kết nối đầy đủ.

Hãy hình dung ranh giới của một mạng lưới thần kinh như trong hình 10.15. Trong sơ đồ này, bạn có thể thấy trình phân loại tương ứng với từng nút. Lưu ý rằng lớp ẩn đầu tiên được hình thành bởi các bộ phân loại tuyến tính và các bộ phân loại trong mỗi lớp kế tiếp phức tạp hơn một chút so với các lớp trước đó.



Hình 10.15 Cách tôi muốn hình dung mạng lưới thần kinh. Mỗi nút tương ứng với một bộ phân loại và bộ phân loại này có ranh giới được xác định rõ ràng. Các nút trong lớp ẩn đầu tiên đều tương ứng với các bộ phân loại tuyến tính (perceptron) nên chúng được vẽ dưới dạng các đường thẳng. Ranh giới của các nút trong mỗi lớp được hình thành bằng cách kết hợp các ranh giới từ lớp trước đó. Vì vậy, ranh giới trong mỗi lớp ẩn ngày càng phức tạp hơn. Trong sơ đồ này, chúng tôi đã loại bỏ các nút thiên vị.

Một công cụ tuyệt vời để sử dụng để hiểu mạng lưới thần kinh là TensorFlow Playground, có thể tìm thấy tại <https://playground.tensorflow.org>. Một số bộ dữ liệu đồ họa có sẵn ở đó và có thể huấn luyện các mạng thần kinh với các kiến trúc và siêu tham số khác nhau.

Đào tạo mạng lưới thần kinh

Trong chương này, chúng ta đã thấy mạng lưới thần kinh nói chung trông như thế nào và chúng không hề bí ẩn như vẻ ngoài của nó. Làm thế nào để chúng ta huấn luyện một trong những con quái vật này? Về lý thuyết, quá trình này không phức tạp, mặc dù nó có thể được tính toán

đất. Chúng tôi có một số thủ thuật và phương pháp phỏng đoán mà chúng tôi có thể sử dụng để tăng tốc độ. Trong phần này, chúng ta tìm hiểu quá trình đào tạo này. Huấn luyện mạng lưới thần kinh không khác mấy so với huấn luyện các mô hình khác, chẳng hạn như perceptron hoặc bộ phân loại logistic. Chúng ta bắt đầu bằng việc khởi tạo tất cả các trọng số và độ lệch một cách ngẫu nhiên. Tiếp theo, chúng tôi xác định hàm lỗi để đo hiệu suất của mạng nơ-ron. Cuối cùng, chúng tôi liên tục sử dụng hàm lỗi để điều chỉnh trọng số và độ lệch của mô hình nhằm giảm hàm lỗi.

Chức năng lỗi: Một cách để đo lường hiệu suất của mạng lưới thần kinh

Trong phần này, chúng ta tìm hiểu về hàm lỗi được sử dụng để huấn luyện mạng nơ-ron. May mắn thay, chúng ta đã từng thấy hàm này trước đây—hàm log-loss trong phần “Bộ phân loại logistic” ở chương 6. Hãy nhớ lại rằng công thức tính tổn thất log là

$$\log \text{ loss} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}),$$

trong đó y là nhãn và $\hat{y}$ là dự đoán.

Xin nhắc lại, lý do chính đáng để sử dụng tính năng mất nhật ký cho các vấn đề phân loại là nó trả về một giá trị nhỏ khi dự đoán và nhãn ở gần nhau và một giá trị lớn khi chúng ở xa.

Lan truyền ngược: Bước quan trọng trong việc huấn luyện mạng lưới thần kinh

Trong phần này, chúng ta tìm hiểu bước quan trọng nhất trong quá trình huấn luyện mạng lưới thần kinh. Hãy nhớ lại rằng trong các chương 3, 5 và 6 (hồi quy tuyến tính, thuật toán perceptron và hồi quy logistic), chúng tôi đã sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình của mình. Đây cũng là trường hợp của mạng lưới thần kinh. Thuật toán huấn luyện được gọi là *thuật toán lan truyền ngược* và mã giả của nó như sau:

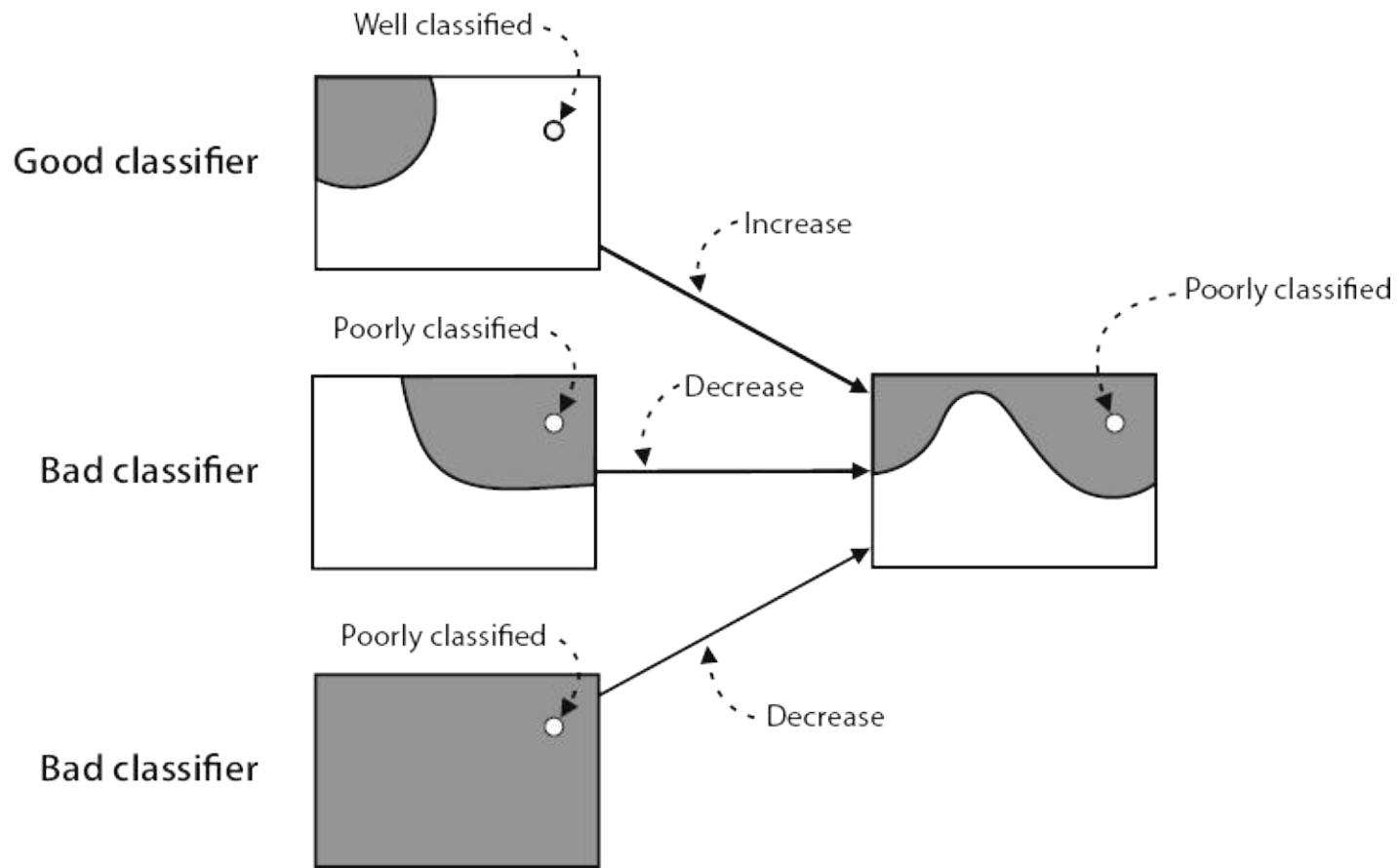
Mã giả cho thuật toán lan truyền ngược

- Khởi tạo mạng lưới thần kinh với các trọng số và độ lệch ngẫu nhiên.

- Lặp lại nhiều lần: Tính hàm mất mát và độ dốc của nó (cụ thể là các đạo hàm đối với từng trọng số và độ lệch). Thực hiện một bước nhỏ theo hướng ngược lại với gradient để giảm hàm mất mát đi một lượng nhỏ. Các trọng số bạn thu được tương ứng với một mạng lưới thần kinh (có thể) rất phù hợp với dữ liệu.
-

Hàm mất mát của mạng nơ-ron rất phức tạp vì nó liên quan đến logarit của dự đoán và bản thân dự đoán đó là một hàm phức tạp. Hơn nữa, chúng ta cần tính đạo hàm theo nhiều biến, tương ứng với từng trọng số và độ lệch của mạng nơ-ron. Trong phụ lục B, “Sử dụng phương pháp giảm độ dốc để huấn luyện mạng nơ-ron”, chúng tôi xem xét các chi tiết toán học của thuật toán lan truyền ngược cho mạng nơ-ron có một lớp ẩn hoặc kích thước tùy ý. Xem một số tài nguyên được đề xuất trong phụ lục C để đi sâu vào phép toán lan truyền ngược cho các mạng nơ-ron sâu hơn. Trong thực tế, các gói lớn như Keras, TensorFlow và PyTorch đã triển khai thuật toán này với tốc độ và hiệu suất cao.

Hãy nhớ lại rằng khi chúng ta học các mô hình hồi quy tuyến tính (chương 3), các perceptron rời rạc (chương 5) và các perceptron liên tục (chương 6), quá trình này luôn có một bước trong đó chúng ta di chuyển một đường theo cách chúng ta cần để mô hình hóa dữ liệu của mình một cách tốt nhất. Loại hình học này khó hình dung hơn đối với mạng lưới thần kinh vì nó xảy ra ở các chiều cao hơn nhiều. Tuy nhiên, chúng ta vẫn có thể hình dung một bức tranh tinh thần về lan truyền ngược và để làm được điều này, chúng ta chỉ cần tập trung vào một trong các nút của mạng nơ-ron và một điểm dữ liệu. Hãy tưởng tượng một bộ phân loại giống như cái bên phải trong hình 10.16. Bộ phân loại này được lấy từ ba bộ phân loại ở bên trái (bộ phân loại dưới cùng tương ứng với độ lệch mà chúng tôi biểu thị bằng một bộ phân loại luôn trả về dự đoán là 1). Trình phân loại kết quả phân loại sai điểm, như được hiển thị. Trong ba bộ phân loại đầu vào, bộ phân loại đầu tiên phân loại điểm tốt, nhưng hai bộ phân loại còn lại thì không. Do đó, bước lan truyền ngược sẽ tăng trọng số ở cạnh tương ứng với bộ phân loại trên cùng và giảm trọng số tương ứng với hai bộ phân loại ở phía dưới. Điều này đảm bảo trình phân loại kết quả sẽ trông giống trình phân loại trên cùng hơn và do đó, khả năng phân loại điểm của nó sẽ được cải thiện.



Hình 10.16 Một bức tranh tưởng tượng về lan truyền ngược. Ở mỗi bước của quá trình huấn luyện, trọng số của các cạnh được cập nhật. Nếu một bộ phân loại tốt, trọng số của nó sẽ tăng lên một lượng nhỏ, còn nếu nó xấu thì trọng số của nó sẽ giảm.

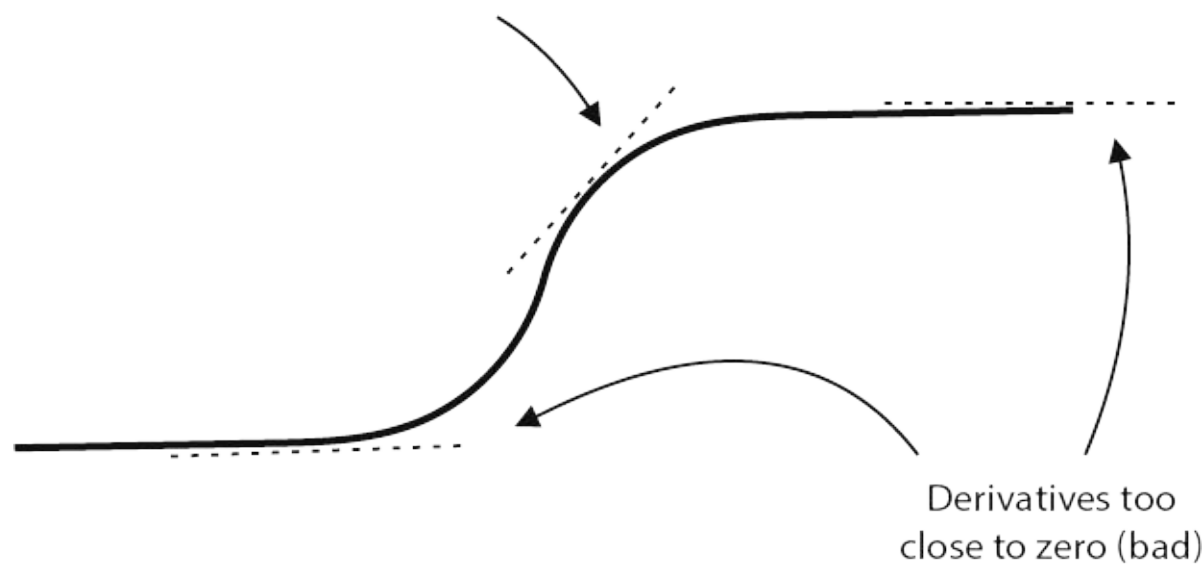
Các vấn đề tiềm ẩn: Từ trạng bị quá mức đến biến mất độ dốc

Trong thực tế, mạng lưới thần kinh hoạt động rất tốt. Tuy nhiên, do tính phức tạp của chúng nên nhiều vấn đề nảy sinh trong quá trình đào tạo. May mắn thay, chúng ta có thể có giải pháp cho những vấn đề cấp bách nhất. Một vấn đề

mà mạng lưới thần kinh có đang được trang bị quá mức—các kiến trúc thực sự lớn có thể ghi nhớ dữ liệu của chúng ta mà không cần khái quát hóa tốt. Trong phần tiếp theo, chúng ta sẽ thấy một số kỹ thuật để giảm tình trạng trang bị quá mức khi huấn luyện mạng lưới thần kinh.

Một vấn đề nghiêm trọng khác mà mạng lưới thần kinh có thể gặp phải là độ dốc biến mất. Lưu ý rằng hàm sigmoid rất phẳng ở hai đầu, điều này biểu thị rằng đạo hàm (tiếp tuyến của đường cong) quá phẳng (xem hình 10.17). Điều này có nghĩa là độ dốc của chúng rất gần bằng 0.

Large derivative (good)



Hình 10.17 Hàm sigmoid phẳng ở hai đầu, nghĩa là đối với các giá trị dương và âm lớn thì đạo hàm của nó rất nhỏ, cản trở việc huấn luyện.

Trong quá trình lan truyền ngược, chúng tôi kết hợp nhiều hàm sigmoid này (có nghĩa là chúng tôi cắm đầu ra của hàm sigmoid làm đầu vào cho một hàm sigmoid khác nhiều lần). Đúng như dự đoán, thành phần này dẫn đến các đạo hàm rất gần bằng 0, có nghĩa là các bước thực hiện trong quá trình truyền ngược là rất nhỏ. Nếu đúng như vậy, chúng ta có thể phải mất một thời gian rất dài để có được một bộ phân loại tốt, đây là một vấn đề.

Chúng ta có một số giải pháp cho vấn đề gradient biến mất và cho đến nay một trong những giải pháp hiệu quả nhất là thay đổi hàm kích hoạt. Trong phần “Các hàm kích hoạt khác nhau”, chúng ta tìm hiểu một số hàm kích hoạt mới để giúp chúng ta giải quyết vấn đề độ dốc biến mất.

Các kỹ thuật huấn luyện mạng nơ-ron: Chính quy hóa và loại bỏ

Như đã đề cập ở phần trước, mạng nơ-ron có xu hướng bị trang bị quá mức. Trong phần này, chúng ta thảo luận về một số kỹ thuật để giảm lượng trang bị quá mức trong quá trình đào tạo mạng lưới thần kinh.

Làm thế nào để chúng ta chọn kiến trúc chính xác? Đây là một câu hỏi khó, không có câu trả lời cụ thể. Nguyên tắc chung là bạn nên chọn một kiến trúc lớn hơn mức chúng ta có thể cần và sau đó áp dụng các kỹ thuật để giảm lượng trang bị quá mức mà mạng của bạn có thể có. Theo một cách nào đó, nó giống như việc chọn một chiếc quần, trong đó bạn chỉ có những lựa chọn là quá nhỏ hoặc quá lớn. Nếu chúng ta chọn quần

quá nhỏ, chúng ta không thể làm được gì nhiều. Mặt khác, nếu chọn quần quá rộng, chúng ta có thể đeo thắt lưng để vừa vặn hơn. Nó không phải là lý tưởng, nhưng đó là tất cả những gì chúng tôi có bây giờ. Chọn kiến trúc chính xác dựa trên tập dữ liệu là một vấn đề phức tạp và hiện có rất nhiều nghiên cứu đang được thực hiện theo hướng này. Để tìm hiểu thêm về điều này, hãy xem các tài nguyên trong phụ lục C.

QUY ĐỊNH: MỘT CÁCH GIẢM THIỂU TÌM HIỂU BẰNG CÁCH PHẠT TRỌNG LƯỢNG CAO HƠN

Như chúng ta đã học trong chương 4, chúng ta có thể sử dụng chính quy hóa L1 và L2 để giảm tình trạng quá khớp trong các mô hình hồi quy và phân loại, và mạng nơ-ron cũng không phải là ngoại lệ. Cách người ta áp dụng chính quy hóa trong mạng nơ-ron cũng giống như cách người ta áp dụng nó trong hồi quy tuyến tính—bằng cách thêm một thuật ngữ chính quy hóa vào hàm lỗi. Nếu chúng ta đang thực hiện chính quy hóa L1, thuật ngữ chính quy hóa sẽ bằng tham số chính quy hóa (λ) nhân với tổng các giá trị tuyệt đối của tất cả các trọng số trong mô hình của chúng ta (không bao gồm các độ lệch). Nếu chúng ta đang thực hiện chính quy hóa L2 thì chúng ta lấy tổng bình phương thay vì giá trị tuyệt đối. Ví dụ, lỗi chính quy L2 của mạng nơ-ron trong ví dụ ở phần “Mạng nơ-ron có ví dụ” là

$$\log \text{loss} + \lambda \cdot (6^2 + 10^2 + 10^2 + 6^2 + 1^2 + 1^2) = \log \text{loss} + 274\lambda.$$

Bỏ học: ĐẢM BẢO MỘT SỐ NÚT MẠNH KHÔNG CHIA SẺ ĐÀO TẠO

Dropout là một kỹ thuật thú vị được sử dụng để giảm việc trang bị quá mức trong mạng lưới thần kinh và để hiểu nó, chúng ta hãy xem xét phép tương tự sau: hãy tưởng tượng rằng chúng ta thuận tay phải và chúng ta thích đi đến phòng tập thể dục. Sau một thời gian, chúng ta bắt đầu nhận thấy bắp tay bên phải của mình đang phát triển rất nhiều, còn bắp tay bên trái thì không. Sau đó, chúng ta bắt đầu chú ý hơn đến việc tập luyện và nhận ra rằng vì thuận tay phải nên chúng ta có xu hướng luôn nâng tạ bằng tay phải và không cho tay trái tập luyện nhiều. Chúng tôi quyết định thế là đủ nên chúng tôi thực hiện một biện pháp quyết liệt. Một số ngày, chúng tôi quyết định trói tay phải của mình vào lưng và buộc mình phải thực hiện toàn bộ động tác mà không cần sử dụng cánh tay phải. Sau đó, chúng ta bắt đầu thấy cánh tay trái bắt đầu phát triển như mong muốn. Bây giờ, để cả hai cánh tay đều hoạt động, chúng ta làm như sau: mỗi ngày trước khi đến phòng tập thể dục, chúng ta tung hai đồng xu, một đồng xu cho mỗi cánh tay. Nếu đồng xu bên trái rơi vào mặt đầu, chúng ta buộc cánh tay trái vào lưng, và nếu cánh tay phải rơi vào mặt đầu, chúng ta buộc cánh tay phải vào lưng. Một số ngày chúng tôi sẽ

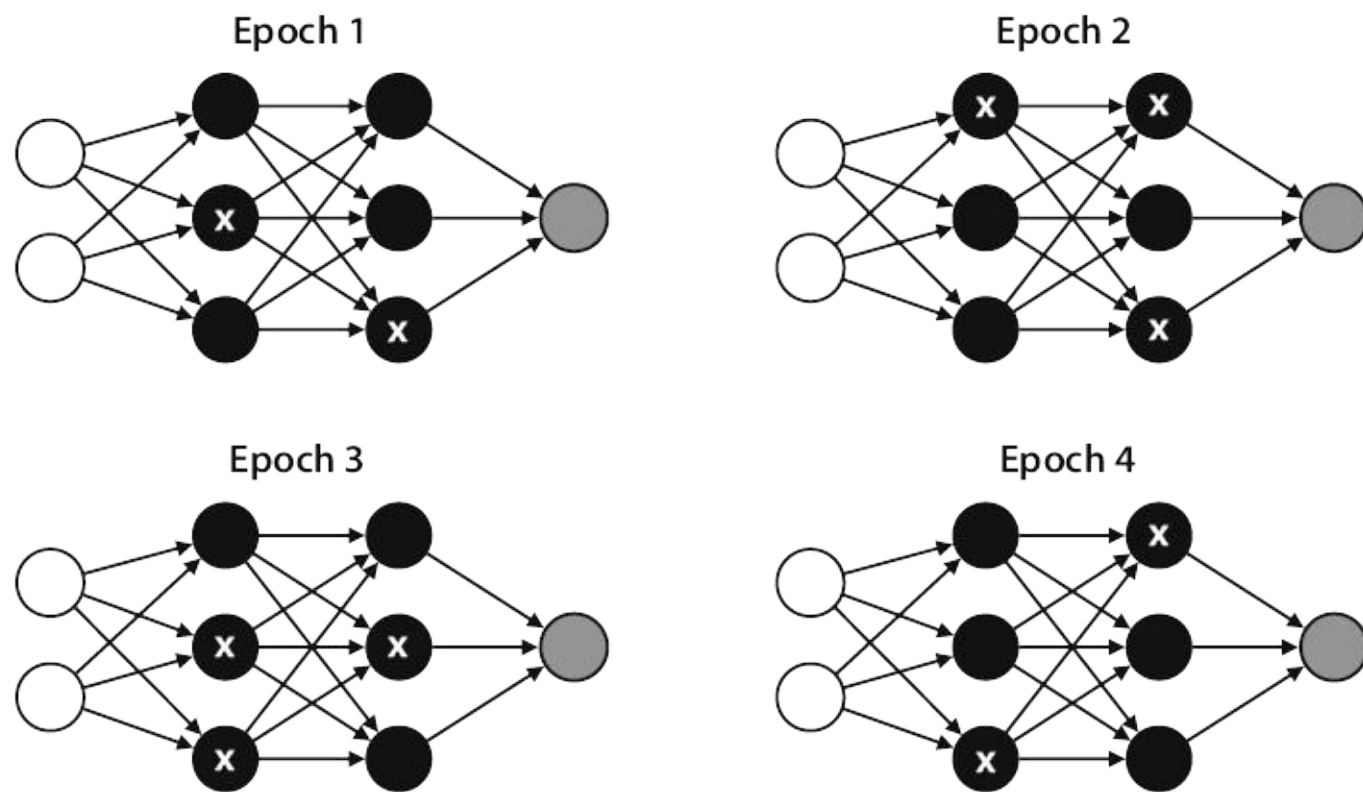
làm việc bằng cả hai tay, một số ngày chỉ dùng một tay và một số ngày không dùng tay nào (có lẽ đó là những ngày dùng chân). Tính ngẫu nhiên của các đồng xu sẽ đảm bảo rằng, về trung bình, chúng ta đang làm việc gần như bằng nhau ở cả hai nhánh.

Dropout sử dụng logic này, ngoại trừ thay vì cánh tay, chúng tôi đang huấn luyện các trọng lượng trong mạng lưới thần kinh. Khi mạng nơ-ron có quá nhiều nút, một số nút sẽ chọn các mẫu trong dữ liệu hữu ích để đưa ra dự đoán chính xác, trong khi các nút khác chọn các mẫu không phù hợp hoặc nhiễu. Quá trình loại bỏ sẽ loại bỏ một số nút ngẫu nhiên ở mỗi giai đoạn và thực hiện một bước giảm độ dốc trên các nút còn lại. Bằng cách loại bỏ một số nút ở mỗi kỷ nguyên, có khả năng là đôi khi chúng ta có thể loại bỏ những nút đã chọn được các mẫu hữu ích, do đó buộc các nút khác phải xử lý phần thiếu sót.

Cụ thể hơn, quá trình loại bỏ gán một xác suất nhỏ p cho mỗi nơ-ron. Trong mỗi kỷ nguyên của quá trình huấn luyện, mỗi nơ-ron bị loại bỏ với xác suất p và mạng nơ-ron chỉ được huấn luyện với những nơ-ron còn lại. Bỏ học chỉ được sử dụng trên ẩn

các lớp, không phải trên các lớp đầu vào hoặc đầu ra. Quá trình loại bỏ được minh họa trong hình 10.18, trong đó một số nơ-ron bị loại bỏ trong mỗi giai đoạn trong số bốn giai đoạn huấn luyện.

Dropout đã đạt được thành công lớn trong quá trình thực hành và tôi khuyến khích bạn sử dụng nó mỗi khi huấn luyện mạng lưới thần kinh. Các gói mà chúng tôi sử dụng để huấn luyện mạng nơ-ron làm cho nó dễ sử dụng, như chúng ta sẽ thấy ở phần sau của chương này.



Hình 10.18 Quá trình bỏ học. Ở các thời điểm khác nhau, chúng tôi chọn các nút ngẫu nhiên để loại bỏ khỏi quá trình đào tạo nhằm tạo cơ hội cho tất cả các nút cập nhật trọng số của chúng và không để một vài nút duy nhất chiếm ưu thế trong quá trình đào tạo.

Các hàm kích hoạt khác nhau: Tiếp tuyến hyperbol ($\tanh$) và đơn vị tuyến tính chỉnh lưu (ReLU)

Như chúng ta đã thấy trong phần “Các vấn đề tiềm ẩn”, hàm sigmoid hơi quá phẳng, điều này gây ra các vấn đề về độ biến thiên biến mất. Giải pháp cho vấn đề này là sử dụng các chức năng kích hoạt khác nhau. Trong phần này, chúng tôi đề cập đến hai hàm kích hoạt khác nhau rất quan trọng để cải thiện quá trình đào tạo của chúng tôi: tang hyperbol ($\tanh$) và đơn vị tuyến tính chỉnh lưu (ReLU)

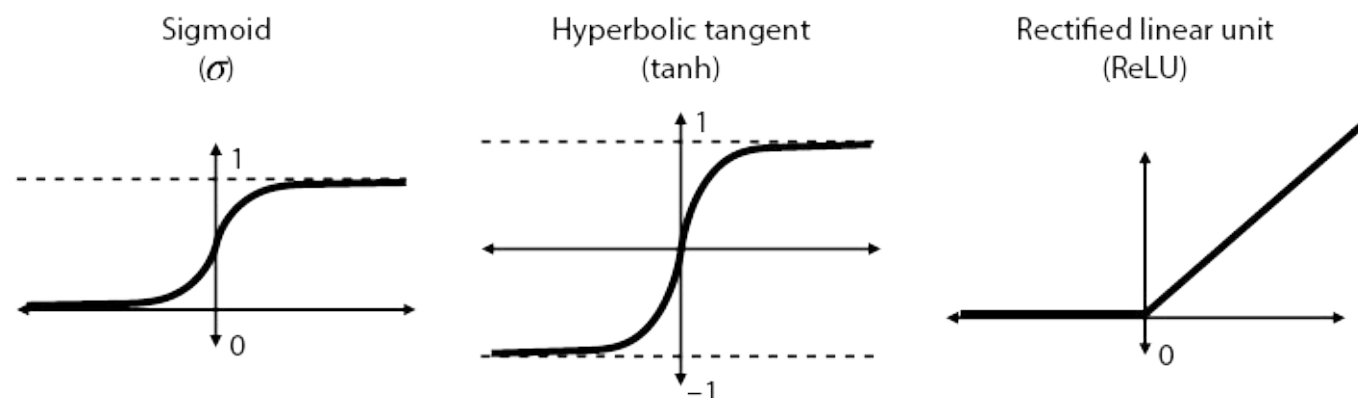
TANG TUYỆT VỜI (TANH)

Hàm *tang hyperbol* có xu hướng hoạt động tốt hơn hàm sigmoid trong thực tế do hình dạng của nó và được cho bởi công thức sau:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tính kém phẳng hơn sigmoid một chút, nhưng nó vẫn có hình dạng tương tự, như minh họa trong hình 10.19. Nó cung cấp một sự cải tiến so với sigmoid, nhưng nó vẫn gặp phải vấn đề

vấn đề độ dốc biến mất.



Hình 10.19 Ba hàm kích hoạt khác nhau được sử dụng trong mạng nơ-ron. Bên trái: Hàm sigmoid, được biểu thị bằng chữ cái Hy Lạp *sigma*. Ở giữa: Tiếp tuyến hyperbol, hay tanh. Bên phải: Đơn vị tuyến tính được chỉnh lưu, hoặc ReLU.

ĐƠN VỊ TUYẾN TÍNH CHỈNH LƯU (RELU)

Một kích hoạt phổ biến hơn nhiều thường được sử dụng trong mạng nơ-ron là *đơn vị tuyến tính được chỉnh lưu* hoặc ReLU. Điều này rất đơn giản: nếu đầu vào âm thì đầu ra bằng 0; mặt khác, đầu ra bằng đầu vào. Nói cách khác, nó để lại các số không âm và biến tất cả các số âm thành 0. Đối với $x \geq 0$, $ReLU(x) = x$ và đối với $x < 0$, $ReLU(x) = 0$. ReLU là một giải pháp tốt để bài toán gradient biến mất, vì đạo hàm của nó bằng 1 khi đầu vào dương, và do đó, nó được sử dụng rộng rãi trong các mạng nơ-ron lớn.

Điều tuyệt vời về các chức năng kích hoạt này là chúng ta có thể kết hợp các chức năng khác nhau trong cùng một mạng lưới thần kinh. Trong một trong những kiến trúc phổ biến nhất, mọi nút đều sử dụng chức năng kích hoạt ReLU ngoại trừ nút cuối cùng sử dụng sigmoid. Lý do cuối cùng cho sigmoid này là vì nếu vấn đề của chúng ta là vấn đề phân loại thì đầu ra của mạng nơ-ron phải nằm trong khoảng từ 0 đến 1.

Mạng thần kinh có nhiều hơn một đầu ra: Hàm softmax

Cho đến nay, các mạng nơ-ron mà chúng tôi làm việc cùng chỉ có một đầu ra. Tuy nhiên, không khó để xây dựng một mạng nơ-ron tạo ra nhiều đầu ra bằng hàm softmax mà chúng ta đã học trong phần “Phân loại thành nhiều lớp: Hàm softmax” ở chương 6. Hàm softmax là một phần mở rộng đa biến của sigmoid và chúng ta có thể sử dụng nó để biến điểm số thành xác suất.

Cách tốt nhất để minh họa hàm softmax là bằng một ví dụ. Hãy tưởng tượng rằng chúng ta có một mạng lưới thần kinh có nhiệm vụ xác định xem một hình ảnh có chứa một con lợn đất, một con chim, một con mèo hay một con chó hay không. Ở lớp cuối cùng, chúng ta có bốn nút, một nút tương ứng với mỗi con vật. Thay vì áp dụng hàm sigmoid cho các điểm đến từ lớp trước, chúng tôi áp dụng hàm softmax cho tất cả chúng. Ví dụ: nếu điểm là 0, 3, 1 và 1, softmax trả về như sau:

$$\bullet \text{ Probability(aardvark)} = \frac{e^0}{e^0 + e^3 + e^1 + e^1} = 0.0377$$

$$\bullet \text{ Probability(bird)} = \frac{e^3}{e^0 + e^3 + e^1 + e^1} = 0.7573$$

$$\bullet \text{ Probability(cat)} = \frac{e^1}{e^0 + e^3 + e^1 + e^1} = 0.1025$$

$$\bullet \text{ Probability(dog)} = \frac{e^1}{e^0 + e^3 + e^1 + e^1} = 0.1025$$

Những kết quả này chỉ ra rằng mạng lưới thần kinh tin tưởng mạnh mẽ rằng hình ảnh tương ứng với một con chim.

Siêu tham số

Giống như hầu hết các thuật toán học máy, mạng lưới thần kinh sử dụng nhiều siêu tham số mà chúng ta có thể tinh chỉnh để chúng hoạt động tốt hơn. Các siêu tham số này xác định cách chúng tôi thực hiện quá trình đào tạo, cụ thể là chúng tôi muốn quá trình diễn ra trong bao lâu, tốc độ như thế nào và cách chúng tôi chọn nhập dữ liệu vào mô hình. Một số siêu tham số quan trọng nhất trong mạng lưới thần kinh như sau:

- **Tốc độ học tập η :** kích thước của bước chúng tôi sử dụng trong quá trình đào tạo **Số kỷ**
- **nguyên:** số bước chúng tôi sử dụng cho quá trình đào tạo của mình **Hàng loạt so với lô nhỏ so**
- **với giảm độ dốc ngẫu nhiên:** có bao nhiêu điểm cùng một lúc trong quá trình đào tạo—cụ thể là, chúng tôi nhập từng điểm một, theo đợt hay tất cả cùng một lúc?
- **Kiến trúc:**
 - Số lớp trong mạng nơ-ron Số lượng nút trên mỗi lớp
 - Các hàm kích hoạt được sử dụng trong mỗi nút
 -
- **Các tham số chính quy hóa:**
 - Chính quy hóa L1 hoặc L2
 - Thuật ngữ chính quy hóa λ

- **Xác suất bỏ học p**

Chúng tôi điều chỉnh các siêu tham số này giống như cách chúng tôi điều chỉnh chúng cho các thuật toán khác, sử dụng các phương pháp như tìm kiếm dạng lưới. Trong chương 13, chúng tôi sẽ trình bày chi tiết hơn về các phương pháp này bằng một ví dụ thực tế.

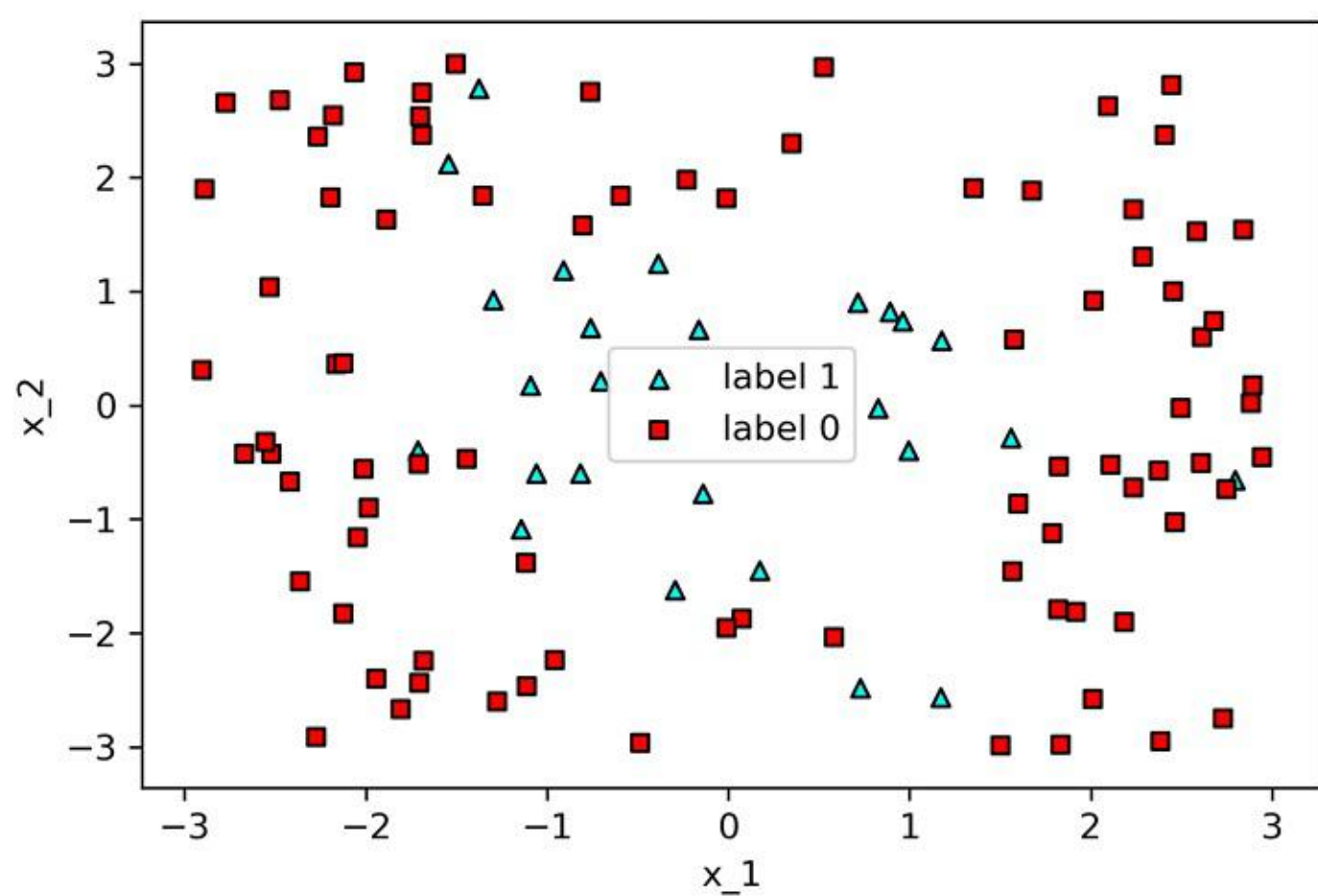
Mã hóa mạng nơ-ron trong Keras

Bây giờ chúng ta đã học được lý thuyết đằng sau mạng lưới thần kinh, đã đến lúc áp dụng chúng vào thực tế! Nhiều gói tuyệt vời đã được viết cho mạng lưới thần kinh, chẳng hạn như Keras, TensorFlow và PyTorch. Cả ba đều mạnh mẽ và trong chương này, chúng ta sẽ sử dụng Keras do tính đơn giản của nó. Chúng tôi sẽ xây dựng hai mạng lưới thần kinh cho hai bộ dữ liệu khác nhau. Tập dữ liệu đầu tiên chứa các điểm có hai đặc điểm và nhãn 0 và 1. Tập dữ liệu có hai chiều, vì vậy chúng ta có thể xem xét ranh giới phi tuyến do mô hình tạo ra. Tập dữ liệu thứ hai là tập dữ liệu phổ biến được sử dụng trong nhận dạng hình ảnh được gọi là tập dữ liệu MNIST (Viện Tiêu chuẩn và Công nghệ Quốc gia đã sửa đổi). Tập dữ liệu MNIST chứa các chữ số viết tay mà chúng ta có thể phân loại bằng mạng thần kinh.

Ví dụ đồ họa hai chiều

Trong phần này, chúng ta sẽ huấn luyện mạng nơ-ron trong Keras trên tập dữ liệu được hiển thị trong hình 10.20. Tập dữ liệu chứa hai nhãn 0 và 1. Các điểm có nhãn 0 được vẽ dưới dạng hình vuông và các điểm có nhãn 1 được vẽ dưới dạng hình tam giác. Lưu ý rằng các điểm có nhãn 1 hầu hết nằm ở trung tâm, trong khi các điểm có nhãn 0 nằm ở hai bên. Đối với loại tập dữ liệu này, chúng ta cần một bộ phân loại có ranh giới phi tuyến, điều này làm cho nó trở thành một ví dụ điển hình cho mạng nơ-ron. Mã cho phần này như sau:

- **Sổ tay:** Graphical_example.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_
- **Tập dữ liệu:** one_circle.csv



Hình 10.20 Mạng nơ-ron rất phù hợp cho các tập hợp có thể phân tách phi tuyến tính. Để kiểm tra điều này, chúng tôi sẽ huấn luyện mạng lưới thần kinh trên tập dữ liệu vòng tròn này.

Trước khi huấn luyện mô hình, hãy xem xét một số hàng ngẫu nhiên trong dữ liệu của chúng tôi. Đầu vào sẽ được gọi là x , với các tính năng x_1 và x_2 , và đầu ra sẽ được gọi là y . Bảng 10.3 có một số điểm dữ liệu mẫu. Tập dữ liệu có 110 hàng.

Bảng 10.3 Tập dữ liệu gồm 110 hàng, 2 đối tượng và nhãn 0 và 1

x_1	x_2	y
-0.759416	2.753240	0
-1.885278	1.629527	0
...	...	...
0.729767	-2.479655	1
-1.715920	-0.393404	1

Trước khi xây dựng và huấn luyện mạng lưới thần kinh, chúng ta phải thực hiện một số quá trình tiền xử lý dữ liệu.

PHÂN LOẠI DỮ LIỆU CỦA CHÚNG TÔI: CHUYỂN ĐỔI KHÔNG NHỊ PHÂN

Tính năng thành dạng nhị phân

Trong tập dữ liệu này, đầu ra là một số từ 0 đến 1, nhưng nó đại diện cho hai lớp. Trong Keras, nên phân loại loại đầu ra này. Điều này đơn giản có nghĩa là các điểm có nhãn 0 giờ đây sẽ có nhãn [1,0] và các điểm có nhãn 1 giờ đây sẽ có nhãn [0,1]. Chúng tôi thực hiện việc này bằng cách sử dụng hàm như

`to_categorical`

sau:

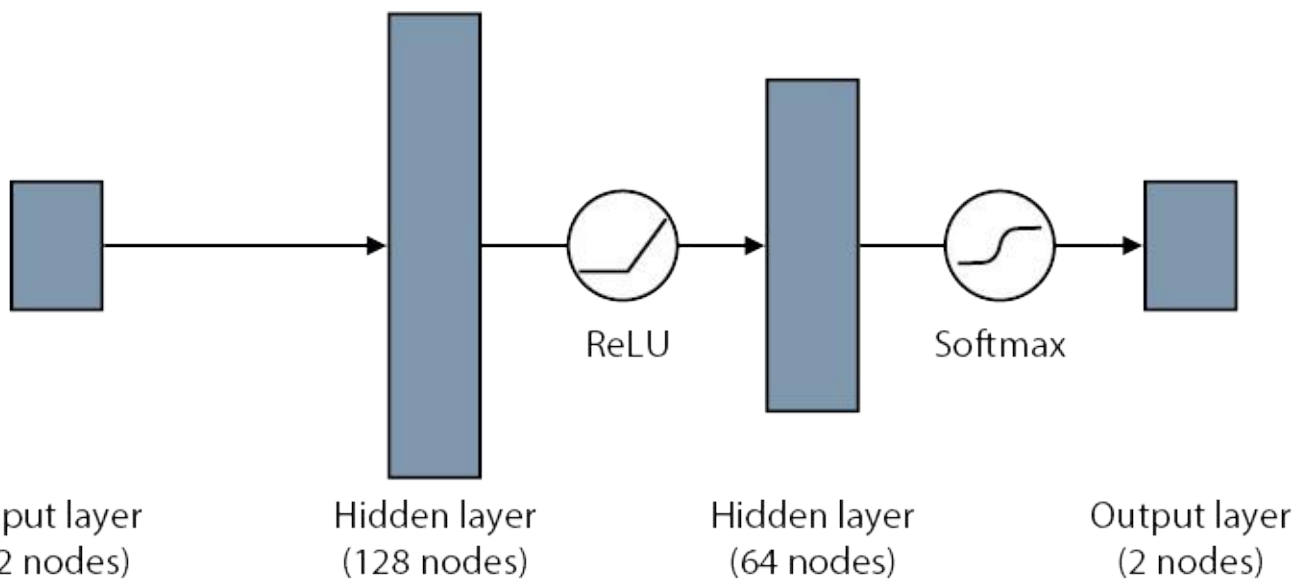
```
from tensorflow.keras.utils import to_categorical
categorized_y = np.array(to_categorical(y, 2))
```

Các nhãn mới được gọi là `categorized_y`.

CẤU TRÚC CỦA MẠNG THẦN KINH

Trong phần này, chúng tôi xây dựng kiến trúc mạng nơ-ron cho tập dữ liệu này. Việc quyết định sử dụng kiến trúc nào không phải là một môn khoa học chính xác, nhưng người ta thường khuyên nên lớn hơn một chút thay vì nhỏ hơn một chút. Đối với tập dữ liệu này, chúng tôi sẽ sử dụng kiến trúc sau với hai lớp ẩn (hình 10.21):

- Kích thước lớp đầu vào: 2 Lớp ẩn đầu tiên Kích thước: 128 Chức năng kích hoạt: ReLU
- Lớp ẩn thứ hai Kích thước: 64 Chức năng kích hoạt: ReLU
- Kích thước lớp đầu ra: 2 Chức năng kích hoạt: softmax
-
-
-
-
-
-



Hình 10.21 Kiến trúc mà chúng tôi sẽ sử dụng để phân loại tập dữ liệu của mình. Nó chứa hai lớp ẩn: một trong 128 nút và một trong 64 nút. Hàm kích hoạt giữa chúng là ReLU và hàm kích hoạt cuối cùng là softmax.

Hơn nữa, chúng tôi sẽ thêm các lớp bỏ học giữa các lớp ẩn của chúng tôi với xác suất bỏ học là 0,2 để ngăn chặn tình trạng trang bị quá mức.

XÂY DỰNG MÔ HÌNH TRONG KERAS

Việc xây dựng mạng lưới thần kinh chỉ mất một vài dòng mã trong Keras. Đầu tiên chúng ta nhập các gói và chức năng cần thiết như sau:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Activation
```

Bây giờ, hãy bắt đầu xác định mô hình với kiến trúc mà chúng ta đã xác định trong tiêu mục trước. Đầu tiên, chúng ta xác định mô hình với dòng sau:

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(2,)))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(2, activation='softmax'))
```

☐ Xác định mô hình ☐ Thêm lớp ẩn đầu tiên với chức năng kích hoạt ReLU
☐ Thêm dropout với xác suất 0,2 ☐ Thêm lớp ẩn thứ hai với chức năng kích hoạt ReLU ☐ Thêm lớp đầu ra với chức năng kích hoạt softmax

Khi mô hình được xác định, chúng ta có thể biên dịch nó, như được hiển thị ở đây:

```
model.compile(loss = 'categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Các tham số trong hàm như `compile`

- `loss = 'categorical_crossentropy'`: đây là hàm mất mát mà chúng ta đã được xác định là mất nhật ký.Vì nhãn của chúng tôi có nhiều hơn một cột nên chúng tôi cần sử dụng phiên bản đa biến cho hàm mất nhật ký, được gọi là entropy chéo phân loại.
- `optimizer = 'adam'`: các gói như Keras có nhiều thủ thuật tích hợp sẵn giúp chúng tôi đào tạo một mô hình một cách tối ưu.Việc thêm trình tối ưu hóa vào quá trình đào tạo của chúng tôi luôn là một ý tưởng hay.Một số cái tốt nhất là Adam, SGD, RMSProp và AdaGrad. Hãy thử khóa đào tạo tương tự này với các trình tối ưu hóa khác và xem họ thực hiện như thế nào.
- `metrics = ['accuracy']`: Trong quá trình đào tạo, chúng tôi nhận được báo cáo về cách thức mô hình đang làm ở mỗi thời đại.Cờ này cho phép chúng tôi xác định những số liệu nào chúng tôi muốn xem trong quá trình đào tạo và đối với ví dụ này, chúng tôi đã chọn độ chính xác.

Khi chạy mã, chúng ta nhận được bản tóm tắt về kiến trúc và số lượng tham số như sau:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 128)	384

dropout (Dropout)	(None, 128)	0

dense_1 (Dense)	(None, 64)	8256

dropout_1 (Dropout)	(None, 64)	0

dense_2 (Dense)	(None, 2)	130
=====		
Total params: 8,770		
Trainable params: 8,770		
Non-trainable params: 0		

Mỗi hàng trong kết quả đầu ra trước đó là một lớp (các lớp bỏ đi được coi là các lớp riêng biệt cho mục đích mô tả).Các cột tương ứng với loại lớp, hình dạng (số nút) và số lượng tham số, chính xác là số lượng trọng số cộng với số lượng độ lệch.Mô hình này có tổng cộng 8.770 tham số có thể huấn luyện.

ĐÀO TẠO NGƯỜI MẪU

Để đào tạo, một dòng mã đơn giản là đủ, được hiển thị bên dưới:

```
model.fit(x, categorized_y, epochs=100, batch_size=10)
```

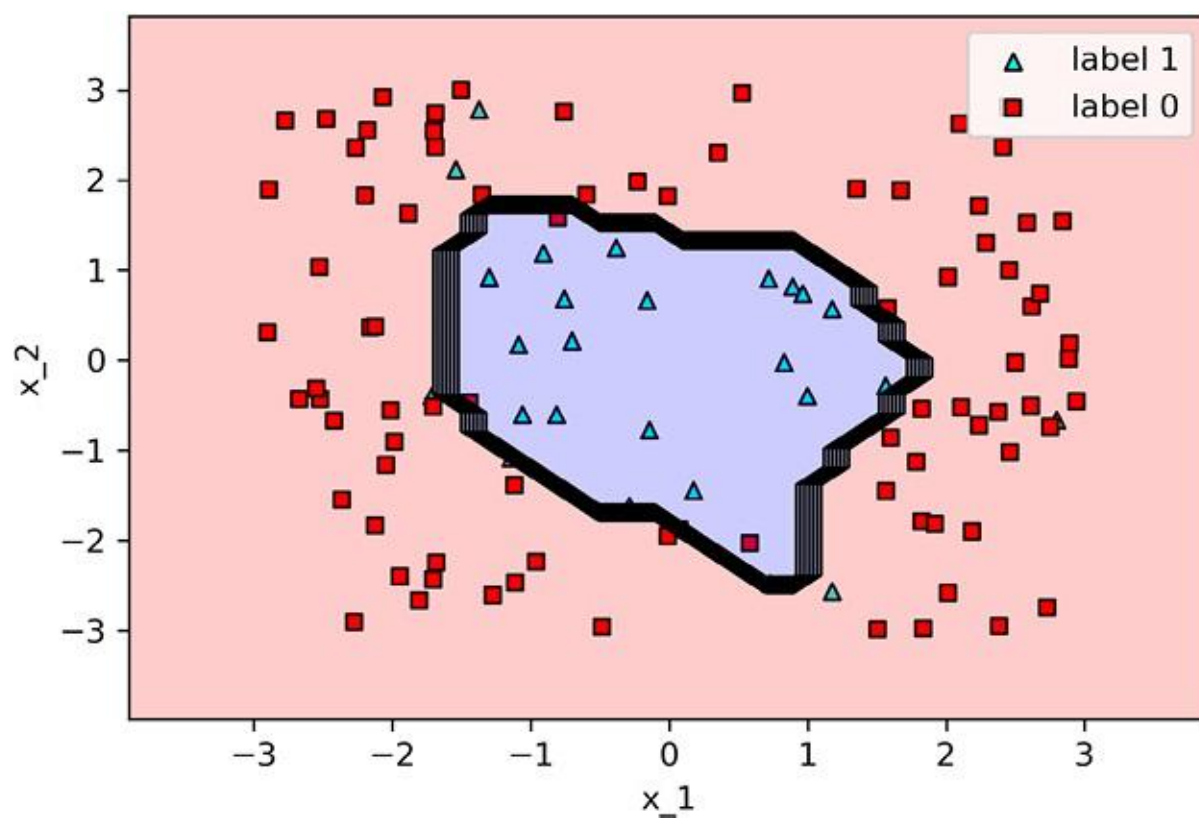
Hãy kiểm tra từng đầu vào của hàm phù hợp này.

- `x` and `categorized_y`: các tính năng và nhãn tương ứng.
- `epochs`: số lần chúng tôi chạy lan truyền ngược trên toàn bộ tập dữ liệu của mình. Ở đây chúng tôi làm điều đó 100 lần. : độ dài của các lô mà chúng tôi sử dụng để huấn luyện mô hình của mình. Ở đây, chúng tôi đang giới thiệu dữ liệu của mình cho mô hình theo lô 10. Đối với một tập dữ liệu nhỏ như thế này, chúng tôi không cần nhập dữ liệu theo lô, nhưng trong ví dụ này, chúng tôi đang thực hiện việc đó để hiển thị.

Khi mô hình huấn luyện, nó đưa ra một số thông tin ở mỗi thời điểm, cụ thể là độ mất (hàm lỗi) và độ chính xác. Ngược lại, hãy chú ý tiếp theo rằng kỷ nguyên đầu tiên có độ suy hao cao và độ chính xác thấp, trong khi kỷ nguyên cuối cùng có kết quả tốt hơn nhiều ở cả hai chỉ số:

```
Epoch 1/100
11/11 [=====] - 0s 2ms/step - loss: 0.5473 - accuracy: 0.7182
...
Epoch 100/100
11/11 [=====] - 0s 2ms/step - loss: 0.2110 - accuracy: 0.9000
```

Độ chính xác cuối cùng của mô hình trong quá trình huấn luyện là 0,9. Điều này là tốt, mặc dù hãy nhớ rằng độ chính xác phải được tính toán trong bộ thử nghiệm. Tôi sẽ không làm điều đó ở đây, nhưng hãy thử chia tập dữ liệu thành tập huấn luyện và tập kiểm tra, đồng thời huấn luyện lại mạng thần kinh này để xem bạn đạt được độ chính xác khi kiểm tra như thế nào. Hình 10.22 cho thấy sơ đồ ranh giới của mạng lưới thần kinh.



Hình 10.22 Ranh giới của bộ phân loại mạng thần kinh mà chúng tôi đã đào tạo. Lưu ý rằng nó phân loại chính xác hầu hết các điểm, với một vài ngoại lệ.

Lưu ý rằng mô hình đã phân loại dữ liệu khá tốt, bao quanh các hình tam giác và để các hình vuông ở bên ngoài. Nó đã mắc một số lỗi do dữ liệu bị nhiễu, nhưng điều này không sao cả. Ranh giới gian lận gợi ý mức độ trang bị quá mức nhỏ, nhưng nhìn chung nó có vẻ là một mô hình tốt.

Tàu hoả thiết lập mạng lưới thần kinh để nhận dạng hình ảnh

Trong phần này, chúng ta tìm hiểu cách huấn luyện mạng lưới thần kinh để nhận dạng hình ảnh. Tập dữ liệu chúng tôi sử dụng là MNIST, một tập dữ liệu phổ biến để nhận dạng hình ảnh, chứa 70.000 chữ số viết tay từ 0 đến 9. Nhãn của mỗi hình ảnh là chữ số tương ứng. Mỗi hình ảnh thang độ xám xuất hiện dưới dạng ma trận 28×28 gồm các số từ 0 đến 255, trong đó 0 đại diện cho màu trắng, 255 đại diện cho màu đen và bất kỳ số nào ở giữa đại diện cho một màu xám. Mã cho phần này như sau:

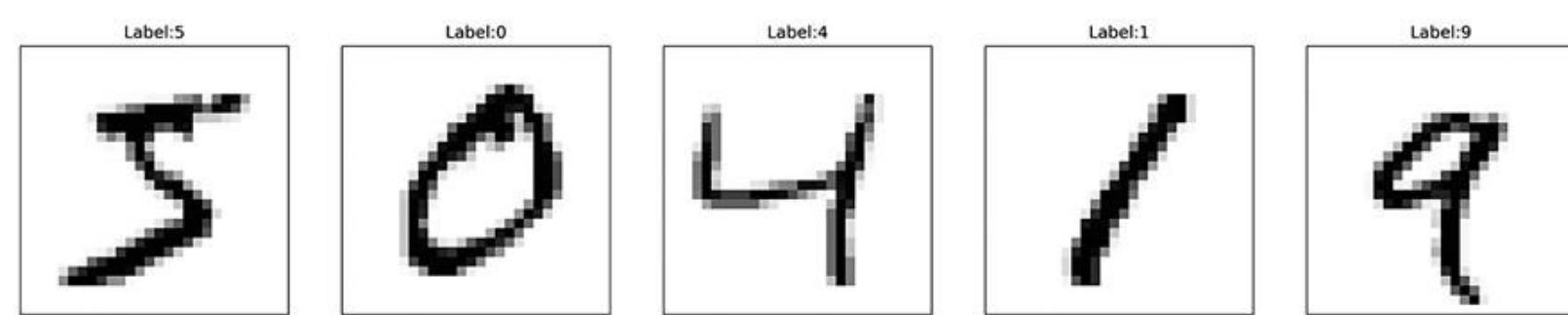
- **Sổ tay:** Image_recognition.ipynb
https://github.com/luiguiserrano/manning/blob/master/Chapter_10_Neural_
- **Bộ dữ liệu:** MNIST (được tải sẵn Keras)

TÀI DỮ LIỆU

Tập dữ liệu này được tải sẵn trong Keras, vì vậy rất dễ tải nó vào mảng NumPy. Trên thực tế, nó đã được tách thành các tập huấn luyện và kiểm tra có kích thước lần lượt là 60.000 và 10.000. Các dòng mã sau sẽ tải chúng vào mảng NumPy:

```
from tensorflow import keras
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

Trong hình 10.23, bạn có thể thấy năm hình ảnh đầu tiên trong tập dữ liệu kèm theo nhãn của chúng



Hình 10.23 Một số ví dụ về chữ số viết tay trong MNIST có nhãn

XỬ LÝ TRƯỚC DỮ LIỆU

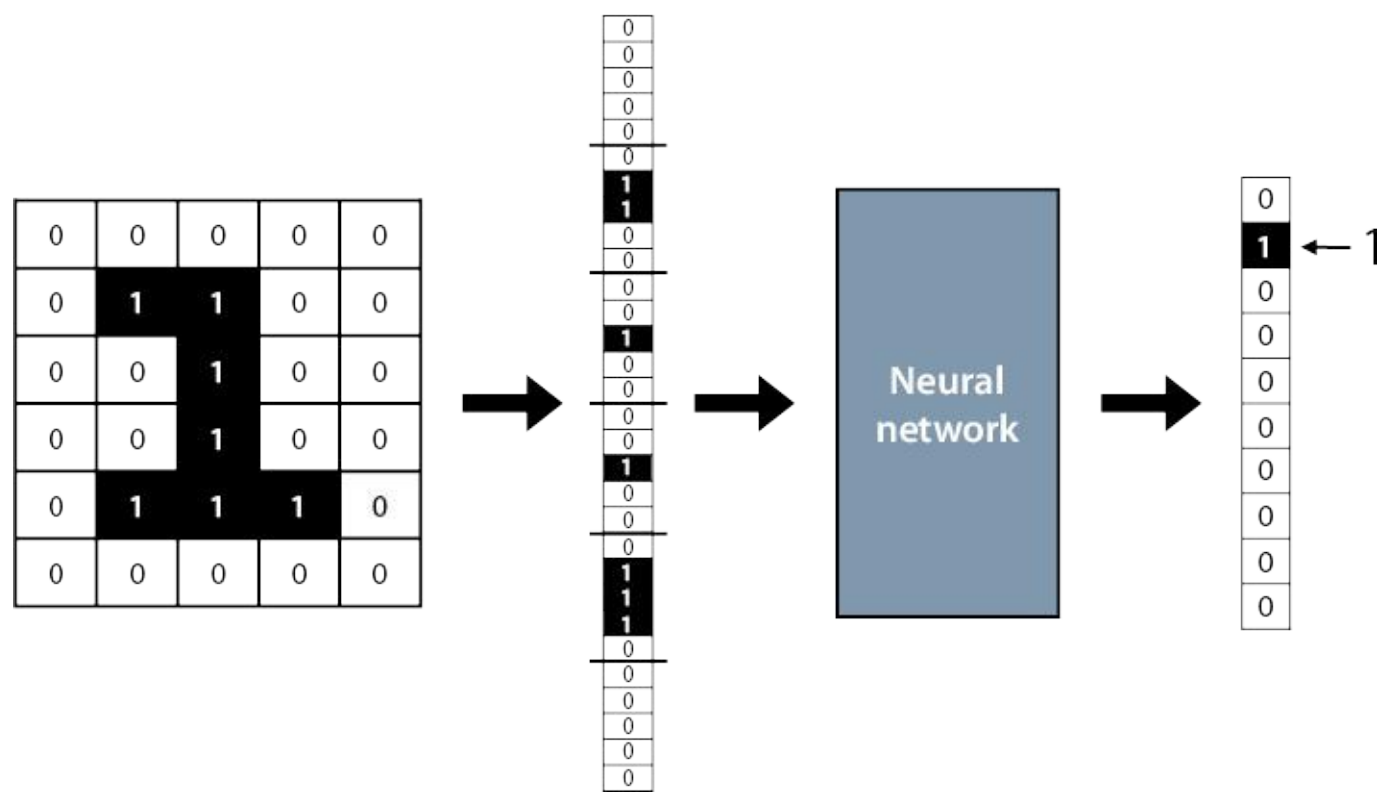
Mạng thần kinh nhận vector làm đầu vào thay vì ma trận, vì vậy chúng ta phải biến mỗi hình ảnh 28×28 thành một vector dài có độ dài $28^2 = 784$. Chúng ta có thể sử dụng hàm cho việc này, như minh họa tiếp theo:

```
x_train_resaped = x_train.reshape(-1, 28*28)
x_test_resaped = x_test.reshape(-1, 28*28)
```

Như ví dụ trước, chúng ta cũng phải phân loại nhãn. Vì nhãn là một số từ 0 đến 9 nên chúng ta phải biến nó thành một vector có độ dài 10, trong đó mục nhập tương ứng với nhãn là 1 và phần còn lại là 0. Chúng ta có thể thực hiện điều này bằng các dòng mã sau:

```
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

Quá trình này được minh họa trong hình 10.24.



Hình 10.24 Trước khi huấn luyện mạng nơ-ron, chúng tôi xử lý trước các hình ảnh và nhãn theo cách sau. Chúng ta định hình hình ảnh hình chữ nhật thành một vector dài bằng cách ghép các hàng. Sau đó, chúng tôi chuyển đổi mỗi nhãn thành một vector có độ dài 10 với chỉ một mục khác 0 ở vị trí của nhãn tương ứng.

XÂY DỰNG VÀ ĐÀO TẠO MÔ HÌNH

Chúng ta có thể sử dụng cùng một kiến trúc mà chúng ta đã sử dụng trong mô hình trước đó, với một thay đổi nhỏ, vì đầu vào hiện có kích thước 784. Trong các dòng mã tiếp theo, chúng ta xác định mô hình và kiến trúc của nó:

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(28*28,)))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(10, activation='softmax'))
```

Bây giờ chúng tôi biên dịch và huấn luyện mô hình cho 10 kỷ nguyên với kích thước lô là 10, như được hiển thị ở đây. Mô hình này có 109.386 tham số có thể huấn luyện được, vì vậy việc huấn luyện trong 10 kỷ nguyên có thể mất vài phút trên máy tính của bạn.

```
model.compile(loss = 'categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(x_train_reshaped, y_train_cat, epochs=10, batch_size=10)
```

Nhìn vào kết quả đầu ra, chúng ta có thể thấy rằng mô hình có độ chính xác huấn luyện là

0,9164 là tốt nhưng hãy đánh giá độ chính xác của thử nghiệm để đảm bảo mô hình không bị quá khớp.

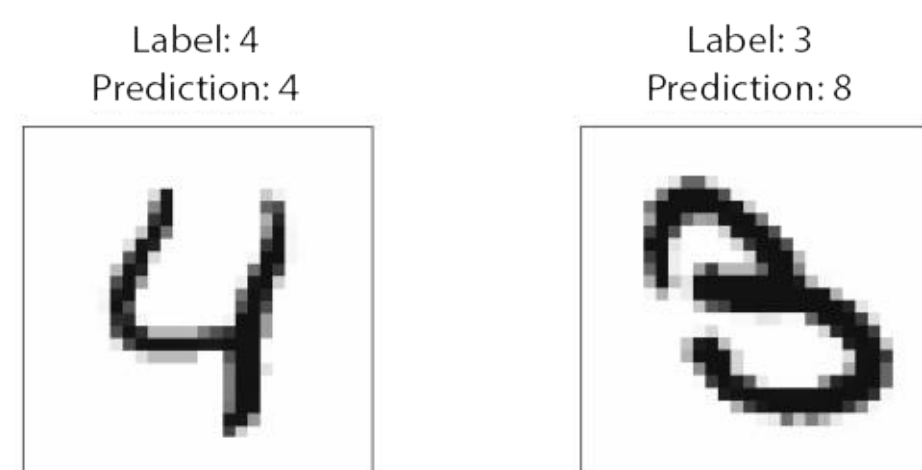
ĐÁNH GIÁ MÔ HÌNH

Chúng ta có thể đánh giá độ chính xác trong bộ thử nghiệm bằng cách đưa ra dự đoán trong tập dữ liệu thử nghiệm và so sánh chúng với nhãn. Mạng nơ-ron tạo ra các vector có độ dài 10 với xác suất mà nó gán cho từng nhãn, vì vậy chúng ta có thể thu được dự đoán bằng cách xem xét mục nhập có giá trị lớn nhất trong vector này, như sau:

```
predictions_vector = model.predict(x_test_resaped)
predictions = [np.argmax(pred) for pred in predictions_vector]
```

Khi chúng tôi so sánh chúng với nhãn, chúng tôi nhận được độ chính xác thử nghiệm là 0,942, khá tốt. Chúng ta có thể làm tốt hơn thế này với các kiến trúc phức tạp hơn, chẳng hạn như mạng nơ-ron tích chập (xem thêm về điều này trong phần tiếp theo), nhưng thật tốt khi biết rằng với một mạng nơ-ron nhỏ, được kết nối đầy đủ, chúng ta có thể xử lý khá tốt vấn đề nhận dạng hình ảnh.

Bây giờ chúng ta hãy xem xét một số dự đoán. Trong hình 10.25, chúng ta có thể thấy cái đúng (trái) và cái sai (phải). Lưu ý rằng hình không chính xác là hình ảnh được viết kém của số 3, trông hơi giống số 8.



Hình 10.25 Bên trái: Hình ảnh số 4 đã được mạng lưới thần kinh phân loại chính xác. Phải: Hình ảnh số 3 đã bị phân loại không chính xác thành số 8.

Với bài tập này, chúng ta có thể thấy rằng quá trình đào tạo một mạng lưới thần kinh lớn như vậy rất đơn giản chỉ với một vài dòng mã trong Keras! Tất nhiên, có nhiều hơn nữa người ta có thể làm ở đây. Chơi với máy tính xách tay, thêm nhiều lớp hơn vào mạng lưới thần kinh,

thay đổi các siêu tham số và xem bạn có thể cải thiện độ chính xác thử nghiệm cho mô hình này đến mức nào!

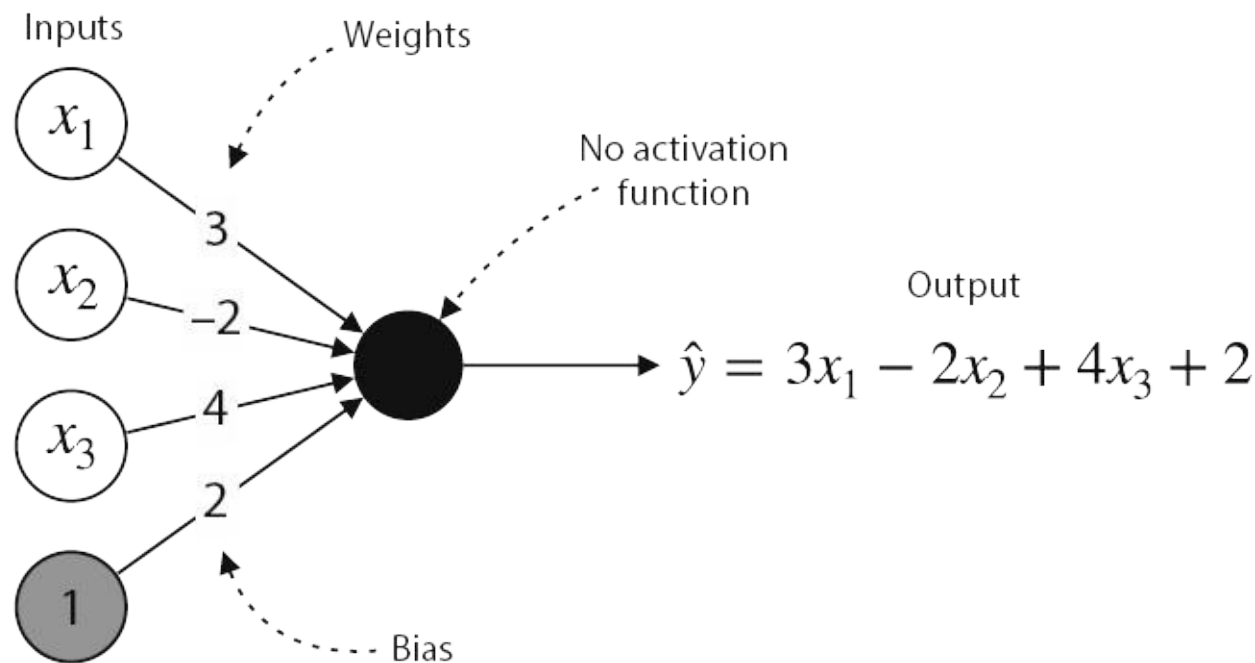
Mạng lưới thần kinh cho hồi quy

Trong suốt chương này, chúng ta đã thấy cách sử dụng mạng nơ-ron làm mô hình phân loại, nhưng mạng nơ-ron cũng hữu ích như mô hình hồi quy. May mắn thay, chúng tôi chỉ có hai điều chỉnh nhỏ để áp dụng cho mạng nơ-ron phân loại để có được mạng nơ-ron hồi quy. Điều chỉnh đầu tiên là loại bỏ hàm sigmoid cuối cùng khỏi mạng lưới thần kinh. Vai trò của điều này

Chức năng của nó là biến đầu vào thành một số từ 0 đến 1, vì vậy nếu chúng ta loại bỏ nó, mạng nơ-ron sẽ có thể trả về bất kỳ số nào. Điều chỉnh thứ hai là thay đổi hàm lỗi thành sai số tuyệt đối hoặc sai số bình phương trung bình, vì đây là các hàm lỗi liên quan đến hồi quy. Mọi thứ khác sẽ vẫn như cũ, bao gồm cả quá trình đào tạo.

Ví dụ: hãy xem perceptron trong hình 10.7 trong phần “Biểu diễn đồ họa của perceptron”. Perceptron này đưa ra dự đoán $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$. Nếu chúng ta loại bỏ chức năng kích hoạt sigmoid, perceptron mới sẽ đưa ra dự đoán $\hat{y} = 3x_1 - 2x_2 + 4x_3 + 2$. Perceptron này là

minh họa trong hình 10.26. Lưu ý rằng perceptron này đại diện cho một mô hình hồi quy tuyến tính.



Hình 10.26 Nếu chúng ta loại bỏ hàm kích hoạt khỏi perceptron, chúng ta sẽ biến mô hình phân loại thành mô hình hồi quy tuyến tính. Mô hình hồi quy tuyến tính dự đoán bất kỳ giá trị số nào, không chỉ một giá trị từ 0 đến 1.

Để minh họa quá trình này, chúng tôi huấn luyện mạng lưới thần kinh ở Keras trên một tập dữ liệu quen thuộc: tập dữ liệu về giá nhà đất ở Hyderabad. Hãy nhớ lại điều đó trong phần “Ứng dụng thực tế: Sử dụng Turi Create để dự đoán giá nhà đất ở Ấn Độ” ở chương 3, chúng tôi đã đào tạo mô hình hồi quy tuyến tính để phù hợp với tập dữ liệu này. Mã cho phần này như sau:

- **Sổ tay:** House_price_predictions_neural_network.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_
- **Tập dữ liệu:** Hyderabad.csv

Bạn có thể tìm thấy chi tiết về cách tải tập dữ liệu và chia tập dữ liệu thành các tính năng và nhãn trong sổ ghi chép. Kiến trúc của mạng lưới thần kinh mà chúng ta sẽ sử dụng như sau:

- Lớp đầu vào có kích thước 38 (số cột trong tập dữ liệu) Lớp ẩn có kích thước 128 với chức năng kích hoạt ReLU và tham số dropout là 0,2
- Lớp ẩn có kích thước 64 với chức năng kích hoạt ReLU và tham số dropout là 0,2
- Lớp đầu ra có kích thước 1 không có chức năng kích hoạt
-

```
model = Sequential()
model.add(Dense(38, activation='relu', input_shape=(38,)))
model.add(Dropout(.2))
```

```
model.add(Dense(128, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(1))
```

Để huấn luyện mạng nơ-ron, chúng tôi sử dụng hàm lỗi bình phương trung bình và trình tối ưu hóa Adam. Chúng tôi sẽ đào tạo trong 10 kỷ nguyên với kích thước lô là 10, như được hiển thị ở đây:

```
model.compile(loss = 'mean_squared_error', optimizer='adam')
model.fit(features, labels, epochs=10, batch_size=10)
```

Mạng nơ-ron này báo cáo sai số bình phương trung bình gốc là 5.535.425 trong tập dữ liệu huấn luyện. Hãy nghiên cứu sâu hơn về mô hình này bằng cách thêm một bộ thử nghiệm, thử nghiệm với kiến trúc và xem bạn có thể cải thiện nó đến mức nào!

Các kiến trúc khác dành cho các tập dữ liệu phức tạp hơn

Mạng lưới thần kinh rất hữu ích trong nhiều ứng dụng, có lẽ còn hữu ích hơn bất kỳ thuật toán học máy nào khác hiện nay. Một trong những phẩm chất quan trọng nhất của mạng lưới thần kinh là tính linh hoạt của chúng. Chúng tôi có thể sửa đổi kiến trúc theo những cách rất thú vị để phù hợp hơn với dữ liệu của mình và giải quyết vấn đề của chúng tôi. Để tìm hiểu thêm về các kiến trúc này, hãy xem *Grokking Deep Learning* của Andrew Trask (Manning, 2019) và bộ video có trong phụ lục C hoặc tại <https://serrano.academy/neural-networks/>.

Mạng nơ-ron nhìn như thế nào: Mạng nơ-ron tích chập (CNN)

Như chúng ta đã học trong chương này, mạng nơ-ron có khả năng xử lý hình ảnh rất tốt và chúng ta có thể sử dụng chúng trong nhiều ứng dụng, chẳng hạn như sau:

- **Nhận dạng hình ảnh:** đầu vào là hình ảnh, đầu ra là nhãn trên hình ảnh. Một số bộ dữ liệu nổi tiếng được sử dụng để nhận dạng hình ảnh như sau:
 - MNIST: chữ số viết tay ở dạng ảnh thang độ xám 28 x 28 CIFAR-10: ảnh màu, với 10
 - nhãn như máy bay, ô tô, v.v., ở dạng ảnh 32 x 32 CIFAR-100: tương tự CIFAR-10, nhưng có 100 nhãn như động vật có vú sống dưới nước, hoa, v.v.
 -
- **Phân đoạn ngữ nghĩa:** đầu vào là một hình ảnh và đầu ra không chỉ là nhãn của những thứ tìm thấy trong hình ảnh mà còn là vị trí của chúng bên trong hình ảnh. Thông thường, mạng nơ-ron xuất vị trí này dưới dạng hình chữ nhật có giới hạn trong ảnh.

Trong phần “Huấn luyện mạng nơ-ron để nhận dạng hình ảnh”, chúng tôi đã xây dựng một mạng nơ-ron nhỏ, được kết nối đầy đủ để phân loại khá tốt tập dữ liệu MNIST. Tuy nhiên, đối với những hình ảnh phức tạp hơn, chẳng hạn như hình ảnh và khuôn mặt, mạng lưới thần kinh như thế này sẽ không hoạt động tốt vì việc biến hình ảnh thành một vector dài sẽ làm mất rất nhiều thông tin. Đối với những hình ảnh phức tạp này, chúng ta cần các kiến trúc khác nhau và đây là lúc mạng lưới thần kinh tích chập đến để trợ giúp chúng ta.

Để biết chi tiết về mạng lưới thần kinh, hãy xem lại các tài nguyên trong phụ lục C, nhưng đây là bản phác thảo sơ bộ về cách chúng hoạt động. Hãy tưởng tượng rằng chúng ta có một hình ảnh lớn mà chúng ta muốn xử lý. Chúng tôi lấy một cửa sổ nhỏ hơn, chẳng hạn như 5 x 5 hoặc 7 x 7 pixel và vượt nó qua hình ảnh lớn. Mỗi lần chúng tôi chuyển nó qua, chúng tôi áp dụng một công thức gọi là *tích chập*. Do đó, chúng ta kết thúc bằng một hình ảnh được lọc nhỏ hơn một chút, theo cách nào đó, hình ảnh này tóm tắt hình ảnh trước đó—một lớp chập. Mạng nơ-ron tích chập bao gồm một số lớp tích chập này, theo sau là một số lớp được kết nối đầy đủ.

Khi nói đến những hình ảnh phức tạp, thông thường chúng ta sẽ không đào tạo mạng lưới thần kinh từ đầu. Một kỹ thuật hữu ích được gọi là *học chuyển giao* bao gồm việc bắt đầu với một mạng được huấn luyện trước và sử dụng dữ liệu của chúng tôi để điều chỉnh một số tham số của mạng đó (thường là lớp cuối cùng). Kỹ thuật này có xu hướng hoạt động tốt và với chi phí tính toán thấp. Các mạng như InceptionV3, ImageNet, ResNet và VGG đã được đào tạo bởi các công ty và nhóm nghiên cứu có sức mạnh tính toán lớn nên chúng tôi rất khuyến khích sử dụng chúng.

Mạng thần kinh hoạt động như thế nào: Mạng thần kinh tái phát (RNN), đơn vị tái phát có kiểm soát (GRU),

và mạng bộ nhớ ngắn hạn dài (LSTM)

Một trong những ứng dụng hấp dẫn nhất của mạng lưới thần kinh là khi chúng ta có thể khiến chúng nói chuyện với chúng ta hoặc hiểu những gì chúng ta nói. Điều này liên quan đến việc lắng nghe những gì chúng ta nói hoặc đọc những gì chúng ta viết, phân tích nó và có thể phản hồi hoặc hành động. Khả năng máy tính hiểu và xử lý ngôn ngữ được gọi là *xử lý ngôn ngữ tự nhiên*. Mạng lưới thần kinh đã đạt được nhiều thành công trong xử lý ngôn ngữ tự nhiên. Ví dụ về phân tích cảm xúc ở đầu chương này là một phần của quá trình xử lý ngôn ngữ tự nhiên, bởi vì nó đòi hỏi phải hiểu các câu và xác định xem chúng có cảm xúc tích cực hay tiêu cực. Như bạn có thể tưởng tượng, còn có nhiều ứng dụng tiên tiến hơn, chẳng hạn như sau:

- **Dịch máy:** dịch các câu từ nhiều ngôn ngữ khác nhau sang ngôn ngữ khác. **Nhận dạng giọng nói:** giải mã giọng nói của con người và biến nó thành văn bản. **Tóm tắt văn bản:** tóm tắt các văn bản lớn thành một vài đoạn văn. **Chatbots:** một hệ thống có thể nói chuyện với con người và trả lời các câu hỏi. Chúng chưa được hoàn thiện nhưng các chatbot hữu ích hoạt động trong các chủ đề cụ thể, chẳng hạn như hỗ trợ khách hàng.

Các kiến trúc hữu ích nhất hoạt động tốt để xử lý văn bản là *mạng thần kinh tái phát* và một số phiên bản nâng cao hơn của chúng được gọi là *mạng bộ nhớ ngắn hạn dài (LSTM)* và *các đơn vị tái phát có kiểm soát (GRU)*. Để hiểu chúng là gì, hãy tưởng tượng một mạng lưới thần kinh nơi đầu ra được cắm lại vào mạng như một phần của đầu vào. Bằng cách này, mạng lưới thần kinh có một bộ nhớ và khi được đào tạo đúng cách, bộ nhớ này có thể giúp chúng hiểu được chủ đề trong văn bản.

Mạng lưới thần kinh vẽ tranh như thế nào: Mạng đối thủ sáng tạo (GAN)

Một trong những ứng dụng hấp dẫn nhất của mạng lưới thần kinh là thể hệ. Cho đến nay, mạng lưới thần kinh (và hầu hết các mô hình ML khác trong cuốn sách này) đã hoạt động tốt trong việc học máy dự đoán, cụ thể là có thể trả lời các câu hỏi như “Cái đó bao nhiêu?” hoặc “Đây là con mèo hay con chó?” Tuy nhiên, trong những năm gần đây, nhiều tiến bộ đã xuất hiện trong một lĩnh vực hấp dẫn được gọi là *học máy sáng tạo*. Học máy sáng tạo là lĩnh vực học máy dạy các

máy tính cách tạo ra mọi thứ thay vì chỉ trả lời các câu hỏi. Các hành động như vẽ một bức tranh, sáng tác một bài hát hoặc viết một câu chuyện thể hiện mức độ hiểu biết cao hơn về thế giới.

Không còn nghi ngờ gì nữa, một trong những tiên bộ quan trọng nhất trong vài năm qua là sự phát triển của *mạng đối thủ tổng hợp* hay GAN. Mạng đối thủ sáng tạo đã cho thấy kết quả hấp dẫn khi tạo hình ảnh. GAN bao gồm hai mạng cạnh tranh, mạng tạo và mạng phân biệt đối xử. Trình tạo cố gắng tạo ra hình ảnh trông giống thật, trong khi trình phân biệt cố gắng phân biệt hình ảnh thật và hình ảnh giả. Trong quá trình đào tạo, chúng tôi cung cấp hình ảnh thật cho bộ phân biệt đối xử cũng như hình ảnh giả do bộ tạo tạo ra. Khi áp dụng cho tập dữ liệu về khuôn mặt người, quy trình này tạo ra một trình tạo có thể tạo ra một số khuôn mặt trông rất thật. Trên thực tế, chúng trông thật đến mức con người thường khó có thể phân biệt được chúng. Hãy tự kiểm tra GAN—<https://www.whichfaceisreal.com>.

Tóm tắt

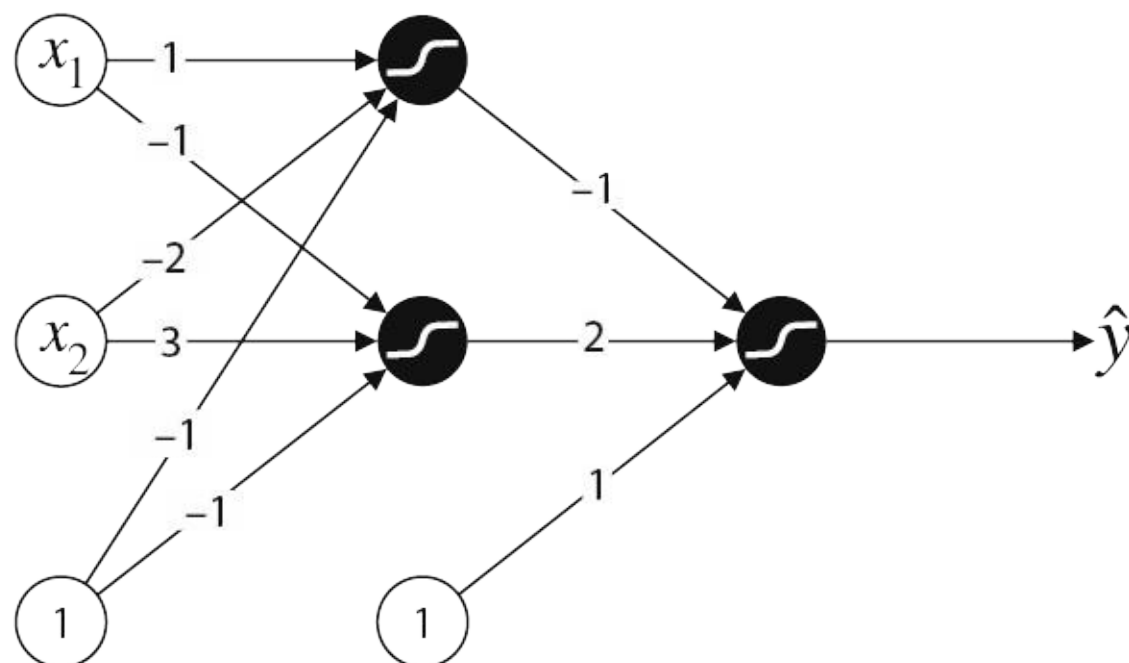
- Mạng lưới thần kinh là một mô hình mạnh mẽ được sử dụng để phân loại và hồi quy. Mạng nơ-ron bao gồm một tập hợp các tri giác được tổ chức thành các lớp, trong đó đầu ra của một lớp đóng vai trò là đầu vào cho lớp tiếp theo. Sự phức tạp của chúng cho phép chúng đạt được thành công lớn trong các ứng dụng mà các mô hình học máy khác gặp khó khăn.
- Mạng lưới thần kinh có những ứng dụng tiên tiến trong nhiều lĩnh vực, bao gồm nhận dạng hình ảnh và xử lý văn bản.
- Khối xây dựng cơ bản của mạng lưới thần kinh là perceptron. Perceptron nhận một số giá trị làm đầu vào và xuất ra một giá trị bằng cách nhân đầu vào với trọng số, thêm độ lệch và áp dụng hàm kích hoạt.
- Các hàm kích hoạt phổ biến bao gồm sigmoid, tang hyperbol, softmax và đơn vị tuyến tính chỉnh lưu (ReLU). Chúng được sử dụng giữa các lớp trong mạng lưới thần kinh để phá vỡ tính tuyến tính và giúp chúng ta xây dựng các ranh giới phức tạp hơn.
- Hàm sigmoid là một hàm đơn giản gửi bất kỳ số thực nào đến khoảng từ 0 đến 1. Tiếp tuyến hyperbol tương tự, ngoại trừ đầu ra là khoảng giữa -1 và 1 . Mục tiêu của chúng là chia đầu vào của chúng ta thành một khoảng nhỏ để các câu trả lời của chúng ta có thể được hiểu là một phạm trù. Chúng chủ yếu được sử dụng cho lớp (đầu ra) cuối cùng trong mạng lưới thần kinh. Do tính phẳng của các đạo hàm của chúng, chúng có thể gây ra các vấn đề về độ dốc biến mất.

- Hàm ReLU là hàm gửi các số âm về 0 và các số không âm về chính chúng. Nó cho thấy thành công lớn trong việc giảm vấn đề độ dốc biến mất và do đó nó được sử dụng nhiều hơn trong việc huấn luyện mạng lưới thần kinh hơn là hàm sigmoid hoặc hàm tiếp tuyến hyperbol.
- Mạng lưới thần kinh có cấu trúc rất phức tạp nên khó đào tạo. Quá trình chúng tôi sử dụng để huấn luyện chúng, được gọi là lan truyền ngược, đã cho thấy thành công lớn. Lan truyền ngược bao gồm việc lấy đạo hàm của hàm mất mát và tìm tất cả các đạo hàm riêng theo tất cả các trọng số của mô hình. Sau đó, chúng tôi sử dụng các dẫn xuất này để cập nhật lặp đi lặp lại các trọng số của mô hình nhằm cải thiện hiệu suất của mô hình.
- Mạng lưới thần kinh có xu hướng bị trang bị quá mức và các vấn đề khác như biến mất độ dốc, nhưng chúng ta có thể sử dụng các kỹ thuật như chính quy hóa và bỏ học để giúp giảm thiểu những vấn đề này.
- Chúng tôi có một số gói hữu ích để đào tạo mạng lưới thần kinh, chẳng hạn như Keras, TensorFlow và PyTorch. Các gói này giúp chúng ta huấn luyện mạng nơ-ron rất dễ dàng vì chúng ta chỉ phải xác định kiến trúc của mô hình và các hàm lỗi, còn chúng đảm nhiệm việc huấn luyện. Hơn nữa, họ có nhiều trình tối ưu hóa tiên tiến tích hợp sẵn mà chúng ta có thể tận dụng.

Bài tập

BÀI TẬP 10.1

Hình ảnh sau đây cho thấy một mạng lưới thần kinh trong đó tất cả các kích hoạt đều là hàm sigmoid.



What would this neural network predict for the input (1,1)?

EXERCISE 10.2

As we learned in exercise 5.3, it is impossible to build a perceptron that mimics the XOR gate. In other words, it is impossible to fit the following dataset with a perceptron and obtain 100% accuracy:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

This is because the dataset is not linearly separable. Using a neural network of depth 2, build a perceptron that mimics the XOR gate shown previously. As the activation functions, use the step function instead of the sigmoid function to get discrete outputs.

hint This will be hard to do using a training method; instead, try eyeballing the weights. Try (or search online how) to build an XOR gate using AND, OR, and NOT gates, and use the results of exercise 5.3 to help you.

EXERCISE 10.3

Ở cuối phần “Biểu diễn đồ họa của mạng nơ-ron”, chúng ta thấy rằng mạng nơ-ron trong hình 10.13 với hàm kích hoạt không phù hợp với tập dữ liệu trong bảng 10.1 vì điểm (1,1) bị phân loại sai.

Một. Xác minh rằng đây là trường hợp. b. Thay đổi trọng số để mạng lưới thần kinh phân loại chính xác mọi điểm.

11 Tìm ranh giới với kiểu dáng: Hỗ trợ máy vectơ và phương thức kernel

Trong chương này

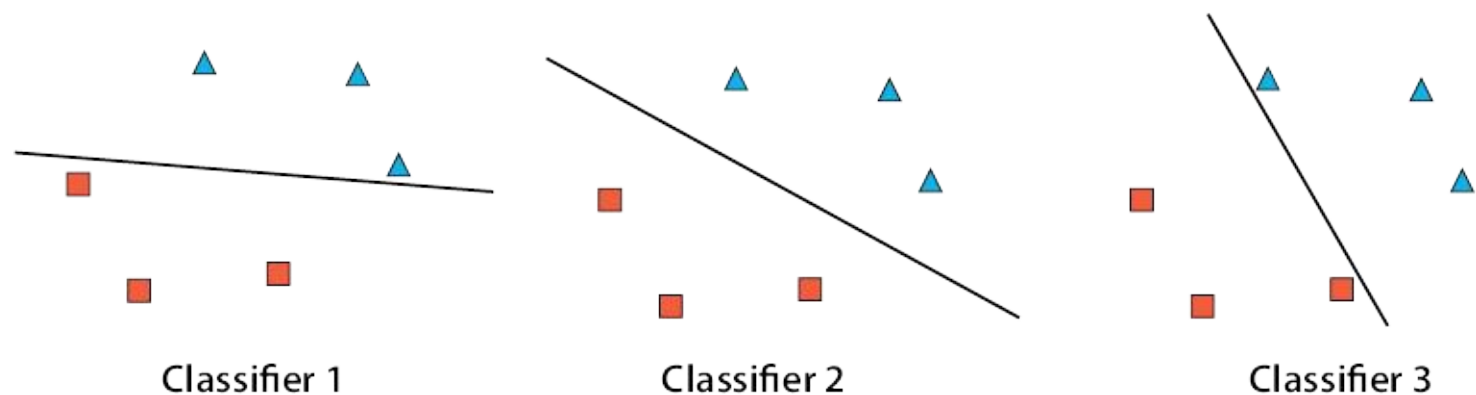
- máy vectơ hỗ trợ là gì, bộ phân loại tuyến tính nào cho tập dữ liệu có ranh giới tốt nhất
- bằng cách sử dụng phương pháp hạt nhân để xây dựng bộ phân loại phi tuyến mã hóa
- máy vectơ hỗ trợ và phương thức hạt nhân trong Scikit-Learn
-



Các chuyên gia khuyên dùng phương pháp kernel khi cố gắng tách các tập dữ liệu về gà.

Trong chương này, chúng ta thảo luận về một mô hình phân loại mạnh mẽ được gọi là *máy vectơ hỗ trợ* (viết tắt là SVM). SVM tương tự như perceptron, ở chỗ nó phân tách một tập dữ liệu thành hai lớp bằng ranh giới tuyến tính. Tuy nhiên, SVM nhằm mục đích tìm ra ranh giới tuyến tính nằm càng xa các điểm trong tập dữ liệu càng tốt. Chúng tôi cũng đề cập đến phương pháp kernel, phương pháp này rất hữu ích khi được sử dụng cùng với SVM và nó có thể giúp phân loại các tập dữ liệu bằng cách sử dụng các ranh giới phi tuyến tính cao.

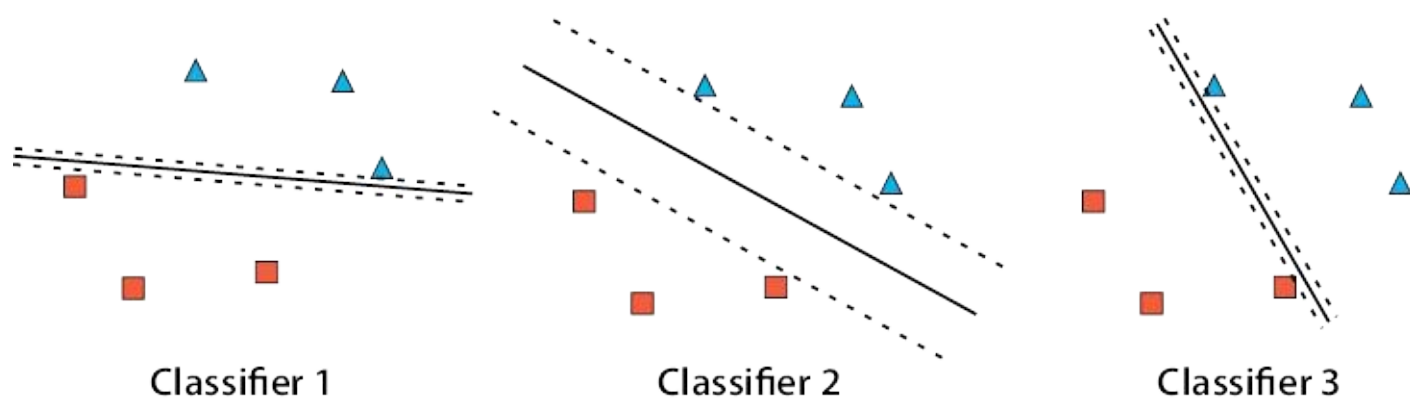
Trong chương 5, chúng ta đã tìm hiểu về bộ phân loại tuyến tính, hay còn gọi là perceptron. Với dữ liệu hai chiều, chúng được xác định bằng một đường phân cách tập dữ liệu bao gồm các điểm có hai nhãn. Tuy nhiên, chúng tôi có thể nhận thấy rằng nhiều dòng khác nhau có thể phân tách một tập dữ liệu và điều này đặt ra câu hỏi sau: làm sao chúng tôi biết đâu là dòng tốt nhất? Trong hình 11.1, chúng ta có thể thấy ba bộ phân loại tuyến tính khác nhau phân tách tập dữ liệu này. Bạn thích cái nào hơn, loại 1, 2 hay 3?



Hình 11.1 Ba bộ phân loại giúp phân loại chính xác tập dữ liệu của chúng ta. Chúng ta nên chọn loại nào, loại 1, 2 hay 3?

Nếu bạn nói phân loại 2, chúng tôi đồng ý. Cả ba dòng đều phân tách rõ ràng tập dữ liệu, nhưng dòng thứ hai được đặt tốt hơn. Dòng thứ nhất và thứ ba rất gần với một số điểm, trong khi dòng thứ hai ở xa tất cả các điểm. Nếu chúng ta dịch chuyển ba dòng này một chút, thì dòng thứ nhất và dòng thứ ba có thể lướt qua một số điểm, phân loại sai một số điểm trong quá trình này, trong khi dòng thứ hai vẫn sẽ phân loại tất cả chúng một cách chính xác. Do đó, phân loại 2 mạnh hơn phân loại 1 và 3.

Đây là lúc các máy vectơ hỗ trợ phát huy tác dụng. Bộ phân loại SVM sử dụng hai đường thẳng song song thay vì một đường. Mục tiêu của SVM có hai mặt; nó cố gắng phân loại dữ liệu một cách chính xác và cũng cố gắng giãn cách các dòng nhiều nhất có thể. Trong hình 11.2, chúng ta có thể thấy hai đường thẳng song song của ba bộ phân loại, cùng với đường giữa của chúng để tham khảo. Hai đường bên ngoài (chấm) trong bộ phân loại 2 cách xa nhau nhất, điều này làm cho bộ phân loại này trở thành bộ phân loại tốt nhất.



Hình 11.2 Chúng ta vẽ bộ phân loại của mình thành hai đường thẳng song song, càng xa nhau càng tốt. Chúng ta có thể thấy rằng phân loại 2 là phân loại mà các đường thẳng song song cách xa nhau nhất. Điều này có nghĩa là đường giữa trong phân loại 2 là đường nằm giữa các điểm tốt nhất.

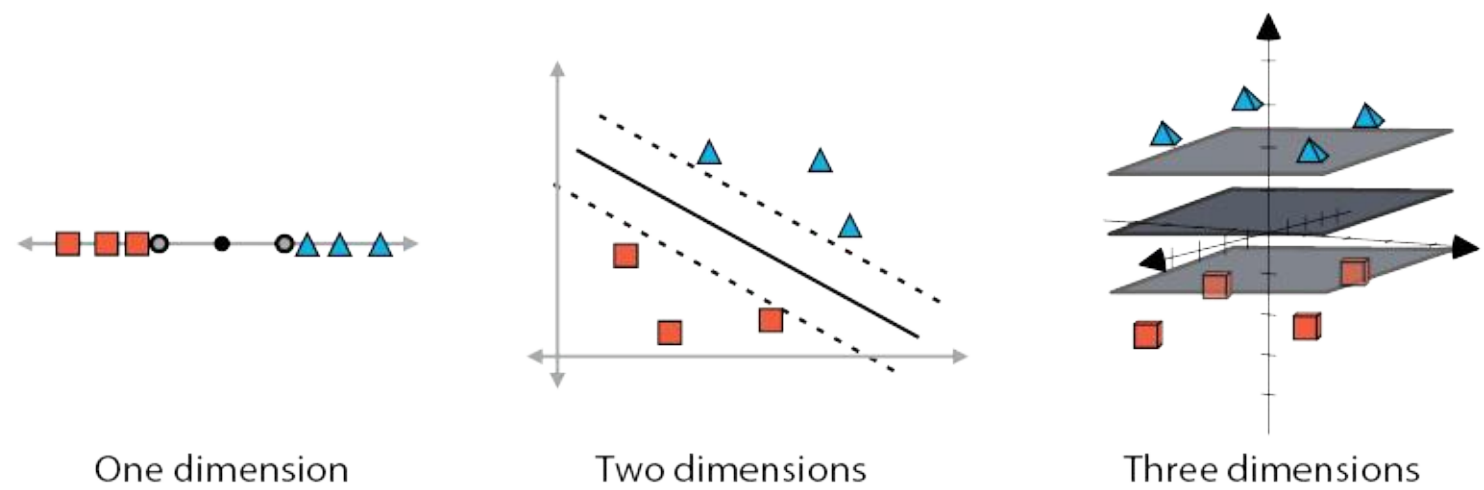
Chúng ta có thể muốn hình dung SVM như một đường ở giữa cố gắng cách xa các điểm nhất có thể. Chúng ta cũng có thể tưởng tượng nó như hai đường thẳng song song bên ngoài đang cố gắng cách xa nhau nhất có thể. Trong chương này, chúng ta sẽ sử dụng cả hai cách hình dung vào những thời điểm khác nhau, vì mỗi cách hình dung đều hữu ích trong những tình huống nhất định.

Làm thế nào để chúng ta xây dựng một bộ phân loại như vậy? Chúng ta có thể thực hiện điều này theo cách tương tự như trước đây, với hàm lỗi hơi khác và bước lặp hơi khác.

lưu ý Trong chương này, tất cả các bộ phân loại đều rời rạc, cụ thể là đầu ra của chúng là 0 hoặc 1. Đôi khi chúng được mô tả bằng dự đoán $\hat{y} = \text{bước}(f(x))$ và những lần khác bằng phương trình biên của chúng $f(x) = 0$, cụ thể là biểu đồ của hàm cố gắng tách các điểm dữ liệu của chúng ta thành hai lớp. Ví dụ: perceptron đưa ra dự đoán $\hat{y} = \text{bước}(3x_1 + 4x_2 - 1)$ đôi khi chỉ được mô tả bằng phương trình tuyến tính $3x_1 + 4x_2 - 1 = 0$. Đối với một số bộ phân loại trong chương này,

đặc biệt là những phần trong phần “Huấn luyện SVM có ranh giới phi tuyến: Phương pháp hạt nhân”, phương trình biên sẽ không nhất thiết phải là hàm tuyến tính.

Trong chương này, chúng ta thấy lý thuyết này chủ yếu dựa trên các tập dữ liệu một và hai chiều (các điểm trên đường thẳng hoặc trên mặt phẳng). Tuy nhiên, máy vectơ hỗ trợ hoạt động tốt như nhau trong các tập dữ liệu có kích thước cao hơn. Ranh giới tuyến tính trong một chiều là các điểm và trong hai chiều là các đường. Tương tự như vậy, các ranh giới tuyến tính trong ba chiều là các mặt phẳng và ở các chiều cao hơn, chúng là các siêu mặt phẳng có kích thước nhỏ hơn một chiều so với không gian mà các điểm tồn tại. Trong mỗi trường hợp này, chúng tôi cố gắng tìm ranh giới xa các điểm nhất. Trong hình 11.3, bạn có thể xem các ví dụ về ranh giới cho một, hai và ba chiều.



Hình 11.3 Ranh giới tuyến tính cho các tập dữ liệu theo một, hai và ba chiều. Trong một chiều, ranh giới được hình thành bởi hai điểm, trong hai chiều bởi hai đường thẳng và trong ba chiều bởi hai mặt phẳng. Trong mỗi trường hợp, chúng tôi cố gắng tách biệt hai điều này càng nhiều càng tốt. Ranh giới ở giữa (điểm, đường hoặc mặt phẳng) được minh họa rõ ràng.

Tất cả mã cho chương này đều có trong kho GitHub này:
https://github.com/luisguiserrano/manning/tree/master/Chapter_11_Support_Vector_

Sử dụng hàm lỗi mới để xây dựng bộ phân loại tốt hơn

Như thường thấy trong các mô hình học máy, SVM được xác định bằng hàm lỗi. Trong phần này, chúng ta thấy hàm lỗi của SVM rất đặc biệt vì nó cố gắng tối đa hóa hai thứ cùng lúc: phân loại điểm và khoảng cách giữa các đường.

Để huấn luyện SVM, chúng ta cần xây dựng hàm lỗi cho bộ phân loại gồm hai dòng, cách nhau càng xa càng tốt. Khi nghĩ đến việc xây dựng một hàm lỗi, chúng ta phải luôn tự hỏi: “Chúng ta muốn mô hình đạt được điều gì?” Sau đây là hai điều chúng tôi muốn đạt được:

- Mỗi dòng trong số hai dòng nên phân loại các điểm tốt nhất có thể. Hai dòng nên
- càng xa nhau càng tốt.

Hàm lỗi sẽ phạt bất kỳ mô hình nào không đạt được những điều này. Bởi vì chúng tôi muốn hai điều, nên hàm lỗi SVM của chúng tôi phải là tổng của hai hàm lỗi: hàm đầu tiên xử phạt các điểm bị phân loại sai và hàm

người thứ hai phạt các hàng quá gần nhau. Do đó, hàm lỗi của chúng tôi có thể trông như thế này:

$$\text{Lỗi} = \text{Lỗi phân loại} + \text{Lỗi khoảng cách}$$

ở đây Lỗi xt thành hai phần, chúng tôi sẽ phát triển từng phần trong hai thuật ngữ này một cách riêng biệt.

Chức năng lỗi phân loại: Cố gắng phân loại điểm chính xác

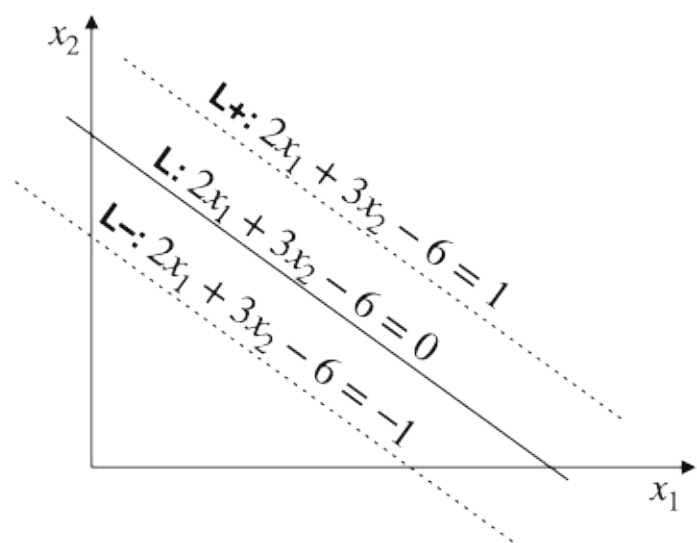
Trong phần này chúng ta tìm hiểu hàm lỗi phân loại. Đây là một phần của hàm lỗi đẩy bộ phân loại phân loại chính xác các điểm. Nói tóm lại, lỗi này được tính như sau. Bởi vì bộ phân loại được hình thành bởi hai dòng nên chúng ta coi chúng như hai perceptron rời rạc riêng biệt (chương 5). Sau đó, chúng tôi tính tổng sai số của bộ phân loại này bằng tổng của hai lỗi perceptron (phần “Làm thế nào để so sánh các bộ phân loại? Hàm lỗi” ở chương 5). Chúng ta hãy xem một ví dụ.

SVM sử dụng hai đường thẳng song song và may mắn thay, các đường thẳng song song có phương trình tương tự; chúng có cùng trọng số nhưng có độ lệch khác nhau. Do đó, trong SVM của chúng tôi, chúng tôi sử dụng đường trung tâm làm khung tham chiếu L với phương trình $w_1x_1 + w_2x_2 + b = 0$ và xây dựng hai dòng, một ở trên nó và một ở dưới nó, với các phương trình tương ứng:

- L^+ : $w_1x_1 + w_2x_2 + b = 1$, và
- L^- : $w_1x_1 + w_2x_2 + b = -1$

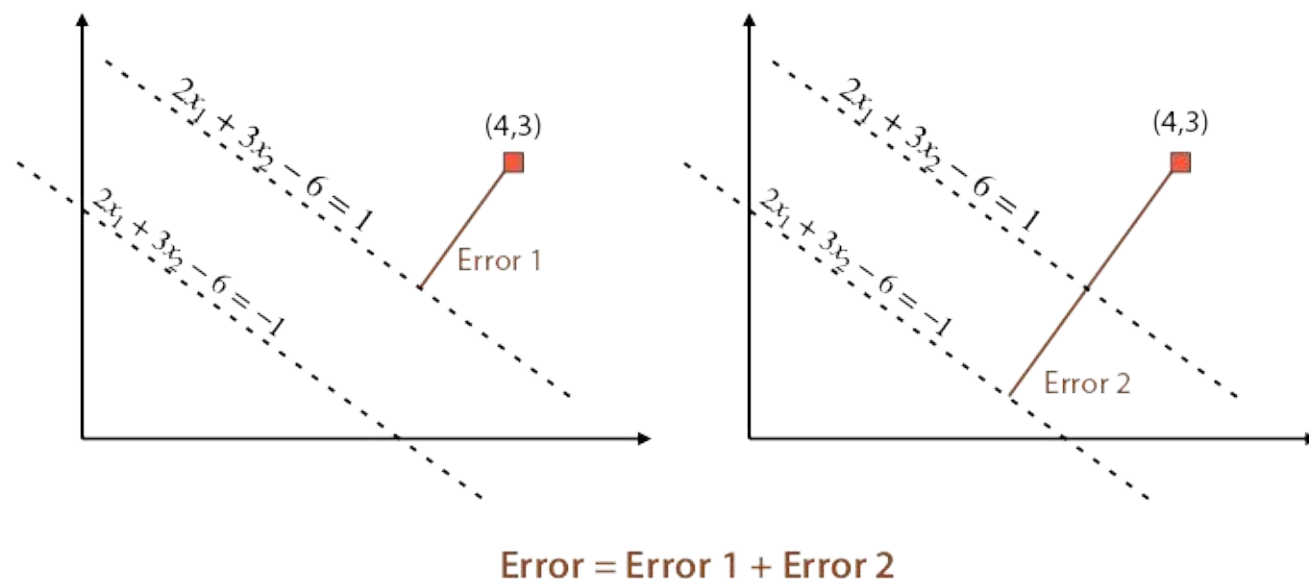
Ví dụ, hình 11.4 cho thấy ba đường thẳng song song L, L^+ và L^- với các phương trình sau:

- L : $2x_1 + 3x_2 - 6 = 0$
- L^+ : $2x_1 + 3x_2 - 6 = 1$
- L^- : $2x_1 + 3x_2 - 6 = -1$



Hình 11.4 Dòng chính L của chúng ta là dòng ở giữa. Chúng ta dựng hai đường thẳng song song cách đều L+ và L- bằng cách thay đổi một chút phương trình của L.

Trình phân loại của chúng tôi hiện bao gồm các dòng L+ và L-. Chúng ta có thể coi L+ và L- là hai bộ phân loại perceptron độc lập và mỗi bộ đều có cùng mục tiêu là phân loại các điểm một cách chính xác. Mỗi bộ phân loại đều có hàm lỗi perceptron riêng, do đó hàm phân loại được định nghĩa là tổng của hai hàm lỗi này, như minh họa trong hình 11.5.



Hình 11.5 Bây giờ bộ phân loại của chúng ta bao gồm hai dòng, sai số của một điểm bị phân loại sai sẽ được đo đối với cả hai dòng. Sau đó chúng tôi thêm hai lỗi để có được lỗi phân loại. Lưu ý rằng sai số không phải là độ dài của đoạn vuông góc với đường biên như minh họa mà là tỷ lệ thuận với nó.

Lưu ý rằng trong SVM, cả hai dòng phải phân loại điểm tốt. Do đó, một điểm nằm giữa hai dòng luôn bị phân loại sai bởi một trong các dòng, do đó nó không được tính là điểm được phân loại chính xác bởi SVM.

Nhớ lại ở phần “So sánh các bộ phân loại như thế nào? Hàm lỗi” ở chương 5 hàm lỗi cho perceptron rời rạc với dự đoán $\hat{y} = \text{bước}(w_1x_1 + w_2x_2 + b)$ tại điểm (p, q) được cho bởi:

- 0 nếu điểm được phân loại chính xác và $|w_1x_1 + w_2x_2 + b|$ nếu
- điểm đó là phân loại sai

Ví dụ, hãy xem xét điểm $(4,3)$ có nhãn là 0. Điểm này được phân loại không chính xác bởi cả hai perceptron trong hình 11.5. Lưu ý rằng hai perceptron đưa ra những dự đoán sau:

- L+: $\hat{y} = \text{bước}(2x_1 + 3x_2 - 7)$
- L-: $\hat{y} = \text{bước}(2x_1 + 3x_2 - 5)$

Do đó, lỗi phân loại của nó đối với SVM này là

$$|2 \cdot 4 + 3 \cdot 3 - 7| + |2 \cdot 4 + 3 \cdot 3 - 5| = 10 + 12 = 22.$$

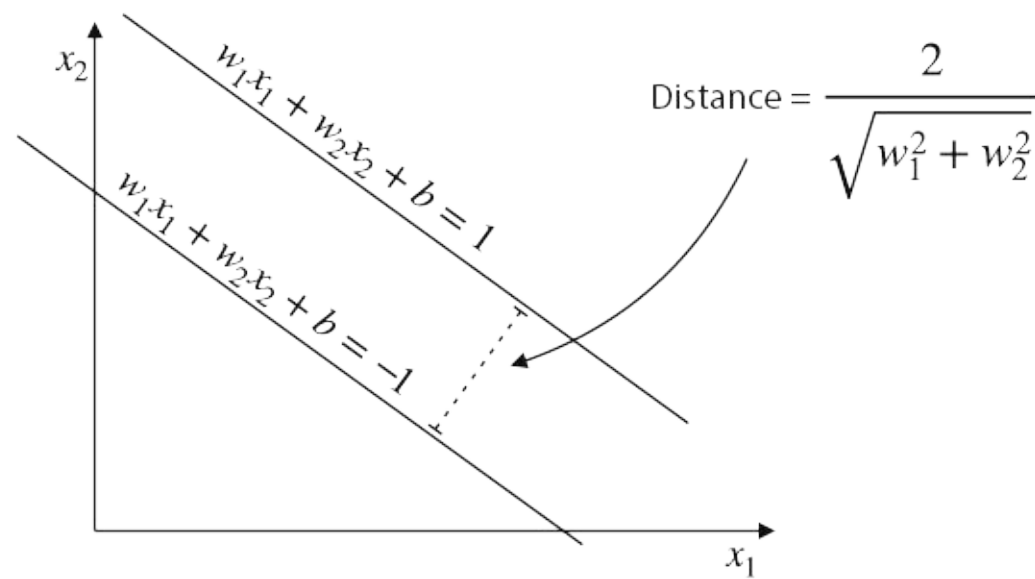
Chức năng lỗi khoảng cách: Cố gắng tách hai dòng của chúng ta càng xa nhau càng tốt

Bây giờ chúng ta đã tạo một hàm lỗi để đo các lỗi phân loại, chúng ta cần xây dựng một hàm xem xét khoảng cách giữa hai đường và đưa ra cảnh báo nếu khoảng cách này nhỏ. Trong phần này, chúng ta thảo luận về một hàm lỗi đơn giản đáng ngạc nhiên, hàm này lớn khi hai đường gần nhau và nhỏ khi chúng ở xa.

Hàm lỗi này được gọi là *hàm lỗi khoảng cách* và chúng ta đã thấy nó trước đây; đó là thuật ngữ chính quy hóa mà chúng ta đã học trong phần “Sửa đổi hàm lỗi để giải quyết vấn đề” ở chương 4. Cụ thể hơn, nếu các đường thẳng của chúng ta có phương trình $w_1x_1 + w_2x_2 + b = 1$ và $w_1x_1 + w_2x_2 + b = -1$ thì hàm lỗi là

$w_1^2 + w_2^2$. Tại sao? Chúng ta sẽ sử dụng thực tế sau: khoảng cách vuông góc

giữa hai đường thẳng chính xác là $\frac{2}{\sqrt{w_1^2 + w_2^2}}$, như minh họa trong hình 11.6. Nếu bạn muốn tìm hiểu chi tiết cách tính khoảng cách này, vui lòng xem bài tập 11.1 ở cuối chương này.



Hình 11.6 Khoảng cách giữa hai đường thẳng song song có thể được tính dựa trên phương trình của các đường thẳng.

Biết được điều này, hãy chú ý những điều sau:

- Khi $w_1^2 + w_2^2$ lớn thì nhỏ. $\frac{2}{\sqrt{w_1^2 + w_2^2}}$
- Khi $w_1^2 + w_2^2$ là nhỏ, là lớn. $\frac{2}{\sqrt{w_1^2 + w_2^2}}$

Vì chúng ta muốn các dòng cách nhau càng xa càng tốt nên thuật ngữ này $w_1^2 + w_2^2$ là một hàm sai số tốt, vì nó mang lại cho chúng ta các giá trị lớn cho các bộ phân loại kém (những bộ phân loại có các đường gần nhau) và các giá trị nhỏ cho các bộ phân loại tốt (những bộ phân loại có các đường ở xa).

Trong hình 11.7, chúng ta có thể thấy hai ví dụ về bộ phân loại SVM. Phương trình của chúng như sau:

- SVM 1:
 - L+: $3x_1 + 4x_2 + 5 = 1$
 - L-: $3x_1 + 4x_2 + 5 = -1$
- SVM 2:
 - L+: $30x_1 + 40x_2 + 50 = 1$
 - L-: $30x_1 + 40x_2 + 50 = 1$

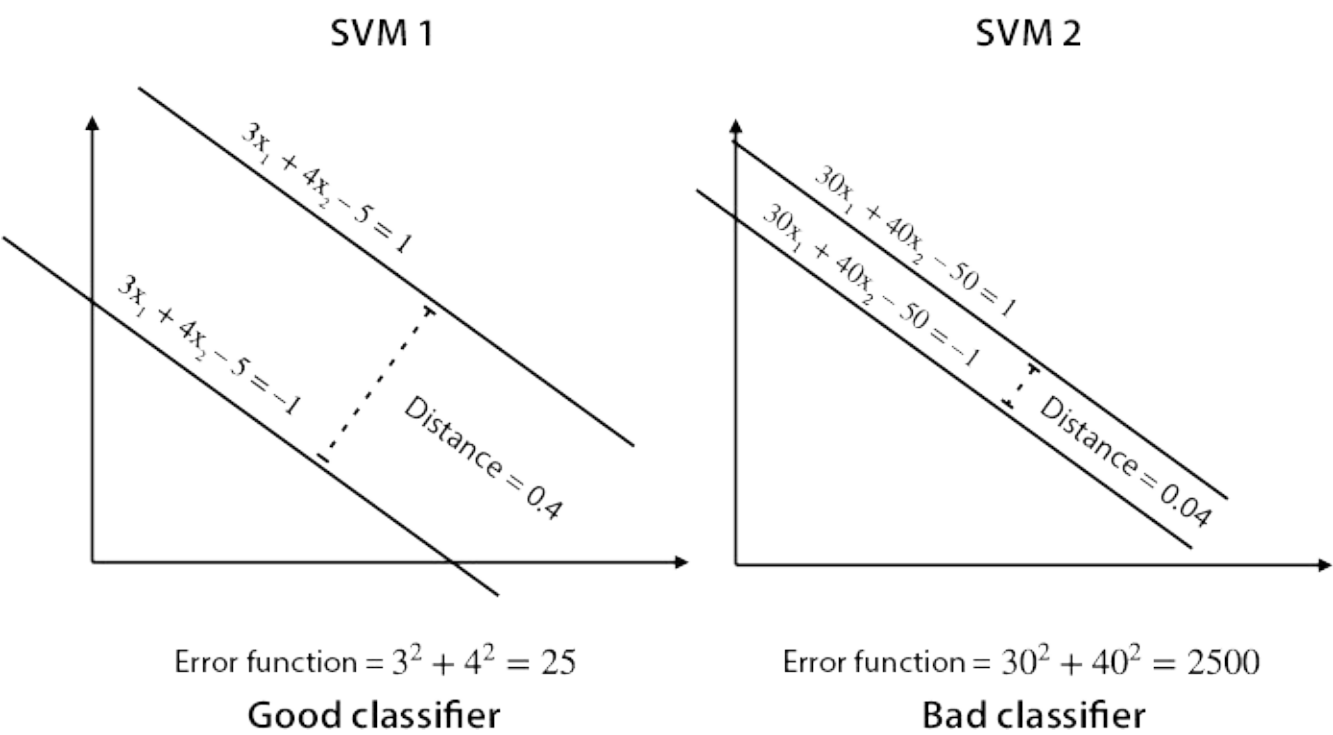
Các hàm lỗi khoảng cách của chúng được hiển thị tiếp theo:

- SVM 1: Hàm lỗi khoảng cách = $3^2 + 4^2 = 25$
-

- SVM 2:
 - Distance error function = $30^2 + 40^2 = 2500$

Cũng lưu ý từ hình 11.7 rằng các đường trong SVM 2 gần hơn nhiều so với trong SVM 1, điều này làm cho SVM 1 trở thành một bộ phân loại tốt hơn nhiều (từ góc độ khoảng cách).các

khoảng cách giữa các dòng trong SVM 1 là $\frac{2}{\sqrt{3^2 + 4^2}} = 0.4$, trong khi ở SVM 2 là $\frac{2}{\sqrt{30^2 + 40^2}} = 0.04$.



Hình 11.7 Bên trái: Một SVM trong đó các đường cách nhau 0,4, với sai số 25. Phải: Một SVM trong đó các đường cách nhau 0,04, với sai số 2500. Lưu ý rằng trong so sánh này, bộ phân loại ở bên trái tốt hơn nhiều so với bộ phân loại ở bên phải, vì các đường cách xa nhau hơn.Điều này dẫn đến sai số khoảng cách nhỏ hơn.

Thêm hai hàm lỗi để có được hàm lỗi

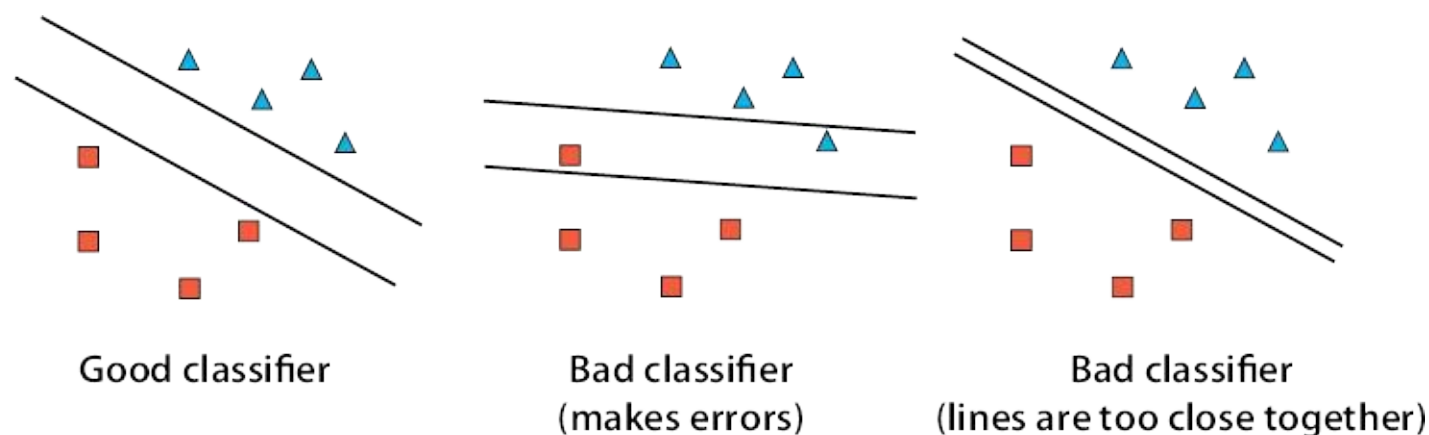
Bây giờ chúng ta đã xây dựng hàm lỗi phân loại và hàm lỗi khoảng cách, hãy xem cách kết hợp chúng để xây dựng hàm lỗi giúp chúng ta đảm bảo rằng chúng ta đã đạt được cả hai mục tiêu: phân loại điểm tốt và có hai đường cách xa nhau.

Để có được hàm lỗi này, chúng tôi thêm hàm lỗi phân loại và

hàm lỗi khoảng cách và nhận được công thức sau:

$$\text{Lỗi} = \text{Lỗi phân loại} + \text{Lỗi khoảng cách}$$

Sau đó, một SVM tốt giúp giảm thiểu hàm lỗi này phải cố gắng tạo ra ít lỗi phân loại nhất có thể, đồng thời cố gắng giữ các dòng càng xa nhau càng tốt.



Hình 11.8 Bên trái: Một SVM tốt bao gồm hai dòng cách đều nhau giúp phân loại chính xác tất cả các điểm. Giữa: Một SVM xấu phân loại sai hai điểm. Đúng: Một SVM xấu bao gồm hai dòng quá gần nhau.

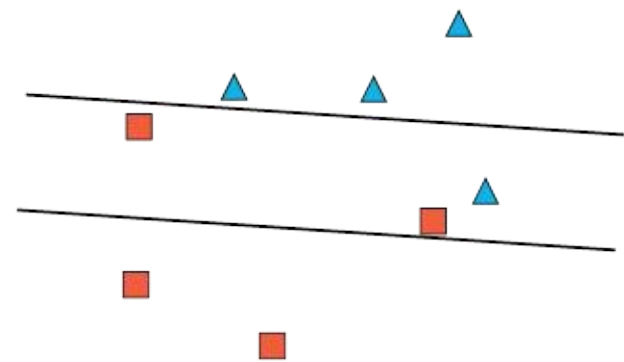
Trong hình 11.8, chúng ta có thể thấy ba bộ phân loại SVM cho cùng một tập dữ liệu. Cái bên trái là một công cụ phân loại tốt vì nó phân loại dữ liệu tốt và các dòng cách xa nhau, giúp giảm khả năng xảy ra lỗi. Cái ở giữa mắc một số lỗi (vì có một hình tam giác bên dưới dòng trên cùng và một hình vuông ở dòng dưới cùng), vì vậy nó không phải là một công cụ phân loại tốt. Cái bên phải phân loại các điểm một cách chính xác, nhưng các đường quá gần nhau nên nó cũng không phải là một công cụ phân loại tốt.

Chúng ta có muốn SVM của mình tập trung nhiều hơn vào phân loại hoặc khoảng cách không? Tham số C có thể giúp ích cho chúng ta

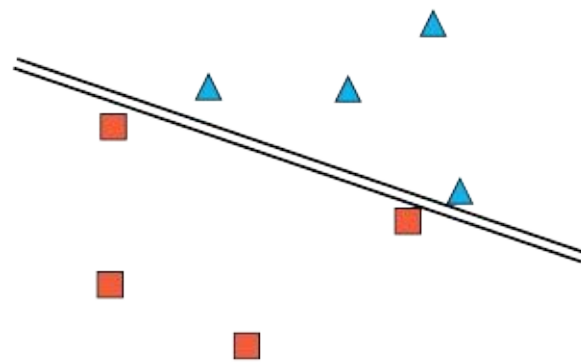
Trong phần này, chúng ta tìm hiểu một kỹ thuật hữu ích để điều chỉnh và cải thiện mô hình của mình, bao gồm việc giới thiệu tham số C. Tham số C được sử dụng trong trường hợp chúng tôi muốn huấn luyện một SVM chú ý đến việc phân loại hơn là khoảng cách (hoặc ngược lại).

Cho đến nay, có vẻ như tất cả những gì chúng ta phải làm để xây dựng một bộ phân loại SVM tốt là theo dõi hai điều. Chúng tôi muốn đảm bảo rằng trình phân loại mắc ít lỗi nhất có thể, đồng thời giữ các dòng càng xa nhau càng tốt. Nhưng nếu chúng ta phải hy sinh

cái này vì lợi ích của cái kia? Trong hình 11.9, chúng ta có hai bộ phân loại cho cùng một tập dữ liệu. Cái bên trái mắc một số lỗi nhưng các dòng cách xa nhau. Cái bên phải không mắc lỗi gì nhưng các dòng quá gần nhau. Chúng ta nên thích cái nào hơn?



Lines are far apart (good).
Makes errors (bad).



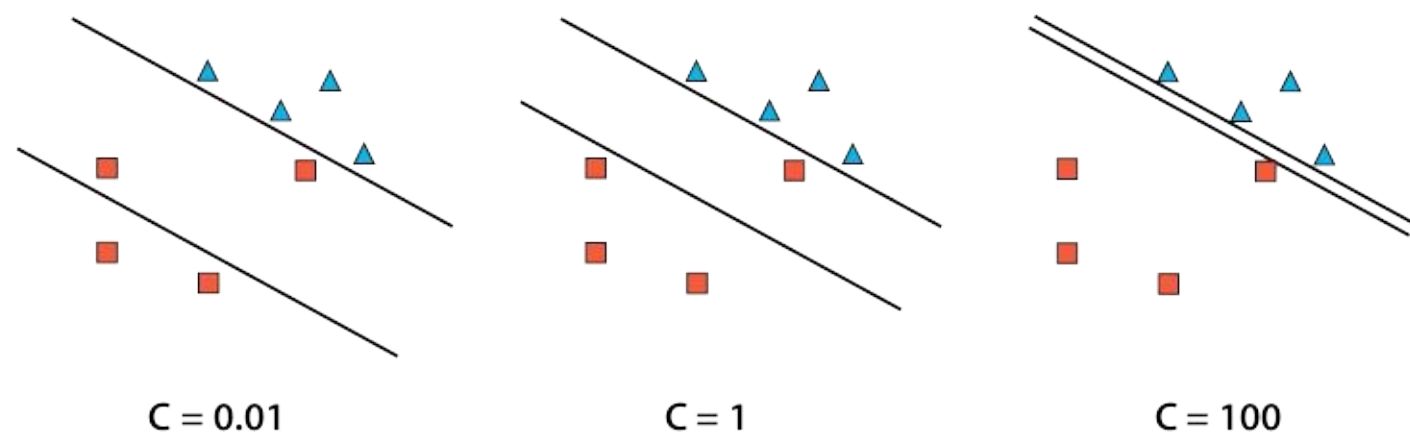
Lines are too close (bad).
Doesn't make errors (good).

Hình 11.9 Cả hai cách phân loại này đều có một ưu và một nhược điểm. Cái bên trái bao gồm các dòng có khoảng cách đều nhau (pro), nhưng nó phân loại sai một số điểm (con). Cái bên phải bao gồm các đường quá gần nhau (con), nhưng nó phân loại tất cả các điểm một cách chính xác (pro).

Hóa ra câu trả lời cho điều này phụ thuộc vào vấn đề chúng ta đang giải quyết. Đôi khi, chúng tôi muốn một trình phân loại tạo ra ít lỗi nhất có thể, ngay cả khi các dòng quá gần nhau và đôi khi chúng tôi muốn một trình phân loại giữ các dòng tách biệt nhau, ngay cả khi nó mắc một vài lỗi. Làm thế nào để chúng ta kiểm soát điều này? Chúng tôi sử dụng một tham số mà chúng tôi gọi là tham số C. Chúng tôi sửa đổi một chút công thức lỗi bằng cách nhân lỗi phân loại với C để có được công thức sau:

$$\text{Công thức lỗi} = C \cdot (\text{Lỗi phân loại}) + (\text{Lỗi khoảng cách})$$

Nếu C lớn thì công thức lỗi bị chi phối bởi lỗi phân loại, do đó bộ phân loại của chúng tôi tập trung nhiều hơn vào việc phân loại các điểm một cách chính xác. Nếu C nhỏ thì công thức bị chi phối bởi lỗi khoảng cách, do đó bộ phân loại của chúng tôi tập trung nhiều hơn vào việc giữ các đường cách xa nhau.



Hình 11.10 Các giá trị khác nhau của C chuyển đổi giữa một bộ phân loại có các dòng cách đều nhau và một bộ phân loại các điểm một cách chính xác. Bộ phân loại ở bên trái có giá trị nhỏ là C (0,01) và các dòng cách đều nhau nhưng lại mắc lỗi. Bộ phân loại ở bên phải có giá trị lớn là C (100) và nó phân loại các điểm một cách chính xác nhưng các đường quá gần nhau. Bộ phân loại ở giữa mắc một lỗi nhưng lại tìm thấy hai dòng cách nhau rất xa.

Trong hình 11.10, chúng ta có thể thấy ba bộ phân loại: một bộ có giá trị C lớn phân loại chính xác tất cả các điểm, một bộ có giá trị C nhỏ giúp giữ các đường cách xa nhau và một bộ có $C = 1$, cố gắng thực hiện cả hai. Trong đời thực, C là một siêu tham số mà chúng ta có thể điều chỉnh bằng các phương pháp như biểu đồ độ phức tạp của mô hình (phần “Một cách số để quyết định mức độ phức tạp của mô hình” trong chương 4) hoặc kiến thức của chúng ta về vấn đề chúng ta đang giải quyết, dữ liệu và mô hình.

Mã hóa máy vector hỗ trợ trong Scikit- Learn

Bây giờ chúng ta đã biết SVM là gì, chúng ta đã sẵn sàng viết mã và sử dụng nó để lập mô hình một số dữ liệu. Trong Scikit-Learn, việc viết mã SVM rất đơn giản và đó là những gì chúng ta học trong phần này. Chúng tôi cũng tìm hiểu cách sử dụng tham số C trong mã của mình.

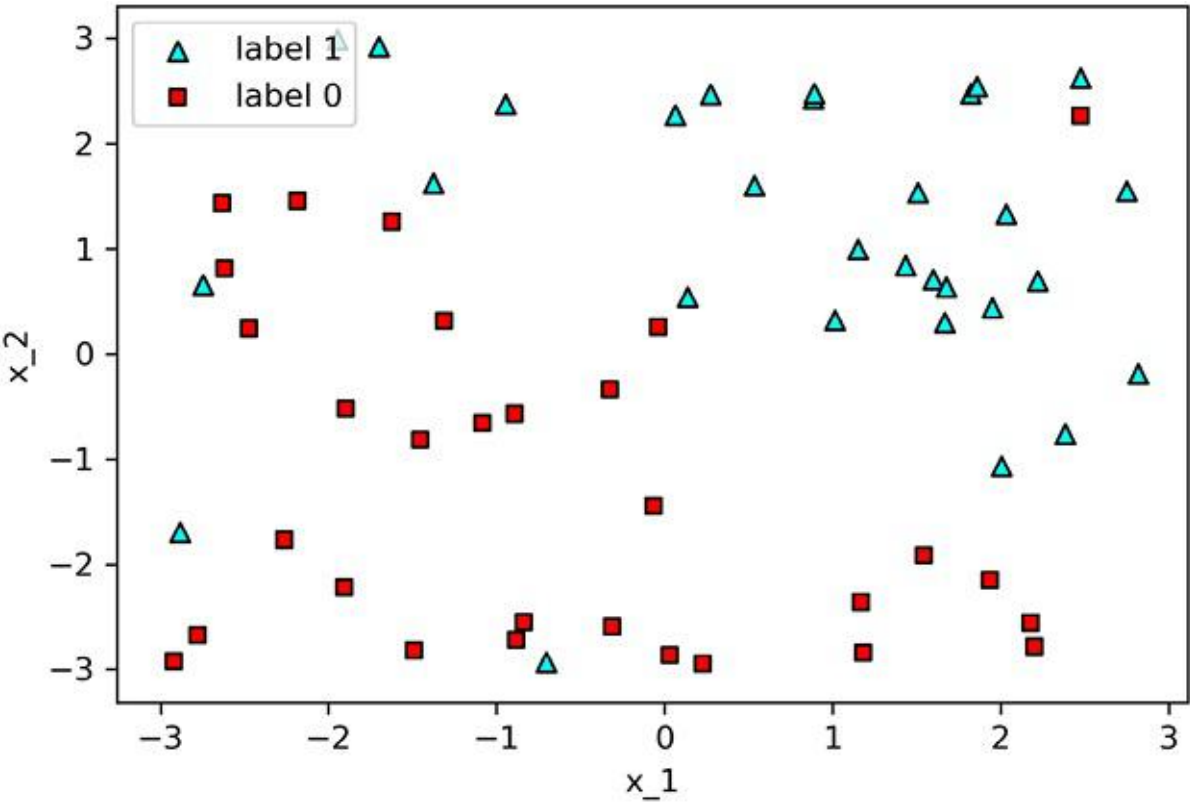
Mã hóa một SVM đơn giản

Chúng tôi bắt đầu bằng cách mã hóa một SVM đơn giản trong tập dữ liệu mẫu và sau đó chúng tôi sẽ thêm nhiều tham số hơn. Tập dữ liệu được gọi là Linear.csv và sơ đồ của nó được hiển thị trong hình 11.11. Mã cho phần này như sau:

- **Sổ tay:** SVM_graphical_example.ipynb
[https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support](https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support_Vectors_SVM.ipynb)
- **Tập dữ liệu:** tuyến tính.csv

Trước tiên, chúng tôi nhập từ gói trong Scikit-Learn và tải dữ liệu của mình như sau:

```
from sklearn.svm import SVC
```

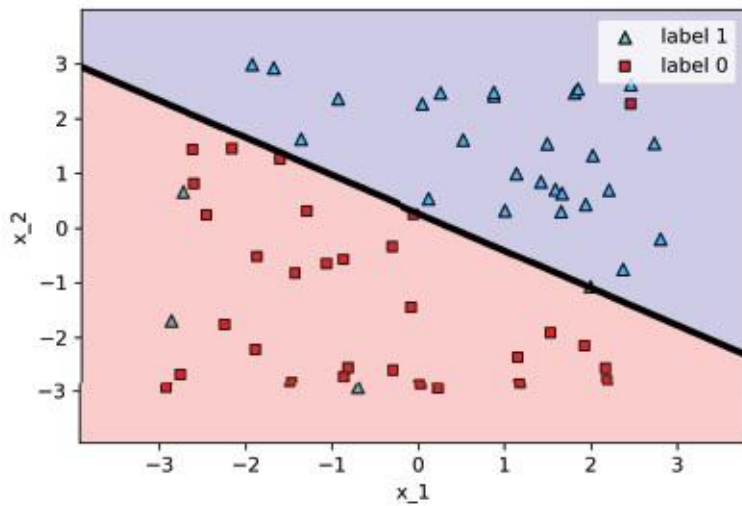


Hình 11.11 Một tập dữ liệu gần như có thể phân tách tuyến tính, với một số giá trị ngoại lệ

Sau đó, như được hiển thị trong đoạn mã tiếp theo, chúng tôi tải dữ liệu của mình vào hai Khung dữ liệu Pandas được gọi là `features` và `labels`, sau đó chúng tôi xác định mô hình của mình được gọi và huấn luyện nó. Độ chính xác chúng tôi thu được là 0,933 và đồ thị được hiển thị trong hình 11.12.

```
svm_linear = SVC(kernel='linear')
svm_linear.fit(features, labels)
```

Accuracy = 0.933



Hình 11.12 Sơ đồ của bộ phân loại SVM mà chúng tôi đã xây dựng trong Scikit-Learn bao gồm một đường. Độ chính xác của mô hình này là 0,933.

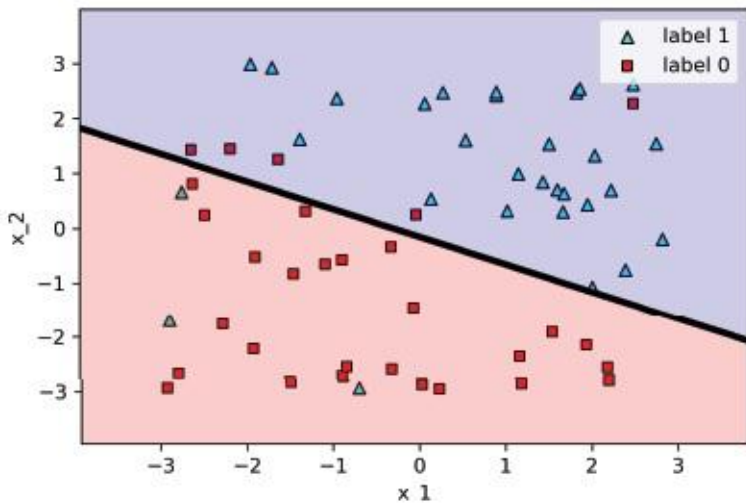
Thông số C

Trong Scikit-Learn, chúng ta có thể dễ dàng đưa tham số C vào mô hình. Ở đây chúng ta huấn luyện và vẽ đồ thị hai mô hình, một mô hình có giá trị rất nhỏ là 0,01 và một mô hình khác có giá trị lớn là 100, được hiển thị trong đoạn mã sau và trong hình 11.13:

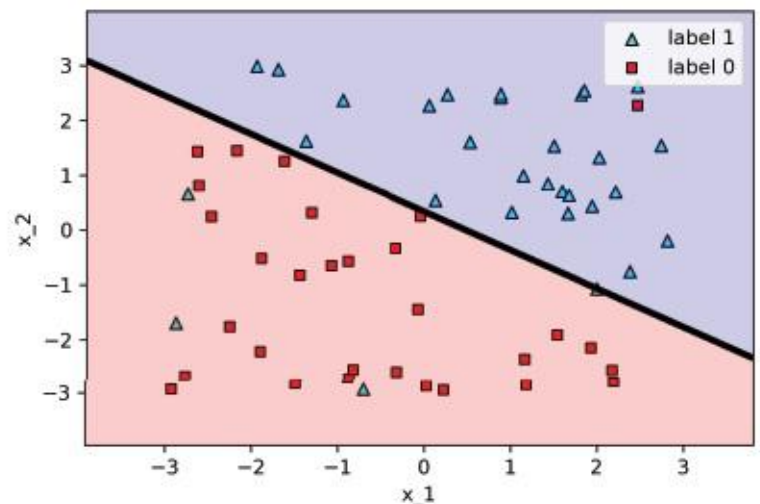
```
svm_c_001 = SVC(kernel='linear', C=0.01)
svm_c_001.fit(features, labels)

svm_c_100 = SVC(kernel='linear', C=100)
svm_c_100.fit(features, labels)
```

$C = 0.01$
Accuracy = 0.867



$C = 100$
Accuracy = 0.917



Hình 11.13 Bộ phân loại ở bên trái có giá trị C nhỏ và nó đặt khoảng cách giữa các điểm một cách hợp lý, nhưng nó mắc một số lỗi. Bộ phân loại ở bên phải có giá trị C lớn và nó không mắc lỗi, mặc dù đường này đi quá gần một số điểm.

Chúng ta có thể thấy rằng mô hình có giá trị C nhỏ không chú trọng nhiều đến việc phân loại các điểm một cách chính xác và nó mắc một số lỗi, thể hiện rõ ở độ chính xác thấp (0,867). Thật khó để nói trong ví dụ này, nhưng bộ phân loại này nhấn mạnh rất nhiều vào việc đường càng cách xa các điểm càng tốt. Ngược lại, bộ phân loại có giá trị C lớn sẽ cố gắng phân loại chính xác tất cả các điểm, điều này phản ánh độ chính xác cao hơn của nó.

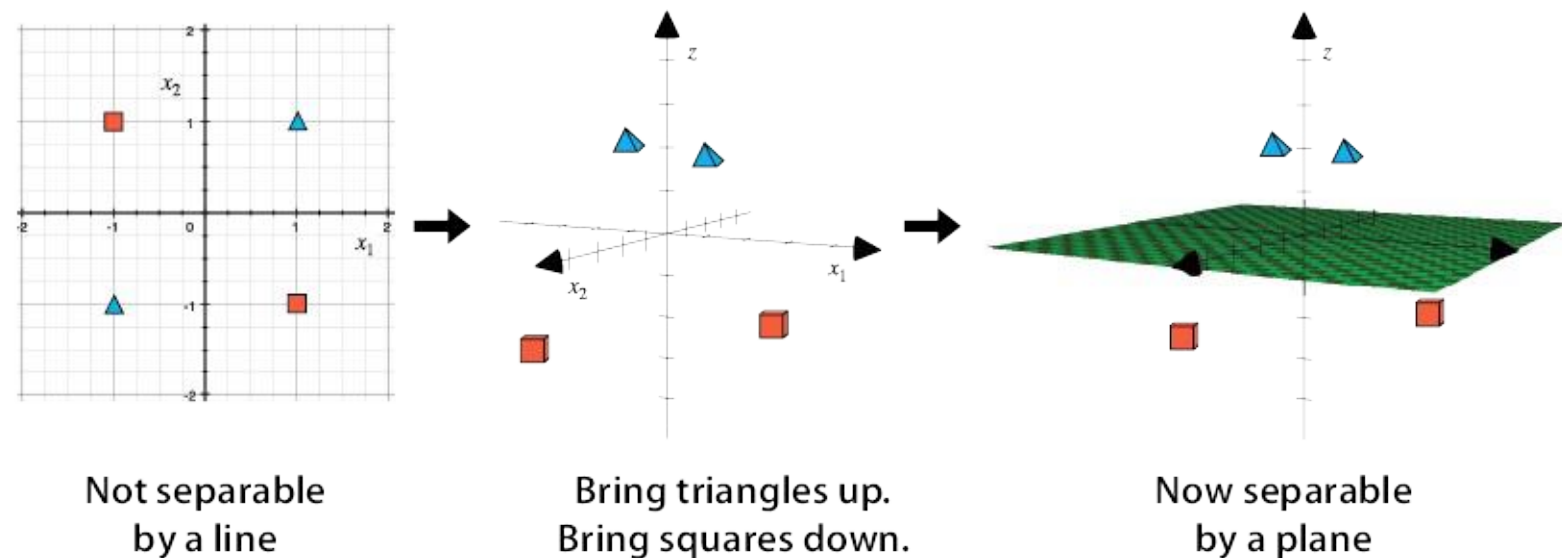
Huấn luyện SVM có ranh giới phi tuyến: Phương pháp kernel

Như chúng ta đã thấy trong các chương khác của cuốn sách này, không phải mọi tập dữ liệu đều có thể phân tách tuyến tính và nhiều khi chúng ta cần xây dựng các bộ phân loại phi tuyến tính để nắm bắt được độ phức tạp của dữ liệu. Trong phần này, chúng tôi nghiên cứu một phương pháp mạnh mẽ liên quan đến SVM được gọi là *phương pháp hạt nhân*, giúp chúng tôi xây dựng các bộ phân loại phi tuyến tính cao.

Nếu chúng ta có một tập dữ liệu và nhận thấy rằng không thể tách nó bằng bộ phân loại tuyến tính, chúng ta có thể làm gì? Một ý tưởng là thêm nhiều cột hơn vào tập dữ liệu này và hy vọng rằng tập dữ liệu phong phú hơn có thể phân tách tuyến tính. Phương thức kernel bao gồm việc thêm nhiều cột một cách thông minh, xây dựng bộ phân loại tuyến tính trên tập dữ liệu mới này và sau đó

xóa các cột mà chúng tôi đã thêm trong khi theo dõi trình phân loại (hiện là phi tuyến tính).

Điều đó thật thú vị, nhưng chúng ta có một cách hình học hay để xem phương pháp này. Hãy tưởng tượng rằng tập dữ liệu có hai chiều, có nghĩa là đầu vào có hai cột. Nếu chúng ta thêm cột thứ ba, tập dữ liệu bây giờ sẽ có dạng ba chiều, giống như nếu các điểm trên giấy của bạn đột nhiên bắt đầu bay vào không gian ở các độ cao khác nhau. Biết đâu nếu chúng ta nâng các điểm ở các độ cao khác nhau một cách khéo léo thì chúng ta có thể tách chúng ra bằng một mặt phẳng. Đây là phương pháp kernel và được minh họa trong hình 11.14.



Hình 11.14 Bên trái: Tập hợp không thể phân tách bằng một dòng. Giữa: Chúng ta nhìn nó theo không gian ba chiều và tiến hành nâng hai hình tam giác lên và hạ thấp hai hình vuông xuống. Đúng: Tập dữ liệu mới của chúng tôi hiện có thể phân tách bằng mặt phẳng. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Hạt nhân S , BẢN ĐỒ ĐẶC ĐIỂM VÀ LÝ THUYẾT ĐIỀU HÀNH

Lý thuyết đằng sau phương pháp hạt nhân xuất phát từ một lĩnh vực toán học gọi là *lý thuyết toán tử*. Hạt nhân là một hàm tương tự, nói tóm lại, là hàm cho chúng ta biết hai điểm giống nhau hay khác nhau (ví dụ: gần hay xa). Hạt nhân có thể tạo ra *bản đồ đặc trưng*, là bản đồ giữa không gian nơi tập dữ liệu của chúng ta tồn tại và không gian (thường) có chiều cao hơn.

Không cần phải có lý thuyết đầy đủ về hạt nhân và bản đồ đặc trưng để hiểu các bộ phân loại. Nếu bạn muốn tìm hiểu sâu hơn về những điều này, hãy xem các tài nguyên trong phụ lục C. Với mục đích của chương này, chúng ta xem phương pháp kernel như một cách thêm các cột vào tập dữ liệu của chúng ta để làm cho các điểm có thể phân tách được. Ví dụ: tập dữ liệu trong

hình 11.14 có hai cột, x_1 và x_2 và chúng tôi đã thêm cột thứ ba với giá trị x_1x_2 . Tương tự, nó cũng có thể được coi là chức năng gửi điểm (x_1, x_2) trên mặt phẳng đến điểm (x_1, x_2, x_1x_2) trong không gian 3 chiều, chúng ta có thể tách chúng bằng mặt phẳng bên phải của hình 11.14. Để nghiên cứu ví dụ này chi tiết hơn, hãy xem bài tập 11.2 ở cuối chương.

Hai hạt nhân và bản đồ tính năng tương ứng của chúng mà chúng ta thấy trong chương này là *hạt nhân đa thức* và *hàm cơ sở xuyên tâm (RBF) kernel*. Cả hai đều bao gồm việc thêm các cột vào tập dữ liệu của chúng tôi theo những cách khác nhau nhưng rất hiệu quả.

Sử dụng phương trình đa thức vì lợi ích của chúng ta: Hạt nhân đa thức

Trong phần này, chúng ta thảo luận về hạt nhân đa thức, một hạt nhân hữu ích sẽ giúp chúng ta lập mô hình các tập dữ liệu phi tuyến. Cụ thể hơn, phương pháp kernel giúp chúng ta mô hình hóa dữ liệu bằng các phương trình đa thức như đường tròn, parabol và hyperbol. Chúng ta sẽ minh họa hạt nhân đa thức bằng hai ví dụ.

VÍ DỤ 1: TẬP DỮ LIỆU TRÒN

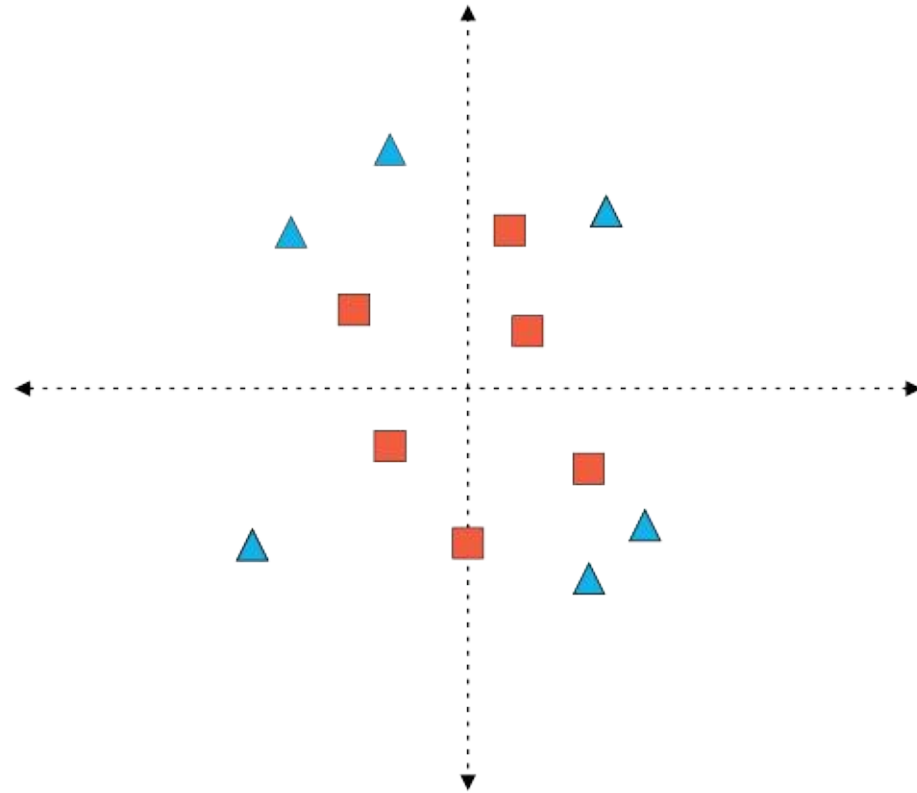
Trong ví dụ đầu tiên của chúng ta, hãy thử phân loại tập dữ liệu trong bảng 11.1.

Bảng 11.1 Một tập dữ liệu nhỏ, được mô tả trong hình 11.15

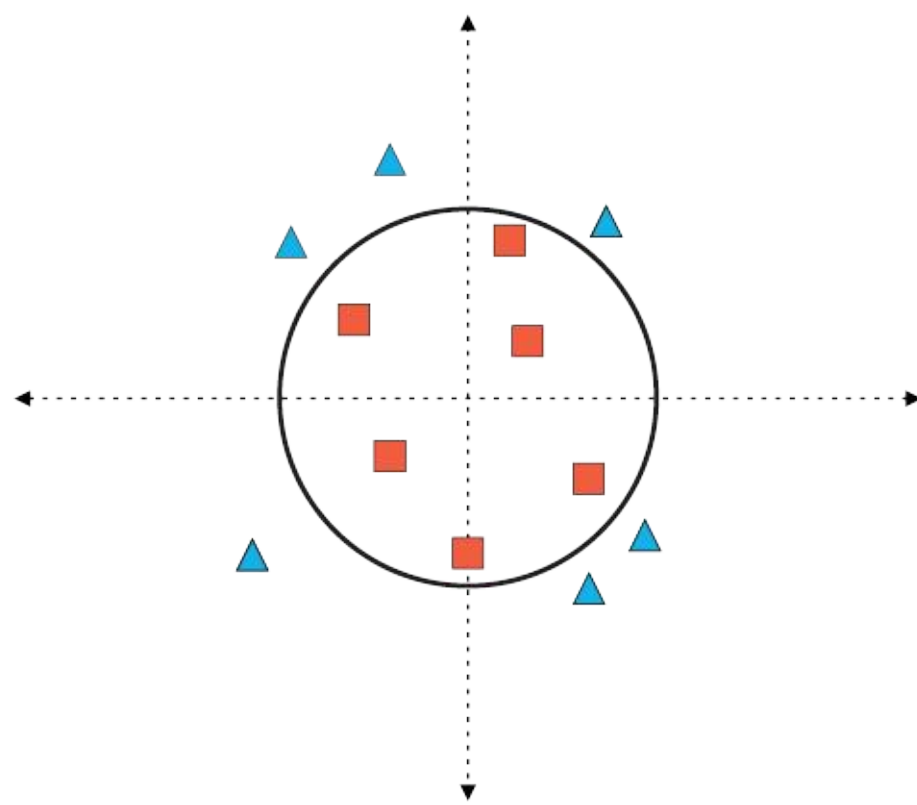
x_1	x_2	y
0.3	0.3	0
0.2	0.8	0
-0.6	0.4	0
0.6	-0.4	0
-0.4	-0.3	0
0	-0.8	0
-0.4	1.2	1
0.9	-0.7	1
-1.1	-0.8	1
0.7	0.9	1
-0.9	0.8	1
0.6	-1	1

Đồ thị được thể hiện trong hình 11.15, trong đó các điểm có nhãn 0 được vẽ dưới dạng hình vuông và các điểm có nhãn 1 được vẽ dưới dạng hình tam giác.

Khi chúng ta nhìn vào đồ thị trong hình 11.15, rõ ràng là một đường thẳng sẽ không thể tách các hình vuông khỏi các hình tam giác. Tuy nhiên, sẽ có một đường tròn (xem trong hình 11.16). Bây giờ câu hỏi là, nếu một máy vector hỗ trợ chỉ có thể vẽ các ranh giới tuyến tính, thì làm thế nào chúng ta có thể vẽ đường tròn này?



Hình 11.15 Sơ đồ của tập dữ liệu trong bảng 11.1. Lưu ý rằng nó không thể phân tách bằng một dòng. Do đó, tập dữ liệu này là một ứng cử viên sáng giá cho phương pháp kernel.



Hình 11.16 Phương thức kernel cung cấp cho chúng ta một bộ phân loại có ranh giới hình tròn, giúp phân tách các điểm này một cách rõ ràng.

Để vẽ ranh giới này, chúng ta hãy suy nghĩ. Đặc điểm phân biệt hình vuông với hình tam giác là gì? Từ việc quan sát cốt truyện, có vẻ như các hình tam giác ở xa gốc tọa độ hơn các hình tròn. Công thức đo khoảng cách đến gốc tọa độ là căn bậc hai của tổng bình phương của hai tọa độ. Nếu như

tọa độ này là x_1 và x_2 thì khoảng cách này là $\sqrt{x_1^2 + x_2^2}$. Hãy quên căn bậc hai và chỉ nghĩ đến $x_1^2 + x_2^2$. Bây giờ hãy thêm một cột vào bảng 11.1 với giá trị này và xem điều gì sẽ xảy ra. Tập dữ liệu kết quả được thể hiện trong bảng 11.2.

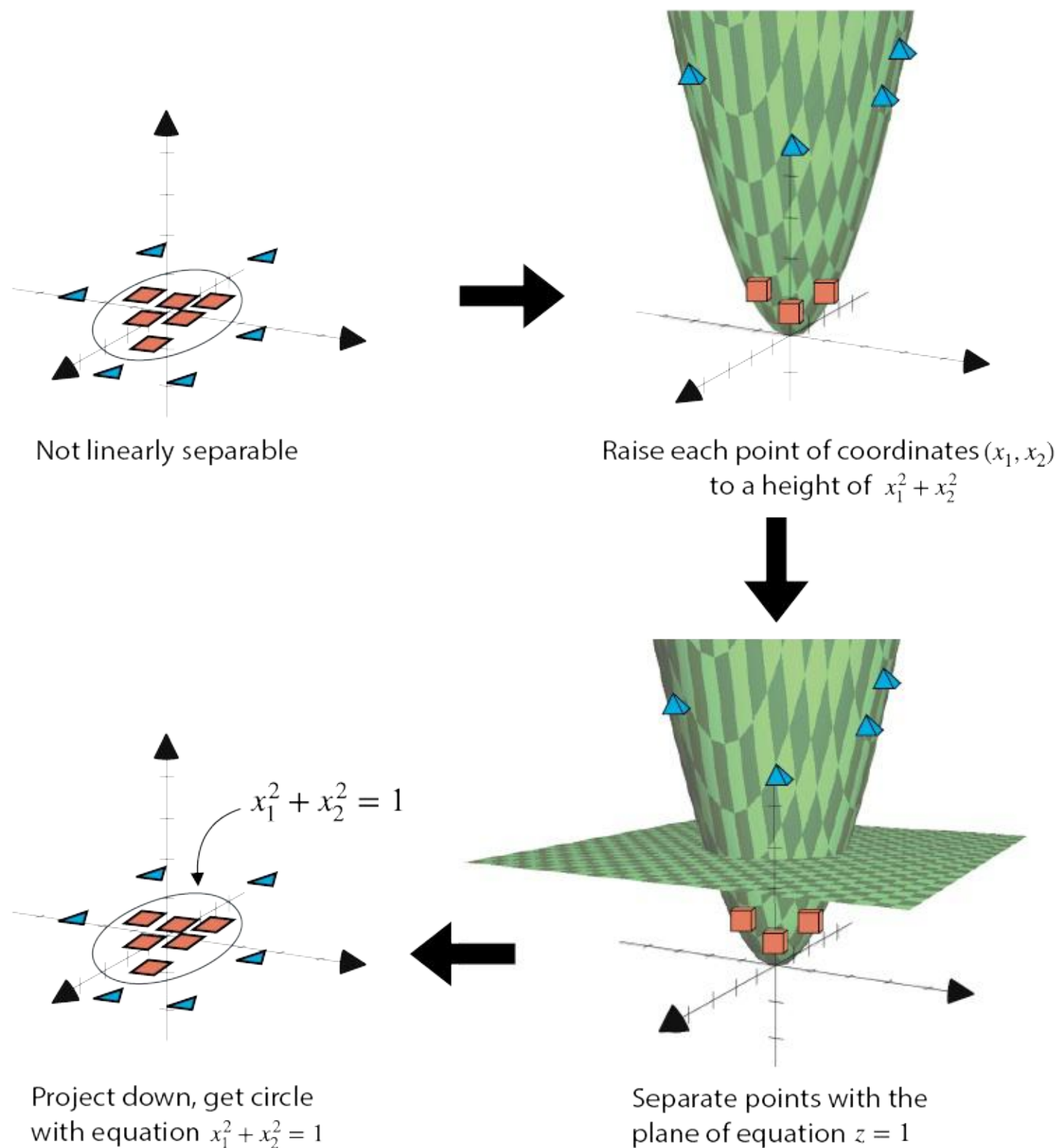
Bảng 11.2 Chúng tôi đã thêm một cột nữa vào bảng 11.1. Cột này bao gồm tổng bình phương của các giá trị của hai cột đầu tiên.

x_1	x_2	$x_1^2 + x_2^2$	y
0.3	0.3	0.18	0
0.2	0.8	0.68	0
-0.6	0.4	0.52	0
0.6	-0.4	0.52	0
-0.4	-0.3	0.25	0
0	-0.8	0.64	0
-0.4	1.2	1.6	1
0.9	-0.7	1.3	1
-1.1	-0.8	1.85	1
0.7	0.9	1.3	1
-0.9	0.8	1.45	1
0.6	-1	1.36	1

Sau khi nhìn vào bảng 11.2, chúng ta có thể thấy được xu hướng.Tất cả các điểm có nhãn 0 đều thỏa mãn rằng tổng bình phương của các tọa độ nhỏ hơn 1 và các điểm có nhãn 1 thỏa mãn rằng tổng này lớn hơn 1. Do đó, phương trình trên tọa độ phân cách các điểm chính xác là $x_1^2 + x_2^2 = 1$. Lưu ý rằng đây không phải một phương trình tuyến tính, bởi vì các biến được nâng lên lũy thừa lớn hơn một.Trên thực tế, đây chính xác là phương trình của một đường tròn.

Cách hình học để tưởng tượng điều này được mô tả trong hình 11.17.Tập hợp ban đầu của chúng ta tồn tại trong mặt phẳng và không thể tách hai lớp bằng một đường thẳng.Nhưng nếu chúng ta nâng mỗi điểm (x_1, x_2) lên độ cao $x_1^2 + x_2^2$, điều này cũng giống như việc đặt các điểm trong paraboloid có phương trình $z = x_1^2 + x_2^2$ (được vẽ trong hình).Khoảng cách chúng ta nâng lên mỗi điểm chính xác là bình phương khoảng cách từ điểm đó đến gốc tọa độ.Do đó, các hình vuông được nâng lên một lượng nhỏ vì chúng ở gần gốc tọa độ và các hình tam giác được nâng lên một lượng lớn vì chúng ở xa gốc tọa độ.Bây giờ, các hình vuông và hình tam giác ở cách xa nhau và do đó, chúng ta có thể tách chúng bằng mặt phẳng nằm ngang ở độ cao 1—nói cách khác là mặt phẳng có phương trình $z = 1$. Bước cuối cùng, chúng ta chiếu mọi thứ xuống mặt phẳng.Giao điểm giữa paraboloid và mặt phẳng trở thành đường tròn có phương trình $x_1^2 + x_2^2 = 1$.

$x_1^2 + x_2^2 = 1$. Chú ý rằng phương trình này không tuyến tính, vì nó có các số hạng bậc hai.Cuối cùng, dự đoán mà bộ phân loại này đưa ra được đưa ra bởi $\hat{y} = \text{bước}(x_1^2 + x_2^2 - 1)$.



Hình 11.17 Phương thức kernel. Bước 1: Chúng tôi bắt đầu với tập dữ liệu không thể phân tách tuyến tính. Bước 2: Sau đó, chúng ta nâng mỗi điểm lên một khoảng bằng bình phương khoảng cách của điểm đó đến gốc tọa độ. Điều này tạo ra một paraboloid. Bước 3: Bây giờ hình tam giác ở mức cao, còn hình vuông ở mức thấp. Chúng tôi tiến hành tách chúng bằng một mặt phẳng ở độ cao 1. Bước 4. Chúng tôi chiếu mọi thứ xuống. Giao điểm giữa paraboloid và mặt phẳng tạo thành một đường tròn. Hình chiếu của vòng tròn này cho chúng ta ranh giới hình tròn của bộ phân loại của chúng ta. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC;

EXAMPLE 2: THE MODIFIED XOR DATASET

Circles are not the only figure we can draw. Let’s consider a very simple dataset, illustrated in table 11.3 and plotted in figure 11.18. This dataset is similar to the one that corresponds to the XOR operator from exercises 5.3 and 10.2. If you’d like to solve the same problem with the original XOR dataset, you can do it in exercise 11.2 at the end of the chapter.

Table 11.3 The modified XOR dataset

x_1	x_2	y
-1	-1	1
-1	1	0
1	-1	0
1	1	1

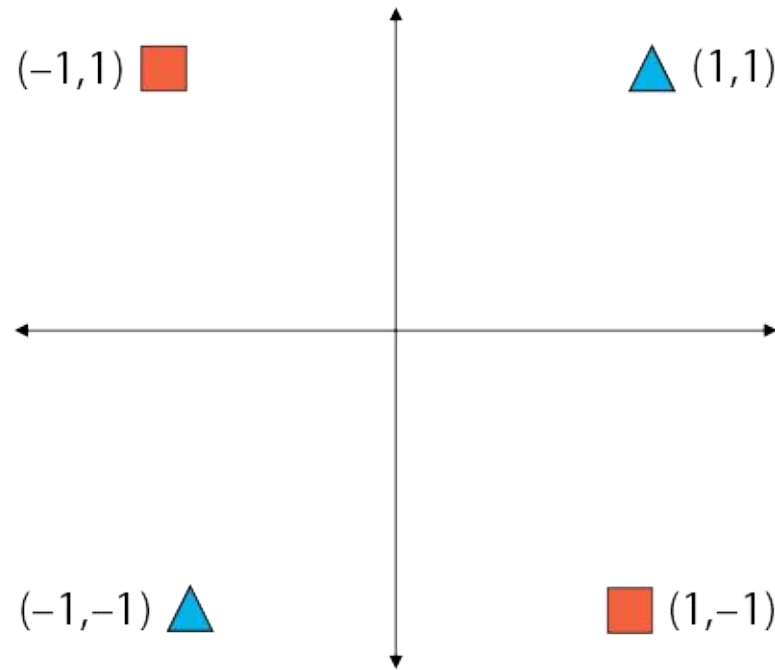
To see that this dataset is not linearly separable, take a look at figure 11.18. The two triangles lie on opposite corners of a large square, and the two squares lie on the remaining two corners. It is impossible to draw a line that separates the triangles from the squares. However, we can use a polynomial equation to help us, and this time we’ll use the product of the two features. Let’s add the column corresponding to the product x_1x_2 to the original dataset. The result is shown in table 11.4.

Table 11.4 We have added a column to table 11.3, which consists of the product of the first two columns. Notice that there is a strong relation between the rightmost two columns on the table.

x_1	x_2	$x_1 x_2$	y
-1	-1	1	1
-1	1	-1	0
1	-1	-1	0
1	1	1	1

Notice that the column corresponding to the product x_1x_2 is very similar to the

cột nhãn. Bây giờ chúng ta có thể thấy rằng bộ phân loại tốt cho dữ liệu này là bộ phân loại có phương trình biên sau: $x_1x_2 = 1$. Đồ thị của phương trình này là sự kết hợp của trục ngang và trục dọc, và lý do cho điều này là để tích x_1x_2 bằng 0, chúng ta cần $x_1 = 0$ hoặc $x_2 = 0$. Dự đoán mà trình phân loại này đưa ra được đưa ra bởi $\hat{y} = \text{bước}(x_1x_2)$, và là 1 cho các điểm ở phía đông bắc và góc phần tư phía tây nam của mặt phẳng và 0 ở nơi khác.



Hình 11.18 Sơ đồ của tập dữ liệu trong bảng 11.3. Bộ phân loại phân tách hình vuông khỏi hình tam giác có phương trình biên $x_1x_2 = 0$, tương ứng với sự kết hợp của trục ngang và trục dọc.

Vượt xa các phương trình bậc hai: hạt nhân đa thức

Trong cả hai ví dụ trước, chúng tôi đã sử dụng biểu thức đa thức để giúp chúng tôi phân loại tập dữ liệu không thể phân tách tuyến tính. Trong ví dụ đầu tiên, biểu thức này là x_1

$+ x_2^2$, bởi vì giá trị đó nhỏ đối với các điểm gần gốc và lớn đối với các điểm ở xa gốc. Trong ví dụ thứ hai, biểu thức là x_1x_2 , biểu thức này giúp chúng ta phân tách các điểm trong các góc phần tư khác nhau của mặt phẳng.

Làm thế nào chúng ta tìm thấy những biểu thức này? Trong một tập dữ liệu phức tạp hơn, chúng ta có thể không đủ điều kiện để xem xét một âm mưu và xem một biểu thức sẽ giúp ích cho chúng ta. Chúng ta cần một phương pháp hay nói cách khác là một thuật toán. Những gì chúng ta sẽ làm là xem xét tất cả các đơn thức có thể có bậc 2 (bậc hai), chứa x_1 và x_2 . Đây là

ba đơn thức sau: x_1^2 , x_1x_2 và x_2^2 . Chúng tôi gọi những biến mới này là x_3 , x_4 , và x_5 và chúng tôi coi chúng như thể chúng không có mối quan hệ nào với x_1 và x_2 . Hãy áp dụng điều này cho ví dụ đầu tiên (hình tròn). Tập dữ liệu trong bảng 11.1 với các cột mới được thêm vào này được hiển thị trong bảng 11.5.

Bây giờ chúng ta có thể xây dựng một SVM để phân loại tập dữ liệu nâng cao này. Cách huấn luyện SVM là sử dụng các phương pháp đã học ở phần trước. Tôi khuyến khích bạn xây dựng một trình phân loại như vậy bằng Scikit-Learn, Turi Create hoặc gói bạn chọn. Bằng cách kiểm tra, đây là một phương trình của bộ phân loại hoạt động:

$$0x_1 + 0x_2 + 1x_3 + 0x_4 + 1x_5 - 1 = 0$$

Bảng 11.5 Chúng tôi đã thêm ba cột hàng nữa vào bảng 11.1, một cột tương ứng với mỗi đơn thức bậc 2 của hai biến x_1 và x_2 . Các đơn thức này là x_1^2 , x_1x_2 và x_2^2 .

x_1	x_2	$x_3 = x_1^2$	$x_4 = x_1 x_2$	$x_5 = x_2^2$	y
0.3	0.3	0.09	0.09	0.09	0
0.2	0.8	0.04	0.16	0.64	0
-0.6	0.4	0.36	-0.24	0.16	0
0.6	-0.4	0.36	-0.24	0.16	0
-0.4	-0.3	0.16	0.12	0.09	0
0	-0.8	0	0	0.64	0
-0.4	1.2	0.16	-0.48	1.44	1
0.9	-0.7	0.81	-0.63	0.49	1
-1.1	-0.8	1.21	0.88	0.64	1
0.7	0.9	0.49	0.63	0.81	1
-0.9	0.8	0.81	-0.72	0.64	1
0.6	-1	0.36	-0.6	1	1

Nhớ rằng $x_3 = x_1^2$ và $x_4 = x_2^2$, ta được phương trình đường tròn mong muốn, như được hiển thị tiếp theo:

$$x_1^2 + x_2^2 = 1$$

Nếu chúng ta muốn hình dung quá trình này về mặt hình học, giống như chúng ta đã làm với những quá trình trước, thì nó sẽ phức tạp hơn một chút. Tập dữ liệu hai chiều tuyệt vời của chúng tôi đã trở thành tập dữ liệu năm chiều. Trong phần này, các điểm được gán nhãn 0 và 1 hiện ở rất xa và có thể được phân tách bằng siêu phẳng bốn chiều. Khi chúng ta chiếu hình này xuống hai chiều, chúng ta sẽ có được hình tròn mong muốn.

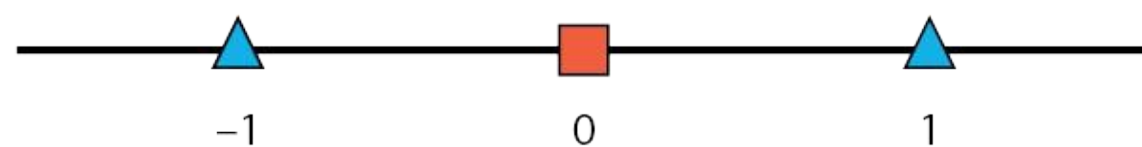
Hạt nhân đa thức tạo ra bản đồ gửi mặt phẳng 2-D đến không gian 5-D. Bản đồ là bản đồ gửi điểm (x_1, x_2) đến điểm $(x_1, x_2, x_1^2, x_1x_2, x_2^2)$. Vì bậc lớn nhất của mỗi đơn thức là 2 nên ta nói đây là hạt nhân đa thức bậc 2. Đối với hạt nhân đa thức, ta luôn phải xác định bậc.

Chúng ta sẽ thêm những cột nào vào tập dữ liệu nếu chúng ta đang sử dụng hạt nhân đa thức ở mức độ cao hơn, chẳng hạn như k ? Chúng ta thêm một cột cho mỗi đơn thức trong tập hợp biến đã cho, có bậc nhỏ hơn hoặc bằng k . Ví dụ: nếu chúng ta đang sử dụng hạt nhân đa thức bậc 3 trên các biến x_1 và x_2 , thì chúng ta sẽ thêm các cột

tương ứng với các đơn thức $\{x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3\}$. Chúng tôi có thể cũng làm điều này cho nhiều biến hơn theo cách tương tự. Ví dụ: nếu chúng ta sử dụng hạt nhân đa thức bậc 2 trên các biến x_1, x_2 và x_3 , thì chúng ta sẽ thêm các cột có các đơn thức sau: $\{x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3, x_2^2, x_2x_3, x_3^2\}$.

Sử dụng các điểm va chạm ở các chiều cao hơn để mang lại lợi ích cho chúng ta: Hạt nhân hàm cơ sở xuyên tâm (RBF)

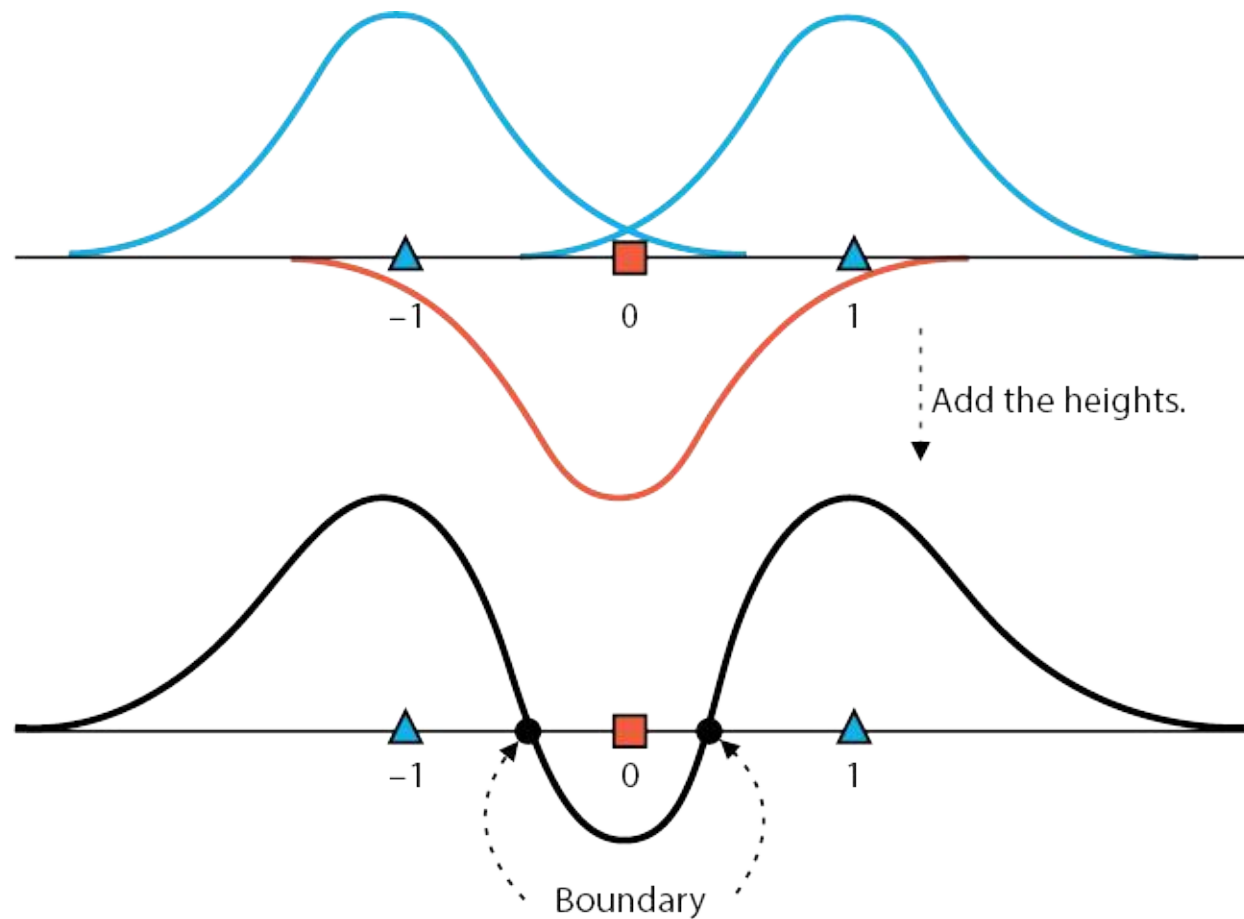
Hạt nhân tiếp theo mà chúng ta sẽ thấy là hạt nhân hàm cơ sở xuyên tâm. Hạt nhân này cực kỳ hữu ích trong thực tế vì nó có thể giúp chúng ta xây dựng các ranh giới phi tuyến bằng cách sử dụng các hàm đặc biệt nhất định tập trung vào mỗi điểm dữ liệu. Để giới thiệu nhân RBF, trước tiên chúng ta hãy xem ví dụ một chiều được hiển thị trong hình 11.19. Tập dữ liệu này không thể phân tách tuyến tính—hình vuông nằm chính xác giữa hai hình tam giác.



Hình 11.19 Tập dữ liệu theo một chiều không thể được phân loại bằng bộ phân loại tuyến tính. Lưu ý rằng bộ phân loại tuyến tính là một điểm chia đường thành hai phần và không có điểm nào mà chúng ta có thể xác định được trên đường có tất cả các hình tam giác ở một bên và hình vuông ở phía bên kia.

Cách chúng ta xây dựng bộ phân loại cho tập dữ liệu này là tưởng tượng việc xây dựng một ngọn núi hoặc một thung lũng trên mỗi điểm. Đối với các điểm được gắn nhãn 1 (hình tam giác), chúng tôi sẽ đặt một ngọn núi và đối với những điểm được gắn nhãn 0 (hình vuông), chúng tôi sẽ đặt một thung lũng. Những ngọn núi và thung lũng này được gọi là các hàm cơ sở xuyên tâm. Hình kết quả được hiển thị ở đầu hình 11.20. Bây giờ, chúng ta vẽ một dãy núi sao cho tại

mỗi điểm, độ cao là tổng của tất cả độ cao của các ngọn núi và thung lũng tại điểm đó. Chúng ta có thể thấy dãy núi thu được ở cuối hình 11.20. Cuối cùng, ranh giới của bộ phân loại của chúng tôi tương ứng với các điểm mà dãy núi này có độ cao bằng 0, cụ thể là hai điểm được đánh dấu ở phía dưới. Trình phân loại này phân loại mọi thứ trong khoảng giữa hai điểm đó dưới dạng hình vuông và mọi thứ bên ngoài khoảng dưới dạng hình tam giác.

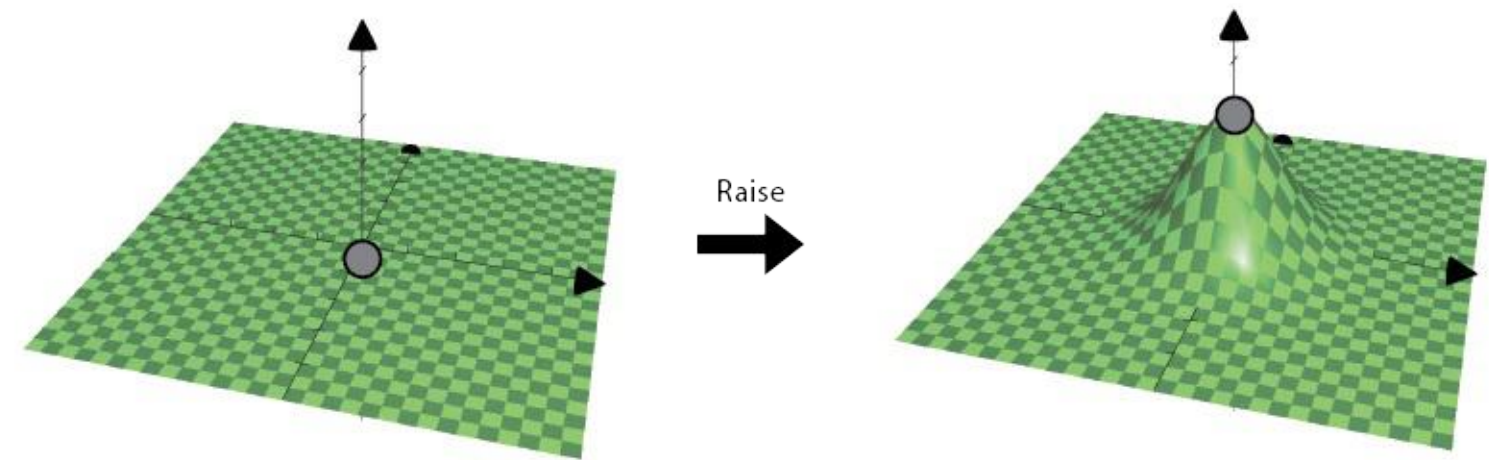


Hình 11.20 Sử dụng SVM với nhân RBF để phân tách tập dữ liệu phi tuyến theo một chiều. Trên cùng: Chúng tôi vẽ một ngọn núi (hàm cơ sở hướng tâm) tại mỗi điểm có nhãn 1 và một thung lũng tại mỗi điểm của nhãn 0. Dưới cùng: Chúng tôi thêm các hàm cơ sở hướng tâm từ hình trên cùng. Hàm kết quả cắt trục hai lần. Hai điểm giao nhau là ranh giới của bộ phân loại SVM của chúng tôi. Chúng tôi phân loại từng điểm giữa chúng là hình vuông (nhãn 0) và mọi điểm bên ngoài là hình tam giác (nhãn 1).

Điều này (cộng với một số phép toán xung quanh nó sẽ được trình bày trong phần tiếp theo) là bản chất của hạt nhân RBF. Bây giờ, hãy sử dụng nó để xây dựng một bộ phân loại tương tự trong tập dữ liệu hai chiều.

Để xây dựng những ngọn núi và thung lũng trên mặt phẳng, hãy tưởng tượng mặt phẳng như một tấm chăn (như minh họa trong hình 11.21). Nếu chúng ta véo chăn vào thời điểm đó và nâng nó lên, chúng ta

lấy được ngọn núi. Nếu chúng ta đẩy nó xuống, chúng ta sẽ có được thung lũng. Những ngọn núi và thung lũng này là các hàm cơ sở xuyên tâm. Chúng được gọi là các hàm cơ sở bán kính vì giá trị của hàm tại một điểm chỉ phụ thuộc vào khoảng cách giữa điểm đó và tâm. Chúng ta có thể nâng tấm chắn lên bất kỳ điểm nào chúng ta muốn và điều đó mang lại cho chúng ta một hàm cơ sở bán kính khác nhau cho mỗi điểm. *Hạt nhân hàm cơ sở xuyên tâm* (còn gọi là hạt nhân RBF) tạo ra một bản đồ sử dụng các hàm xuyên tâm này để thêm một số cột vào tập dữ liệu của chúng ta theo cách giúp chúng ta tách nó ra.



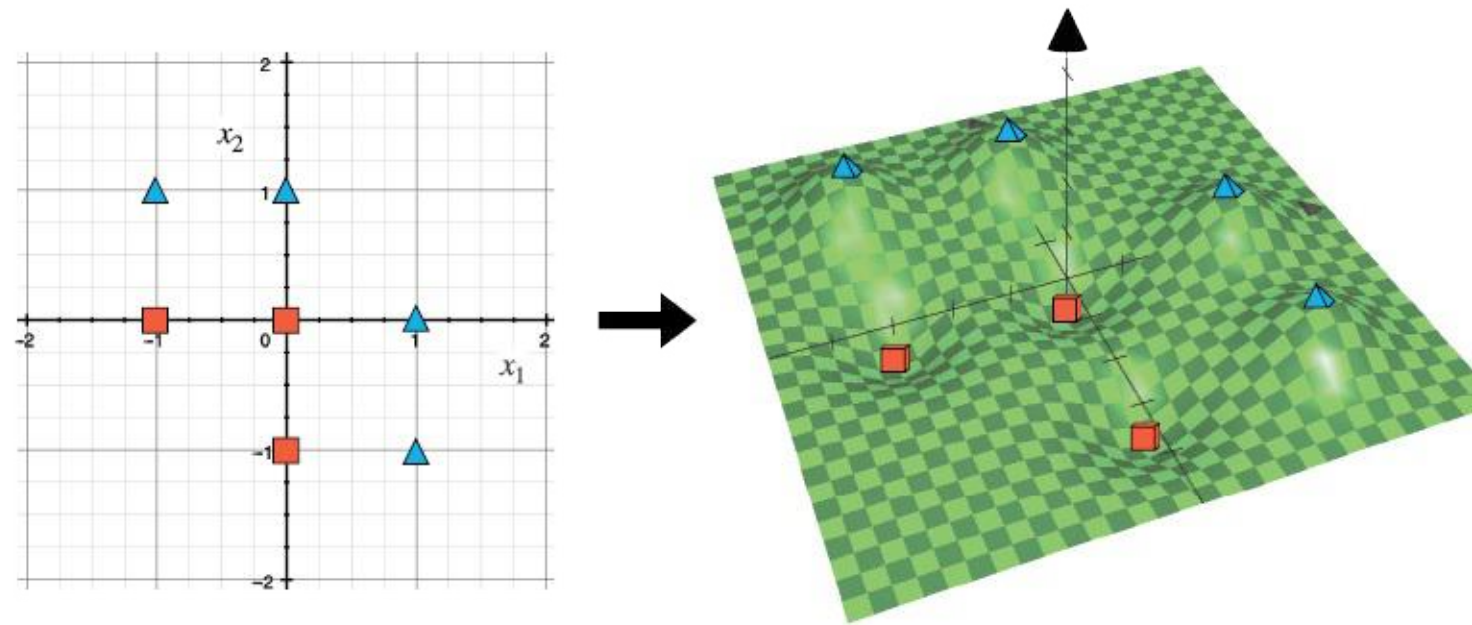
Hình 11.21 Hàm cơ sở hướng tâm bao gồm việc nâng mặt phẳng lên tại một điểm cụ thể. Đây là nhóm hàm mà chúng ta sẽ sử dụng để xây dựng các bộ phân loại phi tuyến. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Làm thế nào để chúng ta sử dụng điều này như một công cụ phân loại? Hãy tưởng tượng như sau: chúng ta có tập dữ liệu ở bên trái của hình 11.22, trong đó, như thường lệ, các hình tam giác biểu thị các điểm có nhãn 1 và các hình vuông biểu thị các điểm có nhãn 0. Bây giờ, chúng ta nâng mặt phẳng ở mọi tam giác và đẩy nó xuống ở mọi ô vuông. Chúng ta có được đồ thị ba chiều ở bên phải của hình 11.22.

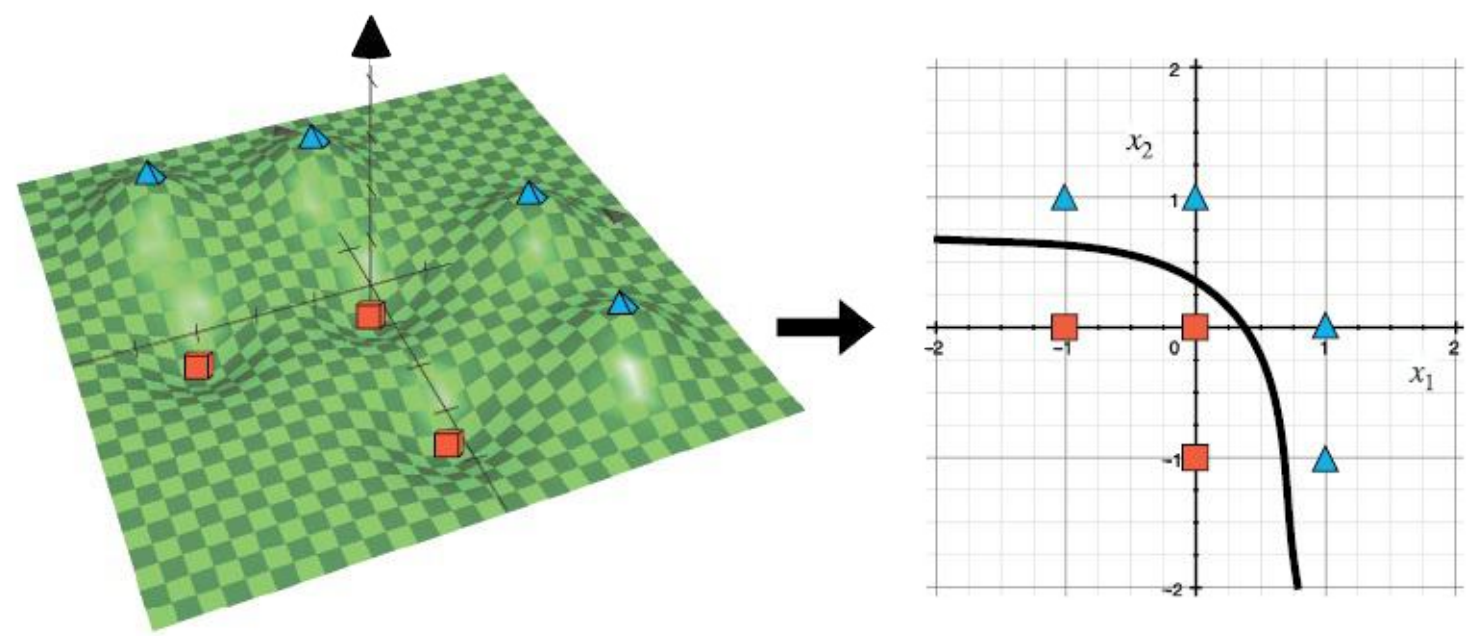
Để tạo bộ phân loại, chúng ta vẽ một mặt phẳng ở độ cao 0 và giao nó với bề mặt của chúng ta. Điều này cũng giống như việc nhìn vào đường cong được tạo bởi các điểm ở độ cao 0. Hãy tưởng tượng nếu có một phong cảnh có núi và biển. Đường cong sẽ tương ứng với đường bờ biển, cụ thể là nơi nước và đất gặp nhau. Đường bờ biển này là đường cong thể hiện bên trái trong hình 11.23. Sau đó, chúng ta chiếu mọi thứ trở lại mặt phẳng và thu được bộ phân loại mong muốn, được hiển thị bên phải trong hình 11.23.

Đó là ý tưởng đằng sau hạt nhân RBF. Tất nhiên, chúng ta phải phát triển môn toán mà chúng ta sẽ thực hiện trong một số phần tiếp theo. Nhưng về nguyên tắc, nếu chúng ta có thể tưởng tượng việc nâng

và đẩy một tấm chắn xuống, sau đó xây dựng bộ phân loại bằng cách nhìn vào ranh giới của các điểm nằm ở một độ cao cụ thể, khi đó chúng ta có thể hiểu hạt nhân RBF là gì.



Hình 11.22 Bên trái: Một tập dữ liệu trong mặt phẳng không thể phân tách tuyến tính.Đúng: Chúng ta đã sử dụng các hàm cơ sở bán kính để nâng từng hình tam giác và hạ thấp từng hình vuông.Lưu ý rằng bây giờ chúng ta có thể phân tách tập dữ liệu bằng một mặt phẳng, điều đó có nghĩa là tập dữ liệu đã sửa đổi của chúng ta có thể phân tách tuyến tính.(Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

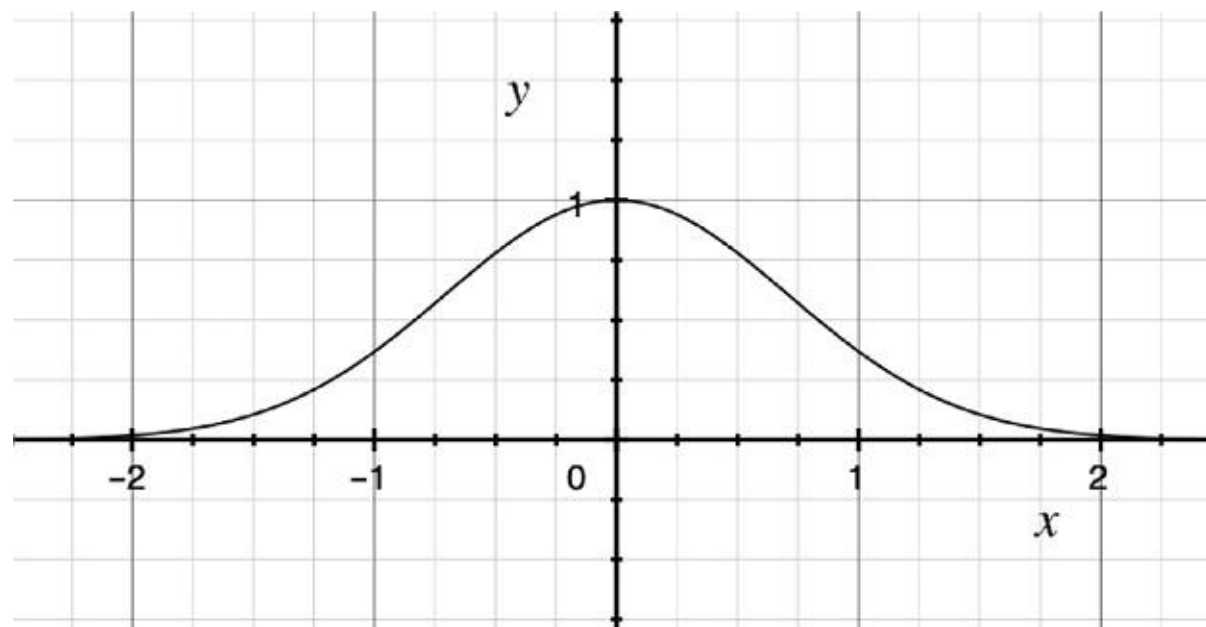


Hình 11.23 Bên trái: Nếu chúng ta nhìn vào các điểm ở độ cao 0, chúng tạo thành một đường cong. Nếu chúng ta coi những điểm cao là đất liền và những điểm thấp là biển thì đường cong này là đường bờ biển. Đúng: Khi chúng ta chiếu (làm phẳng) các điểm trở lại mặt phẳng, đường bờ biển hiện là bộ phân loại của chúng ta giúp phân tách các hình tam giác khỏi các hình vuông. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Cái nhìn sâu sắc hơn về các chức năng cơ bản xuyên tâm

Các hàm cơ sở bán kính có thể tồn tại với số lượng biến bất kỳ. Ở phần đầu của phần này, chúng ta đã thấy chúng có một và hai biến. Đối với một biến, cách đơn giản nhất

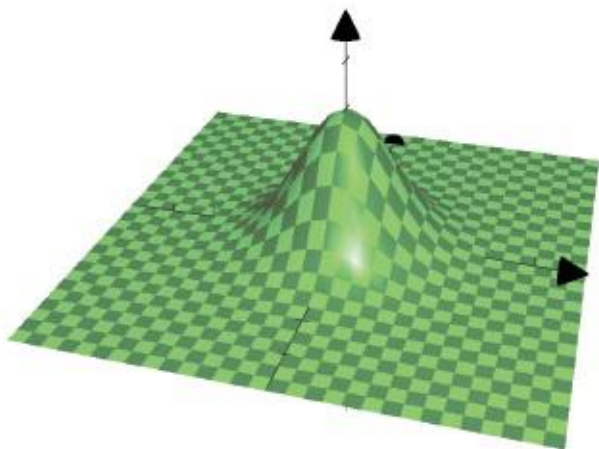
hàm cơ sở bán kính có công thức $y = e^{-x^2}$. Điều này trông giống như một vết lồi trên đường (hình 11.24). Nó trông rất giống một bản phân phối chuẩn (Gaussian) tiêu chuẩn. Phân phối chuẩn chuẩn hóa cũng tương tự, nhưng nó có công thức hơi khác một chút, sao cho diện tích bên dưới nó là 1.



Hình 11.24 Một ví dụ về hàm cơ sở hướng tâm. Nó trông rất giống một bản phân phối (Gaussian) bình thường.

Lưu ý rằng vết sung này xảy ra ở mức 0. Nếu chúng ta muốn nó xuất hiện ở bất kỳ thời điểm nào khác điểm, giả sử p , chúng ta có thể dịch công thức và nhận được $y = e^{-(x-p)^2}$. Ví dụ, Hàm cơ sở xuyên tâm có tâm tại điểm 5 chính xác là $y = e^{-(x-5)^2}$.

Đối với hai biến, công thức hàm cơ sở bán kính cơ bản nhất là $z = e^{-(x^2 + y^2)}$, và nó trông giống như đồ thị trong hình 11.25. Một lần nữa, bạn có thể nhận thấy rằng nó trông rất giống một phân phối chuẩn đa biến. Một lần nữa, đây là phiên bản sửa đổi của phân phối chuẩn đa biến.



Hình 11.25 Hàm cơ sở bán kính trên hai biến. Nó lại trông rất giống một phân phối bình thường. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Sự va chạm này xảy ra chính xác tại điểm (0,0). Nếu chúng ta muốn nó xuất hiện bất cứ lúc nào

điểm khác, chẳng hạn (p, q) , chúng ta có thể dịch công thức và nhận được $y = e^{-(x-p)^2 + (y-q)^2}$.

Ví dụ: hàm cơ sở bán kính có tâm tại điểm $(2, -3)$ chính xác là

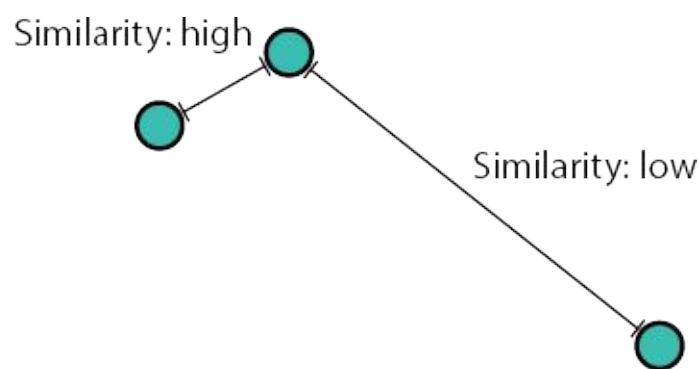
$$y = e^{-[(x-2)^2 + (y+3)^2]}.$$

Với n biến, công thức hàm cơ sở hướng tâm cơ bản là $y = e^{-(x_1^2 + \dots + x_n^2)}$. Chúng ta không thể vẽ đồ thị theo $n+1$ chiều, nhưng nếu chúng ta tưởng tượng vẽ một tấm chăn có n chiều và dùng ngón tay nhắc nó lên thì nó trông như thế này. Tuy nhiên, vì thuật toán mà chúng ta sử dụng hoàn toàn là toán học nên máy tính không gặp khó khăn gì khi chạy nó với bao nhiêu biến tùy thích. Như thường lệ, phần lớn n chiều này có tâm ở điểm 0, nhưng nếu chúng ta muốn nó tập trung ở điểm $(p_1, \dots, p_n)$, thì

$$\text{công thức là } y = e^{-[(x_1 - p_1)^2 + \dots + (x_n - p_n)^2]}.$$

BIỆN PHÁP VỀ CÁC ĐIỂM ĐÓNG NHƯ THẾ NÀO: ĐỒNG ĐỒNG

Để xây dựng một SVM sử dụng nhân RBF, chúng ta cần một khái niệm: khái niệm *sự tương tự*. Chúng ta nói rằng hai điểm giống nhau nếu chúng ở gần nhau và không tương tự nếu chúng ở xa (hình 11.26). Nói cách khác, độ tương tự giữa hai điểm là cao nếu chúng ở gần nhau và thấp nếu chúng ở xa nhau. Nếu cặp điểm là cùng một điểm thì độ tương tự là 1. Về lý thuyết, độ tương tự giữa hai điểm cách nhau một khoảng vô hạn là 0.

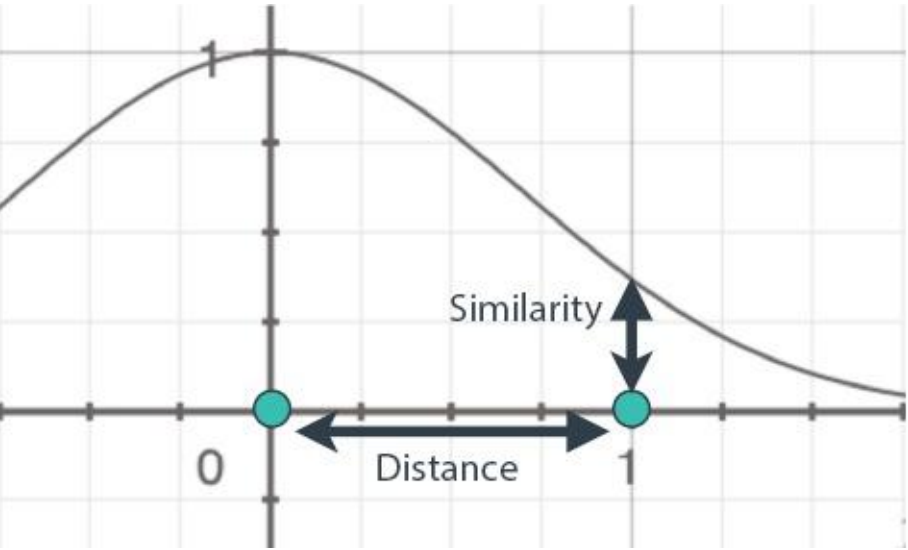


Hình 11.26 Hai điểm gần nhau được xác định là có độ tương tự cao. Hai điểm ở xa nhau được xác định là có độ tương tự thấp.

Bây giờ chúng ta cần tìm một công thức cho sự giống nhau. Như chúng ta có thể thấy, độ tương tự giữa hai điểm giảm khi khoảng cách giữa chúng tăng lên. Vì vậy, nhiều công thức về sự tương tự sẽ có tác dụng miễn là chúng thỏa mãn điều kiện đó. Bởi vì chúng ta đang sử dụng các hàm mũ trong phần này nên hãy định nghĩa nó như sau. Đối với các điểm p và q , độ tương đồng giữa p và q như sau:

$$\text{similarity}(p, q) = e^{-\text{distance}(p, q)^2}$$

Điều đó có vẻ giống như một công thức phức tạp về sự giống nhau, nhưng có một cách rất hay để xem xét nó. Nếu chúng ta muốn tìm sự giống nhau giữa hai điểm, chẳng hạn p và q , thì sự giống nhau này chính xác là chiều cao của hàm cơ sở bán kính có tâm tại p và được áp dụng tại điểm q . Nghĩa là, nếu ta véo chần tại điểm p và nhấc lên thì chiều cao của chần tại điểm q là cao nếu q gần với p và thấp nếu q ở xa p . Trong hình 11.27, chúng ta có thể thấy điều này đối với một biên, nhưng hãy tưởng tượng nó với bất kỳ số lượng biến nào bằng cách sử dụng phép loại suy chung.



Hình 11.27 Độ tương tự được định nghĩa là chiều cao của một điểm trong hàm cơ sở bán kính, trong đó đầu vào là khoảng cách. Lưu ý rằng khoảng cách càng cao thì độ tương tự càng thấp và ngược lại.

Đào tạo SVM bằng nhân RBF

Bây giờ chúng ta đã có tất cả các công cụ để huấn luyện SVM bằng hạt nhân RBF, hãy xem cách kết hợp tất cả lại với nhau. Trước tiên chúng ta hãy xem tập dữ liệu đơn giản được hiển thị trong hình 11.19. Bản thân tập dữ liệu xuất hiện trong bảng 11.6.

Bảng 11.6 Bộ dữ liệu một chiều được trình bày trong hình 11.19. Lưu ý rằng nó không thể phân tách tuyến tính vì điểm có nhãn 0 nằm ngay giữa hai điểm có nhãn 1.

Điểm	x	y (nhãn)
1	-1	1
2	0	0
3	1	1

Như chúng ta đã thấy, tập dữ liệu này không thể phân tách tuyến tính. Để làm cho nó có thể phân tách tuyến tính,

chúng tôi sẽ thêm một vài cột.Ba cột chúng tôi đang thêm là các cột tương tự và chúng ghi lại sự tương đồng giữa các điểm.Sự giống nhau giữa

hai điểm có x-tọa độ x_1 và x_2 được đo bằng $e^{(x_1 + x_2)^2}$, như đã nêu trong phần “Sử dụng các điểm và chạm ở các chiều cao hơn để mang lại lợi ích cho chúng ta”.Ví dụ,

độ tương đồng giữa điểm 1 và điểm 2 là $e^{(-1 - 0)^2} = 0,368$.Trong cột Sim1, chúng ta sẽ ghi lại sự tương đồng giữa điểm 1 và ba điểm còn lại, v.v.Tập dữ liệu mở rộng được thể hiện trong bảng 11.7.

Bảng 11.7 Chúng tôi mở rộng tập dữ liệu trong bảng 11.6 bằng cách thêm ba cột mới. Mỗi cột tương ứng với sự giống nhau của tất cả các điểm đối với từng điểm. Tập dữ liệu mở rộng này tồn tại trong không gian bốn chiều và có thể phân tách tuyến tính.

Điểm	x	Sim1	Sim2	Sim3	y
1	-1	1	0.368	0.018	1
2	0	0.368	1	0.368	0
3	1	0.018	0.368	1	1

Tập dữ liệu mở rộng này hiện có thể phân tách tuyến tính!Nhiều bộ phân loại sẽ tách tập hợp này, nhưng cụ thể, bộ phân loại có phương trình biên sau sẽ:

$$\hat{y} = step(Sim1 - Sim2 + Sim3)$$

Hãy xác minh điều này bằng cách dự đoán nhãn tại mọi điểm như minh họa tiếp theo:

- **Điểm 1:** $\hat{y} = bước(1 - 0,368 + 0,018) = bước(0,65) = 1$
- **Điểm 2:** $\hat{y} = bước(0,368 - 1 + 0,368) = bước(-0.264) = 0$
- **Điểm 3:** $\hat{y} = bước(0,018 - 0,368 + 1) = bước(0,65) = 1$

Hơn nữa, vì $Sim1=e^{(x + 1)^2}$, $Sim2=e^{(x - 0)^2}$ và $Sim3=e^{(x - 1)^2}$ nên cuối cùng của chúng tôi bộ phân loại đưa ra các dự đoán sau:

$$\hat{y} = step(e^{(x + 1)^2} - e^{x^2} + e^{(x - 1)^2})$$

Bây giờ, hãy thực hiện quy trình tương tự nhưng theo hai chiều.Phần này không yêu cầu mã, nhưng các phép tính rất lớn, vì vậy nếu bạn muốn xem qua chúng, chúng có trong sổ tay sau:

https://github.com/luiguiserrano/manning/blob/master/Chapter_11_Support_Vecto

Bảng 11.8 A tập dữ liệu đơn giản hai chiều, được vẽ trong hình 11.28

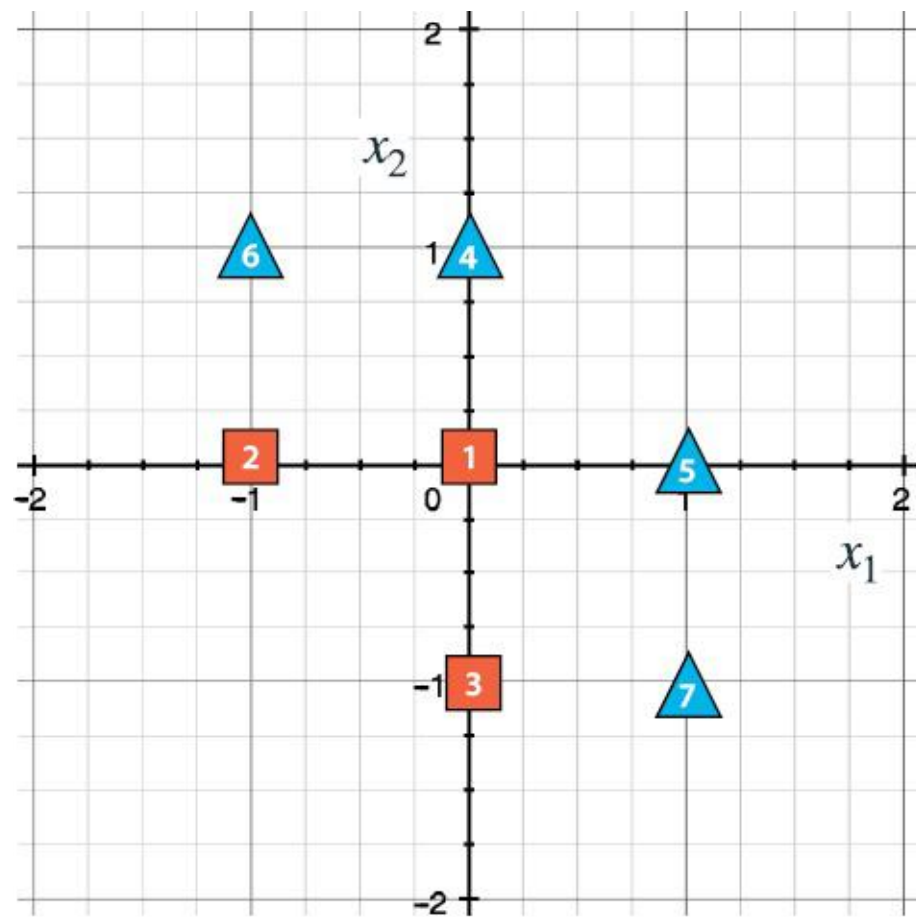
We'll use an SVM with an RBF kernel to classify this dataset.

Point	x_1	x_2	y
1	0	0	0
2	-1	0	0
3	0	-1	0
4	0	1	1
5	1	0	1
6	-1	1	1
7	1	-1	1

Consider the dataset in table 11.8, which we already classified graphically (figures 11.22 and 11.23). For convenience, it is plotted again in figure 11.28. In this plot, the points with label 0 appear as squares and those with label 1 as triangles.

Notice that in the first column of table 11.8 and in figure 11.28, we have numbered every point. This is not part of the data; we did it only for convenience. We will now add seven columns to this table. The columns are the similarities with respect to every point. For example, for point 1, we add a similarity column named Sim1. The entry for every point in this column is the amount of similarity between that point and point 1. Let's calculate one of them, for example, the similarity with point 6. The distance between point 1 and point 6, by the Pythagorean theorem follows:

$$distance(point\ 1, point\ 6) = \sqrt{(0+1)^2 + (0-1)^2} = \sqrt{2}$$



Hình 11.28 Sơ đồ của tập dữ liệu trong bảng 11.8, trong đó các điểm có nhãn 0 là hình vuông và các điểm có nhãn 1 là hình tam giác. Lưu ý rằng hình vuông và hình tam giác không thể được phân tách bằng một đường thẳng. Chúng tôi sẽ sử dụng SVM với nhân RBF để phân tách chúng bằng đường viền cong.

Vì vậy, sự giống nhau chính xác là

$$\text{similarity}(\text{point } 1, \text{point } 6) = e^{-\text{distance}(q,p)^2} = e^{-2} = 0.135.$$

Số này nằm ở hàng 1 và cột Sim6 (và theo tính đối xứng, cũng ở hàng 6 và cột Sim1). Điền thêm một vài giá trị vào bảng này để thuyết phục bản thân rằng trường hợp này xảy ra hoặc hãy xem sổ ghi chép nơi toàn bộ bảng được tính toán. Kết quả được thể hiện ở bảng 11.9.

Bảng 11.9 Chúng tôi đã thêm bảy cột tương tự vào tập dữ liệu trong bảng 11.8. Mỗi điểm ghi lại những điểm tương đồng với tất cả sáu điểm còn lại.

Point	x_1	x_2	Sim1	Sim2	Sim3	Sim4	Sim5	Sim6	Sim7	y
1	0	0	1	0.368	0.368	0.368	0.368	0.135	0.135	0
2	-1	0	0.368	1	0.135	0.135	0.018	0.368	0.007	0
3	0	-1	0.368	0.135	1	0.018	0.135	0.007	0.368	0
4	0	1	0.368	0.135	0.018	1	0.135	0.368	0.007	1
5	1	0	0.368	0.018	0.135	0.135	1	0.007	0.368	1
6	-1	1	0.135	0.368	0.007	0.367	0.007	1	0	1
7	1	-1	0.135	0.007	0.368	0.007	0.368	0	1	1

Notice the following things:

1. The similarity between each point and itself is always 1.
2. For each pair of points, the similarity is high when they are close in the plot and low when they are far.
3. The table consisting of the columns Sim1 to Sim7 is symmetric, because the similarity between p and q is the same as the similarity between q and p (as it depends only on the distance between p and q).
4. The similarity between points 6 and 7 appears as 0, but in reality, it is not. The distance between points 6 and 7 is $\sqrt{2^2 + 2^2} = \sqrt{8}$, so their similarity is $e^{-8} = 0.00033546262$, which rounds to zero because we are using three significant figures.

Now, on to building our classifier! Notice that for the data in the small table 11.8, no linear classifier works (because the points can't be split by a line), but on the much larger table 11.9, which has a lot more features (columns), we can fit such a classifier. We proceed to fit an SVM to this data. Many SVMs can classify this dataset correctly, and in the notebook, I've used Turi Create to build one. However, a simpler one works as well. This classifier has the following weights:

- The weights of x_1 and x_2 are 0.
- The weight of Sim p is 1, for $p = 1, 2$, and 3.
- The weight of Sim p is -1, for $p = 4, 5, 6$, and 7.
- The bias is $b = 0$.

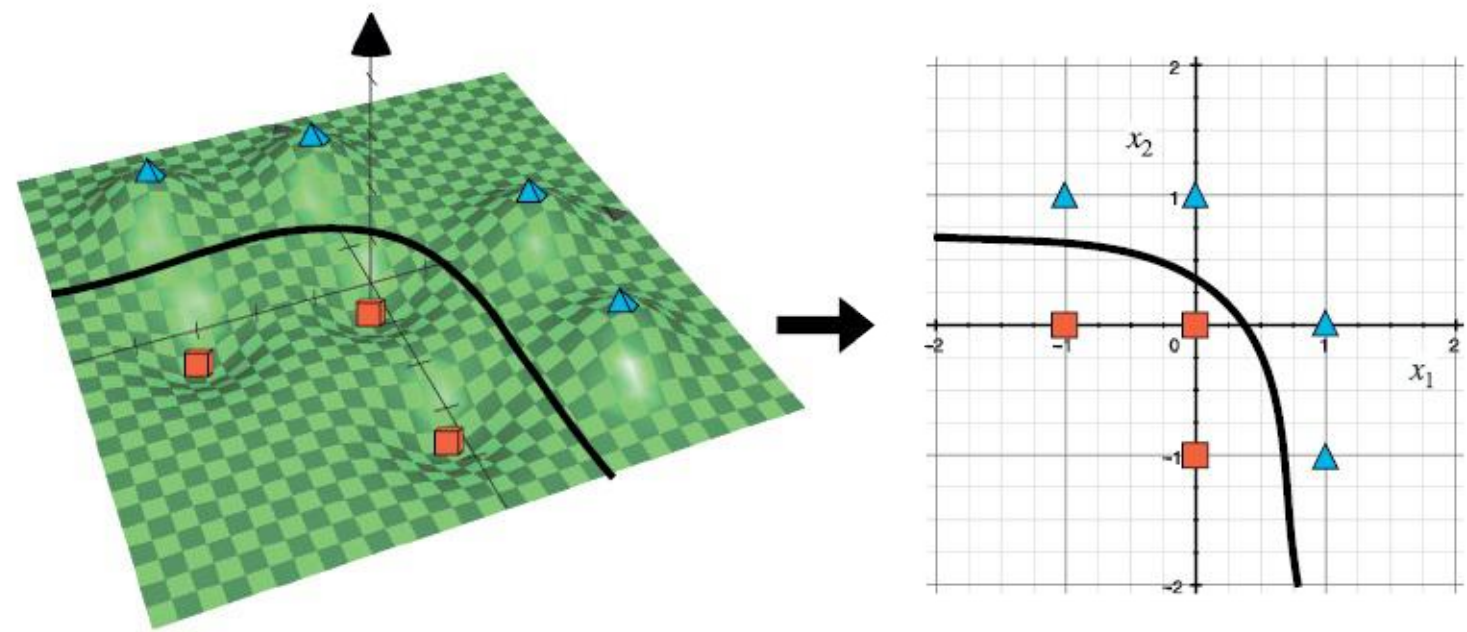
We find the classifier was adding a label -1 to the columns corresponding to the points labeled 0, and a +1 to the columns corresponding to the points labeled 1.

Điều này tương đương với quá trình thêm một ngọn núi tại bất kỳ điểm nào của nhãn 1 và một thung lũng tại mọi điểm của nhãn 0, như trong hình 11.29. Để kiểm tra về mặt toán học xem điều này có hiệu quả hay không, hãy lấy bảng 11.7, cộng các giá trị của các cột Sim4, Sim5, Sim6 và Sim7, sau đó trừ các giá trị của các cột Sim1, Sim2 và Sim3. Bạn sẽ nhận thấy rằng bạn nhận được số âm ở ba hàng đầu tiên và số dương ở bốn hàng cuối cùng. Do đó, chúng tôi có thể sử dụng ngưỡng 0 và chúng tôi có một bộ phân loại giúp phân loại chính xác tập dữ liệu này, vì các điểm được gán nhãn 1 nhận được điểm dương và các điểm được gán nhãn 0 nhận được điểm âm. Sử dụng ngưỡng 0 tương đương với việc sử dụng đường bờ biển để phân tách các điểm trong đồ thị trên hình 11.29.

Nếu chúng ta cắm hàm tương tự vào, bộ phân loại chúng ta thu được sẽ như sau:

$$\hat{y} = \text{step}(-e^{x_1^2 + x_2^2} - e^{(x_1 + 1)^2 + x_2^2} - e^{x_1^2 + (x_2 + 1)^2} + e^{x_1^2 + (x_2 - 1)^2} + e^{(x_1 - 1)^2 + x_2^2} + e^{(x_1 + 1)^2 + (x_2 - 1)^2} + e^{(x_1 - 1)^2 + (x_2 + 1)^2})$$

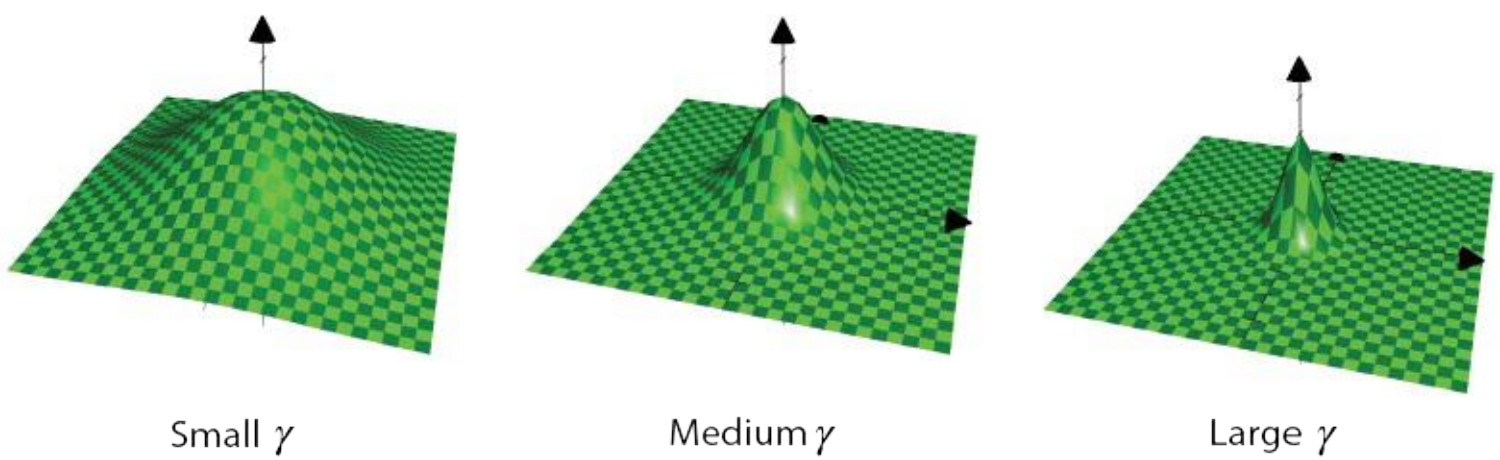
Tóm lại, chúng tôi đã tìm thấy một tập dữ liệu không thể phân tách tuyến tính. Chúng tôi đã sử dụng các hàm cơ sở xuyên tâm và độ tương tự giữa các điểm để thêm một số cột vào tập dữ liệu. Điều này đã giúp chúng tôi xây dựng một bộ phân loại tuyến tính (trong một không gian có chiều cao hơn nhiều). Sau đó, chúng tôi chiếu bộ phân loại tuyến tính chiều cao hơn vào mặt phẳng để có được bộ phân loại mà chúng tôi muốn. Chúng ta có thể thấy kết quả phân loại cong trong hình 11.29.



Hình 11.29 Trong tập dữ liệu này, chúng tôi nâng từng hình tam giác lên và hạ xuống từng hình vuông. Sau đó, chúng ta vẽ một mặt phẳng ở độ cao 0, ngăn cách các hình vuông và hình tam giác. Mặt phẳng cắt bề mặt theo một ranh giới cong. Sau đó, chúng tôi chiếu mọi thứ trở lại hai chiều và ranh giới cong này là ranh giới ngăn cách hình tam giác với hình vuông của chúng tôi. Ranh giới được vẽ ở bên phải. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

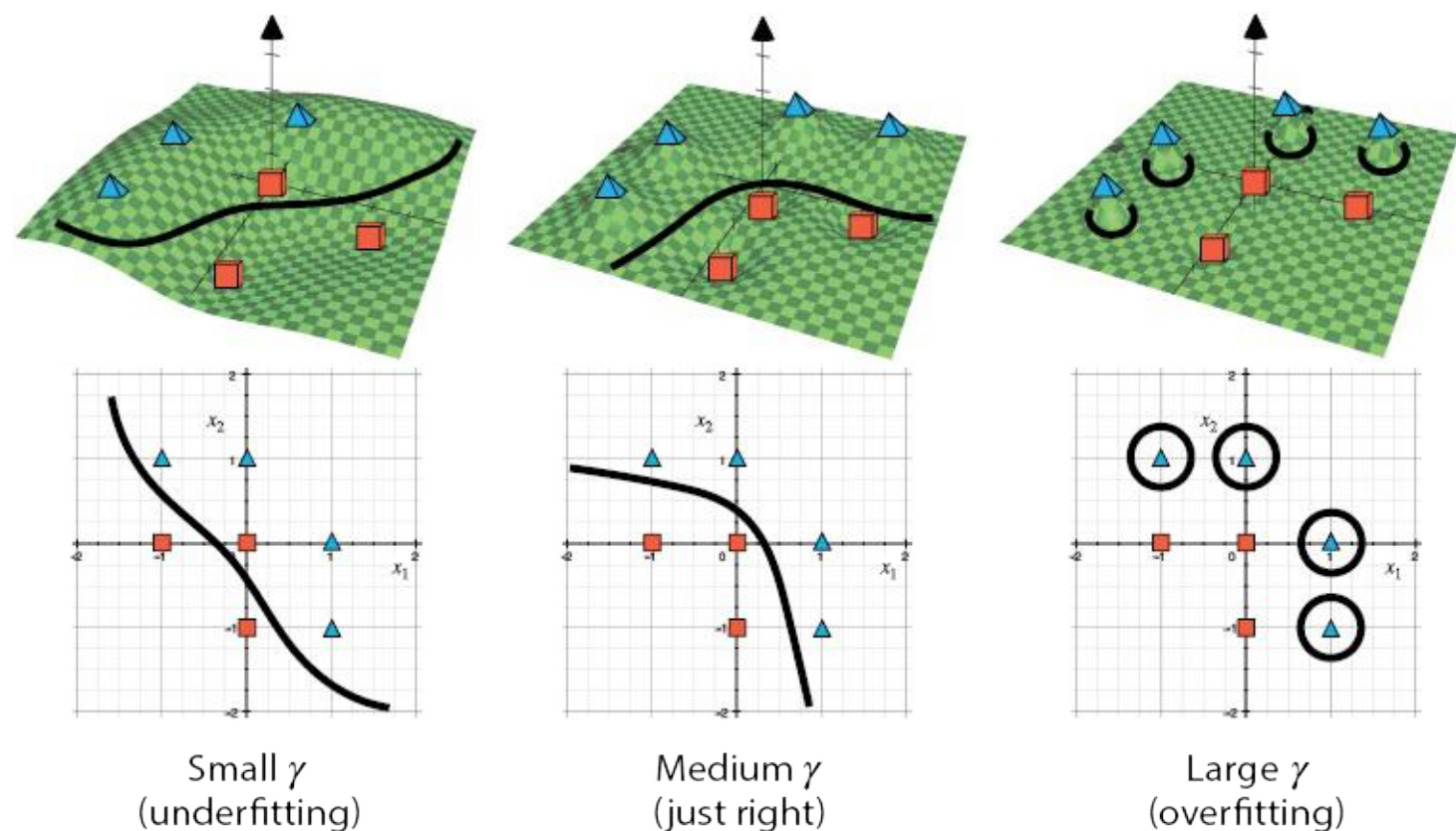
TRANG ĐẸP VÀ HƯỚNG DẪN VỚI NHÂN RBF: THÔNG SỐ GAMMA

Ở phần đầu của phần này, chúng ta đã đề cập rằng tồn tại nhiều hàm cơ sở bán kính khác nhau, cụ thể là một hàm cho mỗi điểm trong mặt phẳng. Thực tế còn rất nhiều nữa. Một số trong số chúng nâng mặt phẳng tại một điểm và tạo thành một bề mặt hẹp, còn một số khác tạo thành một bề mặt rộng. Một số ví dụ có thể được nhìn thấy trong hình 11.30. Trong thực tế, độ rộng của các hàm cơ sở xuyên tâm là điều chúng tôi muốn điều chỉnh. Để làm điều này, chúng tôi sử dụng một tham số được gọi là *tham số gamma*. Khi gamma nhỏ, bề mặt hình thành rất rộng và khi gamma lớn, bề mặt rất hẹp.



Hình 11.30 Tham số gamma xác định bề rộng của bề mặt. Lưu ý rằng đối với các giá trị gamma nhỏ, bề mặt rất rộng và đối với các giá trị gamma lớn, bề mặt rất hẹp. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Gamma là một siêu tham số. Hãy nhớ lại rằng siêu tham số là các thông số kỹ thuật mà chúng ta sử dụng để huấn luyện mô hình của mình. Cách chúng tôi điều chỉnh siêu tham số này là sử dụng các phương pháp mà chúng tôi đã thấy trước đây, chẳng hạn như biểu đồ độ phức tạp của mô hình (phần “Một cách bằng số để quyết định mức độ phức tạp của mô hình của chúng tôi” trong chương 4). Các giá trị gamma khác nhau có xu hướng phù hợp quá mức và không phù hợp. Chúng ta hãy nhìn lại ví dụ ở đầu phần này, với ba giá trị gamma khác nhau. Ba mô hình được vẽ trong hình 11.31.



Hình 11.31 Ba bộ phân loại SVM được hiển thị với nhân RBF và các giá trị gamma khác nhau. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Lưu ý rằng đối với giá trị gamma rất nhỏ, mô hình sẽ phù hợp quá mức vì đường cong quá đơn giản và nó không phân loại tốt dữ liệu của chúng ta. Đối với giá trị gamma lớn, mô hình này cực kỳ phù hợp vì nó xây dựng một ngọn núi nhỏ cho mỗi hình tam giác và một thung lũng nhỏ cho mỗi hình vuông. Điều này làm cho nó phân loại hầu hết mọi thứ thành hình vuông, ngoại trừ các khu vực xung quanh hình tam giác. Giá trị trung bình của gamma dường như hoạt động tốt vì nó xây dựng một ranh giới đủ đơn giản nhưng vẫn phân loại các điểm một cách chính xác.

Phương trình của hàm cơ sở bán kính không thay đổi nhiều khi chúng ta cộng tham số gamma—tất cả những gì chúng ta phải làm là nhân số mũ với gamma. Trong trường hợp tổng quát, phương trình của hàm cơ sở bán kính như sau:

$$y = e^{-\gamma[(x_1 - p_1)^2 + \dots + (x_n - p_n)^2]}$$

Đừng lo lắng nhiều về việc học công thức này—chỉ cần nhớ rằng ngay cả ở những chiều không gian cao hơn, những chỗ va chạm mà chúng ta tạo ra có thể rộng hoặc hẹp. Như thường lệ, có một cách để viết mã này và làm cho nó hoạt động, đó là những gì chúng ta sẽ làm trong phần tiếp theo.

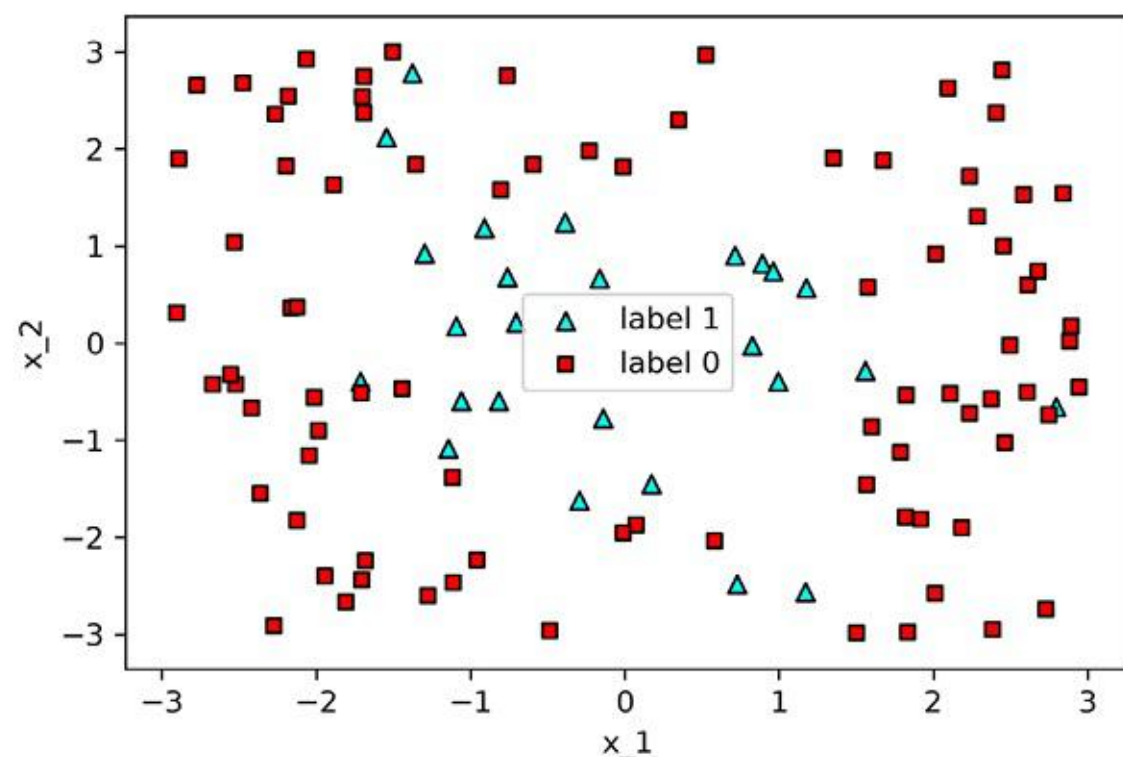
Mã hóa phương thức kernel

Bây giờ chúng ta đã học được phương pháp hạt nhân cho SVM, chúng ta học cách viết mã chúng trong Scikit- Tìm hiểu và huấn luyện mô hình trong một tập dữ liệu phức tạp hơn bằng cách sử dụng hạt nhân đa thức và RBF. Để huấn luyện một SVM trong Scikit-Learn bằng một hạt nhân cụ thể, tất cả những gì chúng ta làm là thêm hạt nhân làm tham số khi xác định SVM. Mã cho phần này như sau:

- **Sổ tay:** SVM_graphical_example.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support
- Bộ dữ liệu:
one_circle.csv
two_circles.csv

Mã hóa hạt nhân đa thức để phân loại tập dữ liệu hình tròn

Trong tiêu mục này, chúng ta sẽ xem cách mã hóa hạt nhân đa thức trong Scikit-Learn. Để làm điều này, chúng tôi sử dụng tập dữ liệu có tên one_circle.csv, được hiển thị trong hình 11.32.



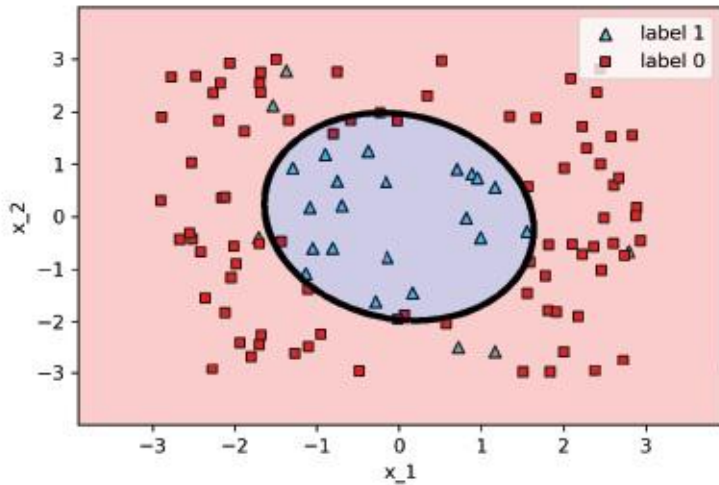
Hình 11.32 Một tập dữ liệu hình tròn có một số nhiễu. Chúng tôi sẽ sử dụng SVM với hạt nhân đa thức để phân loại tập dữ liệu này.

Lưu ý rằng ngoài một số ngoại lệ, tập dữ liệu này chủ yếu là hình tròn. Chúng tôi huấn luyện một trình phân loại SVM trong đó chúng tôi chỉ định tham số là 2 và tham số là 2, như được hiển thị trong đoạn mã tiếp theo. Lý do chúng tôi muốn

bậc bằng 2 là do phương trình đường tròn là đa thức bậc 2. Kết quả được thể hiện trên hình 11.33.

```
svm_degree_2 = SVC(kernel='poly', degree=2)
svm_degree_2.fit(features, labels)
```

Kernel = Polynomial
Degree = 2
Accuracy = 0.891

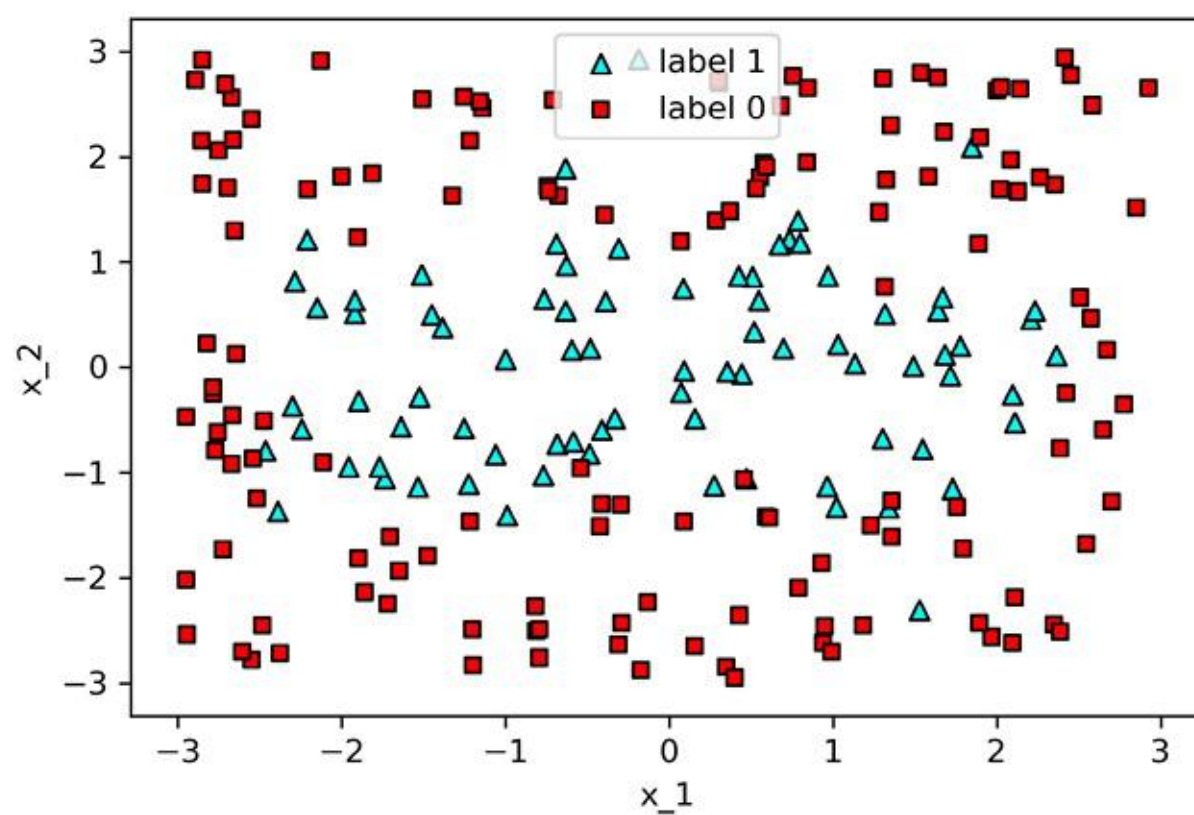


Hình 11.33 Bộ phân loại SVM với hạt nhân đa thức bậc 2

Lưu ý rằng SVM này với hạt nhân đa thức bậc 2 quản lý để xây dựng một vùng chủ yếu là hình tròn để ràng buộc tập dữ liệu như mong muốn.

Mã hóa hạt nhân RBF để phân loại tập dữ liệu được hình thành bởi hai vòng tròn giao nhau và chơi với tham số GAMMA

Chúng ta đã vẽ một vòng tròn, nhưng hãy phức tạp hơn. Trong tiểu mục này, chúng ta tìm hiểu cách mã hóa một số SVM bằng nhân RBF để phân loại tập dữ liệu có hình dạng hai vòng tròn giao nhau. Tập dữ liệu này, được gọi là two_circles.csv, được minh họa trong hình 11.34.



Hình 11.34 Một tập dữ liệu bao gồm hai vòng tròn giao nhau, với một số ngoại lệ. Chúng tôi sẽ sử dụng SVM với nhân RBF để phân loại tập dữ liệu này.

Để sử dụng kernel RBF, chúng tôi chỉ định γ . Chúng ta cũng có thể chỉ định một giá trị cho γ . Chúng ta sẽ huấn luyện bốn bộ phân loại SVM khác nhau cho các giá trị γ sau: 0,1, 1, 10 và 100, như minh họa tiếp theo:

```
svm_gamma_01 = SVC(kernel='rbf', gamma=0.1) ☐
svm_gamma_01.fit(features, labels)

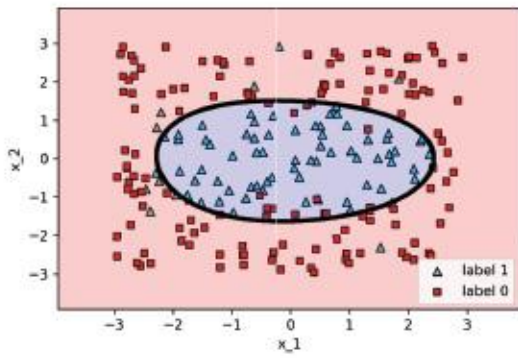
svm_gamma_1 = SVC(kernel='rbf', gamma=1) ☐
svm_gamma_1.fit(features, labels)

svm_gamma_10 = SVC(kernel='rbf', gamma=10) ☐
svm_gamma_10.fit(features, labels)

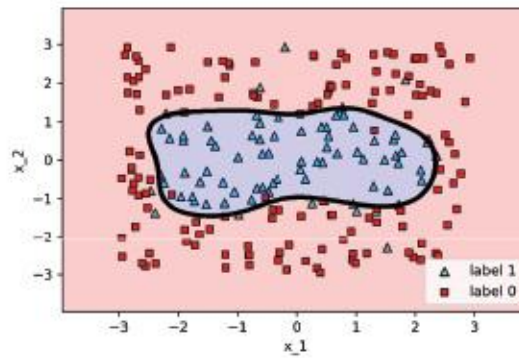
svm_gamma_100 = SVC(kernel='rbf', gamma=100) ☐
svm_gamma_100.fit(features, labels)
```

☐ Gamma = 0,1 ☐
☐ Gamma = 1 ☐
☐ Gamma = 10 ☐
☐ Gamma = 100

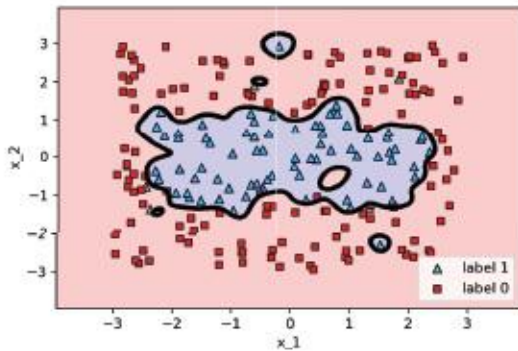
Kernel = RBF, Gamma = 0.1
Accuracy = 0.877



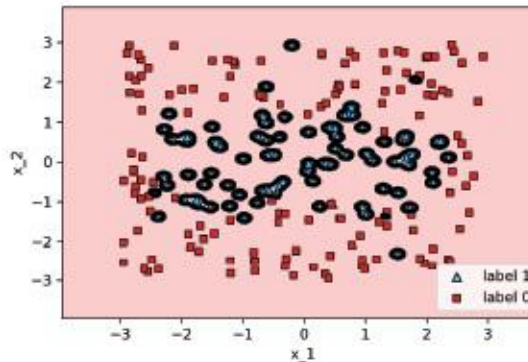
Kernel = RBF, Gamma = 1
Accuracy = 0.905



Kernel = RBF, Gamma = 10
Accuracy = 0.964



Kernel = RBF, Gamma = 100
Accuracy = 0.991



Hình 11.35 Bốn bộ phân loại SVM với nhân RBF và các giá trị gamma khác nhau

Bốn bộ phân loại xuất hiện trong hình 11.35. Lưu ý rằng đối với $\gamma = 0,1$, mô hình không phù hợp một chút vì nó cho rằng ranh giới là một hình bầu dục và nó mắc một số lỗi. $\gamma = 1$ đưa ra một mô hình tốt, nắm bắt dữ liệu tốt. Vào thời điểm chúng ta đạt đến $\gamma = 10$, chúng ta có thể thấy rằng mô hình bắt đầu quá phù hợp. Lưu ý cách nó cố gắng phân loại chính xác mọi điểm, bao gồm cả các điểm ngoại lệ mà nó bao quanh riêng lẻ. Vào thời điểm đạt tới $\gamma = 100$, chúng ta có thể thấy một số trang bị quá mức nghiêm trọng. Trình phân loại này chỉ bao quanh mỗi tam giác bằng một vùng hình tròn nhỏ và phân loại mọi thứ khác thành hình vuông. Do đó, đối với mô hình này, $\gamma = 1$ dường như là giá trị tốt nhất trong số những giá trị mà chúng tôi đã thử.

Tóm tắt

- Máy vector hỗ trợ (SVM) là một bộ phân loại bao gồm khớp hai đường thẳng song song (hoặc siêu phẳng) và cố gắng đặt chúng cách xa nhau nhất có thể, trong khi vẫn cố gắng phân loại dữ liệu một cách chính xác.

- Cách xây dựng các máy vector hỗ trợ là sử dụng hàm lỗi bao gồm hai thuật ngữ: tổng của hai lỗi perceptron, một lỗi trên mỗi đường song song và lỗi khoảng cách, cao khi hai đường song song cách xa nhau và thấp khi chúng ở gần nhau. Chúng tôi sử dụng tham số C để điều chỉnh giữa việc cố gắng phân loại các điểm một cách chính xác và cố gắng giãn cách các dòng. Điều này rất hữu ích trong quá trình đào tạo vì nó cho phép chúng ta kiểm soát các tùy chọn của mình, cụ thể là nếu chúng ta muốn xây dựng một bộ phân loại phân loại dữ liệu rất tốt hoặc một bộ phân loại có ranh giới cách đều nhau. Phương thức kernel là một công cụ hữu ích và rất mạnh để xây dựng các bộ phân loại phi tuyến. Phương thức kernel bao gồm việc sử dụng các hàm để giúp chúng tôi nhúng tập dữ liệu của mình vào trong một không gian có chiều cao hơn, trong đó các điểm có thể dễ dàng phân loại hơn bằng bộ phân loại tuyến tính. Điều này tương đương với việc thêm các cột vào tập dữ liệu của chúng tôi một cách thông minh để làm cho tập dữ liệu nâng cao có thể phân tách tuyến tính. Một số hạt nhân khác nhau, chẳng hạn như hạt nhân đa thức và hạt nhân RBF, đều có sẵn. Hạt nhân đa thức cho phép chúng ta xây dựng các vùng đa thức như hình tròn, parabol và hyperbol. Nhân RBF cho phép chúng ta xây dựng các vùng cong phức tạp hơn.
-

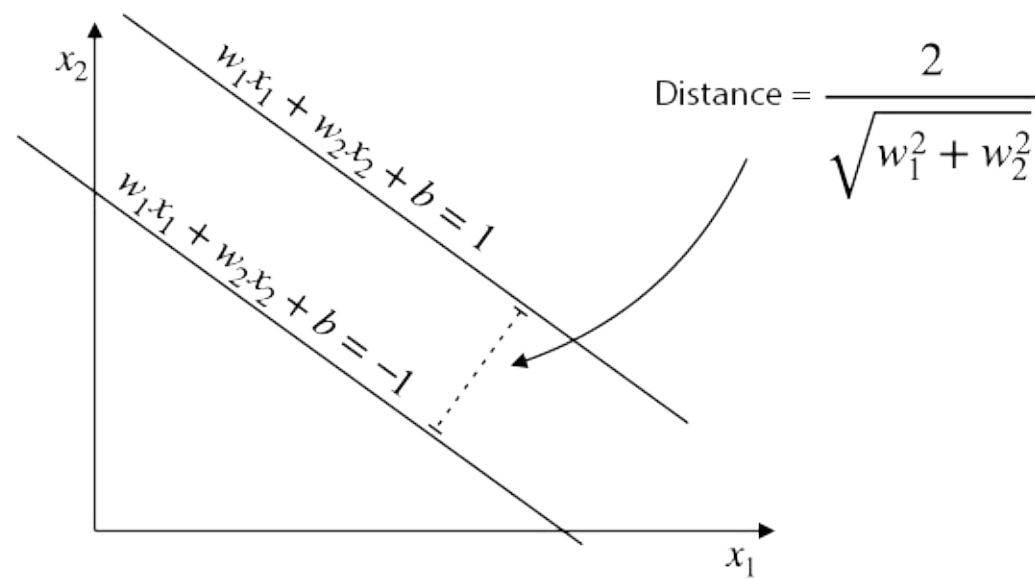
Bài tập

BÀI TẬP 11.1

(Bài tập này hoàn thành phép tính cần thiết trong phần “Hàm sai số khoảng cách.”)

Chúng minh rằng khoảng cách giữa các đường thẳng có phương trình $w_1x_1 + w_2x_1 + b = 1$ và

$$w_1x_1 + w_2x_1 + b = -1 \text{ chính xác là } \frac{2}{\sqrt{w_1^2 + w_2^2}}$$



EXERCISE 11.2

As we learned in exercise 5.3, it is impossible to build a perceptron model that mimics the XOR gate. In other words, it is impossible to fit the following dataset (with 100% accuracy) with a perceptron model:

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{y}$
0	0	0
0	1	1
1	0	1
1	1	0

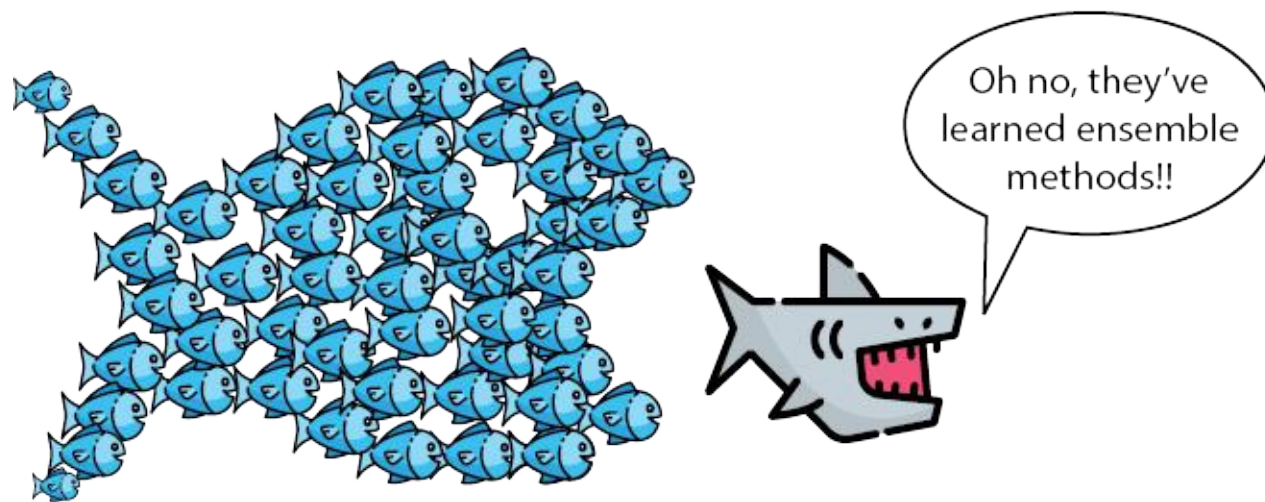
This is because the dataset is not linearly separable. An SVM has the same problem, because an SVM is also a linear model. However, we can use a kernel to help us out. What kernel should we use to turn this dataset into a linearly separable one? What would the resulting SVM look like?

hint Look at example 2 in the section “Using polynomial equations to your benefit,” which solves a very similar problem.

12 Kết hợp các mô hình để tối đa hóa kết quả: *Ensemble learning*

Trong chương này

- Học tập hợp là gì và cách nó được sử dụng để kết hợp các bộ phân loại yếu thành một bộ phân loại mạnh hơn bằng cách sử dụng đóng bao để kết hợp các bộ phân loại theo cách ngẫu nhiên
- bằng cách sử dụng tính năng tăng cường để kết hợp các bộ phân loại theo cách thông minh hơn
- một số phương pháp tập hợp phổ biến nhất: rừng ngẫu nhiên, AdaBoost, tăng cường độ dốc và XGBoost



Sau khi tìm hiểu nhiều mô hình học máy thú vị và hữu ích, người ta tự hỏi liệu có thể kết hợp các bộ phân loại này hay không. Rất may là chúng ta có thể làm được, và trong chương này, chúng ta học được một số cách để xây dựng các mô hình mạnh hơn bằng cách kết hợp các mô hình yếu hơn. Hai phương pháp chính chúng ta học trong chương này là đóng bao và tăng tốc. Tóm lại, đóng bao bao gồm việc xây dựng một vài mô hình một cách ngẫu nhiên và nối chúng lại với nhau. Mặt khác, việc tăng cường bao gồm việc xây dựng các mô hình này theo cách thông minh hơn bằng cách chọn từng mô hình một cách chiến lược để tập trung vào những sai lầm của các mô hình trước đó. Kết quả mà các phương pháp tập hợp này đã thể hiện trong các bài toán học máy quan trọng là rất lớn. Ví dụ: Giải thưởng Netflix, được trao cho mô hình tốt nhất phù hợp với tập dữ liệu lớn về dữ liệu lượng người xem Netflix, đã giành được bởi một nhóm sử dụng tập hợp nhiều mô hình khác nhau.

Trong chương này, chúng ta tìm hiểu một số mô hình tăng cường và đóng bao mạnh mẽ và phổ biến nhất, bao gồm các khu rừng ngẫu nhiên, AdaBoost, tăng cường độ dốc và

XGBoost. Phần lớn trong số này được mô tả để phân loại và một số được mô tả để hồi quy. Tuy nhiên, hầu hết các phương pháp tập hợp đều hoạt động trong cả hai trường hợp.

Một chút về thuật ngữ: trong suốt cuốn sách này, chúng tôi đã gọi các mô hình học máy là mô hình, hoặc đôi khi là các biến hồi quy hoặc bộ phân loại, tùy thuộc vào nhiệm vụ của chúng. Trong chương này, chúng tôi giới thiệu thuật ngữ *người học*, thuật ngữ này cũng đề cập đến mô hình học máy. Trong tài liệu, người ta thường sử dụng thuật ngữ *người học yếu* và *người học mạnh* khi nói về các phương pháp tập hợp. Tuy nhiên, không có sự khác biệt giữa mô hình học máy và người học.

Tất cả mã cho chương này đều có sẵn trong kho GitHub này:
https://github.com/luisguiserrano/manning/tree/master/Chapter_12_Ensemble_Meth

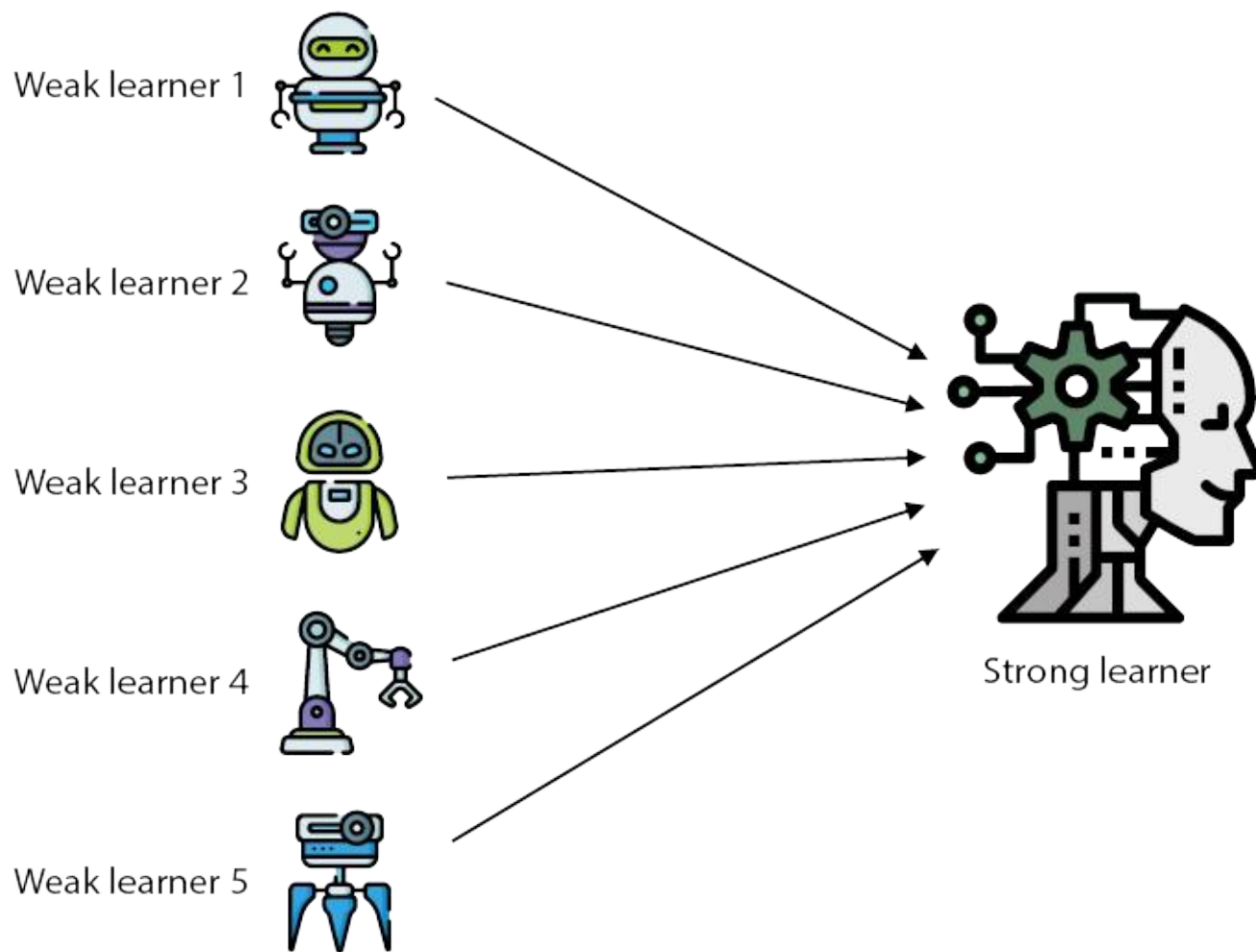
Với một chút giúp đỡ từ bạn bè

Hãy hình dung các phương pháp tổng hợp bằng cách tương tự sau: Hãy tưởng tượng rằng chúng ta phải làm một bài kiểm tra bao gồm 100 câu hỏi đúng/sai về nhiều chủ đề khác nhau, bao gồm toán, địa lý, khoa học, lịch sử và âm nhạc. May mắn thay, chúng tôi được phép gọi năm người bạn của mình—Adriana, Bob, Carlos, Dana và Emily—để giúp đỡ. Có một hạn chế nhỏ, đó là tất cả họ đều làm việc toàn thời gian và họ không có thời gian để trả lời tất cả 100 câu hỏi, nhưng họ rất sẵn lòng giúp chúng tôi giải quyết một số câu hỏi trong số đó. Chúng ta có thể sử dụng những kỹ thuật nào để nhận được sự giúp đỡ của họ? Hai kỹ thuật có thể thực hiện sau:

Kỹ thuật 1: Với mỗi người bạn, hãy chọn một số câu hỏi ngẫu nhiên và yêu cầu họ trả lời (đảm bảo mỗi câu hỏi đều nhận được câu trả lời từ ít nhất một người bạn của chúng ta). Sau khi chúng tôi nhận được câu trả lời, hãy trả lời bài kiểm tra bằng cách chọn tùy chọn phổ biến nhất trong số những người đã trả lời câu hỏi đó. Ví dụ: nếu hai người bạn của chúng tôi trả lời “Đúng” và một người trả lời “Sai” ở câu hỏi 1, thì chúng tôi trả lời câu hỏi 1 là “Đúng” (nếu hòa, chúng tôi có thể chọn ngẫu nhiên một trong các câu trả lời thắng).

Kỹ thuật 2: Chúng tôi giao bài thi cho Adriana và yêu cầu cô ấy chỉ trả lời những câu hỏi mà cô ấy chắc chắn nhất. Chúng tôi cho rằng những câu trả lời đó là tốt và loại chúng khỏi bài kiểm tra. Bây giờ chúng tôi đưa ra những câu hỏi còn lại cho Bob với hướng dẫn tương tự. Chúng tôi tiếp tục theo cách này cho đến khi chuyển nó cho cả năm người bạn.

Kỹ thuật 1 giống với thuật toán đóng bao và kỹ thuật 2 giống với thuật toán tăng cường. Cụ thể hơn, việc đóng gói và tăng cường sử dụng một tập hợp các mô hình được gọi là *người học yếu* và kết hợp chúng thành *người học mạnh* (như minh họa trong hình 12.1).



Hình 12.1 Phương pháp tập hợp bao gồm việc ghép một số người học yếu để xây dựng một người học mạnh.

Đóng gói: Xây dựng các tập hợp ngẫu nhiên bằng cách lấy các điểm ngẫu nhiên từ tập dữ liệu (có thay thế). Huấn luyện một mô hình khác nhau trên mỗi bộ. Những mô hình này là những người học yếu. Sau đó, mô hình học mạnh được hình thành dưới dạng kết hợp của các mô hình yếu và việc dự đoán được thực hiện bằng cách bỏ phiếu (nếu đó là mô hình phân loại) hoặc tính trung bình các dự đoán (nếu đó là mô hình hồi quy).

Tăng cường: Bắt đầu bằng cách huấn luyện một mô hình ngẫu nhiên, là mô hình học yếu đầu tiên. Đánh giá nó trên toàn bộ tập dữ liệu. Thu nhỏ những điểm có dự đoán tốt và phóng to những điểm có dự đoán kém. Huấn luyện người học yếu thứ hai trên tập dữ liệu đã sửa đổi này. Chúng tôi tiếp tục theo cách này cho đến khi xây dựng được một số mô hình. Cách kết hợp chúng thành một người học mạnh cũng giống như cách đóng gói, cụ thể là bằng cách bỏ phiếu hoặc lấy trung bình các dự đoán của người học yếu. Hơn

cụ thể, nếu người học là người phân loại, người học mạnh sẽ dự đoán lớp phổ biến nhất được dự đoán bởi người học yếu (do đó thuật ngữ *bỏ phiếu*) và nếu có mối quan hệ ràng buộc thì bằng cách chọn ngẫu nhiên trong số đó. Nếu người học là người hồi quy, người học mạnh sẽ dự đoán mức trung bình của các dự đoán do người học yếu đưa ra.

Hầu hết các mô hình trong chương này sử dụng cây quyết định (cả hồi quy và phân loại) như những mô hình học yếu. Chúng tôi làm điều này vì cây quyết định rất phù hợp với kiểu tiếp cận này. Tuy nhiên, khi bạn đọc chương này, tôi khuyến khích bạn nghĩ về cách kết hợp các loại mô hình khác, chẳng hạn như perceptron và SVM.

Chúng tôi đã dành cả một cuốn sách để xây dựng những người học rất giỏi. Tại sao chúng ta muốn kết hợp nhiều người học yếu thay vì chỉ xây dựng một người học giỏi ngay từ đầu? Một lý do là các phương pháp tập hợp đã được chứng minh là có mức độ phù hợp thấp hơn nhiều so với các mô hình khác. Tóm lại, một mô hình rất dễ bị quá khớp, nhưng nếu bạn có nhiều mô hình cho cùng một tập dữ liệu thì sự kết hợp của chúng sẽ ít phù hợp hơn. Theo một nghĩa nào đó, có vẻ như nếu một người học mắc lỗi thì những người khác có xu hướng sửa lỗi đó và nhìn chung, họ làm việc tốt hơn.

Chúng ta tìm hiểu các mô hình sau trong chương này. Thuật toán đầu tiên là thuật toán đóng bao và ba thuật toán cuối cùng là tăng cường:

- Rừng ngẫu nhiên
- AdaBoost Tăng cường
- độ dốc XGBoost
-

Tất cả các mô hình này đều hoạt động để hồi quy và phân loại. Vì mục đích giáo dục, chúng tôi học hai cái đầu tiên làm mô hình phân loại và hai cái cuối cùng là mô hình hồi quy. Quá trình này tương tự cho cả phân loại và hồi quy. Tuy nhiên, hãy đọc từng phần và tưởng tượng nó sẽ hoạt động như thế nào trong cả hai trường hợp. Để tìm hiểu cách tất cả các thuật toán này hoạt động để phân loại và hồi quy, hãy xem các liên kết tới video và tài liệu đọc trong phụ lục C giải thích chi tiết cả hai trường hợp.

Bagging: Tham gia ngẫu nhiên một số người học yếu để xây dựng một người học mạnh

Trong trường hợp này chúng ta thấy một trong những mô hình đóng bao nổi tiếng nhất: *ngẫu nhiên*

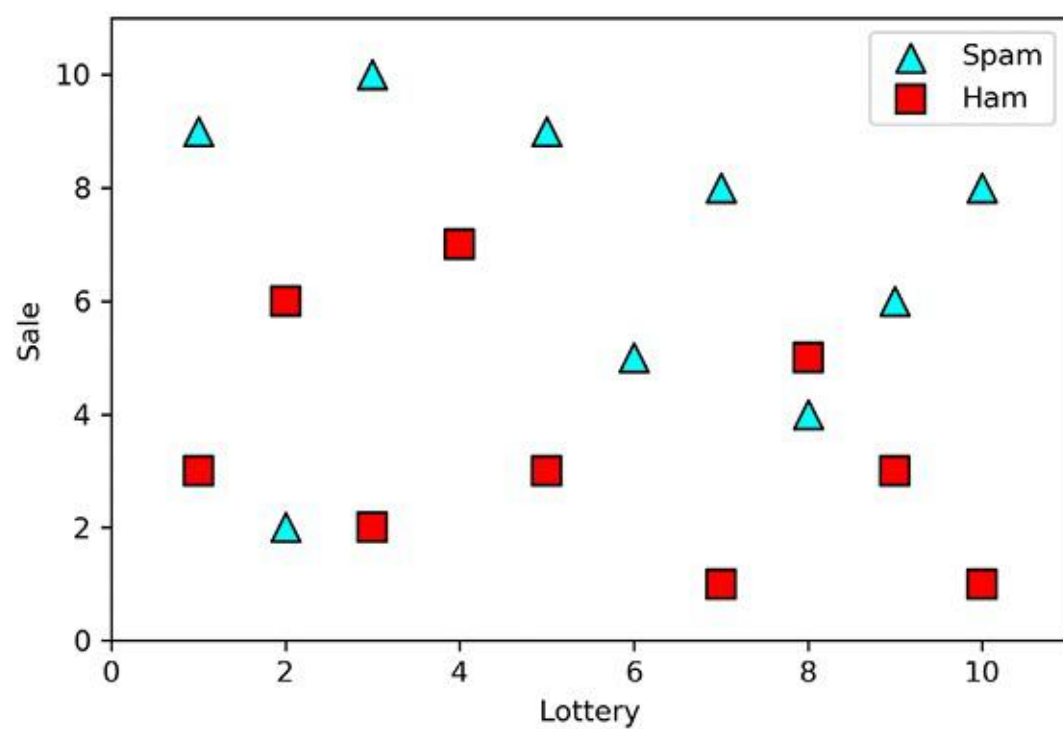
forest. In a random forest, the weak learners are small decision trees trained on random subsets of the dataset. Random forests work well for classification and regression problems, and the process is similar. We will see random forests in a classification example. The code for this section follows:

- **Notebook:** Random_forests_and_AdaBoost.ipynb
 - https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemb

We use a small dataset of spam and ham emails, similar to the one we used in chapter 8 with the naive Bayes model. The dataset is shown in table 12.1 and plotted in figure 12.2. The features of the dataset are the number of times the words “lottery” and “sale” appear in the email, and the “yes/no” label indicates whether the email is spam (yes) or ham (no).

Table 12.1 Table of spam and ham emails, together with the number of appearances of the words “lottery” and “sale” on each email

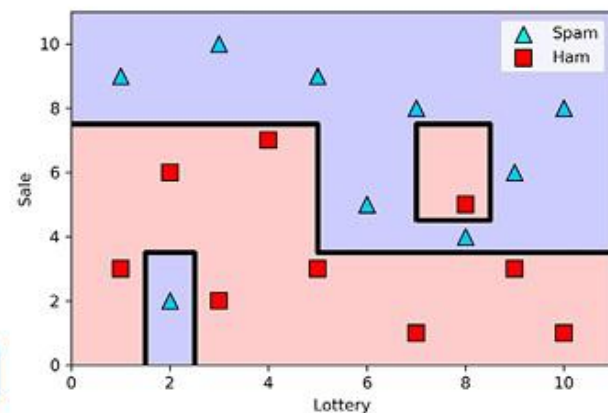
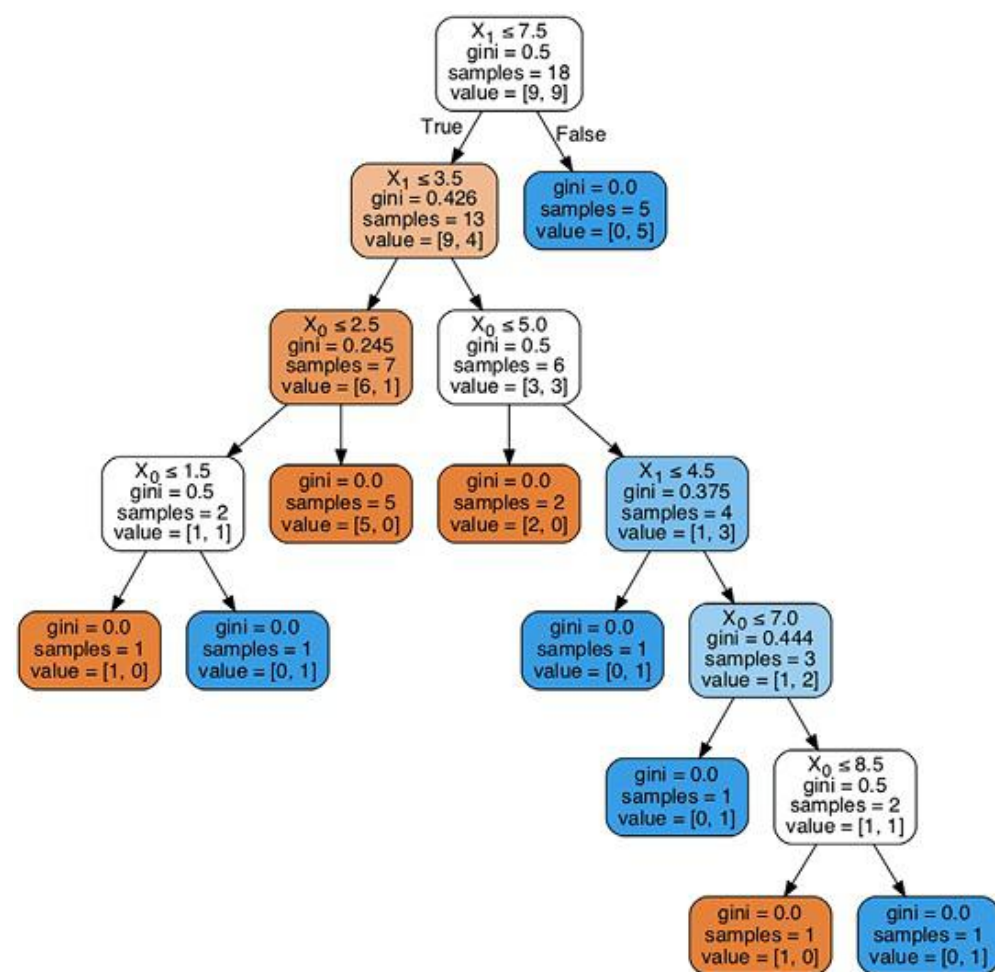
Lottery	Sale	Spam
7	8	1
3	2	0
8	4	1
2	6	0
6	5	1
9	6	1
8	5	0
7	1	0
1	9	1
4	7	0
1	3	0
3	10	1
2	2	1
9	3	0
5	3	0
10	1	0
5	9	1
10	8	1



Hình 12.2 Hình 12.2 Sơ đồ của tập dữ liệu trong bảng 12.1.Email rác được thể hiện bằng hình tam giác và email ham được thể hiện bằng hình vuông.Trục ngang và trục dọc biểu thị số lần xuất hiện của các từ “xổ số” và “bán hàng”.

ĐẦU TIÊN, (TRÊN) LẤP CÂY QUYẾT ĐỊNH

Trước khi chúng ta đi vào các khu rừng ngẫu nhiên, hãy lắp bộ phân loại cây quyết định cho dữ liệu này và xem nó hoạt động tốt như thế nào.Vì chúng ta đã học điều này trong chương 9 nên hình 12.3 chỉ hiển thị kết quả cuối cùng nhưng chúng ta có thể thấy mã trong sổ ghi chép.Ở bên trái của hình 12.3, chúng ta có thể thấy cây thật (khá sâu!), và ở bên phải, chúng ta có thể thấy đồ thị của ranh giới.Lưu ý rằng nó rất phù hợp với tập dữ liệu, với độ chính xác huấn luyện 100%, mặc dù rõ ràng là nó quá phù hợp.Việc trang bị quá mức có thể được nhận thấy ở hai ngoại lệ mà mô hình cố gắng phân loại chính xác mà không nhận thấy chúng là các ngoại lệ.



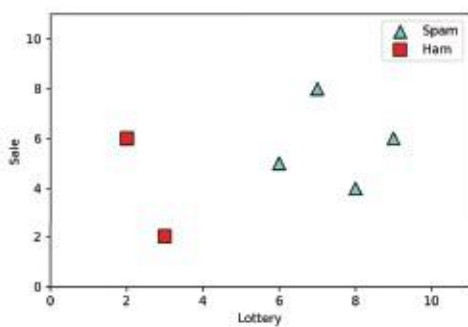
Hình 12.3 Bên trái: Cây quyết định phân loại tập dữ liệu của chúng tôi. Bên phải: Ranh giới được xác định bởi cây quyết định này. Lưu ý rằng nó phân chia dữ liệu rất tốt, mặc dù nó gợi ý về việc trang bị quá mức, vì một mô hình tốt sẽ coi hai điểm biệt lập là các điểm ngoại lệ, thay vì cố gắng phân loại chúng một cách chính xác.

Trong các phần tiếp theo, chúng ta sẽ xem cách giải quyết vấn đề trang bị quá mức này bằng cách khớp một khu rừng ngẫu nhiên.

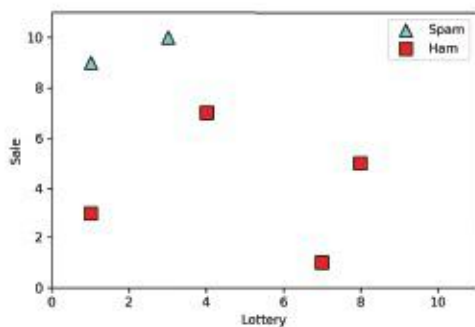
Gắn rừng ngẫu nhiên theo cách thủ công

Trong phần này, chúng ta tìm hiểu cách điều chỉnh một khu rừng ngẫu nhiên theo cách thủ công, mặc dù điều này chỉ nhằm mục đích giáo dục vì đây không phải là cách thực hiện trong thực tế. Tóm lại, chúng tôi chọn các tập hợp con ngẫu nhiên từ tập dữ liệu của mình và huấn luyện một trình học yếu (cây quyết định) trên mỗi tập hợp con đó. Một số điểm dữ liệu có thể thuộc về một số tập hợp con và những điểm khác có thể không thuộc về tập hợp con nào. Sự kết hợp của chúng là người học mạnh mẽ của chúng tôi. Cách người học giỏi đưa ra dự đoán là để người học yếu bỏ phiếu. Đối với tập dữ liệu này, chúng tôi sử dụng ba người học yếu. Vì tập dữ liệu có 18 điểm nên hãy xem xét ba tập con, mỗi tập có 6 điểm dữ liệu, như minh họa trong hình 12.4.

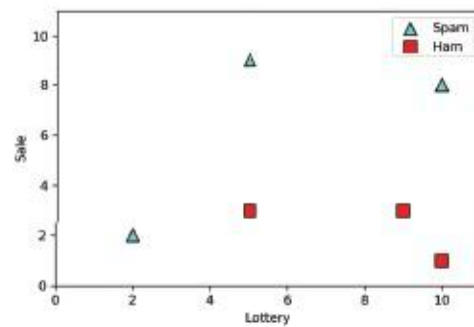
Subset 1



Subset 2



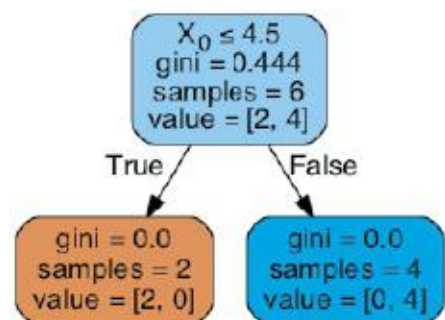
Subset 3



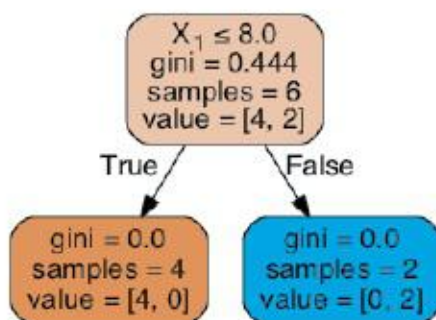
Hình 12.4 Bước đầu tiên để xây dựng một rừng ngẫu nhiên là chia dữ liệu của chúng ta thành ba tập hợp con. Đây là phân tách của tập dữ liệu được hiển thị trong hình 12.2.

Tiếp theo, chúng tôi tiến hành xây dựng ba người học yếu. Phù hợp với cây quyết định có độ sâu 1 trên mỗi tập hợp con này. Nhớ lại chương 9 rằng cây quyết định có độ sâu 1 chỉ chứa một nút và hai lá. Ranh giới của nó bao gồm một đường ngang hoặc dọc giúp phân chia tập dữ liệu một cách tốt nhất có thể. Những học sinh yếu được minh họa ở hình 12.5.

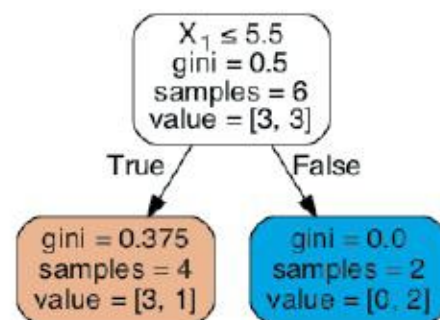
Weak learner 1



Weak learner 2



Weak learner 3



Hình 12.5 Ba người học yếu tạo thành rừng ngẫu nhiên của chúng tôi là những cây quyết định có độ sâu 1. Mỗi cây quyết định phù hợp với một trong ba tập hợp con tương ứng từ hình 12.4.

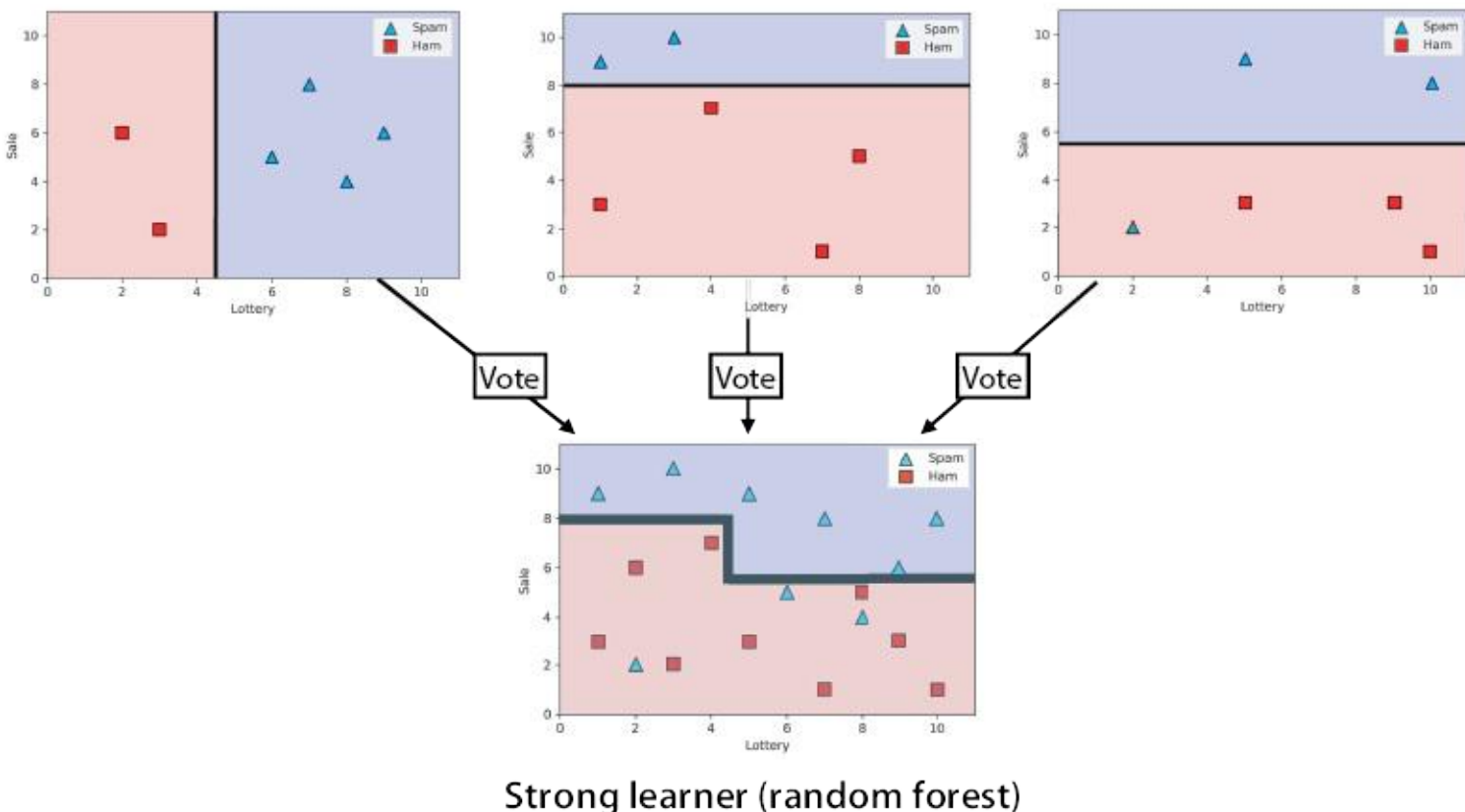
Chúng tôi kết hợp những điều này thành một người học hồi mạnh mẽ bằng cách bỏ phiếu. Nói cách khác, đối với bất kỳ đầu vào nào, mỗi người học yếu đều dự đoán giá trị 0 hoặc 1. Dự đoán mà người học giỏi đưa ra là đầu ra phổ biến nhất trong ba giá trị đó. Sự kết hợp này có thể được thấy trong hình 12.6, trong đó những người học yếu ở trên và những người học giỏi ở phía dưới.

Lưu ý rằng rừng ngẫu nhiên là một công cụ phân loại tốt vì nó phân loại hầu hết các điểm một cách chính xác, nhưng nó cho phép mắc một số lỗi để không làm dữ liệu bị khớp quá mức. Tuy nhiên, chúng ta không cần huấn luyện các khu rừng ngẫu nhiên này theo cách thủ công vì Scikit-Learn có các chức năng cho việc này mà chúng ta sẽ thấy trong phần tiếp theo.

Weak learner 1

Weak learner 2

Weak learner 3



Hình 12.6 Cách để có được dự đoán của rừng ngẫu nhiên là kết hợp dự đoán của ba người học yếu. Ở phía trên, chúng ta có thể thấy ba ranh giới của cây quyết định từ hình 12.5. Ở phía dưới, chúng ta có thể thấy ba cây quyết định bỏ phiếu như thế nào để có được ranh giới của khu rừng ngẫu nhiên tương ứng.

Huấn luyện một khu rừng ngẫu nhiên trong Scikit-Learn

Trong phần này, chúng ta sẽ xem cách huấn luyện một khu rừng ngẫu nhiên bằng Scikit-Learn. Trong đoạn mã sau, chúng tôi sử dụng gói `RandomForestClassifier` để bắt đầu, chúng tôi có dữ liệu của mình trong hai Khung dữ liệu Pandas được gọi là `features` và `labels`, như

hiển thị tiếp theo:

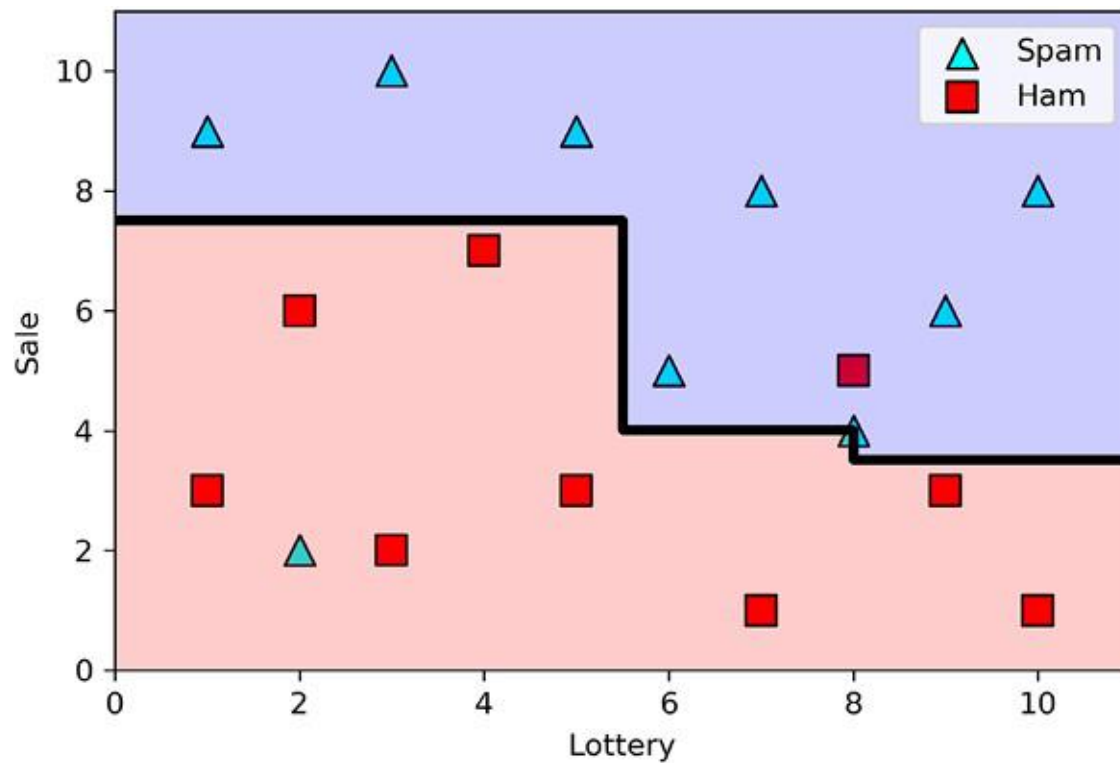
```
from sklearn.ensemble import RandomForestClassifier
random_forest_classifier = RandomForestClassifier(random_state=0, n_estimators=5, max_depth=1)
random_forest_classifier.fit(features, labels)
random_forest_classifier.score(features, labels)
```

Trong mã trước, chúng tôi đã chỉ định rằng chúng tôi muốn năm học viên yếu có siêu tham số. Những người học yếu này lại là cây quyết định, và

chúng tôi đã chỉ định rằng độ sâu của chúng là 1 với siêu tham số `max_depth`.

Sơ đồ của mô hình được thể hiện trên hình 12.7. Lưu ý rằng mô hình này mắc một số lỗi nhưng vẫn tìm được ranh giới hợp lý, nơi chứa các email spam

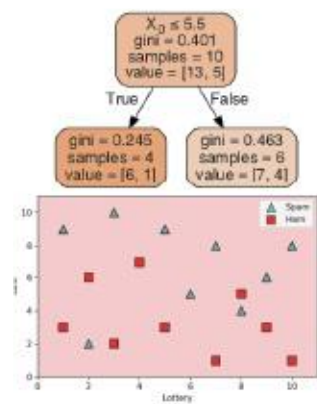
với sự xuất hiện nhiều của các từ “xổ số” và “bán hàng” (phía trên bên phải của cốt truyện) và các email ham là những email không xuất hiện nhiều các từ này (phía dưới bên trái của hình).



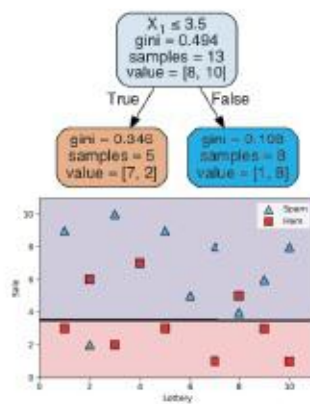
Hình 12.7 Ranh giới của khu rừng ngẫu nhiên thu được bằng Scikit-Learn. Lưu ý rằng nó phân loại tốt tập dữ liệu và coi hai điểm bị phân loại sai là các điểm ngoại lệ, thay vì cố gắng phân loại chúng một cách chính xác.

Scikit-Learn cũng cho phép chúng ta hình dung và vẽ đồ thị cho từng học viên yếu (xem mã để biết mã). Những học sinh yếu được thể hiện ở hình 12.8. Lưu ý rằng không phải tất cả những người học yếu đều hữu ích. Ví dụ: điểm đầu tiên phân loại mọi điểm là ham.

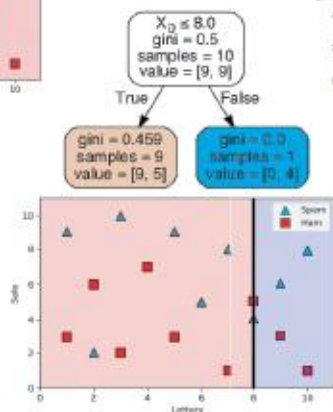
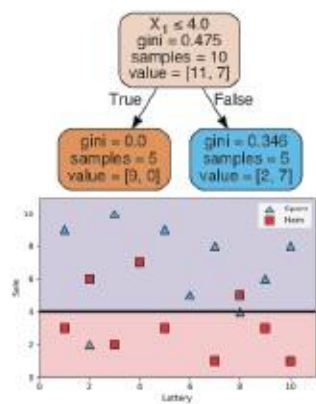
Weak learner 1



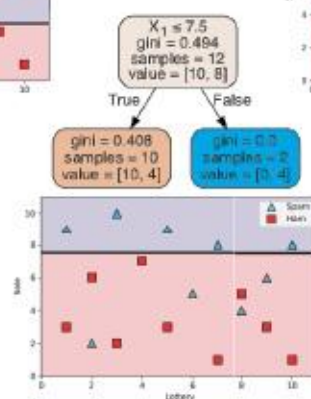
Weak learner 3



Weak learner 5



Weak learner 2



Weak learner 4

Hình 12.8 Rừng ngẫu nhiên được hình thành bởi năm học sinh yếu thu được bằng Scikit-Learn. Mỗi cây là một cây quyết định có độ sâu 1. Chúng kết hợp với nhau để tạo thành cây học mạnh như minh họa trong hình 12.7.

Trong phần này, chúng ta sử dụng cây quyết định có độ sâu 1 cho những người học yếu, nhưng nói chung, chúng ta có thể sử dụng cây có độ sâu bất kỳ mà chúng ta muốn. Hãy thử đào tạo lại mô hình này bằng cách sử dụng cây quyết định có độ sâu cao hơn bằng cách thay đổi siêu tham số `max_depth` và xem khu rừng ngẫu nhiên trông như thế nào!

AdaBoost: Kết nối những người học yếu một cách thông minh để xây dựng người học giỏi

Tăng cường cũng tương tự như đóng gói ở chỗ chúng ta tham gia cùng một số người học yếu để xây dựng một người học giỏi. Sự khác biệt là chúng tôi không chọn ngẫu nhiên những học sinh yếu. Thay vào đó, mỗi người học được xây dựng bằng cách tập trung vào điểm yếu của những người học trước. Trong phần này, chúng ta tìm hiểu một kỹ thuật tăng tốc mạnh mẽ tên là AdaBoost, được phát triển bởi Freund và Schapire vào năm 1997 (xem phụ lục C để tham khảo). AdaBoost là viết tắt của tăng cường thích ứng và nó hoạt động để hội quy và phân loại. Tuy nhiên, chúng tôi sẽ sử dụng nó trong một ví dụ phân loại minh họa rất rõ ràng về thuật toán huấn luyện.

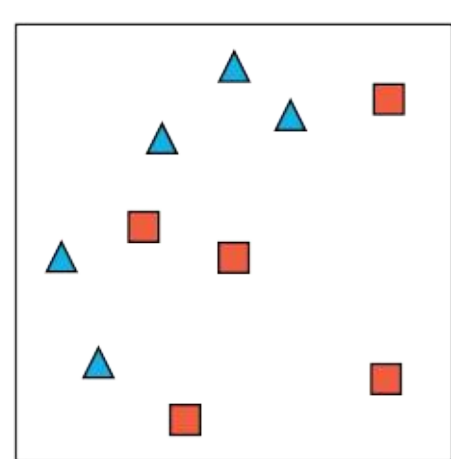
Trong AdaBoost, giống như trong các khu rừng ngẫu nhiên, mỗi người học yếu là một cây quyết định có chiều sâu

1. Không giống như các khu rừng ngẫu nhiên, mỗi người học yếu được đào tạo trên toàn bộ tập dữ liệu chứ không phải trên một phần của tập dữ liệu đó. Lưu ý duy nhất là sau khi đào tạo mỗi học viên yếu, chúng tôi sửa đổi tập dữ liệu bằng cách mở rộng các điểm đã được phân loại không chính xác để những học viên yếu trong tương lai chú ý hơn đến những điểm này. Tóm lại, AdaBoost hoạt động như sau:

Mã giả để huấn luyện mô hình AdaBoost

- Huấn luyện người học yếu đầu tiên trên tập dữ liệu đầu tiên. Lặp lại bước sau cho mỗi
- học viên yếu mới: Sau khi đào tạo một học viên yếu, các điểm được sửa đổi như sau: Các điểm được phân loại không chính xác sẽ được phóng to. Huấn luyện một người học yếu mới về tập dữ liệu đã sửa đổi này.

Trong phần này, chúng tôi phát triển mã giả này chi tiết hơn qua một ví dụ. Tập dữ liệu chúng tôi sử dụng có hai lớp (hình tam giác và hình vuông) và được vẽ trong hình 12.9.



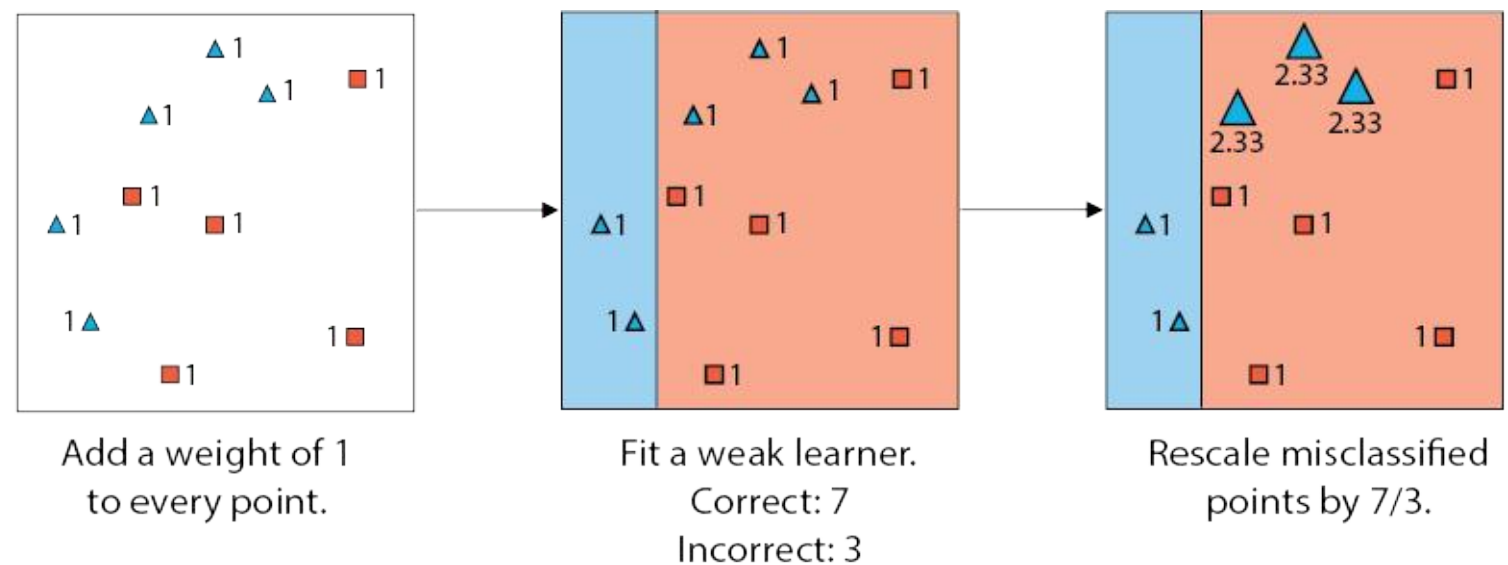
Hình 12.9 Tập dữ liệu mà chúng tôi sẽ phân loại bằng AdaBoost. Nó có hai nhãn được biểu thị bằng hình tam giác và hình vuông.

Bức tranh toàn cảnh về AdaBoost: Bồi dưỡng người học yếu

Trong hai phần phụ tiếp theo, chúng ta sẽ thấy cách xây dựng mô hình AdaBoost để phù hợp với tập dữ liệu được hiển thị trong hình 12.9. Đầu tiên, chúng tôi xây dựng những người học yếu, sau đó chúng tôi sẽ kết hợp thành một người học mạnh.

Bước đầu tiên là gán cho mỗi điểm một trọng số là 1, như minh họa bên trái của hình 12.10. Tiếp theo, chúng tôi xây dựng một trình học yếu trên tập dữ liệu này. Hãy nhớ rằng kẻ yếu

Người học là cây quyết định có độ sâu 1. Cây quyết định có độ sâu 1 tương ứng với đường ngang hoặc đường dọc phân chia các điểm tốt nhất. Một số cây như vậy thực hiện được công việc này, nhưng chúng ta sẽ chọn một cây—đường thẳng đứng minh họa ở giữa hình 12.10—phân loại chính xác hai hình tam giác ở bên trái và năm hình vuông ở bên phải, đồng thời phân loại sai ba hình tam giác ở bên phải. Bước tiếp theo là mở rộng ba điểm được phân loại không chính xác để khiến chúng trở nên quan trọng hơn dưới con mắt của những học sinh yếu kém trong tương lai. Để mở rộng chúng, hãy nhớ rằng mỗi điểm ban đầu có trọng số là 1. Chúng tôi xác định *hệ số thay đổi tỷ lệ* của người học yếu này bằng số điểm được phân loại đúng chia cho số điểm được phân loại sai. Trong trường hợp này, hệ số thay đổi tỷ lệ là $7/3 = 2,33$. Chúng tôi tiến hành định lại tỷ lệ cho mọi điểm bị phân loại sai bằng hệ số thay đổi tỷ lệ này, như được minh họa ở bên phải của hình 12.10.

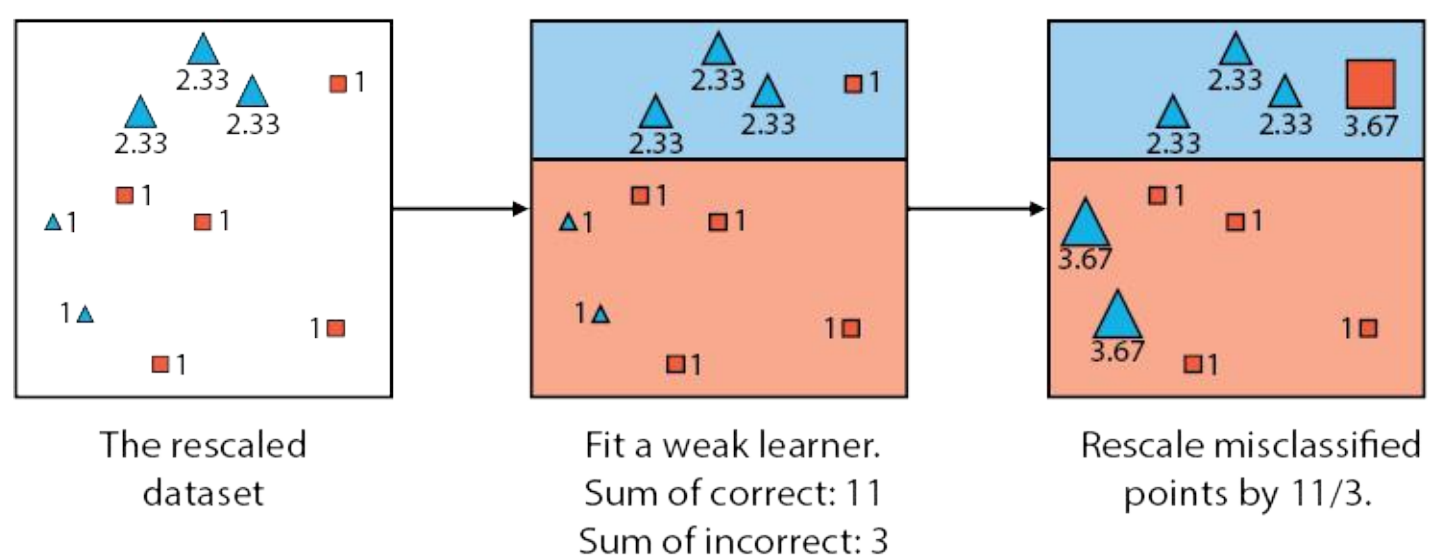


Hình 12.10 Lắp máy học yếu đầu tiên của mô hình AdaBoost. Bên trái: Tập dữ liệu, trong đó mỗi điểm được gán trọng số là 1. Ở giữa: Một học sinh yếu phù hợp nhất với tập dữ liệu này. Phải: Tập dữ liệu đã được thay đổi tỷ lệ, trong đó chúng tôi đã mở rộng các điểm bị phân loại sai theo hệ số thay đổi tỷ lệ là $7/3$.

Bây giờ chúng ta đã xây dựng được bộ học yếu đầu tiên, chúng ta sẽ xây dựng những bộ học tiếp theo theo cách tương tự. Người học yếu thứ hai được minh họa trong hình 12.11. Ở bên trái của hình, chúng ta có tập dữ liệu đã được thay đổi tỷ lệ. Người học yếu thứ hai là người phù hợp nhất với tập dữ liệu này. Chúng ta có ý gì qua điều đó? Vì các điểm có trọng số khác nhau nên chúng tôi muốn học viên yếu có tổng trọng số của các điểm được phân loại chính xác là cao nhất. Người học yếu này là

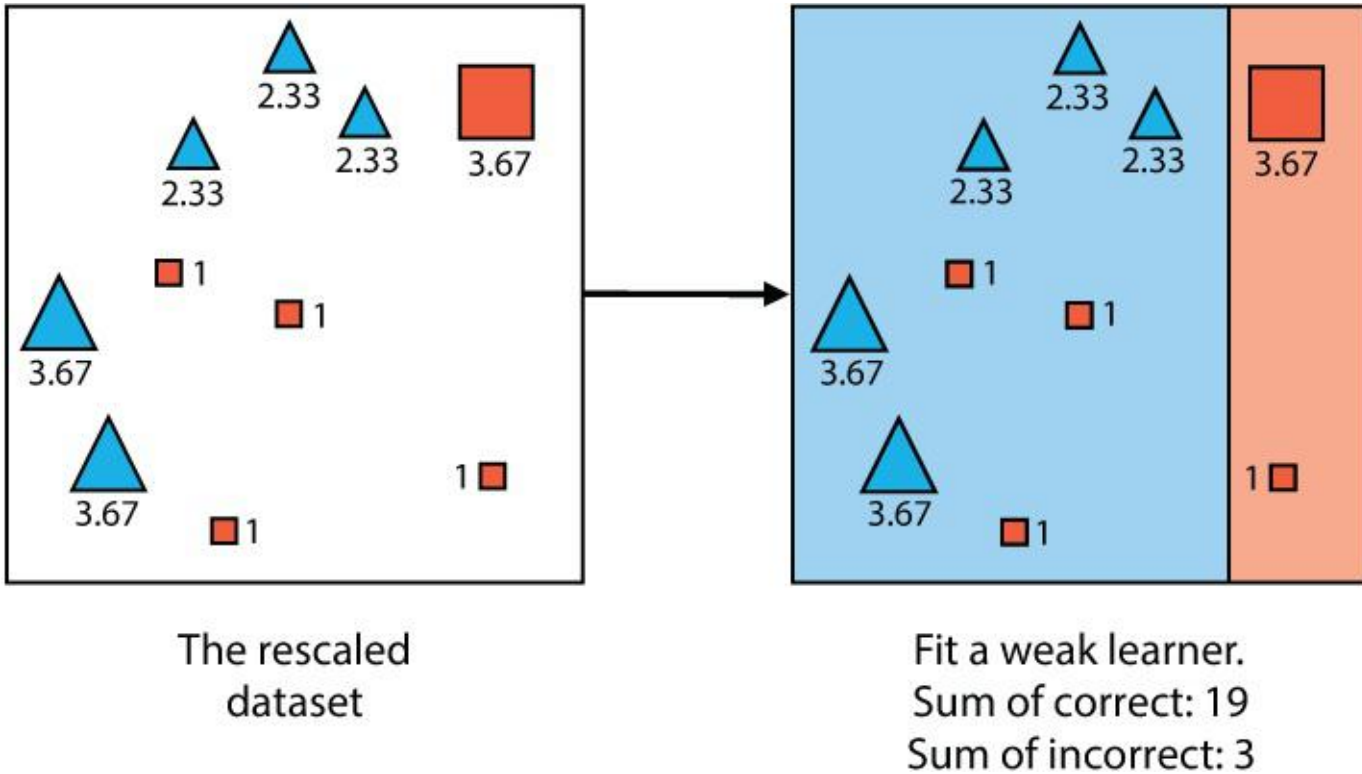
đường ngang ở giữa hình 12.11. Bây giờ chúng ta tiến hành tính hệ số thay đổi tỷ lệ. Chúng ta cần sửa đổi một chút định nghĩa của nó, vì các điểm hiện đã có trọng số. Hệ số thay đổi tỷ lệ là tỷ lệ giữa tổng trọng số của

các điểm được phân loại đúng và tổng trọng số của các điểm được phân loại sai. Số hạng thứ nhất là $2,33 + 2,33 + 2,33 + 1 + 1 + 1 + 1 = 11$, và số hạng thứ hai là $1 + 1 + 1 = 3$. Như vậy, hệ số thay đổi tỷ lệ là $11/3 = 3,67$. Chúng ta tiến hành nhân trọng số của ba điểm bị phân loại sai với hệ số $3,67$ này, như minh họa bên phải của hình 12.11.



Hình 12.11 Lắp máy học yếu thứ hai của mô hình AdaBoost. Bên trái: Tập dữ liệu được thay đổi tỷ lệ từ hình 12.10. Giữa: Trình học yếu phù hợp nhất với tập dữ liệu đã được thay đổi tỷ lệ—điều này có nghĩa là trình học yếu có tổng trọng số của các điểm được phân loại chính xác là lớn nhất. Phải: Tập dữ liệu được thay đổi tỷ lệ mới, trong đó chúng tôi đã mở rộng các điểm bị phân loại sai theo hệ số thay đổi tỷ lệ là $11/3$.

Chúng tôi tiếp tục theo cách này cho đến khi xây dựng được số lượng học viên yếu như mong muốn. Đối với ví dụ này, chúng tôi chỉ xây dựng ba người học yếu. Người học yếu thứ ba là một đường thẳng đứng, được minh họa trong hình 12.12.



Hình 12.12 Lắp máy học yếu thứ ba của mô hình AdaBoost. Bên trái: Tập dữ liệu được thay đổi tỷ lệ từ hình 12.11. Đúng: Một người học yếu phù hợp nhất với tập dữ liệu đã được thay đổi tỷ lệ này.

Đây là cách chúng tôi xây dựng những người học yếu. Bây giờ, chúng ta cần kết hợp chúng thành một người học mạnh mẽ. Điều này tương tự như những gì chúng tôi đã làm với các khu rừng ngẫu nhiên, nhưng sử dụng nhiều phép toán hơn một chút, như được trình bày trong phần tiếp theo.

Kết hợp người học yếu thành người học giỏi

Bây giờ chúng ta đã xây dựng được những người học yếu, trong phần này, chúng ta tìm hiểu cách hiệu quả để kết hợp chúng thành một người học mạnh. Ý tưởng là để những người phân loại bỏ phiếu, giống như họ đã làm trong bộ phân loại rừng ngẫu nhiên, nhưng lần này, những người học giỏi sẽ có nhiều tiếng nói hơn những người học kém. Trong trường hợp một trình phân loại *thực sự* kém thì phiếu bầu của nó thực sự sẽ là âm.

Để hiểu điều này, hãy tưởng tượng chúng ta có ba người bạn: Teresa trung thực, Umberto khó đoán và Lenny nói dối. Teresa trung thực hầu như luôn nói sự thật, Lenny nói dối hầu như luôn nói dối, và Umberto không thể đoán trước hầu như nói sự thật trong một nửa thời gian và nửa còn lại nói dối. Trong ba người bạn này, ai là người ít hữu ích nhất?

Theo cách nhìn của tôi, Teresa thật thà rất đáng tin cậy, bởi vì cô ấy hầu như luôn nói sự thật nên chúng ta có thể tin tưởng cô ấy. Trong số hai cái còn lại, tôi thích Lying Lenny hơn. Nếu anh ấy

hầu như luôn nói dối khi chúng tôi hỏi anh ấy một câu hỏi có hoặc không, chúng tôi chỉ đơn giản coi đó là sự thật ngược lại với những gì anh ấy nói với chúng tôi và chúng tôi sẽ luôn đúng! Mặt khác, Umberto không thể đoán trước sẽ chẳng có ích gì nếu chúng ta không biết liệu anh ta đang nói thật hay nói dối. Trong trường hợp đó, nếu chúng ta chấm điểm cho những gì mỗi người bạn nói, tôi sẽ cho Teresa trung thực điểm tích cực cao, Lenny nói dối là điểm tiêu cực cao và Umberto khó đoán là điểm 0.

Bây giờ hãy tưởng tượng rằng ba người bạn của chúng ta là những người học yếu được đào tạo trong một tập dữ liệu có hai lớp. Truthful Teresa là máy phân loại có độ chính xác rất cao, Lying Lenny là máy phân loại có độ chính xác rất thấp và Umberto Không thể đoán trước là máy phân loại có độ chính xác gần 50%. Chúng tôi muốn xây dựng một người học giỏi trong đó dự đoán được đưa ra bằng phiếu bầu có trọng số từ ba người học yếu. Vì vậy, đối với mỗi người học yếu, chúng tôi chấm điểm và đó là số phiếu bầu của người học sẽ được tính trong lần bình chọn cuối cùng. Hơn nữa, chúng tôi muốn chỉ định những điểm số này theo cách sau:

- Công cụ phân loại Truthful Teresa nhận được điểm tích cực cao. Trình phân
- loại Umberto không thể đoán trước có điểm gần bằng 0. Trình phân loại Lying
- Lenny nhận được điểm âm cao.

Nói cách khác, điểm của người học yếu là một con số có các tính chất sau:

1. Là dương khi độ chính xác của người học lớn hơn 0,5
2. Là 0 khi độ chính xác của mô hình là 0,5
3. Âm khi độ chính xác của người học nhỏ hơn 0,5
4. Là số dương lớn khi độ chính xác của người học gần bằng 1
5. Là số âm lớn khi độ chính xác của người học gần bằng 0

Để đạt được điểm cao cho học sinh yếu thỏa mãn các đặc tính từ 1–5 ở trên, chúng tôi sử dụng một khái niệm phổ biến trong xác suất được gọi là *logit* hoặc *log-odds* mà chúng ta sẽ thảo luận tiếp theo.

Xác suất, tỷ lệ cược và tỷ lệ cược LOG

Bạn có thể đã thấy trong cờ bạc rằng xác suất không bao giờ được đề cập đến, nhưng họ luôn nói về tỷ lệ cược. Những tỷ lệ cược này là gì? Chúng tương tự như xác suất theo nghĩa sau: nếu chúng ta chạy một thử nghiệm nhiều lần và ghi lại số lần một kết quả cụ thể xảy ra thì xác suất của kết quả này là số lần nó xảy ra chia cho tổng số lần chúng ta chạy thử nghiệm. Tỷ lệ xảy ra kết quả này là số lần nó xảy ra chia cho

số lần nó đã không xảy ra.

Ví dụ: xác suất nhận được 1 khi chúng ta tung xúc xắc là $1/6$ nhưng tỷ lệ cược là $1/5$. Nếu một con ngựa cụ thể thắng 3 trong 4 cuộc đua thì xác suất con ngựa đó thắng một cuộc đua là $3/4$ và tỷ lệ cược là $3/1 = 3$. Công thức tính tỷ lệ cược là

đơn giản: nếu xác suất của một sự kiện là x thì tỷ lệ cược là $\frac{x}{1-x}$. Ví dụ, trong ví dụ về xúc xắc, xác suất là $1/6$ và tỷ lệ cược là

$$\frac{\frac{1}{6}}{1 - \frac{1}{6}} = \frac{1}{5}.$$

Lưu ý rằng vì xác suất là một số nằm trong khoảng từ 0 đến 1 nên tỷ lệ cược là một số nằm trong khoảng từ 0 đến ∞ .

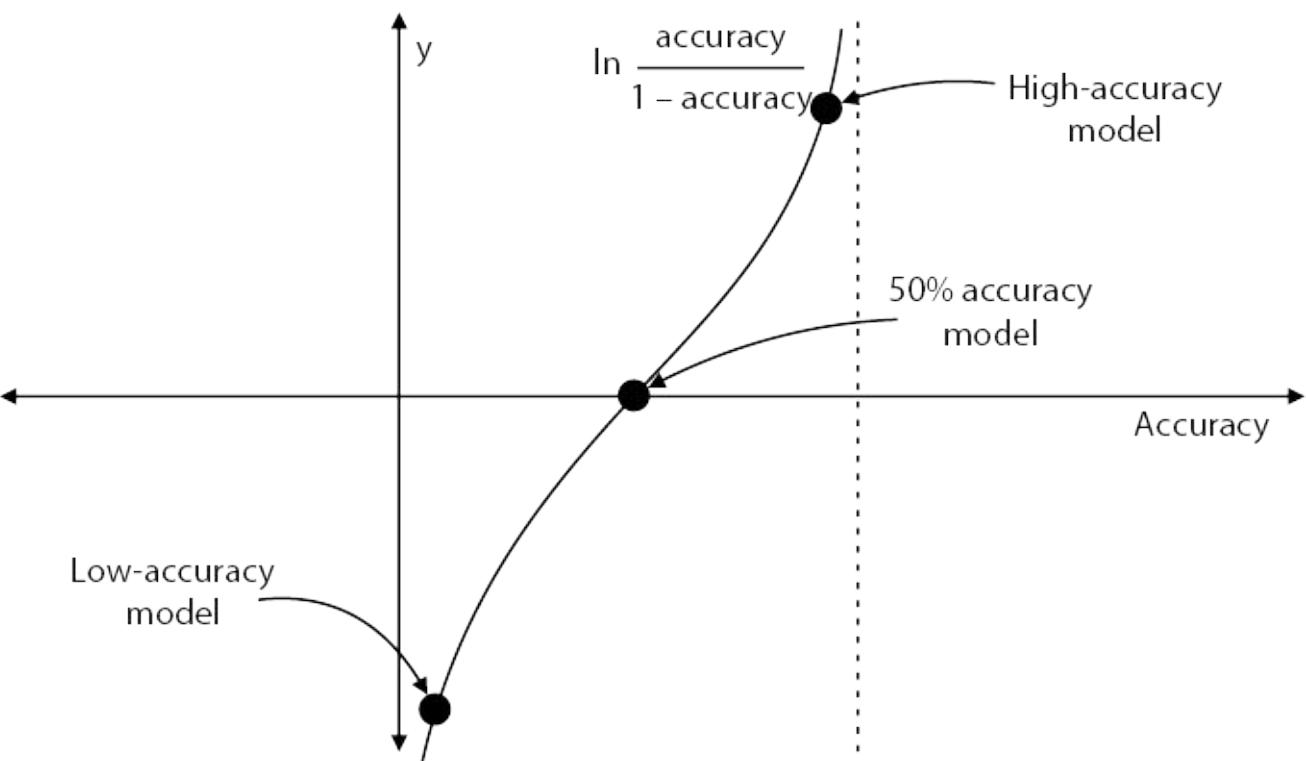
Bây giờ hãy quay lại mục tiêu ban đầu của chúng ta. Chúng tôi đang tìm kiếm một hàm thỏa mãn các tính chất 1–5 ở trên. Hàm tỷ lệ cược gần giống nhưng không hoàn toàn ở đó vì nó chỉ đưa ra các giá trị dương. Cách biến tỷ lệ cược thành một hàm thỏa mãn các tính chất 1–5 ở trên là lấy logarit. Vì vậy, chúng ta thu được tỷ lệ logarit, còn được gọi là logit, được xác định như sau:

$$\text{log-odds}(x) = \ln \frac{x}{1-x}$$

Hình 12.13 thể hiện đồ thị của hàm log-odds. Lưu ý rằng hàm này thỏa mãn các thuộc tính 1–5.

$$y = \ln \left(\frac{x}{1-x} \right)$$

Vì vậy, tất cả những gì chúng ta cần làm là sử dụng hàm log-odds để tính điểm của từng học sinh yếu. Chúng tôi áp dụng hàm tỷ lệ log này cho độ chính xác. Bảng 12.2 chứa một số giá trị về độ chính xác của người học yếu và tỷ lệ log của độ chính xác này. Lưu ý rằng, như mong muốn, các mô hình có độ chính xác cao có điểm dương cao, mô hình có độ chính xác thấp có điểm âm cao và mô hình có độ chính xác gần 0,5 có điểm gần 0.



Hình 12.13 Đường cong thể hiện đồ thị của hàm log-odds về độ chính xác.Lưu ý rằng đối với các giá trị chính xác nhỏ, tỷ lệ logarit là số âm rất lớn và đối với các giá trị chính xác cao hơn, đó là số dương rất lớn.Khi độ chính xác là 50% (hoặc 0,5), tỷ lệ log chính xác bằng 0.

Bảng 12.2 Một số giá trị về độ chính xác của bộ phân loại yếu, với điểm số tương ứng, được tính bằng log-odds. Lưu ý rằng các mô hình có độ chính xác rất thấp sẽ nhận được điểm âm lớn, các giá trị có độ chính xác rất cao sẽ nhận được điểm dương lớn và các giá trị có độ chính xác gần 0,5 sẽ nhận được điểm gần bằng 0.

Độ chính xác	Tỷ lệ log (điểm của học sinh yếu)
0.01	-4.595
0.1	-2.197
0.2	-1.386
0.5	0
0.8	1.386
0.9	2.197
0.99	4.595

TỔNG HỢP CÁC PHÂN LOẠI

Bây giờ chúng ta đã giải quyết được tỷ lệ log là cách để xác định điểm cho tất cả những người học yếu, chúng ta có thể tiến hành tham gia cùng họ để xây dựng người học giỏi.Nhớ lại điều đó

độ chính xác của một học sinh yếu là tổng điểm của các điểm được phân loại đúng chia cho tổng điểm của tất cả các điểm, như trong hình 12.10–12.12.

- Học yếu 1:

- Accuracy: $\frac{7}{10}$

- Score: $\ln\left(\frac{7}{3}\right) = 0.847$

- Học yếu 2:

- Accuracy: $\frac{11}{14}$

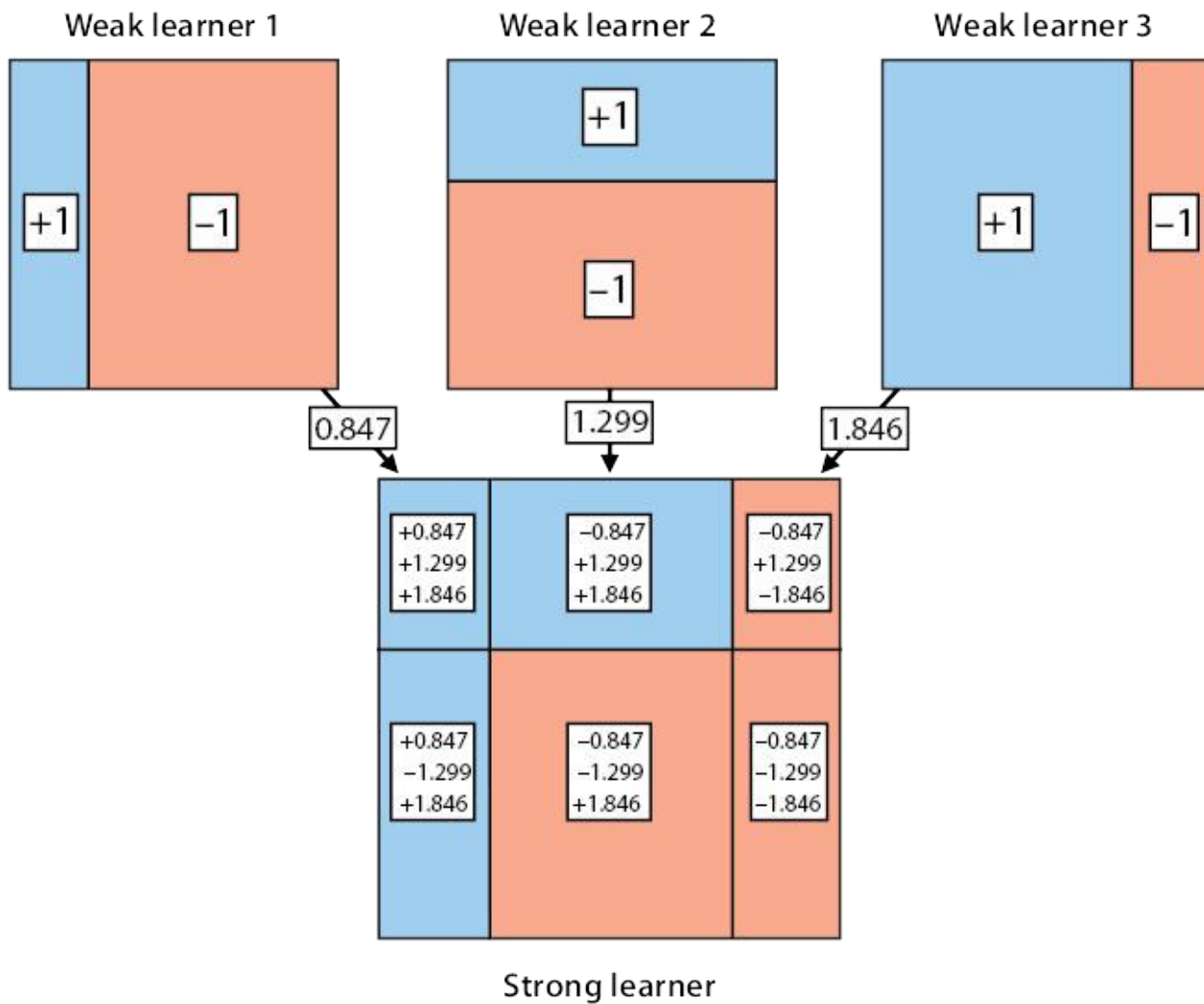
- Score: $\ln\left(\frac{11}{3}\right) = 1.299$

- Học yếu 3:

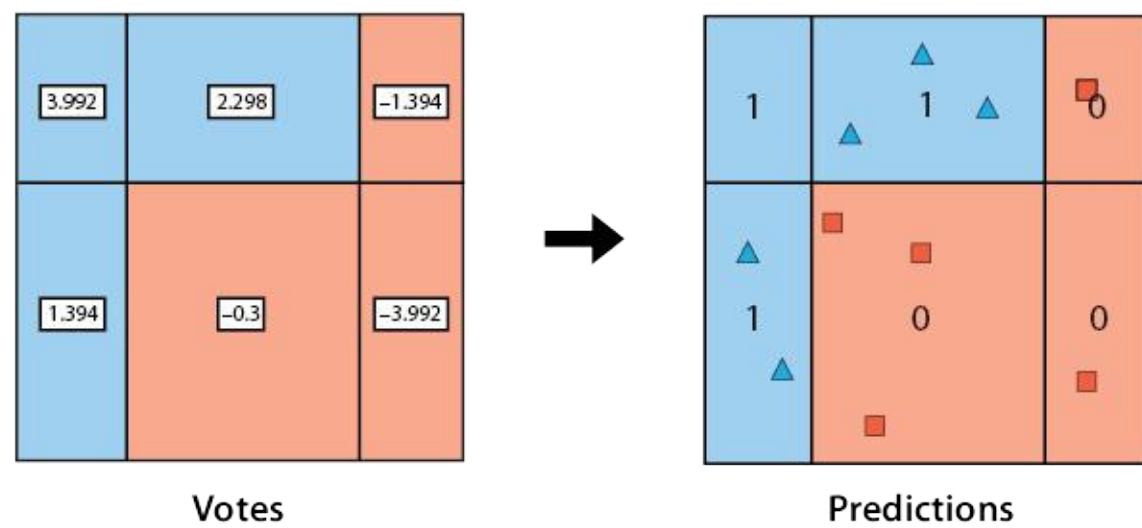
- Accuracy: $\frac{19}{22}$

- Score: $\ln\left(\frac{19}{3}\right) = 1.846$

Dự đoán mà người học mạnh đưa ra có được bằng phiếu bầu có trọng số của các bộ phân loại yếu, trong đó phiếu bầu của mỗi bộ phân loại là điểm của nó. Một cách đơn giản để thấy điều này là thay đổi dự đoán của học sinh yếu từ 0 và 1 thành -1 và 1 , nhân mỗi dự đoán với điểm của học sinh yếu rồi cộng chúng lại. Nếu dự đoán kết quả lớn hơn hoặc bằng 0 thì máy học mạnh sẽ dự đoán số 1 và nếu nó âm thì dự đoán số 0. Quá trình bỏ phiếu được minh họa trong hình 12.14 và dự đoán trong hình 12.15. Cũng lưu ý trong hình 12.15 rằng bộ phân loại kết quả đã phân loại chính xác mọi điểm trong tập dữ liệu.



Hình 12.14 Cách kết hợp người học yếu thành người học mạnh trong mô hình AdaBoost. Chúng tôi chấm điểm từng người học yếu bằng cách sử dụng tỷ lệ cược log và yêu cầu họ bỏ phiếu dựa trên điểm số của họ (điểm càng lớn thì người học cụ thể đó càng có nhiều quyền biểu quyết). Mỗi khu vực trong sơ đồ phía dưới có tổng điểm của những học sinh yếu. Lưu ý rằng để đơn giản hóa phép tính của chúng ta, dự đoán của những học sinh yếu là +1 và -1, thay vì 1 và 0.

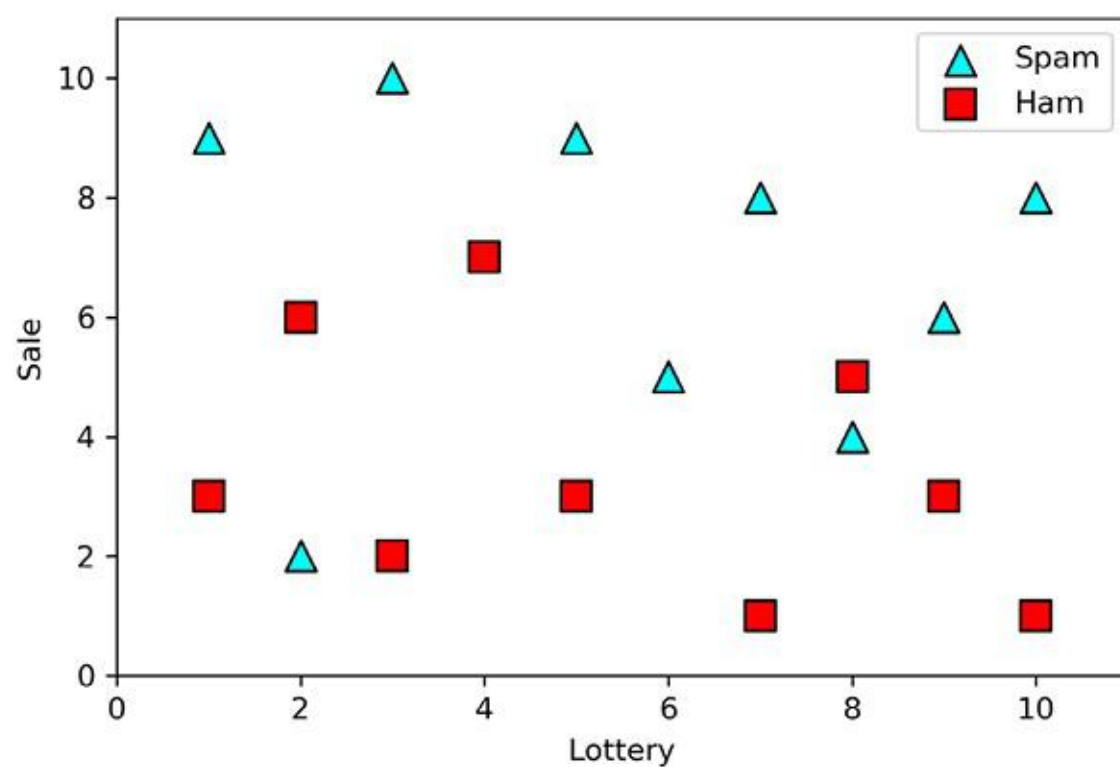


Hình 12.15 Cách lấy dự đoán cho mô hình AdaBoost. Sau khi cộng điểm của những học sinh yếu (như trong hình 12.14), chúng tôi chỉ định dự đoán là 1 nếu tổng điểm lớn hơn hoặc bằng 0 và dự đoán là 0 nếu ngược lại.

Mã hóa AdaBoost trong Scikit-Learn

Trong phần này, chúng ta xem cách sử dụng Scikit-Learn để huấn luyện mô hình AdaBoost. Chúng tôi huấn luyện nó trên cùng một tập dữ liệu email spam mà chúng tôi đã sử dụng trong phần “Điều chỉnh một nhóm ngẫu nhiên theo cách thủ công” và được vẽ trong hình 12.16. Chúng ta tiếp tục sử dụng sổ ghi chép sau đây từ các phần trước:

- **Sổ tay:** `Random_forests_and_AdaBoost.ipynb`
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemb



Hình 12.16 Trong tập dữ liệu này, chúng tôi huấn luyện bộ phân loại AdaBoost bằng Scikit-Learn. Đây chính là tập dữ liệu thư rác từ phần “Bagging”, trong đó các đặc điểm là số lần xuất hiện của các từ “xổ số” và “thư rác” và các email spam được biểu thị bằng hình tam giác và email ham được biểu thị bằng hình vuông.

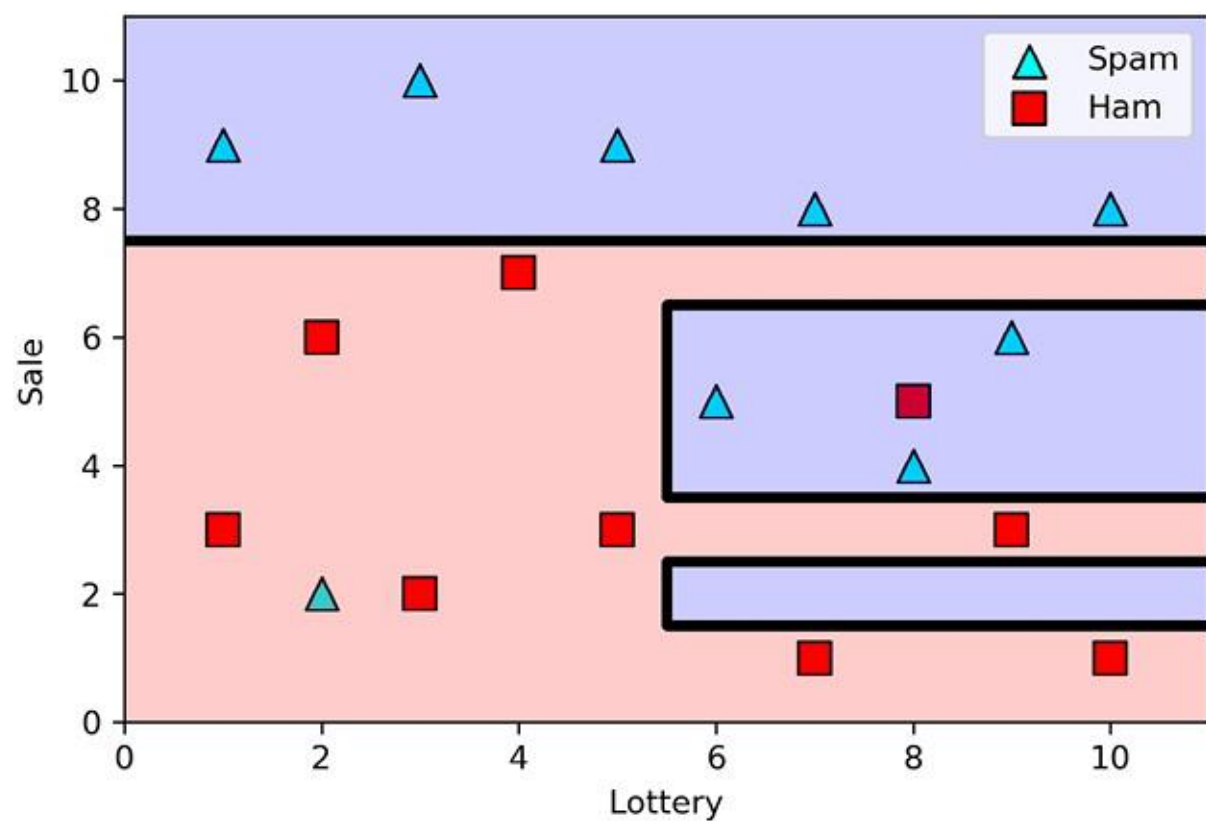
Tập dữ liệu nằm trong hai Pandas DataFrames được gọi là `features` và `labels`. Quá trình đào tạo được thực hiện bằng gói trong Scikit-Learn. Chúng tôi

xác định rằng mô hình này sẽ sử dụng sáu học sinh yếu với `n_estimators` siêu tham số, như hiển thị tiếp theo:

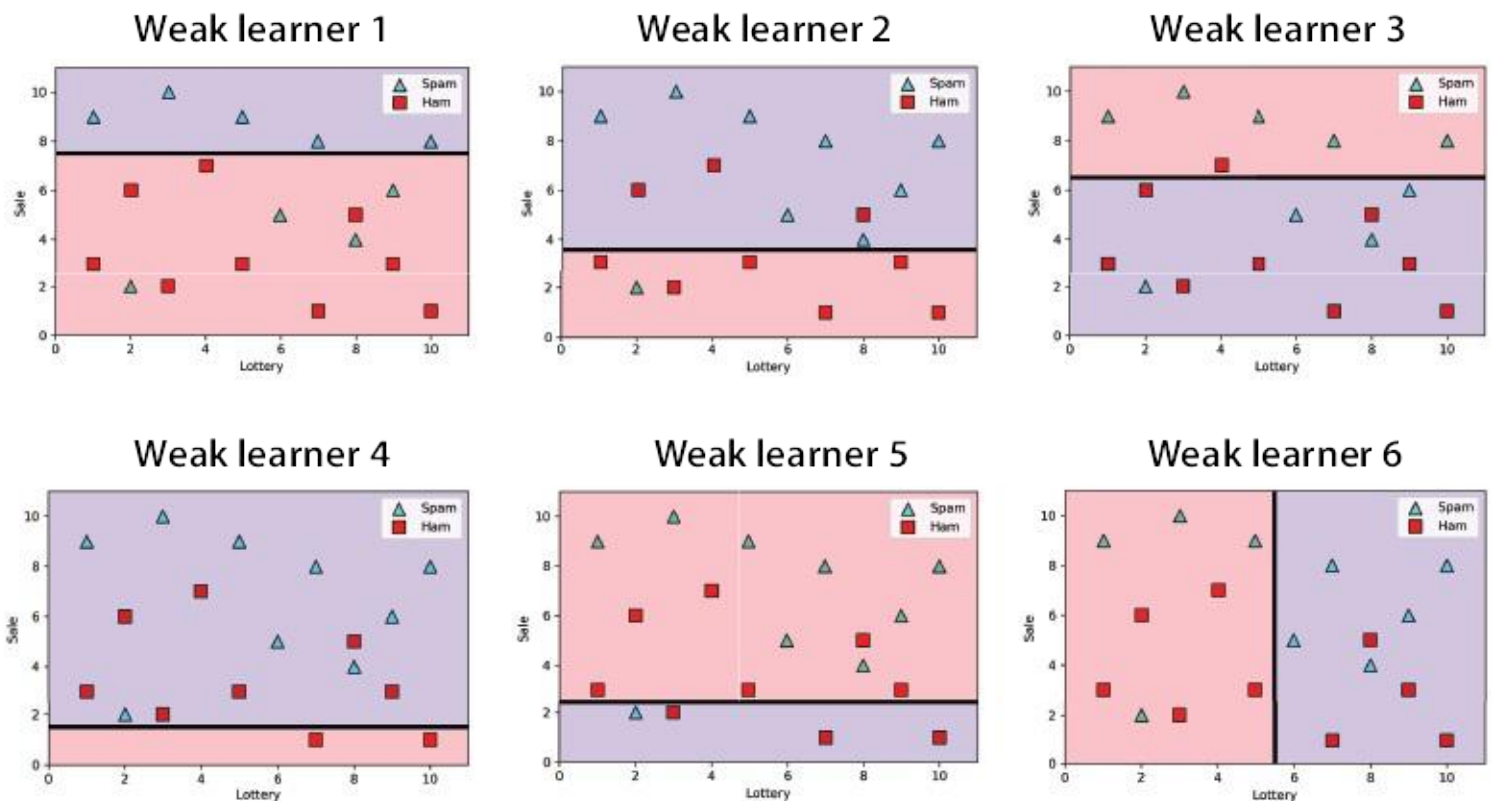
```
from sklearn.ensemble import AdaBoostClassifier
adaboost_classifier = AdaBoostClassifier(n_estimators=6)
adaboost_classifier.fit(features, labels)
adaboost_classifier.score(features, labels)
```

Ranh giới của mô hình kết quả được vẽ trong hình 12.17.

Chúng ta có thể đi xa hơn một chút và khám phá sáu học sinh yếu kém và điểm số của họ (xem số tay để biết mã). Ranh giới của họ được vẽ trong hình 12.18, và như được thấy rõ trong cuốn sổ, điểm của tất cả những học sinh yếu đều là 1.



Hình 12.17 Kết quả phân loại AdaBoost trên tập dữ liệu thư rác trong hình 12.16. Lưu ý rằng trình phân loại thực hiện rất tốt việc khớp tập dữ liệu và không khớp quá nhiều.



Hình 12.18 Sáu người học yếu trong mô hình AdaBoost của chúng tôi. Mỗi một trong số chúng là một cây quyết định có độ sâu 1. Chúng kết hợp thành Strong Learner trong hình 12.17.

Lưu ý rằng người học giỏi trong hình 12.17 đạt được bằng cách cho mỗi người học yếu trong hình 12.18 điểm 1 và để họ bình chọn.

Tăng cường độ dốc: Sử dụng cây quyết định để xây dựng những người học giỏi

Trong phần này, chúng ta thảo luận về việc tăng cường độ dốc, một trong những mô hình học máy phổ biến và thành công nhất hiện nay. Tăng cường độ dốc tương tự như AdaBoost, trong đó những người học yếu là cây quyết định và mục tiêu của mỗi người học yếu là học hỏi từ những sai lầm của những người trước đó. Một điểm khác biệt giữa tăng cường độ dốc và AdaBoost là trong việc tăng cường độ dốc, chúng tôi cho phép cây quyết định có độ sâu lớn hơn 1. Tăng cường độ dốc có thể được sử dụng để hồi quy và phân loại, nhưng để rõ ràng, chúng tôi sử dụng ví dụ hồi quy. Để sử dụng nó cho việc phân loại, chúng ta cần thực hiện một số điều chỉnh nhỏ. Để tìm hiểu thêm về điều này, hãy xem các liên kết tới video và tài liệu đọc trong phụ lục C. Mã cho phần này như sau:

- **Sổ tay:** `gradient_boosting_and_XGBoost.ipynb`
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemb

Ví dụ chúng tôi sử dụng giống với ví dụ trong phần “Cây quyết định hồi quy” ở chương 9, trong đó chúng tôi đã nghiên cứu mức độ tương tác của một số người dùng nhất định với một ứng dụng. Đặc điểm này là độ tuổi của người dùng và nhãn là số ngày người dùng tương tác với ứng dụng (bảng 12.3). Sơ đồ của tập dữ liệu được thể hiện trong hình 12.19.

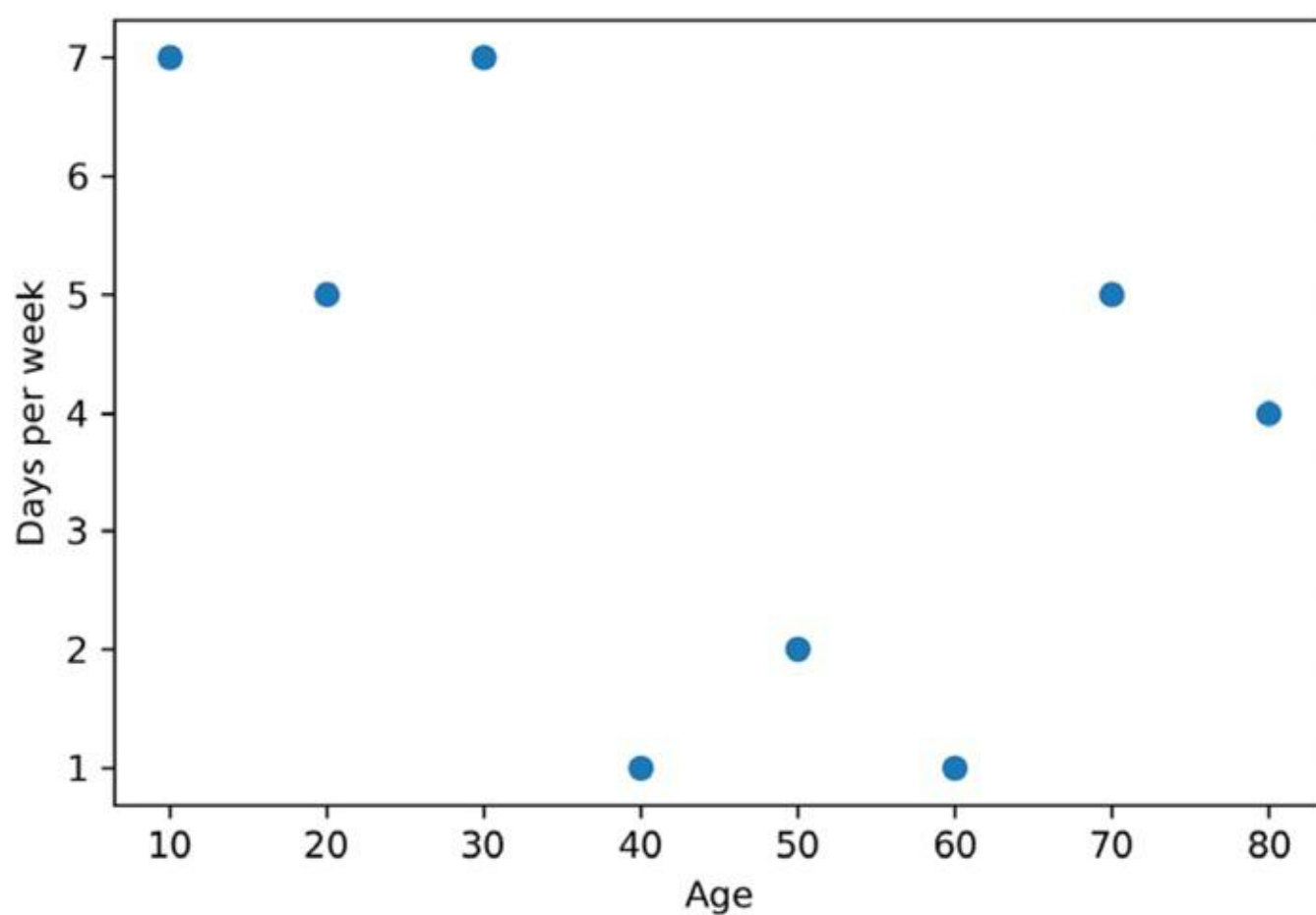


Figure 12.19 The plot of the user engagement dataset from table 12.3. The horizontal axis represents the age of the users, and the vertical axis represents the days per week that the user uses our app.

Table 12.3 A small dataset with eight users, their age, and their engagement with our app. The engagement is measured in the number of days when they opened the app in one week. We'll fit this dataset using gradient boosting.

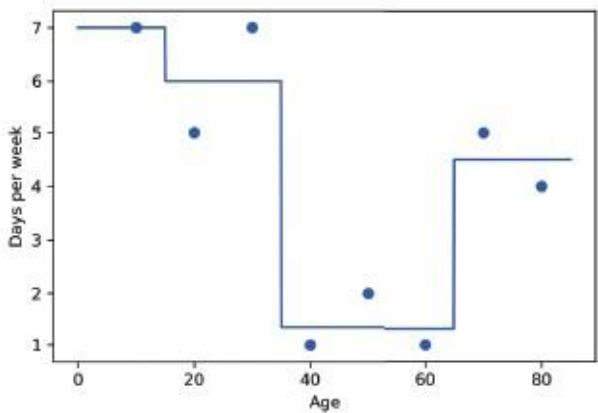
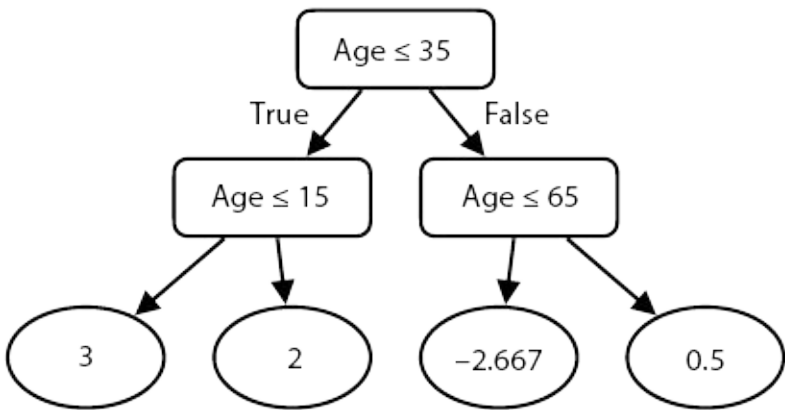
Feature (age)	Label (engagement)
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

The idea of gradient boosting is that we'll create a sequence of trees that fit this dataset. The two hyperparameters that we'll use for now are the number of trees,

mà chúng tôi đặt thành 5 và tốc độ học tập mà chúng tôi đặt thành 0,8.Học yếu đầu tiên rất đơn giản: đó là cây quyết định có độ sâu 0 phù hợp nhất với tập dữ liệu.Cây quyết định có độ sâu 0 chỉ đơn giản là một nút gán cùng một nhãn cho mỗi điểm trong tập dữ liệu.Bởi vì hàm lỗi mà chúng ta đang giảm thiểu là sai số bình phương trung bình nên giá trị tối ưu cho dự đoán này là giá trị trung bình của các nhãn. Giá trị trung bình của các nhãn của tập dữ liệu này là 4, vì vậy bộ học yếu đầu tiên của chúng ta là một nút gán dự đoán là 4 cho mọi điểm.

Bước tiếp theo là tính toán phần dư, đó là sự khác biệt giữa nhãn và dự đoán được đưa ra bởi người học yếu đầu tiên này và điều chỉnh cây quyết định mới cho các phần dư này.Như bạn có thể thấy, điều này đang làm là huấn luyện cây quyết định để lấp đầy những khoảng trống mà cây đầu tiên để lại.Các nhãn, dự đoán và phần dư được trình bày trong bảng 12.4.

Người học yếu thứ hai là cây phù hợp với các phần dư này.Cây có thể sâu như chúng ta muốn, nhưng với ví dụ này, chúng ta sẽ đảm bảo tất cả những người học yếu đều có độ sâu tối đa là 2. Cây này được hiển thị trong hình 12.20 (cùng với ranh giới của nó) và các dự đoán của nó nằm ở cột ngoài cùng bên phải của bảng 12.4.Cây này đã được lấy bằng Scikit-Learn;xem sổ ghi chép để biết quy trình.



Hình 12.20 Bộ học yếu thứ hai trong mô hình tăng cường độ dốc.Học viên này là một cây quyết định có độ sâu 2 như hình bên trái.Những dự đoán của người học yếu này được thể hiện ở cột truyện bên phải.

Bảng 12.4 Dự đoán từ người học yếu thứ nhất là trung bình của các nhãn. Người học yếu thứ hai được đào tạo để phù hợp với phần dư của người học yếu thứ nhất.

Feature (age)	Label (engagement)	Prediction from weak learner 1	Residual	Prediction from weak learner 2
10	7	4	3	3
20	5	4	2	2
30	7	4	3	2
40	1	4	-3	-2.667
50	2	4	-2	-2.667
60	1	4	-3	-2.667
70	5	4	1	0.5
80	4	4	0	0.5

The idea is to continue in this fashion, calculating new residuals and training a new weak learner to fit these residuals. However, there’s a small caveat—to calculate the prediction from the first two weak learners, we first multiply the prediction of the second weak learner by the learning rate. Recall that the learning rate we’re using is 0.8. Thus, the combined prediction of the first two weak learners is the prediction of the first one (4) plus 0.8 times the prediction of the second one. We do this because we don’t want to overfit by fitting our training data too well. Our goal is to mimic the gradient descent algorithm, by slowly walking closer and closer to the solution, and this is what we achieve by multiplying the prediction by the learning rate. The new residuals are the original labels minus the combined predictions of the first two weak learners. These are calculated in table 12.5.

Table 12.5 The labels, the predictions from the first two weak learners, and the residual. The prediction from the first weak learner is the average of the labels. The prediction from the second weak learner is shown in figure 12.20. The combined prediction is equal to the prediction of the first weak learner plus the learning rate (0.8) times the prediction of the second weak learner. The residual is the difference between the label and the combined prediction from the first two weak learners.

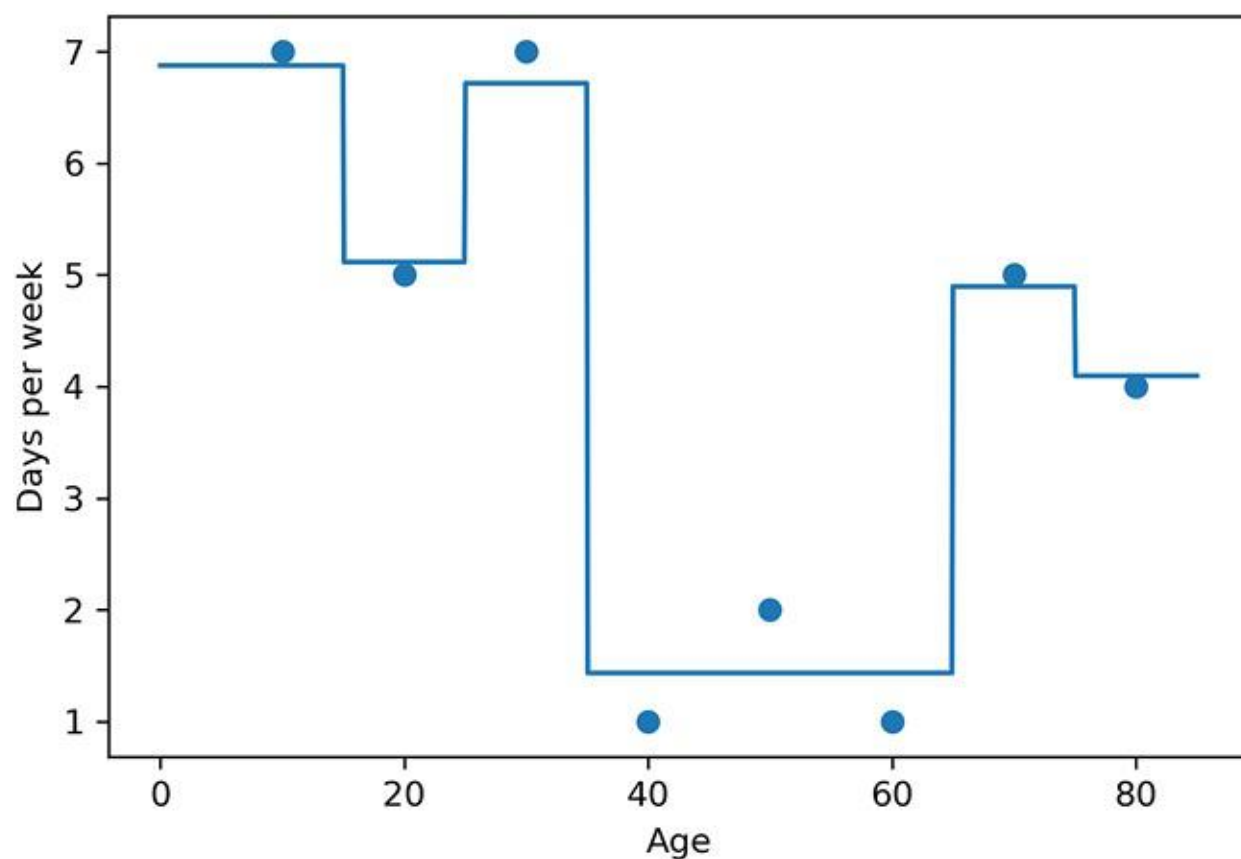
Nhãn	Dự đoán từ người học yếu 1	Dự đoán từ người học yếu 2	Dự đoán từ người học yếu gấp 2 lần tốc độ học	Dự đoán của học sinh yếu 1 và 2	Dư lượng
7	4	3	2.4	6.4	0.6
5	4	2	1.6	5.6	-0.6
7	4	2	1.6	5.6	1.4
1	4	-2.667	-2.13	1.87	-0.87
2	4	-2.667	-2.13	1.87	0.13
1	4	-2.667	-2.13	1.87	-0.87
5	4	0.5	0.4	4.4	0.6
4	4	0.5	0.4	4.4	-0.4

Bây giờ chúng ta có thể tiến hành điều chỉnh một học sinh yếu mới trên các phần dư mới và tính toán dự đoán kết hợp của hai học sinh yếu đầu tiên.Chúng tôi đạt được điều này bằng cách cộng dự đoán cho người học yếu thứ nhất và 0,8 (tỷ lệ học) nhân với tổng dự đoán của người học yếu thứ hai và thứ ba.Chúng tôi lặp lại quá trình này cho mọi người học yếu mà chúng tôi muốn xây dựng.Thay vì làm thủ công, chúng ta có thể sử dụng gói trong Scikit-Learn (mã có trong sổ tay).Một số dòng mã tiếp theo cho thấy cách điều chỉnh mô hình và đưa ra dự đoán.Lưu ý rằng chúng tôi đã đặt độ sâu của cây tối đa là 2, số lượng cây là 5 và tốc độ học là 0,8.Các siêu tham số được sử dụng cho việc này là , , và .Cũng lưu ý rằng nếu

`max_depth` `n_estimators` `learning_rate`
chúng ta muốn có năm cây, chúng ta phải đặt siêu tham số thành bốn, vì cây đầu tiên không được tính.

```
from sklearn.ensemble import GradientBoostingRegressor
gradient_boosting_regressor = GradientBoostingRegressor(max_depth=2, n_estimators=4, learning_rate=0.8)
gradient_boosting_regressor.fit(features, labels)
gradient_boosting_regressor.predict(features)
```

Biểu đồ cho kết quả là người học giỏi được thể hiện trong hình 12.21.Lưu ý rằng nó thực hiện tốt công việc dự đoán các giá trị.

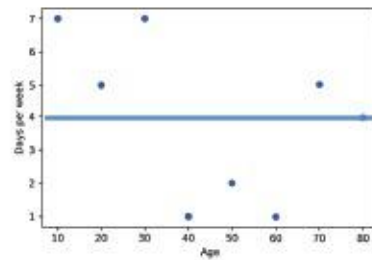


Hình 12.21 Sơ đồ dự đoán của người học mạnh trong bộ hồi quy tăng cường độ dốc của chúng tôi. Lưu ý rằng mô hình khá phù hợp với tập dữ liệu.

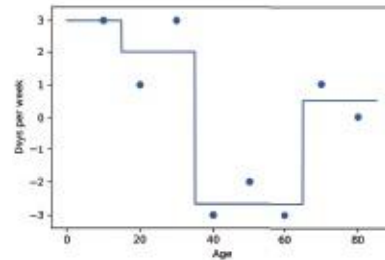
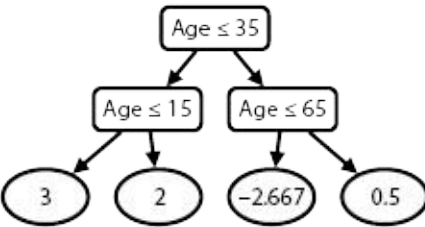
Tuy nhiên, chúng ta có thể đi xa hơn một chút và thực sự vẽ sơ đồ về 5 học sinh yếu mà chúng ta thu được. Chi tiết về việc này có trong sổ tay và năm học sinh yếu được thể hiện trong hình 12.22. Lưu ý rằng dự đoán của những người học yếu cuối cùng nhỏ hơn nhiều so với dự đoán của những người học yếu đầu tiên, bởi vì mỗi người học yếu đang dự đoán lỗi của những người học trước và những lỗi này ngày càng nhỏ hơn ở mỗi bước.

Weak learner 1

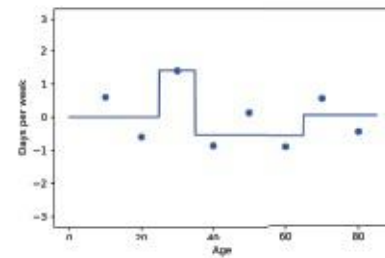
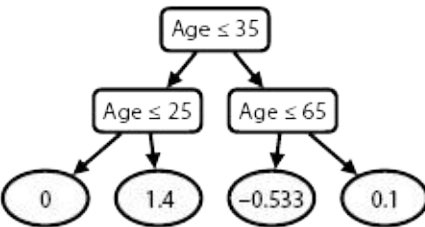
4



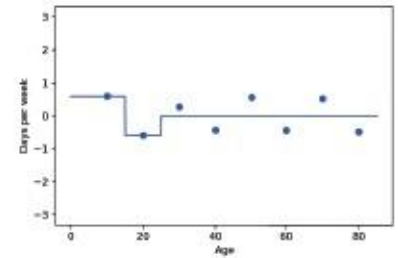
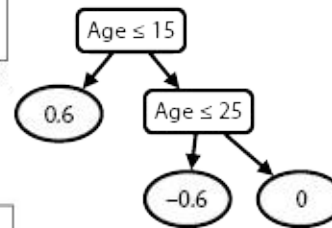
Weak learner 2



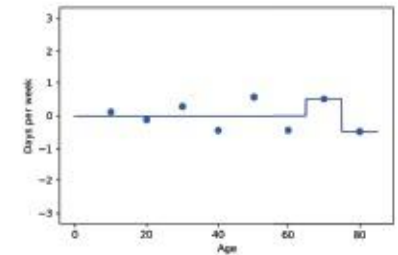
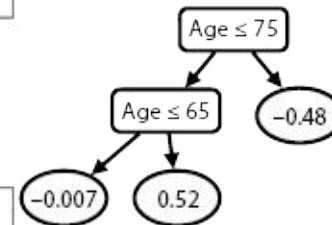
Weak learner 3



Weak learner 4



Weak learner 5



Hình 12.22 Năm người học yếu trong mô hình tăng cường độ dốc. Cây đầu tiên là cây quyết định có độ sâu 0 luôn dự đoán mức trung bình của các nhãn. Mỗi người học yếu liên tiếp là một cây quyết định có độ sâu tối đa là 2, phù hợp với phần dư từ dự đoán được đưa ra bởi những người học yếu trước đó. Lưu ý rằng dự đoán của người học yếu sẽ nhỏ hơn, bởi vì phần dư sẽ nhỏ hơn khi dự đoán của người học mạnh tiến gần đến nhãn hơn.

Cuối cùng, chúng ta có thể sử dụng Scikit-Learn hoặc tính toán thủ công để thấy rằng các dự đoán như sau:

- Tuổi = 10, dự đoán = 6,87 Tuổi = 20, dự đoán = 5,11 Tuổi = 30, dự đoán = 6,71 Tuổi = 40, dự đoán = 1,43 Tuổi = 50, dự đoán = 1,43 Tuổi = 60, dự đoán = 1,43 Tuổi = 70, dự đoán = 4,90 Tuổi = 80, dự đoán = 4,10

XGBoost: Một cách cực đỉnh để tăng cường độ dốc

XGBoost, viết tắt của *tăng cường độ dốc cực cao*, là một trong những triển khai tăng cường độ dốc phổ biến, mạnh mẽ và hiệu quả nhất. Được tạo bởi Tianqi Chen và Carlos Guestrin vào năm 2016 (xem phụ lục C để tham khảo), các mô hình XGBoost thường hoạt động tốt hơn các mô hình phân loại và hồi quy khác. Trong phần này, chúng ta thảo luận về cách hoạt động của XGBoost, sử dụng ví dụ hồi quy tương tự từ phần “Tăng cường độ dốc: Sử dụng cây quyết định để xây dựng những người học giỏi”.

XGBoost sử dụng cây quyết định làm người học yếu và giống như các phương pháp tăng cường trước đây mà chúng ta đã học, mỗi người học yếu được thiết kế để tập trung vào điểm yếu của những người học trước. Cụ thể hơn, mỗi cây được xây dựng để phù hợp với phần dư dự đoán của các cây trước đó. Tuy nhiên, có một số khác biệt nhỏ, chẳng hạn như cách chúng tôi xây dựng cây, sử dụng thước đo gọi là *điểm tương đồng*. Hơn nữa, chúng tôi thêm một bước cắt tỉa để ngăn chặn việc trang bị quá mức, trong đó chúng tôi loại bỏ các cành cây nếu chúng không đáp ứng một số điều kiện nhất định. Trong phần này, chúng tôi đề cập đến điều này chi tiết hơn.

Điểm tương đồng XGBoost: Một cách mới và hiệu quả để đo lường mức độ tương đồng trong một tập hợp

Trong tiểu mục này, chúng ta thấy khối xây dựng chính của XGBoost, đây là một cách để đo lường mức độ giống nhau của các phần tử của một tập hợp. Số liệu này được gọi một cách thích hợp là *điểm tương đồng*. Trước khi học nó, chúng ta hãy làm một bài tập nhỏ. Trong ba bộ sau, bộ nào có nhiều điểm giống nhau nhất và bộ nào có ít điểm giống nhau nhất?

- **Bộ 1:** {10, −10, 4 **Bộ 2:**
- {7, 7, 7 **Bộ 3:** {7}
-

Nếu bạn nói rằng tập 2 có nhiều điểm tương đồng nhất và tập 1 có ít điểm tương đồng nhất thì trực giác của bạn đã đúng. Trong tập 1, các phần tử rất khác nhau nên phần này có ít điểm giống nhau nhất. Giữa bộ 2 và bộ 3 thì không rõ ràng lắm, vì cả hai bộ đều có cùng một phần tử nhưng có số lần khác nhau. Tuy nhiên, bộ 2 có số bảy xuất hiện ba lần, trong khi bộ 3 chỉ xuất hiện một lần. Do đó, ở tập 2, các phần tử có nhiều hơn

đồng nhất hoặc giống nhau hơn trong bộ 3.

Để định lượng sự tương đồng, hãy xem xét số liệu sau. Cho một tập $\{a_1, a_2, \dots, a_n\}$, điểm tương đồng là bình phương của tổng các phần tử chia cho số phần tử

$$\frac{(a_1 + a_2 + \dots + a_n)^2}{n}$$
 các phần tử, cụ thể là, .Hãy tính điểm tương đồng cho ba bộ trên, được hiển thị bên dưới:

- Set 1: $\text{Similarity score} = \frac{(10 - 10 + 4)^2}{3} = 5.33$
- Set 2: $\text{Similarity score} = \frac{(7 + 7 + 7)^2}{3} = 147$
- Set 3: $\text{Similarity score} = \frac{7^2}{1} = 49$

Lưu ý rằng đúng như dự đoán, điểm tương đồng của tập 2 là cao nhất và điểm tương đồng của tập 1 là thấp nhất.

lưu ý Điểm tương đồng này không hoàn hảo. Người ta có thể lập luận rằng tập hợp $\{1, 1, 1\}$ giống với tập hợp $\{7, 8, 9\}$, tuy nhiên điểm tương tự của $\{1, 1, 1\}$ là 3 và điểm tương tự của $\{7, 8, 9\}$ là 192. Tuy nhiên, theo mục đích của thuật toán của chúng tôi, điểm này vẫn hoạt động. Mục tiêu chính của điểm tương đồng là có thể phân tách tốt các giá trị lớn và nhỏ và mục tiêu này đã đạt được, như chúng ta sẽ thấy trong ví dụ hiện tại.

Có một siêu tham số λ liên kết với điểm tương tự, giúp ngăn ngừa tình trạng tràn bị quá mức. Khi sử dụng nó được cộng vào mẫu số của độ tương tự

$$\frac{(a_1 + a_2 + \dots + a_n)^2}{n + 2\lambda}$$
 điểm, đưa ra công thức .Vì vậy, ví dụ, nếu $\lambda = 2$, thì

$$\frac{(10 - 10 + 4)^2}{3 + 4} = 3.2$$
 điểm tương đồng của tập 1 hiện tại. Chúng tôi sẽ không sử dụng siêu tham số λ trong ví dụ của mình, nhưng khi lấy mã, chúng tôi sẽ xem cách đặt nó thành bất kỳ giá trị nào chúng tôi muốn.

Xây dựng những người học yếu

Trong tiêu mục này, chúng ta sẽ thấy cách xây dựng từng học viên yếu. Để minh họa quá trình này, chúng tôi sử dụng ví dụ tương tự từ phần “Tăng cường độ dốc”, được hiển thị trong bảng 12.3. Để thuận tiện, tập dữ liệu tương tự được hiển thị ở hai cột ngoài cùng bên trái của bảng 12.6. Đây là tập dữ liệu về người dùng của một ứng dụng, trong đó

tính năng là độ tuổi của người dùng và nhãn là số ngày mỗi tuần mà họ tương tác với ứng dụng.Sơ đồ của tập dữ liệu này được thể hiện trong hình 12.19.

Bảng 12.6 Tập dữ liệu tương tự như trong bảng 12.3, bao gồm người dùng, độ tuổi của họ và số ngày mỗi tuần họ tương tác với ứng dụng của chúng tôi. Cột thứ ba chứa các dự đoán từ người học yếu đầu tiên trong mô hình XGBoost của chúng tôi. Những dự đoán này đều có giá trị mặc định là 0,5. Cột cuối cùng chứa phần dư, là sự khác biệt giữa nhãn và dự đoán.

Đặc điểm (tuổi)	Nhãn (tương tác)	Dự đoán từ người học yếu đầu tiên	Dư lượng
10	7	0.5	6.5
20	5	0.5	4.5
30	7	0.5	6.5
40	1	0.5	0.5
50	2	0.5	1.5
60	1	0.5	0.5
70	5	0.5	4.5
80	4	0.5	3.5

Quá trình huấn luyện mô hình XGBoost tương tự như quá trình huấn luyện cây tăng cường độ dốc.Học yếu đầu tiên là một cây đưa ra dự đoán 0,5 cho mỗi điểm dữ liệu.Sau khi xây dựng bộ học yếu này, chúng tôi tính toán phần dư, đó là sự khác biệt giữa nhãn và nhãn được dự đoán.Hai đại lượng này có thể tìm thấy ở hai cột ngoài cùng bên phải của bảng 12.6.

Trước khi bắt đầu xây dựng những cây còn lại, hãy quyết định xem chúng ta muốn chúng sâu bao nhiêu.Để giữ cho ví dụ này nhỏ gọn, chúng ta hãy lại sử dụng độ sâu tối đa là 2. Điều đó có nghĩa là khi đạt đến độ sâu 2, chúng ta sẽ ngừng xây dựng những người học yếu.Đây là một siêu tham số mà chúng ta sẽ thấy chi tiết hơn trong phần “Đào tạo mô hình XGBoost bằng Python”.

Để xây dựng bộ học yếu thứ hai, chúng ta cần làm cho cây quyết định phù hợp với phần dư. Chúng tôi thực hiện việc này bằng cách sử dụng điểm tương đồng.Như thường lệ, ở nút gốc, chúng ta có toàn bộ tập dữ liệu.Do đó, chúng tôi bắt đầu bằng cách tính điểm tương tự của toàn bộ tập dữ liệu như sau:

$$Similarity = \frac{(6.5 + 4.5 + 6.5 + 0.5 + 1.5 + 0.5 + 4.5 + 3.5)^2}{8} = 98$$

Bây giờ, chúng ta tiến hành phân chia nút bằng cách sử dụng tính năng tuổi theo tất cả các cách có thể, như chúng ta đã làm với cây quyết định. Đối với mỗi lần phân chia, chúng tôi tính điểm tương tự của các tập hợp con tương ứng với từng lá và cộng chúng lại. Đó là điểm tương đồng tổng hợp tương ứng với phân chia đó. Điểm số như sau:

Phân chia cho nút gốc, với tập dữ liệu {6.5, 4.5, 6.5, 0.5, 1.5, 0.5, 4.5, 3.5} và điểm tương đồng = 98:

- Chia ở phút 15:
 - Nút trái: {6,5};điểm tương đồng: 42,25 Nút phải: {4,5, 6,5, 0,5, 1,5, 0,5, 4,5, 3,5};điểm tương đồng: 66,04 Điểm tương đồng tổng hợp: 108,29
 -
- Chia tay ở phút 25:
 - Nút trái: {6,5, 4,5};điểm tương đồng: 60,5 Nút phải: {6,5, 0,5, 1,5, 0,5, 4,5, 3,5};điểm tương đồng: 48,17 Điểm tương tự tổng hợp: 108,67
 -
- Chia ở 35:
 - Nút trái: {6,5, 4,5, 6,5};điểm tương đồng: 102,08 Nút phải: {0,5, 1,5, 0,5, 4,5, 3,5};điểm tương đồng: 22,05 **Điểm tương đồng tổng hợp: 124,13**
 -
- Chia ở 45:
 - Nút trái: {6,5, 4,5, 6,5, 0,5};điểm tương đồng: 81 Nút phải: {1,5, 0,5, 4,5, 3,5};điểm tương đồng: 25 Điểm tương đồng tổng hợp: 106
- Chia ở 55:
 - Nút trái: {6,5, 4,5, 6,5, 0,5, 1,5};điểm tương đồng: 76,05 Nút phải: {0,5, 4,5, 3,5};điểm tương đồng: 24,08 Điểm tương đồng tổng hợp: 100,13
 -
- Chia ở 65:
 - Nút trái: {6,5, 4,5, 6,5, 0,5, 1,5, 0,5};điểm tương đồng: 66,67 Nút phải: {4,5, 3,5};điểm tương đồng: 32 Điểm tương đồng tổng hợp: 98,67
 -
- Chia ở 75:
 - Nút trái: {6,5, 4,5, 6,5, 0,5, 1,5, 0,5, 4,5};điểm tương đồng: 85,75 Nút phải: {3,5};điểm tương đồng: 12,25 Điểm tương đồng tổng hợp: 98
 -

Như được hiển thị trong các tính toán này, phần phân chia có điểm tương đồng tổng hợp tốt nhất là ở độ tuổi = 35. Đây sẽ là phần phân chia ở nút gốc.

Tiếp theo, chúng tôi tiến hành phân chia tập dữ liệu tại mỗi nút theo cách tương tự.

Chia cho nút bên trái, với tập dữ liệu {6,5, 4,5, 6,5} và điểm tương đồng 102,08:

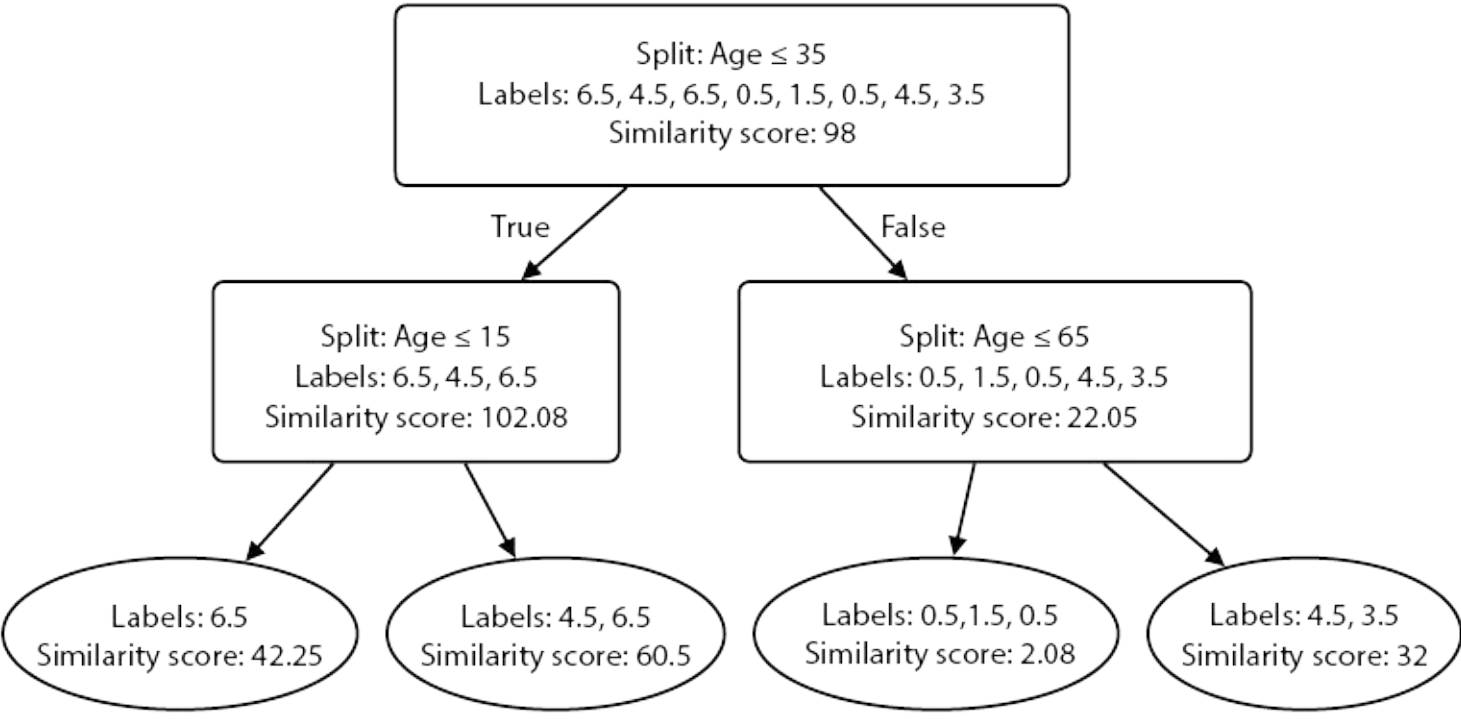
- Tách lúc 15: Nút trái: {6,5};điểm tương đồng: 42,25 Nút phải: {4,5, 6,5};điểm tương đồng: 60,5 Điểm tương đồng: 102,75
 -
- Tách ở 25: Nút trái: {6,5, 4,5};điểm tương đồng: 60,5 Nút phải: {6,5};Điểm tương đồng: 42,25 Điểm tương đồng: 102,75
 -

Cả hai phần chia đều cho chúng ta điểm tương tự tổng hợp như nhau, vì vậy chúng ta có thể sử dụng bất kỳ phần nào trong hai phần.Hãy sử dụng phần chia ở mức 15. Bây giờ, sang nút bên phải.

Chia cho nút bên phải, với tập dữ liệu {0,5, 1,5, 0,5, 4,5, 3,5} và điểm tương tự 22,05:

- Chia ở 45:
 - Nút trái: {0,5};điểm tương đồng: 0,25 Nút phải: {1,5, 0,5, 4,5, 3,5};điểm tương đồng: 25 Điểm tương đồng: 25,25
 -
- Chia ở 55:
 - Nút trái: {0,5, 1,5};điểm tương đồng: 2 Nút phải: {0,5, 4,5, 3,5};Điểm tương đồng: 24,08 Điểm tương đồng: 26,08
 -
- Tách ở 65: Nút trái: {0,5, 1,5, 0,5};điểm tương đồng: 2,08 Nút phải: {4,5, 3,5};Điểm tương đồng: 32 **Điểm tương đồng: 34,08**
 -
 -
- Chia ở 75:
 - Nút trái: {0,5, 1,5, 0,5, 4,5};điểm tương đồng: 12,25 Nút phải: {3,5};điểm tương đồng: 12,25 Điểm tương đồng: 24,5
 -

Từ đây, chúng ta kết luận rằng sự phân chia tốt nhất là ở tuổi = 65. Cây hiện có độ sâu 2 nên chúng ta ngừng phát triển nó, vì đây là điều chúng ta đã quyết định khi bắt đầu thuật toán.Cây kết quả, cùng với điểm tương tự tại các nút, được thể hiện trong hình 12.23.

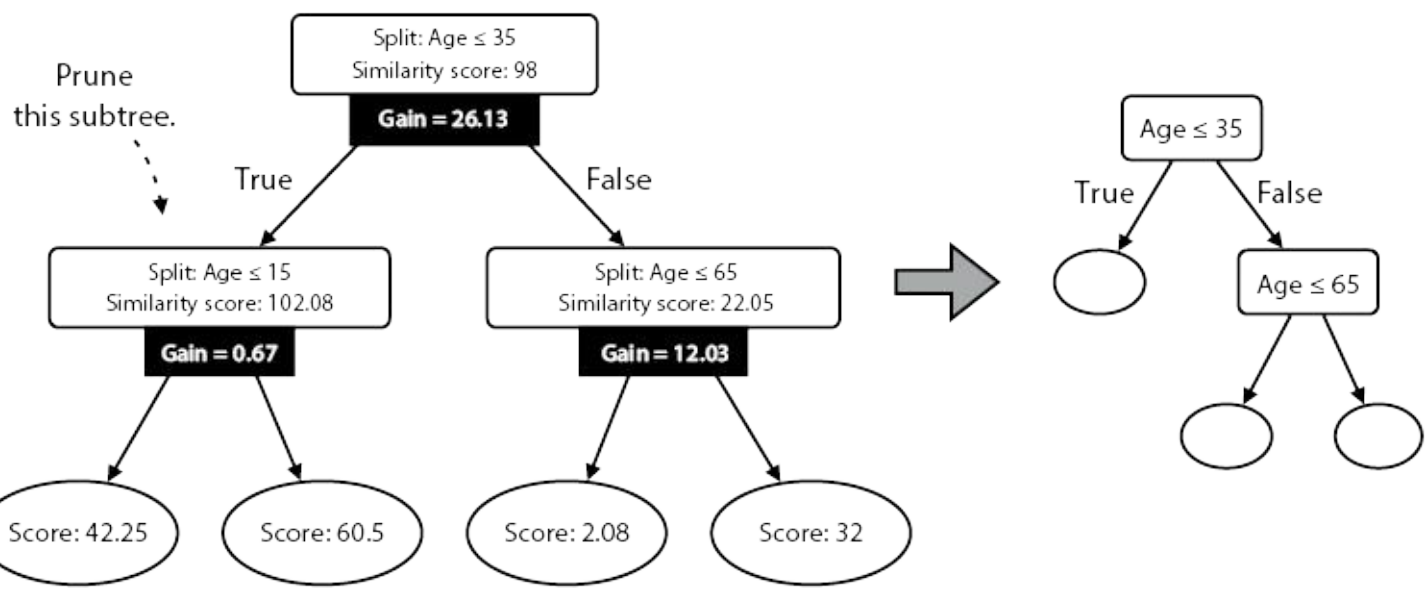


Hình 12.23 Trình học yếu thứ hai trong bộ phân loại XGBoost của chúng tôi. Đối với mỗi nút, chúng ta có thể thấy sự phân chia dựa trên đặc điểm độ tuổi, các nhãn tương ứng với nút đó và điểm tương đồng cho từng bộ nhãn. Phân phân chia được chọn cho mỗi nút là phân phân chia tối đa hóa điểm tương tự tổng hợp của các lá. Đối với mỗi lá, bạn có thể xem các nhãn tương ứng và điểm tương đồng của chúng.

Đó (gần như) là người học yếu thứ hai của chúng tôi. Trước khi tiếp tục xây dựng thêm các bộ học yếu, chúng ta cần thực hiện thêm một bước nữa để giúp giảm bớt việc trang bị quá mức.

Cắt tỉa cây: Một cách để giảm bớt việc trang bị quá mức bằng cách đơn giản hóa những người học yếu

Một tính năng tuyệt vời của XGBoost là nó không quá phù hợp. Để làm điều này, nó sử dụng một số siêu tham số được mô tả chi tiết trong phần “Đào tạo mô hình XGBoost bằng Python”. Một trong số đó, tổn thất phân chia tối thiểu, ngăn chặn sự phân chia xảy ra nếu điểm tương tự tổng hợp của các nút kết quả không lớn hơn đáng kể so với điểm tương tự của nút ban đầu. Sự khác biệt này được gọi là *mức tăng tương tự*. Ví dụ: trong nút gốc của cây của chúng tôi, điểm tương tự là 98 và điểm tương tự tổng hợp của các nút là 124,13. Như vậy, độ lợi tương tự là 124,13 – 98=26,13. Tương tự, độ lợi tương tự của nút bên trái là 0,67 và của nút bên phải là 12,03, như thể hiện trong hình 12.24.



Hình 12.24 Ở bên trái, chúng ta có cùng một cây từ hình 12.23, với một thông tin bổ sung: mức tăng tương tự. Chúng tôi đạt được điều này bằng cách trừ điểm tương tự cho mỗi nút khỏi điểm tương tự tổng hợp của các lá. Chúng tôi chỉ cho phép phân tách có mức tăng tương tự cao hơn 1 (siêu tham số tổn thất phân chia tối thiểu của chúng tôi), do đó, một trong các phân tách không còn được phép. Điều này dẫn đến cây bị tỉa ở bên phải, cây bây giờ trở thành cây học yếu của chúng ta.

Chúng tôi sẽ đặt tổn thất phân chia tối thiểu thành 1. Với giá trị này, phân chia duy nhất bị ngăn chặn là phân chia ở nút bên trái ($\text{tuổi} \leq 15$). Do đó, người học yếu thứ hai trông giống như người ở bên phải của hình 12.24.

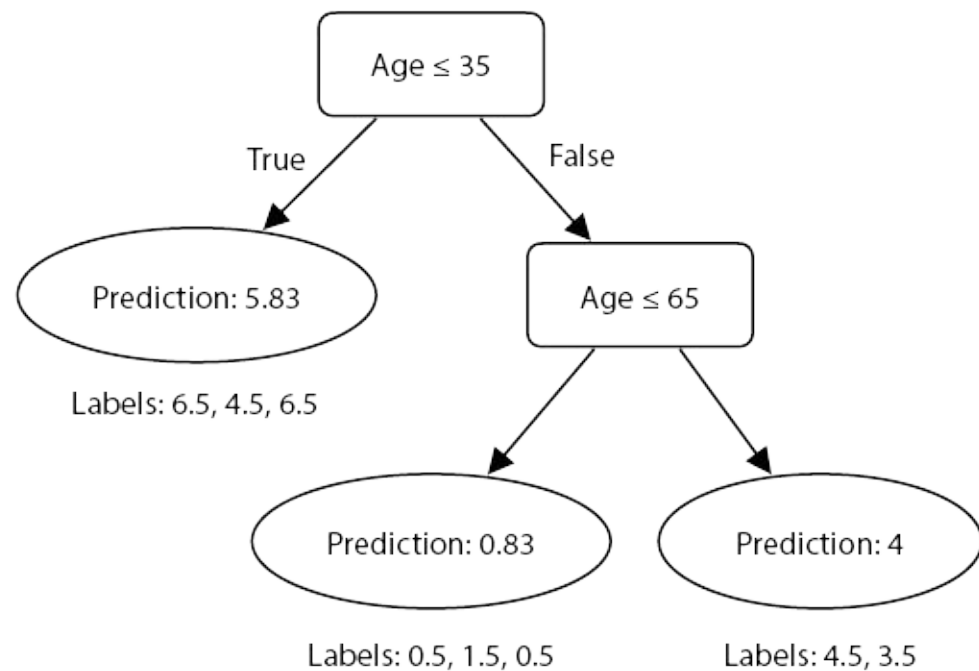
Đưa ra dự đoán

Bây giờ chúng ta đã xây dựng được bộ học yếu thứ hai, đã đến lúc sử dụng nó để đưa ra dự đoán. Chúng ta thu được các dự đoán giống như cách chúng ta thu được chúng từ bất kỳ cây quyết định nào, cụ thể là bằng cách lấy trung bình các nhãn trong lá tương ứng. Những dự đoán cho học sinh yếu thứ hai của chúng ta được thấy trong hình 12.25.

Bây giờ, hãy tính dự đoán tổng hợp cho hai học sinh yếu đầu tiên. Để tránh trang bị quá mức, chúng tôi sử dụng kỹ thuật tương tự mà chúng tôi đã sử dụng trong tăng cường độ dốc, đó là nhân

dự đoán của tất cả những người học yếu (trừ người đầu tiên) theo tốc độ học tập. Điều này nhằm mô phỏng phương pháp giảm độ dốc, trong đó chúng tôi dần hội tụ đến một dự đoán tốt sau vài lần lặp. Chúng tôi sử dụng tỷ lệ học tập là 0,7. Như vậy, tổng dự đoán của 2 người học yếu đầu tiên bằng dự đoán của người học yếu thứ nhất cộng với dự đoán của người học yếu thứ 2 nhân 0,7. Ví dụ: đối với điểm dữ liệu đầu tiên, dự đoán này là

$$0.5 + 5.83 \cdot 0.7 = 4.58.$$



Hình 12.25 Trình học yếu thứ hai trong mô hình XGBoost của chúng tôi sau khi được cắt bớt. Đây chính là cây trong hình 12.24 với những dự đoán của nó. Dự đoán ở mỗi lá là giá trị trung bình của các nhãn tương ứng với lá đó.

Cột thứ năm của bảng 12.7 chứa dự đoán tổng hợp của hai học sinh yếu đầu tiên.

Bảng 12.7 Các nhãn, dự đoán của hai học sinh yếu đầu tiên và phần dư. Dự đoán kết hợp có được bằng cách cộng dự đoán từ người học yếu thứ nhất (luôn là 0,5) cộng với tỷ lệ học tập (0,7) nhân với dự đoán từ người học yếu thứ hai. Phần dư một lần nữa là sự khác biệt giữa nhãn và dự đoán kết hợp.

Nhãn (tương tác)	Dự đoán từ người học yếu 1	Dự đoán từ người học yếu 2	Dự đoán từ người học yếu gấp 2 lần tốc độ học	Dự đoán kết hợp	Dư lượng
7	0.5	5.83	4.08	4.58	2.42
5	0.5	5.83	4.08	4.58	0.42
7	0.5	5.83	4.08	4.58	2.42
1	0.5	0.83	0.58	1.08	-0.08
2	0.5	0.83	0.58	1.08	0.92
1	0.5	0.83	0.58	1.08	-0.08
5	0.5	4	2.8	3.3	1.7
4	0.5	4	2.8	3.3	0.7

Lưu ý rằng các dự đoán kết hợp gần với nhãn hơn so với dự đoán của người học yếu đầu tiên.Bước tiếp theo là lặp lại.Chúng tôi tính toán phần dư mới cho tất cả các điểm dữ liệu, ghép cây cho chúng, tĩa cây, tính toán các dự đoán kết hợp mới và tiếp tục theo cách này.Số lượng cây chúng ta muốn là một siêu tham số khác mà chúng ta có thể chọn ngay từ đầu.Để tiếp tục xây dựng những cây này, chúng tôi sử dụng gói Python hữu ích có tên .

```
xgboost
```

Đào tạo mô hình XGBoost bằng Python

Trong phần này, chúng ta tìm hiểu cách huấn luyện mô hình để phù hợp với tập dữ liệu hiện tại bằng gói Python.Mã cho phần này nằm trong cùng sổ ghi chép với phần xgboost cái trước đó, được hiển thị ở đây:

- **Sổ tay:** gradient_boosting_and_XGBoost.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemb

Trước khi bắt đầu, hãy xem lại các siêu tham số mà chúng tôi đã xác định cho mô hình này:

số lượng ước tính Số lượng học sinh yếu. Lưu ý: trong gói, người học yếu đầu tiên không được tính trong số các bộ ước tính. Đối với ví dụ này, chúng tôi đặt nó thành 3, sẽ cho chúng tôi bốn học sinh yếu.

độ sâu tối đa Độ sâu tối đa được phép cho mỗi cây quyết định (người học yếu).Chúng tôi đặt nó thành 2.

tham số lambda Số được thêm vào mẫu số của điểm tương đồng. Chúng tôi đặt nó thành 0.

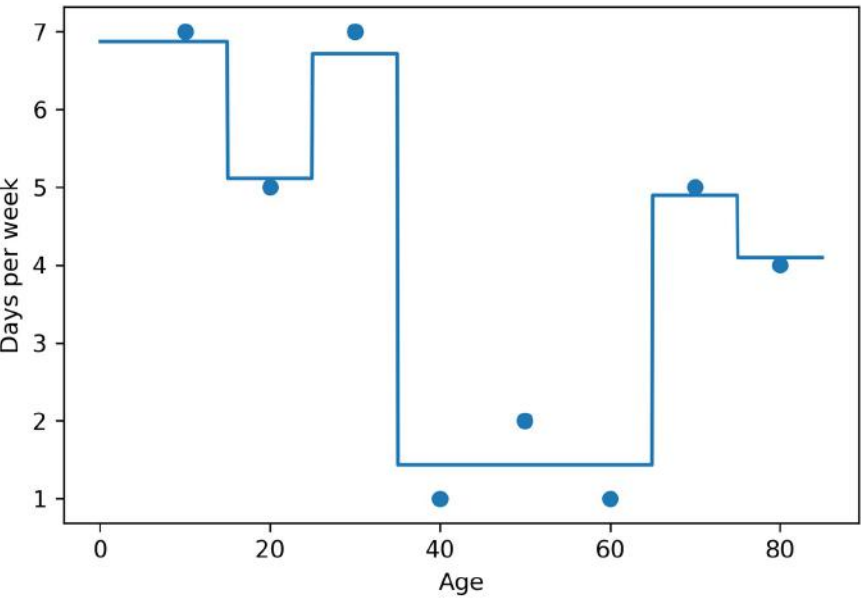
mất chia nhỏ tối thiểu Mức tăng tối thiểu về điểm tương đồng để cho phép xảy ra sự chia tách.Chúng tôi đặt nó thành 1.

tốc độ học tập Dự đoán từ người học yếu thứ hai đến cuối cùng được nhân với tốc độ học tập.Chúng tôi đặt nó thành 0,7.

Với các dòng mã sau, chúng tôi nhập gói, xây dựng một mô hình có tên và khớp nó với tập dữ liệu của chúng tôi:

```
import xgboost
from xgboost import XGBRegressor
xgboost_regressor = XGBRegressor(random_state=0,
                                  n_estimators=3,
                                  max_depth=2,
                                  reg_lambda=0,
                                  min_split_loss=1,
                                  learning_rate=0.7)
xgboost_regressor.fit(features, labels)
```

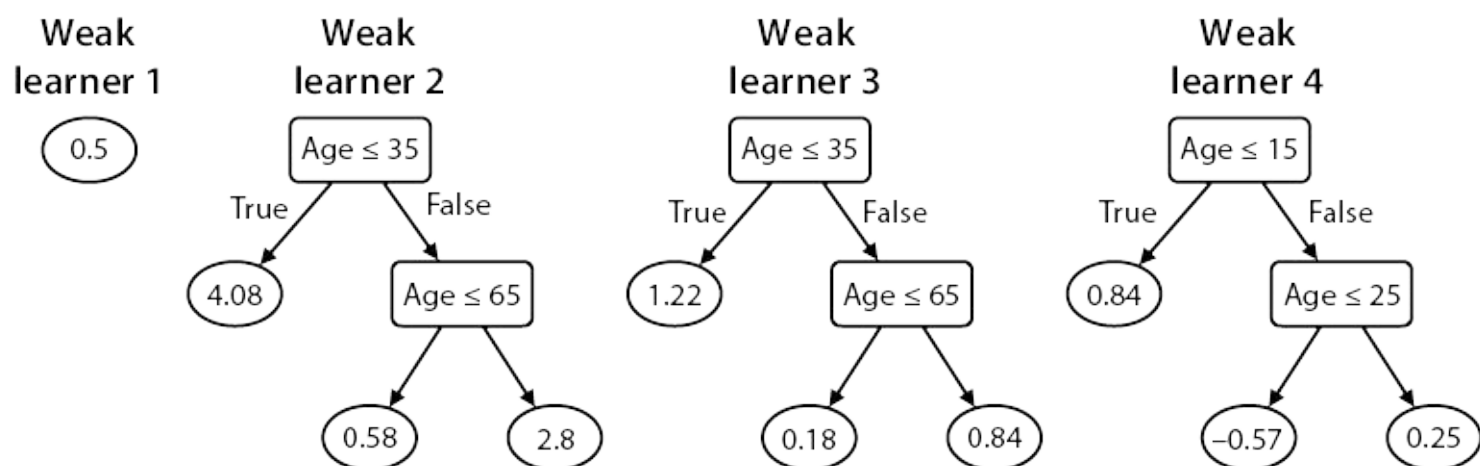
Sơ đồ của mô hình được thể hiện trong hình 12.26.Lưu ý rằng nó rất phù hợp với tập dữ liệu.



Hình 12.26 Sơ đồ dự đoán của mô hình XGBoost của chúng tôi.Lưu ý rằng nó rất phù hợp với tập dữ liệu.

Gói này cũng cho phép chúng tôi xem xét những người học yếu và họ xuất hiện trong hình 12.24.Các cây thu được theo cách này đã có nhãn nhân với tỷ lệ học là 0,7, điều này rõ ràng khi so sánh với các dự đoán của cây thu được thủ công trong hình 12.25 và cây thứ hai từ

bên trái trong hình 12.27.



Hình 12.27 Bốn người học yếu tạo nên người học mạnh trong mô hình XGBoost của chúng tôi. Lưu ý rằng giá trị đầu tiên luôn dự đoán là 0,5. Ba chiếc còn lại có hình dáng khá giống nhau, đây là một sự trùng hợp ngẫu nhiên. Tuy nhiên, hãy lưu ý rằng các dự đoán từ mỗi cây sẽ nhỏ hơn vì mỗi lần chúng ta điều chỉnh các phần dư nhỏ hơn. Hơn nữa, hãy lưu ý rằng cây học yếu thứ hai chính là cây mà chúng ta thu được theo cách thủ công trong hình 12.25, trong đó điểm khác biệt duy nhất là ở cây này, các dự đoán đã được nhân với tốc độ học là 0,7.

Vì vậy, để có được dự đoán của người học mạnh, chúng ta chỉ cần thêm dự đoán của mỗi cây. Ví dụ: đối với người dùng 20 tuổi, các dự đoán như sau:

- Học yếu 1: 0,5 Học yếu 2:
- 4,08 Học yếu 3: 1,22 Học
- yếu 4: -0,57
-

Như vậy dự đoán là $0,5 + 5,83 + 1,22 - 0,57 = 5,23$. Dự đoán cho các điểm khác như sau:

- Tuổi=10; dự đoán = 6,64 Tuổi =
- 20; dự đoán = 5,23 Tuổi = 30; dự
- đoán = 6,05 Tuổi = 40; dự đoán =
- 1,51 Tuổi = 50; dự đoán = 1,51
- Tuổi = 60; dự đoán = 1,51 Tuổi =
- 70; dự đoán = 4,39 Tuổi = 80; dự
- đoán = 4.39
-
-

Ứng dụng của phương pháp tập hợp

Phương pháp tập hợp là một số kỹ thuật học máy hữu ích nhất được sử dụng hiện nay vì chúng thể hiện mức độ hiệu suất cao với chi phí tương đối thấp. Một trong những nơi mà các phương pháp tập hợp được sử dụng nhiều nhất là trong các thử thách học máy, chẳng hạn như Thử thách Netflix. Thử thách Netflix là một cuộc thi do Netflix tổ chức, nơi họ ẩn danh một số dữ liệu và công khai nó. Mục tiêu của các đối thủ cạnh tranh là xây dựng một hệ thống đề xuất tốt hơn chính Netflix; hệ thống tốt nhất sẽ giành được một triệu đô la. Đội chiến thắng đã sử dụng sự kết hợp mạnh mẽ của những người học trong một nhóm để giành chiến thắng. Để biết thêm thông tin về điều này, hãy xem tài liệu tham khảo trong phụ lục C.

Tóm tắt

- Phương pháp tập hợp bao gồm việc đào tạo một số học viên yếu và kết hợp chúng thành một học viên mạnh. Chúng là một cách hiệu quả để xây dựng các mô hình mạnh mẽ mang lại kết quả tuyệt vời với các bộ dữ liệu thực. Các phương pháp tập hợp có thể được sử dụng để hồi quy và phân loại. Có hai loại phương pháp tập hợp chính: đóng gói và tăng cường. Đóng gói hoặc tổng hợp bootstrap, bao gồm việc xây dựng những người học liên tiếp trên các tập hợp con ngẫu nhiên của dữ liệu của chúng tôi và kết hợp chúng thành một người học mạnh mẽ để đưa ra dự đoán dựa trên đa số phiếu bầu. Tăng cường bao gồm việc xây dựng một chuỗi người học, trong đó mỗi người học tập trung vào điểm yếu của người trước đó và kết hợp chúng thành một bộ phân loại mạnh giúp đưa ra dự đoán dựa trên phiếu bầu có trọng số của người học. AdaBoost, tăng cường độ dốc và XGBoost là ba thuật toán tăng cường nâng cao mang lại kết quả tuyệt vời với các bộ dữ liệu thực. Các ứng dụng của phương pháp tập hợp rất đa dạng, từ các thuật toán khuyến nghị đến các ứng dụng trong y học và sinh học.
-
-

Bài tập

BÀI TẬP 12.1

Một người học mạnh được tăng cường L được hình thành bởi ba người học yếu là L_1 , L_2 và L_3 . Của họ

trọng số lần lượt là 1, 0,4 và 1,2.Đối với một điểm cụ thể, L_1 và L_2 dự đoán rằng nhãn của nó là dương và L_3 dự đoán rằng nhãn của nó là âm.cuối cùng là gì dự đoán của người học L về điểm này?

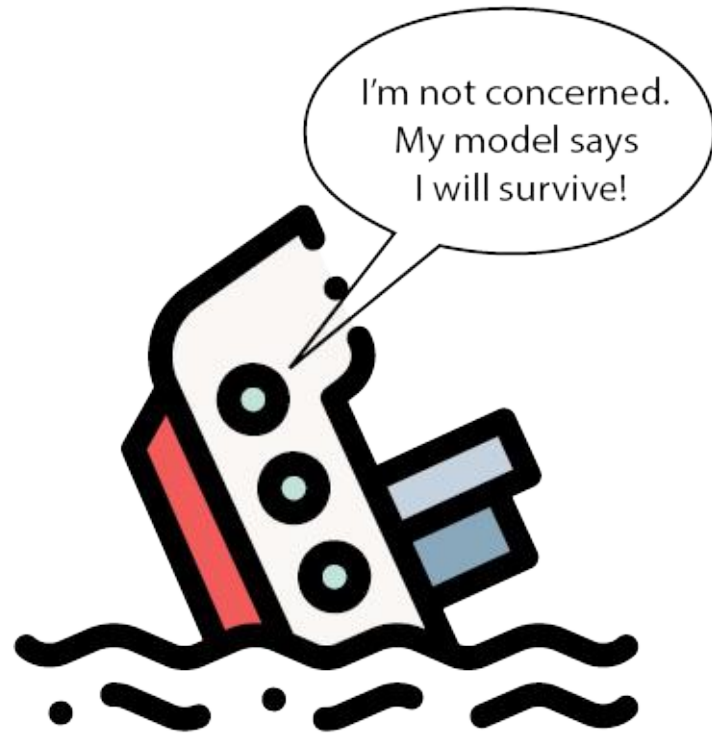
BÀI TẬP 12.2

Chúng tôi đang trong quá trình đào tạo mô hình AdaBoost trên tập dữ liệu có kích thước 100. Trình học yếu hiện tại phân loại chính xác 68 trên 100 điểm dữ liệu.Trọng số mà chúng ta sẽ gán cho người học này trong mô hình cuối cùng là bao nhiêu?

13 Áp dụng tất cả vào thực tế: Một ví dụ thực tế về kỹ thuật dữ liệu và học máy

Trong chương này

- dọn dẹp và xử lý trước dữ liệu để mô hình của chúng tôi có thể đọc được bằng cách sử dụng Scikit-Learn để đào tạo và đánh giá một số mô hình bằng cách sử dụng tìm kiếm lưới để chọn siêu tham số tốt cho mô hình của chúng tôi bằng cách sử dụng xác thực chéo k-Fold để có thể sử dụng dữ liệu của chúng tôi cho việc đào tạo và xác thực đồng thời



Trong suốt cuốn sách này, chúng tôi đã tìm hiểu một số thuật toán quan trọng nhất trong học tập có giám sát và chúng tôi đã có cơ hội mã hóa chúng và sử dụng chúng để đưa ra dự đoán trên một số tập dữ liệu. Tuy nhiên, quá trình đào tạo một mô hình trên dữ liệu thực đòi hỏi nhiều bước hơn và đây là những gì chúng ta sẽ thảo luận trong chương này.

Một trong những công việc cơ bản nhất của nhà khoa học dữ liệu là làm sạch và xử lý trước dữ liệu. Điều này rất quan trọng vì máy tính không thể thực hiện được toàn bộ công việc đó. Để làm sạch dữ liệu đúng cách, cần có kiến thức tốt về dữ liệu và vấn đề đang được giải quyết. Trong chương này, chúng ta thấy một số kỹ thuật quan trọng nhất để dọn dẹp và tiền xử lý dữ liệu. Sau đó, chúng tôi xem xét kỹ hơn các tính năng

và áp dụng một số kỹ thuật tính năng để giúp chúng sẵn sàng cho mô hình. Bước tiếp theo, chúng tôi chia mô hình thành các tập huấn luyện, xác thực và kiểm tra, huấn luyện một số mô hình trên tập dữ liệu của chúng tôi và đánh giá chúng. Bằng cách này, chúng tôi sẽ có thể chọn mô hình hoạt động tốt nhất cho tập dữ liệu này. Cuối cùng, chúng ta tìm hiểu các phương pháp quan trọng như tìm kiếm dạng lưới để tìm ra siêu tham số tốt nhất cho mô hình của mình.

Chúng tôi áp dụng tất cả các bước này trên tập dữ liệu phổ biến để học và thực hành các kỹ thuật học máy: tập dữ liệu Titanic. Chúng tôi trình bày chi tiết về tập dữ liệu này trong phần sau. Chương này bao gồm rất nhiều mã hóa. Hai gói Python mà chúng tôi sử dụng là Pandas và Scikit-Learn, chúng tôi đã sử dụng rộng rãi trong suốt cuốn sách này. Gói Pandas rất tốt để xử lý dữ liệu, bao gồm mở tệp, tải dữ liệu và sắp xếp dữ liệu dưới dạng bảng, được gọi là DataFrames. Gói Scikit-Learn rất tốt cho việc đào tạo và đánh giá các mô hình, đồng thời nó chứa các triển khai chắc chắn của hầu hết các thuật toán mà chúng ta học trong cuốn sách này.

Mã và tập dữ liệu mà chúng ta sẽ làm việc trong toàn bộ chương này như sau:

- **Sổ tay:** End_to_end_example.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_13_End_to_
- **Tập dữ liệu:** titanic.csv

Bộ dữ liệu Titanic

Trong phần này, chúng tôi tải và nghiên cứu tập dữ liệu. Tải và xử lý dữ liệu là một kỹ năng quan trọng đối với một nhà khoa học dữ liệu, bởi vì sự thành công của mô hình phụ thuộc rất nhiều vào cách xử lý trước dữ liệu đưa vào đó. Chúng tôi sử dụng gói Pandas để thực hiện việc này.

Trong suốt chương này, chúng tôi làm việc với một ví dụ phổ biến cho việc học máy: bộ dữ liệu Titanic. Ở cấp độ cao, tập dữ liệu chứa thông tin về nhiều hành khách trên *Titanic*, bao gồm tên, tuổi, tình trạng hôn nhân, cảng lên tàu và hạng của họ. Quan trọng nhất, nó còn chứa thông tin về sự sống sót của hành khách. Bạn có thể tìm thấy tập dữ liệu này trên Kaggle (www.kaggle.com), một cộng đồng trực tuyến phổ biến với các tập dữ liệu và cuộc thi tuyệt vời mà tôi thực sự khuyên bạn nên xem qua.

lưu ý Tập dữ liệu chúng tôi sử dụng là tập dữ liệu lịch sử, như bạn có thể tưởng tượng, chứa nhiều thành kiến xã hội từ năm 1912. Tập dữ liệu lịch sử không trình bày

cơ hội sửa đổi hoặc lấy mẫu bổ sung để phản ánh các chuẩn mực xã hội hiện tại và sự hiểu biết về thế giới. Một số ví dụ được tìm thấy ở đây là việc thiếu sự bao gồm các giới tính phi nhị phân, cách đối xử khác nhau đối với hành khách liên quan đến giới tính và tầng lớp xã hội, v.v. Chúng tôi sẽ đánh giá tập dữ liệu này như thể nó là một bảng số vì chúng tôi tin rằng đây là tập dữ liệu rất phong phú và thường được sử dụng để xây dựng mô hình và đưa ra dự đoán. Tuy nhiên, với tư cách là nhà khoa học dữ liệu, nhiệm vụ của chúng ta là luôn lưu ý đến những thành kiến trong dữ liệu của mình, chẳng hạn như những thành kiến liên quan đến chủng tộc, bản dạng giới, khuynh hướng tình dục, địa vị xã hội, khả năng, quốc tịch, tín ngưỡng, v.v. và làm mọi thứ trong khả năng của mình để đảm bảo rằng các mô hình chúng ta xây dựng sẽ không duy trì những thành kiến lịch sử.

Các tính năng của tập dữ liệu của chúng tôi

Tập dữ liệu Titanic mà chúng tôi đang sử dụng chứa tên và thông tin của 891 hành khách trên *Titanic*, bao gồm cả việc họ có sống sót hay không. Dưới đây là các cột của tập dữ liệu:

- **PassengerID:** số nhận dạng từng hành khách, từ 1 đến 891
- **Tên:** tên đầy đủ của hành khách
- **Giới tính:** giới tính của hành khách (nam hoặc nữ)
- **Tuổi:** tuổi của hành khách là số nguyên
- **Hạng p:** hạng mà hành khách đi: thứ nhất, thứ hai hoặc thứ ba
- **SibSP:** số anh chị em và vợ/chồng của hành khách (0 nếu hành khách đi một mình)
- **Parch:** số cha mẹ và con cái của hành khách (0 nếu hành khách đi một mình)
- **Vé:** số vé
- **Giá vé:** giá vé hành khách trả bằng bảng Anh
- **Cabin:** cabin mà hành khách đi
- **Đã bắt đầu:** cảng nơi hành khách đã lên tàu: “C” đối với Cherbourg, “Q” đối với Queenstown và “S” đối với Southampton
- **Sống sót:** thông tin hành khách có sống sót (1) hay không (0)
-
-
-
-
-
-

Sử dụng Pandas để tải tập dữ liệu

Trong phần này, chúng ta tìm hiểu cách mở tập dữ liệu bằng Pandas và tải nó vào DataFrame, đây là đối tượng mà Pandas sử dụng để lưu trữ các bảng dữ liệu. Tôi đã tải xuống dữ liệu từ www.kaggle.com và lưu trữ dưới dạng CSV (dấu phẩy-

giá trị riêng biệt) có tên titanic.csv. Trước khi thực hiện bất kỳ điều gì trên Pandas, chúng ta phải nhập Pandas bằng lệnh sau:

```
import pandas
```

Bây giờ chúng ta đã tải Pandas, chúng ta cần tải tập dữ liệu. Để lưu trữ tập dữ liệu, Pandas sử dụng hai đối tượng: *DataFrame* và *Series*. Về cơ bản, chúng giống nhau, ngoại trừ Sê-ri được sử dụng cho các tập dữ liệu chỉ có một cột và DataFrame được sử dụng cho các tập dữ liệu có nhiều cột.

We có thể tải tập dữ liệu dưới dạng DataFrame bằng lệnh sau :

```
raw_data = pandas.read_csv('./titanic.csv', index_col="PassengerId")
```

Lệnh này lưu trữ tập dữ liệu vào Pandas DataFrame có tên `raw_data`. Chúng tôi gọi nó là dữ liệu thô vì mục tiêu của chúng tôi là làm sạch nó và xử lý trước nó sau này. Sau khi tải nó, chúng ta có thể thấy các hàng đầu tiên trông giống như bảng 13.1. Nói chung, Pandas thêm một cột bổ sung đánh số tất cả các phần tử trong tập dữ liệu. Vì tập dữ liệu đã có sẵn cách đánh số này nên chúng ta có thể đặt chỉ mục này thành cột đó bằng cách chỉ định `index_col="PassengerId"` trong tập dữ liệu, các hàng được lập chỉ mục bắt đầu từ 1 thay vì bắt đầu từ 0 như thông thường trong thực tế.

Bảng 13.1 Bộ dữ liệu Titanic chứa thông tin về các hành khách trên tàu Titanic, bao gồm cả việc họ có sống sót hay không. Ở đây chúng tôi sử dụng Pandas để mở tập dữ liệu và in ra các hàng và cột của nó. Lưu ý rằng nó có 891 hàng và 12 cột.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17
3	1	3	Heikkinen, Miss Laina	female	26.0	0	0	STON/ 31012
...	...	...	...	...	...	...	...	...
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	11136
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	37037

SAVING AND LOADING THE DATASET

Before we embark on studying our dataset, here’s a small step that will help us. At the end of each section, we’ll save the dataset in a CSV file, and we’ll load it again at the beginning of the next section. This is so we can put down the book or quit the Jupyter Notebook and come back to work on it later at any checkpoint, without having to rerun all the commands from the beginning. With a small dataset like this one, it is not a big deal to rerun the commands, but imagine if we were processing large volumes of data. Serializing and saving data is important there, because it saves time and processing power.

Here are the names of the datasets saved at the end of each section:

- “The Titanic dataset”: raw_data
- “Cleaning up our dataset”: clean_data
- “Feature engineering”: preprocessed_data

The commands for saving and loading follow:

```
tablename.to_csv('./filename.csv', index=None)
tablename = pandas.read_csv('./filename.csv')
```

When Pandas loads a dataset, it adds an index column that numbers each of the

các phần tử.Chúng ta có thể bỏ qua cột này nhưng khi lưu tập dữ liệu phải đặt tham số để tránh lưu các cột chỉ mục không cần thiết.

Tập dữ liệu đã có cột chỉ mục có tên là PassengerId.Thay vào đó, nếu chúng tôi muốn sử dụng cột này làm cột chỉ mục mặc định trong Pandas, chúng tôi có thể chỉ định thời điểm tải tập dữ liệu (nhưng chúng tôi sẽ không làm điều này).

Sử dụng Pandas để nghiên cứu tập dữ liệu của chúng tôi

Trong phần này, tôi sẽ hướng dẫn bạn một số phương pháp hữu ích để nghiên cứu tập dữ liệu của chúng tôi.Đầu tiên là hàm độ dài, hoặc len. Hàm này trả về số hàng trong tập dữ liệu như sau:

```
len(raw_data)
Output: 891
```

Điều này có nghĩa là tập dữ liệu của chúng tôi có 891 hàng.Để xuất tên của các cột, chúng tôi sử dụng thuộc tính của DataFrame, như sau:

```
raw_data.columns
Output: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'], dtype='object')
```

Bây giờ hãy khám phá một trong các cột.Với lệnh sau, chúng ta có thể khám phá cột Sống sót:

```
raw_data['Survived']
Output:
0, 1, 1, 1, 0, ..., 0, 1, 0, 1, 0
Name: Survived, Length: 891, dtype: int64
```

Cột đầu tiên là chỉ số của hành khách (1 đến 891).Giá trị thứ hai là 0 nếu hành khách không sống sót và 1 nếu hành khách sống sót.Tuy nhiên, nếu chúng ta muốn có hai cột—ví dụ Tên và Tuổi—chúng ta có thể sử dụng

yêu cầu:

```
raw_data[['Name', 'Age']]
```

và điều này sẽ trả về một DataFrame chỉ có hai cột đó.

Bây giờ giả sử chúng ta muốn tìm hiểu xem có bao nhiêu hành khách sống sót.Chúng ta có thể tính tổng các giá trị trong cột Survived bằng hàm như sau:

sum


```
sum(raw_data['Survived'])
Output: 342
```

Điều này chỉ ra Trong số 891 hành khách trong tập dữ liệu của chúng tôi, chỉ có 342 người sống sót.

Đây chỉ là phần nổi của tảng băng chìm về tất cả các chức năng mà Pandas cung cấp để xử lý các tập dữ liệu. Truy cập trang tài liệu tại <https://pandas.pydata.org> để tìm hiểu thêm về nó.

Dọn dẹp tập dữ liệu của chúng tôi: Thiếu giá trị và cách xử lý chúng

Bây giờ chúng ta đã biết cách xử lý DataFrames, chúng ta sẽ thảo luận về một số kỹ thuật để dọn dẹp tập dữ liệu của mình. Tại sao điều này lại quan trọng? Trong cuộc sống thực, dữ liệu có thể lộn xộn và việc đưa dữ liệu lộn xộn vào một mô hình thường dẫn đến một mô hình tồi. Điều quan trọng là trước khi đào tạo mô hình, nhà khoa học dữ liệu phải khám phá kỹ tập dữ liệu và thực hiện một số thao tác dọn dẹp để dữ liệu sẵn sàng cho mô hình.

Vấn đề đầu tiên chúng tôi gặp phải là tập dữ liệu bị thiếu giá trị. Do lỗi của con người hoặc máy tính hoặc đơn giản là do sự cố thu thập dữ liệu, các bộ dữ liệu không phải lúc nào cũng có tất cả các giá trị trong đó. Việc cố gắng khớp mô hình với tập dữ liệu bị thiếu giá trị có thể sẽ dẫn đến lỗi. Bộ dữ liệu Titanic không phải là ngoại lệ khi bị thiếu dữ liệu. Ví dụ: hãy xem cột Cabin trong tập dữ liệu của chúng tôi, được hiển thị ở đây:

```
raw_data['Cabin']
Output:
0      NaN
1     C85
2      NaN
3    C123
4      NaN
...
886   NaN
887   B42
888   NaN
889   C148
890   NaN
Name: Cabin, Length: 891, dtype: object
```

Một số tên cabin có mặt, chẳng hạn như C123 hoặc C148, nhưng phần lớn các giá trị là NaN. NaN hoặc “không phải số” có nghĩa là mục nhập bị thiếu, không thể đọc được hoặc đơn giản là một loại khác không thể chuyển đổi thành số. Điều này có thể xảy ra do lỗi văn thư; người ta có thể tưởng tượng rằng

records of the *Titanic* are old and some information has been lost, or they simply didn't record the cabin number for every passenger to begin with. Either way, we don't want to have NaN values in our dataset. We are at a decision point: should we deal with these NaN values or remove the column completely? First let's check how many NaN values are in each column of our dataset. Our decision will depend on the answer to that question.

To find out how many values in each column are NaN, we use the `is_na` (or `is_null`) function. The `is_na` function returns a 1 if the entry is NaN, and a 0 otherwise. Therefore, if we sum over these values, we get the number of entries that are NaN in every column, as shown here:

```
raw_data.isna().sum()
Output:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
```

This tells us that the only columns with missing data are Age, which is missing 177 values; Cabin, which is missing 687 values; and Embarked, which is missing 2 values. We can deal with missing data using a few methods, and we'll apply different ones to different columns for this dataset.

Dropping columns with missing data

When a column is missing too many values, the corresponding feature may not be useful to our model. In this case, Cabin does not look like a good feature. Out of 891 rows, 687 don't have a value. This feature should be removed. We can do it with the `drop` function in Pandas as follows. We'll make a new DataFrame called `clean_data` to store the data we're about to clean up:

```
clean_data = raw_data.drop('Cabin', axis=1)
```

The arguments to the `drop` function follow:

- The name of the column we want to drop

- Tham số là 1 khi chúng ta muốn xóa một cột và 0 khi chúng ta muốn xóa một hàng

Sau đó chúng ta gán đầu ra của hàm này cho biến , biểu thị `clean_data` rằng chúng tôi muốn thay thế DataFrame cũ được gọi bằng cái mới bằng cột đã xóa.

Cách không mất toàn bộ cột: Điền dữ liệu còn thiếu

Không phải lúc nào chúng tôi cũng muốn xóa các cột có dữ liệu bị thiếu vì chúng tôi có thể mất thông tin quan trọng.Chúng ta cũng có thể điền vào dữ liệu những giá trị có ý nghĩa.Ví dụ: chúng ta hãy xem cột Tuổi, được hiển thị bên dưới:

```
clean_data['Age']
Output:
0      22.0
1      38.0
2      26.0
3      35.0
4      35.0
...
886    27.0
887    19.0
888     NaN
889    26.0
890    32.0
Name: Age, Length: 891, dtype: float64
```

Như chúng tôi đã tính toán trước đây, cột Tuổi chỉ thiếu 177 giá trị trong tổng số 891 giá trị, con số này không nhiều.Cột này hữu ích nên chúng ta đừng xóa nó.Vậy chúng ta có thể làm gì với những giá trị còn thiếu này?Có rất nhiều điều chúng ta có thể làm, nhưng phổ biến nhất là điền chúng với giá trị trung bình hoặc trung vị của các giá trị khác. Hãy làm điều sau.Đầu tiên, chúng ta tính trung vị bằng cách sử dụng hàm trung vị và thu được 28. Tiếp theo, chúng ta sử dụng hàm điền vào các giá trị còn thiếu

```
clean_data.fillna
```

với giá trị chúng tôi cung cấp cho nó, như được hiển thị trong đoạn mã tiếp theo:

```
median_age = clean_data["Age"].median()
clean_data["Age"] = clean_data["Age"].fillna(median_age)
```

Cột thứ ba thiếu giá trị là Embarked, thiếu hai giá trị. Chúng ta có thể làm gì ở đây?Chúng ta không thể sử dụng số trung bình vì đây là những chữ cái, không phải số.May mắn thay, chỉ có hai hàng thiếu con số này trong số 891 hàng trong số đó nên chúng ta không mất quá nhiều thông tin ở đây.Đề nghị của tôi là gộp tất cả lại

hành khách không có giá trị trong cột Embarked vào cùng hạng.Chúng ta có thể gọi lớp này là U, nghĩa là “Không xác định”.Dòng mã sau đây sẽ làm điều đó:

```
clean_data["Embarked"] = clean_data["Embarked"].fillna('U')
```

Cuối cùng, chúng ta có thể lưu DataFrame này trong tệp CSV có tên clean_titanic_data để sử dụng trong phần tiếp theo:

```
clean_data.to_csv('./clean_titanic_data.csv', index=None)
```

Kỹ thuật tính năng: Chuyển đổi các tính năng trong tập dữ liệu của chúng tôi trước khi đào tạo mô hình

Bây giờ chúng tôi đã làm sạch tập dữ liệu của mình, chúng tôi đã tiến gần hơn nhiều đến việc có thể đào tạo một mô hình.Tuy nhiên, chúng ta vẫn cần thực hiện một số thao tác dữ liệu quan trọng mà chúng ta sẽ thấy trong phần này.Đầu tiên là chuyển đổi loại dữ liệu từ số sang phân loại và ngược lại.Thứ hai là lựa chọn tính năng, trong đó chúng tôi quyết định thủ công những tính năng nào cần loại bỏ để cải thiện việc đào tạo mô hình của mình.

Hãy nhớ lại ở chương 2 rằng có hai loại đặc điểm, số và phân loại.Đối tượng số là đối tượng được lưu trữ dưới dạng số.Trong tập dữ liệu này, các đặc điểm như tuổi, giá vé và hạng là những con số.Đặc điểm phân loại là đặc điểm chứa nhiều danh mục hoặc lớp.Ví dụ: tính năng giới tính có hai lớp: nữ và nam.Tính năng bắt tay có ba lớp, C dành cho Cherbourg, Q dành cho Queenstown và S dành cho Southampton.

Như chúng ta đã thấy trong suốt cuốn sách này, các mô hình học máy lấy các con số làm đầu vào.Nếu đúng như vậy, làm thế nào để chúng ta nhập từ “nữ” hoặc chữ “Q”?Chúng ta cần có cách biến các đặc điểm phân loại thành các đặc điểm số.Ngoài ra, dù bạn có tin hay không, đôi khi chúng ta có thể quan tâm đến việc coi các đặc điểm số là mang tính phân loại để hỗ trợ chúng ta trong quá trình đào tạo, chẳng hạn như đặt chúng vào các nhóm, chẳng hạn như độ tuổi 1–10, 11–20, v.v.Chúng tôi đề cập đến vấn đề này nhiều hơn trong phần “Biến dữ liệu số thành dữ liệu phân loại”.

Hơn nữa, khi chúng ta nghĩ về một đặc điểm chẳng hạn như hạng hành khách (được gọi là Pclass), đây có thực sự là một đặc điểm số hay nó mang tính phân loại?Chúng ta có nên coi lớp học như một

number between one and three, or as three classes: first, second, and third? We answer all those questions in this section.

In this section, we call the DataFrame `preprocessed_data`. The first few rows of this dataset are shown in table 13.2

Table 13.2 The first five rows of the cleaned-up dataset. We will proceed to preprocess this data for training.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/ 31012
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	11380
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	37345

Turning categorical data into numerical data: One-hot encoding

As was mentioned previously, machine learning models perform a lot of mathematical operations, and to perform mathematical operations in our data, we must make sure all the data is numerical. If we have any columns with categorical data, we must turn them into numbers. In this section, we learn a way to do this effectively using a technique called *one-hot encoding*.

But before we delve into one-hot encoding, here’s a question: why not simply attach a different number to each one of the classes? For example, if our feature

có 10 lớp sao không đánh số 0, 1, 2,..., 9? Lý do là điều này buộc phải có trật tự trong các tính năng mà chúng ta có thể không mong muốn. Ví dụ: nếu cột Embarked có ba lớp C, Q và S, tương ứng với Cherbourg, Queenstown và Southampton, thì việc gán các số 0, 1 và 2 cho các lớp này sẽ ngầm cho mô hình biết rằng giá trị của Queenstown nằm giữa các giá trị của Cherbourg và Southampton, điều này không nhất thiết đúng. Một mô hình phức tạp có thể giải quyết được thứ tự ngầm định này, nhưng các mô hình đơn giản hơn (chẳng hạn như mô hình tuyến tính) sẽ bị ảnh hưởng. Chúng tôi muốn làm cho các giá trị này độc lập hơn với nhau và đây là lúc mã hóa một nóng xuất hiện.

Mã hóa một lần hoạt động theo cách sau: Đầu tiên, chúng tôi xem xét tính năng này có bao nhiêu lớp và xây dựng bao nhiêu cột mới. Ví dụ: một cột có hai danh mục là nữ và nam sẽ biến thành hai cột, một dành cho nữ và một dành cho nam. Chúng ta có thể gọi các cột này là giới tính_nam và giới tính_nữ cho rõ ràng. Sau đó, chúng tôi nhìn vào từng hành khách. Nếu hành khách là nữ thì cột giới tính_nữ sẽ có giá trị 1, cột giới tính_nam sẽ có giá trị 0. Nếu hành khách là nam thì chúng ta làm ngược lại.

Điều gì sẽ xảy ra nếu chúng ta có một cột có nhiều lớp hơn, chẳng hạn như cột đã bắt tay? Bởi vì cột đó có ba lớp (C dành cho Cherbourg, Q dành cho Queenstown và S dành cho Southampton), nên chúng tôi chỉ cần tạo ba cột có tên là đã bắt tay_c, đã bắt tay_q và đã bắt tay_s. Theo cách đó, nếu một hành khách lên tàu, chẳng hạn như Southampton, cột thứ ba sẽ có số 1 và hai cột còn lại có số 0. Quá trình này được minh họa trong hình 13.1.

	gender			gender_female	gender_male
Passenger 1	F	➔	Passenger 1	1	0
Passenger 2	M		Passenger 2	0	1
Passenger 3	M		Passenger 3	0	1
Passenger 4	F		Passenger 4	1	0

	embarked			embarked_c	embarked_q	embarked_s
Passenger 1	Q	➔	Passenger 1	0	1	0
Passenger 2	S		Passenger 2	0	0	1
Passenger 3	C		Passenger 3	1	0	0
Passenger 4	S		Passenger 4	0	0	1

Hình 13.1 Mã hóa một lần dữ liệu của chúng tôi để biến tất cả thành số để mô hình học máy đọc.Ở bên trái, chúng tôi có các cột với các đặc điểm phân loại như giới tính hoặc cảng lên tàu.Ở bên phải, chúng tôi đã biến các đặc điểm phân loại này thành các đặc điểm số.

Hàm Pandas giúp chúng ta mã hóa một lần. Chúng tôi sử dụng nó để tạo một số cột mới, sau đó chúng ta đính kèm các cột này vào tập dữ liệu và không được quên xóa cột gốc vì thông tin đó là dư thừa. Tiếp theo là mã để thực hiện mã hóa một lần theo giới tính và các cột được bắt đầu:

```
gender_columns = pandas.get_dummies(data['Sex'], prefix='Sex')
embarked_columns = pandas.get_dummies(data["Pclass"], prefix="Pclass")

preprocessed_data = pandas.concat([preprocessed_data, gender_columns], axis=1)
preprocessed_data = pandas.concat([preprocessed_data, embarked_columns], axis=1)

preprocessed_data = preprocessed_data.drop(['Sex', 'Embarked'], axis=1)
```

☐ Tạo các cột với các cột được mã hóa một lần ☐ Ghép nối tập dữ liệu với các cột mới được tạo ☐ Xóa các cột cũ khỏi tập dữ liệu

Đôi khi quá trình này có thể tốn kém. Hãy tưởng tượng có một cột có 500 lớp. Điều đó sẽ thêm 500 cột mới vào bảng của chúng tôi! Không chỉ vậy, các hàng sẽ rất thưa thớt, tức là chúng sẽ chứa hầu hết các số 0. Bây giờ hãy tưởng tượng nếu chúng ta có nhiều cột với hàng trăm lớp mỗi cột—bảng của chúng ta sẽ trở nên quá lớn để xử lý. Trong trường hợp này, với tư cách là nhà khoa học dữ liệu, hãy sử dụng tiêu chí của bạn để đưa ra quyết định.

Nếu có đủ sức mạnh tính toán và không gian lưu trữ để xử lý hàng nghìn hoặc thậm chí hàng triệu cột thì mã hóa một lần không có vấn đề gì. Nếu những tài nguyên này bị hạn chế, có lẽ chúng ta có thể mở rộng các lớp của mình để tạo ra ít cột hơn. Ví dụ: nếu chúng ta có một cột có 100 loại động vật, chúng ta có thể gộp chúng thành sáu cột được tạo thành bởi động vật có vú, chim, cá, động vật lưỡng cư, động vật không xương sống và bò sát.

CHÚNG TÔI CÓ THỂ MÃ HÓA MỘT TÍNH NĂNG SỐ SỐNG KHÔNG? NẾU CÓ, TẠI SAO CHÚNG TÔI MUỐN LÀM?

Rõ ràng, nếu một đối tượng địa lý có các danh mục như nam hoặc nữ, chiến lược tốt nhất của chúng tôi là mã hóa nó một lần. Tuy nhiên, có một số tính năng số mà chúng ta vẫn có thể muốn xem xét mã hóa một lần. Ví dụ, hãy xem cột Pclass. Cột này có các lớp 0, 1 và 2 dành cho hạng nhất, hạng nhì và hạng ba. Chúng ta nên giữ nó dưới dạng một tính năng số hay chúng ta nên mã hóa nó thành ba tính năng Pclass1, Pclass2 và Pclass3? Điều này chắc chắn còn gây tranh cãi và chúng ta có thể đưa ra những lập luận tốt cho cả hai bên. Người ta có thể lập luận rằng chúng tôi không muốn phóng to tập dữ liệu một cách không cần thiết nếu nó không mang lại cho mô hình sự cải thiện tiềm năng về hiệu suất. Có một quy tắc chung mà chúng ta có thể sử dụng để quyết định xem có nên chia một cột thành nhiều cột hay không. Chúng ta có thể tự hỏi: đặc điểm này có liên quan trực tiếp đến kết quả không? Nói cách khác, việc tăng giá trị của tính năng này có làm cho hành khách có nhiều khả năng (hoặc ít có khả năng) sống sót hơn không? Người ta có thể tưởng tượng rằng có lẽ hạng ghế càng cao thì khả năng hành khách sống sót càng cao. Hãy xem liệu trường hợp này có xảy ra hay không bằng cách thực hiện một số phép tính (xem mã trong sổ ghi chép), được hiển thị bên dưới:

- Ở khoang hạng nhất, 62,96% hành khách sống sót. Ở hạng hai,
- 40,38% hành khách sống sót. Ở hạng ba, 55% hành khách sống
- sót.

Lưu ý rằng khả năng sống sót thấp nhất là của hành khách ở khoang hạng hai. Vì vậy, việc tăng (hoặc giảm) lớp sẽ tự động nâng cao cơ hội sống sót là không đúng. Vì lý do này, tôi đề xuất mã hóa một lần tính năng này như sau:

```
categorized_pclass_columns = pd.get_dummies(preprocessed_data['Pclass'], prefix='Pclass')
preprocessed_data = pd.concat([preprocessed_data, categorized_pclass_columns], axis=1)
preprocessed_data = preprocessed_data.drop(['Pclass'], axis=1)
```


Chuyển dữ liệu số thành dữ liệu phân loại (và tại sao chúng ta lại muốn làm điều này?): Xếp chồng

Trong phần trước, chúng ta đã học cách biến dữ liệu phân loại thành dữ liệu số. Trong phần này, chúng ta thấy cách đi theo hướng khác. Tại sao chúng ta lại muốn điều này? Hãy xem một ví dụ.

Hãy nhìn vào cột Tuổi. Nó đẹp và số. Một mô hình máy học trả lời câu hỏi sau: “Tuổi tác quyết định khả năng sống sót trên *Titanic* bao nhiêu?” Hãy tưởng tượng rằng chúng ta có một mô hình tuyến tính để sinh tồn. Một mô hình như vậy sẽ dẫn đến một trong hai kết luận sau:

- Hành khách càng lớn tuổi thì khả năng sống sót càng cao. Hành khách càng
- lớn tuổi thì càng ít có khả năng sống sót.

Tuy nhiên, điều này có luôn luôn như vậy không? Điều gì sẽ xảy ra nếu mối quan hệ giữa tuổi tác và khả năng sống sót không đơn giản như vậy? Điều gì sẽ xảy ra nếu khả năng sống sót cao nhất là khi hành khách ở độ tuổi từ 20 đến 30 và thấp đối với tất cả các nhóm tuổi khác. Điều gì sẽ xảy ra nếu khả năng sống sót thấp nhất là từ 20 đến 30? Chúng ta cần cung cấp cho mô hình toàn quyền tự do để xác định độ tuổi nào quyết định liệu một hành khách có khả năng sống sót nhiều hay ít. Chúng ta có thể làm gì?

Nhiều mô hình phi tuyến có thể giải quyết vấn đề này, nhưng chúng ta vẫn nên sửa đổi cột Tuổi thành thứ gì đó giúp mô hình có nhiều tự do hơn để khám phá dữ liệu. Một kỹ thuật hữu ích mà chúng ta có thể làm là phân loại các độ tuổi, cụ thể là chia chúng thành nhiều nhóm khác nhau. Ví dụ: chúng ta có thể biến cột tuổi thành như sau:

- Từ 0 đến 10 tuổi Từ 11 đến
- 20 tuổi Từ 21 đến 30 tuổi Từ
- 31 đến 40 tuổi Từ 41 đến 50
- tuổi Từ 51 đến 60 tuổi Từ 61
- đến 70 tuổi Từ 71 đến 80 tuổi
- Từ 81 tuổi trở lên
-
-
-
-

Điều này tương tự như mã hóa một lần, theo nghĩa là nó sẽ biến cột Tuổi thành chín cột mới. Mã để làm điều này như sau:

```
bins = [0, 10, 20, 30, 40, 50, 60, 70, 80]
categorized_age = pandas.cut(preprocessed_data['Age'], bins)
preprocessed_data['Categorized_age'] = categorized_age
preprocessed_data = preprocessed_data.drop(["Age"], axis=1)
```

Lựa chọn tính năng: Loại bỏ các tính năng không cần thiết

Trong tiểu mục “Xóa các cột có dữ liệu bị thiếu”, chúng tôi đã loại bỏ một số cột trong bảng của mình vì chúng có quá nhiều giá trị bị thiếu. Tuy nhiên, chúng ta cũng nên bỏ đi một số cột khác, vì chúng không cần thiết cho mô hình của chúng ta, hoặc tệ hơn là chúng có thể làm hỏng hoàn toàn mô hình của chúng ta! Trong phần này, chúng ta thảo luận về những tính năng nào nên bị loại bỏ. Nhưng trước đó, hãy xem xét các tính năng và nghĩ xem tính năng nào trong số chúng sẽ không tốt cho mô hình của chúng ta. Họ lại ở đây:

- **PassengerID**: một số duy nhất tương ứng với mỗi hành khách **Tên**: tên đầy đủ của hành khách **Giới tính (hai loại)**: giới tính của hành khách là nam hay nữ **Tuổi (một số loại)**: tuổi của hành khách là số nguyên
-
- **Pclass (nhiều loại)**: hạng mà hành khách đi: thứ nhất, thứ hai hoặc thứ ba **SibSP**: số anh chị em và vợ/chồng của hành khách (0 nếu hành khách đi một mình) **Parch**: số cha mẹ và con cái của hành khách (0 nếu hành khách đi một mình) **Vé**: số vé **Giá vé**: giá vé hành khách đã thanh toán bằng bảng Anh **Cabin**: cabin mà hành khách đang đi **Đã lên tàu**: cảng mà hành khách đã lên tàu: C đến Cherbourg, Q cho Queenstown và S cho Southampton **Sống sót**: thông tin liệu hành khách có sống sót (1) hay không (0)
-
-
-
-
-

Đầu tiên, chúng ta hãy nhìn vào tính năng tên. Chúng ta có nên xem xét nó trong mô hình của chúng tôi? Hoàn toàn không, và đây là lý do: mỗi hành khách có một cái tên khác nhau (có lẽ có một số rất ít trường hợp ngoại lệ, không đáng kể). Do đó, mô hình sẽ được đào tạo để chỉ học tên của những hành khách sống sót và nó sẽ không thể cho chúng tôi biết bất cứ điều gì về những hành khách mới mà nó chưa thấy tên. Mô hình này đang ghi nhớ dữ liệu - nó không học được điều gì có ý nghĩa

về các tính năng của nó.Điều này có nghĩa là nó quá phù hợp và do đó, chúng ta nên loại bỏ hoàn toàn cột Tên.

Vé và các tính năng PassengerID có cùng vấn đề với tính năng tên, vì mỗi hành khách chỉ có một tên duy nhất.Chúng tôi cũng sẽ xóa hai cột đó.Hàm sẽ giúp chúng ta thực hiện việc này, như minh họa bên dưới:

```
drop
```

```
preprocessed_data = preprocessed_data.drop(['Name', 'Ticket', 'PassengerId'], axis=1)
```

Còn tính năng Survived thì sao—chúng ta cũng không nên loại bỏ tính năng đó sao?Chắc chắn! Việc giữ cột Sống sót trong tập dữ liệu của chúng tôi trong khi đào tạo sẽ quá phù hợp vì mô hình sẽ chỉ sử dụng tính năng này để xác định xem hành khách có sống sót hay không. Điều này giống như gian lận trong bài kiểm tra bằng cách nhìn vào lời giải.Chúng tôi sẽ không xóa nó khỏi tập dữ liệu vì sau này chúng tôi sẽ xóa nó khi chia tập dữ liệu thành các tính năng và nhãn để đào tạo.

Như thường lệ, chúng ta có thể lưu tập dữ liệu này trong tệp csv preprocessed_titanic_data.csv để sử dụng trong phần tiếp theo.

Đào tạo người mẫu của chúng tôi

Bây giờ dữ liệu của chúng tôi đã được xử lý trước, chúng tôi có thể bắt đầu đào tạo các mô hình khác nhau trên dữ liệu.Chúng ta nên chọn những mô hình nào từ những mô hình đã học trong cuốn sách này: cây quyết định, máy vectơ hỗ trợ, bộ phân loại hậu cần?Câu trả lời cho điều này nằm ở việc đánh giá các mô hình của chúng tôi.Trong phần này, chúng ta xem cách đào tạo một số mô hình khác nhau, đánh giá chúng trên tập dữ liệu xác thực và chọn mô hình tốt nhất phù hợp với tập dữ liệu của chúng ta.

Như thường lệ, chúng tôi tải dữ liệu của mình từ tệp mà chúng tôi đã lưu trong phần trước, như minh họa tiếp theo.Chúng tôi sẽ gọi nó.

```
data
```

```
data = pandas.read_csv('preprocessed_titanic_data.csv')
```

Bảng 13.3 chứa một vài hàng đầu tiên của dữ liệu được xử lý trước.Lưu ý rằng không phải tất cả các cột đều được hiển thị vì có 27 cột trong số đó.

Bảng 13.3 Năm hàng đầu tiên của dữ liệu được xử lý trước của chúng tôi, sẵn sàng được đưa vào mô hình. Lưu ý rằng nó có 21 cột, nhiều hơn trước. Các cột bổ sung này được tạo khi chúng tôi mã hóa và đóng thùng một lần

several of the existing features.

Survived	SibSp	Parch	Fare	Sex_female	Sex_male	Pclass_C	Pclass_Q
0	1	0	7.25000	0	1	0	0
1	1	0	71.2833	1	0	1	0
1	0	0	7.9250	1	0	0	0
1	1	0	53.1000	1	0	0	0
0	0	0	8.0500	0	1	0	0

aside If you run the code from the notebook, you may get different numbers.

Splitting the data into features and labels, and training and validation

Our dataset is a table with the features and labels together. We need to perform two splits. First, we need to separate the features from the labels to feed this to the model. Next, we need to form a training and a testing set. This is what we cover in this subsection.

To split the dataset into two tables called `features` and `labels`, we use the `drop` function as follows:

```
features = data.drop(["Survived"], axis=1)
labels = data["Survived"]
```

Next, we split the data into training and validation sets. We'll use 60% of our data for training, 20% for validation, and 20% for testing. For splitting the data, we use the Scikit-Learn function `train_test_split`. In this function, we specify the percentage of data we want for validation with the `test_size` parameter. The output is the four tables called `features_train`, `features_test`, `labels_train`, `labels_test`.

If we wanted to split our data into 80% training and 20% testing, we would use the following code:

```
from sklearn.model_selection import train_test_split
features_train, features_test, labels_train, labels_test =
    train_test_split(features, labels, test_size=0.2)
```

Tuy nhiên, vì chúng ta muốn đào tạo 60%, xác nhận 20% và kiểm tra 20% nên chúng ta cần sử dụng hàm này hai lần: một lần để tách đào tạo dữ liệu và một lần để phân tách các bộ xác thực và kiểm tra, như được hiển thị ở đây:

```
features_train, features_validation_test, labels_train, labels_validation_test =
    train_test_split(features, labels, test_size=0.4)
features_validation, features_test, labels_validation, labels_test =
    train_test_split(features_validation_test, labels_validation_test, test_size=0.5)
```

ngoài ra Bạn có thể thấy rằng trong số ghi chép chúng tôi đã chỉ định một lỗi cố định `random_state` trong chức năng này. Lý do là nó xáo trộn dữ liệu khi nó `train_test_split` chia tách nó.Chúng tôi sửa trạng thái ngẫu nhiên để đảm bảo chúng tôi luôn nhận được sự phân chia giống nhau.

Chúng ta có thể kiểm tra độ dài của các DataFrame này và nhận thấy rằng độ dài của tập huấn luyện là 534, của tập xác thực là 178 và của tập kiểm tra là 179. Bây giờ, hãy nhớ lại chương 4 rằng Quy tắc Vàng là không bao giờ sử dụng dữ liệu thử nghiệm của chúng tôi để đào tạo hoặc đưa ra quyết định về các mô hình của chúng tôi.Do đó, chúng tôi sẽ lưu tập kiểm tra đến cuối cùng khi chúng tôi quyết định sử dụng mô hình nào.Chúng tôi sẽ sử dụng tập huấn luyện để huấn luyện các mô hình và tập xác thực để đưa ra bất kỳ quyết định nào về việc chọn mô hình nào.

Đào tạo một số mô hình trên tập dữ liệu của chúng tôi

Cuối cùng chúng ta cũng đến phần thú vị: đào tạo người mẫu!Trong phần này, chúng ta sẽ xem cách đào tạo một số mô hình khác nhau trong Scikit-Learn chỉ bằng một vài dòng mã.

Đầu tiên, chúng ta bắt đầu bằng việc huấn luyện mô hình hồi quy logistic.Chúng ta có thể làm điều này bằng cách tạo một thể hiện và sử dụng phương thức này như sau:

```
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression()
lr_model.fit(features_train, labels_train)
```

Chúng ta cũng hãy huấn luyện cây quyết định, mô hình Bayes đơn giản, máy vector hỗ trợ, rừng ngẫu nhiên, cây được tăng cường độ dốc và mô hình AdaBoost, như được hiển thị trong đoạn mã tiếp theo:

```
from sklearn.tree import DecisionTreeClassifier, GaussianNB, SVC,
    RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier

dt_model = DecisionTreeClassifier()
dt_model.fit(features_train, labels_train)

nb_model = GaussianNB()
```

```
nb_model.fit(features_train, labels_train)

svm_model = SVC()
svm_model.fit(features_train, labels_train)

rf_model = RandomForestClassifier()
rf_model.fit(features_train, labels_train)

gb_model = GradientBoostingClassifier()
gb_model.fit(features_train, labels_train)

ab_model = AdaBoostClassifier()
ab_model.fit(features_train, labels_train)
```

Cái gì mô hình nào tốt hơn?Đánh giá các mô hình

Bây giờ chúng ta đã đào tạo một số mô hình, chúng ta cần chọn mô hình tốt nhất.Trong phần này, chúng tôi sử dụng các số liệu khác nhau để đánh giá chúng bằng các bộ xác thực.Hãy nhớ lại rằng trong chương 4, chúng ta đã học về độ chính xác, khả năng thu hồi, độ chính xác và điểm F_1 .Để làm mới bộ nhớ của bạn, các định nghĩa sau:

độ chính xác Tỷ lệ giữa số điểm được dán nhãn đúng và tổng số điểm.

thu hồi Trong số các điểm được gán nhãn tích cực, tỷ lệ chúng được phân loại chính xác.Nói cách khác, $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$, trong đó TP là số lượng kết quả dương tính thật và FN là số lượng kết quả âm tính giả.

độ chính xác Trong số các điểm được phân loại là tích cực, tỷ lệ các điểm đó được phân loại đúng. Nói cách khác, $\text{Độ chính xác} = \text{TP} / (\text{TP} + \text{FP})$, trong đó FP là số lượng kết quả dương tính giả.

Điểm F1 Giá trị trung bình hài hòa của độ chính xác và khả năng thu hồi. Đây là một con số giữa độ chính xác và thu hồi, nhưng nó gần với mức nhỏ hơn trong cả hai.

KIỂM TRA ĐỘ CHÍNH XÁC CỦA MỖI MÔ HÌNH

Hãy bắt đầu bằng việc đánh giá độ chính xác của những mô hình này.Chúng ta sẽ sử dụng Scikit-Learn sẽ thực hiện việc này, như minh họa tiếp theo:

```
lr_model.score(features_validation, labels_validation)
Output:
0.7932960893854749
```

Chúng tôi tính toán nó cho tất cả các mô hình khác và nhận được kết quả sau đây.

được làm tròn thành hai chữ số (xem sổ ghi chép để biết toàn bộ quy trình):

Độ chính xác

- **Hồi quy logistic:** 0,77 **Cây quyết**
- **định:** 0,78 **Naive Bayes:** 0,72
- **SVM:** 0,68 **Rừng ngẫu nhiên:**
- 0,7875 **Tăng cường độ dốc:** 0,81
- **AdaBoost:** 0,76
-
-
-

Điều này gợi ý rằng mô hình tốt nhất trong tập dữ liệu này là cây được tăng cường độ dốc, vì nó mang lại cho chúng tôi độ chính xác cao nhất trên tập xác thực (81%, tốt cho tập dữ liệu Titanic).Điều này không có gì đáng ngạc nhiên vì thuật toán này thường hoạt động rất tốt.

Bạn có thể làm theo quy trình tương tự để tính toán độ thu hồi, độ chính xác và điểm F_1 .Tôi sẽ để bạn làm điều đó để thu hồi và chính xác, và chúng ta sẽ cùng nhau làm điều đó để đạt được điểm F_1 .

KIỂM TRA F_1 -Score của MỖI MÔ HÌNH

Đây là cách chúng tôi kiểm tra điểm F_1 .Đầu tiên, chúng ta phải đưa ra dự đoán của mô hình, sử dụng hàm `predict` và sau đó chúng ta sử dụng hàm như sau: `f1_score`

```
lr_predicted_labels = lr_model.predict(features_validation)
f1_score(labels_validation, lr_predicted_labels)
```

Output :
0.6870229007633588

☐ Sử dụng mô hình để đưa ra dự đoán ☐ Tính điểm F_1

Như trước đây, chúng ta có thể thực hiện việc này cho tất cả các mô hình và nhận được kết quả sau:

F1-Điểm

- **Hồi quy logistic:** 0,69 **Cây quyết định:**
- 0,71 **Naive Bayes:** 0,63 **Máy vector hỗ**
- **trợ:** 0,42
-

- **Rừng ngẫu nhiên:** 0,68 **Tăng**
- **cường độ dốc:** 0,74 **AdaBoost:**
- 0,69

Một lần nữa, cây được tăng cường độ dốc đã giành chiến thắng với số điểm F_1 là 0,74. Với thực tế là con số của nó cao hơn nhiều so với các mô hình khác, chúng ta có thể kết luận một cách an toàn rằng trong số tám mô hình này, cây được tăng cường độ dốc là mô hình tốt nhất. Lưu ý rằng các mô hình dựa trên cây nhìn chung hoạt động tốt, điều này không có gì đáng ngạc nhiên do tính phi tuyến tính cao của tập dữ liệu. Sẽ rất thú vị khi đào tạo mạng nơ-ron và mô hình XGBoost trên tập dữ liệu này và tôi khuyến khích bạn làm như vậy!

Thử nghiệm mô hình

Sau khi so sánh các mô hình bằng cách sử dụng bộ xác thực, cuối cùng chúng tôi đã quyết định và đã chọn cây được tăng cường độ dốc làm mô hình tốt nhất cho tập dữ liệu này. Đừng ngạc nhiên; cây được tăng cường độ dốc (và người anh em họ gần của chúng, XGBoost) giành chiến thắng trong hầu hết các cuộc thi. Nhưng để xem liệu chúng ta có thực sự làm tốt hay không hay chúng ta vô tình tập quá mức, chúng ta cần thực hiện bài kiểm tra cuối cùng cho mô hình này: chúng ta cần kiểm tra mô hình trong bộ kiểm tra mà chúng ta chưa chạm tới.

Trước tiên, hãy đánh giá độ chính xác như sau:

```
gb_model.score(features_test, labels_test)
Output :
0.8324022346368715
```

Và bây giờ chúng ta hãy xem điểm F_1 được hiển thị bên dưới:

```
gb_predicted_test_labels = gb_model.predict(features_test)
f1_score(labels_test, gb_predicted_test_labels)
Output :
0.8026315789473685
```

Những điểm số này khá tốt đối với bộ dữ liệu Titanic. Vì vậy, chúng ta có thể thoải mái nói rằng mô hình của chúng ta là tốt.

Tuy nhiên, chúng tôi đã đào tạo các mô hình này mà không chạm vào siêu tham số của chúng, điều đó có nghĩa là Scikit-Learn đã chọn một số siêu tham số tiêu chuẩn cho chúng. Có cách nào để chúng ta có thể tìm ra các siêu tham số tốt nhất cho một mô hình không? Trong phần tiếp theo, chúng ta biết rằng có.

Điều chỉnh các siêu tham số để tìm ra mô hình tốt nhất: Tìm kiếm dạng lưới

Trong phần trước, chúng tôi đã đào tạo một số mô hình và nhận thấy rằng cây tăng cường độ dốc hoạt động tốt nhất trong số đó. Tuy nhiên, chúng tôi chưa khám phá nhiều cách kết hợp siêu tham số khác nhau, vì vậy chúng tôi có chỗ cần cải thiện trong quá trình đào tạo của mình. Trong phần này, chúng ta thấy một kỹ thuật hữu ích để tìm kiếm trong số nhiều tổ hợp siêu tham số nhằm tìm ra mô hình tốt cho dữ liệu của chúng ta.

Hiệu suất của cây được tăng cường độ dốc cao đến mức người ta có thể đạt được đối với tập dữ liệu Titanic, vì vậy hãy để nó yên. Tuy nhiên, SVM kém lại hoạt động ở vị trí cuối cùng với độ chính xác là 69% và điểm F_1 là 0,42. Chúng tôi tin vào

Tuy nhiên, SVM là một mô hình học máy mạnh mẽ. Có lẽ hiệu suất kém của SVM này là do các siêu tham số mà nó đang sử dụng. Có thể có sự kết hợp tốt hơn giữa chúng để hoạt động.

ngoài ra Trong suốt phần này, chúng tôi đưa ra một số lựa chọn về tham số. Một số dựa trên kinh nghiệm, một số dựa trên thông lệ tiêu chuẩn và một số là tùy ý. Tôi khuyến khích bạn thử làm theo quy trình tương tự với bất kỳ lựa chọn nào bạn quyết định thực hiện và cố gắng đánh bại số điểm hiện tại của các mô hình!

Để cải thiện hiệu suất của SVM, chúng tôi sử dụng một phương pháp có tên *tìm kiếm dạng lưới*, bao gồm đào tạo mô hình của chúng tôi nhiều lần qua các kết hợp siêu tham số khác nhau và chọn một phương pháp hoạt động tốt nhất trên bộ xác thực của chúng tôi.

Hãy bắt đầu bằng cách chọn một hạt nhân. Trong thực tế, tôi nhận thấy rằng hạt nhân RBF (hàm cơ sở xuyên tâm) có xu hướng hoạt động tốt, vì vậy hãy chọn hạt nhân đó. Hãy nhớ lại ở chương 9 rằng siêu tham số đi kèm với nhân RBF là γ , là một số thực. Hãy thử huấn luyện một SVM với hai giá trị cho γ , đó là 1 và 10. Tại sao lại là 1 và 10? Thông thường, khi tìm kiếm siêu tham số, chúng tôi có xu hướng thực hiện tìm kiếm theo cấp số nhân, vì vậy, chúng tôi sẽ thử các giá trị như 0,1, 1, 10, 100, 1000, v.v., trái ngược với 1, 2, 3, 4, 5. Tìm kiếm theo cấp số nhân này bao trùm một không gian lớn hơn và cho chúng ta cơ hội tìm thấy siêu tham số tốt hơn và thực hiện kiểu tìm kiếm này là thông lệ tiêu chuẩn của các nhà khoa học dữ liệu.

Hãy nhớ lại chương 9 rằng một siêu tham số khác liên quan đến SVM là tham số C . Chúng ta cũng hãy thử huấn luyện các mô hình với $C = 1$ và $C = 10$. Điều này cho chúng ta bốn mô hình có thể huấn luyện sau đây:

Mô hình 1: hạt nhân = RBF, gamma = 1, C = 1

Mẫu 2: hạt nhân = RBF, gamma = 1, C = 10

Mẫu 3: hạt nhân = RBF, gamma = 10, C = 1

Mẫu 4: nhân = RBF, gamma = 10, C= 10

Chúng ta có thể dễ dàng huấn luyện tất cả những thứ đó trong tập huấn luyện của mình bằng tám dòng mã sau:

```
svm_1_1 = SVC(kernel='rbf', C=1, gamma=1)
svm_1_1.fit(features_train, labels_train)

svm_1_10 = SVC(kernel='rbf', C=1, gamma=10)
svm_1_10.fit(features_train, labels_train)

svm_10_1 = SVC(kernel='rbf', C=10, gamma=1)
svm_10_1.fit(features_train, labels_train)

svm_10_10 = SVC(kernel='rbf', C=10, gamma=10)
svm_10_10.fit(features_train, labels_train)
```

Bây giờ chúng ta đánh giá chúng bằng cách sử dụng độ chính xác (một lựa chọn tùy ý khác—chúng ta cũng có thể sử dụng F_1 -điểm, độ chính xác hoặc thu hồi).Điểm số được ghi ở bảng 13.4.

Bảng 13.4 Phương pháp tìm kiếm dạng lưới rất hữu ích để tìm kiếm qua nhiều tổ hợp siêu tham số và chọn ra mô hình tốt nhất. Ở đây, chúng tôi sử dụng tìm kiếm dạng lưới để chọn sự kết hợp tốt nhất giữa tham số C và gamma trong SVM. Chúng tôi đã sử dụng độ chính xác để so sánh các mô hình trong bộ xác thực. Lưu ý rằng mô hình tốt nhất trong số này là mô hình có gamma = 0,1 và C = 10, với độ chính xác là 0,72.

	C = 1	C = 10
gamma= 0,1	0.69	0.72
gamma= 1	0.70	0.70
gamma= 10	0.67	0.65

Lưu ý từ bảng 13.4 rằng độ chính xác tốt nhất là 0,72, được đưa ra bởi mô hình có gamma = 0,1 và C = 1. Đây là sự cải thiện so với 0,68 mà chúng tôi thu được trước đây khi chúng tôi không chỉ định bất kỳ siêu tham số nào.

Nếu có nhiều tham số hơn, chúng ta chỉ cần tạo một lưới với chúng và huấn luyện tất cả các mô hình có thể.Lưu ý rằng khi chúng ta khám phá nhiều lựa chọn hơn, số lượng mô hình sẽ tăng lên nhanh chóng.Ví dụ: nếu chúng ta muốn khám phá năm giá trị cho

gamma và bốn giá trị cho C, chúng ta sẽ phải huấn luyện 20 mô hình (năm lần bốn). Chúng tôi cũng có thể thêm nhiều siêu tham số hơn—ví dụ: nếu có siêu tham số thứ ba mà chúng tôi muốn thử và có bảy giá trị cho siêu tham số này, chúng tôi sẽ phải huấn luyện tổng cộng 140 mô hình (năm nhân bốn nhân bảy). Khi số lượng mô hình tăng quá nhanh, điều quan trọng là phải chọn các lựa chọn theo cách nó khám phá tốt không gian siêu tham số mà không cần đào tạo một số lượng lớn mô hình.

Scikit-Learn cung cấp một cách đơn giản để thực hiện việc này: sử dụng `GridSearchCV`. Đầu tiên, chúng ta định nghĩa siêu tham số là một từ điển, trong đó khóa của từ điển là tên của tham số và giá trị tương ứng với khóa này là danh sách các giá trị mà chúng ta muốn thử cho siêu tham số của mình. Trong trường hợp này, hãy cùng khám phá các kết hợp siêu tham số sau:

Hạt nhân: RBF

C: 0,01, 0,1, 1, 10, 100

gamma: 0,01, 0,1, 1, 10, 100

Đoạn mã sau sẽ làm điều đó:

```
svm_parameters = {'kernel': ['rbf'],
                  'C': [0.01, 0.1, 1 , 10, 100],
                  'gamma': [0.01, 0.1, 1, 10, 100]
                  } ❶

svm = SVC() ❷

svm_gs = GridSearchCV(estimator = svm,
                      param_grid = svm_parameters) ❸

svm_gs.fit(features_train, labels_train) ❹
```

- ❑ Một từ điển có siêu tham số và các giá trị mà chúng tôi muốn thử
- ❑ Một SVM thông thường không có siêu tham số
- ❑ Đối tượng `GridSearchCV` nơi chúng tôi chuyển SVM và từ điển siêu tham số
- ❑ Chúng tôi điều chỉnh mô hình `GridSearchCV` theo cách tương tự như cách chúng tôi điều chỉnh mô hình thông thường trong Scikit-Learn.

Phần này đào tạo 25 mô hình với tất cả sự kết hợp của siêu tham số được đưa ra trong từ điển siêu tham số. Bây giờ, chúng tôi chọn mô hình tốt nhất trong số này và gọi nó là `svm_winner`. Hãy tính độ chính xác của mô hình này trên bộ xác thực là

sau:

```
svm_winner = svm_gs.best_estimator_
svm_winner.score(features_validation, labels_validation)
```

Output :
0.7303370786516854

Mô hình chiến thắng của chúng tôi đạt được độ chính xác 0,73, tốt hơn so với 0,68 ban đầu. Chúng tôi vẫn có thể cải thiện mô hình này bằng cách chạy tìm kiếm siêu tham số lớn hơn và tôi khuyến khích bạn tự mình thử nó. Bây giờ, hãy cùng khám phá siêu tham số nào đã được sử dụng bởi mô hình SVM chiến thắng gần đây nhất, như được hiển thị ở đây:

```
svm_winner
Output :
SVC(C=10, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma=0.01, kernel='rbf',
    max_iter=-1, probability=False, random_state=None, shrinking=True,
    tol=0.001, verbose=False)
```

Mô hình chiến thắng đã sử dụng hạt nhân RBF có $\gamma = 0,01$ và $C = 10$.

thử thách Tôi khuyến khích bạn thử sử dụng tìm kiếm dạng lưới trên các mô hình khác và xem bạn có thể cải thiện độ chính xác và *điểm* F_1 của mô hình chiến thắng đến mức nào! Nếu bạn có điểm cao, hãy chạy nó trên tập dữ liệu Kaggle và gửi dự đoán của bạn bằng liên kết sau: <https://www.kaggle.com/c/titanic/submit>.

Còn một điều nữa: CV ở cuối là gì? Nó là viết tắt của `crossGridSearchCV` *xác thực* mà chúng ta sẽ tìm hiểu trong phần tiếp theo.

Sử dụng xác thực chéo K-Fold để sử dụng lại dữ liệu của chúng tôi dưới dạng đào tạo và xác thực

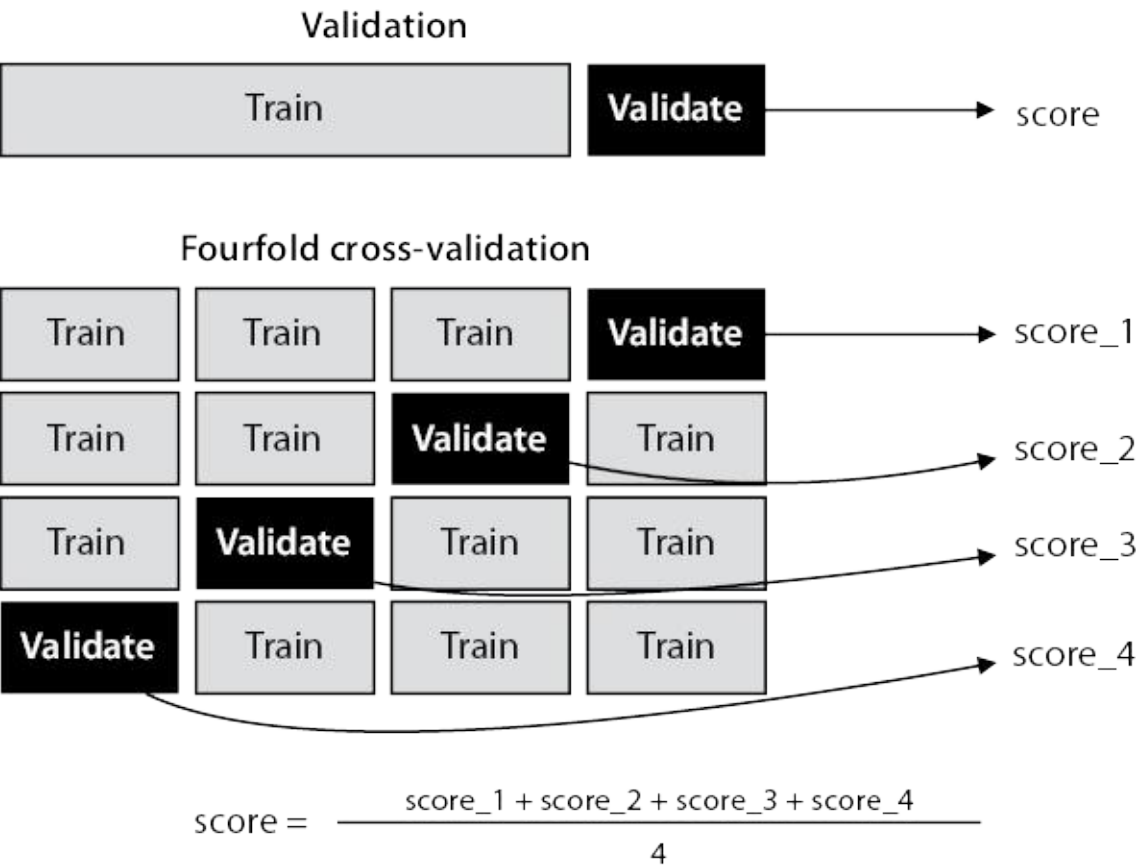
Trong phần này, chúng ta tìm hiểu một phương pháp thay thế cho phương pháp đào tạo-xác thực-kiểm tra truyền thống mà chúng ta đã sử dụng trong chương này. Nó được gọi là *xác thực chéo k-fold* và rất hữu ích trong nhiều trường hợp, đặc biệt khi tập dữ liệu của chúng tôi nhỏ.

Trong suốt ví dụ này, chúng tôi đã sử dụng 60% dữ liệu để đào tạo, 20% để xác thực và 20% cuối cùng để thử nghiệm. Điều đó có hiệu quả trong thực tế, nhưng có vẻ như chúng ta đang mất một số dữ liệu, phải không? Cuối cùng, chúng tôi đào tạo mô hình chỉ với 60% dữ liệu, điều này có thể ảnh hưởng đến mô hình của chúng tôi, đặc biệt khi tập dữ liệu nhỏ. Xác thực chéo K là một cách sử dụng tất cả dữ liệu cho mục đích đào tạo và kiểm tra bằng cách tái sử dụng dữ liệu đó nhiều lần. Nó hoạt động như sau:

1. Chia dữ liệu thành k phần bằng nhau (hoặc gần bằng nhau).

2. Huấn luyện mô hình k lần, sử dụng kết hợp $k - 1$ trong các phần làm tập huấn luyện và phần còn lại làm tập xác thực. 3. Điểm cuối cùng của mô hình đó là điểm trung bình của các điểm xác thực từ các bước k .

Hình 13.2 cho thấy một bức tranh về xác nhận chéo bốn lần.



Hình 13.2 K xác thực chéo lần lượt là một phương pháp hữu ích giúp tái chế dữ liệu của chúng tôi để sử dụng dữ liệu đó cho cả mục đích đào tạo và xác thực.Ở trên cùng, chúng ta thấy sự phân chia đào tạo-xác thực cổ điển.Ở phía dưới, chúng ta thấy hình minh họa về xác thực chéo bốn lần, trong đó chúng ta chia dữ liệu thành bốn phần bằng nhau (hoặc gần như bằng nhau).Sau đó chúng tôi huấn luyện mô hình của mình bốn lần.Mỗi lần chúng tôi chọn ba phần làm tập huấn luyện và phần còn lại làm tập xác thực.Điểm của mô hình này là điểm trung bình của bốn điểm đạt được trên mỗi bộ xác nhận.

Phương pháp này được sử dụng và kết quả của quá trình này có thể được kiểm tra bằng cách gõ `print(model.score)`.Chúng tôi sẽ không hiển thị kết quả ở đây, vì chúng rất dài nhưng bạn có thể xem chúng trong sổ tay.

Tóm tắt

- Pandas là gói Python hữu ích để mở, thao tác và lưu tập dữ liệu. Việc dọn dẹp dữ liệu của chúng
- tôi là cần thiết vì nó có thể đi kèm với các vấn đề như thiếu giá trị. Các tính năng có thể là số hoặc phân loại. Các đặc tính số là những con số, chẳng hạn như tuổi tác. Đặc điểm phân loại là các danh mục hoặc loại, chẳng hạn như chó/mèo/chim. Các mô hình học máy chỉ lấy số, vì vậy để cung cấp dữ liệu phân loại cho mô hình học máy, chúng ta phải biến nó thành dữ liệu số. Một cách để làm điều này là mã hóa một lần. Trong một số trường hợp, chúng tôi có thể muốn coi các đặc điểm số của mình cũng như các đặc điểm phân loại, điều mà chúng tôi có thể thực hiện bằng cách gộp dữ liệu. Điều quan trọng là sử dụng lựa chọn tính năng để loại bỏ các tính năng không cần thiết trong dữ liệu của chúng tôi. Scikit-Learn là gói hữu ích để đào tạo, kiểm tra và đánh giá các mô hình học máy. Trước khi huấn luyện mô hình, chúng ta phải chia dữ liệu thành huấn luyện, xác thực và kiểm tra. Chúng tôi có chức năng Pandas để thực hiện việc này. Tìm kiếm dạng lưới là một phương pháp được sử dụng để tìm các siêu tham số tốt nhất cho một mô hình. Nó bao gồm việc đào tạo một số mô hình trên một tập hợp siêu tham số (đôi khi lớn). *Xác thực chéo K* là một phương pháp được sử dụng để tái chế dữ liệu và sử dụng dữ liệu đó để đào tạo và xác thực. Nó bao gồm việc đào tạo và thử nghiệm một số mô hình trên các phần khác nhau của dữ liệu.
-
-
-
-

Bài tập

BÀI TẬP 13.1

Kho lưu trữ chứa một tệp có tên test.csv. Đây là tệp có nhiều hành khách hơn trên *Titanic*, ngoại trừ việc nó không có cột Sống sót.

1. Xử lý trước dữ liệu trong tệp này như chúng tôi đã làm trong chương này.
2. Sử dụng bất kỳ mô hình nào để dự đoán nhãn trong tập dữ liệu này. Theo mô hình của bạn, có bao nhiêu hành khách sống sót?
3. So sánh hiệu suất của tất cả các mô hình trong chương này, bạn nghĩ có bao nhiêu hành khách trong bộ thử nghiệm thực sự sống sót?

Chương 2: Các loại máy học

Đối với các câu hỏi trong chương này, câu trả lời của bạn không cần phải giống câu trả lời của tôi. Nếu bạn có những ý tưởng khác nhau cho các mô hình được sử dụng trong các ứng dụng này, chúng có thể rất tuyệt! Tôi khuyến khích bạn tra cứu chúng trong tài liệu và nếu chúng không tồn tại, hãy thử áp dụng chúng.

Bài tập 2.1

Đối với mỗi tình huống sau đây, hãy nêu rõ đó là ví dụ về học tập có giám sát hay không giám sát. Giải thích câu trả lời của bạn. Trong trường hợp không rõ ràng, hãy chọn một và giải thích lý do tại sao bạn chọn nó.

Một. Hệ thống giới thiệu trên mạng xã hội giới thiệu bạn bè tiềm năng cho người dùng b. Một hệ thống trong một trang tin tức chia tin tức thành các chủ đề c. Tính năng tự động hoàn thành câu của Google d. Hệ thống khuyến nghị của nhà bán lẻ trực tuyến đề xuất cho người dùng những gì nên mua dựa trên lịch sử mua hàng trước đây của họ e. Một hệ thống trong công ty thẻ tín dụng có chức năng ghi lại các giao dịch gian lận

GIẢI PHÁP

Tùy thuộc vào cách bạn diễn giải vấn đề và tập dữ liệu, mỗi dữ liệu này có thể được coi là một ví dụ về học có giám sát hoặc không giám sát. Hoàn toàn có thể (và được khuyến khích!) Có những câu trả lời khác nhau, miễn là lý do đằng sau chúng là chính xác.

Một. Đây là một ví dụ về cả học tập có giám sát và không giám sát. Học có giám sát: đối với một người dùng cụ thể, chúng tôi có thể xây dựng mô hình phân loại trong đó nhãn của mọi người dùng khác là tích cực nếu họ là bạn bè tiềm năng và tiêu cực nếu họ không phải là bạn bè tiềm năng.

b. Đây cũng là một ví dụ về cả học tập có giám sát và không giám sát. Học có giám sát: mô hình phân loại trong đó nhãn của mỗi bài báo là chủ đề, ví dụ: chính trị, thể thao hoặc khoa học. Học không giám sát: chúng tôi có thể phân cụm các bài viết rồi kiểm tra thủ công xem các chủ đề trong mỗi cụm có giống nhau hay không. Nếu đúng như vậy thì chúng ta có thể gán nhãn thủ công cho từng cụm theo chủ đề phổ biến nhất. Có một số kỹ thuật học không giám sát nâng cao hơn như phân bổ Dirichlet tiềm ẩn mà bạn có thể tìm hiểu trong video này: <https://www.youtube.com/watch?v=T05t-SqKArY>.

c. Đây là một nhiệm vụ học tập có giám sát. Chúng ta có thể xây dựng một mô hình phân loại trong đó các đặc điểm là những từ cuối cùng mà người dùng đã nhập và nhãn là từ tiếp theo họ sẽ nhập. Theo cách đó, dự đoán của mô hình chính là từ mà chúng tôi sẽ gợi ý cho người dùng.

d. Điều này tương tự như a) và nó có thể được coi là vấn đề học có giám sát hoặc không có giám sát. Học có giám sát: đối với một người dùng cụ thể, chúng tôi có thể xây dựng mô hình phân loại cho tất cả các sản phẩm, trong đó đối với mỗi sản phẩm, chúng tôi dự đoán liệu người dùng có mua sản phẩm đó hay không. Chúng tôi cũng có thể xây dựng mô hình hồi quy trong đó chúng tôi dự đoán số tiền người dùng sẽ chi cho sản phẩm cụ thể đó. Học tập không giám sát: chúng tôi có thể phân cụm người dùng. Nếu người dùng đã mua một sản phẩm, chúng tôi có thể giới thiệu sản phẩm đó cho những người dùng khác trong cụm. Chúng tôi cũng có thể phân cụm các sản phẩm và nếu người dùng đã mua một sản phẩm, chúng tôi sẽ đề xuất các sản phẩm trong cùng một cụm.

đ. Đây là một nhiệm vụ học tập có giám sát. Chúng ta có thể xây dựng mô hình phân loại dự đoán một giao dịch nào đó có gian lận hay không dựa trên đặc điểm của giao dịch đó. Nó cũng có thể được coi là một nhiệm vụ học tập không giám sát, trong đó chúng tôi phân cụm các giao dịch và những giao dịch còn lại ở dạng ngoại lệ có nguy cơ gian lận cao hơn.

Bài tập 2.2

Đối với mỗi ứng dụng sau đây của học máy, bạn sẽ sử dụng hồi quy hoặc phân loại để giải quyết nó? Giải thích câu trả lời của bạn. Trong trường hợp không rõ ràng, hãy chọn một và giải thích lý do tại sao bạn chọn nó.

Một. Một cửa hàng trực tuyến dự đoán số tiền người dùng sẽ chi tiêu trên trang web của họ b. Trợ lý giọng nói giải mã giọng nói và chuyển nó thành văn bản c. Bán hoặc mua cổ phiếu từ một công ty cụ thể d. YouTube đề xuất video cho người dùng

GIẢI PHÁP

Một. Hồi quy, bởi vì chúng tôi đang cố gắng dự đoán số tiền mà người dùng chi tiêu và đây là một tính năng bằng số. b. Phân loại, bởi vì chúng tôi đang cố gắng dự đoán xem câu mà người dùng đã nói có hướng đến Alexa hay không và đây là một tính năng phân loại. c. Đây có thể là hồi quy hoặc phân loại. Nếu chúng ta đang cố gắng dự đoán lợi nhuận kỳ vọng hoặc rủi ro kỳ vọng để giúp chúng ta đưa ra quyết định thì đó là sự hồi quy. Nếu chúng ta đang cố gắng dự đoán liệu chúng ta có nên mua cổ phiếu hay không thì đó là sự phân loại. d. Điều này một lần nữa có thể là hồi quy hoặc phân loại. Nếu chúng tôi đang cố gắng dự đoán người dùng sẽ dành bao nhiêu thời gian để xem video để giới thiệu video đó thì đó là sự hồi quy. Nếu chúng tôi đang cố gắng dự đoán liệu người dùng có xem video hay không thì đó là phân loại.

Bài tập 2.3

Nhiệm vụ của bạn là chế tạo một chiếc ô tô tự lái. Đưa ra ít nhất ba ví dụ về các vấn đề về máy học mà bạn sẽ phải giải quyết để xây dựng nó. Trong mỗi ví dụ, hãy giải thích xem bạn đang sử dụng phương pháp học có giám sát/không giám sát và nếu được giám sát thì bạn đang sử dụng phương pháp hồi quy hay phân loại. Nếu bạn đang sử dụng các loại máy học khác, hãy giải thích loại nào và tại sao.

GIẢI PHÁP

- Một mô hình phân loại, dựa trên hình ảnh, xác định xem có người đi bộ, biển báo dừng, làn đường, xe ô tô khác, v.v. Đây là một lĩnh vực rộng lớn của máy học được gọi là thị giác máy tính, tôi rất khuyến khích bạn khám phá thêm! Một mô hình phân loại tương tự như mô hình trước, xác định những vật thể nào ở xung quanh ô tô dựa trên tín hiệu từ tất cả các cảm biến khác nhau trong ô tô (lidar, v.v.). Một mô hình học máy tìm ra con đường gần nhất đến đích mong muốn của chúng ta. Đây không phải là học tập có giám sát hoặc không giám sát chính xác. Có một số thuật toán trí tuệ nhân tạo cổ điển hơn như tìm kiếm A* (A-star) có thể được sử dụng ở đây.
-

Chương 3: Vẽ đường gần các điểm: Hồi quy tuyến tính

Bài tập 3.1

Một trang web đã đào tạo mô hình hồi quy tuyến tính để dự đoán số phút mà người dùng sẽ dành cho trang web. Công thức họ thu được là

$$\hat{t} = 0.8d + 0.5m + 0.5y + 0.2a + 1.5$$

trong đó $\hat{t}$ là thời gian dự đoán tính bằng phút và d, m, y và a là các biến chỉ báo (cụ thể là chúng chỉ lấy giá trị 0 hoặc 1) được xác định như sau:

- d là biến cho biết người dùng có sử dụng máy tính để bàn hay không. m là biến cho biết người dùng có sử dụng thiết bị di động hay không. y là biến cho biết người dùng có còn trẻ (dưới 21 tuổi) hay không. a là biến cho biết người dùng có phải là người lớn (21 tuổi trở lên) hay không.

Ví dụ: Nếu người dùng 30 tuổi và sử dụng máy tính để bàn thì $d = 1, m = 0, y = 0$ và $a = 1$.

Nếu một người dùng 45 tuổi xem trang web từ điện thoại của họ, họ sẽ dành bao nhiêu thời gian trên trang web?

GIẢI PHÁP

Trong trường hợp này, giá trị của các biến như sau:

- $d = 0$ vì người dùng không sử dụng máy tính để bàn. $m = 1$ vì người dùng sử dụng thiết bị di động.
- $y = 0$ vì người dùng không dưới 21 tuổi. $a = 1$ vì người dùng trên 21 tuổi.

Khi chúng ta thế chúng vào công thức, chúng ta nhận được

$$\hat{t} = 0.8 \cdot 0 + 0.5 \cdot 1 + 0.5 \cdot 0 + 0.2 \cdot 1 + 1.5 = 2.2.$$

Điều này có nghĩa là mô hình dự đoán rằng người dùng này sẽ dành 2,2 phút cho

Bài tập 3.2

Hãy tưởng tượng rằng chúng tôi đã đào tạo mô hình hồi quy tuyến tính trong tập dữ liệu y tế. Mô hình dự đoán thời gian sống dự kiến của bệnh nhân. Đối với mỗi tính năng trong tập dữ liệu của chúng tôi, mô hình sẽ chỉ định trọng số.

a) Đối với các đại lượng sau đây, hãy cho biết trọng lượng gắn liền với đại lượng này là số dương, số âm hay bằng 0. Lưu ý: nếu bạn tin rằng trọng số là một con số rất nhỏ, dù dương hay âm, bạn có thể nói số không.

1. Số giờ tập thể dục của bệnh nhân mỗi tuần
2. Số điều thuốc bệnh nhân hút mỗi tuần
3. Số thành viên trong gia đình có vấn đề về tim
4. Số anh chị em của bệnh nhân
5. Bệnh nhân có nhập viện hay không

b) Mô hình cũng có độ lệch. Bạn nghĩ độ lệch là dương, âm hay bằng 0?

GIẢI PHÁP

a) Chúng ta sẽ đưa ra một số khái quát dựa trên kiến thức y học tổng quát. Đối với một bệnh nhân cụ thể, những điều sau đây không nhất thiết đúng, nhưng chúng tôi sẽ giả định rằng chúng đúng với dân số nói chung:

1. Một bệnh nhân tập thể dục nhiều sẽ sống lâu hơn một bệnh nhân tương tự không tập luyện. Vì vậy, trọng số này phải là một số dương.
2. Một bệnh nhân hút nhiều thuốc lá mỗi tuần dự kiến sẽ sống ngắn hơn so với một bệnh nhân tương tự không hút thuốc. Vì vậy, trọng số này phải là số âm.
3. Một bệnh nhân có nhiều thành viên trong gia đình mắc bệnh tim thì khả năng mắc các vấn đề về tim cao hơn và do đó họ được cho là sẽ sống ngắn hơn so với một bệnh nhân tương tự không mắc bệnh này. Vì vậy, trọng số này phải là số âm.
4. Số lượng anh chị em có xu hướng độc lập với thời gian sống dự kiến, vì vậy chúng tôi kỳ vọng trọng số này sẽ là một con số rất nhỏ hoặc bằng 0.

5. Bệnh nhân đã từng nhập viện trước đây rất có thể đã có vấn đề về sức khỏe trước đó. Do đó, thời gian sống dự kiến của họ ngắn hơn so với một bệnh nhân tương tự chưa từng nhập viện trước đó. Vì vậy, trọng số này phải là số âm. Tất nhiên, việc nhập viện có thể vì một lý do không ảnh hưởng đến tuổi thọ dự kiến (chẳng hạn như gãy chân), nhưng nhìn chung, chúng ta có thể nói rằng nếu một bệnh nhân đã từng đến bệnh viện trước đây thì họ có xác suất gặp vấn đề về sức khỏe cao hơn.

b) Độ lệch là dự đoán cho một bệnh nhân mà mọi đặc điểm đều bằng 0 (tức là bệnh nhân không hút thuốc, không tập thể dục, không có thành viên nào trong gia đình mắc bệnh tim, không có anh chị em ruột và chưa bao giờ nhập viện). Vì bệnh nhân này được kỳ vọng sẽ sống được một số năm dương nên độ lệch của mô hình này phải là một số dương.

Bài tập 3.3

Sau đây là tập dữ liệu về các ngôi nhà có kích thước (tính bằng feet vuông) và giá (tính bằng đô la).

	(Các) kích thước	Giải thưởng (p)
Nhà 1	100	200
Nhà 2	200	475
Nhà 3	200	400
Nhà 4	250	520
Nhà 5	325	735

Giả sử chúng ta đã huấn luyện mô hình trong đó dự đoán giá căn nhà dựa trên kích thước như sau:

$$\hat{p} = 2s + 50$$

Một. Tính toán các dự đoán mà mô hình này đưa ra trên tập dữ liệu. b. Tính sai số tuyệt đối trung bình của mô hình này. c. Tính sai số bình phương trung bình gốc của mô hình này.

GIẢI PHÁP

Một. Giá dự đoán dựa trên mô hình như sau:

- o Nhà 1: $= 2 \cdot \hat{p}100 + 50 = 250$ Nhà 2: $= 2$
- o $\cdot 200 + 50 = 450$ Nhà 3: $= 2 \cdot 200 +$
- o $50 = 450$ Nhà 4: $= 2 \cdot 250 + 50 = 550$
- o Nhà 5: $= 2 \cdot 325 + 50 = 700$
- o $\hat{p}$

b. Sai số tuyệt đối trung bình là

$$\frac{1}{5} (|200 - 250| + |475 - 450| + |400 - 450| + |520 - 550| + |735 - 700|) \\ = \frac{1}{5} (50 + 25 + 50 + 30 + 35) = 38.$$

c. Sai số bình phương trung bình là

$$\frac{1}{5} ((200 - 250)^2 + (475 - 450)^2 + (400 - 450)^2 + (520 - 550)^2 + (735 - 700)^2) \\ = \frac{1}{5} (2500 + 625 + 2500 + 900 + 1225) = 1550.$$

Do đó, sai số bình phương trung bình gốc là $\sqrt{1550} = 39,37$.

Bài tập 3.4

Mục tiêu của chúng ta là di chuyển đường thẳng có phương trình $\hat{y} = 2x + 3$ đến gần điểm $(x, y) = (5, 15)$ bằng cách sử dụng các thủ thuật đã học trong chương này. Đối với hai bài toán sau, sử dụng tốc độ học $\eta = 0,01$.

Một. Áp dụng thủ thuật tuyệt đối để sửa đổi dòng trên để gần điểm hơn. b. Áp dụng thủ thuật vuông để sửa đổi dòng trên để gần điểm hơn.

GIẢI PHÁP

The dự đoán mà mô hình này đưa ra tại thời điểm đó là $\hat{y} = 2 \cdot 5 + 3 = 13$.

Một. Vì dự đoán là 13, nhỏ hơn nhãn 15 nên điểm nằm bên dưới đường kẻ.

Trong mô hình này, độ dốc là $m = 2$ và phần chặn y là $b = 3$. Bí quyết tuyệt đối là thêm $x\eta = 5 \cdot 0,01 = 0,05$ vào độ dốc và $\eta = 0,01$ đến phần chặn y -, do đó thu được mô hình có phương trình

$$\hat{y} = 2.05x + 3.01.$$

b. Thủ thuật hình vuông liên quan đến việc thêm $(y - \hat{y})x\eta = (15 - 13) \cdot 5 \cdot 0,01 = 0,1$ vào độ dốc và $(y - \hat{y})\eta = (15 - 13) \cdot 0,01 = 0,02$ tới điểm chặn y -, do đó thu được mô hình có phương trình

$$\hat{y} = 2.1x + 3.02.$$

Chương 4: Tối ưu hóa quy trình đào tạo: Underfitting, Overfitting, testing và Regularization

Bài tập 4.1

Chúng tôi đã đào tạo bốn mô hình trong cùng một tập dữ liệu với các siêu tham số khác nhau. Trong bảng sau, chúng tôi đã ghi lại các lỗi huấn luyện và kiểm tra cho từng mô hình.

Mô hình	Lỗi đào tạo	Lỗi kiểm tra
1	0.1	1.8
2	0.4	1.2
3	0.6	0.8
4	1.9	2.3

Một. Bạn sẽ chọn mô hình nào cho tập dữ liệu này? b. Mô hình nào có vẻ không phù hợp với dữ liệu? c. Mô hình nào có vẻ như phù hợp quá mức với dữ liệu?

GIẢI PHÁP

Một. Mô hình tốt nhất là mô hình có sai số kiểm tra nhỏ nhất, đó là mô hình 3. b. Mô hình 4 có vẻ như không phù hợp vì nó có nhiều lỗi đào tạo và kiểm tra. c. Mô hình 1 và 2 trông giống như đang trang bị quá mức vì chúng có lỗi huấn luyện nhỏ nhưng lỗi kiểm tra lớn.

Bài tập 4.2

Chúng tôi được cung cấp bộ dữ liệu sau:

x	y
1	2
2	2.5
3	6
4	14.5
5	34

We train the polynomial regression model that predicts the value of y as $\hat{y}$, where

$$\hat{y} = 2x^2 - 5x + 4.$$

If the regularization parameter is $\lambda = 0.1$ and the error function we've used to train this dataset is the mean absolute value (MAE), determine the following:

- a. The lasso regression error of our model (using the L1-norm)
- b. The ridge regression error of our model (using the L2-norm)

SOLUTION

First we need to find the predictions to calculate the mean absolute error of the model. In the following table, we can find the prediction calculated by the formula $\hat{y} = 2x^2 - 5x + 4$, and the absolute value of the difference between the prediction and the label $|y - \hat{y}|$.

x	y	$\hat{y}$	$ y - \hat{y} $
1	2	1	1
2	2.5	2	0.5
3	6	7	1
4	14.5	16	1.5
5	34	29	5

Thus, the mean absolute error is the average of the numbers in the fourth row, namely

$$\frac{1}{5}(1 + 0.5 + 1 + 1.5 + 5) = 1.8.$$

Một. Đầu tiên chúng ta cần tìm chuẩn L1 của đa thức. Đây là tổng giá trị tuyệt đối của các hệ số không cố định, cụ thể là $|2| + |-5| = 7$. Để tìm chi phí chính quy hóa L1 của mô hình, chúng ta cộng sai số tuyệt đối trung bình và định mức L1 nhân với tham số chính quy hóa, để thu được $1,8 + 0,1 \cdot 7 = 2,5$.

b. Tương tự, ta tìm chuẩn L1 của đa thức bằng cách cộng bình phương các hệ số không cố định để được $2^2 + (-5)^2 = 29$. Như trước, chi phí chính quy hóa L2 của mô hình là $1,8 + 0,1 \cdot 29 = 4,7$.

Chương 5: Dùng đường kẻ để phân chia điểm: Thuật toán perceptron

Bài tập 5.1

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán (D)
Bệnh nhân 1		X	X	X	Bệnh
Bệnh nhân 2	X	X		X	Bệnh
Bệnh nhân 3	X		X	X	Bệnh
Bệnh nhân 4	X	X	X		Bệnh
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

Xây dựng mô hình perceptron để phân loại tập dữ liệu này.

gợi ý Bạn có thể sử dụng thuật toán perceptron, nhưng bạn có thể nhắm đến một mô hình perceptron tốt hoạt động được.

GIẢI PHÁP

Nếu chúng tôi đếm xem mỗi bệnh nhân có bao nhiêu triệu chứng, chúng tôi nhận thấy rằng những bệnh nhân bị bệnh có ba triệu chứng trở lên, trong khi những bệnh nhân khỏe mạnh có hai triệu chứng trở xuống. Do đó, mô hình sau đây có tác dụng dự đoán chẩn đoán D :

$$\hat{D} = \text{step}(C + F + B + T - 2.5)$$

Bài tập 5.2

Xét mô hình perceptron gán cho điểm (x_1, x_2) dự đoán $\hat{y} =$

$\text{bước}(2x_1 + 3x_2 - 4)$. Mô hình này có một đường biên với phương trình $2x_1 + 3x_2 - 4 = 0$. Ta có điểm $p = (1, 1)$ có nhãn 0.

Một. Xác minh rằng điểm p bị mô hình phân loại sai. b. Tính sai số perceptron mà mô hình tạo ra tại điểm p . c. Sử dụng thủ thuật perceptron để thu được mô hình mới vẫn phân loại sai p nhưng tạo ra lỗi nhỏ hơn. Bạn có thể sử dụng $\eta = 0,01$ làm tốc độ học tập. d. Tìm dự đoán mà mô hình mới đưa ra tại điểm p và xác minh rằng sai số perceptron thu được nhỏ hơn sai số ban đầu.

GIẢI PHÁP

Một. Dự đoán cho điểm p là

$$\hat{y} = \text{step}(2x_1 + 3x_2 - 4) = \text{step}(2 \cdot 1 + 3 \cdot 1 - 4) = \text{step}(1) = 1.$$

Vì nhãn của điểm là 0 nên điểm đó bị phân loại sai.

b. Lỗi perceptron là giá trị tuyệt đối của điểm. Tỉ số là $2x_1 + 3x_2 -$

$$4 = 2 \cdot 1 + 3 \cdot 1 - 4 = 1, \text{ vậy lỗi perceptron là } 1.$$

c. Trọng số của mô hình là 2, 3 và -4 và tọa độ của điểm là $(1, 1)$. Thủ thuật perceptron thực hiện như sau: Thay thế 2 bằng $2 - 0,01 \cdot 1 = 1,99$ Thay thế 3 bằng $3 - 0,01 \cdot 1 = 2,99$ Thay thế -4 bằng $-1 - 0,01 \cdot 1 = -4,01$

◦

◦

Như vậy, mô hình mới là mô hình đưa ra dự đoán $\hat{y} = \text{bước}(1,99x_1 + 2,99x_2 - 4,01)$.

- d. Note that at our point, the new prediction is $\hat{y} = \text{step}(1.99x_1 + 2.99x_2 - 4.01) = \text{step}(0.97) = 0$, which means the model still misclassifies the point. However, the new perceptron error is $|1.99 \cdot 1 + 2.99 \cdot 1 - 4.01| = 0.97$, which is smaller than 1, the previous error.

Exercise 5.3

Perceptrons are particularly useful for building logical gates such as AND and OR.

- a. Build a perceptron that models the AND gate. In other words, build a perceptron to fit the following dataset (where x_1 , x_2 are the features and y is the label):

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- b. Similarly, build a perceptron that models the OR gate, given by the following dataset:

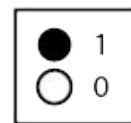
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

- c. Show that there is no perceptron that models the XOR gate, given by the following dataset:

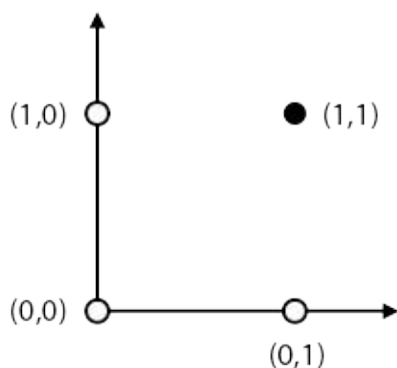
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

SOLUTION

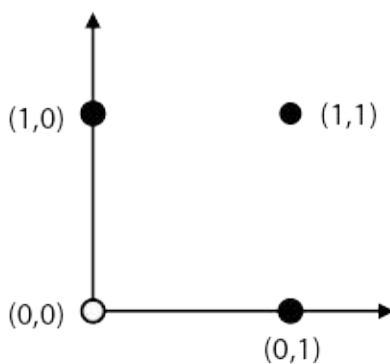
For simplicity, we plot the data points in the figure that follows.



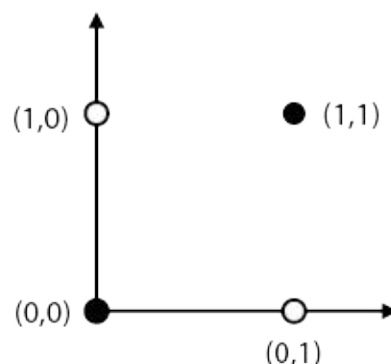
AND



OR



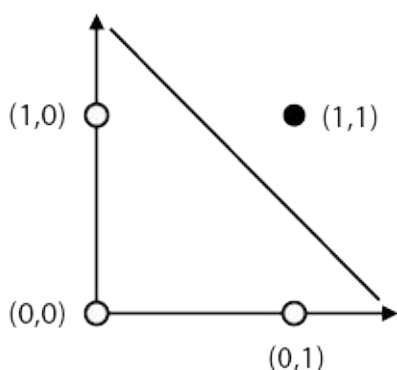
XOR



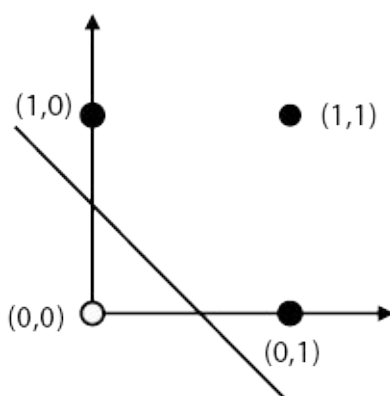
Lưu ý rằng bộ phân loại perceptron chính xác là một đường phân chia các chấm đen và trắng trong các ô trên.

Đối với bộ dữ liệu AND và OR, chúng ta có thể dễ dàng phân chia các điểm đen và trắng bằng một đường thẳng, như được thấy tiếp theo.

AND



OR



Một. Nhiều phương trình hoạt động cho đường phân cách tập dữ liệu AND. Ta chọn đoạn thẳng có phương trình $x_1 + x_2 - 1.5$. Vì vậy, perceptron phân loại này

tập dữ liệu đưa ra dự đoán $\hat{y} = \text{bước}(x_1 + x_2 - 1.5)$.

b. Tương tự, nhiều phương trình hoạt động với tập dữ liệu OR và chúng tôi chọn dòng có phương trình $x_1 + x_2 - 0.5$. Phương trình dự đoán là $\hat{y} = \text{bước}(x_1 + x_2 - 0.5)$.

c. Lưu ý rằng tập dữ liệu cho XOR không thể phân tách bằng một dòng duy nhất. Do đó, không có mô hình perceptron nào hoàn toàn phù hợp với tập dữ liệu XOR. Tuy nhiên, sự kết hợp của các perceptron có thể tách tập dữ liệu này. Chúng còn được gọi là perceptron đa lớp, hay mạng lưới thần kinh, và chúng ta sẽ thấy chúng trong chương 10. Nếu bạn tò mò, hãy xem bài tập 10.2.

Chương 6: Cách tiếp cận liên tục để phân chia điểm: Phân loại logistic

Bài tập 6.1

Một nha sĩ đã đào tạo một bộ phân loại logistic trên tập dữ liệu của bệnh nhân để dự đoán xem họ có bị sâu răng hay không. Mô hình đã xác định được xác suất để một bệnh nhân bị sâu răng là

$$\sigma(d + 0.5c - 0.8),$$

Ở đây

- d là biến cho biết trước đây bệnh nhân có bị sâu răng nữa hay không và c là biến cho biết bệnh nhân có ăn kẹo hay không.
-

Ví dụ: nếu một bệnh nhân ăn kẹo thì $c = 1$, còn nếu không thì $c = 0$. Xác suất để một bệnh nhân ăn kẹo và đã được điều trị răng sâu năm ngoái sẽ bị sâu răng hôm nay là bao nhiêu?

GIẢI PHÁP

Nếu bệnh nhân ăn kẹo thì $c = 1$. Nếu bệnh nhân đã được điều trị sâu răng vào năm ngoái thì $d = 1$. Như vậy, theo mô hình, xác suất để bệnh nhân bị sâu răng là

$$\sigma(1 + 0.5 \cdot 1 - 0.8) = \sigma(0.7) = 0.668.$$

Bài tập 6.2

Hãy xem xét trình phân loại logistic gán cho điểm (x_1, x_2) dự đoán $\hat{y} = \sigma(2x_1 + 3x_2 - 4)$ và điểm $p = (1, 1)$ có nhãn 0.

Một. Tính toán dự đoán $\hat{y}$ mà mô hình đưa ra cho điểm p . b. Tính log loss mà mô hình tạo ra tại điểm p .

c. Sử dụng thủ thuật logistic để có được mô hình mới tạo ra tổn thất log nhỏ hơn. Bạn có thể sử dụng $\eta = 0,1$ làm tốc độ học tập. d. Tìm dự đoán mà mô hình mới đưa ra tại điểm p và xác minh rằng log loss thu được nhỏ hơn ban đầu.

GIẢI PHÁP

Một. Dự đoán là $\hat{y} = \sigma(2 \cdot 1 + 3 \cdot 1 - 4) = \sigma(1) = 0,731$ b. Mất nhật ký là

$$\begin{aligned} \log \text{loss} &= -y \ln (\hat{y}) - (1 - y) \ln (1 - \hat{y}) \\ &= -0 \ln (0.731) - (1 - 0) \ln (1 - 0.731) \\ &= 1.313. \end{aligned}$$

c. Nhớ lại thủ thuật perceptron cho mô hình hồi quy logistic với dự đoán $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$ cung cấp cho chúng ta các trọng số mới sau:

- $w_1' = w_1 + \eta(y - \hat{y})x_1$ cho $i = 1, 2$
- $b' = b + \eta(y - \hat{y})$ cho $i = 1, 2$

Đây là những giá trị để thế vào các công thức trước đó:

- $y = 0$
- $\hat{y} = 0.731$
- $w_1 = 2$
- $w_2 = 3$
- $b = -4$
- $\eta = 0.1$
- $x_1 = 1$
- $x_2 = 1$

Chúng tôi thu được các trọng số mới sau đây cho bộ phân loại của mình:

- $w_1' = 2 + 0.1 \cdot (0 - 0.731) \cdot 1 = 1.9269$
- $w_2' = 3 + 0.1 \cdot (0 - 0.731) \cdot 1 = 2.9269$
- $b = -4 + 0.1 \cdot (0 - 0.731) = -4.0731$

Do đó, bộ phân loại mới của chúng tôi là bộ đưa ra dự đoán $\hat{y} = \sigma(1.9269x_1 + 2.9269x_2 - 4.0731)$.

Dự đoán tại điểm p là $\hat{y} = \sigma(1.9269 \cdot 1 + 2.9269 \cdot 1 - 4.0731) = 0.686$. Lưu ý rằng vì nhãn là 0 nên dự đoán đã được cải thiện từ 0,731 ban đầu lên 0,686 thực tế.

d. Log loss cho dự đoán này là $-y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}) = -0 \ln(0.686) - (1 - 0) \ln(1 - 0.686) = 1,158$. Lưu ý rằng giá trị này nhỏ hơn mức mất log ban đầu là 1,313.

Bài tập 6.3

Sử dụng mô hình đầu tiên trong bài tập 6.2, hãy xây dựng một điểm có giá trị dự đoán là 0,8.

gợi ý Trước tiên hãy tìm điểm sẽ đưa ra dự đoán là 0,8 và nhớ lại rằng dự đoán đó là $\hat{y} = \sigma(\text{điểm})$.

GIẢI PHÁP

Đầu tiên chúng ta cần tìm số điểm sao cho $\sigma(\text{điểm}) = 0,8$. Điều này tương đương với

$$\frac{e^{\text{score}}}{1 + e^{\text{score}}} = 0.8$$

$$e^{\text{score}} = 0.8(1 + e^{\text{score}})$$

$$e^{\text{score}} = 0.8 + 0.8 \cdot e^{\text{score}}$$

$$0.2e^{\text{score}} = 0.8$$

$$e^{\text{score}} = 4$$

$$\text{score} = \ln(4) = 1.386$$

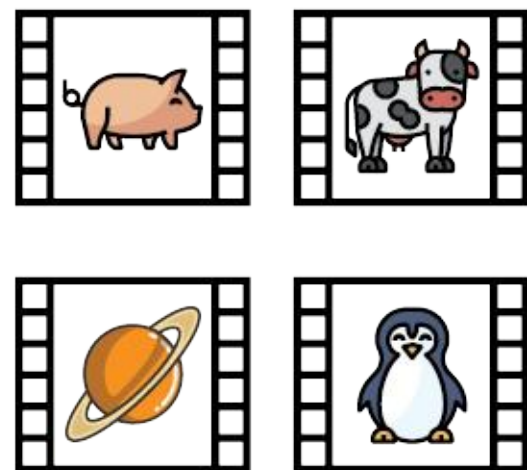
Nhớ lại đối với điểm (x_1, x_2) , điểm là $2x_1 + 3x_2 - 4$. Nhiều điểm (x_1, x_2) thỏa mãn rằng số điểm là 1.386, nhưng cụ thể, hãy chọn một trong đó $x_2 = 0$ để thuận tiện. Ta cần giải phương trình $2x_1 + 3 \cdot 0 - 4 = 1.386$, có nghiệm là $x_1 = 2.693$. Do đó, điểm đưa ra dự đoán là 0,8 là điểm $(2,693, 0)$.

Chương 7: Bạn đo lường các mô hình phân loại như thế nào? Độ chính xác và những người bạn của nó

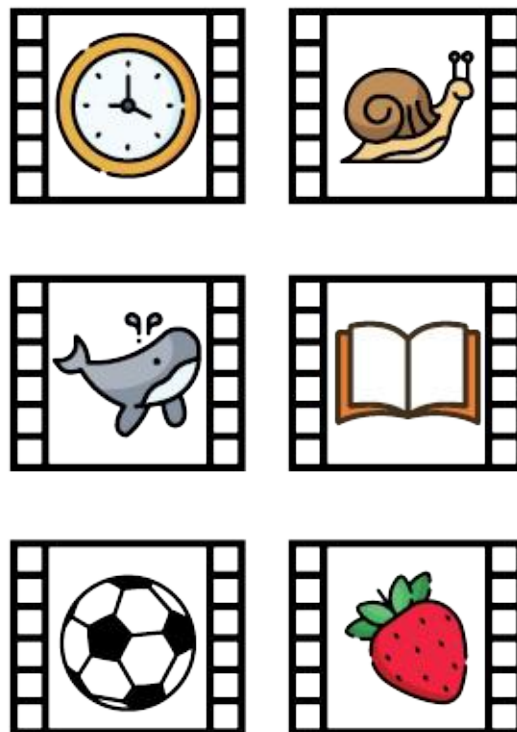
Bài tập 7.1

Một trang web video đã xác nhận rằng một người dùng cụ thể thích video về động vật và hoàn toàn không thích gì khác. Trong hình tiếp theo, chúng ta có thể thấy các đề xuất mà người dùng này nhận được khi đăng nhập vào trang web.

Recommended



Not recommended



Nếu điều này là tất cả dữ liệu chúng ta có trên mô hình, hãy trả lời các câu hỏi sau:

Một. Độ chính xác của mô hình là gì? b. Việc thu hồi mô hình là gì? c. Độ chính xác của mô hình là gì? d. Điểm F_1 của mô hình là bao nhiêu? đ. Bạn có cho rằng đây là một mô hình khuyến nghị tốt không?

GIẢI PHÁP

Đầu tiên, hãy viết ma trận nhầm lẫn.Trong trường hợp này, chúng tôi gắn nhãn những video nói về động vật là *tích cực* và những video được đề xuất là *được dự đoán là tích cực*.

- Có bốn video được đề xuất.Trong số đó, ba cuốn là về động vật, có nghĩa là chúng là những gợi ý tốt.Câu chuyện còn lại không nói về động vật nên nó là một kết quả dương tính giả. Có sáu video không được khuyến khích.Trong số đó, có hai cuốn về động vật, lẽ ra phải được khuyến khích.Vì vậy, chúng là những âm tính giả.Bốn cuốn còn lại không nói về động vật nên việc không giới thiệu chúng là đúng đắn.

Do đó, ma trận nhầm lẫn như sau:

	Dự đoán dương tính (được khuyến nghị)	Dự đoán là âm tính (không khuyến khích)
Tích cực (về động vật)	3	2
Tiêu cực (không phải về động vật)	1	4

Bây giờ chúng ta có thể tính toán các số liệu.

Một. $Accuracy = \frac{7}{10} = 0.7$

b. $Recall = \frac{3}{5} = 0.6$

c. $Precision = \frac{3}{4} = 0.75$

d. $F_1\text{-score} = \frac{2\left(\frac{3}{4}\right)\left(\frac{3}{5}\right)}{\frac{3}{4} + \frac{3}{5}} = \frac{2}{3} = 0.67$

đ. Đây là một câu trả lời chủ quan.Một mô hình y tế với những số liệu này có thể không đủ tốt.Tuy nhiên, nếu một mô hình đề xuất có độ chính xác và khả năng thu hồi ở mức khá thì đó được coi là một mô hình tốt vì việc mắc một vài lỗi trong mô hình đề xuất không quá nghiêm trọng.

Bài tập 7.2

Tìm độ nhảy và độ đặc hiệu của mô hình y tế với ma trận nhầm lẫn sau:

	Dự đoán bị bệnh	Dự đoán là khỏe mạnh
Bệnh	120	22
Khỏe mạnh	63	795

GIẢI PHÁP

Độ nhảy là số người bị bệnh được dự đoán chính xác chia cho tổng số người bị bệnh.Đây là $120/142=0,845$.

Độ đặc hiệu là số người khỏe mạnh được dự đoán chính xác chia cho tổng số người khỏe mạnh.Đây là $795/858=0,927$.

Bài tập 7.3

Đối với các mô hình sau, hãy xác định lỗi nào nghiêm trọng hơn, dương tính giả hay âm tính giả.Dựa vào đó, xác định chỉ số nào trong hai chỉ số, độ chính xác hay mức thu hồi, chúng ta nên nhấn mạnh khi đánh giá từng mô hình.

1. Hệ thống đề xuất phim dự đoán liệu người dùng có xem phim hay không. 2. Một mô hình phát hiện hình ảnh được sử dụng trong ô tô tự lái để phát hiện xem hình ảnh có chứa người đi bộ hay không. 3. Trợ lý giọng nói tại nhà dự đoán liệu người dùng có ra lệnh hay không.

GIẢI PHÁP

lưu ý Trong tất cả các mô hình sau, âm tính giả và dương tính giả đều không tốt và chúng tôi muốn tránh cả hai mô hình đó.Tuy nhiên, chúng tôi đưa ra một lập luận cho rằng một trong hai điều đó tệ hơn. Đây đều là những câu hỏi mang tính khái niệm, vì vậy nếu bạn có ý kiến khác, miễn là bạn có thể tranh luận tốt thì nó có giá trị!Đây là những cuộc thảo luận nảy sinh trong một nhóm các nhà khoa học dữ liệu và điều quan trọng là phải có những ý kiến và lập luận lành mạnh ủng hộ từng quan điểm.

1. Trong mô hình này, chúng tôi gắn nhãn những bộ phim mà người dùng muốn xem là phim tích cực. Kết quả dương tính giả xảy ra bất cứ khi nào chúng tôi đề xuất một bộ phim mà người dùng không muốn xem. Kết quả âm tính giả xảy ra bất cứ khi nào có một bộ phim mà người dùng muốn xem nhưng chúng tôi không khuyến khích điều đó. Điều nào tệ hơn, âm tính giả hay dương tính giả? Vì trang chủ hiển thị nhiều đề xuất và người dùng bỏ qua hầu hết trong số đó nên mô hình này có nhiều tiêu cực sai, không ảnh hưởng nhiều đến trải nghiệm người dùng. Tuy nhiên, nếu có một bộ phim hay mà người dùng muốn xem thì điều quan trọng là phải giới thiệu bộ phim đó cho họ. Do đó, trong mô hình này, kết quả âm tính giả còn tệ hơn kết quả dương tính giả, vì vậy chúng ta nên đánh giá mô hình này bằng cách sử dụng **thu hồi**.

2. Trong mô hình này, chúng tôi coi sự tồn tại của người đi bộ là tích cực. Kết quả dương tính giả xảy ra khi không có người đi bộ nhưng xe lại cho rằng có người đi bộ. Kết quả âm tính giả xảy ra khi ô tô không phát hiện được người đi bộ ở phía trước ô tô. Trong trường hợp âm tính giả, xe có thể tông vào người đi bộ. Trong trường hợp dương tính giả, ô tô có thể phanh gấp một cách không cần thiết, điều này có thể dẫn đến tai nạn hoặc không. Mặc dù cả hai đều nghiêm trọng nhưng việc tông vào người đi bộ còn tệ hơn nhiều. Do đó, trong mô hình này, kết quả âm tính giả còn tệ hơn kết quả dương tính giả, vì vậy chúng ta nên đánh giá mô hình này bằng cách sử dụng **thu hồi**.

3. Trong mô hình này, chúng tôi gắn nhãn lệnh thoại là tích cực. Kết quả dương tính giả xảy ra khi người dùng không nói chuyện với trợ lý giọng nói nhưng trợ lý giọng nói lại phản hồi. Âm tính giả xảy ra khi người dùng đang nói chuyện với trợ lý giọng nói nhưng trợ lý giọng nói không phản hồi. Theo lựa chọn cá nhân, tôi thích phải lặp lại với trợ lý giọng nói của mình hơn là để cô ấy nói chuyện với tôi một cách bất ngờ. Do đó, trong mô hình này, kết quả dương tính giả còn tệ hơn kết quả âm tính giả, vì vậy chúng ta nên đánh giá mô hình này bằng **độ chính xác**.

Bài tập 7.4

Chúng ta được cung cấp các mô hình sau:

1. Mô hình ô tô tự lái để phát hiện người đi bộ dựa trên hình ảnh từ camera của ô tô
2. Mô hình y tế chẩn đoán căn bệnh chết người dựa trên các triệu chứng của bệnh nhân
3. Hệ thống đề xuất phim dựa trên các bộ phim trước đây của người dùng đã xem
4. Trợ lý giọng nói xác định xem người dùng có cần hỗ trợ khi ra lệnh bằng giọng nói hay không

5. Mô hình phát hiện thư rác xác định xem email có phải là thư rác hay không dựa trên các từ trong email

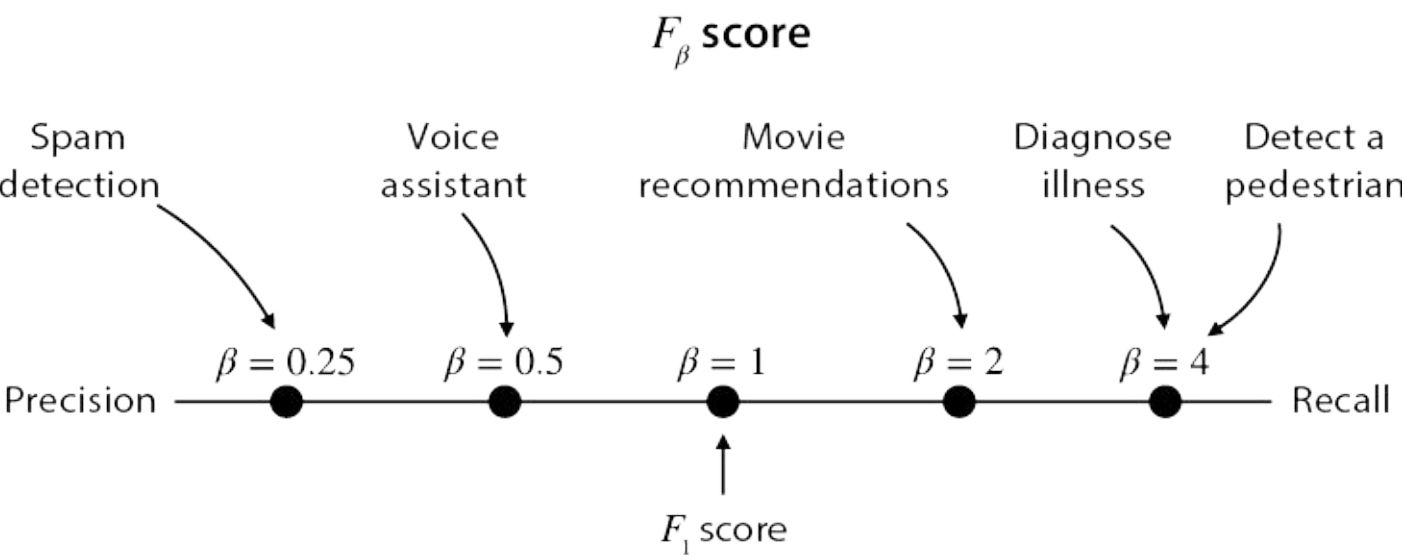
Chúng tôi được giao nhiệm vụ đánh giá các mô hình này bằng cách sử dụng điểm F_β . Tuy nhiên, chúng tôi chưa được cấp giá trị của β để sử dụng. Bạn sẽ sử dụng giá trị nào của β để đánh giá từng mô hình?

GIẢI PHÁP

Hãy nhớ rằng đối với các mô hình trong đó độ chính xác quan trọng hơn khả năng thu hồi, chúng tôi sử dụng điểm F_β với giá trị nhỏ là β . Ngược lại, đối với những mẫu xe có khả năng thu hồi quan trọng hơn độ chính xác, chúng tôi sử dụng điểm F_β với giá trị lớn là β .

lưu ý Nếu bạn có điểm khác với giải pháp này thì điều đó hoàn toàn ổn, miễn là bạn có lập luận cho thấy cái nào quan trọng hơn giữa độ chính xác và khả năng thu hồi cũng như giá trị β mà bạn chọn.

- Đối với xe tự lái và các mẫu xe y tế, việc thu hồi là cực kỳ quan trọng vì chúng tôi muốn có rất ít kết quả âm tính giả. Vì vậy, tôi sẽ sử dụng giá trị lớn β , chẳng hạn như 4. Đối với mô hình phát hiện thư rác, độ chính xác là quan trọng vì chúng tôi muốn có rất ít kết quả dương tính giả. Vì vậy, tôi sẽ sử dụng giá trị nhỏ β , chẳng hạn như 0,25. Đối với hệ thống khuyến nghị, việc thu hồi là quan trọng hơn (xem bài tập 7.3), mặc dù độ chính xác cũng rất quan trọng. Vì vậy, tôi sẽ sử dụng giá trị lớn β , chẳng hạn như 2. Đối với trợ lý giọng nói, độ chính xác là quan trọng hơn, mặc dù việc thu hồi cũng quan trọng (xem bài tập 7.3). Vì vậy, tôi sẽ sử dụng một giá trị nhỏ cho β , chẳng hạn như 0,5.
-



Chương 8: Sử dụng xác suất tối đa: Mô hình Bayes ngây thơ

Bài tập 8.1

Với mỗi cặp biến cố A và B, hãy xác định xem chúng độc lập hay phụ thuộc. Đối với (a) đến (d), hãy đưa ra sự chứng minh bằng toán học. Đối với (e) và (f) đưa ra lời giải thích bằng lời nói.

Ném ba đồng xu công bằng:

Một. A: Người đầu tiên rơi vào đầu. B: Con thứ ba rơi vào đuôi. b. A: Người đầu tiên rơi vào đầu. B: Có một số lẻ mặt ngửa trong ba lần ném.

Tung hai con xúc xắc:

c. A: Cái đầu tiên hiển thị số 1. B: Cái thứ hai hiển thị số 2. d. A: Cái đầu tiên hiển thị số 3. B: Cái thứ hai hiển thị giá trị cao hơn cái đầu tiên.

Đối với những điều sau đây, hãy đưa ra lời biện minh bằng lời nói. Giả sử rằng đối với bài toán này, chúng ta đang sống ở một nơi có các mùa.

đ. A: Ngoài trời đang mưa. B: Hôm nay là thứ Hai.
f. A: Ngoài trời đang mưa. B: Bây giờ là tháng Sáu.

GIẢI PHÁP

Một số điều sau đây có thể được suy luận bằng trực giác. Tuy nhiên, đôi khi trực giác không thành công khi xác định xem hai sự kiện có độc lập hay không. Vì lý do này, trừ khi các sự kiện rõ ràng là độc lập, chúng ta sẽ tiếp tục kiểm tra xem hai sự kiện A và B có độc lập hay không nếu $P(A \cap B) = P(A)P(B)$.

Một. Vì A và B tương ứng với việc tung các đồng xu khác nhau nên chúng là những sự kiện độc lập.

b. $P(A) = \frac{1}{2}$ bởi vì việc tung đồng xu công bằng sẽ dẫn đến hai tình huống có khả năng xảy ra như nhau. Để tính $P(B)$, chúng ta sẽ sử dụng “h” cho mặt ngửa và “t” cho mặt sấp. Bằng cách này, sự kiện “hth” tương ứng với lần tung đồng xu thứ nhất và thứ ba là mặt úp và lần thứ hai là mặt ngửa. Vì vậy, nếu chúng ta ném ba đồng xu thì tám đồng

các khả năng có khả năng xảy ra như nhau là {hhh, hht, hth, htt, thh, tht, tth, ttt}. bởi vì $P(B) = \frac{4}{8} = \frac{1}{2}$ tám khả năng có khả năng xảy ra như nhau (hhh, hht, hth, htt, thh, tht, tth, ttt), chỉ có bốn khả năng có số mặt ngửa là lẻ, cụ thể là {hhh, htt,

th, tth}. vì $P(A \cap B) = \frac{2}{8} = \frac{1}{4}$ hai khả năng xảy ra, chỉ có hai khả năng thỏa mãn là khả năng đầu tiên rơi vào mặt ngửa và có số lần mặt ngửa là lẻ,

cụ thể là {hhh, htt}. , do đó $P(A) \cdot P(B) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4} = P(A \cap B)$ và B độc lập. c. Vì A và B tương ứng với việc tung những viên xúc xắc khác nhau nên chúng là những sự kiện độc lập.

d. , bởi vì $P(A) = \frac{1}{2}$ nó tương ứng với việc tung một con xúc xắc và nhận được một con xúc xắc cụ thể

giá trị. vì lý do sau. Lưu ý rằng 36 khả năng có khả năng như nhau về điểm số của hai viên xúc xắc là {11, 12, 13, ..., 56, 66}. Trong sáu trong số này, hai con xúc xắc có cùng giá trị. 30 còn lại tương ứng với 15 trong đó giá trị thứ nhất cao hơn và 15 trong đó giá trị thứ hai cao hơn, theo tính đối xứng. Vì vậy, có 15 trường hợp mà xúc xắc thứ hai thể hiện

giá trị cao hơn giá trị thứ ba, vì vậy . vì lý do sau. Nếu xúc xắc đầu tiên rơi vào con số 3, chúng ta có tổng cộng sáu kịch bản có khả năng xảy ra như nhau, đó là {31, 32, 33, 34, 35, 36}. Trong sáu số đó, số thứ hai là

cao hơn cho ba người trong số họ. Như vậy, $P(A) \cdot P(B) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36} \neq P(A \cap B) = \frac{3}{36} = \frac{1}{12}$ nên biến cố A và B phụ thuộc nhau.

đ. Đối với vấn đề này, chúng ta sẽ giả sử rằng A và B độc lập, tức là thời tiết không phụ thuộc vào ngày trong tuần. Đây là một giả định hợp lý dựa trên kiến thức của chúng ta về thời tiết, nhưng nếu muốn chắc chắn hơn, chúng ta có thể xem xét các tập dữ liệu thời tiết và xác minh điều này bằng cách tính toán các xác suất tương ứng.

f. Bởi vì chúng ta cho rằng chúng ta sống ở một nơi có các mùa nên tháng 6 là mùa hè ở bán cầu bắc và mùa đông ở bán cầu nam. Tùy thuộc vào nơi chúng ta sống, trời có thể mưa nhiều hơn vào mùa đông hoặc mùa hè. Vì vậy, chúng ta có thể giả sử rằng các sự kiện A và B là phụ thuộc.

Có một văn phòng nơi chúng tôi phải đến thường xuyên để làm một số giấy tờ. Văn phòng này có hai nhân viên, Aisha và Beto. Chúng tôi biết rằng Aisha làm việc ở đó ba ngày một tuần và Beto làm việc hai ngày còn lại. Tuy nhiên, lịch trình thay đổi hàng tuần, vì vậy chúng tôi không bao giờ biết Aisha ở đó ba ngày và Beto ở đó hai ngày.

Một. Nếu chúng ta ngẫu nhiên đến văn phòng vào một ngày nào đó thì xác suất để Aisha là nhân viên bán hàng là bao nhiêu?

Chúng tôi nhìn từ bên ngoài và nhận thấy người bán hàng đang mặc một chiếc áo len màu đỏ, mặc dù chúng tôi không thể biết người bán hàng là ai. Chúng tôi đã đến văn phòng đó rất nhiều nên biết rằng Beto có xu hướng mặc đồ màu đỏ thường xuyên hơn Aisha. Trên thực tế, Aisha mặc màu đỏ một trong ba ngày (một phần ba thời gian), và Beto mặc màu đỏ một trong hai ngày (một nửa thời gian).

b. Xác suất để Aisha là nhân viên bán hàng biết rằng hôm nay nhân viên đó mặc đồ màu đỏ là bao nhiêu?

GIẢI PHÁP

Hãy sử dụng ký hiệu sau cho các sự kiện:

- A: sự kiện nhân viên bán hàng là Aisha B: Sự kiện nhân viên bán hàng là Beto R: Sự kiện nhân viên bán hàng mặc đồ màu đỏ

Một. Vì Aisha làm việc ở văn phòng ba ngày và Beto làm việc hai ngày nên

xác suất để Aisha là nhân viên bán hàng là $P(A) = \frac{3}{5}$, hoặc 60%. Ngoài ra, xác suất rằng Beto là nhân viên bán hàng là $P(B) = \frac{2}{5}$, hoặc 40%.

b. Theo trực giác, vì Beto thường xuyên mặc đồ màu đỏ hơn Aisha nên chúng ta tưởng tượng rằng xác suất người bán hàng là Aisha thấp hơn ở phần a). Hãy kiểm tra xem toán học có đồng ý với chúng tôi không. Chúng ta biết nhân viên bán hàng mặc đồ màu đỏ, vì vậy chúng ta cần tìm xác suất để người bán hàng đó là Aisha *biết rằng* nhân viên bán hàng mặc đồ màu đỏ. Đây là $P(A|R)$.

Xác suất để Aisha mặc đồ màu đỏ là $\frac{1}{3}$, vì vậy $P(R|A) = \frac{1}{3}$.
Xác suất để Beto mặc đồ màu đỏ là $\frac{1}{2}$, vì vậy $P(R|B) = \frac{1}{2}$.

Chúng ta có thể sử dụng định lý Bayes để thu được

$$P(A \mid R) = \frac{P(R \mid A)P(A)}{P(R \mid A)P(A) + P(R \mid B)P(B)} = \frac{\frac{1}{3} \cdot \frac{3}{5}}{\frac{1}{3} \cdot \frac{3}{5} + \frac{1}{2} \cdot \frac{2}{5}} = \frac{\frac{1}{5}}{\frac{1}{5} + \frac{1}{5}} = \frac{1}{2}, \text{ or } 50\%$$

Một phép tính tương tự cho thấy xác suất Beto là nhân viên bán hàng là , hay 50%. $P(B \mid R) = \frac{1}{2}$

Trên thực tế, xác suất để Aisha là nhân viên bán hàng nhỏ hơn xác suất có được ở phần a), vì vậy trực giác của chúng tôi đã đúng.

Bài tập 8.3

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán
Bệnh nhân 1		X	X	X	Đau ốm
Bệnh nhân 2	X	X		X	Đau ốm
Bệnh nhân 3	X		X	X	Đau ốm
Bệnh nhân 4	X	X	X		Đau ốm
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

Mục tiêu của bài tập này là xây dựng một mô hình Bayes đơn giản để dự đoán chẩn đoán từ các triệu chứng.Sử dụng thuật toán Bayes ngây thơ để tìm các xác suất sau:

lưu ý Đối với các câu hỏi sau, chúng tôi hoàn toàn không biết các triệu chứng không được đề cập.Ví dụ: nếu chúng ta biết rằng bệnh nhân bị ho nhưng không nói gì về việc họ bị sốt thì điều đó không có nghĩa là bệnh nhân không bị sốt.

Một. Xác suất để một bệnh nhân bị bệnh khi bệnh nhân đó bị ho b. Xác suất để một bệnh nhân bị ốm nếu bệnh nhân đó không mệt c. Xác suất một bệnh nhân bị bệnh khi bệnh nhân đó bị ho và sốt d. Xác suất một bệnh nhân bị bệnh khi bệnh nhân đó ho và sốt nhưng không khó thở

GIẢI PHÁP

Đối với vấn đề này, chúng tôi có các sự kiện sau:

- C: trường hợp bệnh nhân bị ho F: trường hợp bệnh nhân bị sốt B: trường hợp bệnh nhân khó thở T: trường hợp bệnh nhân mệt mỏi S: trường hợp bệnh nhân được chẩn đoán là bị bệnh H: trường hợp bệnh nhân được chẩn đoán là khỏe mạnh
-
-
-

Hơn nữa, A^c biểu thị phần bù (ngược lại) của biến cố A. Vì vậy, ví dụ T^c biểu thị sự kiện bệnh nhân không mệt mỏi.

Đầu tiên, hãy tính $P(S)$ và $P(H)$. Lưu ý rằng vì tập dữ liệu chứa bốn

khỏe mạnh và bốn bệnh nhân bị bệnh, cả hai xác suất (trước) này là $\frac{1}{2}$, hoặc 50%.

Một. Vì 4 bệnh nhân bị ho và 3 người trong số họ bị bệnh, chiếm tỷ lệ 75%. $P(S|C) = \frac{3}{4}$

Tương tự, chúng ta có thể sử dụng định lý Bayes theo cách sau: đầu tiên, chúng ta

tính bằng cách nhận thấy rằng có bốn bệnh nhân bị bệnh và ba trong số đó

họ bị ho. Chúng tôi cũng nhận thấy rằng, vì có bốn bệnh nhân khỏe mạnh và chỉ một người trong số họ bị ho. $P(C|H) = \frac{1}{4}$

Bây giờ chúng ta có thể sử dụng công thức

$$P(S|C) = \frac{P(C|S)P(S)}{P(C|S)P(S) + P(C|H)P(H)} = \frac{\frac{3}{4} \cdot \frac{1}{2}}{\frac{3}{4} \cdot \frac{1}{2} + \frac{1}{4} \cdot \frac{1}{2}} = \frac{3}{4}$$

$$P(S|T^c) = \frac{P(T^c|S)P(S)}{P(T^c|S)P(S) + P(T^c|H)P(H)} = \frac{\frac{1}{4} \cdot \frac{1}{2}}{\frac{1}{4} \cdot \frac{1}{2} + \frac{2}{4} \cdot \frac{1}{2}} = \frac{\frac{1}{8}}{\frac{1}{8} + \frac{2}{8}} = \frac{1}{3}$$

$$P(S | T^c) = \frac{1}{3}$$

b. Vì 4 bệnh nhân bị ho và 3 người trong số đó bị bệnh, chiếm tỷ lệ 33,3%.

Chúng ta cũng có thể sử dụng định lý Bayes như trước đây. Chú ý rằng $P(T^c | S) = \frac{1}{4}$ bởi vì chỉ

một trong bốn bệnh nhân bị bệnh không mệt mỏi. Ngoài ra, bởi vì $P(T^c | H) = \frac{2}{4}$ hai trong số bốn bệnh nhân khỏe mạnh không hề mệt mỏi.

Theo định lý Bayes,

$$P(S | T^c) = \frac{P(T^c | S)P(S)}{P(T^c | S)P(S) + P(T^c | H)P(H)} = \frac{\frac{1}{4} \cdot \frac{1}{2}}{\frac{1}{4} \cdot \frac{1}{2} + \frac{2}{4} \cdot \frac{1}{2}} = \frac{\frac{1}{8}}{\frac{1}{8} + \frac{2}{8}} = \frac{1}{3}$$

c. $C \cap F$ biểu thị trường hợp bệnh nhân ho và sốt nên ta cần tính $P(S | C \cap F)$.

Nhớ lại phần a) rằng và $P(S | C) = \frac{3}{4}$ $P(C | H) = \frac{1}{4}$

Bây giờ chúng ta cần tính $P(F | S)$ và $P(F | H)$. Lưu ý rằng vì có bốn

bệnh nhân bị bệnh và ba người trong số họ bị sốt, $P(F | S) = \frac{3}{4}$ Tương tự, hai trong số

bốn bệnh nhân khỏe mạnh bị sốt, vì vậy. $P(F | H) = \frac{2}{4} = \frac{1}{2}$

Chúng tôi sẵn sàng sử dụng thuật toán Bayes ngây thơ để ước tính xác suất bệnh nhân bị bệnh khi họ bị ho và sốt. Sử dụng công thức trong phần “Hai từ thì sao? Thuật toán Bayes ngây thơ” ở chương 8, ta có

$$\begin{aligned} P(S | C \cap F) &= \frac{P(C | S)P(F | S)P(S)}{P(C | S)P(F | S)P(S) + P(C | H)P(F | H)P(H)} \\ &= \frac{\frac{3}{4} \cdot \frac{3}{4} \cdot \frac{1}{2}}{\frac{3}{4} \cdot \frac{3}{4} \cdot \frac{1}{2} + \frac{1}{4} \cdot \frac{2}{4} \cdot \frac{1}{2}} = \frac{\frac{9}{32}}{\frac{9}{32} + \frac{2}{32}} = \frac{9}{11} \text{ or } 81.82\%. \end{aligned}$$

d. Đối với bài tập này chúng ta cần tìm $P(S|C \cap F \cap B^c)$

Lưu ý rằng vì có bốn bệnh nhân bị bệnh và chỉ có một người trong số họ không có khó thở, .Tương tự, có 4 bệnh nhân khỏe mạnh và ba người trong số họ không bị khó thở nên .

$$P(B^c | S) = \frac{1}{4}$$

$$P(B^c | H) = \frac{3}{4}$$

Như trước đây, chúng ta có thể sử dụng thuật toán Bayes đơn giản.

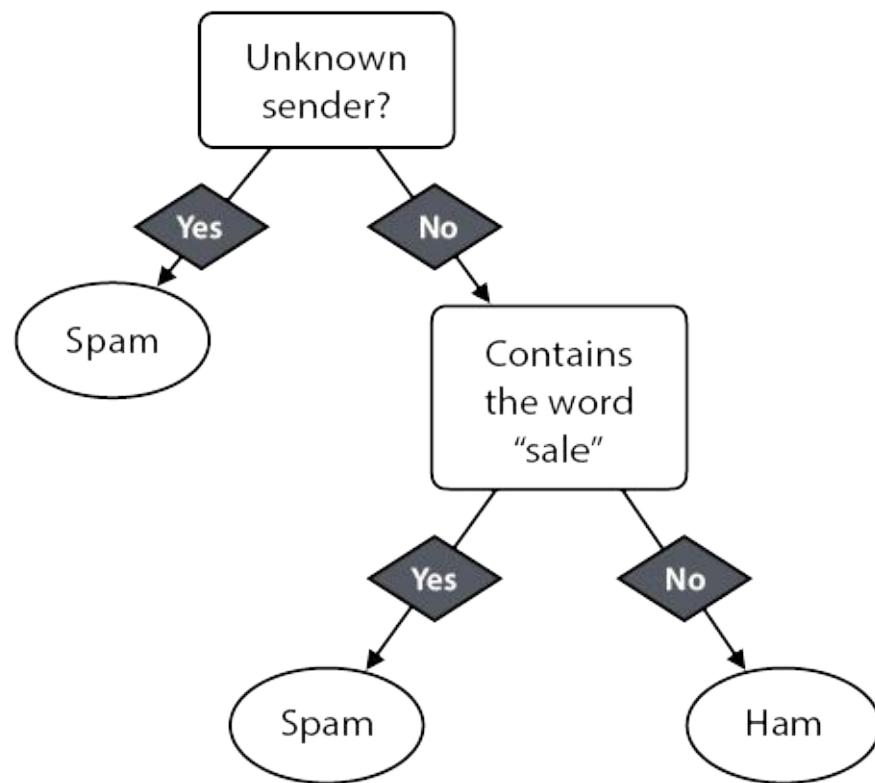
$$P(S | C \cap F \cap B^c) = \frac{P(C | S)P(F | S)P(B^c | S)P(S)}{P(C | S)P(F | S)P(B^c | S)P(S) + P(C | H)P(F | H)P(B^c | H)P(H)}$$

$$= \frac{\frac{3}{4} \cdot \frac{3}{4} \cdot \frac{1}{4} \cdot \frac{1}{2}}{\frac{3}{4} \cdot \frac{3}{4} \cdot \frac{1}{4} \cdot \frac{1}{2} + \frac{1}{4} \cdot \frac{2}{4} \cdot \frac{3}{4} \cdot \frac{1}{2}} = \frac{\frac{9}{32}}{\frac{9}{32} + \frac{6}{32}} = \frac{9}{15}, \text{ or } 60\%.$$

Chương 9: Phân tách dữ liệu bằng cách đặt câu hỏi: Cây quyết định

Bài tập 9.1

Trong mô hình cây quyết định phát hiện thư rác sau đây, hãy xác định xem email từ mẹ bạn có dòng chủ đề “Hãy đến cửa hàng, đang có đợt giảm giá” có bị phân loại là thư rác hay không.



GIẢI PHÁP

Đầu tiên chúng tôi kiểm tra xem người gửi có phải là không xác định hay không. Vì người gửi là mẹ chúng ta nên người gửi không phải là người không biết. Vì vậy, chúng tôi lấy nhánh bên phải. Chúng tôi phải kiểm tra xem email có chứa từ “bán hàng” hay không. Email chứa từ "bán hàng", do đó bộ phân loại (không chính xác) phân loại nó là thư rác.

Bài tập 9.2

Mục tiêu của chúng tôi là xây dựng mô hình cây quyết định để xác định xem các giao dịch thẻ tín dụng có gian lận hay không. Chúng tôi sử dụng tập dữ liệu về các giao dịch thẻ tín dụng bên dưới,

với các tính năng sau:

- **Giá trị:** giá trị giao dịch.**Nhà cung cấp được phê duyệt:** công ty thẻ tín dụng có danh sách các
- nhà cung cấp được phê duyệt. Biến này cho biết nhà cung cấp có nằm trong danh sách này hay không.

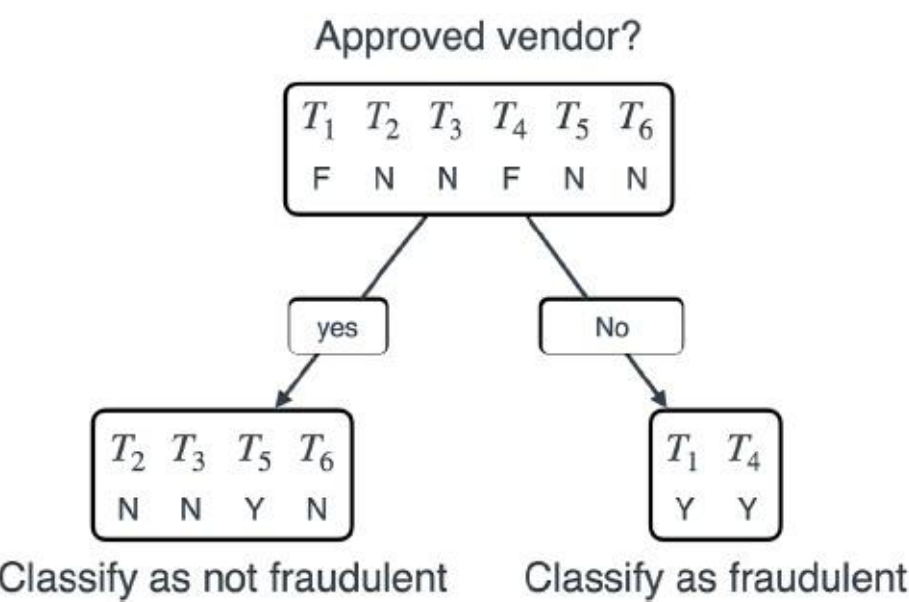
	Giá trị	Nhà cung cấp được phê duyệt	Lừa đảo
Giao dịch 1	\$100	Không được phê duyệt	Yes
Giao dịch 2	\$100	Tán thành	No
Giao dịch 3	\$10,000	Tán thành	No
Giao dịch 4	\$10,000	Không được phê duyệt	Yes
Giao dịch 5	\$5,000	Tán thành	Yes
Giao dịch 6	\$100	Tán thành	No

Xây dựng nút đầu tiên của cây quyết định theo các thông số kỹ thuật sau:

Một. Sử dụng chỉ số tạp chất Gini b. Sử dụng entropy

GIẢI PHÁP

Trong cả hai trường hợp, mức phân chia tốt nhất sẽ đạt được bằng cách sử dụng tính năng Nhà cung cấp được phê duyệt, như trong hình ảnh tiếp theo.



Hãy gọi các giao dịch T_1, T_2, T_3, T_4, T_5 và T_6 .

Trước tiên, chúng ta hãy xem tất cả các phép chia sau đây mà chúng ta có thể thực hiện. Việc phân chia bằng cách sử dụng Nhà cung cấp được phê duyệt rất dễ dàng vì đây là biến phân loại có hai danh mục. Cột Giá trị phức tạp hơn—chúng ta có thể sử dụng cột này để phân chia dữ liệu theo hai cách có thể. Một là khi giới hạn có giá trị nào đó trong khoảng từ 100 đến 5.000 đô la, và thứ hai là khi nó có giá trị nào đó trong khoảng từ 5.000 đến 10.000 đô la. Tóm lại, đây là tất cả các cách phân chia có thể xảy ra:

- **Giá trị 1:** trong đó giá trị giới hạn nằm trong khoảng từ 100 USD đến 5.000 USD. Hai lớp ở đây là $\{T_1, T_2, T_6\}$ và $\{T_3, T_4, T_5\}$. **Giá trị 2:** trong đó giá trị giới hạn là một giá trị nào đó trong khoảng từ 5.000 USD đến 10.000 USD. Hai lớp ở đây là $\{T_1, T_2, T_5, T_6\}$ và $\{T_3, T_4\}$. **Nhà cung cấp được phê duyệt:** hai lớp là “được phê duyệt” và “không được phê duyệt” hoặc tương đương, $\{T_2, T_3, T_5, T_6\}$ và $\{T_1, T_4\}$.
-

Một. Hãy tính toán xác định chỉ số tạp chất Gini cho từng phần trong bốn phần sau:

Giá trị 1: giá trị ngưỡng từ 100 USD đến 5.000 USD

Lưu ý rằng đối với loại đầu tiên $\{T_1, T_2, T_6\}$, các nhãn trong cột Lừa đảo là

{“có”, “không”, “không”}. Chỉ số tạp chất Gini của sự phân chia này là . $1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = \frac{4}{9}$

Lưu ý rằng đối với loại thứ hai $\{T_3, T_4, T_5\}$, các nhãn ở cột Lừa đảo

là {“không”, “có”, “có”}. Chỉ số tạp chất Gini của sự phân chia này là . $1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = \frac{4}{9}$

Do đó, chỉ số tạp chất Gini có trọng số cho sự phân chia này là . $\frac{3}{6} \cdot \frac{4}{9} + \frac{3}{6} \cdot \frac{4}{9} = \frac{4}{9} = 0.444$

Giá trị 2: giá trị giới hạn từ \$5.000 đến \$10.000

Đối với loại đầu tiên $\{T_1, T_2, T_5, T_6\}$, các nhãn trong cột Lừa đảo là

{“có”, “không”, “có”, “không”}. Chỉ số tạp chất Gini của sự phân chia này là

$$1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = \frac{1}{2}.$$

Lưu ý rằng đối với loại thứ hai $\{T_3, T_4\}$, các nhãn trong cột Lừa đảo là

{“không”, “có”}.Chỉ số tạp chất Gini của sự phân chia này là .

$$1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = \frac{1}{2}$$

Như vậy, chỉ số tạp chất Gini có trọng số cho lần phân chia này là .

$$\frac{4}{6} \cdot \frac{1}{2} + \frac{2}{6} \cdot \frac{1}{2} = \frac{1}{2} = 0.5$$

Nhà cung cấp được phê duyệt:

Đối với loại đầu tiên $\{T_2, T_3, T_5, T_6\}$, các nhãn trong cột Lừa đảo là

{“không”, “không”, “có”, “không”}.Chỉ số tạp chất Gini của sự phân chia này là

$$1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = \frac{6}{16}.$$

Đối với loại thứ hai $\{T_1, T_4\}$, các nhãn trong cột Lừa đảo là {“có”, “có”}.Chỉ số tạp chất Gini của lần phân chia này là $1 - 1^2 = 0$.

Do đó, chỉ số tạp chất Gini có trọng số cho sự phân chia này là .

$$\frac{4}{6} \cdot \frac{6}{16} + \frac{2}{6} \cdot 0 = \frac{1}{4} = 0.25$$

Lưu ý rằng trong ba giá trị này, giá trị thấp nhất là 0,25, tương ứng với cột Nhà cung cấp được phê duyệt.Điều này ngụ ý rằng cách tốt nhất để phân chia dữ liệu này là sử dụng tính năng Nhà cung cấp được phê duyệt.

b. Về phần này, chúng ta đã thực hiện hầu hết các công việc nặng nhọc. Chúng ta sẽ thực hiện theo quy trình tương tự như ở phần a), ngoại trừ việc tính entropy ở mỗi giai đoạn thay vì tính chỉ số tập chất Gini.

Giá trị 1: giá trị ngưỡng từ 100 USD đến 5.000 USD

Entropy của tập {"có", "không", "không"} là .

$$-\frac{1}{3}\log_2\left(\frac{1}{3}\right)-\frac{2}{3}\log_2\left(\frac{2}{3}\right)=0.918$$

Entropy của tập {"không", "có", "có"} cũng là .

$$-\frac{2}{3}\log_2\left(\frac{2}{3}\right)-\frac{1}{3}\log_2\left(\frac{1}{3}\right)=0.918$$

Do đó, entropy có trọng số cho phép chia này là .

$$\frac{3}{6}\cdot 0.918+\frac{3}{6}\cdot 0.918=0.918$$

Giá trị 2: giá trị giới hạn từ \$5.000 đến \$10.000

Entropy của tập {"có", "không", "có", "không"} là .

$$-\frac{2}{4}\log_2\left(\frac{2}{4}\right)-\frac{2}{4}\log_2\left(\frac{2}{4}\right)=1$$

Entropy của tập {"không", "có"} là .

$$-\frac{1}{2}\log_2\left(\frac{1}{2}\right)-\frac{1}{2}\log_2\left(\frac{1}{2}\right)=1$$

Do đó, entropy có trọng số cho phép chia này là .

$$\frac{4}{6}\cdot 1+\frac{2}{6}\cdot 1=1$$

Nhà cung cấp được phê duyệt:

Entropy của tập {"không", "không", "có", "không"} là .

$$-\frac{1}{4}\log_2\left(\frac{1}{4}\right)-\frac{3}{4}\log_2\left(\frac{3}{4}\right)=0.811$$

Entropy của tập {"có", "có"} là .

$$-\frac{2}{2}\log_2\left(\frac{2}{2}\right)=0$$

Do đó, entropy có trọng số cho phép chia này là .

$$\frac{4}{6}\cdot 0.811+\frac{2}{6}\cdot 0=0.541$$

Lưu ý rằng trong số ba giá trị này, entropy nhỏ nhất là 0,541, tương ứng với cột Nhà cung cấp được phê duyệt. Do đó, cách tốt nhất để phân chia dữ liệu này là sử dụng tính năng Nhà cung cấp được phê duyệt.

Bài tập 9.3

Sau đây là tập dữ liệu về những bệnh nhân đã có kết quả xét nghiệm dương tính hoặc âm tính với COVID-19. Các triệu chứng của họ là ho (C), sốt (F), khó thở (B) và mệt mỏi (T).

	Ho (C)	Sốt (F)	Khó thở (B)	Mệt mỏi (T)	Chẩn đoán
Bệnh nhân 1		X	X	X	Đau ốm
Bệnh nhân 2	X	X		X	Đau ốm
Bệnh nhân 3	X		X	X	Đau ốm
Bệnh nhân 4	X	X	X		Đau ốm
Bệnh nhân 5	X			X	Khỏe mạnh
Bệnh nhân 6		X	X		Khỏe mạnh
Bệnh nhân 7		X			Khỏe mạnh
Bệnh nhân 8				X	Khỏe mạnh

Sử dụng độ chính xác, xây dựng cây quyết định có độ cao 1 (gốc quyết định) để phân loại dữ liệu này.Độ chính xác của bộ phân loại này trên tập dữ liệu là gì?

GIẢI PHÁP

Hãy gọi bệnh nhân là P_1 đến P_8 .Bệnh nhân bị bệnh sẽ được ký hiệu là “s” và những người khỏe mạnh bởi “h.”

Đầu tiên hãy lưu ý rằng phép phân tách đầu tiên có thể là bất kỳ đặc điểm nào trong bốn đặc điểm C, F, B và T. Trước tiên, hãy tính độ chính xác của bộ phân loại thu được bằng cách phân tách dữ liệu trên đặc điểm C, cụ thể là bộ phân loại mà chúng tôi xây dựng dựa trên câu hỏi, “Bệnh nhân có bị ho không?”

Chia dựa trên đặc điểm C:

- Bệnh nhân ho: $\{P_2, P_3, P_4, P_5\}$.Nhãn của họ là $\{s, s, s, h\}$.
- Bệnh nhân không ho: $\{P_1, P_6, P_7, P_8\}$.Nhãn của chúng là $\{s, h, h, h\}$.

Nhìn vào đây, chúng ta có thể thấy rằng bộ phân loại chính xác nhất (chỉ dựa trên đặc điểm C) là bộ phân loại mọi người bị ho là bị bệnh và mọi người không bị ho là khỏe mạnh.Bộ phân loại này phân loại chính xác sáu trong số tám bệnh nhân (ba bệnh và ba khỏe mạnh), do đó độ chính xác của nó là $6/8$, hay 75%.

Bây giờ, hãy làm theo quy trình tương tự với ba tính năng còn lại.

Chia dựa trên đặc điểm F:

- Bệnh nhân sốt: $\{P_1, P_2, P_4, P_6, P_7\}$.Nhãn của chúng là $\{s, s, s, h, h\}$.
- Bệnh nhân không sốt: $\{P_3, P_5, P_8\}$.Nhãn của chúng là $\{s, h, h\}$.

Nhìn vào điều này, chúng ta có thể thấy rằng bộ phân loại chính xác nhất (chỉ dựa trên đặc điểm F) là bộ phân loại mọi bệnh nhân bị sốt là bị bệnh và mọi bệnh nhân không bị sốt là khỏe mạnh.Bộ phân loại này phân loại chính xác năm trong số tám bệnh nhân (ba bệnh và hai khỏe mạnh), do đó độ chính xác của nó là $5/8$, hay 62,5%.

Phân chia dựa trên đặc điểm B:

- Bệnh nhân có biểu hiện khó thở: $\{P_1, P_3, P_4, P_5\}$.Nhãn của họ là $\{s, s, s, h\}$. Bệnh nhân không có biểu hiện khó thở: $\{P_2, P_6, P_7, P_8\}$.Nhãn của họ là $\{s, h, h, h\}$.
-

Nhìn vào điều này, chúng ta có thể thấy rằng bộ phân loại chính xác nhất (chỉ dựa trên đặc điểm B) là bộ phân loại mọi bệnh nhân có biểu hiện khó thở là bị bệnh và mọi bệnh nhân không có biểu hiện khó thở là khỏe mạnh.Bộ phân loại này phân loại chính xác sáu trong số tám bệnh nhân (ba bệnh và ba khỏe mạnh), do đó độ chính xác của nó là $6/8$, hay 75%.

Phân chia dựa trên đặc điểm T:

- Bệnh nhân mệt mỏi: $\{P_1, P_2, P_3, P_5, P_8\}$.Nhãn của chúng là $\{s, s, s, h, h\}$.
- Bệnh nhân không mệt: $\{P_4, P_6, P_7\}$.Nhãn của chúng là $\{s, h, h\}$.

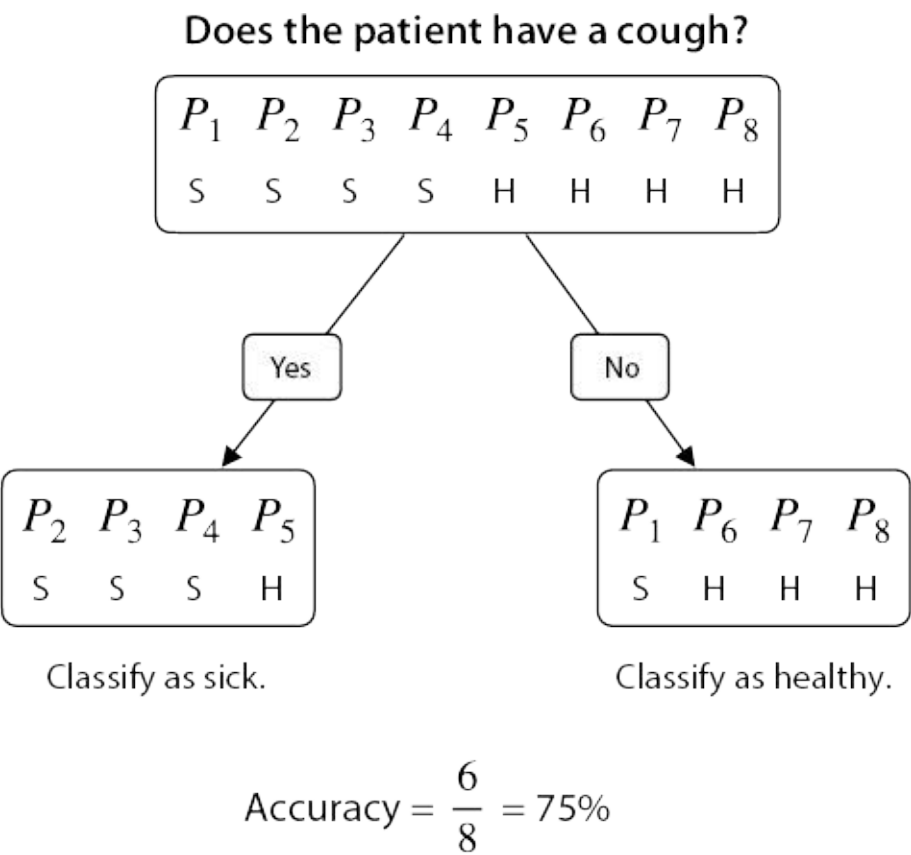
Nhìn vào này, chúng ta có thể thấy rằng bộ phân loại chính xác nhất (chỉ dựa trên F

tính năng) là đặc điểm phân loại mọi bệnh nhân một mọi là bệnh tật và mọi bệnh nhân không một mọi là khỏe mạnh. Bộ phân loại này phân loại chính xác năm trong số tám bệnh nhân (ba bệnh và hai khỏe mạnh), do đó độ chính xác của nó là $5/8$, hay 62,5%.

Lưu ý hai đặc điểm cho chúng ta độ chính xác cao nhất là C (ho) và B (khó thở). Cây quyết định sẽ chọn ngẫu nhiên một trong số đó. Hãy chọn cái đầu tiên, C. Sau khi phân chia dữ liệu bằng tính năng C, chúng ta thu được hai tập dữ liệu sau:

- Bệnh nhân ho: $\{P_2, P_3, P_4, P_5\}$. Nhãn của họ là $\{s, s, s, h\}$.
- Bệnh nhân không ho: $\{P_1, P_6, P_7, P_8\}$. Nhãn của chúng là $\{s, h, h, h\}$.

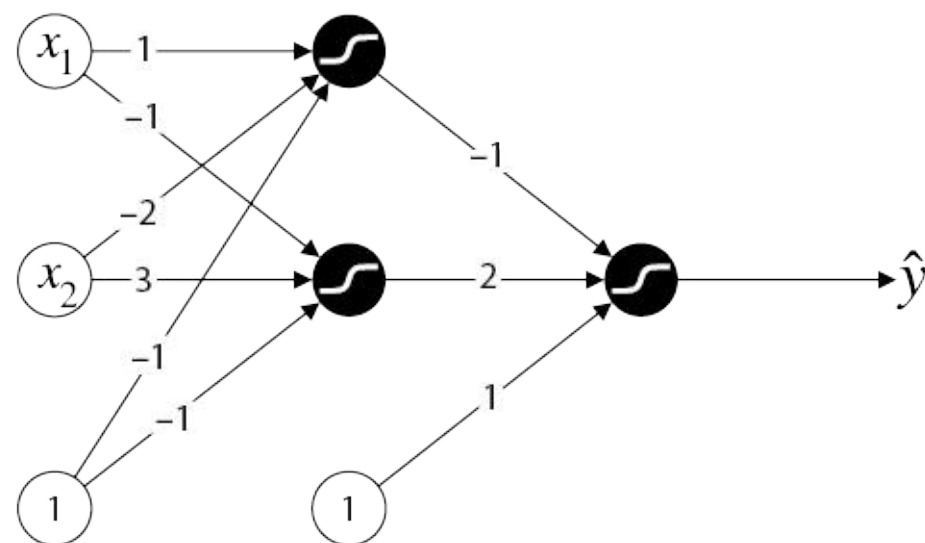
Điều này cung cấp cho chúng tôi cây độ sâu 1 để phân loại dữ liệu với độ chính xác 75%. Cây được mô tả trong hình tiếp theo.



Chương 10: Kết hợp các khối xây dựng để có thêm sức mạnh: Mạng lưới thần kinh

Bài tập 10.1

Hình ảnh sau đây cho thấy một mạng lưới thần kinh trong đó tất cả các kích hoạt đều là hàm sigmoid.



Mạng nơ-ron này sẽ dự đoán điều gì cho đầu vào (1,1)?

GIẢI PHÁP

Hãy gọi đầu ra của các nút ở giữa là η_1 và η_2 . Những điều này được tính như sau:

$$h_1 = \sigma(1 \cdot x_1 - 2 \cdot x_2 - 1)$$

$$h_2 = \sigma(-1 \cdot x_1 + 3 \cdot x_2 - 1)$$

Cắm $x_1 = 1$ và $x_2 = 1$ vào, ta được kết quả sau:

$$h_1 = \sigma(-2) = 0.119$$

$$h_2 = \sigma(1) = 0.731$$

Lớp cuối cùng là

$$\hat{y} = \sigma(-1 \cdot h_1 + 2 \cdot h_2 + 1).$$

Replacing the values previously obtained for h_1 and h_2 , we get

$$\hat{y} = \sigma(-0.119 + 2 \cdot 0.731 + 1) = \sigma(2.343) = 0.912.$$

Thus, the output of the neural network is 0.912.

Exercise 10.2

As we learned in exercise 5.3, it is impossible to build a perceptron that mimics the XOR gate. In other words, it is impossible to fit the following dataset with a perceptron and obtain 100% accuracy:

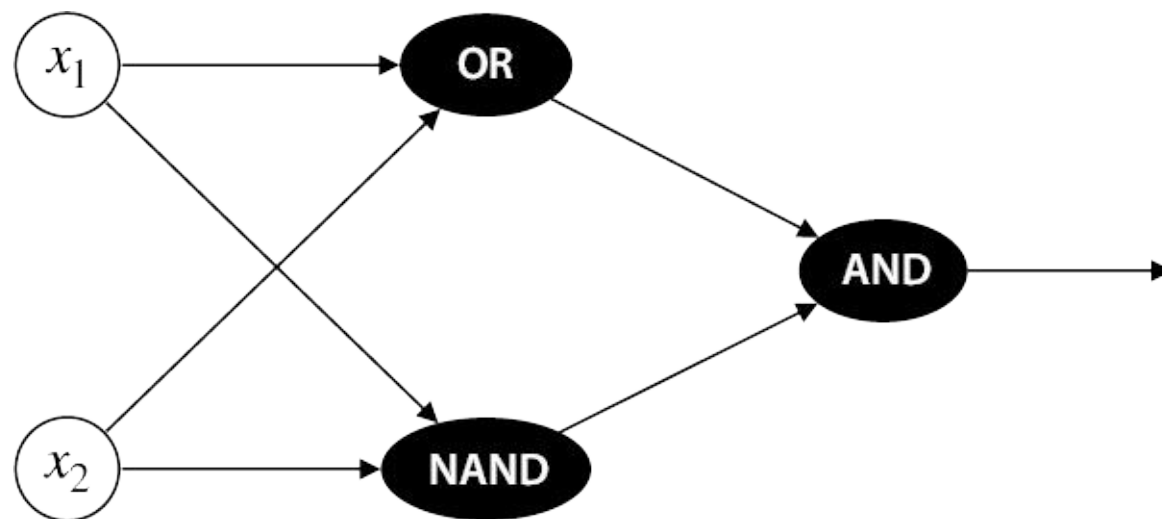
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

This is because the dataset is not linearly separable. Using a neural network of depth 2, build a perceptron that mimics the XOR gate shown previously. As the activation functions, use the step function instead of the sigmoid function to get discrete outputs.

hint This will be hard to do using a training method; instead, try eyeballing the weights. Try (or search online how) to build an XOR gate using AND, OR, and NOT gates, and use the results of exercise 5.3 to help you.

SOLUTION

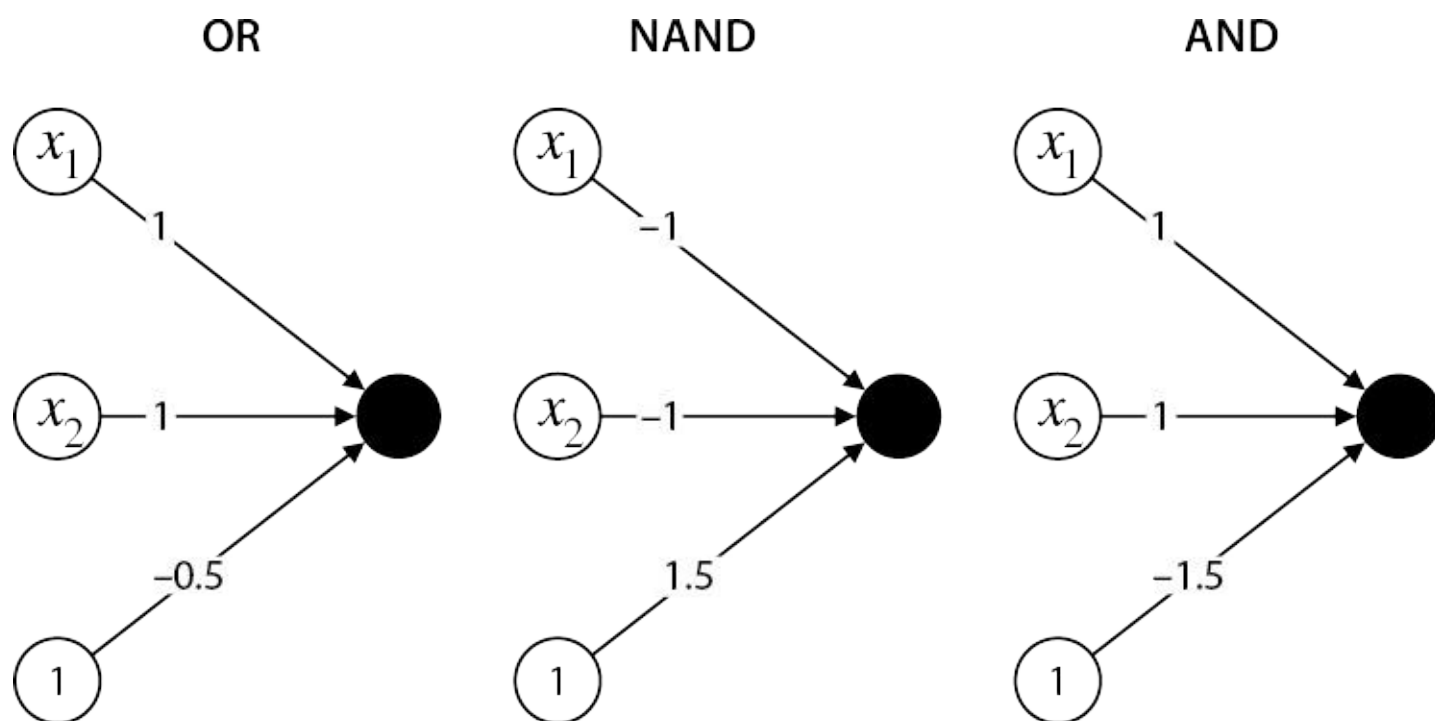
Note that the following combination of AND, OR, and NOT gates forms an XOR gate (where the NAND gate is the combination of an AND gate and a NOT gate).



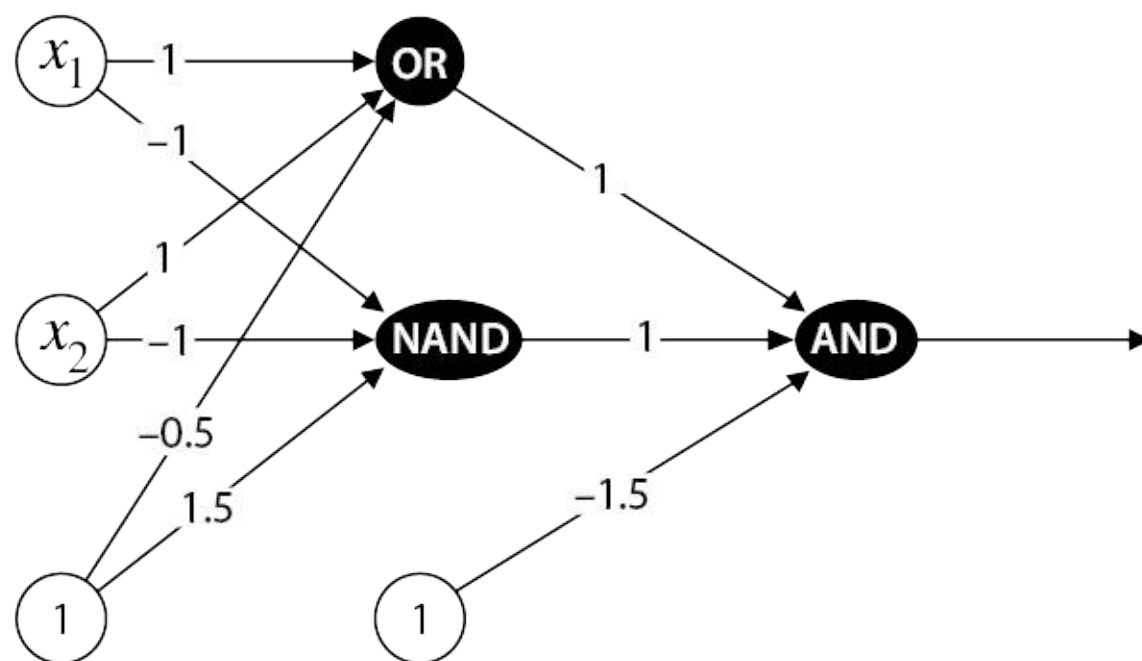
The following truth table illustrates it.

x_1	x_2	$h_1 = x_1 \text{ OR } x_2$	$h_2 = x_1 \text{ NAND } x_2$	$h_1 \text{ AND } h_2$	$x_1 \text{ XOR } x_2$
0	0	0	1	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	1	0	0	0

As we did in exercise 5.3, here are perceptrons that mimic the OR, NAND, and AND gates. The NAND gate is obtained by negating all the weights in the AND gate.



Joining these together, we get the neural network shown in the next figure.



Tôi khuyến khích bạn xác minh rằng mạng này thực sự bắt chước cổng logic XOR. Điều này được thực hiện bằng cách nhập bốn vector (0,0), (0,1), (1,0), (1,1) qua mạng và xác minh rằng các đầu ra là 0, 1, 1, 0.

Bài tập 10.3

Ở cuối phần “Biểu diễn đồ họa của mạng nơ-ron”, chúng ta thấy rằng mạng nơ-ron trong hình 10.13 với hàm kích hoạt không phù hợp với tập dữ liệu trong bảng 10.1, vì điểm (1,1) bị phân loại sai.

Một. Xác minh rằng đây là trường hợp. b. Thay đổi trọng số để mạng lưới thần kinh phân loại chính xác mọi điểm.

GIẢI PHÁP

Một. Đối với điểm $(x_a, x_b) = (1, 1)$, dự đoán như sau:

$$C = \sigma(6 \cdot 1 + 10 \cdot 1 - 15) = \sigma(1) = 0.731$$

$$F = \sigma(10 \cdot 1 + 6 \cdot 1 - 15) = \sigma(1) = 0.731$$

$$\hat{y} = \sigma(1 \cdot 0.731 + 1 \cdot 0.731 - 1.5) = \sigma(-0.39) = 0.404$$

Vì dự đoán gần 0 hơn 1 nên điểm bị phân loại sai.

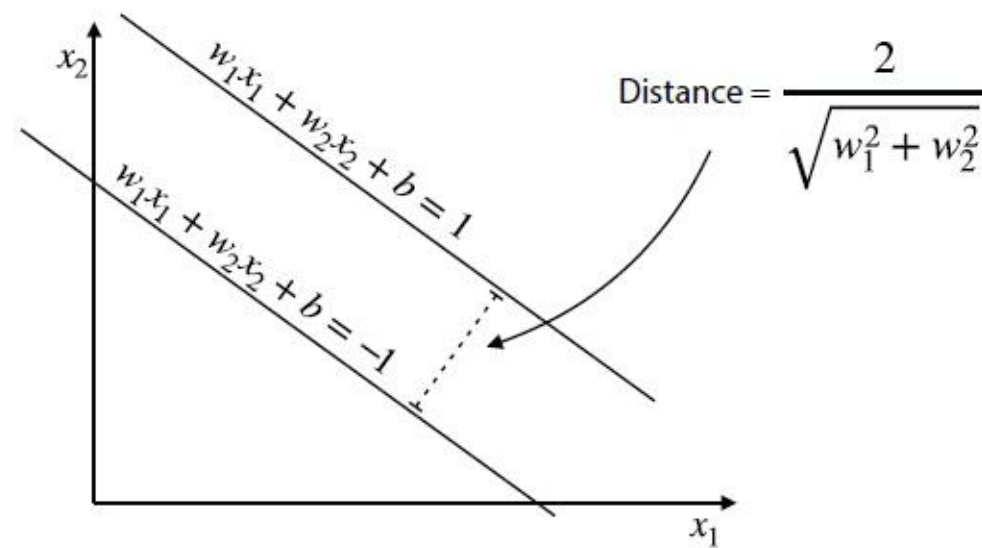
b. Giảm độ lệch ở nút cuối cùng xuống mức nhỏ hơn $2 \cdot 0,731 = 1,461$ là được. Ví dụ: nếu độ lệch này là 1,4 thì dự đoán tại điểm (1,1) sẽ cao hơn 0,5. Như một bài tập, tôi khuyến khích bạn xác minh rằng mạng nơ-ron mới này dự đoán chính xác nhãn cho các điểm còn lại.

Chương 11: Tìm ranh giới với phong cách: Hỗ trợ máy vectơ và phương thức kernel

Bài tập 11.1

(Bài tập này hoàn thành phép tính cần thiết trong phần “Hàm sai số khoảng cách.”)

Chứng minh rằng khoảng cách giữa các đường thẳng có phương trình $w_1x_1 + w_2x_2 + b = 1$ và $w_1x_1 + w_2x_2 + b = -1$ chính xác là $\frac{2}{\sqrt{w_1^2 + w_2^2}}$.



GIẢI PHÁP

Đầu tiên chúng ta gọi các dòng như sau:

- L_1 là đường thẳng có phương trình $w_1x_1 + w_2x_2 + b = 1$.
- L_2 là đường thẳng có phương trình $w_1x_1 + w_2x_2 + b = -1$.

Lưu ý rằng chúng ta có thể viết lại phương trình $w_1x_1 + w_2x_2 + b = 0$ như với $x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$. Mọi đường vuông góc với đường này đều có độ dốc $\frac{w_1}{w_2}$. Đặc biệt, dòng với

$$x_2 = \frac{w_2}{w_1} x_1$$

phương trình vuông góc với cả L_1 và L_2 . Chúng ta sẽ gọi dòng này là L_3 .

Tiếp theo, ta giải các giao điểm của L_3 với từng đường thẳng L_1 và L_2 .

Giao điểm của L_1 và L_3 là nghiệm của các phương trình sau:

- $w_1 x_1 + w_2 x_2 + b = 1$
- $x_2 = \frac{w_1}{w_2} x_1$

Chúng ta có thể thế phương trình thứ hai vào phương trình thứ nhất để thu được

$$w_1 x_1 + w_2 \cdot \frac{w_1}{w_2} x_1 + b = 1,$$

và sau đó giải tìm x_1 để có được

$$x_1 = \frac{1-b}{2w_1}.$$

Do đó, vì mọi điểm trong L_2 đều có dạng giao điểm của L_1 và L_3 là điểm có tọa độ $(x, \frac{w_1}{w_2} x)$ $\left(\frac{1-b}{w_1}, \frac{1-b}{w_2} \right)$.

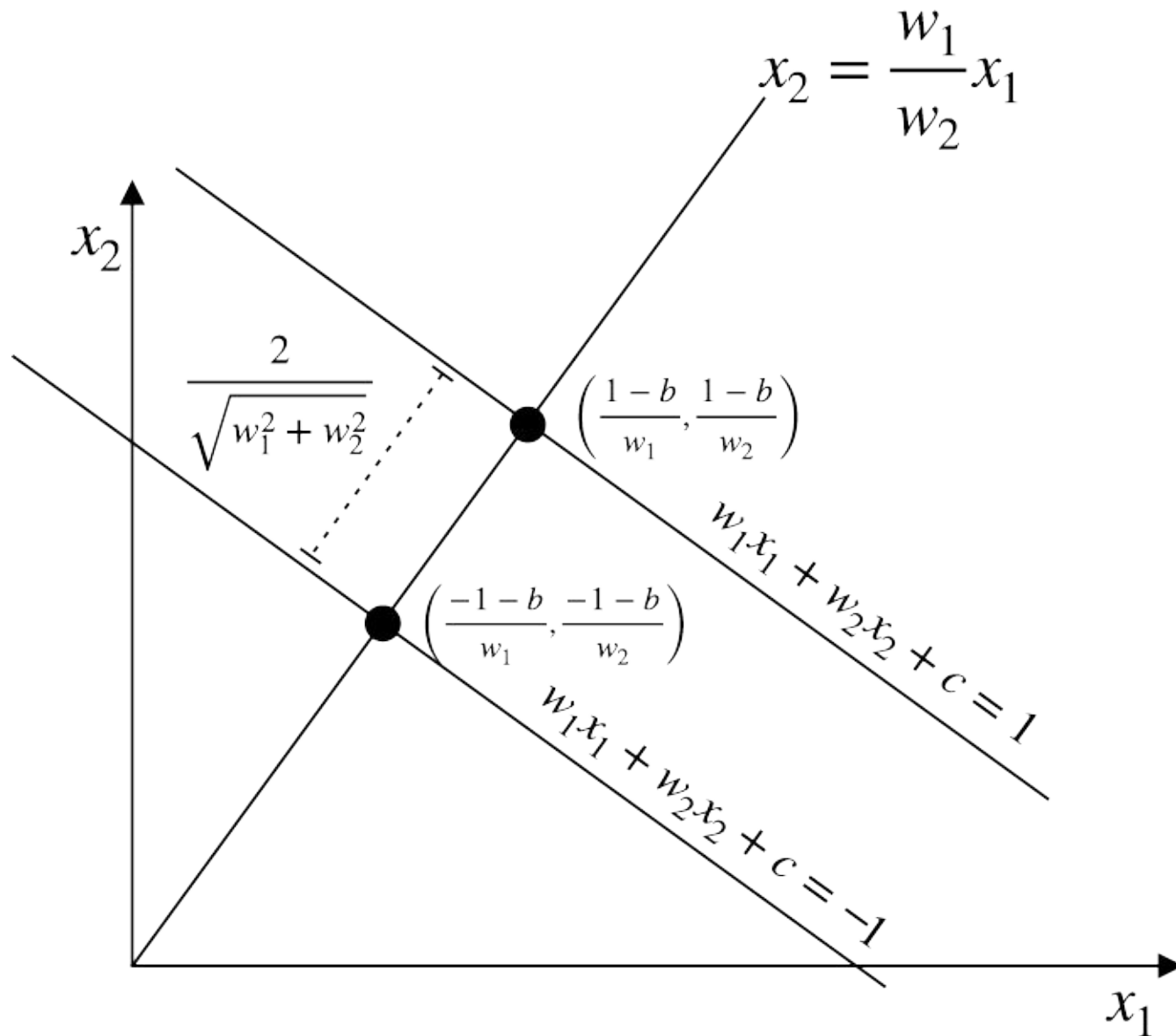
Tính toán tương tự sẽ cho thấy giao điểm của L_2 và L_3 là điểm

với tọa độ $\left(\frac{1-b}{w_1}, \frac{1-b}{w_2} \right)$.

Để tìm khoảng cách giữa hai điểm này, chúng ta có thể sử dụng định lý Pythagore. Khoảng cách này là

$$\sqrt{\left(\frac{1-b}{w_1} - \frac{-1-b}{w_1} \right)^2 + \left(\frac{1-b}{w_2} - \frac{-1-b}{w_2} \right)^2} = \sqrt{\left(\frac{2}{w_1} \right)^2 + \left(\frac{2}{w_2} \right)^2} = \frac{2}{\sqrt{w_1^2 + w_2^2}}$$

như mong muốn.



Exercise 11.2

As we learned in exercise 5.3, it is impossible to build a perceptron model that mimics the XOR gate. In other words, it is impossible to fit the following dataset (with 100% accuracy) with a perceptron model:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

This is because the dataset is not linearly separable. An SVM has the same problem, because an SVM is also a linear model. However, we can use a kernel to help us out. What kernel should we use to turn this dataset into a linearly separable one? What would the resulting SVM look like?

hint Look at example 2 in the section “Using polynomial equations to your benefit,” which solves a very similar problem.

SOLUTION

Considering the polynomial kernel of degree two, we get the following dataset:

x_1	x_2	x_1^2	$x_1 x_2$	x_2^2	y
0	0	0	0	0	0
0	1	0	0	1	1
1	1	1	0	0	1
1	1	1	1	1	0

Several classifiers work on this modified dataset. For example, the one with equation $\hat{y} = step(x_1 + x_2 - 2x_1x_2 - 0.5)$ classifies the data correctly.

Chương 12: Kết hợp các mô hình để tối đa hóa kết quả: Học tập theo nhóm

Bài tập 12.1

Một người học mạnh được tăng cường L được hình thành bởi ba người học yếu là L_1 , L_2 và L_3 . Của họ trọng số lần lượt là 1, 0,4 và 1,2. Đối với một điểm cụ thể, L_1 và L_2 dự đoán rằng nhãn của nó là dương và L_3 dự đoán rằng nhãn của nó là âm. cuối cùng là gì dự đoán của người học L về điểm này?

GIẢI PHÁP

Vì L_1 và L_2 dự đoán nhãn là tích cực và L_3 dự đoán nhãn là phủ định nên tổng số phiếu bầu là

$$1 + 0.4 - 1.2 = 0.2.$$

Kết quả này là dương, có nghĩa là người học giỏi dự đoán rằng nhãn của điểm này là dương.

Bài tập 12.2

Chúng tôi đang trong quá trình đào tạo mô hình AdaBoost trên tập dữ liệu có kích thước 100. Trình học yếu hiện tại phân loại chính xác 68 trên 100 điểm dữ liệu. Trọng số mà chúng ta sẽ gán cho người học này trong mô hình cuối cùng là bao nhiêu?

GIẢI PHÁP

Trọng số này là tỷ lệ cược log hoặc logarit tự nhiên của tỷ lệ cược. Tỷ lệ cược là 68/32, vì bộ phân loại phân loại đúng 68 điểm và phân loại sai 32 điểm còn lại. Do đó, trọng số được gán cho người học yếu này là

$$weight = \ln \left(\frac{68}{32} \right) = 0.754$$

Chương 13: Áp dụng tất cả vào thực tế: Một ví dụ thực tế về kỹ thuật dữ liệu và học máy

Bài tập 13.1

Kho lưu trữ chứa một tệp có tên test.csv. Đây là tệp có nhiều hành khách hơn trên *Titanic*, ngoại trừ việc nó không có cột Sống sót.

1. Xử lý trước dữ liệu trong tệp này như chúng tôi đã làm trong chương này. 2. Sử dụng bất kỳ mô hình nào để dự đoán nhãn trong tập dữ liệu này. Theo mô hình của bạn, có bao nhiêu hành khách sống sót? 3. So sánh hiệu suất của tất cả các mô hình trong chương này, bạn nghĩ có bao nhiêu hành khách trong bộ thử nghiệm thực sự sống sót?

GIẢI PHÁP

Giải pháp nằm ở cuối sổ ghi chép sau:

https://github.com/luisguiserrano/manning/tree/master/Chapter_13_End_to_end_exa

Phụ lục B. Toán học đằng sau việc giảm độ dốc: Xuống núi bằng cách sử dụng đạo hàm và độ dốc

Trong phụ lục này, chúng ta sẽ xem xét các chi tiết toán học của việc giảm độ dốc. Phụ lục này khá kỹ thuật và không cần thiết phải hiểu nó để theo dõi phần còn lại của cuốn sách. Tuy nhiên, nó ở đây để mang lại cảm giác đầy đủ cho những độc giả muốn hiểu hoạt động bên trong của một số thuật toán học máy cốt lõi. Kiến thức toán học cần thiết cho phụ lục này cao hơn so với phần còn lại của cuốn sách. Cụ thể hơn, cần có kiến thức về vector, đạo hàm và quy tắc dây chuyền.

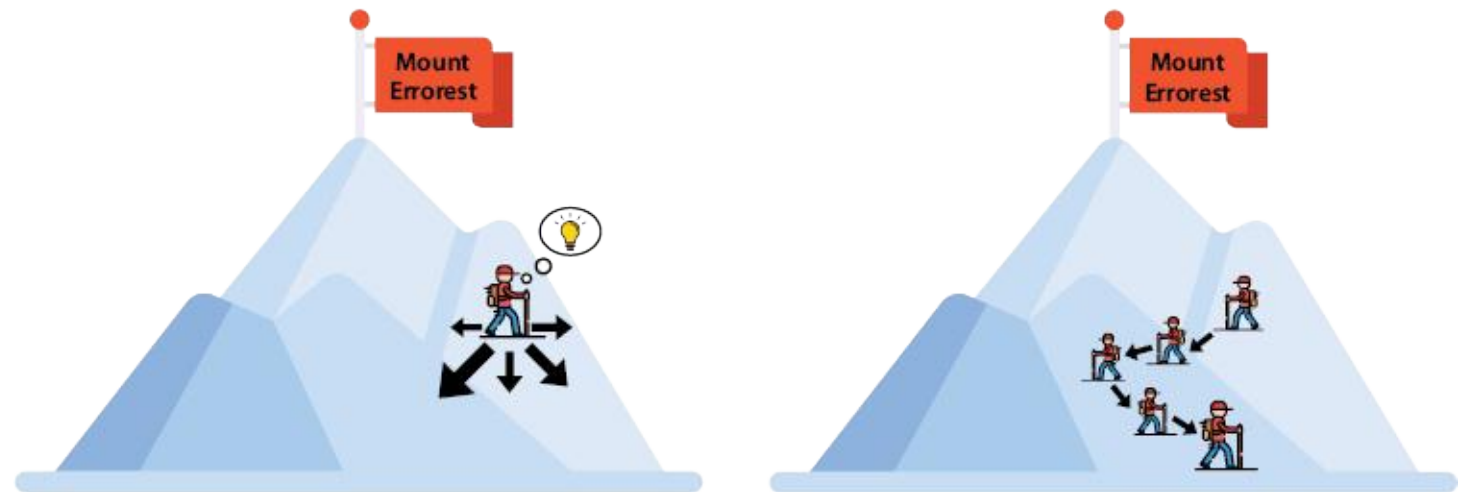
Trong các chương 3, 5, 6, 10 và 11, chúng tôi đã sử dụng phương pháp giảm độ dốc để giảm thiểu các hàm lỗi trong các mô hình của mình. Cụ thể hơn, chúng tôi đã sử dụng phương pháp giảm độ dốc để giảm thiểu các hàm lỗi sau:

- Chương 3: hàm lỗi tuyệt đối và bình phương trong mô hình hồi quy tuyến tính
- Chương 5: hàm lỗi perceptron trong mô hình perceptron
- Chương 6: mất log trong bộ phân loại logistic
- Chương 10: mất log trong mạng nơ-ron
- Chương 11: lỗi phân loại (perceptron) và lỗi khoảng cách (chính quy hóa) trong SVM
-

Như chúng ta đã học ở chương 3, 5, 6, 10 và 11, hàm lỗi đo lường mức độ hoạt động kém của mô hình. Do đó, việc tìm giá trị tối thiểu cho hàm sai số này - hoặc ít nhất là một giá trị thực sự nhỏ, ngay cả khi nó không phải là giá trị tối thiểu - sẽ là công cụ để tìm ra một mô hình tốt.

Sự tương tự mà chúng tôi sử dụng là việc đi xuống một ngọn núi—Núi Errorrest, được minh họa trong hình B.1. Kịch bản như sau: Bạn đang ở đâu đó trên đỉnh một ngọn núi và bạn muốn xuống chân ngọn núi này. Trời rất nhiều mây nên bạn không thể nhìn xa xung quanh mình. Cách tốt nhất bạn có thể làm là đi xuống núi từng bước một. Bạn tự hỏi: “Nếu tôi chỉ đi một bước thì tôi nên đi theo hướng nào để đi xuống nhiều nhất?” Bạn tìm ra hướng đó và thực hiện bước đó. Sau đó, bạn hỏi lại câu hỏi tương tự và thực hiện một bước khác và lặp lại quá trình này nhiều lần. Có thể tưởng tượng rằng nếu bạn luôn đi một bước giúp bạn đi xuống nhiều nhất thì bạn sẽ phải tụt xuống vị trí thấp nhất. Bạn có thể cần một chút may mắn để thực sự đến được chân núi, ngược lại

đến việc bị mắc kẹt trong thung lũng, nhưng chúng ta sẽ giải quyết vấn đề này sau trong phần “Bị mắc kẹt ở mức cực tiểu cục bộ”.



Hình B.1 Trong bước giảm độ dốc, chúng ta muốn đi xuống từ một ngọn núi có tên là Mount Erorest.

Trong các phần sau, chúng tôi sẽ mô tả toán học đằng sau việc giảm độ dốc và sử dụng nó để giúp chúng tôi đào tạo một số thuật toán học máy bằng cách giảm các hàm lỗi của chúng.

Sử dụng phương pháp giảm độ dốc để giảm chức năng

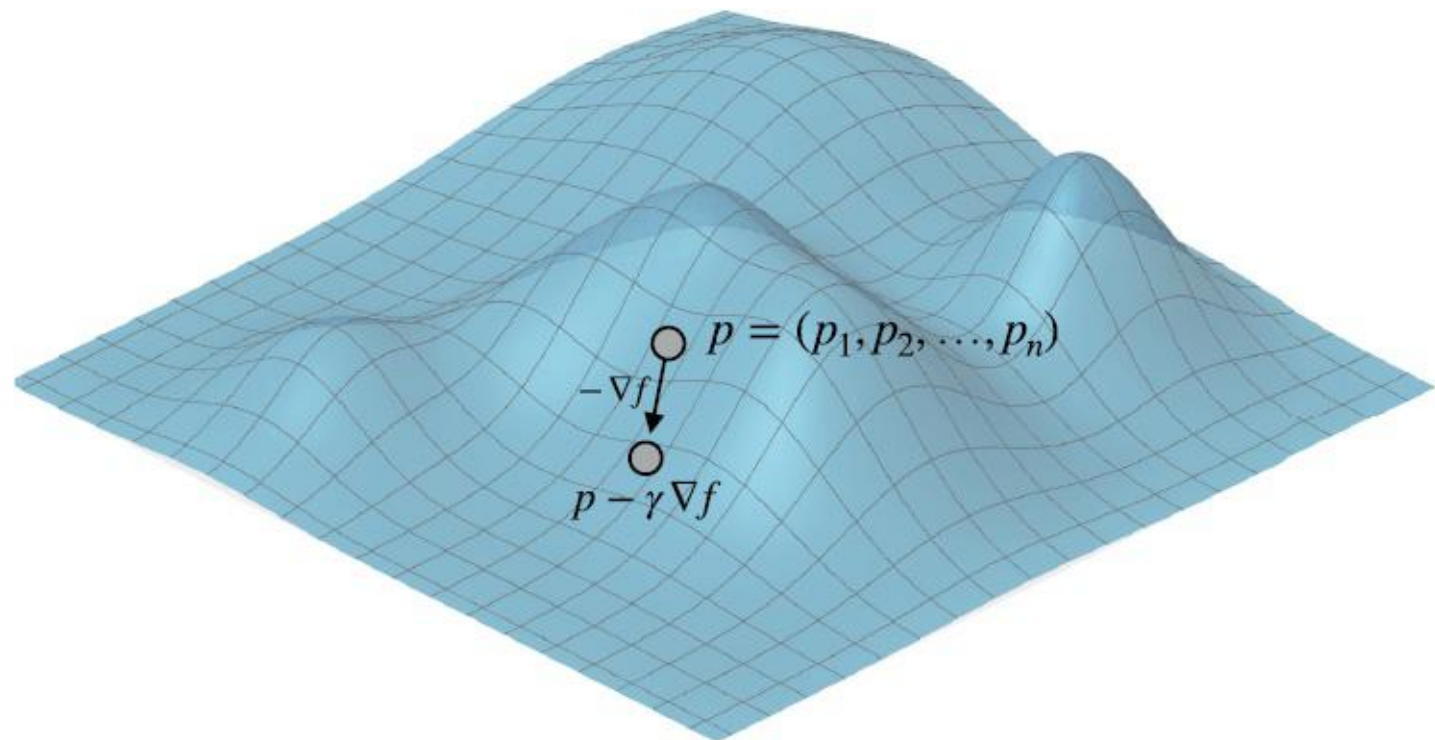
Hình thức toán học của việc giảm độ dốc như sau: giả sử bạn muốn cực tiểu hóa hàm $f(x_1, x_2, \dots, x_n)$ trên các biến n $x_1, x_2, \dots, x_n$. Chúng tôi giả sử hàm số là liên tục và khả vi trên từng biến n .

Hiện tại chúng ta đang đứng tại điểm p có tọa độ $(p_1, p_2, \dots, p_n)$, và chúng ta muốn tìm hướng mà hàm số giảm nhiều nhất để thực hiện bước đó. Điều này được minh họa trong hình B.2. Để tìm hướng mà hàm số giảm nhiều nhất, chúng ta sử dụng *độ dốc* của hàm. Độ dốc là vector n -chiều được hình thành bởi đạo hàm riêng của f đối với từng biến $x_1, x_2, \dots, x_n$. Độ dốc này được ký hiệu là ∇f , như sau:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

Độ dốc là một vector chỉ hướng tăng trưởng lớn nhất, cụ thể là

hướng mà hàm *tăng* nhiều nhất. Như vậy, âm của gradient là hướng mà hàm *giảm* nhiều nhất. Đây là bước chúng tôi muốn thực hiện. Chúng tôi xác định quy mô của bước bằng cách sử dụng *tốc độ học tập* mà chúng tôi đã học ở chương 3 và chúng tôi biểu thị bằng η . Bước giảm độ dốc bao gồm việc thực hiện một bước có độ dài $\eta|\nabla f|$ theo hướng âm của độ dốc ∇f . Như vậy, nếu điểm ban đầu của chúng ta là p thì sau khi áp dụng bước giảm độ dốc, chúng ta thu được điểm $p - \eta\nabla f$. Hình B.2 minh họa bước chúng tôi đã thực hiện để giảm hàm f .



Hình B.2 Ban đầu chúng ta đang ở điểm p . Chúng ta tiến một bước theo hướng âm của gradient và kết thúc ở một điểm mới. Đây là hướng mà hàm số giảm nhiều nhất. (Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>).

Bây giờ chúng ta đã biết cách thực hiện một bước để giảm nhẹ hàm số, chúng ta có thể chỉ cần lặp lại quá trình này nhiều lần để giảm thiểu hàm số của mình. Do đó, mã giả của thuật toán giảm độ dốc như sau:

Mã giả cho thuật toán giảm độ dốc

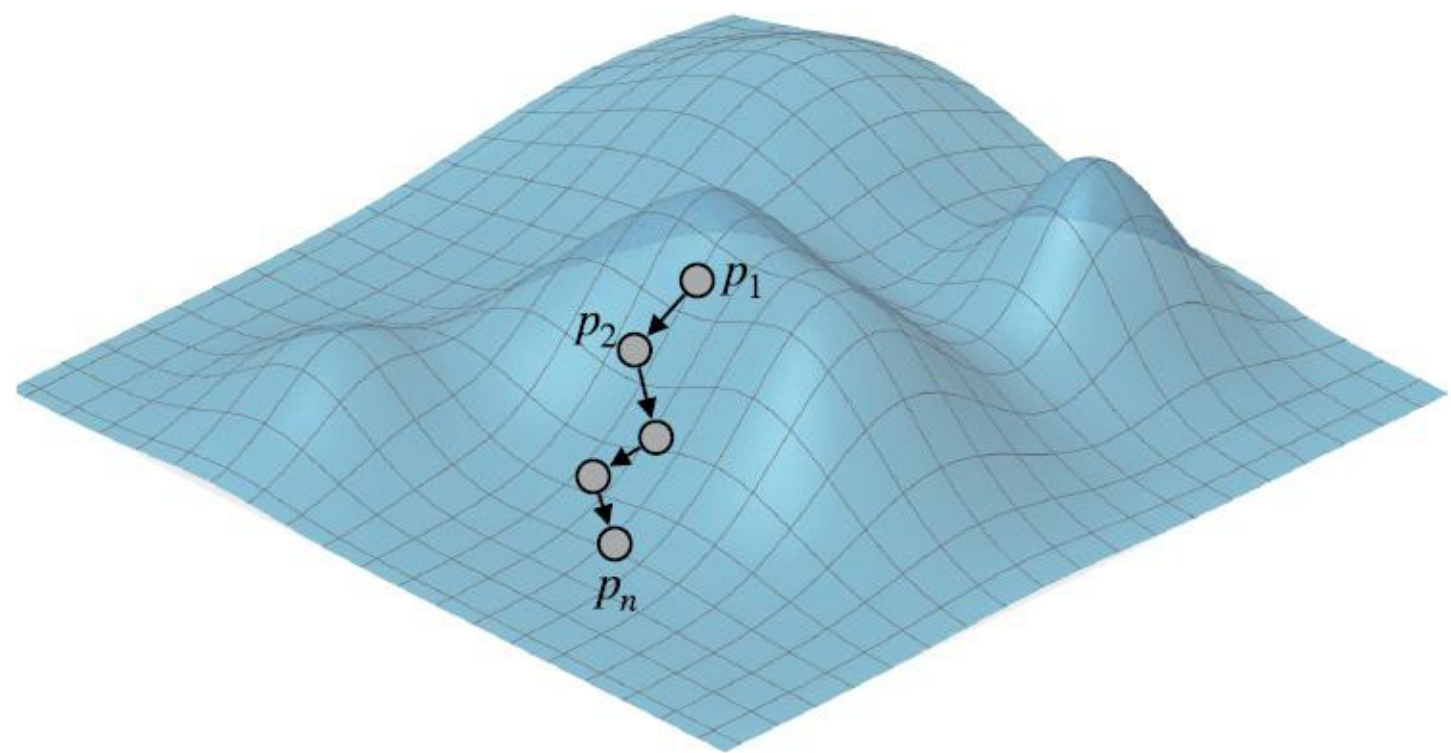
Mục tiêu: Giảm thiểu chức năng f .

Siêu tham số:

- Số kỷ nguyên (lặp lại) N Tốc độ học η
-

Quy trình:

- Chọn một điểm ngẫu nhiên p_0 .
- Với $i = 0, \dots, N - 1$: – Tính gradient $\nabla f(p_i)$.
 - – Chọn điểm $p_{i+1} = p_i - \eta \nabla f(p_i)$.
- Kết thúc bằng dấu p_n .



Hình B.3 Nếu lặp lại bước giảm độ dốc nhiều lần, chúng ta có nhiều khả năng tìm được giá trị nhỏ nhất của hàm. Trong hình này, p_1 đại diện cho điểm bắt đầu và p_n đại diện cho điểm chúng ta đạt được bằng cách sử dụng phương pháp giảm độ dốc.

(Nguồn: Hình ảnh được tạo với sự hỗ trợ của Grapher™ từ Golden Software, LLC;
<https://www.goldensoftware.com/products/grapher>).

Quá trình này *luôn luôn* tìm giá trị nhỏ nhất của hàm số phải không? Thật không may, không.

Một số vấn đề có thể xảy ra khi cố gắng thu nhỏ một hàm bằng cách sử dụng phương pháp giảm độ dốc, chẳng hạn như bị kẹt ở mức tối thiểu cục bộ (thung lũng). Chúng ta sẽ học một kỹ thuật rất hữu ích để giải quyết vấn đề này trong phần “Bị mắc kẹt ở cực tiểu cục bộ”.

Us giảm độ dốc cho mô hình huấn luyện

Bây giờ chúng ta đã biết cách giảm độ dốc giúp chúng ta giảm thiểu (hoặc ít nhất là tìm các giá trị nhỏ cho) một hàm, trong phần này chúng ta sẽ xem cách sử dụng nó để huấn luyện một số mô hình học máy. Các mô hình chúng tôi sẽ đào tạo như sau:

- Hồi quy tuyến tính (từ chương 3). Perceptron (từ chương 5). Bộ phân loại logistic (từ chương 6).
- Mạng lưới thần kinh (từ chương 10). Chính quy hóa (từ chương 4 và 11). Đây không phải là một mô hình, nhưng chúng ta vẫn có thể thấy những tác động của bước giảm độ dốc đối với mô hình sử dụng chính quy.
-

Cách chúng tôi sử dụng phương pháp giảm độ dốc để huấn luyện một mô hình là đặt f là hàm lỗi tương ứng của mô hình và sử dụng phương pháp giảm độ dốc để giảm thiểu f . Giá trị của hàm lỗi được tính toán trên tập dữ liệu. Tuy nhiên, như chúng ta đã thấy trong các phần “Chúng ta đào tạo bằng cách sử dụng một điểm tại một thời điểm hay nhiều điểm” trong chương 3, “Giảm độ dốc ngẫu nhiên, lô nhỏ và lô” ở chương 6 và “Siêu tham số” ở chương 10, nếu tập dữ liệu quá lớn, chúng ta có thể tăng tốc độ đào tạo bằng cách chia tập dữ liệu thành các lô nhỏ có cùng kích thước (gần) và ở mỗi bước, hãy chọn một lô nhỏ khác nhau để tính hàm lỗi.

Dưới đây là một số ký hiệu chúng tôi sẽ sử dụng trong phụ lục này. Hầu hết các thuật ngữ đã được giới thiệu ở chương 1 và 2:

- **kích thước** của tập dữ liệu hoặc số lượng hàng là m . **kích thước** của tập dữ liệu hoặc số cột là n . Tập dữ liệu được hình thành bởi **tính năng** và **nhãn**. **Đặc điểm** là các vectơ $m \times x_i = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$ cho $i = 1, 2, \dots, m$.
-
- Các **nhãn** y_i , cho $i = 1, 2, \dots, m$.

- **mô hình** được cho bởi vectơ n trọng số $w = (w_1, w_2, \dots, w_n)$ và độ lệch b (một đại lượng vô hướng) (trừ khi mô hình là một mạng lưới thần kinh, sẽ có nhiều trọng số và độ lệch hơn). Những **dự đoán** $\hat{y}_i$, cho $i = 1, 2, \dots, m$. **tốc độ học tập** của mô hình là η . Các **các lô nhỏ** dữ liệu là $B_1, B_2, \dots, B_l$, với một số l . Mỗi mini-lô có độ dài q . Các điểm trong một lô nhỏ (để tiện ký hiệu) được ký hiệu là $x^{(1)}, \dots, x^{(q)}$ và nhãn là $y_1, \dots, y_q$.

các Sau đây là thuật toán giảm độ dốc mà chúng tôi sẽ sử dụng cho các mô hình đào tạo :

Thuật toán GIẢM ĐỘ gradien cho MÔ HÌNH HỌC TẬP MÁY ĐÀO TẠO

Siêu tham số:

- Số kỳ nguyên (lặp lại) N Tốc độ học η

Quy trình:

- Chọn các trọng số ngẫu nhiên $w_1, w_2, \dots, w_n$ và độ lệch ngẫu nhiên b .
- Đối với $i = 0, \dots, N - 1$: Đối với mỗi lô nhỏ $B_1, B_2, \dots, B_l$:

- Tính hàm lỗi $f(w, b)$ trên lô nhỏ cụ thể đó.

$$\nabla f(w_1, \dots, w_n, b) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_n}, \frac{\partial f}{\partial b} \right)$$

- Tính gradient Thay thế trọng số và độ lệch như sau:

- w_1 được thay thế bởi
$$w_i' = w_i - \eta \frac{\partial f}{\partial w_i}$$

- b được thay thế bởi
$$b' = b - \eta \frac{\partial f}{\partial b}$$

Xuyên suốt các phần phụ sau, chúng tôi sẽ thực hiện quy trình này một cách chi tiết cho từng mô hình và hàm lỗi sau:

- Mô hình hồi quy tuyến tính với hàm sai số tuyệt đối trung bình (phần sau)

- Mô hình hồi quy tuyến tính với hàm sai số bình phương trung bình (phần sau) Mô hình Perceptron với hàm lỗi perceptron (phần “Sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình phân loại”)
- Mô hình hồi quy logistic với hàm log loss (phần “Sử dụng phương pháp giảm độ dốc để đào tạo các mô hình phân loại”)
- Mạng nơ-ron với chức năng mất log (phần “Sử dụng phương pháp giảm độ dốc để huấn luyện mạng nơ-ron”)
- Các mô hình có tính chính quy hóa (phần “Sử dụng phương pháp giảm độ dốc để chuẩn hóa”)

•

•

Sử dụng phương pháp giảm độ dốc để huấn luyện mô hình hồi quy tuyến tính

Trong phần này, chúng tôi sử dụng phương pháp giảm độ dốc để huấn luyện mô hình hồi quy tuyến tính, sử dụng cả hai hàm sai số mà chúng tôi đã học trước đây: sai số tuyệt đối trung bình và sai số bình phương trung bình. Nhớ lại chương 3 rằng trong hồi quy tuyến tính, các dự đoán $\hat{y}_1, \hat{y}_1, \dots, \hat{y}_q$ được đưa ra theo công thức sau:

$$\hat{y}_i = \sum_{j=1}^n w_j x_j^{(i)} + b$$

Mục tiêu của mô hình hồi quy của chúng tôi là tìm các trọng số $w_1, \dots, w_n$, tạo ra các dự đoán thực sự gần với nhãn. Do đó, hàm lỗi giúp đo lường khoảng cách từ $\hat{y}$ đến y đối với một tập hợp trọng số cụ thể. Như chúng ta đã thấy trong phần “Sai số tuyệt đối” và “Sai số bình phương” ở chương 3, chúng ta có hai cách khác nhau để tính khoảng cách này. Đầu tiên là giá trị tuyệt đối $|\hat{y} - y|$, và thứ hai là bình phương hiệu $(y - \hat{y})^2$. Cái đầu tiên làm phát sinh sai số tuyệt đối trung bình và cái thứ hai gây ra sai số bình phương trung bình. Hãy nghiên cứu chúng một cách riêng biệt.

Đào tạo mô hình hồi quy tuyến tính sử dụng phương pháp giảm độ dốc để giảm thiểu sai số tuyệt đối trung bình

Trong tiểu mục này, chúng ta sẽ tính toán độ dốc của hàm sai số tuyệt đối trung bình và sử dụng nó để áp dụng phương pháp giảm độ dốc và huấn luyện mô hình hồi quy tuyến tính. Sai số tuyệt đối trung bình là một cách để biết $\hat{y}$ và y cách nhau bao xa. Nó được định nghĩa lần đầu tiên trong

phần “Sai số tuyệt đối” ở chương 3 và công thức của nó như sau:

$$MAE(w,b,x,y)=\frac{1}{q}\sum_{i=1}^q|\hat{y}_i-y_i|$$

Để thuận tiện, chúng tôi sẽ viết tắt $MAE(w,b,x,y)$ là MAE . Để sử dụng phương pháp giảm độ dốc nhằm giảm MAE , chúng ta cần tính gradient ∇MAE , là vector chứa $n+1$ đạo hàm riêng của MAE đối với $w_1, \dots, w_n, b$,

$$\nabla MAE=\left(\frac{\partial MAE}{\partial w_1},\dots,\frac{\partial MAE}{\partial w_n},\frac{\partial MAE}{\partial b}\right)$$

Chúng tôi sẽ tính toán mô phỏng các đạo hàm riêng này bằng cách sử dụng quy tắc dây chuyền. Đầu tiên, hãy chú ý rằng

$$\frac{\partial MAE}{\partial w_j}=\frac{1}{q}\sum_{i=1}^q\frac{\partial |\hat{y}_i-y_i|}{\partial w_j}$$

Đạo hàm của $f(x)=|x|$ là hàm dấu, là $+1$ khi x dương và -1 khi x âm (nó không được xác định tại 0 , nhưng để thuận tiện, chúng ta có thể định nghĩa nó là 0). Vì vậy, chúng ta có thể viết lại phương trình trước đó thành

$$\frac{\partial MAE}{\partial w_i}=\frac{1}{q}\sum_{i=1}^q\overset{sgn(x)=\frac{|x|}{x}}{sgn(\hat{y}_i-y_i)}\frac{\partial \hat{y}_i}{\partial w_j}$$

Để tính giá trị này, hãy tập trung vào phần cuối cùng của phương trình, cụ thể là,

. Vì , thì $\hat{y}_i=\sum_{j=1}^nw_jx_j^{(i)}$

$$\frac{\partial \hat{y}_i}{\partial w_j}=\sum_{j=1}^n\frac{\partial \left(w_jx_j^{(i)}\right)}{\partial w_j}=x_j^{(i)}.$$

Điều này là do đạo hàm của w_j đối với w_i , là 1 nếu $j=i$ và 0 nếu ngược lại.

Do đó, thay thế vào đạo hàm, ta thu được:

$$\frac{\partial MAE}{\partial w_i}=\frac{1}{q}\sum_{i=1}^qsgn\left(\hat{y}_i-y_i\right)x_j^{(i)}$$

Bằng cách sử dụng phân tích tương tự, chúng ta có thể tính đạo hàm của $MAE(w,b)$ đối với b là

$$\frac{\partial MAE}{\partial b} = \frac{1}{q} \sum_{i=1}^q \text{sgn}(\hat{y}_i - y_i)$$

Bước giảm độ dốc như sau:

BƯỚC HUỚNG DỐC:

Thay thế (w, b) bằng (w', b') , trong đó

- $w_j' = w_j + \frac{1}{q} \sum_{i=1}^q \eta \text{sgn}(y_i - \hat{y}_i) x_j^{(i)}$ for $i = 1, 2, \dots, n$
- $b' = b + \eta \sum_{i=1}^q \text{sgn}(y_i - \hat{y}_i)$

Lưu ý một điều thú vị: nếu lô nhỏ có kích thước $q = 1$ và chỉ bao gồm điểm $x = (x_1, x_2, \dots, x_n)$ có nhãn y và dự đoán $\hat{y}$ thì bước này được xác định

như sau:

Thay thế (w, b) bằng (w', b') , trong đó

- $w_j' = w_j + \eta \text{sgn}(y - \hat{y}) x_j$
- $b' = b + \eta \text{sgn}(y - \hat{y})$

Đây chính xác là thủ thuật đơn giản mà chúng tôi đã sử dụng trong phần “Thủ thuật đơn giản” ở chương 3 để huấn luyện thuật toán hồi quy tuyến tính.

Đào tạo mô hình hồi quy tuyến tính sử dụng phương pháp giảm độ dốc để giảm thiểu lỗi bình phương trung bình

Trong tiêu mục này, chúng ta sẽ tính toán độ dốc của hàm sai số bình phương trung bình và sử dụng nó để áp dụng phương pháp giảm độ dốc và huấn luyện mô hình hồi quy tuyến tính. Sai số bình phương trung bình là một cách khác để biết $\hat{y}$ và y cách nhau bao xa. Nó được định nghĩa lần đầu tiên trong phần “Sai số bình phương” ở chương 3 và công thức của nó là

$$MSE(w, b, x, y) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Để thuận tiện, chúng tôi sẽ viết tắt $MSE(w, b, x, y)$ là MSE . Để tính gradient ∇MSE , chúng ta có thể làm theo quy trình tương tự như đã làm đối với sai số tuyệt đối trung bình

được mô tả trước đó, ngoại trừ đạo hàm của $f(x) = x^2$ là $2x$. Do đó, đạo hàm của MSE đối với w_j là

$$\frac{\partial MSE}{\partial w_j} = \frac{1}{2q} \sum_{i=1}^q \frac{\partial (\hat{y}_i - y_i)^2}{\partial w_j} = \frac{1}{2q} \sum_{i=1}^q 2(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j} = \frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j}$$

Tương tự, đạo hàm của $MSE(w, b)$ đối với b là

$$\frac{\partial MSE}{\partial b} = \frac{1}{2q} \sum_{i=1}^q \frac{\partial (\hat{y}_i - y_i)^2}{\partial b} = \frac{1}{2q} \sum_{i=1}^q 2(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial b} = \frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial b}$$

BƯỚC HƯỚNG DỐC:

Thay thế (w, b) bằng (w', b') , trong đó

- $w_j' = w_j + \eta \frac{1}{q} \sum_{i=1}^q (y_i - \hat{y}_i) x_j^{(i)}$ for $i = 1, 2, \dots, n$
- $b' = b + \eta \frac{1}{q} \sum_{i=1}^q (y_i - \hat{y}_i)$

Một lần nữa lưu ý rằng nếu lô nhỏ có kích thước $q = 1$ và chỉ bao gồm điểm $x = (x_1, x_2, \dots, x_n)$ có nhãn y và dự đoán $\hat{y}$ thì bước này được xác định như sau:

Thay thế (w, b) bằng (w', b') , trong đó

- $w_j' = w_j + \eta(y - \hat{y})x_j$
- $b' = b + \eta(y - \hat{y})$

Đây chính xác là thủ thuật bình phương mà chúng tôi đã sử dụng trong phần “Thủ thuật bình phương” ở chương 3 để huấn luyện thuật toán hồi quy tuyến tính.

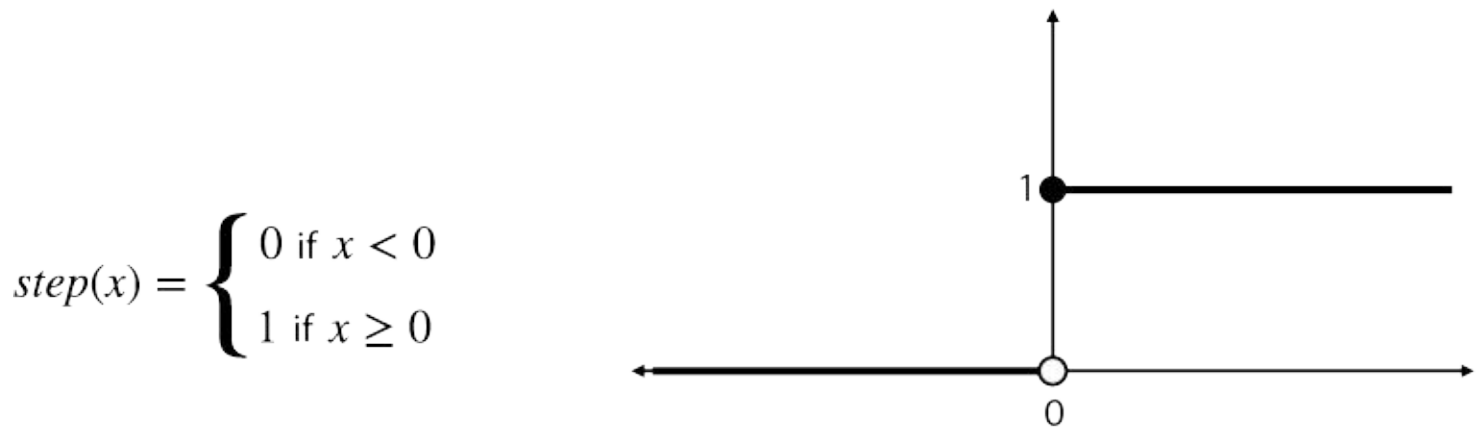
Sử dụng người đi theo mô hình phân loại đào tạo

Trong phần này, chúng ta tìm hiểu cách sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình phân loại. Hai mô hình mà chúng ta sẽ đào tạo là mô hình perceptron (chương 5) và mô hình hồi quy logistic (chương 6). Mỗi một trong số chúng đều có chức năng báo lỗi riêng nên chúng tôi sẽ phát triển chúng một cách riêng biệt.

ĐÀO TẠO MÔ HÌNH PERCEPTRON SỬ DỤNG HƯỚNG DẪN GRADIENT ĐỂ GIẢM THIỂU LỖI PERCEPTRON

Trong tiểu mục này, chúng ta sẽ tính toán độ dốc của hàm lỗi perceptron và sử dụng nó để áp dụng phương pháp giảm độ dốc và huấn luyện mô hình perceptron. Trong mô hình perceptron, các dự đoán là $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q$ trong đó mỗi $\hat{y}_i$ là 0 hoặc 1. Để tính toán

dự đoán, trước tiên chúng ta cần nhớ hàm bước $step(x)$, được giới thiệu ở chương 5. Hàm này lấy bất kỳ số thực x nào làm đầu vào và xuất ra 0 nếu $x < 0$ và 1 nếu $x \geq 0$. Đồ thị của nó được hiển thị trong hình B.4.



Hình B.4 Hàm bước. Đối với số âm, nó xuất ra 0 và đối với các số không âm, nó xuất ra 1.

Mô hình cho mỗi điểm một *điểm*. Điểm số mà mô hình có trọng số ($w_1, w_2, \dots, w_n$) và sự thiên vị b cho điểm $x^{(i)} = (x_1^{(i)}, x_n^{(i)}, \dots, x_n^{(i)})$ là

$$score(w, b, x^{(i)}) = \sum_{j=1}^n w_j x_j^{(i)} + b$$

Các dự đoán $\hat{y}_i$ được đưa ra theo công thức sau:

$$\hat{y}_i = step(score(w, b, x^{(i)})) = step\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right)$$

Nói cách khác, dự đoán là 1 nếu điểm dương và 0 nếu ngược lại.

Hàm lỗi perceptron được gọi là $PE(w, b, x, y)$, chúng ta sẽ viết tắt là PE . Nó được định nghĩa lần đầu trong phần "Làm thế nào để so sánh các bộ phân loại? Hàm lỗi" trong chương 5. Theo cách xây dựng, đó là một con số lớn nếu mô hình đưa ra dự đoán xấu và một con số nhỏ (trong trường hợp này thực tế là 0) nếu mô hình đưa ra dự đoán tốt. Hàm lỗi được định nghĩa như sau.

- $PE(w, b, x, y) = 0$ if $\hat{y} = y$
- $|điểm(w, b, x)|$ if $\hat{y} \neq y$

Nói cách khác, nếu điểm được phân loại chính xác thì sai số sẽ bằng 0. Nếu điểm được phân loại không chính xác thì sai số là giá trị tuyệt đối của điểm. Do đó, các điểm bị phân loại sai có điểm có giá trị tuyệt đối thấp sẽ tạo ra sai số thấp và các điểm bị phân loại sai có điểm có giá trị tuyệt đối cao sẽ tạo ra sai số cao. Điều này là do giá trị tuyệt đối của điểm của một điểm tỷ lệ thuận với khoảng cách giữa điểm đó và ranh giới. Do đó, những điểm có sai số thấp là những điểm ở gần ranh giới, trong khi những điểm có sai số cao nằm ở xa ranh giới.

Để tính gradient ∇PE , chúng ta có thể sử dụng quy tắc tương tự như trước. Một điều chúng ta cần lưu ý là đạo hàm của hàm giá trị tuyệt đối $|x|$ là 1 khi $x \geq 0$ và 0 khi $x < 0$. Đạo hàm này không được xác định tại 0, đây là một vấn đề với tính toán của chúng tôi, nhưng trên thực tế, chúng ta có thể tùy ý định nghĩa nó là 1 mà không gặp vấn đề gì.

Trong chương 10, chúng ta đã giới thiệu hàm $ReLU(x)$ (đơn vị tuyến tính chỉnh lưu), bằng 0 khi $x < 0$ và x khi $x \geq 0$. Lưu ý rằng có hai cách để một điểm có thể bị phân loại sai:

- Nếu $y = 0$ và $\hat{y} = 1$. Điều này có nghĩa là $\text{điểm}(w, b, x) \geq 0$.
- Nếu $y = 1$ và $\hat{y} = 0$. Điều này có nghĩa là $\text{điểm}(w, b, x) < 0$.

Vì vậy, chúng ta có thể viết lại lỗi perceptron một cách thuận tiện như sau:

$$PE = \sum_{i=1}^q y_i ReLU(-score(w, b, x)) + (1 - y_i) ReLU(score(w, b, x))$$

hoặc chi tiết hơn, như

$$PE = \sum_{i=1}^q y_i ReLU\left(-\sum_{j=1}^n w_j x_j^{(i)} - b\right) + (1 - y_i) ReLU\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right)$$

Bây giờ chúng ta có thể tiến hành tính gradient ∇PE bằng quy tắc dây chuyền. Một quan sát quan trọng mà chúng tôi sẽ sử dụng và người đọc có thể xác minh là đạo hàm của $ReLU(x)$ là hàm bước $\text{bước}(x)$. Độ dốc này là

$$\frac{\partial PE}{\partial w_j} = \sum_{i=1}^q y_i \text{step}\left(-\sum_{j=1}^n w_j x_j^{(i)} - b\right) (-x_j^{(i)}) + (1 - y_i) \text{step}\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right) x_j^{(i)}$$

mà chúng ta có thể viết lại thành

$$\frac{\partial PE}{\partial w_j} = \sum_{i=1}^q -y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x))$$

Điều này có vẻ phức tạp nhưng thực ra không khó đến thế. Hãy phân tích từng mệnh đề từ vế phải của biểu thức trước đó. Lưu ý rằng $bước(điểm(w, b, x))$ là 1 khi và chỉ khi $điểm(w, b, x) > 0$, còn ngược lại thì bằng 0. Đây chính xác là khi $\hat{y} = 1$. Tương tự, $bước(-điểm(w, b, x))$ là 1 khi và chỉ khi $điểm(w, b, x) < 0$, và ngược lại, nó là 0. Đây chính xác là khi $\hat{y} = 0$. Do đó

- Nếu $\hat{y}_i = 0$ và $y_i = 0$:

$$-y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x)) = 0$$

- Nếu $\hat{y}_i = 1$ và $y_i = 1$:

$$-y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x)) = 0$$

- Nếu $\hat{y}_i = 0$ và $y_i = 1$:

$$-y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x)) = -x_j^{(i)}$$

- Nếu $\hat{y}_i = 1$ và $y_i = 0$:

$$-y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x)) = x_j^{(i)}$$

Điều này có nghĩa là khi tính toán $\partial PE / \partial x_j$, chỉ có các khoản tiền đến từ điểm phân loại sai sẽ thêm giá trị.

Theo cách tương tự,

$$\frac{\partial PE}{\partial b} = \sum_{i=1}^q -y_i \text{step}(-score(w, b, x)) + (1 - y_i) \text{step}(score(w, b, x))$$

Do đó, bước giảm độ dốc được xác định như sau:

BƯỚC HUỚNG DỐC:

Thay thế (w, b) bằng (w', b') , trong đó

- $w_j' = w_j + \eta \sum_{i=1}^q -y_i x_j^{(i)} \text{step}(-score(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(score(w, b, x))$,
and
- $b' = b + \eta \sum_{i=1}^q -y_i \text{step}(-score(w, b, x)) + (1 - y_i) \text{step}(score(w, b, x))$

Và một lần nữa, nhìn vào phía bên phải của biểu thức trước đó

- Nếu $\hat{y}_i = 0$ và $y_i = 0$:

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 0$$

- Nếu $\hat{y}_i = 1$ và $y_i = 1$:

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 0$$

- Nếu $\hat{y}_i = 0$ và $y_i = 1$:

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = -1$$

- Nếu $\hat{y}_i = 1$ và $y_i = 0$:

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 1$$

Tất cả điều này có thể không có nhiều ý nghĩa, nhưng người ta có thể viết mã này để tính toán tất cả các mục của gradient. Một lần nữa lưu ý rằng nếu lô nhỏ có kích thước $q = 1$ và chỉ bao gồm điểm $x = (x_1, x_2, \dots, x_n)$ có nhãn y và dự đoán $\hat{y}$ thì bước này được xác định như sau:

BƯỚC HUỚNG DỐC:

- Nếu điểm được phân loại chính xác, không thay đổi w và b . Nếu điểm
- có nhãn $y = 0$ và được phân loại là $\hat{y} = 1$: Thay w bằng $w' = w - \eta x$. Thay b bằng $b' = w - \eta$. Nếu điểm có nhãn $y = 1$ và được phân loại là $\hat{y} = 0$: Thay thế w bằng $w' = w + \eta x$. Thay b bằng $b' = w + \eta$.
- -
 -

Lưu ý rằng đây chính xác là thủ thuật perceptron được mô tả trong phần “Thủ thuật perceptron” ở chương 5.

ĐÀO TẠO MÔ HÌNH HỒI CHUYỂN LOGISTIC SỬ DỤNG GIẢM GIÁ GRADIENT ĐỂ GIẢM THIỂU MẤT LOG

Trong tiêu mục này, chúng ta sẽ tính toán độ dốc của hàm mất log và sử dụng nó để

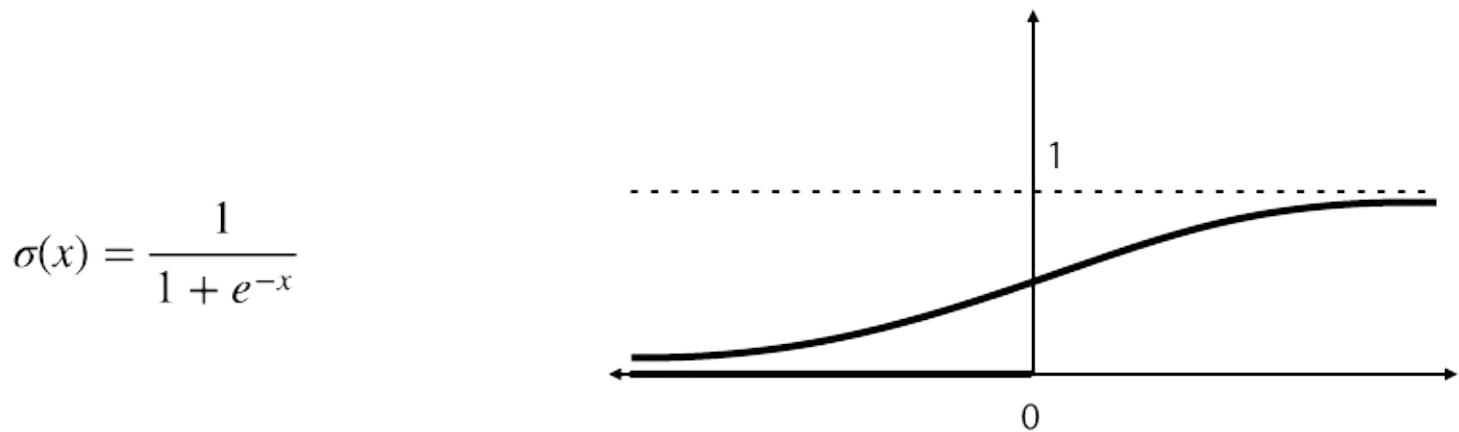
áp dụng phương pháp giảm độ dốc và huấn luyện mô hình hồi quy logistic. Trong mô hình hồi quy logistic, các dự đoán là $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q$ trong đó mỗi $\hat{y}_i$ là một số thực nào đó nằm trong khoảng từ 0 đến 1. Để tính toán các dự đoán, trước tiên chúng ta cần nhớ hàm sigmoid $\sigma(x)$, được giới thiệu trong chương 6. Hàm này lấy bất kỳ số thực x nào làm đầu vào và xuất ra một số từ 0 đến 1. Nếu x là số dương lớn thì $\sigma(x)$ gần bằng 1. Nếu x là số âm lớn thì $\sigma(x)$ gần bằng 0. Công thức của hàm sigmoid là

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Đồ thị của $\sigma(x)$ được minh họa trên hình B.5.

Các dự đoán của mô hình hồi quy logistic chính xác là đầu ra của hàm sigmoid, cụ thể, đối với $i = 1, 2, \dots, q$ chúng được định nghĩa như sau:

$$\hat{y}_i = \sigma(\text{score}(w, b, x^{(i)})) = \sigma\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right)$$



Hình B.5 Hàm sigmoid luôn xuất ra một số từ 0 đến 1. Đầu ra cho số âm gần bằng 0 và đầu ra cho số dương gần bằng 1.

Việc mất nhật ký được ký hiệu là $LL(w, b, x, y)$, chúng tôi sẽ viết tắt là LL . Hàm lỗi này lần đầu tiên được định nghĩa trong phần “Tập dữ liệu và dự đoán” ở chương 6. Nó tương tự như hàm lỗi perceptron vì theo cách xây dựng, đó là một số lớn nếu mô hình đưa ra dự đoán xấu và một số nhỏ nếu mô hình đưa ra dự đoán tốt. Hàm mất log được định nghĩa là

$$LL = -\sum_{i=1}^q y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

Chúng ta có thể tiến hành tính gradient ∇LL bằng quy tắc dây chuyền. Trước đó, hãy lưu ý rằng đạo hàm của hàm sigmoid có thể được viết là $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$. Các chi tiết về phép tính cuối cùng có thể được tính toán bằng cách sử dụng quy tắc thương số để lấy vi phân và chúng dành cho người đọc. Sử dụng cách này, chúng ta có thể tính đạo hàm của $\hat{y}_i$ đối với w_j . Bởi vì $\hat{y}_i = \sigma(\sum_{j=1}^n w_j x_j^{(i)} + b)$, thì theo quy tắc dây chuyền,

$$\frac{\partial \hat{y}_i}{\partial w_j} = \sigma \left(\sum_{j=1}^n (w_j x_j^{(i)} + b) \right) \left[1 - \sigma \left(\sum_{j=1}^n (w_j x_j^{(i)} + b) \right) \right] x_j^{(i)} = y_i(1 - y_i)x_j^{(i)}$$

Bây giờ, hãy tiếp tục phát triển phần log loss. Sử dụng lại quy tắc dây chuyền, chúng ta nhận được

$$\frac{\partial LL}{\partial w_j} = - \sum_{i=1}^q y_i \frac{1}{\hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_j} - (1 - y_i) \frac{-1}{1 - \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_j}$$

Và theo phép tính trước đó cho $\partial \hat{y}_i / \partial w_j$,

$$\frac{\partial LL}{\partial w_j} = \sum_{i=1}^q -y_i \frac{1}{\hat{y}_i} \hat{y}_i (1 - \hat{y}_i) x_j^{(i)} + (1 - y_i) \frac{-1}{1 - \hat{y}_i} \hat{y}_i (1 - \hat{y}_i) x_j^{(i)}$$

Đơn giản hóa, chúng tôi nhận được

$$\frac{\partial LL}{\partial w_j} = \sum_{i=1}^q -y_i (1 - \hat{y}_i) x_j^{(i)} + (1 - y_i) \hat{y}_i x_j^{(i)}$$

điều này thậm chí còn đơn giản hóa hơn nữa vì

$$\frac{\partial LL}{\partial w_j} = \sum_{i=1}^q (\hat{y}_i - y_i) x_j^{(i)}$$

Tương tự, lấy đạo hàm theo b , ta được

$$\frac{\partial LL}{\partial b} = \sum_{i=1}^q (\hat{y}_i - y_i)$$

Do đó, bước giảm độ dốc sẽ trở thành như sau:

BƯỚC HƯỚNG DẪM:

Thay thế (w, b) bằng (w', b') , trong đó

- $w' = w + \eta \sum_{i=1}^q (y_i - \hat{y}_i) x^{(i)}$

- $b' = b + \eta \sum_{i=1}^q (y_i - \hat{y}_i)$

Lưu ý rằng khi các lô nhỏ có kích thước 1, bước giảm độ dốc sẽ như sau:

Thay thế (w, b) bằng (w', b') , trong đó

- $w' = w + \eta (y_i - \hat{y}_i) x^{(i)}$
- $b' = b + \eta (y_i - \hat{y}_i)$

Đây chính xác là thủ thuật hồi quy logistic mà chúng ta đã học trong phần “Làm thế nào để tìm một bộ phân loại logistic tốt?” trong chương 6.

Nhà máy giảm độ dốc để huấn luyện mạng lưới thần kinh

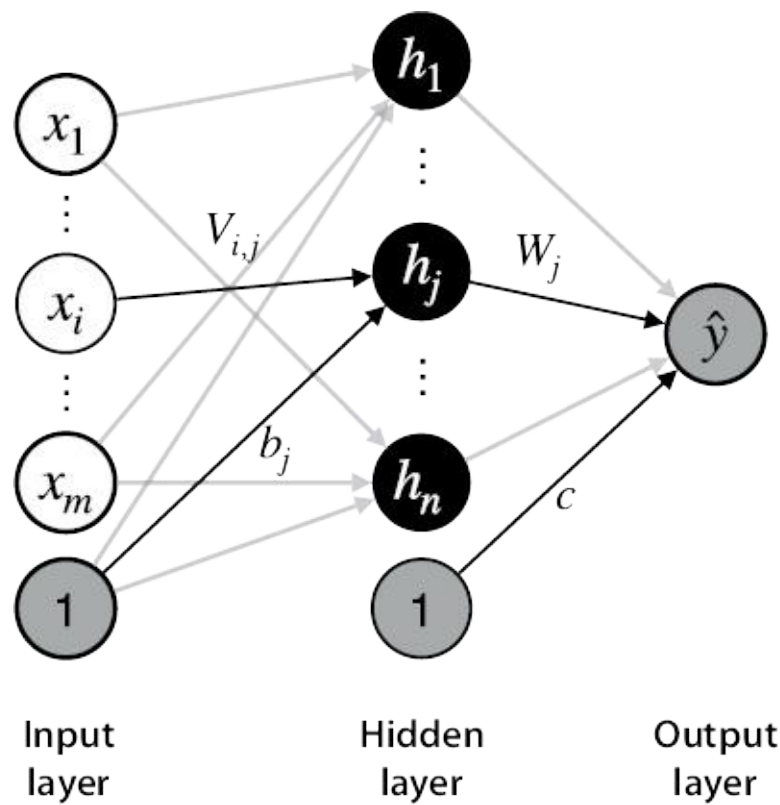
Trong phần “lan truyền ngược” ở chương 10, chúng ta đã tìm hiểu về lan truyền ngược—quá trình huấn luyện mạng lưới thần kinh. Quá trình này bao gồm việc lặp lại bước giảm độ dốc để giảm thiểu mất mát ký. Trong tiểu mục này, chúng ta thấy cách tính toán đạo hàm thực sự để thực hiện bước giảm độ dốc này. Chúng tôi sẽ thực hiện quá trình này trong mạng nơ-ron có độ sâu 2 (một lớp đầu vào, một lớp ẩn và một lớp đầu ra), nhưng ví dụ này đủ lớn để cho thấy cách tính toán các đạo hàm này nói chung. Hơn nữa, chúng tôi sẽ áp dụng giảm độ dốc cho lỗi chỉ cho một điểm (nói cách khác, chúng tôi sẽ thực hiện giảm độ dốc ngẫu nhiên). Tuy nhiên, tôi khuyến khích bạn tìm ra các đạo hàm cho mạng lưới thần kinh gồm nhiều lớp hơn và sử dụng các lô điểm nhỏ (giảm độ dốc của lô nhỏ).

Trong mạng nơ-ron của chúng ta, lớp đầu vào bao gồm m nút đầu vào, lớp ẩn bao gồm n nút ẩn và lớp đầu ra bao gồm một nút đầu ra. Ký hiệu trong này

tiểu mục này khác với các tiểu mục khác, vì mục đích đơn giản, như sau (và được minh họa trong hình B.6):

- Đầu vào là điểm có tọa độ $x_1, x_2, \dots, x_m$.
- Lớp ẩn đầu tiên có trọng số V_{ij} và độ lệch b_j , cho $i = 1, 2, \dots, m$ và $j = 1, 2, \dots, n$. Lớp ẩn thứ hai có các trọng số W_j cho $j = 1, 2, \dots, n$ và độ lệch c .

-



$$h_j = \sigma \left(\sum_{i=1}^m x_i V_{ij} + b_j \right)$$

$$\hat{y} = \sigma \left(\sum_{j=1}^n h_j W_j + c \right)$$

Hình B.6 Quá trình tính toán dự đoán bằng cách sử dụng mạng nơ-ron với một lớp ẩn và các hàm kích hoạt sigmoid

Cách tính đầu ra thông qua hai phương trình sau:

- $h_j = \sigma \left(\sum_{i=1}^m x_i V_{ij} + b_j \right)$
- $\hat{y} = \sigma \left(\sum_{j=1}^n h_j W_j + c \right)$

Để dễ dàng tính đạo hàm, chúng ta sử dụng các biến trợ giúp r_j và s sau:

- $r_j = \sum_{i=1}^m x_i V_{ij} + b_j$
- $h_j = \sigma(r_j)$
- $s = \sum_{j=1}^n h_j W_j + c$
- $\hat{y} = \sigma(s)$

Bằng cách đó, chúng ta có thể tính đạo hàm riêng sau đây (nhớ rằng đạo hàm của hàm sigmoid là $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$ và log loss là $L(y, \hat{y}) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$ —chúng ta sẽ gọi nó là L cho tiện lợi):

$$1. \quad \frac{\partial L}{\partial \hat{y}} = -y \frac{1}{\hat{y}} - (1-y) \frac{-1}{1-\hat{y}} = \frac{-(y-\hat{y})}{\hat{y}(1-\hat{y})}$$

$$2. \quad \frac{\partial \hat{y}}{\partial s} = \sigma(s)[1-\sigma(s)] = y(1-y)$$

$$3. \quad \frac{\partial s}{\partial W_j} = h_j, \text{ and } \frac{\partial s}{\partial b_j} = 1$$

$$4. \quad \frac{\partial s}{\partial h_j} = W_j$$

$$5. \quad \frac{\partial h_j}{\partial r_j} = \sigma(r_j)[1-\sigma(r_j)] = h_j(1-h_j)$$

$$6. \quad \frac{\partial r_j}{\partial V_{ij}} = x_i \text{ and } \frac{\partial r_j}{\partial b_j} = 1$$

Để đơn giản hóa các phép tính, hãy lưu ý rằng nếu chúng ta nhân phương trình 1 và 2 và sử dụng quy tắc dây chuyền, chúng ta sẽ nhận được

$$7. \quad \frac{\partial L}{\partial s} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s} = \frac{-(y-\hat{y})}{\hat{y}(1-\hat{y})} \hat{y}(1-\hat{y}) = -(y-\hat{y})$$

Bây giờ, chúng ta có thể sử dụng quy tắc dây chuyền và các phương trình 3–7 để tính đạo hàm của log loss theo trọng số và độ lệch như sau:

$$8. \quad \frac{\partial L}{\partial W_j} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial W_j} = -(y-\hat{y})h_j$$

$$9. \quad \frac{\partial L}{\partial c} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial c} = -(y-\hat{y})$$

$$10. \quad \frac{\partial L}{\partial V_{ij}} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial h_j} \frac{\partial h_j}{\partial r_j} \frac{\partial r_j}{\partial V_{ij}} = -(y-\hat{y})W_j h_j(1-h_j)x_i$$

$$11. \quad \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial h_j} \frac{\partial h_j}{\partial r_j} \frac{\partial r_j}{\partial b_j} = -(y-\hat{y})W_j h_j(1-h_j)$$

Sử dụng các phương trình trước đó, bước giảm độ dốc như sau:

GRADIENT DESCENT STEP FOR NEURAL NETWORKS:

- Replace V_{ij} by $V_{ij} - \eta \frac{\partial L}{\partial V_{ij}} = V_{ij} + \eta(y - \hat{y})W_j h_j(1 - h_j)x_i$.
- Replace b_j by $b_j - \eta \frac{\partial L}{\partial b_j} = b_j + \eta(y - \hat{y})W_j h_j(1 - h_j)$.
- Replace W_j by $W_j + \eta \frac{\partial L}{\partial W_j} = W_j + \eta(y - \hat{y})h_j$.
- Replace c by $c + \eta \frac{\partial L}{\partial c} = c + \eta(y - \hat{y})$.

Các phương trình trước đây khá phức tạp và các phương trình lan truyền ngược cho mạng lưới thần kinh có nhiều lớp hơn nữa cũng vậy. Rất may, chúng ta có thể sử dụng PyTorch, TensorFlow và Keras để huấn luyện mạng lưới thần kinh mà không cần phải tính toán tất cả các đạo hàm.

Sử dụng tính năng giảm độ dốc để điều chỉnh

Trong phần “Sửa đổi hàm lỗi để giải quyết vấn đề của chúng ta” ở chương 4, chúng ta đã học về chính quy hóa như một cách để giảm việc trang bị quá mức trong các mô hình học máy. Chính quy hóa bao gồm việc thêm một thuật ngữ chính quy hóa vào hàm lỗi, giúp giảm việc trang bị quá mức. Thuật ngữ này có thể là chuẩn L1 hoặc L2 của đa thức được sử dụng trong mô hình. Trong phần “Các kỹ thuật huấn luyện mạng nơ-ron” ở chương 10, chúng ta đã học cách áp dụng chính quy hóa để huấn luyện mạng nơ-ron bằng cách thêm một thuật ngữ chính quy hóa tương tự. Sau đó, trong phần “Hàm lỗi khoảng cách” ở chương 11, chúng ta đã học về hàm lỗi khoảng cách cho SVM, hàm này đảm bảo rằng hai dòng trong bộ phân loại luôn ở gần nhau. Hàm lỗi khoảng cách có dạng giống như thuật ngữ chính quy L2.

Tuy nhiên, trong phần “Một cách trực quan để xem sự chính quy” ở chương 4, chúng ta đã học được một cách trực quan hơn để xem sự chính quy. Nói tóm lại, mỗi bước giảm độ dốc sử dụng chính quy hóa sẽ làm giảm các giá trị của các hệ số của mô hình một lượng nhỏ. Chúng ta hãy nhìn vào toán học đằng sau hiện tượng này.

Đối với mô hình có trọng số $w_1, w_2, \dots, w_n$, các thuật ngữ chính quy hóa như sau:

- L1 chính quy hóa: $W_1 = |w_1| + |w_2| + \dots + |w_n|$
- L2 chính quy hóa: $W_2 = w_1^2 + w_2^2 + \dots + w_n^2$

Hãy nhớ lại rằng để không làm thay đổi các hệ số quá mạnh, thuật ngữ chính quy hóa được nhân với tham số chính quy hóa λ . Do đó, khi chúng ta áp dụng phương pháp giảm độ dốc, các hệ số được sửa đổi như sau:

- L1 chính quy hóa: w_i được thay thế bằng $w_i - \nabla W_1$
- L2 chính quy hóa: w_i được thay thế bằng $w_i - \nabla W_2$

Trong đó ∇ biểu thị độ dốc của thuật ngữ chính quy hóa. Nói cách khác,

$$\nabla(b_1, \dots, b_n) = \left(\frac{\partial b_1}{\partial a_1}, \dots, \frac{\partial b_n}{\partial a_n} \right), \quad \frac{\partial |w_i|}{\partial w_i} = \text{sgn}(w_i) \quad \frac{\partial w_i^2}{\partial w_i} = 2w_i$$

, Vì vậy, và sau đó bước giảm độ dốc như sau:

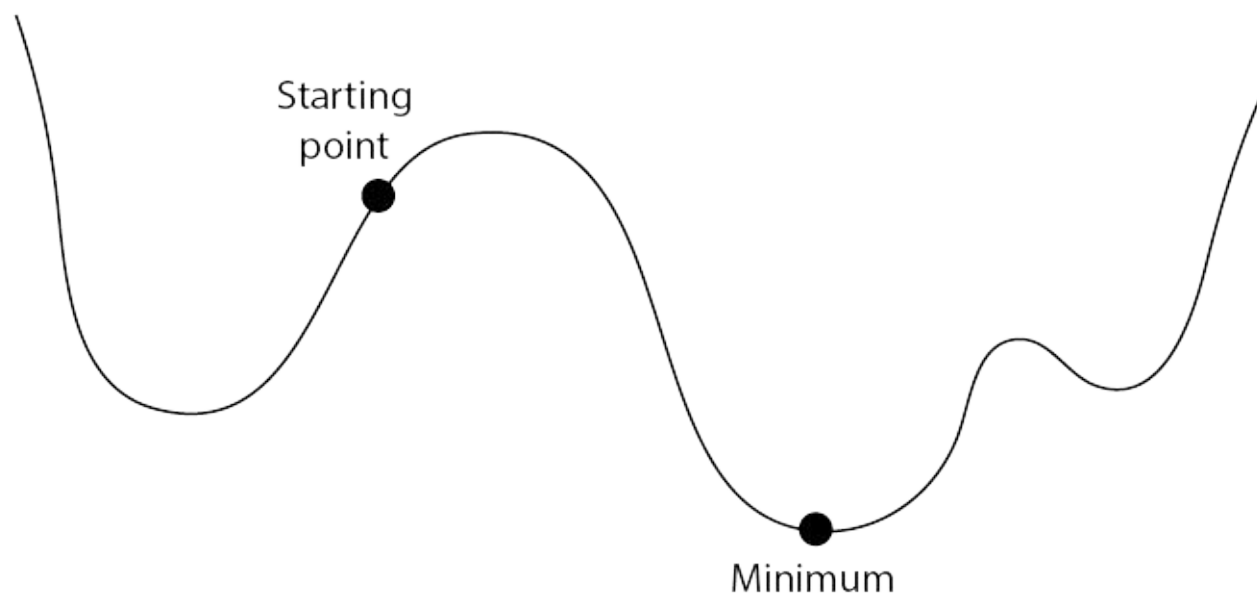
BƯỚC GIẢM ĐỘ CỐ ĐỊNH ĐỂ ĐIỀU CHỈNH:

- Chính quy hóa L1: Thay thế a_i bằng
$$a_i - \lambda \frac{\partial a_i}{\partial a_i} = a_i - \lambda \cdot \text{sgn}(a_i)$$
- Chính quy hóa L2: Thay thế a_i bằng
$$a_i - \lambda \frac{\partial a_i}{\partial a_i} = a_i - 2\lambda a_i = (1 - 2\lambda)a_i$$

Lưu ý rằng bước giảm độ dốc này luôn làm giảm giá trị tuyệt đối của hệ số a_i . Trong quá trình chính quy hóa L1, chúng ta trừ một giá trị nhỏ khỏi a_i nếu nó dương và thêm một giá trị nhỏ nếu nó âm. Trong quá trình chính quy hóa L2, chúng ta nhân a_i với một số nhỏ hơn 1 một chút.

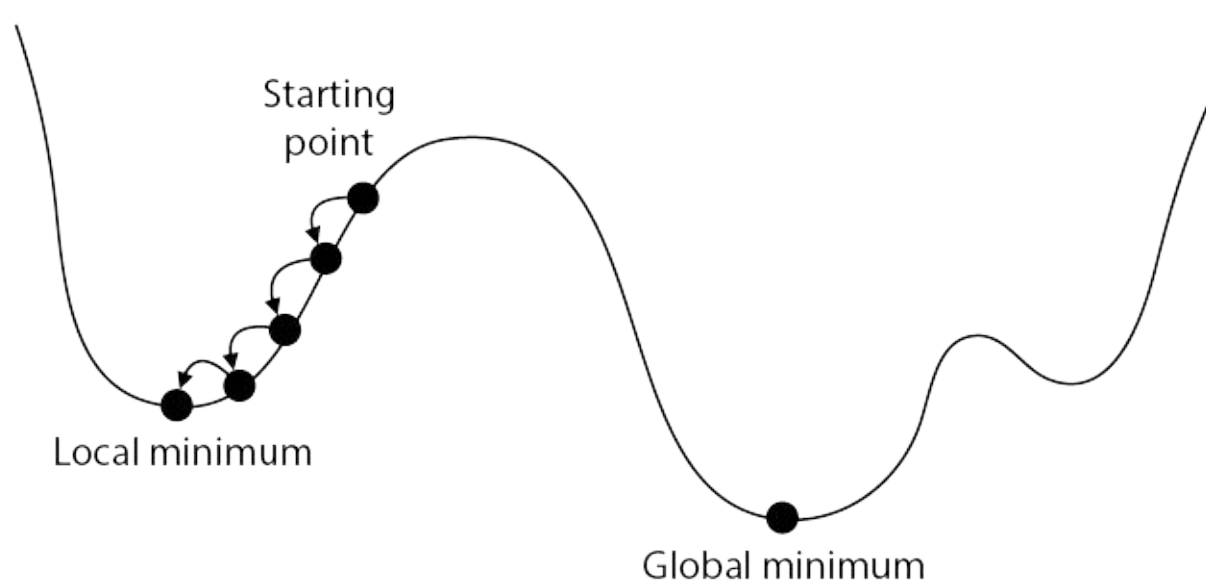
Bị mắc kẹt ở mức tối thiểu cục bộ: Điều đó xảy ra như thế nào và cách chúng tôi giải quyết nó

Như đã đề cập ở phần đầu của phụ lục này, thuật toán giảm độ dốc không nhất thiết phải tìm giá trị tối thiểu của hàm. Ví dụ, xem hình B.7. Giả sử chúng ta muốn tìm giá trị nhỏ nhất của hàm trong hình này bằng cách sử dụng phương pháp giảm độ dốc. Vì bước đầu tiên trong quá trình giảm độ dốc là bắt đầu ở một điểm ngẫu nhiên nên chúng ta sẽ bắt đầu tại điểm được gắn nhãn “Điểm bắt đầu”.



Hình B.7 Chúng ta đang đứng ở điểm được gắn nhãn “Điểm bắt đầu”. Giá trị tối thiểu của hàm là điểm được gắn nhãn “Tối thiểu”. Liệu chúng ta có thể đạt được mức tối thiểu này bằng cách sử dụng phương pháp giảm độ dốc không?

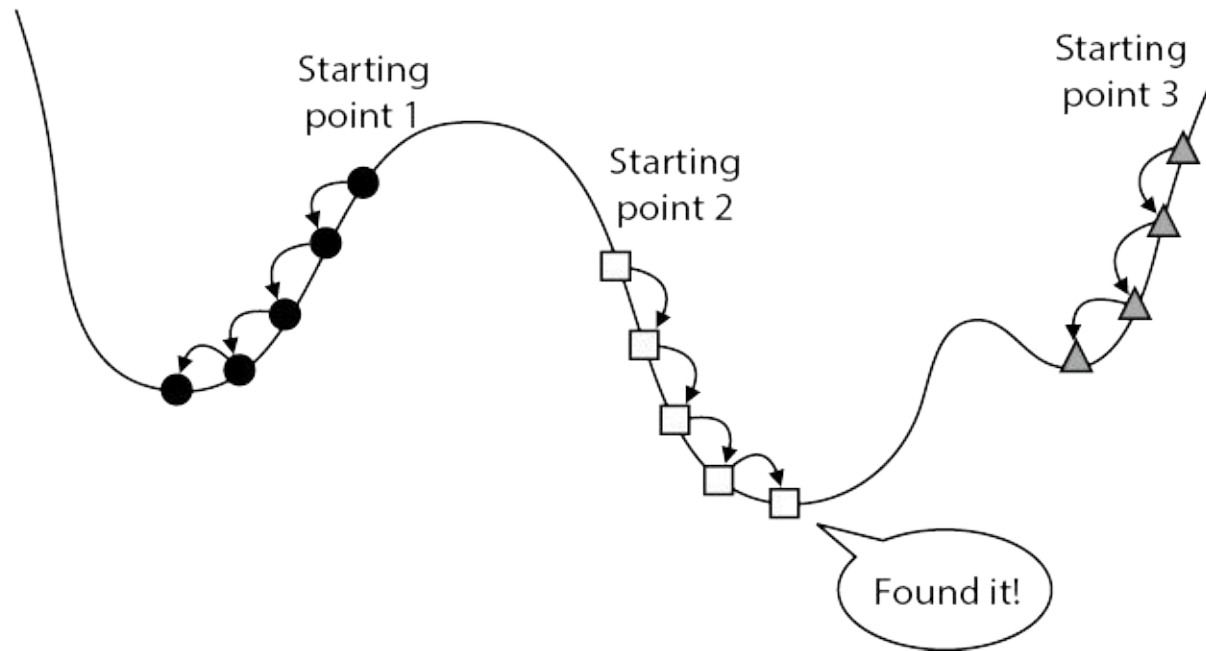
Hình B.8 cho thấy đường dẫn mà thuật toán giảm độ dốc sẽ đi để tìm mức tối thiểu. Lưu ý rằng nó thành công trong việc tìm ra điểm cực tiểu cục bộ gần nhất với điểm đó, nhưng nó hoàn toàn bỏ sót điểm cực tiểu toàn cục ở bên phải.



Hình B.8 Thật không may, việc giảm độ dốc không giúp chúng ta tìm được giá trị tối thiểu của hàm này. Chúng tôi đã cố gắng đi xuống, nhưng chúng tôi bị mắc kẹt ở mức tối thiểu cục bộ (thung lũng). Làm thế nào chúng ta có thể giải quyết vấn đề này?

Làm thế nào để chúng ta giải quyết vấn đề này? Chúng ta có thể sử dụng nhiều kỹ thuật để khắc phục điều này và trong phần này, chúng ta tìm hiểu một kỹ thuật phổ biến được gọi là *khởi động lại ngẫu nhiên*. Giải pháp đơn giản là chạy thuật toán nhiều lần, luôn bắt đầu ở một điểm ngẫu nhiên khác và chọn giá trị nhỏ nhất được tìm thấy tổng thể. Trong hình B.9, chúng tôi sử dụng ngẫu nhiên

khởi động lại để tìm mức tối thiểu toàn cục trên một hàm (lưu ý rằng hàm này chỉ được xác định trên khoảng trong hình và do đó, giá trị thấp nhất trong khoảng thực sự là mức tối thiểu toàn cục). Chúng tôi đã chọn ba điểm bắt đầu ngẫu nhiên, một điểm được minh họa bằng hình tròn, một điểm là hình vuông và một điểm là hình tam giác. Lưu ý rằng nếu chúng ta sử dụng phương pháp giảm độ dốc trên mỗi điểm trong số ba điểm này, hình vuông sẽ tìm được mức tối thiểu toàn cục của hàm.



Hình B.9 Minh họa kỹ thuật khởi động lại ngẫu nhiên. Hàm chỉ được xác định trên khoảng này, với ba đáy và cực tiểu toàn cục nằm ở thung lũng thứ hai. Ở đây chúng tôi chạy thuật toán giảm độ dốc với ba điểm bắt đầu khác nhau: hình tròn, hình vuông và hình tam giác. Lưu ý rằng hình vuông đã tìm được mức tối thiểu toàn cục của hàm.

Phương pháp này vẫn không đảm bảo tìm được mức tối thiểu toàn cục, vì chúng ta có thể không may mắn và chỉ chọn được những điểm bị mắc kẹt trong thung lũng. Tuy nhiên, với đủ điểm bắt đầu ngẫu nhiên, chúng ta có cơ hội tìm được mức tối thiểu toàn cục cao hơn nhiều. Và ngay cả khi chúng ta không thể tìm được mức tối thiểu toàn cục, chúng ta vẫn có thể tìm được mức tối thiểu cục bộ đủ tốt để giúp chúng ta đào tạo một mô hình tốt.

Phụ lục C. Tài liệu tham khảo

Những tài liệu tham khảo này cũng có thể được tìm thấy tại <https://serrano.academy/grokking-machine-learning/>.

Tài liệu tham khảo chung

- Kho GitHub: www.github.com/luisguiserrano/manning Các video trên YouTube:
- www.youtube.com/c/LuisSerrano Thông tin chung: <https://serrano.academy> Thông tin sách: <https://serrano.academy/grokking-machine-learning>
-

Các khóa học

- Udacity machine learning nanodegree: <http://mng.bz/4KE5> Khóa học máy Coursera:
- <https://www.coursera.org/learn/machine-learning> Chuyên ngành học máy Coursera (Đại học Washington): <http://mng.bz/Xryl> Học máy end-to-end:
- <https://end-to-end-machine-learning.teachable.com/courses>
-

Blog và kênh YouTube

- Video học máy của Brandon Rohrer: <https://www.youtube.com/user/BrandonRohrer> StatQuest với Josh Starmer: <https://www.youtube.com/user/joshstarmer> Blog của Chris Olah: <https://colah.github.io/> Blog của Jay Alammar: <https://jalamar.github.io/> Blog của Alexis Cook: <https://alexisbcook.github.io/> Blog của Dhruv Parthasarathy: <https://medium.com/@dhruvp> 3Blue1Brown: <http://youtube.com/c/3blue1Brown> Thành thạo học máy: <https://machinelearningmastery.com> Blog của Andrej Karpathy: <http://karpathy.github.io/>
-
-
-
-

Sách

- *Nhận dạng mẫu và học máy*, của Christopher Bishop: <http://mng.bz/g1DZ>

Chương 1

Video

- Các video về học máy tổng quát: <https://serrano.academy/general-machine-learning/> Video giới thiệu thân thiện về học máy: www.youtube.com/watch?v=IpGxLWOIZy4 Bản phác thảo thư rác Monty Python: www.youtube.com/watch?v=zLih-WQwBSc
-

Chương 2

Video

- Các video học máy được giám sát: [https://serrano.academy/Tuyến tính-models/](https://serrano.academy/Tuyến%20tính-models/) Các video học máy không giám sát: <https://serrano.academy/unsupervised-learning/> Các video học máy sáng tạo: <https://serrano.academy/generative-models> Video học tăng cường: <https://serrano.academy/reinforcement-learning> Video học sâu: <https://serrano.academy/neural-networks>
-
-

Sách

- *Học tăng cường sâu Grokking*, của Miguel Morales: <http://mng.bz/5Zy4>

Các khóa học

- Khóa học UCL về học tập tăng cường, của David Silver: <https://www.davidsilver.uk/teaching/> Chương trình Nanodegree học tập tăng cường sâu của Udacity: <http://mng.bz/6mMG>

Chương 3

Mã

- Kho lưu trữ GitHub: <http://mng.bz/o8lN>

Bộ dữ liệu

- Tập dữ liệu nhà ở ở Hyderabad: Người tạo: Ruchi Bhatia Ngày: 27/08/2020 Phiên bản: 4 Lấy từ <http://mng.bz/nrdv> Giấy phép: CC0: Miễn công cộng
 -
 -

Video

- Video hồi quy tuyến tính: <http://mng.bz/v4Rx> Video hồi quy đa thức: <https://www.youtube.com/watch?v=HmkmA-EFaW0>

Chương 4

Mã

- Kho lưu trữ GitHub: <http://mng.bz/4KXB>

Video

- Học máy: Kiểm tra và đo lường lỗi: <https://www.youtube.com/watch?v=aDW44NPhNw0> Lasso (L1) hồi quy (StatQuest): <https://www.youtube.com/watch?v=H0i quy NGf0voTMIcs> Ridge (L2) (StatQuest): <https://www.youtube.com/watch?v=Q81RR3yKn30>
-

Chương 5

Mã

- Kho lưu trữ GitHub: <http://mng.bz/Qqpm>

Video

- Video hồi quy logistic và thuật toán perceptron:
<https://www.youtube.com/watch?v=jbluHIgBmBo>
- Video mô hình Hidden Markov:
<https://www.youtube.com/watch?v=kqSzLo9fenk>

Chương 6

Mã

- Kho lưu trữ GitHub: <http://mng.bz/Xr9Y>

Bộ dữ liệu

- Tập dữ liệu đánh giá mở rộng về phim IMDB Người tạo: Stefano Leone Ngày: 24/11/2019 Phiên bản: 2 Lấy từ <https://www.kaggle.com/stefanoleone992/imdb-extensive-dataset> Giấy phép: CC0: Public Domain
 -
 -
 -

Video

- Video hồi quy logistic và thuật toán perceptron:
<https://www.youtube.com/watch?v=jbluHIgBmBo> Cross entropy (StatQuest):
- <https://www.youtube.com/watch?v=6ArSys5qHAU> Cross entropy (Aurélien Géron):
- <https://www.youtube.com/watch?v=ErfnhcEV1O8>

Chương 7

Video

- Học máy: Kiểm tra và đo lường lỗi: <https://www.youtube.com/watch?v=aDW44NPhNw0>

Chương 8

Mã

- Kho lưu trữ GitHub: <http://mng.bz/yJRJ>

Bộ dữ liệu

- Tập dữ liệu bộ lọc thư rác Người tạo: Karthik Veerakumar Ngày: 07/17/14 Phiên bản: 1 Lấy từ <https://www.kaggle.com/karthickveerakumar/spam-filter> Chế độ hiển thị: Công khai
 -
 -
 -
 -

Video

- Bayes ngây thơ: <https://www.youtube.com/watch?v=Q8l0Vip5YUw>

Chương 9

Mã

- Kho lưu trữ GitHub: <http://mng.bz/MvM2>

Bộ dữ liệu

- Người tạo tập dữ liệu tuyển sinh: Mohan S. Acharya Ngày: 03/03/2018 Phiên bản: 2 Được lấy từ <http://mng.bz/aZlJ> Bài báo: Mohan S. Acharya, Asfia Armaan và Aneeta S Antony, “So sánh các mô hình hồi quy để dự đoán tuyển sinh sau đại học”, Hội nghị quốc tế của IEEE về trí tuệ tính toán trong khoa học dữ liệu (2019). Giấy phép: CC0: Miễn công cộng
 -
 -
 -

Video

- Cây quyết định (StatQuest): <https://www.youtube.com/watch?v=7VeUPuFGJHk> Cây quyết
- định hồi quy (StatQuest): <https://www.youtube.com/watch?v=g9c66TUylZ4> Cây quyết định (Brandon Rohrer): <https://www.youtube.com/watch?v=9w16p4QmkAI> Chỉ số tạp chất Gini: <https://www.youtube.com/watch?v=u4IxOk2ijSs> Shannon entropy và thông tin thu được: <https://www.youtube.com/watch?v=9r7FIXEAGvs>
-
-

Bài đăng trên blog

- Entropy của Shannon, thu thập thông tin và nhật bóng từ xô: <http://mng.bz/g1lR>

Chương 10

Mã

- Kho lưu trữ GitHub: <http://mng.bz/ePAJ>

Bộ dữ liệu

- Bộ dữ liệu MNIST. Đặng, L. “Cơ sở dữ liệu MNIST về hình ảnh chữ số viết tay cho nghiên cứu học máy.” *Tạp chí xử lý tín hiệu IEEE* 29, số.6 (2012): 141–42. <http://yann.lecun.com/exdb/mnist/>
Dữ liệu về nhà ở của Hyderabad (xem tài liệu tham khảo cho chương 3)

-

Video

- Mạng lưới thần kinh và học sâu: <https://www.youtube.com/watch?v=BR9h47Jtqyw> Mạng nơ-ron tích chập: <https://www.youtube.com/watch?v=2-Ol7ZB0MmU> Mạng nơ-ron tái diễn: <https://www.youtube.com/watch?v=UNmqTiOnRfg> Mạng nơ-ron hoạt động như thế nào (Brandon Rohrer): <https://www.youtube.com/watch?v=ILsA4nyG7I0> Mạng thần kinh tái phát (RNN) và bộ nhớ ngắn hạn dài (LSTM) (Brandon Rohrer): <https://www.youtube.com/watch?v=WCUNPb-5EYI>

-

-

Sách

- *Grokking Deep Learning*, của Andrew Trask: <https://www.manning.com/books/grokking-deep-learning>
- *Deep Learning*, của Ian Goodfellow, Yoshua Bengio và Aaron Courville: <https://www.deeplearningbook.org/>

Các khóa học

- Khóa học sâu của Udacity: <http://mng.bz/p9lP>

Bài đăng trên blog

- “Sử dụng phương pháp học chuyển giao để phân loại hình ảnh bằng Keras”, của Alexis Cook: <http://mng.bz/OQgP> “Các lớp tổng hợp trung bình toàn cầu để bản địa hóa đối tượng”, của Alexis Cook: <http://mng.bz/Ywj7> “Lịch sử tóm tắt về CNN trong phân đoạn hình ảnh: Từ R-CNN đến mặt nạ R-CNN”, của Dhruv Parthasarathy: <http://mng.bz/GOnN> “Mạng thần kinh, đa tạp và cấu trúc liên kết”, của Chris Olah: <http://mng.bz/zERZ> “Tìm hiểu về Mạng LSTM”, của Chris Olah: <http://mng.bz/01nz> “Cách GPT3 hoạt động: Trực quan hóa và Hoạt hình”, của Jay Alammar: <http://mng.bz/KoXn> “Cách định cấu hình tốc độ học tập khi đào tạo mạng thần kinh học sâu”, của Jason Brownlee: <http://mng.bz/9ae8> “Thiết lập tốc độ học tập của mạng thần kinh của bạn”, của Jeremy Jordan: <http://mng.bz/WBKX> “Chọn kiến trúc tốt nhất cho mạng thần kinh nhân tạo” của Ahmed Gad: <http://mng.bz/WBKX> “Công thức đào tạo mạng thần kinh”, của Andrej Karpathy: <http://mng.bz/80gg>
-
-
-
-

Công cụ

- Sân chơi TensorFlow: <https://playground.tensorflow.org/>

Chương 11

Mã

- Kho lưu trữ GitHub: <http://mng.bz/ED6r>

Video

- Máy vectơ hỗ trợ: https://www.youtube.com/watch?v=Lpr__X8zuE8 Hạt nhân đa thức
- (StatQuest): <https://www.youtube.com/watch?v=Toet3EiSFcM> Hạt nhân xuyên tâm (RBF)
- (StatQuest): https://www.youtube.com/watch?v=Qc5IyLW_hns
-

Bài đăng trên blog

- “Hạt nhân và Bản đồ tính năng: Lý thuyết và Trực giác”, của Xavier Bourret Sicotte:
<http://mng.bz/N4aX>

Chương 12

Mã

- Kho lưu trữ GitHub: <http://mng.bz/DK50>

Video

- Rừng ngẫu nhiên (StatQuest): https://www.youtube.com/watch?v=J4Wdy0Wc_xQ
- AdaBoost (StatQuest): <https://www.youtube.com/watch?v=LsK-xG1cLYA>
- Tăng cường độ dốc (StatQuest): <https://www.youtube.com/watch?v=3CC4N4z3GJc>
- XGBoost (StatQuest): <https://www.youtube.com/watch?v=OtD8wVaFm6E>
-

Bài viết và bài đăng trên blog

- “Sự tổng quát hóa về lý thuyết-quyết định của việc học trực tuyến và ứng dụng để tăng cường,” của Yoav Freund và Robert Shapire, *Tạp chí Khoa học Hệ thống và Máy tính*, 55, 119–139. <http://mng.bz/l9Bz>
- “Giải thích về AdaBoost,” của Robert Schapire: <http://rob.schapire.net/papers/explaining-adaboost.pdf>
- “XGBoost: Hệ thống tăng cường cây có thể mở rộng,” của Tianyi Chen và Carlos Guestrin. KDD '16: Kỷ yếu của Hội nghị quốc tế ACM SIGKDD lần thứ 22 về Khám phá tri thức và khai thác dữ liệu, tháng 8 năm 2016, 785–794.
- <https://doi.org/10.1145/2939672.2939785> “Giành giải thưởng Netflix: Tóm tắt”, của Edwin Chen: <http://mng.bz/B1jq>
-

Chương 13

Mã

- Kho lưu trữ GitHub: <http://mng.bz/drlz>

Bộ dữ liệu

- Tập dữ liệu Titanic: Được lấy từ <https://www.kaggle.com/c/titanic/data>
 -

Biểu tượng đồ họa và hình ảnh

Các biểu tượng đồ họa và hình ảnh được cung cấp bởi những người sáng tạo sau đây từ flaticon.com:

- IFC—người đi bộ đường dài, dãy núi: Freepik;flag: Good Ware Lời nói
- đầu—nhạc sĩ: photo3idea_studio;nhà toán học: Freepik Hình 1.2—người: Eucalyp;máy tính: Freepik Hình 1.4—người: Ultimatearm Hình 1.5, 1.6, 1.7, 1.8, 1.9—người trên máy tính: Freepik Hình 2.1, 2.2, 2.3, 2.4—con chó: Freepik;mèo: Smashicons
-
-
- Hình 2.2, 2.3, 2.4—nhà máy: DinosoftLabs Hình 2.5, 2.6—phong bì: Freepik Hình 2.7—nhà: Vectors Market Hình 2.8—điểm bản đồ: Freepik Hình 2.11, 2.12—robot, núi: Freepik;rồng: bạch đàn;Kho báu: Smashicons Hình 3.3, 3.4, 3.5—nhà: Vectors Market Hình 3.20, 3.21, 3.22—người đi bộ đường dài, dãy núi: Freepik;cờ, bóng đèn: Good Ware Hình 4.1—Godzilla: Freepik;vi đập: Smashicons;bay: bạch đàn;bazooka: photo3idea_studio Hình 4.6—house: Chợ Vectors;băng, bệnh zona: Freepik;thanh titan: Vitaly Gorbachev
-
-
-
-

- Các hình 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.10, 5.11, 5.12, 5.13, 5.16, 5.17, 5.18, 5.19
5,20, 5,21, 5,22, 5,23, 5,24, 6,1, 6,4, 6,5, 6,6, 10,2, 10,3—mặt vui, mặt buồn, mặt trung tính:
Số liệu thị trường Vectors 5.15, 5.16—người đi bộ đường dài: Freepik;bóng đèn: Good Ware
- Hình 7.11 (trong bài tập)—các loài động vật khác nhau: surang;cuốn sách: Đồ Tốt; đồng hồ,
dâu tây, hành tinh, bóng đá: Freepik;video: tu sĩ Hình 8.4—mũ đầu bếp: Vitaly Gorbachev;lúa
mì, bánh mì: Freepik Hình 8.5, 8.6, 8.7, 8.8, 8.9, 8.10, 8.11, 8.12, 8.13—phong bì: Smashicons
Hình 9.6, 9.7, 9.8, 9.9, 9.12, 9.13, 9.15, 9.16, 9,17—nguyên tử: mavadee; tổ ong:
- Smashicons;hiệp sĩ cò vua: Freepik Hình 9.13, 9.17—túi xách: Gregor Cresnar Joke trong
- chương 10—cánh tay đầy cơ bắp: Freepik Joke trong chương 11—ngô, gà: Freepik;gà: tu sĩ
- Trò đùa trong chương 12—cá mập, cá: Freepik Hình 12.1—robot lớn: photo3idea_studio;robot
nhỏ: Freepik Joke trong chương 13—ship: Freepik

-
-
-
-
-
-
-

index

A

- absolute error 61, 62, 154
- absolute trick 55–56
- accuracy
 - decision trees
 - with continuous features 260
 - with yes/no questions 243–244
 - examples of models 178–179
 - receiver operating characteristic (ROC) curve 189–200
 - AUC (area under the curve) 194–195
 - making decisions using 195–196
 - sensitivity and specificity 189–194, 198–200
 - testing 403
 - types of errors 179–188
 - choosing 189
 - confusion matrix 182–183
 - false positives and false negatives 180–182
 - F-score 186–189
 - precision 185–186
 - recall 183–180
- Acharya, Mohan S. 476
- activation functions 286, 297–298
- AdaBoost 360–369
 - building weak learners 361–363
 - coding in Scikit-Learn 368–369
 - combining weak learners into strong learner 363–366
 - combining classifiers 366
 - probability, odds, and log-odds 364
- AdaBoostClassifier package 368
- Admissions dataset 475
- AI (artificial intelligence) 3–4, 4
- Alammar, Jay 472, 477
- algorithms 7–13
 - clustering 23–25
 - examples of models that humans use 8–12
 - examples of models that machines use 12–13
 - linear regression 44–45
 - general linear regression algorithm 59–60
 - overview 56
 - using in dataset 58
 - using model to make predictions 59
- alien planet example
 - classification 106
 - bias and y-intercept 119–120
 - classifiers, goal of 114
 - definition of perceptron classifier 117–119
 - general classifier 114–115

phức tạp hơn một chút hành tinh 110–113 hàm bước và các hàm kích hoạt 116
mạng nơ-ron 279–292 kiến trúc của mạng nơ-ron được kết nối đầy đủ 291–292 ranh
giới của mạng nơ-ron 289–291 kết hợp đầu ra của các perceptron thành một perceptron
khác 283–285 biểu diễn đồ họa của mạng nơ-ron 287 biểu diễn đồ họa của các
perceptron 285–287 sử dụng hai dòng để phân loại tập dữ liệu 280–281 AND tập dữ
liệu 421 AND cổng 420 toán tử AND 283, 332 Antony, Aneeta S. 476 hàm apply() 227
hệ thống đề xuất ứng dụng 241–256 đặt câu hỏi hay nhất 241–252 thuật toán cây quyết
định 255–256 siêu tham số 254–255 lặp lại 252–254 tổng quan vấn đề 240 kiến trúc
299 Armaan, Asfia 476 trí tuệ nhân tạo (AI) 3–4 AUC (diện tích dưới đường cong)
194–195

B

lan truyền ngược 293–294 đóng bao 353, 360 cây quyết định phù hợp 356 rừng ngẫu
nhiên phù hợp theo cách thủ công 356–357 huấn luyện rừng ngẫu nhiên trong
Scikit-Learn 358–359 giảm độ dốc theo đợt 67 thuật toán hồi quy logistic 166 thuật
toán perceptron 137 lô so với lô nhỏ so với giảm độ dốc ngẫu nhiên 299 Phân tích
Bayesian 211 Bayesian học 211 Thống kê Bayes 211 Định lý Bayes tìm hậu nghiệm
với 228 triển khai thuật toán Bayes ngây thơ 229–230 Bengio, Yoshua 477 Bhatia,
Ruchi 473 độ lệch 39, 113 cây nhị phân 234 binning 398–399 Bishop, Christopher
472 thúc đẩy 353 nhánh, của các nút quyết định 235 Tóm tắt lịch sử của các nút
quyết định CNN trong phân đoạn hình ảnh, A (Parthasarathy) 477

C

phân loại chéo-entropy 303 dữ liệu phân loại 20 chatbot 310
Chen, Edwin 479 Chen, Tiani 479 tập dữ liệu vòng tròn
328–330, 346–347 phân loại 21–22, 104, 146, 150 vấn đề về
hành tinh ngoài hành tinh 106 độ chệch và y-đánh chặn 119 bộ
phân loại, mục tiêu của 114 định nghĩa của bộ phân loại
perceptron 117–119 bộ phân loại chung 114–115 hành tinh
phức tạp hơn một chút 110–113 hàm bước và các hàm kích hoạt
116 ứng dụng của 142–143 thị giác máy tính 144 chăm sóc sức
khỏe 143–145 hệ thống đề xuất 143 bộ lọc email spam 143 mã
hóa 137–142 tổng quan 140–141 thủ thuật perceptron 139 sử
dụng Turi Tạo 141–142 kết hợp 366 hàm lỗi 121–130, 318–320
so sánh các bộ phân loại 121–129 lỗi perceptron trung bình
126–129 thuật toán perceptron 129–136 giảm độ dốc 137 thủ
thuật perceptron 129–133 mã giả cho 135–136 giảm độ dốc
ngẫu nhiên và theo đợt 137 sử dụng giảm độ dốc để huấn luyện
các mô hình 457–462 để giảm mất log 461–463 để giảm lỗi
perceptron 457–460 mô hình phân loại 20 phân loại 104 thuật
toán phân cụm 23–25 CNN (mạng thần kinh tích chập) 309–310
phương pháp hạt nhân mã hóa 346–350 hạt nhân đa thức để
phân loại tập dữ liệu vòng 346 hạt nhân hàm cơ sở xuyên tâm
(RBF) 347–349 thuật toán hồi quy logistic 166 thuật toán
perceptron 137–142 mã hóa thủ thuật perceptron 139 tổng quan
140–141

sử dụng Turi Create 141 So sánh các mô hình hồi quy để dự đoán tuyển sinh sau đại học, A (Acharya, Armaan và Antony) 476 hàm biên dịch 303 thị giác máy tính 144 xác suất có điều kiện 206 ma trận nhầm lẫn 182–183 tính năng liên tục 258–260, 260 perceptron liên tục 149 mạng nơ-ron tích chập (CNN) 309–310 Cook, Alexis 472, 477 tập dữ liệu về virus corona 179 hàm chi phí 61 Courville, Aaron 477 Tham số C trong Scikit-Learn 326 tổng quan 323–324 tạo hàm 68, 97, 142, 172 siêu tham số tiêu chí 265 entropy chéo 158 xác thực chéo 407 CSV (các giá trị được phân tách bằng dấu phẩy) 389

D

dữ liệu 4, 17–18, 393 được xác định 17–18 tính năng 17 dữ liệu được gắn nhãn và không được gắn nhãn 17–18 nhãn 17 hồi quy tuyến tính 57 dự đoán 17 kỹ thuật tính năng kỹ thuật dữ liệu 395–400 gộp 398–399 lựa chọn tính năng 399–400 mã hóa một lần nóng 396–398 tìm kiếm dạng lưới 405–407 giá trị bị thiếu 392–394 bỏ cột có dữ liệu bị thiếu 393 điền dữ liệu bị thiếu 394 Bộ dữ liệu Titanic 388–392 tính năng của 389 tải với Pandas 389–390 nghiên cứu với Pandas 391–392 mô hình đào tạo 400–404 đánh giá mô hình 402–404 một số mô hình trên tập dữ liệu 402 phân tách dữ liệu 401 mô hình thử nghiệm 404 bằng cách sử dụng xác thực chéo k-fold 408–409 Đối tượng DataFrame 390

DBSCAN (phân cụm không gian dựa trên mật độ) 25 nút quyết định 235 gốc quyết định 234 Khái quát hóa lý thuyết-quyết định của học tập trực tuyến và một

n Ứng dụng để tăng cường, A (Freund và Shapire)

479 Đối tượng DecisionTreeClassifier 263 Đối tượng DecisionTree 264 Đối tượng DecisionTreeRegressor 271 cây quyết định 235 ứng dụng 272–273 chăm sóc sức khỏe 272 hệ thống đề xuất ứng dụng 241–256 đặt câu hỏi hay nhất 241–252 thuật toán cây quyết định 255–256 siêu tham số 254–255 lặp lại 252–254 tổng quan về vấn đề 240–241 phù hợp với 356 cho hồi quy 268–271 tăng cường độ dốc 370–374 ranh giới đồ họa của 261–264 lập mô hình tuyến tính sinh viên với Scikit-Learn 264–267 phân tách dữ liệu với nhiều lớp hơn 257–260 sử dụng các tính năng liên tục 258–260 sử dụng các tính năng phân loại phi nhị phân 257 học sâu (DL) 5, 5–6, 278 Deep Learning (Goodfellow, Bengio và Courville) 477 tham số 70 độ 346 thuật toán phân cụm không gian dựa trên mật độ (DBSCAN) 25 độ sâu, cây quyết định 235 chiều Giảm 26 chiều 25–26 hướng 48 perceptron rời rạc 149 hàm display_tree 263 bộ phân loại so sánh khoảng cách 123–125 hàm lỗi 320–321 DL (học sâu) 5, 5–6, 278 bỏ chức năng 393 bỏ học 296

E

tập dữ liệu email dương tính giả và âm tính giả 182 tổng quan 179 mô hình phát hiện thư rác email 20 học tập tổng hợp AdaBoost 360–369

xây dựng người học yếu 361–363 mã hóa trong Scikit-Learn
368–369 kết hợp người học yếu thành người học mạnh 363–366
tương tự 352–353 ứng dụng của các phương pháp tập hợp 384 đóng
gói 354–355 cây quyết định khớp 356 khớp ngẫu nhiên theo cách thủ
công 356–357 huấn luyện nhóm ngẫu nhiên trong Scikit-Learn
358–359 tăng cường độ dốc 370–374 XGBoost 375–384 xây dựng
cho người học yếu 376–379 đưa ra dự đoán 380–381 điểm tương
đồng 375–376 mô hình đào tạo bằng Python 382–384 cắt tia cây
379–380 entropy 248, 250 kỷ nguyên 56 hàm lỗi 60–67, 121–130 lỗi
tuyệt đối 61 so sánh các phân loại 121–129 khoảng cách 123–125 số
lỗi 121–123 điểm 125–126 giảm độ dốc 63–66 hồi quy lasso và hồi
quy sườn 90–91 phân loại logistic 154 lỗi tuyệt đối 154 mất nhật ký
155 lỗi bình phương 155 lỗi bình phương tuyệt đối và trung bình
(gốc) 63–65 trung bình lỗi perceptron 126–129 đo mạng thần kinh
293 SVM (máy vector hỗ trợ) 318–324 chức năng lỗi phân loại
318–320 Tham số C 323–324 chức năng lỗi khoảng cách 320–321 vẽ
đồ thị 66–67 lỗi vuông 62 giảm độ dốc ngẫu nhiên và hàng loạt 67
loại lỗi 179–188 chọn 189 ma trận nhầm lẫn 182–183 sai dương tính
và âm tính giả 180–182 Điểm F 186–189 tính toán 187 tính toán điểm
Fb 187–189

độ chính xác 185–186 thu hồi 183–180 sự
kiện 210–211 Giải thích về AdaBoost
(Schapire) 479

F

Điểm F1 186–187, 403, 404
hàm f1_score 403 phủ định sai 180–182
dương tính giả 180–182 kỹ thuật tính năng
395–400 tạo nhóm 398–399 lựa chọn tính
năng 399–400 mã hóa một lần 396–398
bản đồ tính năng 327 tính năng 11, 38, 265,
325, 358, 401 hàm fillna 394 hàm khớp
263, 265, 402 công thức 3 công thức 8
Freund, Yoav 479 F-score 186–189 tính
toán 187 tính toán Điểm Fb 187–188

G

Gad, Ahmed 478 trò chơi 31 tham số gamma 344–345 GAN (mạng tạo đối thủ)
29, 311 đơn vị tái phát có cổng (GRU) 310–311 mô hình hỗn hợp Gaussian 25
thuật toán hồi quy tuyến tính tổng quát 59–60 học máy tổng quát 29, 311 di
truyền học 25 get_dummies hàm 397 Chỉ số tạp chất Gini 245, 260 Lớp tổng hợp
trung bình toàn cầu để bản địa hóa đối tượng (Cook) 477 Goodfellow, Ian 477
tăng cường độ dốc 375–384 xây dựng những người học yếu 376 đưa ra dự đoán
380–381 huấn luyện mô hình XGBoost bằng Python 382–384 cắt tia cây
379–380 sử dụng cây quyết định để xây dựng những người học giỏi 370–374
Điểm tương tự XGBoost 375–376 Gói gradientBoostingRegressor 373 giảm độ
dốc 63–66, 449–452 hàm giảm 450–451 để chính quy hóa 466–467 cực tiểu cục
bộ 467–469 thuật toán perceptron 137 mô hình đào tạo 452–466 mô hình phân
loại 457–462

mô hình hồi quy tuyến tính 454–458 mạng thần kinh 463–466
ranh giới đồ họa 261–264 tìm kiếm lưới 255, 405–407
GridSearchCV 407 ví dụ về thế giới lưới 30–31 Grokking Deep
Learning (Trask) 309, 477 Grokking Deep Reinforcement
Learning (Morales) 473 GRU (các đơn vị định kỳ có kiểm soát)
310–311 Guestrin, Carlos 479

H

ham 8 phân loại ứng dụng chăm sóc sức khỏe 143–145 cây quyết định 272 hồi quy 72 phân cụm theo cấp bậc 25 mô hình
có độ chính xác cao 186 mô hình thu hồi cao 180 độ nhạy cao 190 độ đặc hiệu cao 190 HMM (mô hình Markov ẩn) 105
mô hình giá nhà ở 20 mô hình hồi quy nhà ở 20 xây dựng bước 40–41 hồi quy tuyến tính đa biến 42–43 dự đoán giá nhà
37–43 dự đoán bước 42 nhớ bước 39–40 sử dụng Turi Tạo 67–69 Cách GPT3 hoạt động (Alammar) 477 Cách định cấu
hình tốc độ học tập khi đào tạo Mạng thần kinh học sâu (Brownlee) 477 Tập dữ liệu nhà ở Hyderabad 473 cây quyết định
siêu tham số 254–255 được xác định 71–72 tìm kiếm dạng lưới 405–407 cài đặt trong Scikit-Learn 266–268 huấn luyện
mạng lưới thần kinh 299

I

câu lệnh if 140 nhận dạng hình ảnh 305, 309
xây dựng và huấn luyện mô hình 306–307 đánh
giá mô hình 307 tải dữ liệu 305 tiền xử lý dữ
liệu 305–306

Tập dữ liệu đánh giá mở rộng về phim của IMDB 474
cột chỉ mục 172 thu thập thông tin 252 giao điểm 219
hàm is_na 393 lặp lại 252–254

J

Jordan, Jeremy 478

K

Karpathy, Andrej 472, 478 Ví dụ đồ họa Keras 299–307 theo hai chiều
300–305 kiến trúc của mạng nơron 301 xây dựng mô hình trong mô
hình đào tạo Keras 302–303 303–305 biến các đặc tính không nhị
phân thành nhị phân 301 huấn luyện mạng nơron để nhận dạng hình
ảnh 305–307 xây dựng và huấn luyện mô hình 306–307 mô hình đánh
giá 307 tải dữ liệu 305 dữ liệu tiền xử lý 305–306 phương pháp kernel
326–349 mã hóa 346–350 kernel đa thức để phân loại tập dữ liệu tuần
hoàn 346–347 kernel hàm cơ sở xuyên tâm (RBF) 347 kernel đa thức
328–334 AND toán tử trên hệ thống khuyến nghị 332–333 tập dữ liệu
tuần hoàn 328–330 vượt xa các phương trình bậc hai 333–334 hạt
nhân hàm cơ sở xuyên tâm (RBF) 334–339 hàm tương tự 337–339 339
huấn luyện SVM với tham số hạt nhân 340–345 346 Hạt nhân và Bản
đồ tính năng (Sicotte) 478 xác thực chéo k-fold 408–409 Phân cụm
K-means 25

L

Chỉ tiêu L1 và L2 trong các hệ số của mô hình
92–93 đo độ phức tạp của mô hình 89–90 tham
số l1_penalty 97 tham số l2_penalty 97 dữ liệu
được dán nhãn 18–22

mô hình phân loại 21–22 mô hình hồi quy 20–21 nhãn 39, 265, 325, 358, 401 lambda 92 tham số lambda 382 hồi quy lasso 90–91 nút lá 235 người học 352 tốc độ học tập 53, 299, 382, 450 siêu tham số learning_rate 373 lá 234 hàm len 391 Leone, Stefano 474 phương trình tuyến tính 41 thủ thuật tuyệt đối hồi quy tuyến tính 55–56 thuật toán 44–60 thuật toán hồi quy tuyến tính tổng quát 59–60 tổng quan 56–57 sử dụng trong tập dữ liệu 58–59 sử dụng mô hình để đưa ra dự đoán 59 ứng dụng của hồi quy 72 hàm lỗi 60–67 sai số tuyệt đối 61 giảm độ dốc 63–66 sai số bình phương tuyệt đối trung bình và (gốc) trung bình 63–65 vẽ đồ thị 66–67 sai số bình phương 62 giảm độ dốc ngẫu nhiên và theo đợt 67 mô hình hồi quy nhà ở xây dựng bước 40–41 hồi quy tuyến tính đa biến 42–43 dự đoán giá nhà 37–43 dự đoán bước 42 bước 39–40 sử dụng Turi Tạo 67–69 tải dữ liệu và vẽ biểu đồ 57 tham số và siêu tham số 71–72 hồi quy đa thức phù hợp với đường cong đa thức 71 đa thức 69–70 thủ thuật đơn giản 49–52 độ dốc và y-chặn 47–49 thủ thuật vuông 52–56 sử dụng phương pháp giảm độ dốc để huấn luyện các mô hình 454–458 giảm sai số tuyệt đối trung bình 454–456 giảm sai số bình phương trung bình 456–457 gói hồi quy tuyến tính 68 cực tiểu cục bộ 467–469 vị trí 48 đối tượng logistic_classifier 142

gối logistic_classifier 172 bộ phân loại logistic phân loại đánh giá
IMDB với Turi Tạo 171 bộ phân loại so sánh sử dụng mất log 158 bộ
phân loại logistic 1 158 bộ phân loại logistic 2 159 tập dữ liệu và dự
đoán 152 hàm lỗi 154 lỗi tuyệt đối 154 mất nhật ký 155 lỗi bình phương
155 thuật toán hồi quy logistic 160 mã hóa 166 thủ thuật logistic 160
mã giả cho 165 ngẫu nhiên, giảm độ dốc theo lô và theo đợt 166 hàm
sigmoid 150 hàm softmax 173 LogisticRegression 402 thuật toán hồi
quy logistic 150 thủ thuật logistic 160 mất log so sánh phân loại bằng
158 phân loại logistic 1 158 phân loại logistic 2 159 công thức cho 158
tổng quan 155 sử dụng giảm dần độ dốc 461–463 log-odds 364 hàm mất
mát 61 hàm low() 227 LSTM (mạng bộ nhớ ngắn hạn dài) 105,
310–311

M

machine learning AI (trí tuệ nhân tạo) 3–4 nền tảng cần thiết cho
2–3 deep learning 5–6 xác định 4–5 khung ghi nhớ-công
thức-dự đoán 6–13 mô hình hồi quy nhà ở 37–38 cách con người
suy nghĩ 6–7 mô hình và thuật toán 7–8 loại dữ liệu 17–18 học
tăng cường 29–31 học có giám sát với dữ liệu được dán nhãn
18–22 học không giám sát với dữ liệu không được gán nhãn
22–25 dịch máy 310 MAE (giá trị tuyệt đối trung bình) 418

phân khúc thị trường 25 nhân tố ma trận 29 siêu tham số max_deep 266, 358 siêu tham số max_features 266 độ sâu tối đa 382 lỗi tuyệt đối trung bình 63, 454–456 lỗi perceptron trung bình 126 lỗi bình phương trung bình (MSE) 63–65, 269, 456 tập dữ liệu y tế 179 hình ảnh y tế 25 độ dốc giảm dần theo đợt nhỏ 67, 137, 166 học theo đợt nhỏ 67 chức năng giảm thiểu 63 siêu tham số min_impurity_decrease 266 siêu tham số min_impurity_split 266 tổn thất phân chia tối thiểu 382 siêu tham số min_samples_leaf 266 siêu tham số min_samples_split 266 giá trị bị thiếu 392–394 bỏ cột có dữ liệu bị thiếu 393 điền dữ liệu bị thiếu 394 tập dữ liệu MNIST (Viện Tiêu chuẩn và Công nghệ Quốc gia đã sửa đổi) 300 biểu đồ độ phức tạp của mô hình 85–86 mô hình 7–8, 39 mô hình phân loại 21–22 ví dụ về mô hình mà con người sử dụng 8–12 ví dụ về các mô hình mà máy sử dụng 12 mô hình hồi quy 20–21 Morales, Miguel 473 MSE (sai số bình phương trung bình) 63–65, 269, 456 perceptron nhiều lớp 278 hồi quy tuyến tính đa biến 42–43 đề xuất âm nhạc 72

N

mô hình Bayes ngây thơ 206, 232 ví dụ về bệnh tật hoặc khỏe mạnh 207–211 mô hình phát hiện thư rác 212–226 xây dựng với dữ liệu thực 226–230 tìm phần sau 213–214 tìm 212 trước đó biến tỷ lệ thành xác suất 214–216 Cổng NAND 443 vùng âm 109 siêu tham số n_estimators 358 mạng lưới thần kinh 5 ví dụ về hành tinh ngoài hành tinh 279–281 kiến trúc của mạng lưới thần kinh được kết nối đầy đủ 291–292 ranh giới của mạng lưới thần kinh 289–291 kết hợp đầu ra của các perceptron thành một perceptron khác 283–285

biểu diễn đồ họa của các mạng thần kinh 287
biểu diễn đồ họa của các perceptron 285–287
sử dụng hai dòng để phân loại tập dữ liệu 280–282
kiến trúc cho các tập dữ liệu phức tạp 309–311
mạng thần kinh tích chập (CNN) 309–310
mạng đối kháng sinh học (GAN) 311
RNN, GRU, LSTM 310–311
mã hóa trong Keras 299–307
ví dụ đồ họa trong hai kích thước 300–305
đào tạo mạng lưới thần kinh để nhận dạng hình ảnh 305–307
cho hồi quy 308–309
đào tạo 292–299
hàm kích hoạt 297–298
lan truyền ngược 293–294
hàm lỗi 293
siêu tham số 299
vấn đề, trang bị quá mức và biến mất độ dốc 294–295
hàm softmax 298–299
kỹ thuật cho 295–299
có độ dốc gốc 463–466

Nmạng eural, Đa tạp và Cấu trúc liên kết (Olah) 477
Nca ngợi 234
Nđặc điểm phân loại nhị phân 257
NHOẶC cổng 442
Ndữ liệu số 20

O

tỷ lệ cược 364
Olah, Chris 472, 477
mã hóa một lần nóng 257, 396–398
lý thuyết toán tử 327
HOẶC tập dữ liệu 422
HOẶC cổng 420
trang bị quá mức 78, 101
ví dụ sử dụng hồi quy đa thức 79–81
đồ thị độ phức tạp của mô hình 85–86
hồi quy đa thức, thử nghiệm và chính quy hóa với Turi
Tạo giảm 95–100 bằng cách tia cây 379–380
chính quy hóa 86–95
hiệu ứng của L1 và L2
chính quy hóa trong các hệ số của mô hình 92–93
ví dụ về trang bị quá mức 88–89
cách trực quan để xem chính quy hóa 93
Định mức L1 và L2 89–90
hồi quy lasso và hồi quy sườn 90–91
tham số chính quy hóa 91–92
mô hình thử nghiệm 81–84
kiểm tra chọn hàng tập 83
sử dụng dữ liệu thử nghiệm cho mô hình huấn luyện 83–84
huấn luyện mạng noron 294–295
tập xác thực 84

P

Pandas đang tải tập dữ liệu 389–390 tập dữ liệu nghiên cứu 391–392
tham số 71–72 Parthasarathy, Dhruv 472, 477 Nhận dạng mẫu và học
máy (Bishop) 472 perceptron 104 thuật toán perceptron 105 bộ phân
loại perceptron 104 hàm lỗi perceptron 125 mô hình perceptron 104
thuật toán perceptron 129–136 giảm độ dốc 137 thủ thuật perceptron
129–133 mã giả cho 135–136 giảm dần độ dốc ngẫu nhiên và hàng loạt
137 kiến trúc của mạng nơ-ron được kết nối đầy đủ 291–292 ranh giới
của mạng nơ-ron 289–291 kết hợp các đầu ra của một perceptron khác
283–285 bộ phân loại hạnh phúc nghề nghiệp 283 bộ phân loại hạnh
phúc gia đình 284 bộ phân loại hạnh phúc 285 biểu diễn đồ họa của
285–287 biểu diễn đồ họa của mạng nơ-ron 287 sử dụng độ dốc giảm
dần 457–460 sử dụng hai dòng để phân loại tập dữ liệu 281–282 thủ
thuật perceptron 105 vẽ hàm lỗi 66–67 dữ liệu hồi quy tuyến tính 57 hạt
nhân đa thức 328, 334 Toán tử AND trên hệ thống khuyến nghị
332–333 tập dữ liệu vòng 328–330 vượt ra ngoài phương trình bậc hai
333–334 hồi quy đa thức 69, 79–80 vùng tích cực 109 phần sau được
xác định 210–211 mô hình phát hiện thư rác 213–214 với định lý Bayes
228 độ chính xác 403, 427 Điểm F 186–189 tính toán 187 tính điểm Fb
187–189

dự đoán dương 425 hàm dự đoán 266, 403 dự đoán 8, 39, 286 cột dự đoán 173 trước 228 xác định 210–211 mô hình phát hiện thư rác 212 xác suất 214–216, 364 đề xuất sản phẩm 72 quy tắc sản phẩm cho xác suất độc lập 220 mã giả cho thuật toán lan truyền ngược 293–294 cho thuật toán hồi quy logistic 165 cho thuật toán perceptron 135–136 cho thủ thuật perceptron 133 đại số 130–132 hình học 129

R

Gói RandomForestClassifier 358 rừng ngẫu nhiên 354 gói ngẫu nhiên 57 khởi động lại ngẫu nhiên 468 trạng thái ngẫu nhiên 401 tỷ lệ 214–216 RBF (hàm cơ sở xuyên tâm) kernel 328, 339, 405 hàm của 337–339 độ tương tự 339 huấn luyện SVM với 340–345 RBM (máy Boltzmann bị hạn chế) 29 thu hồi 402, 427 Điểm F 186–189 tính toán 187 tính điểm Fb 187–188

tổng quan 183–180 độ nhảy và 198–200 Công thức đào tạo mạng thần kinh, A (Karpathy) 478 hệ thống khuyến nghị VÀ toán tử trên 332–333 phân loại 143 cây quyết định 272–273 hồi quy 72 hồi quy 150 cho cây quyết định 268–271 mô hình 20–21 mạng lưới thần kinh cho mô hình hồi quy 308–309 20 chính quy hóa 86–95 hiệu ứng của chính quy hóa L1 và L2 92–93 ví dụ về trang bị quá mức 88–89

đề huấn luyện mạng noron 295–296 giảm độ dốc cho 466–467
cách trực quan để xem chính quy hóa 93–95 L1 và L2 định mức
89–90 hồi quy lasso và hồi quy sườn 90–91 tham số chính quy hóa
91–92 tham số chính quy hóa 91–92, 299 học tăng cường 29–31
ReLU (đơn vị tuyến tính được chỉnh lưu) 297, 297–298 khung ghi
nhớ-công thức-dự đoán 6–13 cách con người suy nghĩ 6–7 mô hình
và thuật toán 7–13 ví dụ về các mô hình mà con người sử dụng
8–12 ví dụ về các mô hình mà máy móc sử dụng 12 hệ số thay đổi
tỷ lệ 361 kinh nghiệm nghiên cứu 265 hồi quy sườn 90–91 RMSE
(sai số bình phương trung bình gốc) 63, 98 RNN (mạng thần kinh
tái phát) 105, 310–311 robot 31 đường cong ROC (đặc tính vận
hành máy thu) 189–192 AUC (vùng dưới đường cong) 194–195 đưa
ra quyết định sử dụng khả năng thu hồi và độ chính xác 195–198
198–200 độ nhảy và độ đặc hiệu 189–190 Rohrer, Brandon 472,
477 nút gốc 234 quy tắc xác suất bổ sung 215

S

Schapire, Robert 479 Scikit-Learn xây dựng cây quyết định với 263 mã hóa AdaBoost
trong 368–369 mã hóa SVM trong 324–326 C tham số 326 mô hình SVM 324 đơn giản
tuyển sinh sinh viên với 264–267 rừng ngẫu nhiên đào tạo trong 358–359 điểm 125–126,
chức năng điểm 458 266, 403 Chọn kiến trúc tốt nhất cho mạng thần kinh nhân tạo
(Gad) 477 ô tô tự lái 31 phân đoạn ngữ nghĩa 310 độ nhảy 189–194 tính toán 191 tối ưu
hóa trong mô hình 191–194 phân tích tình cảm 21, 104 phân tích tình cảm 281–282 cột
tình cảm 172

Đối tượng chuỗi 390 Thiết lập tốc độ học tập của mạng thần kinh của bạn (Jordan) 478 Shapire, Robert 479 Sicotte, Xavier Bourret 478 hàm sigmoid 150 Silver, David 473 độ tương tự 339 điểm tương tự 375 thủ thuật đơn giản 49–50> đơn giản hóa việc giảm kích thước dữ liệu 25–26 hệ số hóa ma trận và phân tách giá trị số ít 27–29 số ít phân rã giá trị 27–29, 29 dốc 40, 49 phương tiện truyền thông xã hội 21 hàm softmax 173, 298–299 tập dữ liệu email spam 179 bộ lọc 143 mô hình Bayes ngây thơ 212–226 xây dựng với dữ liệu thực 226–230 tìm sau 213–214 tìm trước 212–213 biến tỷ lệ thành xác suất 214–216 Tập dữ liệu bộ lọc thư rác 475 độ đặc hiệu 189–194 tính toán 191 tối ưu hóa trong mô hình 191–194 nhận dạng giọng nói 310 hàm phân tách() 227 phân tách dữ liệu 257–260 thành các tính năng và nhãn, đồng thời đào tạo và xác thực 401 bằng cách sử dụng các tính năng liên tục 258–260 sử dụng các tính năng phân loại phi nhị phân 257 hình vuông lỗi 61, 63–65, 155 thủ thuật bình phương 52–56 Starmer, Josh 472 giảm độ dốc ngẫu nhiên 67 thuật toán hồi quy logistic 166 thuật toán perceptron 137 hàm tổng 392 quy tắc tổng cho xác suất rời rạc 245 học có giám sát 18–22 mô hình phân loại 21–22 mô hình hồi quy 20–21 SVM (hỗ trợ máy vector) mã hóa trong Scikit-Learn 324–326 mã hóa SVM đơn giản 324–325 C tham số 326 phương thức kernel 326–349 mã hóa 346–350

kernel đa thức 328–334 kernel cơ sở xuyên tâm (RBF) 334–339 sử dụng hàm lỗi để xây dựng các bộ phân loại tốt hơn 318–324 thêm hai hàm lỗi để thu được hàm lỗi 322–323 hàm lỗi phân loại 318–320 Tham số C 323–324 hàm lỗi khoảng cách 320–321

T

hàm tanh (hyperbol tangent) 297 lỗi kiểm tra 82 mô hình kiểm tra 81–84, 402–404 độ chính xác 403 Điểm F1 403–404 chọn bộ kiểm tra 83 sử dụng dữ liệu kiểm tra cho mô hình huấn luyện 83–84 bộ kiểm tra 81 gói text_analytics 171 tóm tắt văn bản 310 ngưỡng 120 tập dữ liệu Titanic 388–392 tính năng của 389 tải bằng Pandas 389–390 nghiên cứu với Pandas 391–392 lỗi đào tạo 82 đào tạo mô hình kỹ thuật dữ liệu 400–404 đánh giá mô hình 402–404 một số mô hình trên tập dữ liệu 402 phân tách dữ liệu thành các tính năng và nhãn, đào tạo và xác nhận 401 mô hình thử nghiệm 404 với độ dốc giảm dần 452–466 mô hình phân loại 457–462 mô hình hồi quy tuyến tính 454–458 mạng noron 463–466 huấn luyện mạng noron 292–299 hàm kích hoạt 297–298 lan truyền ngược 293–294 hàm lỗi 293 siêu tham số 299 vấn đề, trang bị quá mức và biến mất gradient 294–295 hàm softmax 298 kỹ thuật cho 295–296 dropout 296 chính quy hóa 295 tập huấn luyện 81 hàm train_test_split 401 học chuyển tiếp 310 Trask, Andrew 309, 477 Turi Tạo 67–69

phân loại các đánh giá IMDB với 171 thuật toán perceptron mã hóa
141–142 hồi quy, kiểm tra và chuẩn hóa đa thức với 95–96

U

trang bị thiếu 78, 101 ví dụ sử dụng hồi quy đa thức 79–81 biểu đồ độ phức tạp của mô hình
85–86 hồi quy, kiểm tra và chuẩn hóa đa thức với Turi Create 95–100 chính quy hóa 86–95
hiệu ứng của chính quy hóa L1 và L2 trong các hệ số của mô hình 92–93 ví dụ về trang bị quá
mức 88–89 cách trực quan để xem chính quy hóa 93–95 định mức L1 và L2 89–90 hồi quy
lasso và hồi quy sườn 90–91 tham số chính quy hóa 91–92 mô hình thử nghiệm 81–84 bộ thử
nghiệm chọn 83 sử dụng dữ liệu thử nghiệm cho mô hình đào tạo 83–84 bộ xác thực 84 Tìm
hiểu Mạng LSTM (Olah) 477 dữ liệu chưa được gắn nhãn 22–25 thuật toán phân cụm 23–25
giảm kích thước 25–26 học máy tổng quát 29 nhân tố ma trận và phân rã giá trị số ít 27–28
học không giám sát 22–25 thuật toán phân cụm 23–25 giảm kích thước 25–26 học máy tổng
quát 29 phân tích nhân tử ma trận và phân tách giá trị số ít 27–29 Sử dụng Ttransfer Learning
để phân loại hình ảnh bằng Keras (Cook) 477

V

bộ xác nhận 84 Veerakumar,
Karthik 475

W

người học yếu 353 lưu lượng truy cập trang
web 21 trọng số 39, 113 Giành giải Netflix
(Chen) 479

X

XGBoost (extreme gradient boosting) 375, 384, 479

xây dựng những người học yếu 376–379
 đưa ra dự đoán 380–381 điểm tương đồng
 375–376 mô hình đào tạo bằng Python
 382–384 cắt tỉa cây 379–380 gói xgboost
 383 Mô hình XGBRegressor 382 Tập dữ
 liệu XOR 332, 422